

INDICE COMPLETO DEL SISTEMA MICROBOT

SEZIONE 1 — Visione e Origine del Sistema

1. Nascita del progetto MicroBot
2. Idea iniziale e transizione da robot singolo a sistema distribuito
3. Differenza tra robotica classica e swarm robotics
4. Cambio di paradigma: da sistemi autonomi a sistemi emergenti
5. Concetto di materia programmabile
6. Forma fisica controllata tramite logica software
7. Obiettivi tecnici del progetto
8. Obiettivi scientifici del progetto
9. Obiettivi imprenditoriali del progetto
10. Evoluzione dell'idea nel tempo
11. Trasformazione da prototipo a ecosistema

SEZIONE 2 — Struttura delle Cartelle e dello ZIP

12. Analisi della cartella principale (workspace ZIP)
13. Struttura generale del sistema di file
14. Organizzazione per domini progettuali
15. Separazione tra MicroBot e progetti esterni
16. Cartelle di documentazione tecnica
17. PDF, relazioni e tesi
18. Cartelle di codice sorgente
19. Progetti web, Python, C e sistemi embedded
20. Cartelle di simulazione
21. Ambienti di test e prototipi software
22. Ruolo del file INDEX.txt come mappa globale
23. Versionamento implicito dei progetti
24. Evoluzione delle versioni nel tempo

SEZIONE 3 — Architettura Generale del Sistema

25. Architettura a livelli del sistema
26. Interfaccia utente
27. Controller centrale
28. MicroBot come nodi distribuiti
29. Separazione delle responsabilità
30. Design modulare
31. Indipendenza dei componenti
32. Sistema centralizzato vs distribuito
33. Flusso delle informazioni
34. Input, elaborazione e output
35. Interazione uomo-macchina
36. Interfacce VR e gesture

SEZIONE 4 — Il MicroBot (Unità Base)

37. Definizione del MicroBot
38. Nodo intelligente distribuito
39. Struttura geometrica del robot
40. Configurazione a coni troncati e sfera

- 41. Componenti interni
- 42. Microcontrollore
- 43. Batteria
- 44. Magneti
- 45. LED
- 46. Distribuzione dei volumi interni
- 47. Vincoli di spazio
- 48. Peso e dimensionamento

SEZIONE 5 — Architettura Elettronica

- 49. Microcontrollore ESP32
- 50. Gestione computazionale
- 51. Sistema di comunicazione
- 52. Sistema di alimentazione Li-Po
- 53. Gestione dell'energia
- 54. Regolazione della tensione
- 55. Convertitori e stabilizzazione
- 56. Driver MOSFET
- 57. Controllo delle bobine
- 58. Diodo di flyback
- 59. Protezione elettrica
- 60. LED RGB come sistema di feedback

SEZIONE 6 — Sistema Magnetico

- 61. Magneti permanenti
- 62. Allineamento passivo
- 63. Bobine elettromagnetiche
- 64. Controllo attivo del campo
- 65. Relazione tra corrente, spire e campo
- 66. Energia magnetica
- 67. Accumulo e rilascio energetico
- 68. Forza di attrazione
- 69. Interazione tra dipoli
- 70. Dipendenza dalla distanza
- 71. Dimensionamento delle coil

SEZIONE 7 — Modello Matematico

- 72. Rappresentazione nello spazio
- 73. Coordinate dei MicroBot
- 74. Modello discreto del moto
- 75. Aggiornamento delle posizioni
- 76. Distanza euclidea
- 77. Interazione tra unità
- 78. Soglie di attacco
- 79. Soglie di distacco
- 80. Meccanismo di hysteresis
- 81. Forze dello swarm
- 82. Coesione
- 83. Separazione

- 84. Allineamento
- 85. Sistema multi-agente
- 86. Emergenza del comportamento collettivo

SEZIONE 8 — Comunicazione

- 87. Rete Wi-Fi
- 88. MicroBot come nodi di rete
- 89. Protocollo UDP
- 90. Comunicazione a bassa latenza
- 91. Struttura dei pacchetti
- 92. Identificatore
- 93. Comando
- 94. Timestamp
- 95. Modalità broadcast
- 96. Modalità unicast

SEZIONE 9 — Sincronizzazione

- 97. Controller come riferimento temporale
- 98. Clock globale del sistema
- 99. Slot temporali
- 100. Coordinazione delle azioni
- 101. Gestione del ritardo
- 102. Allineamento tra dispositivi

SEZIONE 10 — Pattern e Comportamenti

- 103. Formazioni geometriche
- 104. Linee
- 105. Cerchi
- 106. Strutture complesse
- 107. Assegnazione dei ruoli
- 108. Distribuzione delle posizioni
- 109. Adattamento dinamico
- 110. Correzione degli errori

SEZIONE 11 — Safety Layer

- 111. Controllo della corrente
- 112. Protezione delle bobine
- 113. Monitoraggio della temperatura
- 114. Prevenzione del surriscaldamento
- 115. Gestione della batteria
- 116. Protezione energetica
- 117. Watchdog timer
- 118. Ripristino automatico

SEZIONE 12 — Simulazione Software

- 119. Ambiente di simulazione
- 120. Implementazione web
- 121. Visualizzazione dei MicroBot
- 122. Modelli virtuali
- 123. Test dei pattern
- 124. Validazione del comportamento
- 125. Iterazione veloce
- 126. Sviluppo sperimentale

SEZIONE 13 — Telemetria

- 127. Monitoraggio dello stato
- 128. Raccolta dati in tempo reale
- 129. Analisi del sistema
- 130. Debug dello swarm

SEZIONE 14 — MicroBot OS

- 131. Sistema operativo embedded
- 132. Evoluzione delle versioni
- 133. Gestione delle risorse
- 134. Astrazione hardware
- 135. Scheduling
- 136. Shell interna
- 137. Diagnostica

SEZIONE 15 — Sicurezza Biometrica

- 138. Autenticazione tramite fotocamera
- 139. Controllo degli accessi
- 140. Riconoscimento facciale
- 141. Landmark detection
- 142. Confronto dei dati
- 143. Decisione tramite soglia

SEZIONE 16 — Progetti Esterni

- 144. Intelligenza artificiale
- 145. Clustering e modelli
- 146. Giochi
- 147. Sistemi interattivi

- 148. Simulazioni evolutive
- 149. Algoritmi genetici
- 150. Progetti web
- 151. Interfacce avanzate
- 152. Tool diagnostici
- 153. Script di sistema

SEZIONE 17 — Integrazione dei Progetti

- 154. Collegamento tra sistemi
- 155. Integrazione funzionale
- 156. Supporto reciproco
- 157. Crescita delle competenze
- 158. Evoluzione tecnica

SEZIONE 18 — Livello Teorico Avanzato

- 159. Interpretazione delle dimensioni
- 160. Quarta dimensione (tempo)
- 161. Quinta dimensione (configurazioni)
- 162. Tempo come struttura
- 163. Sistema come processo
- 164. Forma come evento dinamico

SEZIONE 19 — Scalabilità

- 165. Espansione dello swarm
- 166. Aumento del numero di unità
- 167. Limiti hardware
- 168. Limiti comunicativi
- 169. Limiti energetici
- 170. Sviluppi futuri
- 171. Integrazione AI
- 172. Sensori avanzati

SEZIONE 20 — Visione Finale

- 173. MicroBot come piattaforma
- 174. Sistema modulare ed estensibile
- 175. Applicazioni tecnologiche
- 176. Applicazioni nella robotica
- 177. Applicazioni nella medicina
- 178. Applicazioni nella realtà virtuale
- 179. Potenziale imprenditoriale
- 180. Sviluppo startup
- 181. Ricerca scientifica
- 182. Possibilità di pubblicazione
- 183. Evoluzione futura del sistema

1.1 Nascita del progetto MicroBot

La nascita del progetto MicroBot si colloca all'interno di un processo evolutivo che parte da una riflessione fondamentale sulla natura dei sistemi tecnologici e sul loro rapporto con la realtà fisica. L'idea originaria non emerge come una semplice intuizione isolata, ma come risultato di una progressiva trasformazione del modo di concepire la robotica, passando da una visione tradizionale basata su unità autonome e indipendenti a una prospettiva sistemica fondata sulla cooperazione tra molteplici entità semplici.

In una fase iniziale, il concetto di robot veniva associato a una singola macchina dotata di sensori, attuatori e capacità decisionali, progettata per eseguire compiti specifici in modo autonomo. Tuttavia, questa impostazione presenta limiti strutturali evidenti, in particolare in termini di scalabilità, adattabilità e robustezza. Un singolo sistema, per quanto avanzato, rimane vincolato alla propria complessità interna e alla difficoltà di gestire dinamiche ambientali variabili.

Da questa consapevolezza nasce la necessità di un cambio di paradigma: invece di aumentare la complessità di un singolo robot, si introduce l'idea di distribuire l'intelligenza e il comportamento su un insieme di unità elementari. Questo approccio si ispira ai sistemi naturali, come gli sciame di insetti o i sistemi cellulari, nei quali comportamenti complessi emergono dall'interazione locale tra componenti semplici.

Il progetto MicroBot prende forma proprio a partire da questa intuizione: realizzare un sistema composto da molteplici micro-unità, ciascuna dotata di funzionalità limitate ma capace di interagire con le altre attraverso meccanismi fisici e logici ben definiti. In questo contesto, il robot non è più una singola entità, ma diventa una collettività dinamica, in cui la forma, la funzione e il comportamento emergono dall'organizzazione del sistema nel suo complesso.

Un elemento chiave nella nascita del progetto è rappresentato dall'introduzione del concetto di materia programmabile. L'idea centrale consiste nel considerare la materia non come qualcosa di statico e passivo, ma come un insieme di unità riconfigurabili, in grado di modificare la propria struttura in risposta a input esterni o a logiche interne. I MicroBot, in questa visione, costituiscono i "mattoni" elementari di una materia dinamica, la cui configurazione può essere controllata tramite software.

Parallelamente, lo sviluppo del progetto è stato influenzato da un approccio fortemente sperimentale, basato sulla realizzazione di prototipi progressivi sia a livello hardware che software. Le prime versioni non sono state pensate come sistemi completi, ma come strumenti per validare singoli aspetti del modello, come la comunicazione tra nodi, la sincronizzazione temporale e la rappresentazione visiva del comportamento collettivo. In particolare, l'utilizzo di simulazioni software e prototipi basati su LED ha permesso di tradurre concetti astratti in comportamenti osservabili, facilitando la comprensione delle dinamiche emergenti.

Un ulteriore aspetto rilevante nella genesi del progetto è l'integrazione tra diversi ambiti disciplinari. MicroBot non si configura esclusivamente come un progetto di robotica, ma come un sistema interdisciplinare che combina elementi di informatica, elettronica, fisica e modellazione matematica. Questa integrazione consente di affrontare il problema da più prospettive, rendendo il sistema non solo più completo, ma anche più flessibile e aperto a sviluppi futuri.

Infine, la nascita del progetto è strettamente legata a una visione a lungo termine, che va oltre la realizzazione di un semplice prototipo. MicroBot è concepito fin dall'inizio come un ecosistema in evoluzione, capace di crescere nel tempo attraverso l'aggiunta di nuove funzionalità, nuovi moduli e nuovi livelli di complessità. In questo senso, ogni fase di sviluppo non rappresenta un punto di arrivo, ma un passo intermedio verso la costruzione di un sistema più ampio, in cui tecnologia, teoria e applicazione convergono in un'unica architettura coerente.

1.2 Idea iniziale e transizione da robot singolo a sistema distribuito

L'idea iniziale alla base del progetto MicroBot si sviluppa a partire da una concezione classica della robotica, in cui il sistema è identificato con una singola unità autonoma, progettata per eseguire compiti specifici attraverso l'integrazione di sensori, attuatori e logiche di controllo. In questa configurazione, il robot rappresenta un'entità completa, dotata di tutte le componenti necessarie per percepire l'ambiente, elaborare informazioni e agire di conseguenza.

Tuttavia, questa impostazione evidenzia rapidamente una serie di limiti strutturali. L'aumento della complessità del robot comporta un incremento significativo delle difficoltà progettuali, sia in termini di integrazione hardware che di gestione software. Inoltre, la dipendenza da un'unica unità rende il sistema intrinsecamente fragile: un guasto o un malfunzionamento compromette l'intero comportamento del sistema.

Allo stesso tempo, la scalabilità risulta limitata, poiché ogni nuova funzionalità richiede un'espansione diretta della complessità interna del robot stesso.

A partire da queste considerazioni emerge la necessità di un cambiamento radicale nell'approccio progettuale. Invece di concentrare tutte le capacità in un'unica entità, si introduce l'idea di suddividere il sistema in una molteplicità di unità elementari, ciascuna caratterizzata da una struttura semplice e da funzionalità limitate. Questa transizione segna il passaggio da un modello centralizzato e monolitico a un modello distribuito e modulare.

Nel paradigma distribuito, ogni MicroBot non è più un sistema completo, ma un nodo all'interno di una rete. La sua funzione non è quella di operare in modo indipendente, bensì di contribuire al comportamento globale attraverso interazioni locali con altre unità. Questo implica una ridefinizione del concetto stesso di intelligenza: essa non è più localizzata in un singolo elemento, ma emerge dall'insieme delle relazioni tra i componenti del sistema.

La transizione da robot singolo a sistema distribuito comporta anche un cambiamento nella logica di progettazione del comportamento. Nel modello tradizionale, il comportamento è programmato in modo esplicito e centralizzato; nel modello MicroBot, invece, esso è il risultato di regole semplici applicate localmente e ripetute su larga scala. Questo approccio consente di ottenere proprietà emergenti, ovvero comportamenti complessi che non sono direttamente codificati, ma che derivano dall'interazione tra le unità.

Un ulteriore elemento chiave di questa transizione è rappresentato dalla modularità. Ogni MicroBot può essere considerato come un modulo indipendente, facilmente replicabile e sostituibile. Ciò introduce vantaggi significativi in termini di robustezza del sistema: la perdita o il malfunzionamento di una singola unità non compromette l'intero sistema, ma viene compensato dalle altre. Inoltre, la scalabilità diventa un aspetto intrinseco del sistema, poiché è possibile aumentare il numero di unità senza modificare la struttura logica di base.

Dal punto di vista architetturale, il sistema assume quindi una forma distribuita, in cui le informazioni e le decisioni sono condivise tra i nodi. Tuttavia, nel caso specifico del progetto MicroBot, viene mantenuta una componente di coordinamento centrale, rappresentata dal controller, che funge da riferimento globale per la sincronizzazione e la gestione delle operazioni. Questo porta a una configurazione ibrida, in cui coesistono elementi di centralizzazione e distribuzione, permettendo di combinare i vantaggi di entrambi gli approcci.

Infine, la transizione verso un sistema distribuito apre la strada a una nuova modalità di interazione con la materia. Non si tratta più di controllare direttamente un oggetto, ma di orchestrare un insieme di unità che, nel loro complesso, assumono una determinata forma e funzione. Questo rappresenta il fondamento concettuale su cui si sviluppa l'intero progetto MicroBot, e costituisce il punto di partenza per l'introduzione del concetto di swarm robotics e, successivamente, di materia programmabile.

1.3 Differenza tra robotica classica e swarm robotics

La distinzione tra robotica classica e swarm robotics rappresenta uno degli elementi concettuali fondamentali per comprendere il modello alla base del sistema MicroBot. I due approcci si differenziano non solo per l'architettura dei sistemi, ma anche per la filosofia progettuale, la gestione dell'intelligenza e il modo in cui emerge il comportamento.

La robotica classica si basa su un paradigma centralizzato e unitario, in cui il robot è concepito come un sistema autonomo completo. In questo modello, tutte le funzionalità necessarie al funzionamento sono integrate all'interno di un'unica entità: sensori per la percezione dell'ambiente, attuatori per l'interazione fisica e un'unità di controllo per l'elaborazione delle informazioni e la presa di decisione. Il comportamento del sistema è definito in modo esplicito attraverso algoritmi deterministici o adattivi, e l'intelligenza è localizzata all'interno del robot stesso.

Dal punto di vista formale, un sistema di robotica classica può essere descritto come una funzione di trasformazione tra input e output:

$$f: I \rightarrow O$$

dove I rappresenta l'insieme degli input sensoriali e O l'insieme delle azioni eseguite. In questo contesto, la complessità del comportamento è direttamente proporzionale alla complessità della funzione f , che deve essere progettata per gestire tutte le possibili situazioni operative.

Al contrario, la swarm robotics introduce un paradigma distribuito, ispirato ai sistemi naturali come colonie di insetti, stormi di uccelli o sistemi cellulari. In questo modello, il sistema è composto da un insieme di agenti semplici, detti unità o nodi, ciascuno dotato di capacità limitate e di una conoscenza locale dell'ambiente. L'intelligenza non è più concentrata in un singolo elemento, ma distribuita tra tutte le unità.

Il comportamento globale del sistema non è programmato direttamente, ma emerge dall'interazione tra gli agenti. Ogni unità applica un insieme di regole locali relativamente semplici, e la ripetizione di queste regole su larga scala genera dinamiche collettive complesse. Formalmente, il sistema può essere descritto come:

$$S = \{a_1, a_2, \dots, a_n\}$$

dove ogni agente a_i segue una funzione locale:

$$a_i(t+1) = g(a_i(t), N_i(t), E(t))$$

in cui N_i rappresenta il vicinato locale dell'agente e E lo stato dell'ambiente. Il comportamento globale del sistema S non è definito da una funzione unica, ma emerge dall'insieme delle interazioni tra gli agenti.

Una differenza fondamentale tra i due approcci riguarda quindi la natura del controllo. Nella robotica classica il controllo è centralizzato e top-down: esiste una logica globale che governa il comportamento del sistema. Nella swarm robotics, invece, il controllo è distribuito e bottom-up: il comportamento globale è il risultato di interazioni locali, senza una regia centrale esplicita.

Questa differenza si riflette anche nelle proprietà dei sistemi. I sistemi di robotica classica tendono a essere più precisi e controllabili, ma meno robusti e scalabili. Al contrario, i sistemi swarm presentano elevata robustezza, grazie alla ridondanza delle unità, e alta scalabilità, poiché il numero di agenti può essere aumentato senza modificare le regole di base. Tuttavia, essi risultano più complessi da progettare e analizzare, in quanto il comportamento emergente non è sempre facilmente prevedibile.

Nel contesto del progetto MicroBot, viene adottato un approccio ibrido che combina elementi di entrambi i paradigmi. Da un lato, i MicroBot operano come unità distribuite, seguendo regole locali e contribuendo a un comportamento emergente collettivo. Dall'altro, è presente un controller centrale che introduce un livello di coordinamento globale, in particolare per quanto riguarda la sincronizzazione temporale e la gestione delle comunicazioni. Questa scelta consente di mantenere i vantaggi della distribuzione, come robustezza e modularità, senza rinunciare al controllo necessario per applicazioni pratiche.

In conclusione, la differenza tra robotica classica e swarm robotics non riguarda semplicemente il numero di robot coinvolti, ma rappresenta un cambiamento profondo nel modo di concepire l'intelligenza, il controllo e il comportamento nei sistemi artificiali. Il progetto MicroBot si inserisce in questo contesto come un sistema che sfrutta i principi della swarm robotics, adattandoli a un'architettura controllata e progettata per applicazioni reali.

1.4 Cambio di paradigma: da sistemi autonomi a sistemi emergenti

Il passaggio da sistemi autonomi a sistemi emergenti rappresenta un cambiamento di paradigma fondamentale nella progettazione dei sistemi robotici e, più in generale, nei sistemi complessi artificiali. Questo cambiamento non riguarda soltanto l'architettura tecnica, ma coinvolge direttamente il modo in cui vengono concepiti il controllo, l'intelligenza e la relazione tra le parti e il tutto.

Nel paradigma dei sistemi autonomi, ogni entità è progettata per essere autosufficiente. Il robot viene dotato di tutti i componenti necessari per operare in modo indipendente: percezione, elaborazione e azione sono integrati all'interno della stessa unità. Il comportamento è definito in modo esplicito e diretto, attraverso algoritmi che stabiliscono come il sistema deve reagire a determinati input. In questa visione, il sistema è completamente descrivibile a partire dalle sue componenti interne e dalle regole che le governano.

Tuttavia, questo approccio presenta limiti significativi quando si tenta di affrontare sistemi complessi o ambienti altamente variabili. La necessità di prevedere tutte le possibili condizioni operative porta a un aumento esponenziale della complessità del sistema, rendendo difficile sia la progettazione che la gestione del comportamento. Inoltre, la rigidità delle regole esplicite limita la capacità del sistema di adattarsi a situazioni non previste.

Il paradigma dei sistemi emergenti introduce una prospettiva radicalmente diversa. In questo modello, il comportamento complessivo del sistema non è definito direttamente, ma emerge dall'interazione tra un insieme di componenti semplici. Ogni unità segue regole locali, spesso molto basilari, e non possiede una conoscenza globale dello stato del sistema. Nonostante ciò, l'insieme delle interazioni genera strutture, pattern e comportamenti complessi a livello macroscopico.

Formalmente, si può distinguere tra due livelli di descrizione: il livello locale e il livello globale. Il livello locale è definito dalle regole che governano il comportamento delle singole unità, mentre il livello globale è rappresentato dal comportamento emergente del sistema nel suo complesso. La relazione tra questi due livelli non è lineare né immediata: piccoli cambiamenti nelle regole locali possono produrre effetti significativi sul comportamento globale, e viceversa.

Questo implica una ridefinizione del concetto di controllo. Nei sistemi emergenti, il controllo non avviene più attraverso la specifica diretta del comportamento finale, ma attraverso la progettazione delle regole locali e delle interazioni tra le unità. In altre parole, il progettista non definisce cosa il sistema deve fare in ogni istante, ma stabilisce le condizioni affinché un determinato comportamento possa emergere spontaneamente.

Nel contesto del progetto MicroBot, questo paradigma si traduce nella progettazione di unità semplici, dotate di capacità limitate, che interagiscono tra loro attraverso meccanismi fisici (come l'attrazione magnetica) e logici (come la comunicazione tra nodi). Il comportamento globale, come la formazione di strutture o l'organizzazione spaziale dello swarm, non è imposto direttamente, ma emerge dall'insieme delle interazioni tra i MicroBot.

Un aspetto particolarmente rilevante di questo approccio è la sua connessione con il concetto di materia dinamica. Se nei sistemi tradizionali la forma è una proprietà statica dell'oggetto, nei sistemi emergenti la forma diventa il risultato di un processo. La configurazione spaziale del sistema non è predefinita, ma evolve nel tempo in funzione delle interazioni tra le unità. In questo senso, la materia stessa può essere interpretata come un sistema computazionale distribuito, in cui la forma è una manifestazione visibile delle dinamiche interne.

Questo cambiamento di paradigma introduce anche nuove sfide. La progettazione di sistemi emergenti richiede un approccio sperimentale e iterativo, in quanto il comportamento globale non è sempre prevedibile a partire dalle regole locali. Inoltre, la validazione del sistema diventa più complessa, poiché è necessario analizzare non solo il comportamento delle singole unità, ma anche le dinamiche collettive.

Nonostante queste difficoltà, i vantaggi del paradigma emergente sono significativi. I sistemi risultano intrinsecamente più robusti, grazie alla distribuzione delle funzionalità, e più adattabili, poiché il comportamento può evolvere in risposta alle condizioni ambientali. Inoltre, la scalabilità è una proprietà naturale del sistema: l'aggiunta di nuove unità non richiede modifiche sostanziali alle regole di base.

In conclusione, il passaggio da sistemi autonomi a sistemi emergenti rappresenta il fondamento teorico su cui si costruisce l'intero progetto MicroBot. Esso consente di superare i limiti della robotica tradizionale, introducendo una nuova modalità di progettazione in cui complessità e comportamento nascono dall'interazione di elementi semplici, aprendo la strada a sistemi più flessibili, dinamici e potenzialmente riconfigurabili.

1.5 Concetto di materia programmabile

Il concetto di materia programmabile rappresenta uno dei pilastri teorici fondamentali del progetto MicroBot e costituisce il punto di convergenza tra robotica, informatica e modellazione fisica. Esso introduce una visione innovativa della materia, non più intesa come entità statica e passiva, ma come sistema dinamico, riconfigurabile e controllabile attraverso logiche computazionali.

Tradizionalmente, la materia viene descritta come un insieme di particelle organizzate secondo leggi fisiche che determinano proprietà come forma, struttura e comportamento. In questo contesto, la forma di un

oggetto è una caratteristica intrinseca, definita dalla sua composizione e difficilmente modificabile senza interventi esterni diretti. Anche nei sistemi robotici classici, la struttura fisica del robot è progettata a priori e rimane invariata durante il funzionamento.

Il paradigma della materia programmabile rompe questa visione introducendo l'idea che la forma possa essere trattata come una variabile, e non come una costante. In altre parole, la configurazione spaziale della materia diventa un parametro controllabile, modificabile in funzione di input esterni o di logiche interne al sistema. Questo implica una separazione concettuale tra la materia stessa e la forma che essa assume nel tempo.

Nel contesto del progetto MicroBot, la materia programmabile è realizzata attraverso un insieme di unità discrete, i MicroBot, che possono essere considerati come elementi base di un sistema più ampio. Ogni unità è dotata di capacità limitate, come la comunicazione, il controllo magnetico e la segnalazione luminosa, ma è progettata per interagire con le altre unità in modo coordinato. La forma globale del sistema emerge dalla disposizione spaziale di queste unità e può essere modificata attraverso il controllo delle interazioni tra di esse.

Formalmente, si può descrivere la configurazione del sistema come un insieme di posizioni nello spazio:

$$C(t) = \{ (x_1(t), y_1(t)), (x_2(t), y_2(t)), \dots, (x_n(t), y_n(t)) \}$$

dove ogni coppia (x_i, y_i) rappresenta la posizione di un MicroBot al tempo t . La forma del sistema è quindi una funzione della configurazione $C(t)$, e la sua evoluzione nel tempo dipende dalle regole di interazione tra le unità e dagli input di controllo.

Questo approccio consente di trattare la forma come un'informazione, piuttosto che come una proprietà fisica fissa. In altre parole, la forma diventa programmabile, nel senso che può essere modificata attraverso istruzioni software che influenzano il comportamento delle singole unità. Il sistema si comporta quindi come un "materiale attivo", in cui la struttura è il risultato di un processo computazionale distribuito.

Un aspetto particolarmente rilevante è il ruolo delle interazioni locali nella determinazione della forma globale. I MicroBot non possiedono una conoscenza completa della configurazione del sistema, ma operano sulla base di informazioni locali, come la distanza dalle unità vicine o lo stato dei loro magneti. Nonostante ciò, l'insieme di queste interazioni locali consente di ottenere configurazioni coerenti a livello globale, dimostrando come la complessità possa emergere da regole semplici.

Il concetto di materia programmabile introduce anche una nuova relazione tra software e hardware. Nei sistemi tradizionali, il software controlla il comportamento del sistema senza modificarne la struttura fisica. Nel caso dei MicroBot, invece, il software influisce direttamente sulla configurazione della materia, determinando la forma e l'organizzazione spaziale del sistema. Questo porta a una fusione tra livello logico e livello fisico, in cui il codice non descrive soltanto azioni, ma contribuisce a definire la struttura stessa del sistema.

Dal punto di vista applicativo, la materia programmabile apre scenari estremamente ampi. Un sistema basato su MicroBot potrebbe adattare la propria forma in funzione del contesto, passando da una configurazione compatta a una distribuita, oppure assumendo strutture specifiche per svolgere determinati compiti. Questa capacità di riconfigurazione rende il sistema potenzialmente utilizzabile in ambiti come la robotica avanzata, la medicina, l'esplorazione e la realtà virtuale.

Infine, il concetto di materia programmabile si collega direttamente a una visione più ampia del sistema come processo dinamico. La forma non è più un punto di arrivo, ma uno stato temporaneo all'interno di un'evoluzione continua. In questo senso, il sistema MicroBot può essere interpretato come una manifestazione concreta di una materia che non è definita una volta per tutte, ma che esiste attraverso le trasformazioni che è in grado di compiere.

1.6 Forma fisica controllata tramite logica software

Nel contesto del sistema MicroBot, il controllo della forma fisica rappresenta uno degli aspetti più innovativi e distintivi dell'intero progetto. A differenza dei sistemi tradizionali, in cui la forma è determinata esclusivamente da vincoli meccanici e costruttivi, nel paradigma adottato la configurazione spaziale del

sistema è direttamente influenzata da logiche software, rendendo possibile una manipolazione dinamica della struttura fisica attraverso il controllo computazionale.

Questo approccio si basa sull'idea che la forma globale del sistema non sia predefinita, ma emerga dall'interazione tra le singole unità e dai segnali di controllo generati dal sistema centrale. Ogni MicroBot, infatti, non possiede una conoscenza completa della configurazione globale, ma reagisce a input locali e a comandi ricevuti, contribuendo in modo incrementale alla formazione della struttura complessiva.

Il controllo della forma può essere descritto come un processo a più livelli, in cui il software agisce indirettamente sulla configurazione fisica attraverso una catena di trasformazioni. In primo luogo, il controller centrale genera una serie di istruzioni che definiscono un obiettivo globale, ad esempio la formazione di una determinata figura o la distribuzione spaziale delle unità. Queste istruzioni vengono tradotte in comandi specifici indirizzati ai singoli MicroBot, contenenti informazioni come la posizione target, lo stato magnetico e i parametri di comportamento.

A livello locale, ogni MicroBot elabora i comandi ricevuti e li integra con le informazioni provenienti dall'ambiente e dalle unità vicine. In particolare, il sistema magnetico svolge un ruolo fondamentale nel tradurre le decisioni logiche in interazioni fisiche. L'attivazione delle bobine elettromagnetiche, controllata tramite segnali PWM e driver MOSFET, genera forze di attrazione o repulsione che influenzano direttamente la posizione relativa delle unità.

Questo meccanismo consente di realizzare una catena causale ben definita:

software → segnale elettrico → campo magnetico → forza fisica → variazione della posizione → modifica della forma

In questo contesto, il software non agisce direttamente sulla materia, ma controlla i parametri fisici che governano le interazioni tra le unità. La forma del sistema è quindi il risultato finale di una sequenza di trasformazioni che partono dal livello logico e si propagano fino al livello fisico.

Un aspetto cruciale di questo processo è la presenza di un feedback continuo. I MicroBot possono trasmettere informazioni sul proprio stato, come posizione, livello di batteria o condizioni operative, permettendo al controller di aggiornare dinamicamente le istruzioni. Questo introduce un ciclo di controllo chiuso, in cui la forma del sistema viene costantemente monitorata e corretta in funzione delle deviazioni rispetto all'obiettivo.

Dal punto di vista matematico, il sistema può essere modellato come un sistema dinamico discreto:

$$C(t+1) = F(C(t), U(t))$$

dove $C(t)$ rappresenta la configurazione del sistema al tempo t e $U(t)$ l'insieme dei comandi generati dal controller. La funzione F descrive l'evoluzione del sistema in base alle regole di interazione tra le unità e ai segnali di controllo.

Un ulteriore elemento rilevante è la presenza di vincoli fisici, come limiti di energia, capacità delle bobine e tempi di risposta dei componenti elettronici. Il software deve quindi operare all'interno di questi vincoli, ottimizzando i parametri di controllo per ottenere il comportamento desiderato senza compromettere la stabilità del sistema. Questo introduce un livello di complessità aggiuntivo, in cui la progettazione del software deve tenere conto delle caratteristiche fisiche dell'hardware.

Inoltre, la natura distribuita del sistema implica che il controllo della forma non è mai completamente centralizzato. Anche se il controller fornisce una guida globale, l'effettiva realizzazione della forma dipende dalle interazioni locali tra i MicroBot. Questo porta a una combinazione di controllo top-down e dinamiche bottom-up, in cui il comportamento finale emerge dalla collaborazione tra livello globale e livello locale.

Infine, il controllo della forma tramite software apre la possibilità di implementare comportamenti adattivi e riconfigurabili. Il sistema può modificare la propria struttura in tempo reale in risposta a input esterni, cambiando configurazione in funzione delle condizioni operative o degli obiettivi richiesti. In questo senso, la forma diventa una variabile dinamica, controllata non da vincoli statici, ma da logiche computazionali.

In conclusione, il sistema MicroBot realizza una nuova modalità di interazione tra software e materia, in cui il codice non si limita a descrivere azioni, ma diventa uno strumento per modellare direttamente la struttura fisica del sistema. Questo rappresenta un passo significativo verso la realizzazione di sistemi in cui la distinzione tra livello logico e livello fisico tende progressivamente a ridursi, aprendo la strada a nuove forme di progettazione e controllo nei sistemi tecnologici complessi.

1.7 Obiettivi tecnici del progetto

Gli obiettivi tecnici del progetto MicroBot definiscono l'insieme delle funzionalità, delle capacità operative e dei vincoli ingegneristici necessari per la realizzazione di un sistema distribuito di unità robotiche cooperative. Tali obiettivi costituiscono il collegamento diretto tra il modello teorico del sistema e la sua implementazione concreta, guidando le scelte progettuali a livello hardware, software e architetturale.

Un primo obiettivo fondamentale è la realizzazione di unità modulari e replicabili. Ogni MicroBot deve essere progettato come un nodo autonomo dal punto di vista operativo, ma allo stesso tempo semplice nella struttura e facilmente producibile. Questo implica la definizione di un'architettura hardware compatta, basata su un microcontrollore a basso consumo, una batteria integrata e un insieme minimo di componenti per la comunicazione e l'interazione fisica. La modularità consente non solo la scalabilità del sistema, ma anche la sostituibilità delle unità, aumentando la robustezza complessiva.

Un secondo obiettivo riguarda la comunicazione tra le unità e il controller centrale. Il sistema deve garantire uno scambio di informazioni affidabile e a bassa latenza, utilizzando protocolli leggeri e adatti a dispositivi embedded. L'adozione di tecnologie come il Wi-Fi in modalità SoftAP e l'utilizzo del protocollo UDP permette di ridurre l'overhead comunicativo, mantenendo al contempo una buona capacità di coordinamento tra i nodi. La struttura dei pacchetti deve includere identificatori univoci, timestamp e comandi specifici, in modo da consentire una gestione precisa delle operazioni.

Un ulteriore obiettivo tecnico è la sincronizzazione temporale del sistema. Poiché molte operazioni, come l'attivazione dei magneti o l'esecuzione di pattern collettivi, richiedono un'elevata coordinazione, è necessario introdurre un riferimento temporale globale. Il controller centrale assume quindi il ruolo di clock di sistema, distribuendo segnali temporali ai MicroBot e permettendo l'esecuzione sincronizzata delle azioni attraverso una struttura a slot temporali.

Dal punto di vista fisico, un obiettivo chiave è il controllo delle interazioni magnetiche tra le unità. Le bobine elettromagnetiche devono essere dimensionate in modo da generare forze sufficienti per l'attrazione e il mantenimento del contatto tra i MicroBot, senza compromettere l'efficienza energetica. Ciò richiede un'attenta progettazione dei parametri elettrici, come corrente, numero di spire e resistenza, nonché l'utilizzo di driver adeguati per la gestione delle bobine.

Parallelamente, il sistema deve garantire una gestione efficiente dell'energia. Ogni MicroBot è alimentato da una batteria Li-Po di capacità limitata, il che rende necessario ottimizzare il consumo energetico di tutti i componenti. Questo include l'adozione di modalità di risparmio energetico, la gestione dinamica dell'attività delle bobine e il monitoraggio continuo dello stato della batteria per prevenire condizioni di scarica critica.

Un altro obiettivo tecnico rilevante è la capacità di rappresentare e controllare la posizione delle unità nello spazio. Anche in assenza di sistemi di localizzazione avanzati, il sistema deve essere in grado di stimare le relazioni spaziali tra i MicroBot, utilizzando informazioni locali come la distanza tra unità o la presenza di contatto. Questo consente di implementare algoritmi di coordinamento basati su modelli matematici, come il calcolo della distanza euclidea e l'applicazione di soglie di attacco e distacco.

Dal punto di vista software, un obiettivo essenziale è la realizzazione di un sistema di controllo distribuito basato su macchine a stati. Ogni MicroBot deve essere in grado di gestire autonomamente il proprio stato operativo (ad esempio idle, attivo, errore), reagendo sia ai comandi del controller sia alle condizioni locali. Questo approccio permette di strutturare il comportamento del sistema in modo chiaro e modulare, facilitando lo sviluppo e il debugging.

Inoltre, il sistema deve supportare la simulazione e la validazione del comportamento prima dell'implementazione fisica. La creazione di ambienti di simulazione software, sia in ambito web che tramite linguaggi come Python, consente di testare algoritmi, pattern e dinamiche di swarm in modo rapido ed economico. Questo approccio riduce i tempi di sviluppo e permette di individuare eventuali criticità prima della realizzazione hardware.

Infine, un obiettivo tecnico trasversale riguarda la progettazione di un sistema scalabile e aperto a future estensioni. L'architettura deve essere pensata fin dall'inizio per supportare l'integrazione di nuovi moduli, come sensori avanzati, algoritmi di intelligenza artificiale o interfacce uomo-macchina basate su realtà virtuale. Questo richiede una separazione chiara tra i diversi livelli del sistema e l'adozione di interfacce ben definite tra i componenti.

In conclusione, gli obiettivi tecnici del progetto MicroBot delineano un sistema complesso ma coerente, in cui ogni scelta progettuale è orientata alla realizzazione di un'architettura distribuita, modulare e riconfigurabile. Essi rappresentano la base operativa su cui si sviluppano tutte le fasi successive del progetto, dalla prototipazione alla validazione fino all'implementazione completa del sistema.

1.8 Obiettivi scientifici del progetto

Gli obiettivi scientifici del progetto MicroBot si collocano nell'ambito dello studio dei sistemi complessi, della robotica distribuita e della modellazione della materia come sistema dinamico riconfigurabile. A differenza degli obiettivi tecnici, che riguardano la realizzazione concreta del sistema, gli obiettivi scientifici mirano a comprendere, formalizzare e generalizzare i principi che governano il comportamento del sistema stesso.

Un primo obiettivo scientifico consiste nello studio dei fenomeni emergenti all'interno di sistemi multi-agente. Il progetto MicroBot rappresenta un caso di studio in cui un insieme di unità semplici, dotate di capacità limitate, è in grado di generare comportamenti collettivi complessi attraverso interazioni locali. L'analisi di tali fenomeni permette di approfondire la relazione tra regole locali e comportamento globale, con particolare attenzione alle condizioni che favoriscono la formazione di pattern stabili e coerenti.

Un secondo obiettivo riguarda la formalizzazione del comportamento dello swarm attraverso modelli matematici. Il sistema può essere descritto come un insieme di agenti discreti che evolvono nel tempo secondo regole di interazione, rendendolo assimilabile a un sistema dinamico multi-agente. In questo contesto, è possibile studiare proprietà come la convergenza verso configurazioni stabili, la stabilità delle strutture formate e la sensibilità del sistema a variazioni dei parametri. L'introduzione di concetti come soglie di attacco e distacco e meccanismi di hysteresis consente di modellare in modo più accurato le dinamiche reali del sistema.

Un ulteriore obiettivo scientifico è l'analisi del ruolo della comunicazione nei sistemi distribuiti. Il progetto MicroBot integra un sistema di comunicazione a bassa latenza tra le unità e il controller centrale, permettendo di studiare come l'informazione si propaga all'interno dello swarm e come influisce sul comportamento collettivo. In particolare, è possibile confrontare approcci diversi, come comunicazione centralizzata, distribuita o ibrida, valutandone l'impatto su robustezza, scalabilità e capacità di coordinamento.

Un aspetto centrale della ricerca riguarda anche la relazione tra livello logico e livello fisico. Il sistema MicroBot consente di osservare come decisioni computazionali, espresse sotto forma di segnali e algoritmi, si traducano in interazioni fisiche tra le unità. Questo apre la possibilità di studiare la materia come un sistema computazionale distribuito, in cui la forma emerge da processi di elaborazione dell'informazione. In questo senso, il progetto si colloca all'intersezione tra informatica, fisica e scienza dei materiali.

Un obiettivo particolarmente rilevante è la definizione operativa del concetto di materia programmabile. Sebbene questo concetto sia stato introdotto in ambito teorico, la sua realizzazione pratica è ancora oggetto di ricerca. Il sistema MicroBot rappresenta un tentativo di concretizzare tale idea attraverso un approccio modulare e scalabile, fornendo una piattaforma sperimentale per testare ipotesi e modelli legati alla riconfigurabilità della materia.

Inoltre, il progetto si propone di esplorare il legame tra spazio, tempo e configurazione nei sistemi dinamici. La configurazione del sistema non è statica, ma evolve nel tempo secondo regole definite, rendendo possibile interpretare la forma come una funzione temporale. Questo introduce una dimensione dinamica nello studio delle strutture, in cui il tempo non è solo un parametro esterno, ma parte integrante del sistema stesso. Tale prospettiva si collega a modelli più avanzati in cui le configurazioni possono essere viste come stati all'interno di uno spazio delle possibilità.

Un altro obiettivo scientifico è lo sviluppo di metodologie di simulazione e validazione per sistemi complessi. Il progetto prevede l'utilizzo di ambienti di simulazione per testare il comportamento dello swarm prima della realizzazione fisica, consentendo di confrontare modelli teorici e risultati sperimentali. Questo approccio permette di identificare discrepanze tra teoria e pratica e di migliorare progressivamente i modelli utilizzati.

Infine, il progetto MicroBot si pone come piattaforma per l'integrazione di tecniche avanzate di intelligenza artificiale. L'introduzione di algoritmi di apprendimento automatico potrebbe permettere alle unità di adattare il proprio comportamento in funzione dell'esperienza, aprendo la strada a sistemi in grado di evolvere nel tempo. Questo rappresenta un possibile sviluppo futuro, ma al tempo stesso un obiettivo scientifico rilevante, in quanto consente di esplorare nuove forme di intelligenza distribuita.

In conclusione, gli obiettivi scientifici del progetto MicroBot vanno oltre la realizzazione di un sistema tecnologico, proponendosi di contribuire alla comprensione dei principi che governano i sistemi complessi e la materia dinamica. Il progetto si configura quindi non solo come un prototipo ingegneristico, ma come una piattaforma di ricerca in cui teoria e pratica si integrano in modo continuo, permettendo di esplorare nuove frontiere nella robotica e nelle scienze dei sistemi.

1.9 Obiettivi imprenditoriali del progetto

Gli obiettivi imprenditoriali del progetto MicroBot riguardano la trasformazione del sistema da prototipo sperimentale a piattaforma tecnologica scalabile, capace di generare valore economico, applicativo e innovativo. In questo contesto, il progetto non viene considerato esclusivamente come un esercizio accademico o tecnico, ma come una base concreta per lo sviluppo di un ecosistema imprenditoriale fondato su tecnologie emergenti.

Un primo obiettivo consiste nella definizione di MicroBot come piattaforma modulare. Il sistema è progettato in modo tale da poter essere esteso, adattato e personalizzato per diversi contesti applicativi, mantenendo una struttura di base comune. Questo approccio consente di sviluppare prodotti e servizi differenti partendo da una stessa architettura, riducendo i costi di sviluppo e aumentando la flessibilità del sistema. La modularità rappresenta quindi un elemento chiave per la sostenibilità economica del progetto nel lungo periodo.

Un secondo obiettivo riguarda l'individuazione di ambiti applicativi ad alto valore. Il concetto di materia programmabile e di sistemi riconfigurabili apre possibilità in diversi settori, tra cui la robotica avanzata, la medicina, la logistica e la realtà virtuale. In ambito robotico, il sistema può essere utilizzato per la creazione di strutture dinamiche e adattive; in ambito medico, può essere esplorato per applicazioni di micro-intervento o trasporto mirato; nel contesto della realtà virtuale e aumentata, può fungere da interfaccia fisica dinamica tra utente e ambiente digitale. L'identificazione di questi ambiti è fondamentale per orientare lo sviluppo del progetto verso applicazioni concrete e sostenibili.

Un ulteriore obiettivo imprenditoriale è la costruzione di un prototipo dimostrativo efficace. Per attrarre investimenti e interesse industriale, è necessario dimostrare in modo chiaro e tangibile le capacità del sistema. In questo senso, le versioni prototipali basate su simulazioni software e sistemi LED rappresentano un primo passo, ma devono evolvere verso implementazioni fisiche in grado di evidenziare il potenziale della tecnologia. Il prototipo diventa quindi uno strumento di comunicazione oltre che di validazione tecnica.

Dal punto di vista organizzativo, il progetto prevede la formazione di un team interdisciplinare. La natura complessa del sistema richiede competenze in ambiti diversi, tra cui elettronica, informatica, modellazione matematica e design dell'interazione. La costruzione di un team con competenze complementari consente di affrontare le diverse sfide del progetto in modo efficace, favorendo lo sviluppo di soluzioni integrate e innovative.

Un aspetto centrale degli obiettivi imprenditoriali è la definizione di un modello di business sostenibile. Il valore del sistema MicroBot può essere espresso attraverso diverse modalità, come la vendita di hardware, la fornitura di piattaforme software, o lo sviluppo di soluzioni personalizzate per specifici clienti. Inoltre, la possibilità di creare un ecosistema aperto, in cui sviluppatori esterni possano contribuire con moduli e applicazioni, rappresenta un'opportunità per ampliare la diffusione del sistema e generare nuove fonti di valore.

Un altro elemento rilevante è la protezione e valorizzazione della proprietà intellettuale. Le soluzioni sviluppate all'interno del progetto, sia a livello hardware che software, possono costituire la base per brevetti o altre forme di tutela, contribuendo a rafforzare la posizione competitiva del sistema. Allo stesso tempo, la pubblicazione di risultati scientifici e la condivisione di parte delle tecnologie possono favorire la visibilità del progetto e l'interesse della comunità.

Il progetto si propone inoltre di costruire una narrazione coerente e convincente, in grado di comunicare efficacemente il valore della tecnologia. La capacità di presentare il sistema in modo chiaro, sia a livello

tecnico che concettuale, è fondamentale per coinvolgere investitori, partner e collaboratori. In questo senso, la documentazione, le demo interattive e le presentazioni svolgono un ruolo strategico nel processo di sviluppo imprenditoriale.

Infine, un obiettivo a lungo termine è la creazione di una startup basata sul sistema MicroBot. Questa entità avrebbe il compito di sviluppare, commercializzare e distribuire la tecnologia, trasformando il progetto in un prodotto reale. La startup rappresenterebbe il punto di convergenza tra ricerca, sviluppo e mercato, permettendo di portare il sistema da una fase sperimentale a una fase operativa.

In conclusione, gli obiettivi imprenditoriali del progetto MicroBot delineano un percorso che va oltre la realizzazione tecnica, puntando alla creazione di un sistema capace di generare impatto reale. Il progetto si configura quindi come una piattaforma con potenziale di sviluppo industriale, in cui innovazione tecnologica e visione imprenditoriale si integrano per dare forma a nuove opportunità nel campo dei sistemi riconfigurabili e della materia programmabile.

1.10 Evoluzione dell'idea nel tempo

L'evoluzione dell'idea alla base del progetto MicroBot rappresenta un processo progressivo e iterativo, caratterizzato da una continua trasformazione del modello concettuale e delle soluzioni tecniche adottate. Questo percorso non segue una linea rigidamente predefinita, ma si sviluppa attraverso fasi successive di sperimentazione, validazione e riformulazione, in cui ogni versione del sistema contribuisce a ridefinire la direzione del progetto.

In una fase iniziale, l'idea si configura come un sistema relativamente semplice, basato su unità elementari che simulano comportamenti coordinati attraverso segnali visivi, come nel caso dei prototipi LED. Queste prime implementazioni hanno lo scopo di rendere osservabili dinamiche altrimenti astratte, permettendo di studiare pattern collettivi e logiche di sincronizzazione in un contesto controllato. In questa fase, l'attenzione è focalizzata principalmente sul comportamento del sistema, piuttosto che sulla sua realizzazione fisica completa.

Successivamente, il progetto evolve verso una maggiore strutturazione sia a livello hardware che software. L'introduzione di microcontrollori come l'ESP32 consente di passare da semplici simulazioni a sistemi embedded reali, in cui ogni unità è in grado di eseguire codice, comunicare e gestire il proprio stato. Questo passaggio segna una transizione importante: il sistema non è più soltanto una rappresentazione, ma diventa un oggetto fisico dotato di capacità operative.

Parallelamente, si sviluppa una crescente attenzione verso l'architettura del sistema. Il modello iniziale, basato su comportamenti locali relativamente indipendenti, viene progressivamente integrato con un livello di coordinamento centrale, rappresentato dal controller. Questa scelta introduce una struttura ibrida, in cui coesistono elementi distribuiti e centralizzati, permettendo di migliorare la sincronizzazione e il controllo delle operazioni senza perdere i vantaggi della modularità.

Un'altra fase significativa dell'evoluzione riguarda l'integrazione tra simulazione e implementazione reale. L'utilizzo di ambienti software, come simulazioni in Python o applicazioni web, permette di testare rapidamente nuove idee e algoritmi, riducendo il costo e il tempo necessari per la sperimentazione. Queste simulazioni non sostituiscono il sistema fisico, ma lo affiancano, creando un ciclo continuo di validazione tra modello teorico e comportamento reale.

Con il progredire del progetto, emerge anche una maggiore formalizzazione dei concetti alla base del sistema. Idee inizialmente intuitive, come l'interazione tra unità o la formazione di pattern, vengono progressivamente tradotte in modelli matematici e strutture logiche più definite. Questo processo consente di rendere il sistema non solo funzionante, ma anche descrivibile e analizzabile in modo rigoroso.

Un ulteriore elemento di evoluzione è rappresentato dall'ampliamento del contesto progettuale. Il progetto MicroBot non rimane isolato, ma si integra con una serie di iniziative parallele, tra cui lo sviluppo di interfacce web, sistemi di simulazione avanzati, algoritmi di intelligenza artificiale e strumenti di visualizzazione. Questa espansione trasforma il progetto in un ecosistema, in cui diverse componenti contribuiscono a un obiettivo comune.

Nel tempo, cambia anche la prospettiva con cui il sistema viene interpretato. Da semplice progetto tecnico, MicroBot si evolve in un modello concettuale più ampio, che include elementi di teoria dei sistemi complessi, materia programmabile e rappresentazione dinamica della forma. Questo passaggio segna una maturazione

del progetto, che non si limita più alla realizzazione di un dispositivo, ma si configura come una piattaforma di ricerca e sperimentazione.

Infine, l'evoluzione dell'idea è caratterizzata da una crescente consapevolezza delle potenzialità future del sistema. Le versioni successive non sono più viste come semplici miglioramenti incrementali, ma come passi verso una visione più ampia, in cui il sistema può essere esteso, adattato e applicato in contesti diversi. Questo orientamento verso il futuro guida le scelte progettuali, favorendo soluzioni flessibili e aperte all'integrazione di nuove tecnologie.

In conclusione, l'evoluzione del progetto MicroBot rappresenta un processo dinamico in cui teoria, sperimentazione e implementazione si influenzano reciprocamente. Ogni fase contribuisce a definire non solo cosa il sistema è, ma anche cosa può diventare, rendendo il progetto un esempio di sviluppo iterativo orientato alla costruzione di un sistema complesso e in continua trasformazione.

1.11 Trasformazione da prototipo a ecosistema

La trasformazione del progetto MicroBot da prototipo a ecosistema rappresenta una delle fasi più significative del suo sviluppo, in quanto segna il passaggio da un insieme di componenti sperimentali a un sistema strutturato, integrato e in continua evoluzione. Questo processo non è semplicemente un'evoluzione quantitativa, legata all'aggiunta di nuove funzionalità, ma implica un cambiamento qualitativo nella natura stessa del progetto.

Nelle fasi iniziali, il sistema si configura come un prototipo, ovvero un insieme di implementazioni parziali finalizzate alla validazione di specifiche idee. I prototipi LED, le prime simulazioni software e le implementazioni embedded su microcontrollore rappresentano esempi di questa fase, in cui l'obiettivo principale è testare singoli aspetti del sistema, come la sincronizzazione, la comunicazione o la formazione di pattern. In questo contesto, le diverse componenti sono spesso sviluppate in modo indipendente, senza una completa integrazione tra loro.

Con il progredire del progetto, emerge la necessità di unificare queste componenti all'interno di una struttura coerente. La semplice coesistenza di moduli separati non è sufficiente a garantire il funzionamento del sistema nel suo complesso; è necessario definire un'architettura che permetta alle diverse parti di interagire in modo coordinato. Questo porta alla definizione di un sistema a livelli, in cui ogni componente ha un ruolo specifico e interfacce ben definite.

In questa fase, il controller centrale assume un ruolo fondamentale come elemento di coordinamento, mentre i MicroBot operano come nodi distribuiti all'interno del sistema. Le simulazioni software, le interfacce web e i moduli di visualizzazione non sono più strumenti isolati, ma diventano parte integrante del sistema, contribuendo alla sua comprensione, al suo controllo e alla sua evoluzione.

La trasformazione in ecosistema implica anche un cambiamento nel modo in cui il progetto viene sviluppato e gestito. Non si tratta più di implementare singole funzionalità, ma di costruire un'infrastruttura in grado di supportare lo sviluppo continuo. Questo richiede l'adozione di pratiche come la modularità, la separazione delle responsabilità e la definizione di standard di comunicazione tra i componenti.

Un aspetto centrale di questa trasformazione è l'integrazione tra hardware e software. Nel sistema MicroBot, le componenti fisiche e quelle digitali non sono indipendenti, ma strettamente interconnesse. Le decisioni software influenzano direttamente il comportamento fisico delle unità, mentre i vincoli hardware determinano le possibilità e i limiti del sistema. L'ecosistema si configura quindi come un sistema cyber-fisico, in cui informazione e materia interagiscono in modo continuo.

Inoltre, il concetto di ecosistema introduce l'idea di apertura e estensibilità. Il sistema non è chiuso, ma progettato per accogliere nuove componenti, nuove funzionalità e nuovi contributi. Questo può includere l'integrazione di sensori avanzati, algoritmi di intelligenza artificiale, interfacce uomo-macchina basate su realtà virtuale o aumentata, e moduli di analisi dei dati. L'ecosistema diventa quindi una piattaforma su cui è possibile costruire ulteriori sviluppi.

Un altro elemento distintivo è la presenza di diversi livelli di astrazione. A un livello basso si trovano le componenti hardware e le interazioni fisiche; a un livello intermedio si collocano i protocolli di comunicazione e le logiche di controllo; a un livello alto si trovano le interfacce utente e i sistemi di visualizzazione. Questa stratificazione consente di gestire la complessità del sistema e di sviluppare nuove funzionalità senza dover intervenire su tutte le componenti.

La trasformazione in ecosistema comporta anche una maggiore attenzione alla documentazione e alla rappresentazione del sistema. La creazione di documenti tecnici, relazioni e modelli descrittivi diventa parte integrante del progetto, in quanto consente di formalizzare le conoscenze acquisite e di facilitare la collaborazione tra diversi attori. La documentazione non è quindi un elemento accessorio, ma uno strumento fondamentale per la crescita del sistema.

Infine, il passaggio da prototipo a ecosistema implica una visione a lungo termine. Il sistema non è più considerato come un prodotto finito, ma come una piattaforma in evoluzione, capace di adattarsi a nuove esigenze e di incorporare nuove tecnologie. Questo orientamento permette di mantenere il progetto aperto e dinamico, evitando di vincolarlo a una configurazione specifica.

In conclusione, la trasformazione del progetto MicroBot in un ecosistema segna il passaggio da una fase esplorativa a una fase strutturata e orientata allo sviluppo continuo. Il sistema diventa una piattaforma integrata in cui hardware, software e teoria convergono, creando le basi per ulteriori evoluzioni sia in ambito tecnico che scientifico e imprenditoriale.

SEZIONE 2 — Struttura delle Cartelle e dello ZIP

2.1 Analisi della cartella principale (workspace ZIP)

La cartella principale del workspace MicroBot rappresenta il punto di accesso all'intero sistema progettuale e costituisce la base organizzativa su cui si sviluppano tutte le componenti del progetto. Essa non si limita a contenere file e directory, ma riflette direttamente la struttura logica, tecnica e concettuale del sistema, fungendo da mappa operativa dell'intero ecosistema.

Il contenuto dello ZIP è organizzato secondo una logica modulare, in cui le diverse aree del progetto sono separate in base alla loro funzione e al dominio di appartenenza. Questa suddivisione consente di isolare i vari livelli del sistema, facilitando sia la comprensione che lo sviluppo incrementale. La presenza di cartelle dedicate a specifiche componenti, come codice sorgente, simulazioni e documentazione, permette di mantenere una chiara distinzione tra le diverse attività progettuali.

Un elemento centrale della cartella principale è la distinzione tra il core del sistema MicroBot e i progetti esterni o complementari. Il core comprende tutte le componenti direttamente coinvolte nella realizzazione del sistema, come il firmware dei MicroBot, il controller centrale e i moduli di comunicazione. I progetti esterni, invece, includono strumenti di supporto, simulazioni, interfacce web e altri sviluppi paralleli che contribuiscono all'ecosistema ma non ne costituiscono il nucleo operativo.

Dal punto di vista organizzativo, la struttura della cartella principale segue un principio di separazione delle responsabilità. Ogni directory è progettata per contenere elementi omogenei, riducendo la complessità e migliorando la navigabilità del progetto. Questo approccio è particolarmente importante in un sistema in continua evoluzione, in cui nuove componenti vengono aggiunte nel tempo.

Un altro aspetto rilevante è la presenza di una documentazione integrata all'interno dello stesso workspace. File come relazioni tecniche, appunti e documenti descrittivi non sono separati dal codice, ma coesistono con esso, creando un ambiente in cui teoria e implementazione sono strettamente collegate. Questa integrazione facilita la comprensione del sistema, permettendo di accedere rapidamente sia alle specifiche concettuali che alle relative implementazioni.

La cartella principale svolge inoltre un ruolo fondamentale nel versionamento implicito del progetto. Anche in assenza di un sistema di controllo versione esplicito, come Git, l'organizzazione delle directory e la presenza di versioni multiple di uno stesso componente permettono di tracciare l'evoluzione del sistema nel tempo. Le diverse iterazioni del progetto, identificate attraverso nomi o strutture di cartelle differenti, rappresentano le varie fasi di sviluppo e consentono di analizzare il percorso evolutivo del sistema.

Dal punto di vista operativo, la cartella principale è progettata per essere utilizzata come ambiente di lavoro completo. Essa contiene tutti gli elementi necessari per sviluppare, testare e comprendere il sistema, evitando la dipendenza da risorse esterne. Questo approccio rende il progetto portatile e facilmente condivisibile, permettendo ad altri utenti di accedere all'intero ecosistema semplicemente attraverso lo ZIP.

Un ulteriore elemento di interesse è la presenza di file di indice, come ad esempio un INDEX.txt, che fungono da guida alla navigazione del progetto. Questi file forniscono una visione d'insieme della struttura

del workspace, facilitando l'orientamento all'interno di un sistema complesso e riducendo il tempo necessario per individuare specifiche componenti.

Infine, la cartella principale riflette il carattere evolutivo del progetto MicroBot. La sua struttura non è statica, ma si adatta nel tempo in funzione delle esigenze del sistema. Nuove directory possono essere aggiunte, altre riorganizzate, in un processo continuo di ottimizzazione che accompagna lo sviluppo del progetto.

In conclusione, l'analisi della cartella principale evidenzia come l'organizzazione del workspace non sia un aspetto secondario, ma una componente fondamentale del sistema. Essa rappresenta la traduzione concreta dell'architettura progettuale in una struttura operativa, permettendo di gestire la complessità del progetto e di supportarne l'evoluzione nel tempo.

2.2 Struttura generale del sistema di file

La struttura generale del sistema di file del progetto MicroBot rappresenta la traduzione operativa dell'architettura concettuale del sistema. Essa definisce come le diverse componenti vengono organizzate, collegate e rese accessibili all'interno del workspace, permettendo di gestire un insieme eterogeneo di risorse in modo coerente e scalabile.

Il sistema di file è organizzato secondo una gerarchia multilivello, in cui ogni livello rappresenta un diverso grado di astrazione. Al livello più alto si trovano le macro-categorie del progetto, che suddividono il sistema in domini principali, come il core MicroBot, i progetti di supporto, la documentazione e gli ambienti di simulazione. Questa suddivisione consente di ottenere una visione globale del sistema, facilitando l'individuazione delle aree di interesse.

A un livello intermedio, ogni macro-categoria è ulteriormente suddivisa in sottocartelle che rappresentano componenti specifiche. Ad esempio, all'interno del core MicroBot possono essere presenti directory dedicate al firmware dei nodi, al controller centrale e ai moduli di comunicazione. Allo stesso modo, le cartelle relative ai progetti di supporto possono contenere applicazioni web, script di analisi o strumenti di visualizzazione.

Al livello più basso si trovano i file effettivi, che includono codice sorgente, configurazioni, documenti e risorse multimediali. Questa struttura a più livelli consente di isolare i diversi elementi del sistema, riducendo le interferenze tra componenti e migliorando la manutenibilità del progetto. Ogni file è collocato in una posizione che riflette il suo ruolo all'interno del sistema, evitando ambiguità e duplicazioni.

Un aspetto fondamentale della struttura del sistema di file è la coerenza nella nomenclatura. I nomi delle cartelle e dei file seguono convenzioni che permettono di identificare rapidamente il contenuto e la funzione di ciascun elemento. Questo è particolarmente importante in un progetto complesso, in cui la chiarezza organizzativa contribuisce in modo significativo alla produttività e alla riduzione degli errori.

La struttura del sistema di file supporta inoltre lo sviluppo parallelo di diverse componenti. Grazie alla separazione delle aree progettuali, è possibile lavorare su moduli differenti senza compromettere il funzionamento del sistema complessivo. Questo approccio è coerente con la natura modulare del progetto e favorisce l'espansione futura del sistema.

Un altro elemento rilevante è la presenza di file di configurazione e di avvio, che permettono di eseguire o testare specifiche parti del sistema. Questi file fungono da punto di ingresso per le diverse componenti, consentendo di avviare simulazioni, eseguire script o compilare codice in modo strutturato. La loro collocazione all'interno della struttura del progetto è progettata per facilitare l'accesso e l'utilizzo.

Dal punto di vista della tracciabilità, la struttura del sistema di file consente di seguire l'evoluzione delle diverse componenti nel tempo. La presenza di versioni multiple, file di backup o directory dedicate a specifiche iterazioni del progetto permette di mantenere uno storico delle modifiche, anche in assenza di un sistema di versionamento formale. Questo aspetto è particolarmente utile per analizzare il processo di sviluppo e per recuperare versioni precedenti in caso di necessità.

Inoltre, la struttura del sistema di file riflette l'integrazione tra le diverse tecnologie utilizzate nel progetto. Linguaggi e ambienti differenti, come C/C++ per i sistemi embedded, Python per le simulazioni e HTML/CSS/JavaScript per le interfacce web, coesistono all'interno dello stesso workspace. La loro organizzazione in cartelle dedicate consente di gestire questa eterogeneità in modo ordinato e funzionale.

Infine, la struttura del sistema di file è progettata per essere facilmente estendibile. Nuove cartelle e nuovi file possono essere aggiunti senza compromettere l'organizzazione esistente, grazie alla modularità della struttura. Questo permette al progetto di crescere nel tempo mantenendo un alto livello di ordine e coerenza.

In conclusione, la struttura generale del sistema di file del progetto MicroBot costituisce un elemento essenziale per la gestione della complessità del sistema. Essa non solo organizza le risorse, ma riflette l'architettura e la filosofia del progetto, supportando lo sviluppo, la manutenzione e l'evoluzione dell'intero ecosistema.

2.3 Organizzazione per domini progettuali

L'organizzazione per domini progettuali all'interno del sistema MicroBot rappresenta una scelta architeturale fondamentale per la gestione della complessità e per lo sviluppo scalabile dell'intero ecosistema. In un progetto caratterizzato dalla presenza di molteplici tecnologie, linguaggi e obiettivi, la suddivisione per domini consente di strutturare il lavoro in aree funzionalmente omogenee, riducendo le interdipendenze e migliorando la comprensione globale del sistema.

Un dominio progettuale può essere definito come un insieme coerente di componenti che condividono uno stesso obiettivo, una stessa logica operativa o uno stesso contesto tecnologico. Nel caso del progetto MicroBot, i domini principali emergono in modo naturale dall'evoluzione del sistema e riflettono le diverse dimensioni del progetto: hardware, software, simulazione, interfaccia e sperimentazione.

Il primo dominio, centrale e prioritario, è rappresentato dal core MicroBot. Esso include tutte le componenti direttamente coinvolte nella realizzazione del sistema robotico distribuito, come il firmware dei MicroBot, il controller centrale e i moduli di comunicazione. Questo dominio costituisce il nucleo operativo del progetto, in cui vengono implementate le logiche fondamentali di funzionamento, come la gestione degli stati, la sincronizzazione e il controllo delle interazioni tra le unità.

Un secondo dominio riguarda la simulazione e la modellazione del sistema. In questa area rientrano gli ambienti software sviluppati per rappresentare il comportamento dello swarm, testare algoritmi e visualizzare le dinamiche collettive. Le simulazioni in Python, le implementazioni web e i sistemi di visualizzazione costituiscono strumenti essenziali per l'analisi del comportamento emergente e per la validazione delle ipotesi progettuali. Questo dominio svolge un ruolo di ponte tra teoria e implementazione, permettendo di esplorare soluzioni prima della loro realizzazione fisica.

Un ulteriore dominio è quello delle interfacce e della comunicazione uomo-macchina. Questo include applicazioni web, sistemi di visualizzazione interattiva e potenziali integrazioni con ambienti di realtà virtuale o aumentata. In questo contesto, il sistema MicroBot non è visto solo come un insieme di unità fisiche, ma come un sistema interattivo, in cui l'utente può osservare, controllare e influenzare il comportamento dello swarm. Questo dominio è fondamentale per rendere il sistema accessibile e comprensibile anche a utenti non tecnici.

Accanto a questi, si sviluppa un dominio dedicato agli strumenti e ai progetti di supporto. Esso comprende script diagnostici, tool di analisi, esperimenti con algoritmi e progetti paralleli che, pur non essendo direttamente parte del core MicroBot, contribuiscono allo sviluppo delle competenze e alla costruzione dell'ecosistema. Esempi di questo dominio includono progetti di intelligenza artificiale, simulazioni evolutive, strumenti di analisi dati e applicazioni sperimentali.

Un altro dominio rilevante è quello della documentazione e della formalizzazione. In esso rientrano relazioni tecniche, appunti, documenti strutturati e materiali descrittivi che accompagnano lo sviluppo del sistema. Questo dominio svolge un ruolo cruciale nel rendere il progetto comprensibile, replicabile e comunicabile, sia in ambito accademico che imprenditoriale.

L'organizzazione per domini non implica una separazione rigida tra le componenti, ma piuttosto una struttura flessibile in cui i diversi domini interagiscono tra loro. Ad esempio, le simulazioni possono influenzare lo sviluppo del firmware, mentre le interfacce utente possono essere utilizzate per controllare il sistema fisico. Questa interconnessione riflette la natura integrata del progetto, in cui le diverse aree contribuiscono a un obiettivo comune.

Dal punto di vista metodologico, la suddivisione per domini consente di adottare strategie di sviluppo parallelo e incrementale. Ogni dominio può evolvere indipendentemente, pur mantenendo una coerenza con

l'architettura generale del sistema. Questo approccio favorisce la sperimentazione, permettendo di testare nuove idee in contesti isolati prima di integrarle nel sistema principale.

Infine, l'organizzazione per domini progettuali rappresenta una base solida per l'espansione futura del sistema. Nuovi domini possono essere aggiunti nel tempo, in funzione delle esigenze e delle opportunità che emergono durante lo sviluppo. Questo rende il progetto MicroBot non solo un sistema strutturato, ma anche un ambiente aperto, in grado di adattarsi e crescere nel tempo.

In conclusione, la suddivisione del sistema MicroBot in domini progettuali consente di gestire la complessità in modo efficace, mantenendo al tempo stesso un alto grado di flessibilità e integrazione. Essa rappresenta una scelta strategica che supporta lo sviluppo del progetto su più livelli, favorendo la collaborazione, la scalabilità e l'evoluzione continua dell'ecosistema.

2.4 Separazione tra MicroBot e progetti esterni

All'interno del workspace MicroBot, la separazione tra il core del sistema e i progetti esterni rappresenta un principio organizzativo fondamentale, volto a distinguere le componenti direttamente coinvolte nel funzionamento del sistema da quelle che svolgono un ruolo di supporto, sperimentazione o estensione. Questa distinzione non è soltanto formale, ma riflette una differenza sostanziale nella funzione e nell'impatto delle diverse parti del progetto.

Il core MicroBot comprende tutte le componenti essenziali per il funzionamento del sistema robotico distribuito. In questa categoria rientrano il firmware dei MicroBot, il controller centrale, i protocolli di comunicazione e le logiche di coordinamento dello swarm. Questi elementi costituiscono l'infrastruttura principale del sistema e rappresentano il nucleo tecnologico su cui si basa l'intero progetto. Qualsiasi modifica o evoluzione del core ha un impatto diretto sul comportamento del sistema e sulla sua capacità operativa.

I progetti esterni, invece, includono una serie di sviluppi paralleli che, pur non essendo indispensabili per il funzionamento del sistema, contribuiscono in modo significativo alla sua comprensione, al suo sviluppo e alla sua presentazione. Questi progetti possono essere considerati come strumenti di supporto o come estensioni dell'ecosistema, e comprendono simulazioni software, interfacce web, applicazioni sperimentali, algoritmi di intelligenza artificiale e tool diagnostici.

La distinzione tra core ed esterno è particolarmente importante per la gestione della complessità. Separando le componenti critiche da quelle sperimentali, è possibile lavorare su nuove idee e funzionalità senza compromettere la stabilità del sistema principale. Questo approccio consente di mantenere un equilibrio tra innovazione e affidabilità, permettendo al progetto di evolversi in modo controllato.

Dal punto di vista architettureale, questa separazione si traduce in una struttura del sistema di file in cui le cartelle relative al core sono chiaramente distinte da quelle dedicate ai progetti esterni. Le directory del core sono generalmente più stabili, con una struttura definita e una maggiore attenzione alla qualità del codice e alla documentazione. Al contrario, le cartelle dei progetti esterni possono essere più dinamiche, riflettendo la natura sperimentale e iterativa di queste componenti.

Un aspetto rilevante è il ruolo dei progetti esterni come ambiente di sperimentazione. Molte delle idee sviluppate in questi contesti possono essere successivamente integrate nel core, dopo essere state validate e ottimizzate. In questo senso, i progetti esterni fungono da laboratorio, in cui è possibile testare nuove soluzioni senza i vincoli imposti dal sistema principale.

Inoltre, i progetti esterni svolgono un ruolo importante nella comunicazione e nella valorizzazione del sistema. Le simulazioni, le demo interattive e le interfacce visive permettono di rappresentare il funzionamento del sistema in modo intuitivo, facilitando la comprensione da parte di utenti non tecnici e potenziali stakeholder. Questo aspetto è particolarmente rilevante in un contesto imprenditoriale, in cui la capacità di presentare il progetto in modo efficace è fondamentale.

Dal punto di vista dello sviluppo delle competenze, i progetti esterni rappresentano anche un'opportunità per esplorare nuove tecnologie e metodologie. Lavorare su ambiti diversi, come l'intelligenza artificiale, la visualizzazione o lo sviluppo web, consente di ampliare le conoscenze e di arricchire il progetto con nuove prospettive. Questa diversificazione contribuisce alla crescita complessiva dell'ecosistema.

È importante sottolineare che, nonostante la separazione, esiste una forte interconnessione tra il core e i progetti esterni. Le informazioni e le soluzioni sviluppate in un ambito possono influenzare gli altri, creando un flusso continuo di conoscenza all'interno del sistema. Questa relazione dinamica rappresenta uno degli elementi distintivi del progetto MicroBot, che si configura come un sistema aperto e in evoluzione.

Infine, la distinzione tra core e progetti esterni assume un ruolo strategico anche in prospettiva futura. Essa permette di definire in modo chiaro quali componenti costituiscono il prodotto principale e quali invece rappresentano strumenti di supporto o possibili linee di sviluppo indipendenti. Questo è particolarmente utile in un contesto imprenditoriale, in cui è necessario identificare il valore centrale del sistema e le sue possibili estensioni.

In conclusione, la separazione tra MicroBot e progetti esterni consente di organizzare il workspace in modo efficace, mantenendo una chiara distinzione tra le componenti critiche e quelle sperimentali. Questa scelta favorisce lo sviluppo modulare, la sperimentazione controllata e la crescita dell'ecosistema, contribuendo a rendere il progetto più robusto, flessibile e orientato al futuro.

2.5 Cartelle di documentazione tecnica

Le cartelle di documentazione tecnica all'interno del workspace MicroBot rappresentano un elemento fondamentale per la comprensione, la formalizzazione e la trasmissione del sistema. Esse costituiscono il punto di incontro tra l'attività progettuale e la sua rappresentazione strutturata, permettendo di trasformare idee, esperimenti e implementazioni in contenuti descrittivi coerenti e analizzabili. In questo contesto, la documentazione non assume un ruolo accessorio, ma diventa parte integrante dell'architettura del progetto stesso, contribuendo alla sua scalabilità, riproducibilità e comunicabilità.

All'interno di tali cartelle vengono raccolti documenti eterogenei, organizzati secondo criteri gerarchici e tematici, che riflettono le diverse dimensioni del sistema MicroBot. Tra questi si includono descrizioni architetture, specifiche tecniche, modelli matematici, analisi dei componenti hardware e software, protocolli di comunicazione, diagrammi di flusso, nonché report sperimentali e simulativi. Questa organizzazione consente di mantenere una chiara separazione tra i livelli concettuali e operativi del progetto, facilitando sia la consultazione che l'aggiornamento continuo delle informazioni.

Dal punto di vista metodologico, le cartelle di documentazione tecnica svolgono una funzione di standardizzazione del sapere prodotto. Ogni documento segue una struttura coerente, spesso articolata in sezioni numerate, che consente di mantenere uniformità espositiva e rigore formale. Questo approccio favorisce non solo la leggibilità, ma anche l'integrazione progressiva di nuovi contenuti, rendendo il sistema documentale modulare ed estendibile nel tempo. La presenza di indici, nomenclature e riferimenti incrociati tra documenti permette inoltre di costruire una rete informativa interconnessa, in cui ogni elemento contribuisce alla comprensione globale del sistema.

Un ulteriore aspetto rilevante riguarda la funzione di tracciabilità e versionamento. Le cartelle documentali permettono di registrare l'evoluzione del progetto, documentando le scelte progettuali, le modifiche apportate e i risultati ottenuti nelle diverse fasi di sviluppo. Ciò risulta particolarmente importante in un contesto come quello di MicroBot, caratterizzato da iterazioni continue e da un forte legame tra simulazione, prototipazione e implementazione reale. La documentazione diventa quindi uno strumento di memoria tecnica, indispensabile per evitare perdita di informazioni e per garantire coerenza tra le diverse componenti del sistema.

Infine, tali cartelle svolgono un ruolo chiave nella comunicazione verso l'esterno. Esse costituiscono la base per la redazione di relazioni formali, presentazioni tecniche, materiali divulgativi e pitch progettuali. Una documentazione ben strutturata consente infatti di tradurre la complessità del sistema MicroBot in un linguaggio accessibile e rigoroso, adattabile a diversi interlocutori, come collaboratori tecnici, revisori accademici o potenziali investitori. In questo senso, le cartelle di documentazione tecnica non sono soltanto un archivio interno, ma rappresentano un vero e proprio strumento strategico per la valorizzazione e la diffusione del progetto.

2.5.1 PDF, relazioni e tesi

All'interno delle cartelle di documentazione tecnica, la sottosezione dedicata a PDF, relazioni e tesi rappresenta il livello più formale e strutturato dell'intero sistema documentale MicroBot. In essa vengono raccolti tutti i documenti a carattere esteso e consolidato, concepiti non solo come supporto interno allo sviluppo, ma anche come strumenti di comunicazione tecnica verso l'esterno. Questi file costituiscono la

sintesi ad alto livello del progetto, integrando aspetti teorici, progettuali e sperimentali in un formato coerente, leggibile e adatto alla distribuzione.

I documenti PDF svolgono principalmente una funzione di cristallizzazione dello stato del sistema in un determinato momento evolutivo. A differenza dei file sorgente o delle note di sviluppo, che sono soggetti a modifiche continue, i PDF rappresentano versioni stabili e versionate del progetto, utilizzate per presentazioni ufficiali, revisioni accademiche o condivisione con collaboratori. Essi includono tipicamente descrizioni dell'architettura del sistema, schemi funzionali, analisi dei componenti, modelli matematici e risultati delle simulazioni, organizzati secondo una struttura rigorosa e progressiva.

Le relazioni tecniche, spesso incluse in formato PDF o in fase di conversione verso tale formato, costituiscono il nucleo descrittivo del progetto. Esse approfondiscono specifici aspetti del sistema MicroBot, come il comportamento dello swarm, il funzionamento del sistema magnetico, l'architettura elettronica o i protocolli di comunicazione. Ogni relazione è generalmente articolata in sezioni e sottosezioni numerate, con un linguaggio tecnico preciso e una forte attenzione alla chiarezza espositiva. Questo consente di trattare ogni componente del sistema come un modulo documentale autonomo, ma al tempo stesso integrato nel quadro complessivo.

Le tesi rappresentano il livello più avanzato e completo della documentazione, in cui il progetto viene analizzato in maniera sistematica e approfondita. In esse confluiscono tutti gli elementi sviluppati nelle fasi precedenti, riorganizzati secondo una logica accademica che include introduzione, stato dell'arte, metodologia, sviluppo del sistema, risultati e conclusioni. La presenza di una o più tesi all'interno del workspace MicroBot evidenzia la volontà di collocare il progetto non solo in un contesto ingegneristico, ma anche scientifico, rendendolo potenzialmente oggetto di valutazione, pubblicazione o ulteriore ricerca.

Un aspetto fondamentale di questa sottosezione è la coerenza strutturale tra i documenti. Nonostante la varietà dei contenuti, tutti i file tendono a seguire convenzioni comuni in termini di formattazione, numerazione delle sezioni, utilizzo di figure e tabelle, e modalità di citazione dei riferimenti interni. Questo approccio consente di costruire un corpus documentale uniforme, in cui ogni documento può essere facilmente navigato e compreso anche senza una conoscenza completa dell'intero sistema.

Infine, la presenza di PDF, relazioni e tesi all'interno del workspace svolge una funzione strategica per la valorizzazione del progetto. Essa permette di tradurre un insieme complesso di codice, simulazioni e prototipi in una narrazione tecnica strutturata, capace di evidenziare le scelte progettuali, i risultati ottenuti e il potenziale evolutivo del sistema MicroBot. In questo senso, tali documenti rappresentano non solo una forma di documentazione, ma anche uno strumento di legittimazione tecnica e scientifica del progetto nel suo complesso.

2.6 Cartelle di codice sorgente

Le cartelle di codice sorgente costituiscono il nucleo operativo del sistema MicroBot, rappresentando il livello in cui le astrazioni teoriche e progettuali vengono tradotte in implementazioni concrete. All'interno del workspace, esse raccolgono l'insieme dei programmi, degli script e dei moduli che definiscono il comportamento del sistema, sia nella sua componente simulativa che in quella fisica. Queste cartelle non svolgono unicamente una funzione esecutiva, ma rappresentano anche un'estensione diretta della documentazione tecnica, in quanto il codice stesso diventa un mezzo di formalizzazione delle logiche progettuali.

L'organizzazione delle cartelle di codice segue una suddivisione per domini tecnologici, che riflette la natura eterogenea del sistema MicroBot. Tale approccio consente di mantenere separati i diversi livelli di sviluppo, evitando interferenze tra ambienti con requisiti e finalità differenti. Le principali categorie includono progetti web, ambienti di simulazione in Python e moduli a basso livello sviluppati in C o per sistemi embedded, ciascuno con una propria struttura interna e convenzioni specifiche.

I progetti web rappresentano l'interfaccia visiva e interattiva del sistema. Essi includono applicazioni sviluppate in HTML, CSS e JavaScript, finalizzate alla visualizzazione dello swarm, alla simulazione dei comportamenti e alla presentazione del progetto. All'interno di queste cartelle sono presenti componenti per il rendering grafico, sistemi di animazione, dashboard interattive e moduli di controllo, che permettono all'utente di interagire con il sistema in tempo reale. Il codice web svolge quindi una duplice funzione: da un lato consente la prototipazione rapida dei comportamenti, dall'altro funge da strumento di comunicazione visiva.

Le cartelle dedicate a Python contengono invece gli ambienti di simulazione e i modelli computazionali del sistema. In esse vengono implementati gli algoritmi che regolano il comportamento dello swarm, come le regole di coesione, separazione e allineamento, i modelli di interazione tra unità e i sistemi di aggiornamento delle posizioni nello spazio. Python viene utilizzato per la sua flessibilità e per la rapidità di sviluppo, permettendo di testare e validare rapidamente ipotesi progettuali prima della loro eventuale implementazione su hardware reale. Questi ambienti rappresentano quindi un livello intermedio tra teoria e applicazione.

Le componenti sviluppate in C e nei sistemi embedded costituiscono il livello più vicino all'hardware. Esse includono il firmware dei MicroBot, la gestione delle risorse del microcontrollore, il controllo delle periferiche e l'implementazione dei protocolli di comunicazione a basso livello. In questo contesto, il codice è ottimizzato per efficienza, latenza e consumo energetico, e deve rispettare vincoli stringenti legati alle capacità del dispositivo. Questo livello rappresenta la concretizzazione fisica del sistema, in cui le logiche sviluppate negli ambienti superiori vengono adattate e integrate nel contesto reale.

Un elemento distintivo delle cartelle di codice sorgente è la loro evoluzione nel tempo. All'interno del workspace sono spesso presenti più versioni dello stesso progetto, ciascuna rappresentativa di una fase specifica dello sviluppo. Questo versionamento implicito, pur non sempre formalizzato tramite sistemi di controllo versione, consente di tracciare il percorso evolutivo del sistema e di mantenere accessibili le diverse iterazioni progettuali. Tale caratteristica è particolarmente rilevante in un contesto sperimentale, in cui il confronto tra versioni differenti può fornire indicazioni utili per il miglioramento del sistema.

Nel complesso, le cartelle di codice sorgente non devono essere intese come semplici contenitori di programmi, ma come un'infrastruttura dinamica in cui teoria, simulazione e implementazione convergono. Esse rappresentano il punto in cui il sistema MicroBot prende forma operativa, rendendo possibile l'esecuzione, la sperimentazione e l'evoluzione continua dell'intero progetto.

2.6.1 Progetti web

Le cartelle dedicate ai progetti web rappresentano il livello di interfaccia e visualizzazione del sistema MicroBot, costituendo il punto di contatto diretto tra l'utente e l'ecosistema tecnologico sviluppato. All'interno di queste cartelle vengono raccolti tutti i componenti necessari alla costruzione di ambienti interattivi, dashboard di monitoraggio, simulazioni visive e pagine di presentazione, realizzati principalmente mediante tecnologie standard del web quali HTML, CSS e JavaScript.

I progetti web svolgono una funzione cruciale nella traduzione delle logiche interne del sistema in rappresentazioni visive comprensibili. Attraverso grafici, animazioni e interfacce dinamiche, essi permettono di osservare il comportamento dello swarm in tempo reale, rendendo evidenti fenomeni che, a livello puramente numerico o algoritmico, risulterebbero difficilmente interpretabili. In questo senso, il livello web non è soltanto un supporto estetico, ma diventa uno strumento analitico e diagnostico.

Dal punto di vista architetturale, i progetti web sono spesso organizzati in moduli distinti, ciascuno con una responsabilità specifica. Si individuano tipicamente componenti per il rendering grafico, basati su canvas 2D o librerie di visualizzazione avanzata, moduli per la gestione degli input utente, sistemi di aggiornamento in tempo reale e strutture dati per la rappresentazione dello stato del sistema. Questa modularità consente di estendere facilmente le funzionalità dell'interfaccia e di adattarla a diversi scenari applicativi.

Un elemento caratterizzante di tali progetti è l'integrazione con i livelli superiori e inferiori del sistema. Le interfacce web possono infatti ricevere dati da simulazioni Python o da sistemi embedded, visualizzando parametri come posizione dei MicroBot, stato energetico, temperatura o pattern attivi. In alcuni casi, esse possono anche inviare comandi al sistema, assumendo il ruolo di pannelli di controllo attraverso cui l'utente modifica il comportamento dello swarm. Questa bidirezionalità rafforza il ruolo del web come nodo centrale nel flusso informativo del sistema.

Le cartelle web includono inoltre progetti con finalità differenti. Alcuni sono orientati alla simulazione, permettendo di testare e visualizzare algoritmi in ambienti controllati; altri hanno una funzione più espositiva, presentando il progetto MicroBot attraverso interfacce curate, animazioni avanzate e contenuti descrittivi. Questa distinzione evidenzia la doppia natura del livello web, che si colloca sia nel dominio dello sviluppo tecnico che in quello della comunicazione.

Dal punto di vista evolutivo, i progetti web mostrano spesso una progressione in termini di complessità e qualità visiva. Versioni iniziali possono essere caratterizzate da implementazioni semplici e funzionali, mentre iterazioni successive introducono miglioramenti estetici, maggiore interattività e ottimizzazioni delle

prestazioni. Questa evoluzione riflette il processo di affinamento del sistema nel suo complesso e contribuisce alla costruzione di un'identità visiva coerente per il progetto MicroBot.

In conclusione, le cartelle dei progetti web rappresentano un elemento chiave del workspace, in quanto permettono di rendere visibile, interattivo e comprensibile un sistema altrimenti astratto e distribuito. Esse costituiscono il ponte tra la complessità interna del sistema e la percezione esterna, facilitando sia l'analisi tecnica che la comunicazione del progetto.

2.6.2 Progetti Python

Le cartelle dedicate ai progetti Python rappresentano il livello computazionale e simulativo del sistema MicroBot, costituendo l'ambiente in cui vengono formalizzati e testati i modelli dinamici che regolano il comportamento dello swarm. In queste cartelle si sviluppa la transizione tra teoria e implementazione, permettendo di tradurre concetti matematici e logiche distribuite in algoritmi eseguibili e osservabili.

Python viene utilizzato come linguaggio principale per la sua elevata espressività, la disponibilità di librerie scientifiche e la rapidità di prototipazione. Ciò consente di costruire ambienti di simulazione flessibili, in cui è possibile modificare parametri, introdurre nuove regole e osservare immediatamente gli effetti sul sistema. Questo approccio iterativo è fondamentale per un progetto come MicroBot, caratterizzato da comportamenti emergenti difficilmente prevedibili a priori.

All'interno di queste cartelle si trovano tipicamente moduli dedicati alla rappresentazione dello spazio e delle unità. Ogni MicroBot viene modellato come un agente dotato di stato interno, posizione, velocità e insieme di regole comportamentali. Il sistema complessivo è quindi trattato come un sistema multi-agente, in cui l'evoluzione globale emerge dall'interazione locale tra le singole unità. Le simulazioni permettono di analizzare fenomeni quali aggregazione, dispersione, formazione di pattern e adattamento dinamico.

Un aspetto centrale dei progetti Python è l'implementazione dei modelli matematici descritti nella sezione teorica del sistema. Vengono utilizzate funzioni per il calcolo delle distanze, l'applicazione delle forze di interazione e l'aggiornamento discreto delle posizioni nel tempo. Le soglie di attacco e distacco, insieme ai meccanismi di hysteresis, vengono integrate negli algoritmi per simulare il comportamento reale dei MicroBot in presenza di forze magnetiche e vincoli fisici. Questo consente di validare le ipotesi progettuali prima della loro implementazione su hardware.

Le cartelle Python includono anche strumenti per la visualizzazione dei risultati. Attraverso librerie grafiche è possibile rappresentare lo stato dello swarm in tempo reale, osservando l'evoluzione delle configurazioni e identificando eventuali anomalie o instabilità. Queste visualizzazioni svolgono un ruolo fondamentale nel processo di debug e ottimizzazione, permettendo di individuare rapidamente comportamenti non desiderati.

Un ulteriore elemento rilevante è la modularità del codice. I progetti sono generalmente organizzati in sottocomponenti distinti, come moduli per la gestione degli agenti, funzioni di simulazione, sistemi di controllo e interfacce di output. Questa struttura facilita la riusabilità del codice e consente di integrare facilmente nuove funzionalità, come modelli più avanzati, algoritmi di ottimizzazione o sistemi di apprendimento automatico.

Dal punto di vista evolutivo, i progetti Python riflettono in modo diretto il processo di sperimentazione del sistema MicroBot. Essi vengono continuamente modificati, estesi e raffinati, rappresentando una sorta di laboratorio digitale in cui testare idee e approcci differenti. La presenza di più versioni o varianti dello stesso modello evidenzia la natura esplorativa del progetto e contribuisce alla costruzione di una base solida per le fasi successive di sviluppo.

In sintesi, le cartelle dei progetti Python costituiscono il cuore analitico del sistema MicroBot. Esse permettono di esplorare, validare e comprendere il comportamento dello swarm in un ambiente controllato, fungendo da ponte tra la formalizzazione teorica e l'implementazione pratica.

2.6.3 Progetti C e sistemi embedded

Le cartelle dedicate ai progetti in C e ai sistemi embedded rappresentano il livello più basso e critico dell'intero sistema MicroBot, in cui le logiche astratte e i modelli simulativi vengono tradotti in istruzioni direttamente eseguibili sull'hardware. In questo contesto, il codice non ha più una funzione esplorativa o descrittiva, ma assume un ruolo strettamente operativo, dovendo rispettare vincoli reali legati a prestazioni, consumo energetico, latenza e risorse limitate.

Il linguaggio C viene utilizzato per la sua efficienza e per il controllo diretto che offre sull'hardware. A differenza dei livelli superiori, in cui l'astrazione è elevata, qui il programmatore opera in prossimità delle periferiche, gestendo registri, pin, temporizzazioni e interruzioni. Questo consente di implementare funzionalità fondamentali per il funzionamento dei MicroBot, come il controllo delle bobine elettromagnetiche, la gestione dei LED, la lettura di eventuali sensori e la comunicazione con il controller centrale.

All'interno di queste cartelle si trova il firmware dei MicroBot, ovvero l'insieme delle istruzioni che definiscono il comportamento di ogni singola unità fisica. Il firmware è generalmente strutturato secondo un'architettura a stati, in cui il dispositivo può trovarsi in diverse modalità operative, come inattivo, in attesa di comandi, in esecuzione di un pattern o in fase di sicurezza. Questa organizzazione consente di mantenere il controllo del sistema anche in condizioni critiche, garantendo stabilità e prevedibilità.

Un aspetto centrale dei sistemi embedded è la gestione delle risorse. Il microcontrollore dispone di una quantità limitata di memoria, capacità di calcolo ed energia, e ogni operazione deve essere ottimizzata per minimizzare il consumo e massimizzare l'efficienza. Questo implica l'uso di strutture dati compatte, l'evitamento di operazioni costose e la gestione attenta dei cicli di esecuzione. In particolare, il controllo delle bobine elettromagnetiche richiede una regolazione precisa della corrente, che deve essere ottenuta attraverso tecniche come la modulazione PWM, evitando al contempo il surriscaldamento e il danneggiamento dei componenti.

Le cartelle embedded includono anche la gestione della comunicazione a basso livello. I MicroBot devono essere in grado di ricevere comandi e trasmettere informazioni attraverso protocolli leggeri, spesso basati su comunicazioni wireless a bassa latenza. Il codice implementa quindi la codifica e decodifica dei pacchetti, la gestione degli errori e la sincronizzazione con il controller centrale. Questo livello è fondamentale per garantire il coordinamento dello swarm e la coerenza delle azioni tra le diverse unità.

Un ulteriore elemento rilevante è l'integrazione con il sistema operativo embedded, ove presente. Nei casi in cui venga utilizzato un microkernel o un sistema operativo leggero, il codice deve interfacciarsi con meccanismi di scheduling, gestione dei task e astrazione dell'hardware. Questo introduce un ulteriore livello di complessità, ma consente di organizzare il firmware in maniera più modulare e scalabile.

Dal punto di vista evolutivo, i progetti in C e embedded rappresentano il passaggio definitivo dalla simulazione alla realtà fisica. Essi incorporano le conoscenze acquisite nei livelli superiori e le traducono in un sistema funzionante, soggetto a vincoli concreti e verificabile sperimentalmente. Le iterazioni su questo livello sono spesso più lente e richiedono test accurati, ma sono essenziali per la validazione finale del sistema.

In conclusione, le cartelle di progetti C e sistemi embedded costituiscono il fondamento operativo del sistema MicroBot. Esse permettono di trasformare modelli teorici e simulazioni in comportamenti reali, garantendo il funzionamento delle unità fisiche e l'integrazione dell'intero sistema nel mondo reale.

2.7 Cartelle di simulazione

Le cartelle di simulazione rappresentano un livello intermedio fondamentale all'interno del sistema MicroBot, collocandosi tra l'astrazione teorica e l'implementazione concreta. In esse vengono sviluppati e testati ambienti controllati che permettono di osservare il comportamento del sistema in condizioni virtuali, riducendo la complessità e i costi associati alla sperimentazione fisica. Questo livello consente di validare modelli, individuare criticità e ottimizzare le soluzioni prima della loro applicazione su hardware reale.

Le simulazioni svolgono una funzione essenzialmente predittiva e analitica. Attraverso la riproduzione digitale delle dinamiche del sistema, è possibile studiare l'evoluzione dello swarm, verificare la stabilità dei pattern e analizzare l'impatto di parametri come distanza, intensità delle forze, latenza di comunicazione e limiti energetici. Questo approccio permette di anticipare problemi che emergerebbero solo in fase di test fisico, migliorando l'efficienza del processo di sviluppo.

All'interno di queste cartelle si trovano ambienti di simulazione realizzati con diverse tecnologie, a seconda degli obiettivi specifici. Alcuni sono orientati alla visualizzazione e utilizzano interfacce grafiche per rappresentare i MicroBot nello spazio, mostrando in tempo reale le interazioni e i movimenti. Altri sono più focalizzati sull'analisi numerica, eseguendo simulazioni senza componente visiva ma producendo dati utili per lo studio del sistema. Questa varietà consente di affrontare il problema da prospettive differenti, integrando approcci qualitativi e quantitativi.

Un elemento chiave delle simulazioni è la possibilità di controllare in modo preciso le condizioni iniziali e i parametri del sistema. È possibile definire il numero di unità, la loro disposizione iniziale, le regole di interazione e i limiti operativi, osservando come il sistema evolve nel tempo. Questo livello di controllo consente di esplorare scenari difficilmente riproducibili nel mondo reale e di effettuare test sistematici su larga scala.

Le cartelle di simulazione includono inoltre strumenti per il confronto tra modelli diversi. Versioni alternative degli algoritmi possono essere testate in parallelo, permettendo di valutare quale soluzione produca risultati migliori in termini di stabilità, efficienza o adattabilità. Questo approccio comparativo è essenziale per guidare le scelte progettuali e per affinare progressivamente il sistema.

Dal punto di vista metodologico, la simulazione rappresenta un ciclo continuo di iterazione. I risultati ottenuti vengono analizzati, confrontati con le aspettative teoriche e utilizzati per modificare i modelli o i parametri. Questo processo iterativo consente di convergere verso soluzioni sempre più robuste e di costruire una comprensione approfondita del comportamento emergente dello swarm.

Infine, le cartelle di simulazione svolgono un ruolo importante anche nella comunicazione del progetto. Le rappresentazioni visive e le animazioni generate possono essere utilizzate per mostrare il funzionamento del sistema in modo intuitivo, facilitando la comprensione anche da parte di utenti non esperti. In questo senso, la simulazione diventa non solo uno strumento tecnico, ma anche un mezzo di divulgazione.

In sintesi, le cartelle di simulazione costituiscono un ambiente sperimentale virtuale in cui il sistema MicroBot può essere esplorato, testato e ottimizzato. Esse permettono di ridurre il divario tra teoria e pratica, fornendo un contesto controllato in cui sviluppare e validare le logiche alla base del comportamento dello swarm.

2.7.1 Ambienti di test

Gli ambienti di test all'interno delle cartelle di simulazione rappresentano il livello più controllato e sperimentale del sistema MicroBot, in cui specifiche funzionalità o componenti vengono isolati e analizzati in maniera sistematica. A differenza delle simulazioni complete, che mirano a riprodurre il comportamento globale dello swarm, gli ambienti di test si concentrano su porzioni limitate del sistema, permettendo un'analisi più precisa e mirata.

Questi ambienti sono progettati per verificare il corretto funzionamento di singoli moduli, come algoritmi di movimento, regole di interazione tra unità, sistemi di comunicazione o modelli di forza. Isolando tali componenti, è possibile individuare con maggiore facilità eventuali errori logici, instabilità o comportamenti inattesi, riducendo la complessità dell'analisi rispetto a un sistema completo. Questo approccio modulare consente di validare ogni parte del sistema prima della sua integrazione.

Un aspetto fondamentale degli ambienti di test è la definizione di scenari controllati. Vengono impostate condizioni iniziali specifiche, come il numero di MicroBot, la loro disposizione nello spazio e i parametri delle interazioni, al fine di osservare il comportamento del sistema in situazioni ben definite. Questo permette di riprodurre esperimenti in modo consistente e di confrontare i risultati ottenuti in diverse configurazioni.

Gli ambienti di test includono spesso strumenti per la raccolta e l'analisi dei dati. Durante l'esecuzione delle simulazioni, vengono registrati parametri come posizioni, velocità, distanze tra unità e stato interno degli agenti. Questi dati possono essere utilizzati per costruire grafici, identificare pattern ricorrenti e valutare quantitativamente le prestazioni del sistema. In questo modo, il processo di sviluppo si basa non solo su osservazioni qualitative, ma anche su evidenze misurabili.

Un ulteriore elemento rilevante è la possibilità di eseguire test ripetuti con variazioni controllate dei parametri. Attraverso questo approccio, è possibile analizzare la sensibilità del sistema rispetto a determinati fattori, come l'intensità delle forze, le soglie di interazione o la latenza della comunicazione. Questo consente di comprendere meglio i limiti operativi del sistema e di individuare configurazioni ottimali.

Dal punto di vista dello sviluppo, gli ambienti di test svolgono anche una funzione di debug. Essi permettono di identificare rapidamente problemi specifici, facilitando la correzione del codice e il miglioramento degli algoritmi. La presenza di ambienti dedicati al test riduce il rischio di introdurre errori nelle simulazioni principali o nei sistemi reali, contribuendo alla stabilità complessiva del progetto.

In conclusione, gli ambienti di test rappresentano uno strumento essenziale per la validazione incrementale del sistema MicroBot. Essi consentono di analizzare in modo isolato e controllato le diverse componenti, garantendo che ogni elemento funzioni correttamente prima di essere integrato nel sistema globale. Questo approccio metodologico migliora l'affidabilità, la comprensibilità e la qualità complessiva del progetto.

2.7.2 Prototipi software

I prototipi software rappresentano una componente fondamentale all'interno delle cartelle di simulazione, in quanto costituiscono il primo livello di concretizzazione operativa delle idee progettuali. Essi si collocano tra gli ambienti di test, caratterizzati da isolamento e controllo, e le simulazioni complete, orientate alla rappresentazione del sistema globale. I prototipi hanno come obiettivo principale la verifica rapida di concetti, architetture e comportamenti, attraverso implementazioni funzionali ma non necessariamente definitive.

A differenza degli ambienti di test, che si focalizzano su singoli moduli, i prototipi software tendono a integrare più componenti del sistema, offrendo una visione parziale ma già significativa del comportamento complessivo. In essi possono coesistere modelli di interazione tra MicroBot, sistemi di visualizzazione, logiche di controllo e, in alcuni casi, elementi di comunicazione. Questo livello consente di valutare l'efficacia delle soluzioni adottate in un contesto più realistico rispetto ai test isolati.

Un aspetto distintivo dei prototipi software è la loro natura iterativa e sperimentale. Essi vengono sviluppati rapidamente, modificati frequentemente e spesso sostituiti da versioni successive più evolute. L'obiettivo non è la perfezione del codice, ma la velocità di verifica delle idee. Questo approccio permette di esplorare diverse soluzioni progettuali, confrontarle e selezionare quelle più promettenti per un'eventuale implementazione stabile.

I prototipi possono assumere forme diverse a seconda del contesto. Alcuni sono orientati alla simulazione visiva, permettendo di osservare il comportamento dello swarm attraverso rappresentazioni grafiche dinamiche. Altri sono focalizzati sulla logica interna, implementando algoritmi e modelli matematici senza una componente visiva significativa. In entrambi i casi, il prototipo funge da ponte tra il modello teorico e la sua applicazione concreta.

All'interno di queste cartelle è spesso presente una varietà di prototipi, ciascuno dedicato a uno specifico aspetto del sistema. Questa pluralità riflette la natura esplorativa del progetto MicroBot, in cui diverse idee vengono sviluppate in parallelo prima di convergere verso una soluzione unificata. La presenza di versioni multiple dello stesso prototipo consente inoltre di tracciare l'evoluzione delle soluzioni adottate e di mantenere un archivio delle diverse iterazioni.

Dal punto di vista metodologico, i prototipi software svolgono un ruolo chiave nel processo di validazione progressiva. Essi permettono di identificare rapidamente limiti, problemi e opportunità, fornendo un feedback immediato che guida le fasi successive di sviluppo. Questo ciclo continuo di prototipazione e revisione consente di ridurre il rischio di errori nelle fasi più avanzate e di costruire un sistema più robusto.

Infine, i prototipi software hanno anche una funzione comunicativa. Essi possono essere utilizzati per dimostrare il funzionamento del sistema, sia a livello interno che verso interlocutori esterni. La possibilità di mostrare un comportamento funzionante, anche se non definitivo, rappresenta un elemento importante per la comprensione e la valorizzazione del progetto.

In sintesi, i prototipi software costituiscono uno strumento essenziale per l'esplorazione, la verifica e l'evoluzione del sistema MicroBot. Essi permettono di trasformare rapidamente le idee in implementazioni concrete, facilitando il passaggio dalla teoria alla pratica e contribuendo in modo significativo allo sviluppo complessivo del progetto.

2.8 Ruolo del file INDEX.txt come mappa globale

Il file INDEX.txt assume un ruolo centrale all'interno del workspace MicroBot, configurandosi come elemento di coordinamento e orientamento dell'intero sistema documentale e operativo. In un contesto caratterizzato da un'elevata complessità strutturale, dovuta alla presenza di numerose cartelle, progetti eterogenei e versioni multiple, la necessità di una mappa globale diventa fondamentale per garantire navigabilità, comprensione e coerenza.

INDEX.txt può essere interpretato come un punto di ingresso logico al sistema. Esso fornisce una visione sintetica ma strutturata dell'intero workspace, descrivendo l'organizzazione delle cartelle principali, la loro funzione e le relazioni tra i diversi componenti. A differenza di una semplice lista di file, questo documento assume una funzione descrittiva e orientativa, guidando l'utente nella comprensione della struttura complessiva del progetto.

Dal punto di vista funzionale, il file INDEX.txt permette di ridurre significativamente il tempo necessario per individuare informazioni specifiche. In assenza di una mappa globale, l'utente sarebbe costretto a esplorare manualmente le cartelle, con il rischio di perdere tempo o di non comprendere correttamente il ruolo dei diversi elementi. INDEX.txt centralizza queste informazioni, fornendo un riferimento immediato e accessibile.

Un aspetto rilevante è la capacità del file di riflettere la logica progettuale del sistema. La sua struttura non è arbitraria, ma segue criteri coerenti con l'organizzazione del workspace, evidenziando la suddivisione per domini, livelli e funzionalità. In questo senso, INDEX.txt non è soltanto uno strumento di navigazione, ma anche una rappresentazione astratta dell'architettura del sistema MicroBot.

Il file svolge inoltre una funzione di integrazione tra le diverse componenti del progetto. Attraverso riferimenti incrociati, descrizioni sintetiche e indicazioni contestuali, esso collega tra loro documentazione tecnica, codice sorgente, simulazioni e prototipi. Questo contribuisce a creare una visione unitaria del sistema, superando la frammentazione naturale derivante dalla sua complessità.

Dal punto di vista evolutivo, INDEX.txt deve essere aggiornato in modo coerente con lo sviluppo del progetto. Ogni modifica significativa alla struttura del workspace, come l'aggiunta di nuove cartelle, la riorganizzazione dei contenuti o l'introduzione di nuovi moduli, dovrebbe riflettersi nel file. In questo modo, la mappa globale rimane allineata con lo stato reale del sistema, mantenendo la sua utilità nel tempo.

Infine, INDEX.txt svolge un ruolo strategico anche nella comunicazione del progetto verso l'esterno. Esso può essere utilizzato come punto di partenza per presentare il sistema a nuovi collaboratori, revisori o potenziali investitori, offrendo una panoramica chiara e immediata della struttura e delle componenti principali. La sua semplicità formale, unita alla sua funzione informativa, lo rende uno strumento estremamente efficace per introdurre la complessità del sistema in modo ordinato.

In conclusione, il file INDEX.txt rappresenta la mappa globale del sistema MicroBot, un elemento chiave per la navigazione, la comprensione e l'integrazione delle diverse componenti del workspace. Esso consente di trasformare una struttura complessa e distribuita in un sistema leggibile e coerente, facilitando sia lo sviluppo interno che la comunicazione esterna del progetto.

2.9 Versionamento implicito dei progetti

All'interno del workspace MicroBot, il versionamento dei progetti non è necessariamente gestito attraverso sistemi formali di controllo versione, ma emerge in maniera implicita attraverso la struttura stessa delle cartelle e l'evoluzione progressiva dei file. Questo approccio, sebbene meno rigido rispetto a strumenti dedicati, consente comunque di tracciare lo sviluppo del sistema nel tempo, mantenendo accessibili le diverse iterazioni progettuali.

Il versionamento implicito si manifesta principalmente attraverso la presenza di più varianti di uno stesso progetto, spesso differenziate da nomi, directory o modifiche incremental. Queste versioni rappresentano fasi successive di sviluppo, in cui vengono introdotte nuove funzionalità, ottimizzazioni o cambiamenti architettureali. La coesistenza di tali varianti consente di osservare il percorso evolutivo del sistema, evidenziando le scelte progettuali adottate e le alternative esplorate.

Un aspetto rilevante di questo approccio è la conservazione della memoria tecnica. Mantenere versioni precedenti dei progetti permette di recuperare soluzioni funzionanti, confrontare implementazioni diverse e comprendere le motivazioni alla base di determinate decisioni. Questo è particolarmente importante in un contesto sperimentale come MicroBot, in cui l'evoluzione del sistema avviene attraverso iterazioni successive e continui aggiustamenti.

Tuttavia, il versionamento implicito presenta anche alcune criticità. In assenza di una struttura formalizzata, può risultare difficile identificare rapidamente la versione più aggiornata o stabile di un progetto. Inoltre, la mancanza di metadati espliciti, come descrizioni delle modifiche o cronologie dettagliate, può rendere più complessa la comprensione delle differenze tra le varie iterazioni. Questo richiede una gestione attenta delle cartelle e una denominazione coerente dei file, al fine di mantenere un livello adeguato di ordine e chiarezza.

Dal punto di vista metodologico, il versionamento implicito riflette un approccio flessibile e orientato alla sperimentazione. Esso consente di sviluppare rapidamente nuove idee senza vincoli rigidi, favorendo l'esplorazione di soluzioni alternative. Tuttavia, con l'aumentare della complessità del sistema, diventa sempre più importante affiancare a questo approccio strumenti più strutturati, in grado di garantire tracciabilità, controllo e collaborazione.

In questo contesto, il versionamento implicito può essere visto come una fase iniziale del processo di sviluppo, che prepara il terreno per l'introduzione di sistemi più avanzati. L'adozione futura di strumenti di versionamento esplicito, come repository strutturati e sistemi di gestione delle modifiche, potrebbe integrare e potenziare l'organizzazione già presente nel workspace, senza sostituirla completamente.

In conclusione, il versionamento implicito dei progetti rappresenta una modalità naturale e flessibile di gestione dell'evoluzione del sistema MicroBot. Esso consente di tracciare il progresso del progetto, mantenere una memoria storica delle soluzioni adottate e supportare un processo di sviluppo iterativo, pur richiedendo attenzione per evitare disordine e perdita di chiarezza.

3.0 Evoluzione delle versioni nel tempo — formalizzazione del versionamento come struttura del sistema

Il concetto di versionamento, introdotto nella sezione precedente in forma implicita attraverso la struttura del workspace, può essere esteso e formalizzato come componente strutturale del sistema MicroBot. Se nella fase iniziale il versionamento emerge come conseguenza pratica dell'evoluzione dei file e delle cartelle, nella presente sezione esso viene reinterpretato come modello descrittivo del sistema stesso, in cui ogni versione rappresenta uno stato significativo all'interno di uno spazio evolutivo più ampio.

In questa prospettiva, il versionamento non deve essere considerato esclusivamente come uno strumento di gestione del codice, ma come una rappresentazione della dinamica del sistema nel tempo. Ogni versione costituisce infatti una configurazione specifica del sistema, caratterizzata da una determinata architettura, da un insieme di funzionalità e da un preciso livello di integrazione tra le componenti. Il passaggio da una versione all'altra non è semplicemente un aggiornamento incrementale, ma un processo di trasformazione che coinvolge sia il livello tecnico sia quello concettuale.

Formalmente, è possibile descrivere l'evoluzione del sistema come una sequenza di stati discreti:

$$V = \{v_0, v_1, v_2, \dots, v_n\}$$

dove ogni elemento v_i rappresenta una versione del sistema. Tuttavia, questa rappresentazione lineare risulta limitata nel momento in cui si considera la natura reale dello sviluppo, che non segue necessariamente una progressione unidirezionale, ma include ramificazioni, iterazioni parallele e convergenze tra soluzioni diverse. Per questo motivo, è più appropriato rappresentare il versionamento come un grafo:

$$G = (V, E)$$

in cui i nodi V rappresentano le versioni e gli archi E rappresentano le transizioni tra di esse. Questo modello consente di descrivere non solo la sequenza temporale delle versioni, ma anche le relazioni tra differenti percorsi evolutivi, riflettendo in modo più accurato la struttura del workspace MicroBot.

All'interno di questo grafo, ogni versione v_i può essere descritta come uno stato del sistema, definito da un insieme di parametri che includono:

- struttura delle cartelle e organizzazione del codice
- architettura del sistema (centralizzata, distribuita, ibrida)
- modelli matematici adottati
- livello di integrazione tra hardware e software
- strumenti di simulazione e visualizzazione
- capacità di comunicazione e sincronizzazione
- grado di formalizzazione teorica

Questa descrizione permette di interpretare ogni versione come un punto all'interno di uno spazio degli stati, introducendo un collegamento diretto con i concetti sviluppati nelle sezioni successive dell'indice, in particolare con la quinta dimensione intesa come spazio delle configurazioni.

A questo livello, il versionamento può essere riformulato come una funzione di evoluzione dello stato:

$$v(t+1) = \Phi(v(t), \Delta(t))$$

dove $\Delta(t)$ rappresenta l'insieme delle modifiche introdotte nel sistema, che possono includere aggiornamenti del codice, cambiamenti architetturali, introduzione di nuovi moduli o revisione dei modelli teorici. Questa formulazione evidenzia come il sistema non sia statico, ma evolva nel tempo attraverso una successione di trasformazioni discrete, ciascuna delle quali modifica la configurazione complessiva.

Un aspetto particolarmente rilevante di questa interpretazione riguarda la relazione tra versionamento e conoscenza del sistema. Ogni versione non rappresenta soltanto uno stato operativo, ma anche un livello di comprensione raggiunto durante lo sviluppo. Le versioni precedenti contengono informazioni sulle scelte progettuali, sugli errori commessi e sulle soluzioni adottate, costituendo una memoria tecnica che contribuisce alla costruzione del sistema stesso.

In questo senso, il versionamento implicito svolge una funzione analoga a quella della memoria nei sistemi complessi: esso conserva tracce delle configurazioni passate, permettendo di ricostruire il percorso evolutivo e di riutilizzare conoscenze acquisite. Tuttavia, a differenza dei sistemi di versionamento formale, questa memoria non è strutturata in modo esplicito, ma deve essere ricostruita attraverso l'analisi delle cartelle e dei file.

Dal punto di vista architetturale, l'evoluzione delle versioni riflette anche il passaggio da un sistema frammentato a un ecosistema integrato. Nelle versioni iniziali, le componenti del sistema tendono a essere sviluppate in modo indipendente, con una limitata interazione tra di esse. Con il progredire dello sviluppo, emerge la necessità di un'integrazione più stretta, che porta alla definizione di interfacce comuni, protocolli di comunicazione e strutture condivise.

Questo processo può essere interpretato come una transizione da uno spazio delle versioni discreto e disorganizzato a uno spazio strutturato, in cui le configurazioni del sistema sono collegate tra loro da relazioni ben definite. Tale transizione anticipa il concetto di sistema come processo, sviluppato nelle sezioni successive dell'indice, in cui la forma del sistema non è più vista come statica, ma come risultato di un'evoluzione continua.

Un ulteriore elemento di connessione con il livello teorico avanzato riguarda l'interpretazione del versionamento come dimensione temporale estesa. Mentre il tempo fisico rappresenta la successione degli eventi nello spazio-tempo, il versionamento rappresenta una forma di tempo logico, in cui ogni versione corrisponde a un punto nella storia del sistema. Questo introduce una distinzione tra tempo fisico e tempo del sistema, che diventa particolarmente rilevante nel contesto dei sistemi distribuiti e delle simulazioni.

Nel progetto MicroBot, questa distinzione si manifesta chiaramente: il sistema possiede un tempo interno, definito dal clock globale e dalla sincronizzazione tra i nodi, ma possiede anche un tempo evolutivo, rappresentato dalla successione delle versioni. Questi due livelli temporali sono interconnessi, ma non coincidono, e la loro integrazione rappresenta un aspetto fondamentale per la comprensione del sistema.

Infine, la formalizzazione del versionamento come struttura del sistema apre la strada a una reinterpretazione più generale dell'evoluzione progettuale. Il sistema non deve essere visto come un oggetto statico che viene progressivamente migliorato, ma come un processo dinamico che attraversa uno spazio di configurazioni, esplorando diverse soluzioni e convergendo verso strutture sempre più integrate.

In conclusione, il passaggio dal versionamento implicito alla formalizzazione del versionamento come modello evolutivo rappresenta un'estensione naturale del lavoro. Esso consente di integrare il livello tecnico della gestione del codice con il livello teorico della rappresentazione dei sistemi, creando un ponte tra sviluppo pratico e modellazione concettuale. Questa integrazione costituisce la base per le sezioni successive, in cui il sistema verrà analizzato non solo come insieme di componenti, ma come struttura multidimensionale in cui spazio, tempo, stato e informazione sono profondamente interconnessi.

SEZIONE 3 — Architettura Generale del Sistema

3.1 Architettura a livelli del sistema

L'architettura generale del sistema MicroBot rappresenta la struttura fondamentale attraverso cui vengono organizzate, coordinate e integrate tutte le componenti del progetto. A differenza di un sistema monolitico, in cui le funzionalità sono concentrate all'interno di un'unica entità, MicroBot adotta un'architettura a livelli, progettata per gestire la complessità attraverso la separazione delle responsabilità e la modularità dei componenti.

Questa scelta architeturale non è arbitraria, ma deriva direttamente dall'evoluzione del sistema descritta nella sezione precedente. Il passaggio da versionamento implicito a modello evolutivo ha evidenziato la necessità di una struttura che permetta di integrare componenti eterogenee — hardware, software, simulazione, interfacce — mantenendo al contempo coerenza e scalabilità. L'architettura a livelli costituisce quindi la formalizzazione operativa di questa esigenza.

Il sistema può essere descritto come un insieme di livelli interconnessi, ciascuno dei quali svolge una funzione specifica all'interno del processo complessivo. Tali livelli non sono isolati, ma comunicano tra loro attraverso interfacce ben definite, permettendo il flusso continuo di informazioni e comandi.

A livello più basso si colloca il livello fisico, costituito dai MicroBot come unità hardware distribuite. In questo livello rientrano tutti gli elementi materiali del sistema, inclusi microcontrollori, batterie, sistemi magnetici e componenti elettronici. Questo strato rappresenta il punto di contatto diretto con la realtà fisica e definisce i vincoli operativi del sistema, come limiti energetici, capacità computazionali e caratteristiche delle interazioni magnetiche.

Al di sopra del livello fisico si trova il livello di controllo locale, in cui ogni MicroBot gestisce il proprio stato interno e le proprie operazioni. In questo contesto, il comportamento di ciascuna unità è descritto attraverso macchine a stati e logiche di controllo embedded, che permettono di reagire agli input ricevuti e di interagire con le unità vicine. Questo livello rappresenta il primo grado di autonomia del sistema e costituisce la base per l'emergenza del comportamento collettivo.

Il livello successivo è rappresentato dal livello di comunicazione, che consente lo scambio di informazioni tra le unità e tra i MicroBot e il controller centrale. Questo strato include protocolli di rete, strutture dei pacchetti e meccanismi di sincronizzazione, e svolge un ruolo fondamentale nella coordinazione del sistema. La comunicazione non è solo un mezzo di trasmissione dei dati, ma un elemento strutturale che determina la capacità del sistema di funzionare come entità coerente.

Al di sopra del livello di comunicazione si colloca il livello di coordinamento globale, rappresentato dal controller centrale. Questo componente introduce una forma di controllo top-down, complementare alle dinamiche emergenti bottom-up. Il controller non sostituisce il comportamento distribuito, ma fornisce un riferimento temporale e logico che consente di sincronizzare le azioni e di guidare il sistema verso configurazioni desiderate.

Un ulteriore livello è costituito dal livello di simulazione e rappresentazione, in cui il sistema viene modellato, visualizzato e analizzato attraverso strumenti software. Questo livello permette di testare il comportamento dello swarm in ambienti virtuali, riducendo i costi e i tempi di sviluppo. La simulazione svolge anche un ruolo fondamentale nella comprensione del sistema, in quanto consente di osservare dinamiche emergenti difficilmente analizzabili direttamente a livello fisico.

Infine, al livello più alto si colloca il livello di interazione uomo-macchina, che include interfacce utente, dashboard, sistemi web e possibili estensioni in realtà virtuale o gesture control. Questo strato rappresenta il punto di accesso al sistema per l'utente e consente di monitorare, controllare e interpretare il comportamento dello swarm.

L'architettura complessiva può quindi essere interpretata come una gerarchia di livelli, in cui ogni strato contribuisce alla definizione del sistema:

- livello fisico → materia e interazioni reali
- livello locale → stato delle unità
- livello di comunicazione → scambio di informazione
- livello di coordinamento → controllo globale

- livello di simulazione → modellazione e validazione
- livello di interazione → accesso e controllo umano

Questa struttura introduce una separazione chiara tra i diversi aspetti del sistema, ma allo stesso tempo mantiene una forte integrazione tra di essi. Ogni livello dipende dagli altri e contribuisce alla definizione del comportamento complessivo, creando un sistema in cui la complessità è distribuita e gestita in modo modulare.

Dal punto di vista teorico, l'architettura a livelli può essere reinterpretata come una stratificazione dello stato del sistema. Ogni livello rappresenta una proiezione della variabile s (stato), già introdotta nel modello multidimensionale, su un dominio specifico. Il sistema non è quindi soltanto una struttura fisica o logica, ma una configurazione multilivello in cui ogni strato contribuisce alla definizione dello stato globale.

Questa interpretazione permette di collegare direttamente l'architettura del sistema con il modello teorico avanzato sviluppato nelle sezioni successive dell'indice. In particolare, il concetto di sistema come processo e di forma come evento dinamico trova una realizzazione concreta nell'architettura MicroBot, in cui la configurazione del sistema evolve nel tempo attraverso l'interazione tra livelli diversi.

In conclusione, l'architettura a livelli del sistema MicroBot costituisce la base strutturale su cui si sviluppano tutte le funzionalità del progetto. Essa consente di gestire la complessità, facilitare l'evoluzione del sistema e integrare componenti eterogenee all'interno di un framework coerente. Questa struttura rappresenta il punto di partenza per l'analisi dettagliata delle singole componenti, che verrà sviluppata nei punti successivi della sezione.

3.2 Interfaccia utente

L'interfaccia utente rappresenta il punto di contatto tra il sistema MicroBot e l'osservatore esterno, costituendo il livello attraverso cui il comportamento dello swarm viene reso accessibile, interpretabile e controllabile. In un sistema complesso e distribuito come MicroBot, l'interfaccia non può essere considerata un semplice elemento di visualizzazione, ma deve essere progettata come componente strutturale dell'architettura, in grado di riflettere in modo coerente lo stato del sistema e di supportare l'interazione in tempo reale.

Dal punto di vista funzionale, l'interfaccia utente ha il compito di tradurre informazioni ad alta complessità — quali configurazioni dello swarm, stato dei nodi, flussi di comunicazione e dinamiche emergenti — in rappresentazioni comprensibili. Questo implica una trasformazione continua tra livello interno del sistema e livello percettivo, in cui lo stato globale deve essere sintetizzato senza perdere le caratteristiche fondamentali che lo definiscono.

Nel contesto MicroBot, l'interfaccia si configura come un sistema multilivello, composto da diversi sottosistemi di rappresentazione e controllo. Un primo livello è costituito dalla visualizzazione grafica dello swarm, in cui le unità vengono rappresentate come entità discrete nello spazio, con attributi visivi che riflettono il loro stato interno. Ad esempio, parametri come colore, luminosità o dimensione possono essere utilizzati per indicare variabili quali energia, stato operativo o connessioni attive. Questa rappresentazione consente di osservare direttamente la configurazione spaziale del sistema e di identificare pattern emergenti.

Un secondo livello riguarda la rappresentazione dei dati di sistema, che include informazioni di telemetria, stato dei nodi, latenza di comunicazione e parametri di sincronizzazione. Tali dati possono essere visualizzati attraverso dashboard interattive, grafici temporali o indicatori numerici, fornendo una visione più dettagliata del comportamento interno del sistema. Questo livello è particolarmente importante per attività di debug, analisi e validazione, in quanto permette di correlare fenomeni osservabili con variabili interne.

Oltre alla visualizzazione, l'interfaccia utente svolge un ruolo attivo nel controllo del sistema. Attraverso comandi inviati al controller centrale, l'utente può modificare la configurazione dello swarm, attivare pattern specifici o intervenire sul comportamento delle unità. Questo introduce un flusso bidirezionale di informazione, in cui l'interfaccia non si limita a ricevere dati, ma diventa parte integrante del ciclo di controllo.

Dal punto di vista architetturale, l'interfaccia utente si colloca al livello più alto del sistema, ma è strettamente connessa agli altri livelli attraverso interfacce definite. In particolare, essa comunica con il livello di simulazione per ottenere rappresentazioni aggiornate dello stato e con il livello di coordinamento per inviare comandi. Questa posizione le consente di fungere da punto di integrazione tra osservazione e azione.

Un aspetto rilevante riguarda la necessità di gestire la complessità della rappresentazione. In sistemi distribuiti con un numero elevato di unità, la visualizzazione diretta di tutte le informazioni può risultare inefficace o addirittura controproducente. È quindi necessario introdurre meccanismi di astrazione e filtraggio, che permettano di selezionare le informazioni più rilevanti in funzione del contesto. Questo può includere la possibilità di visualizzare sottosistemi, aggregare dati o modificare il livello di dettaglio.

Dal punto di vista teorico, l'interfaccia utente può essere interpretata come una funzione di proiezione dello stato del sistema su uno spazio percettivo. Se lo stato globale è rappresentato dalla variabile s , l'interfaccia realizza una trasformazione del tipo:

$$R = \Psi(s)$$

dove R rappresenta la rappresentazione percepita dall'utente. Questa funzione non è invertibile in generale, in quanto comporta una riduzione dell'informazione, ma deve essere progettata in modo da preservare le caratteristiche essenziali del sistema.

Questa interpretazione introduce un collegamento diretto con i concetti sviluppati nel livello teorico avanzato, in cui la percezione viene considerata come una dimensione aggiuntiva del sistema. L'interfaccia utente non è quindi soltanto un elemento di visualizzazione, ma un mezzo attraverso cui lo stato del sistema diventa accessibile e interpretabile.

Infine, è importante considerare l'estensione dell'interfaccia verso modalità di interazione avanzate. Oltre alle tradizionali interfacce web o desktop, il sistema può essere integrato con tecnologie di realtà virtuale, realtà aumentata o controllo tramite gesture. Queste modalità permettono di superare i limiti delle rappresentazioni bidimensionali, offrendo una percezione più immersiva e intuitiva delle dinamiche dello swarm.

In conclusione, l'interfaccia utente nel sistema MicroBot rappresenta un elemento chiave per la comprensione e il controllo del sistema. Essa svolge una funzione di mediazione tra il livello interno e quello percettivo, trasformando lo stato complesso del sistema in rappresentazioni accessibili e supportando l'interazione in tempo reale. La sua progettazione richiede quindi un approccio integrato, in cui aspetti tecnici, percettivi e teorici sono strettamente interconnessi.

3.3 Controller centrale

Il controller centrale rappresenta il nucleo logico e coordinativo del sistema MicroBot, costituendo il punto di convergenza tra le dinamiche distribuite dello swarm e le esigenze di controllo globale. A differenza di un sistema completamente centralizzato, in cui tutte le decisioni sono prese da un'unica entità, il controller MicroBot opera all'interno di un'architettura ibrida, in cui il controllo top-down coesiste con comportamenti emergenti bottom-up.

La funzione primaria del controller consiste nella gestione dello stato globale del sistema, inteso non come conoscenza completa e perfettamente aggiornata di ogni singola unità, ma come rappresentazione sintetica e operativa della configurazione dello swarm. Tale rappresentazione viene costruita attraverso la raccolta di informazioni provenienti dai nodi distribuiti, la loro elaborazione e la successiva generazione di comandi coordinati.

Dal punto di vista funzionale, il controller può essere descritto come un sistema di orchestrazione, responsabile delle seguenti operazioni fondamentali:

- acquisizione dei dati dai MicroBot (telemetria, stato, eventi)
- mantenimento di una rappresentazione coerente dello stato globale
- elaborazione di strategie di controllo e pattern
- distribuzione di comandi verso i nodi
- gestione della sincronizzazione temporale
- interfaccia con il livello utente e con il sistema di simulazione

Queste funzioni evidenziano come il controller non sia un semplice "centro di comando", ma un sistema complesso che integra capacità di elaborazione, comunicazione e coordinamento.

Dal punto di vista architetturale, il controller si colloca tra il livello di comunicazione e il livello di interfaccia utente, fungendo da ponte tra il mondo interno dello swarm e l'ambiente esterno. Esso riceve input sia dal

sistema (stato dei nodi) sia dall'utente (comandi, configurazioni), e produce output sotto forma di istruzioni operative per i MicroBot.

Un elemento chiave del controller è la gestione del tempo. In un sistema distribuito, la sincronizzazione rappresenta una condizione necessaria per il corretto funzionamento delle operazioni coordinate. Il controller assume quindi il ruolo di riferimento temporale globale, distribuendo segnali di sincronizzazione che permettono ai nodi di allineare le proprie azioni. Questo introduce una struttura temporale discreta, basata su slot o cicli, in cui le operazioni vengono eseguite in modo coordinato.

Dal punto di vista matematico, il comportamento del controller può essere modellato come una funzione di trasformazione dello stato:

$$u(t) = C(s(t), I(t))$$

dove $s(t)$ rappresenta lo stato globale del sistema, $I(t)$ l'insieme degli input esterni (ad esempio comandi utente) e $u(t)$ l'insieme dei comandi generati dal controller. Questa funzione definisce la logica di controllo, che può variare da semplici regole deterministiche a strategie più complesse basate su ottimizzazione o apprendimento.

Un aspetto particolarmente rilevante riguarda la relazione tra controllo centrale e comportamento emergente. Nel sistema MicroBot, il controller non impone direttamente ogni dettaglio del comportamento delle unità, ma definisce condizioni e vincoli entro cui le dinamiche locali possono evolvere. Questo approccio consente di mantenere i vantaggi della distribuzione — robustezza, scalabilità, adattabilità — senza rinunciare alla possibilità di guidare il sistema verso configurazioni specifiche.

Dal punto di vista implementativo, il controller può essere realizzato come un sistema software eseguito su una macchina centrale, che espone interfacce di comunicazione verso i nodi e verso l'interfaccia utente. In questo contesto, tecnologie come server HTTP, WebSocket e API REST possono essere utilizzate per gestire il flusso di dati in tempo reale. La presenza di un endpoint WebSocket, in particolare, consente una comunicazione bidirezionale a bassa latenza, fondamentale per il controllo in tempo reale dello swarm.

Un ulteriore elemento di complessità riguarda la gestione della consistenza dello stato. In un sistema distribuito, le informazioni provenienti dai nodi possono essere parziali, ritardate o incoerenti. Il controller deve quindi implementare meccanismi di aggregazione e filtraggio, in grado di costruire una rappresentazione robusta dello stato globale nonostante queste limitazioni. Questo può includere tecniche di smoothing temporale, gestione delle latenze e rilevamento di anomalie.

Dal punto di vista teorico, il controller può essere interpretato come una funzione di riduzione e proiezione dello spazio degli stati distribuiti. Se lo stato completo del sistema è dato da:

$$s = \{s_1, s_2, \dots, s_n\}$$

il controller costruisce una rappresentazione sintetica:

$$\hat{s} = \Gamma(s)$$

dove Γ è una funzione di aggregazione. Questa rappresentazione viene poi utilizzata per generare i comandi, chiudendo il ciclo di controllo.

Un ulteriore aspetto importante è la scalabilità del controller. Con l'aumentare del numero di unità, il carico computazionale e comunicativo cresce, rendendo necessario un design efficiente e modulare. Possibili estensioni includono la distribuzione del controller stesso o l'introduzione di livelli intermedi di coordinamento, creando una gerarchia di controllo.

Infine, il controller centrale svolge un ruolo fondamentale nell'integrazione tra sistema reale e simulazione. Esso può operare sia su dati provenienti da unità fisiche sia su modelli virtuali, permettendo di utilizzare lo stesso framework per test e implementazione. Questa continuità tra simulazione e realtà rappresenta uno degli elementi distintivi del sistema MicroBot.

In conclusione, il controller centrale costituisce il cuore logico del sistema, responsabile della coordinazione, della gestione dello stato e dell'interazione tra i diversi livelli dell'architettura. La sua progettazione richiede un equilibrio tra controllo e flessibilità, tra centralizzazione e distribuzione, riflettendo la natura complessa e dinamica del sistema MicroBot.

3.4 MicroBot come nodi distribuiti

Nel contesto dell'architettura generale del sistema, i MicroBot non devono essere interpretati come semplici unità hardware indipendenti, ma come nodi all'interno di un sistema distribuito, in cui il comportamento globale emerge dall'interazione tra componenti locali. Questa interpretazione rappresenta un punto centrale del progetto, in quanto definisce la natura stessa del sistema e ne distingue profondamente il funzionamento rispetto ai modelli di robotica tradizionale.

Ogni MicroBot è progettato come un'unità elementare dotata di capacità limitate, sia dal punto di vista computazionale sia dal punto di vista energetico e fisico. Tuttavia, la potenza del sistema non risiede nella complessità della singola unità, ma nella rete di relazioni che si instaura tra le unità stesse. In questa prospettiva, il MicroBot assume il ruolo di agente locale, il cui comportamento è determinato da un insieme di regole semplici applicate in modo distribuito.

Formalmente, il sistema può essere descritto come un insieme di nodi:

$$S = \{n_1, n_2, \dots, n_n\}$$

dove ogni nodo n_i è caratterizzato da uno stato locale s_i e da una funzione di aggiornamento che ne determina l'evoluzione nel tempo. Lo stato locale può includere variabili quali posizione, stato energetico, configurazione magnetica, stato logico interno e informazioni ricevute dagli altri nodi. L'evoluzione dello stato è descritta da una funzione del tipo:

$$s_i(t+1) = f(s_i(t), N_i(t), u_i(t), E(t))$$

dove $N_i(t)$ rappresenta l'insieme dei nodi vicini, $u_i(t)$ i comandi ricevuti dal controller e $E(t)$ lo stato dell'ambiente. Questa formulazione evidenzia come ogni MicroBot operi sulla base di informazioni locali, senza avere accesso diretto alla configurazione globale del sistema.

Un aspetto fondamentale della distribuzione riguarda la limitatezza della conoscenza. Nessun nodo possiede una visione completa dello stato globale, e il comportamento collettivo emerge dalla combinazione delle interazioni locali. Questo introduce una distinzione tra stato locale e stato globale: mentre il primo è direttamente accessibile al nodo, il secondo è una proprietà emergente del sistema nel suo complesso.

Nel progetto MicroBot, questa struttura distribuita è supportata sia da interazioni logiche sia da interazioni fisiche. Le interazioni logiche avvengono attraverso la comunicazione tra nodi, che permette lo scambio di informazioni e la coordinazione delle azioni. Le interazioni fisiche, invece, sono mediate dal sistema magnetico, che consente alle unità di attrarsi, respingersi o mantenere configurazioni stabili. La combinazione di questi due livelli di interazione rappresenta uno degli elementi distintivi del sistema.

Dal punto di vista architettonico, ogni MicroBot può essere visto come un sistema autonomo a livello locale, ma dipendente a livello globale. Esso è in grado di gestire il proprio stato, eseguire operazioni e reagire a stimoli, ma il significato delle sue azioni emerge solo nel contesto della rete di nodi. Questa dualità tra autonomia e dipendenza rappresenta una caratteristica tipica dei sistemi distribuiti e costituisce la base per l'emergenza del comportamento collettivo.

Un ulteriore elemento di rilievo è la modularità dei nodi. Poiché ogni MicroBot è progettato come unità replicabile, il sistema può essere scalato semplicemente aumentando il numero di nodi. Questa proprietà consente di estendere il sistema senza modificare la struttura logica di base, rendendo la scalabilità una caratteristica intrinseca. Tuttavia, l'aumento del numero di nodi introduce anche nuove sfide, in particolare in termini di comunicazione, sincronizzazione e gestione dello stato.

Dal punto di vista dinamico, il sistema dei nodi distribuiti può essere interpretato come un sistema multi-agente, in cui ogni agente segue regole locali e contribuisce a un comportamento globale. Questo comportamento non è programmato direttamente, ma emerge dall'interazione tra gli agenti. In questo senso,

il sistema MicroBot si colloca all'interno della teoria dei sistemi complessi, in cui proprietà macroscopiche derivano da dinamiche microscopiche.

Un aspetto particolarmente interessante riguarda la relazione tra nodi e configurazione dello spazio. Poiché ogni MicroBot occupa una posizione nello spazio fisico, la distribuzione dei nodi determina la forma del sistema. Tuttavia, questa forma non è statica, ma evolve nel tempo in funzione delle interazioni tra le unità. Questo porta a una rappresentazione in cui lo spazio fisico e lo stato del sistema sono strettamente interconnessi.

Dal punto di vista teorico, è possibile interpretare l'insieme dei nodi come una discretizzazione della materia programmabile. Ogni MicroBot rappresenta un elemento discreto di un sistema continuo più ampio, in cui la configurazione globale può essere modificata attraverso il controllo delle interazioni locali. Questa interpretazione permette di collegare direttamente il sistema MicroBot ai concetti sviluppati nelle sezioni teoriche, in particolare alla rappresentazione della forma come funzione dello stato.

Un ulteriore elemento di complessità è introdotto dalla presenza di vincoli fisici e operativi. Ogni nodo è limitato in termini di energia, capacità computazionale e precisione delle interazioni. Questi vincoli influenzano il comportamento del sistema e devono essere considerati nella progettazione delle regole locali. In particolare, è necessario garantire che il sistema rimanga stabile e coerente anche in presenza di errori, ritardi o malfunzionamenti di singole unità.

Infine, i nodi distribuiti svolgono un ruolo fondamentale nell'evoluzione del sistema verso un ecosistema integrato. La loro capacità di interagire, adattarsi e contribuire al comportamento globale rende possibile la costruzione di strutture complesse e dinamiche, che possono essere controllate e analizzate attraverso gli altri livelli dell'architettura.

In conclusione, i MicroBot come nodi distribuiti rappresentano l'elemento base su cui si costruisce l'intero sistema. La loro progettazione e il loro comportamento definiscono le proprietà fondamentali dello swarm, inclusa la capacità di emergenza, la scalabilità e l'adattabilità. Questa struttura costituisce il fondamento per le sezioni successive, in cui verranno analizzati in dettaglio i meccanismi di interazione, comunicazione e coordinamento tra le unità.

3.5 Separazione delle responsabilità

La separazione delle responsabilità rappresenta uno dei principi architetturali fondamentali su cui si basa l'intero sistema MicroBot, e costituisce il meccanismo attraverso cui la complessità viene distribuita, controllata e resa evolvibile nel tempo. In continuità con quanto emerso nella sezione sul versionamento implicito e nella formalizzazione evolutiva del sistema, questo principio non si limita a una scelta organizzativa del codice, ma si configura come una proprietà strutturale del sistema stesso, che coinvolge simultaneamente livello logico, fisico e informativo.

Nel contesto di MicroBot, la complessità deriva dalla coesistenza di molteplici dimensioni operative: interazioni fisiche tra unità, comunicazione distribuita, coordinamento centrale, simulazione, visualizzazione e interazione utente. Senza una chiara separazione delle responsabilità, tali componenti tenderebbero a sovrapporsi, generando dipendenze forti, difficoltà di manutenzione e limitazioni nell'evoluzione del sistema. La separazione introduce invece una decomposizione del sistema in sottosistemi relativamente indipendenti, ciascuno responsabile di un insieme definito di funzioni.

Dal punto di vista formale, è possibile descrivere il sistema come una composizione di funzioni specializzate:

$$S = F_n \circ F_{n-1} \circ \dots \circ F_2 \circ F_1$$

dove ogni funzione F_i rappresenta un dominio di responsabilità. Questa composizione non è necessariamente lineare, ma può assumere una struttura a grafo, in cui diversi moduli interagiscono attraverso interfacce definite. La chiave del modello risiede nel fatto che ogni funzione opera su una rappresentazione specifica dello stato del sistema, senza richiedere accesso diretto all'intero spazio degli stati.

In continuità con la struttura a livelli introdotta nella sezione 3.1, la separazione delle responsabilità può essere interpretata come una segmentazione dello stato globale s in sottospazi:

$s = (s_phys, s_local, s_comm, s_ctrl, s_sim, s_ui)$

dove ciascun termine rappresenta una proiezione dello stato su un dominio specifico: fisico, locale, comunicativo, di controllo, di simulazione e di interfaccia. Ogni sottosistema opera su una di queste componenti, riducendo la complessità e migliorando la coerenza interna.

Un elemento centrale di questo approccio è la definizione delle interfacce. La separazione delle responsabilità non implica isolamento completo, ma comunicazione controllata. Ogni componente espone un insieme di operazioni e dati attraverso interfacce ben definite, che permettono agli altri moduli di interagire senza accedere direttamente all'implementazione interna. Questo principio è particolarmente rilevante nel contesto del sistema reale che stai costruendo, dove moduli come adapters, bridges e layer di orchestrazione rappresentano concretamente queste interfacce.

Gli adapters, ad esempio, possono essere interpretati come moduli di adattamento tra domini diversi dello stato. Essi traducono informazioni tra rappresentazioni eterogenee, come dati provenienti dall'hardware, strutture di simulazione o formati utilizzati dall'interfaccia utente. In termini formali, un adapter realizza una funzione di trasformazione:

$s_A = T(s_B)$

dove s_A e s_B appartengono a sottospazi differenti dello stato globale. Questa trasformazione consente di mantenere la separazione tra domini senza perdere coerenza.

I bridges, invece, rappresentano i canali di comunicazione tra componenti, gestendo il flusso di dati e sincronizzando le operazioni. Essi implementano relazioni dinamiche tra sottosistemi, permettendo la propagazione dello stato e dei comandi lungo l'architettura. In questo senso, i bridges possono essere interpretati come archi del grafo G introdotto nella sezione 3.0, collegando nodi che rappresentano versioni o componenti del sistema.

Un ulteriore livello di separazione è rappresentato dalla distinzione tra logica di controllo e logica di comportamento. Il controller centrale opera a livello globale, definendo strategie e obiettivi, mentre i MicroBot implementano comportamenti locali basati su regole semplici. Questa distinzione consente di combinare controllo top-down e dinamiche emergenti bottom-up, evitando sia l'eccessiva rigidità di un sistema completamente centralizzato sia l'imprevedibilità di un sistema totalmente distribuito.

Dal punto di vista evolutivo, la separazione delle responsabilità facilita l'introduzione di nuove funzionalità e la modifica di componenti esistenti. Poiché ogni modulo è relativamente indipendente, è possibile intervenire su una parte del sistema senza compromettere le altre. Questo è coerente con il modello di versionamento descritto nella sezione precedente, in cui le diverse versioni rappresentano configurazioni alternative del sistema. La separazione consente di esplorare queste configurazioni in modo controllato, mantenendo la possibilità di tornare a versioni precedenti o di combinare soluzioni diverse.

Un aspetto particolarmente rilevante riguarda la gestione degli errori e delle anomalie. In un sistema complesso, è inevitabile che si verifichino malfunzionamenti, ritardi o inconsistenze. La separazione delle responsabilità permette di isolare tali problemi all'interno di specifici moduli, riducendo l'impatto sul sistema globale. Ad esempio, un errore nel livello di comunicazione non deve compromettere il funzionamento del livello fisico o del livello di simulazione, ma deve essere gestito attraverso meccanismi locali di recupero o compensazione.

Dal punto di vista teorico, questo principio si collega direttamente alla concezione del sistema come processo, introdotta nel livello teorico avanzato. La separazione delle responsabilità può essere vista come una decomposizione del processo globale in sottoprocessi, ciascuno dei quali contribuisce alla definizione della dinamica complessiva. In questo senso, il sistema non è una struttura statica, ma una rete di processi interconnessi, ciascuno con una propria funzione e un proprio dominio.

Infine, la separazione delle responsabilità rappresenta un elemento chiave per la trasformazione del progetto in un ecosistema. Solo attraverso una struttura modulare e ben definita è possibile integrare componenti esterne, estendere il sistema e supportare la collaborazione tra diversi sviluppatori o domini applicativi. Questo aspetto diventa particolarmente rilevante nelle sezioni successive, in cui il sistema viene analizzato in termini di integrazione, scalabilità e apertura.

In conclusione, la separazione delle responsabilità costituisce il principio organizzativo che rende possibile la gestione della complessità nel sistema MicroBot. Essa consente di strutturare il sistema in modo modulare, facilitare l'evoluzione, migliorare la robustezza e integrare componenti eterogenee all'interno di un framework coerente. Questo principio rappresenta il collegamento tra architettura, implementazione e teoria, e costituisce la base per l'analisi dei successivi aspetti del sistema.

3.6 Design modulare

Il design modulare rappresenta l'evoluzione operativa del principio di separazione delle responsabilità, costituendo il meccanismo attraverso cui la struttura logica del sistema MicroBot viene tradotta in una configurazione concreta di componenti indipendenti ma interconnessi. Se nella sezione precedente la complessità è stata scomposta in domini funzionali, in questa fase tale decomposizione viene formalizzata in termini di moduli, ovvero unità strutturali autonome che implementano specifiche funzionalità all'interno del sistema.

Nel contesto MicroBot, un modulo può essere definito come un componente che incapsula uno specifico insieme di responsabilità, esponendo un'interfaccia ben definita e nascondendo i dettagli della propria implementazione. Questa definizione implica due proprietà fondamentali: l'incapsulamento e l'interoperabilità. L'incapsulamento garantisce che ogni modulo possa essere sviluppato e modificato indipendentemente, mentre l'interoperabilità assicura che i moduli possano comunicare tra loro in modo coerente.

Dal punto di vista formale, il sistema può essere rappresentato come un insieme di moduli:

$$M = \{M_1, M_2, \dots, M_k\}$$

dove ogni modulo M_i è caratterizzato da una funzione interna F_i e da un'interfaccia I_i . L'interazione tra moduli avviene attraverso composizione e scambio di dati:

$$M_i \rightarrow M_j \text{ attraverso } I_{ij}$$

Questa struttura introduce un livello di astrazione che consente di gestire la complessità del sistema senza dover considerare simultaneamente tutti i dettagli delle singole componenti.

Nel sistema MicroBot, il design modulare si manifesta a diversi livelli dell'architettura. A livello software, esso è visibile nella suddivisione del sistema in componenti quali controller, moduli di comunicazione, sistemi di simulazione, interfacce utente e layer di integrazione. A livello organizzativo, si riflette nella struttura delle cartelle e dei repository, in cui ogni modulo è rappresentato da un insieme coerente di file e risorse.

Un aspetto particolarmente rilevante riguarda la corrispondenza tra moduli e sottospazi dello stato del sistema. Riprendendo la decomposizione introdotta nella sezione precedente, ogni modulo opera su una specifica componente dello stato:

- moduli hardware $\rightarrow s_{\text{phys}}$
- moduli locali $\rightarrow s_{\text{local}}$
- moduli di comunicazione $\rightarrow s_{\text{comm}}$
- moduli di controllo $\rightarrow s_{\text{ctrl}}$
- moduli di simulazione $\rightarrow s_{\text{sim}}$
- moduli di interfaccia $\rightarrow s_{\text{ui}}$

Questa corrispondenza consente di interpretare il design modulare come una partizione dello spazio degli stati, in cui ogni modulo gestisce una porzione specifica della complessità complessiva.

Nel sistema reale, questa struttura si concretizza attraverso pattern architetturali ben definiti. Gli adapters rappresentano moduli di traduzione tra domini diversi, permettendo di collegare componenti eterogenee senza introdurre dipendenze dirette. I bridges implementano canali di comunicazione tra moduli, gestendo il flusso di dati e la sincronizzazione. I core modules, come il controller o il sistema di simulazione, definiscono la logica principale del sistema.

Questa organizzazione introduce una distinzione tra moduli funzionali e moduli infrastrutturali. I primi implementano le funzionalità specifiche del sistema, come il comportamento dello swarm o la gestione della comunicazione. I secondi forniscono servizi di supporto, come logging, gestione degli errori, configurazione e orchestrazione. Questa distinzione consente di isolare le componenti critiche e di migliorare la manutenibilità del sistema.

Un ulteriore elemento chiave del design modulare è la riusabilità. Poiché i moduli sono progettati per essere indipendenti, essi possono essere riutilizzati in contesti diversi o combinati per creare nuove configurazioni del sistema. Questo è particolarmente importante nel contesto evolutivo descritto nella sezione 3.0, in cui il sistema attraversa diverse versioni e configurazioni. La modularità consente di preservare componenti esistenti e di integrarli in nuove versioni senza doverli riscrivere.

Dal punto di vista dinamico, il design modulare supporta l'evoluzione del sistema come processo. Ogni modifica può essere interpretata come una trasformazione di uno o più moduli, mantenendo invariata la struttura complessiva. Questo permette di modellare l'evoluzione del sistema come una sequenza di trasformazioni locali, piuttosto che come una ricostruzione globale.

Un aspetto critico riguarda la gestione delle dipendenze tra moduli. Un design modulare efficace richiede che le dipendenze siano limitate e ben definite, evitando accoppiamenti forti che possano compromettere l'indipendenza dei componenti. Questo implica l'adozione di principi come l'inversione delle dipendenze e l'utilizzo di interfacce astratte, che permettono di modificare l'implementazione di un modulo senza influenzare gli altri.

Nel contesto MicroBot, questo principio è particolarmente importante per garantire la scalabilità del sistema. Con l'aumentare del numero di moduli e della complessità delle interazioni, una struttura modulare ben progettata consente di mantenere il sistema gestibile e di evitare fenomeni di degrado architetturale.

Dal punto di vista teorico, il design modulare può essere interpretato come una decomposizione del sistema in sottosistemi accoppiati debolmente, ciascuno dei quali contribuisce alla dinamica complessiva. Questa interpretazione si collega direttamente alla concezione del sistema come processo e alla rappresentazione multidimensionale dello stato, in cui la complessità viene distribuita su più dimensioni e livelli.

Infine, il design modulare rappresenta la base per la trasformazione del sistema in un ecosistema aperto. La presenza di moduli indipendenti consente di integrare componenti esterne, sviluppate anche da terze parti, e di estendere il sistema in nuove direzioni. Questo aspetto è fondamentale per l'evoluzione futura del progetto, in quanto permette di passare da un sistema chiuso a una piattaforma modulare e collaborativa.

In conclusione, il design modulare costituisce il principio organizzativo che permette di tradurre la separazione delle responsabilità in una struttura concreta e operativa. Esso consente di gestire la complessità, facilitare l'evoluzione, migliorare la riusabilità e supportare la scalabilità del sistema MicroBot. Questa struttura rappresenta il fondamento su cui si basano le successive analisi dell'architettura, in particolare per quanto riguarda l'indipendenza dei componenti e la gestione delle interazioni tra moduli.

3.7 Indipendenza dei componenti

L'indipendenza dei componenti rappresenta una proprietà fondamentale dell'architettura MicroBot e costituisce la naturale estensione del design modulare e della separazione delle responsabilità. Se la modularità definisce la struttura del sistema in termini di unità funzionali distinte, l'indipendenza stabilisce il grado di autonomia con cui tali unità possono operare, evolvere e interagire senza introdurre vincoli rigidi o dipendenze critiche.

Nel contesto di un sistema complesso e distribuito come MicroBot, l'indipendenza non deve essere interpretata come isolamento totale, ma come capacità di ciascun componente di funzionare correttamente all'interno del sistema anche in presenza di variazioni, ritardi o malfunzionamenti delle altre componenti. Questa proprietà è essenziale per garantire robustezza, scalabilità e adattabilità, soprattutto in un ambiente caratterizzato da interazioni multiple e dinamiche emergenti.

Dal punto di vista formale, è possibile descrivere l'indipendenza tra due componenti M_i e M_j in termini di dipendenza funzionale. Un componente M_i è indipendente da M_j se la sua funzione interna F_i non richiede accesso diretto allo stato interno di M_j , ma interagisce esclusivamente attraverso interfacce definite. In altre parole:

$$F_i = F_i(s_i, I_i)$$

dove s_i rappresenta lo stato locale del componente e I_i l'insieme degli input ricevuti attraverso interfacce. Questa formulazione garantisce che modifiche interne a M_j non influenzino direttamente il comportamento di M_i , a condizione che l'interfaccia rimanga invariata.

Nel sistema MicroBot, questo principio si applica a diversi livelli. I MicroBot, come nodi distribuiti, operano sulla base del proprio stato locale e delle informazioni ricevute dai vicini o dal controller, senza dipendere dalla conoscenza completa dello stato globale. Analogamente, i moduli software — come controller, simulazione, interfaccia utente e sistemi di comunicazione — sono progettati per interagire attraverso API e protocolli definiti, evitando accoppiamenti diretti.

Un aspetto centrale dell'indipendenza riguarda la gestione delle modifiche. In un sistema evolutivo, come quello descritto nella sezione 3.0, le versioni successive introducono variazioni nei moduli e nelle loro interazioni. L'indipendenza dei componenti consente di applicare tali modifiche in modo localizzato, riducendo l'impatto sul resto del sistema. Questo è particolarmente rilevante nel contesto del versionamento, in cui diverse configurazioni del sistema possono coesistere e essere confrontate.

Dal punto di vista dinamico, l'indipendenza contribuisce alla stabilità del sistema. In presenza di errori o anomalie, un componente indipendente può continuare a funzionare o adottare strategie di fallback senza compromettere l'intero sistema. Ad esempio, un ritardo nella comunicazione non deve impedire ai MicroBot di eseguire comportamenti locali, ma deve essere gestito attraverso meccanismi di tolleranza agli errori.

Questa proprietà è strettamente legata al concetto di sistema distribuito robusto, in cui la perdita o il malfunzionamento di singole unità non compromette il comportamento globale. Nel caso MicroBot, ciò si traduce nella capacità dello swarm di mantenere configurazioni coerenti anche in presenza di nodi inattivi o comunicazioni incomplete. Questo comportamento emerge proprio dall'indipendenza delle unità, che non dipendono da un singolo punto di controllo.

Un ulteriore elemento di rilievo è la relazione tra indipendenza e scalabilità. In un sistema altamente accoppiato, l'aggiunta di nuove componenti richiede modifiche significative alla struttura esistente. Al contrario, in un sistema composto da componenti indipendenti, l'estensione avviene in modo naturale, attraverso l'integrazione di nuovi moduli che rispettano le interfacce esistenti. Questo principio è particolarmente importante per MicroBot, dove il numero di unità può crescere significativamente.

Dal punto di vista teorico, l'indipendenza dei componenti può essere interpretata come una proprietà dello spazio degli stati. Se lo stato globale del sistema è rappresentato da:

$$s = (s_1, s_2, \dots, s_k)$$

l'indipendenza implica che la dinamica di ciascuna componente s_i dipende solo da un sottoinsieme limitato delle altre componenti. Questo porta a una decomposizione dello spazio degli stati in sottospazi debolmente accoppiati, rendendo il sistema più gestibile e analizzabile.

Un aspetto importante riguarda anche la relazione tra indipendenza e osservabilità. Poiché ogni componente gestisce una parte dello stato, la conoscenza globale deve essere ricostruita attraverso l'aggregazione delle informazioni locali. Questo introduce un trade-off tra indipendenza e completezza dell'informazione: maggiore indipendenza implica minore conoscenza globale diretta, ma maggiore robustezza e flessibilità.

Nel contesto MicroBot, questo trade-off è gestito attraverso il controller centrale, che funge da punto di aggregazione senza annullare l'autonomia dei nodi. Il sistema risultante combina indipendenza locale e coordinamento globale, realizzando un equilibrio tra distribuzione e controllo.

Infine, l'indipendenza dei componenti rappresenta un requisito fondamentale per l'apertura del sistema. Solo un sistema composto da componenti indipendenti può essere esteso, modificato o integrato con moduli esterni senza compromettere la coerenza interna. Questo aspetto è essenziale per la trasformazione del progetto in un ecosistema, in cui diversi sviluppatori o sistemi possono contribuire in modo autonomo.

In conclusione, l'indipendenza dei componenti costituisce una proprietà chiave dell'architettura MicroBot, che consente di gestire la complessità, supportare l'evoluzione, migliorare la robustezza e garantire la scalabilità.

del sistema. Essa rappresenta il complemento necessario del design modulare e prepara il terreno per l'analisi delle relazioni tra componenti, che verrà sviluppata nei punti successivi della sezione.

3.8 Sistema centralizzato vs distribuito

La distinzione tra sistemi centralizzati e sistemi distribuiti rappresenta uno dei nodi concettuali fondamentali nella progettazione dell'architettura MicroBot. Essa non riguarda soltanto una scelta implementativa, ma implica un diverso modo di concepire il controllo, la gestione dell'informazione e la relazione tra le componenti del sistema. Nel contesto del progetto, questa distinzione non viene risolta attraverso l'adozione esclusiva di uno dei due paradigmi, ma attraverso la definizione di un modello ibrido, in cui centralizzazione e distribuzione coesistono e si integrano.

Un sistema centralizzato è caratterizzato dalla presenza di un'entità principale che possiede una visione globale dello stato e che prende decisioni per l'intero sistema. Formalmente, il comportamento può essere descritto come:

$$u(t) = C(s(t))$$

dove C rappresenta il controller centrale e $s(t)$ lo stato globale. In questo modello, il flusso di controllo è di tipo top-down: le informazioni vengono raccolte, elaborate centralmente e trasformate in comandi che guidano le unità operative. I vantaggi di questo approccio risiedono nella coerenza del comportamento, nella facilità di coordinazione e nella possibilità di implementare strategie globali complesse.

Tuttavia, il modello centralizzato presenta limiti significativi. In primo luogo, esso introduce un punto singolo di fallimento: un malfunzionamento del controller può compromettere l'intero sistema. Inoltre, la scalabilità è limitata, poiché l'aumento del numero di unità comporta un incremento del carico computazionale e comunicativo sul nodo centrale. Infine, la dipendenza da una visione globale può ridurre la capacità del sistema di adattarsi a variazioni locali e condizioni non previste.

Al contrario, un sistema distribuito è composto da un insieme di nodi che operano sulla base di informazioni locali, senza un controllo centrale dominante. In questo caso, il comportamento globale emerge dalle interazioni tra le unità:

$$s_i(t+1) = f(s_i(t), N_i(t), E(t))$$

dove ogni nodo aggiorna il proprio stato in funzione del proprio stato precedente, del vicinato e dell'ambiente. Il controllo è di tipo bottom-up, e non esiste una funzione globale che determini direttamente il comportamento dell'intero sistema.

I sistemi distribuiti offrono vantaggi significativi in termini di robustezza e scalabilità. La perdita di singole unità non compromette il funzionamento globale, e l'aggiunta di nuovi nodi non richiede modifiche strutturali al sistema. Inoltre, la natura locale delle interazioni consente una maggiore adattabilità e flessibilità.

Tuttavia, anche il modello distribuito presenta criticità. In particolare, il comportamento emergente può risultare difficile da prevedere e controllare, e la mancanza di una visione globale può rendere complessa l'implementazione di strategie coordinate. Inoltre, la gestione della sincronizzazione e della coerenza tra i nodi può richiedere meccanismi aggiuntivi.

Nel sistema MicroBot, questi due paradigmi non vengono considerati come alternative, ma come componenti complementari di un'unica architettura. Il sistema adotta infatti un modello ibrido, in cui il controller centrale fornisce un livello di coordinamento globale, mentre i MicroBot operano come nodi distribuiti che implementano comportamenti locali.

In questo modello, il comportamento del sistema può essere descritto come:

$$\begin{aligned} s_i(t+1) &= f(s_i(t), N_i(t), u_i(t), E(t)) \\ u_i(t) &= C(\hat{s}(t)) \end{aligned}$$

dove $\hat{s}(t)$ rappresenta una rappresentazione aggregata dello stato globale. Il controller non impone direttamente il comportamento delle unità, ma genera comandi che influenzano le dinamiche locali, lasciando spazio all'emergenza del comportamento collettivo.

Questa struttura introduce una distinzione tra controllo globale e comportamento locale. Il controllo globale definisce obiettivi, vincoli e strategie, mentre il comportamento locale realizza tali obiettivi attraverso interazioni distribuite. Il risultato è un sistema in cui il comportamento finale non è completamente determinato né completamente emergente, ma il risultato di un'interazione tra i due livelli.

Dal punto di vista architetturale, il modello ibrido si riflette nella distribuzione delle responsabilità tra i diversi componenti. Il controller centrale gestisce la sincronizzazione, la raccolta dei dati e la definizione delle strategie, mentre i MicroBot implementano le regole locali e le interazioni fisiche. I moduli di comunicazione e simulazione fungono da intermediari, collegando i diversi livelli e garantendo la coerenza del sistema.

Un aspetto particolarmente rilevante riguarda la gestione dell'informazione. Nel modello centralizzato, l'informazione è concentrata in un unico punto, mentre nel modello distribuito è dispersa tra i nodi. Nel sistema MicroBot, l'informazione è parzialmente distribuita e parzialmente aggregata, creando una struttura in cui la conoscenza globale è approssimata attraverso la raccolta di dati locali. Questo introduce un livello di astrazione che consente di bilanciare completezza e efficienza.

Dal punto di vista teorico, il modello ibrido può essere interpretato come una combinazione di due dinamiche nello spazio degli stati. Da un lato, la dinamica globale definita dal controller:

$$s(t+1) = F_{\text{global}}(s(t))$$

dall'altro, la dinamica locale dei nodi:

$$s_i(t+1) = F_{\text{local}}(s_i(t), N_i(t))$$

Il comportamento del sistema emerge dall'interazione tra queste due funzioni, creando una struttura dinamica complessa.

Un ulteriore elemento di connessione riguarda il concetto di sistema come processo, sviluppato nelle sezioni teoriche avanzate. Il modello ibrido riflette una visione in cui il sistema non è governato da una singola funzione, ma da una rete di processi interconnessi, ciascuno dei quali contribuisce alla definizione dello stato globale.

Infine, il modello centralizzato-distribuito rappresenta la base per la scalabilità e l'evoluzione del sistema. La presenza di un controller consente di mantenere il controllo e la coordinazione, mentre la distribuzione delle unità permette di estendere il sistema senza limiti strutturali rigidi. Questa combinazione rende il sistema MicroBot particolarmente adatto a contesti dinamici e complessi.

In conclusione, la distinzione tra sistemi centralizzati e distribuiti, e la loro integrazione in un modello ibrido, costituisce uno degli elementi chiave dell'architettura MicroBot. Essa consente di combinare controllo e flessibilità, coerenza e adattabilità, creando un sistema capace di gestire la complessità attraverso l'interazione tra livelli diversi. Questo approccio rappresenta non solo una scelta tecnica, ma una visione progettuale che permea l'intero sistema.

3.9 Flusso delle informazioni

Il flusso delle informazioni all'interno del sistema MicroBot rappresenta la struttura dinamica attraverso cui lo stato del sistema viene generato, trasformato, distribuito e reso osservabile. In un'architettura complessa e multilivello come quella descritta nelle sezioni precedenti, l'informazione non può essere considerata un semplice dato statico, ma deve essere interpretata come un'entità dinamica che attraversa diversi domini, assumendo forme differenti a seconda del livello in cui viene elaborata.

Il sistema MicroBot può essere descritto come una rete di trasformazioni informative, in cui ogni componente riceve, elabora e trasmette informazioni secondo regole specifiche. Questo processo può essere formalizzato come una composizione di funzioni:

$$s(t+1) = \Phi_n \circ \Phi_{n-1} \circ \dots \circ \Phi_2 \circ \Phi_1 (s(t))$$

dove ciascuna funzione Φ_i rappresenta una trasformazione applicata a uno specifico livello dell'architettura. Questa formulazione evidenzia come lo stato del sistema non sia determinato da un singolo passaggio, ma da una sequenza di elaborazioni distribuite.

Per comprendere il flusso delle informazioni, è necessario distinguere tra diverse tipologie di dati che attraversano il sistema:

- dati di stato locale (stato dei MicroBot, parametri interni)
- dati di comunicazione (pacchetti, comandi, timestamp)
- dati di controllo (strategie, pattern, configurazioni)
- dati di simulazione (modelli virtuali, stati previsti)
- dati di visualizzazione (output per interfaccia utente)
- dati di logging e telemetria (eventi, metriche, errori)

Queste categorie non sono isolate, ma interagiscono continuamente, generando un flusso complesso che attraversa tutti i livelli del sistema.

A livello più basso, il flusso ha origine nei MicroBot, che generano dati relativi al proprio stato locale. Questi dati includono informazioni come posizione, stato energetico, configurazione magnetica e stato logico interno. Nei tuoi zip, questa parte è riflessa chiaramente nei moduli di firmware e nei sistemi di telemetria, dove ogni nodo produce continuamente aggiornamenti che rappresentano la sua condizione operativa.

Questi dati vengono trasmessi attraverso il livello di comunicazione, utilizzando protocolli leggeri come UDP o connessioni persistenti come WebSocket nel sistema unificato. La struttura dei pacchetti — contenente identificatore, comando e timestamp — definisce il formato minimo attraverso cui l'informazione può essere interpretata correttamente dagli altri componenti. Nei tuoi progetti, questa struttura è visibile nei moduli di rete e nei layer di comunicazione presenti sia nella simulazione che nella piattaforma web.

Una volta raccolti, i dati vengono aggregati dal controller centrale, che costruisce una rappresentazione sintetica dello stato globale. Questo processo non è una semplice somma dei dati locali, ma una trasformazione che tiene conto di latenze, inconsistenze e perdita di informazioni. Nei tuoi sistemi reali (ad esempio il backend con endpoint e WebSocket), questa fase è implementata attraverso moduli di orchestrazione che mantengono uno stato condiviso e aggiornato.

Il controller, a sua volta, genera dati di controllo sotto forma di comandi, pattern o configurazioni. Questi dati vengono distribuiti nuovamente ai nodi, chiudendo il ciclo di controllo. Questo ciclo rappresenta il cuore dinamico del sistema:

stato locale → comunicazione → aggregazione → decisione → comando → aggiornamento stato

Nei tuoi zip, questa pipeline è chiaramente visibile nella presenza di moduli distinti per input, elaborazione e output, così come nei file di configurazione e nei sistemi di logging che tracciano ogni fase del processo.

Parallelamente al sistema reale, esiste un flusso informativo associato alla simulazione. I modelli virtuali replicano il comportamento dello swarm, permettendo di generare dati previsionali e di testare strategie senza coinvolgere direttamente l'hardware. Nei tuoi progetti Python e web, questo è evidente nei moduli di simulazione e nei dataset utilizzati per rappresentare lo stato dello swarm nel tempo. Il flusso della simulazione può essere visto come un ramo parallelo:

$$s_sim(t+1) = \Phi_sim(s_sim(t), u(t))$$

che interagisce con il flusso reale attraverso confronti, validazioni e sincronizzazioni.

Un ulteriore livello del flusso riguarda la rappresentazione e la visualizzazione. I dati aggregati vengono trasformati in formati adatti all'interfaccia utente, come dashboard, grafici o visualizzazioni dello swarm. Nei tuoi sistemi web, questo è implementato attraverso componenti frontend che ricevono dati in tempo reale

tramite WebSocket e li trasformano in rappresentazioni visive dinamiche. Questa trasformazione può essere descritta come:

$$R(t) = \Psi(s(t))$$

dove $R(t)$ rappresenta la rappresentazione percepita dall'utente.

Un elemento fondamentale del flusso informativo è rappresentato dal sistema di logging e telemetria. Nei tuoi zip, la presenza di cartelle come data logs, telemetry e runtime evidenzia un'attenzione particolare alla raccolta e all'analisi dei dati. Questi sistemi registrano eventi, errori e metriche, creando una memoria del comportamento del sistema nel tempo. Questo introduce una dimensione temporale estesa, in cui il flusso informativo non è solo istantaneo, ma anche storico.

Dal punto di vista teorico, il flusso delle informazioni può essere interpretato come una dinamica nello spazio degli stati. Ogni trasformazione Φ_i modifica la configurazione del sistema, spostandolo da uno stato all'altro. Questo porta a una visione in cui il sistema non è un oggetto statico, ma un processo continuo di trasformazione informazionale.

Un aspetto particolarmente rilevante riguarda la coerenza del flusso. In presenza di latenze, perdita di pacchetti o errori, il sistema deve garantire che le informazioni rimangano consistenti e utilizzabili. Questo richiede l'introduzione di meccanismi come timestamp, sincronizzazione, buffering e correzione degli errori, che sono già presenti nei tuoi moduli di comunicazione e nei sistemi di gestione del runtime.

Infine, il flusso delle informazioni rappresenta il punto di integrazione tra tutti i livelli del sistema: nodi distribuiti, controller centrale, simulazione, interfaccia utente e sistemi di logging. Ogni componente contribuisce a questo flusso, creando una rete complessa di trasformazioni che definisce il comportamento complessivo del sistema.

In conclusione, il flusso delle informazioni nel sistema MicroBot non è un semplice passaggio di dati, ma una struttura dinamica che integra tutti i livelli dell'architettura. Esso rappresenta il meccanismo attraverso cui lo stato del sistema viene generato, trasformato e reso osservabile, costituendo il fondamento per la comprensione e il controllo del comportamento dello swarm. Questa struttura prepara il terreno per l'analisi successiva dei processi di input, elaborazione e output, che verranno approfonditi nei punti seguenti della sezione.

3.10 Input, elaborazione e output

Il sistema MicroBot, considerato nella sua totalità, può essere formalizzato come una pipeline computazionale distribuita, in cui ogni fase del processo — acquisizione dei dati, trasformazione e produzione di output — è realizzata attraverso l'interazione tra componenti eterogenee. In continuità con il flusso informativo descritto nella sezione precedente, il sistema non si limita a trasmettere dati, ma li trasforma continuamente, generando uno stato dinamico che evolve nel tempo.

Questa pipeline può essere descritta come una funzione composta:

$$O(t) = \Lambda(\Phi(I(t)))$$

dove $I(t)$ rappresenta l'insieme degli input, Φ la funzione di elaborazione e Λ la funzione di output. Tuttavia, nel caso MicroBot, ciascuna di queste funzioni non è localizzata in un singolo punto del sistema, ma distribuita su più livelli, rendendo la pipeline intrinsecamente parallela e dinamica.

La fase di input rappresenta il punto di ingresso dell'informazione nel sistema. Gli input possono provenire da diverse fonti, ciascuna associata a un dominio specifico. Una prima categoria è costituita dagli input provenienti dai MicroBot, che includono dati di telemetria, stato locale e eventi generati dai nodi. Questi dati riflettono direttamente la condizione fisica del sistema e costituiscono la base per qualsiasi elaborazione successiva. Nei tuoi archivi, questa tipologia di input è rappresentata dai sistemi di telemetry, dai file di data logs e dai moduli runtime che raccolgono informazioni in tempo reale.

Una seconda categoria è rappresentata dagli input utente, che includono comandi, configurazioni e interazioni attraverso l'interfaccia. Questi input introducono una dimensione esterna al sistema, permettendo

di guidare il comportamento dello swarm e di modificare i parametri operativi. Nei tuoi progetti web e nelle piattaforme unificate, questa componente è gestita attraverso dashboard, interfacce interattive e endpoint di controllo.

Una terza categoria è costituita dagli input provenienti dalla simulazione. I modelli virtuali generano dati che possono essere utilizzati per testare strategie, validare comportamenti e prevedere l'evoluzione del sistema. Questi input rappresentano una forma di informazione sintetica, che può essere integrata con i dati reali per migliorare la qualità delle decisioni.

Una volta acquisiti, gli input vengono sottoposti alla fase di elaborazione, che rappresenta il cuore computazionale del sistema. Questa fase è distribuita tra diversi componenti, tra cui il controller centrale, i moduli locali dei MicroBot e i sistemi di simulazione. L'elaborazione include operazioni di aggregazione, filtraggio, trasformazione e decisione.

Il controller centrale svolge un ruolo chiave in questa fase, aggregando i dati provenienti dai nodi e costruendo una rappresentazione coerente dello stato globale. Questa rappresentazione, indicata come $\hat{s}(t)$, non coincide necessariamente con lo stato reale, ma costituisce una stima operativa che tiene conto delle limitazioni del sistema. Su questa base, il controller applica funzioni decisionali:

$$u(t) = C(\hat{s}(t), I_{\text{ext}}(t))$$

dove $I_{\text{ext}}(t)$ rappresenta gli input esterni, come comandi utente o parametri di configurazione.

Parallelamente, ogni MicroBot esegue una propria elaborazione locale, basata sul proprio stato e sulle informazioni ricevute. Questo introduce una seconda dimensione della computazione, in cui decisioni locali vengono prese indipendentemente dal controller, contribuendo al comportamento emergente del sistema. La combinazione di elaborazione globale e locale rappresenta una caratteristica distintiva della pipeline MicroBot.

Un ulteriore livello di elaborazione è rappresentato dai moduli intermedi, come adapters e bridges, che trasformano i dati tra diversi formati e domini. Questi moduli permettono di collegare componenti eterogenee, come hardware, simulazione e interfaccia utente, garantendo la coerenza del sistema. Nei tuoi archivi, questa funzione è chiaramente visibile nei layer di integrazione tra moduli Python, sistemi web e componenti embedded.

La fase di output rappresenta la trasformazione finale dell'informazione, in cui i risultati dell'elaborazione vengono tradotti in azioni o rappresentazioni. Gli output possono assumere diverse forme, a seconda del livello del sistema. A livello fisico, gli output corrispondono ai comandi inviati ai MicroBot, che determinano il loro comportamento, come l'attivazione delle bobine, il movimento o il cambiamento di stato. A livello logico, gli output includono aggiornamenti dello stato, eventi e dati di sistema.

A livello di interfaccia, gli output sono rappresentati da visualizzazioni e feedback per l'utente. Nei tuoi sistemi web, questo include rendering dello swarm, aggiornamenti in tempo reale tramite WebSocket e visualizzazione di metriche. Questa trasformazione può essere formalizzata come:

$$R(t) = \Psi(\hat{s}(t))$$

dove $R(t)$ rappresenta la rappresentazione percepita.

Un elemento fondamentale della pipeline è la presenza di cicli di feedback. Gli output generati influenzano lo stato del sistema, che a sua volta produce nuovi input, creando un ciclo continuo:

$$I(t) \rightarrow \Phi \rightarrow O(t) \rightarrow I(t+1)$$

Questo ciclo rappresenta la natura dinamica del sistema, in cui ogni iterazione modifica la configurazione globale. Nei tuoi sistemi, questo comportamento è evidente nei loop di simulazione, nei cicli runtime e nei processi di aggiornamento continuo dei dati.

Dal punto di vista teorico, la pipeline input-elaborazione-output può essere interpretata come una dinamica nello spazio degli stati, in cui ogni trasformazione corrisponde a una funzione che modifica la configurazione del sistema. Questo collega direttamente la struttura computazionale con il modello multidimensionale introdotto nelle sezioni teoriche, in cui lo stato s evolve nel tempo attraverso trasformazioni successive.

Un aspetto critico riguarda la latenza e la sincronizzazione. Poiché le diverse fasi della pipeline sono distribuite, è necessario garantire che le informazioni siano coerenti nel tempo. Questo richiede l'utilizzo di timestamp, buffer e meccanismi di sincronizzazione, già presenti nei tuoi moduli di comunicazione e nei sistemi di gestione del tempo.

Infine, la pipeline MicroBot non è statica, ma evolutiva. Le diverse versioni del sistema, descritte nella sezione 3.0, possono essere viste come configurazioni alternative della pipeline, in cui vengono introdotte nuove trasformazioni o modificate quelle esistenti. Questo rende la pipeline non solo un modello operativo, ma anche uno strumento per descrivere l'evoluzione del sistema.

In conclusione, la struttura input-elaborazione-output rappresenta una formalizzazione del comportamento del sistema MicroBot come processo computazionale distribuito. Essa integra tutte le componenti dell'architettura — nodi, controller, simulazione, interfaccia e logging — in un'unica pipeline dinamica, costituendo la base per la comprensione e il controllo del sistema. Questa struttura prepara il terreno per l'analisi dell'interazione uomo-macchina, in cui il ruolo dell'utente viene integrato all'interno del ciclo operativo.

3.11 Interazione uomo-macchina

L'interazione uomo-macchina nel sistema MicroBot rappresenta il livello più alto dell'architettura e costituisce il punto di convergenza tra percezione, controllo e rappresentazione del sistema. In continuità con quanto sviluppato nei punti precedenti, essa non deve essere interpretata come un semplice strato di input/output, ma come una componente attiva del sistema, integrata nel ciclo operativo e capace di influenzarne direttamente l'evoluzione.

Nel modello generale definito nella sezione 3.10, l'utente può essere considerato come una sorgente esterna di input che agisce sul sistema attraverso una funzione di controllo:

$$u(t) = C(\hat{s}(t), I_user(t))$$

dove $I_user(t)$ rappresenta l'insieme delle azioni e delle decisioni dell'utente. Tuttavia, questa rappresentazione è riduttiva, in quanto non considera il ruolo dell'interfaccia come mediatore tra stato del sistema e percezione umana. È quindi necessario introdurre una struttura più completa, in cui l'interazione viene modellata come un ciclo chiuso:

utente $\rightarrow$ interfaccia $\rightarrow$ sistema $\rightarrow$ rappresentazione $\rightarrow$ utente

Questo ciclo evidenzia come l'utente non agisca direttamente sul sistema, ma attraverso una serie di trasformazioni che traducono lo stato interno in forme percepibili e viceversa.

Dal punto di vista architetturale, l'interazione uomo-macchina si basa su una serie di componenti già presenti nei tuoi sistemi, in particolare nei moduli web, nelle dashboard e nei sistemi di comunicazione in tempo reale. L'utilizzo di tecnologie come WebSocket consente di stabilire un canale bidirezionale a bassa latenza, attraverso cui l'utente può ricevere aggiornamenti continui dello stato del sistema e inviare comandi in tempo reale. Questo tipo di comunicazione trasforma l'interfaccia da elemento passivo a componente dinamico del sistema.

Un aspetto fondamentale dell'interazione riguarda la rappresentazione dello stato. Come già introdotto nella sezione 3.2, lo stato globale $s(t)$ deve essere trasformato in una forma percepibile $R(t)$ attraverso una funzione di rappresentazione:

$$R(t) = \Psi(s(t))$$

Questa trasformazione può assumere diverse forme, tra cui visualizzazione grafica dello swarm, rappresentazioni simboliche o feedback dinamici. Nei tuoi progetti web, questa funzione è implementata

attraverso rendering in tempo reale, visualizzazioni interattive e componenti UI che riflettono lo stato del sistema.

Un ulteriore livello di complessità è introdotto dalla possibilità di estendere l'interazione oltre le interfacce tradizionali. Tecnologie come realtà virtuale, realtà aumentata e sistemi di gestire control permettono di creare modalità di interazione immersive, in cui l'utente può percepire e manipolare il sistema in modo più diretto. In questo contesto, lo spazio dell'interfaccia non è più bidimensionale, ma tridimensionale o addirittura multimodale, includendo input visivi, spaziali e gestuali.

Questa evoluzione dell'interfaccia introduce una trasformazione nel ruolo dell'utente, che passa da osservatore a partecipante attivo nel sistema. L'utente non si limita a inviare comandi, ma interagisce con lo stato del sistema in modo continuo, influenzandone la dinamica. Questo porta a una ridefinizione del ciclo operativo, in cui l'interazione diventa parte integrante del processo:

$$s(t+1) = \Phi(s(t), u_{\text{system}}(t), u_{\text{user}}(t))$$

dove $u_{\text{user}}(t)$ rappresenta l'influenza diretta dell'utente.

Dal punto di vista teorico, l'interazione uomo-macchina può essere interpretata come una dimensione aggiuntiva del sistema, in cui la percezione e l'azione umana si integrano con la dinamica interna. Questo collega direttamente l'architettura MicroBot con il modello multidimensionale sviluppato nelle sezioni teoriche, in cui lo stato del sistema non è solo una variabile interna, ma una configurazione che può essere osservata e modificata.

Un aspetto particolarmente rilevante riguarda la gestione della complessità percettiva. Poiché il sistema può generare una quantità elevata di informazioni, è necessario progettare l'interfaccia in modo da evidenziare gli elementi più significativi, riducendo il rumore e facilitando la comprensione. Questo richiede l'utilizzo di tecniche di astrazione, filtraggio e aggregazione, che permettano di adattare la rappresentazione al contesto e all'obiettivo dell'utente.

Nel contesto dei tuoi sistemi, questa esigenza è evidente nella presenza di dashboard strutturate, sistemi di logging e visualizzazioni modulari, che consentono di analizzare il comportamento del sistema a diversi livelli di dettaglio. L'interfaccia diventa quindi uno strumento di analisi oltre che di controllo.

Un ulteriore elemento di connessione riguarda il ruolo dell'interazione nel processo di sviluppo. Nei tuoi progetti, l'interfaccia non è solo utilizzata per il controllo del sistema finale, ma anche come strumento per il debug, la validazione e l'esplorazione delle dinamiche dello swarm. Questo introduce una dimensione metaprogettuale, in cui l'interazione contribuisce direttamente all'evoluzione del sistema.

Infine, l'interazione uomo-macchina rappresenta un elemento chiave per la trasformazione del sistema MicroBot in una piattaforma applicativa. La possibilità di controllare, osservare e manipolare il sistema in modo intuitivo apre la strada a numerose applicazioni, che verranno approfondite nelle sezioni successive. In particolare, l'integrazione con tecnologie immersive e sistemi di controllo avanzati può estendere significativamente le capacità del sistema.

In conclusione, l'interazione uomo-macchina nel sistema MicroBot non è un semplice livello di interfaccia, ma una componente strutturale che integra percezione, controllo e rappresentazione. Essa consente di collegare il comportamento interno del sistema con l'esperienza dell'utente, trasformando un sistema complesso in un'entità osservabile, controllabile ed evolvibile. Questa integrazione rappresenta il punto culminante della Sezione 3 e prepara il passaggio alle sezioni successive, in cui verranno analizzate in dettaglio le componenti fisiche ed elettroniche del sistema.

SEZIONE 4 — Il MicroBot (Unità Base)

4.1 Definizione del MicroBot

Il MicroBot rappresenta l'unità fondamentale del sistema, ovvero il blocco elementare a partire dal quale viene costruita l'intera architettura distribuita. In continuità con quanto sviluppato nella Sezione 3, il MicroBot non deve essere interpretato come un robot autonomo nel senso tradizionale, ma come un nodo fisico all'interno di un sistema complesso, la cui funzione emerge dall'interazione con altre unità analoghe.

Dal punto di vista concettuale, il MicroBot costituisce una discretizzazione della materia programmabile. Ogni unità è infatti progettata per essere semplice, replicabile e controllabile, in modo tale che il comportamento globale del sistema non sia determinato dalla complessità del singolo elemento, ma dalla configurazione collettiva dello swarm. Questo approccio riflette direttamente il passaggio da sistemi centralizzati a sistemi distribuiti, già discusso nella sezione 3.8.

Formalmente, un MicroBot può essere definito come un sistema dinamico locale:

$$n_i = (s_i, a_i, c_i)$$

dove s_i rappresenta lo stato interno, a_i l'insieme delle azioni possibili e c_i i canali di comunicazione. Lo stato interno include variabili fisiche ed elettroniche, mentre le azioni rappresentano le operazioni che il nodo può eseguire, come attivare un campo magnetico o modificare il proprio comportamento. I canali di comunicazione definiscono le modalità attraverso cui il MicroBot interagisce con gli altri nodi e con il controller.

Un aspetto fondamentale della definizione del MicroBot riguarda la limitazione delle sue capacità. Ogni unità è progettata per operare con risorse ridotte, sia in termini di potenza computazionale sia di energia disponibile. Questa scelta non rappresenta un limite, ma un vincolo progettuale che favorisce l'emergenza del comportamento collettivo. Infatti, riducendo la complessità del singolo nodo, si rende necessario il coordinamento tra più unità per ottenere funzionalità avanzate.

Dal punto di vista architetturale, il MicroBot si colloca al livello fisico del sistema, ma è strettamente connesso ai livelli superiori attraverso il flusso delle informazioni. I dati generati dal nodo contribuiscono alla costruzione dello stato globale, mentre i comandi ricevuti influenzano il comportamento locale. Questo rende il MicroBot un elemento attivo all'interno della pipeline input-elaborazione-output descritta nella sezione 3.10.

Un ulteriore elemento distintivo è la relazione tra MicroBot e spazio. Poiché ogni unità occupa una posizione fisica, la distribuzione dei nodi nello spazio determina direttamente la forma del sistema. Tuttavia, questa forma non è statica, ma evolve nel tempo in funzione delle interazioni tra le unità. Questo porta a una rappresentazione in cui la geometria del sistema è una funzione dello stato:

$$\text{Forma}(t) = F(\{s_1(t), s_2(t), \dots, s_n(t)\})$$

Questa relazione collega direttamente il livello fisico con il modello teorico multidimensionale, in cui la forma è interpretata come manifestazione dello stato del sistema.

Nel contesto dei tuoi progetti reali, questa definizione trova riscontro nei prototipi hardware basati su microcontrollori (come ESP32), LED e componenti magnetici. Anche nelle versioni più semplici, come i prototipi a LED, il MicroBot svolge già il ruolo di nodo distribuito, mostrando come sia possibile simulare comportamenti complessi a partire da unità elementari.

Un aspetto importante riguarda la replicabilità del MicroBot. Poiché il sistema è progettato per essere scalabile, ogni unità deve poter essere riprodotta in modo consistente, mantenendo le stesse caratteristiche funzionali. Questo implica una standardizzazione della struttura e dei componenti, che verrà approfondita nei punti successivi della sezione.

Dal punto di vista dinamico, il MicroBot contribuisce al comportamento del sistema attraverso l'aggiornamento del proprio stato locale:

$$s_i(t+1) = f(s_i(t), N_i(t), u_i(t), E(t))$$

Questa equazione, già introdotta nella sezione 3.4, evidenzia come il nodo operi sulla base di informazioni locali e comandi ricevuti, contribuendo al comportamento globale senza avere una visione completa del sistema.

Infine, il MicroBot rappresenta il punto di contatto tra teoria e implementazione. Esso incarna i principi sviluppati nelle sezioni precedenti — distribuzione, modularità, indipendenza e flusso informativo —

traducendoli in un oggetto fisico concreto. Questa doppia natura rende il MicroBot non solo un componente tecnico, ma anche un elemento chiave per la validazione del modello teorico.

In conclusione, il MicroBot può essere definito come un nodo fisico intelligente, progettato per operare all'interno di un sistema distribuito, la cui funzione emerge dall'interazione con altre unità. Esso rappresenta l'unità base del sistema MicroBot e costituisce il fondamento per tutte le strutture e i comportamenti analizzati nelle sezioni successive.

4.2 Nodo intelligente distribuito

Nel contesto del sistema MicroBot, il concetto di nodo intelligente distribuito rappresenta una ridefinizione del significato di "intelligenza" in ambito robotico. A differenza dei modelli tradizionali, in cui l'intelligenza è concentrata in un'unità centrale o in un singolo agente autonomo, nel sistema MicroBot essa è distribuita tra una molteplicità di unità semplici, ciascuna delle quali contribuisce al comportamento globale attraverso interazioni locali.

Un MicroBot non è intelligente nel senso classico del termine, ovvero non possiede una capacità di ragionamento complesso o una conoscenza globale del sistema. La sua "intelligenza" deriva invece dalla capacità di reagire a stimoli locali, aggiornare il proprio stato e partecipare a dinamiche collettive. Questa forma di intelligenza può essere definita come intelligenza emergente distribuita, in cui il comportamento globale non è programmato direttamente, ma emerge dalla combinazione delle azioni locali.

Dal punto di vista formale, ogni nodo può essere modellato come un sistema di decisione locale, descritto da una funzione:

$$a_i(t) = \pi(s_i(t), N_i(t), u_i(t))$$

dove $a_i(t)$ rappresenta l'azione eseguita dal nodo, $s_i(t)$ il suo stato interno, $N_i(t)$ l'insieme dei nodi vicini e $u_i(t)$ i comandi ricevuti dal controller. La funzione π rappresenta la politica locale, ovvero l'insieme delle regole che determinano il comportamento del nodo.

Questa formulazione evidenzia come l'intelligenza del sistema non risieda nella complessità della funzione π per il singolo nodo, ma nella struttura delle interazioni tra i nodi. Anche con politiche locali relativamente semplici, l'interazione tra molte unità può generare comportamenti complessi, adattivi e difficilmente prevedibili.

Un aspetto fondamentale dell'intelligenza distribuita è la località dell'informazione. Ogni MicroBot ha accesso solo a una porzione limitata dello stato del sistema, costituita dal proprio stato interno e dalle informazioni provenienti dai nodi vicini o dal controller. Questo implica che le decisioni vengono prese in condizioni di informazione incompleta, rendendo il sistema intrinsecamente robusto rispetto a errori o perdita di dati.

Nel sistema MicroBot, questa località è rafforzata sia a livello logico sia a livello fisico. A livello logico, i nodi comunicano attraverso messaggi limitati e strutturati, senza accesso diretto allo stato globale. A livello fisico, le interazioni magnetiche avvengono solo tra unità vicine, introducendo un vincolo naturale alla propagazione delle informazioni. Questa combinazione crea una struttura in cui l'informazione si diffonde gradualmente attraverso il sistema.

Un altro elemento chiave riguarda la temporalità delle decisioni. Ogni nodo opera in un ciclo discreto, in cui osserva il proprio stato, riceve input e aggiorna la propria configurazione. Questo ciclo può essere sincronizzato attraverso il controller centrale o gestito localmente, introducendo una dinamica temporale che contribuisce al comportamento globale. Nei tuoi sistemi, questo è evidente nei loop di runtime e nei cicli di aggiornamento della simulazione.

Dal punto di vista implementativo, l'intelligenza del nodo è realizzata attraverso firmware embedded, che esegue logiche di controllo basate su stati e condizioni. Nei prototipi che hai sviluppato, questo si manifesta in forma semplificata, ad esempio nel controllo dei LED o nella gestione di segnali, ma costituisce la base per comportamenti più complessi. L'evoluzione futura del sistema può includere logiche più avanzate, mantenendo però il principio di semplicità locale.

Un aspetto particolarmente rilevante è la relazione tra intelligenza locale e comportamento emergente. Nel sistema MicroBot, il comportamento collettivo — come la formazione di strutture, l'allineamento o la

separazione — non è imposto direttamente, ma emerge dalle interazioni tra nodi. Questo fenomeno può essere descritto come una funzione emergente:

$$B(t) = E(\{a_1(t), a_2(t), \dots, a_n(t)\})$$

dove $B(t)$ rappresenta il comportamento globale e E una funzione che descrive l'emergenza. Questa funzione non è esplicitamente definita, ma deriva dalla struttura del sistema e dalle regole locali.

Un ulteriore elemento di connessione riguarda il ruolo del controller centrale. Sebbene il sistema sia distribuito, il controller introduce un livello di coordinamento che influenza le decisioni locali. Questo crea una struttura ibrida, in cui l'intelligenza è sia distribuita sia guidata. Il controller non sostituisce l'intelligenza locale, ma la orienta, definendo condizioni e vincoli.

Dal punto di vista teorico, il nodo intelligente distribuito può essere interpretato come una componente di uno spazio degli stati multidimensionale. Ogni nodo rappresenta una variabile del sistema, e l'interazione tra nodi definisce la dinamica complessiva. Questa interpretazione collega direttamente il livello fisico con il modello teorico sviluppato nelle sezioni precedenti.

Un aspetto critico riguarda la gestione della complessità emergente. Poiché il comportamento globale non è completamente prevedibile, è necessario introdurre meccanismi di monitoraggio e controllo che permettano di analizzare e, se necessario, correggere le dinamiche del sistema. Nei tuoi progetti, questo ruolo è svolto dai sistemi di telemetria, logging e simulazione, che permettono di osservare il comportamento e di intervenire.

Infine, il concetto di nodo intelligente distribuito rappresenta il fondamento per l'evoluzione del sistema verso forme più avanzate di computazione distribuita. Integrando sensori, algoritmi più sofisticati o tecniche di apprendimento, ogni nodo può aumentare la propria capacità di adattamento, mantenendo comunque la struttura distribuita del sistema.

In conclusione, il MicroBot come nodo intelligente distribuito rappresenta un elemento fondamentale dell'architettura, in cui l'intelligenza non è concentrata, ma emerge dall'interazione tra unità semplici. Questa struttura consente di combinare robustezza, scalabilità e adattabilità, costituendo la base per lo sviluppo di sistemi complessi e dinamici.

4.3 Struttura geometrica del robot

La struttura geometrica del MicroBot rappresenta uno degli elementi più critici dell'intero sistema, in quanto definisce non solo la forma fisica dell'unità, ma anche le modalità attraverso cui essa interagisce con le altre unità e con l'ambiente. In continuità con quanto sviluppato nei punti precedenti, la geometria non può essere considerata un semplice aspetto estetico o costruttivo, ma costituisce una componente funzionale che influenza direttamente il comportamento del sistema distribuito.

Nel contesto MicroBot, la progettazione geometrica deve soddisfare una serie di requisiti fondamentali. In primo luogo, ogni unità deve essere compatta e simmetrica, al fine di garantire uniformità nelle interazioni. In secondo luogo, la forma deve facilitare l'aggregazione tra unità, permettendo la formazione di strutture stabili. Infine, la geometria deve essere compatibile con i vincoli interni, come la disposizione dei componenti elettronici e dei sistemi magnetici.

Dal punto di vista teorico, la geometria del MicroBot può essere interpretata come una funzione che collega lo stato interno del nodo con la sua capacità di interazione nello spazio. Se si considera lo stato del nodo s_i , la forma fisica può essere vista come una mappatura:

$$G: s_i \rightarrow \mathbb{R}^3$$

che determina come il nodo occupa lo spazio e come interagisce con altri nodi. Questa interpretazione collega direttamente la struttura geometrica con il modello multidimensionale del sistema, in cui la forma è una manifestazione dello stato.

Un aspetto fondamentale della progettazione geometrica riguarda la gestione delle superfici di contatto. Poiché le interazioni tra MicroBot sono mediate da forze magnetiche, la forma deve garantire che tali forze

possano essere applicate in modo efficace. Questo implica la presenza di superfici o punti di contatto ben definiti, che permettano l'allineamento tra unità e la trasmissione delle forze.

Nel sistema MicroBot, la geometria è progettata per supportare configurazioni multiple. Ogni unità deve poter interagire con altre in diverse orientazioni, senza richiedere un allineamento perfetto. Questo introduce una forma di tolleranza geometrica, che è fondamentale per il funzionamento reale del sistema, dove le condizioni ideali raramente si verificano.

Dal punto di vista fisico, la struttura geometrica influenza direttamente la distribuzione delle forze. La posizione dei magneti, la distanza tra le unità e l'orientamento delle superfici determinano l'intensità e la direzione delle interazioni. Questo rende la geometria un elemento chiave nella progettazione del sistema magnetico, che verrà approfondito nelle sezioni successive.

Un ulteriore elemento di rilievo riguarda la relazione tra geometria e stabilità. Le configurazioni dello swarm devono essere stabili nel tempo, evitando collassi o separazioni indesiderate. La forma del MicroBot deve quindi favorire configurazioni in cui le forze si equilibrano, creando strutture robuste. Questo introduce un collegamento tra geometria e comportamento emergente, in quanto la forma delle unità influenza direttamente le dinamiche collettive.

Dal punto di vista implementativo, la progettazione geometrica deve tenere conto dei vincoli di fabbricazione. Nei prototipi iniziali, la forma può essere semplificata per facilitare la costruzione, utilizzando componenti standard e strutture facilmente realizzabili. Tuttavia, con l'evoluzione del sistema, è possibile introdurre forme più complesse, ottimizzate per migliorare le prestazioni.

Un aspetto importante riguarda la scalabilità della geometria. Poiché il sistema è progettato per essere composto da molte unità, la forma del singolo MicroBot deve essere tale da permettere la costruzione di strutture più grandi senza introdurre inefficienze. Questo implica una progettazione modulare anche a livello geometrico, in cui ogni unità si integra naturalmente con le altre.

Dal punto di vista teorico, la struttura geometrica può essere interpretata come una discretizzazione di una forma continua. Lo swarm nel suo insieme può assumere configurazioni che approssimano superfici o volumi complessi, mentre ogni MicroBot rappresenta un elemento discreto di questa struttura. Questo collega direttamente il sistema al concetto di materia programmabile, in cui la forma globale è il risultato della configurazione delle unità elementari.

Un ulteriore collegamento riguarda la relazione tra geometria e percezione. La forma del sistema, risultante dalla disposizione dei MicroBot, è ciò che l'utente osserva attraverso l'interfaccia. Questo introduce un legame tra livello fisico e livello percettivo, in cui la geometria diventa un mezzo per rappresentare lo stato del sistema.

Infine, la progettazione geometrica deve considerare anche l'integrazione con gli altri componenti del sistema. La disposizione interna dei componenti elettronici, dei magneti e delle fonti di energia deve essere compatibile con la forma esterna, evitando interferenze e garantendo un funzionamento corretto. Questo richiede un approccio integrato, in cui geometria e architettura elettronica vengono progettate congiuntamente.

In conclusione, la struttura geometrica del MicroBot rappresenta un elemento centrale nella progettazione del sistema, influenzando le interazioni, la stabilità e il comportamento emergente dello swarm. Essa costituisce il punto di collegamento tra livello fisico e modello teorico, e rappresenta la base per le scelte progettuali che verranno approfondite nei punti successivi, in particolare per quanto riguarda la configurazione specifica della forma e la distribuzione dei componenti interni.

4.4 Configurazione a coni troncati e sfera

La configurazione geometrica del MicroBot basata su coni troncati e sfera rappresenta una scelta progettuale mirata a ottimizzare simultaneamente stabilità, interazione magnetica e flessibilità di aggregazione. In continuità con quanto discusso nella sezione precedente, la forma non è definita esclusivamente da criteri estetici o costruttivi, ma costituisce una componente funzionale che influenza direttamente il comportamento del sistema distribuito.

La struttura può essere descritta come una composizione di tre elementi principali: un corpo centrale approssimabile a una sfera e due estremità caratterizzate da superfici coniche troncate. Questa configurazione introduce una simmetria assiale che facilita l'orientamento del nodo nello spazio e ne semplifica le modalità di interazione con le unità adiacenti.

Dal punto di vista geometrico, la presenza della sfera centrale consente una distribuzione uniforme dei volumi interni e una maggiore libertà di orientamento. Una forma sferica riduce infatti le dipendenze direzionali, permettendo al MicroBot di interagire con altre unità indipendentemente dalla sua rotazione. Questo è particolarmente importante in un sistema distribuito, in cui non è possibile garantire un allineamento perfetto tra i nodi.

Le estremità a cono troncato introducono invece superfici di contatto controllate, che svolgono un ruolo fondamentale nelle interazioni magnetiche. A differenza di una superficie completamente sferica, che avrebbe punti di contatto indefiniti e instabili, i coni troncati forniscono aree ben definite in cui le forze possono essere applicate in modo più prevedibile. Questo permette di migliorare l'allineamento tra unità e di stabilizzare le configurazioni dello swarm.

Formalmente, la geometria del MicroBot può essere descritta come una funzione composta:

$$G = G_{\text{sfera}} \oplus G_{\text{cono}_1} \oplus G_{\text{cono}_2}$$

dove ciascun termine rappresenta una componente della struttura. Questa composizione definisce non solo la forma esterna, ma anche la distribuzione interna dei componenti, influenzando direttamente le proprietà fisiche del nodo.

Un aspetto fondamentale della configurazione riguarda la relazione tra geometria e campo magnetico. I magneti o le bobine elettromagnetiche, posizionati all'interno o in prossimità delle estremità, generano campi che interagiscono con quelli delle unità vicine. La presenza dei coni troncati consente di orientare tali campi lungo direzioni preferenziali, aumentando l'efficacia delle interazioni.

La distanza tra le superfici di contatto e la loro inclinazione determinano l'intensità delle forze e la stabilità delle connessioni. In particolare, una superficie troppo piccola può ridurre l'area di contatto e quindi la forza di attrazione, mentre una superficie troppo ampia può rendere difficile il distacco tra le unità. La configurazione a cono troncato rappresenta un compromesso tra questi due estremi, garantendo sia stabilità sia capacità di riconfigurazione.

Dal punto di vista dinamico, la geometria influenza direttamente le configurazioni possibili dello swarm. Le unità possono aggregarsi in strutture lineari, planari o tridimensionali, a seconda dell'orientamento reciproco e delle forze in gioco. La presenza di superfici coniche facilita la formazione di catene e cluster, mentre la sfera centrale consente una certa libertà di rotazione, permettendo al sistema di adattarsi a configurazioni diverse.

Un ulteriore elemento di rilievo riguarda la distribuzione interna dei componenti. La sfera centrale può ospitare il microcontrollore, la batteria e altri elementi elettronici, mentre le estremità possono essere dedicate ai sistemi magnetici e ai sensori. Questa separazione spaziale consente di ottimizzare l'organizzazione interna, riducendo interferenze e migliorando l'efficienza.

Dal punto di vista costruttivo, la configurazione a coni troncati e sfera presenta vantaggi anche in termini di fabbricazione. Le superfici semplici e regolari possono essere realizzate attraverso tecniche di stampa 3D o lavorazioni standard, permettendo una prototipazione rapida e iterativa. Nei tuoi prototipi iniziali, anche se semplificati, questo principio è già presente nella ricerca di forme che facilitino l'assemblaggio e l'interazione.

Un aspetto importante riguarda la tolleranza geometrica. In un sistema reale, le unità non saranno perfettamente identiche e le superfici non saranno perfettamente allineate. La scelta di una geometria che includa superfici inclinate e una componente sferica consente di assorbire queste imperfezioni, migliorando la robustezza del sistema.

Dal punto di vista teorico, la configurazione geometrica può essere interpretata come una funzione che mappa lo stato interno del nodo nello spazio delle interazioni. In altre parole, la forma del MicroBot determina quali configurazioni sono possibili e quali sono stabili, influenzando la dinamica complessiva del

sistema. Questo collega direttamente la geometria al modello multidimensionale, in cui la forma è una manifestazione dello stato.

Infine, la configurazione a coni troncati e sfera rappresenta un passo verso la realizzazione di unità di materia programmabile. La capacità di aggregarsi, separarsi e riconfigurarsi in modo controllato è direttamente legata alla forma del nodo, rendendo la geometria un elemento chiave per l'evoluzione futura del sistema.

In conclusione, la configurazione geometrica basata su coni troncati e sfera costituisce una soluzione progettuale che bilancia esigenze di stabilità, interazione e flessibilità. Essa rappresenta un elemento fondamentale per il funzionamento del sistema MicroBot e fornisce una base solida per lo sviluppo delle componenti interne e delle interazioni fisiche, che verranno analizzate nei punti successivi.

4.5 Componenti interni

La definizione dei componenti interni del MicroBot rappresenta il passaggio dalla modellazione astratta del nodo alla sua realizzazione fisica. In continuità con la struttura geometrica definita nella sezione precedente, i componenti interni devono essere progettati e disposti in modo coerente con i vincoli spaziali, energetici e funzionali del sistema. Ogni elemento non è indipendente, ma contribuisce alla definizione dello stato del nodo e, di conseguenza, al comportamento globale dello swarm.

Il MicroBot può essere interpretato come un sistema integrato in cui componenti elettronici, sistemi di attuazione e elementi strutturali coesistono in uno spazio limitato. Formalmente, lo stato interno del nodo può essere espresso come:

$$s_i = (s_compute, s_energy, s_magnetic, s_feedback)$$

dove ciascun termine rappresenta un sottosistema funzionale. Questa decomposizione riflette direttamente la distribuzione fisica dei componenti all'interno del MicroBot.

Il primo elemento fondamentale è il microcontrollore, che rappresenta il nucleo computazionale del nodo. Nei tuoi prototipi, l'utilizzo di piattaforme come ESP32 consente di integrare capacità di elaborazione, comunicazione e gestione degli input/output in un unico componente. Il microcontrollore esegue il firmware che implementa la logica locale, gestisce i segnali e coordina le operazioni interne. Dal punto di vista architetturale, esso realizza la funzione π descritta nella sezione 4.2, determinando le azioni del nodo in base allo stato e agli input.

Un secondo componente essenziale è il sistema di alimentazione, tipicamente basato su batterie Li-Po. Questo sottosistema definisce i limiti energetici del nodo e influisce direttamente sulla durata operativa e sulle capacità di attuazione. La gestione dell'energia non si limita alla fornitura di corrente, ma include anche regolazione della tensione, protezione e monitoraggio dello stato della batteria. Nei tuoi sistemi, questa componente è fondamentale per evitare instabilità e garantire un funzionamento sicuro.

Il sistema magnetico rappresenta uno degli elementi distintivi del MicroBot. Esso può essere realizzato attraverso magneti permanenti o bobine elettromagnetiche controllate. Nel primo caso, si ottiene un comportamento passivo, in cui le interazioni sono determinate dalla disposizione dei poli magnetici. Nel secondo caso, è possibile introdurre un controllo attivo, variando il campo magnetico attraverso l'applicazione di corrente. Questo sottosistema è responsabile delle forze di attrazione e repulsione che permettono l'aggregazione e la riconfigurazione dello swarm.

Un ulteriore componente è rappresentato dal sistema di feedback visivo, tipicamente implementato tramite LED RGB. Nei prototipi che hai sviluppato, i LED non sono solo un elemento estetico, ma svolgono una funzione informativa, permettendo di visualizzare lo stato del nodo. Il colore, l'intensità e il pattern di illuminazione possono essere utilizzati per rappresentare variabili come stato operativo, energia o ruolo all'interno dello swarm. Questo introduce un collegamento diretto tra stato interno e rappresentazione percepibile.

Dal punto di vista strutturale, la disposizione dei componenti deve rispettare la geometria definita nella sezione 4.4. La sfera centrale ospita tipicamente il microcontrollore e la batteria, mentre le estremità sono dedicate ai sistemi magnetici. Questa distribuzione consente di separare le funzioni e di ridurre interferenze

tra componenti. Inoltre, permette di mantenere un centro di massa equilibrato, migliorando la stabilità del nodo.

Un aspetto critico riguarda l'interazione tra i diversi sottosistemi. Il funzionamento del MicroBot richiede una coordinazione precisa tra computazione, energia e attuazione. Ad esempio, l'attivazione delle bobine magnetiche richiede una gestione accurata della corrente, che deve essere compatibile con le capacità della batteria e del circuito. Questo introduce vincoli progettuali che devono essere considerati fin dalle fasi iniziali.

Dal punto di vista termico, la presenza di componenti attivi può generare calore, che deve essere dissipato per evitare danni. Nei sistemi reali, questo richiede l'introduzione di strategie di gestione termica, come limitazione della corrente o distribuzione dei carichi. Anche se nei prototipi iniziali questo aspetto può essere trascurabile, diventa rilevante con l'aumentare della complessità.

Un ulteriore elemento di rilievo riguarda la miniaturizzazione. Poiché il sistema è basato su un elevato numero di unità, la dimensione del singolo MicroBot deve essere contenuta. Questo implica una selezione attenta dei componenti e una progettazione efficiente del layout interno. Nei tuoi prototipi, l'utilizzo di componenti compatti rappresenta già un primo passo in questa direzione.

Dal punto di vista della comunicazione, il microcontrollore integra moduli wireless che permettono lo scambio di dati con il controller e con gli altri nodi. Questo sottosistema collega direttamente il livello fisico con il flusso informativo descritto nella Sezione 3, rendendo il MicroBot un nodo attivo nella rete di comunicazione.

Un aspetto importante è la modularità interna. Anche all'interno del singolo nodo, i componenti possono essere organizzati in moduli funzionali, facilitando la manutenzione e l'evoluzione del design. Questo approccio riflette a livello micro la modularità del sistema globale.

Infine, i componenti interni rappresentano il punto di collegamento tra teoria e implementazione. Le variabili dello stato s_i trovano una corrispondenza diretta nei parametri fisici ed elettronici del nodo, rendendo possibile una traduzione concreta del modello teorico. Questo permette di validare le ipotesi e di osservare il comportamento del sistema in condizioni reali.

In conclusione, i componenti interni del MicroBot costituiscono l'infrastruttura fisica su cui si basa il funzionamento del nodo. La loro progettazione richiede un approccio integrato, che tenga conto di vincoli spaziali, energetici e funzionali, garantendo al contempo coerenza con l'architettura generale del sistema. Questa base consente di sviluppare le funzionalità avanzate del MicroBot e rappresenta il punto di partenza per l'analisi dettagliata dei singoli componenti, che verrà sviluppata nei punti successivi.

4.6 Microcontrollore

Il microcontrollore rappresenta il nucleo computazionale del MicroBot e costituisce il componente attraverso cui vengono implementate tutte le logiche di controllo locale, gestione delle risorse e interazione con l'ambiente. In continuità con quanto definito nella sezione 4.2, esso realizza concretamente la funzione di decisione locale del nodo, trasformando lo stato interno e gli input ricevuti in azioni operative.

Nel contesto del sistema MicroBot, la scelta del microcontrollore non è casuale, ma deve soddisfare una serie di requisiti specifici. Tra questi, la capacità di eseguire codice in tempo reale, la disponibilità di interfacce di comunicazione wireless, la gestione efficiente degli input/output e un consumo energetico compatibile con le limitazioni del sistema. Nei prototipi sviluppati, l'utilizzo di piattaforme come ESP32 risponde a queste esigenze, offrendo un compromesso tra prestazioni, integrazione e flessibilità.

Dal punto di vista architetturale, il microcontrollore può essere interpretato come una macchina a stati, in cui il comportamento del nodo è determinato da una sequenza di transizioni tra stati interni. Formalmente, il sistema può essere descritto come:

$$s_i(t+1) = f(s_i(t), \text{input}(t))$$

dove la funzione f è implementata dal firmware. Questo modello evidenzia come il microcontrollore non esegua semplicemente istruzioni, ma gestisca un processo continuo di aggiornamento dello stato.

Il firmware del MicroBot è strutturato in modo da gestire più funzioni contemporaneamente, riflettendo la natura multifunzionale del nodo. Tra le principali responsabilità del microcontrollore si possono individuare:

- gestione della comunicazione con il controller e con altri nodi
- elaborazione dei comandi ricevuti
- controllo dei sistemi magnetici (attivazione bobine)
- gestione del sistema di feedback (LED)
- monitoraggio dello stato energetico
- esecuzione della logica locale

Queste funzioni non vengono eseguite in modo sequenziale rigido, ma attraverso un modello di esecuzione concorrente, in cui diverse attività vengono gestite in parallelo o in modo cooperativo. Nei sistemi basati su ESP32, questo può essere realizzato attraverso task separati o loop non bloccanti, garantendo una risposta rapida agli eventi.

Un aspetto fondamentale riguarda la gestione degli input/output. Il microcontrollore deve interfacciarsi con diversi componenti, tra cui sensori, attuatori e moduli di comunicazione. Questo richiede una configurazione accurata dei pin e l'utilizzo di periferiche hardware dedicate, come moduli PWM per il controllo delle bobine o interfacce seriali per la comunicazione.

Nel contesto dei tuoi prototipi, l'utilizzo di PWM per il controllo del campo magnetico rappresenta un esempio concreto di questa integrazione. La modulazione del segnale consente di regolare l'intensità del campo, introducendo un livello di controllo fine sulle interazioni tra i nodi. Questo collega direttamente il livello computazionale con quello fisico.

Un ulteriore elemento di rilievo è la gestione del tempo. Il microcontrollore opera in un contesto temporale discreto, in cui le operazioni vengono eseguite secondo cicli o interrupt. La sincronizzazione con il controller centrale, descritta nella Sezione 3, richiede che il nodo sia in grado di allineare le proprie operazioni con segnali esterni, mantenendo al contempo una certa autonomia.

Dal punto di vista energetico, il microcontrollore deve operare in modo efficiente, minimizzando il consumo quando possibile. Questo può includere l'utilizzo di modalità di risparmio energetico o la gestione dinamica delle risorse. Nei sistemi distribuiti, l'efficienza energetica è un fattore critico, in quanto influisce direttamente sulla durata operativa del nodo.

Un aspetto particolarmente importante riguarda la robustezza del firmware. Poiché il sistema è composto da molte unità, è inevitabile che si verifichino errori o anomalie. Il microcontrollore deve quindi essere in grado di gestire tali situazioni, ad esempio attraverso meccanismi di watchdog, reset automatici o gestione degli errori. Nei tuoi progetti, la presenza di sistemi di logging e diagnostica rappresenta un supporto fondamentale per questo tipo di gestione.

Dal punto di vista della comunicazione, il microcontrollore integra moduli wireless che permettono lo scambio di dati in tempo reale. Questo collega direttamente il nodo al flusso informativo globale, rendendolo parte attiva della rete. La gestione della comunicazione deve tenere conto di latenza, perdita di pacchetti e sincronizzazione, introducendo una complessità che va oltre la semplice trasmissione di dati.

Un ulteriore elemento di connessione riguarda l'integrazione con il sistema di simulazione. Il firmware del microcontrollore può essere progettato in modo da riflettere il comportamento dei modelli virtuali, permettendo una continuità tra simulazione e implementazione reale. Questo è particolarmente importante nel tuo sistema, dove simulazione e realtà sono strettamente integrate.

Dal punto di vista teorico, il microcontrollore rappresenta il punto in cui la funzione π del nodo intelligente viene implementata concretamente. Esso traduce il modello matematico in operazioni reali, rendendo possibile l'esecuzione del sistema. Questa connessione tra teoria e implementazione è uno degli elementi più importanti del progetto.

Infine, il microcontrollore costituisce la base per l'evoluzione futura del MicroBot. L'introduzione di funzionalità più avanzate, come sensori o algoritmi di apprendimento, dipende direttamente dalle capacità di questo componente. Tuttavia, è importante mantenere il principio di semplicità locale, evitando di concentrare troppa complessità nel singolo nodo.

In conclusione, il microcontrollore rappresenta il cuore computazionale del MicroBot, responsabile della gestione dello stato, dell'esecuzione delle logiche e dell'interazione con gli altri componenti. La sua progettazione e implementazione devono essere coerenti con l'architettura distribuita del sistema, garantendo efficienza, robustezza e integrazione con i livelli superiori.

4.7 Sistema di alimentazione (batteria e gestione energetica)

Il sistema di alimentazione del MicroBot rappresenta uno degli elementi più critici dell'intera architettura, in quanto definisce i limiti operativi del nodo e condiziona direttamente tutte le altre componenti, dal microcontrollore ai sistemi magnetici. In un sistema distribuito composto da un elevato numero di unità, la gestione dell'energia non è un aspetto secondario, ma un vincolo progettuale fondamentale che influenza prestazioni, durata e stabilità del sistema.

Nel contesto MicroBot, il sistema di alimentazione è tipicamente basato su batterie ricaricabili di tipo Li-Po (Lithium Polymer), scelte per la loro elevata densità energetica e per la possibilità di essere integrate in volumi ridotti. Questa scelta è coerente con i requisiti di miniaturizzazione e mobilità del nodo, ma introduce anche una serie di problematiche legate alla gestione della sicurezza, della tensione e della corrente.

Dal punto di vista formale, il sistema energetico del nodo può essere descritto come una variabile di stato:

$s_energy(t)$

che evolve nel tempo in funzione del consumo delle componenti e degli eventuali meccanismi di ricarica. L'evoluzione di questa variabile può essere modellata come:

$$s_energy(t+1) = s_energy(t) - P(t) \cdot \Delta t$$

dove $P(t)$ rappresenta la potenza assorbita dal nodo. Questa semplice relazione evidenzia come ogni operazione eseguita dal MicroBot abbia un costo energetico, introducendo un vincolo diretto sul comportamento del sistema.

Un elemento fondamentale del sistema di alimentazione è la gestione della tensione. Le batterie Li-Po forniscono una tensione variabile in funzione dello stato di carica, che deve essere stabilizzata per garantire il corretto funzionamento dei componenti elettronici. Questo richiede l'utilizzo di regolatori di tensione, che convertono la tensione della batteria in valori stabili e compatibili con il microcontrollore e gli altri dispositivi.

Oltre alla regolazione, è necessario implementare meccanismi di protezione. Le batterie Li-Po sono sensibili a condizioni di sovraccarico, scarica eccessiva e cortocircuito, che possono causare danni o situazioni pericolose. Il sistema di alimentazione deve quindi includere circuiti di protezione che monitorano la tensione e interrompono il flusso di corrente in caso di anomalie.

Un aspetto particolarmente rilevante riguarda la gestione della corrente. I sistemi magnetici, in particolare le bobine elettromagnetiche, possono richiedere correnti elevate per generare campi sufficienti. Questo introduce un problema di bilanciamento tra capacità della batteria e richiesta energetica. Il microcontrollore deve quindi gestire l'attivazione delle bobine in modo controllato, evitando picchi di corrente che potrebbero compromettere il sistema.

Nei tuoi prototipi, questo si traduce nell'utilizzo di driver come MOSFET, che permettono di controllare il flusso di corrente verso le bobine senza sovraccaricare il microcontrollore. Questo collegamento tra sistema di alimentazione e sistema di attuazione è fondamentale per il corretto funzionamento del nodo.

Un ulteriore elemento di rilievo è il monitoraggio dello stato energetico. Il MicroBot deve essere in grado di rilevare il livello di carica della batteria e di adattare il proprio comportamento in funzione di esso. Ad esempio, in condizioni di energia limitata, il nodo può ridurre l'attività, disattivare componenti non essenziali o segnalare la propria condizione al sistema. Nei tuoi sistemi, questa informazione può essere integrata nel flusso di telemetria e visualizzata attraverso l'interfaccia.

Dal punto di vista architetturale, il sistema di alimentazione è strettamente integrato con il microcontrollore e con gli altri componenti. La distribuzione dell'energia deve essere progettata in modo da minimizzare perdite

e interferenze, garantendo al contempo flessibilità. Questo richiede una disposizione accurata dei componenti e una progettazione attenta dei circuiti.

Un aspetto importante riguarda la dissipazione termica. Il passaggio di corrente, in particolare nei sistemi magnetici e nei regolatori di tensione, genera calore che deve essere gestito per evitare surriscaldamenti. Anche se nei prototipi iniziali questo fenomeno può essere limitato, diventa significativo con l'aumentare della potenza e della densità dei componenti.

Dal punto di vista del sistema distribuito, l'energia introduce una dimensione ulteriore nella dinamica dello swarm. Ogni nodo ha una capacità energetica limitata, e la distribuzione dell'energia tra le unità può influenzare il comportamento globale. Questo apre la possibilità di strategie avanzate, in cui il sistema tiene conto dello stato energetico dei nodi per ottimizzare le configurazioni e le operazioni.

Un ulteriore elemento di connessione riguarda la scalabilità. Con l'aumentare del numero di MicroBot, la gestione dell'energia diventa sempre più complessa, richiedendo soluzioni efficienti e standardizzate. Questo implica che il sistema di alimentazione debba essere progettato fin dall'inizio con una visione scalabile.

Dal punto di vista teorico, la variabile energetica può essere integrata nel modello dello stato del nodo, contribuendo alla definizione della funzione di comportamento. In questo senso, l'energia non è solo una risorsa, ma una componente dello stato che influenza direttamente le dinamiche del sistema.

Infine, il sistema di alimentazione rappresenta un elemento chiave per l'evoluzione futura del MicroBot. L'introduzione di tecnologie di ricarica wireless, sistemi di gestione energetica avanzati o fonti alternative potrebbe estendere significativamente le capacità del sistema, rendendolo più autonomo e adattabile.

In conclusione, il sistema di alimentazione del MicroBot non è semplicemente una fonte di energia, ma una componente strutturale che influenza tutte le altre parti del nodo. La sua progettazione richiede un equilibrio tra capacità, efficienza e sicurezza, e rappresenta un elemento fondamentale per il funzionamento e l'evoluzione del sistema.

4.8 Sistema magnetico

Il sistema magnetico del MicroBot rappresenta il meccanismo fisico attraverso cui le unità interagiscono tra loro, permettendo l'aggregazione, la separazione e la riconfigurazione dello swarm. In continuità con quanto definito nelle sezioni precedenti, esso costituisce il livello di interazione materiale del sistema, traducendo le decisioni computazionali in forze reali che agiscono nello spazio.

A differenza di sistemi robotici tradizionali, in cui il movimento e l'interazione sono realizzati tramite attuatori meccanici complessi, il sistema MicroBot utilizza campi magnetici come mezzo principale di interazione. Questa scelta introduce una modalità di controllo indiretta, in cui le unità non si muovono attraverso motori o articolazioni, ma attraverso forze generate e modulate elettricamente o configurate passivamente.

Dal punto di vista fisico, le interazioni tra MicroBot possono essere modellate come interazioni tra dipoli magnetici. Ogni unità può essere vista come una sorgente di campo magnetico, la cui intensità e direzione dipendono dalla configurazione interna. La forza tra due unità è funzione della distanza, dell'orientamento e dell'intensità dei campi:

$$F \propto (m_1 \cdot m_2) / r^4$$

dove m_1 e m_2 rappresentano i momenti magnetici delle unità e r la distanza tra di esse. Questa relazione evidenzia come le forze diminuiscano rapidamente con la distanza, rendendo le interazioni fortemente locali.

Nel sistema MicroBot, il sistema magnetico può essere implementato in due modalità principali: passiva e attiva. Nel caso passivo, si utilizzano magneti permanenti, che generano campi costanti nel tempo. Questa soluzione è semplice e a basso consumo energetico, ma offre un controllo limitato, in quanto le interazioni sono determinate dalla configurazione fisica dei magneti.

Nel caso attivo, si utilizzano bobine elettromagnetiche, che permettono di generare campi controllabili attraverso la corrente elettrica. Questa soluzione introduce un livello di controllo significativamente maggiore,

consentendo di attivare, modulare o invertire il campo magnetico. Tuttavia, essa richiede una gestione energetica più complessa e introduce problematiche legate alla dissipazione termica.

Un aspetto fondamentale del sistema magnetico riguarda la relazione con la geometria del MicroBot. Come discusso nella sezione 4.4, la configurazione a coni troncati e sfera è progettata per facilitare l'allineamento dei campi e migliorare la stabilità delle interazioni. Le superfici di contatto definiscono le direzioni preferenziali lungo cui le forze vengono applicate, influenzando direttamente le configurazioni possibili dello swarm.

Dal punto di vista dinamico, il sistema magnetico costituisce il collegamento tra livello locale e comportamento globale. Le forze generate tra singole unità determinano la formazione di strutture più grandi, come catene, cluster o configurazioni tridimensionali. Questo processo è un esempio di comportamento emergente, in cui interazioni semplici producono risultati complessi.

Un elemento chiave riguarda la stabilità delle configurazioni. Le forze magnetiche devono essere sufficienti a mantenere le unità aggregate, ma non così elevate da impedire la riconfigurazione. Questo introduce un equilibrio tra attrazione e separazione, che può essere regolato attraverso la scelta dei magneti o il controllo della corrente nelle bobine.

Dal punto di vista del controllo, il sistema magnetico è direttamente collegato al microcontrollore. Nei sistemi attivi, il firmware gestisce l'attivazione delle bobine attraverso segnali PWM, permettendo di modulare l'intensità del campo. Questo crea un collegamento diretto tra decisione computazionale e azione fisica, realizzando concretamente la funzione π del nodo.

Un ulteriore aspetto di rilievo è la dissipazione energetica. La generazione di campi magnetici, in particolare attraverso bobine, comporta un consumo significativo di energia e la produzione di calore. Questo richiede una gestione attenta, sia a livello hardware sia a livello software, per evitare condizioni di surriscaldamento.

Dal punto di vista del sistema distribuito, il sistema magnetico introduce una forma di comunicazione implicita. Anche in assenza di scambio di dati espliciti, le unità influenzano il comportamento reciproco attraverso le forze fisiche. Questo crea un canale di interazione parallelo alla comunicazione digitale, contribuendo alla complessità del sistema.

Un aspetto particolarmente interessante riguarda la possibilità di combinare interazioni magnetiche e comunicazione digitale. Il controller centrale può inviare comandi che modificano il comportamento magnetico dei nodi, influenzando indirettamente la struttura dello swarm. Questo permette di guidare il sistema verso configurazioni specifiche, mantenendo al contempo la natura distribuita.

Dal punto di vista teorico, il sistema magnetico può essere interpretato come una funzione che collega lo stato interno dei nodi con le forze nello spazio fisico. In questo senso, esso rappresenta una mappatura tra lo spazio degli stati e lo spazio delle interazioni:

$$F = M(s)$$

dove M è la funzione che descrive il comportamento magnetico. Questa interpretazione collega direttamente il livello fisico con il modello multidimensionale del sistema.

Infine, il sistema magnetico rappresenta uno degli elementi chiave per la realizzazione della materia programmabile. La capacità di controllare le interazioni tra unità permette di modificare la forma e la struttura del sistema in modo dinamico, aprendo la strada a applicazioni avanzate.

In conclusione, il sistema magnetico del MicroBot costituisce il meccanismo fondamentale attraverso cui le unità interagiscono nello spazio. Esso integra principi fisici e logiche di controllo, permettendo la formazione e la riconfigurazione dello swarm. La sua progettazione rappresenta una delle sfide principali del sistema e costituisce la base per gli approfondimenti successivi, in cui verranno analizzate le componenti specifiche e le relazioni quantitative tra i parametri.

4.9 Magnet permanenti

I magneti permanenti rappresentano la forma più semplice e immediata di implementazione del sistema magnetico nel MicroBot. In questa configurazione, ogni unità integra uno o più magneti con polarità definita, generando un campo magnetico costante nel tempo. Questa scelta progettuale consente di realizzare interazioni fisiche tra nodi senza richiedere consumo energetico attivo per la generazione del campo, riducendo significativamente la complessità del sistema di alimentazione.

Dal punto di vista fisico, un magnete permanente può essere modellato come un dipolo magnetico caratterizzato da un momento magnetico m , che determina l'intensità e la direzione del campo generato. Il campo prodotto da un dipolo nello spazio decresce rapidamente con la distanza, rendendo le interazioni fortemente locali. Questo comportamento è coerente con l'architettura distribuita del sistema MicroBot, in cui le interazioni avvengono principalmente tra unità vicine.

La scelta del tipo di magnete è un elemento critico. Materiali come il neodimio (NdFeB) offrono una densità di flusso magnetico elevata in volumi ridotti, rendendoli particolarmente adatti a sistemi miniaturizzati. Tuttavia, magneti più potenti introducono anche forze di attrazione più elevate, che possono rendere difficile la separazione delle unità o causare instabilità durante la riconfigurazione. Questo introduce un vincolo progettuale che richiede un bilanciamento tra forza e controllabilità.

Un aspetto fondamentale riguarda la configurazione dei poli. La disposizione dei magneti all'interno del MicroBot determina le modalità di interazione con le altre unità. Configurazioni simmetriche possono favorire l'allineamento automatico, mentre configurazioni asimmetriche possono introdurre direzionalità nelle interazioni. Nel caso della geometria a coni troncati, i magneti possono essere posizionati lungo l'asse principale, creando superfici di contatto con polarità complementari.

Dal punto di vista dinamico, i magneti permanenti introducono un comportamento passivo. Le unità si attraggono o si respingono in base alla loro configurazione, senza possibilità di modificare attivamente il campo. Questo implica che il comportamento del sistema è in gran parte determinato dalla geometria e dalla disposizione dei magneti, piuttosto che da decisioni computazionali. In altre parole, una parte della logica del sistema viene "codificata" direttamente nella struttura fisica.

Questo approccio presenta vantaggi significativi in termini di semplicità e affidabilità. L'assenza di componenti attivi riduce il consumo energetico e elimina la necessità di controlli complessi. Inoltre, i magneti permanenti non richiedono manutenzione e sono meno soggetti a guasti rispetto ai sistemi elettromagnetici.

Tuttavia, questa semplicità introduce anche limitazioni importanti. In particolare, l'impossibilità di controllare il campo magnetico rende difficile implementare comportamenti dinamici complessi. Le unità non possono "decidere" quando attrarsi o separarsi, ma reagiscono esclusivamente alle condizioni fisiche. Questo limita la flessibilità del sistema e rende più complessa la gestione di configurazioni avanzate.

Nel contesto MicroBot, i magneti permanenti possono essere interpretati come una base strutturale su cui costruire comportamenti più complessi. Essi forniscono un livello minimo di interazione, che può essere successivamente esteso attraverso l'introduzione di componenti attivi. Questo approccio modulare consente di sviluppare il sistema in modo incrementale, partendo da prototipi semplici e aumentando progressivamente la complessità.

Un aspetto particolarmente interessante riguarda la relazione tra magneti permanenti e comportamento emergente. Anche con interazioni puramente passive, è possibile osservare fenomeni complessi, come l'auto-organizzazione e la formazione di pattern. Questo evidenzia come la struttura fisica possa contribuire in modo significativo alla dinamica del sistema, riducendo la necessità di controllo esplicito.

Dal punto di vista progettuale, è importante considerare anche gli effetti indesiderati. Forze troppo elevate possono causare aggregazioni incontrollate, mentre forze troppo deboli possono impedire la formazione di strutture stabili. Inoltre, la presenza di magneti può introdurre interferenze con altri componenti elettronici, richiedendo una disposizione attenta all'interno del nodo.

Un ulteriore elemento di rilievo è la scalabilità. In un sistema composto da molte unità, le interazioni magnetiche possono diventare complesse, con effetti cumulativi difficili da prevedere. Questo richiede un'analisi attenta delle configurazioni possibili e dei limiti operativi.

Dal punto di vista teorico, i magneti permanenti rappresentano una funzione statica che collega lo stato del nodo alle forze generate nello spazio. A differenza dei sistemi attivi, in cui questa funzione può essere modificata nel tempo, qui essa è fissa, introducendo una forma di vincolo strutturale nel sistema.

Infine, i magneti permanenti costituiscono un punto di partenza ideale per la sperimentazione. Nei tuoi prototipi, l'utilizzo di componenti semplici consente di osservare direttamente gli effetti delle interazioni, fornendo una base empirica per lo sviluppo del sistema.

In conclusione, i magneti permanenti rappresentano una soluzione semplice ed efficace per implementare interazioni fisiche tra MicroBot. Essi offrono vantaggi in termini di semplicità ed efficienza, ma introducono limitazioni nella capacità di controllo. La loro integrazione nel sistema deve essere valutata in funzione degli obiettivi progettuali, e costituisce la base per l'introduzione di sistemi più avanzati, che verranno analizzati nelle sezioni successive.

4.10 Bobine elettromagnetiche (controllo attivo del campo)

Le bobine elettromagnetiche rappresentano l'evoluzione naturale del sistema magnetico passivo basato su magneti permanenti, introducendo la possibilità di controllare dinamicamente le interazioni tra i MicroBot. In questa configurazione, il campo magnetico non è più una proprietà statica della struttura, ma una variabile controllabile nel tempo, direttamente collegata alle decisioni computazionali del nodo.

Dal punto di vista fisico, una bobina percorsa da corrente genera un campo magnetico la cui intensità è proporzionale al numero di spire e alla corrente che la attraversa. Questo comportamento può essere descritto, in forma semplificata, come:

$$B \propto N \cdot I$$

dove B è il campo magnetico, N il numero di spire e I la corrente. Questa relazione evidenzia come il controllo della corrente rappresenti il principale strumento per modulare il campo.

Nel contesto del MicroBot, l'introduzione delle bobine trasforma il sistema magnetico in un sistema di attuazione controllato. Il microcontrollore, attraverso segnali PWM e driver di potenza (come MOSFET), può attivare o disattivare le bobine, modulando l'intensità del campo e, di conseguenza, le forze tra le unità. Questo crea un collegamento diretto tra livello computazionale e livello fisico, rendendo possibile un controllo fine del comportamento dello swarm.

A differenza dei magneti permanenti, le bobine permettono di modificare la polarità del campo, invertendo la direzione della corrente. Questo consente non solo di attrarre altre unità, ma anche di respingerle, introducendo una dinamica molto più ricca. La possibilità di attivare selettivamente le interazioni apre la strada a comportamenti complessi, come aggregazione controllata, rilascio selettivo e riconfigurazione dinamica.

Dal punto di vista architetturale, le bobine si integrano con il microcontrollore attraverso un sottosistema di potenza. Poiché la corrente richiesta può superare le capacità dirette dei pin del microcontrollore, è necessario utilizzare componenti intermedi, come MOSFET e diodi di protezione (flyback), che permettono di gestire il flusso di corrente in modo sicuro. Questo sottosistema rappresenta un punto critico nella progettazione, in quanto deve garantire efficienza, affidabilità e sicurezza.

Un aspetto fondamentale riguarda il consumo energetico. A differenza dei magneti permanenti, le bobine richiedono energia continua per mantenere il campo. Questo introduce un vincolo diretto sul sistema di alimentazione, rendendo necessario un bilanciamento tra prestazioni e autonomia. Nei sistemi distribuiti, questo aspetto è particolarmente rilevante, in quanto il consumo di ogni nodo contribuisce al comportamento globale.

Dal punto di vista termico, il passaggio di corrente nelle bobine genera calore, che deve essere dissipato per evitare surriscaldamenti. Questo richiede una progettazione attenta sia a livello hardware sia a livello firmware, introducendo limiti operativi e strategie di gestione, come l'attivazione temporizzata o la modulazione della potenza.

Un elemento particolarmente interessante riguarda la relazione tra controllo locale e comportamento emergente. Con le bobine, ogni MicroBot può modificare attivamente le proprie interazioni, influenzando il

comportamento dello swarm. Tuttavia, il risultato globale non è determinato esclusivamente dal controllo centrale, ma emerge dall'interazione tra le decisioni locali dei nodi. Questo mantiene la natura distribuita del sistema, pur introducendo un livello di controllo più elevato.

Dal punto di vista dinamico, il sistema può essere descritto come:

$$F_i(t) = M(s_i(t), I_i(t))$$

dove $F_i(t)$ rappresenta le forze generate dal nodo e M la funzione che collega stato e corrente al campo magnetico. Questa formulazione evidenzia come il comportamento fisico del nodo sia direttamente influenzato dalla logica implementata nel firmware.

Un ulteriore aspetto riguarda la possibilità di modulazione temporale. Utilizzando segnali PWM, è possibile variare rapidamente l'intensità del campo, creando effetti dinamici che possono essere utilizzati per migliorare la stabilità o facilitare la separazione delle unità. Questo introduce una dimensione temporale nel controllo delle interazioni, che non è presente nei sistemi passivi.

Dal punto di vista progettuale, la disposizione delle bobine all'interno del MicroBot deve essere coerente con la geometria definita in precedenza. Le estremità a cono troncato rappresentano posizioni ideali per l'integrazione delle bobine, in quanto consentono di orientare il campo lungo direzioni specifiche, migliorando l'efficacia delle interazioni.

Un elemento critico riguarda la complessità del sistema. L'introduzione delle bobine aumenta significativamente il numero di componenti e la difficoltà di progettazione. Tuttavia, essa rappresenta un passo fondamentale verso la realizzazione di un sistema completamente controllabile, in cui il comportamento non è solo emergente, ma anche guidato.

Dal punto di vista teorico, le bobine introducono una funzione dinamica nel sistema, in cui la relazione tra stato e forze può essere modificata nel tempo. Questo estende il modello del sistema, introducendo una dimensione di controllo che collega direttamente computazione e fisica.

Infine, le bobine elettromagnetiche rappresentano uno degli elementi chiave per la realizzazione della materia programmabile. La possibilità di controllare le interazioni tra unità in modo dinamico permette di modificare la forma e la struttura del sistema in tempo reale, aprendo la strada a applicazioni avanzate.

In conclusione, le bobine elettromagnetiche trasformano il sistema MicroBot da una struttura passiva a un sistema attivo e controllabile. Esse introducono un livello di complessità maggiore, ma offrono un potenziale significativamente superiore, permettendo di realizzare comportamenti dinamici e adattivi. Questa evoluzione rappresenta un passo fondamentale nello sviluppo del sistema e prepara il terreno per l'analisi delle relazioni quantitative tra corrente, campo e forze, che verranno approfondite nei punti successivi.

4.11 Relazione tra corrente, spire e campo magnetico

Nel contesto del sistema MicroBot, la comprensione quantitativa della relazione tra corrente, numero di spire e campo magnetico rappresenta un passaggio fondamentale per il dimensionamento e il controllo delle interazioni tra unità. In continuità con la sezione precedente, in cui è stato introdotto il controllo attivo tramite bobine, è ora necessario formalizzare il legame tra le grandezze elettriche e le forze fisiche generate.

Il campo magnetico generato da una bobina può essere descritto, in prima approssimazione, attraverso la legge di Ampère. Nel caso di una bobina solenoidale ideale, il campo interno è dato da:

$$B = \mu_0 \cdot \mu_r \cdot (N / L) \cdot I$$

dove B è il campo magnetico, μ_0 la permeabilità del vuoto, μ_r la permeabilità relativa del materiale, N il numero di spire, L la lunghezza della bobina e I la corrente. Questa relazione evidenzia come il campo sia proporzionale sia alla corrente sia al numero di spire, introducendo due parametri progettuali fondamentali.

Nel MicroBot, tuttavia, le bobine non sono solenoidi ideali, ma strutture compatte integrate in uno spazio limitato. Questo implica che il campo non è uniforme e che la geometria reale influisce significativamente

sulla distribuzione del campo. Di conseguenza, la relazione teorica deve essere considerata come una base, da adattare alle condizioni reali attraverso sperimentazione e simulazione.

Un aspetto fondamentale riguarda il compromesso tra numero di spire e corrente. Aumentare il numero di spire consente di ottenere un campo più intenso a parità di corrente, ma aumenta anche la resistenza della bobina, riducendo l'efficienza e aumentando la dissipazione di calore. Al contrario, aumentare la corrente permette di ottenere campi più intensi con meno spire, ma richiede una maggiore capacità del sistema di alimentazione e introduce rischi termici.

Dal punto di vista energetico, la potenza dissipata nella bobina è data da:

$$P = R \cdot I^2$$

dove R è la resistenza della bobina. Questa relazione evidenzia come l'aumento della corrente comporti un incremento quadratico della potenza dissipata, rendendo la gestione termica un elemento critico. Nei tuoi sistemi, questo si collega direttamente alla sezione sull'alimentazione e alla necessità di limitare i picchi di corrente.

Un ulteriore elemento di rilievo riguarda il campo generato nello spazio circostante. Il campo magnetico non agisce solo all'interno della bobina, ma si estende all'esterno, influenzando le unità vicine. La distribuzione del campo dipende dalla geometria della bobina e dalla presenza di materiali ferromagnetici, che possono concentrare o deviare il flusso magnetico.

Nel sistema MicroBot, la progettazione delle bobine deve tenere conto della distanza tipica tra le unità. Poiché le forze magnetiche decrescono rapidamente con la distanza, è necessario generare un campo sufficientemente intenso da garantire interazioni efficaci. Questo introduce un vincolo diretto sul dimensionamento delle bobine e sulla corrente utilizzata.

Dal punto di vista del controllo, la relazione tra corrente e campo consente di modulare le interazioni in modo continuo. Utilizzando segnali PWM, il microcontrollore può variare la corrente media nella bobina, regolando l'intensità del campo. Questo permette di implementare strategie di controllo più sofisticate, in cui le forze vengono adattate in funzione dello stato del sistema.

Un aspetto particolarmente interessante riguarda la linearità della relazione. In condizioni ideali, il campo è proporzionale alla corrente, ma nella pratica possono verificarsi fenomeni di saturazione, soprattutto in presenza di materiali ferromagnetici. Questo introduce non linearità che devono essere considerate nella progettazione e nel controllo.

Dal punto di vista del sistema distribuito, la relazione tra corrente e campo rappresenta il collegamento tra decisione locale e interazione fisica. Ogni nodo può modificare la propria corrente, influenzando il campo e quindi le forze che agiscono sulle unità vicine. Questo crea una dinamica in cui le decisioni locali si propagano attraverso il sistema sotto forma di interazioni fisiche.

Un ulteriore elemento di connessione riguarda la simulazione. Nei tuoi sistemi software, è possibile modellare il campo magnetico e le forze tra unità utilizzando approssimazioni di queste relazioni. Questo consente di testare diverse configurazioni e di ottimizzare i parametri prima dell'implementazione reale.

Dal punto di vista progettuale, il dimensionamento delle bobine richiede la scelta di parametri specifici, come diametro del filo, numero di spire e dimensioni della bobina. Questi parametri devono essere selezionati in modo da bilanciare intensità del campo, consumo energetico e dimensioni del nodo.

Infine, la relazione tra corrente, spire e campo magnetico rappresenta uno degli elementi chiave per la comprensione del sistema MicroBot. Essa collega direttamente l'elettronica alla fisica delle interazioni, fornendo gli strumenti necessari per progettare e controllare il comportamento dello swarm.

In conclusione, la formalizzazione di questa relazione consente di passare da un approccio qualitativo a uno quantitativo, permettendo di dimensionare il sistema in modo preciso e di sviluppare strategie di controllo efficaci. Questo rappresenta un passo fondamentale verso la realizzazione di un sistema completamente controllabile e scalabile.

SEZIONE 5 — Architettura Elettronica

5.1 Microcontrollore ESP32

Il microcontrollore ESP32 rappresenta il nucleo computazionale dell'architettura elettronica del sistema MicroBot. Esso costituisce il componente incaricato di eseguire il firmware locale, gestire gli ingressi e le uscite digitali, controllare eventuali attuatori, comunicare con il controller centrale e coordinare le operazioni interne del nodo. In questa prospettiva, l'ESP32 non deve essere considerato soltanto come una scheda elettronica programmabile, ma come il centro operativo del singolo MicroBot, cioè il punto in cui le istruzioni software vengono tradotte in segnali elettrici e, successivamente, in comportamento fisico osservabile.

La scelta dell'ESP32 risulta coerente con l'impostazione generale del progetto perché combina capacità computazionale, connettività wireless, disponibilità di pin GPIO e supporto a periferiche hardware utili al controllo embedded. In un sistema distribuito come MicroBot, ogni unità deve poter ricevere comandi, elaborare uno stato locale e rispondere in modo autonomo o semi-autonomo. L'ESP32 consente di realizzare questa logica grazie alla possibilità di eseguire codice residente sul nodo, mantenendo al tempo stesso una connessione con il livello superiore del sistema.

Dal punto di vista architetturale, l'ESP32 si colloca tra il livello software e il livello fisico. Da un lato riceve comandi logici, parametri di configurazione o segnali di sincronizzazione; dall'altro genera segnali elettrici destinati ai componenti collegati, come LED, driver, sensori o circuiti di potenza. Questa posizione intermedia lo rende un elemento di conversione tra informazione e azione. Il MicroBot, infatti, non agisce direttamente a partire da un comando astratto, ma attraverso una catena tecnica precisa: comando digitale, interpretazione firmware, uscita GPIO o PWM, attivazione del circuito, risposta fisica del componente.

Nel progetto MicroBot, l'ESP32 può essere modellato come una macchina a stati locale. Ogni nodo può trovarsi in condizioni operative differenti, ad esempio inattivo, attivo, in attesa di comando, in esecuzione di pattern, in errore o in modalità diagnostica. Il firmware deve quindi essere strutturato in modo da gestire queste transizioni senza creare ambiguità. Una rappresentazione generale può essere espressa come:

`stato_successivo = funzione(stato_corrente, comando_ricevuto, condizioni_locali)`

Questa forma evidenzia come il comportamento del nodo non dipenda soltanto dai comandi esterni, ma anche dalla condizione interna del MicroBot. Se, ad esempio, il livello energetico è basso o viene rilevata una condizione anomala, il firmware può decidere di limitare alcune operazioni, segnalare lo stato attraverso LED o inviare un evento al controller.

Un aspetto fondamentale dell'ESP32 è la gestione dei pin di input/output. I pin GPIO permettono di collegare il microcontrollore ai componenti esterni e di controllarli in modo programmabile. Nel caso dei prototipi iniziali, questi pin possono essere utilizzati per pilotare LED o segnali semplici; nelle versioni più avanzate, possono controllare driver MOSFET, leggere sensori, acquisire segnali di stato o gestire linee di comunicazione. La corretta assegnazione dei pin diventa quindi una scelta progettuale importante, perché influenza l'espandibilità futura del nodo.

Un elemento particolarmente importante è il supporto al PWM, cioè alla modulazione di larghezza d'impulso. Attraverso il PWM, l'ESP32 può generare segnali controllati nel tempo, utili per regolare l'intensità luminosa dei LED o per pilotare indirettamente sistemi di potenza. Anche se la fisica magnetica verrà trattata in modo rigoroso nella Sezione 6, dal punto di vista elettronico è importante chiarire che il microcontrollore non alimenta direttamente carichi significativi, ma fornisce segnali di controllo a circuiti dedicati. Questa distinzione è essenziale per evitare errori progettuali: l'ESP32 decide e modula, mentre il circuito di potenza eroga la corrente necessaria.

La connettività wireless dell'ESP32 è un altro elemento centrale. Poiché il sistema MicroBot è pensato come rete di nodi distribuiti, il microcontrollore deve poter comunicare con il controller centrale o con altri dispositivi. La rete può essere utilizzata per ricevere comandi, inviare telemetria, sincronizzare operazioni e aggiornare lo stato del sistema. In questo senso, l'ESP32 non è soltanto un processore locale, ma anche un nodo di rete. La sua funzione elettronica si intreccia quindi con la Sezione 8, dedicata alla comunicazione, ma in questa sede è importante evidenziare il ruolo hardware del componente come base abilitante della comunicazione stessa.

Dal punto di vista energetico, l'ESP32 deve essere integrato con attenzione nel sistema di alimentazione. Il suo consumo non è trascurabile, soprattutto quando sono attive le funzioni wireless. Per questo motivo, la progettazione elettronica deve considerare modalità di risparmio energetico, gestione dei cicli di attività e

separazione tra componenti sempre attivi e componenti attivati solo quando necessario. In un sistema composto da molte unità, anche piccole inefficienze energetiche sul singolo nodo possono diventare rilevanti su scala di swarm.

Un altro aspetto importante riguarda la robustezza. Il microcontrollore deve essere in grado di gestire condizioni non ideali: perdita di comunicazione, comando non valido, tensione instabile, ritardi, blocchi software o stati incoerenti. Per questo motivo, il firmware dovrebbe includere meccanismi di controllo come watchdog, validazione dei messaggi, timeout e modalità sicure. In un sistema distribuito, la robustezza del singolo nodo contribuisce direttamente alla robustezza complessiva dello swarm.

L'ESP32 svolge anche una funzione diagnostica. Attraverso segnali seriali, LED di stato o messaggi di telemetria, può comunicare informazioni utili sul funzionamento del nodo. Questa capacità è fondamentale durante le fasi di prototipazione, perché permette di comprendere se il problema si trova nel codice, nella comunicazione, nell'alimentazione o nel circuito collegato. La diagnostica locale costituisce quindi una parte integrante dell'architettura elettronica, non un'aggiunta secondaria.

Infine, l'ESP32 rappresenta una scelta coerente con lo sviluppo incrementale del progetto. Nelle prime versioni può essere utilizzato per controllare LED e simulare stati del nodo; nelle versioni successive può essere integrato con circuiti di potenza, sensori, moduli di comunicazione più strutturati e sistemi di feedback avanzati. Questa progressione rispecchia il metodo generale del progetto MicroBot: partire da una base semplice, validare ogni livello e aumentare gradualmente la complessità mantenendo la coerenza architetturale.

In conclusione, il microcontrollore ESP32 costituisce il cuore elettronico e computazionale del singolo MicroBot. Esso permette di trasformare il nodo da componente passivo a unità programmabile, capace di ricevere comandi, elaborare stati, controllare componenti esterni e partecipare alla rete distribuita. La sua funzione non è isolata, ma attraversa tutto il sistema: collega hardware, firmware, comunicazione, telemetria, controllo e futura espansione. Per questo motivo, la progettazione attorno all'ESP32 deve essere affrontata con rigore, considerando non solo ciò che il componente può fare singolarmente, ma il ruolo che esso assume all'interno dell'intero ecosistema MicroBot.

5.2 Gestione computazionale

La gestione computazionale all'interno del MicroBot rappresenta il livello logico attraverso cui il microcontrollore organizza, esegue e coordina le operazioni del nodo. In continuità con la sezione precedente, in cui è stato definito il ruolo dell'ESP32 come nucleo computazionale, è ora necessario formalizzare come le risorse di calcolo vengono effettivamente utilizzate per sostenere il comportamento del sistema.

Nel contesto MicroBot, la computazione non è centralizzata in un unico sistema, ma distribuita tra molte unità. Questo implica che ogni nodo deve essere in grado di gestire autonomamente una parte del processo complessivo, eseguendo operazioni locali in modo efficiente e coerente con lo stato globale. La gestione computazionale diventa quindi un problema di organizzazione delle risorse limitate del microcontrollore, piuttosto che di pura potenza di calcolo.

Dal punto di vista formale, il comportamento del nodo può essere descritto come una sequenza di operazioni applicate allo stato interno:

$$s_i(t+1) = f(s_i(t), \text{input}(t))$$

dove la funzione f rappresenta l'insieme delle operazioni computazionali eseguite dal microcontrollore. Questa funzione non è monolitica, ma composta da più sottoprocessi, ciascuno responsabile di una specifica parte del comportamento del nodo.

Nel sistema MicroBot, la gestione computazionale può essere suddivisa in diverse categorie funzionali. Una prima categoria riguarda la gestione dello stato interno, che include l'aggiornamento delle variabili, il controllo delle condizioni operative e la gestione delle transizioni tra stati. Questa componente è fondamentale per garantire coerenza e prevedibilità nel comportamento del nodo.

Una seconda categoria è costituita dall'elaborazione degli input, che include la ricezione di comandi dal controller, la lettura di eventuali sensori e la gestione dei segnali provenienti dall'ambiente. Il microcontrollore

deve essere in grado di interpretare questi input e di integrarli nello stato interno, evitando conflitti o ambiguità.

Una terza categoria riguarda il controllo degli output, cioè la generazione di segnali verso i componenti esterni. Questo include l'attivazione dei pin GPIO, la generazione di segnali PWM e la gestione dei dispositivi collegati. La corretta sincronizzazione tra input e output è essenziale per garantire un comportamento coerente.

Un aspetto fondamentale della gestione computazionale è la struttura del ciclo di esecuzione. Nei sistemi embedded, è comune utilizzare un loop principale (main loop) che viene eseguito in modo continuo, all'interno del quale vengono gestite tutte le operazioni del nodo. Tuttavia, in un sistema complesso come MicroBot, è spesso necessario introdurre una gestione più articolata, basata su task o funzioni non bloccanti, per evitare ritardi e garantire reattività.

Nel caso dell'ESP32, è possibile utilizzare un modello basato su task concorrenti, in cui diverse funzioni vengono eseguite in parallelo o in modo cooperativo. Questo permette, ad esempio, di gestire la comunicazione mentre si controllano gli output, senza bloccare il sistema. Nei tuoi progetti, questo approccio è già visibile nei sistemi runtime e nei loop di simulazione, dove più processi operano simultaneamente.

Un ulteriore elemento di rilievo riguarda la gestione del tempo. Il microcontrollore deve eseguire operazioni in modo temporizzato, rispettando intervalli e sincronizzazioni. Questo può essere realizzato attraverso timer, interrupt o controlli basati sul tempo trascorso. La gestione del tempo è particolarmente importante per garantire la coerenza tra nodi, soprattutto in presenza di un controller centrale che fornisce riferimenti temporali.

Dal punto di vista delle risorse, il microcontrollore dispone di memoria limitata e capacità di elaborazione finite. La gestione computazionale deve quindi essere progettata in modo efficiente, evitando operazioni inutili o ridondanti. Questo implica una progettazione attenta del firmware, con strutture dati semplici e algoritmi leggeri.

Un aspetto critico riguarda la gestione degli errori. In un sistema reale, è inevitabile che si verifichino condizioni anomale, come input non validi, perdita di comunicazione o stati incoerenti. Il microcontrollore deve essere in grado di rilevare queste condizioni e di reagire in modo appropriato, ad esempio ignorando un comando, ripristinando uno stato sicuro o segnalando un errore.

Dal punto di vista del sistema distribuito, la gestione computazionale locale contribuisce al comportamento globale. Ogni nodo esegue le proprie operazioni in modo indipendente, ma il risultato complessivo emerge dalla combinazione di queste attività. Questo implica che il firmware deve essere progettato non solo per funzionare correttamente a livello locale, ma anche per interagire in modo coerente con gli altri nodi.

Un ulteriore elemento di connessione riguarda la simulazione. Nei tuoi sistemi software, il comportamento del nodo viene modellato attraverso funzioni che rappresentano la logica locale. La gestione computazionale del microcontrollore può essere vista come l'implementazione reale di queste funzioni, creando una continuità tra simulazione e hardware.

Dal punto di vista architetturale, la gestione computazionale rappresenta il livello in cui vengono implementati i principi di modularità e separazione delle responsabilità. Le diverse funzioni del nodo devono essere organizzate in moduli logici, che possono essere sviluppati e testati in modo indipendente. Questo approccio facilita l'evoluzione del sistema e riduce la complessità.

Infine, la gestione computazionale costituisce la base per l'introduzione di funzionalità più avanzate. L'integrazione di sensori, algoritmi di controllo più sofisticati o tecniche di apprendimento dipende direttamente dalla capacità del microcontrollore di gestire la complessità computazionale. Tuttavia, è importante mantenere un equilibrio tra funzionalità e semplicità, per evitare di compromettere l'efficienza e la robustezza del sistema.

In conclusione, la gestione computazionale del MicroBot rappresenta il meccanismo attraverso cui il microcontrollore organizza ed esegue le operazioni del nodo. Essa integra gestione dello stato, elaborazione degli input, controllo degli output e sincronizzazione temporale, contribuendo direttamente al comportamento del sistema distribuito. Una progettazione efficace di questo livello è essenziale per garantire prestazioni, affidabilità ed evoluzione del sistema MicroBot.

5.3 Sistema di comunicazione

Il sistema di comunicazione rappresenta una componente essenziale dell'architettura elettronica del MicroBot, in quanto consente lo scambio di informazioni tra i nodi distribuiti e il controller centrale. In un sistema come MicroBot, dove il comportamento globale emerge dall'interazione tra unità, la comunicazione non è un semplice supporto funzionale, ma un elemento strutturale che determina la capacità del sistema di coordinarsi, sincronizzarsi e adattarsi.

In continuità con quanto definito nelle sezioni precedenti, il MicroBot opera come nodo all'interno di una rete distribuita. Questo implica che ogni unità deve essere in grado di trasmettere e ricevere dati, integrandoli nella propria gestione computazionale. Il sistema di comunicazione costituisce quindi il canale attraverso cui lo stato locale viene condiviso e i comandi globali vengono distribuiti.

Dal punto di vista hardware, il sistema di comunicazione è realizzato attraverso le capacità wireless integrate nel microcontrollore ESP32. Questo consente di evitare connessioni fisiche tra i nodi, mantenendo la flessibilità e la scalabilità del sistema. La comunicazione può avvenire utilizzando protocolli di rete basati su Wi-Fi, che permettono di creare una rete locale in cui i MicroBot e il controller centrale sono connessi.

Dal punto di vista funzionale, la comunicazione può essere suddivisa in due direzioni principali. La prima è la comunicazione dal nodo al controller, che include l'invio di dati di stato, telemetria ed eventuali eventi. Questo flusso consente al controller di costruire una rappresentazione del sistema e di monitorarne il comportamento. La seconda è la comunicazione dal controller ai nodi, che include l'invio di comandi, configurazioni e segnali di sincronizzazione. Questo flusso consente di coordinare le operazioni e di guidare il comportamento dello swarm.

Formalmente, il sistema può essere descritto come uno scambio di messaggi:

messaggio = (id_nodo, tipo, contenuto, timestamp)

dove l'identificatore del nodo consente di distinguere le unità, il tipo indica la natura del messaggio (stato, comando, evento), il contenuto rappresenta i dati trasmessi e il timestamp permette di gestire la dimensione temporale. Questa struttura è coerente con quanto osservato nei tuoi sistemi, in cui i dati vengono organizzati in formati strutturati per essere trasmessi e interpretati correttamente.

Un aspetto fondamentale della comunicazione è la gestione della latenza. In un sistema distribuito, i messaggi non vengono ricevuti istantaneamente, ma con un certo ritardo. Questo implica che i nodi devono essere progettati per operare anche in presenza di informazioni non aggiornate, mantenendo un comportamento coerente. La gestione della latenza diventa quindi parte integrante della progettazione del sistema.

Un ulteriore elemento di rilievo è la perdita di pacchetti. Le comunicazioni wireless non sono sempre affidabili, e alcuni messaggi possono non essere ricevuti. Il sistema deve quindi essere robusto rispetto a queste condizioni, evitando di dipendere da singoli messaggi critici. Questo può essere ottenuto attraverso ridondanza, aggiornamenti periodici dello stato o strategie di fallback.

Dal punto di vista del firmware, la comunicazione deve essere integrata nella gestione computazionale senza bloccare l'esecuzione del nodo. Questo richiede l'utilizzo di operazioni non bloccanti o task separati, che permettano di gestire la ricezione e l'invio dei messaggi in parallelo con le altre operazioni. Nei tuoi sistemi, questo approccio è già presente nei moduli runtime e nelle strutture basate su loop e callback.

Un aspetto particolarmente importante riguarda la scalabilità della comunicazione. Con l'aumentare del numero di nodi, il volume di dati scambiati cresce, introducendo un carico maggiore sulla rete. Il sistema deve quindi essere progettato per gestire un numero elevato di unità senza degradare le prestazioni. Questo può richiedere l'adozione di strategie come la limitazione della frequenza dei messaggi o l'aggregazione dei dati.

Dal punto di vista architetturale, il sistema di comunicazione si integra con il controller centrale, che funge da nodo di coordinamento. Il controller riceve dati dai nodi, li elabora e invia comandi in risposta. Questa struttura riflette il modello ibrido centralizzato-distribuito definito nella Sezione 3, in cui il controllo globale è combinato con l'autonomia locale.

Un ulteriore elemento di connessione riguarda il sistema di simulazione. Nei tuoi progetti, la comunicazione è utilizzata anche per collegare il sistema reale con quello virtuale, permettendo di visualizzare lo stato dello swarm e di testare comportamenti. Questo crea una continuità tra simulazione e implementazione, rendendo il sistema più flessibile.

Dal punto di vista della sicurezza, la comunicazione deve essere progettata per evitare interferenze o accessi non autorizzati. Anche se nei prototipi iniziali questo aspetto può essere secondario, diventa rilevante in sistemi più complessi, in cui la rete potrebbe essere esposta a condizioni esterne.

Infine, il sistema di comunicazione rappresenta il collegamento tra i diversi livelli dell'architettura MicroBot. Esso connette i nodi distribuiti, il controller centrale, il sistema di simulazione e l'interfaccia utente, creando una rete di scambio di informazioni che sostiene il comportamento complessivo del sistema.

In conclusione, il sistema di comunicazione del MicroBot non è semplicemente un mezzo per trasmettere dati, ma una componente strutturale che consente al sistema di funzionare come entità coerente. La sua progettazione deve tenere conto di latenza, affidabilità, scalabilità e integrazione con gli altri componenti, garantendo al contempo flessibilità ed efficienza.

5.4 Sistema di alimentazione Li-Po

Il sistema di alimentazione basato su batterie Li-Po (Lithium Polymer) rappresenta il fondamento energetico dell'architettura elettronica del MicroBot, costituendo la sorgente primaria di energia per tutte le componenti del nodo. In un sistema distribuito composto da molte unità autonome, la gestione dell'energia assume un ruolo centrale, in quanto determina direttamente la durata operativa, l'affidabilità e la capacità del sistema di sostenere comportamenti complessi.

Le batterie Li-Po sono particolarmente adatte al contesto MicroBot grazie alla loro elevata densità energetica, alla possibilità di essere realizzate in formati compatti e alla loro flessibilità di integrazione all'interno di strutture geometriche ridotte. Questa caratteristica è coerente con i vincoli di miniaturizzazione definiti nella Sezione 4, in cui ogni nodo deve contenere tutti i componenti essenziali in uno spazio limitato.

Dal punto di vista elettrico, una batteria Li-Po fornisce una tensione nominale tipicamente compresa tra 3.7 V e 4.2 V (a piena carica), che varia nel tempo in funzione dello stato di carica. Questa variabilità introduce la necessità di un sistema di regolazione della tensione, che verrà approfondito nel punto successivo, ma che già a questo livello deve essere considerato come parte integrante dell'architettura.

Il comportamento energetico del nodo può essere descritto attraverso una variabile di stato:

$E(t)$

che rappresenta l'energia disponibile nel tempo. L'evoluzione di questa variabile dipende dal consumo delle componenti elettroniche e può essere approssimata come:

$$E(t+1) = E(t) - P_{\text{tot}}(t) \cdot \Delta t$$

dove $P_{\text{tot}}(t)$ rappresenta la potenza totale assorbita dal nodo. Questa relazione evidenzia come ogni operazione — comunicazione, elaborazione, attivazione di componenti — contribuisca alla riduzione dell'energia disponibile.

Un aspetto fondamentale del sistema Li-Po è la gestione della sicurezza. Le batterie di questo tipo sono sensibili a condizioni operative non corrette, come sovraccarico, scarica eccessiva o cortocircuito. Per questo motivo, è necessario integrare circuiti di protezione che monitorano la tensione e interrompono il flusso di corrente in caso di anomalie. Questo sistema di protezione è essenziale per garantire l'affidabilità del nodo e la sicurezza dell'intero sistema.

Dal punto di vista funzionale, la batteria deve essere in grado di fornire energia a componenti con requisiti differenti. Il microcontrollore richiede una tensione stabile e una corrente relativamente contenuta, mentre altri sottosistemi, come quelli di attuazione, possono richiedere correnti più elevate. Questo introduce la necessità di una distribuzione dell'energia che tenga conto delle diverse esigenze, evitando interferenze tra i componenti.

Un elemento particolarmente rilevante riguarda la capacità della batteria, espressa tipicamente in milliampere-ora (mAh). Questa grandezza determina la quantità totale di energia disponibile e quindi la durata operativa del nodo. La scelta della capacità deve bilanciare due esigenze opposte: da un lato, aumentare la capacità migliora l'autonomia; dall'altro, batterie più grandi occupano più spazio e aumentano il peso, influenzando la geometria e il comportamento del MicroBot.

Nel contesto MicroBot, la gestione dell'energia non è solo un problema locale, ma ha implicazioni a livello di sistema. Poiché ogni nodo ha una capacità energetica limitata, lo stato energetico può influenzare il comportamento dello swarm. Ad esempio, nodi con energia ridotta potrebbero ridurre la propria attività o segnalare la propria condizione al controller. Questo introduce una dimensione energetica nel modello dello stato, che può essere integrata nella gestione del sistema.

Un ulteriore aspetto riguarda la ricarica. Nei prototipi iniziali, la ricarica può essere effettuata manualmente, ma in un sistema più avanzato potrebbe essere necessario introdurre meccanismi di ricarica automatica, come contatti fisici o sistemi wireless. Anche se questo aspetto non è centrale nella fase iniziale, rappresenta una direzione importante per l'evoluzione del progetto.

Dal punto di vista architetturale, la batteria deve essere integrata in modo coerente con la struttura del MicroBot. La sua posizione influisce sul centro di massa e sulla distribuzione dei volumi interni, già discussa nella Sezione 4. Una disposizione equilibrata è fondamentale per garantire stabilità e uniformità nelle interazioni.

Un aspetto critico è la gestione delle variazioni di tensione. Poiché la tensione della batteria diminuisce con il tempo, i componenti elettronici devono essere progettati per operare correttamente in un intervallo di tensione definito. Questo introduce la necessità di regolatori e sistemi di stabilizzazione, che saranno trattati nel punto successivo.

Dal punto di vista della progettazione elettronica, il sistema Li-Po rappresenta il punto di partenza per la definizione di tutto il circuito. La scelta della batteria influisce sui regolatori, sui driver e sui componenti collegati, creando una catena di dipendenze che deve essere gestita con attenzione.

Infine, il sistema di alimentazione Li-Po si collega direttamente alla gestione computazionale. Il microcontrollore può monitorare lo stato della batteria e adattare il comportamento del nodo in funzione dell'energia disponibile. Questo permette di implementare strategie di risparmio energetico e di migliorare l'efficienza complessiva del sistema.

In conclusione, il sistema di alimentazione Li-Po costituisce la base energetica del MicroBot, influenzando tutte le componenti del nodo. La sua progettazione richiede un equilibrio tra capacità, dimensioni, sicurezza e integrazione con gli altri sistemi, e rappresenta un elemento fondamentale per il funzionamento e l'evoluzione dell'architettura elettronica.

5.5 Gestione dell'energia

La gestione dell'energia rappresenta il livello logico e operativo attraverso cui il sistema MicroBot controlla, distribuisce e ottimizza l'utilizzo della risorsa energetica disponibile. In continuità con il punto precedente, in cui è stata definita la batteria Li-Po come sorgente primaria di energia, è ora necessario analizzare come questa energia venga effettivamente utilizzata all'interno del nodo e come il sistema possa adattare il proprio comportamento in funzione dei vincoli energetici.

In un sistema distribuito come MicroBot, ogni nodo è energeticamente autonomo ma limitato. Questo implica che la gestione dell'energia non può essere affidata a un controllo globale centralizzato, ma deve essere implementata localmente, all'interno di ciascun MicroBot. Tuttavia, le decisioni energetiche locali contribuiscono al comportamento globale dello swarm, rendendo la gestione dell'energia un problema sia locale che sistemico.

Dal punto di vista formale, il nodo può essere descritto come un sistema in cui l'energia disponibile $E(t)$ influenza direttamente le operazioni eseguibili. Ogni azione del nodo ha un costo energetico, e la gestione dell'energia consiste nel selezionare le operazioni in modo da massimizzare l'efficienza e la durata operativa. Questo può essere espresso come:

$$E(t+1) = E(t) - \sum P_i(t) \cdot \Delta t$$

dove $P_i(t)$ rappresenta la potenza assorbita da ciascun sottosistema. Questa relazione evidenzia come il consumo totale sia la somma dei contributi delle singole componenti, rendendo necessario un controllo coordinato.

Un primo aspetto della gestione dell'energia riguarda la distribuzione tra i sottosistemi. Il microcontrollore, il sistema di comunicazione, i dispositivi di feedback e i circuiti di potenza hanno esigenze energetiche differenti. Il sistema deve quindi garantire che ogni componente riceva energia sufficiente per operare correttamente, evitando al contempo sprechi o sovraccarichi.

Un secondo aspetto riguarda la priorità delle operazioni. In condizioni di energia limitata, il nodo deve essere in grado di distinguere tra operazioni essenziali e non essenziali. Ad esempio, la comunicazione e la gestione dello stato possono essere considerate prioritarie, mentre il feedback visivo o altre funzioni accessorie possono essere ridotte o disattivate. Questa distinzione introduce una gerarchia delle funzioni, che deve essere definita a livello di firmware.

Un ulteriore elemento di rilievo è la gestione dinamica del consumo. Il microcontrollore può adattare il proprio comportamento in funzione dell'energia disponibile, modificando la frequenza delle operazioni, riducendo l'attività o entrando in modalità di risparmio energetico. Nei sistemi embedded, queste modalità permettono di ridurre significativamente il consumo quando il nodo non è attivamente coinvolto in operazioni critiche.

Nel contesto MicroBot, la gestione dell'energia può essere integrata con il sistema di comunicazione. Il nodo può trasmettere informazioni sul proprio stato energetico al controller, permettendo una visione globale della distribuzione dell'energia nello swarm. Questo consente di implementare strategie avanzate, in cui il sistema tiene conto delle condizioni energetiche dei singoli nodi per coordinare le operazioni.

Un aspetto importante riguarda la stabilità del sistema. Una gestione inefficiente dell'energia può portare a comportamenti instabili, come spegnimenti improvvisi o malfunzionamenti. Il firmware deve quindi includere meccanismi di protezione, che impediscano al nodo di operare in condizioni critiche. Ad esempio, può essere definita una soglia minima di energia al di sotto della quale alcune operazioni vengono disabilitate.

Dal punto di vista temporale, la gestione dell'energia è strettamente legata al ciclo di esecuzione del nodo. Le operazioni devono essere distribuite nel tempo in modo da evitare picchi di consumo e garantire un utilizzo uniforme della batteria. Questo può essere ottenuto attraverso scheduling delle attività o limitazione della frequenza di alcune operazioni.

Un ulteriore elemento di connessione riguarda la progettazione hardware. La gestione dell'energia non è solo una questione software, ma dipende anche dalla scelta dei componenti e dalla configurazione del circuito. Componenti più efficienti e regolatori ben progettati possono ridurre il consumo complessivo, migliorando le prestazioni del sistema.

Dal punto di vista teorico, l'energia può essere considerata come una dimensione dello stato del nodo, che influenza direttamente le sue capacità operative. Questa dimensione si integra con le altre variabili dello stato, contribuendo alla definizione del comportamento del sistema. In questo senso, la gestione dell'energia non è solo un problema tecnico, ma una parte integrante del modello del sistema.

Un aspetto particolarmente interessante riguarda la possibilità di strategie cooperative. In un sistema distribuito, i nodi potrebbero adattare il proprio comportamento in funzione dello stato energetico degli altri, ad esempio riducendo il carico su nodi con energia limitata. Anche se questa funzionalità non è presente nei prototipi iniziali, rappresenta una direzione di sviluppo avanzata.

Infine, la gestione dell'energia rappresenta un elemento chiave per la scalabilità del sistema. Con l'aumentare del numero di nodi, l'efficienza energetica diventa sempre più importante, in quanto piccoli sprechi su singole unità si amplificano a livello globale. Una progettazione attenta consente di mantenere il sistema efficiente anche su larga scala.

In conclusione, la gestione dell'energia nel MicroBot rappresenta il meccanismo attraverso cui il sistema controlla e ottimizza l'utilizzo della risorsa energetica. Essa integra aspetti hardware e software, influenzando direttamente il comportamento del nodo e del sistema distribuito. Una gestione efficace è essenziale per garantire durata operativa, stabilità e scalabilità, costituendo un elemento fondamentale dell'architettura elettronica.

5.6 Regolazione della tensione

La regolazione della tensione rappresenta una componente fondamentale dell'architettura elettronica del MicroBot, in quanto garantisce che tutti i dispositivi interni ricevano un'alimentazione stabile e compatibile con i loro requisiti operativi. In continuità con i punti precedenti relativi alla batteria Li-Po e alla gestione dell'energia, questo livello introduce il meccanismo attraverso cui una sorgente energetica variabile viene trasformata in una distribuzione di tensioni controllate.

Le batterie Li-Po, come descritto nella sezione 5.4, non forniscono una tensione costante, ma una tensione che varia nel tempo in funzione dello stato di carica. Questo comportamento è incompatibile con il funzionamento diretto di molti componenti elettronici, che richiedono valori di tensione specifici e relativamente stabili. Il microcontrollore, ad esempio, opera tipicamente a 3.3 V, e variazioni significative rispetto a questo valore possono causare malfunzionamenti o danni permanenti.

Per questo motivo, è necessario introdurre un sistema di regolazione della tensione che trasformi la tensione della batteria in uno o più livelli stabili. Questo sistema è generalmente realizzato attraverso regolatori di tensione, che possono essere di tipo lineare o switching (a commutazione). La scelta tra queste soluzioni dipende da diversi fattori, tra cui efficienza, complessità e dissipazione termica.

I regolatori lineari operano riducendo la tensione in ingresso a un valore inferiore attraverso dissipazione di energia sotto forma di calore. Essi sono semplici da implementare e producono una tensione molto stabile, ma risultano inefficienti quando la differenza tra tensione in ingresso e in uscita è elevata. Nei sistemi MicroBot, possono essere utilizzati per alimentare componenti sensibili, dove la stabilità è prioritaria rispetto all'efficienza.

I regolatori switching, al contrario, utilizzano tecniche di commutazione ad alta frequenza per convertire la tensione con maggiore efficienza. Essi possono ridurre o aumentare la tensione (step-down o step-up) e risultano particolarmente utili quando è necessario ottimizzare il consumo energetico. Tuttavia, introducono maggiore complessità circuitale e possono generare rumore elettrico, che deve essere gestito con attenzione.

Dal punto di vista architetturale, il sistema di regolazione della tensione può essere visto come un livello intermedio tra la batteria e i componenti del nodo. La batteria fornisce una tensione non regolata, mentre i regolatori distribuiscono tensioni specifiche ai vari sottosistemi. Questa distribuzione può includere, ad esempio, una linea a 3.3 V per il microcontrollore e altre linee per eventuali componenti aggiuntivi.

Un aspetto fondamentale della regolazione della tensione è la stabilità. I regolatori devono essere in grado di mantenere la tensione costante anche in presenza di variazioni nel carico, cioè quando i componenti collegati assorbono corrente in modo variabile. Questo è particolarmente importante in un sistema come MicroBot, in cui le operazioni del nodo possono cambiare rapidamente, generando variazioni di consumo.

Un ulteriore elemento di rilievo riguarda la risposta dinamica. Il sistema di regolazione deve reagire rapidamente alle variazioni di carico, evitando cadute di tensione che potrebbero influenzare il funzionamento del microcontrollore o di altri componenti. Questo richiede una progettazione attenta del circuito, inclusa la scelta di condensatori di filtro e altri elementi passivi.

Dal punto di vista della sicurezza, la regolazione della tensione contribuisce alla protezione del sistema. Limitando la tensione fornita ai componenti, si riduce il rischio di sovratensioni che potrebbero danneggiarli. Inoltre, alcuni regolatori includono protezioni integrate contro sovracorrente e surriscaldamento, migliorando l'affidabilità complessiva.

Un aspetto importante riguarda l'integrazione con la gestione dell'energia. Il sistema di regolazione deve essere progettato in modo da minimizzare le perdite e massimizzare l'efficienza, contribuendo alla durata operativa del nodo. Nei sistemi distribuiti, anche piccoli miglioramenti di efficienza possono avere un impatto significativo su larga scala.

Dal punto di vista della progettazione fisica, i regolatori e i componenti associati devono essere posizionati in modo da ridurre interferenze e migliorare la dissipazione del calore. Questo si collega direttamente alla distribuzione dei volumi interni del MicroBot, già discussa nella Sezione 4.

Un ulteriore elemento di connessione riguarda la robustezza del sistema. Una regolazione inadeguata della tensione può portare a comportamenti instabili, come reset del microcontrollore o malfunzionamenti dei componenti. Per questo motivo, la progettazione di questo livello deve essere particolarmente accurata.

Dal punto di vista teorico, la regolazione della tensione può essere interpretata come una funzione che mappa una sorgente energetica variabile in un insieme di risorse stabili, utilizzabili dai diversi sottosistemi. Questa funzione rappresenta un elemento di stabilizzazione all'interno del sistema, riducendo la complessità introdotta dalla variabilità della batteria.

Infine, la regolazione della tensione rappresenta un prerequisito per il corretto funzionamento di tutte le altre componenti elettroniche. Senza un sistema di regolazione efficace, non sarebbe possibile garantire la stabilità e l'affidabilità del nodo.

In conclusione, la regolazione della tensione costituisce un elemento chiave dell'architettura elettronica del MicroBot, permettendo di trasformare una sorgente energetica variabile in un sistema di alimentazione stabile e affidabile. Essa integra aspetti di efficienza, sicurezza e stabilità, contribuendo in modo determinante al funzionamento complessivo del nodo.

5.7 Convertitori e stabilizzazione

I convertitori e i sistemi di stabilizzazione rappresentano il livello avanzato della gestione dell'alimentazione all'interno del MicroBot, completando e approfondendo quanto introdotto nella sezione precedente sulla regolazione della tensione. Se la regolazione garantisce un valore nominale stabile, i convertitori e i circuiti di stabilizzazione assicurano che tale valore venga mantenuto in condizioni reali, caratterizzate da variazioni dinamiche di carico, rumore elettrico e transitori.

Nel contesto MicroBot, la presenza di componenti con comportamenti differenti — microcontrollore, moduli di comunicazione, sistemi di attuazione e dispositivi di feedback — introduce un carico variabile nel tempo. Questo significa che il sistema di alimentazione non opera in condizioni statiche, ma deve adattarsi continuamente a richieste energetiche che cambiano in modo rapido. I convertitori e i circuiti di stabilizzazione sono progettati proprio per gestire queste variazioni, garantendo continuità e qualità dell'alimentazione.

Dal punto di vista funzionale, i convertitori possono essere classificati in base al tipo di trasformazione che effettuano. I convertitori step-down (buck) riducono la tensione in ingresso a un valore inferiore, mentre i convertitori step-up (boost) la aumentano. In alcuni casi, possono essere utilizzati convertitori buck-boost, che permettono di ottenere una tensione stabile anche quando quella in ingresso è superiore o inferiore al valore desiderato. Questa flessibilità è particolarmente utile nel caso di batterie Li-Po, la cui tensione varia durante il ciclo di scarica.

Nel sistema MicroBot, l'utilizzo di convertitori step-down è comune per alimentare il microcontrollore e i circuiti logici a partire dalla tensione della batteria. In configurazioni più avanzate, potrebbero essere necessari convertitori step-up per alimentare componenti che richiedono tensioni superiori, anche se questo scenario dipende dalle scelte progettuali specifiche.

Un aspetto fondamentale dei convertitori switching è l'efficienza. A differenza dei regolatori lineari, essi minimizzano la dissipazione di energia, rendendoli particolarmente adatti a sistemi alimentati a batteria. Tuttavia, questa efficienza è accompagnata dalla generazione di rumore elettrico dovuto alla commutazione ad alta frequenza. Questo rumore può interferire con il funzionamento del microcontrollore e dei moduli di comunicazione, rendendo necessaria l'introduzione di sistemi di filtraggio.

La stabilizzazione della tensione non si limita quindi alla conversione, ma include anche la riduzione del rumore e delle oscillazioni. Questo viene ottenuto attraverso l'utilizzo di componenti passivi, come condensatori e induttori, che filtrano le variazioni indesiderate e rendono la tensione più uniforme. I condensatori di bypass, ad esempio, vengono posizionati vicino ai pin di alimentazione del microcontrollore per assorbire variazioni rapide di corrente, migliorando la stabilità locale.

Un ulteriore elemento di rilievo riguarda la gestione dei transitori. Quando un componente viene attivato o disattivato, può verificarsi un cambiamento improvviso nel consumo di corrente, che si traduce in una variazione temporanea della tensione. I sistemi di stabilizzazione devono essere in grado di assorbire questi transitori, evitando che si propaghino nel circuito e influenzino altri componenti.

Dal punto di vista architetturale, i convertitori e i circuiti di stabilizzazione costituiscono un livello di mediazione tra la sorgente energetica e i carichi. Essi distribuiscono l'energia in modo controllato, adattandosi alle condizioni operative e garantendo che ogni componente riceva un'alimentazione adeguata. Questa funzione è particolarmente importante in un sistema distribuito, dove la ripetizione del nodo su larga scala amplifica l'impatto di eventuali instabilità.

Un aspetto critico riguarda la progettazione fisica del circuito. I convertitori switching operano ad alta frequenza e richiedono una disposizione attenta dei componenti per minimizzare le interferenze elettromagnetiche. La lunghezza delle tracce, la posizione dei componenti e la qualità delle connessioni influenzano direttamente le prestazioni del sistema. Questo richiede un approccio progettuale preciso, soprattutto nelle versioni più avanzate del MicroBot.

Dal punto di vista energetico, l'efficienza dei convertitori influisce direttamente sulla durata operativa del nodo. Perdite anche moderate possono ridurre significativamente l'autonomia, soprattutto in sistemi con molte unità. Per questo motivo, la scelta dei convertitori e la loro configurazione devono essere ottimizzate in funzione dei requisiti del sistema.

Un ulteriore elemento di connessione riguarda la gestione computazionale. Il microcontrollore può monitorare lo stato dell'alimentazione e adattare il comportamento del nodo in funzione della stabilità della tensione. Ad esempio, in presenza di instabilità, può ridurre il carico o ritardare alcune operazioni, contribuendo alla stabilizzazione del sistema.

Dal punto di vista della robustezza, i convertitori e i sistemi di stabilizzazione contribuiscono a prevenire malfunzionamenti. Una tensione instabile può causare reset del microcontrollore, errori di comunicazione o comportamenti imprevedibili. Garantire un'alimentazione stabile è quindi essenziale per il corretto funzionamento del nodo.

Infine, i convertitori e i sistemi di stabilizzazione rappresentano un elemento chiave per la scalabilità del sistema MicroBot. In un sistema composto da molte unità, la qualità dell'alimentazione deve essere garantita in modo uniforme, evitando variazioni che potrebbero compromettere il comportamento collettivo.

In conclusione, i convertitori e i sistemi di stabilizzazione costituiscono il livello avanzato della gestione dell'alimentazione nel MicroBot, garantendo efficienza, stabilità e qualità dell'energia fornita ai componenti. Essi completano il sistema di regolazione della tensione e rappresentano un elemento fondamentale per il funzionamento affidabile e scalabile dell'architettura elettronica.

5.8 Driver MOSFET

Il driver MOSFET rappresenta il componente elettronico attraverso cui il microcontrollore può controllare carichi che richiedono una corrente superiore rispetto a quella erogabile direttamente dai pin GPIO. Nel sistema MicroBot, questo elemento assume un ruolo fondamentale perché consente di separare il livello logico, gestito dall'ESP32, dal livello di potenza, necessario per alimentare dispositivi come bobine elettromagnetiche, LED ad alta intensità o altri attuatori futuri. Questa separazione è essenziale per garantire sicurezza, stabilità e corretto funzionamento del nodo.

L'ESP32, come la maggior parte dei microcontrollori, è progettato per generare segnali logici a bassa potenza. I suoi pin possono fornire o assorbire correnti limitate e non devono essere utilizzati direttamente per pilotare carichi significativi. Collegare una bobina o un carico induttivo direttamente a un GPIO sarebbe una scelta errata, perché potrebbe danneggiare il microcontrollore, causare instabilità elettrica o produrre comportamenti imprevedibili. Il MOSFET viene quindi inserito come elemento intermedio, con la funzione di interruttore elettronico controllato.

Dal punto di vista funzionale, il MOSFET permette al microcontrollore di controllare un circuito di potenza senza dover fornire direttamente la corrente richiesta dal carico. Il principio è semplice: il pin GPIO dell'ESP32 pilota il gate del MOSFET, mentre la corrente principale scorre tra drain e source, attraversando il carico collegato. In questo modo, un segnale logico a bassa corrente può governare un flusso energetico molto più elevato. Questa architettura consente di mantenere il microcontrollore protetto e di affidare la gestione della potenza a un componente progettato appositamente per tale scopo.

Nel contesto MicroBot, il MOSFET può essere impiegato soprattutto nel controllo delle bobine elettromagnetiche. Poiché una bobina richiede corrente per generare un campo magnetico, il microcontrollore non deve alimentarla direttamente, ma deve soltanto decidere quando e con quale intensità

attivarla. Il MOSFET diventa quindi il componente che traduce il comando logico in attivazione elettrica del carico. Questo realizza concretamente la catena funzionale già definita nelle sezioni precedenti:

comando software → segnale GPIO/PWM → driver MOSFET → corrente nel carico → effetto fisico

Questa catena è uno dei punti più importanti dell'intera architettura elettronica, perché mostra come il sistema passi dal dominio informazionale al dominio fisico. Il comando non rimane una semplice istruzione software, ma viene trasformato in una variazione reale dello stato elettrico del nodo. Nel caso delle bobine, tale variazione produce un campo magnetico; nel caso dei LED, produce un feedback visivo; in eventuali versioni future, potrebbe produrre movimento, vibrazione o altre forme di attuazione.

Un aspetto importante riguarda il tipo di MOSFET da utilizzare. Nei sistemi embedded a bassa tensione è opportuno utilizzare MOSFET logic-level, cioè componenti capaci di attivarsi correttamente con tensioni di gate compatibili con i livelli logici del microcontrollore, tipicamente 3.3 V nel caso dell'ESP32. Non tutti i MOSFET sono adatti a questo scopo: alcuni richiedono tensioni di gate più elevate per entrare pienamente in conduzione. Se si utilizzasse un MOSFET non logic-level, il componente potrebbe non aprirsi completamente, aumentando la resistenza interna, dissipando più calore e riducendo l'efficienza del circuito.

La resistenza in conduzione del MOSFET, indicata spesso come $R_{DS(on)}$, è un parametro fondamentale. Quando il MOSFET è attivo, esso non si comporta come un interruttore ideale, ma introduce una piccola resistenza. La potenza dissipata può essere approssimata come:

$$P = I^2 \cdot R_{DS(on)}$$

Questa relazione mostra che, al crescere della corrente, anche una resistenza molto bassa può produrre dissipazione termica significativa. Per questo motivo, nella scelta del MOSFET è necessario considerare la corrente prevista, la tensione di alimentazione, il tipo di carico e le condizioni termiche del nodo. In un MicroBot, dove lo spazio è ridotto e la dissipazione del calore è limitata, questo aspetto diventa particolarmente rilevante.

Un altro elemento essenziale riguarda la configurazione del circuito. In molti casi, il MOSFET viene utilizzato come low-side switch, cioè viene posizionato tra il carico e massa. In questa configurazione, il carico è collegato al polo positivo dell'alimentazione, mentre il MOSFET controlla il collegamento verso GND. Quando il gate viene portato a livello alto dal microcontrollore, il MOSFET conduce e permette alla corrente di attraversare il carico. Quando il gate viene portato a livello basso, il MOSFET interrompe il circuito. Questa soluzione è semplice, efficiente e adatta a molti prototipi embedded.

Nel caso di carichi induttivi, come le bobine, il MOSFET deve essere accompagnato da un sistema di protezione. Una bobina immagazzina energia nel proprio campo magnetico; quando la corrente viene interrotta rapidamente, tale energia tende a generare una sovratensione di ritorno. Questo fenomeno può danneggiare il MOSFET, il microcontrollore o altri componenti del circuito. Per evitare questo problema, viene utilizzato un diodo di flyback, che verrà approfondito nel punto successivo dell'indice. In questa sede è importante chiarire che il MOSFET non deve mai essere considerato isolatamente: il suo corretto funzionamento dipende dalla progettazione dell'intero circuito di potenza.

Il MOSFET è inoltre compatibile con il controllo PWM. Questo aspetto è particolarmente importante perché permette di modulare la potenza fornita al carico. Invece di limitarsi a uno stato acceso/spento, il microcontrollore può generare un segnale a duty cycle variabile, regolando la corrente media nel carico. Nel caso di un LED, questo permette di controllare la luminosità; nel caso di una bobina, permette di modulare l'intensità media del campo magnetico, entro i limiti fisici ed elettrici del sistema. Il MOSFET diventa quindi non solo un interruttore, ma un elemento di controllo dinamico.

Dal punto di vista firmware, il controllo del MOSFET deve essere gestito con attenzione. Non basta attivare o disattivare un pin: è necessario definire stati sicuri, tempi massimi di attivazione, limiti di duty cycle e condizioni di spegnimento automatico. Questo è particolarmente importante quando il MOSFET controlla una bobina o un carico ad assorbimento elevato. Il firmware dovrebbe impedire attivazioni prolungate non necessarie e dovrebbe riportare il sistema in uno stato sicuro in caso di errore, perdita di comunicazione o instabilità dell'alimentazione.

La presenza del MOSFET rafforza anche il principio di separazione delle responsabilità, già introdotto nella Sezione 3. Il microcontrollore si occupa della decisione logica, mentre il MOSFET si occupa della

commutazione di potenza. Questa distinzione rende il sistema più robusto e più facilmente espandibile. Se in futuro il carico cambia, ad esempio passando da una piccola bobina a una bobina più potente, non è necessario modificare la logica generale del firmware, ma può essere sufficiente riprogettare il circuito di potenza mantenendo invariata l'interfaccia di controllo.

Dal punto di vista dell'integrazione fisica, il MOSFET deve essere posizionato in modo coerente con il layout interno del MicroBot. Deve trovarsi vicino al carico controllato, riducendo la lunghezza delle connessioni di potenza, ma deve anche essere disposto in modo da favorire la dissipazione termica e minimizzare interferenze con il microcontrollore e i circuiti di comunicazione. Questa scelta progettuale collega direttamente l'architettura elettronica con la distribuzione dei volumi interni descritta nella Sezione 4.

Un ulteriore aspetto riguarda la diagnostica. Il sistema può essere progettato per monitorare indirettamente il comportamento del carico controllato dal MOSFET. Ad esempio, attraverso misure di corrente o tensione, il microcontrollore potrebbe rilevare condizioni anomale come circuito aperto, corto circuito, surriscaldamento o carico non responsivo. Anche se nei prototipi iniziali questa funzione può essere semplificata, essa rappresenta un elemento importante nelle versioni più mature del sistema.

Nel modello complessivo MicroBot, il driver MOSFET rappresenta quindi un punto di conversione tra informazione e potenza. Esso permette al sistema di passare dal comando astratto all'azione elettrica controllata, mantenendo separati il livello logico e quello energetico. Questa funzione è indispensabile per qualunque nodo che debba interagire fisicamente con l'ambiente o con altri nodi, perché consente di controllare componenti reali senza compromettere la sicurezza del microcontrollore.

In conclusione, il driver MOSFET è una componente fondamentale dell'architettura elettronica del MicroBot. La sua funzione non è semplicemente quella di accendere o spegnere un carico, ma di consentire il controllo sicuro, efficiente e modulabile dei sottosistemi di potenza. La corretta progettazione di questo livello garantisce che il nodo possa trasformare le decisioni computazionali in effetti fisici controllati, mantenendo robustezza, scalabilità e coerenza con l'intera architettura distribuita del sistema.

5.9 Controllo delle bobine

Il controllo delle bobine rappresenta uno dei punti centrali dell'architettura elettronica del MicroBot, poiché costituisce il meccanismo attraverso cui il sistema può trasformare una decisione digitale in un'interazione fisica controllata. In continuità con il punto precedente, dedicato ai driver MOSFET, questa sezione analizza il modo in cui una bobina elettromagnetica può essere comandata in sicurezza dal microcontrollore, senza confondere il livello logico del firmware con il livello di potenza necessario alla generazione del campo magnetico. La bobina, infatti, non deve essere interpretata come un semplice componente collegato al circuito, ma come un attuatore elettromagnetico, cioè come un dispositivo capace di produrre un effetto fisico reale a partire da una grandezza elettrica controllata.

Nel sistema MicroBot, il controllo delle bobine ha una funzione strategica perché permette di superare il limite dei magneti permanenti. Un magnete permanente genera un campo costante, non modificabile dinamicamente dal software, mentre una bobina può essere accesa, spenta o modulata in funzione dello stato del nodo, dei comandi ricevuti e delle condizioni operative. Questo rende possibile introdurre una forma di controllo attivo dell'interazione tra MicroBot, mantenendo coerenza con l'idea generale di materia programmabile: la forma del sistema non dipende soltanto dalla geometria o dalla disposizione passiva dei componenti, ma può essere influenzata da istruzioni software che modificano localmente le forze in gioco.

Dal punto di vista elettronico, una bobina si comporta come un carico induttivo. Quando viene attraversata da corrente, genera un campo magnetico; quando la corrente varia, l'induttanza tende a opporsi alla variazione stessa. Questo comportamento distingue la bobina da un carico puramente resistivo, come un semplice LED con resistenza, e richiede una progettazione più attenta. La bobina immagazzina energia nel campo magnetico e può generare transitori elettrici quando viene disattivata rapidamente. Per questo motivo, il controllo della bobina non può essere ridotto a un semplice comando acceso/spento, ma deve essere progettato considerando corrente, tempi di attivazione, dissipazione termica, protezione del circuito e stabilità dell'alimentazione.

La catena di controllo può essere descritta come una successione di trasformazioni tecniche:

comando logico → segnale PWM/GPIO → MOSFET → corrente nella bobina → campo magnetico → forza/interazione

Questa catena mostra chiaramente la natura multilivello del sistema MicroBot. Il comando logico appartiene al dominio software; il segnale GPIO o PWM appartiene al dominio elettronico di controllo; il MOSFET appartiene al dominio di potenza; la corrente nella bobina appartiene al dominio elettrico; il campo magnetico appartiene al dominio fisico; la forza risultante appartiene al dominio meccanico e interattivo. In questo senso, il controllo delle bobine è uno dei punti in cui l'intero modello MicroBot diventa concreto, perché collega direttamente informazione, energia e materia.

Dal punto di vista firmware, la bobina può essere controllata in modalità digitale semplice oppure in modalità modulata. Nel primo caso, il microcontrollore attiva o disattiva la bobina portando il segnale di controllo del MOSFET a livello alto o basso. Questa modalità è utile nei prototipi iniziali, perché permette di verificare il funzionamento del circuito e osservare l'effetto fisico dell'attivazione. Tuttavia, un controllo puramente binario limita la precisione del sistema, perché consente soltanto due stati: bobina attiva o bobina inattiva. In un sistema avanzato, invece, è preferibile utilizzare un controllo PWM, che permette di regolare la potenza media applicata alla bobina variando il duty cycle del segnale.

Il PWM consente di modulare nel tempo l'attivazione della bobina senza richiedere un controllo analogico diretto. Se il duty cycle è basso, la bobina riceve energia per una frazione ridotta del periodo e genera un campo medio più debole; se il duty cycle aumenta, la corrente media cresce e con essa aumenta l'intensità media del campo magnetico. Questa tecnica è particolarmente adatta all'ESP32, che dispone di periferiche hardware dedicate alla generazione di segnali PWM. Tuttavia, è importante sottolineare che il PWM non elimina i vincoli fisici della bobina: la corrente, la temperatura e la risposta induttiva devono comunque rimanere entro limiti sicuri.

Un aspetto cruciale riguarda il tempo massimo di attivazione. Una bobina percorsa da corrente dissipa potenza sotto forma di calore secondo la relazione generale:

$$P = R \cdot I^2$$

dove R rappresenta la resistenza della bobina e I la corrente che la attraversa. Questa relazione mostra che la dissipazione cresce con il quadrato della corrente. Di conseguenza, anche un piccolo aumento della corrente può produrre un aumento significativo del calore generato. In un MicroBot, dove lo spazio interno è ridotto e la dissipazione termica è limitata, questo vincolo diventa particolarmente importante. Il firmware deve quindi impedire attivazioni prolungate non controllate, introducendo limiti temporali, soglie di sicurezza e strategie di spegnimento automatico.

Il controllo delle bobine deve inoltre essere integrato con la gestione dell'energia. Ogni attivazione della bobina rappresenta un consumo significativo rispetto ad altri componenti del nodo. Se più bobine o più nodi vengono attivati simultaneamente, il consumo complessivo può aumentare rapidamente, influenzando l'autonomia del sistema e la stabilità dell'alimentazione. Per questo motivo, il controllo delle bobine non deve essere considerato una funzione isolata, ma deve essere coordinato con il sistema di alimentazione Li-Po, con i convertitori, con i regolatori e con la logica di gestione energetica descritta nei punti precedenti.

Dal punto di vista della sicurezza elettronica, una bobina deve sempre essere accompagnata da protezioni adeguate. Il componente fondamentale, che verrà analizzato nel punto successivo, è il diodo di flyback. La sua funzione è proteggere il circuito dalla sovratensione generata quando la corrente nella bobina viene interrotta. Senza questa protezione, il MOSFET e il microcontrollore potrebbero essere esposti a picchi di tensione dannosi. Questo dimostra che il controllo delle bobine non riguarda soltanto il comando della corrente, ma anche la gestione dell'energia immagazzinata e rilasciata dal campo magnetico.

Un altro aspetto importante riguarda la sincronizzazione. Nel sistema MicroBot, l'attivazione delle bobine può avere senso solo se coordinata nel tempo con lo stato degli altri nodi. Se una bobina viene attivata troppo presto, troppo tardi o senza coerenza rispetto alla configurazione dello swarm, l'effetto fisico può risultare inefficace o addirittura destabilizzante. Per questo motivo, il controllo delle bobine deve essere collegato al sistema di sincronizzazione generale, in cui il controller centrale può fornire riferimenti temporali o comandi coordinati. Questo consente di ottenere attivazioni simultanee, sequenziali o selettive, a seconda del pattern desiderato.

Dal punto di vista dello stato del nodo, la bobina può essere trattata come una variabile controllata. Lo stato magnetico locale del MicroBot può essere rappresentato come:

`s_magnetic = (coil_enabled, duty_cycle, activation_time, thermal_state, safety_state)`

Questa rappresentazione permette di integrare il controllo della bobina nella macchina a stati del firmware. Il nodo non deve semplicemente “accendere” una bobina, ma deve sapere se essa è attiva, da quanto tempo è attiva, con quale intensità media viene pilotata, se il sistema si trova in condizioni termiche accettabili e se eventuali protezioni sono intervenute. Questa formalizzazione è utile perché rende il controllo più robusto e più facilmente integrabile con telemetria e diagnostica.

Nei prototipi iniziali del sistema MicroBot, il controllo delle bobine può essere simulato o sostituito temporaneamente da LED, proprio come già avvenuto nelle versioni dimostrative basate su segnali luminosi. Questo approccio è metodologicamente corretto perché consente di validare la pipeline logica prima di introdurre carichi elettrici più complessi. Il LED può rappresentare visivamente lo stato della bobina: acceso significa bobina attiva, spento significa bobina inattiva, luminosità variabile significa duty cycle variabile. Questa fase permette di verificare firmware, segnali, sincronizzazione e interfaccia prima di passare alla componente elettromagnetica reale.

Dal punto di vista dell'integrazione con la simulazione, il controllo delle bobine può essere modellato come una funzione che modifica le forze tra nodi. Nel simulatore, invece di generare realmente un campo magnetico, il sistema può associare al valore del duty cycle una forza virtuale o una tendenza all'aggregazione. Questa corrispondenza tra controllo reale e modello simulato è fondamentale perché permette di sviluppare e testare strategie prima dell'implementazione hardware. In questo senso, la bobina reale e la bobina simulata devono condividere una stessa astrazione logica, anche se operano in domini fisici differenti.

Un ulteriore aspetto riguarda il controllo selettivo. In una versione avanzata del MicroBot, un nodo potrebbe disporre di più bobine orientate in direzioni diverse. In questo caso, il firmware dovrebbe decidere non solo se attivare il sistema magnetico, ma quale bobina attivare, con quale intensità e per quanto tempo. Questo permetterebbe di controllare direzionalmente l'interazione, favorendo l'aggancio con un nodo specifico o la formazione di determinate configurazioni geometriche. Anche se questa funzionalità può essere considerata futura, è importante progettare l'architettura elettronica in modo da non escluderla.

Il controllo delle bobine deve anche essere compatibile con il safety layer. Qualunque attuatore capace di assorbire corrente significativa deve essere sottoposto a limiti e controlli. Il sistema deve impedire l'attivazione delle bobine in caso di batteria scarica, temperatura elevata, comando non valido o perdita di comunicazione. In assenza di un controllo sicuro, la bobina potrebbe rimanere attiva oltre il necessario, consumando energia e generando calore. Per questo motivo, ogni comando magnetico dovrebbe avere una durata massima predefinita o richiedere un rinnovo esplicito da parte del controller.

Dal punto di vista architetturale, il controllo delle bobine rappresenta un esempio perfetto di separazione delle responsabilità. Il controller centrale può decidere la strategia globale, il firmware locale può tradurre il comando in parametri operativi, il MOSFET può gestire la commutazione di potenza, la bobina può generare il campo fisico e il sistema di telemetria può monitorare l'esito dell'operazione. Nessun livello svolge da solo l'intera funzione, ma tutti contribuiscono a una catena coerente. Questa struttura riflette l'intera filosofia del progetto MicroBot.

In conclusione, il controllo delle bobine costituisce uno dei punti di maggiore importanza dell'architettura elettronica del MicroBot. Esso permette di trasformare il nodo da semplice unità computazionale a elemento fisicamente attivo, capace di modificare le proprie interazioni con gli altri nodi. La sua progettazione richiede attenzione a firmware, potenza, protezione, energia, calore, sincronizzazione e sicurezza. Proprio per questo motivo, il controllo delle bobine deve essere sviluppato in modo progressivo: prima simulato, poi rappresentato tramite LED, successivamente testato con piccoli carichi controllati e infine integrato nel sistema magnetico reale. In questa progressione si manifesta pienamente il metodo del progetto MicroBot: costruire complessità attraverso livelli verificabili, mantenendo sempre coerenza tra teoria, elettronica, software e comportamento fisico.

5.10 Diodo di flyback

Il diodo di flyback rappresenta un componente di protezione fondamentale nell'architettura elettronica del MicroBot quando vengono controllati carichi induttivi, come le bobine elettromagnetiche. In continuità con il punto precedente, dedicato al controllo delle bobine, questa sezione analizza il problema della sovratensione generata dall'interruzione della corrente in un'induttanza e il modo in cui il diodo di flyback permette di proteggere il MOSFET, il microcontrollore e l'intero circuito di alimentazione. All'interno del sistema MicroBot, questo componente non deve essere interpretato come un dettaglio secondario, ma come una condizione necessaria per rendere sicuro e affidabile il controllo attivo del campo magnetico.

Una bobina elettromagnetica non si comporta come un carico puramente resistivo. Quando una corrente attraversa la bobina, essa genera un campo magnetico e immagazzina energia all'interno di tale campo. Questa energia può essere espressa, in forma semplificata, come:

$$E = 1/2 \cdot L \cdot I^2$$

dove E rappresenta l'energia immagazzinata, L l'induttanza della bobina e I la corrente che la attraversa. Questa relazione evidenzia un punto essenziale: la bobina non consuma soltanto energia, ma ne accumula temporaneamente una parte nel proprio campo magnetico. Quando il MOSFET interrompe il passaggio di corrente, questa energia non può scomparire istantaneamente. La bobina tende quindi a mantenere la corrente in circolazione, generando una tensione inversa potenzialmente molto elevata.

Il fenomeno può essere descritto attraverso la relazione fondamentale dell'induttanza:

$$V = L \cdot di/dt$$

Questa equazione mostra che la tensione ai capi della bobina dipende dalla rapidità con cui la corrente cambia nel tempo. Se il MOSFET spegne improvvisamente la bobina, la variazione di corrente è molto rapida, quindi di/dt assume un valore elevato. Di conseguenza, la tensione generata dalla bobina può raggiungere valori molto superiori alla tensione di alimentazione del circuito. Questo picco di tensione, chiamato spesso back EMF o tensione di ritorno induttiva, può danneggiare il MOSFET, disturbare l'alimentazione, causare reset del microcontrollore o introdurre rumore elettrico nel sistema.

Il diodo di flyback viene inserito proprio per fornire un percorso sicuro alla corrente induttiva quando il MOSFET viene spento. In una configurazione tipica con MOSFET low-side, la bobina è collegata tra il positivo dell'alimentazione e il drain del MOSFET, mentre il source del MOSFET è collegato a massa. Il diodo viene posto in parallelo alla bobina, con il catodo collegato al lato positivo dell'alimentazione e l'anodo collegato al lato della bobina vicino al MOSFET. Durante il funzionamento normale, quando la bobina è alimentata, il diodo è polarizzato inversamente e non conduce. Quando il MOSFET si spegne e la bobina tenta di mantenere la corrente, il diodo diventa polarizzato direttamente e permette alla corrente di ricircolare in modo controllato.

Questo meccanismo protegge il circuito perché limita la tensione generata dalla bobina a un valore relativamente sicuro. Invece di produrre un picco incontrollato, l'energia immagazzinata nel campo magnetico viene dissipata gradualmente attraverso il percorso formato dalla bobina e dal diodo. Il diodo di flyback agisce quindi come una valvola di sicurezza elettrica: non modifica il comportamento normale della bobina durante l'attivazione, ma interviene nel momento critico dello spegnimento.

Nel contesto del MicroBot, questa protezione è particolarmente importante per tre motivi. Il primo riguarda la fragilità del microcontrollore. L'ESP32 lavora con livelli logici bassi, tipicamente 3.3 V, e non deve essere esposto a disturbi o sovratensioni provenienti dai carichi di potenza. Anche se il microcontrollore non alimenta direttamente la bobina, un transitorio forte può propagarsi attraverso massa, alimentazione o gate del MOSFET, causando instabilità. Il secondo motivo riguarda la protezione del MOSFET, che deve commutare il carico senza superare la propria tensione massima drain-source. Il terzo motivo riguarda la stabilità complessiva del nodo, perché un disturbo generato dalla bobina potrebbe influenzare comunicazione wireless, logging, telemetria o stati del firmware.

La presenza del diodo di flyback si collega direttamente alla filosofia di separazione delle responsabilità già definita nella Sezione 3. Il microcontrollore decide il comportamento, il MOSFET commuta il carico, la bobina genera il campo magnetico e il diodo gestisce il rilascio sicuro dell'energia induttiva. Ogni componente ha quindi una funzione precisa all'interno della catena elettronica. Senza il diodo, il sistema sarebbe incompleto, perché la fase di spegnimento della bobina resterebbe non controllata. In un sistema MicroBot maturo, non basta progettare l'attivazione di un componente: bisogna progettare anche la sua disattivazione sicura.

Un aspetto importante riguarda il tempo di decadimento della corrente. Il diodo di flyback protegge il circuito, ma rende più lento lo spegnimento effettivo della bobina. Poiché la corrente continua a circolare nel percorso di ricircolo, il campo magnetico non collassa istantaneamente, ma decade gradualmente. Questo significa che la bobina può continuare a generare un campo residuo per un breve intervallo anche dopo lo spegnimento del MOSFET. In applicazioni semplici questo comportamento è accettabile e spesso desiderabile, perché riduce lo stress elettrico. Tuttavia, in applicazioni che richiedono tempi di risposta molto

rapidi, può essere necessario adottare circuiti di clamp più avanzati, come diodi Zener, TVS o reti di protezione progettate per accelerare il decadimento della corrente.

Nel MicroBot, questa distinzione diventa rilevante perché il controllo delle interazioni magnetiche può richiedere precisione temporale. Se una bobina viene usata per attrarre o rilasciare un altro nodo, il tempo con cui il campo magnetico cresce e decade influenza la qualità del movimento o dell'aggancio. Un sistema molto semplice con solo diodo di flyback è adatto alle prime fasi prototipali, perché garantisce protezione e stabilità. In versioni successive, se il sistema richiederà attivazioni magnetiche più rapide e selettive, sarà possibile valutare soluzioni più sofisticate, mantenendo però lo stesso principio di base: l'energia induttiva deve sempre avere un percorso controllato.

La scelta del diodo deve essere coerente con la corrente della bobina e con la velocità richiesta dal circuito. Non basta inserire un diodo qualsiasi: esso deve sopportare la corrente di ricircolo generata dalla bobina e deve avere caratteristiche compatibili con la frequenza di commutazione prevista. In molti prototipi si possono utilizzare diodi standard per basse frequenze, ma se il controllo PWM è frequente o se le commutazioni sono rapide, può essere preferibile utilizzare diodi più veloci o Schottky, riducendo le perdite e migliorando la risposta del circuito. La selezione del componente deve quindi essere legata al dimensionamento della bobina, alla corrente prevista, alla tensione di alimentazione e alla strategia di controllo.

Dal punto di vista termico, anche il diodo può dissipare energia. Durante il ricircolo della corrente, il diodo conduce e presenta una caduta di tensione diretta. La potenza dissipata può essere approssimata come:

$$P_{\text{diodo}} = V_f \cdot I$$

dove V_f rappresenta la caduta di tensione diretta del diodo e I la corrente che lo attraversa. Anche se questa dissipazione è generalmente temporanea, può diventare rilevante se la bobina viene attivata e disattivata frequentemente. In un MicroBot, dove lo spazio interno è limitato e la dissipazione del calore è critica, anche questo aspetto deve essere considerato.

Il diodo di flyback contribuisce inoltre alla riduzione dei disturbi elettromagnetici. Una bobina che viene spenta senza protezione può generare picchi molto rapidi, capaci di introdurre rumore nel circuito e nell'ambiente circostante. Questo rumore può disturbare sensori, comunicazione wireless o segnali logici. Inserendo il diodo, il transitorio viene reso più controllato e meno aggressivo. In un sistema composto da molte unità, questo aspetto diventa ancora più importante, perché disturbi generati da più nodi potrebbero sommarsi e compromettere la stabilità dello swarm.

Dal punto di vista della diagnostica, il circuito di flyback può essere considerato parte del safety layer elettronico. Se il sistema rileva reset improvvisi, instabilità durante lo spegnimento delle bobine, interferenze nella comunicazione o malfunzionamenti del MOSFET, una delle prime aree da verificare è proprio la protezione del carico induttivo. Per questo motivo, nella documentazione tecnica del MicroBot è importante indicare chiaramente la presenza del diodo, il suo orientamento e il suo ruolo. Un errore di polarità o un diodo sottodimensionato possono compromettere il funzionamento dell'intero nodo.

In una fase di prototipazione, il diodo di flyback permette di sperimentare il controllo delle bobine con maggiore sicurezza. Prima ancora di ottimizzare il campo magnetico o il dimensionamento delle coil, è necessario garantire che il circuito possa sopportare cicli ripetuti di attivazione e disattivazione. Questo approccio è coerente con il metodo incrementale del progetto MicroBot: prima si stabilizza il livello elettrico, poi si passa all'ottimizzazione fisica e comportamentale. La protezione viene quindi prima della prestazione.

Il ruolo del diodo è importante anche nel collegamento tra hardware reale e simulazione. Nel simulatore, lo spegnimento di una bobina può essere modellato come una variazione istantanea dello stato magnetico, ma nel sistema fisico reale esiste sempre una dinamica di decadimento. Questo significa che il modello simulativo, nelle versioni più avanzate, potrebbe includere una costante di tempo associata alla bobina e al circuito di flyback. In questo modo, la simulazione diventerebbe più aderente al comportamento reale del nodo, migliorando la coerenza tra software e hardware.

In conclusione, il diodo di flyback è un componente essenziale per il controllo sicuro delle bobine elettromagnetiche nel sistema MicroBot. La sua funzione è proteggere il circuito dai picchi di tensione generati dai carichi induttivi, fornendo un percorso controllato alla corrente quando il MOSFET viene spento. Esso collega direttamente sicurezza elettronica, stabilità del firmware, affidabilità del nodo e qualità del

controllo magnetico. La sua presenza consente di trasformare il controllo delle bobine da esperimento rischioso a sottosistema elettronico robusto, coerente con l'obiettivo più ampio del progetto: costruire un sistema distribuito, modulare e fisicamente controllabile, in cui ogni livello tecnico sia verificabile, protetto e pronto per evoluzioni successive.

5.11 Protezione elettrica

La protezione elettrica rappresenta uno dei livelli fondamentali dell'architettura elettronica del MicroBot, perché consente di garantire il funzionamento sicuro del nodo in presenza di condizioni non ideali, variazioni di carico, errori di collegamento, disturbi elettrici e possibili anomalie generate dai sottosistemi di potenza. Dopo aver analizzato il ruolo del MOSFET, il controllo delle bobine e il diodo di flyback, diventa necessario formalizzare l'insieme delle strategie attraverso cui il circuito viene protetto da sovracorrenti, sovratensioni, inversioni di polarità, instabilità dell'alimentazione e transitori elettrici. In un sistema distribuito come MicroBot, la protezione elettrica non è un'aggiunta opzionale, ma una condizione strutturale per rendere il singolo nodo affidabile e, di conseguenza, per rendere stabile l'intero swarm.

Il principio di base della protezione elettrica è impedire che un'anomalia locale si trasformi in un guasto permanente o in un comportamento imprevedibile del sistema. Ogni MicroBot è un nodo autonomo, alimentato da una batteria, dotato di microcontrollore, linee di comunicazione, componenti di feedback e circuiti di potenza. Questa concentrazione di componenti in uno spazio ridotto rende il nodo particolarmente sensibile agli errori di progettazione e ai fenomeni elettrici indesiderati. Un picco di tensione, una corrente eccessiva o una massa mal gestita possono causare reset del microcontrollore, instabilità della comunicazione, danneggiamento dei componenti o perdita temporanea del controllo locale. Per questo motivo, ogni sottosistema deve essere progettato considerando non solo il funzionamento normale, ma anche le condizioni di errore.

Dal punto di vista architetturale, la protezione elettrica può essere interpretata come un livello trasversale. Essa non appartiene esclusivamente al sistema di alimentazione, né soltanto al circuito di potenza o al microcontrollore. Al contrario, attraversa tutti i livelli elettronici del nodo, perché ogni componente può essere sorgente o vittima di un'anomalia. La batteria può fornire correnti elevate in caso di corto circuito; i convertitori possono generare instabilità se mal dimensionati; le bobine possono produrre sovratensioni induttive; i MOSFET possono dissipare calore o essere sovraccaricati; l'ESP32 può bloccarsi o resettarsi se l'alimentazione scende sotto una soglia critica. La protezione elettrica deve quindi essere progettata come un sistema coordinato di difese, non come un singolo componente isolato.

Un primo livello di protezione riguarda la sovracorrente. La sovracorrente si verifica quando un carico assorbe più corrente di quella prevista o quando si crea un percorso anomalo tra alimentazione e massa. Nel MicroBot, questo rischio è particolarmente rilevante perché il nodo può includere carichi a maggiore assorbimento, come bobine elettromagnetiche o LED ad alta intensità. Se la corrente supera i limiti del circuito, possono verificarsi surriscaldamento, cadute di tensione, danni ai componenti o scarica rapida della batteria. Per ridurre questo rischio, è possibile utilizzare fusibili, polyfuse autoripristinanti, limitatori di corrente o circuiti di monitoraggio dedicati. Nei prototipi iniziali, anche una semplice scelta conservativa dei componenti e l'inserimento di resistenze di limitazione può rappresentare un primo livello di protezione.

Un secondo livello riguarda la sovratensione. Le sovratensioni possono derivare dalla batteria, da transitori generati dai carichi induttivi o da errori di collegamento. Il caso delle bobine è particolarmente significativo, perché l'interruzione rapida della corrente può generare tensioni molto superiori a quella nominale del circuito. Il diodo di flyback, analizzato nel punto precedente, rappresenta una protezione specifica contro questo fenomeno, ma non esaurisce il problema. In versioni più avanzate, può essere opportuno utilizzare diodi TVS, clamp di tensione, Zener o reti RC per assorbire transitori più rapidi e proteggere linee sensibili. L'obiettivo è impedire che tensioni anomale raggiungano il microcontrollore, i pin GPIO o i regolatori.

Un terzo aspetto fondamentale riguarda l'inversione di polarità. Durante la prototipazione, soprattutto quando si lavora con batterie e collegamenti manuali, esiste il rischio di collegare l'alimentazione al contrario. Anche un singolo errore può danneggiare immediatamente componenti sensibili. Per questo motivo, il circuito può includere protezioni contro l'inversione di polarità, realizzate tramite diodi in serie, MOSFET di protezione o connettori polarizzati. Nei sistemi miniaturizzati come MicroBot, l'utilizzo di connettori fisicamente non invertibili è una scelta particolarmente utile, perché riduce la probabilità dell'errore già a livello meccanico.

Un quarto livello riguarda la stabilità della massa e dei riferimenti elettrici. In un circuito embedded, la massa non è soltanto un collegamento generico, ma il riferimento comune rispetto al quale vengono interpretati segnali, tensioni e livelli logici. Se la massa è disturbata da correnti elevate o da ritorni impulsivi provenienti dai carichi di potenza, il microcontrollore può interpretare segnali errati o subire reset. Per questo motivo, è

importante separare concettualmente e fisicamente le correnti di potenza dalle correnti logiche, mantenendo collegamenti di massa ordinati e riducendo i percorsi condivisi tra carichi rumorosi e componenti sensibili. Questo principio diventa particolarmente importante quando le bobine o altri attuatori vengono controllati tramite MOSFET.

Un quinto aspetto riguarda la protezione dei pin GPIO. I pin del microcontrollore sono progettati per segnali logici, non per sopportare tensioni o correnti elevate. Ogni linea collegata a componenti esterni dovrebbe essere progettata in modo da evitare che un'anomalia possa propagarsi verso l'ESP32. Questo può includere resistenze in serie sul gate del MOSFET, resistenze di pull-down per garantire stati definiti all'avvio, partitori di tensione per segnali superiori a 3.3 V, optoisolatori in casi particolari o circuiti di interfaccia dedicati. Il principio fondamentale è che il microcontrollore deve comandare il sistema, ma non deve mai essere esposto direttamente alle condizioni elettriche più aggressive del circuito.

Nel caso specifico dei MOSFET, la protezione elettrica include anche la gestione dello stato del gate. Un gate lasciato flottante può portare il MOSFET in conduzione parziale o imprevedibile, causando attivazioni indesiderate del carico. Per questo motivo, è buona pratica inserire una resistenza di pull-down tra gate e massa, in modo che il MOSFET rimanga spento quando il microcontrollore non sta pilotando attivamente il segnale. Questo è particolarmente importante durante l'avvio del nodo, quando i pin del microcontrollore possono trovarsi temporaneamente in stato indefinito. In un MicroBot, un'attivazione involontaria di una bobina durante il boot potrebbe consumare energia, generare calore o produrre un'interazione fisica non prevista.

Un altro punto importante riguarda la protezione termica. Anche se la temperatura verrà trattata in modo più esteso nel safety layer, dal punto di vista elettronico è già necessario considerare la dissipazione dei componenti. Regolatori, MOSFET, diodi e bobine possono generare calore durante il funzionamento. In un contenitore piccolo, con spazio ridotto e ventilazione limitata, il calore può accumularsi rapidamente. La protezione elettrica deve quindi includere margini adeguati nella scelta dei componenti, limitazione dei tempi di attivazione e, nelle versioni più avanzate, monitoraggio termico tramite sensori o stime firmware. Un componente che funziona correttamente su banco può comportarsi in modo diverso quando viene inserito all'interno di un volume chiuso e miniaturizzato.

La protezione elettrica è inoltre collegata alla qualità dell'alimentazione. Cadute improvvise di tensione, ripple e rumore possono causare instabilità, soprattutto durante l'attivazione di carichi impulsivi. I condensatori di disaccoppiamento e di bulk svolgono una funzione essenziale in questo contesto. I condensatori di bypass vicini al microcontrollore assorbono variazioni rapide e stabilizzano la linea logica, mentre condensatori di capacità maggiore possono supportare il circuito durante transitori di carico. Questa stabilizzazione non sostituisce una buona progettazione dell'alimentazione, ma ne costituisce una parte essenziale.

Dal punto di vista del firmware, la protezione elettrica deve essere accompagnata da strategie software coerenti. Il software non può sostituire le protezioni hardware, ma può ridurre il rischio di condizioni pericolose. Ad esempio, il firmware può limitare il duty cycle massimo delle bobine, imporre un tempo massimo di attivazione, impedire comandi ripetuti troppo ravvicinati, verificare lo stato della batteria prima di attivare carichi pesanti e riportare il nodo in modalità sicura in caso di errore. Questa integrazione tra hardware e software è fondamentale perché MicroBot non è un circuito statico, ma un sistema dinamico controllato dal codice.

Formalmente, si può descrivere la protezione elettrica come un insieme di vincoli applicati allo stato operativo del nodo:

`operazione_consentita = funzione(stato_elettrico, stato_energetico, stato_termico, comando)`

Questa formulazione evidenzia che un comando ricevuto dal controller non deve essere eseguito automaticamente se le condizioni locali non sono sicure. Il nodo deve possedere una logica minima di validazione: se la tensione è insufficiente, se il carico è in errore, se la temperatura è elevata o se il comando richiede troppa corrente, il firmware deve poter rifiutare o limitare l'operazione. Questo rende il MicroBot non solo esecutore, ma nodo responsabile del proprio stato fisico.

La protezione elettrica contribuisce anche alla scalabilità dello swarm. In un sistema composto da molti nodi, l'affidabilità del singolo nodo ha un impatto diretto sull'affidabilità complessiva. Se ogni unità è fragile o sensibile a piccoli disturbi, l'aumento del numero di unità aumenta la probabilità di malfunzionamenti. Al contrario, se ogni nodo è progettato con protezioni adeguate, lo swarm può crescere mantenendo una

maggiore stabilità. La protezione elettrica diventa quindi una condizione per passare da prototipo singolo a sistema distribuito realmente scalabile.

Un ulteriore aspetto riguarda la diagnostica. Un sistema protetto deve anche essere osservabile. Quando si verifica un'anomalia, il nodo dovrebbe essere in grado di segnalare l'evento, ad esempio attraverso LED, messaggi seriali, log o telemetria. Un reset improvviso senza informazione diagnostica rende difficile capire il problema; un evento registrato con causa probabile, tensione rilevata e stato operativo permette invece di migliorare progressivamente il progetto. In questo senso, la protezione elettrica non è soltanto prevenzione del danno, ma anche produzione di conoscenza tecnica sul comportamento del sistema.

Nel ciclo di sviluppo MicroBot, la protezione elettrica deve essere introdotta fin dalle prime fasi, anche quando il sistema viene simulato o rappresentato tramite LED. Nei prototipi iniziali, le correnti possono essere basse e i rischi limitati, ma il metodo progettuale deve già prevedere separazione tra logica e potenza, limitazione delle correnti, stati sicuri e diagnostica. Questo consente di passare gradualmente verso componenti più complessi senza dover ripensare l'intera architettura. In altre parole, la sicurezza non deve essere aggiunta alla fine, ma incorporata nel modo stesso in cui il nodo viene progettato.

Dal punto di vista documentale, questa sezione ha anche una funzione metodologica. Essa chiarisce che il progetto MicroBot non si limita a immaginare un comportamento desiderato, ma affronta i vincoli reali della sua implementazione elettronica. La differenza tra un concetto teorico e un sistema ingegneristico funzionante passa proprio da questi livelli: alimentazione stabile, protezioni, margini di sicurezza, gestione dei transistori, controllo dei carichi e robustezza del firmware. Ogni elemento contribuisce a trasformare l'idea di materia programmabile in un sistema realizzabile.

In conclusione, la protezione elettrica costituisce un livello essenziale dell'architettura elettronica del MicroBot. Essa garantisce che i componenti possano operare in sicurezza, impedisce che anomalie locali provochino guasti permanenti, protegge il microcontrollore dai carichi di potenza e aumenta l'affidabilità complessiva del nodo. La sua progettazione deve integrare componenti hardware, strategie firmware, diagnostica e vincoli energetici. In un sistema distribuito, proteggere il singolo nodo significa proteggere lo swarm nel suo complesso: per questo motivo, la protezione elettrica rappresenta uno dei passaggi fondamentali nella transizione da prototipo sperimentale a piattaforma MicroBot realmente scalabile.

5.12 LED RGB come sistema di feedback

Il LED RGB rappresenta il componente attraverso cui il MicroBot può rendere visibile il proprio stato interno, trasformando informazioni logiche, energetiche, operative e diagnostiche in segnali luminosi immediatamente interpretabili. All'interno dell'architettura elettronica del sistema, il LED non deve essere considerato un semplice elemento estetico, ma un dispositivo di feedback locale, utile sia nella fase di prototipazione sia nelle versioni più avanzate del nodo. In continuità con i punti precedenti, dedicati al microcontrollore, all'alimentazione, ai circuiti di potenza e alla protezione elettrica, il LED RGB chiude la Sezione 5 mostrando come il sistema possa comunicare non soltanto attraverso rete e telemetria, ma anche attraverso un canale visivo incorporato direttamente nel MicroBot.

Nel progetto MicroBot, il feedback luminoso assume una funzione particolarmente importante perché consente di osservare il comportamento del nodo senza dover necessariamente accedere a strumenti esterni di debug. Un MicroBot può trovarsi in molti stati differenti: avvio, attesa, connessione, ricezione di comando, esecuzione di un pattern, errore, batteria bassa, modalità diagnostica o stato di sicurezza. Senza un sistema di feedback locale, questi stati resterebbero nascosti all'utente o sarebbero visibili soltanto attraverso log seriali, dashboard o telemetria. Il LED RGB permette invece di associare a ogni stato una rappresentazione cromatica o luminosa, rendendo il nodo immediatamente leggibile anche durante test rapidi, dimostrazioni fisiche o fasi di assemblaggio.

Dal punto di vista funzionale, il LED RGB può essere interpretato come un dispositivo di proiezione dello stato interno. Se lo stato locale del nodo è indicato come $s_i(t)$, il feedback luminoso può essere descritto come una funzione:

$$\text{colore}(t) = L(s_i(t))$$

dove L rappresenta la funzione di mapping tra stato interno e uscita luminosa. Questa formulazione è coerente con il modello complessivo del sistema, in cui l'informazione interna viene trasformata in una forma percepibile. Il LED, quindi, non genera comportamento fisico nello swarm come una bobina, ma genera osservabilità, cioè rende leggibile ciò che avviene all'interno del nodo. In un sistema complesso,

l'osservabilità è fondamentale quanto il controllo, perché senza feedback non è possibile verificare se il sistema sta funzionando correttamente.

Un LED RGB consente di rappresentare più stati rispetto a un LED monocromatico, perché combina tre canali fondamentali: rosso, verde e blu. Variando l'intensità relativa di questi tre canali è possibile ottenere un insieme ampio di colori, ciascuno associabile a una condizione operativa. Ad esempio, il verde può indicare funzionamento normale, il blu può indicare connessione attiva o ricezione dati, il rosso può indicare errore o condizione critica, il giallo può rappresentare avviso energetico, il viola può indicare modalità diagnostica o sincronizzazione. Questa codifica cromatica deve essere definita in modo coerente e documentata, così che il colore non sia arbitrario ma diventi parte del linguaggio operativo del sistema.

Dal punto di vista elettronico, il LED RGB può essere controllato direttamente dall'ESP32 tramite pin GPIO o, in modo più raffinato, attraverso segnali PWM. Il controllo PWM permette di regolare l'intensità di ciascun canale cromatico, ottenendo variazioni graduali, dissolvenze, pulsazioni e pattern luminosi. Questo è particolarmente utile perché non tutti gli stati devono essere rappresentati soltanto da colori fissi. Alcuni stati dinamici, come connessione in corso, sincronizzazione o attesa di comando, possono essere indicati attraverso lampeggi o pulsazioni. In questo modo, il feedback luminoso diventa temporale oltre che cromatico.

La gestione del LED RGB deve però rispettare gli stessi principi di robustezza e protezione descritti nei punti precedenti. Ogni canale del LED deve essere correttamente limitato in corrente, generalmente tramite resistenze dedicate o driver appropriati. Collegare un LED direttamente a un pin senza limitazione può danneggiare sia il LED sia il microcontrollore. Anche se il consumo di un LED è molto più basso rispetto a quello di una bobina, esso resta comunque un carico elettrico e deve essere progettato con attenzione. In un sistema con molti nodi, anche il consumo luminoso complessivo può diventare significativo, soprattutto se i LED restano accesi ad alta intensità per lunghi periodi.

Dal punto di vista firmware, il LED RGB dovrebbe essere gestito attraverso un modulo dedicato, separato dalla logica principale. Questa scelta rispetta il principio di separazione delle responsabilità già definito nella Sezione 3. Il firmware non dovrebbe accendere manualmente i colori in punti sparsi del codice, ma dovrebbe utilizzare una funzione o un servizio di stato, ad esempio:

```
set_status_led(stato)
```

In questo modo, la logica applicativa comunica lo stato del nodo, mentre il modulo LED decide come rappresentarlo. Questo approccio rende il sistema più ordinato, più modificabile e più facile da estendere. Se in futuro si volesse cambiare la codifica dei colori o introdurre pattern più complessi, sarebbe sufficiente intervenire sul modulo di feedback senza modificare tutta la logica del nodo.

Il LED RGB ha anche una funzione diagnostica fondamentale. Durante la fase di avvio, può indicare se il firmware è partito correttamente. Durante la connessione, può mostrare se il nodo è riuscito a collegarsi alla rete. Durante l'esecuzione, può confermare la ricezione di comandi o segnalare errori. In caso di malfunzionamento, un pattern luminoso può aiutare a distinguere rapidamente tra problemi diversi: alimentazione insufficiente, errore di comunicazione, comando non valido, reset del microcontrollore, stato di sicurezza o guasto del circuito di potenza. Questo rende il LED uno strumento di debugging immediato, particolarmente utile nei prototipi fisici.

Nel contesto del progetto MicroBot, i prototipi basati su LED hanno già un valore metodologico molto importante. Essi permettono di validare la logica del sistema prima di introdurre attuatori più complessi. Un LED può simulare lo stato di una bobina, lo stato di un nodo, il ruolo all'interno dello swarm o la ricezione di un comando. Questa fase non è "meno importante" della parte magnetica, ma rappresenta un passaggio corretto nello sviluppo incrementale: prima si verifica che il nodo riceva, interpreti e rappresenti correttamente i segnali; solo dopo si collega quella logica a componenti fisici più impegnativi dal punto di vista elettrico e meccanico.

Il LED RGB può inoltre essere integrato con la telemetria. Lo stato mostrato localmente dal nodo dovrebbe essere coerente con lo stato inviato al controller o visualizzato nella dashboard. Se un nodo mostra rosso per errore, anche il sistema software dovrebbe registrare e mostrare lo stesso errore. Se il nodo mostra blu per connessione attiva, la dashboard dovrebbe indicare che il nodo è online. Questa coerenza tra feedback fisico e feedback digitale è essenziale per evitare ambiguità. Il MicroBot deve comunicare lo stesso stato su più canali: LED locale, log, telemetria e interfaccia utente.

Dal punto di vista dell'interazione uomo-macchina, il LED RGB svolge un ruolo molto forte perché avvicina il sistema all'utente. In uno swarm reale, l'utente non osserva soltanto una dashboard, ma vede molte unità fisiche che cambiano colore, lampeggiano o reagiscono ai comandi. Questo crea una percezione immediata del comportamento collettivo. Pattern luminosi sincronizzati possono rappresentare sincronizzazione temporale, propagazione di comandi, formazione di gruppi, stati energetici o ruoli distribuiti. In questo senso, il LED RGB non è solo feedback del singolo nodo, ma può diventare feedback dello swarm.

Un aspetto interessante riguarda la rappresentazione dei ruoli. In un sistema distribuito, non tutti i nodi devono necessariamente svolgere la stessa funzione nello stesso momento. Alcuni possono essere nodi attivi, altri passivi, altri in attesa, altri in errore, altri ancora coinvolti in una specifica formazione. Il LED permette di rendere visibile questa differenziazione. Ad esempio, un nodo leader o coordinatore locale potrebbe avere un colore diverso rispetto ai nodi periferici; un nodo che partecipa a un pattern potrebbe lampeggiare con una frequenza specifica; un nodo escluso temporaneamente da un'operazione potrebbe usare un colore neutro. Questa codifica rafforza la leggibilità dello swarm.

Il feedback luminoso è utile anche per il safety layer. In presenza di batteria bassa, temperatura elevata, errore di comunicazione o condizione di protezione attiva, il LED può segnalare immediatamente che il nodo non è in stato operativo normale. Questo è importante perché alcune condizioni di sicurezza devono essere visibili anche senza aprire una console di debug. Un MicroBot che entra in modalità protetta dovrebbe comunicarlo chiaramente, sia al controller sia all'utente. In questo senso, il LED RGB diventa parte del sistema di sicurezza, non soltanto del sistema di visualizzazione.

Dal punto di vista energetico, il LED deve essere gestito con criterio. Un feedback troppo luminoso o sempre attivo può consumare energia inutilmente. Il firmware dovrebbe quindi prevedere livelli di intensità differenziati, modalità di spegnimento automatico o pattern a basso duty cycle. Nei momenti in cui il nodo è in idle, il LED può lampeggiare lentamente o ridurre la luminosità. Nei momenti diagnostici, può aumentare l'intensità o utilizzare pattern più evidenti. Questa gestione consente di mantenere il feedback utile senza compromettere l'autonomia del nodo.

Dal punto di vista della progettazione fisica, il LED RGB deve essere posizionato in modo visibile ma protetto. La sua collocazione deve permettere all'utente di osservare lo stato del nodo anche quando più MicroBot sono aggregati. Allo stesso tempo, non deve interferire con la disposizione interna dei componenti, con i magneti, con la batteria o con eventuali superfici di contatto. La scelta della posizione del LED è quindi sia funzionale sia ergonomica: deve rendere il nodo leggibile senza compromettere la struttura.

In una versione più avanzata, il feedback luminoso potrebbe diventare parte dell'identità del nodo. Ogni MicroBot potrebbe usare pattern o colori specifici per indicare il proprio identificatore, il proprio gruppo o il proprio ruolo nel sistema. Questo sarebbe utile soprattutto durante test con molti nodi, dove diventa difficile distinguere fisicamente un'unità dall'altra. Una breve sequenza luminosa potrebbe indicare l'ID del nodo, oppure la dashboard potrebbe inviare un comando di "identificazione" che fa lampeggiare un nodo specifico. Questa funzione sarebbe molto utile in fase di manutenzione e debug.

Il LED RGB rappresenta anche un ponte tra hardware reale e simulazione. Nelle simulazioni web o Python, lo stato del nodo può essere rappresentato graficamente con lo stesso codice cromatico del LED fisico. Se il nodo reale usa verde, blu, rosso o giallo per determinati stati, anche il modello virtuale dovrebbe usare gli stessi colori. Questo crea continuità tra prototipo fisico, simulazione e interfaccia. L'utente impara un linguaggio visivo unico, valido in tutti i livelli del sistema.

Dal punto di vista documentale, è opportuno definire una tabella di mapping tra stato e feedback luminoso. Questa tabella potrà essere inserita in una sezione tecnica o in un file di configurazione del progetto. Per esempio: avvio = bianco pulsante, idle = verde tenue, connessione = blu lampeggiante, comando ricevuto = ciano breve, errore = rosso lampeggiante, batteria bassa = giallo/arancione, modalità sicurezza = rosso fisso, diagnostica = viola. La cosa importante non è scegliere questi colori specifici, ma definire una convenzione stabile e coerente.

In conclusione, il LED RGB come sistema di feedback rappresenta l'ultimo componente fondamentale dell'architettura elettronica di base del MicroBot. Esso rende visibile lo stato interno del nodo, supporta debug, diagnostica, sicurezza, interazione e dimostrazione del comportamento collettivo. La sua importanza non deve essere sottovalutata: in un sistema distribuito, vedere lo stato dei nodi è essenziale per comprenderne il funzionamento. Il LED RGB trasforma il MicroBot da oggetto elettronico opaco a nodo osservabile, interpretabile e comunicativo. Con questo punto si chiude in modo coerente la Sezione 5, perché l'architettura elettronica viene completata non solo come insieme di alimentazione, controllo e

protezione, ma anche come sistema capace di restituire all'utente una rappresentazione immediata del proprio stato operativo.

SEZIONE 6 — Sistema Magnetico

Il sistema magnetico rappresenta uno dei nuclei funzionali più importanti dell'intero progetto MicroBot, perché costituisce il livello fisico attraverso cui le singole unità possono interagire, agganciarsi, mantenere una configurazione, separarsi e contribuire alla formazione di strutture collettive. Dopo aver definito nella Sezione 5 l'architettura elettronica del nodo, cioè microcontrollore, alimentazione, regolazione, driver, protezioni e feedback visivo, è ora possibile analizzare in modo rigoroso il sottosistema che traduce parte di quella struttura elettronica in interazione materiale tra MicroBot.

Nel sistema MicroBot, il magnetismo non deve essere considerato soltanto come un meccanismo di aggancio, ma come un vero livello di comportamento fisico. Il progetto non si limita infatti a far comunicare nodi tramite segnali digitali, ma mira a costruire un sistema in cui la forma collettiva emerga anche dalle forze reali tra unità. In questa prospettiva, il sistema magnetico è il punto in cui la materia programmabile smette di essere solo concetto teorico e diventa problema ingegneristico concreto: se le unità devono formare strutture, devono disporre di un meccanismo fisico capace di generare vincoli, attrazioni, allineamenti e, nelle versioni più avanzate, riconfigurazioni controllate.

Il sistema magnetico può essere sviluppato secondo due livelli principali. Il primo è il livello passivo, basato su magneti permanenti, in cui le forze sono determinate dalla geometria, dalla polarità e dalla distanza tra i nodi. Il secondo è il livello attivo, basato su bobine elettromagnetiche, in cui il campo può essere controllato attraverso corrente elettrica e quindi tramite logica software. Questa distinzione è fondamentale perché consente di mantenere una progressione progettuale coerente: prima si valida l'aggregazione fisica semplice, poi si introduce il controllo dinamico del campo.

Dal punto di vista del modello generale del sistema, il magnetismo può essere interpretato come una funzione che collega lo stato interno del nodo alle forze nello spazio fisico. Se lo stato locale del nodo è indicato con $s_i(t)$, il sottosistema magnetico produce una configurazione di campo e quindi una possibile interazione con i nodi vicini. In forma generale, questa relazione può essere espressa come:

$$F_{ij}(t) = M(s_i(t), s_j(t), r_{ij}(t), \text{orientamento})$$

dove $F_{ij}(t)$ rappresenta la forza tra due MicroBot, M rappresenta la funzione magnetica del sistema, $s_i(t)$ e $s_j(t)$ rappresentano gli stati dei due nodi, $r_{ij}(t)$ rappresenta la distanza tra essi e l'orientamento definisce la disposizione relativa dei poli o delle superfici magnetiche. Questa formulazione mostra subito che il magnetismo non dipende da un solo parametro, ma da una combinazione di stato, geometria, distanza e orientamento.

Nel contesto del progetto, questa sezione deve quindi svolgere una funzione duplice. Da un lato deve spiegare i principi fisici essenziali che rendono possibile l'interazione magnetica; dall'altro deve collegare questi principi alla struttura concreta del MicroBot, cioè alla sua geometria, ai componenti interni, all'elettronica di controllo e alla futura scalabilità dello swarm. Il magnetismo, infatti, non è separabile dal resto dell'architettura: dipende dalla batteria, dal driver MOSFET, dalla protezione elettrica, dalla disposizione interna dei volumi, dal firmware e dal safety layer. Tuttavia, in questa sezione viene analizzato nella sua logica specifica, seguendo l'indice in modo rigoroso.

6.1 Magneti permanenti

I magneti permanenti rappresentano il primo livello di implementazione del sistema magnetico MicroBot e costituiscono la soluzione più semplice, stabile ed energeticamente efficiente per ottenere interazioni fisiche tra unità. A differenza delle bobine elettromagnetiche, che richiedono corrente per generare un campo controllabile, i magneti permanenti producono un campo magnetico costante senza consumo energetico continuo. Questa caratteristica li rende particolarmente adatti alla fase prototipale e alla validazione iniziale del concetto di aggregazione tra MicroBot.

Nel contesto MicroBot, un magnete permanente può essere interpretato come un componente fisico che introduce una relazione di attrazione o repulsione tra nodi in funzione della polarità, della distanza e dell'orientamento. Ogni MicroBot, integrando uno o più magneti permanenti, diventa capace di interagire meccanicamente con altre unità senza richiedere un comando attivo da parte del microcontrollore. Questo permette di realizzare un primo livello di aggregazione passiva, in cui la stabilità delle connessioni deriva direttamente dalle proprietà fisiche dei componenti.

Dal punto di vista fisico, un magnete permanente può essere modellato in prima approssimazione come un dipolo magnetico. Il dipolo possiede un momento magnetico, indicato generalmente con m , che descrive l'intensità e l'orientamento del campo prodotto. Quando due dipoli magnetici si trovano vicini, essi interagiscono generando forze e momenti torcenti che tendono ad allinearli secondo configurazioni energeticamente favorevoli. Questo comportamento è estremamente utile nel sistema MicroBot, perché permette ai nodi di orientarsi e agganciarsi senza una gestione complessa da parte del software.

Il vantaggio principale dei magneti permanenti è la loro semplicità. Essi non richiedono circuiti di pilotaggio, non consumano corrente durante il mantenimento dell'aggancio e non generano calore per effetto Joule come avviene nelle bobine percorse da corrente. In una fase iniziale del progetto, questa semplicità è molto importante, perché consente di separare il problema fisico dell'aggancio dal problema elettronico del controllo attivo. In altre parole, i magneti permanenti permettono di verificare prima se la geometria del MicroBot e le superfici di contatto sono adatte alla formazione di connessioni stabili.

Tuttavia, questa semplicità introduce anche un limite fondamentale: il campo magnetico generato da un magnete permanente non può essere spento o modulato via software. La forza è sempre presente e dipende quasi esclusivamente dalla configurazione fisica. Questo significa che il sistema può attrarre o respingere secondo regole definite dalla disposizione dei magneti, ma non può decidere dinamicamente quando attivare o disattivare l'interazione. Per questo motivo, i magneti permanenti sono molto utili per l'allineamento passivo e per il mantenimento di configurazioni, ma risultano meno adatti quando l'obiettivo diventa la riconfigurazione dinamica controllata.

La scelta del materiale magnetico è un aspetto progettuale importante. I magneti al neodimio, ad esempio, offrono una forza elevata in dimensioni ridotte e sono quindi compatibili con un sistema miniaturizzato. Tuttavia, una forza troppo elevata può diventare un problema: se l'aggancio è troppo forte, i MicroBot potrebbero non riuscire a separarsi facilmente, oppure potrebbero aggregarsi in modo incontrollato. Al contrario, una forza troppo debole potrebbe non essere sufficiente a mantenere una struttura stabile. Il dimensionamento del magnete deve quindi trovare un equilibrio tra stabilità dell'aggancio e possibilità di riconfigurazione.

Questo equilibrio è uno dei punti centrali del sistema magnetico. Il magnete permanente deve essere abbastanza forte da superare piccole perturbazioni, vibrazioni, errori di posizionamento e attrito, ma non così forte da bloccare il sistema in una configurazione rigida. Nel progetto MicroBot, infatti, la forma non deve essere semplicemente fissata una volta per tutte: deve poter evolvere. Di conseguenza, l'aggancio magnetico deve essere interpretato come un vincolo reversibile, non come un'unione permanente.

Dal punto di vista geometrico, la posizione dei magneti all'interno del MicroBot è determinante. Se i magneti sono collocati lungo l'asse principale del nodo, possono favorire la formazione di catene lineari o configurazioni orientate. Se invece sono distribuiti su più facce o superfici, possono permettere aggregazioni più articolate. La configurazione a coni troncati e sfera descritta nella Sezione 4 si collega direttamente a questo problema: le superfici terminali possono ospitare magneti orientati in modo da favorire l'allineamento tra unità, mentre il corpo centrale può contenere elettronica e alimentazione.

Un aspetto importante riguarda l'orientamento dei poli. Due magneti si attraggono se poli opposti sono rivolti l'uno verso l'altro, mentre si respingono se poli uguali si fronteggiano. Nel MicroBot, questa proprietà può essere usata per codificare una logica fisica di aggancio. Ad esempio, una disposizione alternata dei poli può favorire connessioni corrette e scoraggiare orientamenti indesiderati. In questo senso, la polarità dei magneti diventa una forma di informazione incorporata nella materia: il nodo non "calcola" esplicitamente l'orientamento corretto, ma la struttura fisica tende naturalmente verso configurazioni compatibili.

Questo è un punto molto importante dal punto di vista teorico. Nei sistemi MicroBot, non tutta l'intelligenza deve essere nel firmware. Una parte del comportamento può essere delegata alla fisica del sistema. I magneti permanenti introducono infatti una forma di computazione fisica passiva: la disposizione dei poli, la geometria delle superfici e la forza magnetica guidano il sistema verso determinati stati senza richiedere

elaborazione digitale. Questo riduce il carico computazionale e rende il comportamento più naturale, ma limita anche la flessibilità.

Dal punto di vista del comportamento emergente, i magneti permanenti possono generare configurazioni collettive anche in assenza di controllo attivo. Se molte unità sono dotate di magneti compatibili, possono formare cluster, catene o strutture più complesse attraverso semplici interazioni locali. Questo comportamento è coerente con il paradigma swarm: il sistema globale non è costruito pezzo per pezzo da un controllo centralizzato, ma emerge da regole locali e forze fisiche. La differenza è che, in questo caso, la regola locale non è solo software, ma anche materiale.

Un limite operativo dei magneti permanenti riguarda però la gestione degli errori di aggregazione. Se due unità si agganciano in modo non desiderato, il sistema potrebbe non essere in grado di correggere autonomamente l'errore senza un meccanismo attivo di distacco. Questo rende necessario introdurre, nelle versioni successive, bobine elettromagnetiche, vibrazione, geometrie di sgancio o strategie meccaniche che permettano di superare il vincolo passivo. Per questo motivo, i magneti permanenti devono essere considerati come una base, non come soluzione definitiva per tutte le funzioni.

Dal punto di vista energetico, i magneti permanenti hanno un vantaggio enorme: mantengono l'aggancio senza consumare energia. Questo li rende molto utili per stati stabili e configurazioni mantenute nel tempo. Una strategia efficiente potrebbe prevedere l'uso di magneti permanenti per il mantenimento passivo della struttura e l'uso di bobine elettromagnetiche solo per fasi temporanee di aggancio, distacco o riconfigurazione. In questo modello ibrido, il campo passivo fornisce stabilità di base, mentre il campo attivo introduce controllo.

La presenza di magneti permanenti richiede comunque attenzione nell'integrazione elettronica. Campi magnetici intensi, se collocati male, potrebbero influenzare sensori magnetici, bussole digitali, componenti sensibili o eventuali sistemi di misura futuri. Anche se nei prototipi iniziali basati su ESP32, LED e circuiti semplici questo problema può essere limitato, in una versione più avanzata del sistema la disposizione dei magneti dovrà essere studiata con cura per evitare interferenze con sensori e circuiti.

Un altro aspetto riguarda la ripetibilità. In uno swarm, ogni unità deve comportarsi in modo simile alle altre. Se i magneti non sono montati con lo stesso orientamento, la stessa distanza dalla superficie o la stessa forza nominale, i nodi potrebbero avere comportamenti diversi. Questo renderebbe difficile prevedere l'aggregazione e validare i modelli simulativi. Per questo motivo, la documentazione tecnica del MicroBot dovrebbe includere una specifica precisa sulla posizione, sulla polarità e sul tipo di magnete utilizzato.

Dal punto di vista della simulazione, i magneti permanenti possono essere modellati in modo semplificato attraverso una forza attrattiva locale dipendente dalla distanza. In una prima versione, si può usare una relazione qualitativa in cui la forza cresce quando i nodi sono vicini e decresce rapidamente quando si allontanano. In versioni più avanzate, il modello può includere orientamento, polarità, soglie di aggancio e soglie di distacco. Questo permetterebbe di collegare meglio il comportamento del simulatore al comportamento del prototipo fisico.

La presenza di soglie è particolarmente importante. In un sistema reale, l'aggancio non avviene in modo continuo e perfetto, ma spesso presenta una transizione: sotto una certa distanza, l'attrazione diventa sufficiente a unire le unità; sopra una certa distanza, l'interazione diventa trascurabile. Questo concetto si collega direttamente alle future sezioni del modello matematico, dove saranno introdotte soglie di attacco, soglie di distacco e meccanismi di hysteresis. I magneti permanenti forniscono quindi una base fisica per tali concetti.

Un altro elemento da considerare è la sicurezza nella manipolazione. Magnetici piccoli ma potenti possono attirarsi improvvisamente, pizzicare componenti, danneggiare parti fragili o rendere complesso l'assemblaggio. In una fase prototipale, questo aspetto deve essere gestito con attenzione, soprattutto se il sistema viene mostrato a persone esterne o utilizzato in ambienti didattici. Anche qui emerge il principio generale del progetto: ogni scelta fisica deve essere valutata non solo per ciò che permette di fare, ma anche per i vincoli e i rischi che introduce.

Dal punto di vista documentale, i magneti permanenti devono essere descritti come componente di primo livello del sistema magnetico, ma anche come fase di transizione verso configurazioni più controllabili. Essi permettono di validare la logica di aggregazione, studiare il comportamento delle superfici di contatto, testare la stabilità dei cluster e costruire prototipi dimostrativi robusti. Tuttavia, non risolvono da soli il

problema della riconfigurazione dinamica, che richiederà l'integrazione con bobine elettromagnetiche e strategie di controllo attivo.

In conclusione, i magneti permanenti costituiscono il primo strato fisico del sistema magnetico MicroBot. Essi offrono semplicità, efficienza energetica e capacità di aggancio passivo, rendendoli particolarmente utili nelle fasi iniziali di prototipazione e validazione. Allo stesso tempo, introducono vincoli legati alla mancanza di controllo dinamico, alla forza di aggancio, alla polarità e alla possibilità di aggregazioni indesiderate. Il loro ruolo deve quindi essere interpretato in modo preciso: non sono la soluzione definitiva dell'interazione MicroBot, ma il fondamento fisico su cui costruire successivamente allineamento passivo, controllo attivo del campo e riconfigurazione dello swarm.

6.2 Allineamento passivo

L'allineamento passivo rappresenta il secondo livello del sistema magnetico MicroBot e costituisce il meccanismo attraverso cui le unità tendono a disporsi in configurazioni geometricamente e magneticamente coerenti senza richiedere un intervento attivo continuo da parte del microcontrollore. Dopo aver analizzato il ruolo dei magneti permanenti come sorgenti di campo costante, è necessario comprendere in che modo tali magneti possano generare non solo attrazione, ma anche orientamento, stabilizzazione e auto-correzione locale tra unità vicine. In questo senso, l'allineamento passivo non coincide semplicemente con l'aggancio magnetico, ma rappresenta una proprietà più profonda del sistema: la capacità della struttura fisica di guidare spontaneamente i MicroBot verso posizioni compatibili con la configurazione desiderata.

Nel progetto MicroBot, il concetto di allineamento passivo è particolarmente importante perché permette di delegare una parte del comportamento alla fisica del sistema. In un'architettura distribuita, ogni nodo dispone di risorse limitate, non possiede una conoscenza completa dello stato globale e non può calcolare continuamente tutte le configurazioni possibili. Per questo motivo, è vantaggioso progettare la forma, la polarità e la disposizione dei magneti in modo tale che alcune correzioni avvengano automaticamente. Se due MicroBot si avvicinano con un orientamento leggermente errato, l'interazione magnetica può generare una coppia di rotazione che tende ad allinearli. Questo comportamento riduce il carico computazionale e rende il sistema più robusto rispetto a errori di posizionamento.

Dal punto di vista fisico, l'allineamento passivo nasce dal fatto che due dipoli magnetici tendono a disporsi secondo configurazioni di energia minima. Quando i poli opposti sono orientati correttamente, il sistema tende a stabilizzarsi; quando l'orientamento è sfavorevole, si generano forze e momenti che spingono i magneti a ruotare o a respingersi. Questa dinamica può essere interpretata come una forma di auto-organizzazione locale, in cui le unità non devono "sapere" esplicitamente quale sia l'orientamento corretto, perché la configurazione fisica rende alcune disposizioni più probabili e più stabili di altre.

Nel caso dei MicroBot, questa proprietà può essere sfruttata progettando superfici di contatto e polarità in modo coerente. La configurazione a coni troncati e sfera descritta nella Sezione 4 può contribuire a questo processo, perché le superfici terminali possono guidare il contatto tra unità e ridurre l'ambiguità dell'aggancio. Se i magneti sono posizionati lungo l'asse principale del nodo, l'allineamento tende a favorire configurazioni assiali, cioè catene o strutture orientate. Se invece i magneti sono distribuiti su più punti della superficie, il sistema può supportare configurazioni più articolate, ma richiede una progettazione più attenta per evitare agganci non desiderati.

L'allineamento passivo può essere descritto come una funzione che agisce sulla posizione e sull'orientamento relativo dei nodi. Se due MicroBot vengono indicati come n_i e n_j , la loro interazione non dipende soltanto dalla distanza r_{ij} , ma anche dall'orientamento θ_{ij} tra i rispettivi assi magnetici. In forma concettuale, si può scrivere:

$\text{stabilità_connessione} = \text{funzione}(\text{distanza}, \text{polarità}, \text{orientamento}, \text{geometria_contatto})$

Questa relazione mostra che l'aggancio non è determinato da un solo fattore. Due nodi possono essere vicini ma non correttamente allineati; possono avere polarità compatibili ma superfici di contatto sfavorevoli; possono essere geometricamente compatibili ma troppo lontani per generare una forza sufficiente. L'allineamento passivo nasce quindi dall'interazione simultanea tra campo magnetico e geometria del corpo.

Un elemento centrale è la tolleranza agli errori. In un sistema reale, i MicroBot non si avvicinano mai in condizioni perfettamente ideali. Possono esserci piccoli errori di posizione, rotazioni indesiderate, attrito, imperfezioni di stampa, differenze tra magneti, ritardi nella simulazione o disturbi ambientali. Un sistema basato esclusivamente su controllo attivo dovrebbe correggere ciascuno di questi errori con sensori, calcolo

e attuazione. L'allineamento passivo permette invece di assorbire una parte di queste imprecisioni attraverso la fisica stessa del contatto. Questo rende il sistema più semplice, più economico e più affidabile nella fase prototipale.

L'allineamento passivo è strettamente collegato al concetto di vincolo reversibile. Quando due MicroBot si agganciano, il sistema non deve creare una connessione rigida e permanente, ma una relazione stabile finché le condizioni restano favorevoli. La connessione deve mantenere la struttura, ma deve anche poter essere superata da una forza esterna, da una riconfigurazione o da un futuro sistema di distacco attivo. Per questo motivo, l'allineamento passivo deve essere progettato come equilibrio tra stabilità e reversibilità. Se il vincolo è troppo debole, la struttura si rompe facilmente; se è troppo forte, il sistema perde capacità di riconfigurazione.

Dal punto di vista energetico, il vantaggio dell'allineamento passivo è notevole. Una volta che due unità sono agganciate tramite magneti permanenti, il mantenimento della connessione non richiede consumo elettrico continuo. Questo è molto importante nel sistema MicroBot, perché ogni nodo dispone di una batteria limitata e deve riservare energia per comunicazione, gestione dello stato, feedback e eventuali attuazioni. L'allineamento passivo permette quindi di usare la fisica come "memoria strutturale" della configurazione: una volta formata, la struttura rimane stabile senza dover essere continuamente alimentata.

Questa idea si collega direttamente al concetto di materia programmabile. In un materiale tradizionale, la forma è definita dalla struttura fisica permanente. In MicroBot, invece, la forma è il risultato di configurazioni temporanee tra unità. L'allineamento passivo permette a queste configurazioni di esistere fisicamente, trasformando una decisione o una disposizione spaziale in una struttura mantenuta dalle interazioni locali. In questo senso, il sistema magnetico diventa il mezzo attraverso cui la forma collettiva assume stabilità materiale.

Un aspetto importante riguarda la codifica fisica dell'orientamento. Attraverso la disposizione dei poli magnetici, è possibile favorire certi tipi di connessione e scoraggiarne altri. Per esempio, una disposizione con polarità alternata può rendere naturalmente compatibili alcune superfici e incompatibili altre. Questo meccanismo può essere interpretato come una forma di "protocollo fisico" del nodo. Mentre nella comunicazione digitale un protocollo definisce quali messaggi sono validi, nel sistema magnetico la polarità e la geometria definiscono quali agganci sono fisicamente favoriti. Il sistema incorpora quindi una parte della logica direttamente nella materia.

Dal punto di vista dello swarm, l'allineamento passivo contribuisce alla formazione di pattern collettivi. Se ogni nodo segue semplici regole fisiche di attrazione e orientamento, l'insieme delle unità può formare strutture più grandi senza che ogni singola connessione venga controllata esplicitamente dal controller centrale. Questo è coerente con il paradigma emergente sviluppato nelle sezioni precedenti: il comportamento globale deriva da interazioni locali ripetute. L'allineamento passivo rappresenta quindi una componente fisica del comportamento emergente.

Un limite dell'allineamento passivo è la difficoltà di controllo selettivo. Poiché i magneti permanenti sono sempre attivi, il sistema può produrre agganci anche quando non sono desiderati. Questo può causare cluster non pianificati, configurazioni bloccate o difficoltà di separazione. Per questo motivo, l'allineamento passivo deve essere pensato come primo livello del sistema magnetico, non come soluzione completa. Le sezioni successive, dedicate alle bobine elettromagnetiche e al controllo attivo del campo, introdurranno strumenti per superare questo limite, ma senza eliminare il valore del livello passivo.

Un altro limite riguarda la dipendenza dalla distanza. Le forze magnetiche decrescono rapidamente all'aumentare della separazione tra i nodi. L'allineamento passivo è quindi efficace solo entro un raggio relativamente limitato. Questo significa che i MicroBot devono comunque essere portati entro una certa distanza critica affinché la fisica magnetica possa completare l'aggancio. In una versione iniziale, questa distanza può essere raggiunta manualmente o tramite disposizione controllata; in versioni successive, potrebbe essere ottenuta attraverso movimento, vibrazione, superfici guidate o altre forme di attuazione.

La qualità dell'allineamento dipende anche dall'attrito tra superfici. Se l'attrito è troppo elevato, i magneti potrebbero non riuscire a ruotare o correggere l'orientamento delle unità. Se l'attrito è troppo basso, invece, la connessione potrebbe risultare instabile o soggetta a scivolamenti. Questo dimostra che il sistema magnetico non può essere progettato isolatamente: deve essere considerato insieme alla geometria esterna, ai materiali, alla rugosità delle superfici e al peso del nodo. L'allineamento passivo è quindi un fenomeno magneto-meccanico, non puramente magnetico.

Dal punto di vista della simulazione, l'allineamento passivo può essere modellato introducendo una forza attrattiva e una coppia di orientamento. Nei modelli più semplici, è possibile rappresentarlo attraverso una soglia di aggancio: se due nodi sono abbastanza vicini e l'orientamento è compatibile, essi vengono considerati connessi. Nei modelli più avanzati, si può simulare un progressivo riallineamento, in cui l'angolo relativo tra i nodi tende verso una configurazione stabile. Questa distinzione è importante perché permette di far evolvere il simulatore insieme al prototipo fisico.

Un possibile modello concettuale può essere espresso così:

se distanza < soglia_attacco e orientamento compatibile → aggancio stabile
se distanza < soglia_attacco ma orientamento non compatibile → coppia di riallineamento
se distanza > soglia_distacco → connessione persa

Questa logica introduce già i concetti che saranno approfonditi nella Sezione 7, in particolare soglie di attacco, soglie di distacco e hysteresis. L'allineamento passivo diventa quindi un ponte tra sistema magnetico e modello matematico: da un lato è una proprietà fisica; dall'altro può essere trasformato in regole computazionali per la simulazione e il controllo.

Nel contesto del progetto reale, i prototipi LED e le simulazioni software possono rappresentare l'allineamento passivo in modo simbolico prima ancora che esso venga realizzato fisicamente. Per esempio, due nodi virtuali possono cambiare colore quando entrano nella zona di aggancio, oppure una dashboard può mostrare una connessione quando la distanza e l'orientamento soddisfano certe condizioni. Questa rappresentazione permette di testare la logica del sistema prima dell'implementazione magnetica completa, mantenendo coerenza con il metodo incrementale del progetto.

L'allineamento passivo ha anche un valore dimostrativo importante. In una presentazione del progetto, mostrare che due unità tendono spontaneamente ad agganciarsi in modo corretto rende immediatamente comprensibile il concetto di sistema emergente. L'utente o l'osservatore vede che la struttura non viene forzata manualmente in ogni dettaglio, ma nasce dall'interazione tra componenti progettati per cooperare. Questo rafforza la narrazione del progetto MicroBot come materia programmabile e non come semplice insieme di circuiti.

Dal punto di vista della documentazione tecnica, è importante distinguere chiaramente tra allineamento passivo e controllo attivo. L'allineamento passivo riguarda ciò che il sistema fa grazie alla geometria e ai magneti permanenti; il controllo attivo riguarda ciò che il sistema può modificare tramite corrente e software. Questa distinzione evita confusione e rende più chiara l'evoluzione del progetto: prima viene progettata una base fisica naturalmente stabile, poi viene aggiunto un livello di controllo elettronico per aumentare la flessibilità.

Un ulteriore aspetto riguarda la riproducibilità. Se l'allineamento passivo deve funzionare su più unità, la posizione dei magneti, l'orientamento dei poli e la geometria delle superfici devono essere standardizzati. Piccole differenze tra nodi possono generare comportamenti non uniformi, rendendo difficile la formazione di strutture affidabili. Per questo motivo, in una futura fase di produzione o prototipazione avanzata, sarà necessario definire tolleranze geometriche e procedure di assemblaggio precise.

L'allineamento passivo può inoltre contribuire alla sicurezza del sistema. Poiché non richiede energia continua, riduce il rischio di surriscaldamento rispetto a un sistema basato solo su bobine attive. Inoltre, in caso di perdita di alimentazione, i nodi possono mantenere una parte della struttura grazie ai magneti permanenti. Questo comportamento può essere utile in modalità fallback: se il sistema elettronico si spegne, la struttura non collassa immediatamente, ma resta parzialmente vincolata dalle interazioni passive.

Tuttavia, questa stessa proprietà può diventare un limite se il sistema deve disassemblarsi rapidamente in caso di emergenza. Per questo motivo, nelle versioni avanzate sarà necessario valutare strategie di distacco controllato o geometrie che consentano separazione meccanica con forza limitata. Anche qui emerge il principio generale del progetto: ogni proprietà utile introduce anche un vincolo, e il compito dell'architettura è bilanciare questi aspetti.

In conclusione, l'allineamento passivo costituisce un livello fondamentale del sistema magnetico MicroBot. Esso permette ai nodi di orientarsi, agganciarsi e stabilizzarsi attraverso la combinazione di magneti permanenti, geometria e superfici di contatto, senza richiedere consumo energetico continuo o controllo software diretto. La sua funzione è ridurre la complessità computazionale, aumentare la robustezza locale e

fornire una base fisica alla formazione di strutture emergenti. Allo stesso tempo, esso presenta limiti legati alla mancanza di controllo dinamico, alla dipendenza dalla distanza, alla polarità e alla possibilità di agganci indesiderati. Per questo motivo, l'allineamento passivo deve essere considerato come il primo strato fisico dell'interazione MicroBot: una base stabile e intelligente nella materia, sopra la quale potranno essere costruiti i livelli successivi di controllo attivo, riconfigurazione e comportamento collettivo.

6.3 Bobine elettromagnetiche

Le bobine elettromagnetiche rappresentano il secondo grande livello del sistema magnetico MicroBot e costituiscono il passaggio dalla pura interazione passiva alla possibilità di generare campi magnetici controllabili. Dopo aver analizzato i magneti permanenti e l'allineamento passivo, diventa necessario introdurre un componente capace di modificare il proprio comportamento magnetico nel tempo, in funzione di una corrente elettrica e, quindi, in funzione di una logica di controllo. La bobina elettromagnetica è precisamente questo elemento: un avvolgimento conduttivo che, se attraversato da corrente, genera un campo magnetico orientato e modulabile.

Nel contesto del progetto MicroBot, la bobina non deve essere considerata soltanto come un componente elettronico, ma come un dispositivo di conversione fisica tra energia elettrica e interazione magnetica. Essa riceve energia dal sistema di alimentazione, viene controllata indirettamente dal microcontrollore attraverso il circuito di potenza descritto nella Sezione 5, e produce un campo magnetico capace di influenzare altri nodi. In questo senso, la bobina è uno dei punti in cui il progetto MicroBot collega in modo diretto software, elettronica e materia. Il comando software non rimane informazione astratta, ma viene trasformato in corrente; la corrente genera campo; il campo produce forza; la forza modifica la configurazione dello swarm.

Dal punto di vista fisico, una bobina può essere interpretata come un insieme di spire percorse da corrente. Ogni spira contribuisce alla generazione del campo magnetico, e l'effetto complessivo dipende dal numero di spire, dalla corrente, dalla geometria dell'avvolgimento e dal materiale eventualmente presente al suo interno. In una forma ideale e semplificata, il campo magnetico generato da una bobina solenoidale può essere espresso come:

$$B = \mu_0 \cdot \mu_r \cdot (N / L) \cdot I$$

dove B rappresenta il campo magnetico, μ_0 la permeabilità magnetica del vuoto, μ_r la permeabilità relativa del materiale interno, N il numero di spire, L la lunghezza della bobina e I la corrente che la attraversa. Questa relazione non descrive perfettamente una bobina reale miniaturizzata, ma fornisce una base concettuale importante: il campo cresce aumentando il numero di spire, aumentando la corrente o utilizzando materiali capaci di concentrare meglio il flusso magnetico.

Nel MicroBot, tuttavia, la bobina non può essere progettata come un solenoide ideale di grandi dimensioni. Il nodo ha vincoli spaziali molto stretti, deve contenere microcontrollore, batteria, sistemi di comunicazione, eventuali LED, protezioni e struttura fisica. La bobina deve quindi essere compatta, leggera e integrabile nella geometria del robot. Questo rende il dimensionamento più complesso, perché non basta massimizzare il campo magnetico: bisogna bilanciare campo prodotto, volume occupato, consumo energetico, temperatura, massa e compatibilità con gli altri componenti.

Un primo parametro fondamentale è il numero di spire. Aumentare il numero di spire consente, a parità di corrente, di aumentare il campo magnetico prodotto. Tuttavia, più spire significano anche maggiore lunghezza del filo, quindi maggiore resistenza elettrica. Una resistenza più alta riduce la corrente disponibile a parità di tensione e aumenta la dissipazione termica. Di conseguenza, il numero di spire non può essere scelto in modo arbitrario: deve essere determinato in relazione alla tensione della batteria, alla corrente massima ammessa, alla dimensione del filo, al volume disponibile e al duty cycle di utilizzo.

Un secondo parametro fondamentale è il diametro del filo. Un filo più sottile permette di inserire più spire nello stesso volume, ma presenta una resistenza maggiore e può sopportare meno corrente senza surriscaldarsi. Un filo più spesso riduce la resistenza e può trasportare più corrente, ma occupa più spazio e limita il numero di spire. Questa scelta è particolarmente delicata nel MicroBot, perché lo spazio interno è ridotto e ogni aumento di volume sottrae spazio ad altri componenti. La bobina deve quindi essere dimensionata come parte dell'intero nodo, non come componente isolato.

Un terzo parametro è la geometria dell'avvolgimento. La bobina può essere più lunga e stretta, oppure più corta e larga, e queste configurazioni producono distribuzioni di campo differenti. Nel caso del MicroBot, la geometria deve essere coerente con la forma esterna del nodo e con le superfici di interazione. Se la bobina

è collocata vicino a una delle estremità del MicroBot, può generare un campo orientato lungo l'asse principale, utile per attrarre o respingere un altro nodo in direzione controllata. Se invece la bobina è collocata in posizione centrale, il campo potrebbe risultare meno focalizzato rispetto al punto di contatto.

La posizione della bobina all'interno del MicroBot è quindi un aspetto progettuale essenziale. Una bobina troppo interna potrebbe generare un campo debole sulla superficie esterna, perché la distanza dal punto di interazione sarebbe maggiore. Una bobina troppo vicina alla superficie potrebbe invece interferire con la struttura meccanica, con i magneti permanenti o con la distribuzione dei volumi interni. La scelta più coerente con la configurazione a coni troncati e sfera è collocare le bobine nelle zone terminali o in prossimità delle superfici di aggancio, lasciando la parte centrale per batteria, microcontrollore e cablaggio.

Un elemento importante riguarda la possibile presenza di un nucleo ferromagnetico. Inserire un materiale ad alta permeabilità all'interno della bobina può concentrare il flusso magnetico e aumentare il campo prodotto a parità di corrente. Questo può rendere la bobina più efficace e ridurre la corrente necessaria per ottenere una certa interazione. Tuttavia, il nucleo introduce anche massa, volume, possibili fenomeni di saturazione magnetica e vincoli costruttivi. Nel MicroBot, l'uso di nuclei ferromagnetici deve quindi essere valutato con attenzione, perché può migliorare la forza magnetica ma peggiorare peso, ingombro e risposta dinamica.

La bobina introduce inoltre una proprietà che i magneti permanenti non possiedono: la possibilità di essere disattivata. Questo aspetto è fondamentale. Un magnete permanente mantiene sempre il proprio campo; una bobina, invece, genera campo solo quando attraversata da corrente. Ciò significa che il sistema può decidere quando produrre interazione magnetica e quando ridurla. Anche se il controllo attivo verrà approfondito nel punto successivo dell'indice, è già importante riconoscere che la bobina cambia la natura stessa del sistema magnetico: da struttura passiva a struttura condizionata dallo stato elettrico del nodo.

Dal punto di vista dello stato del MicroBot, una bobina può essere rappresentata come una variabile fisica controllabile. Lo stato magnetico del nodo non dipende più soltanto dalla presenza di magneti permanenti, ma anche da parametri dinamici come corrente, polarità, durata di attivazione e temperatura. Si può quindi descrivere lo stato magnetico locale come:

$s_{\text{magnetic}} = (B_{\text{statico}}, B_{\text{attivo}}, I_{\text{coil}}, \text{polarità}, \text{tempo_attivazione}, \text{temperatura})$

dove B_{statico} rappresenta l'eventuale contributo dei magneti permanenti, B_{attivo} il campo generato dalla bobina, I_{coil} la corrente nella bobina, la polarità indica la direzione del campo, il tempo di attivazione descrive la durata del comando e la temperatura rappresenta un vincolo fisico di sicurezza. Questa rappresentazione mostra che la bobina trasforma il magnetismo in una parte esplicita dello stato dinamico del nodo.

Un aspetto centrale è la risposta temporale della bobina. Una bobina non passa istantaneamente da campo nullo a campo massimo. A causa dell'induttanza, la corrente cresce progressivamente secondo una dinamica determinata dal circuito. In un modello semplificato, il tempo caratteristico della risposta è legato alla costante:

$$\tau = L / R$$

dove L è l'induttanza della bobina e R la resistenza del circuito. Questo significa che la bobina possiede una memoria fisica breve: il campo non compare e non scompare in modo perfettamente istantaneo, ma evolve nel tempo. Questa proprietà deve essere considerata sia nel firmware sia nella simulazione, perché influisce sulla precisione dei comandi magnetici e sulla sincronizzazione tra nodi.

La dissipazione termica rappresenta uno dei limiti più importanti delle bobine elettromagnetiche. Quando la corrente attraversa il filo, parte dell'energia viene dissipata sotto forma di calore secondo la relazione:

$$P = R \cdot I^2$$

Questa formula è molto importante perché mostra che la potenza dissipata cresce con il quadrato della corrente. Se la corrente raddoppia, la potenza dissipata quadruplica. In un MicroBot miniaturizzato, questo vincolo è critico: una bobina troppo energizzata può riscaldarsi rapidamente, danneggiare componenti vicini, ridurre l'efficienza energetica o attivare condizioni di sicurezza. Per questo motivo, l'uso delle bobine deve essere progettato con limiti temporali, duty cycle controllato e monitoraggio dello stato operativo.

Dal punto di vista energetico, la bobina è uno dei componenti più esigenti del nodo. Mentre il microcontrollore e il LED possono operare con consumi relativamente contenuti, una bobina richiede corrente significativa per generare un campo utile. Questo significa che le bobine non dovrebbero essere mantenute attive continuamente, ma utilizzate in modo selettivo, per fasi specifiche come aggancio, correzione, distacco o rinforzo temporaneo dell'interazione. In questo senso, una strategia efficace potrebbe combinare magneti permanenti per il mantenimento passivo e bobine per l'intervento dinamico.

Questa combinazione tra magneti permanenti e bobine è una delle direzioni più interessanti del sistema MicroBot. I magneti permanenti forniscono una base stabile e senza consumo energetico; le bobine aggiungono controllo, temporizzazione e possibilità di modulazione. Il sistema magnetico non deve quindi essere pensato come scelta esclusiva tra passivo e attivo, ma come architettura ibrida. Questa architettura rispecchia anche il modello generale del progetto, che combina controller centrale e nodi distribuiti, simulazione e hardware, comportamento emergente e controllo guidato.

Le bobine permettono inoltre di introdurre polarità variabile, se il circuito di controllo lo consente. In una configurazione semplice con MOSFET low-side, la bobina viene accesa e spenta sempre con la stessa polarità. In una configurazione più avanzata, ad esempio con ponte H o circuiti equivalenti, la direzione della corrente può essere invertita, modificando la polarità del campo. Questo permetterebbe non solo attrazione, ma anche repulsione controllata, aprendo la strada a comportamenti molto più complessi. Tuttavia, questa possibilità aumenta la complessità elettronica e deve essere valutata come fase successiva, non come requisito iniziale.

Nel contesto dei prototipi iniziali, è metodologicamente corretto rappresentare la bobina attraverso componenti più semplici, come LED o simulazioni software. Un LED controllato in PWM può rappresentare visivamente l'intensità di una bobina virtuale; un nodo simulato può trasformare il duty cycle in una forza attrattiva virtuale. Questa strategia permette di validare prima la logica del controllo e solo successivamente introdurre il componente elettromagnetico reale. È una progressione rigorosa: prima si verifica il comportamento informazionale, poi il comportamento elettrico, infine il comportamento fisico-magnetico.

Dal punto di vista della simulazione, la bobina può essere modellata come una sorgente di campo dipendente da un parametro di controllo. In una prima approssimazione, si può associare l'intensità del campo al valore del comando:

$$B(t) = k \cdot u(t)$$

dove $u(t)$ rappresenta il comando normalizzato, ad esempio un duty cycle compreso tra 0 e 1, e k rappresenta un coefficiente di scala. In modelli più avanzati, si può includere la risposta temporale, la saturazione, la distanza e l'orientamento. Questo permette di costruire una continuità tra simulazione e hardware reale, evitando che il modello virtuale diventi troppo distante dal comportamento fisico.

Le bobine hanno anche un ruolo importante nella futura scalabilità dello swarm. Se ogni nodo possiede bobine controllabili, il sistema può teoricamente modificare le proprie connessioni in modo distribuito. Tuttavia, aumentando il numero di nodi aumentano anche i consumi, i disturbi elettromagnetici, la complessità di sincronizzazione e la necessità di sicurezza. Questo significa che il controllo delle bobine deve essere progettato fin dall'inizio con una logica scalabile, evitando soluzioni che funzionano solo per un singolo prototipo ma diventano impraticabili quando i nodi diventano molti.

Dal punto di vista dell'interferenza, le bobine possono generare disturbi elettrici e magnetici. Le variazioni rapide di corrente possono introdurre rumore nel circuito, influenzare linee di alimentazione, disturbare sensori o interferire con altri componenti. Per questo motivo, la progettazione delle bobine deve essere coordinata con protezioni, layout, filtri e grounding. La bobina non è soltanto una sorgente di campo utile: è anche una possibile sorgente di rumore. La differenza tra un prototipo instabile e un sistema robusto dipende dalla capacità di gestire entrambi gli aspetti.

Un altro punto importante riguarda la ripetibilità costruttiva. Se le bobine vengono avvolte manualmente, piccole differenze nel numero di spire, nella tensione del filo, nella posizione o nella geometria possono produrre campi diversi tra un nodo e l'altro. In uno swarm, questa variabilità può rendere difficile ottenere comportamenti uniformi. Per questo motivo, una versione più matura del progetto dovrebbe prevedere specifiche precise per la bobina: diametro del filo, numero di spire, diametro interno, lunghezza, resistenza misurata, posizione nel nodo e modalità di fissaggio. Questa documentazione è necessaria per passare da prototipo sperimentale a sistema riproducibile.

Dal punto di vista meccanico, la bobina deve essere fissata in modo stabile. Vibrazioni, urti o forze magnetiche potrebbero spostarla leggermente, modificando l'orientamento del campo e riducendo la coerenza del sistema. La struttura interna del MicroBot deve quindi prevedere un alloggiamento dedicato, capace di mantenere la bobina nella posizione corretta e di proteggerla da deformazioni. Questo collega la bobina alla geometria del nodo e alla distribuzione dei volumi interni.

Un ulteriore aspetto riguarda la relazione tra bobina e superficie di contatto. Il campo magnetico utile deve essere generato nella zona in cui il MicroBot interagisce con gli altri nodi. Se la bobina è orientata male, se il campo si disperde o se la distanza dalla superficie è troppo grande, l'effetto sull'altro nodo può essere debole. Per questo motivo, la bobina deve essere progettata non solo come componente elettrico, ma come parte di un circuito magnetico orientato verso l'interazione. Questo richiede coerenza tra posizione della bobina, direzione del campo e geometria esterna.

Dal punto di vista teorico, le bobine elettromagnetiche introducono una dimensione fondamentale nel modello MicroBot: la possibilità di modificare localmente le condizioni fisiche del sistema attraverso una variabile controllata. Questo è molto vicino al concetto di materia programmabile. La materia non viene modificata in modo diretto, ma le interazioni tra le sue unità vengono regolate attraverso campi controllabili. Il risultato è una forma di controllo indiretto della struttura: non si impone manualmente la forma finale, ma si modificano localmente le forze che rendono alcune configurazioni più probabili di altre.

Questa idea è centrale perché collega il sistema magnetico al modello emergente del progetto. Il controller centrale o il firmware locale non devono necessariamente specificare ogni singolo movimento; possono invece modificare parametri locali, come intensità e durata del campo, lasciando che le dinamiche fisiche producano la configurazione. In questo senso, la bobina non è un motore classico, ma un regolatore di interazione. Essa non sposta direttamente il nodo lungo una traiettoria obbligata, ma cambia il paesaggio delle forze all'interno dello swarm.

In conclusione, le bobine elettromagnetiche rappresentano un elemento fondamentale nell'evoluzione del sistema magnetico MicroBot. Esse permettono di passare da un'interazione passiva, basata su magneti permanenti, a un'interazione potenzialmente controllabile e modulabile. La loro progettazione richiede attenzione a geometria, numero di spire, corrente, resistenza, dissipazione termica, risposta temporale, posizione interna, protezione elettrica e compatibilità con la simulazione. Le bobine non sostituiscono necessariamente i magneti permanenti, ma li completano, creando una possibile architettura magnetica ibrida in cui stabilità passiva e controllo attivo lavorano insieme. Questo punto prepara direttamente il passaggio successivo dell'indice: il controllo attivo del campo, in cui verrà analizzato come il sistema possa modulare e coordinare nel tempo il campo magnetico prodotto dalle bobine.

6.4 Controllo attivo del campo

Il controllo attivo del campo rappresenta il passaggio concettuale e tecnico attraverso cui il sistema magnetico del MicroBot diventa programmabile. Dopo aver definito il ruolo dei magneti permanenti, dell'allineamento passivo e delle bobine elettromagnetiche, è ora necessario analizzare il modo in cui il campo magnetico può essere generato, modulato, temporizzato e integrato nella logica complessiva del nodo. In questa fase il magnetismo non è più soltanto una proprietà fisica della struttura, ma diventa una variabile controllabile del sistema, direttamente collegata al firmware, all'elettronica di potenza, alla sincronizzazione e alla strategia globale dello swarm.

Nel sistema MicroBot, il controllo attivo del campo può essere interpretato come la capacità del nodo di modificare localmente le proprie condizioni di interazione. Un MicroBot dotato soltanto di magneti permanenti possiede un comportamento magnetico definito in fase di costruzione: la forza, la polarità e la disposizione dei poli sono fissate dalla struttura fisica. Un MicroBot dotato di bobine controllabili, invece, può variare il proprio stato magnetico nel tempo. Questa differenza è fondamentale perché introduce una separazione tra geometria fisica e comportamento magnetico: la forma del nodo resta la stessa, ma le sue interazioni possono cambiare in base allo stato del sistema.

Dal punto di vista funzionale, il controllo attivo del campo si basa sulla possibilità di regolare la corrente che attraversa la bobina. Poiché il campo magnetico generato da una bobina dipende dalla corrente, controllare la corrente significa controllare l'intensità del campo. In forma semplificata, questa relazione può essere rappresentata come:

$$B(t) = k \cdot I(t)$$

dove $B(t)$ rappresenta il campo magnetico prodotto in un certo istante, $I(t)$ rappresenta la corrente nella bobina e k è un coefficiente che dipende dalla geometria della bobina, dal numero di spire, dal materiale interno e dalla disposizione fisica del sistema. Questa formulazione è volutamente semplificata, ma permette di evidenziare il principio fondamentale: il campo magnetico diventa una grandezza comandabile.

Nel sistema reale, il microcontrollore non controlla direttamente una corrente analogica continua, ma genera un segnale logico o PWM che pilota il MOSFET, il quale a sua volta regola il passaggio di corrente nella bobina. La catena può essere descritta così:

firmware → segnale PWM → MOSFET → corrente nella bobina → campo magnetico → interazione fisica

Questa catena mostra che il controllo attivo del campo è un processo multilivello. Non dipende soltanto dal software, né soltanto dall'elettronica, né soltanto dalla fisica della bobina: nasce dalla coordinazione di tutti questi livelli. Un errore in uno di essi compromette l'intero comportamento. Se il firmware genera un comando non sicuro, la bobina può surriscaldarsi; se il MOSFET è dimensionato male, il controllo diventa inefficiente; se la bobina è progettata male, il campo generato può essere insufficiente; se la protezione elettrica è assente, lo spegnimento può danneggiare il circuito.

Il controllo PWM è una delle tecniche più importanti per rendere modulabile il campo. Attraverso il duty cycle, cioè la percentuale del periodo in cui il segnale resta attivo, il microcontrollore può controllare la potenza media applicata alla bobina. Un duty cycle basso produce un campo medio più debole, mentre un duty cycle alto produce un campo medio più intenso. In termini concettuali, si può descrivere il comando come un valore normalizzato:

$$u(t) \in [0, 1]$$

dove 0 indica bobina completamente disattivata e 1 indica attivazione massima consentita. Il campo può allora essere modellato, in prima approssimazione, come:

$$B(t) = B_{\text{max}} \cdot u(t)$$

Questa relazione permette di integrare il campo magnetico nel modello di controllo del nodo. Il firmware non deve ragionare necessariamente in ampere o tesla, ma può operare su livelli normalizzati di intensità, più facili da gestire a livello software. Il livello elettronico e fisico si occupa poi di tradurre quel comando in corrente reale e campo effettivo. Questa separazione è coerente con la struttura modulare del progetto: il controller decide il comportamento, il firmware locale applica il comando, l'elettronica pilota il carico e la bobina produce il campo.

Un aspetto fondamentale riguarda però il limite tra comando teorico e comportamento reale. La relazione tra PWM e campo magnetico non è sempre perfettamente lineare. La bobina ha una resistenza, una induttanza, una risposta temporale e una dissipazione termica. Inoltre, la corrente non segue istantaneamente il segnale PWM, ma viene filtrata dalla dinamica elettrica del circuito. Se la frequenza PWM è molto bassa, il campo può oscillare in modo evidente; se è sufficientemente alta, il sistema può comportarsi come se generasse un campo medio più stabile. La scelta della frequenza PWM diventa quindi un parametro progettuale importante, perché influenza rumorosità elettrica, risposta della bobina, efficienza e qualità del controllo.

Il controllo attivo del campo richiede inoltre una gestione precisa dei tempi di attivazione. Una bobina non dovrebbe restare attiva indefinitamente, perché assorbe corrente e genera calore. Per questo motivo, ogni comando magnetico dovrebbe essere associato a una durata massima o a un ciclo operativo definito. Invece di inviare un comando generico del tipo "attiva bobina", è più corretto definire comandi completi, ad esempio:

attiva bobina A al 40% per 200 ms
attiva bobina B al 70% per 100 ms
disattiva tutte le bobine
applica impulso magnetico breve
mantieni campo minimo per aggancio stabile

Questo tipo di comando è più sicuro e più controllabile, perché impedisce stati indefiniti. In un sistema distribuito, dove possono verificarsi perdita di pacchetti, ritardi o errori di sincronizzazione, un comando

senza durata potrebbe lasciare il nodo in uno stato pericoloso. Un comando temporizzato, invece, consente al firmware di tornare automaticamente a uno stato sicuro anche se il controller smette di comunicare.

Dal punto di vista dello stato interno, il campo controllato può essere integrato nello stato magnetico del nodo. Una rappresentazione più completa può essere:

$s_magnetic(t) = (coil_id, enabled, duty_cycle, polarity, activation_timer, thermal_limit, safety_lock)$

Questa struttura permette al firmware di sapere quale bobina è attiva, con quale intensità, con quale polarità, da quanto tempo, entro quali limiti termici e con quali vincoli di sicurezza. In questo modo il campo magnetico non è trattato come un effetto secondario, ma come una parte esplicita dello stato del MicroBot. Questa formalizzazione è importante perché collega direttamente il sistema magnetico al modello generale dello stato, già sviluppato nelle sezioni precedenti del documento.

Un altro aspetto decisivo riguarda la polarità. In una configurazione elettronica semplice, la corrente attraversa la bobina sempre nello stesso verso, quindi il campo generato ha una polarità fissa quando la bobina è attiva. In una configurazione più avanzata, usando un ponte H o un circuito equivalente, è possibile invertire la direzione della corrente e quindi invertire la polarità del campo. Questa possibilità introduce un controllo molto più ricco: il nodo può scegliere se attrarre o respingere, rafforzare un aggancio, indebolire una connessione o correggere un orientamento. Tuttavia, l'inversione di polarità aumenta la complessità circuitale e deve essere considerata una fase avanzata del progetto, non necessariamente il requisito della prima versione fisica.

Il controllo attivo del campo può assumere diverse modalità operative. Una prima modalità è l'attivazione impulsiva, in cui la bobina viene accesa per un breve intervallo per generare una forza temporanea. Questa modalità è utile per attrarre un nodo vicino, correggere un orientamento o favorire un aggancio. Una seconda modalità è il mantenimento controllato, in cui il campo viene mantenuto a bassa intensità per stabilizzare una connessione. Una terza modalità è il rilascio o distacco, in cui il campo viene ridotto, annullato o invertito per favorire la separazione. Una quarta modalità è la modulazione dinamica, in cui il campo varia nel tempo secondo una sequenza, producendo comportamenti più complessi.

Queste modalità devono essere progettate in modo coerente con il livello passivo. In una strategia ibrida, i magneti permanenti possono garantire l'allineamento e il mantenimento di base, mentre le bobine intervengono solo quando serve modificare temporaneamente l'interazione. Questo modello è molto più efficiente rispetto a un sistema basato esclusivamente su bobine sempre attive. Il campo permanente fornisce stabilità senza consumo; il campo attivo fornisce controllo quando necessario. La combinazione dei due livelli permette di ottenere una materia programmabile più realistica, perché bilancia energia, semplicità e flessibilità.

Un punto importante riguarda la sincronizzazione tra nodi. Il controllo attivo del campo diventa molto più efficace se le attivazioni sono coordinate nel tempo. Se due MicroBot devono agganciarsi, può essere utile che entrambi attivino le bobine in una finestra temporale comune. Se uno deve rilasciare e l'altro deve attrarre, le sequenze devono essere ordinate. Se più nodi devono formare una struttura, l'attivazione simultanea o sequenziale dei campi può guidare la configurazione. Questo collega direttamente la Sezione 6 alla Sezione 9, dedicata alla sincronizzazione. Il campo magnetico non è solo locale: quando viene coordinato tra più nodi, diventa un meccanismo collettivo.

Dal punto di vista del controller centrale, il controllo attivo del campo può essere gestito a due livelli. Il primo livello è basso, in cui il controller invia comandi diretti alle bobine dei singoli nodi. Il secondo livello è alto, in cui il controller invia un obiettivo, per esempio "forma una linea" o "aggancia nodo 3 a nodo 4", e il firmware locale traduce quell'obiettivo in attivazioni specifiche. La seconda modalità è più scalabile, perché evita di sovraccaricare il controller con dettagli troppo fini. Essa è anche più coerente con l'architettura ibrida del progetto, in cui il controller orienta il comportamento ma i nodi mantengono capacità locali.

Dal punto di vista della simulazione, il controllo attivo del campo può essere modellato come una forza variabile nel tempo. Un nodo virtuale può avere un parametro $magnetic_power$ che modifica la forza attrattiva verso altri nodi. In una prima versione, si può usare un modello semplice:

$$F_attiva = k \cdot u(t) / r^2$$

oppure, in forma più coerente con interazioni locali molto rapide:

$$F_{attiva} = k \cdot u(t) \cdot \text{funzione_decadimento}(r)$$

dove r è la distanza tra i nodi e $u(t)$ è il comando magnetico. In versioni successive, il modello può includere orientamento, polarità, saturazione, ritardo di risposta e limiti termici. L'importante è che la simulazione mantenga la stessa logica del sistema reale: il campo non è fisso, ma dipende da uno stato controllato.

Il controllo attivo del campo introduce anche un problema di stabilità. Se le forze vengono attivate in modo troppo brusco o troppo intenso, i nodi potrebbero muoversi in modo non desiderato, urtare, agganciarsi in configurazioni errate o destabilizzare la struttura. Per questo motivo, il firmware dovrebbe prevedere rampe di attivazione, limiti di intensità e sequenze progressive. Invece di passare istantaneamente da 0 a 100%, può essere preferibile aumentare gradualmente il duty cycle, osservare la risposta e correggere. Questo approccio è coerente con i sistemi di controllo reali, in cui l'attuazione brusca spesso produce instabilità.

Un altro elemento importante è il feedback. Idealmente, il controllo attivo dovrebbe essere collegato a informazioni sul risultato dell'azione. Se il sistema attiva una bobina per agganciare un nodo, dovrebbe sapere se l'aggancio è riuscito. Questa informazione potrebbe provenire da sensori di contatto, variazioni di corrente, sensori di prossimità, telemetria esterna o simulazione. Nei prototipi iniziali, il feedback può essere manuale o visivo; nelle versioni successive, può diventare automatico. Senza feedback, il controllo attivo resta open-loop; con feedback, può diventare closed-loop, cioè capace di correggere il comportamento in base al risultato ottenuto.

Questa distinzione è fondamentale. Un controllo open-loop invia un comando e assume che l'effetto sia avvenuto. Un controllo closed-loop misura o stima l'effetto e modifica il comando se necessario. Nel progetto MicroBot, una prima versione può ragionevolmente partire da controllo open-loop, soprattutto per semplicità. Tuttavia, la visione avanzata del sistema dovrebbe puntare progressivamente verso feedback e correzione, perché uno swarm reale richiede capacità di adattamento.

Il controllo attivo del campo deve essere integrato anche con il safety layer. Una bobina non deve essere attivata se la batteria è troppo bassa, se la temperatura è alta, se il MOSFET è in errore, se il comando è fuori limite o se il nodo si trova in uno stato non sicuro. Questo significa che il comando magnetico deve passare attraverso una funzione di validazione locale. Formalmente:

$$\text{comando_eseguito} = \text{validazione}(\text{comando_ricevuto}, \text{stato_energia}, \text{stato_termico}, \text{stato_sicurezza})$$

Questa relazione evidenzia che il nodo non deve essere un esecutore cieco. Anche se il controller invia un comando, il MicroBot deve poterlo limitare o rifiutare quando le condizioni locali non sono sicure. Questo è essenziale per evitare che un errore software globale danneggi i nodi fisici.

Dal punto di vista energetico, il controllo attivo deve minimizzare il tempo in cui la bobina resta alimentata. Una strategia efficiente consiste nell'utilizzare impulsi brevi e mirati, seguiti da mantenimento passivo tramite magneti permanenti. Un'altra strategia consiste nel coordinare le attivazioni per evitare che molte bobine siano accese contemporaneamente, riducendo i picchi di consumo nello swarm. In una versione avanzata, il controller potrebbe distribuire il carico magnetico nel tempo, attivando gruppi di nodi in sequenza anziché tutti insieme.

Il controllo attivo del campo ha anche implicazioni sulla documentazione e sul versionamento del sistema. Nelle prime versioni, il comando magnetico può essere rappresentato da un LED o da un parametro simulato. In una versione intermedia, può controllare un piccolo carico o una bobina di test. In una versione avanzata, può pilotare bobine integrate nel MicroBot. Questa progressione deve essere documentata, perché mostra la maturazione del sistema dal livello simbolico al livello fisico. Non è una debolezza partire dai LED: è una scelta corretta di sviluppo incrementale.

Un aspetto particolarmente interessante riguarda il legame tra campo controllato e forma collettiva. Se le interazioni magnetiche determinano quali nodi si agganciano, allora controllare il campo significa influenzare la forma dello swarm. La forma del sistema può essere vista come funzione dello stato magnetico dei nodi:

$$\text{Forma}(t) = F(s_{\text{magnetic}_1}(t), s_{\text{magnetic}_2}(t), \dots, s_{\text{magnetic}_n}(t))$$

Questa formulazione collega direttamente il sistema magnetico al concetto di forma come evento dinamico, sviluppato nel livello teorico avanzato dell'indice. La forma non è statica: è il risultato temporaneo di campi, stati, connessioni, vincoli e decisioni distribuite.

Dal punto di vista pratico, il controllo attivo può essere usato per implementare diversi comportamenti. Un primo comportamento è l'aggancio assistito, in cui la bobina aumenta temporaneamente l'attrazione quando due nodi sono vicini. Un secondo comportamento è il distacco assistito, in cui il campo viene invertito o ridotto per facilitare la separazione. Un terzo comportamento è il riallineamento, in cui impulsi magnetici correggono l'orientamento. Un quarto comportamento è la stabilizzazione temporanea, in cui una connessione viene rinforzata durante una transizione. Un quinto comportamento è la propagazione di pattern, in cui gruppi di nodi attivano i campi in sequenza per produrre una trasformazione collettiva.

Questi comportamenti non devono essere implementati tutti nella prima versione, ma devono essere previsti nella logica architetturale. Un sistema ben progettato non realizza subito tutto, ma lascia spazio all'evoluzione. Per questo motivo, anche se la prima implementazione fisica può essere semplice, il modello deve già distinguere tra comando magnetico, intensità, durata, polarità, target e stato di sicurezza.

In conclusione, il controllo attivo del campo costituisce uno dei passaggi più importanti nell'evoluzione del sistema magnetico MicroBot. Esso trasforma il campo magnetico da proprietà statica a variabile programmabile, collegando firmware, elettronica di potenza, bobine, energia, sicurezza e comportamento dello swarm. Attraverso il controllo attivo, il MicroBot può modificare le proprie interazioni locali e contribuire alla formazione di strutture dinamiche. Questo livello non sostituisce l'allineamento passivo, ma lo completa: il passivo garantisce stabilità ed efficienza, l'attivo introduce selettività, temporizzazione e riconfigurazione. Insieme, questi due livelli rendono possibile una forma concreta di materia programmabile, in cui la struttura collettiva non è imposta rigidamente, ma emerge da campi controllati, interazioni locali e stati distribuiti.

6.5 Relazione tra corrente, spire e campo

La relazione tra corrente, numero di spire e campo magnetico rappresenta il fondamento quantitativo del sistema magnetico attivo del MicroBot. Dopo aver definito il ruolo delle bobine elettromagnetiche e il principio del controllo attivo del campo, è necessario comprendere in modo più rigoroso quali grandezze determinano l'intensità del campo generato e in che modo tali grandezze influenzano il comportamento fisico del nodo. Questa sezione costituisce quindi il passaggio dalla descrizione funzionale del sistema magnetico alla sua modellazione elettromagnetica di base, mantenendo sempre il collegamento con i vincoli reali del progetto: miniaturizzazione, alimentazione Li-Po, controllo tramite ESP32, driver MOSFET, dissipazione termica e integrazione nello swarm.

Nel caso di una bobina elettromagnetica, il campo magnetico prodotto dipende principalmente dalla corrente che attraversa il filo conduttore, dal numero di spire dell'avvolgimento, dalla geometria della bobina e dal materiale eventualmente posto al suo interno. In forma semplificata, per un solenoide ideale, la relazione può essere espressa come:

$$B = \mu_0 \cdot \mu_r \cdot (N / L) \cdot I$$

dove B rappresenta il campo magnetico, μ_0 è la permeabilità magnetica del vuoto, μ_r è la permeabilità relativa del materiale interno, N è il numero di spire, L è la lunghezza della bobina e I è la corrente che la attraversa. Anche se il MicroBot non utilizza necessariamente bobine ideali, questa equazione fornisce una struttura concettuale essenziale: il campo cresce all'aumentare della corrente, cresce all'aumentare del numero di spire e può essere amplificato dalla presenza di materiali con elevata permeabilità magnetica.

Il primo elemento da analizzare è la corrente. La corrente I rappresenta la grandezza direttamente controllabile dal sistema elettronico, anche se in pratica il microcontrollore non regola la corrente in modo analogico diretto, ma pilota il circuito attraverso segnali digitali o PWM. Quando il MOSFET viene attivato, la corrente può fluire nella bobina; quando viene disattivato, il flusso si interrompe o decade secondo la dinamica dell'induttanza e del circuito di protezione. La corrente è quindi il parametro operativo più immediato attraverso cui il firmware può influenzare il campo magnetico. Aumentando la corrente, il campo tende ad aumentare; diminuendola, il campo tende a ridursi.

Tuttavia, l'aumento della corrente non è gratuito. La bobina possiede una resistenza elettrica R, determinata dalla lunghezza del filo, dalla sezione del conduttore e dal materiale utilizzato. Quando la corrente attraversa la resistenza della bobina, una parte dell'energia viene dissipata sotto forma di calore. La potenza dissipata è data dalla relazione:

$$P = R \cdot I^2$$

Questa formula è fondamentale per il progetto MicroBot perché mostra che il costo termico della corrente cresce quadraticamente. Se la corrente raddoppia, la potenza dissipata non raddoppia, ma quadruplica. Questo significa che il controllo del campo non può essere basato semplicemente sull'aumento della corrente fino a ottenere la forza desiderata. Bisogna invece trovare un compromesso tra intensità del campo, durata dell'attivazione, capacità della batteria, temperatura della bobina e sicurezza del nodo.

Il secondo parametro fondamentale è il numero di spire N . A parità di corrente, aumentare il numero di spire aumenta il contributo magnetico complessivo della bobina, perché ogni spira attraversata da corrente contribuisce alla generazione del campo. In teoria, quindi, un numero maggiore di spire permette di ottenere un campo più intenso senza aumentare la corrente. Questo è particolarmente interessante in un sistema alimentato a batteria, dove limitare la corrente è importante per ridurre consumi e calore. Tuttavia, anche il numero di spire introduce vincoli. Più spire significano più lunghezza di filo, quindi maggiore resistenza. Una resistenza più alta riduce la corrente disponibile a parità di tensione e può aumentare la dissipazione complessiva. Il numero di spire deve quindi essere dimensionato insieme alla corrente, non separatamente.

Il terzo parametro è la lunghezza della bobina L . Nella formula del solenoide ideale, il campo dipende dal rapporto N/L , cioè dalla densità di spire per unità di lunghezza. Una bobina con molte spire concentrate in poco spazio può generare un campo più intenso nella regione interna rispetto a una bobina più lunga con la stessa quantità di spire distribuite. Nel MicroBot, questa osservazione è rilevante perché lo spazio interno è limitato e la bobina deve essere collocata in zone specifiche, probabilmente vicino alle superfici di interazione. Tuttavia, concentrare troppe spire in uno spazio ridotto può aumentare il calore locale, rendere difficile la dissipazione e complicare la costruzione meccanica dell'avvolgimento.

Il quarto parametro è la permeabilità magnetica del materiale interno, indicata con μ_r . Se la bobina contiene aria, la permeabilità relativa è circa 1. Se invece contiene un nucleo ferromagnetico, il campo può essere concentrato e amplificato. Questo può permettere di ottenere un campo più elevato con la stessa corrente o, viceversa, di ottenere lo stesso campo con minore corrente. Nel MicroBot, l'uso di un nucleo ferromagnetico può essere molto interessante, ma introduce altri vincoli: aumenta il peso, occupa volume, può saturarsi, modifica la risposta dinamica della bobina e può interferire con magneti permanenti o sensori futuri. Per questo motivo, la scelta del nucleo deve essere valutata come parte del dimensionamento complessivo, non come miglioramento automatico.

Nel caso reale del MicroBot, la relazione tra corrente, spire e campo deve essere considerata in modo non ideale. Le bobine saranno probabilmente piccole, non perfettamente lunghe rispetto al loro diametro, collocate vicino a superfici curve o coniche, integrate in una struttura compatta e influenzate dalla presenza di altri componenti. Di conseguenza, il campo prodotto non sarà uniforme come nel modello teorico. La formula del solenoide resta utile come riferimento iniziale, ma il comportamento reale dovrà essere validato tramite test, simulazione o misure sperimentali. Questo è coerente con il metodo generale del progetto MicroBot: partire da un modello semplice, implementare un prototipo, osservare il comportamento reale e correggere progressivamente il modello.

Dal punto di vista del firmware, la relazione tra corrente e campo può essere rappresentata attraverso una funzione di controllo. In una prima versione, il comando magnetico può essere normalizzato in un valore compreso tra 0 e 1, dove 0 rappresenta bobina spenta e 1 rappresenta attivazione massima consentita. Il firmware può quindi tradurre questo comando in un duty cycle PWM. In forma semplificata:

`duty_cycle = funzione(comando_magnetico)`

e, idealmente:

$$B \approx B_{\text{max}} \cdot \text{duty_cycle}$$

Questa relazione permette di astrarre il controllo della bobina senza obbligare il software di alto livello a ragionare direttamente in termini di corrente o campo fisico. Tuttavia, il livello embedded deve comunque rispettare i limiti fisici: duty cycle massimo, durata massima di attivazione, temperatura stimata, tensione della batteria e corrente ammissibile. Il comando software deve sempre essere filtrato da una logica di sicurezza.

Un aspetto importante riguarda il fatto che la corrente effettiva nella bobina non dipende solo dal duty cycle, ma anche dalla tensione disponibile, dalla resistenza della bobina, dall'induttanza, dalla frequenza PWM e dalla risposta del circuito. Se la batteria è carica, la tensione disponibile sarà maggiore; se la batteria è scarica, la tensione sarà minore. Questo significa che lo stesso comando PWM potrebbe produrre campi leggermente diversi in momenti diversi. Per ottenere un controllo più preciso, nelle versioni avanzate potrebbe essere necessario monitorare la tensione della batteria o stimare la corrente assorbita, correggendo il comando in funzione delle condizioni reali.

Questo introduce un punto essenziale: nel MicroBot, il campo magnetico non è soltanto funzione del comando, ma dello stato complessivo del nodo. Una rappresentazione più corretta è:

$B(t) = \text{funzione}(N, L, \mu_r, R, V_{\text{battery}}(t), \text{duty_cycle}(t), \text{temperatura}(t))$

Questa formulazione mostra che il campo è il risultato di una combinazione tra parametri costruttivi e variabili operative. I parametri costruttivi sono definiti durante la progettazione della bobina; le variabili operative cambiano durante il funzionamento. Il sistema deve quindi essere progettato per funzionare in modo robusto anche quando le condizioni non sono perfettamente costanti.

La relazione tra corrente, spire e campo si collega direttamente alla forza di interazione tra MicroBot. Il campo magnetico generato da una bobina non è l'obiettivo finale: l'obiettivo è produrre una forza utile per attrarre, respingere, allineare o stabilizzare altre unità. Tuttavia, la forza dipende non solo dal campo prodotto, ma anche dal gradiente del campo, dalla distanza tra i nodi, dalla presenza di magneti permanenti, dall'orientamento e dalle proprietà magnetiche dell'altra unità. Per questo motivo, un campo più intenso non garantisce automaticamente una migliore interazione. È necessario progettare il campo nella zona giusta, con la direzione giusta e con intensità compatibile con la configurazione desiderata.

Nel progetto MicroBot, questo significa che la bobina deve essere dimensionata non solo per "generare tanto campo", ma per generare un campo utile. Un campo disperso internamente al nodo o orientato male potrebbe consumare energia senza produrre una forza efficace sull'altro MicroBot. Al contrario, una bobina ben posizionata vicino alla superficie di contatto può produrre un'interazione più efficace anche con potenza inferiore. La relazione tra corrente, spire e campo deve quindi essere sempre collegata alla geometria del nodo.

Un altro punto fondamentale è la differenza tra campo statico e campo dinamico. Una bobina attivata con corrente costante tende a generare un campo stabile; una bobina controllata in PWM genera un campo medio modulato nel tempo. Se la frequenza PWM è adeguata rispetto alla risposta elettrica e meccanica del sistema, il campo può essere trattato come quasi continuo. Se invece la frequenza è troppo bassa o il sistema è molto sensibile, la variazione può produrre effetti pulsanti. In alcuni casi, questa pulsazione potrebbe essere indesiderata; in altri, potrebbe essere sfruttata per facilitare il distacco, il riallineamento o la vibrazione locale. Questa possibilità apre una direzione futura interessante, ma deve essere affrontata con cautela per evitare instabilità.

La scelta tra aumentare corrente o aumentare spire rappresenta uno dei compromessi più importanti del dimensionamento. Aumentare la corrente rende il controllo più immediato, ma aumenta rapidamente calore e consumo. Aumentare le spire può aumentare il campo a parità di corrente, ma aumenta resistenza, volume e complessità costruttiva. Usare un nucleo ferromagnetico può aumentare l'efficienza magnetica, ma introduce saturazione, peso e vincoli meccanici. Ridurre la bobina facilita l'integrazione, ma riduce il campo generabile. Ogni scelta ha conseguenze su tutto il sistema.

Dal punto di vista della simulazione, questa relazione può essere introdotta in modo progressivo. In una prima versione del simulatore, il campo magnetico può essere modellato come proporzionale a un parametro di potenza magnetica. In una versione più avanzata, il modello può includere numero di spire, corrente e distanza. In una versione ancora più realistica, può includere saturazione, limiti termici, risposta temporale e orientamento. Questa progressione permette di mantenere coerenza tra documentazione, simulazione e hardware, evitando di costruire un modello troppo complesso prima di avere dati reali.

Un possibile modello simulativo semplificato può essere:

$$B_{\text{virtual}} = k \cdot N \cdot I$$

dove k è un coefficiente calibrato sperimentalmente. Successivamente, si può aggiungere il termine geometrico:

$$B_{\text{virtual}} = k \cdot (N / L) \cdot I$$

e poi eventualmente una correzione per la distanza dal punto di interazione. In questo modo, il modello resta leggibile e modificabile, ma può crescere insieme al progetto.

Un aspetto molto importante riguarda la misurazione sperimentale. Per passare da teoria a prototipo, il progetto dovrebbe prevedere test controllati in cui si misura l'effetto della bobina al variare del duty cycle, della corrente, della distanza e del numero di spire. Anche senza strumenti avanzati, è possibile realizzare test comparativi: verificare a quale distanza una bobina riesce ad attrarre un piccolo magnete, misurare il riscaldamento dopo determinati tempi di attivazione, confrontare bobine con diverso numero di spire, osservare il consumo della batteria. Questi dati permetterebbero di calibrare il modello e rendere il documento ancora più solido.

Dal punto di vista energetico, questa sezione dimostra perché il sistema magnetico deve essere progettato insieme al safety layer. La corrente nella bobina non può essere considerata un parametro libero. Deve essere limitata da soglie operative. Si può definire una corrente massima sicura, una durata massima di attivazione e un duty cycle massimo. Per esempio, il sistema potrebbe consentire impulsi brevi ad alta intensità, ma impedire mantenimenti prolungati con la stessa potenza. Questo tipo di logica permette di ottenere prestazioni utili senza compromettere la sicurezza.

Dal punto di vista dello stato del nodo, la corrente nella bobina e il campo prodotto diventano variabili interne fondamentali. Il MicroBot può essere descritto come un nodo il cui stato magnetico dipende da parametri controllati e parametri costruttivi. In una rappresentazione più completa:

$$s_i(t) = (s_{\text{compute}}, s_{\text{energy}}, s_{\text{comm}}, s_{\text{magnetic}}, s_{\text{safety}})$$

dove:

$$s_{\text{magnetic}} = (N, R, L, I(t), \text{duty_cycle}(t), B(t), \text{thermal_estimate})$$

Questa formulazione mostra che il magnetismo non è un elemento isolato, ma una parte dello stato complessivo del nodo. Il controller, il firmware e la telemetria dovrebbero quindi trattarlo come variabile osservabile o almeno stimabile.

La relazione tra corrente, spire e campo ha anche un valore metodologico per l'evoluzione delle versioni. Nelle prime versioni, il progetto può limitarsi a rappresentare il campo in modo simbolico tramite LED o simulazioni. In una fase successiva, può introdurre una bobina di test con pochi parametri controllati. In una fase più avanzata, può confrontare diverse bobine e costruire una tabella sperimentale. Infine, può arrivare a un dimensionamento più preciso, in cui la bobina viene progettata per soddisfare una forza minima richiesta a una certa distanza. Questa progressione è coerente con l'intero sistema MicroBot, che evolve per livelli verificabili.

Un altro elemento importante è la tolleranza. Anche se due bobine vengono progettate con lo stesso numero di spire, nella pratica possono avere differenze di resistenza, disposizione del filo, densità dell'avvolgimento e posizione nel nodo. Queste differenze possono generare campi leggermente diversi. In uno swarm composto da molti nodi, la variabilità tra unità deve essere considerata. Il sistema di controllo potrebbe dover accettare una certa tolleranza, oppure calibrare i nodi individualmente. Questo tema diventerà sempre più importante con l'aumentare della scala del progetto.

La relazione tra corrente, spire e campo si collega infine al concetto più generale di materia programmabile. In un materiale programmabile, la forma collettiva non deriva soltanto dalla posizione degli elementi, ma dalle interazioni controllabili tra essi. La bobina rende queste interazioni dipendenti da una variabile elettrica, quindi programmabile. La corrente diventa un mezzo attraverso cui il software modifica il campo; il campo modifica le forze; le forze modificano la configurazione. Questa catena è il cuore concettuale del sistema:

software → corrente → campo → forza → forma

In conclusione, la relazione tra corrente, spire e campo rappresenta il fondamento quantitativo del controllo magnetico nel MicroBot. Essa permette di comprendere come le scelte elettriche e costruttive della bobina influenzino il campo generato e, indirettamente, le interazioni tra nodi. Il campo magnetico non dipende da un singolo parametro, ma da una combinazione di corrente, numero di spire, geometria, resistenza, materiale, temperatura e condizioni operative. Per questo motivo, il dimensionamento del sistema magnetico deve essere affrontato in modo integrato, bilanciando prestazione, sicurezza, consumo energetico e miniaturizzazione. Questa sezione prepara direttamente il passaggio successivo dell'indice, dedicato all'energia magnetica, dove verrà analizzato non solo come il campo viene generato, ma anche quanta energia viene immagazzinata, trasformata e dissipata durante il funzionamento del sistema.

6.6 Energia magnetica

L'energia magnetica rappresenta il livello fisico attraverso cui il campo generato dal sistema magnetico del MicroBot può essere interpretato non solo come grandezza spaziale, ma come riserva energetica temporanea capace di produrre effetti reali sulle unità vicine. Dopo aver analizzato la relazione tra corrente, numero di spire e campo magnetico, diventa necessario comprendere che il campo non è soltanto una distribuzione geometrica nello spazio, ma anche una forma di energia immagazzinata. Questa energia è ciò che permette alla bobina di generare una forza, influenzare la posizione relativa dei nodi, contribuire all'aggancio, stabilizzare una configurazione o favorire una riconfigurazione. Nel sistema MicroBot, l'energia magnetica costituisce quindi il ponte tra circuito elettrico, campo fisico e comportamento meccanico dello swarm.

Nel caso di una bobina elettromagnetica, l'energia immagazzinata nel campo magnetico può essere espressa, in forma semplificata, dalla relazione:

$$E = 1/2 \cdot L \cdot I^2$$

dove E rappresenta l'energia magnetica immagazzinata, L rappresenta l'induttanza della bobina e I rappresenta la corrente che la attraversa. Questa formula è fondamentale perché mostra che l'energia magnetica dipende dal quadrato della corrente, esattamente come la potenza dissipata per effetto Joule dipende da I^2 . Tuttavia, i due fenomeni non devono essere confusi: la dissipazione termica rappresenta energia persa sotto forma di calore, mentre l'energia magnetica rappresenta energia temporaneamente conservata nel campo e potenzialmente utilizzabile per produrre un'interazione fisica.

Nel MicroBot, questa distinzione è molto importante. Quando una bobina viene attivata, una parte dell'energia fornita dalla batteria viene dissipata come calore nella resistenza del filo, mentre un'altra parte viene immagazzinata nel campo magnetico. La parte immagazzinata è quella che contribuisce all'effetto utile del sistema, cioè alla generazione di un campo capace di interagire con magneti, bobine o materiali ferromagnetici vicini. La progettazione del sistema deve quindi cercare di massimizzare l'effetto utile riducendo le perdite, compatibilmente con i vincoli di volume, peso, corrente e temperatura.

Dal punto di vista concettuale, l'energia magnetica può essere vista come una forma di energia di interazione. Non è energia meccanica già trasformata in movimento, ma energia presente nel campo, pronta a produrre effetti su elementi sensibili al magnetismo. Quando due MicroBot si avvicinano, il campo generato da uno o entrambi può modificare la configurazione energetica del sistema complessivo. Le unità tendono a disporsi verso configurazioni a energia minore, cioè verso stati più stabili dal punto di vista magnetico. Questo è il principio fisico che rende possibile l'allineamento, l'attrazione e il mantenimento dell'aggancio.

Nel caso dei magneti permanenti, l'energia magnetica è già incorporata nella configurazione materiale del magnete. Il sistema non deve fornire corrente per mantenere il campo, perché il campo è una proprietà stabile del materiale. Nel caso delle bobine, invece, l'energia magnetica deve essere prodotta attivamente a partire dall'energia elettrica fornita dalla batteria. Questa differenza è centrale per comprendere il ruolo complementare dei due approcci: i magneti permanenti offrono energia magnetica stabile senza consumo continuo, mentre le bobine permettono di generare energia magnetica controllabile, ma al costo di un consumo elettrico e di dissipazione termica.

Nel sistema MicroBot, una strategia progettuale intelligente consiste nel distinguere tra energia magnetica di mantenimento ed energia magnetica di transizione. L'energia di mantenimento è quella necessaria per conservare una struttura già formata, ad esempio mantenere due nodi agganciati. Questa funzione può essere svolta efficacemente dai magneti permanenti, perché non richiedono alimentazione continua. L'energia di transizione, invece, è quella necessaria per modificare lo stato del sistema: avvicinare due nodi, rinforzare temporaneamente un aggancio, favorire un distacco, correggere un orientamento o guidare una

riconfigurazione. Questa funzione è più adatta alle bobine elettromagnetiche, perché il loro campo può essere acceso, spento o modulato.

Questa distinzione consente di interpretare il sistema magnetico come una struttura ibrida. I magneti permanenti definiscono un paesaggio energetico di base, cioè una serie di configurazioni naturalmente favorite; le bobine modificano temporaneamente questo paesaggio, rendendo possibili transizioni che altrimenti sarebbero difficili. In termini più semplici, il livello passivo stabilizza, mentre il livello attivo modifica. Questa architettura è coerente con l'intera filosofia MicroBot: non forzare ogni comportamento dall'alto, ma progettare regole fisiche locali e intervenire attivamente solo quando serve.

Dal punto di vista matematico, si può immaginare che ogni configurazione dello swarm abbia una certa energia associata. Le configurazioni più stabili corrispondono a minimi energetici locali, cioè stati in cui le forze tra nodi tendono a mantenere la struttura. Se il sistema deve passare da una configurazione a un'altra, deve superare una barriera energetica. L'attivazione delle bobine può essere interpretata come un modo per modificare temporaneamente questa barriera, rendendo più facile il passaggio verso una nuova configurazione. Questa idea è estremamente importante perché collega il magnetismo alla riconfigurazione del sistema.

Nel MicroBot, quindi, il controllo non deve necessariamente imporre direttamente la posizione finale di ogni nodo. Può invece modificare le condizioni energetiche locali affinché alcune configurazioni diventino più probabili, più stabili o più facili da raggiungere. Questo è un modo più naturale di concepire la materia programmabile: il software non sposta brutalmente la materia, ma modifica i vincoli fisici che guidano la materia verso una nuova forma. In questa prospettiva, l'energia magnetica diventa uno strumento di progettazione della forma.

Un aspetto fondamentale riguarda la quantità di energia realmente disponibile nel nodo. Ogni MicroBot è alimentato da una batteria di capacità limitata, quindi non può generare campi intensi per lunghi periodi. L'energia magnetica attiva deve essere usata in modo selettivo, con impulsi brevi, controllati e finalizzati. Questo porta a una logica di efficienza: non si attiva una bobina senza uno scopo preciso; non si mantiene corrente quando il campo non è più necessario; non si tenta di compensare con potenza elettrica ciò che può essere risolto con geometria, polarità e allineamento passivo.

Dal punto di vista del firmware, l'energia magnetica può essere gestita attraverso parametri operativi come durata dell'impulso, intensità del PWM, intervallo tra attivazioni e stato termico stimato. Una sequenza magnetica non dovrebbe essere definita soltanto da "accendi" o "spegni", ma da una quantità controllata di energia trasferita al sistema. Per esempio, un impulso breve ad alta intensità può avere un effetto diverso rispetto a un'attivazione più lunga a bassa intensità. Il firmware deve quindi permettere una gestione temporale del campo, non soltanto una gestione binaria.

In forma concettuale, si può descrivere un comando magnetico come:

comando_magnetico = (intensità, durata, direzione, vincoli_sicurezza)

Questa rappresentazione evidenzia che l'effetto magnetico dipende dalla combinazione di potenza e tempo. La quantità totale di energia coinvolta non dipende soltanto dal livello di attivazione, ma anche dalla durata dell'attivazione. Questo è fondamentale per evitare surriscaldamenti e consumi non controllati. Un comando ad alta intensità può essere accettabile se molto breve, ma pericoloso se prolungato.

Dal punto di vista termico, l'energia magnetica non può essere separata dall'energia dissipata. Ogni volta che la bobina viene attivata, parte dell'energia elettrica non diventa campo utile ma calore. Questo significa che l'efficienza del sistema magnetico non è mai perfetta. In un MicroBot miniaturizzato, dove la dissipazione del calore è difficile, il bilancio energetico deve essere trattato con particolare attenzione. Il calore generato può influenzare la batteria, il microcontrollore, il corpo del robot e la stabilità complessiva del nodo. Per questo motivo, l'energia magnetica deve essere sempre collegata al safety layer.

Un modello operativo utile consiste nel distinguere tra tre grandezze: energia fornita dalla batteria, energia immagazzinata nel campo magnetico ed energia dissipata come calore. In forma concettuale:

energia_elettrica_ingresso = energia_magnetica_utile + energia_dissipata + perdite

Questa relazione non serve necessariamente per un calcolo preciso nelle prime versioni, ma aiuta a mantenere chiara la logica del sistema. Non tutta l'energia prelevata dalla batteria produce effetto utile; una parte viene persa. Il progetto deve quindi migliorare progressivamente il rapporto tra energia utile e perdite, attraverso scelta delle bobine, geometria, materiali, driver, frequenza PWM e strategie di controllo.

Nel caso dei magneti permanenti, il discorso energetico assume una forma diversa. Il magnete non consuma energia durante il mantenimento del campo, ma la sua interazione con altri magneti o materiali può essere descritta comunque in termini di energia potenziale magnetica. Quando due magneti si attraggono, il sistema tende verso una configurazione di energia minore. Per separarli, è necessario fornire lavoro dall'esterno. Questo lavoro di separazione rappresenta una misura indiretta della stabilità dell'aggancio. Se il lavoro richiesto è troppo basso, la connessione è instabile; se è troppo alto, il sistema diventa difficile da riconfigurare.

Questo concetto sarà importante anche nelle sezioni successive su accumulo e rilascio energetico, forza di attrazione e dipendenza dalla distanza. Per ora è sufficiente stabilire che ogni connessione magnetica introduce una forma di energia potenziale nel sistema. L'aggancio tra due MicroBot non è soltanto un contatto fisico, ma uno stato energeticamente favorito. Il distacco richiede invece il superamento di una barriera energetica. Questo rende il sistema magnetico naturalmente adatto alla costruzione di strutture stabili ma riconfigurabili.

Dal punto di vista dello swarm, l'energia magnetica non riguarda solo la coppia di nodi, ma la configurazione collettiva. Quando molti MicroBot sono vicini, le interazioni magnetiche possono sommarsi, competere o generare configurazioni inattese. Una struttura può essere stabile non perché ogni singola connessione sia molto forte, ma perché l'insieme delle connessioni distribuisce le forze in modo coerente. Questo è un principio tipico dei sistemi complessi: la stabilità globale non deriva necessariamente da un singolo vincolo dominante, ma dalla rete di interazioni locali.

Nel modello matematico dello swarm, l'energia magnetica può essere utilizzata come criterio per valutare la stabilità di una configurazione. Una configurazione con molte connessioni compatibili e ben allineate può essere considerata a energia più bassa rispetto a una configurazione disordinata o instabile. Questo approccio può essere utile nelle simulazioni, dove il sistema può cercare configurazioni che minimizzano una funzione energetica semplificata. Anche se nelle prime versioni il modello sarà approssimato, l'idea è molto potente: il comportamento dello swarm può essere studiato come evoluzione verso configurazioni magneticamente più favorevoli.

Una possibile funzione semplificata di energia per una simulazione potrebbe considerare la distanza tra nodi, l'orientamento e lo stato magnetico. Per esempio, coppie di nodi vicini e correttamente allineati riducono l'energia del sistema, mentre nodi troppo vicini ma orientati male possono aumentare instabilità o generare repulsione. Questo tipo di modello permette di simulare l'auto-organizzazione senza descrivere ogni dettaglio fisico del campo. È un compromesso coerente con la metodologia del progetto: partire da una rappresentazione utile, poi aumentarne il realismo.

Un altro aspetto importante riguarda l'energia magnetica come forma di memoria temporanea. Quando una bobina viene attivata, essa immagazzina energia nel campo; quando viene disattivata, questa energia deve essere rilasciata. Il rilascio non è istantaneo e non può essere ignorato. Il diodo di flyback, analizzato nella Sezione 5, serve proprio a fornire un percorso sicuro a questa energia. Questo dimostra che l'energia magnetica non è soltanto utile quando il campo viene generato, ma deve essere gestita anche quando il campo viene spento. Accensione e spegnimento sono entrambe fasi progettuali.

La gestione dell'energia magnetica è quindi legata alla sicurezza elettrica. Se l'energia immagazzinata nel campo non viene rilasciata correttamente, può generare sovratensioni, disturbi, danni ai MOSFET o reset del microcontrollore. Il sistema deve quindi prevedere un percorso controllato per l'energia residua. Questo è un esempio perfetto di come il progetto MicroBot richieda una visione integrata: una scelta fisica, come usare una bobina, produce conseguenze elettroniche, firmware, termiche e di sicurezza.

Dal punto di vista del controllo, l'energia magnetica può essere pensata come una risorsa da assegnare. Il controller o il firmware possono decidere dove e quando usare energia attiva, evitando sprechi. Per esempio, se due nodi sono già agganciati passivamente, non ha senso mantenere una bobina attiva al massimo. Se invece due nodi sono vicini ma non ancora allineati, un impulso magnetico può essere giustificato. Se un nodo ha batteria bassa, il sistema può ridurre la sua partecipazione ad azioni magnetiche attive. In questo modo, l'energia magnetica diventa parte della logica di coordinamento dello swarm.

Questo porta a una possibile strategia avanzata: il sistema potrebbe assegnare ruoli energetici ai nodi. Alcuni nodi con batteria più alta potrebbero svolgere temporaneamente funzioni attive, mentre nodi con batteria più bassa potrebbero limitarsi a mantenere connessioni passive o trasmettere stato. Questa idea collega il sistema magnetico alla telemetria e alla gestione energetica, mostrando come l'energia non sia soltanto una proprietà locale, ma una variabile globale dello swarm.

Nel contesto dei prototipi, l'energia magnetica può essere introdotta progressivamente. In una prima fase, i LED possono rappresentare simbolicamente l'attivazione energetica: luminosità maggiore indica maggiore comando magnetico virtuale. In una seconda fase, una bobina di test può essere attivata con impulsi brevi, misurando riscaldamento e consumo. In una terza fase, si può confrontare la capacità di attrazione a distanze diverse. In una quarta fase, si può integrare il dato nel simulatore. Questa progressione permette di costruire conoscenza sperimentale senza saltare direttamente a un sistema complesso e fragile.

Dal punto di vista documentale, questa sezione ha anche il compito di chiarire che il magnetismo non è "gratis". I magneti permanenti non consumano energia elettrica durante il funzionamento, ma introducono vincoli fisici; le bobine offrono controllo, ma richiedono energia, generano calore e necessitano di protezione. La progettazione deve quindi evitare due errori opposti: pensare che basti aumentare la corrente per ottenere un sistema migliore, oppure pensare che i magneti permanenti siano sufficienti per un sistema realmente programmabile. La soluzione più coerente è un equilibrio controllato tra energia passiva e energia attiva.

Un ulteriore aspetto riguarda la scala del sistema. In un singolo MicroBot, l'energia magnetica può sembrare piccola, ma in uno swarm con molte unità la somma delle attivazioni può diventare significativa. Se molti nodi attivano bobine contemporaneamente, il consumo totale cresce, i disturbi elettromagnetici aumentano e la gestione termica diventa più complessa. La scalabilità richiede quindi politiche di attivazione coordinate, limiti globali e strategie temporali. Questo punto sarà importante nelle sezioni future dedicate alla scalabilità e ai limiti energetici.

Dal punto di vista teorico avanzato, l'energia magnetica si collega direttamente al concetto di forma come evento dinamico. La forma dello swarm non è semplicemente una disposizione geometrica, ma il risultato di un bilancio energetico tra attrazione, repulsione, vincoli meccanici, comandi attivi e condizioni ambientali. La forma esiste perché una certa configurazione energetica la sostiene. Se l'energia attiva viene modificata, o se le connessioni passive vengono superate, la forma può cambiare. In questo senso, la forma non è oggetto statico, ma stato fisico mantenuto da interazioni.

Questa interpretazione è molto coerente con il concetto di MicroBot come materia programmabile. In una materia programmabile reale, il software non cambia magicamente la forma della materia; modifica parametri fisici che rendono alcune forme possibili, stabili o instabili. L'energia magnetica è uno di questi parametri. Attraverso la sua generazione, distribuzione e dissipazione, il sistema può attraversare configurazioni diverse. La programmazione della materia passa quindi per la programmazione delle interazioni energetiche tra unità.

In conclusione, l'energia magnetica rappresenta uno dei concetti centrali del sistema magnetico MicroBot. Essa descrive la quantità di energia immagazzinata nel campo, la capacità del sistema di produrre interazioni fisiche e il costo energetico associato al controllo attivo. Nei magneti permanenti, l'energia magnetica fornisce stabilità passiva senza consumo continuo; nelle bobine, essa diventa una risorsa controllabile ma limitata, legata a corrente, induttanza, dissipazione e sicurezza. Comprendere l'energia magnetica significa comprendere il passaggio tra elettronica, campo, forza e forma. Questa sezione prepara quindi il punto successivo dell'indice, dedicato all'accumulo e rilascio energetico, dove verrà approfondito il modo in cui l'energia viene temporaneamente immagazzinata nel sistema e successivamente restituita, dissipata o trasformata durante le transizioni magnetiche.

6.7 Accumulo e rilascio energetico

L'accumulo e il rilascio energetico costituiscono uno dei meccanismi fondamentali attraverso cui il sistema magnetico del MicroBot passa da una condizione elettrica interna a un effetto fisico osservabile. Dopo aver definito l'energia magnetica come energia temporaneamente immagazzinata nel campo, è necessario analizzare il comportamento dinamico di questa energia: come viene accumulata durante l'attivazione della bobina, come viene mantenuta per un intervallo limitato, come viene rilasciata durante lo spegnimento e quali conseguenze produce sul circuito, sul firmware, sulla sicurezza e sull'interazione tra nodi. Questo punto è essenziale perché chiarisce che il sistema magnetico non è istantaneo e non è privo di inerzia: ogni attivazione implica una fase di carica energetica, una fase di effetto utile e una fase di scarica o dissipazione.

Nel caso di una bobina elettromagnetica, l'accumulo energetico avviene quando una corrente inizia a scorrere nell'avvolgimento. La bobina, per sua natura induttiva, si oppone alla variazione rapida della corrente. Questo significa che, quando il MOSFET viene attivato e il circuito si chiude, la corrente non raggiunge immediatamente il suo valore massimo teorico, ma cresce progressivamente. Durante questa crescita, parte dell'energia fornita dalla batteria viene immagazzinata nel campo magnetico generato dalla bobina. Il campo diventa quindi una forma temporanea di energia fisica, presente nello spazio intorno al componente e capace di interagire con magneti, materiali ferromagnetici o altre bobine.

La relazione fondamentale che descrive l'energia accumulata nel campo magnetico è:

$$E = 1/2 \cdot L \cdot I^2$$

dove E rappresenta l'energia magnetica, L l'induttanza della bobina e I la corrente. Questa equazione mostra che l'energia accumulata cresce con il quadrato della corrente. Di conseguenza, piccoli aumenti di corrente possono produrre incrementi significativi dell'energia immagazzinata, ma anche maggiori rischi di dissipazione termica e stress elettrico. Nel MicroBot, questo vincolo è particolarmente importante perché il nodo è miniaturizzato, alimentato a batteria e dotato di capacità limitata di dissipazione del calore.

L'accumulo energetico non deve essere considerato soltanto come fenomeno elettrico, ma anche come fase funzionale del comportamento magnetico. Quando una bobina si attiva, il campo cresce e modifica progressivamente le condizioni di interazione con le unità vicine. Se un altro MicroBot si trova entro una distanza utile, il campo può contribuire all'attrazione, al riallineamento o alla stabilizzazione della connessione. Tuttavia, poiché la corrente cresce con una certa costante di tempo, anche l'effetto magnetico può avere una dinamica progressiva. In questo senso, l'attivazione della bobina non è un evento puntuale, ma un processo.

Questa dinamica può essere collegata alla costante di tempo del circuito induttivo:

$$\tau = L / R$$

dove τ rappresenta il tempo caratteristico, L l'induttanza e R la resistenza del circuito. Una bobina con induttanza maggiore accumula più energia, ma può rispondere più lentamente alle variazioni di comando. Una bobina con resistenza maggiore limita la corrente, ma dissipa più potenza sotto forma di calore. La progettazione deve quindi trovare un equilibrio tra rapidità di risposta, energia accumulabile, intensità del campo e sicurezza termica.

Nel sistema MicroBot, questo equilibrio è decisivo perché le interazioni tra nodi possono richiedere tempi brevi e coordinati. Se il campo cresce troppo lentamente, il comando magnetico potrebbe non produrre l'effetto desiderato nel momento previsto. Se invece il campo cresce rapidamente ma con correnti elevate, il sistema potrebbe consumare troppa energia o generare calore eccessivo. Il firmware deve quindi gestire l'attivazione delle bobine non come semplice stato acceso/spento, ma come processo temporizzato e limitato.

Una strategia coerente consiste nell'utilizzare impulsi magnetici controllati. Un impulso magnetico è una breve attivazione della bobina, definita da intensità e durata. In forma concettuale, può essere descritto come:

impulso = (potenza, durata, direzione, condizione_di_sicurezza)

Questa rappresentazione mostra che l'energia trasferita al sistema dipende sia dall'intensità sia dal tempo di attivazione. Un impulso breve ad alta intensità può essere utile per favorire un aggancio o una correzione di orientamento; un impulso più lungo ma meno intenso può essere utile per stabilizzare temporaneamente una connessione; un impulso troppo lungo o troppo intenso può invece diventare inefficiente o pericoloso. La gestione dell'accumulo energetico deve quindi essere sempre collegata a limiti operativi.

Il rilascio energetico avviene quando la corrente nella bobina viene ridotta o interrotta. In questa fase, l'energia immagazzinata nel campo magnetico deve essere restituita, dissipata o trasferita attraverso un percorso elettrico sicuro. Se il circuito interrompesse bruscamente la corrente senza protezione, la bobina genererebbe una tensione elevata nel tentativo di mantenere il flusso di corrente. Questo fenomeno è stato già collegato al ruolo del diodo di flyback nella Sezione 5: il diodo fornisce un percorso di ricircolo alla

corrente e permette all'energia residua di dissiparsi gradualmente senza danneggiare MOSFET, microcontrollore o alimentazione.

Il rilascio energetico è quindi una fase critica quanto l'accumulo. In un sistema progettato male, lo spegnimento della bobina può essere più pericoloso della sua accensione, perché genera transitori elettrici, rumore, sovratensioni e possibili disturbi sul circuito. Nel MicroBot, dove il microcontrollore ESP32, i moduli di comunicazione e i sistemi di feedback condividono uno spazio elettrico ridotto, questi transitori devono essere gestiti con particolare attenzione. Il rilascio controllato dell'energia è una condizione necessaria per rendere il sistema affidabile.

Dal punto di vista fisico, il rilascio del campo magnetico ha anche conseguenze sull'interazione tra MicroBot. Quando la bobina viene disattivata, il campo non scompare necessariamente in modo istantaneo. A seconda dell'induttanza, della resistenza e del circuito di protezione, il campo può decadere con una certa gradualità. Questo significa che l'effetto magnetico può persistere per un breve intervallo anche dopo il comando di spegnimento. In alcuni casi, questa persistenza è utile perché evita variazioni brusche; in altri casi, può limitare la precisione temporale del controllo.

Nel contesto dello swarm, il tempo di decadimento del campo può influenzare la qualità della riconfigurazione. Se un nodo deve rilasciare una connessione in un determinato istante, ma il campo residuo permane troppo a lungo, il distacco può risultare ritardato. Se invece il campo decade troppo rapidamente e senza controllo, la struttura può diventare instabile. Questo dimostra che accumulo e rilascio energetico non sono dettagli secondari, ma influenzano direttamente il comportamento collettivo.

Una possibile evoluzione del sistema consiste nell'introdurre diverse modalità di rilascio. Una modalità lenta e protetta, basata su diodo di flyback semplice, privilegia la sicurezza elettrica e riduce lo stress sui componenti. Una modalità più rapida, basata su circuiti di clamp più avanzati, può accelerare il decadimento della corrente e migliorare la risposta dinamica del sistema. La scelta dipende dagli obiettivi della versione progettuale. Nei prototipi iniziali è preferibile privilegiare la sicurezza; nelle versioni più avanzate, quando il comportamento magnetico dovrà essere più preciso, sarà possibile ottimizzare la velocità di rilascio.

L'accumulo e il rilascio energetico devono essere considerati anche nel modello di simulazione. Una simulazione troppo semplice potrebbe trattare il campo come grandezza istantanea: comando attivo significa campo presente, comando disattivo significa campo nullo. Questo modello è utile per iniziare, ma non rappresenta fedelmente la fisica reale della bobina. Una simulazione più accurata dovrebbe introdurre una dinamica di crescita e decadimento del campo, ad esempio attraverso una funzione graduale:

$$B(t+1) = B(t) + \alpha \cdot (B_{\text{target}} - B(t))$$

dove B_{target} rappresenta il campo richiesto dal comando e α rappresenta un coefficiente di risposta. Questa formulazione permette di simulare il fatto che il campo non cambia istantaneamente, ma si avvicina progressivamente al valore desiderato. In questo modo, il comportamento virtuale diventa più coerente con quello fisico.

Dal punto di vista del firmware, la gestione energetica della bobina può essere implementata attraverso stati specifici. Il sistema non dovrebbe trattare la bobina solo come ON o OFF, ma come sottosistema con fasi operative. Una possibile macchina a stati può includere:

- bobina inattiva
- bobina in attivazione
- bobina attiva
- bobina in decadimento
- bobina bloccata per sicurezza
- bobina in errore

Questa struttura consente al firmware di gestire meglio i tempi, i limiti e le condizioni di sicurezza. Per esempio, dopo un'attivazione intensa, la bobina potrebbe entrare temporaneamente in una fase di cooldown, durante la quale nuove attivazioni vengono limitate o rifiutate. Questo evita accumulo termico e riduce il rischio di danneggiamento.

Il concetto di cooldown è importante perché il rilascio energetico non riguarda solo il campo magnetico, ma anche il calore prodotto. Una parte dell'energia elettrica fornita alla bobina viene dissipata nella resistenza

del filo. Questo calore non scompare immediatamente quando la bobina viene spenta, ma resta nel materiale e si disperde gradualmente nell'ambiente. Se la bobina viene riattivata troppo spesso, il calore può accumularsi anche se ogni singolo impulso sembra breve. Il firmware deve quindi considerare non solo l'energia del singolo impulso, ma anche la storia recente delle attivazioni.

Una possibile variabile interna del nodo può essere:

`thermal_load(t)`

Questa variabile rappresenta una stima del carico termico accumulato dalla bobina. Ogni attivazione la aumenta; il tempo di inattività la riduce gradualmente. Anche senza un sensore di temperatura reale, una stima software può offrire un primo livello di protezione. In versioni più avanzate, questa stima potrebbe essere integrata con misure reali di temperatura. Questo approccio è coerente con la filosofia incrementale del progetto: prima si implementa un modello prudente, poi si aggiunge sensoristica per migliorarlo.

L'accumulo energetico ha anche un impatto sulla batteria. Attivare una bobina richiede corrente; se la corrente richiesta è elevata, la tensione della batteria può diminuire temporaneamente. Questo può causare instabilità se il sistema di regolazione non è adeguato. In particolare, un picco di assorbimento potrebbe produrre una caduta di tensione sufficiente a disturbare l'ESP32 o i moduli di comunicazione. Per questo motivo, l'accumulo energetico della bobina deve essere coordinato con la gestione dell'alimentazione, i condensatori di stabilizzazione e i limiti di corrente.

Dal punto di vista dello swarm, il problema diventa ancora più importante. Se molti MicroBot attivano bobine contemporaneamente, l'assorbimento globale aumenta e il sistema può generare disturbi distribuiti. Anche se ogni nodo è alimentato autonomamente, la sincronizzazione di molti impulsi magnetici può produrre effetti collettivi: consumo simultaneo, interferenze magnetiche, cambiamenti improvvisi nella configurazione e possibili instabilità. Per questo motivo, il controller centrale potrebbe dover pianificare le attivazioni nel tempo, evitando che troppe bobine operino nello stesso istante.

Una strategia possibile è l'attivazione a finestre temporali. Invece di attivare tutti i nodi simultaneamente, il sistema può suddividere lo swarm in gruppi e attivare le bobine in sequenza. Questo approccio riduce i picchi energetici, migliora la controllabilità e permette di osservare l'effetto di ogni fase prima di passare alla successiva. Tale logica si collega direttamente alla Sezione 9 sulla sincronizzazione e alla Sezione 10 sui pattern e comportamenti.

L'accumulo e rilascio energetico possono essere interpretati anche in termini di transizioni di stato. Ogni configurazione dello swarm può essere vista come uno stato con una certa stabilità energetica. Per passare a un'altra configurazione, il sistema deve modificare temporaneamente le interazioni locali. Le bobine forniscono energia per superare barriere locali, favorire nuovi agganci o rompere connessioni esistenti. Una volta raggiunta la nuova configurazione, i magneti permanenti o altre condizioni passive possono mantenerla. Questo modello è molto importante perché descrive il MicroBot non come un sistema che "sposta pezzi", ma come un sistema che guida transizioni energetiche tra configurazioni.

In questa prospettiva, il software non controlla direttamente la forma finale in modo rigido, ma controlla l'energia disponibile per attraversare certi passaggi. Il comando magnetico diventa quindi un comando di transizione. Non dice semplicemente "vai lì", ma modifica localmente il paesaggio fisico affinché una trasformazione diventi possibile. Questo è molto coerente con il concetto di sistema emergente: il comportamento finale nasce dall'interazione tra comando, fisica locale, geometria e stato precedente.

Un ulteriore punto riguarda la reversibilità. Un sistema realmente riconfigurabile deve poter accumulare energia per formare una connessione, ma anche rilasciare o ridistribuire energia per scioglierla. Se il sistema è capace solo di agganciarsi ma non di separarsi, la riconfigurazione resta limitata. Per questo motivo, il rilascio energetico deve essere progettato non solo come protezione elettrica, ma come parte funzionale della dinamica di distacco. Nelle versioni avanzate, la bobina potrebbe essere usata per ridurre l'effetto di un magnete permanente, invertire temporaneamente il campo o generare una forza di repulsione. Queste funzioni richiedono circuiti più complessi, ma devono essere previste nel modello teorico.

Dal punto di vista documentale, è importante chiarire che l'accumulo energetico utile e l'accumulo termico indesiderato sono due fenomeni diversi ma collegati. L'energia nel campo magnetico può produrre lavoro utile; il calore è invece una conseguenza dissipativa da limitare. Un buon progetto magnetico cerca di aumentare il rapporto tra effetto utile e perdita termica. Questo può essere ottenuto attraverso bobine meglio

dimensionate, nuclei adeguati, controllo temporizzato, duty cycle limitato, geometria ottimizzata e uso combinato di magneti permanenti.

La sperimentazione sarà essenziale per validare questa parte. Un documento progettuale può definire relazioni teoriche, ma il comportamento reale dipenderà dai componenti effettivi. Sarà utile misurare quanto una bobina si scalda dopo impulsi diversi, quanto velocemente decade l'effetto magnetico, quale distanza utile viene raggiunta, quanto consumo produce ogni attivazione e quale configurazione risulta più efficiente. Questi test potranno diventare una base per tabelle sperimentali future e per calibrare il simulatore.

Dal punto di vista del modello dei dati, ogni evento magnetico potrebbe essere registrato in telemetria. Un log utile potrebbe includere: nodo interessato, bobina attivata, duty cycle, durata, tensione batteria prima e dopo, stima del carico termico, esito dell'operazione e timestamp. Questo permetterebbe di collegare il comportamento fisico alla storia del sistema. Nei progetti più maturi, questa telemetria sarebbe fondamentale per debugging, ottimizzazione e sicurezza.

L'accumulo e il rilascio energetico si collegano infine alla visione più ampia del MicroBot come materia programmabile. Una materia programmabile non è definita soltanto dai suoi componenti, ma dalla capacità di attraversare configurazioni diverse in modo controllato. Queste trasformazioni richiedono energia. Il sistema deve quindi sapere quando accumulare energia, dove accumularla, quanta energia usare, come rilasciarla e come evitare che si trasformi in instabilità o danno. In questo senso, la gestione energetica del campo magnetico è una delle basi fisiche della programmabilità della forma.

In conclusione, l'accumulo e il rilascio energetico descrivono il comportamento dinamico del sistema magnetico durante le fasi di attivazione e disattivazione delle bobine. L'energia viene accumulata nel campo magnetico quando la corrente cresce, viene utilizzata per produrre interazioni fisiche e deve essere rilasciata in modo controllato quando il comando termina. Questo processo coinvolge induttanza, corrente, protezione elettrica, calore, firmware, alimentazione, sicurezza, simulazione e telemetria. Comprendere questa dinamica è essenziale per progettare un MicroBot affidabile: il campo magnetico non deve solo essere generato, ma deve essere gestito nel tempo. Questa sezione prepara il passaggio successivo dell'indice, dedicato alla forza di attrazione, dove verrà analizzato come l'energia e il campo si traducano concretamente in forza tra unità e quindi in capacità di aggancio, stabilizzazione e riconfigurazione dello swarm.

6.8 Forza di attrazione

La forza di attrazione rappresenta il risultato fisico più immediato del sistema magnetico MicroBot, perché costituisce il meccanismo attraverso cui due o più unità possono avvicinarsi, agganciarsi, stabilizzarsi e contribuire alla formazione di una configurazione collettiva. Dopo aver analizzato il campo magnetico, l'energia magnetica e i processi di accumulo e rilascio energetico, diventa necessario comprendere come tali grandezze si traducano concretamente in una forza capace di modificare la posizione relativa dei nodi. In questa prospettiva, la forza di attrazione non deve essere interpretata come un fenomeno isolato, ma come il punto di convergenza tra geometria, campo, distanza, orientamento, energia disponibile e stato del sistema.

Nel sistema MicroBot, la forza di attrazione svolge una funzione strutturale: permette alle unità di passare da una condizione indipendente a una condizione connessa. Questo passaggio è essenziale per il concetto di materia programmabile, perché senza una forza capace di creare vincoli locali tra nodi non sarebbe possibile ottenere strutture collettive stabili. La forma globale dello swarm nasce infatti dalla combinazione di molte interazioni locali; ogni attrazione tra due MicroBot rappresenta un legame elementare che, sommato agli altri, contribuisce alla configurazione complessiva del sistema.

Dal punto di vista fisico, la forza di attrazione magnetica dipende dal campo generato, dalla natura del secondo elemento coinvolto, dalla distanza tra le unità e dall'orientamento reciproco. Nel caso più semplice, due magneti permanenti con poli opposti rivolti l'uno verso l'altro tendono ad attrarsi. Nel caso di una bobina e di un magnete, il campo prodotto dalla bobina può generare una forza sul magnete. Nel caso di due bobine, l'interazione dipende dalla corrente che attraversa entrambe e dalla direzione dei rispettivi campi. In ogni caso, il principio generale è che il sistema tende verso configurazioni in cui l'energia magnetica complessiva risulta più favorevole.

Una rappresentazione concettuale della forza di attrazione può essere espressa come:

$F_{\text{attrazione}} = \text{funzione}(B, \text{distanza}, \text{orientamento}, \text{materiale}, \text{geometria})$

dove B rappresenta il campo magnetico, la distanza indica la separazione tra i nodi, l'orientamento descrive la disposizione relativa dei poli o degli assi magnetici, il materiale indica la risposta magnetica degli elementi coinvolti e la geometria descrive la forma delle superfici di contatto. Questa formulazione è importante perché evita una semplificazione eccessiva: la forza non dipende soltanto dall'intensità del campo, ma dall'intero contesto fisico in cui il campo agisce.

Nel caso di magneti permanenti, la forza di attrazione è sempre presente e deriva dalla configurazione del materiale magnetizzato. Questo rende il sistema semplice ed energeticamente efficiente, perché non è necessario consumare corrente per mantenere l'attrazione. Tuttavia, la forza non può essere modificata dinamicamente. Essa è determinata dalla dimensione del magnete, dal materiale, dalla distanza e dall'orientamento. Nel MicroBot, questo significa che i magneti permanenti possono fornire una forza di aggancio stabile, ma non permettono da soli un controllo selettivo dell'attrazione.

Nel caso delle bobine elettromagnetiche, invece, la forza di attrazione può essere controllata indirettamente variando la corrente. Poiché il campo magnetico prodotto dalla bobina dipende dalla corrente e dal numero di spire, il firmware può modificare l'intensità dell'interazione regolando il duty cycle PWM o il tempo di attivazione. In questo caso, la forza di attrazione diventa una variabile programmabile. Questo è un passaggio fondamentale: il nodo non possiede soltanto una forza fisica fissa, ma può generare una forza temporanea in funzione dello stato del sistema.

In forma semplificata, si può rappresentare questa relazione come:

$$F_{\text{attrazione}}(t) = K \cdot u(t) \cdot g(r)$$

dove K è un coefficiente dipendente dalla bobina, dai materiali e dalla geometria, $u(t)$ è il comando magnetico normalizzato, e $g(r)$ è una funzione di decadimento rispetto alla distanza r . Questa equazione non pretende di descrivere con precisione assoluta il comportamento reale, ma è utile per modellare il sistema in simulazione e per comprendere il ruolo dei parametri principali. Il comando $u(t)$ permette di collegare la forza al firmware, mentre $g(r)$ rappresenta il fatto che l'interazione magnetica diminuisce rapidamente quando i nodi si allontanano.

Il decadimento con la distanza è uno degli aspetti più importanti. La forza magnetica non agisce in modo uniforme a qualunque distanza, ma diventa rapidamente più debole man mano che i nodi si separano. Questo significa che il sistema magnetico è naturalmente locale. I MicroBot possono attrarsi efficacemente solo quando si trovano entro una certa distanza operativa. Questa proprietà è coerente con il paradigma dello swarm, in cui il comportamento globale emerge da interazioni locali, ma introduce anche un vincolo pratico: il sistema deve prevedere strategie per portare i nodi abbastanza vicini affinché l'attrazione possa diventare efficace.

Nel modello MicroBot, questa distanza operativa può essere interpretata come una soglia di influenza magnetica. Al di sopra di tale soglia, il campo può essere troppo debole per produrre un effetto significativo; al di sotto, la forza può crescere rapidamente e favorire l'aggancio. Questa osservazione prepara direttamente i concetti successivi di dipendenza dalla distanza, soglia di attacco e soglia di distacco, che saranno fondamentali anche nel modello matematico dello swarm. La forza di attrazione, quindi, non è solo un valore fisico: è la base per definire regole di comportamento.

Un aspetto fondamentale riguarda l'orientamento. Due MicroBot possono trovarsi alla distanza corretta, ma se l'orientamento dei poli o delle bobine è sfavorevole, la forza di attrazione può essere ridotta, annullata o trasformata in repulsione. Questo significa che il sistema deve considerare non solo la posizione dei nodi, ma anche la loro rotazione e l'allineamento relativo. La geometria a coni troncati e sfera, già discussa nella Sezione 4, assume qui un ruolo importante perché può aiutare a guidare meccanicamente l'allineamento, favorendo configurazioni in cui la forza magnetica agisce nella direzione desiderata.

La forza di attrazione può inoltre generare momenti torcenti. Quando due magneti o due campi non sono perfettamente allineati, la forza non produce soltanto traslazione, ma anche rotazione. Questo fenomeno è alla base dell'allineamento passivo: i nodi tendono a ruotare verso una configurazione energeticamente più favorevole. Nel MicroBot, questo comportamento può essere sfruttato per correggere piccoli errori di orientamento senza richiedere sensori o attuatori complessi. La forza magnetica diventa quindi non solo forza di aggancio, ma anche forza di correzione locale.

Dal punto di vista meccanico, la forza di attrazione deve essere confrontata con altre forze presenti nel sistema, come attrito, peso, urti, resistenza delle superfici e vincoli geometrici. Una forza magnetica sufficiente in condizioni ideali potrebbe non essere sufficiente se l'attrito impedisce lo scorrimento o la rotazione del nodo. Al contrario, una forza troppo elevata potrebbe causare agganci bruschi, urti, instabilità o difficoltà di separazione. Il dimensionamento della forza deve quindi tenere conto dell'intero ambiente fisico, non solo del campo magnetico generato.

Una condizione minima per l'aggancio può essere espressa concettualmente come:

$$F_{\text{attrazione}} > F_{\text{resistenza}}$$

dove $F_{\text{resistenza}}$ rappresenta l'insieme delle forze che si oppongono all'avvicinamento o all'allineamento dei nodi. Questa resistenza può includere attrito statico, peso relativo, vincoli di contatto e imperfezioni meccaniche. Se la forza di attrazione non supera questa soglia, il nodo può percepire il campo ma non muoversi o non agganciarsi. Se invece la forza supera la soglia, l'aggancio diventa possibile. Questa relazione è molto importante perché collega il sistema magnetico alla progettazione meccanica.

Dal punto di vista energetico, la forza di attrazione può essere vista come il risultato di una tendenza del sistema a ridurre la propria energia potenziale magnetica. Quando due nodi compatibili si avvicinano, il sistema può passare a una configurazione più stabile. Per separarli, è necessario fornire lavoro dall'esterno o modificare attivamente il campo. Questo significa che l'aggancio magnetico crea un vincolo energetico: la struttura formata non è semplicemente una disposizione spaziale, ma uno stato fisico sostenuto da una differenza di energia.

Questa idea è centrale per il concetto di forma come evento dinamico. Una forma MicroBot non è un oggetto rigido nel senso tradizionale, ma una configurazione mantenuta da molte forze locali. Ogni connessione magnetica contribuisce alla stabilità della forma, e ogni variazione della forza può modificare la struttura. Se alcune forze vengono rinforzate, la forma può consolidarsi; se vengono ridotte o invertite, la forma può riconfigurarsi. La forza di attrazione diventa quindi uno strumento per controllare indirettamente la geometria collettiva.

Nel caso di controllo attivo tramite bobine, la forza di attrazione può essere applicata in modo temporaneo. Questo consente di distinguere tra attrazione di aggancio, attrazione di mantenimento e attrazione di correzione. L'attrazione di aggancio serve ad avvicinare o connettere due nodi; l'attrazione di mantenimento serve a stabilizzare una connessione; l'attrazione di correzione serve a migliorare orientamento o posizione. Queste tre funzioni possono richiedere intensità e durate diverse. Un impulso breve può essere sufficiente per favorire l'aggancio; una forza moderata può mantenere la connessione; una sequenza modulata può correggere l'allineamento.

Il firmware dovrebbe quindi trattare la forza di attrazione come effetto di un comando parametrico, non come semplice attivazione. Un comando magnetico può includere intensità, durata, eventuale direzione, bobina selezionata e condizione di sicurezza. In una versione avanzata, il comando potrebbe anche includere un obiettivo, ad esempio "favorisci aggancio con nodo vicino" oppure "rinforza connessione esistente". Questa distinzione tra comando fisico basso livello e comando comportamentale alto livello sarà importante per rendere il sistema scalabile.

Dal punto di vista della simulazione, la forza di attrazione può essere modellata introducendo una funzione tra coppie di nodi. In una simulazione iniziale, si può usare un modello semplice basato sulla distanza: se due nodi sono entro una certa soglia, viene applicata una forza attrattiva. In una simulazione più avanzata, si possono includere orientamento, stato magnetico, intensità del comando e saturazione della forza. Un possibile modello concettuale è:

$$F_{ij} = k \cdot m_i \cdot m_j \cdot h(r_{ij}, \theta_{ij})$$

dove m_i e m_j rappresentano l'intensità magnetica dei due nodi, r_{ij} la distanza e θ_{ij} l'orientamento relativo. La funzione h descrive come distanza e orientamento modificano la forza effettiva. Anche se semplificato, questo modello permette di rappresentare il comportamento emergente in modo più realistico rispetto a un semplice aggancio binario.

Un aspetto rilevante è la presenza di saturazione. Nella realtà, non è sempre utile aumentare indefinitamente il comando magnetico. Oltre una certa intensità, l'aumento della forza può non produrre

vantaggi proporzionali, mentre aumenta consumo, calore e rischio di instabilità. Per questo motivo, il sistema dovrebbe prevedere una forza massima operativa, oltre la quale il comando viene limitato. Questo limite può essere definito sulla base di test sperimentali e integrato nel firmware come vincolo di sicurezza.

La forza di attrazione deve anche essere integrata con il concetto di hysteresis. In molti sistemi fisici, la soglia necessaria per creare una connessione non coincide con la soglia alla quale la connessione si rompe. Una volta che due MicroBot sono agganciati, può essere necessaria una forza esterna maggiore per separarli rispetto a quella richiesta inizialmente per unirli. Questo comportamento può essere utile perché evita oscillazioni continue tra aggancio e distacco quando i nodi si trovano vicino alla soglia. Il sistema può quindi definire una soglia di attacco e una soglia di distacco distinte, migliorando la stabilità.

Dal punto di vista pratico, la forza di attrazione dovrà essere validata sperimentalmente. Sarà necessario osservare a quale distanza i nodi iniziano ad attrarsi, quale forza è necessaria per separarli, come varia l'aggancio con l'orientamento, quanto influisce la superficie di contatto e quali configurazioni risultano più stabili. Questi dati saranno fondamentali per calibrare il modello matematico e la simulazione. Senza misurazioni, il sistema rimane qualitativo; con misurazioni anche semplici, può diventare progressivamente quantitativo.

Un test iniziale potrebbe consistere nel misurare la distanza massima di attrazione tra due magneti o tra bobina e magnete. Un secondo test potrebbe valutare la forza di distacco usando piccoli pesi o dinamometri. Un terzo test potrebbe confrontare orientamenti diversi. Un quarto test potrebbe analizzare l'effetto del duty cycle sulla capacità di attrazione. Anche se questi test sono semplici, permettono di costruire una base empirica utile per migliorare il progetto.

Nel contesto dello swarm, la forza di attrazione tra due nodi deve essere analizzata anche in relazione alle altre connessioni. Un nodo può essere attratto simultaneamente da più unità, e la forza risultante può dipendere dalla somma vettoriale delle interazioni. Questo può produrre configurazioni desiderate, ma anche effetti inattesi. Se più nodi attraggono lo stesso elemento da direzioni diverse, il sistema può stabilizzarsi in una posizione intermedia oppure oscillare. La simulazione è fondamentale per studiare questi casi prima di realizzarli fisicamente.

La forza di attrazione contribuisce inoltre alla robustezza del sistema. In una struttura composta da molti nodi, la perdita di una singola connessione può non causare il collasso se altre connessioni compensano. Questo significa che la stabilità globale può emergere da una rete di forze relativamente deboli ma distribuite. È un principio importante: non sempre serve una singola forza molto grande; spesso è più efficace una distribuzione di molte interazioni moderate. Questo è coerente con la filosofia MicroBot, in cui la complessità nasce dall'insieme delle unità, non dalla potenza del singolo nodo.

Dal punto di vista del safety layer, la forza di attrazione deve essere controllata per evitare comportamenti pericolosi o indesiderati. Una forza troppo elevata può rendere difficile la separazione manuale, danneggiare componenti, creare urti o bloccare la riconfigurazione. Una forza troppo debole può invece rendere il sistema instabile. Il controllo deve quindi operare all'interno di una fascia utile, definita da limiti inferiori e superiori. Questa fascia rappresenta la zona operativa sicura del sistema magnetico.

Un altro aspetto riguarda la percezione e il feedback. Nei prototipi e nelle simulazioni, la forza di attrazione può essere rappresentata visivamente tramite LED, colori, linee di connessione o indicatori nella dashboard. Un nodo che sta esercitando attrazione può cambiare colore; una connessione magnetica attiva può essere mostrata come linea luminosa; la forza stimata può essere rappresentata con intensità grafica. Questa rappresentazione aiuta a comprendere il comportamento del sistema e collega la fisica reale all'interfaccia utente.

Nel modello dati del sistema, una connessione magnetica potrebbe essere rappresentata come una relazione tra due nodi:

```
connection = (node_a, node_b, force_estimate, state, timestamp)
```

dove `force_estimate` rappresenta la forza stimata o simulata, `state` indica se la connessione è potenziale, attiva, stabile o in distacco, e `timestamp` consente di tracciare l'evoluzione nel tempo. Questa struttura sarebbe utile sia nella simulazione sia nella telemetria, perché permetterebbe di analizzare come si formano e si rompono le connessioni nello swarm.

Dal punto di vista teorico, la forza di attrazione rappresenta uno dei punti in cui il concetto di informazione diventa fisico. Un comando software può modificare la corrente in una bobina; la corrente modifica il campo; il campo modifica la forza; la forza modifica la configurazione. La catena completa è:

informazione → corrente → campo → forza → posizione → forma

Questa catena è centrale per tutto il progetto MicroBot. Essa mostra che la materia programmabile non è una metafora astratta, ma una sequenza di trasformazioni fisiche e computazionali. Il software non cambia direttamente la forma: cambia parametri fisici che modificano le forze locali, e le forze locali producono una nuova configurazione.

In conclusione, la forza di attrazione è il meccanismo attraverso cui il sistema magnetico del MicroBot produce effetti meccanici concreti tra le unità. Essa dipende da campo, distanza, orientamento, geometria, materiali e stato operativo del nodo. Può essere passiva, quando generata da magneti permanenti, oppure attiva e modulabile, quando generata da bobine controllate. La sua progettazione richiede equilibrio tra stabilità e riconfigurabilità, tra forza utile e sicurezza, tra consumo energetico e comportamento emergente. Comprendere la forza di attrazione significa comprendere come il MicroBot passa dal livello elettronico e magnetico al livello strutturale dello swarm. Questa sezione prepara il punto successivo dell'indice, dedicato all'interazione tra dipoli, dove la forza verrà analizzata in modo più specifico considerando orientamento, polarità e configurazioni relative dei campi magnetici.

6.9 Interazione tra dipoli

L'interazione tra dipoli rappresenta il modello fisico fondamentale attraverso cui è possibile descrivere, almeno in prima approssimazione, il comportamento magnetico tra due MicroBot. Dopo aver analizzato la forza di attrazione come effetto macroscopico del sistema magnetico, è necessario entrare nel livello più specifico della relazione tra sorgenti magnetiche orientate. Un magnete permanente, una bobina percorsa da corrente o una combinazione dei due possono infatti essere modellati come dipoli magnetici, cioè come sistemi caratterizzati da un momento magnetico dotato di intensità e direzione. Questa rappresentazione consente di comprendere perché l'interazione magnetica non dipenda soltanto dalla distanza, ma anche dall'orientamento reciproco, dalla polarità, dalla geometria e dalla configurazione spaziale dei nodi.

Nel contesto MicroBot, il modello a dipoli è particolarmente utile perché permette di collegare il comportamento fisico del singolo elemento magnetico alla dinamica collettiva dello swarm. Ogni unità può essere vista come un nodo che possiede una o più sorgenti magnetiche. Queste sorgenti possono essere passive, nel caso dei magneti permanenti, oppure attive, nel caso delle bobine elettromagnetiche. In entrambi i casi, ciò che conta non è soltanto "quanto" campo viene generato, ma anche "in quale direzione" esso viene generato. Il momento magnetico diventa quindi una variabile vettoriale, non una semplice grandezza scalare.

Formalmente, un dipolo magnetico può essere rappresentato tramite un momento magnetico m , che indica sia l'intensità sia l'orientamento della sorgente. Nel caso di una bobina, il momento magnetico può essere approssimato come proporzionale al numero di spire, alla corrente e all'area della spira:

$$m = N \cdot I \cdot A$$

dove N rappresenta il numero di spire, I la corrente e A l'area dell'avvolgimento. Questa relazione mostra che il momento magnetico della bobina cresce aumentando la corrente, aumentando il numero di spire o aumentando l'area effettiva dell'avvolgimento. Nel MicroBot, tuttavia, l'area e il numero di spire sono limitati dalle dimensioni fisiche del nodo, mentre la corrente è limitata dalla batteria, dal MOSFET, dalla dissipazione termica e dal safety layer. Il momento magnetico non è quindi un parametro libero, ma il risultato di un compromesso progettuale.

Nel caso di un magnete permanente, il momento magnetico dipende dal materiale, dal volume del magnete e dalla magnetizzazione interna. Anche in questo caso, il MicroBot non può semplicemente adottare magneti sempre più potenti, perché una forza eccessiva può impedire la riconfigurazione, aumentare il rischio di agganci indesiderati e rendere difficile il distacco. La progettazione deve quindi cercare un momento magnetico compatibile con la scala del sistema e con gli obiettivi funzionali.

L'interazione tra due dipoli dipende dalla loro disposizione reciproca. Se i momenti magnetici sono allineati in modo favorevole, il sistema può generare attrazione e stabilizzazione; se sono orientati in modo sfavorevole, può generare repulsione, rotazione o configurazioni instabili. Questo significa che due MicroBot non

interagiscono semplicemente perché sono vicini: interagiscono in modo diverso a seconda di come sono orientati. La direzione dei poli e la posizione relativa dei nodi diventano quindi parte integrante della dinamica.

Questa proprietà è fondamentale per l'allineamento passivo. Quando due dipoli si avvicinano con orientamento non perfetto, le forze magnetiche possono generare una coppia, cioè un momento torcente, che tende a ruotarli verso una configurazione più stabile. Questo comportamento è estremamente utile nel progetto MicroBot, perché consente ai nodi di correggere piccoli errori di orientamento senza richiedere sensori complessi o controllo attivo continuo. La fisica del dipolo agisce come meccanismo di auto-correzione locale.

Dal punto di vista concettuale, l'interazione tra dipoli può essere interpretata come una ricerca locale di configurazioni a energia minore. Il sistema tende spontaneamente verso disposizioni in cui i campi risultano compatibili. Questo è esattamente ciò che rende i magneti utili in un sistema emergente: non serve calcolare esplicitamente ogni dettaglio dell'allineamento, perché alcune configurazioni sono fisicamente favorite. Tuttavia, questa stessa proprietà può generare vincoli. Se il sistema entra in una configurazione magneticamente stabile ma non desiderata, può restarvi bloccato. Per questo motivo, il controllo attivo del campo e le strategie di distacco diventano importanti nelle versioni avanzate.

Nel MicroBot, l'interazione tra dipoli può essere classificata in diverse configurazioni operative. La prima è la configurazione attrattiva, in cui poli opposti o momenti compatibili favoriscono l'aggancio. La seconda è la configurazione repulsiva, in cui poli uguali o momenti sfavorevoli tendono ad allontanare i nodi. La terza è la configurazione rotazionale, in cui la forza principale non produce immediatamente avvicinamento o allontanamento, ma una coppia che modifica l'orientamento. La quarta è la configurazione neutra o debole, in cui la distanza o l'orientamento riducono l'interazione a valori trascurabili. Queste configurazioni devono essere considerate sia nella progettazione fisica sia nel modello matematico.

Una rappresentazione concettuale dell'interazione può essere scritta come:

$$F_{ij} = \text{funzione}(m_i, m_j, r_{ij}, \theta_{ij})$$

dove m_i e m_j rappresentano i momenti magnetici dei due nodi, r_{ij} la distanza tra essi e θ_{ij} l'orientamento relativo. Questa relazione mostra che la forza non dipende solo dalla distanza, ma anche dall'allineamento dei momenti. In forma ancora più generale, il sistema deve considerare non soltanto la forza, ma anche la coppia:

$$\tau_{ij} = \text{funzione}(m_i, B_j)$$

dove τ_{ij} rappresenta il momento torcente sul dipolo i generato dal campo prodotto dal dipolo j . Questa grandezza è importante perché descrive il comportamento di riallineamento.

Nel sistema MicroBot, la coppia magnetica può essere sfruttata per favorire l'orientamento corretto delle superfici di contatto. Se due nodi si avvicinano con un angolo non perfettamente allineato, la coppia può ruotarli fino a portare le facce magnetiche in una posizione più compatibile. Questo riduce la necessità di precisione meccanica estrema nel movimento o nel posizionamento. In altre parole, il sistema magnetico introduce una forma di tolleranza fisica: non è necessario che i nodi arrivino già perfettamente orientati, perché l'interazione stessa può completarli.

Questa proprietà si collega alla geometria del MicroBot. La configurazione a coni troncati e sfera può aiutare a trasformare la coppia magnetica in allineamento meccanico effettivo. Le superfici inclinate o guidate possono ridurre gli errori di contatto e favorire l'incastro orientato. Il magnetismo fornisce la tendenza fisica, mentre la geometria fornisce la guida meccanica. Il comportamento risultante è quindi magneto-geometrico: non dipende soltanto dal campo, ma dalla combinazione tra campo e forma.

Nel caso di nodi dotati di più magneti o più bobine, l'interazione tra dipoli diventa ancora più complessa. Ogni MicroBot non è più un singolo dipolo ideale, ma un insieme di dipoli distribuiti. Questo può essere utile perché permette di creare configurazioni più ricche, ad esempio nodi che si agganciano soltanto in certi orientamenti o che distinguono tra superfici diverse. Tuttavia, aumenta anche la complessità del modello e il rischio di interazioni indesiderate. Per questo motivo, nelle prime versioni del sistema è opportuno mantenere una configurazione magnetica semplice e comprensibile, per poi aumentare progressivamente la complessità.

Un aspetto importante riguarda l'interazione tra dipoli passivi e dipoli attivi. Un magnete permanente ha momento magnetico fisso; una bobina ha momento magnetico variabile in funzione della corrente. Quando un MicroBot dotato di bobina interagisce con un altro dotato di magnete permanente, il sistema può modulare temporaneamente l'attrazione o la repulsione. Questo crea una struttura molto interessante: un dipolo fisso fornisce riferimento e stabilità, mentre un dipolo controllato fornisce adattabilità. La combinazione dei due consente di ottenere un comportamento più flessibile rispetto a un sistema puramente passivo.

Nel caso di due bobine attive, il sistema può teoricamente coordinare entrambe le correnti per generare configurazioni di attrazione, repulsione o riallineamento. Questo scenario è più potente ma anche più complesso, perché richiede sincronizzazione, controllo della polarità, gestione della corrente e comunicazione tra nodi. In una fase avanzata, due MicroBot potrebbero negoziare o ricevere dal controller una sequenza di attivazione coordinata, in modo da creare un aggancio controllato. In una fase iniziale, invece, è più realistico usare bobine in modo semplice, come rinforzo temporaneo di interazioni passive.

L'interazione tra dipoli è fortemente dipendente dalla distanza. Sebbene la dipendenza esatta possa essere complessa, il comportamento generale è che l'interazione decresce rapidamente all'aumentare della distanza. Questo significa che i dipoli sono efficaci soprattutto a corto raggio. Tale proprietà è coerente con l'idea di interazione locale dello swarm, ma richiede che il sistema consideri soglie operative precise. Un nodo lontano non deve essere trattato come realmente influente; un nodo vicino può invece entrare in una zona di interazione significativa. Questo punto prepara direttamente la sezione successiva sulla dipendenza dalla distanza.

Dal punto di vista della simulazione, modellare i dipoli in modo completo può essere complesso. Tuttavia, per il progetto MicroBot è possibile introdurre un modello progressivo. In una prima fase, ogni nodo può essere rappresentato come un punto con intensità magnetica scalare e una soglia di attrazione. In una seconda fase, si può aggiungere un vettore di orientamento. In una terza fase, si può distinguere tra attrazione, repulsione e coppia di riallineamento. In una quarta fase, si possono introdurre più poli per nodo. Questo approccio incrementale permette di mantenere il simulatore utile senza renderlo subito ingestibile.

Un possibile modello intermedio può associare a ogni MicroBot una posizione p_i e un vettore orientamento d_i . La forza tra due nodi può dipendere dalla distanza tra p_i e p_j e dalla compatibilità tra d_i e d_j . Se gli orientamenti sono compatibili, la forza attrattiva aumenta; se sono incompatibili, la forza diminuisce o diventa repulsiva. Questo tipo di modello è sufficiente per simulare molte dinamiche interessanti, pur restando abbastanza semplice da implementare.

Dal punto di vista del controllo, l'interazione tra dipoli richiede che il sistema distingua tra "vicino" e "allineato". Due nodi vicini ma non allineati potrebbero non essere pronti per l'aggancio stabile. Una strategia intelligente potrebbe prima usare il campo per riallineare e poi aumentare l'attrazione per completare l'aggancio. Questo introduce una sequenza comportamentale:

rilevamento vicinanza → riallineamento → aggancio → stabilizzazione

Questa sequenza è molto coerente con il comportamento fisico del sistema e potrebbe diventare una regola futura nel firmware o nella simulazione.

Nel contesto del safety layer, l'interazione tra dipoli deve essere limitata per evitare comportamenti troppo bruschi. Magneti molto forti o bobine attivate ad alta intensità possono generare movimenti rapidi, urti o connessioni difficili da separare. Il sistema deve quindi operare entro una fascia di forza controllata. Questo è particolarmente importante se il progetto viene usato in contesti dimostrativi o didattici, dove il comportamento deve essere chiaro, sicuro e ripetibile.

Dal punto di vista della documentazione, l'interazione tra dipoli permette di spiegare in modo rigoroso perché la polarità e l'orientamento dei magneti devono essere standardizzati. Se ogni MicroBot viene assemblato con poli non coerenti, il comportamento dello swarm diventa imprevedibile. Alcuni nodi potrebbero attrarsi quando dovrebbero respingersi, oppure rifiutare connessioni corrette. La riproducibilità dell'assemblaggio magnetico è quindi una condizione necessaria per ottenere un sistema coerente.

Questa osservazione si collega anche al versionamento del progetto. Una versione iniziale può usare magneti montati manualmente e testati singolarmente; una versione più avanzata dovrebbe prevedere guide di montaggio, marcatura dei poli, procedure di verifica e documentazione della polarità. Il sistema magnetico

deve quindi evolvere da prototipo artigianale a componente standardizzato. Senza standardizzazione, la scalabilità dello swarm risulterebbe compromessa.

L'interazione tra dipoli può anche essere vista come una forma di comunicazione fisica implicita. Due nodi non si scambiano un pacchetto digitale, ma "sentono" reciprocamente il campo magnetico. Questa interazione modifica il loro comportamento fisico senza passare necessariamente dal controller. In questo senso, il sistema MicroBot possiede due canali di interazione: uno informativo, basato su comunicazione digitale, e uno fisico, basato su campi e forze. Il comportamento emergente nasce dalla combinazione di entrambi.

Questa doppia natura è uno degli aspetti più forti del progetto. La comunicazione digitale permette coordinamento, timestamp, comandi e telemetria; l'interazione dipolare permette aggancio, allineamento, stabilità e configurazione fisica. Il primo canale lavora sul piano dell'informazione; il secondo sul piano della materia. MicroBot diventa interessante proprio perché cerca di unire questi due piani in un unico sistema.

Dal punto di vista teorico avanzato, l'interazione tra dipoli si collega al concetto di forma come risultato di relazioni. La forma dello swarm non è definita soltanto dalle posizioni dei nodi, ma anche dalle relazioni energetiche e magnetiche tra essi. Due configurazioni geometricamente simili possono avere stabilità diversa se le interazioni dipolari sono diverse. Questo significa che la forma non è solo geometria visibile, ma anche struttura invisibile di campi, orientamenti e vincoli.

Nel modello dati del sistema, una connessione magnetica potrebbe quindi includere non solo distanza e forza stimata, ma anche orientamento relativo e compatibilità dipolare. Una rappresentazione possibile potrebbe essere:

```
magnetic_relation = (node_a, node_b, distance, orientation_alignment, polarity_match, force_estimate, torque_estimate, connection_state)
```

Questa struttura permetterebbe alla simulazione o alla telemetria di descrivere meglio il motivo per cui una connessione è stabile, instabile o in fase di riallineamento. Sarebbe molto utile nelle versioni avanzate del software MicroBot.

Un ulteriore aspetto riguarda la possibilità di usare l'interazione dipolare per codificare ruoli o direzioni. Se alcune facce del MicroBot hanno polarità diverse o intensità diverse, il nodo può distinguere implicitamente tra lati preferenziali di aggancio. Questo potrebbe permettere la formazione di strutture orientate, non semplicemente cluster casuali. Tuttavia, questa possibilità richiede una progettazione molto accurata, perché aumenta la complessità dell'assemblaggio e della simulazione.

Per le prime versioni fisiche, una scelta più prudente è definire una configurazione dipolare semplice, testabile e ripetibile. Per esempio, un asse magnetico principale potrebbe essere sufficiente per validare catene lineari, agganci frontali e allineamento base. Successivamente, si potranno aggiungere più poli o configurazioni più complesse per ottenere pattern tridimensionali o strutture più articolate. Questo approccio mantiene coerenza con lo sviluppo incrementale del progetto.

Dal punto di vista sperimentale, l'interazione tra dipoli può essere studiata con test semplici ma molto utili. Si possono misurare orientamenti preferiti, distanza di attrazione, forza di distacco, comportamento con poli invertiti, effetto di piccoli angoli di disallineamento e stabilità della connessione sotto perturbazioni. Questi test permetteranno di passare da descrizione qualitativa a modellazione più precisa. Ogni dato raccolto potrà essere usato per calibrare la simulazione e migliorare la documentazione.

In conclusione, l'interazione tra dipoli costituisce il modello fisico fondamentale per comprendere l'orientamento, l'attrazione, la repulsione e il riallineamento tra MicroBot. Essa mostra che il sistema magnetico non dipende soltanto dall'intensità del campo, ma anche dalla direzione dei momenti magnetici, dalla polarità, dalla distanza e dalla geometria. Nel progetto MicroBot, questa interazione permette di trasformare il magnetismo in una regola fisica locale capace di sostenere comportamento emergente. I dipoli forniscono al sistema una forma di intelligenza materiale: non calcolano, ma orientano; non comunicano digitalmente, ma influenzano; non impongono una forma, ma rendono alcune configurazioni più stabili di altre. Questa sezione prepara il punto successivo dell'indice, dedicato alla dipendenza dalla distanza, dove verrà analizzato in modo specifico come l'intensità dell'interazione magnetica cambia con la separazione tra i nodi e quali soglie operative ne derivano per il comportamento dello swarm.

6.10 Dipendenza dalla distanza

La dipendenza dalla distanza rappresenta uno degli aspetti più importanti del sistema magnetico MicroBot, perché definisce il raggio effettivo entro cui le unità possono influenzarsi fisicamente. Dopo aver analizzato l'interazione tra dipoli, diventa necessario comprendere che la forza magnetica non agisce in modo uniforme nello spazio, ma decresce rapidamente all'aumentare della separazione tra i nodi. Questa proprietà ha conseguenze fondamentali sulla progettazione del sistema, sulla simulazione dello swarm, sulle soglie di aggancio, sulla stabilità delle configurazioni e sulla futura capacità di riconfigurazione del sistema.

Nel progetto MicroBot, la distanza tra due unità non è un semplice parametro geometrico, ma una variabile operativa che determina se l'interazione magnetica è significativa, debole o trascurabile. Due MicroBot molto vicini possono attrarsi, riallinearsi e agganciarsi; due MicroBot troppo lontani, anche se dotati di magneti o bobine, possono non esercitare alcuna influenza utile l'uno sull'altro. Questo significa che il sistema magnetico introduce naturalmente una logica di prossimità: il comportamento fisico dello swarm nasce principalmente da interazioni locali, non da forze a lungo raggio.

Questa caratteristica è perfettamente coerente con il paradigma dei sistemi distribuiti e della swarm robotics. In uno swarm, ogni nodo non deve necessariamente interagire con tutti gli altri nodi, ma solo con quelli che si trovano nel proprio vicinato operativo. La distanza diventa quindi il criterio attraverso cui si definisce il vicinato fisico del MicroBot. Se due nodi sono abbastanza vicini, possono entrare in una relazione magnetica; se sono lontani, vengono considerati indipendenti dal punto di vista fisico. Questa proprietà riduce la complessità del sistema, perché limita il numero di interazioni rilevanti in ogni istante.

Dal punto di vista fisico, il campo magnetico generato da un dipolo decresce rapidamente con la distanza. Senza entrare in una modellazione troppo pesante in questa fase, è sufficiente stabilire che il magnetismo utile per il MicroBot è fortemente locale. Nel caso di dipoli magnetici, l'intensità del campo e delle forze associate tende a diminuire con potenze elevate della distanza. Questo significa che piccoli aumenti della separazione possono produrre grandi riduzioni dell'interazione. Per esempio, raddoppiare la distanza tra due elementi magnetici può ridurre drasticamente la forza disponibile, rendendo l'aggancio molto meno probabile.

Questa rapidità di decadimento ha una conseguenza progettuale decisiva: il sistema MicroBot deve essere progettato affinché le superfici magnetiche arrivino abbastanza vicine da entrare nella zona utile di interazione. Non basta inserire magneti o bobine nel nodo; bisogna anche garantire che la geometria, la disposizione interna e la meccanica del contatto permettano di ridurre la distanza effettiva tra sorgente magnetica e unità vicina. Se il magnete o la bobina sono troppo interni al corpo del MicroBot, la distanza reale dal componente dell'altro nodo aumenta e la forza diminuisce. Per questo motivo, la posizione dei componenti magnetici deve essere progettata insieme alla geometria esterna.

Nel caso della configurazione a coni troncati e sfera, le estremità del MicroBot possono essere interpretate come zone preferenziali di interazione. Collocare magneti o bobine vicino a queste superfici permette di ridurre la distanza tra sorgenti magnetiche durante il contatto. La geometria, quindi, non ha solo valore strutturale, ma anche elettromagnetico: determina quanto i campi possono essere efficaci nella zona di aggancio. Una geometria mal progettata potrebbe aumentare inutilmente la distanza tra i componenti magnetici, rendendo debole l'interazione anche se i magneti o le bobine sono teoricamente sufficienti.

Dal punto di vista operativo, è utile introdurre il concetto di raggio di influenza magnetica. Questo raggio non deve essere inteso come un confine fisico netto, ma come una soglia pratica oltre la quale l'effetto magnetico diventa troppo debole per produrre un comportamento utile. In forma concettuale, si può distinguere tra tre zone:

zona lontana, in cui l'interazione è trascurabile;

zona intermedia, in cui il campo è percepibile ma non sufficiente a garantire aggancio stabile;

zona vicina, in cui la forza supera le resistenze locali e può produrre attrazione, riallineamento o connessione.

Questa tripartizione è estremamente utile per la simulazione e per il firmware. In un modello computazionale, non ha senso calcolare interazioni magnetiche complete tra tutti i nodi se molti sono troppo lontani. Si può definire una soglia di vicinanza oltre la quale l'interazione viene ignorata o fortemente semplificata. Questo permette di rendere la simulazione più efficiente e più coerente con la natura locale del sistema fisico.

Una rappresentazione concettuale della forza in funzione della distanza può essere:

$$F(r) = K \cdot g(r)$$

dove $F(r)$ rappresenta la forza magnetica alla distanza r , K rappresenta un coefficiente legato all'intensità del sistema magnetico e $g(r)$ rappresenta una funzione di decadimento. La funzione $g(r)$ deve diminuire rapidamente all'aumentare di r . In una simulazione iniziale, non è necessario usare una formula fisica complessa; può essere sufficiente una funzione semplificata che riduce progressivamente l'interazione con la distanza e la annulla oltre una certa soglia. L'importante è mantenere il principio: vicino significa influente, lontano significa quasi indipendente.

La dipendenza dalla distanza introduce direttamente il concetto di soglia di attacco. La soglia di attacco è la distanza entro cui due MicroBot iniziano a considerarsi potenzialmente agganciabili. Se la distanza tra due nodi scende sotto tale soglia e l'orientamento è compatibile, il sistema può attivare una logica di aggancio, visualizzare una connessione nella simulazione o eventualmente rinforzare l'interazione tramite bobine. Questa soglia non è puramente geometrica, ma dipende dalla forza magnetica disponibile, dall'attrito, dalla massa del nodo, dall'orientamento e dalla superficie di contatto.

Allo stesso modo, la distanza introduce il concetto di soglia di distacco. Una volta che due MicroBot sono agganciati, la connessione può persistere anche se la distanza aumenta leggermente o se vi sono piccole perturbazioni. La soglia di distacco può quindi essere diversa dalla soglia di attacco. Questa distinzione è alla base del meccanismo di hysteresis, che verrà approfondito nella Sezione 7. In termini pratici, il sistema evita di oscillare continuamente tra stato agganciato e stato non agganciato quando i nodi si trovano vicino al limite. Questo comportamento rende lo swarm più stabile.

La differenza tra soglia di attacco e soglia di distacco è molto importante per rappresentare un sistema fisico realistico. Se si usasse una sola soglia, due nodi potrebbero connettersi e disconnettersi ripetutamente a causa di piccole variazioni di posizione, generando instabilità nel modello. Invece, definendo due soglie distinte, il sistema introduce una memoria dello stato precedente. Una connessione già formata tende a rimanere stabile entro un certo margine, mentre una connessione non ancora formata richiede condizioni più favorevoli per nascere. Questo è coerente con il comportamento reale dei magneti.

Dal punto di vista fisico, la distanza effettiva tra due elementi magnetici non coincide sempre con la distanza tra i centri geometrici dei MicroBot. Questa distinzione è fondamentale. Due nodi possono avere centri relativamente distanti ma superfici magnetiche vicine, oppure possono avere centri vicini ma componenti magnetici interni lontani dalla zona di contatto. Per questo motivo, nel modello tecnico è opportuno distinguere tra distanza centro-centro e distanza funzionale di interazione. La seconda è quella realmente rilevante per il sistema magnetico.

In una simulazione semplificata, si può iniziare usando la distanza tra i centri dei nodi, perché è facile da calcolare. Tuttavia, in una versione più avanzata, il modello dovrebbe considerare punti di aggancio specifici sulla superficie del MicroBot. Ogni nodo potrebbe avere una o più porte magnetiche, ciascuna con posizione, orientamento e stato. La distanza utile sarebbe allora calcolata tra porte magnetiche, non tra centri. Questo renderebbe il modello più coerente con la struttura fisica reale.

Un possibile modello avanzato potrebbe rappresentare ogni MicroBot come:

nodo = (posizione, orientamento, porte_magnetiche, stato)

dove ogni porta magnetica possiede:

porta = (posizione_locale, orientamento_locale, polarità, intensità, stato_attivo)

In questo modo, la distanza tra due porte diventa il criterio per determinare l'interazione. Questa struttura sarebbe molto utile nelle versioni future del simulatore, perché permetterebbe di rappresentare agganci direzionali, connessioni multiple e configurazioni geometriche più realistiche.

La dipendenza dalla distanza è influenzata anche dall'orientamento. Due sorgenti magnetiche vicine possono non attrarsi in modo efficace se gli assi magnetici non sono compatibili. Per questo motivo, la distanza non deve essere considerata da sola. Una relazione più corretta può essere espressa come:

interazione = funzione(distanza, orientamento, polarità, intensità)

Questa relazione mostra che la distanza definisce la possibilità dell'interazione, mentre orientamento e polarità ne determinano la qualità. Un nodo vicino ma orientato male può generare una coppia di riallineamento invece di un aggancio diretto. Un nodo vicino con polarità incompatibile può generare repulsione. Un nodo lontano, anche se perfettamente orientato, può non produrre alcun effetto utile.

Nel contesto del MicroBot, la distanza agisce quindi come filtro primario. Prima si verifica se i nodi sono abbastanza vicini; poi si valuta se sono orientati correttamente; poi si stabilisce se la connessione è possibile, instabile o stabile. Questa logica può diventare una regola di simulazione e controllo:

se distanza > $r_{\text{influenza}}$ → nessuna interazione significativa;

se distanza ≤ $r_{\text{influenza}}$ ma distanza > r_{attacco} → interazione debole o avvicinamento;

se distanza ≤ r_{attacco} e orientamento compatibile → aggancio possibile;

se connessione attiva e distanza ≤ r_{distacco} → connessione mantenuta;

se connessione attiva e distanza > r_{distacco} → connessione persa.

Questa sequenza sarà molto utile quando si arriverà alla Sezione 7, perché permette di trasformare il comportamento magnetico in un modello matematico discreto. Il sistema fisico fornisce la base; il modello matematico formalizza le condizioni.

Dal punto di vista del controllo attivo, la distanza può determinare quando conviene attivare le bobine. Una bobina consumerebbe energia inutilmente se attivata quando il nodo bersaglio è troppo lontano. Il firmware o il controller dovrebbero quindi evitare attivazioni magnetiche fuori dalla zona utile. In una versione avanzata, il sistema potrebbe attivare la bobina solo quando la distanza stimata è inferiore a una soglia, riducendo consumi e calore. Questo collega la dipendenza dalla distanza alla gestione energetica.

Una strategia intelligente potrebbe essere questa: usare la simulazione o sensori esterni per rilevare nodi vicini; attivare il campo solo quando la vicinanza supera una soglia; mantenere l'attivazione per un tempo limitato; verificare se l'aggancio è avvenuto; disattivare o ridurre il campo una volta raggiunta la connessione. In questo modo, il sistema non spreca energia per tentare interazioni fisicamente improbabili. La distanza diventa quindi una condizione di abilitazione del comando magnetico.

Dal punto di vista energetico, la dipendenza dalla distanza è anche un criterio di efficienza. Se due nodi sono lontani, aumentare moltissimo la corrente nella bobina per ottenere attrazione può essere inefficiente e pericoloso. È spesso più sensato ridurre prima la distanza con altri meccanismi, oppure progettare la geometria affinché i nodi entrino in contatto passivamente, e solo dopo usare il campo attivo per aggancio o stabilizzazione. Questa logica evita di usare potenza elettrica per compensare problemi geometrici o meccanici.

Dal punto di vista del comportamento emergente, la dipendenza dalla distanza crea naturalmente cluster locali. Nodi vicini tendono a interagire più fortemente tra loro rispetto a nodi lontani. Questo può portare alla formazione di gruppi, catene o strutture compatte. Se il sistema è progettato bene, questi cluster possono diventare moduli intermedi dello swarm. Se invece il sistema è mal bilanciato, possono formarsi aggregazioni indesiderate o blocchi difficili da riconfigurare. La distanza, quindi, influenza direttamente la topologia dello swarm.

La topologia dello swarm può essere vista come un grafo in cui i nodi sono MicroBot e gli archi rappresentano connessioni magnetiche o potenziali interazioni. La distanza determina quali archi possono esistere. Se due nodi sono troppo lontani, non c'è arco; se sono vicini, l'arco può essere potenziale; se sono agganciati, l'arco diventa attivo. In forma concettuale:

$G(t) = (V, E(t))$

dove V è l'insieme dei MicroBot ed $E(t)$ è l'insieme delle connessioni dipendenti dalla distanza e dallo stato magnetico. Questo modello collega il sistema magnetico alla teoria dei grafi e alla rappresentazione dello swarm come rete dinamica.

La distanza influisce anche sulla robustezza. Una struttura in cui i nodi sono collegati solo da connessioni al limite della soglia può essere fragile, perché piccole perturbazioni possono rompere i legami. Una struttura in cui i nodi sono entro una distanza più favorevole risulta più stabile. Questo significa che il sistema può valutare la qualità di una configurazione anche in base ai margini di distanza tra nodi connessi. Non basta sapere se due nodi sono collegati; è utile sapere quanto sono vicini al limite di distacco.

Nel modello dati, una connessione magnetica potrebbe quindi includere una misura di margine:

$\text{connection_margin} = r_{\text{distacco}} - \text{distanza_attuale}$

Se il margine è alto, la connessione è stabile; se è vicino a zero, la connessione è fragile. Questo dato potrebbe essere mostrato in dashboard, usato nella simulazione o integrato nel controllo. Sarebbe particolarmente utile per capire quali parti dello swarm sono stabili e quali rischiano di separarsi.

Dal punto di vista sperimentale, la dipendenza dalla distanza richiede test specifici. Sarà necessario misurare a quale distanza inizia l'attrazione percepibile, a quale distanza avviene l'aggancio effettivo, a quale distanza la connessione si rompe e come questi valori cambiano con orientamento, duty cycle, tipo di magnete, numero di spire e geometria del nodo. Questi dati saranno fondamentali per definire soglie realistiche nel modello matematico.

Un test iniziale può essere molto semplice: posizionare due unità o due componenti magnetici a distanze crescenti e osservare quando l'attrazione riesce a vincere l'attrito. Un secondo test può misurare la forza di distacco a contatto. Un terzo test può valutare come cambia il comportamento se tra i magneti è presente il corpo del MicroBot, quindi una distanza strutturale aggiuntiva. Un quarto test può confrontare bobina spenta, bobina attiva a vari duty cycle e magnete permanente. Queste prove permetterebbero di trasformare la dipendenza dalla distanza da concetto teorico a dato ingegneristico.

La distanza è anche importante nella comunicazione tra sistema reale e simulazione. Se il simulatore usa soglie troppo grandi, mostrerà agganci facili che nella realtà non avverranno. Se usa soglie troppo piccole, sottostimerà la capacità del sistema. Il modello deve quindi essere calibrato progressivamente sui test reali. Questo è un punto metodologico fondamentale: la simulazione non deve essere fantasia grafica, ma rappresentazione semplificata di un comportamento fisico misurabile.

Dal punto di vista del firmware, se il nodo non dispone di sensori di distanza reali, la dipendenza dalla distanza può essere gestita inizialmente dalla simulazione o dal controller. Nei prototipi fisici semplici, la distanza può essere controllata manualmente durante i test. In versioni future, sensori di prossimità, contatto, corrente o variazioni del campo potrebbero permettere al nodo di stimare la vicinanza di altri MicroBot. Questo aprirebbe la strada a un controllo magnetico più autonomo.

Un possibile approccio futuro è usare variazioni nella risposta elettrica della bobina per inferire la presenza di materiale magnetico vicino. In alcuni sistemi elettromagnetici, la presenza di un oggetto ferromagnetico modifica l'induttanza o la risposta del circuito. Anche se questa idea è più avanzata e richiede test specifici, potrebbe diventare una direzione interessante per trasformare la bobina non solo in attuatore, ma anche in sensore indiretto di prossimità. Per ora, però, è corretto trattarla come sviluppo futuro, non come funzione già presente.

Dal punto di vista della sicurezza, la distanza deve essere usata per evitare attivazioni inutili o aggressive. Se il sistema non ha conferma della vicinanza di un altro nodo, può limitare l'intensità del campo. Se invece una connessione è già stabile, può ridurre il campo attivo e lasciare il mantenimento ai magneti permanenti. Questo riduce consumo, calore e rischio di movimenti improvvisi. La distanza diventa quindi anche una variabile di sicurezza.

Un altro aspetto importante riguarda la scala dei MicroBot. Se le unità diventano più piccole, anche le distanze operative diminuiscono e le tolleranze diventano più severe. Un errore di pochi millimetri può essere trascurabile in un prototipo grande, ma enorme in un sistema miniaturizzato. Questo significa che la dipendenza dalla distanza diventa sempre più critica man mano che il sistema si avvicina alla scala micro.

La progettazione deve quindi considerare non solo la distanza assoluta, ma la distanza relativa rispetto alle dimensioni del nodo.

Dal punto di vista teorico avanzato, la dipendenza dalla distanza rafforza l'idea di forma come risultato di relazioni locali. La forma globale dello swarm non viene definita da un campo unico che agisce ovunque allo stesso modo, ma da una rete di interazioni che esistono solo entro certe distanze. La configurazione finale dipende da quali nodi entrano nel raggio di influenza di quali altri nodi. La forma, quindi, è anche una proprietà topologica: dipende da connessioni che appaiono e scompaiono in funzione della distanza.

Questa idea si collega direttamente al concetto di sistema come processo. Lo swarm non è una struttura fissa; è una rete dinamica in cui i nodi si avvicinano, entrano in relazione, si agganciano, si separano e formano nuove connessioni. La distanza è la variabile che regola l'accesso a queste relazioni. In un certo senso, la distanza decide quando un altro nodo diventa "rilevante" per il sistema locale.

Nel contesto della documentazione, questa sezione è importante perché prepara il terreno sia per il dimensionamento delle coil sia per il modello matematico. Prima di calcolare una bobina, bisogna sapere a quale distanza si vuole ottenere un effetto. Una bobina progettata per attrarre a 2 mm può essere molto diversa da una progettata per attrarre a 2 cm. Senza specificare la distanza operativa, il dimensionamento magnetico resta vago. La domanda corretta non è soltanto "quanto campo genera la bobina?", ma "quanto campo utile genera alla distanza richiesta?".

Questo porta a una definizione più ingegneristica del requisito magnetico:

il sistema magnetico deve generare una forza sufficiente a superare le resistenze locali entro una distanza operativa definita.

Questa frase è centrale. Significa che forza, distanza e resistenza devono essere progettate insieme. Non basta avere un magnete potente; bisogna avere una forza utile nel punto giusto, alla distanza giusta, per il comportamento giusto.

In conclusione, la dipendenza dalla distanza è un principio fondamentale del sistema magnetico MicroBot. Essa determina il raggio utile delle interazioni, definisce il vicinato fisico dei nodi, influenza le soglie di aggancio e distacco, guida la formazione del grafo dello swarm e impone vincoli concreti sul posizionamento di magneti e bobine. Il magnetismo del MicroBot è locale, rapido nel decadere e fortemente sensibile alla geometria; proprio per questo deve essere modellato con soglie, margini e funzioni di decadimento. Comprendere la dipendenza dalla distanza significa comprendere quando due MicroBot possono realmente interagire, quando una connessione può nascere, quando può stabilizzarsi e quando invece deve essere considerata fisicamente irrilevante. Questa sezione prepara il punto successivo dell'indice, dedicato al dimensionamento delle coil, dove i concetti di corrente, spire, campo, energia, forza e distanza verranno integrati in una logica progettuale più concreta per definire bobine compatibili con i vincoli del sistema.

6.11 Dimensionamento delle coil

Il dimensionamento delle coil rappresenta il punto conclusivo della Sezione 6 e costituisce il passaggio in cui tutti i concetti precedenti vengono integrati in una logica progettuale concreta. Dopo aver analizzato magneti permanenti, allineamento passivo, bobine elettromagnetiche, controllo attivo del campo, relazione tra corrente, spire e campo, energia magnetica, forza di attrazione, interazione tra dipoli e dipendenza dalla distanza, è ora necessario definire come una bobina possa essere progettata in modo coerente con i vincoli reali del sistema MicroBot. Il dimensionamento non consiste semplicemente nello scegliere un numero elevato di spire o una corrente elevata, ma nel trovare un equilibrio tra campo utile, consumo energetico, temperatura, volume disponibile, tensione della batteria, capacità del MOSFET, sicurezza elettrica e obiettivo funzionale dell'interazione.

Nel progetto MicroBot, una coil deve essere progettata partendo da una domanda precisa: quale effetto fisico deve produrre? Una bobina non deve essere dimensionata in astratto, ma in funzione del comportamento richiesto. Se l'obiettivo è soltanto segnalare o simulare uno stato, può bastare un carico simbolico o un LED. Se l'obiettivo è produrre una debole attrazione a distanza molto ravvicinata, la bobina può essere piccola e attivata con impulsi brevi. Se invece l'obiettivo è contribuire realmente all'aggancio o al distacco tra due unità, la bobina deve generare un campo sufficiente a produrre una forza utile nella zona di interazione. Il requisito fisico precede quindi il calcolo elettrico.

Il primo parametro da definire è la distanza operativa. Come discusso nella sezione precedente, il campo magnetico e la forza associata decrescono rapidamente con la distanza. Di conseguenza, non ha senso dimensionare una coil senza specificare a quale distanza deve agire. Una bobina progettata per influenzare un magnete a pochi millimetri richiede requisiti molto diversi rispetto a una bobina progettata per agire a distanze maggiori. Nel caso MicroBot, dato il vincolo di miniaturizzazione, è ragionevole pensare a interazioni a corto raggio, concentrate nelle zone di contatto o pre-contatto tra unità. Questo significa che la bobina deve essere collocata il più vicino possibile alla superficie di interazione, riducendo la distanza tra il centro del campo e l'altro nodo.

Il secondo parametro da definire è la forza minima richiesta. La coil deve produrre un effetto magnetico capace di superare le resistenze locali del sistema, come attrito, peso relativo, imperfezioni geometriche e disallineamenti. In forma concettuale, il requisito può essere espresso come:

$$F_{\text{magnetica}} \geq F_{\text{resistenza}} + \text{margine}_{\text{sicurezza}}$$

Questa relazione indica che la forza magnetica utile deve essere superiore alla somma delle forze che impediscono l'avvicinamento o l'aggancio. Il margine di sicurezza è importante perché nella realtà il sistema non opera in condizioni ideali: la batteria può essere parzialmente scarica, le superfici possono non essere perfette, l'orientamento può non essere ottimale e la bobina può produrre un campo leggermente diverso da quello previsto. Un dimensionamento senza margine rischia di funzionare solo in condizioni di laboratorio molto favorevoli.

Il terzo parametro è il volume disponibile. Una coil occupa spazio, e nel MicroBot lo spazio è una risorsa critica. La struttura interna deve contenere microcontrollore, batteria, regolatori, MOSFET, protezioni, LED,

eventuali sensori e cablaggio. La bobina deve quindi essere progettata come parte dell'intero layout interno, non come componente indipendente. In particolare, se la geometria del nodo prevede una parte centrale sferica e due estremità a cono troncato, la coil dovrebbe essere collocata preferibilmente nelle zone terminali, vicino alle superfici di aggancio, mentre la parte centrale dovrebbe restare dedicata ai componenti elettronici principali e alla batteria. Questa distribuzione riduce la distanza utile e migliora l'efficienza dell'interazione.

Il quarto parametro è il numero di spire. Aumentare il numero di spire aumenta, a parità di corrente, il campo magnetico teoricamente generato dalla bobina. Tuttavia, più spire significano più filo, quindi maggiore resistenza elettrica, maggiore ingombro e possibile aumento della dissipazione termica. La relazione semplificata:

$$B = \mu_0 \cdot \mu_r \cdot (N / L) \cdot I$$

mostra che il campo cresce con N e con I, ma non dice da sola quale sia la scelta migliore. Nel MicroBot, aumentare N può essere utile per ridurre la corrente necessaria, ma può diventare controproducente se la resistenza cresce troppo e limita il flusso di corrente disponibile dalla batteria. Il numero di spire deve quindi essere scelto insieme al diametro del filo, alla tensione disponibile e alla durata prevista degli impulsi.

Il quinto parametro è il diametro del filo. Un filo più sottile consente di realizzare più spire nello stesso volume, ma presenta una resistenza più alta e sopporta meno corrente. Un filo più spesso presenta resistenza minore e può sopportare correnti maggiori, ma occupa più spazio e riduce il numero di spire possibili. Questa scelta produce un compromesso fondamentale: molte spire con poca corrente oppure poche spire con più corrente. Nel MicroBot, dove energia e temperatura sono vincoli forti, la soluzione migliore non è necessariamente quella con il campo massimo, ma quella con il miglior rapporto tra effetto magnetico utile e costo energetico.

Il sesto parametro è la resistenza della coil. La resistenza determina quanta corrente attraversa la bobina a una certa tensione, secondo una forma semplificata della legge di Ohm:

$$I = V / R$$

Tuttavia, nel caso reale con PWM, induttanza e transistori, questa relazione rappresenta solo un'approssimazione statica. Rimane comunque utile per capire il limite generale del circuito. Una resistenza troppo bassa può portare a correnti elevate, stressando batteria, MOSFET e bobina. Una resistenza troppo

alta può limitare troppo la corrente, generando un campo debole. Il dimensionamento deve quindi individuare un intervallo di resistenza compatibile con la tensione della batteria Li-Po, con la capacità del driver MOSFET e con la corrente massima sicura.

Il settimo parametro è l'induttanza. L'induttanza determina quanta energia può essere immagazzinata nel campo magnetico e quanto rapidamente la corrente cresce o decade. La relazione:

$$E = 1/2 \cdot L \cdot I^2$$

mostra che una maggiore induttanza permette di immagazzinare più energia a parità di corrente, ma la costante di tempo:

$$\tau = L / R$$

mostra anche che la risposta può diventare più lenta. Nel MicroBot, la bobina deve essere abbastanza efficace da produrre un campo utile, ma non così lenta da rendere imprecisa la temporizzazione degli impulsi magnetici. Se il sistema deve realizzare agganci o rilasci coordinati, la risposta temporale della coil diventa importante quanto l'intensità del campo.

L'ottavo parametro è il tipo di nucleo. Una bobina in aria è più semplice, leggera e priva di saturazione ferromagnetica significativa, ma produce un campo meno concentrato. Una bobina con nucleo ferromagnetico può aumentare il campo utile, concentrando il flusso magnetico, ma introduce peso, volume, possibile saturazione, isteresi magnetica e maggiore complessità costruttiva. Nel MicroBot, l'uso di un nucleo può essere vantaggioso se permette di ridurre la corrente necessaria, ma deve essere valutato rispetto al peso e allo spazio. In una fase iniziale, una bobina semplice può essere più adatta per test controllati; in una fase successiva, nuclei piccoli e ben posizionati possono migliorare l'efficienza.

Il nono parametro è la durata di attivazione. Una coil può essere progettata per funzionamento continuo oppure per impulsi brevi. Nel MicroBot, il funzionamento continuo è generalmente sfavorevole, perché consuma energia e produce calore. È più coerente progettare le coil per impulsi controllati, utilizzati durante aggancio, riallineamento, distacco o stabilizzazione temporanea. Questo cambia completamente il dimensionamento: una bobina può sopportare correnti più alte per tempi brevi rispetto a correnti permanenti. Tuttavia, il firmware deve garantire che tali impulsi restino entro limiti sicuri.

La durata di attivazione deve essere collegata al duty cycle. Se il microcontrollore usa PWM, il duty cycle determina la frazione di tempo in cui la bobina riceve energia durante ogni periodo. Tuttavia, oltre al duty cycle del PWM esiste anche un duty cycle operativo più ampio: per quanto tempo, nel ciclo globale del sistema, la bobina viene usata. Per esempio, una bobina potrebbe essere pilotata al 50% per 200 ms e poi rimanere spenta per diversi secondi. Questo comportamento produce un carico termico molto diverso rispetto a una bobina mantenuta al 50% continuamente. Il dimensionamento deve quindi considerare sia il duty cycle elettrico sia il duty cycle operativo.

Il decimo parametro è il limite termico. La potenza dissipata dalla coil è:

$$P = R \cdot I^2$$

e questa potenza si trasforma in calore. In un nodo piccolo, il calore può accumularsi rapidamente. Il dimensionamento della coil deve quindi definire una zona operativa sicura, nella quale la temperatura resti compatibile con i materiali, la batteria e l'elettronica vicina. In assenza di sensori termici reali, il firmware può usare una stima conservativa del carico termico, aumentando una variabile interna a ogni attivazione e riducendola progressivamente durante il tempo di riposo. Questa strategia permette di introdurre un primo safety layer anche nelle fasi iniziali.

Un possibile modello operativo può essere:

$$\text{thermal_load}(t+1) = \text{thermal_load}(t) + k_{\text{heat}} \cdot \text{duty_cycle} \cdot \text{durata}$$

durante l'attivazione, e:

$$\text{thermal_load}(t+1) = \text{thermal_load}(t) - k_{\text{cool}} \cdot \Delta t$$

durante il raffreddamento. Anche se semplificato, questo modello permette al firmware di impedire attivazioni troppo frequenti. Quando `thermal_load` supera una soglia, la bobina entra in stato di cooldown o safety lock. Questo approccio è coerente con la filosofia MicroBot: ogni funzione fisica deve essere accompagnata da una logica di sicurezza locale.

L'undicesimo parametro è il driver di potenza. La coil non può essere collegata direttamente all'ESP32. Come definito nella Sezione 5, il MOSFET deve essere scelto in modo compatibile con la corrente della bobina, la tensione della batteria, la frequenza PWM e la dissipazione termica. Il MOSFET deve essere logic-level, cioè controllabile correttamente con i 3.3 V del microcontrollore. Inoltre, deve avere una resistenza `R_DS(on)` sufficientemente bassa per non dissipare troppa potenza. Se il MOSFET è sottodimensionato, anche una coil correttamente progettata può causare instabilità o surriscaldamento.

Il dodicesimo parametro è la protezione elettrica. Ogni coil, essendo un carico induttivo, deve avere un percorso sicuro per il rilascio dell'energia quando viene spenta. Il diodo di flyback rappresenta la soluzione minima per proteggere il circuito dai picchi di tensione. In versioni più avanzate, possono essere usati clamp più rapidi o protezioni più sofisticate. Il dimensionamento della coil deve quindi includere anche il dimensionamento del sistema di protezione: corrente del diodo, tensione supportata, velocità di risposta e dissipazione. Una bobina più potente richiede protezioni più robuste.

Il tredicesimo parametro è la compatibilità con la batteria. Una Li-Po piccola può fornire correnti significative, ma non illimitate. Se la coil richiede correnti troppo alte, la tensione può cadere, il regolatore può entrare in instabilità, l'ESP32 può resettarsi o la batteria può essere stressata. Il dimensionamento della coil deve quindi rispettare la capacità della batteria e la capacità del sistema di alimentazione di sostenere picchi di carico. In pratica, la coil deve essere progettata non solo per generare campo, ma per non compromettere il resto del nodo.

Il quattordicesimo parametro è la ripetibilità costruttiva. Una coil avvolta manualmente può variare molto da un esemplare all'altro. Numero di spire, tensione dell'avvolgimento, diametro effettivo, resistenza finale e posizione nel nodo possono cambiare. In uno swarm, questa variabilità può produrre nodi con forza magnetica diversa. Per questo motivo, il dimensionamento deve includere procedure di misura e verifica. Ogni coil dovrebbe essere caratterizzata almeno tramite resistenza misurata, numero di spire stimato e test di risposta. In versioni più mature, la produzione dovrebbe usare supporti o guide per rendere l'avvolgimento ripetibile.

Il quindicesimo parametro è la strategia di integrazione con i magneti permanenti. Una coil può essere progettata come unico elemento magnetico attivo, oppure come rinforzo di un magnete permanente. La seconda soluzione è spesso più efficiente. Il magnete permanente fornisce attrazione e mantenimento passivo; la coil fornisce impulso, correzione o modulazione temporanea. In questo caso, la coil non deve necessariamente generare da sola tutta la forza di aggancio, ma solo modificare una condizione già vicina alla soglia. Questo riduce i requisiti energetici e rende il sistema più realistico.

In una configurazione ibrida, il dimensionamento può partire da questa logica:

magnete permanente = forza di mantenimento base;

coil = forza temporanea aggiuntiva o correttiva;

firmware = controllo della durata e dell'intensità;

safety layer = limite termico ed energetico.

Questo schema è molto coerente con l'architettura generale del progetto, perché distribuisce le responsabilità tra fisica passiva, elettronica attiva e controllo software.

Dal punto di vista della simulazione, il dimensionamento delle coil può essere rappresentato inizialmente tramite parametri astratti. Ogni nodo può avere un valore `magnetic_strength_max`, un valore `coil_response_time`, un valore `thermal_limit` e un valore `energy_cost_per_activation`. Questi parametri permettono di simulare il comportamento senza entrare subito nei dettagli fisici. Successivamente, quando

saranno disponibili dati sperimentali, i parametri potranno essere calibrati. Questo permette di mantenere coerenza tra modello virtuale e prototipo reale.

Un esempio di stato simulativo potrebbe essere:

```
coil = {  
max_field: valore_normalizzato,  
response_time: valore,  
energy_cost: valore,  
heat_rate: valore,  
cooldown_rate: valore,  
max_duty_cycle: valore  
}
```

Questa struttura consente alla simulazione di considerare non solo la forza generata, ma anche costo energetico e limiti termici. Il comportamento dello swarm diventa così più realistico, perché le bobine non possono essere usate senza conseguenze.

Dal punto di vista della procedura progettuale, il dimensionamento delle coil dovrebbe seguire una sequenza ordinata. Prima si definisce l'obiettivo fisico: aggancio, rinforzo, distacco o riallineamento. Poi si definisce la distanza operativa. Successivamente si stima la forza minima necessaria. Poi si sceglie una geometria compatibile con il nodo. Dopo si selezionano filo, numero di spire e possibile nucleo. Quindi si verifica la resistenza, si stima la corrente, si valuta la potenza dissipata, si controlla la compatibilità con MOSFET e batteria, si definiscono limiti firmware e infine si esegue una prova sperimentale. Questa sequenza evita l'errore di partire dal componente senza aver definito il requisito.

Una possibile sequenza metodologica è:

- definire funzione della coil;
- definire distanza operativa;
- definire forza minima richiesta;
- definire volume massimo disponibile;
- scegliere geometria della bobina;
- scegliere diametro del filo;
- stimare numero di spire;
- misurare resistenza;
- stimare corrente a tensione disponibile;
- calcolare dissipazione termica;
- verificare MOSFET e protezioni;
- definire duty cycle massimo;
- testare fisicamente;
- calibrare simulazione;
- aggiornare documentazione.

Questa procedura rende il dimensionamento tracciabile e coerente con il metodo ingegneristico del progetto.

Un aspetto molto importante è che il dimensionamento deve essere iterativo. La prima coil non sarà necessariamente quella definitiva. È normale costruire una bobina di test, misurarne il comportamento, correggere il numero di spire, cambiare filo, modificare la posizione, ridurre o aumentare duty cycle e ripetere. Questa iterazione non è un errore, ma parte della progettazione. Il sistema MicroBot, essendo sperimentale, deve evolvere attraverso test progressivi e documentati.

Dal punto di vista dei dati sperimentali, sarebbe utile creare una tabella di caratterizzazione per ogni coil testata. La tabella potrebbe includere: codice della coil, diametro filo, numero di spire, resistenza misurata, tensione applicata, corrente stimata o misurata, duty cycle, durata impulso, riscaldamento osservato, distanza di attrazione, forza di distacco e note. Questa tabella diventerebbe un elemento prezioso della documentazione tecnica, perché permetterebbe di collegare teoria, prototipo e simulazione.

Un esempio di campi utili per una tabella tecnica potrebbe essere:

ID coil;

diametro filo;

numero spire;

diametro bobina;

lunghezza bobina;

resistenza;

tensione test;

corrente stimata;

duty cycle massimo;

durata massima impulso;

temperatura osservata;

distanza utile;

risultato del test;

note progettuali.

Questa struttura consentirebbe di confrontare più coil e scegliere quella più adatta alla versione corrente del MicroBot.

Nel progetto MicroBot, il dimensionamento delle coil deve essere prudente. La tentazione naturale è cercare subito una bobina forte, capace di attrarre con decisione. Tuttavia, una bobina troppo aggressiva può creare più problemi che vantaggi: consumo elevato, calore, interferenze, difficoltà di controllo, urti, instabilità e rischio di danneggiamento. Una scelta più intelligente consiste nel partire da coil piccole, sicure, con impulsi brevi e comportamento osservabile. Solo dopo aver validato il circuito e la logica si può aumentare la potenza.

Questo approccio è coerente con tutto ciò che è stato fatto finora nel progetto: prima simulazioni e LED, poi segnali, poi carichi semplici, poi bobine reali, poi interazioni fisiche. Il dimensionamento delle coil non deve

interrompere questa progressione; deve esserne la continuazione naturale. Ogni versione deve aggiungere complessità senza perdere controllo.

Dal punto di vista della sicurezza, la coil deve sempre avere limiti predefiniti. Anche durante test manuali, il firmware dovrebbe impedire attivazioni continue non intenzionali. Un comando magnetico dovrebbe avere durata massima, intensità massima e condizioni di blocco. Se la batteria è bassa, se il nodo è in errore, se il `thermal_load` è alto o se la comunicazione è persa, la coil dovrebbe restare spenta. Questa logica rende il sistema più robusto e protegge sia il componente sia l'utente.

Dal punto di vista dello swarm, il dimensionamento della singola coil deve essere compatibile con la scala del sistema. Una coil che funziona bene su un singolo nodo potrebbe non essere adatta quando molti nodi operano insieme. Se ogni nodo genera campi troppo forti, il sistema può diventare caotico; se ogni nodo consuma troppo, l'autonomia globale diventa scarsa; se ogni nodo scalda troppo, il sistema non è praticabile. Il dimensionamento deve quindi pensare già alla moltiplicazione: quello che succede una volta nel singolo MicroBot può succedere decine o centinaia di volte nello swarm.

Questo porta a un principio fondamentale: la coil migliore non è quella più potente, ma quella più controllabile, efficiente, sicura e ripetibile. Nel MicroBot, il valore di una coil non si misura solo dalla forza massima che può generare, ma dalla sua capacità di integrarsi nel sistema senza destabilizzarlo. Una coil moderata ma ben controllata è più utile di una coil potente ma ingestibile.

Dal punto di vista teorico, il dimensionamento delle coil conclude il collegamento tra informazione e forma. Il software produce un comando; il comando produce corrente; la corrente produce campo; il campo produce forza; la forza modifica le connessioni; le connessioni modificano la forma dello swarm. Tuttavia, ogni passaggio di questa catena è vincolato da grandezze fisiche reali. Il dimensionamento delle coil è ciò che impedisce alla teoria di restare astratta: obbliga il progetto a confrontarsi con tensione, corrente, resistenza, calore, spazio e materiali.

In conclusione, il dimensionamento delle coil rappresenta il punto di sintesi della Sezione 6. Esso integra tutti i concetti del sistema magnetico in una procedura progettuale concreta, capace di guidare la realizzazione di bobine compatibili con i vincoli del MicroBot. Una coil deve essere dimensionata in funzione dell'effetto richiesto, della distanza operativa, della forza minima, del volume disponibile, della corrente massima, della dissipazione termica, della batteria, del MOSFET, delle protezioni e della strategia di controllo. Il suo obiettivo non è generare il massimo campo possibile, ma produrre un'interazione utile, sicura, ripetibile e integrabile nello swarm. Con questo punto si chiude la Sezione 6: il sistema magnetico è stato analizzato come base fisica della materia programmabile, preparando il passaggio alla Sezione 7, dove le interazioni verranno formalizzate nel modello matematico del sistema.

SEZIONE 7 — Modello Matematico

La Sezione 7 introduce il livello formale attraverso cui il sistema MicroBot viene descritto, analizzato e reso simulabile. Dopo aver definito l'architettura generale, la struttura fisica del nodo, l'elettronica interna e il sistema magnetico, diventa necessario costruire un modello matematico capace di rappresentare le unità nello spazio, descrivere la loro evoluzione nel tempo e formalizzare le condizioni che determinano interazione, aggancio, distacco, coesione e comportamento collettivo. Questa sezione rappresenta quindi il passaggio dal progetto fisico al modello astratto del sistema, cioè dal MicroBot come oggetto reale al MicroBot come entità calcolabile.

Il modello matematico non deve essere inteso come un'aggiunta teorica separata dal progetto, ma come lo strumento che consente di collegare prototipo, simulazione e controllo. Senza un modello, il comportamento dello swarm resterebbe descritto soltanto in modo qualitativo; con un modello, invece, diventa possibile definire regole, soglie, funzioni di aggiornamento, stati e relazioni tra unità. Questo permette di costruire simulazioni coerenti, progettare strategie di controllo e confrontare il comportamento atteso con quello osservato nei prototipi fisici.

Nel sistema MicroBot, il modello matematico deve rispettare una caratteristica fondamentale: il sistema è distribuito, discreto e dinamico. È distribuito perché composto da molte unità indipendenti; è discreto perché ogni MicroBot è un'entità separata, identificabile e numerabile; è dinamico perché la configurazione cambia nel tempo. La forma dello swarm non viene descritta come un oggetto statico, ma come una configurazione temporanea generata dall'interazione tra nodi. Di conseguenza, il modello deve essere in grado di rappresentare sia lo stato del singolo MicroBot sia lo stato globale del sistema.

In continuità con le sezioni precedenti, il modello matematico deve includere più livelli. Il primo livello è geometrico e riguarda la posizione dei MicroBot nello spazio. Il secondo livello è dinamico e riguarda l'evoluzione delle posizioni nel tempo. Il terzo livello è relazionale e riguarda le interazioni tra unità, come distanza, vicinanza, attrazione e connessione. Il quarto livello è comportamentale e riguarda le regole che determinano coesione, separazione, allineamento e formazione di strutture. Il quinto livello è sistemico e riguarda l'emergenza del comportamento collettivo a partire da regole locali. Questa stratificazione rende il modello coerente con l'intera architettura MicroBot: anche la matematica, come il sistema, è modulare e multilivello.

7.1 Rappresentazione nello spazio

La rappresentazione nello spazio costituisce il primo elemento del modello matematico del sistema MicroBot, perché ogni unità deve essere descritta attraverso una posizione, una configurazione e, nelle versioni più avanzate, un orientamento. Prima ancora di analizzare forze, regole di movimento o comportamenti emergenti, è necessario stabilire come un MicroBot viene collocato all'interno dello spazio di riferimento. Questa scelta è fondamentale perché tutte le relazioni successive, come distanza, vicinanza, aggancio e interazione magnetica, dipendono dalla rappresentazione spaziale adottata.

Nella forma più semplice, il sistema può essere rappresentato in uno spazio bidimensionale, indicato tramite coordinate x e y . Questa scelta è particolarmente utile nelle prime simulazioni, nelle visualizzazioni web e nei prototipi concettuali, perché permette di osservare il comportamento dello swarm su un piano. In questo caso, ogni MicroBot viene descritto come un punto o come un piccolo corpo circolare all'interno di un piano cartesiano. La posizione del MicroBot i -esimo può essere indicata come:

$$p_i(t) = (x_i(t), y_i(t))$$

dove $p_i(t)$ rappresenta la posizione del nodo i al tempo t , mentre $x_i(t)$ e $y_i(t)$ rappresentano le sue coordinate spaziali. Questa rappresentazione è sufficiente per modellare molte dinamiche iniziali, come avvicinamento, distanza, formazione di linee, cluster, cerchi o pattern semplici. Inoltre, è direttamente compatibile con le simulazioni grafiche su schermo, dove lo spazio visuale è naturalmente bidimensionale.

Tuttavia, il sistema MicroBot non è concettualmente limitato al piano. La visione completa del progetto prevede unità fisiche capaci di interagire in uno spazio tridimensionale. In una rappresentazione più avanzata, ogni nodo deve quindi essere descritto tramite tre coordinate:

$$p_i(t) = (x_i(t), y_i(t), z_i(t))$$

In questo caso, $z_i(t)$ rappresenta la profondità o altezza del nodo nello spazio. L'introduzione della terza coordinata permette di modellare strutture volumetriche, aggregazioni tridimensionali, sovrapposizioni, architetture spaziali più complesse e configurazioni non riducibili a una superficie piana. Questo livello sarà particolarmente importante nelle versioni future del sistema, soprattutto se MicroBot dovrà essere rappresentato in ambienti 3D, realtà virtuale, simulazioni avanzate o modelli fisici più complessi.

La scelta tra rappresentazione bidimensionale e tridimensionale non deve essere vista come una contraddizione, ma come una progressione metodologica. Il modello bidimensionale rappresenta una semplificazione controllata, utile per validare le regole fondamentali dello swarm senza introdurre subito tutta la complessità dello spazio reale. Il modello tridimensionale rappresenta invece l'estensione naturale del sistema verso una descrizione più completa. Questo approccio è coerente con il metodo generale del progetto MicroBot: partire da un livello semplice e verificabile, poi aumentare progressivamente il realismo del modello.

Oltre alla posizione, in molti casi è necessario rappresentare anche l'orientamento del MicroBot. Questo aspetto diventa particolarmente importante quando si considera il sistema magnetico, perché l'interazione tra unità non dipende soltanto dalla distanza, ma anche dalla direzione dei poli, dall'asse della bobina, dalla geometria delle superfici di contatto e dalla compatibilità tra orientamenti. Due MicroBot possono essere

vicini nello spazio, ma non correttamente allineati. Per questo motivo, una rappresentazione completa del nodo deve includere anche una variabile di orientamento.

Nel caso bidimensionale, l'orientamento può essere rappresentato da un angolo $\theta_i(t)$, che indica la rotazione del nodo rispetto a un asse di riferimento. La configurazione spaziale del MicroBot diventa quindi:

$$q_i(t) = (x_i(t), y_i(t), \theta_i(t))$$

Nel caso tridimensionale, l'orientamento richiede una rappresentazione più complessa, ad esempio tramite angoli di Eulero, quaternioni o vettori direzionali. Per una prima formalizzazione del progetto, è sufficiente indicare che il nodo possiede una direzione principale $d_i(t)$, associata all'asse magnetico o alla geometria principale del corpo. Questa direzione consente di valutare se due MicroBot sono orientati in modo compatibile per l'aggancio.

La rappresentazione completa del nodo può quindi essere estesa da semplice posizione a configurazione:

$$q_i(t) = (p_i(t), d_i(t), s_i(t))$$

dove $p_i(t)$ rappresenta la posizione, $d_i(t)$ rappresenta l'orientamento e $s_i(t)$ rappresenta lo stato interno del nodo. Questa formulazione è molto importante perché collega la geometria al modello generale dello stato. Il MicroBot non è soltanto un punto nello spazio, ma un'entità dotata di posizione, orientamento, stato energetico, stato magnetico, stato comunicativo e stato operativo. La rappresentazione spaziale diventa quindi il primo livello di una descrizione più ampia.

Nel sistema MicroBot, lo spazio non è soltanto contenitore passivo, ma ambiente di relazione. La posizione di un nodo assume significato solo rispetto alla posizione degli altri nodi. Un MicroBot isolato ha una posizione assoluta, ma il suo comportamento nello swarm dipende soprattutto dalle relazioni spaziali locali: quali nodi sono vicini, quali sono lontani, quali sono allineati, quali sono entro la soglia magnetica, quali appartengono alla stessa struttura. Per questo motivo, il modello matematico non deve limitarsi a memorizzare coordinate, ma deve calcolare relazioni tra coordinate.

La rappresentazione spaziale permette infatti di costruire il grafo dinamico dello swarm. Ogni MicroBot è un nodo del grafo; ogni possibile interazione fisica o logica può essere rappresentata come un arco tra due nodi. La presenza di un arco dipende da condizioni spaziali, come distanza e orientamento. In forma generale:

$$G(t) = (V, E(t))$$

dove V rappresenta l'insieme dei MicroBot ed $E(t)$ rappresenta l'insieme delle relazioni attive o potenziali al tempo t . Questo grafo cambia nel tempo perché i nodi si muovono, si avvicinano, si separano e modificano il proprio stato. La rappresentazione nello spazio diventa quindi la base per rappresentare anche la struttura relazionale del sistema.

Un aspetto rilevante riguarda la scelta del sistema di riferimento. Nelle simulazioni iniziali, può essere utilizzato un sistema di riferimento globale, in cui tutte le posizioni sono espresse rispetto a un'origine comune. Questo approccio è semplice e utile per visualizzazione, debug e calcolo delle distanze. Tuttavia, in un sistema fisico distribuito, ogni MicroBot potrebbe non conoscere la propria posizione globale. Potrebbe invece conoscere soltanto informazioni locali, come distanza da nodi vicini, contatto, orientamento relativo o comandi ricevuti dal controller. Questa distinzione è fondamentale: il simulatore può avere una visione globale, ma il nodo reale può avere una conoscenza locale.

Questa differenza tra rappresentazione globale e conoscenza locale deve essere mantenuta chiara. Il modello matematico può descrivere l'intero sistema da un punto di vista esterno, utile per simulazione e controllo centrale. Il firmware del singolo MicroBot, invece, può operare su una versione ridotta e locale dello stesso modello. Questo rispecchia il paradigma ibrido già definito nell'architettura generale: il controller e la simulazione possono osservare lo stato globale, mentre i nodi distribuiti reagiscono a informazioni locali e comandi parziali.

Nel modello matematico, la posizione può essere continua o discreta. Una rappresentazione continua considera le coordinate come valori reali, permettendo movimenti fluidi nello spazio. Una rappresentazione

discreta divide invece lo spazio in celle, griglie o posizioni finite. Entrambi gli approcci sono utili. Il modello continuo è più naturale per rappresentare fisica, distanza e movimento; il modello discreto può essere più semplice per simulazioni logiche, pattern geometrici e assegnazione di posizioni target. MicroBot può utilizzare entrambi i livelli: continuo per la simulazione fisica, discreto per la pianificazione di configurazioni.

Ad esempio, una formazione a griglia può essere descritta tramite posizioni discrete target, mentre il movimento dei nodi verso quelle posizioni può essere simulato in modo continuo. Questo permette di separare il problema in due parti: definizione della forma desiderata e dinamica di raggiungimento della forma. La forma desiderata può essere un insieme di posizioni target:

$$T = \{\tau_1, \tau_2, \dots, \tau_n\}$$

dove ogni τ_i rappresenta una posizione obiettivo. Lo stato reale dello swarm è invece:

$$P(t) = \{p_1(t), p_2(t), \dots, p_n(t)\}$$

Il controllo consiste nel ridurre progressivamente la differenza tra $P(t)$ e T , tenendo conto dei vincoli fisici e magnetici. Questa formulazione sarà utile nelle sezioni successive dedicate ai pattern e ai comportamenti.

Un altro elemento importante riguarda i confini dello spazio. In una simulazione, lo spazio può essere limitato da un'area rettangolare, circolare o da un volume definito. I MicroBot devono quindi rispettare vincoli di bordo. Se un nodo raggiunge il limite dello spazio simulato, può essere respinto, bloccato o reinserito. Questi confini non rappresentano necessariamente pareti fisiche reali, ma condizioni operative del modello. Nei prototipi fisici, i confini potrebbero essere rappresentati da una superficie di lavoro, un'arena, un piano magnetico o una zona controllata.

La rappresentazione nello spazio deve anche considerare la dimensione fisica del MicroBot. Se i nodi vengono rappresentati come punti, il modello diventa più semplice, ma perde informazioni importanti: due punti possono sovrapporsi senza vincoli, mentre due oggetti fisici non possono occupare lo stesso spazio. Per rendere il modello più realistico, ogni MicroBot dovrebbe avere un raggio o un volume di esclusione. Nel caso bidimensionale, il nodo può essere approssimato come un cerchio di raggio R ; nel caso tridimensionale, come una sfera o un corpo composto. Questo permette di evitare collisioni non fisiche e di modellare la separazione minima tra unità.

La distanza minima tra due nodi può quindi essere definita come:

$$d_{\min} = R_i + R_j$$

dove R_i e R_j rappresentano i raggi effettivi dei due MicroBot. Se la distanza tra i centri è minore di $d_{\min}$, i nodi si sovrappongono nel modello e deve essere applicata una correzione di separazione. Questo concetto sarà importante nella sezione sulle forze dello swarm, in particolare per il comportamento di separazione, che impedisce ai nodi di collassare tutti nello stesso punto.

La rappresentazione spaziale deve inoltre supportare il concetto di porta magnetica o punto di aggancio. Come discusso nella Sezione 6, la distanza rilevante per l'interazione magnetica non è sempre quella tra i centri dei nodi, ma quella tra le superfici o i punti magnetici. Per un modello più avanzato, ogni MicroBot può essere dotato di punti di interazione locali, definiti rispetto al proprio centro e orientamento. Questi punti si muovono e ruotano insieme al nodo. In questo modo, il modello può distinguere tra nodi vicini ma non correttamente orientati e nodi realmente pronti all'aggancio.

Una rappresentazione possibile è:

$$porta_{ik} = p_i + R(\theta_i) \cdot offset_k$$

dove $porta_{ik}$ rappresenta la k -esima porta magnetica del nodo i , p_i è la posizione del nodo, $R(\theta_i)$ è la rotazione associata all'orientamento e $offset_k$ è la posizione locale della porta rispetto al centro del MicroBot. Questa formalizzazione consente di costruire modelli molto più realistici senza abbandonare la struttura matematica semplice.

Nel contesto dei file e dei sistemi già sviluppati, questa rappresentazione spaziale si collega direttamente alle simulazioni web e Python, ai dataset di swarm, agli snapshot live e ai modelli virtuali del sistema. Ogni file di stato dello swarm, ogni simulazione e ogni dashboard deve in qualche modo rappresentare i nodi nello spazio. Anche quando il sistema mostra soltanto LED o punti grafici, sta già usando una rappresentazione spaziale implicita. Formalizzarla significa rendere il sistema più coerente, più estendibile e più facile da collegare al modello fisico futuro.

Dal punto di vista della telemetria, la rappresentazione spaziale può diventare parte dello stato trasmesso dal nodo o stimato dal controller. In una fase iniziale, la posizione può essere simulata o assegnata dal software. In una fase successiva, potrebbe essere stimata tramite sensori, visione esterna, tracking, contatti o altri metodi. In ogni caso, la struttura dei dati dovrebbe prevedere campi coerenti: id del nodo, posizione, orientamento, velocità, stato e timestamp. Questa coerenza permette di collegare simulazione, dashboard e sistema fisico.

Un possibile record di stato spaziale potrebbe essere:

```
node_state = (id, x, y, z, orientation, velocity, status, timestamp)
```

Questa struttura è abbastanza generale da supportare sia simulazioni bidimensionali sia estensioni tridimensionali. Nei sistemi iniziali, z può essere omesso o posto a zero; orientation può essere semplificato; velocity può essere calcolata dal simulatore. Nelle versioni avanzate, tutti questi campi possono assumere significato fisico più preciso. La cosa importante è progettare il modello dati in modo che possa crescere senza essere riscritto da zero.

La rappresentazione nello spazio deve anche essere coerente con il concetto di tempo. Una posizione isolata descrive solo un istante; il comportamento del sistema emerge dalla sequenza delle posizioni nel tempo. Per questo motivo, la rappresentazione spaziale è sempre funzione del tempo. Il sistema non deve essere descritto solo come P, ma come P(t). Questa dipendenza temporale prepara direttamente i punti successivi della Sezione 7, in cui verranno analizzati modello discreto del moto e aggiornamento delle posizioni.

Dal punto di vista teorico, la rappresentazione nello spazio è il primo livello attraverso cui la forma dello swarm diventa calcolabile. La forma non è ancora interpretata come comportamento emergente, ma come insieme di posizioni. Tuttavia, questa base è necessaria. Senza posizioni, non esistono distanze; senza distanze, non esistono interazioni; senza interazioni, non esistono coesione, separazione, allineamento o aggancio. La rappresentazione spaziale è quindi il fondamento di tutta la matematica successiva.

In conclusione, la rappresentazione nello spazio definisce il modo in cui i MicroBot vengono collocati, descritti e resi calcolabili all'interno del modello. Essa può iniziare come rappresentazione bidimensionale semplice, utile per simulazioni e visualizzazioni, ma deve essere progettata per estendersi verso il tridimensionale, includendo posizione, orientamento, dimensione fisica e punti di aggancio. Questo livello collega geometria, sistema magnetico, simulazione, telemetria e controllo. Con esso, il MicroBot passa da oggetto fisico a entità matematica, pronta per essere analizzata nei punti successivi attraverso coordinate, movimento, distanza, interazione e comportamento collettivo.

7.2 Coordinate dei MicroBot

Le coordinate dei MicroBot rappresentano la formalizzazione numerica della loro posizione all'interno dello spazio di riferimento e costituiscono il livello immediatamente successivo rispetto alla rappresentazione spaziale generale. Se nel punto precedente è stato definito il MicroBot come entità collocata nello spazio, in questa sezione si specifica in che modo tale collocazione viene descritta, memorizzata, aggiornata e utilizzata dal sistema matematico, dalla simulazione e dai livelli di controllo. Le coordinate non sono quindi semplici valori geometrici, ma variabili fondamentali dello stato del sistema: attraverso esse diventa possibile calcolare distanze, interazioni, connessioni, forze, pattern e configurazioni collettive.

Nel modello MicroBot, ogni unità viene identificata da un indice o identificatore univoco. Se il sistema contiene n unità, esse possono essere indicate come:

$$\text{MicroBot} = \{m_1, m_2, \dots, m_n\}$$

A ciascun MicroBot m_i viene associata una posizione nello spazio. Nel caso bidimensionale, la posizione è descritta da due coordinate:

$$p_i(t) = (x_i(t), y_i(t))$$

Nel caso tridimensionale, la posizione viene estesa a tre coordinate:

$$p_i(t) = (x_i(t), y_i(t), z_i(t))$$

Questa distinzione è importante perché consente al modello di essere utilizzato in più fasi dello sviluppo. Le simulazioni iniziali, le dashboard web e i modelli dimostrativi possono operare in due dimensioni, mantenendo semplicità e leggibilità. Le versioni avanzate, invece, possono estendere lo stesso schema a tre dimensioni, senza modificare il principio generale della rappresentazione. In questo modo, il modello resta scalabile anche dal punto di vista matematico.

Le coordinate x , y e z devono essere interpretate come grandezze relative a un sistema di riferimento. In una simulazione, tale sistema di riferimento può essere il piano dello schermo, una griglia virtuale, un'arena di test o un volume tridimensionale. In un prototipo fisico, invece, il sistema di riferimento potrebbe coincidere con una superficie di lavoro, con una struttura di tracking esterna o con un sistema locale definito dal controller. La scelta del riferimento influenza direttamente il significato delle coordinate e deve quindi essere dichiarata con precisione.

In una rappresentazione globale, tutte le coordinate sono espresse rispetto alla stessa origine. Questo è il caso più semplice e più utile per la simulazione, perché permette di confrontare direttamente le posizioni dei nodi. Se due MicroBot hanno coordinate p_i e p_j , la loro distanza può essere calcolata immediatamente. Il controller o il simulatore possiedono quindi una visione globale dello stato spaziale. Questo modello è adatto per visualizzare lo swarm, calcolare pattern, rilevare collisioni e stimare configurazioni.

In una rappresentazione locale, invece, ogni MicroBot può conoscere soltanto coordinate relative o relazioni con i nodi vicini. Questo modello è più realistico per il sistema fisico distribuito, perché un singolo nodo reale potrebbe non conoscere la propria posizione assoluta. Potrebbe sapere, o stimare, che un altro nodo si trova vicino, davanti, dietro, a destra, a sinistra o in contatto, ma non necessariamente conoscere coordinate globali precise. Questa distinzione tra coordinate globali e coordinate locali è fondamentale perché separa ciò che il simulatore può sapere da ciò che il singolo nodo può realmente utilizzare.

Nel sistema MicroBot è quindi utile mantenere due livelli di coordinate: coordinate globali per la simulazione e il controller, coordinate relative per le interazioni locali. Le coordinate globali possono essere indicate come p_i , mentre una relazione locale tra due nodi può essere indicata come:

$$r_{ij}(t) = p_j(t) - p_i(t)$$

Il vettore $r_{ij}(t)$ descrive la posizione del nodo j rispetto al nodo i . Questo vettore è molto importante perché permette di calcolare direzione, distanza e relazione spaziale tra due MicroBot. Nel comportamento distribuito, ciò che conta spesso non è la posizione assoluta del nodo, ma la posizione relativa dei suoi vicini. Un nodo non deve necessariamente sapere dove si trova nel mondo; deve sapere come comportarsi rispetto agli altri nodi vicini.

Nel caso bidimensionale, il vettore relativo è:

$$r_{ij}(t) = (x_j(t) - x_i(t), y_j(t) - y_i(t))$$

Nel caso tridimensionale diventa:

$$r_{ij}(t) = (x_j(t) - x_i(t), y_j(t) - y_i(t), z_j(t) - z_i(t))$$

Questa semplice differenza tra coordinate è alla base di molte funzioni successive del modello. Da r_{ij} si può calcolare la distanza, la direzione dell'interazione, la forza attrattiva o repulsiva, la vicinanza magnetica, la soglia di aggancio e la configurazione locale dello swarm. Le coordinate, quindi, non sono valori statici, ma il punto di partenza per costruire tutte le relazioni dinamiche del sistema.

Un ulteriore elemento importante è l'orientamento. Le coordinate descrivono la posizione del centro del MicroBot, ma non dicono come il MicroBot è ruotato nello spazio. Tuttavia, come già emerso nella Sezione 6, il sistema magnetico dipende fortemente dall'orientamento dei poli, delle bobine e delle superfici di contatto. Per questo motivo, a ciascun MicroBot può essere associata una variabile di orientamento. Nel modello bidimensionale, questa variabile può essere un angolo:

$$\theta_i(t)$$

La configurazione del nodo diventa quindi:

$$q_i(t) = (x_i(t), y_i(t), \theta_i(t))$$

Nel modello tridimensionale, l'orientamento può essere rappresentato tramite un vettore direzionale, una matrice di rotazione o un quaternione. Per una prima formalizzazione, è sufficiente introdurre un vettore direzione:

$$d_i(t)$$

che rappresenta l'asse principale del MicroBot, ad esempio l'asse lungo cui sono disposti i magneti o le bobine principali. Questo vettore permette di valutare se due nodi sono orientati in modo compatibile.

Le coordinate devono inoltre essere collegate alla dimensione fisica del nodo. Se un MicroBot viene rappresentato come punto matematico, le coordinate indicano semplicemente una posizione ideale. Se invece viene rappresentato come corpo fisico, le coordinate indicano generalmente il centro geometrico o il centro di massa. Questa distinzione è importante perché il comportamento fisico dipende dal volume occupato. Due MicroBot non possono sovrapporsi, e la distanza tra i loro centri deve essere confrontata con la dimensione reale dei corpi.

Se ogni MicroBot viene approssimato come una sfera o un disco di raggio R , allora la condizione minima per evitare sovrapposizione è:

$$d_{ij} \geq 2R$$

nel caso di nodi uguali. Se i nodi hanno dimensioni diverse, la condizione diventa:

$$d_{ij} \geq R_i + R_j$$

Questa condizione è alla base del comportamento di separazione, che verrà analizzato successivamente. Essa impedisce al modello di generare configurazioni fisicamente impossibili, in cui due nodi occupano lo stesso spazio. Anche se nelle simulazioni iniziali i MicroBot possono essere rappresentati graficamente come punti o LED, il modello matematico deve già prevedere il vincolo di volume, perché sarà essenziale nelle versioni fisiche.

Le coordinate possono essere continue o discrete. Nel modello continuo, x , y e z sono numeri reali e il MicroBot può assumere qualunque posizione nello spazio simulato. Questo modello è adatto a descrivere movimento fluido, forze, attrazioni e traiettorie. Nel modello discreto, invece, lo spazio è diviso in celle o posizioni finite. Ogni MicroBot occupa una cella o un nodo di una griglia. Questo modello è utile per rappresentare pattern geometrici, formazioni, stati semplificati e algoritmi di assegnazione.

Nel sistema MicroBot, entrambi i modelli possono coesistere. Una formazione desiderata può essere definita in coordinate discrete, mentre il movimento verso quella formazione può essere simulato in coordinate continue. Per esempio, una linea può essere descritta come insieme di posizioni target:

$$T_1, T_2, \dots, T_n$$

mentre ogni MicroBot si muove progressivamente dalla posizione corrente $p_i(t)$ verso una posizione obiettivo T_k . Il problema del controllo diventa quindi assegnare ogni MicroBot a un target e aggiornare la sua posizione nel tempo. Questa struttura sarà molto utile nella Sezione 10, dedicata ai pattern e comportamenti.

Un concetto importante è la matrice delle posizioni. Lo stato spaziale dell'intero swarm può essere rappresentato come una matrice $P(t)$, in cui ogni riga contiene le coordinate di un MicroBot. Nel caso bidimensionale:

$$P(t) = \begin{bmatrix} x_1(t) & y_1(t) \\ x_2(t) & y_2(t) \\ \dots & \dots \\ x_n(t) & y_n(t) \end{bmatrix}$$

Nel caso tridimensionale:

$$P(t) = \begin{bmatrix} x_1(t) & y_1(t) & z_1(t) \\ x_2(t) & y_2(t) & z_2(t) \\ \dots & \dots & \dots \\ x_n(t) & y_n(t) & z_n(t) \end{bmatrix}$$

Questa rappresentazione è utile perché permette di trattare lo stato dello swarm come struttura dati compatta e manipolabile. Una simulazione può aggiornare $P(t)$ a ogni ciclo; una dashboard può visualizzare $P(t)$; un algoritmo può calcolare distanze e connessioni a partire da $P(t)$. La matrice delle posizioni diventa quindi uno degli oggetti fondamentali del sistema matematico.

Accanto alla matrice delle posizioni, si può definire un vettore degli orientamenti:

$$\Theta(t) = [\theta_1(t), \theta_2(t), \dots, \theta_n(t)]$$

nel caso bidimensionale, oppure una matrice/vettore di direzioni nel caso tridimensionale. Insieme, $P(t)$ e $\Theta(t)$ descrivono la configurazione geometrica dello swarm. Se si aggiungono gli stati interni dei nodi, si ottiene una rappresentazione più completa:

$$X(t) = (P(t), \Theta(t), S(t))$$

dove $S(t)$ rappresenta l'insieme degli stati interni dei MicroBot. Questa struttura mostra che la posizione è solo una parte dello stato totale, ma una parte indispensabile.

Dal punto di vista dei dati, le coordinate dei MicroBot devono essere associate a timestamp. Una posizione senza tempo descrive un istante isolato; una sequenza di posizioni con timestamp descrive il movimento. Ogni record di stato dovrebbe quindi includere almeno:

id del nodo;

coordinate spaziali;

orientamento;

stato operativo;

timestamp.

Questa struttura consente di ricostruire l'evoluzione dello swarm nel tempo, calcolare velocità, rilevare ritardi e confrontare simulazioni diverse. Nel progetto MicroBot, dove esistono già idee di telemetria, snapshot e runtime state, questa rappresentazione è fondamentale per mantenere coerenza tra modello teorico e file di dati.

La velocità può essere derivata dalle coordinate successive. Se un nodo si trova in $p_i(t)$ e poi in $p_i(t+\Delta t)$, la sua velocità media può essere stimata come:

$$v_i(t) = [p_i(t+\Delta t) - p_i(t)] / \Delta t$$

Questa grandezza sarà utile nel modello discreto del moto. Anche se i MicroBot fisici iniziali potrebbero non avere movimento autonomo completo, la simulazione può già trattare la velocità come variabile dinamica. Questo permette di sviluppare modelli di movimento, attrazione, separazione e stabilizzazione prima dell'implementazione fisica completa.

Le coordinate sono anche fondamentali per calcolare il centro dello swarm. Il centro geometrico o baricentro del sistema può essere definito come:

$$c(t) = (1/n) \cdot \sum p_i(t)$$

dove la somma è estesa a tutti i MicroBot. Il centro dello swarm è utile per analizzare la distribuzione collettiva, misurare la dispersione, definire movimenti globali o valutare se lo swarm resta compatto. Una dashboard potrebbe mostrare non solo i singoli nodi, ma anche il centro e il raggio medio dello swarm.

La dispersione può essere definita come la distanza media dei nodi dal centro:

$$D(t) = (1/n) \cdot \sum ||p_i(t) - c(t)||$$

Questa misura permette di capire se lo swarm è compatto o disperso. Un valore basso indica nodi vicini al centro; un valore alto indica distribuzione ampia. Questo tipo di metrica sarà utile per valutare coesione e separazione. Anche qui si vede come le coordinate siano il fondamento di metriche più avanzate.

Un altro concetto importante è la distanza locale dai vicini. Per ogni MicroBot, si può definire un insieme di vicini:

$$N_i(t) = \{m_j \mid d_{ij}(t) < R_neighbor\}$$

dove $R_neighbor$ rappresenta un raggio di vicinato. Questo insieme non dipende soltanto dalle coordinate del nodo i , ma dalle coordinate di tutti gli altri nodi. Il vicinato è centrale nei sistemi multi-agente, perché molte regole locali vengono applicate solo ai nodi vicini. Coesione, separazione e allineamento funzionano proprio attraverso insiemi $N_i(t)$, non attraverso l'intero swarm.

Dal punto di vista magnetico, si può definire un vicinato ancora più specifico:

$$N_i^{mag}(t) = \{m_j \mid d_{ij}(t) < R_magnetic \text{ e orientamento compatibile}\}$$

Questo insieme rappresenta i nodi che sono non solo vicini, ma potenzialmente rilevanti per l'interazione magnetica. La distinzione tra vicinato geometrico e vicinato magnetico è importante: due nodi possono essere vicini nello spazio ma non magneticamente compatibili, oppure possono essere entro il raggio visivo della simulazione ma fuori dal raggio utile di attrazione.

Le coordinate devono essere gestite con attenzione anche nei bordi dello spazio simulato. Se lo swarm opera in un'arena virtuale, ogni posizione deve rispettare limiti:

$$\begin{aligned} x_{min} &\leq x_i \leq x_{max} \\ y_{min} &\leq y_i \leq y_{max} \\ z_{min} &\leq z_i \leq z_{max} \end{aligned}$$

Quando un nodo raggiunge un bordo, il modello può applicare diverse strategie: blocco, rimbalzo, correzione, reinserimento o vincolo morbido. La scelta dipende dal tipo di simulazione. In un sistema fisico, i bordi potrebbero rappresentare pareti, limiti del piano di test o confini operativi. Nei modelli futuri, sarà importante distinguere tra confini rigidi e confini virtuali.

Un ulteriore livello riguarda le coordinate target. Molti comportamenti MicroBot possono essere descritti come passaggio da una configurazione iniziale a una configurazione desiderata. In questo caso, ogni nodo può avere una posizione target:

$$\tau_i = (x_i^*, y_i^*, z_i^*)$$

La differenza tra posizione attuale e posizione target è:

$$e_i(t) = \tau_i - p_i(t)$$

Questo vettore errore indica quanto il nodo è lontano dalla posizione desiderata. Un sistema di controllo può usare $e_i(t)$ per generare un movimento o una forza virtuale verso il target. Anche se nelle prime simulazioni il movimento può essere puramente grafico, questa formalizzazione prepara il terreno per algoritmi più avanzati di formazione.

La precisione delle coordinate è un altro elemento importante. Nel simulatore, le coordinate possono essere molto precise perché calcolate numericamente. Nel sistema reale, invece, le coordinate potrebbero essere affette da errore. Questo errore può derivare da sensori, ritardi, rumore, stime approssimate o tracking esterno. Per questo motivo, in una rappresentazione più realistica, la posizione non dovrebbe essere considerata sempre esatta, ma stimata:

$$\hat{p}_i(t)$$

dove $\hat{p}$ indica una stima della posizione reale. Questa distinzione sarà importante nelle versioni avanzate, soprattutto se il sistema userà visione esterna o sensori per localizzare i nodi.

La presenza di errore nelle coordinate impone una progettazione robusta. Le soglie di aggancio e distacco, ad esempio, non devono essere definite troppo vicine tra loro, altrimenti piccoli errori di misura potrebbero causare oscillazioni. Anche i pattern devono tollerare deviazioni. Una formazione non deve fallire se un nodo è leggermente fuori posizione. Questo introduce il concetto di tolleranza geometrica, cioè un margine entro cui la posizione è considerata accettabile.

Un possibile criterio è:

$$\|p_i(t) - \tau_i\| < \varepsilon$$

dove ε rappresenta la tolleranza. Se la distanza tra posizione attuale e target è minore di ε , il nodo viene considerato correttamente posizionato. Questo criterio sarà utile nei test dei pattern e nella validazione del comportamento.

Le coordinate dei MicroBot devono anche essere coerenti tra diversi livelli del sistema: simulazione, backend, frontend, telemetria e documentazione. Se il simulatore usa un sistema di coordinate e la dashboard ne usa un altro, possono nascere ambiguità. È quindi importante definire una convenzione: origine, assi, unità di misura, scala, orientamento dell'asse y, eventuale normalizzazione. Per esempio, in grafica web l'asse y spesso cresce verso il basso, mentre in matematica cresce verso l'alto. Questa differenza deve essere gestita attraverso trasformazioni chiare.

Una funzione di conversione può essere necessaria tra coordinate matematiche e coordinate visuali:

$$p_{\text{visual}} = T_{\text{display}}(p_{\text{model}})$$

dove T_{display} rappresenta la trasformazione dal modello alla visualizzazione. Questa separazione evita che la logica matematica venga contaminata da dettagli grafici. Il modello può mantenere coordinate coerenti dal punto di vista fisico, mentre l'interfaccia le trasforma per mostrarle correttamente sullo schermo.

Dal punto di vista dei file e dei dataset, ogni snapshot dello swarm dovrebbe contenere coordinate in formato chiaro. Un esempio concettuale di record JSON potrebbe essere:

```
{
  "id": "node_01",
  "position": { "x": 0.15, "y": 0.32, "z": 0.00 },
  "orientation": 1.57,
  "state": "active",
}
```



```
"timestamp": 123456.78
}
```

Questa struttura è leggibile, estendibile e compatibile con simulazioni, dashboard e logging. Anche se il documento non deve trasformarsi in codice, indicare una struttura dati coerente aiuta a collegare il modello matematico all'implementazione reale.

Un altro aspetto importante riguarda la normalizzazione delle coordinate. In alcune simulazioni, può essere utile rappresentare lo spazio in coordinate normalizzate tra 0 e 1. Questo rende il modello indipendente dalla dimensione dello schermo o dell'arena. In altri casi, può essere preferibile usare unità fisiche reali, come millimetri o centimetri, soprattutto quando si collega la simulazione al prototipo fisico. La scelta dipende dallo scopo. Per dimostrazioni grafiche, coordinate normalizzate sono comode; per dimensionamento magnetico, servono unità fisiche reali.

Una soluzione rigorosa consiste nel distinguere tra coordinate normalizzate e coordinate fisiche. Le coordinate normalizzate servono alla visualizzazione; le coordinate fisiche servono al modello. Una trasformazione di scala collega i due livelli:

$$p_{\text{physical}} = \text{scale} \cdot p_{\text{normalized}}$$

Questa distinzione sarà utile quando il progetto passerà da simulazione visuale a confronto con prototipi reali.

Dal punto di vista teorico, le coordinate dei MicroBot rappresentano la proiezione spaziale dello stato del sistema. Il sistema completo non è fatto solo di coordinate, ma senza coordinate non può esistere una forma. La forma dello swarm può essere definita come l'insieme delle posizioni dei nodi e delle relazioni tra esse. In forma sintetica:

$$\text{Forma}(t) = \text{funzione}(P(t), E(t))$$

dove $P(t)$ rappresenta le posizioni ed $E(t)$ rappresenta le connessioni o interazioni tra nodi. Questo conferma che la forma non coincide soltanto con la posizione dei nodi, ma anche con il modo in cui tali nodi sono collegati. Tuttavia, le coordinate restano il primo livello necessario.

In conclusione, le coordinate dei MicroBot costituiscono la base numerica del modello matematico del sistema. Esse permettono di localizzare ogni unità, calcolare relazioni spaziali, definire vicinati, misurare distanze, costruire grafi di connessione, rappresentare pattern e integrare simulazione, telemetria e controllo. Una rappresentazione rigorosa deve distinguere tra coordinate globali e locali, bidimensionali e tridimensionali, continue e discrete, matematiche e visuali, reali e stimate. Questo livello prepara direttamente il punto successivo della Sezione 7, dedicato al modello discreto del moto, in cui le coordinate non saranno più considerate valori statici, ma variabili dinamiche aggiornate nel tempo attraverso regole di movimento, interazione e controllo.

7.3 Modello discreto del moto

Il modello discreto del moto rappresenta il passaggio dalla semplice descrizione delle coordinate alla descrizione dell'evoluzione temporale del sistema MicroBot. Nei punti precedenti sono state definite la rappresentazione spaziale e le coordinate dei singoli nodi; tuttavia, uno swarm non è caratterizzato soltanto dalla posizione istantanea delle sue unità, ma dal modo in cui tali posizioni cambiano nel tempo. Il moto è quindi il processo attraverso cui il sistema attraversa una successione di configurazioni, trasformando la forma dello swarm da uno stato iniziale a uno stato successivo. In questa prospettiva, il MicroBot non viene più trattato come un punto statico nello spazio, ma come un'entità dinamica dotata di posizione, velocità, stato interno e regole di aggiornamento.

Nel contesto del progetto MicroBot, la scelta di un modello discreto è coerente sia con la simulazione software sia con il funzionamento dei sistemi embedded. Un computer, un microcontrollore o una simulazione web non aggiornano il sistema in modo continuo nel senso matematico ideale, ma attraverso cicli successivi. A ogni ciclo, il sistema legge lo stato corrente, calcola nuove condizioni, aggiorna le posizioni e produce una nuova configurazione. Il tempo viene quindi rappresentato come una sequenza di istanti discreti:

$t_0, t_1, t_2, \dots, t_k$

dove ogni istante rappresenta un passo della simulazione o del controllo. La distanza temporale tra due istanti consecutivi può essere indicata con Δt . Questo intervallo definisce la risoluzione temporale del modello: più Δt è piccolo, più il moto risulta fluido e vicino a una rappresentazione continua; più Δt è grande, più l'evoluzione diventa approssimata e potenzialmente instabile. La scelta di Δt è quindi un parametro importante del sistema.

Nel modello discreto, la posizione di ogni MicroBot viene aggiornata passo dopo passo. Se $p_i(t)$ rappresenta la posizione del nodo i al tempo t , la posizione al passo successivo può essere espressa come:

$$p_i(t + \Delta t) = p_i(t) + \Delta p_i(t)$$

dove $\Delta p_i(t)$ rappresenta lo spostamento del nodo durante l'intervallo temporale considerato. Questo spostamento può essere calcolato in molti modi, a seconda del livello di realismo desiderato. In una simulazione semplice, $\Delta p_i(t)$ può essere determinato direttamente da una regola comportamentale, ad esempio muoversi verso un target o allontanarsi da un nodo troppo vicino. In un modello più fisico, invece, lo spostamento deriva dalla velocità e dalle forze agenti sul nodo.

Una formulazione leggermente più completa introduce la velocità:

$$p_i(t + \Delta t) = p_i(t) + v_i(t) \cdot \Delta t$$

dove $v_i(t)$ rappresenta la velocità del MicroBot i al tempo t . Questa relazione permette di distinguere tra posizione e moto: la posizione indica dove si trova il nodo, mentre la velocità indica come sta cambiando la sua posizione. Il sistema può quindi aggiornare prima la velocità in base alle forze o alle regole locali, e poi aggiornare la posizione in base alla velocità. Questo schema è molto comune nelle simulazioni di sistemi dinamici perché consente di rappresentare movimenti progressivi, inerzia semplificata e transizioni più naturali.

Se si vuole includere anche l'accelerazione, il modello può essere esteso ulteriormente:

$$v_i(t + \Delta t) = v_i(t) + a_i(t) \cdot \Delta t$$

$$p_i(t + \Delta t) = p_i(t) + v_i(t + \Delta t) \cdot \Delta t$$

dove $a_i(t)$ rappresenta l'accelerazione del nodo, cioè la variazione della velocità prodotta dalle forze o dalle regole del sistema. In un modello fisico classico, l'accelerazione dipende dalla forza risultante e dalla massa:

$$a_i(t) = F_i(t) / m_i$$

Nel caso MicroBot, tuttavia, non è sempre necessario introdurre subito un modello fisico completo. Nelle prime simulazioni, può essere più utile trattare il moto come comportamento controllato, non come dinamica meccanica perfettamente realistica. Questo significa che i nodi possono essere aggiornati attraverso regole semplificate, mantenendo comunque una coerenza concettuale con le forze reali che verranno introdotte nelle versioni più avanzate.

Il modello discreto del moto può quindi essere sviluppato su più livelli. Il primo livello è puramente cinematico: si aggiornano le posizioni in base a velocità definite dal comportamento. Il secondo livello è dinamico semplificato: si aggiornano le velocità in base a forze virtuali, come attrazione, repulsione, coesione e allineamento. Il terzo livello è fisico più realistico: si includono massa, attrito, inerzia, vincoli di contatto e forze magnetiche approssimate. Questa progressione è coerente con l'evoluzione del progetto MicroBot, perché permette di partire da una simulazione controllabile e poi aumentare gradualmente la complessità.

Nel contesto dello swarm, il moto di ogni MicroBot non deve essere considerato indipendente. La variazione della posizione di un nodo dipende dallo stato degli altri nodi, specialmente da quelli vicini. Se un MicroBot è isolato, può mantenere la propria posizione, muoversi verso un target o seguire un comando del controller. Se invece si trova vicino ad altri nodi, il suo moto può essere influenzato da attrazione magnetica, repulsione

per evitare collisioni, allineamento o vincoli di connessione. La funzione di aggiornamento del moto deve quindi includere il vicinato locale.

Una forma generale del modello può essere:

$$p_i(t + \Delta t) = p_i(t) + f_motion(s_i(t), N_i(t), u_i(t), E(t)) \cdot \Delta t$$

dove $s_i(t)$ rappresenta lo stato interno del nodo, $N_i(t)$ l'insieme dei nodi vicini, $u_i(t)$ il comando ricevuto dal controller o dal sistema di controllo, ed $E(t)$ lo stato dell'ambiente. Questa formulazione è molto importante perché mantiene coerenza con il modello distribuito già definito nelle sezioni precedenti: ogni MicroBot aggiorna il proprio comportamento in base allo stato locale, al vicinato, ai comandi e al contesto.

Nel modello matematico MicroBot, il vicinato $N_i(t)$ svolge un ruolo centrale. Il nodo non deve necessariamente considerare tutti gli altri MicroBot, ma solo quelli entro una certa distanza. Questo riduce la complessità computazionale e riflette il carattere locale delle interazioni magnetiche. Il vicinato può essere definito come:

$$N_i(t) = \{m_j \mid d_{ij}(t) < R_neighbor\}$$

dove $d_{ij}(t)$ rappresenta la distanza tra il nodo i e il nodo j , mentre $R_neighbor$ rappresenta il raggio entro cui un altro nodo viene considerato rilevante. Questo insieme può cambiare a ogni passo temporale, perché i nodi si muovono. Di conseguenza, anche il grafo dello swarm cambia nel tempo. Il modello discreto del moto non aggiorna soltanto posizioni, ma aggiorna implicitamente anche le relazioni tra nodi.

Un aspetto fondamentale è la stabilità numerica del modello. Quando un sistema viene aggiornato a passi discreti, valori troppo grandi di Δt o forze troppo intense possono produrre comportamenti instabili: nodi che saltano oltre il target, oscillazioni eccessive, collisioni non realistiche o esplosioni del sistema simulato. Per evitare questo, il modello deve includere limiti alla velocità, limiti allo spostamento massimo per ciclo e, se necessario, fattori di smorzamento. Questi elementi non sono soltanto accorgimenti tecnici, ma rappresentano una forma di realismo: anche un MicroBot fisico non può cambiare posizione istantaneamente o muoversi con velocità infinita.

Si può quindi introdurre un vincolo di velocità massima:

$$\|v_i(t)\| \leq v_max$$

e un vincolo di spostamento massimo per passo:

$$\|\Delta p_i(t)\| \leq \Delta p_max$$

Questi vincoli impediscono al modello di produrre movimenti troppo bruschi. Nel contesto della simulazione, migliorano la leggibilità visiva e la stabilità del comportamento. Nel contesto del sistema fisico, rappresentano una semplificazione dei limiti reali imposti da massa, attrito, potenza disponibile e geometria dell'ambiente.

Un altro elemento importante è lo smorzamento. In un sistema fisico reale, attrito e dissipazione riducono progressivamente l'energia del movimento. In una simulazione semplificata, se si applicano forze attrattive senza smorzamento, i nodi possono oscillare indefinitamente o attraversarsi. Per questo motivo, è utile introdurre un coefficiente di damping:

$$v_i(t + \Delta t) = \alpha \cdot v_i(t) + a_i(t) \cdot \Delta t$$

dove α è un coefficiente compreso tra 0 e 1. Se α è vicino a 1, il sistema conserva molta velocità; se è più basso, il moto viene smorzato rapidamente. Questo parametro permette di controllare la fluidità e la stabilità della simulazione. Nel caso MicroBot, lo smorzamento può rappresentare attrito con la superficie, resistenza del mezzo o semplicemente una scelta modellistica per evitare instabilità.

Il modello discreto del moto deve anche gestire i target. Molti comportamenti del sistema MicroBot possono essere descritti come movimento verso una posizione desiderata. Se τ_i rappresenta il target assegnato al nodo i , il vettore errore è:

$$e_i(t) = \tau_i - p_i(t)$$

Un movimento semplice verso il target può essere definito come:

$$v_target_i(t) = k_target \cdot e_i(t)$$

dove k_target è un coefficiente che determina quanto rapidamente il nodo tende verso la posizione desiderata. Tuttavia, questo movimento deve essere combinato con le altre regole del sistema, come separazione, coesione e vincoli magnetici. Un nodo non può muoversi verso il target ignorando completamente gli altri, perché potrebbe collidere o rompere connessioni esistenti. Il moto reale dello swarm nasce dalla combinazione di più contributi.

Una formulazione generale può essere:

$$v_i(t) = w_target \cdot v_target_i(t) + w_sep \cdot v_sep_i(t) + w_coh \cdot v_coh_i(t) + w_align \cdot v_align_i(t) + w_mag \cdot v_mag_i(t)$$

dove ogni termine rappresenta un contributo comportamentale: movimento verso target, separazione, coesione, allineamento e interazione magnetica. I coefficienti w definiscono il peso relativo di ciascun comportamento. Questa struttura sarà approfondita nei punti successivi della Sezione 7, ma è importante introdurla già qui perché mostra come il moto discreto sia il risultato di una composizione di regole.

Nel MicroBot, il modello del moto deve distinguere tra movimento libero e movimento vincolato. Un nodo libero può aggiornare la propria posizione in base alle regole di movimento. Un nodo agganciato ad altri nodi, invece, non può muoversi indipendentemente senza considerare il vincolo della connessione. Se due MicroBot sono connessi magneticamente, il loro moto deve essere coordinato: o si muovono insieme, o il sistema deve valutare se la forza applicata supera la soglia di distacco. Questo introduce la necessità di rappresentare le connessioni come vincoli dinamici.

Se due nodi i e j sono agganciati, si può imporre una distanza desiderata d_attach :

$$\|p_i(t) - p_j(t)\| \approx d_attach$$

Questa relazione non deve essere necessariamente perfetta, ma rappresenta un vincolo elastico. Se la distanza aumenta troppo, la connessione può diventare instabile; se diminuisce troppo, i nodi possono sovrapporsi. In una simulazione, questo può essere gestito attraverso una forza elastica o una correzione geometrica. Il modello discreto del moto deve quindi tenere conto non solo delle posizioni libere, ma anche dei legami già formati.

Una connessione magnetica può essere modellata come una molla virtuale:

$$F_attach = -k_attach \cdot (d_{ij} - d_attach)$$

dove d_{ij} è la distanza attuale tra i nodi, d_attach è la distanza desiderata di aggancio e k_attach è un coefficiente di rigidità. Questa rappresentazione non pretende di descrivere esattamente il magnetismo reale, ma è utile per simulare il mantenimento di una connessione. Se i nodi si allontanano troppo, la forza li riavvicina; se si comprimono troppo, la correzione li separa. In questo modo, la connessione diventa un vincolo dinamico invece di una semplice informazione binaria.

Il modello discreto del moto deve inoltre gestire le collisioni. Se due nodi si avvicinano troppo senza essere correttamente agganciati, il sistema deve evitare sovrapposizioni. Questo può essere fatto attraverso una forza di separazione che cresce quando la distanza scende sotto una soglia minima. La separazione non è solo un comportamento dello swarm, ma anche una necessità geometrica. Senza di essa, i nodi potrebbero collapsare nello stesso punto nella simulazione, producendo risultati non fisici.

Una forza di separazione semplice può essere definita come contributo che spinge i nodi lontano quando:

$$d_{ij} < d_{\min}$$

dove $d_{\min}$ rappresenta la distanza minima accettabile. Il vettore di separazione agisce nella direzione opposta al nodo troppo vicino. Questa regola è fondamentale nelle simulazioni multi-agente, perché impedisce accumuli irrealistici e rende il comportamento più leggibile.

Il modello discreto del moto deve anche considerare il caso in cui un nodo raggiunge il proprio target. Se il sistema continua ad applicare una velocità verso il target anche quando il nodo è molto vicino, possono verificarsi oscillazioni. Per evitarle, si introduce una tolleranza ε :

se $\|p_i(t) - \tau_i\| < \varepsilon$, allora il nodo è considerato arrivato.

Quando il nodo è arrivato, la sua velocità verso il target può essere ridotta o annullata. Questa logica è importante nella formazione di pattern, perché permette di stabilizzare la configurazione finale. Il sistema non deve inseguire infinitamente una precisione perfetta, ma accettare una tolleranza compatibile con la scala del modello e con i limiti fisici del prototipo.

Dal punto di vista della simulazione, il modello discreto del moto può essere implementato come ciclo iterativo. A ogni passo temporale, il sistema esegue una sequenza logica:

lettura dello stato corrente;

calcolo dei vicini;

calcolo delle distanze;

calcolo dei contributi di movimento;

applicazione dei vincoli;

aggiornamento delle velocità;

aggiornamento delle posizioni;

aggiornamento delle connessioni;

registrazione dello stato.

Questa sequenza corrisponde alla pipeline computazionale descritta nella Sezione 3, ma applicata specificamente al moto dello swarm. Il modello matematico diventa quindi struttura operativa per il simulatore.

Un aspetto importante riguarda l'ordine degli aggiornamenti. Se i nodi vengono aggiornati uno alla volta e ogni aggiornamento modifica subito lo stato usato dai nodi successivi, il risultato può dipendere dall'ordine di calcolo. Per evitare questo problema, è preferibile usare un aggiornamento sincrono: prima si calcolano tutti i nuovi contributi usando lo stato al tempo t , poi si aggiornano tutte le posizioni contemporaneamente al tempo $t + \Delta t$. Questo è coerente con il concetto di slot temporali e sincronizzazione del sistema. In forma generale:

calcola tutti $\Delta p_i(t)$ usando $P(t)$;

poi aggiorna tutti $p_i(t + \Delta t)$.

Questo garantisce maggiore coerenza e riduce artefatti della simulazione.

Nel sistema reale, gli aggiornamenti potrebbero non essere perfettamente sincroni, perché ogni nodo ha il proprio microcontrollore e può ricevere comandi con piccoli ritardi. Tuttavia, il modello sincrono resta utile come astrazione iniziale. Successivamente, si potranno introdurre ritardi, latenze e asincronie per aumentare il realismo. Anche qui, la progressione deve essere controllata: prima modello pulito e comprensibile, poi complessità distribuita.

Il modello discreto del moto deve essere collegato anche alla telemetria. Ogni aggiornamento dello stato può produrre uno snapshot contenente posizioni, velocità, connessioni, stati magnetici e timestamp. Questa sequenza di snapshot permette di ricostruire il comportamento dello swarm nel tempo, analizzare errori, confrontare simulazioni e visualizzare l'evoluzione. Nel progetto MicroBot, la presenza di data logs e snapshot live si inserisce perfettamente in questa logica: il sistema non si limita a muoversi, ma registra la propria evoluzione.

Una struttura dati concettuale per uno snapshot del moto potrebbe includere:

tempo della simulazione;

lista dei nodi;

posizione di ogni nodo;

velocità di ogni nodo;

stato operativo;

connessioni attive;

eventi generati nel passo.

Questa struttura rende il modello verificabile. Se una simulazione produce un comportamento inatteso, è possibile analizzare gli snapshot per capire quando e perché il sistema ha deviato. La matematica diventa quindi anche strumento di debug.

Dal punto di vista fisico, il modello discreto del moto deve essere considerato una semplificazione. I MicroBot reali non si muoveranno necessariamente secondo le stesse equazioni della simulazione, soprattutto nelle prime versioni, dove il movimento può essere manuale, vincolato, simulato o prodotto indirettamente da interazioni magnetiche. Tuttavia, il modello resta necessario perché definisce il comportamento atteso e permette di progettare la transizione verso sistemi più fisici. La simulazione non deve coincidere subito con la realtà, ma deve fornire un linguaggio rigoroso per descriverla.

Un punto molto importante è che il modello del moto non deve essere confuso con il modello dell'attuazione. Il moto descrive come cambiano le posizioni; l'attuazione descrive quale meccanismo fisico produce quel cambiamento. In una simulazione, un nodo può muoversi direttamente perché il codice aggiorna le coordinate. In un prototipo reale, lo stesso movimento potrebbe richiedere magneti, vibrazioni, superfici, ruote, microattuatori o intervento esterno. La matematica del moto definisce la trasformazione desiderata; l'hardware determina come ottenerla.

Questa distinzione è essenziale per mantenere il documento scientificamente corretto. MicroBot, nelle prime versioni, può simulare il moto e rappresentarlo graficamente; nelle versioni fisiche iniziali, può mostrare stati tramite LED o agganci magnetici; nelle versioni avanzate, potrà integrare attuazione reale. Il modello discreto del moto resta valido come livello astratto comune, anche quando il meccanismo fisico cambia.

Dal punto di vista teorico, il moto discreto rappresenta la forma come processo. La forma dello swarm non è un'immagine statica, ma una sequenza di configurazioni:

$\text{Forma}(t_0), \text{Forma}(t_1), \text{Forma}(t_2), \dots, \text{Forma}(t_k)$

Ogni configurazione deriva dalla precedente attraverso regole di aggiornamento. Questo collega direttamente la Sezione 7 con il livello teorico avanzato dell'indice, in cui il sistema viene interpretato come processo e la forma come evento dinamico. Il modello discreto del moto è la formalizzazione matematica di questa idea: la forma esiste perché viene continuamente aggiornata.

Nel contesto MicroBot, il moto discreto permette anche di rappresentare versioni diverse del sistema. Una versione iniziale può usare regole semplici e dirette; una versione intermedia può aggiungere forze virtuali; una versione avanzata può includere vincoli magnetici e soglie fisiche; una versione futura può integrare dati reali da sensori. Il modello non è quindi definitivo e rigido, ma evolutivo. Come il progetto stesso, può crescere per livelli.

In conclusione, il modello discreto del moto descrive come le coordinate dei MicroBot evolvono nel tempo attraverso passi successivi. Esso introduce il concetto di Δt , posizione aggiornata, velocità, accelerazione, vicinato, vincoli, target, collisioni, connessioni e stabilità numerica. Questo modello costituisce il fondamento operativo della simulazione dello swarm e prepara il passaggio ai punti successivi, in cui verrà approfondito l'aggiornamento delle posizioni. Attraverso il moto discreto, il MicroBot passa da entità collocata nello spazio a entità dinamica capace di partecipare a una trasformazione collettiva. La forma dello swarm diventa così una traiettoria nel tempo, non un semplice disegno geometrico.

7.4 Aggiornamento delle posizioni

L'aggiornamento delle posizioni rappresenta il meccanismo operativo attraverso cui il modello discreto del moto diventa effettivamente eseguibile. Dopo aver definito il moto come evoluzione temporale delle coordinate, è necessario precisare come il sistema calcola, a ogni passo, la nuova posizione di ciascun MicroBot. Questo punto è fondamentale perché collega direttamente la rappresentazione matematica alla simulazione software, alla futura telemetria e alla logica di controllo dello swarm. In altre parole, l'aggiornamento delle posizioni è il processo attraverso cui lo stato geometrico del sistema viene trasformato da una configurazione corrente a una configurazione successiva.

Nel sistema MicroBot, ogni nodo possiede una posizione al tempo t , indicata come $p_i(t)$. Tale posizione può essere bidimensionale, se il sistema viene simulato su un piano, oppure tridimensionale, se si considera uno spazio fisico completo. L'obiettivo dell'aggiornamento è determinare la posizione al tempo successivo, $p_i(t + \Delta t)$, tenendo conto di tutti i contributi rilevanti: velocità corrente, forze virtuali, interazioni magnetiche, vincoli di collisione, target, connessioni e limiti del sistema. La forma generale dell'aggiornamento può essere espressa come:

$$p_i(t + \Delta t) = p_i(t) + \Delta p_i(t)$$

dove $\Delta p_i(t)$ rappresenta lo spostamento calcolato per il nodo i nel passo temporale considerato. Questa formula è semplice, ma contiene tutta la logica dinamica del sistema: la complessità non è nella somma finale, ma nel modo in cui viene determinato lo spostamento.

In una simulazione elementare, $\Delta p_i(t)$ può essere calcolato direttamente come movimento verso una direzione desiderata. Per esempio, se un MicroBot deve muoversi verso un target τ_i , lo spostamento può essere proporzionale al vettore errore:

$$e_i(t) = \tau_i - p_i(t)$$

Il nodo si muove quindi nella direzione del target, con una velocità limitata. Tuttavia, questa soluzione è insufficiente per descrivere uno swarm realistico, perché ignora la presenza degli altri nodi. Nel sistema MicroBot, il movimento di una singola unità non può essere calcolato come se essa fosse isolata. Ogni nodo deve considerare il contesto locale: nodi vicini, connessioni magnetiche, rischio di collisione, necessità di mantenere coesione e obiettivi globali del pattern.

Per questo motivo, l'aggiornamento delle posizioni deve essere costruito come composizione di contributi diversi. In forma generale:

$$\Delta p_i(t) = \Delta p_{\text{target}} + \Delta p_{\text{sep}} + \Delta p_{\text{coh}} + \Delta p_{\text{align}} + \Delta p_{\text{mag}} + \Delta p_{\text{constraints}}$$

dove Δp_{target} rappresenta lo spostamento verso un obiettivo, Δp_{sep} rappresenta la separazione da nodi troppo vicini, Δp_{coh} rappresenta la tendenza a mantenere coesione con il gruppo, Δp_{align} rappresenta

l'allineamento con i vicini, Δp_{mag} rappresenta l'effetto delle interazioni magnetiche e $\Delta p_{\text{constraints}}$ rappresenta le correzioni dovute a vincoli fisici o limiti dello spazio. Questa struttura è importante perché rende il modello modulare: ogni contributo può essere progettato, testato e modificato separatamente.

Il contributo verso target è utile quando il sistema deve raggiungere una formazione specifica, come una linea, un cerchio, una griglia o una struttura più complessa. In questo caso, ogni nodo può ricevere una posizione obiettivo e muoversi gradualmente verso di essa. Il vettore di movimento verso target può essere scritto come:

$$\Delta p_{\text{target}} = k_{\text{target}} \cdot (\tau_i - p_i(t)) \cdot \Delta t$$

dove k_{target} controlla la velocità di convergenza. Se k_{target} è troppo basso, il sistema raggiunge il pattern lentamente; se è troppo alto, i nodi possono oscillare o attraversare il target. Per questo motivo, il coefficiente deve essere scelto in modo prudente, specialmente quando il movimento verso target viene combinato con altre regole.

Il contributo di separazione serve a impedire che i MicroBot si sovrappongano o si avvicinino troppo. Ogni nodo ha una dimensione fisica e quindi occupa uno spazio. Se due nodi entrano sotto una distanza minima, il modello deve introdurre una correzione che li spinga in direzioni opposte. Questo contributo può essere calcolato considerando tutti i vicini troppo vicini:

$$\Delta p_{\text{sep}} = \sum \text{funzione_separazione}(p_i, p_j)$$

per tutti i nodi j tali che $d_{ij} < d_{\text{min}}$. La separazione è fondamentale perché senza di essa il sistema potrebbe produrre configurazioni irrealistiche, in cui molti nodi collassano nello stesso punto a causa dell'attrazione o del movimento verso uno stesso target.

Il contributo di coesione opera invece in senso opposto: impedisce allo swarm di disperdersi eccessivamente. Mentre la separazione evita l'accumulo locale, la coesione mantiene il gruppo unito. Un modo semplice per calcolare la coesione è considerare il centro medio dei vicini del nodo e generare un movimento verso tale centro. Se $N_i(t)$ è l'insieme dei vicini del nodo i , il centro locale può essere definito come:

$$c_i(t) = \text{media delle posizioni dei nodi in } N_i(t)$$

Il contributo di coesione può quindi essere:

$$\Delta p_{\text{coh}} = k_{\text{coh}} \cdot (c_i(t) - p_i(t)) \cdot \Delta t$$

Questo termine rende il comportamento dello swarm più compatto e naturale. Tuttavia, se la coesione è troppo forte, i nodi possono collassare; se è troppo debole, lo swarm può disperdersi. Per questo motivo, deve essere bilanciata con la separazione.

Il contributo di allineamento riguarda la tendenza dei nodi ad assumere orientamenti o direzioni di movimento compatibili con i vicini. Nelle simulazioni di swarm, l'allineamento è una regola fondamentale perché consente ai nodi di muoversi in modo coordinato senza controllo centralizzato totale. Nel sistema MicroBot, l'allineamento ha anche un significato magnetico, perché due nodi non devono solo essere vicini, ma anche orientati correttamente per agganciarsi. In una versione iniziale, l'allineamento può agire sulla velocità; in una versione più avanzata, può agire anche sull'orientamento $\theta_i(t)$ o sul vettore direzione $d_i(t)$.

Il contributo magnetico è direttamente collegato alla Sezione 6. Esso rappresenta l'effetto delle attrazioni, repulsioni e connessioni tra nodi. In una simulazione semplificata, questo contributo può dipendere dalla distanza e dallo stato magnetico dei nodi:

$$\Delta p_{\text{mag}} = \text{funzione}(\text{distanza}, \text{polarità}, \text{intensità}, \text{orientamento}, \text{stato_connessione})$$

Se due MicroBot sono entro la soglia di attrazione e hanno orientamento compatibile, il contributo magnetico può avvicinarli. Se sono troppo vicini o polarizzati in modo incompatibile, può respingerli o generare una correzione. Se sono già agganciati, il contributo magnetico può agire come vincolo, mantenendo la distanza

di connessione. Questo rende l'aggiornamento delle posizioni coerente con la fisica magnetica descritta nella sezione precedente.

Il contributo dei vincoli comprende tutte le correzioni necessarie per mantenere il sistema entro condizioni ammissibili. Questi vincoli possono riguardare i bordi dell'arena, la velocità massima, la distanza minima, la conservazione delle connessioni, la stabilità dei cluster o la compatibilità con un pattern. Per esempio, se dopo l'aggiornamento un nodo uscisse dallo spazio simulato, la posizione deve essere corretta. Se lo spostamento calcolato è troppo grande, deve essere limitato. Se una connessione magnetica verrebbe stirata oltre la soglia di distacco, il sistema deve decidere se mantenere il vincolo o romperlo.

Un punto estremamente importante è l'ordine con cui vengono applicate queste operazioni. Un aggiornamento robusto non dovrebbe modificare immediatamente la posizione di un nodo mentre gli altri stanno ancora usando il vecchio stato. Per evitare dipendenze dall'ordine di calcolo, è preferibile usare una strategia a due fasi. Nella prima fase, il sistema calcola tutti gli spostamenti $\Delta p_i(t)$ usando lo stato corrente $P(t)$. Nella seconda fase, applica simultaneamente tutti gli aggiornamenti:

$$P(t + \Delta t) = P(t) + \Delta P(t)$$

Questo aggiornamento sincrono è più coerente con il modello matematico e riduce artefatti nella simulazione. Se invece i nodi venissero aggiornati uno alla volta, il primo nodo aggiornato influenzerebbe il secondo usando già una posizione nuova, mentre il secondo influenzerebbe il terzo in condizioni ancora diverse. Questo potrebbe produrre comportamenti dipendenti dall'ordine interno del programma, non dalla logica del sistema.

Un'altra regola fondamentale è la limitazione dello spostamento massimo. Dopo aver sommato tutti i contributi, il vettore $\Delta p_i(t)$ potrebbe risultare troppo grande. Questo può accadere se più forze si sommano nella stessa direzione, se un nodo è molto lontano dal target o se una correzione di collisione è intensa. Per evitare salti irrealistici, si introduce un limite:

$$||\Delta p_i(t)|| \leq \Delta p_max$$

Se lo spostamento supera tale valore, viene normalizzato e ridotto al massimo consentito. Questa regola rende il moto più stabile e leggibile, e rappresenta anche un vincolo fisico: un MicroBot reale non potrebbe spostarsi arbitrariamente lontano in un singolo ciclo.

Lo stesso principio vale per la velocità. Se il modello utilizza velocità esplicite, è utile imporre:

$$||v_i(t)|| \leq v_max$$

La velocità massima impedisce comportamenti estremi e permette di controllare la fluidità della simulazione. In un swarm, questo è particolarmente importante perché movimenti troppo rapidi possono generare collisioni, instabilità e connessioni magnetiche non realistiche. La velocità deve essere compatibile con la scala del sistema, con il tempo di aggiornamento e con il tipo di comportamento che si vuole rappresentare.

L'aggiornamento delle posizioni deve anche gestire lo stato dei nodi. Non tutti i MicroBot devono essere aggiornati nello stesso modo. Un nodo inattivo può restare fermo; un nodo in errore può essere escluso dal movimento; un nodo agganciato può seguire il cluster; un nodo leader può muoversi verso un target globale; un nodo in modalità safety può disattivare comportamenti dinamici. Questo significa che la funzione di aggiornamento deve dipendere dallo stato operativo:

$$p_i(t + \Delta t) = \text{update_position}(p_i(t), s_i(t), \text{contesto})$$

dove $s_i(t)$ include informazioni come stato energetico, stato magnetico, stato comunicativo, ruolo e modalità di sicurezza. Il modello matematico deve quindi essere collegato al modello dello stato, non solo alla geometria.

Un esempio importante riguarda i nodi agganciati. Se due o più MicroBot formano un cluster magnetico stabile, il loro aggiornamento non dovrebbe considerarli completamente indipendenti. Il cluster può essere trattato come gruppo, o le connessioni possono essere modellate come vincoli interni. In una simulazione

semplice, si può mantenere costante la distanza tra nodi connessi. In una simulazione più avanzata, si può usare una forza elastica che permette piccole deformazioni ma impedisce separazioni eccessive. Questa distinzione sarà importante per modellare strutture flessibili e riconfigurabili.

Le connessioni magnetiche possono quindi modificare l'aggiornamento delle posizioni attraverso una matrice di adiacenza $A(t)$, dove $A_{ij} = 1$ indica che il nodo i e il nodo j sono connessi. Il sistema può usare questa matrice per applicare vincoli solo alle coppie effettivamente agganciate. La forma dello swarm diventa quindi funzione sia delle posizioni sia della rete di connessioni:

$\text{Forma}(t) = \text{funzione}(P(t), A(t))$

Questa formulazione è più completa rispetto a una semplice lista di coordinate, perché due swarm con le stesse posizioni ma connessioni diverse possono avere stabilità e comportamento differenti.

Un altro aspetto importante riguarda la gestione delle collisioni durante l'aggiornamento. Può accadere che, dopo aver calcolato un movimento verso target o attrazione magnetica, due nodi finiscano troppo vicini. Il sistema deve allora applicare una correzione post-update. Una strategia possibile è eseguire prima l'aggiornamento provvisorio, poi controllare tutte le coppie di nodi e correggere eventuali sovrapposizioni. Questo metodo è comune nelle simulazioni fisiche semplificate. Tuttavia, bisogna evitare correzioni troppo aggressive, perché potrebbero generare oscillazioni. Anche qui, smorzamento e limiti di spostamento aiutano a mantenere stabilità.

L'aggiornamento delle posizioni deve tenere conto anche delle soglie magnetiche. Se due nodi entrano entro la soglia di attacco, il sistema può creare una connessione potenziale o attiva. Se due nodi connessi superano la soglia di distacco, il sistema può rompere la connessione. Questo significa che, dopo ogni aggiornamento, il sistema deve rivalutare le relazioni tra nodi. La sequenza diventa:

aggiorna posizioni;

calcola nuove distanze;

verifica soglie di attacco;

verifica soglie di distacco;

aggiorna grafo delle connessioni.

Questo ciclo collega il moto alla topologia dello swarm. I nodi non cambiano solo posizione: cambiano anche le relazioni tra loro. Questa è una delle caratteristiche fondamentali del sistema MicroBot.

Dal punto di vista computazionale, il calcolo delle interazioni tra tutti i nodi può diventare costoso quando il numero di unità cresce. Se ci sono n MicroBot, confrontare tutte le coppie richiede, in modo diretto, un numero di confronti proporzionale a n^2 . Per piccoli prototipi questo non è un problema, ma per swarm più grandi può diventare inefficiente. In futuro, si potranno usare ottimizzazioni come griglie spaziali, quad-tree, octree o suddivisioni dell'arena per calcolare solo i vicini realmente rilevanti. Questo tema sarà collegato alla scalabilità nelle sezioni successive.

Per le versioni iniziali, tuttavia, un calcolo completo delle distanze tra coppie può essere accettabile e metodologicamente utile, perché è semplice da implementare e da verificare. Anche qui vale il principio incrementale: prima un modello chiaro, poi ottimizzazione. Non serve introdurre strutture complesse prima di avere uno swarm numericamente grande. La complessità deve crescere quando il sistema lo richiede.

Un elemento molto importante è la coerenza tra aggiornamento delle posizioni e visualizzazione. La dashboard o la simulazione web non devono inventare posizioni indipendenti dal modello; devono rappresentare $P(t)$ o una sua trasformazione grafica. Questo evita disallineamenti tra backend, simulazione e interfaccia. Se il modello calcola una posizione, l'interfaccia deve mostrarla; se l'interfaccia consente all'utente di spostare un nodo, tale modifica deve rientrare nello stato del modello. L'aggiornamento delle posizioni è quindi anche una questione di coerenza software.

Dal punto di vista dei file di stato, ogni aggiornamento può generare uno snapshot. Uno snapshot può contenere la posizione prima dell'aggiornamento, la posizione dopo l'aggiornamento, i contributi principali allo spostamento e gli eventi generati. Questo sarebbe estremamente utile per il debug. Se un nodo si muove in modo inatteso, il log potrebbe mostrare se lo spostamento è dovuto a target, separazione, magnetismo, collisione o vincolo. In questo modo il modello matematico diventa osservabile e verificabile.

Un esempio concettuale di informazione registrabile per nodo potrebbe essere:

id nodo;

posizione precedente;

posizione aggiornata;

velocità;

contributo target;

contributo separazione;

contributo magnetico;

connessioni attive;

stato finale;

timestamp.

Questa struttura non deve essere necessariamente implementata subito in modo completo, ma rappresenta una direzione molto utile per la maturazione del sistema. Il progetto MicroBot è già basato su documentazione, logging, simulazione e runtime: collegare l'aggiornamento delle posizioni a log strutturati aumenterebbe molto la qualità tecnica.

Un ulteriore aspetto riguarda la differenza tra aggiornamento ideale e aggiornamento fisico. Nella simulazione, il sistema può aggiornare direttamente le coordinate. Nel mondo reale, invece, un MicroBot non cambia posizione perché una variabile viene modificata, ma perché una forza o un attuatore produce movimento. Questo significa che il modello di aggiornamento deve essere interpretato come rappresentazione del comportamento atteso, non necessariamente come descrizione diretta dell'hardware attuale. Nelle prime versioni, il moto può essere simulato; nelle versioni fisiche successive, il movimento potrà essere confrontato con quello reale.

Questa distinzione è fondamentale per evitare ambiguità documentale. Il modello matematico descrive come il sistema dovrebbe evolvere nella simulazione e nel controllo. L'hardware reale potrà implementare gradualmente questa evoluzione attraverso magneti, bobine, superfici, vibrazioni o altri meccanismi. Il valore del modello è proprio quello di fornire una struttura comune anche quando l'implementazione fisica cambia.

L'aggiornamento delle posizioni può essere usato anche per valutare la qualità di un comportamento. Per esempio, se lo swarm deve formare una linea, si può misurare quanto le posizioni aggiornate si avvicinano alla configurazione desiderata. Se deve mantenere coesione, si può misurare la dispersione. Se deve evitare collisioni, si può controllare la distanza minima. Ogni aggiornamento produce quindi dati utili per valutare prestazioni. Il modello non serve solo a muovere i nodi, ma a misurare se il movimento è corretto.

Una metrica utile può essere l'errore medio rispetto ai target:

$$E_pattern(t) = (1/n) \cdot \sum ||p_i(t) - \tau_i||$$

Se questo errore diminuisce nel tempo, lo swarm sta convergendo verso la formazione. Se resta alto o oscilla, il comportamento deve essere corretto. Questa metrica potrà essere collegata ai test dei pattern nella Sezione 12 e alle metriche di telemetria nella Sezione 13.

Un'altra metrica utile è la distanza minima tra nodi:

$$d_{\min_current}(t) = \min d_{ij}(t)$$

per tutte le coppie i, j . Se questa distanza scende sotto la soglia fisica minima, il sistema sta producendo collisioni o sovrapposizioni. Una terza metrica è il numero di connessioni attive, che indica quanto lo swarm è aggregato. Queste metriche derivano direttamente dall'aggiornamento delle posizioni e rendono il sistema analizzabile.

Dal punto di vista teorico avanzato, l'aggiornamento delle posizioni rende concreta l'idea di forma come evento dinamico. La forma non viene disegnata una volta sola, ma continuamente ricalcolata. Ogni passo temporale produce una piccola trasformazione della configurazione. Lo swarm è quindi una traiettoria nello spazio delle configurazioni:

$$P(t_0) \rightarrow P(t_1) \rightarrow P(t_2) \rightarrow \dots \rightarrow P(t_k)$$

Questa traiettoria è influenzata da regole locali, vincoli fisici, comandi globali e interazioni magnetiche. Il sistema MicroBot non è un oggetto fermo, ma un processo matematico e fisico che attraversa stati successivi.

L'aggiornamento delle posizioni rappresenta anche un ponte verso il concetto di stato globale. La posizione di un nodo è una variabile locale, ma l'insieme delle posizioni di tutti i nodi costituisce uno stato globale. Il controller, il simulatore e la dashboard possono osservare tale stato globale, mentre il singolo nodo può operare su informazioni locali. Questa doppia rappresentazione è coerente con l'architettura ibrida del sistema: globale per analisi e coordinamento, locale per comportamento distribuito.

In conclusione, l'aggiornamento delle posizioni è il meccanismo matematico-operativo attraverso cui il sistema MicroBot evolve nel tempo. Esso trasforma coordinate statiche in dinamica, integra target, vicinato, separazione, coesione, allineamento, magnetismo e vincoli, e produce a ogni ciclo una nuova configurazione dello swarm. La sua progettazione deve essere sincrona, stabile, limitata da velocità e spostamenti massimi, coerente con le connessioni magnetiche e osservabile tramite log e metriche. Questo punto prepara direttamente il successivo, dedicato alla distanza euclidea, perché ogni aggiornamento delle posizioni richiede di calcolare continuamente le distanze tra i nodi, e tali distanze sono la base di quasi tutte le regole di interazione del sistema.

7.5 Distanza euclidea

La distanza euclidea rappresenta una delle grandezze fondamentali del modello matematico MicroBot, perché consente di quantificare in modo preciso la separazione spaziale tra due unità dello swarm. Dopo aver definito la rappresentazione nello spazio, le coordinate dei MicroBot, il modello discreto del moto e l'aggiornamento delle posizioni, diventa necessario introdurre una misura formale della distanza, poiché quasi tutte le regole successive del sistema dipendono da essa. La distanza permette infatti di stabilire se due nodi sono vicini, lontani, potenzialmente interagenti, in collisione, entro la soglia di aggancio, oltre la soglia di distacco o appartenenti allo stesso vicinato operativo.

Nel modello MicroBot, ogni nodo possiede una posizione $p_i(t)$, definita tramite coordinate spaziali. Nel caso bidimensionale, la posizione del nodo i è:

$$p_i(t) = (x_i(t), y_i(t))$$

mentre la posizione del nodo j è:

$$p_j(t) = (x_j(t), y_j(t))$$

La distanza euclidea tra i due nodi è allora definita come:

$$d_{ij}(t) = \sqrt{[(x_j(t) - x_i(t))^2 + (y_j(t) - y_i(t))^2]}$$

Questa formula misura la lunghezza del segmento che collega i due punti nello spazio bidimensionale. Essa rappresenta la distanza geometrica diretta tra i centri dei due MicroBot, assumendo che ciascun nodo sia approssimato, almeno inizialmente, come punto o corpo simmetrico centrato nella propria posizione. Questa approssimazione è utile nelle prime simulazioni perché consente di calcolare rapidamente relazioni spaziali e interazioni tra nodi.

Nel caso tridimensionale, la formula viene estesa aggiungendo la coordinata z:

$$d_{ij}(t) = \sqrt{[(x_j(t) - x_i(t))^2 + (y_j(t) - y_i(t))^2 + (z_j(t) - z_i(t))^2]}$$

Questa forma consente di rappresentare configurazioni volumetriche e interazioni nello spazio reale, preparando il sistema a evoluzioni future in cui i MicroBot non saranno più limitati a una superficie bidimensionale. Anche se molte simulazioni iniziali possono restare su un piano, la formulazione tridimensionale è importante perché mantiene il modello coerente con la visione complessiva del progetto: MicroBot non è soltanto un sistema di punti su schermo, ma un sistema fisico potenzialmente capace di organizzarsi nello spazio.

La distanza euclidea è essenziale perché trasforma le coordinate in relazioni. Le coordinate di un nodo, prese singolarmente, descrivono soltanto dove si trova; la distanza tra due nodi descrive invece il rapporto spaziale tra essi. In uno swarm, questo rapporto è più importante della posizione assoluta, perché il comportamento collettivo nasce dalle relazioni locali. Un MicroBot può non conoscere la propria posizione globale, ma la logica del sistema può comunque essere costruita intorno alla distanza rispetto ai nodi vicini. La distanza diventa quindi una misura operativa della vicinanza.

Dal punto di vista del modello distribuito, la distanza euclidea permette di definire il vicinato di un nodo. Dato un raggio di vicinanza $R_{neighbor}$, l'insieme dei vicini del nodo i può essere definito come:

$$N_i(t) = \{m_j \mid d_{ij}(t) < R_{neighbor}\}$$

Questa definizione è fondamentale perché consente di limitare le interazioni ai nodi rilevanti. Invece di considerare l'intero swarm, ogni MicroBot o ogni funzione di simulazione può concentrarsi sui nodi che si trovano entro una certa distanza. Questo è coerente con la natura fisica del sistema, in cui le interazioni magnetiche, gli urti, l'allineamento e la coesione hanno significato soprattutto localmente. La distanza euclidea diventa quindi il criterio matematico per costruire la rete locale dello swarm.

La distanza euclidea è anche la base delle soglie operative. Nel sistema MicroBot, molte condizioni possono essere definite confrontando la distanza tra due nodi con un valore soglia. Per esempio, se la distanza è inferiore a una soglia minima, i nodi sono troppo vicini e deve essere applicata una regola di separazione. Se la distanza è inferiore a una soglia magnetica, i nodi possono entrare nella zona di influenza del sistema magnetico. Se la distanza è inferiore alla soglia di attacco e l'orientamento è compatibile, può avvenire un aggancio. Se due nodi già connessi superano la soglia di distacco, la connessione può essere considerata persa.

In forma sintetica, la distanza consente di definire condizioni del tipo:

se $d_{ij} < d_{min} \rightarrow$ separazione necessaria

se $d_{ij} < d_{influence} \rightarrow$ interazione possibile

se $d_{ij} < d_{attach} \rightarrow$ aggancio possibile

se $d_{ij} > d_{detach} \rightarrow$ distacco

Queste condizioni mostrano come una grandezza geometrica semplice possa generare stati discreti del sistema. La distanza continua viene trasformata in decisioni logiche: vicino, lontano, connesso, non connesso, stabile, instabile. Questo passaggio è molto importante perché collega il modello matematico continuo alle macchine a stati e alle regole discrete del firmware o della simulazione.

Un aspetto importante è la distinzione tra distanza centro-centro e distanza funzionale. La distanza euclidea calcolata tra p_i e p_j misura normalmente la distanza tra i centri dei MicroBot. Tuttavia, nel sistema magnetico reale, ciò che conta spesso non è la distanza tra i centri, ma la distanza tra superfici, magneti, bobine o punti di aggancio. Se ogni MicroBot ha raggio R , la distanza tra le superfici può essere approssimata come:

$$d_{\text{superficie}} = d_{ij} - 2R$$

nel caso di due nodi uguali e approssimati come sfere o dischi. Questa distanza è più significativa per l'interazione fisica, perché un campo magnetico agisce tra componenti collocati vicino alla superficie, non necessariamente tra centri geometrici astratti. In una simulazione iniziale si può usare la distanza centro-centro; in una simulazione più avanzata, sarà utile usare la distanza tra porte magnetiche o superfici di contatto.

Questa distinzione diventa fondamentale quando il sistema passa da rappresentazione simbolica a modello fisico. Se il simulatore considera agganci possibili sulla base della distanza centro-centro senza tener conto della dimensione reale dei nodi, potrebbe produrre connessioni non realistiche. Per esempio, due nodi di raggio significativo potrebbero risultare “vicini” come centri, ma le loro superfici potrebbero già sovrapporsi oppure essere ancora troppo lontane dalle sorgenti magnetiche interne. Per evitare questo, il modello deve progressivamente includere il raggio, la geometria e i punti di interazione.

Nel caso di porte magnetiche, la distanza euclidea può essere calcolata non tra i centri dei nodi, ma tra punti specifici associati alla superficie del MicroBot. Se porta_{ik} rappresenta la k -esima porta magnetica del nodo i e porta_{jh} rappresenta la h -esima porta del nodo j , la distanza utile diventa:

$$d_{\text{porta}} = \|\text{porta}_{ik} - \text{porta}_{jh}\|$$

Questo modello consente di rappresentare agganci direzionali, orientamento magnetico e compatibilità tra superfici. È più complesso, ma più coerente con la struttura fisica del MicroBot descritta nella Sezione 4 e con il sistema magnetico della Sezione 6. In una prima versione, tuttavia, la distanza tra centri resta sufficiente per definire le regole di base.

La distanza euclidea è anche utilizzata per calcolare direzioni di movimento. Il vettore relativo tra due nodi è:

$$r_{ij}(t) = p_j(t) - p_i(t)$$

La distanza è la norma di questo vettore:

$$d_{ij}(t) = \|r_{ij}(t)\|$$

Il vettore normalizzato:

$$n_{ij}(t) = r_{ij}(t) / d_{ij}(t)$$

indica la direzione dal nodo i verso il nodo j . Questo vettore è fondamentale per applicare forze o spostamenti. Se il nodo i deve avvicinarsi al nodo j , il movimento sarà lungo n_{ij} . Se deve allontanarsi, il movimento sarà lungo $-n_{ij}$. In questo modo, la distanza non fornisce solo una misura scalare, ma permette anche di costruire una direzione d'azione.

Nel comportamento di separazione, ad esempio, se due nodi sono troppo vicini, il nodo i viene spinto nella direzione opposta a j :

$$\Delta p_{\text{sep}} \propto -n_{ij}$$

Nel comportamento di attrazione, invece, il nodo i può essere spinto verso j :

$$\Delta p_{\text{attr}} \propto n_{ij}$$

Questa relazione mostra che distanza e direzione sono inseparabili nel modello del moto. La distanza determina l'intensità o la necessità dell'interazione; il vettore normalizzato determina la direzione in cui applicarla.

La distanza euclidea è inoltre alla base delle funzioni di decadimento. Come visto nel sistema magnetico, molte interazioni diminuiscono con la distanza. Nel modello matematico, questo comportamento può essere rappresentato attraverso funzioni del tipo:

$$w(d) = 1 / (d + \epsilon)$$

oppure:

$$w(d) = e^{(-\lambda d)}$$

oppure ancora tramite funzioni a soglia, in cui l'interazione vale zero oltre un certo raggio. La scelta della funzione dipende dal livello di realismo desiderato. Una funzione continua produce transizioni graduali; una funzione a soglia produce comportamenti più discreti e facili da controllare. Nel progetto MicroBot, può essere utile partire con soglie semplici e poi introdurre decadimenti più realistici quando la simulazione diventa più avanzata.

Un problema pratico è evitare divisioni per zero. Se due nodi occupano la stessa posizione, la distanza d_{ij} diventa zero e il vettore normalizzato non è definito. Anche se una sovrapposizione perfetta non dovrebbe avvenire in un modello fisico corretto, può accadere nelle simulazioni iniziali o in condizioni di errore. Per questo motivo, molte formule includono un piccolo valore ϵ al denominatore, in modo da evitare instabilità numeriche. Questo dettaglio è importante perché mostra la differenza tra formula matematica ideale e implementazione computazionale robusta.

Dal punto di vista computazionale, calcolare tutte le distanze tra n MicroBot richiede il confronto tra tutte le coppie di nodi. Il numero di coppie cresce come $n(n-1)/2$. Per piccoli swarm questo calcolo è semplice; per swarm grandi può diventare costoso. Tuttavia, nelle fasi iniziali del progetto, il calcolo completo delle distanze è accettabile e utile per chiarezza. In futuro, per aumentare la scalabilità, sarà possibile introdurre strutture spaziali come griglie, quad-tree o octree, che permettono di calcolare solo le distanze tra nodi vicini. Questo tema sarà ripreso nella Sezione 19 dedicata alla scalabilità.

La distanza euclidea permette anche di costruire una matrice delle distanze. Per uno swarm con n nodi, si può definire una matrice $D(t)$ in cui l'elemento $D_{ij}(t)$ rappresenta la distanza tra il nodo i e il nodo j :

$$D_{ij}(t) = d_{ij}(t)$$

Questa matrice è simmetrica, perché la distanza tra i e j è uguale alla distanza tra j e i :

$$D_{ij} = D_{ji}$$

Inoltre, la diagonale è nulla, perché la distanza di un nodo da sé stesso è zero:

$$D_{ii} = 0$$

La matrice delle distanze è uno strumento molto utile per analisi, simulazione e debug. Essa permette di vedere rapidamente quali nodi sono vicini, quali sono isolati e quali coppie possono interagire. Può essere utilizzata per costruire il grafo dello swarm, per calcolare metriche di coesione e per determinare le connessioni magnetiche potenziali.

A partire dalla matrice delle distanze, si può costruire una matrice di adiacenza $A(t)$, in cui:

$$A_{ij}(t) = 1 \text{ se } d_{ij}(t) < R_{\text{neighbor}}$$

$$A_{ij}(t) = 0 \text{ altrimenti}$$

Questa matrice rappresenta il grafo di vicinanza dello swarm. Se invece la soglia usata è quella di aggancio magnetico, la matrice rappresenta le possibili connessioni fisiche. In questo modo, la distanza euclidea diventa il ponte tra geometria e teoria dei grafi. Lo swarm può essere analizzato non solo come insieme di punti, ma come rete dinamica di relazioni.

La distanza euclidea è fondamentale anche per misurare la coesione globale dello swarm. Una misura semplice è la distanza media tra coppie di nodi:

$D_{\text{media}}(t) = \text{media di } d_{ij}(t) \text{ per tutte le coppie } i, j$

Se questa distanza è alta, lo swarm è disperso; se è bassa, lo swarm è compatto. Tuttavia, una distanza media troppo bassa può indicare collisione o collasso. Per questo motivo, la coesione deve essere interpretata insieme alla separazione minima. Uno swarm ben organizzato non è semplicemente quello con distanza media bassa, ma quello in cui le distanze sono compatibili con la struttura desiderata.

Un'altra metrica utile è la distanza dal centro dello swarm. Se $c(t)$ è il centro geometrico dello swarm, la distanza di ogni nodo dal centro è:

$d_{ic}(t) = ||p_i(t) - c(t)||$

La media di queste distanze fornisce una misura della dispersione radiale. Questa metrica è utile per pattern come cerchi, cluster o formazioni compatte. Per esempio, in una formazione circolare, i nodi dovrebbero avere distanza simile dal centro; in una formazione lineare, invece, la distanza dal centro non basta e devono essere usate altre metriche. La distanza euclidea è quindi base comune per molte misure diverse.

Nel contesto dei pattern, la distanza euclidea permette di valutare quanto un nodo è vicino alla propria posizione target. Se τ_i è il target del nodo i , l'errore di posizione è:

$e_i(t) = ||p_i(t) - \tau_i||$

La media degli errori su tutti i nodi misura quanto lo swarm è vicino alla configurazione desiderata. Questa metrica sarà importante nelle sezioni dedicate ai pattern e alla validazione del comportamento, perché permette di passare da una valutazione visiva a una valutazione numerica.

La distanza euclidea è anche collegata alla telemetria. Se il sistema registra le posizioni dei nodi nel tempo, può calcolare distanze, vicinati, connessioni e metriche di stabilità. Questi valori possono essere salvati nei log, visualizzati nella dashboard o usati per il debug. Per esempio, se una connessione magnetica si rompe, il log può indicare se la distanza aveva superato la soglia di distacco. Se un nodo collide con un altro, il sistema può rilevare che la distanza era scesa sotto d_{min} . In questo modo, la distanza diventa una grandezza diagnostica.

Dal punto di vista della simulazione web, la distanza euclidea deve essere coerente con la scala grafica. Se le coordinate sono normalizzate o espresse in pixel, le soglie devono essere definite nella stessa unità o convertite correttamente. Se invece il modello usa millimetri o centimetri, l'interfaccia deve trasformare queste unità in coordinate visuali. Questo richiede una funzione di scala. Senza una gestione coerente delle unità, il sistema potrebbe mostrare visivamente nodi vicini mentre il modello li considera lontani, o viceversa. La coerenza tra modello e visualizzazione è quindi essenziale.

Un aspetto più avanzato riguarda l'uso di distanze diverse dalla distanza euclidea. In alcune situazioni, potrebbe essere utile usare distanze su griglia, distanze geodetiche, distanze lungo una struttura o distanze topologiche nel grafo. Tuttavia, per il sistema MicroBot di base, la distanza euclidea resta la scelta più naturale, perché descrive direttamente la separazione fisica nello spazio. Le altre distanze potranno essere introdotte in modelli più specifici, ad esempio per pattern vincolati o percorsi su superfici.

La distanza euclidea deve inoltre essere interpretata con attenzione nel passaggio dal 2D al 3D. In 2D, due nodi possono sembrare vicini sulla proiezione del piano, ma in 3D potrebbero essere separati lungo l'asse z . Se il sistema viene esteso a tre dimensioni, tutte le soglie devono essere ricalibrate. Una soglia valida in un piano potrebbe non avere lo stesso significato in volume. Questo conferma la necessità di progettare il modello in modo estendibile, mantenendo la formula generale basata sulla norma del vettore posizione.

Nel modello distribuito reale, la distanza potrebbe non essere direttamente misurabile dal singolo MicroBot. Nelle prime fasi, può essere calcolata dal simulatore o dal controller. In futuro, potrebbe essere stimata tramite sensori di prossimità, visione esterna, misure magnetiche, contatto o altre tecnologie. Questo significa che il modello matematico può usare una distanza ideale d_{ij} , mentre il sistema reale potrebbe usare una stima:

$$\hat{d}_{ij}$$

La differenza tra distanza reale e distanza stimata introduce errore. Il sistema deve quindi tollerare incertezza. Soglie troppo rigide possono fallire se la misura è rumorosa; soglie con margine e hysteresis rendono il comportamento più robusto. Questo collega direttamente la distanza euclidea ai punti successivi su soglie di attacco, soglie di distacco e meccanismo di hysteresis.

Un'altra considerazione importante riguarda il fatto che la distanza non descrive tutto. Due nodi possono avere la stessa distanza ma orientamenti diversi, polarità diverse, stati energetici diversi o ruoli diversi. La distanza è quindi condizione necessaria ma non sufficiente per determinare l'interazione. Per esempio, due nodi entro la soglia di aggancio non devono necessariamente agganciarsi se l'orientamento magnetico non è compatibile. Il modello deve quindi usare la distanza insieme ad altre variabili. Una regola completa potrebbe essere:

aggancio possibile se distanza < soglia_attacco e orientamento compatibile e stato_magnetico valido e safety ok

Questa formulazione evita di ridurre il comportamento dello swarm a una semplice geometria. La distanza è fondamentale, ma deve essere integrata con stato, orientamento e condizioni operative.

Dal punto di vista teorico, la distanza euclidea è il primo passo verso la costruzione dello spazio relazionale del sistema. Lo spazio geometrico dice dove sono i nodi; la distanza dice quanto sono vicini; le soglie dicono quali relazioni possono esistere; il grafo dice quali relazioni sono attive; il comportamento emergente nasce dall'evoluzione di questo grafo. In questo senso, la distanza è una grandezza semplice ma strutturale: senza di essa non esiste vicinato, senza vicinato non esistono interazioni locali, senza interazioni locali non esiste swarm.

Nel progetto MicroBot, questa catena è centrale. La forma collettiva non nasce da un comando unico che impone tutto dall'alto, ma da molte relazioni locali tra unità. La distanza euclidea è ciò che permette di decidere quali unità sono abbastanza vicine da influenzarsi. Essa definisce, in termini matematici, il limite tra indipendenza e relazione. Due nodi lontani sono quasi indipendenti; due nodi vicini entrano nello spazio delle possibilità interattive.

In conclusione, la distanza euclidea costituisce una grandezza fondamentale del modello matematico MicroBot. Essa permette di misurare la separazione tra nodi, definire vicinati, calcolare direzioni di movimento, applicare regole di attrazione e separazione, costruire matrici di distanza, generare grafi dinamici, valutare pattern e supportare telemetria e debug. Pur essendo una formula semplice, la sua funzione nel sistema è profonda: essa trasforma coordinate isolate in relazioni operative. Il punto successivo, dedicato all'interazione tra unità, utilizzerà proprio la distanza euclidea come base per formalizzare le condizioni attraverso cui i MicroBot passano dall'essere entità separate all'essere componenti di uno swarm realmente interconnesso.

7.6 Interazione tra unità

L'interazione tra unità rappresenta il livello del modello matematico in cui i MicroBot cessano di essere considerati come punti indipendenti nello spazio e vengono descritti come elementi di un sistema collettivo. Dopo aver definito coordinate, moto, aggiornamento delle posizioni e distanza euclidea, diventa necessario formalizzare il modo in cui due o più nodi si influenzano reciprocamente. Questa sezione è centrale perché introduce il passaggio dalla geometria alla dinamica relazionale: uno swarm non è semplicemente un insieme di oggetti distribuiti nello spazio, ma una rete di unità che modificano il proprio comportamento in funzione della presenza, della posizione, dello stato e dell'orientamento delle altre.

Nel sistema MicroBot, l'interazione tra unità può avvenire su più livelli. Il primo livello è geometrico, perché due nodi interagiscono potenzialmente quando la loro distanza rientra entro un certo intervallo operativo. Il secondo livello è fisico, perché magneti permanenti, bobine elettromagnetiche e superfici di contatto possono generare attrazione, repulsione, allineamento o vincoli di aggancio. Il terzo livello è logico, perché i

nodi possono scambiare informazioni, ricevere comandi, assumere ruoli o modificare il proprio stato. Il quarto livello è sistemico, perché molte interazioni locali producono una configurazione globale, cioè una forma collettiva dello swarm.

L'interazione tra due MicroBot può essere rappresentata in forma generale come una funzione:

$$I_{ij}(t) = \text{funzione}(p_i(t), p_j(t), s_i(t), s_j(t), \theta_i(t), \theta_j(t))$$

dove $I_{ij}(t)$ rappresenta l'interazione tra il nodo i e il nodo j al tempo t , p_i e p_j rappresentano le posizioni, s_i e s_j rappresentano gli stati interni, mentre θ_i e θ_j rappresentano gli orientamenti. Questa formulazione mostra che l'interazione non dipende da un solo parametro. Due MicroBot non interagiscono soltanto perché sono vicini, ma perché la loro distanza, il loro stato e il loro orientamento rendono possibile una relazione fisica o logica.

La distanza euclidea, introdotta nel punto precedente, costituisce il primo filtro dell'interazione. Se due nodi sono troppo lontani, la loro influenza fisica può essere considerata nulla o trascurabile. Se invece sono entro un raggio operativo, possono entrare nel vicinato reciproco e diventare rilevanti per il calcolo del comportamento. Questo porta alla definizione dell'insieme dei vicini:

$$N_i(t) = \{m_j \mid d_{ij}(t) < R_{\text{neighbor}}\}$$

dove R_{neighbor} rappresenta il raggio di vicinanza. L'interazione tra unità viene quindi calcolata principalmente tra nodi appartenenti al vicinato locale, non tra tutte le coppie possibili del sistema. Questo è coerente sia con la fisica del magnetismo, che ha effetto significativo a corto raggio, sia con il paradigma dei sistemi distribuiti, in cui ogni agente opera sulla base di informazioni locali.

Una volta stabilita la vicinanza, il modello deve determinare il tipo di interazione. Nel sistema MicroBot si possono distinguere diverse categorie principali: interazione attrattiva, interazione repulsiva, interazione di allineamento, interazione di connessione e interazione informativa. Queste categorie non sono necessariamente alternative; in molti casi agiscono contemporaneamente con pesi diversi. Per esempio, due nodi possono attrarsi magneticamente, ma allo stesso tempo applicare una regola di separazione per evitare sovrapposizione fisica. Oppure possono essere vicini ma non compatibili per l'aggancio, generando un effetto di riallineamento prima della connessione.

L'interazione attrattiva rappresenta la tendenza di due unità ad avvicinarsi. Essa può derivare da magneti permanenti, da bobine attive o da una regola virtuale della simulazione. In forma concettuale, una forza attrattiva tra due nodi può essere espressa come:

$$F_{\text{attr}_{ij}}(t) = k_{\text{attr}} \cdot g(d_{ij}(t)) \cdot n_{ij}(t)$$

dove k_{attr} rappresenta l'intensità dell'attrazione, $g(d_{ij})$ è una funzione che diminuisce con la distanza e n_{ij} è il vettore unitario che punta dal nodo i al nodo j . Questa formulazione indica che l'attrazione agisce nella direzione dell'altro nodo e dipende dalla distanza. Se i nodi sono vicini, l'attrazione può essere significativa; se sono lontani, l'effetto si riduce.

Tuttavia, l'attrazione non può essere applicata senza limiti. Se due MicroBot continuassero ad attrarsi senza una regola di arresto o un vincolo geometrico, potrebbero sovrapporsi nel modello simulativo. Per questo motivo, l'interazione attrattiva deve essere bilanciata dall'interazione repulsiva. La repulsione serve a mantenere una distanza minima tra i nodi, impedendo collisioni o compressioni non realistiche. Una regola semplice può essere definita così:

se $d_{ij}(t) < d_{\text{min}}$, allora applicare separazione.

La forza o lo spostamento repulsivo agisce nella direzione opposta al nodo vicino:

$$F_{\text{rep}_{ij}}(t) \propto -n_{ij}(t)$$

Questa repulsione non deve essere interpretata necessariamente come una forza fisica reale sempre presente, ma come un vincolo del modello per impedire configurazioni impossibili. Nel sistema fisico, la

separazione minima deriva dal volume dei corpi; nella simulazione, deve essere esplicitata attraverso una regola.

L'interazione di allineamento riguarda invece la tendenza dei nodi ad assumere orientamenti compatibili. Questo livello è particolarmente importante per MicroBot, perché l'interazione magnetica dipende fortemente dalla direzione dei poli, delle bobine e delle superfici di contatto. Due nodi possono essere abbastanza vicini da attrarsi, ma se l'orientamento non è corretto, l'aggancio può risultare debole, instabile o impossibile. Per questo motivo, il modello deve includere una misura di compatibilità angolare.

Nel caso bidimensionale, se θ_i e θ_j rappresentano gli orientamenti dei due nodi, si può definire una differenza angolare:

$$\Delta\theta_{ij} = \theta_j - \theta_i$$

L'interazione di allineamento tende a ridurre questa differenza verso un valore desiderato. In alcuni casi, l'orientamento desiderato può essere uguale, se i nodi devono puntare nella stessa direzione; in altri, può essere opposto o complementare, se i poli magnetici devono fronteggiarsi correttamente. Il concetto importante è che l'allineamento non è soltanto estetico, ma funzionale: determina la qualità dell'interazione fisica.

Nel modello MicroBot, l'allineamento può essere espresso come una correzione dell'orientamento:

$$\theta_i(t + \Delta t) = \theta_i(t) + \Delta\theta_i(t)$$

dove $\Delta\theta_i(t)$ dipende dalla compatibilità con i nodi vicini. In un modello semplice, questa correzione può essere simulata direttamente; in un modello fisico più realistico, può derivare da una coppia magnetica generata dall'interazione tra dipoli. In entrambi i casi, il risultato è lo stesso: il nodo tende progressivamente verso una configurazione di orientamento più favorevole.

L'interazione di connessione rappresenta un livello più discreto. Mentre attrazione e allineamento possono essere trattati come grandezze continue, la connessione indica il passaggio a uno stato relazionale stabile. Due nodi possono essere non connessi, potenzialmente connessi, connessi, in distacco o in errore. Questa relazione può essere rappresentata attraverso una variabile binaria o uno stato più ricco. Nel caso più semplice:

$$A_{ij}(t) = 1 \text{ se i nodi sono connessi}$$

$$A_{ij}(t) = 0 \text{ se i nodi non sono connessi}$$

La matrice $A(t)$, formata da tutti i valori A_{ij} , rappresenta la rete delle connessioni dello swarm. Questa matrice è fondamentale perché descrive la topologia fisica del sistema. Due swarm possono avere posizioni simili ma connessioni diverse, e quindi comportamenti completamente diversi. La forma dello swarm non è definita soltanto dalle coordinate, ma anche dalla rete di connessioni.

La transizione da non connesso a connesso dipende da più condizioni. Una regola iniziale può essere:

connessione possibile se $d_{ij} < d_{\text{attach}}$ e orientamento compatibile e stati compatibili.

Questa regola include tre elementi fondamentali: distanza, orientamento e stato. La distanza verifica che i nodi siano sufficientemente vicini; l'orientamento verifica che l'aggancio sia geometricamente e magneticamente plausibile; lo stato verifica che i nodi possano effettivamente connettersi. Per esempio, un nodo in errore, in modalità safety o con batteria troppo bassa potrebbe non partecipare a nuove connessioni attive. Questo rende il modello più realistico e coerente con l'architettura elettronica e il safety layer.

Una volta creata, una connessione può essere mantenuta anche se le condizioni di attacco non sono più perfettamente soddisfatte. Questo introduce il concetto di stabilità della connessione, che sarà approfondito nei punti successivi tramite soglie di attacco, soglie di distacco e hysteresis. Per ora è sufficiente osservare che l'interazione tra unità non è istantanea e priva di memoria: una relazione già formata modifica il

comportamento futuro dei nodi. Il sistema deve sapere non solo dove sono i nodi, ma anche quali relazioni sono già attive.

L'interazione informativa rappresenta il livello logico della relazione tra unità. Anche se il sistema magnetico produce interazioni fisiche, i MicroBot possono anche scambiare dati o ricevere informazioni dal controller. L'interazione informativa può includere identificativo del nodo, stato energetico, stato magnetico, ruolo, timestamp, segnalazione di errore o conferma di comando. Nel modello matematico, questa interazione può essere rappresentata come scambio di stato:

$$s_i(t + \Delta t) = \text{funzione}(s_i(t), \text{messaggi_ricevuti}, \text{vicini})$$

Questo livello sarà approfondito nella Sezione 8 sulla comunicazione, ma è importante riconoscere già qui che l'interazione tra unità non è soltanto fisica. MicroBot è un sistema cyber-fisico: le relazioni tra nodi esistono sia nel dominio materiale sia nel dominio informazionale.

Una caratteristica centrale del sistema è che le interazioni locali possono essere combinate. Il comportamento del nodo i non dipende da un solo vicino, ma dalla somma o composizione delle interazioni con tutti i vicini rilevanti. In forma generale:

$$F_i(t) = \sum F_{ij}(t)$$

per tutti i nodi j appartenenti al vicinato $N_i(t)$. Questa sommatoria rappresenta la forza o influenza risultante sul nodo i . In un modello comportamentale, al posto della forza fisica si può usare uno spostamento desiderato, una velocità o una variazione di stato. Il principio resta lo stesso: il comportamento locale è il risultato della combinazione delle relazioni locali.

Questa composizione è alla base del comportamento emergente. Nessuna singola interazione definisce da sola la forma dello swarm, ma molte interazioni locali, ripetute nel tempo, producono strutture globali. Una coppia di nodi può formare una connessione; molte connessioni possono formare una catena; molte catene possono formare un cluster; più cluster possono riconfigurarsi in pattern più complessi. Il modello matematico deve quindi descrivere l'interazione elementare in modo abbastanza semplice da essere ripetibile, ma abbastanza ricco da produrre comportamento collettivo.

Un aspetto importante è la simmetria dell'interazione. In molti casi, se il nodo i interagisce con il nodo j , anche il nodo j interagisce con il nodo i . La distanza è simmetrica:

$$d_{ij} = d_{ji}$$

e spesso anche la connessione lo è:

$$A_{ij} = A_{ji}$$

Tuttavia, non tutte le interazioni devono essere perfettamente simmetriche. Un nodo può avere un ruolo diverso, una bobina attiva mentre l'altro ha solo un magnete passivo, oppure uno stato energetico differente. In questi casi, l'effetto dell'interazione può essere asimmetrico. Per esempio, un nodo attivo può attirare un nodo passivo, mentre il nodo passivo non genera lo stesso livello di controllo. Il modello deve quindi consentire sia interazioni simmetriche sia interazioni direzionali, a seconda del tipo di relazione.

Nel caso di una connessione fisica stabile, è spesso utile trattare l'interazione come vincolo. Se due nodi sono agganciati, il loro comportamento non è più completamente indipendente. La distanza tra essi deve restare vicina a un valore di equilibrio, indicato come d_{attach} . Se la distanza aumenta troppo, il vincolo produce una forza di richiamo o, se viene superata una soglia, si rompe. Questo può essere modellato come una molla virtuale:

$$F_{\text{connessione}} = -k \cdot (d_{ij} - d_{\text{attach}})$$

Questa formula permette di rappresentare la connessione come relazione elastica: non completamente rigida, ma orientata a mantenere una distanza desiderata. È una semplificazione utile per simulare strutture flessibili e riconfigurabili.

Un altro aspetto rilevante è la compatibilità tra unità. Non tutti i MicroBot devono necessariamente potersi collegare con tutti gli altri nello stesso modo. In versioni avanzate, potrebbero esistere ruoli, polarità, stati o porte magnetiche diverse. La compatibilità può essere rappresentata da una funzione:

$$C_{ij} = \text{compatibilità}(m_i, m_j)$$

Se C_{ij} è alta, l'interazione è favorita; se è bassa, l'interazione è debole o impedita. Questa funzione può dipendere da polarità, orientamento, ruolo, stato operativo o configurazione corrente. Anche se nelle versioni iniziali tutti i nodi possono essere considerati omogenei, introdurre il concetto di compatibilità rende il modello più estendibile.

Il concetto di porte magnetiche rafforza questa idea. Un MicroBot può avere più punti di interazione, ciascuno con orientamento, polarità e stato. L'interazione non avviene allora genericamente tra due nodi, ma tra due porte specifiche. Una relazione più precisa può essere:

$$I_{ij} = \text{funzione}(\text{porta}_{ik}, \text{porta}_{jh})$$

Questo modello permette di rappresentare agganci direzionali e strutture più complesse. Per una prima simulazione può essere troppo dettagliato, ma è coerente con la visione fisica del progetto e potrà essere introdotto nelle versioni successive.

L'interazione tra unità deve anche tenere conto dei limiti energetici. Una bobina attiva può generare un'interazione più forte, ma consuma energia e produce calore. Un nodo con batteria bassa può ridurre la propria partecipazione alle interazioni attive. Questo significa che l'interazione fisica può dipendere dallo stato energetico:

$$I_{ij}(t) = \text{funzione}(\dots, E_i(t), E_j(t))$$

dove E_i ed E_j rappresentano le energie disponibili nei nodi. Questo collegamento tra energia e interazione è molto importante perché evita di descrivere lo swarm come sistema ideale privo di costi. Nel MicroBot reale, ogni interazione attiva ha un costo, e il modello deve poterlo rappresentare.

Il safety layer introduce ulteriori vincoli. Anche se due nodi sono vicini e compatibili, l'interazione può essere vietata se uno dei due è in stato di sicurezza, se la temperatura è troppo alta, se la batteria è troppo bassa o se è stato rilevato un errore elettrico. Una regola completa di interazione deve quindi includere una validazione:

$$\text{interazione_attiva} = \text{condizione_geometrica} \wedge \text{condizione_magnetica} \wedge \text{condizione_stato} \wedge \text{condizione_sicurezza}$$

Questa formulazione è molto coerente con la progettazione rigorosa del sistema. Il comportamento fisico non deve dipendere solo dal desiderio di formare una struttura, ma anche dalla capacità reale e sicura del nodo di partecipare a quella struttura.

Dal punto di vista della simulazione, l'interazione tra unità può essere implementata a diversi livelli di complessità. In una simulazione minima, due nodi interagiscono se sono entro una certa distanza. In una simulazione intermedia, l'intensità dipende dalla distanza. In una simulazione avanzata, l'interazione dipende da distanza, orientamento, polarità, stato magnetico, energia e connessioni precedenti. Questa progressione è coerente con il metodo del progetto MicroBot: partire da regole semplici, verificarle, poi aumentare progressivamente realismo e complessità.

Nel modello dati, ogni interazione rilevante potrebbe essere rappresentata come una relazione esplicita. Per esempio:

$$\text{interaction} = (\text{node_a}, \text{node_b}, \text{distance}, \text{type}, \text{strength}, \text{state}, \text{timestamp})$$

dove type può indicare attrazione, repulsione, allineamento, connessione o comunicazione; strength rappresenta l'intensità stimata; state rappresenta se l'interazione è potenziale, attiva, stabile o in errore.

Questa struttura sarebbe utile per logging, telemetria, dashboard e debug. Uno swarm complesso non deve essere osservato soltanto come insieme di nodi, ma anche come insieme di relazioni.

La rappresentazione delle interazioni come grafo dinamico è particolarmente potente. Ogni nodo del grafo rappresenta un MicroBot; ogni arco rappresenta una relazione. Gli archi possono avere pesi, tipi e stati. Un arco magnetico può indicare connessione fisica; un arco di vicinanza può indicare prossimità; un arco comunicativo può indicare scambio dati. Il sistema completo può quindi essere descritto come un grafo multilivello:

$$G(t) = (V, E_{\text{geo}}(t), E_{\text{mag}}(t), E_{\text{comm}}(t))$$

dove E_{geo} rappresenta relazioni geometriche di vicinanza, E_{mag} relazioni magnetiche e E_{comm} relazioni informative. Questa formalizzazione rende chiaro che lo swarm è una rete dinamica, non solo una nuvola di punti.

La dinamica delle interazioni può essere descritta come evoluzione del grafo nel tempo. Quando i nodi si avvicinano, compaiono nuovi archi di vicinanza; quando entrano nella soglia magnetica, compaiono archi magnetici potenziali; quando si agganciano, gli archi diventano connessioni attive; quando si separano, gli archi vengono rimossi. La forma dello swarm è quindi legata alla topologia del grafo. Cambiare forma significa cambiare connessioni, non soltanto cambiare coordinate.

Questa idea è molto importante per il concetto di materia programmabile. Una materia programmabile non è soltanto un insieme di particelle che si muovono, ma un insieme di unità che modificano le proprie relazioni. La forma emerge dalla rete di relazioni. Nel MicroBot, l'interazione tra unità è il meccanismo attraverso cui questa rete viene costruita, mantenuta e trasformata.

Dal punto di vista dei comportamenti collettivi, l'interazione tra unità prepara le regole di coesione, separazione e allineamento. La coesione nasce da interazioni attrattive che mantengono il gruppo unito. La separazione nasce da interazioni repulsive che impediscono sovrapposizioni. L'allineamento nasce da interazioni orientative che rendono coerenti direzioni e configurazioni. Questi tre comportamenti non sono funzioni isolate, ma tre modi diversi di interpretare e combinare l'interazione locale tra nodi.

Un altro aspetto importante è la temporalità dell'interazione. Un'interazione può essere istantanea, temporanea o persistente. La vicinanza geometrica può durare pochi istanti; una connessione magnetica può durare più a lungo; una relazione informativa può essere aggiornata periodicamente; un ruolo nello swarm può persistere per una fase intera del comportamento. Il modello deve quindi distinguere tra interazioni brevi e stati relazionali stabili. Questo sarà importante per evitare oscillazioni e per introdurre hysteresis.

L'interazione tra unità può anche generare eventi. Se due nodi entrano entro una soglia, il sistema può registrare un evento di `proximity_detected`. Se si agganciano, può registrare `attach_event`. Se si separano, `detach_event`. Se una connessione fallisce, `attach_failed`. Questi eventi sono importanti per il logging e la telemetria. Il modello matematico non deve essere soltanto continuo, ma anche event-driven: alcune transizioni hanno significato discreto e devono essere registrate.

Un possibile insieme di eventi relazionali può includere:

vicinanza rilevata;

interazione magnetica potenziale;

aggancio riuscito;

aggancio fallito;

connessione stabile;

distacco;

collisione evitata;

orientamento incompatibile;

interazione rifiutata per safety.

Questi eventi rendono il sistema osservabile e aiutano a collegare simulazione, dashboard e firmware. Nei tuoi sistemi, dove la documentazione e i log sono parte importante dell'ecosistema, questa struttura sarebbe molto coerente.

Dal punto di vista teorico avanzato, l'interazione tra unità è il punto in cui il concetto di stato individuale si trasforma in stato collettivo. Il singolo MicroBot ha posizione, energia, stato magnetico e logica locale. Tuttavia, lo swarm esiste solo quando questi stati individuali entrano in relazione. Lo stato globale può quindi essere rappresentato come:

$$S_{\text{global}}(t) = (S_{\text{nodes}}(t), E_{\text{relations}}(t))$$

dove $S_{\text{nodes}}(t)$ è l'insieme degli stati dei nodi ed $E_{\text{relations}}(t)$ è l'insieme delle relazioni attive. Questa rappresentazione è più completa rispetto a una semplice lista di nodi, perché include la struttura delle interazioni.

La qualità di uno swarm può essere valutata anche attraverso le sue interazioni. Uno swarm molto frammentato avrà pochi archi attivi; uno swarm troppo collassato avrà molte interazioni troppo ravvicinate; uno swarm ben strutturato avrà connessioni coerenti con il pattern desiderato. Le interazioni diventano quindi metriche di qualità. Non basta sapere dove sono i MicroBot; bisogna sapere come sono collegati e come si influenzano.

In conclusione, l'interazione tra unità costituisce il nucleo relazionale del modello matematico MicroBot. Essa descrive come i nodi si influenzano attraverso distanza, orientamento, magnetismo, stato energetico, sicurezza e comunicazione. Le interazioni possono essere attrattive, repulsive, orientative, informative o di connessione, e la loro composizione genera il comportamento collettivo dello swarm. Formalizzare l'interazione significa trasformare un insieme di unità isolate in un sistema realmente distribuito, capace di costruire forme, mantenere strutture, correggere configurazioni e produrre emergenza. Questo punto prepara direttamente i successivi, dedicati alle soglie di attacco e distacco, dove verranno definite le condizioni precise attraverso cui un'interazione potenziale diventa connessione stabile oppure viene interrotta.

7.7 Soglie di attacco

Le soglie di attacco rappresentano il criterio matematico e operativo attraverso cui il sistema MicroBot stabilisce quando due unità possono passare da una semplice condizione di vicinanza a una condizione di possibile connessione. Dopo aver definito l'interazione tra unità, è necessario introdurre una regola capace di distinguere tra interazioni deboli, interazioni potenziali e interazioni sufficientemente forti da generare un aggancio. La soglia di attacco svolge esattamente questa funzione: definisce il limite entro il quale due MicroBot sono abbastanza vicini, compatibili e coerenti nello stato da poter entrare in una relazione di aggancio.

Nel sistema MicroBot, una soglia di attacco non deve essere interpretata come un semplice valore numerico di distanza. Sarebbe riduttivo dire che due nodi si agganciano solo perché la loro distanza è inferiore a un certo valore. In un sistema magnetico reale, l'aggancio dipende anche dall'orientamento, dalla polarità, dalla geometria delle superfici, dallo stato magnetico attivo o passivo, dall'energia disponibile, dal livello di sicurezza e dalla presenza di eventuali vincoli già attivi. Per questo motivo, la soglia di attacco deve essere considerata come una condizione composita, cioè come l'insieme delle condizioni minime che rendono possibile la creazione di una connessione.

In forma semplificata, la condizione di attacco può essere espressa come:

attacco possibile se $d_{ij}(t) < d_{\text{attack}}$

dove $d_{ij}(t)$ rappresenta la distanza tra il MicroBot i e il MicroBot j al tempo t , mentre d_{attack} rappresenta la soglia di attacco. Questa formulazione è utile come primo livello del modello, perché introduce un criterio chiaro: se i due nodi sono troppo lontani, l'aggancio non può avvenire; se sono abbastanza vicini, l'aggancio diventa possibile. Tuttavia, questa regola deve essere estesa per rappresentare correttamente il sistema MicroBot.

Una formulazione più completa può essere:

attacco possibile se
 $d_{ij}(t) < d_{\text{attack}}$
e orientamento_compatibile(i, j)
e polarità_compatibile(i, j)
e stato_operativo_valido(i, j)
e safety_ok(i, j)

Questa struttura chiarisce che la distanza è condizione necessaria, ma non sufficiente. Due nodi possono trovarsi molto vicini ma non essere orientati correttamente; possono essere vicini e orientati, ma avere polarità incompatibile; possono essere fisicamente compatibili, ma trovarsi in stato di errore, batteria bassa o protezione attiva. La soglia di attacco, quindi, non è soltanto geometrica: è geometrica, magnetica, logica e di sicurezza.

Dal punto di vista fisico, la soglia di attacco rappresenta la distanza entro cui la forza magnetica disponibile può superare le resistenze locali. Come discusso nella Sezione 6, una forza magnetica utile deve essere sufficiente a vincere attrito, piccoli disallineamenti, peso relativo, imperfezioni di superficie e resistenze meccaniche. Di conseguenza, la soglia di attacco può essere interpretata come il limite oltre il quale la forza non è più sufficiente a produrre aggancio. In forma concettuale:

d_{attack} è tale che $F_{\text{magnetica}}(d_{\text{attack}}) \geq F_{\text{resistenza_minima}}$

Questa relazione mostra che la soglia di attacco dipende direttamente dalla forza disponibile. Se il sistema usa magneti più forti, la soglia può aumentare; se usa magneti più deboli, la soglia diminuisce. Se la bobina viene attivata, la soglia effettiva può aumentare temporaneamente. Se la batteria è scarica o il duty cycle è limitato, la soglia effettiva può ridursi. La soglia non è quindi necessariamente un valore fisso in tutte le condizioni, ma può diventare una variabile dipendente dallo stato del sistema.

Nel caso di magneti permanenti, la soglia di attacco è determinata principalmente da geometria, materiale magnetico, orientamento e attrito. Poiché il campo è sempre presente, due MicroBot entrano nella zona di possibile aggancio quando la distanza scende sotto il limite in cui l'attrazione diventa fisicamente efficace. In questo caso, il sistema non decide attivamente di creare il campo: il campo esiste già, e la soglia rappresenta il punto in cui tale campo produce un effetto significativo. Questo comportamento è adatto all'allineamento passivo e all'aggregazione spontanea.

Nel caso di bobine elettromagnetiche, invece, la soglia di attacco può essere modulata. Se la bobina è spenta, la soglia può essere nulla o molto ridotta, a meno che siano presenti magneti permanenti. Se la bobina è attiva a bassa intensità, la soglia di attacco può estendersi leggermente. Se la bobina è attiva con maggiore intensità, la zona di influenza può aumentare, sempre entro i limiti energetici e termici del sistema. Questo permette di introdurre un concetto importante: la soglia di attacco attiva.

La soglia di attacco attiva può essere rappresentata come:

$d_{\text{attack_effettiva}}(t) = \text{funzione}(d_{\text{attack_base}}, \text{stato_magnetico}(t), \text{energia}(t), \text{safety}(t))$

dove $d_{\text{attack_base}}$ rappresenta la soglia passiva di base, mentre lo stato magnetico, l'energia disponibile e le condizioni di sicurezza modificano la capacità reale del nodo di agganciarsi. Questa formulazione è molto coerente con MicroBot, perché collega modello matematico, magnetismo, alimentazione e safety layer. Il nodo non ha sempre la stessa capacità di aggancio: essa dipende dal suo stato.

Un aspetto fondamentale riguarda la differenza tra soglia di influenza e soglia di attacco. La soglia di influenza è la distanza entro cui due nodi iniziano a esercitare un effetto debole o potenziale l'uno sull'altro.

La soglia di attacco è più restrittiva: indica la distanza entro cui l'effetto è sufficiente a generare una connessione. È quindi utile distinguere:

$$d_{\text{influence}} > d_{\text{attack}}$$

Nella zona tra $d_{\text{influence}}$ e d_{attack} , i nodi possono percepirsi, avvicinarsi, orientarsi o prepararsi all'aggancio, ma non sono ancora in condizione di connettersi stabilmente. Sotto d_{attack} , invece, il sistema può valutare la creazione effettiva di una connessione. Questa distinzione permette di modellare meglio il processo di aggancio come transizione progressiva, non come evento improvviso privo di preparazione.

La soglia di attacco è strettamente legata all'orientamento. Due nodi entro la distanza di attacco possono comunque non agganciarsi se l'angolo relativo non è compatibile. Per questo motivo, il modello può introdurre una soglia angolare:

$$|\Delta\theta_{ij}| < \theta_{\text{attack}}$$

dove θ_{attack} rappresenta la massima differenza angolare ammessa per considerare l'orientamento compatibile. In questo caso, la condizione completa diventa:

$$d_{ij} < d_{\text{attack}} \\ \text{e } |\Delta\theta_{ij}| < \theta_{\text{attack}}$$

Questa formulazione è particolarmente importante per i MicroBot dotati di superfici magnetiche direzionali, coni troncati, bobine orientate o poli specifici. L'aggancio non deve avvenire in qualunque orientamento, ma solo quando la geometria e il campo rendono stabile la connessione.

Nel caso tridimensionale, la compatibilità di orientamento può essere valutata tramite prodotto scalare tra vettori direzione. Se d_i e d_j rappresentano gli assi principali o magnetici dei due nodi, la compatibilità può essere stimata tramite:

$$\text{compatibilità} = d_i \cdot d_j$$

Il prodotto scalare indica quanto due direzioni sono allineate o opposte. A seconda del tipo di connessione desiderata, il sistema può richiedere direzioni simili, opposte o con un angolo specifico. Questo permette di generalizzare il concetto di soglia angolare anche nello spazio tridimensionale.

Un ulteriore elemento riguarda la polarità. Nel sistema magnetico, poli compatibili possono generare attrazione, mentre poli incompatibili possono generare repulsione o instabilità. La soglia di attacco deve quindi includere una funzione di compatibilità magnetica. Due porte magnetiche possono essere considerate compatibili se le loro polarità sono opposte nel caso di magneti permanenti, o se il campo attivo generato dalla bobina è configurato in modo coerente con l'aggancio. In forma generale:

$$\text{polarità_compatibile} = \text{true oppure false}$$

Questa condizione impedisce al sistema di creare connessioni matematiche tra nodi che, fisicamente, si respingerebbero o non riuscirebbero ad agganciarsi.

Dal punto di vista dello stato operativo, la soglia di attacco deve essere abilitata soltanto se entrambi i nodi sono disponibili. Un MicroBot può trovarsi in stato idle, active, connected, error, safety, low_energy o diagnostic. Non tutti questi stati consentono nuove connessioni. Per esempio, un nodo in errore potrebbe rifiutare l'aggancio; un nodo in safety potrebbe disattivare le bobine; un nodo già connesso a troppi altri nodi potrebbe non accettare ulteriori connessioni; un nodo con batteria insufficiente potrebbe partecipare solo passivamente. Questo introduce una logica di validazione:

$$\text{stato_operativo_valido}(i, j) = \text{true se entrambi i nodi possono connettersi}$$

Questa parte è fondamentale per rendere il modello realistico. In uno swarm maturo, l'aggancio non deve dipendere solo dalla geometria, ma anche dalla capacità effettiva dei nodi di sostenere quella connessione.

Un altro aspetto importante riguarda il numero massimo di connessioni. Ogni MicroBot, a seconda della geometria e delle porte disponibili, può supportare un certo numero di agganci simultanei. Se il nodo è progettato con due estremità magnetiche principali, potrebbe avere due connessioni assiali preferenziali. Se ha più porte, potrebbe avere più connessioni. La soglia di attacco dovrebbe quindi essere verificata solo se esiste una porta libera o una capacità residua di connessione. In forma concettuale:

attacco possibile se $\text{porte_libere}(i) > 0$ e $\text{porte_libere}(j) > 0$

Questo impedisce al modello di creare connessioni infinite o fisicamente impossibili. Anche se nelle simulazioni iniziali questo vincolo può essere semplificato, è importante includerlo nella documentazione perché sarà essenziale per la coerenza fisica futura.

La soglia di attacco può essere rappresentata anche come evento. Quando due nodi passano da distanza maggiore a distanza minore della soglia, il sistema può generare un evento di tipo `attach_candidate` o `attack_threshold_crossed`. Questo evento non significa necessariamente che l'aggancio sia già avvenuto, ma indica che le condizioni geometriche minime sono state raggiunte. Successivamente, il sistema verifica orientamento, polarità e safety. Se tutte le condizioni sono soddisfatte, viene generato un `attach_event`. Questa distinzione tra candidato e aggancio effettivo rende il comportamento più leggibile.

Una possibile sequenza event-driven è:

`proximity_detected` → `attack_candidate` → `orientation_check` → `polarity_check` → `safety_check` → `attach_confirmed`

Se una delle verifiche fallisce, il sistema può generare un evento diverso:

`attach_rejected_orientation`
`attach_rejected_polarity`
`attach_rejected_safety`
`attach_rejected_energy`

Questa struttura sarebbe molto utile per logging e telemetria. Invece di sapere soltanto che due nodi non si sono agganciati, il sistema potrebbe indicare il motivo. Questo aumenta moltissimo la capacità di debug e rende il modello più vicino a un sistema ingegneristico reale.

Dal punto di vista della simulazione, la soglia di attacco può essere inizialmente implementata come valore fisso. Per esempio, tutti i nodi possono avere la stessa soglia `d_attack`. Questa scelta è semplice e utile nelle prime versioni. In una versione più avanzata, ogni nodo o ogni porta magnetica può avere la propria soglia, dipendente dal tipo di magnete, dalla bobina, dallo stato energetico o dal ruolo. In forma generale:

$d_{\text{attack}_{ij}} = \text{funzione}(\text{porta}_i, \text{porta}_j, \text{stato}_i, \text{stato}_j)$

Questo permette di modellare unità non perfettamente identiche o connessioni direzionali. Tuttavia, per mantenere il sistema controllabile, è corretto partire da soglie uniformi e poi introdurre variabilità solo quando necessaria.

Un aspetto delicato è la scelta numerica della soglia. Se `d_attack` è troppo grande, il simulatore potrebbe creare agganci tra nodi che nella realtà sarebbero troppo lontani per interagire. Se è troppo piccola, il sistema potrebbe non agganciare mai o richiedere precisione irrealistica. La soglia deve quindi essere calibrata sulla scala del nodo e sui dati sperimentali. Nelle prime simulazioni può essere scelta empiricamente; nelle versioni successive dovrebbe derivare da test reali di distanza di attrazione e stabilità del contatto.

La soglia di attacco deve essere coerente con la dimensione del MicroBot. Se R è il raggio del nodo, una soglia centro-centro di attacco potrebbe essere vicina a $2R$ più un piccolo margine di attrazione. Se invece si usa la distanza tra superfici, la soglia sarà molto più piccola e rappresenterà lo spazio residuo tra le superfici magnetiche. Questo mostra ancora una volta l'importanza di dichiarare quale tipo di distanza viene usata. Senza questa distinzione, il modello rischia ambiguità.

In un modello basato su distanza centro-centro:

$$d_{\text{attack_center}} \approx 2R + \text{margine_magnetico}$$

In un modello basato su distanza tra superfici:

$$d_{\text{attack_surface}} \approx \text{margine_magnetico}$$

Entrambi i modelli sono validi, ma devono essere usati con coerenza. Per simulazioni semplici, la distanza centro-centro è più comoda; per modelli fisici avanzati, la distanza tra porte o superfici è più corretta.

La soglia di attacco è anche legata al concetto di energia. Due nodi si agganciano se il sistema può passare a una configurazione energeticamente più stabile. La soglia indica il punto in cui il campo magnetico è abbastanza forte da rendere favorevole questa transizione. Se la forza disponibile è bassa, il sistema deve arrivare più vicino per agganciarsi. Se la forza disponibile è alta, la transizione può avvenire a distanza leggermente maggiore. Questo collega soglia di attacco, forza, energia e campo magnetico in un'unica logica fisica.

Dal punto di vista del controllo attivo, la soglia può essere usata per decidere quando attivare una bobina. Il sistema può evitare di accendere una bobina quando il nodo vicino è troppo lontano, perché l'energia sarebbe sprecata. Può invece attivarla quando il nodo entra nella zona di influenza ma non è ancora sotto la soglia passiva di attacco, in modo da estendere temporaneamente la capacità di aggancio. Una strategia possibile è:

se $d_{ij} < d_{\text{influence}}$ ma $d_{ij} > d_{\text{attack_passive}}$, allora attiva impulso di aggancio;

se $d_{ij} < d_{\text{attack_passive}}$, lascia che il magnete permanente completi l'aggancio;

se connessione confermata, disattiva la bobina.

Questa logica combina efficienza energetica e controllo attivo. La bobina non sostituisce il magnete permanente, ma aiuta il sistema nella fase di transizione.

Un altro elemento importante è la prevenzione degli agganci indesiderati. Se due nodi entrano nella soglia di attacco ma non dovrebbero connettersi secondo il pattern corrente, il sistema deve poter rifiutare la connessione. Questo introduce una condizione di compatibilità con l'obiettivo globale:

$$\text{pattern_compatibile}(i, j) = \text{true}$$

Per esempio, in una formazione lineare, un nodo potrebbe dover agganciarsi solo a specifici vicini, non a qualunque nodo vicino. In questo caso, la soglia fisica è necessaria ma non sufficiente: il sistema deve verificare anche che la connessione sia coerente con la struttura desiderata. Questa condizione collega il modello matematico ai pattern della Sezione 10.

In forma completa, una regola di attacco avanzata può essere:

```
attach(i, j) = true se
dij < dattack
e orientation_ok(i, j)
e polarity_ok(i, j)
e state_ok(i, j)
e safety_ok(i, j)
e pattern_ok(i, j)
e capacity_ok(i, j)
```

Questa formula è molto importante perché rappresenta una sintesi rigorosa del sistema. L'aggancio non è un evento casuale né puramente fisico, ma il risultato di più livelli di verifica. Questa è esattamente la natura del progetto MicroBot: fisica, software, stato e controllo cooperano per determinare il comportamento.

Dal punto di vista dell'hysteresis, che verrà approfondita più avanti, la soglia di attacco deve essere distinta dalla soglia di distacco. Questa distinzione è essenziale. Se la stessa soglia venisse usata per attacco e distacco, due nodi vicini al limite potrebbero connettersi e disconnettersi ripetutamente a causa di piccole oscillazioni. Per evitare questo comportamento instabile, si definisce normalmente:

$$d_attack < d_detach$$

Questo significa che per creare una connessione i nodi devono essere abbastanza vicini, ma una volta connessi possono rimanere tali anche se la distanza aumenta leggermente. Il sistema acquisisce così memoria e stabilità. La soglia di attacco determina la nascita della relazione; la soglia di distacco determina la sua fine.

Questa distinzione è molto coerente con il comportamento fisico reale dei magneti. Spesso avvicinare due magneti richiede raggiungere una certa distanza prima che l'attrazione li agganci, ma una volta uniti serve una forza o separazione maggiore per staccarli. La matematica deve rappresentare questa differenza, perché essa contribuisce alla stabilità dello swarm. La soglia di attacco è quindi una parte di una coppia di soglie, non un valore isolato.

La soglia di attacco ha anche un ruolo nella valutazione della qualità della simulazione. Se molti nodi non riescono mai ad agganciarsi, la soglia potrebbe essere troppo restrittiva o le regole di movimento insufficienti. Se invece i nodi si agganciano troppo facilmente e formano cluster caotici, la soglia potrebbe essere troppo permissiva o mancare una verifica di compatibilità. Monitorare quanti attach_candidate diventano attach_confirmed può fornire una metrica utile per il debugging.

Una possibile metrica è:

$$attack_success_rate = attach_confirmed / attach_candidate$$

Se il valore è troppo basso, il sistema genera molte possibilità di aggancio che falliscono; se è troppo alto ma il pattern è sbagliato, il sistema aggancia troppo facilmente. Questa metrica, insieme agli eventi di rifiuto, può aiutare a calibrare soglie, orientamenti e regole di compatibilità.

Nel modello dati, ogni evento di superamento della soglia di attacco potrebbe essere registrato con informazioni precise:

node_a;

node_b;

distance;

attack_threshold;

orientation_error;

polarity_status;

energy_status;

safety_status;

result;

timestamp.

Questa struttura permetterebbe di analizzare non solo quando avviene un aggancio, ma perché avviene o perché fallisce. Nel progetto MicroBot, dove telemetria, logging e documentazione sono elementi centrali, questa granularità sarebbe molto utile.

Dal punto di vista teorico, la soglia di attacco rappresenta il punto in cui una relazione potenziale diventa relazione strutturale. Prima della soglia, due nodi sono semplicemente vicini o influenti; dopo la soglia, possono diventare parti di una struttura comune. La soglia definisce quindi una transizione di stato nel grafo dello swarm: un arco potenziale può diventare arco attivo. In termini di grafo:

se $\text{attach}(i, j) = \text{true}$, allora A_{ij} passa da 0 a 1.

Questo passaggio modifica la topologia dello swarm. Ogni nuova connessione cambia la struttura globale, la stabilità e le possibilità di movimento. La soglia di attacco è quindi una regola locale con conseguenze globali.

In conclusione, le soglie di attacco costituiscono un elemento fondamentale del modello matematico MicroBot. Esse definiscono le condizioni minime affinché due unità possano trasformare una vicinanza spaziale in una connessione fisica o logica stabile. La soglia non dipende soltanto dalla distanza, ma anche da orientamento, polarità, stato operativo, sicurezza, energia, capacità di connessione e compatibilità con il pattern desiderato. In questo modo, il modello evita agganci arbitrari e costruisce una logica rigorosa di formazione delle strutture. Le soglie di attacco rappresentano quindi il punto di nascita delle relazioni nello swarm: il momento in cui due MicroBot smettono di essere semplicemente vicini e diventano potenzialmente parte della stessa forma collettiva. Il punto successivo, dedicato alle soglie di distacco, completerà questa logica definendo quando una connessione già formata deve essere mantenuta, indebolita o interrotta.

7.8 Soglie di distacco

Le soglie di distacco rappresentano il criterio matematico e operativo attraverso cui il sistema MicroBot stabilisce quando una connessione già formata tra due unità deve essere mantenuta, indebolita o interrotta. Dopo aver definito le soglie di attacco, cioè le condizioni necessarie affinché due MicroBot possano entrare in relazione, è necessario formalizzare il processo inverso: la perdita della connessione. Questo punto è fondamentale perché uno swarm riconfigurabile non deve soltanto sapersi aggregare, ma deve anche sapersi separare in modo controllato. Senza una regola di distacco, il sistema rischierebbe di trasformarsi in una struttura rigida, incapace di evolvere nel tempo.

Nel modello MicroBot, una connessione tra due unità può essere rappresentata tramite una variabile relazionale. Se il nodo i e il nodo j sono connessi, si può scrivere:

$$A_{ij}(t) = 1$$

Se invece non sono connessi:

$$A_{ij}(t) = 0$$

La soglia di distacco definisce la condizione che fa passare questa relazione dallo stato attivo allo stato inattivo. In forma semplice, si può esprimere così:

$$\text{distacco se } d_{ij}(t) > d_{\text{detach}}$$

dove $d_{ij}(t)$ rappresenta la distanza tra i due nodi al tempo t e d_{detach} rappresenta la soglia di distacco. Questa formulazione indica che, quando due nodi connessi si allontanano oltre una certa distanza, la connessione non può più essere considerata stabile. Tuttavia, come nel caso dell'attacco, anche il distacco non deve essere interpretato come una regola puramente geometrica. Nel sistema MicroBot reale, il distacco dipende da distanza, forza magnetica residua, orientamento, stato energetico, stato del campo attivo, vincoli meccanici, safety layer e obiettivo comportamentale dello swarm.

La soglia di distacco deve essere distinta dalla soglia di attacco. Questo è uno dei punti più importanti del modello. Se il sistema utilizzasse un'unica soglia per creare e rompere una connessione, i nodi vicini al limite potrebbero oscillare continuamente tra stato connesso e stato disconnesso. Una piccola variazione di

posizione, un errore numerico, una vibrazione o una fluttuazione del modello basterebbero a creare instabilità. Per evitare questo comportamento, il modello introduce due soglie separate:

$$d_{\text{attack}} < d_{\text{detach}}$$

Questa relazione significa che per creare una connessione i nodi devono essere abbastanza vicini, ma una volta che la connessione è stata formata essa può rimanere stabile anche se la distanza aumenta leggermente. Questo comportamento è coerente con la fisica reale dei sistemi magnetici: l'aggancio può richiedere una certa vicinanza iniziale, mentre il distacco può richiedere una separazione maggiore o una forza esterna sufficiente a superare il vincolo.

La differenza tra soglia di attacco e soglia di distacco introduce una memoria nel sistema. Lo stato futuro non dipende soltanto dalla distanza attuale, ma anche dallo stato precedente della relazione. Se due nodi non sono connessi, devono scendere sotto d_{attack} per agganciarsi. Se invece sono già connessi, restano tali finché non superano d_{detach} . Questa memoria è essenziale per stabilizzare il comportamento dello swarm. Senza memoria, il sistema sarebbe troppo sensibile alle oscillazioni; con la memoria, le connessioni diventano più robuste.

In forma logica, il comportamento può essere descritto così:

se $A_{ij}(t) = 0$ e $d_{ij}(t) < d_{\text{attack}}$, allora la connessione può nascere.

se $A_{ij}(t) = 1$ e $d_{ij}(t) < d_{\text{detach}}$, allora la connessione viene mantenuta.

se $A_{ij}(t) = 1$ e $d_{ij}(t) > d_{\text{detach}}$, allora la connessione viene interrotta.

Questa struttura prepara direttamente il meccanismo di hysteresis, che verrà sviluppato nel punto successivo. Tuttavia, già qui è importante riconoscere che il distacco non è il semplice contrario dell'attacco. L'attacco riguarda la nascita di una relazione; il distacco riguarda la fine di una relazione che ha già acquisito stabilità.

Dal punto di vista fisico, la soglia di distacco può essere interpretata come il limite oltre il quale la forza magnetica non è più sufficiente a mantenere la connessione contro le forze che tendono a separare i nodi. In forma concettuale:

$$\text{distacco se } F_{\text{separazione}} > F_{\text{magnetica_residua}}$$

Questa relazione mostra che il distacco non dipende solo dalla distanza, ma dal bilancio delle forze. Se i nodi sono sottoposti a trazione, urti, vibrazioni, attrito o comandi di riconfigurazione, la connessione può rompersi anche prima di raggiungere una distanza teorica molto elevata. Al contrario, se i magneti sono forti o se una bobina attiva rinforza il campo, la connessione può resistere più a lungo. La soglia di distacco è quindi legata alla stabilità reale del vincolo.

Nel caso dei magneti permanenti, la soglia di distacco è determinata principalmente dalla forza del magnete, dalla geometria del contatto, dalla superficie, dall'attrito e dall'orientamento. Una connessione passiva può restare stabile senza consumo energetico, ma può essere difficile da interrompere se il magnete è troppo forte. Questo è un punto molto delicato nel progetto MicroBot: una connessione troppo debole rende instabile lo swarm, mentre una connessione troppo forte rende difficile la riconfigurazione. La soglia di distacco deve quindi essere progettata per mantenere un equilibrio tra stabilità e reversibilità.

Nel caso delle bobine elettromagnetiche, il distacco può essere controllato in modo più attivo. Se la connessione è mantenuta o rinforzata da un campo generato da una bobina, il sistema può ridurre il duty cycle, spegnere la bobina o, in versioni più avanzate, invertire la polarità del campo per favorire la separazione. Questo introduce una soglia di distacco dinamica, dipendente dallo stato magnetico attivo. In forma generale:

$$d_{\text{detach_effettiva}}(t) = \text{funzione}(d_{\text{detach_base}}, \text{campo_attivo}(t), \text{polarità}(t), \text{energia}(t), \text{safety}(t))$$

Questa formulazione indica che la distanza alla quale una connessione si rompe può cambiare in base al controllo del campo. Se il campo attivo rinforza la connessione, la soglia effettiva aumenta; se il campo viene ridotto o invertito, la soglia effettiva diminuisce. Questo rende possibile la riconfigurazione controllata dello swarm.

La soglia di distacco deve essere collegata anche alla qualità dell'orientamento. Due nodi possono rimanere vicini, ma perdere compatibilità angolare. Se l'orientamento relativo diventa troppo sfavorevole, la connessione può diventare instabile anche senza un grande aumento della distanza. Per questo motivo, una regola completa di distacco può includere una soglia angolare:

distacco se $|\Delta\theta_{ij}| > \theta_{\text{detach}}$

dove θ_{detach} rappresenta il massimo errore angolare tollerato per mantenere una connessione. Questo è particolarmente importante nei MicroBot dotati di superfici magnetiche direzionali o porte di aggancio specifiche. Una connessione non deve essere considerata stabile solo perché i centri dei nodi sono vicini; deve essere coerente anche l'orientamento delle superfici e dei campi.

Una condizione più completa può essere:

distacco se
 $d_{ij}(t) > d_{\text{detach}}$
oppure $|\Delta\theta_{ij}(t)| > \theta_{\text{detach}}$
oppure $\text{safety_fail}(i, j)$
oppure $\text{energia_insufficiente}(i, j)$
oppure $\text{pattern_requires_detach}(i, j)$

Questa formula mostra che il distacco può avvenire per cause diverse. Può essere un distacco fisico, dovuto alla distanza; un distacco geometrico, dovuto all'orientamento; un distacco di sicurezza, imposto dal safety layer; un distacco energetico, dovuto alla batteria o alla temperatura; oppure un distacco comportamentale, richiesto dal pattern o dalla strategia globale dello swarm.

Il distacco può quindi essere spontaneo o comandato. Il distacco spontaneo avviene quando le condizioni fisiche non sono più sufficienti a mantenere la connessione. Il distacco comandato avviene quando il sistema decide di interrompere una connessione per riconfigurarsi. Questa distinzione è fondamentale per MicroBot. Un sistema passivo può subire distacchi; un sistema controllato può pianificarli. La riconfigurazione richiede proprio questa seconda capacità: non solo reagire alla perdita di connessione, ma decidere quando una connessione deve essere rimossa per permettere una nuova forma.

Nel caso di un pattern dinamico, ad esempio, lo swarm potrebbe dover passare da una configurazione lineare a una configurazione circolare. Alcune connessioni precedenti dovranno essere sciolte, mentre altre dovranno essere create. Il sistema deve quindi disporre di una logica di distacco controllato. In questo caso, la soglia di distacco non è soltanto un limite fisico, ma diventa una condizione abilitata dal piano di riconfigurazione. Il controller può indicare che una connessione non è più desiderata; il nodo può ridurre o annullare il campo attivo; la fisica completa la separazione.

Dal punto di vista del grafo dello swarm, il distacco corrisponde alla rimozione di un arco. Se una connessione tra i nodi i e j viene interrotta, la matrice di adiacenza cambia:

A_{ij} passa da 1 a 0

Questa modifica può alterare la topologia dell'intera struttura. Una connessione rimossa può separare due cluster, liberare un nodo, interrompere una catena o permettere una nuova aggregazione. Per questo motivo, il distacco deve essere trattato con attenzione. In una struttura complessa, rimuovere una connessione sbagliata potrebbe causare instabilità globale. Il sistema deve quindi distinguere tra connessioni periferiche, connessioni critiche e connessioni ridondanti.

Una connessione critica è una connessione la cui rimozione separa lo swarm in parti non desiderate o riduce drasticamente la stabilità della struttura. Una connessione ridondante, invece, può essere rimossa senza compromettere la forma globale perché altri legami mantengono la struttura. In una versione avanzata, il controller potrebbe analizzare il grafo delle connessioni prima di comandare un distacco, evitando di

rompere legami strutturalmente importanti. Questo collega il modello matematico alla teoria dei grafi e alla scalabilità del sistema.

Dal punto di vista della simulazione, il distacco deve essere rappresentato come evento. Quando una connessione viene persa, il sistema dovrebbe generare un evento del tipo:

`detach_event`

Questo evento può includere informazioni come nodi coinvolti, distanza al momento del distacco, orientamento, causa del distacco, stato energetico, stato magnetico e timestamp. Una struttura di log potrebbe essere:

`detach_event = (node_a, node_b, distance, detach_threshold, cause, timestamp)`

La causa potrebbe essere `distance_exceeded`, `orientation_lost`, `safety_triggered`, `energy_low`, `command_requested` o `pattern_transition`. Questo livello di dettaglio sarebbe molto utile per debug, telemetria e documentazione sperimentale. Sapere che una connessione è stata persa non basta; bisogna sapere perché è stata persa.

La soglia di distacco deve essere calibrata con attenzione. Se `d_detach` è troppo vicina a `d_attack`, il sistema diventa instabile e le connessioni possono oscillare. Se `d_detach` è troppo grande, il sistema mantiene connessioni anche quando i nodi sono ormai troppo lontani o la struttura è deformata. Questo può produrre configurazioni non realistiche nella simulazione. La scelta corretta deve quindi bilanciare stabilità e realismo. Nelle prime versioni, la soglia può essere scelta empiricamente; nelle versioni successive, dovrebbe essere calibrata tramite test fisici.

Un criterio pratico può essere:

`d_detach = d_attack + margine_hysteresis`

dove `margine_hysteresis` rappresenta una distanza aggiuntiva che garantisce stabilità. Questo margine deve essere abbastanza grande da evitare oscillazioni, ma non così grande da mantenere connessioni irrealistiche. Il valore dipende dalla scala del modello, dalla dimensione dei nodi, dalla forza magnetica e dalla tolleranza desiderata.

Nel sistema fisico, la soglia di distacco può essere collegata al lavoro necessario per separare due nodi. Una connessione magnetica crea un vincolo energetico; per romperla, bisogna fornire energia sufficiente a superare tale vincolo. Questo lavoro di distacco può essere interpretato come misura della stabilità della connessione. Se il lavoro richiesto è basso, la connessione è fragile; se è alto, è stabile ma meno riconfigurabile. Anche qui ritorna il compromesso fondamentale del progetto MicroBot: stabilità contro flessibilità.

Dal punto di vista del controllo attivo, il sistema può ridurre il lavoro richiesto per il distacco modificando il campo. Se una bobina può opporsi temporaneamente al magnete permanente o ridurre la forza risultante, il distacco diventa più facile. Questa strategia è importante per rendere il sistema riconfigurabile. In una versione semplice, il distacco può essere manuale o simulato; in una versione avanzata, può essere assistito da controllo elettromagnetico. La documentazione deve mantenere chiara questa progressione.

Il distacco deve essere compatibile con il safety layer. Se una bobina si surriscalda, se la batteria è troppo bassa o se il nodo entra in modalità protezione, il sistema può decidere di interrompere alcune connessioni attive o impedire il mantenimento tramite campo controllato. Tuttavia, bisogna distinguere tra disattivazione della bobina e distacco fisico. Se sono presenti magneti permanenti, la connessione potrebbe restare fisicamente attiva anche dopo lo spegnimento della bobina. Questo significa che il safety layer può disattivare il controllo attivo, ma non sempre può garantire separazione fisica completa. È un dettaglio molto importante e realistico.

Nel caso di connessioni puramente passive, il distacco può richiedere una forza meccanica esterna o una geometria di sgancio. Nel caso di connessioni ibride, il distacco può essere facilitato da bobine, ma il magnete permanente può continuare a opporsi. Nel caso di connessioni puramente attive, lo spegnimento

del campo può essere sufficiente a interrompere il legame, ma il consumo energetico durante il mantenimento sarebbe maggiore. Ogni architettura magnetica produce quindi una diversa logica di distacco.

Dal punto di vista della simulazione, è utile distinguere lo stato della connessione in più livelli, non solo connesso/non connesso. Una connessione può essere:

potenziale;

in attacco;

stabile;

in tensione;

in distacco;

interrotta.

Lo stato "in tensione" può indicare che la distanza è vicina alla soglia di distacco o che l'orientamento sta peggiorando. Lo stato "in distacco" può indicare che il sistema ha deciso di rompere la connessione ma la transizione non è ancora completata. Questa rappresentazione è più realistica perché il distacco fisico può richiedere tempo, soprattutto se il campo decade gradualmente o se il sistema deve superare attrito e vincoli geometrici.

Una possibile macchina a stati della connessione può essere:

disconnected → candidate → attached → strained → detaching → disconnected

Questa macchina a stati permette di evitare transizioni troppo brusche e rende il comportamento più interpretabile. Ogni passaggio può generare eventi e log. Per esempio, quando una connessione passa da attached a strained, la dashboard può mostrare un avviso; quando passa da strained a detaching, il sistema può registrare la causa; quando torna disconnected, il grafo viene aggiornato.

Nel contesto dei pattern, il distacco è essenziale per la riconfigurazione. Uno swarm che può solo creare connessioni ma non romperle è un sistema aggregativo, non pienamente programmabile. La programmabilità della materia richiede reversibilità. Ogni forma deve poter essere costruita, mantenuta e poi trasformata. La soglia di distacco è il parametro che rende possibile questa reversibilità. Essa definisce quando una relazione smette di essere conveniente, stabile o desiderata.

La soglia di distacco ha anche un ruolo nella prevenzione dei danni. Se una connessione viene sottoposta a trazione eccessiva, mantenerla artificialmente potrebbe danneggiare il nodo, deformare la struttura o generare instabilità. In alcuni casi, rompere la connessione è più sicuro che mantenerla. Il sistema deve quindi poter trattare il distacco non solo come fallimento, ma come risposta corretta a una condizione non sostenibile. Questo è molto importante dal punto di vista ingegneristico: non tutte le separazioni sono errori; alcune sono protezioni.

Dal punto di vista della telemetria, monitorare le soglie di distacco permette di valutare la stabilità dello swarm. Si può calcolare quante connessioni sono vicine alla rottura, quante si sono rotte in un certo intervallo, quali nodi perdono connessioni più spesso e quali pattern sono più instabili. Una metrica utile può essere:

$\text{detach_rate} = \text{numero_distacchi} / \text{intervallo_temporale}$

Un detach_rate alto può indicare soglie sbagliate, forza magnetica insufficiente, pattern instabile, eccessiva velocità di movimento o problemi di simulazione. Un detach_rate basso può indicare buona stabilità, ma se è troppo basso durante fasi di riconfigurazione potrebbe indicare che il sistema non riesce a separarsi quando necessario. Anche qui, la metrica deve essere interpretata nel contesto.

Un'altra metrica utile è il margine di distacco:

$$\text{detach_margin} = d_{\text{detach}} - d_{ij}$$

Se il margine è alto, la connessione è lontana dalla rottura; se è vicino a zero, la connessione è fragile; se diventa negativo, la connessione dovrebbe essere interrotta. Questo valore potrebbe essere visualizzato nella dashboard o usato dal controller per decidere azioni correttive. Per esempio, una connessione con margine basso potrebbe essere rinforzata temporaneamente da una bobina oppure sostituita da una nuova connessione più stabile.

Dal punto di vista matematico, la soglia di distacco permette di definire la persistenza delle relazioni. Lo swarm non viene ricostruito da zero a ogni passo temporale; conserva una memoria delle connessioni precedenti. Questa memoria rende possibile la formazione di strutture stabili. Il sistema non decide a ogni istante soltanto in base alla distanza attuale, ma considera se una connessione esisteva già. Questo introduce una dimensione storica nel modello, molto coerente con l'idea del sistema come processo.

Nel passaggio dal simulatore al prototipo fisico, la soglia di distacco dovrà essere calibrata con esperimenti. Sarà necessario osservare a quale distanza o forza una connessione si rompe, quanto influisce l'orientamento, quanto influisce la velocità di separazione, se il distacco è netto o graduale, e come cambiano questi valori con magneti diversi, bobine attive o superfici diverse. Questi test permetteranno di sostituire soglie arbitrarie con soglie sperimentali. Anche misure semplici possono aumentare molto la solidità del modello.

Un test di base può consistere nel misurare la forza necessaria per separare due moduli agganciati. Un altro test può misurare la distanza alla quale l'aggancio non si mantiene più se i nodi vengono allontanati lentamente. Un terzo test può confrontare distacco con bobina spenta, bobina attiva in attrazione e bobina attiva in opposizione. Questi dati potrebbero poi alimentare tabelle tecniche e simulazioni calibrate.

Dal punto di vista teorico avanzato, la soglia di distacco definisce il limite di stabilità di una relazione. Una forma collettiva esiste finché le sue relazioni interne restano sotto i limiti di rottura. Quando alcune connessioni superano la soglia, la forma cambia. In questo senso, la forma dello swarm non è soltanto un insieme di coordinate, ma un equilibrio temporaneo tra connessioni che si formano e connessioni che si rompono. La soglia di distacco è il parametro che decide quando un legame non appartiene più alla forma.

Questa idea è molto forte per il concetto MicroBot: la forma non è permanente, ma condizionata. Esiste finché le relazioni locali la sostengono. Quando il sistema modifica le soglie, le forze o gli stati magnetici, cambia la possibilità stessa della forma. La riconfigurazione non è quindi una magia, ma una sequenza di attacchi e distacchi regolati da soglie, stati e vincoli fisici.

In conclusione, le soglie di distacco completano la logica delle soglie di attacco definendo quando una connessione deve essere interrotta. Esse sono essenziali per garantire stabilità, evitare oscillazioni, supportare riconfigurazione e distinguere tra connessioni robuste, fragili e non più sostenibili. La soglia di distacco non è solo una distanza, ma una condizione composita che coinvolge geometria, orientamento, forza magnetica, stato energetico, sicurezza e obiettivi del pattern. Insieme alla soglia di attacco, essa permette di trasformare il comportamento dello swarm in una rete dinamica di relazioni che nascono, persistono e si sciolgono nel tempo. Il punto successivo, dedicato al meccanismo di hysteresis, formalizzerà proprio questa differenza tra nascita e rottura della connessione, introducendo una memoria fondamentale per la stabilità del sistema.

7.9 Meccanismo di hysteresis

Il meccanismo di hysteresis rappresenta il principio attraverso cui il sistema MicroBot introduce memoria nelle relazioni tra unità, evitando che le connessioni magnetiche oscillino continuamente tra stato attivo e stato inattivo quando i nodi si trovano vicino alle soglie operative. Dopo aver definito separatamente le soglie di attacco e le soglie di distacco, diventa necessario formalizzare il motivo per cui tali soglie non devono coincidere. L'hysteresis consente infatti di distinguere il momento in cui una connessione nasce dal momento in cui una connessione si rompe, rendendo il comportamento dello swarm più stabile, più realistico e più coerente con i fenomeni fisici osservabili nei sistemi magnetici.

Nel modello MicroBot, una connessione tra due unità non deve dipendere soltanto dalla distanza istantanea tra esse, ma anche dallo stato precedente della relazione. Se due MicroBot non sono ancora connessi, devono soddisfare condizioni abbastanza restrittive per generare un aggancio. Se invece sono già connessi,

la relazione può essere mantenuta anche in presenza di piccole variazioni di distanza, orientamento o forza. Questo comportamento riflette una proprietà molto comune nei sistemi fisici reali: creare un legame e romperlo non sono processi perfettamente simmetrici. Nel caso dei magneti, avvicinare due unità fino all'aggancio richiede il raggiungimento di una certa condizione di prossimità; una volta agganciate, però, per separarle può essere necessario superare una soglia diversa, spesso più ampia o più energeticamente significativa.

La forma più semplice di hysteresis nel sistema MicroBot è definita attraverso due soglie distinte:

$$d_{\text{attack}} < d_{\text{detach}}$$

dove d_{attack} rappresenta la soglia sotto la quale due nodi non connessi possono agganciarsi, mentre d_{detach} rappresenta la soglia oltre la quale due nodi già connessi devono separarsi. La presenza di due valori diversi introduce una zona intermedia compresa tra d_{attack} e d_{detach} . Questa zona è fondamentale: al suo interno il comportamento del sistema dipende dallo stato precedente. Se i nodi erano già connessi, restano connessi; se non erano connessi, non si agganciano automaticamente. In questo modo, il sistema evita transizioni continue e instabili.

La logica può essere descritta così:

se $A_{ij}(t) = 0$ e $d_{ij}(t) < d_{\text{attack}}$, allora $A_{ij}(t+1)$ può diventare 1.

se $A_{ij}(t) = 1$ e $d_{ij}(t) < d_{\text{detach}}$, allora $A_{ij}(t+1)$ resta 1.

se $A_{ij}(t) = 1$ e $d_{ij}(t) > d_{\text{detach}}$, allora $A_{ij}(t+1)$ diventa 0.

Questa struttura mostra chiaramente il ruolo della memoria. Lo stato $A_{ij}(t+1)$ non dipende soltanto dalla distanza $d_{ij}(t)$, ma anche dal valore precedente di $A_{ij}(t)$. Questo significa che il grafo delle connessioni dello swarm possiede continuità temporale. Le relazioni non vengono ricreate da zero a ogni ciclo, ma persistono finché non viene soddisfatta una condizione reale di rottura. Questo rende il sistema più coerente con un comportamento fisico effettivo.

Senza hysteresis, il sistema avrebbe una sola soglia. In quel caso, se due nodi si trovassero esattamente vicino al limite, piccole oscillazioni della distanza potrebbero causare una sequenza rapida di aggancio, distacco, aggancio, distacco. Questo fenomeno produrrebbe instabilità sia nella simulazione sia nella logica di controllo. La dashboard mostrerebbe connessioni intermittenti, il modello genererebbe eventi inutilmente ripetuti, il controller potrebbe inviare comandi incoerenti e, in un prototipo reale, il sistema potrebbe consumare energia tentando di stabilizzare una relazione che continua a cambiare stato. L'hysteresis serve proprio a impedire questo comportamento.

Dal punto di vista fisico, l'hysteresis può essere interpretata come effetto di stabilità della connessione. Una volta che due MicroBot si sono agganciati, il contatto meccanico, l'allineamento magnetico, la geometria delle superfici e la forza residua contribuiscono a mantenere il legame. Anche se la distanza tra i centri cambia leggermente, la connessione può restare valida perché il sistema ha già raggiunto una configurazione energeticamente favorevole. Per rompere tale configurazione, non basta tornare appena sopra la soglia di attacco; bisogna superare una soglia di distacco più permissiva. Questo introduce una forma di inerzia relazionale.

Nel sistema MicroBot, questa inerzia relazionale è molto importante perché la forma dello swarm deve avere una certa stabilità. Se ogni connessione fosse troppo fragile, la struttura collettiva non potrebbe mantenersi. Una linea, un cluster o una formazione geometrica si dissolverebbero facilmente a causa di piccoli errori numerici, vibrazioni o variazioni locali. L'hysteresis permette invece alla struttura di tollerare perturbazioni moderate. La forma non cambia a ogni minima oscillazione, ma solo quando le condizioni superano un limite significativo.

Il meccanismo di hysteresis può essere applicato non solo alla distanza, ma anche all'orientamento. Come già discusso, due MicroBot devono essere orientati in modo compatibile per agganciarsi. Tuttavia, una volta connessi, possono tollerare piccole variazioni angolari senza separarsi immediatamente. Anche qui si possono definire due soglie:

$$\theta_{\text{attack}} < \theta_{\text{detach}}$$

dove θ_{attack} rappresenta l'errore angolare massimo ammesso per creare una connessione, mentre θ_{detach} rappresenta l'errore angolare massimo tollerato per mantenerla. Questa distinzione è utile perché evita che una connessione venga persa immediatamente a causa di piccole rotazioni o imprecisioni locali. In un sistema fisico reale, un aggancio può restare stabile anche se l'allineamento non è perfetto; il modello deve quindi rappresentare questa tolleranza.

In forma più completa, l'hysteresis può essere applicata alla funzione di connessione:

se non connesso: richiedi condizioni forti di attacco;

se connesso: richiedi condizioni forti di rottura.

Questa frase riassume l'intero principio. La nascita di una connessione deve essere selettiva, per evitare agganci casuali o indesiderati. Il mantenimento di una connessione deve essere tollerante, per evitare instabilità. Il distacco deve essere controllato, per permettere riconfigurazione senza rendere la struttura fragile. Il sistema deve quindi essere prudente nell'agganciare, stabile nel mantenere e chiaro nel separare.

Dal punto di vista energetico, l'hysteresis può essere interpretata come una barriera tra stati. Lo stato disconnesso e lo stato connesso sono due configurazioni distinte. Per passare dallo stato disconnesso allo stato connesso, il sistema deve superare una certa condizione di attacco. Per tornare allo stato disconnesso, deve superare una condizione diversa di distacco. Questa differenza impedisce al sistema di rimbalzare continuamente tra due stati quando si trova vicino al confine. In termini concettuali, l'hysteresis crea una zona di stabilità tra due transizioni.

Nel contesto delle bobine elettromagnetiche, l'hysteresis può essere usata anche per evitare attivazioni e disattivazioni continue del campo. Se una bobina venisse accesa appena la distanza scende sotto una soglia e spenta appena la distanza sale leggermente sopra, il sistema potrebbe generare oscillazioni energetiche, consumo inutile e calore. È più corretto definire soglie separate anche per il controllo attivo. Per esempio, una bobina potrebbe attivarsi quando un nodo entra in una zona di aggancio e disattivarsi solo quando la connessione è confermata o quando il nodo supera una soglia di abbandono più ampia. Questo rende il controllo più stabile.

Una logica possibile è:

attiva assistenza magnetica se $d_{ij} < d_{\text{assist_on}}$;

mantieni assistenza mentre $d_{ij} < d_{\text{assist_off}}$;

disattiva assistenza se $d_{ij} > d_{\text{assist_off}}$ oppure se la connessione è stabile.

Con $d_{\text{assist_on}} < d_{\text{assist_off}}$, il sistema evita di accendere e spegnere rapidamente la bobina quando la distanza oscilla. Questo tipo di hysteresis applicata al controllo elettrico è particolarmente importante per proteggere batteria, MOSFET, bobina e sistema termico. La stabilità matematica diventa stabilità elettronica.

L'hysteresis è anche utile nella gestione del safety layer. Alcuni stati di sicurezza non devono attivarsi e disattivarsi continuamente quando una variabile oscilla intorno alla soglia. Per esempio, se la temperatura stimata della bobina supera una soglia, il sistema può entrare in cooldown. Tuttavia, non deve uscire dal cooldown appena la temperatura stimata scende di pochissimo. È meglio definire una soglia di ingresso e una soglia di uscita:

$\text{thermal_high} \rightarrow$ ingresso in protezione;

$\text{thermal_low} \rightarrow$ uscita dalla protezione;

con $\text{thermal_low} < \text{thermal_high}$.

Questo è lo stesso principio applicato alla sicurezza. La logica con hysteresis evita instabilità anche negli stati energetici, termici e operativi. Nel MicroBot, quindi, l'hysteresis non è soltanto una proprietà delle connessioni magnetiche, ma un principio generale di progettazione dei sistemi dinamici.

Dal punto di vista del grafo dello swarm, l'hysteresis rende più stabile la matrice di adiacenza $A(t)$. Senza hysteresis, piccoli cambiamenti nelle coordinate potrebbero modificare continuamente gli archi del grafo. Con hysteresis, gli archi hanno persistenza. Una connessione creata resta attiva entro un margine definito, e viene rimossa solo quando la condizione di distacco è chiaramente soddisfatta. Questo rende il grafo meno rumoroso e più interpretabile. La struttura dello swarm diventa una rete dinamica, ma non caotica.

La stabilità del grafo è essenziale per i pattern. Se lo swarm deve formare una linea, una catena o un cluster, le connessioni interne devono restare attive abbastanza a lungo da permettere al pattern di consolidarsi. Se le connessioni oscillano, il pattern non converge. L'hysteresis aiuta la convergenza perché impedisce che le relazioni vengano distrutte da micro-variazioni. In una simulazione, questo può fare la differenza tra un comportamento visivamente instabile e una formazione leggibile.

Dal punto di vista della telemetria, l'hysteresis riduce il numero di eventi inutili. Senza hysteresis, il sistema potrebbe generare molti attach_event e detach_event in rapida successione per la stessa coppia di nodi. Questo renderebbe i log rumorosi e difficili da interpretare. Con hysteresis, gli eventi diventano più significativi: un attach_event indica una vera nascita di connessione, mentre un detach_event indica una vera perdita di stabilità. La qualità dei dati migliora perché il sistema non registra transizioni spurie.

Un parametro importante è l'ampiezza della banda di hysteresis, definita come:

$$H = d_{\text{detach}} - d_{\text{attach}}$$

Se H è troppo piccola, il sistema resta sensibile alle oscillazioni. Se H è troppo grande, le connessioni possono essere mantenute troppo a lungo, anche quando la configurazione fisica non è più realistica. Il valore di H deve quindi essere calibrato. Nelle prime simulazioni può essere scelto empiricamente; nelle versioni successive dovrebbe derivare da test fisici su magneti, bobine, superfici, distanze e distacchi. Il margine di hysteresis deve rappresentare una tolleranza reale, non arbitraria.

Nel caso dell'orientamento, si può definire analogamente:

$$H_{\theta} = \theta_{\text{detach}} - \theta_{\text{attach}}$$

Anche qui il margine deve essere bilanciato. Una banda angolare troppo stretta rende le connessioni fragili; una banda troppo ampia consente connessioni deformate o non coerenti. Il sistema deve tollerare piccoli errori, ma non deve mantenere agganci palesemente incompatibili. Questo equilibrio dipende dalla geometria del nodo e dal tipo di porta magnetica.

Nel modello dei dati, ogni connessione può memorizzare non solo il proprio stato, ma anche il margine rispetto alle soglie. Per esempio:

$$\text{attack_margin} = d_{\text{attach}} - d_{ij}$$

$$\text{detach_margin} = d_{\text{detach}} - d_{ij}$$

Quando i nodi non sono connessi, attack_margin indica quanto sono vicini all'aggancio. Quando sono connessi, detach_margin indica quanto la connessione è lontana dalla rottura. Questi valori possono essere usati dalla dashboard per visualizzare connessioni forti, deboli o in tensione. Una connessione con detach_margin alto è stabile; una con detach_margin vicino a zero è fragile; una con detach_margin negativo deve essere rimossa.

La dashboard potrebbe rappresentare questo stato con colori o spessori diversi delle linee tra nodi. Una connessione stabile potrebbe essere mostrata come linea piena; una connessione in tensione come linea tratteggiata o luminosa; una connessione prossima al distacco come linea rossa o pulsante. Questo collegherebbe il modello matematico alla visualizzazione e renderebbe immediatamente comprensibile il comportamento dello swarm.

Dal punto di vista implementativo, il meccanismo di hysteresis può essere gestito tramite una macchina a stati della connessione. Invece di decidere ogni volta da zero, il sistema mantiene uno stato per ogni coppia rilevante:

disconnected;

candidate;

attached;

strained;

detaching.

Lo stato disconnected può passare a candidate quando i nodi entrano nella zona di influenza. Candidate può passare ad attached solo se vengono soddisfatte le condizioni di attacco. Attached può passare a strained quando la distanza o l'orientamento si avvicinano alla soglia di distacco. Strained può tornare ad attached se la relazione si stabilizza, oppure passare a detaching se la soglia viene superata. Detaching infine ritorna a disconnected quando la connessione viene rimossa. Questa macchina a stati rende l'hysteresis più esplicita e controllabile.

Una struttura del genere evita decisioni troppo brusche e permette di introdurre ritardi, conferme o filtri temporali. Per esempio, il sistema potrebbe richiedere che una condizione di distacco sia vera per più cicli consecutivi prima di rompere la connessione. Questo ulteriore filtro temporale aumenta la robustezza contro rumore e oscillazioni. In forma concettuale:

distacco confermato se $d_{ij} > d_{\text{detach}}$ per K passi consecutivi.

Questa regola è molto utile nelle simulazioni e nei sistemi reali, perché evita che un singolo istante anomalo produca un evento strutturale importante. Si tratta di una forma di hysteresis temporale, complementare a quella spaziale.

Analogamente, l'aggancio potrebbe richiedere stabilità per alcuni cicli:

aggancio confermato se $d_{ij} < d_{\text{attack}}$ e orientamento compatibile per K passi consecutivi.

Questa regola impedisce connessioni casuali quando due nodi passano rapidamente vicini senza realmente stabilizzarsi. Nel sistema fisico, un aggancio magnetico richiede tempo e contatto; nel modello, questo può essere rappresentato tramite conferma temporale. Anche qui il comportamento diventa più realistico.

L'hysteresis può quindi essere spaziale, angolare, energetica, termica e temporale. Nel sistema MicroBot, queste forme non sono separate, ma cooperano. La distanza evita oscillazioni geometriche; l'angolo evita oscillazioni di orientamento; l'energia evita accensioni e spegnimenti inutili; la temperatura evita entrate e uscite continue dal safety layer; il tempo evita reazioni a rumore momentaneo. Questo rende l'hysteresis un principio generale di stabilizzazione del sistema.

Dal punto di vista del controllo globale, il controller può usare l'hysteresis per pianificare riconfigurazioni più robuste. Se deve rompere una connessione, non basta portare i nodi appena oltre la soglia di attacco; deve superare la soglia di distacco o inviare un comando attivo di separazione. Se deve creare una connessione, deve portare i nodi entro la soglia di attacco e mantenerli abbastanza stabili da confermare l'aggancio. Questo rende la pianificazione più realistica e impedisce al controller di assumere che le relazioni cambino istantaneamente.

Nel passaggio tra simulazione e realtà, l'hysteresis è uno degli elementi che renderà il modello più credibile. I sistemi fisici reali raramente hanno soglie perfette e istantanee. Attrito, magnetismo, deformazioni, ritardi, rumore, inerzia e tolleranze fanno sì che i cambiamenti di stato abbiano memoria e margine. Un modello senza hysteresis può sembrare pulito matematicamente, ma spesso risulta fragile e irrealistico. Un modello con hysteresis accetta la complessità fisica e la trasforma in regola controllabile.

Dal punto di vista sperimentale, la banda di hysteresis può essere calibrata osservando la differenza tra distanza di aggancio e distanza di separazione. Si può avvicinare lentamente due moduli e registrare il punto in cui si agganciano; poi separarli lentamente e registrare il punto in cui si staccano. La differenza tra questi valori fornisce una stima empirica dell'hysteresis fisica. Test analoghi possono essere condotti per orientamento, forza e bobine attive. Questi dati permetterebbero di rendere il modello molto più solido rispetto a soglie scelte arbitrariamente.

Nel contesto dei file di log, ogni connessione potrebbe conservare la propria storia recente. Per esempio, il sistema può registrare quando una coppia è entrata nella zona di attacco, quando è stata confermata, quanto tempo è rimasta connessa, quando è entrata in tensione e perché si è staccata. Questi dati permetterebbero di calcolare durata media delle connessioni, stabilità dei pattern, frequenza di oscillazione e robustezza dello swarm. L'hysteresis non è quindi solo una regola di controllo, ma anche una fonte di metriche.

Una metrica utile può essere:

$\text{connection_lifetime} = \text{tempo_distacco} - \text{tempo_attacco}$

Se molte connessioni hanno vita molto breve, il sistema è instabile o le soglie sono mal calibrate. Se le connessioni durano troppo quando il pattern richiede riconfigurazione, il sistema è troppo rigido. Un buon sistema MicroBot deve trovare un equilibrio: connessioni abbastanza stabili da mantenere forma, ma abbastanza reversibili da permettere trasformazione.

Un'altra metrica utile è:

$\text{oscillation_count} = \text{numero di attacchi e distacchi ripetuti tra la stessa coppia in un intervallo}$

Un valore alto indica che l'hysteresis è insufficiente o che esiste rumore nel modello. Ridurre oscillation_count è uno degli obiettivi principali dell'introduzione di hysteresis. Questa metrica potrebbe essere molto utile nei test della simulazione.

Dal punto di vista teorico, l'hysteresis conferma che lo stato del sistema non è definito solo dalle coordinate attuali. Due configurazioni geometricamente simili possono avere comportamenti diversi se la storia delle connessioni è diversa. Questo è un concetto importante: lo swarm possiede memoria strutturale. La stessa distanza tra due nodi può significare "non ancora agganciati" oppure "ancora connessi", a seconda dello stato precedente. La forma del sistema è quindi anche storia delle relazioni, non solo geometria istantanea.

Questa idea si collega direttamente al concetto di forma come evento dinamico. Una forma MicroBot non è soltanto l'insieme delle posizioni dei nodi in un istante, ma la continuità delle relazioni che persistono nel tempo. L'hysteresis permette questa continuità. Senza hysteresis, la forma sarebbe fragile e intermittente; con hysteresis, la forma acquisisce resistenza temporale. Non diventa rigida, ma diventa persistente.

Nel concetto di materia programmabile, questa persistenza è fondamentale. Un materiale programmabile non deve cambiare forma in modo casuale a ogni minimo disturbo; deve mantenere una configurazione finché non riceve condizioni sufficienti per trasformarsi. L'hysteresis fornisce proprio questa proprietà: stabilità locale con possibilità di transizione. La materia non è né completamente rigida né completamente instabile; è configurabile entro soglie.

In conclusione, il meccanismo di hysteresis rappresenta una componente essenziale del modello matematico MicroBot. Esso introduce memoria nelle connessioni, distingue soglie di attacco e soglie di distacco, riduce oscillazioni, stabilizza il grafo dello swarm, migliora la qualità della simulazione, protegge il controllo attivo e rende più realistico il comportamento fisico. L'hysteresis non è un dettaglio matematico secondario, ma il principio che permette alle relazioni tra MicroBot di nascere, persistere e terminare in modo controllato. Grazie a essa, lo swarm può mantenere strutture stabili senza perdere la capacità di riconfigurarsi. Il punto successivo, dedicato alle forze dello swarm, userà questa base per descrivere come coesione, separazione, allineamento e interazioni magnetiche si combinano per produrre il comportamento collettivo del sistema.

7.10 Forze dello swarm

Le forze dello swarm rappresentano l'insieme dei contributi matematici e comportamentali che determinano il movimento, la distribuzione e la configurazione collettiva dei MicroBot nello spazio. Dopo aver definito coordinate, distanza euclidea, interazione tra unità, soglie di attacco, soglie di distacco e hysteresis, diventa necessario descrivere quali grandezze guidano effettivamente l'evoluzione dello swarm nel tempo. In questa sezione il sistema non viene più analizzato soltanto come insieme di nodi e connessioni, ma come campo dinamico di influenze locali, in cui ogni MicroBot riceve contributi da più sorgenti: attrazione verso altri nodi, repulsione da nodi troppo vicini, allineamento con il gruppo, tendenza verso un obiettivo, vincoli magnetici, limiti fisici e condizioni di sicurezza.

Nel modello MicroBot, il termine “forza” può essere interpretato in due modi complementari. Nel primo senso, più fisico, una forza rappresenta una grandezza reale capace di produrre accelerazione, movimento, attrazione, repulsione o vincolo tra corpi. Questo è il caso delle forze magnetiche analizzate nella Sezione 6. Nel secondo senso, più computazionale, una forza può rappresentare un vettore comportamentale virtuale, cioè una direzione di aggiornamento della posizione nella simulazione. In una fase iniziale del progetto, molte forze dello swarm possono essere simulate come vettori artificiali; in una fase fisica più avanzata, alcune di esse potranno essere collegate a fenomeni reali, come campi magnetici, contatto, attrito o attuazione. Questa doppia interpretazione è fondamentale perché consente di mantenere continuità tra simulazione e prototipo senza confondere subito il modello matematico con l'hardware finale.

In forma generale, la forza risultante agente sul MicroBot i -esimo può essere rappresentata come somma di più contributi:

$$F_i(t) = F_{\text{coesione}_i}(t) + F_{\text{separazione}_i}(t) + F_{\text{allineamento}_i}(t) + F_{\text{magnetica}_i}(t) + F_{\text{target}_i}(t) + F_{\text{vincoli}_i}(t)$$

Questa formula sintetizza la logica generale dello swarm. Il comportamento del nodo non deriva da un solo comando, ma dalla composizione di più tendenze. La coesione tende a mantenere il gruppo unito; la separazione evita collisioni e sovrapposizioni; l'allineamento favorisce direzioni e orientamenti coerenti; la forza magnetica rappresenta attrazioni, repulsioni e vincoli fisici; la forza verso target guida il sistema verso una configurazione desiderata; i vincoli correggono il comportamento quando il nodo rischia di uscire dallo spazio operativo, violare una soglia o entrare in uno stato non sicuro.

Questa somma non deve essere interpretata necessariamente come una somma fisica rigorosa nel senso della meccanica classica, almeno non nelle prime versioni del sistema. Nella simulazione, infatti, questi contributi possono essere vettori di movimento normalizzati o pesati. Tuttavia, la forma matematica è estremamente utile perché permette di costruire un modello modulare, in cui ogni comportamento può essere sviluppato, testato e calibrato separatamente. Se lo swarm risulta troppo disperso, si può aumentare il peso della coesione; se collassa troppo, si può aumentare la separazione; se non forma strutture ordinate, si può migliorare l'allineamento; se le connessioni magnetiche sono instabili, si può intervenire sui termini magnetici e sulle soglie.

Per rendere il modello controllabile, ogni forza può essere moltiplicata per un coefficiente di peso:

$$F_i(t) = w_c \cdot F_{\text{coesione}_i} + w_s \cdot F_{\text{separazione}_i} + w_a \cdot F_{\text{allineamento}_i} + w_m \cdot F_{\text{magnetica}_i} + w_t \cdot F_{\text{target}_i} + w_v \cdot F_{\text{vincoli}_i}$$

dove w_c , w_s , w_a , w_m , w_t e w_v rappresentano i pesi dei diversi contributi. Questi coefficienti sono parametri fondamentali della simulazione. Essi permettono di regolare il carattere dello swarm. Un valore alto di w_c rende il gruppo più compatto; un valore alto di w_s rende i nodi più distanziati; un valore alto di w_t rende il sistema più orientato al raggiungimento di pattern imposti; un valore alto di w_m rende dominante il comportamento magnetico. La scelta dei pesi determina quindi il compromesso tra comportamento emergente e controllo guidato.

La forza di coesione rappresenta la tendenza dei nodi a rimanere vicini al gruppo. Essa impedisce allo swarm di disperdersi e contribuisce alla formazione di aggregati. In un modello locale, la coesione viene calcolata rispetto ai vicini del nodo, non rispetto all'intero sistema. Se $N_i(t)$ rappresenta l'insieme dei vicini del nodo i , si può calcolare il centro locale dei vicini:

$$c_i(t) = \text{media delle posizioni dei nodi in } N_i(t)$$

La forza di coesione può allora essere orientata dal nodo verso questo centro locale:

$$F_{\text{coesione}_i}(t) = k_c \cdot (c_i(t) - p_i(t))$$

dove k_c è un coefficiente di intensità. Questa forza spinge il nodo verso la regione media occupata dai suoi vicini, favorendo il mantenimento del gruppo. Tuttavia, la coesione deve essere bilanciata: se è troppo debole, lo swarm si frammenta; se è troppo forte, i nodi tendono a collassare nello stesso punto. Per questo motivo, la coesione deve sempre essere accompagnata dalla separazione.

La forza di separazione rappresenta la tendenza opposta: impedisce ai nodi di avvicinarsi oltre una distanza minima. Essa è necessaria perché i MicroBot sono oggetti fisici dotati di volume e non possono sovrapporsi. Nel modello matematico, la separazione agisce soprattutto quando la distanza tra due nodi è inferiore a una soglia $d_{\min}$. Se il nodo j è troppo vicino al nodo i , il contributo di separazione spinge i due nodi in direzioni opposte. Una forma concettuale può essere:

$$F_{\text{separazione}_i}(t) = \sum k_s \cdot (p_i(t) - p_j(t)) / d_{ij}(t)$$

per tutti i nodi j tali che $d_{ij}(t) < d_{\min}$. Questa formula indica che la forza di separazione cresce quando la distanza diminuisce. Più due nodi sono vicini, più forte deve essere la correzione. Anche qui, nella simulazione si può usare una versione semplificata, ma il principio resta centrale: senza separazione, la coesione e l'attrazione magnetica produrrebbero aggregazioni non realistiche.

La forza di allineamento rappresenta la tendenza dei nodi ad assumere una direzione di movimento o un orientamento compatibile con i vicini. Nei sistemi swarm classici, l'allineamento serve a far muovere gli agenti in modo coordinato. Nel sistema MicroBot, questo concetto ha una doppia funzione. Da un lato può riguardare la direzione del moto, specialmente nella simulazione. Dall'altro può riguardare l'orientamento fisico del nodo, cioè la direzione delle superfici magnetiche, dei poli o delle bobine. Questo secondo aspetto è particolarmente importante perché l'aggancio non dipende soltanto dalla posizione, ma anche dalla compatibilità geometrica e magnetica.

Nel caso del movimento, l'allineamento può essere calcolato come tendenza a seguire la velocità media dei vicini:

$$v_{\text{media}_i}(t) = \text{media delle velocità dei nodi in } N_i(t)$$

$$F_{\text{allineamento}_i}(t) = k_a \cdot (v_{\text{media}_i}(t) - v_i(t))$$

Nel caso dell'orientamento, invece, l'allineamento può essere definito come correzione dell'angolo o del vettore direzione del nodo. Se $\theta_i(t)$ rappresenta l'orientamento del nodo e $\theta_{\text{desiderato}_i}(t)$ rappresenta l'orientamento medio o compatibile con i vicini, allora:

$$\Delta\theta_i(t) = k_{\theta} \cdot (\theta_{\text{desiderato}_i}(t) - \theta_i(t))$$

Questa formulazione permette di modellare l'orientamento come parte della dinamica dello swarm. Il nodo non deve soltanto raggiungere una posizione, ma anche orientarsi in modo utile per l'interazione. Nel caso MicroBot, questa distinzione è cruciale: uno swarm può essere geometricamente vicino alla forma desiderata ma magneticamente instabile se gli orientamenti sono sbagliati.

La forza magnetica rappresenta il contributo direttamente collegato al sistema magnetico. Essa può derivare da magneti permanenti, bobine attive o combinazioni ibride. In una simulazione semplificata, la forza magnetica tra due nodi può essere modellata come una forza attrattiva dipendente dalla distanza e dallo stato magnetico:

$$F_{\text{magnetica}_{ij}}(t) = k_m \cdot M_{ij}(t) \cdot g(d_{ij}(t)) \cdot n_{ij}(t)$$

dove $M_{ij}(t)$ rappresenta la compatibilità magnetica tra i nodi, $g(d_{ij})$ rappresenta il decadimento con la distanza e n_{ij} è la direzione dal nodo i al nodo j . Se i nodi sono compatibili, M_{ij} è positivo e la forza è attrattiva. Se sono incompatibili, M_{ij} può essere nullo o negativo, generando assenza di interazione o repulsione. Questa struttura permette di integrare nel modello matematico polarità, orientamento e stato attivo delle bobine.

Il contributo magnetico non deve però sovrascrivere completamente le altre forze. Se la forza magnetica è troppo dominante, lo swarm può aggregarsi in modo rigido o caotico, ignorando pattern e separazione. Se invece è troppo debole, le connessioni non si formano e il sistema resta solo una simulazione geometrica. Il peso w_m deve quindi essere calibrato con attenzione. Nel sistema reale, questa calibrazione corrisponderà al dimensionamento di magneti, coil, correnti, distanze e superfici di contatto.

La forza verso target rappresenta il contributo del controllo globale o della formazione desiderata. Se il sistema deve costruire una linea, un cerchio, una griglia o una forma complessa, ogni nodo può ricevere una posizione obiettivo τ_i . La forza verso target può essere definita come:

$$F_{\text{target}_i}(t) = k_t \cdot (\tau_i - p_i(t))$$

Questa forza guida il nodo verso la propria posizione assegnata. Tuttavia, il sistema MicroBot non deve essere ridotto a un semplice inseguimento di target, perché questo annullerebbe la natura emergente dello swarm. Il target deve orientare il comportamento, non sostituire completamente le interazioni locali. La formazione più interessante nasce dall'equilibrio tra obiettivo globale e regole locali. Se il target domina troppo, i nodi diventano punti comandati centralmente; se domina troppo l'emergenza locale, il sistema può non raggiungere forme specifiche. Il modello deve quindi bilanciare controllo e autonomia.

La forza dei vincoli comprende tutte le correzioni necessarie per mantenere il sistema in condizioni ammissibili. Essa include limiti di bordo, limiti di velocità, vincoli di connessione, vincoli di safety e vincoli geometrici. Se un nodo sta per uscire dallo spazio operativo, una forza di vincolo lo riporta dentro. Se una connessione magnetica è attiva, una forza di vincolo può mantenere la distanza tra i nodi vicina al valore desiderato. Se un nodo è in stato di sicurezza, il suo movimento può essere ridotto o annullato. I vincoli rappresentano quindi il livello che impedisce al modello di produrre comportamenti impossibili o pericolosi.

Un aspetto centrale delle forze dello swarm è il loro carattere locale. In un sistema distribuito, ogni nodo dovrebbe basare il proprio comportamento principalmente sui vicini, non sull'intero sistema. Anche se il simulatore o il controller possono calcolare informazioni globali, la logica swarm deve restare localmente interpretabile. Questo significa che coesione, separazione, allineamento e magnetismo devono essere calcolati soprattutto su $N_i(t)$, l'insieme dei vicini del nodo. La forza risultante è quindi una sintesi del vicinato locale:

$$F_i(t) = \sum \text{contributi dei nodi in } N_i(t)$$

Questo approccio rende il sistema scalabile e coerente con il comportamento emergente. Se ogni nodo dovesse calcolare interazioni con tutti gli altri nodi in modo completo, la complessità crescerebbe rapidamente. Limitare il calcolo al vicinato rende il modello più realistico e più vicino a ciò che un MicroBot fisico potrebbe effettivamente percepire o subire.

Le forze dello swarm devono inoltre essere normalizzate o limitate. Se molti contributi si sommano, la forza risultante può diventare troppo grande, causando movimenti bruschi o instabilità numerica. Per evitare questo problema, si può imporre un limite massimo:

$$||F_i(t)|| \leq F_{\text{max}}$$

oppure, nel modello cinematico, un limite alla velocità o allo spostamento:

$$||v_i(t)|| \leq v_{\text{max}}$$

$$||\Delta p_i(t)|| \leq \Delta p_{\text{max}}$$

Questi limiti sono essenziali per mantenere il sistema stabile. Dal punto di vista fisico, rappresentano l'impossibilità del nodo di reagire con forza o velocità infinita. Dal punto di vista simulativo, evitano esplosioni numeriche e rendono il comportamento più leggibile.

Un altro elemento importante è lo smorzamento. Le forze possono produrre oscillazioni se non esiste un meccanismo che riduce progressivamente il movimento. Nei sistemi fisici, lo smorzamento deriva da attrito, dissipazione e resistenza. Nel modello, può essere introdotto tramite un coefficiente che riduce la velocità:

$$v_i(t + \Delta t) = \alpha \cdot v_i(t) + F_i(t) \cdot \Delta t$$

dove α è minore di 1. Questo parametro permette di evitare oscillazioni continue intorno a target, connessioni o centri di coesione. Nel MicroBot, lo smorzamento è particolarmente utile perché molte forze possono agire contemporaneamente e generare instabilità se non bilanciate.

Le forze dello swarm devono essere coerenti con le soglie di attacco e distacco. Quando due nodi sono lontani, la forza magnetica può essere nulla o debole. Quando entrano nella soglia di influenza, può comparire un contributo attrattivo. Quando superano la soglia di attacco, la relazione può diventare connessione attiva. Quando una connessione è attiva, la forza non è più solo attrattiva ma vincolante. Quando la distanza supera la soglia di distacco, il vincolo viene rimosso. Questo significa che le forze non sono indipendenti dallo stato relazionale: cambiano in base alla storia della connessione.

In forma concettuale:

se non connessi → forza di attrazione potenziale;

se candidati → forza di avvicinamento e allineamento;

se connessi → forza di vincolo;

se in distacco → forza ridotta o repulsiva;

se disconnessi → nessun vincolo.

Questa logica rende il modello più realistico rispetto a una semplice attrazione continua. Una connessione formata cambia la natura dell'interazione tra due nodi. Il sistema non tratta più i nodi come particelle indipendenti, ma come elementi di una struttura.

Dal punto di vista della programmazione del simulatore, le forze dello swarm possono essere calcolate in una pipeline ordinata. Prima si calcolano i vicini; poi si calcolano separazione, coesione e allineamento; poi si calcolano contributi magnetici e target; poi si sommano i contributi; poi si applicano limiti e smorzamento; infine si aggiornano velocità, posizioni e connessioni. Questa pipeline è coerente con il modello discreto del moto già definito. Una sequenza concettuale può essere:

calcolo del vicinato;

calcolo delle distanze;

calcolo dei contributi locali;

calcolo del contributo target;

calcolo dei vincoli;

somma pesata;

limitazione della forza risultante;

aggiornamento del moto;

aggiornamento delle connessioni.

Questa struttura rende il sistema chiaro e documentabile. Ogni fase può essere testata separatamente. Se la separazione non funziona, si controlla il modulo di separazione; se la coesione è eccessiva, si modifica il peso; se le connessioni sono instabili, si interviene su magnetismo e hysteresis.

Le forze dello swarm possono anche essere interpretate come funzioni di energia. La coesione tende a ridurre l'energia associata alla dispersione; la separazione aumenta l'energia quando i nodi sono troppo vicini; il magnetismo crea minimi energetici locali nelle configurazioni compatibili; il target introduce una funzione di costo rispetto alla forma desiderata. In questa prospettiva, il movimento dello swarm può essere visto come una ricerca di configurazioni che minimizzano una funzione complessiva:

$$E_{\text{totale}} = E_{\text{coesione}} + E_{\text{separazione}} + E_{\text{magnetica}} + E_{\text{target}} + E_{\text{vincoli}}$$

Il sistema evolve cercando configurazioni più stabili rispetto a questa energia complessiva. Anche se non è necessario implementare subito un vero algoritmo di minimizzazione, questa interpretazione teorica è molto utile perché collega il comportamento dello swarm alla fisica e alla matematica dei sistemi complessi.

Dal punto di vista della materia programmabile, le forze dello swarm sono il linguaggio attraverso cui la forma viene costruita. Il software non sposta ogni MicroBot manualmente in modo rigido; imposta regole, pesi, soglie e campi che rendono certe configurazioni più probabili o più stabili. Le forze locali trasformano queste regole in movimento collettivo. La forma finale emerge dall'equilibrio tra attrazione, repulsione, orientamento e vincoli. Questo è il cuore del progetto MicroBot: programmare non solo posizioni, ma condizioni di interazione.

Un esempio concreto può chiarire il ruolo delle forze. Se lo swarm deve formare una linea, il termine target può assegnare posizioni lungo un asse; la separazione impedisce ai nodi di sovrapporsi; la coesione evita che si disperdano; l'allineamento orienta i nodi lungo la direzione della linea; il magnetismo stabilizza le connessioni tra vicini; l'hysteresis evita che le connessioni oscillino durante piccoli movimenti. Nessun singolo contributo basta da solo. La linea emerge dalla cooperazione di tutti questi elementi.

Se invece lo swarm deve formare un cluster, il target può essere debole o assente; la coesione può essere dominante; la separazione mantiene una distribuzione ordinata; il magnetismo crea connessioni locali; l'allineamento può essere meno importante. Questo mostra che i pesi delle forze possono cambiare a seconda del comportamento desiderato. Il sistema MicroBot può quindi possedere modalità dinamiche diverse, ciascuna definita da un set di parametri.

Una modalità "aggregation" potrebbe avere coesione e magnetismo forti. Una modalità "formation" potrebbe avere target e allineamento più forti. Una modalità "safe separation" potrebbe aumentare separazione e ridurre magnetismo. Una modalità "idle" potrebbe ridurre quasi tutte le forze, mantenendo solo vincoli e sicurezza. Questa struttura è molto utile per collegare il modello matematico agli stati operativi del sistema.

Dal punto di vista della telemetria, le forze dello swarm possono diventare grandezze osservabili o stimabili. Una dashboard potrebbe mostrare non solo le posizioni dei nodi, ma anche vettori di forza, intensità delle connessioni, contributi principali al movimento e stati di stabilità. Questo sarebbe molto utile per debug e dimostrazione. Se un nodo si muove in modo inatteso, il sistema potrebbe mostrare se è stato spinto dalla separazione, attratto dal target, vincolato da una connessione magnetica o influenzato dal centro del gruppo.

Un record di debug potrebbe includere:

id nodo;

forza di coesione;

forza di separazione;

forza di allineamento;

forza magnetica;

forza target;

forza risultante;

velocità risultante;

stato connessioni;

timestamp.

Questo tipo di logging renderebbe il modello molto trasparente. Il comportamento emergente non sarebbe una scatola nera, ma una composizione analizzabile di contributi. Nel contesto del progetto MicroBot, dove la documentazione e la tracciabilità sono centrali, questa scelta sarebbe molto coerente.

Dal punto di vista dei limiti, bisogna riconoscere che un modello a forze virtuali non rappresenta automaticamente la fisica reale. In simulazione, è facile creare una forza di coesione o un target ideale; nel prototipo fisico, queste forze devono essere prodotte da meccanismi reali o rappresentare soltanto logiche di controllo. Per questo motivo, il documento deve mantenere chiara la distinzione tra forze simulate e forze fisicamente implementate. Le forze simulate sono strumenti per progettare e visualizzare comportamenti; le forze fisiche sono quelle effettivamente generate da magneti, bobine, contatti o attuatori. La transizione tra i due livelli deve essere progressiva e verificata sperimentalmente.

Questa distinzione non indebolisce il progetto, anzi lo rende più rigoroso. Molti sistemi robotici avanzati vengono prima progettati tramite modelli virtuali e solo successivamente collegati a hardware reale. L'importante è che il modello sia costruito in modo coerente con i vincoli fisici futuri. Nel caso MicroBot, le forze virtuali devono essere pensate come astrazioni di interazioni possibili: coesione come attrazione locale, separazione come vincolo di volume, allineamento come orientamento magnetico o geometrico, magnetismo come campo reale, target come controllo globale.

Le forze dello swarm sono inoltre fondamentali per la scalabilità. Un sistema centralizzato potrebbe calcolare posizione e movimento di ogni nodo in modo dettagliato, ma questo approccio diventerebbe complesso con molti MicroBot. Le regole locali basate su forze sono più scalabili, perché ogni nodo può applicare la stessa logica rispetto ai propri vicini. La complessità globale nasce dalla ripetizione di una regola locale semplice. Questo è uno dei principi fondamentali della swarm robotics e si adatta perfettamente al progetto MicroBot.

Dal punto di vista teorico, si può dire che le forze dello swarm definiscono il campo comportamentale del sistema. Ogni punto dello spazio e ogni configurazione dei nodi genera una certa tendenza di movimento. I MicroBot seguono queste tendenze e, seguendole, modificano il campo stesso, perché cambiano le distanze, i vicini e le connessioni. Il sistema è quindi retroattivo: le forze modificano la configurazione, e la configurazione modifica le forze. Questa retroazione è alla base dell'emergenza.

Una formulazione sintetica del processo è:

configurazione corrente → calcolo delle forze → aggiornamento del moto → nuova configurazione → nuove forze

Questo ciclo si ripete nel tempo. La forma dello swarm nasce dalla ripetizione di questo ciclo, non da una singola istruzione. È una dinamica iterativa, coerente con tutto ciò che è stato definito nel modello discreto del moto.

Nel contesto delle versioni del progetto, le forze dello swarm possono evolvere nel tempo. Una prima versione può includere solo coesione e separazione. Una seconda può aggiungere target e allineamento. Una terza può introdurre magnetismo semplificato. Una quarta può aggiungere soglie di attacco, distacco e hysteresis. Una quinta può includere energia, temperatura e safety. Questa progressione consente di sviluppare il simulatore in modo ordinato, evitando di introdurre tutte le complessità contemporaneamente. Ogni forza diventa un modulo evolutivo.

In conclusione, le forze dello swarm costituiscono il livello dinamico attraverso cui il sistema MicroBot produce movimento, aggregazione, stabilizzazione e riconfigurazione. Esse combinano coesione, separazione, allineamento, magnetismo, target e vincoli in una forza risultante che guida l'evoluzione dei nodi nel tempo. Il loro valore non è soltanto matematico, ma architetturale: permettono di collegare simulazione, fisica, controllo, telemetria e comportamento emergente. Attraverso le forze dello swarm, il modello passa dalla semplice descrizione delle relazioni alla generazione effettiva di forma dinamica. I punti successivi approfondiranno separatamente i tre contributi fondamentali della logica swarm: coesione, separazione e allineamento, che rappresentano la base comportamentale su cui costruire sistemi multi-agente più complessi.

7.11 Coesione

La coesione rappresenta una delle regole fondamentali del comportamento swarm e costituisce il meccanismo attraverso cui i MicroBot tendono a mantenere una struttura collettiva invece di disperdersi nello spazio. Dopo aver introdotto le forze dello swarm come composizione di più contributi dinamici, è necessario analizzare separatamente la coesione perché essa svolge una funzione centrale nella stabilità globale del sistema. Senza coesione, ogni nodo potrebbe muoversi in modo indipendente, perdere relazione con gli altri e impedire la formazione di configurazioni collettive. Con la coesione, invece, il sistema introduce una tendenza locale all'aggregazione controllata, permettendo allo swarm di comportarsi come un insieme organizzato.

Nel progetto MicroBot, la coesione non deve essere interpretata come semplice attrazione indiscriminata tra tutti i nodi. Se tutti i MicroBot fossero attratti da tutti gli altri con la stessa intensità, il sistema rischierebbe di collassare in un unico ammasso compatto, perdendo forma, leggibilità e capacità di riconfigurazione. La coesione deve invece essere intesa come una tendenza regolata a mantenere il gruppo vicino, senza annullare la separazione, l'allineamento, i vincoli magnetici e gli obiettivi di pattern. In altre parole, la coesione non deve dominare il sistema, ma contribuire all'equilibrio dinamico dello swarm.

Dal punto di vista matematico, la coesione può essere descritta come la tendenza di un nodo a muoversi verso il centro medio dei propri vicini. Se $N_i(t)$ rappresenta l'insieme dei vicini del MicroBot i al tempo t , il centro locale del vicinato può essere definito come:

$$c_i(t) = (1 / |N_i(t)|) \cdot \sum_j p_j(t)$$

dove la somma è calcolata su tutti i nodi j appartenenti a $N_i(t)$, mentre $|N_i(t)|$ rappresenta il numero di vicini. Il vettore di coesione può quindi essere espresso come:

$$F_{\text{coesione}_i}(t) = k_c \cdot (c_i(t) - p_i(t))$$

dove k_c è un coefficiente che controlla l'intensità della coesione. Questa relazione indica che il nodo i tende a muoversi verso la posizione media dei suoi vicini. Se il nodo è lontano dal gruppo locale, il vettore di coesione sarà più marcato; se è già vicino al centro del vicinato, il contributo sarà ridotto. Questo comportamento permette allo swarm di mantenere una struttura compatta senza richiedere un controllo centrale rigido.

Un aspetto fondamentale è che la coesione deve essere locale, non globale. In un sistema distribuito, ogni nodo non dovrebbe necessariamente calcolare il centro dell'intero swarm, ma soltanto il centro dei nodi vicini. Questo è coerente con la natura fisica e computazionale del progetto MicroBot. Dal punto di vista fisico, le interazioni magnetiche e meccaniche avvengono a corto raggio; dal punto di vista computazionale, un nodo reale potrebbe non avere accesso alla posizione globale di tutti gli altri MicroBot. Per questo motivo, la coesione locale è più realistica, più scalabile e più vicina al paradigma swarm.

La coesione locale permette inoltre la formazione di strutture più flessibili. Se ogni nodo fosse attratto verso il centro globale dello swarm, il sistema tenderebbe a comprimersi in una massa unica. Se invece ogni nodo considera solo i propri vicini, possono emergere catene, cluster, gruppi locali e strutture distribuite. Questo comportamento è particolarmente importante per MicroBot, perché l'obiettivo non è creare semplicemente un ammasso di unità, ma permettere configurazioni riconfigurabili e potenzialmente geometriche. La coesione deve quindi mantenere le relazioni senza distruggere la forma.

Nel sistema MicroBot, la coesione può avere una componente fisica e una componente simulativa. La componente fisica può derivare da magneti permanenti, bobine elettromagnetiche o superfici di aggancio che mantengono vicine le unità. La componente simulativa, invece, può essere rappresentata tramite vettori di movimento nella simulazione software. Queste due componenti non sono identiche, ma devono essere coerenti. La simulazione può usare la coesione come astrazione del comportamento aggregativo che, nel sistema reale, sarà prodotto da interazioni magnetiche e vincoli meccanici.

Questa distinzione è importante per evitare ambiguità. Nella simulazione, un nodo può essere "spinto" verso il centro dei vicini semplicemente aggiornando le coordinate. Nel prototipo fisico, invece, quella tendenza deve essere prodotta da forze reali o da meccanismi di controllo. La coesione matematica è quindi un modello del comportamento desiderato; la coesione fisica è il modo in cui tale comportamento viene realizzato concretamente. Il progetto MicroBot deve mantenere questi due livelli collegati, ma non confusi.

Dal punto di vista magnetico, la coesione può essere vista come risultato di molte attrazioni locali. Ogni connessione magnetica tra due MicroBot contribuisce alla stabilità del gruppo. Se un nodo è connesso a più vicini, la somma delle interazioni tende a mantenerlo all'interno della struttura. In questo senso, la coesione non è una forza unica applicata dall'esterno, ma una proprietà emergente della rete di connessioni locali. Lo swarm resta insieme perché ogni nodo mantiene relazioni con alcuni vicini, e l'insieme di queste relazioni produce stabilità globale.

Questa interpretazione è coerente con la rappresentazione dello swarm come grafo dinamico. Se i MicroBot sono i vertici del grafo e le connessioni magnetiche o di vicinanza sono gli archi, la coesione può essere misurata anche attraverso la connettività del grafo. Uno swarm coeso non è soltanto uno swarm con nodi vicini, ma uno swarm in cui esistono abbastanza relazioni da mantenere il sistema collegato. Se il grafo si spezza in componenti separate, lo swarm perde coesione globale anche se ogni piccolo gruppo interno resta compatto.

Dal punto di vista matematico, si può distinguere tra coesione geometrica e coesione topologica. La coesione geometrica riguarda le distanze tra nodi: uno swarm è geometricamente coeso se i nodi sono relativamente vicini nello spazio. La coesione topologica riguarda invece le connessioni: uno swarm è topologicamente coeso se il grafo delle relazioni resta connesso o sufficientemente integrato. Nel progetto MicroBot sono importanti entrambe. Uno swarm può essere geometricamente compatto ma con poche connessioni stabili, oppure può essere leggermente esteso ma ben collegato tramite una struttura ordinata. La qualità della coesione dipende quindi da distanza e connessioni.

Una misura semplice della coesione geometrica è la dispersione media dei nodi rispetto al centro dello swarm. Se $c(t)$ rappresenta il centro globale:

$$c(t) = (1/n) \cdot \sum p_i(t)$$

la dispersione può essere definita come:

$$D(t) = (1/n) \cdot \sum ||p_i(t) - c(t)||$$

Un valore basso di $D(t)$ indica che i nodi sono concentrati vicino al centro; un valore alto indica che lo swarm è disperso. Tuttavia, questa metrica da sola non basta. Una dispersione troppo bassa può indicare collasso, non buona coesione. Per questo motivo, la coesione deve essere sempre valutata insieme alla separazione. Uno swarm sano mantiene una distanza media contenuta, ma evita sovrapposizioni e compressioni eccessive.

Una misura topologica può invece considerare il numero di connessioni attive o la presenza di componenti connesse nel grafo. Se il grafo dello swarm è indicato come:

$$G(t) = (V, E(t))$$

la coesione topologica aumenta quando il numero di archi utili $E(t)$ è sufficiente a mantenere il sistema connesso. Tuttavia, anche in questo caso non basta massimizzare il numero di connessioni. Troppe connessioni possono rendere la struttura rigida e difficile da riconfigurare. La coesione migliore non è quella con il massimo numero di legami, ma quella con il numero e la distribuzione di legami necessari a mantenere stabilità e flessibilità.

Nel contesto dei pattern, la coesione assume significati diversi. In un cluster, la coesione deve essere relativamente forte, perché l'obiettivo è mantenere i nodi vicini e compatti. In una linea, la coesione deve essere direzionale: i nodi devono restare collegati lungo una struttura allungata, non collassare verso il centro. In un cerchio, la coesione deve mantenere la continuità della forma senza far convergere tutti i nodi al centro. In una struttura tridimensionale, la coesione deve sostenere relazioni volumetriche. Questo significa che la coesione non è una regola unica identica per tutti i pattern, ma un principio che deve essere adattato al comportamento desiderato.

Per una formazione lineare, ad esempio, la coesione può essere applicata principalmente tra vicini consecutivi, non verso il centro globale. Per una formazione circolare, può essere combinata con una forza radiale che mantiene i nodi su un raggio desiderato. Per un cluster libero, può essere più isotropica. Questa

flessibilità è importante perché permette al sistema MicroBot di usare la stessa regola generale in modi diversi a seconda dello stato operativo.

Dal punto di vista del controllo, il coefficiente k_c o il peso w_c della coesione può cambiare nel tempo. Durante una fase di aggregazione, la coesione può essere aumentata per avvicinare i nodi. Durante una fase di formazione precisa, può essere ridotta per permettere ai nodi di raggiungere target specifici. Durante una fase di separazione o riconfigurazione, può essere abbassata ulteriormente per consentire distacchi controllati. In questo modo, la coesione diventa un parametro dinamico del sistema, non un valore fisso.

Una possibile sequenza operativa può essere:

fase di raccolta: coesione alta;

fase di ordinamento: coesione media, separazione e target bilanciati;

fase di aggancio: coesione locale e magnetismo attivi;

fase di stabilizzazione: coesione moderata e hysteresis attiva;

fase di riconfigurazione: coesione selettiva, distacchi controllati.

Questa logica permette allo swarm di attraversare fasi diverse senza perdere controllo. La coesione viene usata come strumento, non come vincolo permanente.

Un aspetto importante riguarda la relazione tra coesione e magnetismo. I magneti permanenti possono produrre una coesione passiva, perché tendono a mantenere vicini i nodi senza consumo energetico. Le bobine possono produrre una coesione attiva, perché permettono di aumentare temporaneamente l'attrazione tra unità. Tuttavia, una coesione magnetica troppo forte può ridurre la riconfigurabilità. Per questo motivo, una strategia ibrida è più efficace: usare magneti permanenti per una coesione minima stabile e bobine per rinforzi temporanei quando il sistema deve aggregarsi o correggere una struttura.

Questa strategia si collega direttamente al concetto di energia. La coesione attiva ha un costo energetico; la coesione passiva ha un costo strutturale. I magneti permanenti non consumano corrente, ma mantengono sempre una forza che può generare agganci indesiderati. Le bobine consumano energia, ma possono essere controllate. Il sistema deve quindi decidere quando conviene usare coesione passiva, quando coesione attiva e quando ridurre la coesione per permettere una transizione.

Dal punto di vista della sicurezza, la coesione deve essere limitata. Se il sistema tenta di mantenere un gruppo troppo compatto, può generare compressioni, urti o surriscaldamento se vengono usate bobine. Se il nodo entra in stato di safety, la coesione attiva dovrebbe essere ridotta o disabilitata. Tuttavia, la coesione passiva data dai magneti può restare presente. Questo significa che il safety layer deve distinguere tra coesione controllabile e coesione fisicamente permanente. Tale distinzione è importante nella progettazione reale del sistema.

La coesione può essere anche selettiva. In uno swarm complesso, non tutti i nodi devono essere coesi con tutti. Alcuni gruppi possono formare sottostrutture, moduli o cluster temporanei. Ogni cluster può avere una propria coesione interna e una relazione più debole con altri cluster. Questo permette di costruire forme più complesse attraverso aggregati intermedi. Nel modello matematico, si può introdurre una coesione intra-gruppo più forte e una coesione inter-gruppo più debole. Questa logica sarà utile nelle sezioni dedicate ai pattern e alla scalabilità.

Dal punto di vista della rappresentazione grafica e della dashboard, la coesione può essere visualizzata attraverso metriche e segnali visivi. Si possono mostrare il centro dello swarm, il raggio medio, la dispersione, le connessioni attive, i cluster identificati e le aree di densità. Una dashboard efficace potrebbe indicare se lo swarm è disperso, compatto, frammentato o sovracompresso. Questo aiuterebbe a interpretare il comportamento della simulazione e a calibrare i parametri.

Nel sistema di logging, la coesione può essere registrata come metrica temporale. A ogni snapshot, il sistema può salvare valori come dispersione media, distanza media tra vicini, numero di componenti

connesse, numero di connessioni attive e densità locale. Questi dati permettono di analizzare se lo swarm sta diventando più coeso nel tempo, se si sta frammentando o se sta collassando. La coesione diventa quindi non solo una forza, ma anche una misura di qualità del comportamento.

Una possibile metrica complessiva di coesione può combinare distanza media e connettività:

$C(t) = \text{funzione}(\text{distanza_media_locale}, \text{numero_connessioni}, \text{componenti_connesse})$

Il valore $C(t)$ potrebbe aumentare quando lo swarm è ben collegato e le distanze sono entro un intervallo desiderato. Tuttavia, una buona metrica dovrebbe penalizzare sia la dispersione eccessiva sia la compressione eccessiva. Questo conferma che coesione e separazione devono essere valutate insieme.

Un errore comune nei modelli swarm è pensare che più coesione sia sempre meglio. Nel MicroBot, questo sarebbe sbagliato. Troppa coesione riduce la libertà di movimento, ostacola la riconfigurazione, favorisce cluster casuali e può nascondere errori di pattern. Poca coesione, invece, produce dispersione e perdita di struttura. Il valore corretto si trova in una fascia operativa. Questa fascia dipende dal pattern, dal numero di nodi, dalla scala, dalla forza magnetica e dal livello di controllo desiderato.

Dal punto di vista teorico, la coesione rappresenta la tendenza dello swarm a conservare identità collettiva. Uno swarm non è tale se ogni nodo si comporta come elemento totalmente indipendente. La coesione crea il senso di gruppo, cioè mantiene le unità all'interno di uno spazio relazionale comune. Tuttavia, nel progetto MicroBot questa identità collettiva deve restare flessibile: il gruppo deve poter cambiare forma, dividersi, ricomporsi e adattarsi. La coesione deve quindi sostenere l'identità dello swarm senza trasformarla in rigidità.

Questa idea è molto vicina al concetto di materia programmabile. In una materia programmabile, le unità devono restare abbastanza legate da formare una struttura, ma non così legate da impedire trasformazioni. La coesione è la proprietà che impedisce alla materia di disperdersi; la separazione è la proprietà che impedisce alla materia di collassare; l'allineamento è la proprietà che rende la struttura ordinata. Queste tre regole, combinate, permettono alla forma di emergere.

Nel modello matematico, la coesione può essere vista anche come minimizzazione della distanza locale. Il sistema tende a ridurre la distanza tra un nodo e il centro dei suoi vicini, ma non fino a zero. La presenza della separazione introduce infatti un minimo desiderato non nullo. La configurazione stabile non è quella in cui tutti i nodi hanno stessa posizione, ma quella in cui i nodi sono distribuiti a distanze compatibili con il corpo fisico e il pattern. Questo equilibrio è la base delle formazioni stabili.

Dal punto di vista della simulazione, il comportamento della coesione deve essere testato gradualmente. Una prima prova può usare pochi nodi senza target, osservando se il gruppo resta compatto. Una seconda prova può aggiungere separazione, verificando se i nodi formano un cluster ordinato. Una terza prova può aggiungere magnetismo, osservando se si formano connessioni stabili. Una quarta prova può aggiungere target, verificando se lo swarm riesce a mantenere coesione mentre raggiunge una forma. Questa progressione permette di calibrare il modello senza confondere troppe variabili contemporaneamente.

Un test utile consiste nel partire con nodi distribuiti casualmente e misurare la dispersione nel tempo. Se la coesione funziona, la dispersione dovrebbe diminuire fino a stabilizzarsi. Se diminuisce troppo, significa che la separazione è insufficiente. Se non diminuisce, la coesione è troppo debole. Se oscilla molto, servono smorzamento o limiti di velocità. Questo tipo di test può essere registrato e confrontato tra versioni diverse del simulatore.

Nel contesto dei file e dell'evoluzione del progetto, la coesione potrebbe essere rappresentata come parametro di configurazione. Un file di configurazione della simulazione potrebbe contenere valori come `cohesion_weight`, `cohesion_radius`, `cohesion_max_force` e `cohesion_mode`. Questo renderebbe il comportamento modificabile senza cambiare il codice principale. In un ecosistema MicroBot ben strutturato, i parametri comportamentali dovrebbero essere separati dall'implementazione, così da facilitare test, versionamento e confronto tra scenari.

Un esempio concettuale di parametri può essere:

`cohesion_radius` = raggio entro cui considerare i vicini;

cohesion_weight = peso della forza di coesione;

cohesion_max_force = limite massimo del contributo coesivo;

cohesion_mode = globale, locale, cluster, pattern-based.

Questa struttura permetterebbe di sperimentare diverse forme di coesione mantenendo lo stesso modello generale. Una coesione globale può essere utile in demo semplici; una coesione locale è più realistica; una coesione cluster-based è più adatta a strutture modulari; una coesione pattern-based è utile quando la forma desiderata definisce quali nodi devono restare vicini.

Dal punto di vista del futuro hardware, la coesione dovrà essere collegata a capacità fisiche effettive. Se i MicroBot non hanno movimento autonomo, la coesione potrà essere osservata soprattutto nella simulazione o negli agganci manuali/magnetici. Se in futuro avranno attuazione o meccanismi di spostamento, la coesione potrà diventare una vera regola di controllo. In entrambi i casi, documentarla ora è corretto, perché definisce il comportamento obiettivo del sistema. Il modello matematico anticipa la fisica, ma deve restare compatibile con essa.

In conclusione, la coesione è il principio che permette allo swarm MicroBot di mantenere unità collettiva, evitando dispersione e favorendo aggregazione controllata. Essa può essere modellata come tendenza di ogni nodo verso il centro locale dei propri vicini, ma deve essere bilanciata da separazione, allineamento, magnetismo, target e vincoli di sicurezza. La coesione può essere geometrica, topologica, passiva, attiva, locale, globale o selettiva a seconda del livello del sistema. Il suo valore non consiste nel comprimere i nodi, ma nel mantenere relazioni sufficienti a sostenere una forma riconfigurabile. Questa sezione prepara direttamente il punto successivo, dedicato alla separazione, che rappresenta il complemento necessario della coesione: il meccanismo che impedisce allo swarm di collassare e garantisce spazio, ordine e stabilità fisica tra le unità.

7.12 Separazione

La separazione rappresenta il complemento necessario della coesione e costituisce il meccanismo attraverso cui il sistema MicroBot impedisce ai nodi di collassare, sovrapporsi o concentrarsi in configurazioni fisicamente impossibili. Dopo aver definito la coesione come tendenza del sistema a mantenere vicinanza e identità collettiva, è necessario introdurre una regola opposta ma altrettanto fondamentale: ogni MicroBot deve conservare uno spazio minimo rispetto agli altri. Senza separazione, la coesione, l'attrazione magnetica e il movimento verso target potrebbero spingere più unità nello stesso punto, producendo un comportamento non realistico nella simulazione e non realizzabile nel prototipo fisico.

Nel modello MicroBot, la separazione non deve essere interpretata come desiderio di dispersione dello swarm, ma come protezione della struttura locale. Il suo scopo non è allontanare indefinitamente i nodi, ma impedire che la distanza tra due unità scenda sotto una soglia minima compatibile con il loro volume fisico, con la geometria esterna, con le superfici di contatto e con i vincoli magnetici. In altre parole, la separazione non distrugge la coesione: la rende ordinata. Uno swarm senza coesione si disperde; uno swarm senza separazione collassa. Uno swarm correttamente progettato vive nell'equilibrio tra queste due tendenze.

Dal punto di vista geometrico, ogni MicroBot occupa uno spazio. Anche se nella simulazione può essere rappresentato come punto, nella realtà possiede un volume, una massa, una superficie esterna e componenti interni. Se il nodo viene approssimato come un disco nel piano o come una sfera nello spazio, è possibile definire un raggio fisico R . Due MicroBot identici non dovrebbero avere distanza centro-centro inferiore a:

$$d_{\min} = 2R$$

Se i nodi hanno dimensioni diverse, la condizione diventa:

$$d_{\min} = R_i + R_j$$

Questa soglia rappresenta la distanza minima necessaria affinché due unità non si sovrappongano. Nel modello matematico, quando la distanza euclidea $d_{ij}(t)$ scende sotto $d_{\min}$, il sistema deve generare una correzione di separazione. Questa correzione può essere interpretata come forza repulsiva virtuale nella simulazione oppure come vincolo geometrico che impedisce configurazioni non fisiche.

La regola di separazione può essere espressa in forma concettuale come:

se $d_{ij}(t) < d_{\min}$, allora applicare $F_{\text{separazione}}$

dove $F_{\text{separazione}}$ agisce lungo la direzione che allontana i due nodi. Se $r_{ij}(t) = p_j(t) - p_i(t)$ è il vettore relativo tra il nodo i e il nodo j , il vettore unitario nella direzione da i verso j è:

$$n_{ij}(t) = r_{ij}(t) / d_{ij}(t)$$

La separazione applicata al nodo i agisce allora nella direzione opposta:

$$F_{\text{sep}_{ij}}(t) \propto -n_{ij}(t)$$

Questa formulazione indica che, se il nodo j è troppo vicino al nodo i , il nodo i viene spinto lontano da j . Per simmetria, anche il nodo j può ricevere una correzione opposta. In questo modo, la separazione distribuisce la correzione tra le unità coinvolte, evitando che un solo nodo assorba tutta la modifica.

Una forma più precisa del contributo di separazione può dipendere dall'entità della violazione della distanza minima. Se i nodi sono appena sotto la soglia, la correzione può essere debole; se sono quasi sovrapposti, la correzione deve essere più forte. Una funzione concettuale può essere:

$$F_{\text{sep}_{ij}}(t) = k_s \cdot (d_{\min} - d_{ij}(t)) \cdot (-n_{ij}(t))$$

dove k_s rappresenta il coefficiente di separazione. Il termine $(d_{\min} - d_{ij})$ misura quanto i nodi sono penetrati nella zona proibita. Se d_{ij} è uguale a $d_{\min}$, il contributo è nullo. Se d_{ij} diminuisce, la forza cresce. Questo comportamento è coerente con l'idea di repulsione locale: la separazione agisce solo quando serve e diventa più forte quando il sistema viola maggiormente il vincolo spaziale.

Nel caso di più vicini troppo vicini, il contributo totale di separazione per il nodo i è la somma dei contributi prodotti da ciascun vicino:

$$F_{\text{separazione}_i}(t) = \sum F_{\text{sep}_{ij}}(t)$$

per tutti i nodi j tali che $d_{ij}(t) < d_{\min}$. Questa somma produce un vettore risultante che spinge il nodo verso una zona meno affollata. Se un nodo è circondato da più unità, la direzione finale dipenderà dalla distribuzione dei vicini. In questo modo, la separazione non impone una direzione arbitraria, ma deriva dalla geometria locale dello swarm.

Nel sistema MicroBot, la separazione è strettamente collegata alla coesione. La coesione tende ad avvicinare un nodo al centro dei vicini, mentre la separazione impedisce che tale avvicinamento diventi eccessivo. Le due regole devono essere calibrate insieme. Se la coesione è forte e la separazione debole, lo swarm collassa. Se la separazione è forte e la coesione debole, lo swarm si disperde o diventa troppo rarefatto. L'equilibrio tra coesione e separazione produce invece distribuzioni compatte ma ordinate, in cui i nodi restano vicini senza sovrapporsi.

Questo equilibrio può essere interpretato come una distanza desiderata tra nodi. In molti casi, il sistema non deve mantenere semplicemente una distanza maggiore di $d_{\min}$, ma una distanza vicina a un valore ottimale d_{desired} . Tale distanza può dipendere dalla dimensione fisica del MicroBot, dalla lunghezza del vincolo magnetico, dalla geometria della superficie di contatto o dal pattern desiderato. La separazione impedisce distanze troppo piccole; la coesione impedisce distanze troppo grandi. Insieme, esse stabilizzano i nodi intorno a una fascia operativa.

Si può quindi immaginare una zona ideale:

$$d_{\min} < d_{ij} < d_{\max_locale}$$

All'interno di questa zona, i nodi sono abbastanza vicini da appartenere allo stesso vicinato, ma non così vicini da entrare in collisione. La qualità dello swarm dipende dalla capacità del sistema di mantenere molte

coppie di nodi in questa fascia. Questa idea sarà utile anche per valutare metriche di stabilità e densità locale.

Dal punto di vista fisico, la separazione rappresenta il volume escluso dei MicroBot. Ogni nodo occupa una regione dello spazio che gli altri nodi non possono occupare. In una simulazione semplificata, questo volume escluso viene rappresentato tramite una forza repulsiva. Nel prototipo fisico, invece, il volume escluso è garantito dalla materia stessa: due oggetti solidi non possono compenetrarsi. Tuttavia, anche nel mondo reale, la separazione resta importante perché forze magnetiche troppo elevate possono causare urti, compressioni o agganci bruschi. La regola matematica anticipa quindi un vincolo fisico reale.

Nel sistema magnetico, la separazione ha un ruolo particolarmente delicato. Magneti permanenti e bobine possono generare attrazione tra unità, ma questa attrazione deve fermarsi a una configurazione di contatto stabile. Se il modello magnetico non include un vincolo di separazione o una distanza di equilibrio, i nodi possono continuare ad avvicinarsi indefinitamente nella simulazione. Per questo motivo, ogni forza magnetica attrattiva deve essere accompagnata da un limite geometrico. Nel modello reale, tale limite è rappresentato dalle superfici del MicroBot; nel modello matematico, deve essere espresso tramite separazione, vincolo o forza elastica di contatto.

Una connessione magnetica stabile può essere vista come equilibrio tra attrazione e separazione. L'attrazione porta i nodi verso il contatto; la separazione impedisce la sovrapposizione; la geometria definisce la posizione finale. In un modello avanzato, il legame tra due nodi connessi può essere rappresentato come una distanza di equilibrio d_{attach} . Se la distanza è maggiore di d_{attach} , la connessione tende ad avvicinare i nodi; se è minore, la separazione li spinge indietro. Questo comportamento può essere modellato come una molla virtuale con lunghezza di riposo:

$$F_{connessione} = -k \cdot (d_{ij} - d_{attach})$$

Questa formulazione permette di rappresentare connessioni magnetiche non come collasso totale, ma come vincoli elastici intorno a una distanza fisicamente significativa.

La separazione è importante anche per la leggibilità visiva della simulazione. Se i nodi si sovrappongono graficamente, diventa difficile osservare lo stato dello swarm, distinguere le unità, interpretare le connessioni e analizzare il comportamento emergente. Una buona regola di separazione mantiene i nodi visivamente distinguibili, migliorando dashboard, animazioni e debugging. Questo aspetto non è secondario: in un progetto come MicroBot, dove la simulazione e la comunicazione visiva hanno un ruolo forte, la chiarezza grafica supporta la comprensione tecnica.

Dal punto di vista del comportamento collettivo, la separazione contribuisce alla distribuzione uniforme dello swarm. Quando i nodi si avvicinano troppo, la forza di separazione li redistribuisce localmente. Questo può produrre pattern più ordinati, evitare accumuli irregolari e migliorare la stabilità delle formazioni. In una formazione circolare, per esempio, la separazione aiuta a distribuire i nodi lungo la circonferenza invece di farli collassare in punti sovrapposti. In una linea, mantiene distanza tra nodi consecutivi. In un cluster, impedisce che il centro diventi troppo denso.

La separazione può essere locale o strutturale. La separazione locale agisce tra nodi troppo vicini, indipendentemente dal pattern. La separazione strutturale, invece, può dipendere dalla forma desiderata. Per esempio, in una griglia, ogni nodo dovrebbe mantenere una distanza specifica dai vicini; in una catena, il nodo deve restare vicino ai vicini collegati ma separato da quelli non collegati; in una struttura tridimensionale, la distanza desiderata può cambiare in base al livello o alla posizione nella forma. Questo significa che la separazione può evolvere da semplice vincolo anti-collisione a regola geometrica del pattern.

Nel sistema MicroBot, una prima versione può implementare separazione uniforme: tutti i nodi hanno stesso raggio e stessa distanza minima. Una versione successiva può introdurre separazione dipendente dal ruolo o dal tipo di connessione. Ad esempio, nodi connessi possono avere una distanza di equilibrio diversa rispetto a nodi non connessi. Nodi appartenenti allo stesso cluster possono tollerare maggiore vicinanza; nodi di cluster diversi possono mantenere più distanza. Questa flessibilità rende il modello più adatto a strutture complesse.

Dal punto di vista del controllo, il peso della separazione può essere modificato in base allo stato operativo. Durante una fase di aggregazione, la separazione deve restare attiva per evitare collasso, ma non deve impedire ai nodi di avvicinarsi. Durante una fase di espansione o riconfigurazione, può essere aumentata per

favorire distacco e distribuzione. Durante una fase di stabilizzazione, può essere regolata per mantenere distanza ottimale. Durante una modalità safety, può diventare dominante per evitare contatti indesiderati o compressioni.

Una possibile logica operativa è:

fase di aggregazione: separazione moderata, coesione alta;

fase di formazione: separazione bilanciata con target e allineamento;

fase di stabilizzazione: separazione locale precisa;

fase di distacco: separazione aumentata tra nodi da separare;

fase di safety: separazione prioritaria rispetto ad attrazione e target.

Questa struttura rende la separazione non un parametro fisso, ma una funzione del comportamento desiderato. Il sistema può diventare più compatto o più espanso modificando i pesi delle forze.

Dal punto di vista energetico, la separazione può ridurre il bisogno di interventi attivi. Se il modello o la geometria mantengono distanze corrette, le bobine non devono continuamente correggere collisioni o agganci indesiderati. In altre parole, una buona separazione geometrica rende il sistema più efficiente. Al contrario, se i nodi collasano spesso, il controllo attivo dovrà spendere energia per correggere situazioni che potevano essere prevenute dal modello. La separazione è quindi anche una forma di prevenzione energetica.

Nel prototipo fisico, la separazione può essere supportata dalla geometria delle superfici. Coni troncati, curvature, spessori, distanziatori o forme di aggancio possono impedire che i componenti magnetici entrino in contatto in modo troppo aggressivo o non desiderato. Questo significa che la separazione non è solo regola software, ma può essere incorporata nella forma del MicroBot. Il design meccanico può implementare fisicamente una distanza minima, riducendo il carico sul controllo software. Anche qui ritorna il principio della computazione fisica: una parte della logica può essere delegata alla materia.

Dal punto di vista della simulazione, la separazione deve essere implementata con attenzione per evitare instabilità numeriche. Se la forza di separazione cresce troppo rapidamente, i nodi possono essere spinti via con salti improvvisi. Se è troppo debole, possono compenetrarsi. Una soluzione pratica consiste nel limitare il contributo massimo della separazione:

$$||F_{\text{separazione}}|| \leq F_{\text{sep_max}}$$

oppure limitare lo spostamento risultante. Inoltre, può essere utile applicare smorzamento alla velocità, in modo che i nodi non rimbalzino indefinitamente dopo una correzione. La separazione deve correggere, non far esplodere il sistema.

Un problema particolare si verifica quando due nodi occupano esattamente la stessa posizione nella simulazione. In questo caso, la distanza d_{ij} è zero e il vettore direzione non può essere normalizzato. Per evitare errori, il modello deve includere un piccolo valore ϵ oppure una direzione casuale controllata di separazione. Questo dettaglio è tecnico, ma importante: senza gestione del caso $d = 0$, il simulatore può generare valori non validi. Un modello rigoroso deve prevedere anche queste condizioni limite.

Dal punto di vista del logging, la separazione può generare eventi diagnostici. Se due nodi entrano sotto la distanza minima, il sistema può registrare un `proximity_violation` o `collision_warning`. Se la separazione corregge con successo la posizione, può registrare `collision_avoided`. Se invece la sovrapposizione persiste per più cicli, può generare un errore di simulazione o di comportamento. Questi eventi sono utili per capire se le regole di coesione, attrazione e target stanno producendo configurazioni troppo dense.

Un evento di separazione potrebbe includere:

`node_a`;

node_b;

distance;

minimum_distance;

correction_vector;

severity;

timestamp.

La severity può indicare quanto grave è la violazione: lieve, media o critica. Questo tipo di telemetria sarebbe utile per analizzare la qualità delle formazioni e individuare parametri sbagliati. Se molte collisioni avvengono durante una certa modalità, significa che i pesi delle forze devono essere ricalibrati.

Una metrica utile è il numero di violazioni della distanza minima nel tempo:

collision_risk_count(t)

Oppure la distanza minima corrente tra tutte le coppie:

$d_{\min_current}(t) = \min d_{ij}(t)$

Se $d_{\min_current}$ scende spesso sotto la soglia fisica, il sistema non sta rispettando il volume dei nodi. Questa metrica può essere mostrata nella dashboard come indicatore di sicurezza geometrica. Uno swarm ben bilanciato dovrebbe mantenere $d_{\min_current}$ sopra la soglia minima, salvo transizioni molto brevi e corrette immediatamente.

Un'altra metrica utile è la densità locale. Per ogni nodo, si può contare quanti vicini si trovano entro un certo raggio. Se la densità locale è troppo alta, la separazione può essere aumentata o il sistema può espandere la formazione. La densità locale permette di distinguere tra swarm compatto ma ordinato e swarm sovraffollato. In una dashboard, zone ad alta densità potrebbero essere evidenziate come aree di rischio o compressione.

Dal punto di vista della teoria dei grafi, la separazione influisce sulla topologia dello swarm perché limita quante connessioni possono formarsi in una certa area. Se i nodi mantengono distanza sufficiente, le connessioni sono più leggibili e distribuite. Se i nodi collassano, il grafo può diventare troppo denso, con molte connessioni potenziali confuse. La separazione quindi non protegge solo lo spazio fisico, ma anche la chiarezza relazionale del sistema.

La separazione può anche essere selettiva rispetto alle connessioni. Due nodi già agganciati possono mantenere una distanza di equilibrio più piccola rispetto a due nodi non collegati. Questo perché la connessione definisce una relazione fisica specifica e controllata. Al contrario, due nodi non destinati ad agganciarsi potrebbero dover mantenere una distanza maggiore per evitare connessioni indesiderate. Questo porta a una separazione dipendente dal grafo:

$d_{\min}(i, j) = \text{funzione}(\text{stato_connessione}, \text{ruolo}, \text{pattern})$

Se $A_{ij} = 1$, la distanza minima può essere d_{attach} . Se $A_{ij} = 0$, la distanza minima può essere maggiore per evitare contatti non desiderati. Questa distinzione sarà particolarmente utile nelle formazioni complesse.

Nel contesto dei pattern, la separazione può essere usata per migliorare la distribuzione dei nodi su una forma. In una formazione circolare, ogni nodo deve evitare di sovrapporsi ai vicini laterali. In una griglia, la separazione mantiene celle distinte. In una struttura ramificata, impedisce che rami diversi collassino uno sull'altro. In questo senso, la separazione contribuisce non solo alla sicurezza, ma anche alla qualità estetica e geometrica della forma. La forma finale diventa più ordinata perché i nodi rispettano distanze locali.

Dal punto di vista del sistema fisico, la separazione deve tenere conto anche delle tolleranze costruttive. Due MicroBot reali non saranno perfettamente identici: piccole differenze nelle dimensioni, nei magneti, nelle superfici o negli assemblaggi possono modificare la distanza minima effettiva. Per questo motivo, $d_{\min}$ dovrebbe includere un margine di sicurezza. Invece di usare esattamente $2R$, può essere opportuno definire:

$$d_{\min} = 2R + \text{margine_tolleranza}$$

Questo margine permette di evitare contatti troppo stretti o simulazioni troppo ottimistiche. La tolleranza è un elemento importante in qualunque progetto fisico, e MicroBot non fa eccezione.

Dal punto di vista del futuro hardware, la separazione potrebbe essere supportata da sensori di contatto o prossimità. Un nodo potrebbe rilevare quando un altro è troppo vicino e modificare il proprio stato o segnalare l'evento al controller. Nelle prime versioni, questa informazione sarà probabilmente simulata; in versioni avanzate, potrebbe essere fisica. Anche qui, il modello matematico prepara la struttura per funzioni future senza richiedere che esse siano già presenti nei prototipi iniziali.

La separazione è anche importante nella sicurezza umana e dimostrativa. Se i MicroBot usano magneti abbastanza forti, agganci bruschi o collisioni possono danneggiare componenti o rendere difficile la manipolazione. Un sistema ben progettato deve evitare movimenti improvvisi e contatti non controllati. Nella simulazione, questo significa limitare velocità e forze. Nel prototipo, significa dimensionare magneti, bobine e geometrie in modo prudente. La separazione è quindi anche una regola di sicurezza del comportamento.

Dal punto di vista teorico avanzato, la separazione rappresenta il principio che preserva l'individualità dei nodi all'interno della collettività. La coesione costruisce il gruppo; la separazione mantiene le unità distinguibili. Senza separazione, la materia programmabile perderebbe modularità, perché le unità collasserebbero in una massa indistinta. Con separazione, ogni MicroBot resta elemento autonomo, pur partecipando alla forma collettiva. Questo equilibrio tra unità e gruppo è uno dei concetti più importanti del progetto.

In una materia programmabile, infatti, la forma non deve cancellare gli elementi che la compongono. La struttura globale deve emergere dalle unità, ma le unità devono restare riconfigurabili. La separazione garantisce proprio questa proprietà: mantiene spazio sufficiente affinché ogni nodo possa cambiare ruolo, orientarsi, agganciarsi, distaccarsi o partecipare a nuove configurazioni. È una regola di libertà locale all'interno dell'ordine globale.

Dal punto di vista matematico, la separazione può essere vista come una barriera energetica a corto raggio. Quando due nodi sono a distanza accettabile, la barriera è nulla o bassa. Quando si avvicinano troppo, l'energia del sistema aumenta rapidamente, spingendoli a separarsi. Questa interpretazione è coerente con i modelli fisici di particelle, in cui forze attrattive a medio raggio sono spesso bilanciate da repulsioni a corto raggio. Nel MicroBot, questa idea permette di costruire strutture stabili senza collasso.

Una possibile funzione energetica di separazione può essere alta quando $d_{ij} < d_{\min}$ e nulla quando $d_{ij} \geq d_{\min}$. La derivata di questa energia rispetto alla posizione produce una forza repulsiva. Anche se non è necessario implementare subito questo formalismo, esso offre una lettura più profonda del comportamento: la separazione definisce un costo per configurazioni troppo dense, e il sistema tende a evitarle.

In conclusione, la separazione costituisce una regola fondamentale del modello matematico MicroBot. Essa impedisce sovrapposizioni, collisioni, collassi e densità eccessive, mantenendo i nodi entro distanze fisicamente e funzionalmente compatibili. La separazione bilancia la coesione, stabilizza le formazioni, protegge la simulazione da configurazioni non realistiche, migliora la leggibilità grafica, supporta la sicurezza e preserva l'individualità dei MicroBot all'interno dello swarm. Il suo ruolo non è opporsi alla formazione collettiva, ma renderla ordinata, stabile e riconfigurabile. Il punto successivo, dedicato all'allineamento, completerà la triade comportamentale fondamentale dello swarm, introducendo il principio attraverso cui le unità non solo restano vicine e separate correttamente, ma assumono anche direzioni e orientamenti coerenti con la struttura collettiva.

7.13 Allineamento

L'allineamento rappresenta il terzo elemento fondamentale della logica swarm, insieme a coesione e separazione, e definisce il meccanismo attraverso cui i MicroBot tendono ad assumere direzioni, orientamenti o configurazioni compatibili con i nodi vicini. Dopo aver analizzato la coesione come forza che mantiene il gruppo unito e la separazione come vincolo che impedisce il collasso fisico, l'allineamento

introduce una proprietà ulteriore: l'ordine direzionale. Uno swarm non deve soltanto restare compatto e non sovrapporsi; deve anche poter organizzare le proprie unità secondo direzioni coerenti, assi comuni, orientamenti magnetici compatibili e strutture geometriche leggibili.

Nel sistema MicroBot, l'allineamento assume un significato particolarmente importante perché il progetto non riguarda semplicemente il movimento di particelle nello spazio, ma la formazione di strutture riconfigurabili attraverso unità fisiche dotate di geometria, magneti, bobine, superfici di contatto e stati interni. In un modello generico di swarm robotics, l'allineamento può indicare la tendenza degli agenti a muoversi nella stessa direzione. Nel caso MicroBot, invece, il concetto deve essere esteso: i nodi devono eventualmente allineare la propria direzione di movimento, ma anche il proprio orientamento fisico, il proprio asse magnetico, le proprie superfici di aggancio e la propria posizione rispetto alla forma collettiva desiderata. L'allineamento diventa quindi una regola sia dinamica sia strutturale.

Dal punto di vista matematico, l'allineamento può essere introdotto partendo dalla velocità. Se ogni MicroBot possiede una velocità $v_i(t)$, il nodo può confrontare la propria direzione di movimento con quella dei vicini appartenenti all'insieme $N_i(t)$. La velocità media locale può essere definita come:

$$v_media_i(t) = (1 / |N_i(t)|) \cdot \sum v_j(t)$$

dove la somma viene calcolata su tutti i nodi j appartenenti al vicinato del nodo i . Il contributo di allineamento può allora essere espresso come:

$$F_allineamento_i(t) = k_a \cdot (v_media_i(t) - v_i(t))$$

dove k_a è un coefficiente che controlla l'intensità dell'allineamento. Questa relazione indica che il MicroBot tende a ridurre la differenza tra la propria velocità e la velocità media dei vicini. Se il nodo si muove in una direzione molto diversa rispetto al gruppo locale, il contributo di allineamento lo corregge; se si muove già in modo coerente, il contributo è ridotto. Questo principio permette allo swarm di produrre movimenti collettivi più ordinati, evitando traiettorie completamente casuali o divergenti.

Tuttavia, per MicroBot l'allineamento della velocità non basta. Il nodo possiede una geometria fisica e un possibile asse magnetico. Per questo motivo, è necessario introdurre anche l'allineamento dell'orientamento. Nel caso bidimensionale, l'orientamento del nodo può essere rappresentato da un angolo $\theta_i(t)$. Se i vicini hanno orientamenti $\theta_j(t)$, si può definire un orientamento medio locale $\theta_media_i(t)$, e il nodo può correggere il proprio angolo verso tale valore. In forma concettuale:

$$\Delta\theta_i(t) = k_ \theta \cdot differenza_angolare(\theta_media_i(t), \theta_i(t))$$

Questa equazione esprime il fatto che l'orientamento del nodo viene aggiornato progressivamente, non istantaneamente. Il coefficiente $k_ \theta$ stabilisce quanto rapidamente il MicroBot tenta di allinearsi. Se $k_ \theta$ è troppo alto, il sistema può oscillare o produrre rotazioni brusche; se è troppo basso, l'allineamento diventa lento e poco efficace. Anche qui, il modello deve bilanciare reattività e stabilità.

Nel caso tridimensionale, l'orientamento può essere rappresentato tramite un vettore direzione $d_i(t)$, una matrice di rotazione o un quaternione. Per una formalizzazione iniziale, è sufficiente usare un vettore direzionale associato all'asse principale del MicroBot. L'allineamento può allora essere misurato tramite il prodotto scalare tra vettori direzione. Se d_i e d_j sono vettori unitari, il prodotto:

$$d_i \cdot d_j$$

indica quanto i due assi sono allineati. Un valore vicino a 1 indica direzioni simili; un valore vicino a -1 indica direzioni opposte; un valore vicino a 0 indica direzioni perpendicolari. A seconda del tipo di aggancio desiderato, il sistema può cercare allineamento parallelo, antiparallelo o angolare. Questo è molto importante nel caso magnetico, perché poli opposti possono richiedere orientamenti complementari, non necessariamente identici.

Nel sistema magnetico MicroBot, l'allineamento è direttamente collegato alla qualità dell'aggancio. Due nodi possono essere vicini e possedere forza magnetica sufficiente, ma se le superfici di contatto o i poli non sono orientati correttamente, la connessione può risultare instabile o impossibile. Per questo motivo, il

modello matematico deve distinguere tra vicinanza geometrica e compatibilità di allineamento. La condizione di aggancio completa non può essere solo:

$$d_{ij} < d_{\text{attack}}$$

ma deve includere anche:

$$\text{errore_angolare}_{ij} < \theta_{\text{attack}}$$

In questo modo, il sistema aggancia soltanto unità sufficientemente vicine e sufficientemente orientate. L'allineamento diventa quindi condizione necessaria per trasformare una vicinanza in una connessione stabile.

L'allineamento può essere passivo o attivo. L'allineamento passivo deriva dalla geometria e dai magneti permanenti. Se due MicroBot sono dotati di magneti disposti correttamente, la coppia magnetica può generare una rotazione che tende ad allinearli senza intervento software diretto. Questo è stato discusso nella Sezione 6 come proprietà del sistema magnetico. L'allineamento attivo, invece, deriva da comandi, bobine, controllo software, simulazione o eventuali attuatori futuri. In un sistema maturo, i due livelli possono cooperare: la fisica produce una tendenza naturale all'allineamento, mentre il software guida o corregge il processo quando necessario.

Questa distinzione è importante perché evita di attribuire al software ciò che può essere ottenuto tramite progettazione fisica. Nel MicroBot, una parte dell'intelligenza può essere incorporata nella materia stessa. Se la forma del nodo, la posizione dei magneti e la polarità sono progettate bene, molte correzioni di orientamento avvengono naturalmente. Il software non deve controllare ogni dettaglio. Questo riduce la complessità computazionale e rende il sistema più robusto. Tuttavia, l'allineamento passivo da solo può essere insufficiente nei casi in cui lo swarm deve riconfigurarsi dinamicamente o scegliere tra più possibili connessioni. Per questo motivo, il controllo attivo resta importante.

L'allineamento può essere considerato anche come una forza di ordine. Coesione e separazione regolano principalmente le distanze; l'allineamento regola le direzioni. Senza allineamento, uno swarm può restare compatto ma disordinato, con nodi orientati casualmente. Questo può essere accettabile per un cluster semplice, ma non per strutture più complesse. Una linea, una catena, una griglia, una superficie o una struttura modulare richiedono che i nodi abbiano orientamenti coerenti. L'allineamento è quindi ciò che trasforma un aggregato in una struttura ordinata.

Nel caso di una formazione lineare, l'allineamento può orientare i MicroBot lungo l'asse della linea. Ogni nodo tende ad assumere un orientamento compatibile con i vicini precedenti e successivi. Nel caso di una formazione circolare, l'allineamento può essere tangenziale o radiale: i nodi possono orientarsi lungo la circonferenza oppure verso il centro, a seconda della funzione desiderata. Nel caso di una griglia, l'allineamento può favorire assi ortogonali. Nel caso di una struttura tridimensionale, può definire piani, strati o direzioni principali. Questo dimostra che l'allineamento non è una singola regola fissa, ma una famiglia di regole adattabili al pattern.

Una possibile formulazione generale è:

$$\text{allineamento_desiderato}_i(t) = \text{funzione}(\text{pattern}, \text{vicini}, \text{ruolo}, \text{stato})$$

Questa formula indica che l'orientamento desiderato di un nodo può dipendere dal pattern corrente, dai nodi vicini, dal ruolo del nodo e dal suo stato operativo. Un nodo terminale in una catena può avere orientamento diverso da un nodo centrale; un nodo leader può orientarsi verso una direzione globale; un nodo di bordo può orientarsi verso l'interno della struttura; un nodo in modalità diagnostica può non partecipare all'allineamento. Il modello deve quindi essere abbastanza flessibile da supportare diversi casi.

Nel contesto delle connessioni magnetiche, l'allineamento può essere rappresentato tramite una funzione di compatibilità:

$$A_{\text{align}}(i, j) = \text{funzione}(\theta_i, \theta_j, \text{polarità}_i, \text{polarità}_j, \text{porta}_i, \text{porta}_j)$$

Questa funzione restituisce un valore alto quando i due nodi sono orientati in modo favorevole all'aggancio, e basso quando l'orientamento è sfavorevole. Tale valore può essere usato per modulare la forza magnetica. Invece di avere una forza attrattiva dipendente solo dalla distanza, si può scrivere:

$$F_{\text{magnetica}_{ij}} = k_m \cdot g(d_{ij}) \cdot A_{\text{align}}(i, j) \cdot n_{ij}$$

Questa formula è molto più coerente con il comportamento reale. Se i nodi sono vicini ma mal orientati, A_{align} è basso e la forza utile di aggancio diminuisce. Se sono vicini e ben orientati, A_{align} è alto e l'aggancio diventa probabile. In questo modo, l'allineamento entra direttamente nella forza magnetica e non resta una condizione separata.

L'allineamento può anche generare una coppia di correzione. Se due nodi sono vicini ma non orientati correttamente, il sistema può non limitarsi ad attrarli, ma può ruotarli verso la compatibilità. In un modello semplificato, questa rotazione può essere rappresentata come:

$$\tau_{ij} = k_{\text{align}} \cdot \text{errore_angolare}_{ij}$$

dove τ_{ij} rappresenta una coppia virtuale o un contributo di rotazione. In un modello fisico, tale coppia può derivare dall'interazione tra dipoli magnetici. Nella simulazione, può essere implementata come correzione dell'angolo θ . Questo comportamento è estremamente utile perché permette di modellare il riallineamento progressivo prima dell'aggancio.

Una sequenza realistica di connessione può essere descritta così:

vicinanza rilevata;

valutazione dell'orientamento;

applicazione di coppia di riallineamento;

riduzione dell'errore angolare;

ingresso nella soglia di attacco;

aggancio;

stabilizzazione.

Questa sequenza è molto più realistica di una connessione istantanea basata solo sulla distanza. Essa rappresenta l'aggancio come processo, non come evento puramente geometrico. Nel progetto MicroBot, questa distinzione è importante perché permette di collegare simulazione, magnetismo, geometria e firmware.

L'allineamento deve essere bilanciato con la separazione. Se due nodi cercano di orientarsi ma sono troppo vicini o sovrapposti, la rotazione può diventare instabile o impossibile. Prima bisogna mantenere una distanza fisica accettabile, poi correggere l'orientamento. In alcuni casi, la separazione può creare lo spazio necessario affinché l'allineamento avvenga. Questo mostra ancora una volta che coesione, separazione e allineamento non sono regole isolate, ma parti di un equilibrio. La coesione avvicina, la separazione protegge lo spazio, l'allineamento organizza la direzione.

L'allineamento deve essere bilanciato anche con il target. Se un nodo deve raggiungere una posizione specifica, il sistema deve decidere se privilegiare il movimento verso il target o l'orientamento con i vicini. In alcune fasi, il target può essere prioritario; in altre, l'allineamento può essere più importante. Per esempio, durante la fase di avvicinamento a una formazione, il nodo può muoversi prima verso la posizione target. Una volta vicino, può orientarsi correttamente per l'aggancio. Questa distinzione permette di evitare conflitti tra traslazione e rotazione.

Dal punto di vista del controllo, è utile separare il controllo della posizione dal controllo dell'orientamento. La posizione può essere aggiornata tramite vettori di spostamento, mentre l'orientamento può essere aggiornato tramite variazioni angolari. Una struttura generale del nodo può quindi includere:

$p_i(t)$: posizione;

$v_i(t)$: velocità;

$\theta_i(t)$: orientamento;

$\omega_i(t)$: velocità angolare.

Nel modello bidimensionale, la velocità angolare $\omega_i(t)$ descrive quanto rapidamente il nodo ruota. L'aggiornamento può essere:

$$\theta_i(t + \Delta t) = \theta_i(t) + \omega_i(t) \cdot \Delta t$$

e la velocità angolare può essere influenzata da contributi di allineamento:

$$\omega_i(t + \Delta t) = \alpha_{\omega} \cdot \omega_i(t) + \tau_i(t) \cdot \Delta t$$

dove α_{ω} rappresenta uno smorzamento rotazionale. Questa struttura rende il modello più dinamico e meno istantaneo. Il nodo non cambia orientamento immediatamente, ma ruota progressivamente.

In una versione iniziale, si può semplificare tutto aggiornando direttamente θ_i verso $\theta_{\text{desiderato}}$. In una versione avanzata, si può introdurre velocità angolare, smorzamento e coppia. Anche qui, la progressione deve essere coerente: prima comportamento comprensibile, poi realismo maggiore. Il documento deve però già prevedere questa possibile estensione, perché l'orientamento sarà cruciale per il sistema magnetico.

L'allineamento è importante anche nella comunicazione visiva del sistema. Nelle simulazioni web o nelle dashboard, i MicroBot non dovrebbero essere rappresentati solo come punti, ma idealmente come unità dotate di direzione. Una freccia, un asse luminoso, una linea interna o una porta evidenziata possono mostrare l'orientamento del nodo. Questo rende immediatamente visibile se lo swarm è disordinato o allineato. Nel caso di MicroBot, dove l'estetica tecnico-visuale è parte forte del progetto, rappresentare l'orientamento aumenterebbe molto il valore dimostrativo.

Dal punto di vista della telemetria, l'allineamento può essere misurato tramite metriche specifiche. Una possibile metrica è l'errore angolare medio tra nodi vicini:

$$E_{\text{align}}(t) = \text{media di } |\Delta\theta_{ij}(t)| \text{ per coppie vicine}$$

Un valore basso indica che i nodi sono orientati in modo coerente; un valore alto indica disordine. Tuttavia, bisogna interpretare questa metrica rispetto al pattern. In una formazione in cui i nodi devono essere paralleli, l'errore basso è desiderabile. In una formazione circolare, gli orientamenti possono essere diversi ma comunque coerenti con la geometria. Per questo motivo, l'allineamento deve essere valutato rispetto a un orientamento atteso, non sempre rispetto all'uguaglianza tra angoli.

Una metrica più generale può essere:

$$E_{\text{orientamento}}(t) = \text{media di errore rispetto all'orientamento}_{\text{desiderato}_i}$$

dove l'orientamento desiderato dipende dal pattern. Questa metrica può essere usata per valutare la qualità della formazione. Se le posizioni sono corrette ma gli orientamenti sono sbagliati, la struttura può apparire giusta nella dashboard ma essere fisicamente non agganciabile. Questo è un punto molto importante: nel MicroBot reale, posizione senza orientamento corretto non basta.

Nel modello dati, ogni nodo dovrebbe quindi includere l'orientamento. Un record completo può avere:

id;

posizione;

orientamento;

velocità;

velocità angolare;

stato magnetico;

connessioni attive;

timestamp.

Questa struttura permette al simulatore e alla dashboard di valutare non solo dove si trova il nodo, ma anche come è orientato e se può interagire correttamente con altri. In futuro, se saranno presenti porte magnetiche specifiche, ogni porta potrà avere una posizione locale e un orientamento locale derivati da quello del nodo.

L'allineamento può essere usato anche per definire ruoli. In una struttura ordinata, alcuni nodi possono orientarsi verso l'esterno, altri verso l'interno, altri lungo un asse. Un nodo leader potrebbe definire una direzione di riferimento; i nodi vicini potrebbero allinearsi a essa; i nodi più lontani potrebbero propagare l'orientamento lungo la struttura. Questo meccanismo permette di creare pattern direzionali senza controllare ogni singolo nodo dall'alto. L'orientamento può propagarsi localmente, come informazione fisica e geometrica.

Questa propagazione è interessante perché collega l'allineamento alla comunicazione implicita. Un nodo può "trasmettere" un orientamento non tramite pacchetto digitale, ma tramite la propria posizione e relazione magnetica. I vicini si allineano, poi i loro vicini si allineano a loro, e così via. In questo modo, una direzione globale può emergere da regole locali. Questo è un comportamento tipico dei sistemi emergenti ed è coerente con la visione MicroBot.

Dal punto di vista magnetico, l'allineamento può ridurre il consumo energetico. Se i nodi sono ben orientati, le forze magnetiche necessarie per l'aggancio possono essere minori. Se invece sono disallineati, il sistema potrebbe dover usare bobine più intensamente per correggere la posizione o generare attrazione sufficiente. Una buona geometria di allineamento passivo può quindi ridurre il bisogno di controllo attivo. Questo è un principio importante di efficienza: progettare bene la forma e l'orientamento riduce il lavoro richiesto all'elettronica.

L'allineamento può anche prevenire agganci indesiderati. Se una connessione richiede orientamento specifico, due nodi vicini ma orientati male non si agganceranno automaticamente. Questo rende il sistema più selettivo e riduce cluster casuali. Nel modello matematico, questo viene ottenuto usando la compatibilità angolare come filtro. Nel sistema fisico, può essere ottenuto tramite polarità, geometria delle superfici e disposizione dei magneti. Ancora una volta, software e fisica cooperano.

Un aspetto delicato è la presenza di orientamenti multipli validi. Alcune connessioni possono essere compatibili in più configurazioni simmetriche. Per esempio, un nodo con geometria simmetrica può agganciarsi in più rotazioni equivalenti. Questo può essere vantaggioso perché aumenta la tolleranza, ma può anche creare ambiguità nella formazione di pattern specifici. Il modello deve quindi distinguere tra orientamenti "validi" e orientamento "preferito". Un orientamento valido permette l'aggancio; un orientamento preferito massimizza la qualità della struttura.

Questa distinzione può essere rappresentata tramite una funzione di costo angolare. Invece di usare una soglia rigida, si può assegnare un punteggio:

`score_align = funzione(errore_angolare)`

Il punteggio è alto quando l'errore è basso e diminuisce quando l'errore cresce. Sotto una certa soglia, l'aggancio è vietato; sopra la soglia minima, l'aggancio è possibile ma può avere qualità diversa. Questo modello è più flessibile e permette di rappresentare connessioni forti o deboli in base all'allineamento.

Nel sistema di connessioni, l'allineamento può quindi influenzare la forza o la stabilità del legame. Una connessione ben allineata può avere maggiore stabilità e maggiore soglia di distacco; una connessione appena compatibile può essere più fragile. In forma concettuale:

stabilità_connessione = funzione(distanza, forza_magnetica, allineamento)

Questo rende il modello più realistico. Non tutte le connessioni sono uguali. Due nodi agganciati perfettamente sono più stabili di due nodi agganciati con orientamento marginale. La dashboard potrebbe rappresentare questa stabilità con colori o valori numerici.

Dal punto di vista della simulazione, l'allineamento deve essere testato separatamente. Prima si può simulare un gruppo di nodi con orientamenti casuali e osservare se convergono verso una direzione comune. Poi si può aggiungere coesione e separazione. Successivamente si può aggiungere magnetismo e verificare se i nodi si agganciano solo quando l'orientamento è compatibile. Infine si può introdurre un pattern e osservare se il sistema raggiunge sia posizione sia orientamento. Questa progressione permette di calibrare il modello evitando confusione tra errori di posizione ed errori di orientamento.

Un test utile può misurare l'errore angolare medio nel tempo. Se l'allineamento funziona, l'errore dovrebbe diminuire. Se diminuisce troppo lentamente, k_θ è troppo basso. Se oscilla, serve smorzamento. Se converge a un orientamento sbagliato, il modello del pattern o della compatibilità è errato. Questi test possono essere registrati e confrontati tra versioni.

Dal punto di vista del firmware futuro, l'allineamento richiederà strumenti fisici o sensoriali. Un MicroBot reale potrebbe non sapere automaticamente il proprio orientamento assoluto, a meno di avere sensori o tracking esterno. Tuttavia, il modello matematico può comunque definire l'orientamento come variabile ideale nella simulazione. In seguito, il sistema potrà stimarla tramite IMU, visione esterna, contatti, magnetometri o altri meccanismi. Per ora è corretto distinguere tra orientamento simulato e orientamento misurato. Il primo serve per modellare; il secondo sarà necessario per controllo fisico avanzato.

Questa distinzione è molto importante dal punto di vista documentale. Non bisogna dichiarare che il prototipo fisico possiede già capacità di orientamento avanzata se non è stata implementata. Però è corretto definire il modello matematico che guiderà tale capacità futura. Il documento deve dire: l'orientamento è variabile necessaria del modello; la sua misura fisica potrà essere introdotta nelle versioni successive. Questo mantiene rigore e visione.

Dal punto di vista teorico avanzato, l'allineamento rappresenta il passaggio dall'aggregazione all'organizzazione. La coesione crea vicinanza, la separazione crea spazio, ma l'allineamento crea ordine. Senza allineamento, lo swarm può essere un gruppo compatto ma casuale. Con l'allineamento, il gruppo può acquisire direzione, struttura, simmetria e funzione. L'allineamento è quindi una proprietà fondamentale della forma. Una forma non è fatta solo da punti nello spazio, ma anche da orientamenti coerenti.

Nel concetto di materia programmabile, questa proprietà è essenziale. Un materiale non è definito solo dalla posizione delle sue particelle, ma anche dal modo in cui esse sono orientate, collegate e organizzate. Se i MicroBot devono costituire una struttura funzionale, non basta che siano vicini: devono essere disposti secondo relazioni corrette. L'allineamento permette di trasformare un ammasso di unità in una configurazione leggibile e potenzialmente utile.

L'allineamento può anche essere interpretato come riduzione dell'entropia locale. All'inizio, gli orientamenti possono essere casuali; attraverso interazioni locali, diventano più ordinati. Il sistema passa da disordine a ordine senza che il controller debba specificare ogni angolo manualmente. Questo è uno degli aspetti più interessanti dello swarm: l'ordine emerge da regole locali ripetute. MicroBot può sfruttare questa logica per costruire forme che sembrano progettate centralmente, ma che in realtà derivano da interazioni distribuite.

In conclusione, l'allineamento costituisce una regola fondamentale del modello matematico MicroBot. Esso permette ai nodi di coordinare direzioni di movimento, orientamenti fisici, assi magnetici e superfici di aggancio. Completa la triade coesione-separazione-allineamento, trasformando un gruppo compatto e non sovrapposto in una struttura ordinata. L'allineamento può essere simulativo, fisico, passivo, attivo, locale o

dipendente dal pattern; può influenzare forza magnetica, stabilità delle connessioni, soglie di attacco e qualità della forma. Il suo ruolo è essenziale per passare da semplice aggregazione a organizzazione strutturale. Il punto successivo, dedicato al sistema multi-agente, porterà questa logica a un livello superiore, descrivendo come ogni MicroBot possa essere interpretato come agente autonomo inserito in una rete di regole locali, stati interni, relazioni e comportamenti collettivi.

7.14 Sistema multi-agente

Il sistema multi-agente rappresenta il livello concettuale e matematico in cui ogni MicroBot viene interpretato non soltanto come componente fisico o nodo elettronico, ma come agente autonomo inserito in una rete dinamica di relazioni. Dopo aver definito posizione, distanza, interazione, soglie, hysteresis, coesione, separazione e allineamento, diventa necessario unificare questi elementi dentro una visione più generale: lo swarm MicroBot è un sistema composto da più agenti, ciascuno dotato di uno stato interno, di capacità di percezione locale, di regole di aggiornamento e di possibilità di interazione con altri agenti. Il comportamento globale non viene imposto direttamente a ogni livello, ma emerge dalla ripetizione coordinata di regole locali applicate da molte unità.

Nel contesto MicroBot, un agente può essere definito come un'entità autonoma capace di mantenere uno stato, ricevere o stimare informazioni, applicare regole decisionali e produrre un effetto sul sistema. Ogni MicroBot possiede infatti una posizione, un orientamento, uno stato energetico, uno stato magnetico, uno stato comunicativo, eventuali connessioni attive e un ruolo operativo. Queste variabili non sono semplici dati isolati, ma costituiscono la base su cui l'agente decide come comportarsi nel ciclo successivo della simulazione o del controllo. In forma generale, lo stato del MicroBot i -esimo può essere rappresentato come:

$$s_i(t) = (p_i(t), \theta_i(t), v_i(t), E_i(t), M_i(t), C_i(t), R_i(t), \text{safety}_i(t))$$

dove $p_i(t)$ rappresenta la posizione, $\theta_i(t)$ l'orientamento, $v_i(t)$ la velocità, $E_i(t)$ lo stato energetico, $M_i(t)$ lo stato magnetico, $C_i(t)$ lo stato comunicativo, $R_i(t)$ il ruolo operativo e $\text{safety}_i(t)$ lo stato di sicurezza. Questa rappresentazione permette di descrivere il MicroBot come agente complesso, non come semplice punto geometrico.

La caratteristica principale di un sistema multi-agente è che nessun agente, preso singolarmente, contiene l'intero comportamento globale. Ogni MicroBot applica regole relativamente semplici, ma l'insieme di molti MicroBot può produrre strutture e dinamiche più complesse. Questa è una differenza fondamentale rispetto a un sistema centralizzato tradizionale. In un sistema puramente centralizzato, un controller calcola ogni azione e ogni posizione; in un sistema multi-agente, invece, il comportamento globale deriva dalla cooperazione tra decisioni locali, vincoli fisici e comandi di alto livello. MicroBot si colloca in una posizione ibrida: può avere un controller centrale, ma conserva una logica locale distribuita nei nodi.

Questa architettura ibrida è particolarmente importante perché rende il sistema più realistico e scalabile. Il controller centrale può definire obiettivi, pattern, parametri globali o stati operativi, ma non deve necessariamente controllare ogni micro-interazione tra ogni coppia di nodi. I MicroBot possono invece gestire localmente vicinanza, separazione, allineamento, aggancio, distacco e safety. Questo riduce il carico del controller e rende il sistema più robusto rispetto a ritardi, perdita di pacchetti o variazioni locali. L'intelligenza non è concentrata in un solo punto, ma distribuita nella rete.

Dal punto di vista matematico, un sistema multi-agente può essere descritto come insieme di agenti A :

$$A = \{a_1, a_2, \dots, a_n\}$$

dove ogni agente a_i corrisponde a un MicroBot. Ogni agente possiede uno stato $s_i(t)$ e una funzione di aggiornamento:

$$s_i(t + \Delta t) = f_i(s_i(t), N_i(t), u_i(t), \text{ambiente}(t))$$

Questa formula è centrale. Essa indica che lo stato futuro dell'agente i dipende dal suo stato corrente, dal vicinato $N_i(t)$, dal comando ricevuto $u_i(t)$ e dalle condizioni dell'ambiente. Il vicinato rappresenta gli altri agenti che influenzano il nodo; il comando rappresenta l'eventuale intervento del controller o del pattern; l'ambiente rappresenta limiti spaziali, superfici, ostacoli, condizioni operative o parametri globali. In questo modo, il comportamento del MicroBot viene espresso come funzione locale, ma collegata al sistema.

Il vicinato $N_i(t)$ è uno degli elementi fondamentali del sistema multi-agente. Ogni MicroBot non deve necessariamente considerare tutti gli altri nodi, ma soltanto quelli che rientrano entro una certa distanza o che sono connessi a esso. Questo rende il sistema locale. L'agente non agisce sulla base di una visione totale dello swarm, ma sulla base di informazioni parziali e rilevanti. Questa scelta è coerente con la fisica del sistema, perché le interazioni magnetiche sono locali, e con la progettazione embedded, perché un nodo reale avrà risorse limitate.

Nel modello MicroBot, il vicinato può essere geometrico, magnetico, comunicativo o strutturale. Il vicinato geometrico include i nodi vicini nello spazio. Il vicinato magnetico include i nodi entro la zona di influenza del campo e con orientamento compatibile. Il vicinato comunicativo include i nodi con cui è possibile scambiare messaggi o dai quali si ricevono informazioni. Il vicinato strutturale include i nodi già connessi fisicamente tramite aggancio magnetico. Questi vicinati non coincidono sempre. Un nodo può essere geometricamente vicino ma non connesso; può essere connesso magneticamente ma non comunicare direttamente; può ricevere dati dal controller su nodi che non sono fisicamente vicini. Distinguere questi livelli aumenta la precisione del modello.

In forma generale, si possono definire più insiemi:

$N_{i_geo}(t)$ = vicini geometrici

$N_{i_mag}(t)$ = vicini magneticamente rilevanti

$N_{i_comm}(t)$ = vicini comunicativi

$N_{i_conn}(t)$ = nodi fisicamente connessi

Questa distinzione permette di modellare con rigore situazioni diverse. Per esempio, la separazione può dipendere da N_{i_geo} , l'aggancio da N_{i_mag} , lo scambio di stato da N_{i_comm} , la stabilità strutturale da N_{i_conn} . Ogni regola del sistema usa il vicinato appropriato, evitando ambiguità.

Un agente MicroBot deve inoltre possedere una politica decisionale. La politica è la regola che trasforma stato e informazioni locali in azione. In un modello semplice, questa politica può essere deterministica: date le stesse condizioni, il nodo produce sempre la stessa risposta. In un modello più avanzato, può includere priorità, pesi, ruoli, probabilità o apprendimento. Per questa fase del documento, è corretto mantenere una politica deterministica e interpretabile, basata su regole chiare. Questo rende il sistema più facile da simulare, documentare e verificare.

Una politica locale può essere descritta così:

$$azione_i(t) = \pi(s_i(t), N_i(t), u_i(t))$$

dove π rappresenta la funzione decisionale dell'agente. L'azione può includere aggiornamento della posizione, variazione dell'orientamento, attivazione o disattivazione di una bobina, emissione di un messaggio, cambio di stato, richiesta di aggancio, rifiuto di una connessione o ingresso in modalità safety. In questo modo, il MicroBot non è passivo: interpreta il proprio contesto e produce una risposta coerente.

Nel sistema MicroBot, le azioni possono essere classificate in diverse categorie. Le azioni geometriche modificano posizione o orientamento nella simulazione. Le azioni magnetiche attivano o modulano campi, bobine o stati di aggancio. Le azioni comunicative inviano o ricevono messaggi. Le azioni diagnostiche aggiornano log, LED o telemetria. Le azioni di sicurezza limitano o interrompono comportamenti rischiosi. Questa classificazione rispecchia la natura multilivello del progetto: un agente non agisce solo nello spazio, ma anche nel dominio energetico, informativo e operativo.

Un aspetto molto importante del sistema multi-agente è la priorità tra le azioni. Un MicroBot potrebbe ricevere un comando di aggancio, ma trovarsi in stato di batteria bassa; potrebbe essere vicino a un target, ma anche troppo vicino a un altro nodo; potrebbe dover mantenere coesione, ma allo stesso tempo evitare collisione; potrebbe essere chiamato ad attivare una bobina, ma avere carico termico elevato. Per evitare conflitti, il sistema deve stabilire una gerarchia di priorità. In generale, la sicurezza deve prevalere sul comportamento, la separazione deve prevenire collisioni, l'aggancio deve rispettare compatibilità e il target deve essere subordinato ai vincoli fisici.

Una possibile gerarchia è:

safety
protezione elettrica ed energetica
vincoli fisici e separazione
connessioni già attive
obiettivi di pattern
coesione e allineamento
ottimizzazioni o comportamenti secondari

Questa gerarchia garantisce che il MicroBot non insegua un obiettivo globale violando condizioni locali fondamentali. Il nodo deve poter rifiutare o modificare un comando se lo stato locale non lo consente. Questo è un punto cruciale: in un sistema multi-agente serio, gli agenti non devono essere esecutori ciechi. Devono avere autonomia minima per proteggere il proprio stato e mantenere coerenza fisica.

Il sistema multi-agente introduce anche il concetto di ruolo. Non tutti i MicroBot devono necessariamente essere identici nel comportamento in ogni istante. Anche se l'hardware può essere omogeneo, il ruolo operativo può variare. Un nodo può essere leader locale, follower, nodo di bordo, nodo centrale, nodo ponte, nodo inattivo, nodo diagnostico o nodo in safety. Il ruolo modifica la politica decisionale. Un nodo leader può orientare un gruppo; un nodo follower può allinearsi; un nodo ponte può mantenere connessioni tra cluster; un nodo di bordo può proteggere la forma; un nodo in safety può sospendere l'attuazione.

Formalmente, il ruolo $R_i(t)$ può essere parte dello stato:

$$s_i(t) = (\dots, R_i(t), \dots)$$

e la politica diventa:

$$azione_i(t) = \pi(s_i(t), N_i(t), u_i(t), R_i(t))$$

Questo consente di usare la stessa architettura di base per agenti con comportamenti diversi. La differenziazione non deve per forza essere hardware; può essere logica. Questa è una proprietà molto utile per uno swarm, perché permette di creare organizzazione interna senza costruire nodi fisicamente diversi.

Nel contesto delle formazioni, i ruoli possono essere assegnati in base al pattern. In una linea, alcuni nodi possono essere estremità, altri nodi interni. In un cerchio, alcuni nodi possono controllare la curvatura locale. In un cluster, alcuni nodi possono essere centrali e altri periferici. In una struttura più complessa, alcuni MicroBot possono fungere da punti di connessione tra moduli. Questo rende il sistema più ricco rispetto a un insieme di agenti totalmente indistinti.

Un sistema multi-agente può funzionare con ruoli assegnati dal controller oppure con ruoli emergenti. Nel primo caso, il controller decide che il nodo 1 è leader, il nodo 2 è follower, e così via. Nel secondo caso, i ruoli derivano dalla posizione o dallo stato: un nodo con molti vicini può diventare centrale; un nodo con una sola connessione può essere classificato come terminale; un nodo tra due cluster può diventare ponte. La seconda modalità è più emergente e scalabile, ma anche più complessa. Nelle prime versioni, è più semplice usare ruoli espliciti o simulati; nelle versioni avanzate, si può introdurre classificazione automatica.

Dal punto di vista del grafo, il sistema multi-agente può essere rappresentato come una rete dinamica. Gli agenti sono nodi del grafo; le interazioni sono archi. Gli archi possono indicare vicinanza, comunicazione, magnetismo o connessione fisica. Il grafo cambia nel tempo perché gli agenti si muovono, si agganciano, si separano, cambiano ruolo o modificano stato. In forma generale:

$$G(t) = (A, E(t))$$

dove A è l'insieme degli agenti ed $E(t)$ l'insieme delle relazioni al tempo t . Questa rappresentazione è molto potente perché permette di analizzare lo swarm non solo come geometria, ma come struttura relazionale. La forma non è soltanto la posizione degli agenti, ma il grafo delle loro connessioni.

Nel sistema MicroBot, il grafo può essere multilivello:

G_geo(t): grafo di vicinanza geometrica
G_mag(t): grafo di interazione magnetica
G_conn(t): grafo di connessioni fisiche
G_comm(t): grafo comunicativo
G_role(t): grafo funzionale o organizzativo

Questa stratificazione permette di distinguere relazioni diverse. Due nodi possono essere vicini ma non connessi; connessi ma non comunicanti direttamente; comunicanti ma lontani fisicamente; appartenenti allo stesso ruolo ma non vicini. Un modello rigoroso deve poter rappresentare queste differenze. Questo è particolarmente importante per un sistema come MicroBot, in cui fisica, elettronica e software si sovrappongono.

Il sistema multi-agente è anche il livello in cui diventa chiara la differenza tra comportamento locale e comportamento globale. Il comportamento locale è ciò che ogni agente calcola usando il proprio stato e i vicini. Il comportamento globale è ciò che osserviamo nell'intero swarm: forma, pattern, stabilità, aggregazione, dispersione, transizione. La relazione tra questi due livelli non è banale. Piccole modifiche alle regole locali possono produrre grandi cambiamenti nel comportamento globale. Per questo motivo, il modello deve essere testato progressivamente e documentato con precisione.

Un esempio semplice è il bilanciamento tra coesione e separazione. A livello locale, ogni agente si avvicina ai vicini ma evita quelli troppo vicini. A livello globale, questa regola può produrre cluster ordinati. Se il peso della coesione aumenta troppo, il cluster collassa. Se aumenta troppo la separazione, lo swarm si espande. Il comportamento globale dipende quindi da parametri locali. Questa è la logica fondamentale dei sistemi multi-agente.

Il sistema multi-agente consente anche robustezza. Se un nodo fallisce, gli altri possono continuare ad applicare le proprie regole. In un sistema puramente centralizzato, il fallimento di un elemento chiave può compromettere tutto. In uno swarm, invece, la ridondanza distribuita permette maggiore resilienza. Naturalmente, questa resilienza non è automatica: deve essere progettata. I nodi devono poter rilevare assenza, errore, perdita di connessione o stato safety dei vicini, e adattarsi. Tuttavia, il modello multi-agente offre la base per questa capacità.

Nel progetto MicroBot, un nodo in errore può essere rimosso dal vicinato attivo, isolato, segnalato tramite LED o dashboard, oppure trattato come elemento passivo. Gli altri agenti possono ricalcolare coesione, separazione e pattern senza di esso. Questo comportamento sarà importante nelle sezioni su safety, error handling e scalabilità. Già nel modello matematico, però, bisogna prevedere che non tutti gli agenti siano sempre pienamente operativi.

Uno stato operativo semplificato può includere:

idle
active
connected
seeking_connection
stabilizing
detaching
low_energy
safety
error

Ogni stato modifica il comportamento dell'agente. Un nodo idle può muoversi poco o attendere; active può partecipare pienamente; connected mantiene vincoli; seeking_connection cerca agganci; stabilizing riduce movimenti bruschi; detaching favorisce separazione; low_energy limita bobine e comunicazione; safety blocca azioni rischiose; error richiede esclusione o diagnostica. Questa macchina a stati rende il sistema multi-agente più controllabile.

Il sistema multi-agente deve anche prevedere comunicazione. Un agente può basarsi solo su fisica e vicinanza, ma un MicroBot reale può anche ricevere messaggi dal controller o da altri nodi. Questi messaggi possono contenere comandi, timestamp, stato del pattern, ruolo, richiesta di aggancio, stato energetico o informazioni diagnostiche. La comunicazione permette di coordinare agenti che altrimenti reagirebbero solo localmente. Tuttavia, la comunicazione può essere imperfetta: pacchetti persi, ritardi, latenze o asincronie.

Per questo motivo, la politica dell'agente deve essere robusta e non dipendere completamente da un singolo messaggio.

Un principio utile è:

comunicazione per coordinare, regole locali per stabilizzare.

Questo significa che il controller e i messaggi possono orientare il comportamento, ma la stabilità immediata deve derivare anche da regole locali come separazione, safety e hysteresis. Se un comando arriva in ritardo, il nodo deve comunque evitare collisioni, surriscaldamento o agganci non validi. Questo rende il sistema più affidabile.

Dal punto di vista temporale, il sistema multi-agente può essere sincrono o asincrono. In una simulazione sincrona, tutti gli agenti aggiornano il proprio stato nello stesso passo temporale. Questo è più semplice e rende il modello pulito. In un sistema reale, però, gli agenti possono aggiornarsi con piccoli ritardi, frequenze diverse o condizioni diverse. Per le prime versioni, il modello sincrono è corretto e utile. Successivamente, si potranno introdurre latenze e aggiornamenti asincroni per aumentare realismo. Anche qui, la progressione deve essere ordinata.

Nel modello sincrono:

tutti leggono $S(t)$
tutti calcolano azioni
tutti aggiornano $S(t + \Delta t)$

Nel modello asincrono:

ogni agente aggiorna secondo il proprio ciclo, usando informazioni disponibili localmente.

Il primo modello è più adatto alla simulazione e alla documentazione iniziale; il secondo è più vicino all'hardware distribuito. Il progetto MicroBot può utilizzare il modello sincrono come base teorica e considerare quello asincrono come estensione futura.

Un aspetto molto importante è la scalabilità computazionale. Se ogni agente interagisce con tutti gli altri, il costo cresce rapidamente. Un sistema multi-agente scalabile deve limitare le interazioni ai vicini. Questo è coerente con le sezioni precedenti sulla distanza e sul vicinato. La regola locale non solo è più realistica, ma riduce anche il carico computazionale. Per swarm piccoli si possono calcolare tutte le coppie; per swarm grandi sarà necessario usare strutture spaziali o filtri di vicinanza. Questa distinzione prepara le sezioni future sulla scalabilità.

Dal punto di vista dei file e della simulazione, il sistema multi-agente può essere rappresentato tramite configurazioni iniziali, stati dei nodi, parametri globali e regole locali. Un file di configurazione potrebbe definire numero di agenti, raggio di vicinato, pesi delle forze, soglie di attacco/distacco, parametri di hysteresis, ruoli iniziali e modalità operative. Uno snapshot potrebbe salvare lo stato di ogni agente e le relazioni attive. Questa organizzazione è coerente con l'ecosistema MicroBot già basato su dataset, snapshot, runtime state e dashboard.

Un record agente potrebbe contenere:

id
position
orientation
velocity
role
energy_state
magnetic_state
connection_state
safety_state
last_update_timestamp

Un record relazione potrebbe contenere:

```
node_a
node_b
relation_type
distance
strength
state
created_at
updated_at
```

Queste strutture permettono di trattare lo swarm come sistema osservabile. Non si registrano solo posizioni, ma stati e relazioni. Questo è essenziale per debugging, validazione e documentazione tecnica.

Il sistema multi-agente è anche il livello in cui si può introdurre valutazione delle prestazioni. Non basta osservare se i nodi si muovono; bisogna misurare se il comportamento collettivo è corretto. Alcune metriche possibili sono: coesione media, distanza minima, errore rispetto al pattern, numero di connessioni attive, durata media delle connessioni, tasso di distacco, numero di collisioni evitate, energia consumata, nodi in safety, tempo di convergenza. Queste metriche permettono di confrontare versioni del simulatore e configurazioni diverse.

Una metrica importante è il tempo di convergenza, cioè il tempo necessario affinché lo swarm raggiunga una configurazione considerata stabile. Una configurazione può essere considerata stabile se l'errore rispetto al pattern è sotto una soglia, le connessioni richieste sono attive, le collisioni sono assenti e le variazioni di posizione sono piccole. In forma concettuale:

```
stabile se E_pattern <  $\epsilon_{\text{pattern}}$ 
e collision_count = 0
e connection_requirements soddisfatti
e velocità media <  $\epsilon_v$ 
```

Questa definizione permette di trasformare una valutazione visiva in criterio numerico.

Dal punto di vista teorico, il sistema multi-agente mostra come l'intelligenza del MicroBot non risieda in un singolo componente, ma nella relazione tra componenti. Il singolo agente è relativamente semplice, ma la rete di agenti produce complessità. Questo è il principio fondamentale dell'emergenza. Il sistema globale può manifestare comportamenti non esplicitamente programmati in ogni dettaglio, ma derivanti dall'interazione tra regole locali, geometria, magnetismo e comunicazione.

Tuttavia, emergenza non significa assenza di controllo. Nel progetto MicroBot, l'emergenza deve essere guidata, osservabile e limitata da vincoli. Uno swarm completamente libero può produrre comportamenti imprevedibili; uno swarm troppo controllato perde la sua natura distribuita. La soluzione è un equilibrio: regole locali robuste, parametri globali chiari, safety layer forte, telemetria continua e pattern definiti. Il sistema multi-agente fornisce la struttura per questo equilibrio.

Questa visione è particolarmente coerente con la materia programmabile. Una materia programmabile non viene controllata atomo per atomo da un supervisore centrale; le sue unità seguono regole locali che producono una forma complessiva. MicroBot cerca di avvicinarsi a questo paradigma in modo ingegneristico: ogni nodo è un agente, ogni agente ha stato e regole, le relazioni producono forma, la forma può cambiare quando cambiano regole, soglie, campi o ruoli.

Un altro aspetto importante è che il sistema multi-agente consente modularità. Se il modello di un singolo agente è definito bene, si può aumentare il numero di agenti senza riscrivere tutta l'architettura. Ogni nuovo MicroBot aggiunge un nuovo stato s_i e nuove relazioni potenziali con i vicini. Questo è il principio che permette allo swarm di scalare. Naturalmente, la scalabilità reale richiederà ottimizzazioni, ma la base concettuale è questa: il comportamento globale nasce dalla ripetizione modulare di un agente base.

Nel contesto delle versioni del progetto, il sistema multi-agente può evolvere gradualmente. Una prima versione può avere agenti simulati identici, con solo posizione e velocità. Una seconda può aggiungere orientamento. Una terza può introdurre energia, magnetismo e connessioni. Una quarta può aggiungere ruoli. Una quinta può integrare comunicazione, safety e telemetria. Una sesta può collegare agenti simulati a

nodi fisici reali. Questa progressione è coerente con lo sviluppo incrementale già seguito nel progetto MicroBot.

Dal punto di vista documentale, questa sezione ha una funzione importante: chiarisce che MicroBot non è soltanto un robot singolo replicato molte volte, ma un sistema di agenti. Il valore del progetto non è nel singolo modulo isolato, ma nella capacità dei moduli di interagire secondo regole coerenti. Questo sposta il focus dalla costruzione del singolo oggetto alla progettazione dell'ecosistema. Il singolo MicroBot è necessario, ma lo swarm è il vero oggetto del progetto.

In conclusione, il sistema multi-agente costituisce il livello di sintesi del modello matematico MicroBot. Ogni MicroBot viene rappresentato come agente dotato di stato, regole, vicinato, ruolo, capacità di interazione e vincoli di sicurezza. Il comportamento globale nasce dall'evoluzione simultanea di molti agenti collegati da relazioni geometriche, magnetiche, comunicative e strutturali. Questo modello permette di collegare simulazione, firmware, magnetismo, telemetria e pattern in una struttura unica. Il sistema multi-agente è quindi il ponte tra il comportamento locale dei singoli MicroBot e l'emergenza del comportamento collettivo, che verrà approfondita nel punto successivo.

7.15 Emergenza del comportamento collettivo

L'emergenza del comportamento collettivo rappresenta il punto conclusivo della Sezione 7 e costituisce il risultato teorico più importante del modello matematico MicroBot. Dopo aver definito le coordinate, il movimento discreto, l'aggiornamento delle posizioni, la distanza euclidea, l'interazione tra unità, le soglie di attacco e distacco, l'hysteresis, le forze dello swarm, la coesione, la separazione, l'allineamento e il sistema multi-agente, è ora possibile descrivere il fenomeno centrale del progetto: il comportamento globale dello swarm non viene programmato direttamente in ogni singolo dettaglio, ma nasce dalla combinazione di molte regole locali applicate simultaneamente da più MicroBot.

Nel progetto MicroBot, l'emergenza non deve essere intesa come un concetto vago o puramente filosofico. Deve essere trattata come una proprietà tecnica del sistema. Un comportamento emergente si verifica quando l'insieme produce una dinamica che non è riducibile alla semplice osservazione di un singolo nodo isolato. Un MicroBot singolo può avere posizione, orientamento, stato energetico, stato magnetico e logica locale; tuttavia, da solo non può generare una forma collettiva. La forma appare solo quando più MicroBot interagiscono tra loro attraverso distanza, magnetismo, comunicazione, soglie, vincoli e regole di aggiornamento. L'emergenza nasce quindi nella relazione, non nel singolo componente.

Dal punto di vista matematico, lo stato globale dello swarm può essere rappresentato come l'insieme degli stati individuali e delle relazioni attive tra essi:

$$S_{\text{global}}(t) = (S_{\text{nodes}}(t), E_{\text{relations}}(t))$$

dove $S_{\text{nodes}}(t)$ rappresenta l'insieme degli stati dei singoli MicroBot, mentre $E_{\text{relations}}(t)$ rappresenta l'insieme delle relazioni geometriche, magnetiche, comunicative e strutturali presenti nel sistema. Questa formulazione è fondamentale perché chiarisce che lo stato globale non è soltanto una lista di nodi, ma una rete dinamica di nodi e connessioni. Due sistemi con gli stessi MicroBot, nelle stesse posizioni, possono comportarsi in modo diverso se le relazioni tra essi sono diverse. La forma dello swarm è quindi geometrica e relazionale allo stesso tempo.

L'emergenza del comportamento collettivo può essere descritta come il risultato della ripetizione iterativa di un ciclo locale. A ogni passo temporale, ogni MicroBot valuta il proprio stato, osserva o riceve informazioni dal proprio vicinato, calcola contributi di coesione, separazione, allineamento, magnetismo e target, aggiorna posizione e orientamento, verifica soglie di attacco e distacco, aggiorna le connessioni e registra eventuali eventi. Questo ciclo viene eseguito da molti nodi contemporaneamente. Il risultato non è una semplice somma di azioni indipendenti, ma una trasformazione collettiva della configurazione.

In forma concettuale, il ciclo può essere espresso così:

stato locale → interazioni locali → aggiornamento locale → nuova configurazione globale → nuove interazioni locali

Questo ciclo è il motore dell'emergenza. Ogni MicroBot modifica localmente il sistema; la somma di queste modifiche cambia lo stato globale; il nuovo stato globale modifica a sua volta i vicinati e le relazioni locali. Il

sistema è quindi ricorsivo e dinamico: le azioni locali producono una configurazione collettiva, e la configurazione collettiva modifica le condizioni delle azioni locali successive. Questa retroazione è ciò che permette allo swarm di evolvere nel tempo.

Un esempio semplice è la formazione di un cluster. Nessun MicroBot deve necessariamente contenere nel proprio firmware l'immagine completa del cluster finale. Può bastare che ogni nodo tenda a rimanere vicino ai vicini, eviti sovrapposizioni, mantenga connessioni magnetiche stabili e rispetti soglie di sicurezza. Da queste regole locali può emergere una struttura compatta. Il cluster non è stato disegnato nodo per nodo da un comando centrale dettagliato; è nato dal bilanciamento tra coesione, separazione e connessioni. Questo è un comportamento emergente perché la struttura globale è prodotta da regole locali ripetute.

Un secondo esempio è la formazione di una linea. In questo caso, il comportamento emergente non deriva soltanto dalla coesione, perché una coesione eccessiva produrrebbe un ammasso. Serve anche allineamento direzionale, separazione tra nodi, eventuale asse target e connessioni preferenziali tra vicini consecutivi. Il risultato finale, cioè una catena ordinata di MicroBot, nasce dalla combinazione di più regole. Il singolo nodo non deve necessariamente conoscere l'intera linea; deve sapere come relazionarsi con i vicini, mantenere distanza, orientarsi e rispettare il ruolo assegnato o emergente. La linea appare come proprietà globale della rete.

Un terzo esempio è la riconfigurazione. Uno swarm realmente interessante non deve solo formare una struttura, ma deve poter passare da una struttura a un'altra. Questo richiede che alcune connessioni vengano mantenute, altre indebolite, altre sciolte e nuove connessioni create. La riconfigurazione emerge dal coordinamento tra soglie di attacco, soglie di distacco, hysteresis, forza magnetica, target e stato operativo. Il sistema non cambia forma perché ogni nodo viene spostato manualmente in una posizione finale, ma perché le condizioni locali vengono modificate in modo da rendere stabile una nuova configurazione. Questa è una delle idee più forti del progetto MicroBot: programmare la forma significa programmare le condizioni di interazione, non necessariamente ogni singolo movimento.

L'emergenza deve però essere controllata. Un comportamento emergente non è automaticamente utile. Uno swarm può emergere in modo caotico, formando cluster indesiderati, connessioni errate, oscillazioni, collisioni o strutture instabili. Per questo motivo, nel progetto MicroBot l'emergenza deve essere guidata da vincoli. Le soglie impediscono agganci arbitrari; la separazione impedisce collassi; l'hysteresis impedisce oscillazioni; il safety layer impedisce comportamenti pericolosi; i pattern danno una direzione al sistema; la telemetria permette di osservare ciò che accade. L'obiettivo non è lasciare lo swarm completamente libero, ma costruire un sistema in cui l'emergenza sia orientata, leggibile e correggibile.

Questo punto è metodologicamente importante. In un progetto ingegneristico, non basta dire che il comportamento "emerge". Bisogna definire da quali regole emerge, entro quali limiti, con quali metriche e con quali condizioni di sicurezza. MicroBot deve quindi distinguere tra emergenza desiderata ed emergenza indesiderata. L'emergenza desiderata è la formazione di strutture coerenti, stabili e riconfigurabili. L'emergenza indesiderata è la formazione di aggregati casuali, instabilità, collisioni, distacchi continui o consumo energetico incontrollato. Il modello matematico della Sezione 7 serve proprio a separare questi due casi.

Dal punto di vista delle metriche, il comportamento emergente può essere valutato attraverso grandezze misurabili. La coesione può essere misurata tramite dispersione media o connettività del grafo. La separazione può essere misurata tramite distanza minima e numero di collisioni evitate. L'allineamento può essere misurato tramite errore angolare medio. La stabilità può essere misurata tramite durata delle connessioni, detach rate e oscillation count. La qualità del pattern può essere misurata tramite errore medio rispetto alle posizioni target. La sicurezza può essere valutata attraverso numero di eventi safety, limiti termici, energia consumata e comandi rifiutati. In questo modo, l'emergenza non resta un'impressione visiva, ma diventa un fenomeno analizzabile.

Una possibile rappresentazione della qualità globale dello swarm può essere:

$Q(t)$ = funzione(coesione, separazione, allineamento, stabilità_connessioni, errore_pattern, sicurezza, energia)

Questa funzione non deve necessariamente essere implementata subito come formula unica, ma è utile come idea progettuale. Un comportamento emergente è buono se lo swarm è abbastanza coeso, non collassa, mantiene orientamento coerente, forma connessioni stabili, si avvicina al pattern desiderato, resta

sicuro e consuma energia in modo sostenibile. La qualità globale non dipende da una sola metrica, ma da un equilibrio tra più dimensioni.

L'emergenza del comportamento collettivo è collegata anche al concetto di grafo dinamico. Ogni MicroBot è un nodo del grafo; ogni relazione di vicinanza, interazione magnetica, comunicazione o aggancio è un arco. La configurazione dello swarm può quindi essere vista come un grafo che cambia nel tempo:

$$G(t) = (V, E(t))$$

Il comportamento collettivo emerge dall'evoluzione di questo grafo. Quando compaiono nuovi archi, il sistema si aggrega; quando alcuni archi vengono rimossi, il sistema si separa o si riconfigura; quando il grafo diventa più ordinato, emergono pattern; quando il grafo si frammenta, lo swarm perde coesione. Questa rappresentazione è molto potente perché permette di analizzare la forma non solo come disposizione spaziale, ma come topologia di relazioni.

Nel contesto della materia programmabile, questa idea è centrale. Una materia programmabile non è semplicemente un materiale che cambia posizione nello spazio; è un sistema in cui le unità modificano le proprie relazioni. La forma finale dipende da quali connessioni esistono, da quanto sono stabili, da quali orientamenti hanno e da come possono essere sciolte o riformate. Nel MicroBot, la materia programmabile è quindi modellata come rete dinamica di agenti fisici. La programmazione della forma avviene modificando le condizioni locali che regolano la rete.

L'emergenza è inoltre collegata alla scalabilità. Un sistema centralizzato può funzionare bene con pochi nodi, ma diventa complesso quando il numero di unità cresce. Un sistema emergente basato su regole locali può invece scalare meglio, perché ogni nodo applica regole simili rispetto ai propri vicini. Naturalmente, la scalabilità non è automatica: servono ottimizzazione del calcolo dei vicini, gestione della comunicazione, limiti energetici e protocolli robusti. Tuttavia, il modello emergente è la base concettuale corretta per passare da pochi prototipi a swarm più numerosi.

L'emergenza introduce anche robustezza. Se il comportamento globale dipende da molte interazioni locali, il sistema può tollerare il fallimento o l'assenza di alcuni nodi, purché la rete complessiva resti sufficientemente connessa. Un singolo MicroBot guasto non deve necessariamente compromettere l'intera struttura. Gli altri nodi possono adattarsi, ricalcolare vicini, modificare connessioni o compensare il vuoto. Questa proprietà è tipica dei sistemi distribuiti e rappresenta uno dei vantaggi potenziali dello swarm rispetto a un robot monolitico.

Tuttavia, la robustezza emergente richiede regole adatte. Se il sistema dipende troppo da un singolo nodo leader, diventa fragile. Se invece i ruoli possono essere riassegnati o se la struttura possiede connessioni ridondanti, diventa più resiliente. Nel modello MicroBot, questo significa che il sistema multi-agente dovrebbe prevedere ruoli dinamici, fallback, esclusione dei nodi in errore e ricalcolo delle connessioni. L'emergenza utile non è casuale: è progettata per assorbire perturbazioni.

Dal punto di vista del controllo, l'emergenza può essere guidata attraverso parametri globali. Il controller non deve per forza comandare ogni MicroBot individualmente; può modificare pesi, soglie, modalità operative o pattern. Per esempio, aumentando il peso della coesione, lo swarm tende ad aggregarsi; aumentando la separazione, tende a espandersi; aumentando l'allineamento, tende a ordinarsi; modificando le soglie di attacco, si rende più facile o più difficile la formazione di connessioni; cambiando il pattern target, si orienta la trasformazione verso una forma specifica. In questo modo, il controllo globale agisce sui parametri del comportamento emergente.

Questa modalità è molto diversa dal controllo robotico tradizionale. In un sistema tradizionale, il controller può calcolare una traiettoria precisa per ogni attuatore. In MicroBot, il controller può impostare condizioni e lasciare che il sistema converga attraverso regole locali. Questa è una forma di controllo indiretto. Non si ordina direttamente ogni dettaglio della forma; si modificano le regole che rendono quella forma più probabile, più stabile o più favorevole. Questa idea è perfettamente coerente con il concetto di programmable matter.

L'emergenza del comportamento collettivo deve essere osservabile. Se lo swarm produce una forma, il sistema deve poter spiegare almeno in parte come ci è arrivato. Per questo servono log, snapshot, dashboard, metriche e registrazione degli eventi. Ogni `attach_event`, `detach_event`, `collision_warning`, `alignment_update` o `safety_trigger` contribuisce alla storia del comportamento emergente. Senza

osservabilità, l'emergenza diventa una scatola nera. Con osservabilità, diventa un fenomeno tecnico studiabile, migliorabile e documentabile.

Nel progetto MicroBot, questa osservabilità può essere realizzata attraverso snapshot dello stato globale, matrici di distanza, grafi delle connessioni, metriche di coesione, mappe di vicinato e visualizzazioni in dashboard. Ogni frame della simulazione può essere interpretato come uno stato del processo emergente. Ogni log può indicare perché certe connessioni sono nate o morte. Ogni metrica può mostrare se lo swarm sta convergendo, collassando, frammentandosi o stabilizzandosi. Questo rende la documentazione molto più forte, perché collega teoria e implementazione.

L'emergenza è anche una proprietà temporale. La forma non appare necessariamente in un singolo istante, ma si costruisce attraverso una sequenza di transizioni. Lo swarm può attraversare fasi: dispersione iniziale, avvicinamento, rilevamento dei vicini, allineamento, aggancio, stabilizzazione, riconfigurazione. Ogni fase ha regole dominanti diverse. Durante l'avvicinamento può dominare il target o la coesione; durante l'aggancio può dominare il magnetismo; durante la stabilizzazione può dominare l'hysteresis; durante la riconfigurazione possono dominare distacco e riallineamento. La forma finale è il risultato di questa storia.

Questa temporalità può essere rappresentata come:

$\text{Forma}(t_0) \rightarrow \text{Forma}(t_1) \rightarrow \text{Forma}(t_2) \rightarrow \dots \rightarrow \text{Forma}(t_k)$

Ogni forma intermedia non è un errore, ma uno stato del processo. Questo punto è importante dal punto di vista teorico: MicroBot non deve essere descritto solo attraverso la forma finale, ma attraverso la dinamica che porta alla forma. Il sistema non è un oggetto statico; è una trasformazione continua. La forma è un evento dinamico mantenuto da relazioni locali.

Dal punto di vista della simulazione, l'emergenza può essere testata partendo da condizioni iniziali diverse. Se lo stesso set di regole produce una struttura simile da configurazioni iniziali differenti, il comportamento è robusto. Se invece piccole variazioni iniziali producono risultati completamente imprevedibili, il sistema potrebbe essere troppo sensibile. Questa analisi è importante perché uno swarm reale dovrà operare in condizioni non perfette. I nodi non partiranno sempre nella posizione ideale, con orientamento perfetto o distanza precisa. Il comportamento emergente deve quindi tollerare variabilità.

Una procedura di test può prevedere più run con posizioni iniziali casuali, misurando tempo di convergenza, errore finale, numero di connessioni corrette, collisioni e distacchi. Se le metriche restano entro intervalli accettabili, il comportamento emergente è stabile. Se variano troppo, bisogna ricalibrare pesi, soglie o regole. Questa metodologia rende il progetto più scientifico e meno dipendente da una singola demo riuscita.

L'emergenza deve essere distinta dall'improvvisazione casuale. Un comportamento casuale può sembrare interessante visivamente, ma non è necessariamente utile. Il comportamento emergente utile ha una struttura riconoscibile, metriche misurabili e regole generative chiare. Nel MicroBot, l'obiettivo non è creare movimento caotico, ma costruire sistemi in cui regole locali producano forme interpretabili. La differenza è fondamentale: l'emergenza è progettata, non lasciata al caso.

Dal punto di vista documentale, questo punto conclude la Sezione 7 perché mostra il motivo per cui tutti i concetti precedenti sono necessari. Le coordinate permettono di localizzare i nodi; la distanza permette di definire vicinati; le interazioni definiscono relazioni; le soglie trasformano vicinanza in connessione; l'hysteresis stabilizza le connessioni; le forze generano movimento; coesione, separazione e allineamento producono ordine locale; il sistema multi-agente distribuisce le regole tra nodi; l'emergenza trasforma tutto questo in comportamento globale. Senza questo punto finale, la Sezione 7 resterebbe una somma di strumenti matematici. Con questo punto, diventa un modello completo del comportamento dello swarm.

In conclusione, l'emergenza del comportamento collettivo è il principio attraverso cui il sistema MicroBot passa da insieme di unità individuali a organismo distribuito. Essa nasce dalla ripetizione di regole locali, dalla combinazione di forze, dalla formazione e rottura di connessioni, dalla memoria introdotta dall'hysteresis, dalla cooperazione tra fisica e software e dalla dinamica del sistema multi-agente. Il comportamento globale non è scritto direttamente in ogni singolo nodo, ma appare come risultato della rete di interazioni. Questo è il cuore matematico e concettuale del progetto MicroBot: costruire una materia programmabile in cui la forma non è imposta rigidamente, ma emerge da unità autonome, relazioni locali e vincoli controllati. Con questa sezione si chiude il modello matematico di base e si prepara il passaggio alla

Sezione 8, dedicata alla comunicazione, dove verrà analizzato il modo in cui i MicroBot possono scambiare informazioni, ricevere comandi e coordinare il comportamento distribuito attraverso una rete digitale.

SEZIONE 8 — Comunicazione

La Sezione 8 introduce il livello comunicativo del sistema MicroBot, cioè l'insieme dei meccanismi attraverso cui i nodi possono ricevere comandi, trasmettere informazioni, sincronizzare il proprio stato e partecipare a un comportamento collettivo coordinato. Dopo aver definito il modello matematico dello swarm, diventa necessario spiegare come l'informazione possa circolare all'interno del sistema. Il comportamento emergente, infatti, può nascere da forze locali e interazioni fisiche, ma per diventare controllabile, osservabile e riconfigurabile richiede anche un canale digitale capace di collegare controller, nodi, simulazione, dashboard e telemetria.

Nel progetto MicroBot, la comunicazione non deve essere interpretata come semplice trasmissione di comandi da un computer a un robot. Essa rappresenta un livello architetturale fondamentale, perché collega la dimensione logica del sistema alla dimensione fisica. Attraverso la comunicazione, il controller può inviare istruzioni ai MicroBot, i nodi possono dichiarare il proprio stato, la dashboard può visualizzare informazioni aggiornate, il sistema può registrare eventi e lo swarm può coordinare azioni distribuite. Senza comunicazione, ogni MicroBot resterebbe un'unità isolata o puramente passiva; con la comunicazione, invece, il nodo diventa parte di una rete operativa.

La comunicazione deve essere progettata in coerenza con l'intera architettura del sistema. Nelle sezioni precedenti è stato chiarito che MicroBot è un sistema multilivello: esiste un'interfaccia utente, un controller centrale, nodi distribuiti, un sistema magnetico, un modello matematico, una simulazione e una futura telemetria. La comunicazione è il tessuto che collega questi livelli. Un comando generato dall'interfaccia deve raggiungere il controller; il controller deve trasformarlo in un messaggio; il messaggio deve arrivare ai nodi; i nodi devono interpretarlo; l'esecuzione deve produrre uno stato; lo stato deve tornare al sistema sotto forma di feedback. Questo ciclo informativo è ciò che rende il sistema controllabile.

In forma generale, il flusso comunicativo può essere rappresentato così:

utente → interfaccia → controller → rete → MicroBot → stato locale → rete → controller → dashboard

Questa catena mostra che la comunicazione è bidirezionale. Non basta inviare comandi verso i MicroBot; è necessario anche ricevere informazioni dai MicroBot. Un sistema che invia comandi senza ricevere feedback resta cieco. Può ordinare un'azione, ma non sapere se essa è stata eseguita, se il nodo è ancora attivo, se la batteria è bassa, se una connessione è stabile o se il safety layer ha bloccato l'operazione. Per questo motivo, la comunicazione deve essere progettata insieme alla telemetria, anche se la telemetria verrà approfondita nella Sezione 13.

Nel progetto MicroBot, la scelta di una comunicazione wireless è coerente con la natura distribuita dello swarm. Se ogni MicroBot dovesse essere collegato fisicamente tramite cavi, il sistema perderebbe gran parte della propria capacità di movimento, aggregazione e riconfigurazione. La comunicazione senza fili permette invece ai nodi di restare indipendenti dal punto di vista meccanico, mantenendo però un collegamento informativo con il controller. Questo è particolarmente importante in uno swarm, dove il numero di unità può crescere e dove i nodi devono potersi disporre nello spazio senza vincoli cablati.

Tuttavia, la comunicazione wireless introduce anche problemi specifici: latenza, perdita di pacchetti, interferenze, limiti di banda, consumo energetico, sincronizzazione e gestione degli indirizzi. Questi aspetti non possono essere ignorati. In un sistema distribuito, un comando ricevuto in ritardo può produrre un comportamento incoerente; un pacchetto perso può lasciare un nodo in uno stato non aggiornato; una comunicazione troppo frequente può consumare energia; un protocollo mal definito può rendere difficile il debug. Per questo motivo, la comunicazione MicroBot deve essere progettata con messaggi semplici, stati chiari, timestamp e modalità robuste di gestione degli errori.

La Sezione 8 segue quindi una progressione precisa. Prima viene analizzata la rete Wi-Fi come infrastruttura di comunicazione di base. Poi viene definito il MicroBot come nodo di rete, cioè come entità identificabile e raggiungibile. Successivamente viene introdotto il protocollo UDP, scelto come possibile soluzione per comunicazioni rapide e leggere. Verrà poi analizzato il concetto di comunicazione a bassa latenza, essenziale per coordinare lo swarm. Infine saranno descritti la struttura dei pacchetti, i campi fondamentali come identificatore, comando e timestamp, e le modalità broadcast e unicast. In questo modo, la comunicazione verrà trattata non solo come tecnologia, ma come architettura informativa del sistema.

8.1 Rete Wi-Fi

La rete Wi-Fi rappresenta il primo livello dell'infrastruttura comunicativa del sistema MicroBot e costituisce il mezzo attraverso cui controller centrale, nodi distribuiti, dashboard e simulazione possono scambiare informazioni senza connessioni fisiche cablate. La scelta del Wi-Fi è coerente con l'utilizzo di microcontrollori come ESP32, già introdotti nella Sezione 5, perché questo tipo di dispositivo integra capacità wireless e consente di costruire prototipi distribuiti con costi contenuti, buona accessibilità e compatibilità con strumenti software comuni. Nel contesto MicroBot, il Wi-Fi non è soltanto una comodità tecnica, ma un elemento necessario per trasformare più unità fisiche in uno swarm controllabile.

La funzione principale della rete Wi-Fi è creare un canale condiviso attraverso cui il controller possa inviare comandi ai MicroBot e ricevere informazioni di stato. In una prima architettura, il controller può essere un computer, un server locale, una dashboard web, un programma Python o un'applicazione dedicata. I MicroBot, invece, possono essere nodi ESP32 connessi alla stessa rete. Questo consente di costruire una topologia semplice: tutti i dispositivi partecipano a una rete comune e possono scambiarsi pacchetti secondo un protocollo definito. Tale configurazione è adatta alle fasi iniziali del progetto perché riduce la complessità hardware e permette di concentrarsi sulla struttura dei messaggi, sulla logica del controller e sul comportamento dei nodi.

Dal punto di vista architetturale, la rete Wi-Fi può essere organizzata in due modalità principali. La prima modalità prevede l'utilizzo di un router o access point esterno, a cui si collegano sia il controller sia i MicroBot. In questo caso, la rete esiste indipendentemente dai nodi e fornisce indirizzamento, associazione e trasporto dei pacchetti. La seconda modalità prevede che uno dei dispositivi, ad esempio il controller o un ESP32 configurato in modalità access point, generi direttamente la rete a cui gli altri nodi si collegano. Questa seconda soluzione può essere utile in dimostrazioni isolate, ambienti di laboratorio o prototipi portabili, perché rende il sistema meno dipendente da una rete esterna.

Nel caso di rete con router esterno, la struttura è più vicina a un ambiente standard. Il computer controller e i MicroBot ricevono indirizzi IP all'interno della stessa sottorete e possono comunicare direttamente. Questa configurazione è comoda durante lo sviluppo, perché permette di usare strumenti comuni di debug, server web locali, dashboard e script Python. Il limite principale è la dipendenza dall'infrastruttura disponibile: se il router cambia configurazione, se la rete è congestionata o se l'accesso non è stabile, il comportamento comunicativo può variare. Per un progetto sperimentale questo è accettabile, ma per un sistema maturo sarà necessario controllare meglio l'ambiente di rete.

Nel caso di rete generata dal sistema stesso, il MicroBot controller può creare un ambiente più controllato. Un ESP32 può funzionare come access point oppure un computer può generare un hotspot dedicato. I nodi si collegano a questa rete specifica e comunicano con il controller senza passare da infrastrutture esterne. Questa soluzione è più coerente con l'idea di dimostratore autonomo, perché il sistema può essere acceso e utilizzato in luoghi diversi senza richiedere una rete preesistente. Tuttavia, anche questa modalità presenta limiti: il numero di dispositivi gestibili può essere inferiore, la stabilità può dipendere dal dispositivo che genera la rete e la configurazione deve essere documentata con precisione.

La rete Wi-Fi deve essere considerata come livello condiviso e potenzialmente rumoroso. Tutti i nodi utilizzano lo stesso mezzo radio, quindi i pacchetti possono subire ritardi, collisioni, interferenze o perdita. Questo è particolarmente rilevante nello swarm perché il numero di nodi può aumentare. Con pochi MicroBot, la comunicazione può sembrare semplice; con molti nodi, il traffico cresce e diventa necessario progettare messaggi compatti, frequenze di invio ragionevoli e strategie di coordinamento. La scalabilità comunicativa deve quindi essere considerata fin dall'inizio, anche se i primi prototipi usano pochi nodi.

Un principio fondamentale è che il Wi-Fi deve trasportare informazioni essenziali, non dati inutilmente pesanti. Un MicroBot non deve ricevere continuamente messaggi lunghi e complessi se bastano comandi brevi e stati sintetici. Allo stesso modo, non deve inviare telemetria ad alta frequenza se il contenuto non è utile al controllo o alla diagnosi. Ogni pacchetto trasmesso ha un costo: occupa banda, consuma energia, introduce potenziale ritardo e aumenta il carico di elaborazione. Per questo motivo, la comunicazione deve essere progettata con un linguaggio compatto e chiaro.

Nel sistema MicroBot, la rete Wi-Fi può supportare diversi tipi di messaggio. Il primo tipo è il comando diretto, cioè un messaggio inviato dal controller a uno o più nodi per modificare stato, LED, bobina, comportamento o modalità operativa. Il secondo tipo è il messaggio di stato, inviato dal MicroBot al controller per comunicare informazioni come batteria, temperatura stimata, stato magnetico, stato di connessione, errore o conferma di ricezione. Il terzo tipo è il messaggio di sincronizzazione, utilizzato per coordinare azioni nel tempo. Il quarto tipo è il messaggio diagnostico, utile durante test, debug e logging. Queste categorie possono condividere la stessa rete, ma devono essere distinte nella struttura del protocollo.

Il Wi-Fi deve essere collegato al concetto di identificazione dei nodi. In una rete, ogni MicroBot deve essere riconoscibile. Questo può avvenire tramite indirizzo IP, indirizzo MAC, identificatore logico assegnato dal firmware o combinazione di questi elementi. Dal punto di vista del sistema, è utile separare l'identità fisica di rete dall'identità logica del MicroBot. L'indirizzo IP può cambiare a seconda della rete; l'identificatore logico del nodo dovrebbe invece restare stabile, ad esempio node_01, node_02, oppure un ID numerico. Questa distinzione rende il sistema più robusto e più facile da documentare.

Una possibile rappresentazione logica del nodo nella rete è:

node_id = identificatore stabile del MicroBot
ip_address = indirizzo di rete corrente
status = stato operativo
last_seen = ultimo timestamp di comunicazione ricevuta

Questa struttura consente al controller di sapere quali nodi sono online, quali non rispondono, quali hanno cambiato indirizzo e quali sono disponibili per ricevere comandi. Il campo last_seen è particolarmente importante perché permette di rilevare nodi disconnessi o bloccati. Se un nodo non invia heartbeat o messaggi di stato per un certo intervallo, il controller può marcarlo come offline o in stato incerto.

La rete Wi-Fi deve quindi supportare un meccanismo di presenza. In uno swarm, il controller deve sapere quali MicroBot sono attivi. Un modo semplice per ottenere questo è usare messaggi periodici di heartbeat. Ogni nodo invia a intervalli regolari un breve pacchetto che segnala: "sono vivo, sono il nodo X, questo è il mio stato essenziale". Questo meccanismo è molto utile perché consente alla dashboard di visualizzare i nodi attivi e permette al sistema di rilevare guasti, reset o disconnessioni.

Un heartbeat minimale può contenere:

identificatore del nodo, stato operativo, livello batteria stimato, eventuale errore, timestamp.

Anche se la struttura completa dei pacchetti verrà descritta più avanti, è importante introdurre già qui il ruolo della rete Wi-Fi come supporto al monitoraggio continuo. Senza heartbeat, il controller potrebbe non sapere se un nodo non sta rispondendo perché non ha ricevuto comandi, perché è spento, perché è bloccato o perché è fuori rete. Con heartbeat, il sistema acquisisce una forma minima di osservabilità.

Il Wi-Fi introduce anche il problema della latenza. La latenza è il tempo che intercorre tra l'invio di un messaggio e la sua ricezione. Nel sistema MicroBot, la latenza è importante perché i nodi potrebbero dover eseguire azioni coordinate, ad esempio attivare LED, simulare pattern, avviare una sequenza magnetica o partecipare a una formazione. Se i messaggi arrivano con ritardi diversi ai vari nodi, le azioni possono risultare non sincronizzate. Per questo motivo, il protocollo deve includere timestamp o meccanismi di sincronizzazione, che saranno approfonditi nella Sezione 9.

La rete Wi-Fi non garantisce automaticamente tempi perfettamente deterministici. Questo significa che il sistema non deve basarsi sull'idea ingenua che tutti i nodi ricevano e processino i comandi nello stesso istante. Una soluzione più robusta consiste nell'inviare comandi contenenti un tempo di esecuzione futuro. Invece di dire "esegui ora", il controller può dire "esegui al timestamp T". Se i nodi possiedono un riferimento temporale sufficientemente allineato, possono ricevere il comando con piccoli ritardi ma eseguirlo in modo più sincronizzato. Questo principio sarà fondamentale per coordinare azioni distribuite.

Dal punto di vista energetico, il Wi-Fi è un sottosistema significativo. La comunicazione wireless consuma energia, specialmente se il modulo resta sempre attivo, trasmette spesso o mantiene connessioni continue. In un MicroBot alimentato a batteria, la frequenza dei messaggi deve essere bilanciata. Una telemetria troppo frequente può ridurre l'autonomia; una telemetria troppo rara può rendere il sistema poco osservabile. Anche qui serve equilibrio. Nei prototipi iniziali, il consumo può essere meno critico perché l'obiettivo è

validare il comportamento; nelle versioni successive, sarà necessario ottimizzare intervalli, payload e modalità operative.

La rete Wi-Fi deve essere progettata anche in relazione alla sicurezza operativa. Un comando errato, duplicato o ricevuto nel momento sbagliato potrebbe produrre comportamenti indesiderati. Per questo motivo, ogni MicroBot dovrebbe validare i messaggi ricevuti. La validazione può includere controllo del formato, verifica del destinatario, verifica del tipo di comando, controllo del timestamp, controllo dello stato locale e rifiuto di comandi non sicuri. La rete trasporta messaggi, ma il nodo deve decidere se eseguirli. Questa idea è coerente con il modello multi-agente: ogni nodo possiede autonomia minima e safety locale.

Un esempio di logica è:

```
ricevi messaggio;  
verifica che il messaggio sia valido;  
verifica che il destinatario sia corretto;  
verifica che il comando sia compatibile con lo stato locale;  
esegui solo se safety_ok;  
invia conferma o errore.
```

Questa sequenza impedisce che la comunicazione diventi un canale incontrollato. Il controller può inviare istruzioni, ma il MicroBot deve mantenere il diritto di rifiutare ciò che non è sicuro. Questo è particolarmente importante per comandi che attivano bobine, carichi di potenza o comportamenti magnetici.

Nel contesto dei primi prototipi, la rete Wi-Fi può essere usata anche per controllare LED e stati simulati. Anche se la parte magnetica non è ancora completamente fisica, il protocollo può già essere progettato come se dovesse in futuro controllare bobine e attuatori. Questo è un vantaggio metodologico: si può validare la comunicazione su carichi semplici, come LED RGB o stati software, prima di collegarla a sottosistemi più rischiosi. In questo modo, la rete e il protocollo maturano prima dell'attuazione fisica completa.

La rete Wi-Fi deve essere integrata con la dashboard. Una dashboard MicroBot può visualizzare nodi online, stato dei messaggi, latenza stimata, numero di pacchetti ricevuti, errori, heartbeat, comandi inviati e telemetria. Questo rende la comunicazione osservabile. Senza dashboard, il debug della rete può essere difficile; con una dashboard, è possibile vedere quali nodi sono presenti, quali rispondono lentamente, quali perdono pacchetti e quali sono in stato anomalo. La comunicazione deve quindi produrre dati utili alla visualizzazione, non restare nascosta.

Dal punto di vista dei file e dei log, la comunicazione Wi-Fi può generare eventi. Ogni connessione di un nodo, ogni perdita di heartbeat, ogni comando ricevuto, ogni risposta, ogni timeout e ogni errore di parsing possono essere registrati. Questo sarebbe coerente con l'ecosistema MicroBot già orientato a logging, runtime state e documentazione. Un log comunicativo può diventare fondamentale per capire perché uno swarm ha fallito una sequenza: il problema era fisico, matematico, magnetico o semplicemente un messaggio perso?

Un evento di rete può contenere:

timestamp, node_id, tipo evento, indirizzo, messaggio, esito, latenza stimata, eventuale errore.

Questi dati permettono di distinguere problemi diversi. Se un nodo non esegue un comando ma il pacchetto non è mai arrivato, il problema è comunicativo. Se il pacchetto è arrivato ma il nodo lo ha rifiutato per safety, il problema è operativo. Se il nodo lo ha eseguito ma la simulazione non mostra effetto, il problema è nel modello o nell'attuazione. La comunicazione osservabile aiuta quindi a separare cause e responsabilità.

La rete Wi-Fi si collega anche alla scalabilità. Con pochi nodi, un controller può inviare messaggi diretti a ciascun MicroBot senza particolari problemi. Con molti nodi, questa strategia può diventare inefficiente. Per questo motivo, il sistema deve prevedere modalità broadcast e unicast. Il broadcast consente di inviare un messaggio a tutti i nodi, utile per comandi globali come "entra in modalità idle", "sincronizza timestamp", "avvia pattern" o "spegni bobine". L'unicast consente di inviare un messaggio a un nodo specifico, utile per comandi individuali, diagnosi o assegnazione di ruolo. Queste modalità saranno approfondite nei punti finali della Sezione 8.

La scelta tra broadcast e unicast dipende dal tipo di comando. Un comando globale dovrebbe essere broadcast per ridurre traffico e garantire coerenza. Un comando specifico dovrebbe essere unicast per evitare ambiguità. Esiste anche una possibile modalità di gruppo, in cui il comando viene destinato a un sottoinsieme di nodi, ad esempio un cluster o una formazione. Anche se questa modalità non è esplicitamente indicata nell'indice, può essere considerata estensione naturale delle due modalità principali, da introdurre eventualmente nelle versioni successive.

Dal punto di vista della topologia comunicativa, il Wi-Fi permette inizialmente una struttura controller-centrica. Tutti i nodi comunicano con il controller, e il controller coordina il sistema. Questa scelta è semplice e coerente con i prototipi. In futuro, si potrebbe introdurre comunicazione diretta tra nodi, ma questa aumenterebbe la complessità del protocollo. Per ora, la struttura più ordinata è:

controller centrale ↔ MicroBot nodes

Questa architettura mantiene chiaro il flusso dei comandi e semplifica logging e debug. Anche se il comportamento fisico dello swarm è distribuito, la comunicazione può iniziare in modo centralizzato. Questo non contraddice il paradigma multi-agente, perché la distribuzione può esistere a livello di regole locali anche se il controller fornisce coordinamento generale.

Un aspetto importante è la gestione degli errori di rete. Un nodo può disconnettersi, perdere pacchetti, ricevere comandi duplicati o ricevere messaggi non validi. Il sistema deve prevedere comportamenti di fallback. Se un nodo perde comunicazione per troppo tempo, dovrebbe entrare in modalità sicura o mantenere uno stato stabile. Se riceve un comando duplicato, dovrebbe riconoscerlo tramite identificatore di messaggio o timestamp. Se riceve un comando vecchio, dovrebbe ignorarlo. Se riceve un comando non valido, dovrebbe inviare errore e non eseguirlo. Queste regole rendono la rete robusta.

La gestione dei comandi duplicati è particolarmente importante con protocolli leggeri come UDP, che verrà discusso nei punti successivi. Anche se UDP è veloce, non garantisce consegna, ordine o assenza di duplicati a livello applicativo. Per questo motivo, la logica MicroBot deve aggiungere un minimo di controllo: message_id, timestamp, stato di conferma o sequenza. Non serve creare subito un protocollo pesante, ma è necessario evitare che un comando vecchio o ripetuto produca effetti indesiderati.

Dal punto di vista del firmware, il modulo Wi-Fi dovrebbe essere separato dalla logica applicativa. Il codice che gestisce connessione, ricezione pacchetti e invio messaggi non dovrebbe essere mescolato direttamente alla logica magnetica o comportamentale. Una struttura modulare potrebbe distinguere:

network manager;
packet parser;
command dispatcher;
state manager;
safety validator;
telemetry sender.

Questa separazione rispecchia il principio già definito nella Sezione 3: ogni componente deve avere responsabilità chiara. Il network manager si occupa della rete; il parser interpreta i messaggi; il dispatcher assegna i comandi; il safety validator decide se sono eseguibili; il telemetry sender invia feedback. Questo rende il firmware più mantenibile e più facile da espandere.

La rete Wi-Fi deve essere coerente anche con la simulazione. Nei prototipi software, i nodi possono essere simulati come processi, oggetti o entità virtuali che ricevono messaggi dal controller. Anche se non esiste hardware reale, lo stesso modello comunicativo può essere usato per testare comandi, pacchetti e stati. Questo è molto utile perché permette di sviluppare il protocollo prima dell'hardware completo. Una simulazione ben progettata può usare messaggi simili a quelli dei MicroBot fisici, rendendo più facile la transizione dalla demo virtuale al nodo reale.

Questa coerenza tra simulazione e hardware è uno dei punti più importanti del progetto. Se il simulatore usa una struttura di comando e l'hardware ne usa un'altra, il sistema si frammenta. Se invece entrambi condividono lo stesso linguaggio di messaggi, allora la simulazione diventa una vera piattaforma di test. Il controller può inviare lo stesso comando a un nodo simulato o a un ESP32 reale; la dashboard può visualizzare stati simili; i log possono avere formato coerente. Questo trasforma la comunicazione in interfaccia comune dell'ecosistema MicroBot.

Dal punto di vista documentale, la rete Wi-Fi deve essere descritta non come dettaglio tecnico secondario, ma come primo strato del communication layer. Essa fornisce il mezzo; il protocollo fornisce la grammatica; i pacchetti forniscono la struttura; il controller fornisce l'intenzione; i nodi forniscono l'esecuzione; la telemetria fornisce il ritorno informativo. Solo tutti questi elementi insieme producono una comunicazione utile.

In conclusione, la rete Wi-Fi costituisce l'infrastruttura comunicativa di base del sistema MicroBot. Essa consente ai nodi di operare senza vincoli cablati, supporta il collegamento tra controller e MicroBot, permette comandi, heartbeat, telemetria e sincronizzazione, e prepara il sistema a modalità broadcast e unicast. Allo stesso tempo, introduce problemi reali come latenza, perdita di pacchetti, consumo energetico, gestione degli indirizzi, sicurezza dei comandi e scalabilità. Per questo motivo, il Wi-Fi deve essere integrato in un'architettura di comunicazione robusta, modulare e osservabile. Il punto successivo analizzerà il MicroBot come nodo di rete, cioè il modo in cui ogni unità viene identificata, registrata, monitorata e trattata come entità comunicativa all'interno dello swarm.

8.2 MicroBot come nodi di rete

Il MicroBot come nodo di rete rappresenta il passaggio dal concetto fisico di unità distribuita al concetto informatico di dispositivo identificabile, raggiungibile e coordinabile all'interno di un sistema comunicativo. Dopo aver definito la rete Wi-Fi come infrastruttura di base, è necessario descrivere come ogni singolo MicroBot venga visto dal controller, dalla dashboard, dal simulatore e dagli altri livelli del sistema. Un MicroBot non è soltanto un piccolo modulo elettronico dotato di microcontrollore, alimentazione, magneti, LED e possibile coil; nel livello comunicativo esso diventa un nodo della rete, cioè un'entità capace di ricevere messaggi, interpretare comandi, mantenere uno stato locale e trasmettere informazioni di ritorno.

Nel progetto MicroBot, questa distinzione è fondamentale perché permette di collegare il piano fisico con il piano logico. Fisicamente, un MicroBot occupa una posizione nello spazio, possiede una geometria, può agganciarsi ad altre unità e può mostrare feedback tramite LED. Logicamente, invece, deve essere identificato come elemento della rete: deve avere un nome, un identificatore, un indirizzo, uno stato operativo, una cronologia di comunicazione e una relazione con il controller. Il sistema non può gestire uno swarm se non sa distinguere un nodo dall'altro. La nozione di "MicroBot come nodo di rete" nasce proprio da questa esigenza: ogni unità deve essere riconosciuta come partecipante individuale del sistema distribuito.

Il primo elemento da definire è l'identità del nodo. Ogni MicroBot deve possedere un identificatore stabile, indipendente per quanto possibile dall'indirizzo di rete corrente. L'indirizzo IP può cambiare a seconda della rete, del router, della modalità access point o della configurazione DHCP; l'identificatore logico del nodo, invece, dovrebbe rimanere costante. Un nodo può essere chiamato `node_01`, `node_02`, `microbot_001`, oppure avere un codice numerico assegnato in fase di firmware. Questo identificatore è ciò che permette al controller di associare comandi, stati, log e telemetria alla stessa unità nel tempo.

Una rappresentazione minima del nodo di rete può essere:

```
node_id = identificatore logico stabile
ip_address = indirizzo IP corrente
mac_address = identificatore hardware della scheda di rete
status = stato operativo del MicroBot
last_seen = ultimo istante in cui il controller ha ricevuto un messaggio dal nodo
```

Questa struttura è molto semplice, ma già sufficiente per trasformare una serie di dispositivi Wi-Fi in uno swarm gestibile. Il controller può mantenere una tabella dei nodi attivi, aggiornarla a ogni messaggio ricevuto, segnalare nodi offline, inviare comandi mirati e visualizzare lo stato nella dashboard. Senza questa tabella, il sistema sarebbe costretto a trattare i messaggi come eventi isolati, senza una vera memoria dei dispositivi.

Il concetto di `last_seen` è particolarmente importante. In una rete distribuita, un nodo può smettere di comunicare per molte ragioni: batteria scarica, reset del microcontrollore, perdita di connessione Wi-Fi, errore firmware, distanza eccessiva dall'access point, interferenza o spegnimento volontario. Il controller non può sapere immediatamente quale sia la causa, ma può rilevare l'assenza di messaggi. Se un nodo non invia heartbeat o telemetria per un certo intervallo, il sistema può marcarlo come stale, offline o unknown. Questo permette alla dashboard di mostrare chiaramente quali MicroBot sono attivi e quali non sono più affidabili.

Un possibile modello degli stati comunicativi del nodo può includere:

online, quando il nodo comunica regolarmente;

stale, quando il nodo non comunica da un intervallo superiore alla soglia attesa ma non è ancora considerato perso;

offline, quando il nodo non comunica da troppo tempo;

error, quando il nodo comunica ma segnala un errore;

safe, quando il nodo è raggiungibile ma limita le proprie azioni per motivi di sicurezza.

Questi stati comunicativi devono essere distinti dagli stati fisici o comportamentali. Un MicroBot può essere online ma in modalità idle; può essere online ma in safety; può essere offline ma fisicamente ancora agganciato ad altri nodi; può essere in errore firmware ma ancora visibile sulla rete. Questa distinzione è importante perché impedisce al sistema di confondere presenza comunicativa e capacità operativa. Essere raggiungibile non significa necessariamente essere disponibile a eseguire qualunque comando.

Il secondo elemento fondamentale è la registrazione del nodo. Quando un MicroBot si accende e si collega alla rete Wi-Fi, deve rendere nota la propria presenza al controller. Questo processo può essere chiamato registrazione, discovery o handshake iniziale. In una forma semplice, il nodo invia un messaggio di annuncio contenente il proprio node_id, il tipo di firmware, la versione, lo stato iniziale e le capacità disponibili. Il controller riceve il messaggio, crea o aggiorna la scheda del nodo nella propria tabella interna e lo considera disponibile.

Un messaggio iniziale di registrazione potrebbe contenere informazioni come:

node_id;

firmware_version;

hardware_profile;

capabilities;

battery_level;

status;

timestamp.

Il campo capabilities è particolarmente utile perché non tutti i MicroBot devono necessariamente avere le stesse funzioni in tutte le versioni. Un nodo iniziale potrebbe supportare solo LED e heartbeat; un nodo successivo potrebbe supportare PWM, coil, telemetria energetica, sensori o diagnostica avanzata. Dichiarare le capacità permette al controller di sapere quali comandi sono compatibili con ogni nodo. Questo evita di inviare, per esempio, un comando coil_set a un nodo che non possiede coil o che non ha quel modulo abilitato.

Nel progetto MicroBot, dove l'evoluzione per versioni è molto importante, il concetto di capability è essenziale. Le prime versioni fisiche possono essere basate su LED, ESP32 e stati simulati; le versioni successive possono introdurre driver MOSFET, bobine, sensori e sistemi di safety. Se il protocollo prevede già un campo dedicato alle capacità, il sistema può crescere senza rompere la compatibilità. Il controller può trattare ogni nodo in base al suo profilo reale, invece di assumere che tutti abbiano le stesse funzioni.

Una possibile classificazione delle capacità può essere:

led_rgb;

heartbeat;

telemetry_basic;

telemetry_energy;

coil_pwm;

coil_directional;

temperature_estimation;

safety_lock;

magnetic_state;

pattern_execution;

diagnostic_mode.

Questa lista non deve essere considerata definitiva, ma mostra il principio. Ogni MicroBot dichiara quali moduli supporta. Il sistema diventa più flessibile e può gestire nodi misti, prototipi diversi e versioni progressive. In una fase di ricerca e sviluppo, questa è una scelta molto intelligente, perché permette di confrontare unità con caratteristiche differenti senza dover riscrivere tutta l'architettura.

Il terzo elemento è l'indirizzamento. Il controller deve poter inviare un comando a tutti i nodi, a un gruppo di nodi o a un nodo specifico. Per farlo, deve sapere come raggiungerli. L'indirizzo IP è necessario a livello di rete, ma non dovrebbe essere l'unico riferimento logico. Il comando dovrebbe indicare un destinatario tramite `node_id`, gruppo o modalità broadcast, mentre il livello di rete si occupa di tradurre quella destinazione in indirizzo effettivo. Questa separazione tra identità logica e indirizzo fisico rende il sistema più robusto.

Un comando potrebbe quindi avere una struttura concettuale di questo tipo:

target = node_03

oppure:

target = all

oppure:

target = group_A

Il controller interpreta target e decide come inviare il messaggio. Se il target è un nodo specifico, usa l'indirizzo IP noto di quel nodo. Se il target è all, usa broadcast o invia a tutti i nodi registrati. Se il target è un gruppo, invia solo ai nodi appartenenti a quel gruppo. Anche se la modalità di gruppo può essere sviluppata più avanti, prevederla concettualmente rende il sistema più estendibile.

Il MicroBot come nodo di rete deve anche avere una porta di ascolto, cioè un endpoint logico su cui ricevere messaggi. Nel caso di UDP, ogni nodo può ascoltare su una porta definita, ad esempio una porta comune per tutti i MicroBot. Il controller invia pacchetti a quell'indirizzo e a quella porta. In alternativa, ogni nodo può avere una porta diversa, ma questa scelta complica la gestione. Per le prime versioni, una porta comune e un protocollo con campo target è più semplice e coerente.

Il quarto elemento è il ciclo comunicativo del nodo. Un MicroBot non deve restare passivo in attesa di comandi; deve svolgere periodicamente alcune operazioni di rete. Può inviare heartbeat, ricevere pacchetti,

processare comandi, aggiornare stato, inviare telemetria e segnalare errori. Questo ciclo deve essere integrato nel firmware senza bloccare le altre funzioni del nodo. In un sistema embedded, è importante evitare logiche bloccanti, perché un'attesa di rete non deve impedire al nodo di gestire safety, LED, bobine o watchdog.

Un ciclo concettuale del nodo può essere:

controlla connessione Wi-Fi;

ricevi eventuali pacchetti;

valida pacchetti ricevuti;

aggiorna comandi pendenti;

esegui azioni consentite;

aggiorna stato locale;

invia heartbeat o telemetria se necessario;

controlla safety;

ripeti.

Questa struttura deve essere non bloccante. Il nodo deve poter eseguire ogni funzione in piccoli passi, mantenendo reattività. Se il firmware si bloccasse in attesa di un pacchetto, potrebbe non aggiornare il safety layer o non spegnere una bobina in tempo. Per questo motivo, il modulo comunicativo deve essere progettato come parte del loop principale, non come processo che domina tutto il nodo.

Il quinto elemento è la validazione dei messaggi ricevuti. Un MicroBot come nodo di rete non deve eseguire automaticamente qualunque pacchetto arrivi. Deve verificare che il messaggio sia formalmente valido, che sia destinato a lui o al gruppo corretto, che il comando sia supportato, che il timestamp sia accettabile, che il messaggio non sia duplicato e che lo stato locale consenta l'esecuzione. Questa validazione è fondamentale per la sicurezza e la robustezza.

Una sequenza rigorosa può essere:

verifica formato;

verifica versione protocollo;

verifica destinatario;

verifica command_type;

verifica message_id o timestamp;

verifica compatibilità con capabilities;

verifica safety locale;

esegui o rifiuta;

invia ack o errore.

Questa struttura rende il nodo molto più affidabile. Se il controller invia un comando non compatibile, il nodo può rispondere con `unsupported_command`. Se il comando è valido ma non sicuro, può rispondere con `rejected_safety`. Se il messaggio è vecchio, può rispondere con `stale_command` o semplicemente ignorarlo. Se il formato è errato, può generare `parse_error`. In questo modo, il sistema diventa osservabile e diagnosticabile.

Il sesto elemento è la risposta del nodo. Una comunicazione robusta non deve essere solo comando in ingresso, ma anche conferma o feedback. Quando un MicroBot riceve un comando, può inviare un `acknowledgement`, cioè una conferma di ricezione. Tuttavia, bisogna distinguere tra ricezione ed esecuzione. Un nodo può ricevere un comando ma non eseguirlo perché lo ritiene non sicuro. Per questo motivo, il feedback dovrebbe indicare almeno tre livelli: ricevuto, accettato, eseguito. In alcuni casi, può bastare un singolo messaggio di risposta; in altri, soprattutto per azioni più lunghe, possono servire stati progressivi.

Una risposta può includere:

`message_id` originale;

`node_id`;

`result`;

`reason`;

`current_state`;

`timestamp`.

Il campo `result` può essere `accepted`, `executed`, `rejected`, `error` o `pending`. Il campo `reason` spiega il motivo, ad esempio `safety_lock`, `low_battery`, `unsupported_command`, `invalid_target` o `busy`. Questo tipo di feedback è molto utile durante lo sviluppo, perché permette di capire perché un comando non produce l'effetto previsto.

Il settimo elemento è la gestione dello stato locale. Ogni MicroBot come nodo di rete deve mantenere una rappresentazione interna del proprio stato, aggiornata nel tempo. Questo stato non deve essere solo fisico, ma anche comunicativo. Il nodo deve sapere se è connesso alla rete, quando ha ricevuto l'ultimo comando, quale comando sta eseguendo, se ha comandi pendenti, se ha inviato telemetria di recente, se ha errori di comunicazione e se è sincronizzato con il controller. Questo stato locale è la base per decisioni robuste.

Una rappresentazione concettuale può essere:

`network_state` = `connected` / `disconnected` / `reconnecting`;

`last_command_id`;

`last_command_time`;

`last_heartbeat_sent`;

`packet_rx_count`;

`packet_tx_count`;

`packet_error_count`;

`controller_seen`;

sync_status.

Questi dati possono essere usati per diagnostica e telemetria. Un aumento di packet_error_count può indicare problemi di parsing o rete. Un last_command_time troppo vecchio può indicare inattività. Un sync_status non valido può impedire comandi temporizzati. Il MicroBot come nodo di rete deve quindi monitorare anche la propria salute comunicativa.

L'ottavo elemento è il rapporto tra nodo reale e nodo simulato. Nel progetto MicroBot, è molto utile mantenere la stessa astrazione per entrambi. Un nodo simulato può avere node_id, stato, posizione, telemetria e capacità, proprio come un nodo fisico. Questo permette al controller e alla dashboard di trattare nodi reali e virtuali in modo simile. La differenza è che il nodo simulato esegue le azioni aggiornando variabili software, mentre il nodo fisico esegue azioni su LED, bobine, sensori o hardware. Questa astrazione comune è fondamentale per costruire un ecosistema coerente.

Si può quindi definire un concetto generale di MicroBot node, con due implementazioni:

simulated_node;

physical_node.

Entrambi rispondono agli stessi comandi di alto livello, ma li eseguono in modi diversi. Un comando set_led può modificare un colore sullo schermo nel simulatore e un LED reale nell'ESP32. Un comando set_magnetic_state può modificare una forza virtuale nella simulazione e un duty cycle PWM nel nodo fisico, se supportato. Questo permette di sviluppare e testare il protocollo anche prima che tutte le funzioni hardware siano disponibili.

Il nono elemento è la nozione di nodo come partecipante a uno swarm, non come dispositivo isolato. Un MicroBot di rete deve sapere almeno quale sia il proprio ruolo nel sistema globale o ricevere dal controller parametri che lo collocano all'interno di un comportamento collettivo. Per esempio, un nodo può ricevere un ruolo leader, follower, boundary, connector, idle o diagnostic. Può ricevere un gruppo di appartenenza. Può ricevere una posizione target nella simulazione. Può ricevere una fase del pattern. Queste informazioni trasformano il nodo da dispositivo generico a componente di una forma collettiva.

Un comando di assegnazione ruolo potrebbe contenere:

node_id;

role;

group_id;

pattern_id;

target_position;

behavior_mode;

timestamp.

Non tutti questi campi devono essere usati nelle prime versioni, ma la struttura concettuale è importante. Il nodo deve poter partecipare a pattern e comportamenti complessi non solo come ricevitore di impulsi, ma come agente con ruolo. Questo collega la comunicazione alla Sezione 7 sul sistema multi-agente.

Il decimo elemento riguarda la gestione dei gruppi. In uno swarm reale o simulato, può essere utile organizzare i MicroBot in sottoinsiemi. Un gruppo può rappresentare un cluster, una sezione della forma, una categoria di nodi o una fase di test. La rete deve poter indirizzare comandi a gruppi. Anche se l'indice esplicito parla soprattutto di broadcast e unicast, la logica di gruppo è una naturale estensione e può essere

documentata come possibilità futura. Essa permette di coordinare sottostrutture senza inviare comandi individuali a ogni nodo.

Un nodo può quindi avere:

group_id = none / group_A / group_B / cluster_01

Il controller può inviare comandi al gruppo, ad esempio “group_A entra in modalità allineamento” oppure “cluster_01 stabilizza connessioni”. Questo diventa particolarmente importante quando lo swarm cresce, perché il controllo individuale diventa troppo dettagliato e il broadcast globale troppo generico.

L'undicesimo elemento è il heartbeat. Come già introdotto nella sezione sulla rete Wi-Fi, ogni nodo dovrebbe inviare periodicamente un messaggio breve per segnalare la propria presenza. Dal punto di vista del nodo di rete, il heartbeat è la sua firma vitale. Non serve solo a dire “sono acceso”, ma può includere informazioni sintetiche sullo stato. Per evitare traffico eccessivo, il heartbeat deve essere compatto. Una telemetria completa può essere inviata a frequenza più bassa o su richiesta; il heartbeat deve invece essere leggero e regolare.

Un heartbeat può contenere:

node_id;

status;

battery_estimate;

safety_state;

current_mode;

sequence_number;

timestamp.

Il sequence_number aiuta a rilevare pacchetti persi o fuori ordine. Il timestamp aiuta a stimare ritardi e sincronizzazione. Il current_mode indica se il nodo è idle, active, diagnostic, pattern, safety o error. Queste informazioni permettono al controller di mantenere una vista aggiornata dello swarm senza interrogare continuamente ogni nodo.

Il dodicesimo elemento è la gestione del riavvio. Un MicroBot può resettarsi o spegnersi e riaccendersi. Quando torna online, il controller deve capire se si tratta dello stesso nodo già registrato o di un nuovo dispositivo. L'identificatore logico aiuta a riconoscerlo. Tuttavia, lo stato precedente potrebbe non essere più valido. Il nodo potrebbe aver perso connessioni, comandi pendenti, sincronizzazione o stato magnetico. Per questo motivo, dopo un riavvio, il nodo dovrebbe annunciare boot_id o reset_counter. Questo permette al controller di sapere che il nodo è tornato ma non deve assumere continuità totale con lo stato precedente.

Una registrazione dopo reboot può includere:

node_id;

boot_count;

firmware_version;

initial_state;

capabilities;

timestamp.

Se il controller vede lo stesso node_id con boot_count aumentato, può aggiornare la sua tabella e marcare lo stato come reinizializzato. Questo è utile per evitare errori: un comando precedente non deve essere considerato ancora valido dopo un reboot del nodo. Il sistema deve poter distinguere continuità e nuova sessione.

Il tredicesimo elemento riguarda la sicurezza dei comandi. Anche in un ambiente di prototipo, è utile progettare una forma minima di protezione logica. Non necessariamente sicurezza crittografica avanzata nelle prime versioni, ma almeno controllo di formato, validità, destinatario e stato. In futuro, se il sistema dovesse essere usato in ambienti reali o dimostrativi estesi, potrebbe essere necessario introdurre autenticazione dei comandi o reti dedicate per evitare interferenze. Per ora, la cosa importante è che il nodo non esegua messaggi casuali o malformati.

Il quattordicesimo elemento è la coerenza tra nodo di rete e LED RGB di feedback. Un MicroBot può mostrare localmente il proprio stato comunicativo tramite LED. Per esempio, blu lampeggiante può indicare connessione in corso, verde può indicare online, giallo può indicare rete instabile, rosso può indicare errore o offline logico, viola può indicare modalità diagnostica. Questo collega il livello comunicativo alla Sezione 5.12. Il nodo di rete non deve essere osservabile solo dalla dashboard, ma anche fisicamente tramite feedback locale.

Una codifica possibile può essere:

blu pulsante = connessione Wi-Fi in corso;

verde tenue = nodo online e idle;

cyan breve = pacchetto ricevuto;

giallo lampeggiante = heartbeat perso o riconnessione;

rosso = errore comunicativo;

viola = diagnostica.

Questi colori devono essere definiti in modo coerente con l'intero sistema di feedback. L'importante non è scegliere esattamente questi valori, ma stabilire una convenzione stabile. In uno swarm fisico, vedere molti nodi con colori diversi permette di capire subito lo stato della rete.

Il quindicesimo elemento è il rapporto tra nodo e dashboard. La dashboard deve rappresentare ogni MicroBot come entità individuale. Per ogni nodo, può mostrare stato online/offline, ID, IP, ultima comunicazione, batteria, modalità, ruolo, connessioni attive, eventuali errori e comandi recenti. Questa rappresentazione rende lo swarm leggibile. Non basta vedere punti che si muovono: bisogna vedere nodi identificati, con stati e storie. Il nodo di rete è quindi l'unità base della dashboard.

Una scheda nodo nella dashboard potrebbe contenere:

Node ID;

Network Status;

IP Address;

Role;

Current Mode;

Battery;

Safety;

Last Seen;

Packet Count;

Active Connections;

Last Command.

Questa struttura permette debugging rapido. Se un nodo non risponde, si vede last_seen. Se rifiuta comandi, si vede safety. Se non esegue pattern, si vede role o mode. Se ha problemi di rete, si vede packet_count o error_count. La dashboard diventa il volto operativo del sistema comunicativo.

Dal punto di vista del firmware, il MicroBot come nodo di rete deve essere modulare. Una possibile architettura interna può separare:

Wi-Fi connection manager;

node identity manager;

message receiver;

message parser;

command validator;

command executor;

telemetry manager;

heartbeat manager;

error reporter.

Questa separazione è importante perché evita codice caotico. Il Wi-Fi manager si occupa di connessione e riconnessione; l'identity manager conserva node_id e capabilities; il receiver riceve pacchetti; il parser interpreta il formato; il validator controlla sicurezza e compatibilità; l'executor applica il comando; il telemetry manager prepara feedback; l'heartbeat manager invia presenza; l'error reporter registra anomalie. Questa struttura è adatta a crescere nel tempo.

Il MicroBot come nodo di rete deve anche avere comportamento di fallback. Se perde la connessione, deve decidere cosa fare. Una strategia prudente è entrare in una modalità safe o idle dopo un timeout. Se stava eseguendo una bobina attiva, deve spegnerla entro un tempo massimo. Se stava mostrando LED, può indicare perdita di rete. Se era in un pattern, può mantenere stato passivo o fermarsi. Il punto chiave è che la perdita di rete non deve lasciare il nodo in uno stato pericoloso.

Una regola generale può essere:

se connection_lost per più di T_timeout, allora disattiva attuazioni attive, mantieni solo funzioni sicure, segnala errore e tenta riconnessione.

Questa regola è molto importante per il futuro hardware. Se un comando attiva una bobina e poi la rete cade, il nodo non deve tenere la bobina accesa indefinitamente. Ogni comando fisico deve avere durata massima locale, indipendente dalla comunicazione. Questa è una garanzia di sicurezza fondamentale.

Dal punto di vista del protocollo, il nodo deve poter distinguere comandi istantanei, comandi temporizzati e comandi persistenti. Un comando istantaneo produce un effetto breve, come un flash LED o un impulso. Un comando temporizzato ha durata esplicita. Un comando persistente cambia modalità, per esempio idle o diagnostic. I comandi che coinvolgono attuatori energetici dovrebbero preferibilmente essere temporizzati, in modo che il nodo possa terminarli da solo anche senza ulteriore messaggio. Questo rafforza la sicurezza locale.

Il MicroBot come nodo di rete è anche il punto in cui si incontrano comunicazione e sincronizzazione. Un nodo deve sapere non solo cosa fare, ma quando farlo. Per azioni semplici, il comando può essere eseguito subito. Per azioni coordinate, il comando deve contenere un timestamp di esecuzione o uno slot temporale. Il nodo di rete deve quindi mantenere una stima del tempo del sistema e confrontarla con i timestamp ricevuti. Questo sarà approfondito nella Sezione 9, ma già qui si deve riconoscere che un nodo di rete non è solo un ricevitore di messaggi: è anche un partecipante temporale.

Un comando sincronizzato può avere:

`execute_at;`

`duration_ms;`

`sequence_id;`

`timebase_id.`

Il nodo riceve il comando, lo memorizza se valido e lo esegue quando il proprio clock locale raggiunge `execute_at`. Questo permette a più nodi di coordinarsi anche se i pacchetti arrivano con piccoli ritardi. La capacità di memorizzare comandi futuri è quindi una funzione importante del nodo di rete.

Dal punto di vista della scalabilità, ogni nodo deve limitare la quantità di dati che invia. Se cento MicroBot inviano telemetria completa a frequenza elevata, la rete può saturarsi. Per questo motivo, il nodo deve supportare livelli di telemetria. Un livello basic invia solo heartbeat. Un livello normal invia stato essenziale. Un livello debug invia informazioni dettagliate ma solo durante test. Un livello silent può ridurre messaggi quando il nodo è stabile. Questo permette al controller di regolare il traffico.

Una possibile classificazione è:

`telemetry_level = off;`

`telemetry_level = heartbeat_only;`

`telemetry_level = normal;`

`telemetry_level = debug;`

`telemetry_level = trace.`

Il livello trace deve essere usato con cautela, perché può generare molti dati. Nei test è utile; in funzionamento normale può essere eccessivo. Questa logica permette di adattare la comunicazione al contesto.

Il MicroBot come nodo di rete deve infine essere documentato come entità persistente del sistema. Ogni nodo dovrebbe avere un profilo tecnico, anche semplice, che includa ID, versione firmware, componenti supportati, eventuale ruolo, note di calibrazione e stato di test. In un ecosistema con più prototipi, questa

documentazione è fondamentale. Permette di sapere quali nodi sono LED-only, quali supportano PWM, quali sono simulati, quali hanno errori noti, quali sono stati usati in determinate prove.

Un profilo nodo può includere:

node_id;

hardware_version;

firmware_version;

capabilities;

calibration_data;

known_limits;

last_tested;

notes.

Questa struttura collega comunicazione, hardware e documentazione. Il nodo non è più un oggetto anonimo, ma una unità tracciata nel ciclo di sviluppo del progetto.

In conclusione, il MicroBot come nodo di rete è l'unità comunicativa fondamentale dello swarm. Ogni nodo deve possedere identità, indirizzo, stato, capacità dichiarate, heartbeat, validazione dei messaggi, feedback, gestione degli errori, fallback di sicurezza e integrazione con dashboard e telemetria. Questa astrazione permette di trattare MicroBot fisici e simulati dentro lo stesso ecosistema logico, rendendo il sistema modulare, scalabile e osservabile. Il nodo di rete è il punto in cui il MicroBot diventa realmente parte di un sistema distribuito: non solo un componente fisico nello spazio, ma un agente identificabile dentro una rete di comandi, stati, ruoli e relazioni. Il punto successivo, dedicato al protocollo UDP, analizzerà il meccanismo di trasporto scelto per rendere questa comunicazione rapida, leggera e adatta alle prime versioni dello swarm.

8.3 Protocollo UDP

Il protocollo UDP rappresenta uno dei possibili meccanismi di trasporto più adatti alle prime versioni comunicative del sistema MicroBot, perché consente lo scambio di pacchetti in modo leggero, rapido e relativamente semplice da implementare su microcontrollori come ESP32. Dopo aver definito la rete Wi-Fi come infrastruttura e il MicroBot come nodo identificabile all'interno della rete, è necessario specificare quale protocollo possa essere usato per trasportare effettivamente i messaggi tra controller e nodi. In questo contesto, UDP risulta particolarmente coerente con un sistema sperimentale basato su comandi brevi, heartbeat, telemetria leggera, aggiornamenti frequenti e bassa latenza.

UDP, cioè User Datagram Protocol, è un protocollo di trasporto basato sull'invio di datagrammi indipendenti. A differenza di TCP, non stabilisce una connessione persistente tra mittente e destinatario, non garantisce automaticamente la consegna dei pacchetti, non garantisce l'ordine di arrivo e non prevede un meccanismo interno di ritrasmissione. Questa caratteristica potrebbe sembrare inizialmente un limite, ma nel caso MicroBot può diventare un vantaggio se il protocollo applicativo viene progettato correttamente. Uno swarm non ha sempre bisogno di trasportare grandi flussi di dati affidabili; spesso ha bisogno di inviare piccoli messaggi rapidamente, accettando che alcuni aggiornamenti possano essere persi purché il sistema resti stabile, sicuro e aggiornato dai messaggi successivi.

Nel sistema MicroBot, UDP è particolarmente adatto per messaggi di stato, heartbeat, comandi brevi e aggiornamenti di controllo. Un heartbeat periodico, ad esempio, non richiede necessariamente che ogni singolo pacchetto venga consegnato: se un heartbeat viene perso ma quello successivo arriva correttamente, il controller può comunque sapere che il nodo è attivo. Allo stesso modo, un aggiornamento di telemetria ad alta frequenza può tollerare qualche perdita, perché il dato successivo sostituisce quello precedente. Questo tipo di comunicazione è diverso dall'invio di un file o di un documento, dove ogni byte

deve arrivare correttamente e in ordine. Per uno swarm, spesso conta di più ricevere informazioni recenti che ricevere perfettamente tutte le informazioni vecchie.

La natura connectionless di UDP rende il sistema più semplice. Il controller può inviare un pacchetto a un nodo specifico, a un gruppo o a tutti i nodi tramite broadcast, senza dover aprire e mantenere una connessione TCP per ciascun MicroBot. Questo è molto utile quando il numero di nodi cresce, perché mantenere molte connessioni persistenti può diventare più complesso dal punto di vista del firmware e della gestione dello stato. Con UDP, ogni messaggio è autonomo: contiene destinatario, comando, timestamp e dati necessari all'interpretazione. Il nodo riceve il pacchetto, lo valida, decide se eseguirlo e, se previsto, invia una risposta.

Nel progetto MicroBot, la comunicazione UDP può essere organizzata intorno a una porta comune. Per esempio, ogni nodo MicroBot può ascoltare su una porta UDP definita, mentre il controller invia i pacchetti a quella porta. Il nodo può a sua volta inviare heartbeat e telemetria a una porta del controller. Questa struttura semplifica la configurazione: ogni dispositivo conosce la porta del sistema e può partecipare al protocollo. L'identificazione del destinatario non deve dipendere soltanto dalla porta, ma dal contenuto del messaggio, in particolare dal campo target o node_id. In questo modo, la rete può supportare sia messaggi diretti sia messaggi broadcast.

La struttura concettuale può essere descritta così:

controller UDP socket → invio comando → MicroBot UDP listener → validazione → esecuzione → risposta UDP verso controller

Questo flusso mette in evidenza la semplicità del modello. Il controller non deve mantenere una connessione aperta con ogni nodo; deve solo conoscere l'indirizzo o usare broadcast. Il MicroBot non deve gestire una sessione complessa; deve solo ascoltare pacchetti, interpretarli e rispondere quando necessario. Questa semplicità è adatta alle prime versioni del progetto, perché permette di implementare rapidamente un canale di comando e telemetria senza appesantire il firmware.

Tuttavia, proprio perché UDP non garantisce automaticamente affidabilità, il sistema MicroBot deve introdurre alcuni controlli a livello applicativo. Questo è un punto fondamentale. Non si deve usare UDP in modo ingenuo, assumendo che ogni comando arrivi sempre e nell'ordine corretto. Il protocollo MicroBot deve aggiungere campi come message_id, timestamp, sequence_number, command_type e target, così da permettere al nodo di riconoscere messaggi duplicati, vecchi, non destinati a lui o non validi. UDP fornisce il trasporto leggero; il protocollo MicroBot deve fornire la logica di interpretazione.

Una struttura minima di pacchetto UDP MicroBot può essere:

protocol_version;

message_id;

timestamp;

source_id;

target_id;

message_type;

command;

payload;

checksum opzionale o campo di validazione;

Questi campi permettono di trasformare un datagramma UDP generico in un messaggio applicativo comprensibile. `protocol_version` permette di gestire evoluzioni future del protocollo. `message_id` identifica univocamente il messaggio. `timestamp` indica quando il messaggio è stato generato o quando deve essere eseguito. `source_id` indica chi lo ha inviato. `target_id` indica il destinatario. `message_type` distingue comando, heartbeat, telemetria, ack, errore o sincronizzazione. `command` specifica l'azione richiesta. `payload` contiene i parametri. Un campo di validazione può aiutare a rilevare messaggi corrotti o malformati, anche se a livello UDP e IP esistono già controlli di base.

Il campo `message_id` è particolarmente importante per la gestione dei duplicati. Anche se UDP non crea normalmente duplicati come comportamento principale, in reti reali o in logiche di ritrasmissione applicativa può capitare che lo stesso comando venga ricevuto più volte. Se il comando è innocuo, come aggiornare uno stato LED, il problema può essere limitato. Se invece il comando attiva una bobina, avvia un impulso o modifica una connessione, la duplicazione può creare effetti indesiderati. Il nodo deve quindi poter riconoscere che un messaggio è già stato processato e decidere se ignorarlo, confermarlo di nuovo o trattarlo come duplicato.

Il campo `timestamp` è altrettanto importante. Un comando ricevuto troppo tardi potrebbe non essere più valido. Per esempio, se il controller invia un comando per attivare una sequenza sincronizzata a un certo istante, un nodo che riceve il messaggio dopo quell'istante non dovrebbe eseguirlo in ritardo senza controllo. Il `timestamp` permette al nodo di valutare la freschezza del messaggio. In una versione semplice, il `timestamp` può indicare il momento di generazione. In una versione più avanzata, può esistere un campo `execute_at`, che indica il tempo futuro in cui il comando deve essere eseguito. Questa distinzione sarà molto importante nella Sezione 9 dedicata alla sincronizzazione.

Il campo `target_id` consente di distinguere messaggi unicast, broadcast e potenzialmente groupcast. Se `target_id` corrisponde al `node_id` locale, il nodo può processare il comando. Se `target_id` è all, il nodo può trattarlo come broadcast. Se `target_id` è un `group_id` a cui il nodo appartiene, può eseguirlo come comando di gruppo. Se `target_id` non corrisponde, il nodo deve ignorare il pacchetto. Questa logica è fondamentale quando si usa UDP broadcast, perché tutti i nodi possono ricevere lo stesso pacchetto, ma non tutti devono necessariamente eseguirlo.

Il campo `message_type` permette di separare semanticamente i messaggi. Un pacchetto di comando non deve essere interpretato come telemetria; un heartbeat non deve essere trattato come comando; un ack non deve essere confuso con un errore. Una classificazione iniziale può includere:

command;

heartbeat;

telemetry;

ack;

error;

sync;

discovery;

diagnostic.

Questa distinzione rende il protocollo più ordinato. Il firmware può usare un dispatcher interno: prima legge `message_type`, poi invia il pacchetto al modulo corretto. I messaggi command vanno al command dispatcher; heartbeat aggiorna la tabella di presenza; telemetry aggiorna lo stato osservato; ack conferma ricezione; error registra problemi; sync aggiorna il riferimento temporale; discovery gestisce registrazione e presenza; diagnostic supporta debug. Questa struttura è coerente con l'architettura modulare del sistema.

Il campo `command` indica l'azione richiesta. In una fase iniziale, i comandi possono essere semplici: `set_led`, `set_mode`, `ping`, `reset_state`, `request_status`. In una fase più avanzata, possono includere `set_coil_pwm`,

magnetic_pulse, assign_role, execute_pattern_step, sync_slot, enter_safety, release_connection. La cosa importante è che ogni comando abbia parametri chiari e limiti definiti. UDP trasporta il comando, ma non ne definisce il significato: il significato appartiene al protocollo MicroBot.

Il payload deve essere mantenuto compatto. Un payload troppo grande può aumentare frammentazione, latenza e complessità di parsing. Per comandi embedded, è preferibile usare messaggi brevi, con campi essenziali. Il formato può essere JSON nelle prime versioni, perché è leggibile, facile da debuggare e comodo da usare con Python e web dashboard. Tuttavia, JSON è più pesante di formati binari o compatti. Per le prime fasi va benissimo, perché la leggibilità e la rapidità di sviluppo sono più importanti dell'ottimizzazione estrema. In futuro, se il sistema crescerà molto, si potrà valutare un formato più compatto.

Una possibile struttura JSON di comando può essere:

```
{
  "protocol": "microbot_udp",
  "version": "0.1",
  "message_id": "cmd_000123",
  "timestamp": 123456.78,
  "source": "controller",
  "target": "node_01",
  "type": "command",
  "command": "set_led",
  "payload": {
    "r": 0,
    "g": 180,
    "b": 255,
    "duration_ms": 500
  }
}
```

Questa struttura è leggibile e adatta al debug. Un nodo può verificare versione, target, type e command, poi leggere i parametri nel payload. Se il comando non è supportato, può rispondere con un errore. Se è supportato ma non sicuro, può rifiutarlo. Se è valido, può eseguirlo e inviare ack. Anche se il formato potrà cambiare in futuro, questa struttura mostra la logica del protocollo.

Un pacchetto di heartbeat potrebbe essere:

```
{
  "protocol": "microbot_udp",
  "version": "0.1",
  "message_id": "hb_node_01_000456",
  "timestamp": 123457.10,
  "source": "node_01",
  "target": "controller",
  "type": "heartbeat",
  "payload": {
    "status": "online",
    "mode": "idle",
    "battery": 0.82,
    "safety": "ok"
  }
}
```

Questo tipo di messaggio consente al controller di mantenere aggiornata la tabella dei nodi. Il payload è breve ma sufficiente per capire se il nodo è presente, in quale modalità si trova e se ci sono problemi evidenti. Una telemetria più dettagliata può essere inviata meno frequentemente o su richiesta.

Un messaggio di ack potrebbe essere:

```
{
  "protocol": "microbot_udp",
  "version": "0.1",
```

```

"message_id": "ack_000123",
"timestamp": 123456.90,
"source": "node_01",
"target": "controller",
"type": "ack",
"payload": {
  "ref_message_id": "cmd_000123",
  "result": "accepted",
  "reason": "ok"
}
}

```

Questa risposta distingue chiaramente il messaggio di conferma dal comando originale. Il campo `ref_message_id` collega l'ack al comando ricevuto. Il campo `result` indica l'esito. Il campo `reason` fornisce una spiegazione. Questo meccanismo è molto utile per compensare parzialmente l'assenza di affidabilità automatica di UDP. Il controller non può sapere con certezza che un pacchetto è arrivato finché non riceve una risposta; l'ack fornisce questa conferma a livello applicativo.

Bisogna però evitare di trasformare UDP in una copia pesante di TCP. Non tutti i messaggi devono richiedere ack. Un heartbeat, per esempio, può non richiedere risposta: se un pacchetto viene perso, il successivo arriverà. Un comando critico, invece, dovrebbe richiedere ack o almeno conferma di accettazione. Un comando energetico, come attivare una coil, dovrebbe probabilmente richiedere conferma o essere progettato con durata massima locale e sicurezza indipendente. Il protocollo deve quindi distinguere tra messaggi best-effort e messaggi che richiedono conferma.

Una possibile classificazione è:

messaggi senza ack obbligatorio: heartbeat, telemetria periodica, aggiornamenti frequenti;

messaggi con ack consigliato: cambio modalità, assegnazione ruolo, `set_led` persistente;

messaggi con ack necessario: attivazione bobina, comando magnetico, reset, modifica safety, avvio sequenza sincronizzata.

Questa distinzione rende il sistema efficiente. Non si spreca traffico per confermare ogni pacchetto non critico, ma si protegge ciò che può produrre effetti rilevanti.

UDP è adatto anche al broadcast. Il controller può inviare un singolo pacchetto all'indirizzo broadcast della rete, e tutti i nodi possono riceverlo. Questo è utile per comandi globali come `ping_all`, `enter_idle`, `sync_time`, `stop_all`, `start_pattern` o `emergency_disable_active_outputs`. Il broadcast riduce il numero di messaggi necessari quando molti nodi devono ricevere la stessa informazione. Tuttavia, il broadcast deve essere usato con attenzione, perché non garantisce che tutti i nodi abbiano ricevuto il pacchetto. Per comandi critici, può essere utile richiedere ack individuali dai nodi, anche se il comando è stato trasmesso in broadcast.

Un comando broadcast può avere:

```
target = "all"
```

Ogni nodo che riceve il pacchetto verifica `target` e decide se processarlo. Se il comando è `stop_all`, ogni nodo deve spegnere immediatamente uscite attive e rispondere, se possibile, con un ack di sicurezza. Per un comando meno critico, come `update_visual_mode`, si può accettare che qualche nodo perda il pacchetto e venga aggiornato al ciclo successivo.

La comunicazione unicast, invece, viene usata per comandi specifici verso un nodo. È utile per calibrazione, diagnostica, assegnazione di ruolo individuale, test di una singola coil o aggiornamento di uno stato particolare. L'unicast consente maggiore precisione e riduce l'ambiguità. Il controller invia il pacchetto all'indirizzo IP del nodo registrato e imposta `target_id` al `node_id` corrispondente. Il nodo verifica che il pacchetto sia destinato a lui e lo processa.

Il protocollo UDP deve considerare anche la perdita di pacchetti. Se un comando importante non riceve ack entro un timeout, il controller può ritentare. Tuttavia, la ritrasmissione deve essere progettata con attenzione per evitare duplicati pericolosi. Se il controller ritrasmette lo stesso comando, deve usare lo stesso message_id oppure un campo retry_of, così il nodo può capire che si tratta di un tentativo ripetuto dello stesso comando e non di un nuovo comando. Questo è essenziale per comandi non idempotenti, cioè comandi che producono effetti diversi se eseguiti più volte.

Un comando idempotente è un comando che può essere ripetuto senza cambiare ulteriormente il risultato. Per esempio, set_mode idle è generalmente idempotente: eseguirlo due volte lascia il nodo in idle. Un comando non idempotente può essere magnetic_pulse, perché due impulsi consecutivi non equivalgono a uno solo. Per i comandi non idempotenti, la gestione di message_id e ack è obbligatoria. Il nodo deve evitare di eseguire due volte lo stesso impulso a causa di ritrasmissione.

Una regola robusta è:

se message_id già eseguito, non rieseguire; reinvia eventualmente l'ultimo ack.

Questa regola permette al controller di ritentare senza rischiare effetti duplicati. Il nodo mantiene una piccola cache degli ultimi message_id processati. Non serve conservarli per sempre; basta una finestra temporale o un numero limitato di ID recenti. Questa è una soluzione semplice ma molto efficace.

UDP introduce anche il problema dell'ordine dei pacchetti. Due comandi possono arrivare in ordine diverso rispetto a quello di invio. Per questo motivo, ogni messaggio dovrebbe avere sequence_number o timestamp. Se un nodo riceve un comando più vecchio di uno già processato, può ignorarlo o gestirlo secondo regole precise. Questo è particolarmente importante per comandi di stato. Se il controller invia set_mode active e poi set_mode idle, ma i pacchetti arrivano invertiti, il nodo potrebbe finire nello stato sbagliato. Un numero di sequenza aiuta a evitare questo problema.

Una regola possibile è:

accetta un comando di stato solo se sequence_number è maggiore dell'ultimo comando accettato per quella categoria.

Per comandi temporizzati, invece, il riferimento principale può essere execute_at. Se il tempo di esecuzione è già passato, il nodo può rifiutare il comando con reason = command_expired. Questo evita esecuzioni tardive.

Il protocollo UDP deve essere integrato con il safety layer. Il nodo non deve mai affidarsi completamente alla ricezione di un futuro comando di stop per spegnere un'uscita attiva. Ogni comando che attiva una funzione energetica deve contenere una durata massima locale. Per esempio, un comando magnetic_pulse deve includere duration_ms. Il nodo attiva la bobina per quella durata e poi la spegne automaticamente. Anche se il pacchetto di stop viene perso, la bobina non resta attiva indefinitamente. Questo principio è fondamentale.

Un comando sicuro per bobina dovrebbe includere almeno:

coil_id;

duty_cycle;

duration_ms;

max_thermal_policy;

command_id;

timestamp.

Il nodo valida `duty_cycle` e `duration_ms` rispetto ai propri limiti. Se il valore supera il massimo consentito, rifiuta o limita il comando. Questa logica dimostra che UDP può essere usato anche per funzioni fisiche più delicate, purché il protocollo applicativo e il firmware siano progettati con sicurezza locale.

Dal punto di vista della latenza, UDP è vantaggioso perché evita il sovraccarico della connessione TCP e permette invio rapido di datagrammi. Tuttavia, bassa latenza non significa latenza garantita. La rete Wi-Fi può introdurre ritardi variabili. Per questo motivo, UDP deve essere accompagnato da timestamp, sincronizzazione e strategie di esecuzione temporizzata. Il protocollo non deve assumere che “ricevuto” significhi “ricevuto nello stesso istante da tutti”. Deve invece permettere ai nodi di coordinarsi usando il tempo come riferimento condiviso.

Per esempio, un comando di pattern può essere:

`start_pattern` con `execute_at = T`

Il controller invia il comando in anticipo. I nodi lo ricevono in tempi leggermente diversi, ma lo eseguono quando il loro clock raggiunge `T`. Questo riduce l'effetto della latenza variabile. Se un nodo riceve il comando dopo `T`, può rifiutarlo oppure entrare in una fase di recupero. Questo tipo di logica sarà approfondito nella sincronizzazione, ma nasce già dal modo in cui UDP viene usato.

Dal punto di vista del parsing, il nodo ESP32 deve interpretare i messaggi in modo efficiente. Se si usa JSON, il firmware deve avere una libreria adatta e limiti chiari alla dimensione massima dei pacchetti. Un pacchetto troppo grande o malformato deve essere rifiutato. Il buffer di ricezione deve avere dimensione controllata per evitare overflow o comportamento instabile. Questo è particolarmente importante nei microcontrollori, dove memoria e risorse non sono infinite. Il protocollo deve quindi definire una dimensione massima del messaggio.

Una regola possibile è:

`max_packet_size` = valore definito dalla versione del protocollo

Se un pacchetto supera tale dimensione, viene ignorato o classificato come `packet_too_large`. Questo evita che un messaggio errato blocchi il nodo. Anche in un prototipo, questa disciplina è utile perché rende il firmware più robusto.

Il protocollo deve prevedere errori espliciti. Se un nodo riceve un messaggio non valido, non deve fallire silenziosamente quando è possibile rispondere. Può inviare un pacchetto `error` al controller con motivo specifico. Per esempio:

`invalid_json;`

`unsupported_protocol_version;`

`wrong_target;`

`unsupported_command;`

`missing_field;`

`invalid_payload;`

`safety_rejected;`

`command_expired;`

`busy;`

low_energy;

thermal_lock.

Questi errori aiutano moltissimo nel debug. Se il sistema non funziona, il controller può mostrare esattamente perché. Una comunicazione ben progettata non nasconde gli errori; li rende leggibili.

Un messaggio di errore può essere:

```
{
  "protocol": "microbot_udp",
  "version": "0.1",
  "message_id": "err_000124",
  "timestamp": 123456.95,
  "source": "node_01",
  "target": "controller",
  "type": "error",
  "payload": {
    "ref_message_id": "cmd_000123",
    "error_code": "unsupported_command",
    "detail": "command set_coil_pwm not available on this node"
  }
}
```

Questo tipo di errore è prezioso quando il sistema include nodi con capacità diverse. Il controller può ricevere l'errore e aggiornare la propria conoscenza del nodo o segnalare all'utente che quel comando non è supportato.

Dal punto di vista del controller, UDP richiede una logica di gestione dei nodi. Il controller deve mantenere una tabella con node_id, indirizzo IP, porta, ultimo heartbeat, stato, capabilities, sequence_number e comandi pendenti. Quando invia un comando che richiede ack, deve registrarlo come pending. Se riceve ack, lo marca come confermato. Se scade il timeout, decide se ritrasmettere, segnalare errore o applicare fallback. Questa logica trasforma UDP da trasporto semplice a sistema di comunicazione controllato.

Una tabella comandi pendenti può includere:

message_id;

target_node;

command;

sent_at;

retry_count;

ack_required;

status;

timeout_at.

Questa struttura consente al controller di monitorare la comunicazione. Se molti comandi verso un nodo vanno in timeout, il nodo può essere marcato come instabile. Se molti nodi non rispondono, può esserci un problema di rete. Se solo comandi specifici falliscono, può esserci un problema di capability o safety. Il protocollo produce quindi anche dati diagnostici.

UDP è adatto anche a una simulazione locale. Il controller può inviare pacchetti UDP non solo a ESP32 fisici, ma anche a processi software che simulano nodi MicroBot. Questo permette di testare il protocollo in modo rapido. Un nodo simulato può ascoltare su una porta UDP, ricevere comandi, aggiornare il proprio stato virtuale e inviare heartbeat. In questo modo, la stessa logica di comunicazione può essere usata per hardware reale e simulazione. Questo è molto coerente con l'ecosistema MicroBot, perché permette di sviluppare controller, dashboard e protocollo anche prima di completare i nodi fisici.

Nel contesto della dashboard, il protocollo UDP può alimentare una visualizzazione live. Ogni heartbeat aggiorna lo stato del nodo; ogni telemetria aggiorna batteria, temperatura, modo operativo; ogni ack aggiorna il pannello comandi; ogni errore appare nel log; ogni evento di rete viene registrato. La dashboard diventa quindi il monitor del protocollo. Questo è fondamentale per dimostrare il sistema ad Andrea o a un osservatore tecnico: non si vede solo "qualcosa che lampeggia", ma un sistema di nodi comunicanti, identificati e monitorati.

Dal punto di vista della documentazione, è importante dichiarare chiaramente i limiti di UDP. UDP non è affidabile per definizione. Non garantisce consegna, ordine o conferma. Nel progetto MicroBot, questi limiti vengono gestiti con `message_id`, `timestamp`, ack selettivi, `timeout`, `retry`, validazione locale e `safety`. Questa frase è importante perché mostra consapevolezza ingegneristica. Non si sta scegliendo UDP perché "è perfetto", ma perché è leggero e adatto a messaggi rapidi, e viene completato da un livello applicativo robusto.

Il confronto con TCP è utile per chiarire la scelta. TCP garantisce consegna ordinata e affidabile, ma introduce connessioni persistenti, maggiore overhead e possibili ritardi dovuti a ritrasmissioni e controllo di congestione. Per alcune funzioni, TCP potrebbe essere utile, ad esempio configurazione, aggiornamenti, trasferimento di file o interfacce web. Per comandi rapidi e telemetria leggera, UDP può essere più adatto. Il sistema MicroBot potrebbe in futuro usare entrambi: UDP per controllo realtime leggero, HTTP/WebSocket/TCP per dashboard, configurazione e monitoraggio più strutturato. Tuttavia, nella Sezione 8 il focus resta UDP come protocollo di comando rapido.

Questa possibile architettura ibrida può essere descritta così:

UDP = comandi rapidi, heartbeat, telemetria leggera, broadcast;

WebSocket o HTTP = dashboard, configurazione, visualizzazione avanzata, log;

file JSON = configurazioni, dataset, snapshot, replay.

Questa separazione evita di usare un solo protocollo per tutto. Ogni canale ha una funzione. UDP lavora sul piano rapido e leggero; la dashboard può usare protocolli più adatti all'interfaccia; i file mantengono memoria e configurazione. L'ecosistema risulta più ordinato.

Nel contesto del firmware ESP32, UDP è relativamente accessibile. Un ESP32 può connettersi al Wi-Fi, aprire un socket UDP, ascoltare su una porta e inviare pacchetti. Questo lo rende adatto alla prototipazione. Il codice deve però essere scritto in modo non bloccante e con buffer sicuri. La ricezione di pacchetti deve essere parte del loop principale o di un task controllato. Dopo la ricezione, il messaggio deve essere copiato, validato e processato senza bloccare funzioni critiche. La comunicazione non deve compromettere `safety`, PWM, LED o watchdog.

Il watchdog è particolarmente importante. Se il parsing di un pacchetto malformato o troppo grande bloccasse il firmware, il nodo potrebbe smettere di rispondere. Il watchdog può aiutare a recuperare da blocchi, ma il firmware deve comunque essere progettato per evitare situazioni pericolose. Anche in caso di reset, le uscite attive devono tornare in stato sicuro. Questo collega UDP, firmware e `safety layer`.

Un altro aspetto è la frequenza dei messaggi. Il sistema deve definire intervalli ragionevoli. Heartbeat troppo frequenti saturano la rete; troppo rari rendono lenta la rilevazione di nodi offline. Telemetria troppo frequente consuma energia; troppo rara riduce osservabilità. Comandi ripetuti troppo spesso possono creare traffico inutile. Una versione iniziale può usare valori semplici e poi calibrare. Per esempio, heartbeat a intervalli moderati e telemetria dettagliata su richiesta o a frequenza inferiore. Il documento non deve fissare valori assoluti senza test, ma deve definire il principio: la frequenza deve essere bilanciata tra reattività, consumo e stabilità.

UDP può supportare anche messaggi di discovery. Un nodo appena acceso può inviare un broadcast di presenza al controller, oppure il controller può inviare un ping broadcast e i nodi rispondono con il proprio profilo. Questo rende il sistema più plug-and-play. Invece di configurare manualmente ogni indirizzo IP, il controller può scoprire i nodi attivi. Questa funzione è molto utile in dimostrazioni e prototipi.

Una sequenza di discovery può essere:

controller invia discovery_request in broadcast;

ogni nodo risponde con discovery_response;

controller registra node_id, ip, capabilities e firmware_version;

dashboard aggiorna lista dei nodi.

Questa sequenza rende il sistema più professionale e gestibile. Anche se nelle prime versioni si può inserire manualmente l'indirizzo di un nodo, la discovery automatica è una direzione naturale.

Dal punto di vista della sicurezza operativa, un comando globale di emergenza deve essere previsto. Per esempio:

emergency_stop;

stop_all;

disable_active_outputs.

Questo comando dovrebbe essere breve, broadcast e prioritario. Ogni nodo che lo riceve deve spegnere uscite attive, annullare comandi pendenti e passare in stato safe. Dato che UDP non garantisce consegna, nei casi più critici il controller può ripetere il comando più volte per un breve intervallo. Inoltre, ogni nodo deve comunque avere timeout locali per evitare di dipendere solo da emergency_stop. La sicurezza deve essere ridondante.

Un comando emergency_stop non deve richiedere payload complesso. Più il comando è semplice, minore è il rischio di parsing errato. Può avere un tipo e un message_id, e deve essere riconosciuto rapidamente. Nel firmware, questo comando dovrebbe avere priorità alta rispetto ad altri comandi. Se arriva mentre il nodo sta eseguendo un'azione, deve interromperla. Questa logica deve essere documentata chiaramente.

Dal punto di vista dell'evoluzione del protocollo, è importante includere protocol_version. MicroBot crescerà nel tempo: i comandi cambieranno, i payload aumenteranno, i nodi avranno capacità diverse. Senza versionamento, un nodo vecchio potrebbe interpretare male un comando nuovo. Con versionamento, il nodo può rifiutare protocolli non supportati o usare una modalità compatibile. Questo è particolarmente importante in un ecosistema con più repo, versioni e prototipi.

Un nodo può dichiarare:

supported_protocol_versions = ["0.1", "0.2"]

Se riceve un messaggio version "0.3" non supportato, risponde con unsupported_protocol_version. Questo evita errori silenziosi e rende il sistema più robusto.

Il protocollo UDP deve anche essere documentato con tabelle dei comandi. Ogni comando dovrebbe avere nome, direzione, payload richiesto, risposta attesa, ack necessario, condizioni di safety e note. Questo trasforma il protocollo in interfaccia tecnica chiara. Per esempio:

set_led: controller → node, payload RGB e durata, ack consigliato;

heartbeat: node → controller, payload stato base, ack non necessario;

set_mode: controller → node, payload modalità, ack necessario;

magnetic_pulse: controller → node, payload coil, duty, durata, ack necessario, safety obbligatoria;

emergency_stop: controller → all, payload minimo, ack consigliato ma non bloccante, priorità massima.

Questa tabella potrà essere sviluppata più avanti nella documentazione o in un file protocol.md. Il punto essenziale è che UDP non basta da solo: serve una grammatica di messaggi ben definita.

Dal punto di vista teorico, UDP rappresenta il canale attraverso cui l'informazione entra nel sistema distribuito. Le sezioni precedenti hanno descritto come le relazioni fisiche generano forma; UDP consente al controller di modificare parametri, stati e comandi che influenzano tali relazioni. L'informazione digitale diventa quindi causa indiretta di trasformazione fisica. Un pacchetto UDP può cambiare il modo operativo di un nodo; il modo operativo cambia la forza virtuale o magnetica; la forza cambia la connessione; la connessione cambia la forma dello swarm. Anche qui ritorna la catena fondamentale:

messaggio → stato → azione → interazione → forma

Questa catena mostra che la comunicazione non è separata dalla materia programmabile. È uno dei modi attraverso cui la materia viene programmata. Il comando digitale non modifica direttamente la forma, ma modifica le condizioni locali che producono la forma.

In conclusione, il protocollo UDP costituisce un livello di trasporto leggero, rapido e adatto alle prime versioni dello swarm MicroBot. La sua forza principale è la semplicità: permette l'invio di pacchetti brevi, supporta broadcast e unicast, si integra bene con ESP32 e consente comunicazione a bassa latenza. I suoi limiti sono l'assenza di garanzia automatica di consegna, ordine e conferma; per questo motivo, il protocollo applicativo MicroBot deve aggiungere message_id, timestamp, sequence_number, ack selettivi, validazione locale, timeout, retry controllati e safety indipendente dalla rete. Usato in questo modo, UDP diventa uno strumento molto efficace per comandi, heartbeat, telemetria leggera, discovery e sincronizzazione iniziale. Il punto successivo analizzerà proprio la comunicazione a bassa latenza, cioè il motivo per cui tempi rapidi e prevedibili sono fondamentali per coordinare uno swarm distribuito senza perdere stabilità, sicurezza e coerenza comportamentale.

8.4 Comunicazione a bassa latenza

La comunicazione a bassa latenza rappresenta una proprietà fondamentale del sistema MicroBot, perché determina la capacità dei nodi di ricevere comandi, aggiornare stati e coordinare azioni in tempi sufficientemente rapidi rispetto alla dinamica dello swarm. Dopo aver introdotto il protocollo UDP come meccanismo di trasporto leggero, è necessario analizzare perché la rapidità della comunicazione sia così importante e quali limiti debbano essere considerati. In un sistema distribuito, infatti, non basta che i messaggi arrivino: devono arrivare in un intervallo temporale compatibile con il comportamento che si vuole ottenere. Se il ritardo è troppo elevato o troppo variabile, il sistema può perdere sincronizzazione, generare azioni incoerenti, rompere pattern, fallire agganci o produrre stati non allineati tra controller e nodi.

Nel progetto MicroBot, la latenza può essere definita come il tempo che intercorre tra il momento in cui un messaggio viene generato dal controller e il momento in cui il MicroBot lo riceve, lo interpreta e lo rende disponibile alla logica applicativa. Questa definizione deve essere distinta dal semplice tempo di trasmissione sulla rete. La latenza effettiva include più fasi: generazione del comando, serializzazione del pacchetto, invio sulla rete Wi-Fi, ricezione sul microcontrollore, parsing del messaggio, validazione, inserimento nella coda comandi ed eventuale esecuzione. Di conseguenza, la comunicazione a bassa latenza non dipende soltanto dal protocollo UDP, ma dall'intera pipeline comunicativa.

In forma concettuale, il ritardo complessivo può essere rappresentato come:

latenza_totale = tempo_generazione + tempo_trasmissione + tempo_ricezione + tempo_parsing + tempo_validazione + tempo_dispatch + tempo_esecuzione

Questa formulazione è importante perché evita una semplificazione pericolosa: scegliere UDP non garantisce automaticamente un sistema realtime. UDP riduce parte dell'overhead rispetto a protocolli più pesanti, ma il comportamento finale dipende anche dalla rete Wi-Fi, dal firmware, dal carico del microcontrollore, dalla frequenza del loop principale, dalla dimensione dei messaggi e dalla gestione delle priorità. Per ottenere una comunicazione realmente rapida, tutto il percorso del messaggio deve essere progettato con attenzione.

Nel sistema MicroBot, la bassa latenza è importante per tre motivi principali. Il primo riguarda il controllo del comportamento. Se il controller invia un comando di cambio modalità, pattern o stato magnetico, i nodi devono riceverlo in tempi compatibili con la fase corrente. Un comando arrivato troppo tardi può essere inutile o dannoso. Il secondo motivo riguarda la sincronizzazione. Se più nodi devono eseguire una sequenza coordinata, ritardi diversi possono far partire le azioni in momenti differenti. Il terzo motivo riguarda la sicurezza. Se un comando di stop, disattivazione o correzione arriva troppo tardi, un nodo può restare in uno stato non desiderato più a lungo del previsto. La latenza, quindi, non è solo una questione di prestazioni, ma anche di coerenza e sicurezza.

È importante distinguere tra latenza media e jitter. La latenza media indica il ritardo tipico di comunicazione; il jitter indica quanto questo ritardo varia nel tempo. In uno swarm, il jitter può essere persino più problematico della latenza media. Se tutti i messaggi arrivano con un ritardo costante, il sistema può compensarlo. Se invece alcuni messaggi arrivano molto rapidamente e altri molto più tardi, le azioni dei nodi diventano difficili da coordinare. Per questo motivo, il sistema MicroBot deve considerare non solo "quanto è veloce" la comunicazione, ma anche quanto è regolare.

Una rappresentazione concettuale può essere:

latenza_media = media dei ritardi misurati

jitter = variazione dei ritardi rispetto alla media

Un sistema con latenza media moderata ma jitter basso può essere più prevedibile di un sistema con latenza media bassa ma jitter alto. Nel primo caso, i nodi ricevono i messaggi con ritardo abbastanza costante; nel secondo, alcune azioni possono essere quasi immediate e altre ritardate in modo imprevedibile. Per MicroBot, la prevedibilità è fondamentale, soprattutto quando si lavora con pattern, attivazioni magnetiche, impulsi temporizzati o sequenze coordinate.

La comunicazione a bassa latenza deve essere collegata alla scelta del tipo di messaggio. Non tutti i messaggi hanno la stessa urgenza. Un heartbeat può tollerare qualche ritardo; una telemetria di debug può arrivare con frequenza inferiore; un comando di pattern può richiedere tempi più stretti; un comando di emergenza deve avere priorità massima. Per questo motivo, il protocollo MicroBot dovrebbe distinguere classi di priorità. Un messaggio critico non dovrebbe restare bloccato dietro a messaggi diagnostici o telemetria pesante. La rete e il firmware devono trattare i messaggi in base alla loro importanza.

Una classificazione possibile è:

priorità alta: emergency_stop, disable_outputs, safety_command, stop_all;

priorità media: comandi di pattern, set_mode, magnetic_pulse, sync_command;

priorità bassa: telemetria periodica, debug, diagnostica non urgente, aggiornamenti visuali.

Questa classificazione consente al nodo e al controller di decidere quali messaggi processare prima. Se un MicroBot riceve un comando di emergenza, deve interrompere o scavalcare la coda normale dei comandi. Se riceve telemetria o comandi diagnostici, può processarli senza compromettere le funzioni critiche. La bassa latenza non si ottiene solo rendendo veloce tutto, ma dando priorità a ciò che deve essere veloce.

Nel firmware del MicroBot, la gestione non bloccante è essenziale. Se il loop principale del nodo resta bloccato in operazioni lente, la latenza di risposta aumenta. Per esempio, un parsing troppo pesante, un delay usato per lampeggiare un LED, un'attesa sincrona su rete o una routine di diagnostica lunga possono impedire al nodo di ricevere e processare rapidamente nuovi pacchetti. Per questo motivo, tutte le funzioni

temporali devono essere progettate in modo non bloccante. Il nodo deve poter controllare la rete, aggiornare LED, gestire PWM, monitorare safety e inviare telemetria senza fermarsi.

Questo principio è particolarmente importante per ESP32 e sistemi embedded. Un comando come “accendi LED per 500 ms” non dovrebbe essere implementato bloccando il programma per 500 ms. Dovrebbe invece impostare uno stato con una scadenza temporale, lasciando il loop libero di continuare a ricevere messaggi. Lo stesso vale per un impulso magnetico: il firmware deve attivare la bobina, registrare il tempo di fine e continuare a processare safety e rete. Se usasse un delay bloccante, un comando di emergenza ricevuto durante l'impulso potrebbe non essere processato fino alla fine del delay. Questo sarebbe inaccettabile in una logica sicura.

La comunicazione a bassa latenza richiede quindi una struttura firmware basata su stati e timer. Ogni azione temporizzata deve avere un tempo di inizio, una durata e una condizione di completamento. Il loop principale controlla periodicamente se le azioni devono continuare o terminare. In questo modo, il nodo resta reattivo. Una struttura concettuale può essere:

ricezione pacchetti;

aggiornamento timer;

esecuzione step brevi;

controllo safety;

invio telemetria se scaduto intervallo;

ripetizione del ciclo.

Questo schema evita blocchi e mantiene bassa la latenza applicativa. Anche se la rete consegna un pacchetto rapidamente, il vantaggio sarebbe perso se il firmware lo processasse in ritardo. La reattività del nodo è quindi parte integrante della comunicazione.

Un altro elemento importante è la dimensione dei messaggi. Pacchetti più piccoli sono generalmente più rapidi da trasmettere, ricevere, copiare e interpretare. Inoltre riducono il rischio di frammentazione e alleggeriscono il parsing sul microcontrollore. Per questo motivo, i messaggi critici devono essere compatti. Non ha senso includere payload lunghi, testo descrittivo o dati diagnostici dentro un comando di emergenza o sincronizzazione. Il protocollo deve separare messaggi operativi brevi da messaggi diagnostici più ricchi.

Per esempio, un comando `emergency_stop` dovrebbe contenere soltanto i campi indispensabili: `protocol_version`, `message_id`, `timestamp`, `source`, `target`, `type` e `command`. Un pacchetto di telemetria debug può invece contenere dati più lunghi, perché non è critico nello stesso modo. Questa separazione migliora la latenza dei comandi importanti e rende il sistema più ordinato.

La frequenza dei messaggi è un altro fattore decisivo. In uno swarm con molti nodi, se ogni MicroBot invia telemetria dettagliata troppo spesso, la rete può congestionarsi. La congestione aumenta latenza, jitter e perdita di pacchetti. Per questo motivo, la telemetria deve essere regolata. Il controller può richiedere livelli diversi: heartbeat frequente ma leggero, stato normale a frequenza media, debug dettagliato solo su richiesta o durante test specifici. Questa strategia impedisce che i dati non urgenti disturbino i comandi operativi.

Una possibile logica è:

heartbeat leggero e regolare;

telemetria normale a intervalli più ampi;

telemetria debug solo su nodi selezionati;

tracce dettagliate solo in laboratorio;

comandi critici sempre prioritari.

Questa organizzazione è fondamentale per la scalabilità. Con pochi nodi, si potrebbe inviare tutto spesso senza vedere problemi. Con molti nodi, lo stesso comportamento saturerebbe la rete. Progettare fin dall'inizio una gerarchia di frequenze rende il sistema più maturo.

La bassa latenza deve essere considerata anche nella dashboard. Se la dashboard riceve dati dal controller e li mostra con ritardo eccessivo, l'utente potrebbe avere una percezione sbagliata dello stato reale dello swarm. Tuttavia, la dashboard non deve necessariamente aggiornarsi alla stessa frequenza del controllo interno. È possibile che il controller lavori a frequenza più alta e la dashboard visualizzi uno stato filtrato o aggiornato a frequenza inferiore. Questa separazione è utile perché l'interfaccia grafica non deve rallentare la logica di controllo. Il sistema operativo deve distinguere tra frequenza di controllo, frequenza di comunicazione e frequenza di visualizzazione.

In forma concettuale:

$$\text{frequenza_controllo} \geq \text{frequenza_comunicazione_operativa} \geq \text{frequenza_visualizzazione}$$

Questo non significa che debba essere sempre così in modo rigido, ma indica una logica sana: la dashboard è uno strumento di osservazione, non deve diventare collo di bottiglia del sistema. Il controller può ricevere pacchetti e aggiornare lo stato interno rapidamente, mentre l'interfaccia mostra snapshot leggibili. Se si tenta di visualizzare ogni micro-evento in tempo reale, si rischia di appesantire inutilmente il sistema.

Nel MicroBot, la bassa latenza è particolarmente importante per le azioni magnetiche. Se due nodi devono attivare bobine in una finestra temporale coordinata, un ritardo tra i comandi può modificare il risultato fisico. Un nodo potrebbe generare campo prima dell'altro, creando attrazione non bilanciata, mancato aggancio o movimento non previsto. Questo diventa ancora più importante quando le bobine sono usate per impulsi brevi. Se l'impulso dura 100 ms ma la latenza varia di decine di millisecondi, il coordinamento può diventare impreciso. Per questo motivo, le azioni magnetiche distribuite dovrebbero usare comandi temporizzati, non esecuzione immediata basata solo sull'arrivo del pacchetto.

Una strategia robusta è inviare il comando in anticipo con un campo `execute_at`. Il controller non dice "attiva ora", ma "attiva al tempo T". Se i nodi hanno clock sincronizzati in modo sufficiente, possono eseguire l'azione nello stesso intervallo anche se ricevono il messaggio con ritardi diversi. Questo approccio riduce la dipendenza dalla latenza istantanea della rete. La comunicazione deve essere abbastanza rapida da consegnare il comando prima di T, ma non deve essere perfettamente simultanea.

Un comando temporizzato può includere:

`execute_at`;

`duration_ms`;

`tolerance_ms`;

`sequence_id`;

`fallback_policy`.

Il campo `tolerance_ms` indica quanto ritardo è accettabile. Se il nodo riceve il comando oltre la finestra utile, può rifiutarlo o applicare una politica di recupero. Il campo `fallback_policy` può indicare cosa fare in caso di ritardo: `ignore`, `execute_late`, `enter_safe`, `request_resync`. Per comandi fisici delicati, `ignore` o `enter_safe` sono spesso più sicuri di `execute_late`. Questa logica rende la comunicazione più affidabile anche in presenza di latenza variabile.

La comunicazione a bassa latenza è collegata anche alla sincronizzazione, ma i due concetti non coincidono. La bassa latenza riguarda la rapidità con cui i messaggi arrivano. La sincronizzazione riguarda

l'allineamento temporale tra i nodi. Un sistema può avere bassa latenza ma clock non sincronizzati; in questo caso i messaggi arrivano rapidamente, ma i nodi non condividono un riferimento temporale comune. Viceversa, un sistema può avere clock abbastanza sincronizzati ma latenza non bassissima; se i comandi arrivano con anticipo, può comunque eseguire azioni coordinate. Per MicroBot servono entrambi: comunicazione rapida e tempo condiviso sufficientemente coerente.

La latenza deve essere misurata, non soltanto ipotizzata. Il controller può inviare messaggi ping ai nodi e misurare il tempo fino alla risposta pong. Questo fornisce una stima del round-trip time, cioè il tempo andata e ritorno. La latenza unidirezionale non è direttamente uguale alla metà del round-trip time in ogni caso, ma questa misura è comunque utile per diagnosticare la qualità della rete. Se il round-trip aumenta o diventa molto variabile, il sistema può ridurre la complessità delle azioni coordinate, aumentare margini temporali o segnalare rete instabile.

Una metrica utile è:

$RTT = \text{tempo_ricezione_pong} - \text{tempo_invio_ping}$

Il controller può calcolare RTT medio, massimo, minimo e jitter. Questi valori possono essere mostrati nella dashboard. Un nodo con RTT molto più alto degli altri può essere lontano dal router, congestionato, in errore firmware o sovraccarico. La dashboard può evidenziare nodi comunicativamente deboli prima che causino problemi nel comportamento dello swarm.

Per misurare la qualità della comunicazione, si possono usare anche `packet_loss` e `missed_heartbeat`. `Packet_loss` indica quanti pacchetti attesi non vengono ricevuti. `Missed_heartbeat` indica quanti heartbeat consecutivi mancano da un nodo. Un singolo heartbeat perso può essere tollerabile; molti heartbeat persi indicano problema. Queste metriche aiutano il controller a decidere se un nodo è affidabile per azioni sincronizzate. Un nodo con rete instabile può essere escluso da una sequenza critica o assegnato a un ruolo meno attivo.

Una possibile classificazione dello stato comunicativo può essere:

`network_good`: RTT basso, jitter basso, heartbeat regolari;

`network_degraded`: RTT aumentato o qualche heartbeat perso;

`network_unstable`: jitter alto, molti pacchetti persi;

`network_lost`: nessun messaggio per timeout prolungato.

Questa classificazione permette al sistema di adattarsi. In stato `network_good` può eseguire sequenze complesse; in `degraded` può aumentare margini temporali; in `unstable` può evitare azioni magnetiche coordinate; in `lost` deve entrare in fallback o safety. La comunicazione a bassa latenza diventa quindi anche una condizione di abilitazione comportamentale.

Un aspetto importante riguarda il carico del controller. Se il controller deve inviare molti comandi individuali in rapida successione, può diventare esso stesso fonte di ritardo. Per esempio, inviare un comando unicast separato a cinquanta nodi può richiedere più tempo rispetto a un singolo broadcast. Per comandi globali, il broadcast è più adatto. Per comandi specifici, l'unicast è necessario. Il sistema deve quindi scegliere la modalità di invio in base al tipo di azione e alla sensibilità temporale. La bassa latenza non dipende solo dalla rete, ma anche dalla strategia di invio.

Per azioni simultanee globali, il broadcast con `execute_at` è spesso la soluzione migliore. Il controller invia un unico messaggio a tutti; ogni nodo lo esegue al tempo stabilito. Per azioni differenziate ma sincronizzate, il controller può inviare in anticipo comandi individuali con lo stesso `execute_at` o con slot temporali diversi. In questo modo, anche se l'invio richiede tempo, l'esecuzione resta coordinata. Questo principio sarà fondamentale per la Sezione 9 sui slot temporali.

La bassa latenza richiede anche che il nodo processi i messaggi in una coda ordinata. Se arrivano molti pacchetti, il firmware deve decidere l'ordine di gestione. I messaggi critici devono avere priorità; i messaggi

vecchi possono essere scartati; i comandi duplicati non devono essere rieseguiti; i comandi scaduti devono essere ignorati. Una coda semplice ma ben progettata può migliorare molto la reattività. Senza coda, il nodo potrebbe processare messaggi nell'ordine di arrivo senza distinguere importanza, rischiando di ritardare comandi urgenti.

Una logica di coda può prevedere:

prima emergency e safety;

poi sync e comandi temporizzati imminenti;

poi comandi operativi normali;

poi diagnostica e debug;

infine telemetria non urgente.

Questa gerarchia rende il nodo più robusto. Anche se riceve molti messaggi di debug, un comando di stop deve passare davanti. Questa è una scelta architetturale fondamentale.

La comunicazione a bassa latenza deve essere progettata considerando anche il ciclo di controllo del sistema. Se il controller calcola nuovi comandi a una certa frequenza, la rete deve essere abbastanza rapida da trasportarli prima che diventino obsoleti. Se il controller invia aggiornamenti ogni 20 ms ma la latenza media è 100 ms, i nodi riceveranno comandi già vecchi. In questo caso, bisogna ridurre la frequenza di controllo, inviare comandi predittivi o cambiare architettura. La frequenza di comando deve essere compatibile con latenza e jitter reali.

Una regola generale è:

$\text{periodo_comando} > \text{latenza_massima_attesa} + \text{margine}$

Se i comandi sono più frequenti della capacità reale della rete, si accumulano, arrivano tardi o diventano inutili. Per questo motivo, la frequenza non deve essere scelta solo in base al desiderio di fluidità, ma in base a misure concrete. Il sistema deve essere progettato empiricamente: si misura la rete, poi si definiscono frequenze e margini.

Nel caso MicroBot, molte azioni non richiedono controllo ad altissima frequenza. Pattern, LED, heartbeat, stati e comandi magnetici temporizzati possono funzionare con messaggi relativamente brevi e ben pianificati. Questo è un vantaggio. Non serve trasformare il sistema iniziale in un controllo realtime rigido al millisecondo. Serve invece progettare un sistema in cui le azioni critiche sono temporizzate, i messaggi sono compatti, i nodi sono reattivi e la rete viene monitorata. Questa è una scelta pragmatica e coerente con lo sviluppo incrementale.

La bassa latenza ha anche un impatto sulla simulazione. Se i nodi simulati comunicano tramite UDP o WebSocket con il controller, si può testare la latenza software prima ancora dell'hardware. Si possono simulare ritardi artificiali, perdita di pacchetti e jitter per vedere come il protocollo reagisce. Questo è molto utile perché permette di validare la robustezza del sistema in condizioni peggiori rispetto al laboratorio. Una simulazione che funziona solo con latenza zero non è realistica. Inserire latenza controllata rende il modello più serio.

Un test di comunicazione simulata può introdurre:

ritardo medio;

jitter casuale;

perdita pacchetti;

duplicazione messaggi;

riordinamento pacchetti;

nodi che vanno offline.

Se il sistema gestisce correttamente queste condizioni, allora il protocollo è più robusto. Questo tipo di test sarebbe molto coerente con una documentazione avanzata e con un ecosistema MicroBot orientato a dimostrare solidità tecnica.

Dal punto di vista della telemetria, ogni nodo può riportare il tempo locale di ricezione e di esecuzione dei comandi. Il controller può confrontare il tempo previsto e il tempo effettivo, stimando l'errore di esecuzione. Questo permette di valutare se la sincronizzazione e la bassa latenza sono sufficienti. Per esempio, un comando potrebbe avere `execute_at = T`; il nodo risponde indicando `executed_at = T + errore`. Il controller può calcolare `execution_error`. Se l'errore è piccolo, la comunicazione e sincronizzazione sono buone; se è grande, bisogna aumentare margini o correggere il sistema.

Una metrica utile è:

`execution_error = executed_at - execute_at`

Questa metrica è più importante della latenza pura quando si lavora con comandi temporizzati. Anche se un pacchetto arriva con ritardo variabile, ciò che conta è se il nodo riesce a eseguire al tempo previsto. La dashboard può mostrare `execution_error` medio e massimo per nodo. Questo rende visibile la qualità del coordinamento.

Un altro punto importante è la gestione dei comandi obsoleti. In una comunicazione a bassa latenza, non tutti i messaggi vecchi devono essere recuperati. Per alcuni tipi di dati, il valore più recente sostituisce i precedenti. Ad esempio, se il controller invia aggiornamenti di colore LED o stato visuale molto frequenti, un pacchetto vecchio non deve essere eseguito dopo uno nuovo. Il nodo deve poter scartare messaggi superati usando `sequence_number` o `timestamp`. Questo evita che lo stato torni indietro.

Per comandi di stato, una regola può essere:

se `sequence_number < last_sequence_processed`, ignora il messaggio.

Per comandi temporizzati:

se `current_time > execute_at + tolerance_ms`, rifiuta il comando come expired.

Queste regole sono essenziali per mantenere coerenza. La bassa latenza non elimina la necessità di gestire messaggi vecchi; semplicemente riduce la probabilità che arrivino troppo tardi.

La comunicazione a bassa latenza deve essere collegata anche alla modalità broadcast. Il broadcast può ridurre il tempo necessario a raggiungere molti nodi con lo stesso comando, ma non consente di sapere automaticamente chi ha ricevuto. Per comandi globali non critici, questo va bene. Per comandi globali critici, il controller può inviare broadcast e poi aspettare ack dai nodi. Tuttavia, se tutti rispondono esattamente nello stesso istante, le risposte possono congestionare la rete. Una soluzione è distribuire gli ack in piccole finestre temporali o accettare ack solo da alcuni nodi in modalità normale, richiedendoli a tutti solo in diagnostica.

Questo mostra che anche il feedback può creare latenza e congestione. Non basta ottimizzare i comandi in uscita; bisogna progettare anche le risposte. In uno swarm numeroso, se ogni nodo risponde immediatamente a ogni broadcast, il controller può ricevere un picco di pacchetti. Per evitarlo, si può usare un piccolo ritardo casuale controllato negli ack o slot di risposta assegnati. Questa logica sarà utile soprattutto nella scalabilità.

Nel contesto dei comandi di emergenza, però, il comportamento cambia. Un `emergency_stop` deve essere eseguito immediatamente quando ricevuto; l'ack può essere secondario. Il nodo non deve ritardare

l'esecuzione per rispettare una finestra di risposta. Prima spegne le uscite attive, poi invia conferma se possibile. Questo ordine è essenziale:

ricevi emergency_stop;

esegui subito disattivazione locale;

poi invia ack.

La sicurezza viene prima dell'osservabilità. Anche se l'ack si perde, il comportamento sicuro deve essere stato eseguito localmente. Questa è una regola chiave.

La bassa latenza deve essere considerata anche rispetto alla distanza fisica e alla qualità del segnale Wi-Fi. Nodi più lontani dall'access point o schermati da ostacoli possono avere ritardi maggiori e più perdita di pacchetti. In un prototipo su tavolo, il problema può essere minimo; in una dimostrazione più grande, può diventare rilevante. Per questo motivo, la dashboard dovrebbe indicare almeno indirettamente la qualità comunicativa di ogni nodo attraverso RTT, heartbeat e packet loss. In versioni più avanzate, si potrebbe includere anche RSSI, cioè intensità del segnale Wi-Fi, se disponibile dal firmware.

Un nodo con segnale debole può essere marcato come comunicativamente fragile. Il controller può evitare di assegnargli ruoli critici o aumentare margini temporali. Questa è una forma di adattamento intelligente alla qualità della rete. In uno swarm maturo, non tutti i nodi devono essere trattati come identici se la loro comunicazione è diversa.

Dal punto di vista della progettazione sperimentale, la latenza deve essere documentata in tabelle di test. Ogni versione del sistema può registrare: numero di nodi, modalità rete, distanza dall'access point, dimensione pacchetti, frequenza heartbeat, RTT medio, jitter, packet loss e comportamento osservato. Queste tabelle trasformano la comunicazione da ipotesi a dato tecnico. Per una documentazione seria, è molto utile poter dire: con N nodi e pacchetti di dimensione X, il sistema mantiene una latenza media entro un certo intervallo e un jitter accettabile per la demo.

Una tabella di test potrebbe includere:

test_id;

numero_nodi;

modalità rete;

tipo messaggio;

dimensione media pacchetto;

frequenza invio;

RTT medio;

RTT massimo;

jitter;

pacchetti persi;

note.

Questi dati possono essere raccolti progressivamente. Anche se all'inizio i valori saranno empirici e semplici, la loro presenza aumenterà molto la solidità del progetto.

La comunicazione a bassa latenza non deve essere confusa con comunicazione ad alta frequenza indiscriminata. Inviare molti messaggi al secondo non significa necessariamente migliorare il controllo. Se i messaggi sono troppi, il sistema può peggiorare. La vera qualità sta nell'inviare il messaggio giusto, al momento giusto, con payload minimo, priorità corretta e validazione locale. MicroBot deve quindi privilegiare efficienza informativa, non rumore comunicativo. Un buon protocollo comunica ciò che serve, quando serve.

Questo principio è fondamentale per lo swarm. In un sistema distribuito, ogni informazione inutile moltiplicata per molti nodi diventa traffico significativo. Ogni byte, ogni pacchetto e ogni ack devono avere una funzione. La bassa latenza nasce anche da questa disciplina: ridurre ciò che non serve permette a ciò che serve di viaggiare meglio.

Dal punto di vista teorico, la latenza rappresenta il ritardo tra informazione e trasformazione fisica. Un comando nasce come informazione nel controller, attraversa la rete, viene interpretato dal MicroBot e diventa azione. Se questo passaggio è troppo lento, la forma fisica dello swarm può non corrispondere più allo stato logico desiderato. Si crea una distanza tra modello e realtà. La comunicazione a bassa latenza riduce questa distanza, mantenendo più stretto il legame tra intenzione del sistema e comportamento osservato.

La catena può essere descritta così:

intenzione → messaggio → ricezione → stato locale → azione → interazione → forma

La latenza agisce tra intenzione e azione. Più questo intervallo è controllato, più il sistema è coerente. Se l'intervallo è variabile e non misurato, il sistema diventa meno prevedibile. Per questo motivo, latenza e telemetria devono essere integrate: non basta ridurre la latenza, bisogna anche osservarla.

Nel contesto MicroBot, la comunicazione a bassa latenza deve essere vista come condizione abilitante, non come obiettivo isolato. L'obiettivo finale non è avere pacchetti velocissimi per sé stessi, ma permettere allo swarm di coordinarsi, riconfigurarsi, rispondere a comandi, proteggersi e mostrare uno stato coerente nella dashboard. La latenza è importante perché sostiene il comportamento collettivo. Un sistema con ottima latenza ma pessime regole locali non produce uno swarm utile; un sistema con buone regole ma latenza incontrollata può risultare incoerente. Serve equilibrio.

In conclusione, la comunicazione a bassa latenza è un requisito fondamentale del communication layer MicroBot. Essa consente comandi rapidi, feedback aggiornato, coordinamento temporale, sicurezza operativa e coerenza tra controller, nodi e dashboard. Tuttavia, la bassa latenza non dipende solo da UDP: dipende da messaggi compatti, firmware non bloccante, priorità dei pacchetti, frequenze controllate, timestamp, code di comando, misurazione di RTT e jitter, gestione dei messaggi obsoleti e strategie di esecuzione temporizzata. MicroBot deve quindi trattare la latenza come proprietà misurabile e progettuale, non come promessa generica. Il punto successivo, dedicato alla struttura dei pacchetti, definirà in modo più preciso il formato informativo dei messaggi, cioè il modo in cui comandi, identificatori, timestamp, payload e stati vengono organizzati dentro il protocollo di comunicazione.

8.5 Struttura dei pacchetti

La struttura dei pacchetti rappresenta il livello formale attraverso cui la comunicazione MicroBot diventa interpretabile, verificabile e scalabile. Dopo aver definito la rete Wi-Fi, il MicroBot come nodo di rete, il protocollo UDP e la necessità di comunicazione a bassa latenza, è necessario stabilire come devono essere organizzate le informazioni all'interno di ogni messaggio. Un pacchetto non è soltanto un blocco di dati inviato sulla rete: è una unità semantica del sistema, cioè un contenitore strutturato che permette a controller, nodi, simulazione e dashboard di condividere comandi, stati, eventi e informazioni temporali secondo una grammatica comune.

Nel progetto MicroBot, la struttura dei pacchetti è particolarmente importante perché il sistema deve collegare livelli diversi: interfaccia utente, controller centrale, nodi ESP32, simulatore, telemetria, safety layer e dashboard. Se ogni componente usasse messaggi costruiti in modo diverso, il sistema diventerebbe rapidamente fragile e difficile da mantenere. Una struttura uniforme dei pacchetti permette invece di creare un linguaggio comune. Il controller può generare comandi secondo uno schema prevedibile; il MicroBot può validarli; la dashboard può visualizzarli; i log possono registrarli; il simulatore può riprodurli. In questo senso, il pacchetto è il punto di incontro tra software, firmware e comportamento fisico.

Un pacchetto MicroBot deve contenere almeno tre categorie di informazioni: metadati, contenuto operativo e dati di validazione. I metadati descrivono il pacchetto stesso: versione del protocollo, identificatore del messaggio, sorgente, destinatario, timestamp e tipo di messaggio. Il contenuto operativo descrive ciò che il pacchetto vuole comunicare o comandare: comando, parametri, stato, telemetria, errore o evento. I dati di validazione permettono al ricevitore di decidere se il pacchetto è valido, recente, destinato al nodo corretto, compatibile con le capacità locali e sicuro da eseguire. Questa tripartizione rende il messaggio robusto e leggibile.

Una struttura concettuale generale può essere rappresentata così:

packet = header + body + validation

L'header contiene le informazioni necessarie a identificare e instradare il messaggio. Il body contiene il contenuto specifico del messaggio. La validation contiene eventuali campi utili per controllo, coerenza, sicurezza e gestione degli errori. Anche se nelle prime versioni questi tre blocchi possono essere inseriti nello stesso oggetto JSON senza separazione rigida, dal punto di vista documentale è utile distinguerli. Questa distinzione permette di capire quale parte del pacchetto serve alla rete, quale serve alla logica applicativa e quale serve alla robustezza del sistema.

Una possibile struttura iniziale in formato JSON può essere:

```
{
  "protocol": "microbot_udp",
  "version": "0.1",
  "message_id": "cmd_000001",
  "sequence": 1,
  "timestamp": 123456.78,
  "source": "controller",
  "target": "node_01",
  "type": "command",
  "command": "set_led",
  "payload": {
    "r": 0,
    "g": 180,
    "b": 255,
    "duration_ms": 500
  },
  "requires_ack": true
}
```

Questa struttura è leggibile e coerente con le prime fasi del progetto. Il campo protocol indica che il messaggio appartiene al sistema MicroBot. Il campo version permette di distinguere evoluzioni future. message_id identifica il messaggio. sequence fornisce un ordine logico. timestamp indica il tempo di generazione o riferimento. source indica il mittente. target indica il destinatario. type definisce la categoria del messaggio. command specifica l'azione richiesta. payload contiene i parametri. requires_ack indica se il mittente si aspetta una conferma. Anche un pacchetto semplice, quindi, contiene già abbastanza informazioni per essere tracciabile e verificabile.

Il campo protocol è utile perché impedisce al nodo di interpretare pacchetti casuali come messaggi validi. Se un MicroBot riceve sulla porta UDP un datagramma che non contiene protocol = microbot_udp, può ignorarlo immediatamente. Questo riduce il rischio di parsing errato e rende il sistema più robusto. In un ambiente di laboratorio, possono circolare altri pacchetti sulla rete; il nodo deve riconoscere soltanto quelli appartenenti al proprio protocollo applicativo. Il campo protocol è quindi una prima barriera logica.

Il campo version è fondamentale per il versionamento del protocollo. MicroBot è un progetto in evoluzione, e la comunicazione non resterà identica nel tempo. Nuovi comandi, nuovi payload, nuovi stati, nuove capacità e nuove regole di sicurezza potranno essere aggiunti nelle versioni successive. Senza un campo di versione, un nodo con firmware vecchio potrebbe ricevere un pacchetto nuovo e interpretarlo male. Con il versionamento, invece, il nodo può verificare se supporta quella versione del protocollo. Se non la supporta, può rispondere con unsupported_protocol_version o ignorare il messaggio in modo controllato.

Il campo `message_id` è uno dei più importanti dell'intera struttura. Ogni messaggio operativo deve essere identificabile in modo univoco o almeno univoco entro una finestra temporale. Questo permette di collegare comandi, ack, errori e log. Se il controller invia un comando con `message_id = cmd_000123`, il nodo può rispondere con un ack che contiene `ref_message_id = cmd_000123`. In questo modo, il controller sa esattamente quale comando è stato ricevuto, accettato, rifiutato o eseguito. Senza `message_id`, sarebbe difficile gestire conferme, ritrasmissioni e debugging.

Il `message_id` è essenziale anche per evitare duplicati. Se il controller ritrasmette un comando perché non ha ricevuto ack, il nodo potrebbe ricevere lo stesso comando due volte. Se il comando è idempotente, il problema può essere limitato; se il comando non è idempotente, come un impulso magnetico, ripeterlo due volte può produrre effetti reali diversi. Il nodo deve quindi memorizzare una piccola finestra degli ultimi `message_id` processati. Se riceve un ID già elaborato, non deve rieseguire il comando; può eventualmente reinviare l'ack precedente. Questa regola è una protezione indispensabile per un sistema che usa UDP.

Il campo `sequence` o `sequence_number` serve invece a ordinare logicamente i messaggi. Mentre `message_id` identifica un messaggio specifico, `sequence` permette di capire se un messaggio è più recente o più vecchio rispetto ad altri della stessa sorgente o della stessa categoria. Questo è utile quando arrivano pacchetti fuori ordine. Per esempio, se il controller invia `set_mode active` con `sequence 10` e poi `set_mode idle` con `sequence 11`, il nodo deve evitare di eseguire `sequence 10` dopo aver già accettato `sequence 11`. Il numero di sequenza permette di rifiutare comandi obsoleti.

Il campo `timestamp` ha un ruolo ancora più ampio. Esso permette di collocare il messaggio nel tempo. Può indicare il momento in cui il pacchetto è stato generato, il tempo logico del sistema, o il riferimento temporale per l'esecuzione. In una prima versione, `timestamp` può essere usato per debug, logging e rilevamento di messaggi vecchi. In una versione avanzata, può essere affiancato da `execute_at`, che indica il momento futuro in cui il nodo deve eseguire il comando. Questa distinzione è importante: `timestamp` dice quando il messaggio è nato; `execute_at` dice quando l'azione deve avvenire.

Una struttura per comandi temporizzati può quindi includere:

```
{
  "timestamp": 123456.78,
  "execute_at": 123457.50,
  "duration_ms": 200
}
```

In questo modo, il controller può inviare il comando con anticipo e i nodi possono eseguirlo in modo coordinato. Se il pacchetto arriva dopo `execute_at` più una tolleranza definita, il nodo può rifiutarlo come expired. Questo rende il sistema più robusto rispetto alla latenza variabile della rete Wi-Fi.

Il campo `source` identifica chi ha inviato il pacchetto. Può essere controller, dashboard, `node_01`, `simulation_engine`, `diagnostic_tool` o un altro componente del sistema. Questo campo è importante per logging e sicurezza. Un nodo può accettare comandi soltanto da sorgenti autorizzate, oppure può trattare diversamente pacchetti provenienti dal controller e pacchetti provenienti da strumenti diagnostici. Anche se nelle prime versioni la sicurezza può essere semplice, è utile prevedere il campo `source` fin dall'inizio. Esso rende il sistema più tracciabile.

Il campo `target` identifica il destinatario logico. Può essere un `node_id` specifico, all per broadcast, un `group_id` oppure controller per messaggi di ritorno. Un MicroBot che riceve un pacchetto deve verificare `target` prima di processarlo. Se `target` non corrisponde al proprio `node_id`, al proprio gruppo o a all, il pacchetto deve essere ignorato. Questa verifica è particolarmente importante nel caso broadcast, dove tutti i nodi ricevono lo stesso pacchetto ma solo alcuni potrebbero doverlo eseguire.

Il campo `type` definisce la categoria generale del messaggio. È uno dei campi più importanti per il dispatcher interno. Prima di leggere il payload, il nodo deve sapere che tipo di messaggio sta ricevendo. Una classificazione iniziale può includere:

command;

heartbeat;

telemetry;

ack;

error;

sync;

discovery;

event;

diagnostic.

Ogni tipo di messaggio segue una logica diversa. Un command richiede validazione e possibile esecuzione. Un heartbeat aggiorna la presenza del nodo. Una telemetry aggiorna dati di stato. Un ack conferma un messaggio precedente. Un error segnala un problema. Un sync serve ad allineare il tempo. Un discovery permette registrazione e identificazione dei nodi. Un event comunica una transizione importante, come aggancio o distacco. Un diagnostic trasporta dati di test o debug. Separare questi tipi rende il protocollo più ordinato e modulare.

Il campo command deve essere presente nei messaggi di tipo command. Esso indica l'azione richiesta. Il comando non deve essere una frase libera, ma un valore appartenente a un insieme definito. Questo permette al nodo di verificarlo facilmente. Una lista iniziale di comandi può includere:

ping;

set_mode;

set_led;

request_status;

assign_role;

start_pattern;

stop_pattern;

set_telemetry_level;

magnetic_pulse;

set_coil_pwm;

disable_outputs;

emergency_stop.

Non tutti i nodi devono supportare tutti i comandi. La compatibilità dipende dal campo capabilities dichiarato dal nodo in fase di discovery o heartbeat esteso. Se un comando non è supportato, il nodo deve rispondere con unsupported_command. Questo evita fallimenti silenziosi.

Il payload contiene i parametri specifici del messaggio. La struttura del payload dipende da type e command. Per set_led, il payload può contenere valori RGB e durata. Per magnetic_pulse, può contenere coil_id, duty_cycle, duration_ms e safety_policy. Per heartbeat, può contenere stato, batteria, safety e modalità. Per

telemetry, può contenere dati più estesi. La regola fondamentale è che il payload deve essere il più compatto possibile e deve avere campi chiaramente definiti.

Un payload per set_led può essere:

```
{
  "r": 0,
  "g": 180,
  "b": 255,
  "duration_ms": 500
}
```

Un payload per magnetic_pulse può essere:

```
{
  "coil_id": "main",
  "duty_cycle": 0.35,
  "duration_ms": 120,
  "max_thermal_load": 0.70
}
```

Un payload per set_mode può essere:

```
{
  "mode": "idle"
}
```

Questi esempi mostrano che ogni comando deve avere payload specifici e limitati. Un payload troppo generico o non standardizzato renderebbe difficile la validazione. Ogni comando deve avere una definizione chiara dei campi obbligatori, opzionali, dei tipi ammessi e dei limiti numerici.

La validazione del payload è una delle responsabilità più importanti del nodo. Se un comando richiede duration_ms e il campo manca, il nodo non deve eseguire il comando. Se duty_cycle è fuori intervallo, deve rifiutarlo o limitarlo secondo una policy chiara. Se coil_id non esiste, deve restituire errore. Questo è particolarmente importante per i comandi fisici. Un payload non valido non deve mai produrre attivazione di hardware. La regola deve essere semplice: se il messaggio non è validato completamente, non viene eseguito.

Una procedura di validazione può essere:

controlla protocol;

controlla version;

controlla dimensione massima;

controlla campi obbligatori;

controlla target;

controlla type;

controlla command;

controlla payload;

controlla timestamp;

controlla duplicati;

controlla capabilities;

controlla safety;

esegui oppure rifiuta.

Questa sequenza rende il nodo robusto. Anche se sembra lunga, può essere implementata in modo modulare. Ogni controllo riduce il rischio di comportamento non previsto.

Il campo `requires_ack` indica se il mittente si aspetta una conferma. Non tutti i messaggi devono richiedere ack. Heartbeat e telemetria periodica normalmente non ne hanno bisogno. Comandi critici invece sì. Il nodo può usare `requires_ack` per decidere se inviare una risposta. Tuttavia, per alcuni comandi il firmware dovrebbe inviare ack anche se il campo non è presente, perché la policy del comando lo richiede. Ad esempio, `emergency_stop` può generare ack se possibile, ma l'esecuzione non deve dipendere dall'ack. La struttura del protocollo può quindi combinare `requires_ack` e regole interne per tipo di comando.

Un pacchetto ack deve contenere almeno:

`ref_message_id`;

`result`;

`reason`;

`node_state` opzionale;

`timestamp`.

Il campo `result` può assumere valori come `received`, `accepted`, `executed`, `rejected`, `failed`. È importante distinguere `received` da `executed`. Un nodo può ricevere un comando ma non averlo ancora eseguito perché `execute_at` è futuro. Può accettarlo e metterlo in coda. Può eseguirlo e confermare dopo. Per comandi temporizzati, questa distinzione è fondamentale. Il controller deve sapere se il comando è stato solo ricevuto o realmente completato.

Una struttura ack più ricca può essere:

```
{
  "ref_message_id": "cmd_000123",
  "result": "accepted",
  "reason": "scheduled",
  "scheduled_for": 123457.50
}
```

Dopo l'esecuzione, il nodo potrebbe inviare un secondo messaggio:

```
{
  "ref_message_id": "cmd_000123",
  "result": "executed",
  "executed_at": 123457.51,
  "execution_error_ms": 10
}
```

Questo approccio è molto utile per comandi sincronizzati. Permette di verificare se il nodo ha rispettato il tempo previsto.

I pacchetti error devono essere progettati con la stessa precisione. Un errore non deve essere una semplice stringa generica, ma deve avere codice e dettaglio. Il codice permette al controller di reagire automaticamente; il dettaglio aiuta il debug umano. Un pacchetto error può contenere:

ref_message_id;

error_code;

detail;

severity;

current_state;

timestamp.

error_code può essere invalid_format, missing_field, wrong_target, unsupported_command, unsupported_version, safety_rejected, command_expired, duplicate_message, payload_out_of_range, busy, low_energy, thermal_lock. La severity può essere info, warning, error, critical. Questo consente alla dashboard di distinguere errori lievi da problemi seri.

I pacchetti heartbeat devono essere molto compatti. Essi servono a segnalare presenza, non a inviare tutta la telemetria. Un heartbeat può contenere:

node_id;

status;

mode;

battery;

safety;

sequence;

timestamp.

Il controller usa questi dati per aggiornare last_seen e stato base. Se il nodo deve inviare dati più estesi, può usare un pacchetto telemetry. Questa separazione evita di appesantire heartbeat e riduce traffico.

Un pacchetto telemetry può invece includere dati più dettagliati:

battery_voltage;

temperature_estimate;

thermal_load;

coil_state;

led_state;

connection_count;

network_quality;

last_command_id;

error_count.

La frequenza dei pacchetti telemetry può essere configurabile. In modalità normale, può essere moderata; in debug può aumentare; in trace può diventare molto dettagliata ma solo per test. Questa distinzione è essenziale per non saturare la rete.

I pacchetti discovery servono a registrare nodi. Un discovery_response può contenere:

node_id;

firmware_version;

hardware_version;

protocol_versions_supported;

capabilities;

boot_count;

ip_address opzionale;

status;

timestamp.

Questo pacchetto permette al controller di costruire la tabella dei nodi. Il campo boot_count o boot_id aiuta a rilevare riavvii. Se un nodo già noto si registra con boot_count cambiato, il controller può sapere che lo stato precedente potrebbe non essere più valido. Questo è molto importante per comandi pendenti, connessioni e sincronizzazione.

I pacchetti sync servono a sostenere la sincronizzazione temporale. Essi possono contenere:

controller_time;

sync_id;

round_trip_measurement opzionale;

valid_until;

timestamp.

Questi pacchetti saranno approfonditi nella Sezione 9, ma nella struttura generale devono essere previsti. Un sistema distribuito ha bisogno di messaggi temporali espliciti. Anche se la sincronizzazione iniziale sarà semplice, il protocollo deve poter crescere verso comandi temporizzati e slot.

I pacchetti event permettono ai nodi o al simulatore di comunicare transizioni rilevanti. Un evento può essere attach_event, detach_event, safety_triggered, mode_changed, command_executed, collision_warning, network_degraded, thermal_limit_reached. Questo tipo di pacchetto è utile per log e dashboard. La telemetria descrive uno stato; l'evento descrive qualcosa che è accaduto. Distinguere stato ed evento è importante.

Un pacchetto event può essere:

```
{
  "type": "event",
  "event": "attach_event",
  "source": "node_03",
  "payload": {
    "other_node": "node_04",
    "distance": 0.018,
    "connection_state": "attached"
  }
}
```

In una simulazione, questo evento può essere generato dal motore software; in un nodo fisico, potrebbe essere generato da sensori o da logica di stato. Anche se nelle prime versioni non tutti gli eventi saranno fisicamente rilevabili, la struttura permette di integrarli progressivamente.

Dal punto di vista del formato, JSON è adatto per la fase iniziale perché è leggibile e facile da integrare con Python, JavaScript, dashboard e strumenti di debug. Tuttavia, JSON presenta overhead. Ogni campo ha nome testuale, i numeri sono rappresentati come testo e il parsing può essere più pesante su microcontrollori. Per questo motivo, il protocollo può prevedere una fase futura con formato compatto, per esempio CBOR, MessagePack o un formato binario custom. Nella documentazione, è corretto indicare che JSON è il formato di prototipazione e debug, mentre formati più compatti possono essere considerati per versioni successive.

La scelta può essere descritta così:

versione iniziale: JSON leggibile, utile per debug e sviluppo rapido;

versione intermedia: JSON ottimizzato con payload compatti;

versione avanzata: formato binario o semi-binario per scalabilità e riduzione della latenza.

Questa progressione è coerente con lo sviluppo incrementale del progetto. Non serve ottimizzare prematuramente, ma è importante sapere che la struttura dei pacchetti deve poter evolvere.

La dimensione massima del pacchetto deve essere definita. Un microcontrollore non deve accettare pacchetti arbitrariamente grandi. Un pacchetto troppo grande può occupare memoria, rallentare parsing o creare problemi di buffer. Il protocollo deve quindi stabilire un limite massimo. Se un messaggio supera il limite, il nodo lo rifiuta. Il limite può cambiare tra versioni, ma deve essere documentato. Per comandi operativi, è preferibile restare molto sotto il limite massimo di UDP, evitando frammentazione IP.

Un principio utile è:

comandi critici = payload minimo;

stati periodici = payload compatto;

diagnostica = payload più ricco ma frequenza controllata;

snapshot completi = preferibilmente non via UDP realtime, ma tramite canale separato o file.

Questo principio evita di usare UDP per compiti non adatti. Il pacchetto UDP deve rimanere leggero.

La struttura dei pacchetti deve essere coerente anche con la sicurezza locale. Un pacchetto non deve mai poter aggirare il safety layer. Anche se contiene un comando formalmente valido, il nodo deve verificarlo contro limiti locali. Per esempio, se un pacchetto richiede `duty_cycle = 0.95` per una bobina ma il limite sicuro del nodo è 0.40, il nodo deve rifiutare o limitare il comando. Il payload non deve essere autorità assoluta; è una richiesta. L'agente locale decide se può eseguire.

Questa distinzione è centrale:

pacchetto = richiesta informativa;

firmware = interprete e validatore;

safety layer = autorità finale sulle azioni fisiche.

In un sistema fisico, il pacchetto non deve poter comandare direttamente hardware senza controllo. Questa architettura protegge il nodo anche in caso di errore del controller.

Ogni pacchetto dovrebbe essere registrabile nei log in forma essenziale. Non necessariamente tutto il contenuto deve essere salvato sempre, ma per debug è utile registrare message_id, type, source, target, result, timestamp e eventuale errore. Questo permette di ricostruire la storia della comunicazione. Se un comportamento dello swarm è errato, i log possono mostrare se i pacchetti sono arrivati, se sono stati rifiutati, se sono scaduti o se sono stati eseguiti. La struttura dei pacchetti deve quindi essere progettata pensando anche alla tracciabilità.

Un log minimale può contenere:

timestamp;

direction;

message_id;

source;

target;

type;

command;

result;

latency_or_delay;

error_code.

Questi campi permettono analisi successive senza salvare payload enormi. In modalità debug, il payload completo può essere salvato; in modalità normale, basta un riassunto. Questo bilancia osservabilità e volume dei log.

Dal punto di vista della dashboard, la struttura dei pacchetti deve permettere visualizzazioni chiare. La dashboard può mostrare l'ultimo heartbeat di ogni nodo, gli ultimi comandi inviati, gli ack ricevuti, gli errori, i messaggi pending e la latenza stimata. Per fare questo, ogni pacchetto deve avere campi standard. Se i messaggi fossero incoerenti, la dashboard diventerebbe difficile da costruire. La standardizzazione dei pacchetti è quindi anche una scelta di interfaccia.

Un pannello comunicativo può derivare direttamente dai campi:

node_id da source o target;

stato da heartbeat payload;

comandi da type command;

conferme da type ack;

errori da type error;

eventi da type event;

sincronizzazione da type sync.

Questa coerenza rende il sistema leggibile anche a chi lo osserva da fuori.

La struttura dei pacchetti deve essere compatibile con nodi fisici e simulati. Un nodo simulato può ricevere lo stesso pacchetto set_led e aggiornare un LED virtuale; un nodo fisico può aggiornare il LED reale. Un pacchetto magnetic_pulse può modificare una forza virtuale nel simulatore e un PWM reale nel firmware, se disponibile. Questo è possibile solo se la struttura del messaggio è astratta in modo corretto. Il comando deve descrivere l'intenzione operativa; l'implementazione decide come realizzarla.

Questa astrazione è molto potente. Permette al controller di essere indipendente dal tipo di nodo. Può inviare comandi a nodi simulati durante test e poi a nodi fisici durante demo. La differenza è dichiarata nelle capabilities e nell'hardware_profile. Questo rende MicroBot un ecosistema più solido.

Un elemento importante è la compatibilità retroattiva. Quando il protocollo evolve, i pacchetti vecchi non devono diventare immediatamente inutilizzabili se possibile. Una nuova versione può aggiungere campi opzionali senza rompere i nodi vecchi, purché questi ignorino campi sconosciuti in modo sicuro. Tuttavia, cambiamenti strutturali importanti devono aumentare la versione del protocollo. Una regola semplice è:

campi opzionali aggiunti = stessa major version;

cambiamenti incompatibili = nuova major version;

nodi vecchi devono ignorare campi non riconosciuti, ma rifiutare comandi non supportati.

Questa disciplina aiuta molto quando il progetto cresce.

La struttura dei pacchetti può essere definita formalmente tramite uno schema. Anche se nel documento principale basta la descrizione, in un file tecnico futuro si potrebbe creare un JSON Schema o una tabella dei campi. Lo schema definirebbe campi obbligatori, tipi, valori ammessi e limiti. Questo sarebbe utile sia per controller sia per firmware sia per simulatore. Il sistema potrebbe validare messaggi automaticamente prima dell'invio o dopo la ricezione.

Una tabella dei campi principali potrebbe essere descritta così:

protocol: stringa, obbligatoria, identifica il protocollo MicroBot;

version: stringa, obbligatoria, indica la versione del protocollo;

message_id: stringa, obbligatoria, identifica il messaggio;

sequence: numero intero, opzionale ma consigliato, ordina i messaggi;

timestamp: numero o stringa temporale, obbligatorio, indica il tempo di generazione;

source: stringa, obbligatoria, identifica il mittente;

target: stringa, obbligatoria, identifica destinatario o gruppo;

type: stringa, obbligatoria, definisce la categoria del messaggio;

command: stringa, obbligatoria solo per type command;

payload: oggetto, opzionale o obbligatorio secondo type;

requires_ack: booleano, opzionale, indica richiesta di conferma.

Questa tabella può essere espansa con campi specifici per sync, ack, error ed event. Il vantaggio di una definizione formale è che riduce ambiguità durante l'implementazione.

Il pacchetto deve essere progettato anche pensando al fallimento parziale. Se manca un campo non critico, il nodo può usare un valore predefinito. Se manca un campo critico, deve rifiutare il messaggio. Per esempio, se manca requires_ack, il nodo può assumere false o applicare la policy del comando. Se manca target, il pacchetto è invalido. Se manca command in un messaggio type command, è invalido. Se manca payload ma il comando non lo richiede, può essere accettato. Questa differenza tra campi critici e opzionali deve essere chiara.

Un comando emergency_stop, per esempio, dovrebbe richiedere pochi campi e non dipendere da payload complesso. Questo aumenta la probabilità che venga riconosciuto correttamente anche in condizioni difficili. Un comando diagnostico può invece richiedere payload più ricco e può essere rifiutato se incompleto senza conseguenze critiche. La struttura dei pacchetti deve quindi riflettere la priorità e la criticità del messaggio.

Dal punto di vista temporale, ogni pacchetto deve essere valutato rispetto alla sua freschezza. Un messaggio vecchio può essere pericoloso. Se un nodo riceve un comando set_mode o magnetic_pulse generato molto tempo prima, potrebbe non essere più coerente con lo stato corrente. Il campo timestamp consente di applicare una finestra di validità. Per esempio:

se $\text{current_time} - \text{timestamp} > \text{max_age}$, rifiuta il messaggio.

Per comandi temporizzati:

se $\text{current_time} > \text{execute_at} + \text{tolerance}$, rifiuta il messaggio.

Queste regole proteggono il sistema da ritardi e pacchetti obsoleti.

La struttura dei pacchetti deve prevedere anche priorità. Il campo priority può essere esplicito oppure derivato da type e command. Un campo esplicito può essere utile:

priority: "high", "normal", "low"

Tuttavia, bisogna evitare che un messaggio qualsiasi possa dichiararsi high e scavalcare controlli. La priorità deve essere validata in base al comando. emergency_stop è sempre high; telemetry debug è low; magnetic_pulse può essere normal o high secondo contesto ma sempre soggetto a safety. Il nodo può usare la priorità per ordinare la coda interna.

Una struttura estesa può quindi includere:

"priority": "high"

Questo campo non è obbligatorio nella prima versione, ma può essere previsto per evoluzioni future.

Un altro campo utile è ttl, cioè time to live applicativo. Indica per quanto tempo il messaggio resta valido. In alternativa, si può usare expires_at. Questo è utile per comandi che non devono essere eseguiti se arrivano tardi. Per esempio:

"expires_at": 123458.00

Il nodo confronta il proprio tempo con `expires_at` e rifiuta se il messaggio è scaduto. Questo rende il protocollo più sicuro in presenza di ritardi.

La struttura dei pacchetti può supportare anche `correlation_id`. Questo campo collega più messaggi appartenenti alla stessa operazione. Per esempio, l'avvio di un pattern può generare comandi verso più nodi, ack individuali, eventi e telemetria. Tutti questi messaggi possono condividere `correlation_id = pattern_run_001`. Questo rende più facile ricostruire una sequenza complessa nei log. Non è indispensabile nelle prime versioni, ma è molto utile in un ecosistema maturo.

Una struttura con `correlation_id` può essere:

```
{  
  "message_id": "cmd_000201",  
  "correlation_id": "pattern_circle_run_01",  
  ...  
}
```

La dashboard potrebbe raggruppare tutti gli eventi legati a quella run. Questo è molto utile per debug e documentazione sperimentale.

Dal punto di vista del sistema multi-agente, il pacchetto è il mezzo attraverso cui un agente riceve input esterni. Tuttavia, il pacchetto non determina da solo l'azione finale. L'agente integra il messaggio con il proprio stato locale, il vicinato, la safety e le regole. Questo significa che due nodi possono ricevere lo stesso comando broadcast ma reagire in modo diverso se hanno stati diversi. Per esempio, un comando `start_pattern` può essere accettato da nodi online e rifiutato da nodi in safety. La struttura del pacchetto deve quindi permettere risposte individuali.

Questa logica è molto importante per evitare una visione troppo centralizzata. Il controller invia intenzioni o comandi; i nodi validano localmente. Il pacchetto è la proposta informativa; il nodo è l'esecutore responsabile. Questa distinzione rende il sistema più robusto.

Nel contesto del safety layer, ogni pacchetto che richiede attivazione fisica deve essere trattato come potenzialmente critico. Il payload deve includere durata, intensità e limiti. Un comando senza durata massima per una bobina non dovrebbe essere accettato. Un comando senza `duty_cycle` valido non deve essere eseguito. Un comando che supera `thermal_load` massimo deve essere rifiutato. Questa politica può essere definita a livello di comando.

Per esempio, `magnetic_pulse` richiede:

`coil_id` obbligatorio;

`duty_cycle` obbligatorio e compreso tra 0 e limite locale;

`duration_ms` obbligatorio e inferiore a durata massima;

`timestamp` valido;

`message_id` valido;

`safety_ok` true;

`thermal_load` sotto soglia.

Se una di queste condizioni fallisce, il comando viene rifiutato. Questa rigidità è necessaria perché il pacchetto controlla un sottosistema energetico.

La struttura dei pacchetti deve essere compatibile anche con modalità manuale e automatica. In modalità manuale, l'utente o la dashboard può inviare comandi diretti a un nodo. In modalità automatica, il controller

genera pacchetti in base a pattern, simulazione o algoritmi. La struttura deve essere la stessa, così il nodo non deve distinguere troppo l'origine operativa. Può distinguere source, ma il formato resta comune. Questo semplifica il firmware.

Un comando manuale e uno automatico possono differire nel source:

```
source = dashboard_manual
```

```
source = pattern_controller
```

ma entrambi seguono lo stesso schema. Questa uniformità è un vantaggio architetturale.

Dal punto di vista delle versioni future, i pacchetti potrebbero includere firma o autenticazione. Nelle prime versioni, su rete locale controllata, può bastare validazione logica. In versioni più mature, soprattutto se il sistema viene mostrato in ambienti pubblici o con reti non isolate, potrebbe essere utile aggiungere un token, checksum applicativo, HMAC o meccanismo di autorizzazione. L'obiettivo non è appesantire ora il prototipo, ma prevedere che la struttura dei pacchetti possa includere un campo auth o signature in futuro.

Un'estensione possibile è:

```
"auth": {  
  "method": "token",  
  "value": "..."  
}
```

Oppure:

```
"signature": "..."
```

Per ora questi campi possono essere assenti, ma la struttura modulare permette di aggiungerli.

La struttura dei pacchetti deve infine essere documentata come contratto tra componenti. Il controller promette di inviare messaggi conformi allo schema. Il nodo promette di validarli ed eseguirli solo se sicuri. La dashboard promette di interpretarli per visualizzazione. Il simulatore promette di usarli per riprodurre comportamenti. Questo contratto riduce ambiguità e rende il progetto più professionale. In un sistema distribuito, un protocollo non documentato diventa rapidamente fonte di errori; un protocollo documentato diventa interfaccia stabile.

In conclusione, la struttura dei pacchetti costituisce la grammatica informativa del sistema MicroBot. Essa definisce come comandi, heartbeat, telemetria, ack, errori, sincronizzazione, discovery ed eventi vengono organizzati dentro messaggi coerenti e verificabili. Un pacchetto MicroBot deve includere campi di identificazione, versione, sorgente, destinatario, tipo, timestamp, contenuto operativo e, quando necessario, conferma, priorità e validazione. Questa struttura permette di usare UDP in modo robusto, compensando i suoi limiti con message_id, sequence number, timestamp, ack selettivi, controllo dei duplicati, gestione degli errori e safety locale. Il pacchetto diventa così il ponte tra intenzione del controller e azione del nodo: un'unità minima di comunicazione capace di modificare stato, comportamento e, indirettamente, forma dello swarm. Il punto successivo, dedicato all'identificatore, approfondirà il ruolo dell'ID del nodo e del messaggio come base per riconoscimento, tracciabilità, indirizzamento e gestione corretta delle relazioni comunicative nel sistema distribuito.

8.6 Identificatore

L'identificatore rappresenta uno degli elementi fondamentali della comunicazione MicroBot, perché consente al sistema di riconoscere in modo univoco nodi, messaggi, comandi, eventi, sessioni e relazioni all'interno dello swarm. Dopo aver definito la struttura dei pacchetti, diventa necessario approfondire il ruolo dell'ID come campo logico essenziale. In un sistema distribuito, infatti, non basta inviare dati: bisogna sapere chi li invia, chi li riceve, a quale messaggio si riferiscono, quale nodo deve eseguire il comando, quale risposta corrisponde a quale richiesta e quale evento appartiene a quale sequenza operativa. Senza identificatori stabili e coerenti, la comunicazione diventerebbe ambigua, difficile da debuggare e inadatta a uno swarm scalabile.

Nel progetto MicroBot, il concetto di identificatore deve essere applicato su più livelli. Esiste l'identificatore del nodo, che distingue un MicroBot dagli altri. Esiste l'identificatore del messaggio, che distingue un pacchetto da un altro. Esiste l'identificatore del comando, che permette di collegare una richiesta alla sua esecuzione. Esiste l'identificatore della sessione o del boot, che permette di distinguere due avvisi diversi dello stesso nodo. Esiste infine l'identificatore dell'evento, utile per registrare transizioni come agganci, distacchi, errori, timeout o cambi di stato. Questi ID lavorano insieme per creare tracciabilità.

Il primo identificatore fondamentale è il `node_id`. Ogni MicroBot deve possedere un identificatore logico stabile, indipendente dall'indirizzo IP e dalla configurazione temporanea della rete. L'indirizzo IP può cambiare, soprattutto se viene assegnato tramite DHCP o se il sistema viene collegato a reti diverse. Il `node_id`, invece, deve rappresentare l'identità del MicroBot nel progetto. Può essere espresso come `node_01`, `node_02`, `microbot_001`, `MB-001` oppure tramite un codice numerico. La forma precisa può essere scelta in fase implementativa, ma il principio deve rimanere chiaro: il `node_id` identifica il MicroBot come unità logica del sistema.

Una rappresentazione semplice può essere:

```
node_id = "MB-001"
```

Questo valore permette al controller di associare comandi, telemetria, log, ruolo, stato e cronologia sempre allo stesso nodo. Se il MicroBot cambia IP ma mantiene lo stesso `node_id`, il controller può aggiornare l'indirizzo senza perdere l'identità dell'unità. Questo è fondamentale per uno swarm fisico composto da più moduli, perché ogni nodo può avere caratteristiche, calibrazioni, capacità e limiti diversi. Un MicroBot con coil montata, per esempio, non deve essere confuso con un MicroBot LED-only. L'identificatore permette di mantenere questa distinzione.

Il `node_id` deve essere persistente. Questo significa che non dovrebbe essere generato casualmente a ogni avvio, perché in quel caso il controller vedrebbe lo stesso hardware come nodo diverso ogni volta. Una soluzione più corretta consiste nel salvarlo nel firmware, in una configurazione locale o in una memoria persistente. Nelle prime versioni, può essere definito manualmente nel codice del singolo ESP32. In versioni più mature, può essere impostato tramite file di configurazione, procedura di provisioning o interfaccia di setup. La cosa importante è che il `node_id` sia stabile nel tempo.

Accanto al `node_id` può esistere un identificatore hardware, come il MAC address del modulo Wi-Fi. Il MAC address è utile perché è legato all'interfaccia fisica del dispositivo. Tuttavia, non dovrebbe sostituire il `node_id`. Il MAC address può essere lungo, meno leggibile e più legato alla scheda che al ruolo progettuale del MicroBot. Il `node_id` è invece leggibile e significativo per la documentazione. La soluzione migliore è usare entrambi: il `node_id` per la logica del sistema e il MAC address come riferimento hardware secondario.

Una scheda nodo può quindi contenere:

```
node_id = "MB-001"
mac_address = "AA:BB:CC:DD:EE:FF"
ip_address = "192.168.1.42"
hardware_version = "microbot-node-v0.2"
firmware_version = "firmware-0.2.1"
```

Questa struttura permette di collegare identità logica, dispositivo fisico, indirizzo di rete e versione tecnica. In un progetto complesso come MicroBot, questa tracciabilità diventa molto importante perché nel tempo possono esistere più prototipi, più firmware, più configurazioni e più livelli di capacità.

Il secondo identificatore fondamentale è il `message_id`. Ogni pacchetto significativo inviato sulla rete dovrebbe avere un identificatore di messaggio. Questo ID permette di distinguere un pacchetto da tutti gli altri e di collegare eventuali risposte, errori o log al messaggio originale. Per esempio, se il controller invia un comando con `message_id = cmd_000245`, il MicroBot può rispondere con un ack contenente `ref_message_id = cmd_000245`. In questo modo, il controller sa esattamente quale comando è stato ricevuto o rifiutato.

Il `message_id` è particolarmente importante perché MicroBot usa, nelle prime versioni, un protocollo leggero come UDP. UDP non garantisce consegna, ordine o assenza di duplicati. Il `message_id` permette al livello applicativo di compensare parzialmente questi limiti. Se un messaggio viene ritrasmesso, il nodo può

riconoscere che ha già processato quel `message_id` e può evitare di eseguire due volte lo stesso comando. Questo è fondamentale per i comandi non idempotenti, come impulsi magnetici, attivazioni temporizzate o cambi di stato critici.

Una regola robusta è:

se `message_id` è già stato processato, non rieseguire il comando; se necessario, reinvia l'ultimo ack associato.

Questa regola impedisce che una ritrasmissione generi un doppio effetto fisico. Per esempio, se `magnetic_pulse` viene inviato e l'ack si perde, il controller potrebbe ritrasmettere lo stesso comando. Senza `message_id`, il nodo potrebbe eseguire un secondo impulso. Con `message_id`, il nodo riconosce che l'impulso è già stato eseguito o accettato e non lo ripete. Questa è una protezione essenziale quando la comunicazione controlla attuatori fisici.

Il `message_id` dovrebbe essere costruito in modo da essere univoco almeno per un certo intervallo di tempo. Può includere un prefisso che indica il tipo di messaggio, un contatore progressivo e magari il mittente. Per esempio:

```
cmd_controller_000001
hb_MB-001_000042
ack_MB-001_000001
err_MB-002_000005
evt_sim_000231
```

Questa forma rende l'ID leggibile anche nei log. Un identificatore completamente casuale può essere tecnicamente valido, ma meno comodo da analizzare manualmente. Per un progetto in fase di sviluppo, la leggibilità è utile. In futuro, si potranno usare UUID o ID compatti se necessario, ma nelle prime versioni una struttura progressiva e chiara è più adatta alla documentazione e al debug.

Il terzo identificatore importante è il `sequence_number`. Mentre `message_id` identifica il singolo messaggio, il `sequence_number` permette di capire l'ordine relativo dei messaggi inviati da una sorgente. Questo è utile soprattutto quando si ricevono pacchetti fuori ordine. Se il nodo riceve un comando con `sequence` 15 e poi un comando con `sequence` 14, può decidere di ignorare quello più vecchio. Questo impedisce che lo stato torni indietro per effetto di ritardi di rete.

Il `sequence_number` può essere globale per tutti i messaggi inviati dal controller oppure separato per categorie. Una sequenza globale è più semplice da implementare. Una sequenza per categoria può essere più precisa, perché evita che messaggi di tipo diverso interferiscano tra loro. Per esempio, i comandi `set_led` potrebbero avere una sequenza separata dai comandi `set_mode`. Nelle prime versioni può bastare una sequenza globale; nelle versioni avanzate si potrà introdurre una gestione più raffinata.

Un esempio di uso del `sequence_number` è:

```
ultimo comando di modalità accettato: sequence 31
nuovo comando ricevuto: sequence 30
risultato: comando ignorato perché obsoleto.
```

Questo comportamento è importante quando il controller aggiorna rapidamente lo stato dei nodi. Senza sequenza, un pacchetto vecchio potrebbe arrivare tardi e sovrascrivere una decisione più recente. Con la sequenza, il nodo mantiene coerenza temporale.

Il quarto identificatore è il `correlation_id`. Questo ID non identifica un singolo messaggio, ma un'intera operazione composta da più messaggi. Per esempio, l'avvio di un pattern può generare un comando broadcast, ack da vari nodi, eventi di esecuzione, telemetria e log finali. Tutti questi messaggi possono condividere lo stesso `correlation_id`, ad esempio `pattern_circle_run_001`. Questo permette al controller, alla dashboard e ai log di raggruppare tutto ciò che appartiene alla stessa sequenza operativa.

Il `correlation_id` è particolarmente utile per MicroBot perché molte operazioni non sono atomiche. Formare un pattern, sincronizzare uno swarm, avviare una sequenza magnetica o eseguire una fase di diagnostica

richiede più messaggi e più stati intermedi. Senza `correlation_id`, questi messaggi apparirebbero nei log come eventi separati. Con `correlation_id`, diventano parti di una storia unica. Questo migliora enormemente la leggibilità del sistema.

Un esempio concettuale:

```
correlation_id = "formation_line_run_004"
```

Messaggi collegati:

```
cmd_start_pattern
ack_node_01
ack_node_02
event_node_01_pattern_started
event_node_02_pattern_started
telemetry_pattern_progress
event_pattern_completed
```

Tutti questi messaggi possono essere analizzati insieme. Se la formazione fallisce, il sistema può ricostruire quale nodo non ha risposto, quale comando è scaduto, quale connessione non si è formata e quale evento ha generato l'errore. Questo è un salto di qualità nella diagnosi.

Il quinto identificatore è il `boot_id` o `session_id` del nodo. Un MicroBot può spegnersi, riavviarsi o resettarsi. Se mantiene lo stesso `node_id`, il controller lo riconosce come la stessa unità logica, ma deve anche sapere che è avvenuto un nuovo avvio. Il `boot_id` serve proprio a distinguere una sessione di esecuzione dalla precedente. Può essere un contatore di avvio, un timestamp di boot o un ID generato all'accensione. Questo campo è importante perché dopo un reboot il nodo può aver perso comandi pendenti, stato temporaneo, sincronizzazione, connessioni logiche e contatori.

Una rappresentazione può essere:

```
node_id = "MB-001"
boot_id = "boot_00017"
```

Se il controller vede `node_id` uguale ma `boot_id` cambiato, sa che il nodo è lo stesso dispositivo, ma la sessione firmware è nuova. Di conseguenza, può invalidare comandi pendenti, richiedere nuova sincronizzazione, aggiornare stato e segnalare reboot nella dashboard. Senza `boot_id`, il controller potrebbe credere che il nodo sia rimasto sempre nello stesso stato, generando errori logici.

Il `boot_id` è utile anche nei log. Se un errore avviene prima di un reset e un altro dopo, è importante distinguerli. Il log può mostrare che il nodo MB-001 ha avuto un reset tra due eventi. Questo aiuta a capire se un problema è causato da rete, firmware, alimentazione o watchdog.

Il sesto identificatore è il `command_id`, che può coincidere con `message_id` per comandi semplici, ma può essere distinto nei casi più complessi. Un comando può essere inviato in più messaggi, ritrasmesso, programmato o aggiornato. In questi casi, `command_id` identifica l'intenzione operativa, mentre `message_id` identifica il pacchetto specifico. Per le prime versioni, questa distinzione può non essere necessaria; tuttavia, dal punto di vista architetturale, è utile riconoscerla.

Per esempio:

```
command_id = "magpulse_MB-001_045"
message_id = "retry_packet_002"
```

Il comando logico è sempre lo stesso impulso magnetico, ma il pacchetto è una ritrasmissione. Questa distinzione diventa utile quando il protocollo cresce. Nelle prime versioni, si può usare `message_id` anche come `command_id`, mantenendo semplicità. In futuro, se ci saranno operazioni più complesse, si potrà separare.

Il settimo identificatore è l'event_id. Ogni evento rilevante generato dal nodo, dal controller o dal simulatore dovrebbe avere un ID. Gli eventi sono transizioni o fatti importanti: connessione stabilita, distacco, errore, safety trigger, perdita rete, cambio modalità, completamento comando, collision warning, soglia superata. L'event_id permette di registrare ogni evento in modo tracciabile e collegarlo eventualmente a un message_id, command_id o correlation_id.

Un evento può avere struttura:

```
event_id = "evt_MB-001_000098"  
event_type = "safety_triggered"  
source = "MB-001"  
timestamp = ...
```

Questo permette alla dashboard di mostrare una timeline chiara. Se un comando ha generato un errore, l'evento può contenere ref_message_id. Se l'evento appartiene a una run di pattern, può contenere correlation_id. Così la storia del sistema diventa navigabile.

L'ottavo identificatore riguarda le connessioni tra nodi. Quando due MicroBot si agganciano o vengono considerati connessi nel modello, può essere utile assegnare un connection_id. Questo ID rappresenta una relazione, non un nodo. Per esempio, una connessione tra MB-001 e MB-002 può essere identificata come:

```
connection_id = "conn_MB-001_MB-002_0007"
```

Questo permette di tracciare durata, stabilità, eventi di tensione, distacco, margine di hysteresis e telemetria associata alla connessione. Nel modello matematico della Sezione 7, le connessioni sono archi del grafo. L'identificatore di connessione consente di trasformare questi archi in oggetti tracciabili nei log e nella dashboard.

Una connessione può contenere:

```
connection_id;  
node_a;  
node_b;  
created_at;  
state;  
last_distance;  
stability_score;  
detach_margin;  
last_event_id.
```

Questo livello sarà particolarmente utile quando MicroBot passerà da semplici simulazioni a strutture più ricche. Sapere che due nodi sono connessi non basta; bisogna sapere da quanto, in quale stato, con quale stabilità e con quali eventi precedenti.

Il nono identificatore riguarda i gruppi. In uno swarm, può essere utile assegnare group_id a sottoinsiemi di MicroBot. Un gruppo può rappresentare un cluster, una formazione, una zona dello swarm o una categoria funzionale. Per esempio:

```
group_id = "cluster_A"  
group_id = "line_segment_01"  
group_id = "diagnostic_group"
```

Il group_id permette al controller di inviare comandi a un sottoinsieme di nodi senza specificarli uno per uno. Inoltre, permette alla dashboard di visualizzare lo swarm organizzato in gruppi. Il group_id non sostituisce il node_id: ogni nodo mantiene identità individuale, ma può appartenere a uno o più gruppi. Nelle prime versioni può esserci un solo gruppo per nodo; nelle versioni avanzate, si potrebbe prevedere appartenenza multipla.

Il decimo identificatore riguarda i pattern. Quando il sistema esegue una formazione o un comportamento, può essere utile usare pattern_id. Per esempio:

```
pattern_id = "line_basic_v0.1"  
pattern_id = "circle_formation_v0.2"  
pattern_id = "aggregation_demo_v0.3"
```

Il `pattern_id` consente di sapere quale comportamento è attivo, quale configurazione target è stata richiesta e quali parametri sono associati. Nei log, un comando `start_pattern` dovrebbe indicare `pattern_id`. La telemetria può riportare progressi relativi a quel pattern. Gli eventi possono essere collegati alla run specifica tramite `correlation_id`, ma `pattern_id` identifica il tipo di comportamento.

È utile distinguere `pattern_id` e `pattern_run_id`. Il primo indica il modello di pattern; il secondo indica una esecuzione specifica. Per esempio:

```
pattern_id = "circle_formation_v0.2"  
pattern_run_id = "circle_run_2026_04_30_001"
```

Questa distinzione permette di ripetere lo stesso pattern più volte e confrontare i risultati. Ogni run può avere metriche diverse: tempo di convergenza, errori, distacchi, collisioni, nodi offline. Questo è molto importante per la validazione sperimentale.

L'identificatore deve essere progettato con una convenzione coerente. Una convenzione chiara rende i log leggibili e riduce errori. Per esempio:

```
MB-001, MB-002 per nodi fisici;  
SIM-001, SIM-002 per nodi simulati;  
CTRL-001 per controller;  
DASH-001 per dashboard;  
CMD-000001 per comandi;  
ACK-000001 per conferme;  
ERR-000001 per errori;  
EVT-000001 per eventi;  
CONN-000001 per connessioni.
```

Questa convenzione aiuta molto quando il progetto cresce. Nei log, vedere MB-001 è più chiaro di un numero casuale. Vedere SIM-003 permette di distinguere subito un nodo simulato da un nodo reale. Vedere CMD, ACK, ERR o EVT nel prefisso aiuta a riconoscere il tipo di entità. La leggibilità degli ID è un vantaggio tecnico durante sviluppo e documentazione.

Tuttavia, bisogna evitare ID troppo lunghi nei pacchetti realtime se la rete diventa grande. Nelle prime versioni, leggibilità e debug sono prioritari. In versioni avanzate, si potrebbe usare una rappresentazione compatta interna, ad esempio numeri interi, mantenendo una mappa leggibile nel controller e nei log. Questa distinzione è importante: il sistema può usare ID umani nei file e ID compatti nella comunicazione ottimizzata. Per ora, in fase prototipale, gli ID leggibili sono la scelta più adatta.

Il `node_id` deve essere unico all'interno dello swarm attivo. Due nodi con lo stesso `node_id` generano ambiguità grave. Il controller potrebbe inviare un comando a MB-001 e non sapere quale dispositivo lo eseguirà. Per questo motivo, durante discovery o registrazione, il controller deve verificare duplicati. Se due dispositivi dichiarano lo stesso `node_id` ma hanno MAC address diversi o `boot_id` diversi, il controller deve segnalare `conflict_node_id`. In questo caso, può rifiutare uno dei due, mettere entrambi in quarantena logica o richiedere riconfigurazione.

Una regola possibile è:

se `node_id` duplicato con hardware diverso, non inviare comandi operativi finché il conflitto non è risolto.

Questo evita comportamenti pericolosi. L'identità è condizione fondamentale per il controllo.

Il `message_id` deve essere unico abbastanza a lungo da evitare confusione tra comandi recenti. Non è necessario che un contatore duri per sempre, ma non deve ripetersi entro una finestra in cui `ack`, `retry` e `log` sono ancora attivi. Il controller può usare un contatore crescente e, in caso di riavvio, includere `session_id`. Per esempio:

```
message_id = "CTRL-BOOT12-CMD-000045"
```

Così, anche se il contatore riparte dopo un reboot del controller, il boot del controller distingue i messaggi. Questa logica può essere utile in versioni avanzate. Nelle prime versioni, un contatore semplice può bastare.

Anche il controller dovrebbe avere un controller_id o source_id stabile. In una configurazione semplice c'è un solo controller, ma nel tempo potrebbero esserci dashboard, script diagnostici, simulatore, server web e strumenti diversi. Ogni sorgente di messaggi deve essere identificabile. Se un nodo riceve un comando da DASH-001 o da CTRL-001, può applicare policy diverse. Per esempio, solo il controller principale può inviare comandi magnetici; la dashboard può inviare richieste di stato; gli strumenti diagnostici possono essere abilitati solo in modalità test. L'identificatore della sorgente diventa quindi base per autorizzazione logica.

L'identificatore ha anche una funzione nella sicurezza. Anche senza crittografia avanzata, il nodo può rifiutare messaggi provenienti da source non riconosciute. Questo non protegge da attacchi sofisticati, ma riduce errori accidentali e pacchetti non previsti. In futuro, l'identificatore potrà essere combinato con token o firma. Per ora, è già utile come filtro semantico.

Una lista di sorgenti autorizzate può essere:

```
authorized_sources = ["CTRL-001", "DIAG-001"]
```

Se source non appartiene alla lista, il nodo ignora o segnala il pacchetto. Questo è un primo livello di controllo.

L'identificatore è cruciale anche per la telemetria. Quando il controller riceve dati, deve sapere da quale nodo provengono. Un pacchetto telemetry senza source o node_id sarebbe inutilizzabile. Se la dashboard mostra batteria, temperatura, safety o stato coil, deve associarli al nodo giusto. Questo vale anche per i nodi simulati. Ogni dato deve avere origine chiara. La telemetria senza identificazione è rumore; la telemetria identificata è conoscenza operativa.

Un pacchetto telemetry deve quindi contenere almeno:

```
source = node_id  
type = telemetry  
timestamp  
payload
```

Il payload può cambiare, ma source e timestamp devono sempre esserci. Il source identifica chi parla; il timestamp indica quando lo stato è stato misurato o inviato. Questi due campi sono la base della tracciabilità.

L'identificatore permette anche di ricostruire la storia di un singolo nodo. Nei log, si può filtrare per node_id e vedere tutti gli eventi relativi a quel MicroBot: boot, discovery, heartbeat, comandi ricevuti, errori, connessioni, distacchi, safety, telemetria. Questa capacità è molto utile durante lo sviluppo. Se MB-004 si comporta male, non serve leggere tutti i log dello swarm; si filtra MB-004 e si analizza la sua timeline. Senza identificatori coerenti, questo sarebbe impossibile.

Una timeline nodo può apparire concettualmente così:

```
MB-004 boot  
MB-004 discovery_response  
MB-004 heartbeat online  
MB-004 command accepted  
MB-004 event attached to MB-005  
MB-004 warning thermal_load_high  
MB-004 command rejected safety  
MB-004 heartbeat safe
```

Questa storia è chiara perché ogni evento è identificato e collegato allo stesso `node_id`. La documentazione tecnica del progetto può sfruttare questa struttura per mostrare esempi realistici di funzionamento.

L'identificatore è fondamentale anche nel grafo dello swarm. I nodi del grafo devono avere ID. Gli archi devono riferirsi a coppie di ID. La matrice di adiacenza $A(t)$, per essere utile in un sistema reale, deve essere collegata agli ID dei MicroBot. Non basta dire che il nodo 1 è connesso al nodo 2 se non si sa quali siano fisicamente. Il mapping tra indice matematico e `node_id` operativo deve essere chiaro.

Nel modello matematico si può usare m_i, m_j ; nel sistema software si usa `node_id`. Il collegamento è:

$m_i \leftrightarrow \text{node_id} = \text{MB-001}$
 $m_j \leftrightarrow \text{node_id} = \text{MB-002}$

Questa corrispondenza permette di collegare formula, simulazione, log e hardware. È un punto documentale molto importante: il modello matematico astratto deve poter essere mappato su entità reali.

L'identificatore deve essere coerente anche nei file di configurazione. Un file `initial_swarm_state.json`, ad esempio, dovrebbe usare gli stessi `node_id` che il controller e la dashboard riconoscono. Se il file usa nomi diversi, nasce confusione. Ogni nodo definito in configurazione dovrebbe avere un `node_id`, una posizione iniziale, un ruolo, capacità attese e stato iniziale. Questo rende possibile caricare scenari e confrontarli con nodi reali o simulati.

Un esempio concettuale:

```
{
  "node_id": "MB-001",
  "initial_position": { "x": 0.0, "y": 0.0, "z": 0.0 },
  "role": "leader",
  "expected_capabilities": ["led_rgb", "heartbeat", "coil_pwm"]
}
```

Questa struttura collega identificatore, modello spaziale, ruolo e capacità. In uno swarm simulato, il nodo può essere creato da questo record. In uno swarm fisico, il controller può verificare se il nodo reale MB-001 è online e possiede le capacità attese.

L'identificatore è importante anche per le capabilities. Se il controller riceve un comando da inviare a un nodo, deve controllare le capabilities associate a quel `node_id`. Per esempio, se MB-001 supporta solo LED e heartbeat, il controller non dovrebbe inviare `set_coil_pwm`. Se lo fa, il nodo lo rifiuta, ma è meglio che il controller prevenga l'errore. La tabella `node_id → capabilities` è quindi parte del sistema di controllo.

Una tabella interna può essere:

MB-001: led_rgb, heartbeat, telemetry_basic
MB-002: led_rgb, heartbeat, coil_pwm, safety_lock
SIM-001: led_rgb, simulated_magnetic_state, virtual_position

Questa tabella rende il sistema adattivo rispetto a nodi diversi. È molto utile in un progetto con prototipi progressivi.

L'identificatore deve gestire anche nodi sostituiti. Se un MicroBot fisico viene sostituito con un altro hardware, bisogna decidere se mantenere lo stesso `node_id` o assegnarne uno nuovo. Se il nuovo hardware prende il ruolo del vecchio nodo nella stessa posizione progettuale, si potrebbe mantenere `node_id` ma aggiornare `hardware_id` e note. Se invece è un nuovo prototipo, meglio assegnare un nuovo `node_id`. Questa scelta deve essere documentata per evitare confusione storica. In generale, il `node_id` dovrebbe identificare una unità logica/hardware specifica, non solo un ruolo temporaneo. Il ruolo può cambiare; l'identità del nodo dovrebbe restare.

Per ruoli temporanei esiste `role_id` o `role` field. Per identità fisica esiste `node_id`. Confondere ruolo e identità è pericoloso. Un nodo può essere leader oggi e follower domani, ma resta MB-001. Se si chiamasse sempre `leader_01`, il suo ID cambierebbe con il ruolo e i log perderebbero continuità. Quindi:

`node_id` = identità stabile;
`role` = funzione temporanea nel comportamento.

Questa distinzione deve essere mantenuta in tutto il sistema.

Anche i nodi simulati devono avere identificatori stabili durante una simulazione. Se una run crea SIM-001, SIM-002, SIM-003, questi ID devono rimanere coerenti negli snapshot. Se a ogni frame gli ID cambiassero, sarebbe impossibile tracciare movimento, velocità, connessioni e stato. In simulazione, il `node_id` permette di seguire la traiettoria del singolo agente nel tempo. Questo è fondamentale per analisi e replay.

Un replay di simulazione può ricostruire:

posizione di SIM-001 nel tempo;
connessioni di SIM-001;
eventi di SIM-001;
ruolo di SIM-001;
errori o stati di SIM-001.

Senza ID, il replay sarebbe solo una sequenza di punti anonimi. Con ID, diventa una storia multi-agente.

L'identificatore deve essere usato con coerenza nei pacchetti ack. Se un nodo risponde a un comando, l'ack deve includere `source = node_id` e `ref_message_id = message_id` del comando. Questo crea una catena chiara:

controller invia CMD-001 a MB-001;
MB-001 risponde ACK-001 con `ref_message_id` CMD-001;
controller aggiorna stato di CMD-001 come `accepted` o `executed`.

Questa catena è il cuore della comunicazione affidabile sopra UDP. Ogni richiesta ha una risposta collegata. Ogni risposta ha origine. Ogni origine è un nodo identificato.

Gli errori devono seguire lo stesso principio. Un errore senza riferimento è meno utile. Se un nodo rifiuta un comando, deve indicare quale comando ha rifiutato. Quindi:

`source = MB-001`
`type = error`
`ref_message_id = CMD-001`
`error_code = safety_rejected`

Questo permette al controller di aggiornare lo stato del comando e alla dashboard di mostrare un messaggio preciso: MB-001 ha rifiutato CMD-001 per `safety_rejected`. Non è un errore generico; è un evento tracciabile.

L'identificatore permette anche gestione dei comandi pendenti. Il controller mantiene una lista di comandi inviati ma non ancora confermati. Ogni comando è indicizzato da `message_id`. Quando arriva un ack con `ref_message_id`, il controller trova il comando pendente corrispondente e lo aggiorna. Se non arriva ack entro `timeout`, può ritrasmettere o segnalare errore. Questa logica non sarebbe possibile senza ID.

Una tabella pending può essere:

`message_id`;
`target_node`;
`command`;
`sent_at`;
`timeout_at`;

```
retry_count;  
status.
```

Il message_id è la chiave. È il punto che collega invio, attesa, ritrasmissione e conferma.

L'identificatore contribuisce anche alla gestione della scalabilità. In uno swarm piccolo, si potrebbe controllare manualmente ogni nodo. In uno swarm grande, serve automazione. Il controller deve poter filtrare per node_id, group_id, role, status, capabilities. Gli ID sono la base di questi filtri. Senza una struttura chiara, la gestione di molti nodi diventerebbe ingestibile. Con ID coerenti, si possono inviare comandi mirati, creare gruppi, analizzare subset e diagnosticare problemi.

Per esempio:

```
invia comando a tutti i nodi con role = boundary;  
richiedi telemetria a tutti i nodi con capability = coil_pwm;  
metti in safety tutti i nodi del group_id = cluster_A;  
analizza errori solo dei nodi MB-010—MB-020.
```

Queste operazioni richiedono identità e attributi ben definiti.

L'identificatore deve essere pensato anche per l'interfaccia utente. La dashboard non deve mostrare solo indirizzi IP o numeri casuali, ma nomi leggibili. Vedere MB-001 online, MB-002 stale, MB-003 safety è molto più chiaro. L'utente può associare il nome al dispositivo fisico, magari tramite etichetta sul prototipo. In una demo, ogni MicroBot potrebbe avere una piccola etichetta o colore associato al suo ID. Questo rende il debug fisico molto più semplice.

Un LED può persino aiutare a identificare un nodo. Il controller può inviare identify_node a MB-004, e il nodo lampeggia in un pattern specifico. Questo comando è molto utile quando si hanno più dispositivi uguali sul tavolo. L'identificatore logico viene collegato al dispositivo fisico visibile.

Un comando identify_node può avere payload:

```
duration_ms;  
blink_pattern;  
color.
```

Il nodo lampeggia e l'utente capisce quale modulo è MB-004. Questo è un dettaglio pratico ma molto utile in laboratorio.

Dal punto di vista della documentazione dei prototipi, ogni MicroBot fisico dovrebbe avere una scheda associata al node_id. La scheda può contenere hardware, firmware, componenti, limiti, problemi noti, data di test, note di calibrazione, foto e ruolo nei test. Questo collega identificatore e documentazione sperimentale. Nel tempo, se il progetto cresce, questa organizzazione evita di perdere memoria tecnica.

Una scheda può essere:

```
Node ID: MB-001  
Hardware Version: v0.2 LED + coil test  
Firmware Version: 0.2.1  
Capabilities: led_rgb, heartbeat, coil_pwm  
Known Limits: duty cycle massimo 35% nei test iniziali  
Last Test: data  
Notes: coil montata su estremità anteriore
```

Questa scheda rende ogni nodo un oggetto tecnico tracciato, non un componente anonimo.

L'identificatore ha anche un ruolo nella calibrazione. Due MicroBot possono avere leggere differenze hardware: una coil con resistenza diversa, un LED più luminoso, una batteria più degradata, un magnete più forte. Il controller può associare dati di calibrazione al node_id. Per esempio, MB-002 può avere un

coefficiente magnetic_gain diverso da MB-003. In simulazione, questi coefficienti possono essere usati per rendere i nodi più realistici. Nel firmware, possono regolare limiti locali.

Una configurazione di calibrazione può includere:

```
node_id;  
coil_resistance;  
max_safe_duty;  
battery_profile;  
magnetic_gain;  
led_correction;  
temperature_offset.
```

Questo livello diventerà importante quando il progetto passerà da demo uniforme a prototipi fisici reali, dove le differenze tra unità contano.

L'identificatore deve essere gestito anche nei casi di errore. Se un nodo invia un messaggio senza node_id valido, il controller deve trattarlo come unknown_node. Può rispondere con richiesta di discovery o ignorarlo. Se un nodo dichiara un node_id non registrato, il controller può aggiungerlo come nuovo nodo oppure metterlo in pending_registration. Se un nodo cambia improvvisamente node_id durante la stessa sessione, può esserci errore firmware o conflitto di configurazione. La gestione di questi casi evita confusione.

Una policy può essere:

```
node_id mancante → rifiuta pacchetto;  
node_id sconosciuto → richiedi discovery;  
node_id duplicato → segnala conflitto;  
node_id cambiato con stesso MAC → richiedi riconfigurazione.
```

Queste regole rendono il sistema più robusto.

Nel contesto della sicurezza, gli identificatori non devono essere considerati prova assoluta di autenticità. Un dispositivo potrebbe dichiarare un node_id falso se la rete non è protetta. Per un prototipo locale, il rischio può essere accettabile; per versioni future, bisognerà aggiungere autenticazione. L'identificatore è necessario per riconoscere, ma non sufficiente per garantire sicurezza crittografica. Questa distinzione è importante e tecnicamente corretta. Il node_id dice chi il pacchetto afferma di essere; l'autenticazione futura potrà verificare se lo è davvero.

In una fase avanzata, si potrà associare a ogni node_id una chiave, un token o una firma. Per ora, il sistema può operare in rete locale controllata, ma la struttura degli ID deve permettere estensioni. Questo mantiene il progetto aperto verso scenari più maturi.

L'identificatore è importante anche per la sincronizzazione. Quando il controller invia messaggi di sync, deve sapere quali nodi hanno risposto e quali no. Ogni risposta deve essere associata al node_id. Se MB-001 ha offset temporale basso e MB-007 ha offset alto, il controller può trattarli diversamente. Senza identificatore, la sincronizzazione sarebbe anonima e inutilizzabile. Nella Sezione 9 questo diventerà ancora più evidente.

Un pacchetto sync_response può contenere:

```
source = MB-001;  
sync_id;  
local_receive_time;  
local_send_time;  
ref_message_id.
```

Questi campi permettono al controller di stimare offset e ritardi per quel nodo specifico. Il node_id è quindi base del profilo temporale individuale.

L'identificatore permette anche di collegare comunicazione e modello matematico. Nel modello, ogni nodo i ha posizione $p_i(t)$, stato $s_i(t)$, vicinato $N_i(t)$. Nell'implementazione, questi valori devono essere associati a `node_id`. Un record di stato può essere:

```
node_id = MB-001
position = p_MB-001(t)
state = s_MB-001(t)
neighbors = [MB-002, MB-003]
```

Questo rende il modello operativo. Non si parla più di nodi astratti, ma di unità identificabili. La matematica diventa software e telemetria.

Anche il vicinato può essere espresso tramite ID. Invece di dire che il nodo i ha vicini j e k , il sistema può registrare:

```
neighbors_MB-001 = [MB-002, MB-005, MB-008]
```

Questo è leggibile e utile nei log. Se MB-001 perde un vicino, l'evento può indicare quale. Se forma una connessione, il `connection_id` può specificare i due `node_id`. Tutto diventa tracciabile.

Nel contesto dei file snapshot, gli ID sono indispensabili. Uno snapshot dello swarm deve contenere una lista di nodi con ID e stato. Se lo snapshot non mantiene ID stabili, non si può confrontare un frame con il successivo. Ogni snapshot dovrebbe quindi salvare:

```
node_id;
position;
orientation;
velocity;
state;
connections;
timestamp.
```

Le connessioni dovrebbero riferirsi a `node_id`. Questo permette replay, analisi e debug. Un sistema MicroBot maturo dovrebbe poter registrare una run e poi riprodurla, mostrando l'evoluzione di ogni nodo identificato.

L'identificatore è anche essenziale per confrontare simulazione e hardware. Se un nodo reale MB-001 viene rappresentato nella dashboard da un avatar virtuale, quell'avatar deve usare lo stesso ID. Se il nodo invia telemetria, la dashboard aggiorna il suo avatar. Se il simulatore prevede una posizione target per MB-001, il controller può inviare comando al nodo reale MB-001. Questa corrispondenza unifica mondo virtuale e mondo fisico.

Una mappa può essere:

```
hardware_node MB-001 ↔ simulation_entity MB-001 ↔ dashboard_object MB-001
```

Questa mappa è una delle basi dell'ecosistema MicroBot. Permette di trattare fisico, simulato e visuale come viste diverse della stessa entità logica.

In conclusione, l'identificatore è una componente strutturale della comunicazione MicroBot. Esso permette di riconoscere nodi, messaggi, comandi, eventi, connessioni, gruppi, sessioni e pattern. Il `node_id` fornisce identità stabile ai MicroBot; il `message_id` collega pacchetti, ack ed errori; il `sequence_number` protegge dall'arrivo fuori ordine; il `correlation_id` raggruppa operazioni complesse; il `boot_id` distingue sessioni diverse; il `connection_id` rende tracciabili le relazioni dello swarm. Senza identificatori, il sistema sarebbe un flusso ambiguo di dati; con identificatori coerenti, diventa una rete osservabile, debuggabile e scalabile.

L'identificatore è quindi la base della tracciabilità del sistema distribuito: permette al controller di sapere chi parla, chi ascolta, cosa è stato richiesto, cosa è stato eseguito, cosa è fallito e come ogni nodo partecipa alla forma collettiva. Il punto successivo, dedicato al comando, analizzerà il contenuto operativo dei pacchetti, cioè il modo in cui un messaggio identificato diventa istruzione concreta per modificare stato, comportamento, LED, modalità, pattern o attuazione magnetica del MicroBot.

8.7 Comando

Il comando rappresenta il contenuto operativo principale della comunicazione MicroBot, cioè l'elemento del pacchetto che indica al nodo quale azione deve valutare, accettare, programmare o rifiutare. Dopo aver definito la struttura dei pacchetti e il ruolo degli identificatori, diventa necessario analizzare il campo che trasforma un messaggio da semplice informazione di rete a istruzione funzionale. Un pacchetto può contenere sorgente, destinatario, identificatore, timestamp e metadati, ma è il comando che definisce l'intenzione concreta del controller: cambiare modalità, accendere un LED, richiedere telemetria, assegnare un ruolo, avviare un pattern, attivare un impulso magnetico, interrompere un'azione o portare il nodo in stato di sicurezza.

Nel sistema MicroBot, il comando non deve essere interpretato come un ordine cieco ed eseguito automaticamente dal nodo. Questa distinzione è fondamentale. Il comando è una richiesta strutturata inviata dal controller o da un altro componente autorizzato; il MicroBot ricevente deve validarla rispetto al proprio stato, alle proprie capacità, ai limiti energetici, al safety layer e al contesto operativo. In altre parole, il comando è l'intenzione del sistema centrale, ma l'esecuzione finale dipende dalla logica locale del nodo. Questa scelta rende il sistema più robusto, perché impedisce che un pacchetto valido dal punto di vista sintattico produca un comportamento non sicuro dal punto di vista fisico.

Una rappresentazione generale del comando può essere:

`command = azione_richiesta + parametri + vincoli + politica_di_esecuzione`

Questa formulazione mostra che un comando non dovrebbe limitarsi al nome dell'azione. Deve includere anche i parametri necessari, i limiti temporali, l'eventuale richiesta di conferma, la priorità, la durata, la modalità di fallback e le condizioni di validità. Per esempio, un comando che accende un LED può essere relativamente semplice; un comando che attiva una bobina deve invece contenere durata, duty cycle, limite termico e condizioni di sicurezza. Ogni comando deve essere proporzionato al rischio e alla complessità dell'azione che richiede.

Dal punto di vista del pacchetto, il comando può essere espresso tramite un campo testuale normalizzato, per esempio:

`"command": "set_led"`

oppure:

`"command": "magnetic_pulse"`

Il valore del campo `command` deve appartenere a un insieme definito dal protocollo. Non deve essere una frase libera, perché il firmware deve poterlo confrontare in modo preciso con i comandi supportati. Ogni comando deve avere una definizione tecnica: direzione del messaggio, payload richiesto, campi obbligatori, campi opzionali, condizioni di validità, risposta attesa, comportamento in caso di errore e relazione con il safety layer. Questa formalizzazione è ciò che trasforma il comando in una vera interfaccia di controllo.

Nel progetto MicroBot, i comandi possono essere classificati in più famiglie. La prima famiglia comprende i comandi di presenza e diagnostica, come `ping`, `request_status` e `identify_node`. La seconda famiglia comprende i comandi di stato, come `set_mode`, `enter_idle`, `enter_diagnostic` o `enter_safety`. La terza famiglia comprende i comandi visuali, come `set_led` o `set_led_pattern`. La quarta famiglia comprende i comandi comportamentali, come `assign_role`, `start_pattern`, `stop_pattern`, `set_behavior_parameters`. La quinta famiglia comprende i comandi magnetici o attuativi, come `magnetic_pulse`, `set_coil_pwm`, `disable_outputs`. La sesta famiglia comprende i comandi critici di sicurezza, come `emergency_stop` o `stop_all`. Questa classificazione è utile perché non tutti i comandi hanno la stessa priorità, lo stesso rischio o la stessa necessità di conferma.

I comandi di diagnostica sono generalmente a basso rischio. Un `ping` serve a verificare che il nodo sia raggiungibile. Un `request_status` chiede al nodo di inviare il proprio stato. Un `identify_node` può far lampeggiare un LED per permettere all'utente di riconoscere fisicamente quale modulo corrisponde a un certo `node_id`. Questi comandi sono importanti durante lo sviluppo perché permettono di verificare rete, identità e configurazione senza coinvolgere attuatori fisici complessi. Possono richiedere `ack`, ma non hanno lo stesso livello di criticità di un comando magnetico.

Un comando ping può avere una struttura minimale:

```
{
  "type": "command",
  "command": "ping",
  "target": "MB-001",
  "message_id": "CMD-000001",
  "timestamp": 123456.78
}
```

La risposta del nodo può essere un ack o un pong contenente node_id, stato e timestamp. Questo comando è utile per misurare latenza, verificare presenza e testare la comunicazione. Pur essendo semplice, introduce già la logica fondamentale del sistema: il controller invia una richiesta identificata, il nodo risponde con un riferimento al messaggio originale e il controller aggiorna lo stato del nodo.

Il comando request_status è leggermente più ricco, perché chiede al MicroBot di inviare informazioni sul proprio stato. Il payload può specificare il livello di dettaglio richiesto:

```
{
  "command": "request_status",
  "payload": {
    "level": "basic"
  }
}
```

Il livello basic può restituire stato, modalità, batteria stimata e safety. Il livello normal può includere anche rete, ultimo comando, LED e stato delle uscite. Il livello debug può includere contatori, errori, carico termico stimato, pacchetti ricevuti e dati diagnostici. Questa struttura evita di inviare sempre telemetria pesante e permette al controller di richiedere dettagli solo quando servono.

Il comando identify_node ha un valore pratico molto alto nei prototipi fisici. Quando sul tavolo sono presenti più MicroBot simili, può essere difficile capire quale sia MB-001, MB-002 o MB-003. Il controller può inviare identify_node a un nodo specifico, e il nodo può lampeggiare con un colore o pattern. Questo collega identificatore logico e dispositivo fisico. Il comando può avere payload:

```
{
  "command": "identify_node",
  "payload": {
    "color": "cyan",
    "duration_ms": 3000,
    "pattern": "blink"
  }
}
```

Questo comando è sicuro se limitato a LED e durata massima. Anche in questo caso, il nodo deve validare duration_ms per evitare lampeggi indefiniti non desiderati. Il comando mostra un principio generale: anche operazioni semplici devono avere limiti temporali chiari.

I comandi di stato modificano la modalità operativa del nodo. Un MicroBot può essere in idle, active, diagnostic, pattern, safety, error, low_energy o altri stati definiti dal firmware. Il comando set_mode permette al controller di chiedere il passaggio a una modalità. Tuttavia, il nodo non deve accettare qualsiasi transizione. Alcuni passaggi possono essere vietati. Per esempio, un nodo in safety non dovrebbe uscire da safety senza condizioni di ripristino valide. Un nodo in error potrebbe richiedere reset o diagnostica. Un nodo con batteria bassa potrebbe rifiutare active. La macchina a stati del nodo deve quindi validare il comando.

Una struttura possibile è:

```
{
  "command": "set_mode",
  "payload": {
    "mode": "active"
  }
}
```

```
}  
}
```

Il nodo riceve il comando, controlla se la transizione dallo stato corrente ad active è consentita, verifica safety, aggiorna stato se possibile e invia ack. Se la transizione non è valida, invia errore o ack rejected con reason. Questo rende la modalità controllata e tracciabile.

Il comando enter_idle è una forma specifica di cambio stato. Può essere usato per portare il nodo in una condizione stabile e non attiva. In idle, il nodo può continuare a comunicare, inviare heartbeat, mostrare LED di stato e mantenere funzioni passive, ma non deve eseguire attuazioni energetiche o pattern complessi. Questa modalità è utile come base sicura. Un comando globale enter_idle può essere inviato in broadcast per fermare una demo senza entrare necessariamente in emergenza.

Il comando enter_diagnostic può abilitare funzioni di test più dettagliate. In modalità diagnostica, il nodo può inviare telemetria più frequente, rispondere a comandi di test, mostrare contatori o eseguire prove controllate su LED e uscite. Tuttavia, la diagnostica non deve aggirare il safety layer. Se un comando diagnostico vuole testare una coil, deve comunque rispettare limiti di duty cycle, durata e temperatura. La modalità diagnostica aumenta l'osservabilità, non elimina le protezioni.

I comandi visuali, come set_led e set_led_pattern, sono molto utili nelle prime versioni MicroBot perché consentono di simulare stati, ruoli e comportamenti anche quando l'attuazione fisica avanzata non è ancora presente. Il LED diventa una forma di feedback locale e una rappresentazione visibile dello stato del nodo. Un set_led può indicare online, errore, ruolo, comando ricevuto, fase di pattern o stato magnetico simulato. Questo collega il communication layer al feedback hardware definito nella Sezione 5.

Un comando set_led può essere:

```
{  
  "command": "set_led",  
  "payload": {  
    "r": 0,  
    "g": 255,  
    "b": 120,  
    "duration_ms": 500  
  }  
}
```

Il nodo deve validare che r, g e b siano in intervalli validi, per esempio 0–255, e che duration_ms sia sotto una durata massima. Se duration_ms è assente, il comando può essere persistente oppure usare un default, ma questa scelta deve essere definita dal protocollo. Per sicurezza e chiarezza, è spesso meglio specificare esplicitamente se il LED è temporaneo o persistente. Un campo mode può distinguere:

"led_mode": "temporary"

oppure:

"led_mode": "persistent"

Il comando set_led_pattern può essere usato per pattern visivi più complessi: lampeggio, pulsazione, stato di errore, identificazione, sincronizzazione. Anche qui, il payload deve essere limitato. Non conviene inviare pattern lunghissimi via UDP. Meglio definire pattern preinstallati nel firmware e richiamarli tramite nome. Per esempio:

```
{  
  "command": "set_led_pattern",  
  "payload": {  
    "pattern": "heartbeat",  
    "color": "green",  
    "duration_ms": 5000  
  }  
}
```

```
}  
}
```

In questo modo il pacchetto resta compatto. Il firmware conosce il pattern heartbeat e lo esegue localmente in modo non bloccante. Questo principio vale per molti comandi: invece di inviare dettagli pesanti, il controller invia un riferimento a un comportamento già definito nel nodo.

I comandi comportamentali sono più vicini al modello multi-agente. `assign_role` permette di assegnare a un nodo un ruolo operativo, come leader, follower, connector, boundary, idle, diagnostic o bridge. Il ruolo influenza la politica locale del nodo e il modo in cui partecipa al pattern. Un comando `assign_role` può avere payload:

```
{  
  "command": "assign_role",  
  "payload": {  
    "role": "leader",  
    "group_id": "cluster_A"  
  }  
}
```

Il nodo deve verificare che il ruolo sia supportato. Se il firmware non implementa logica leader, deve rifiutare o accettare solo come metadato passivo. Anche questo deve essere documentato nelle capabilities. Un nodo può dichiarare `role_assignment` supportato ma non necessariamente tutti i ruoli. Il controller deve considerare questa informazione.

Il comando `start_pattern` permette di avviare un comportamento collettivo. Questo comando può essere broadcast o inviato a un gruppo. Il payload può indicare `pattern_id`, `pattern_run_id`, parametri, tempo di esecuzione e modalità di fallback. Per esempio:

```
{  
  "command": "start_pattern",  
  "payload": {  
    "pattern_id": "line_basic_v0.1",  
    "pattern_run_id": "line_run_001",  
    "execute_at": 123460.00  
  }  
}
```

Questo comando non deve necessariamente contenere tutta la geometria del pattern. In una prima versione, il controller può gestire il pattern centralmente e inviare comandi specifici ai nodi. In una versione più avanzata, i nodi possono conoscere pattern predefiniti e avviarli localmente. La scelta dipende dalla complessità del firmware. Per le prime versioni, è più semplice che il controller mantenga la logica principale e i nodi ricevano istruzioni più semplici. Tuttavia, documentare `pattern_id` e `pattern_run_id` prepara il sistema a estensioni future.

Il comando `stop_pattern` interrompe un comportamento collettivo. Deve essere distinto da `emergency_stop`. `stop_pattern` ferma il pattern in modo ordinato, ma non necessariamente entra in stato di emergenza. Può riportare i nodi in idle, mantenere LED o connessioni passive e salvare stato finale. `emergency_stop` invece deve disattivare immediatamente uscite attive e privilegiare sicurezza. Questa distinzione è importante: non tutti gli stop sono emergenze.

Un comando `stop_pattern` può avere payload:

```
{  
  "pattern_run_id": "line_run_001",  
  "stop_mode": "graceful"  
}
```

`stop_mode` può essere `graceful`, `immediate` o `safe`. `graceful` permette chiusura ordinata; `immediate` interrompe rapidamente; `safe` applica misure più conservative. Anche qui, il nodo deve validare e comportarsi secondo lo stato locale.

Il comando `set_behavior_parameters` consente di modificare pesi o parametri del comportamento simulato o distribuito, come coesione, separazione, allineamento, raggio di vicinato, soglie di attacco o distacco. Questo comando è molto utile nel simulatore e può essere usato per testare dinamiche diverse. Tuttavia, sui nodi fisici iniziali potrebbe non avere piena applicazione se essi non calcolano localmente queste forze. È quindi un comando più naturale per simulazione, controller o firmware avanzato.

Un esempio:

```
{
  "command": "set_behavior_parameters",
  "payload": {
    "cohesion_weight": 0.45,
    "separation_weight": 0.80,
    "alignment_weight": 0.30
  }
}
```

Il sistema deve distinguere se questi parametri sono applicati globalmente nel simulatore o localmente nei nodi. Nel progetto MicroBot, questa distinzione è importante per evitare confusione tra modello virtuale e prototipo fisico. Il comando può essere supportato dai `simulated_node` e non dai `physical_node`, oppure essere interpretato dal controller invece che dal nodo.

I comandi magnetici o attuativi sono i più delicati. Essi coinvolgono bobine, MOSFET, PWM, corrente, calore e potenzialmente interazione fisica. Per questo motivo devono avere una struttura molto rigorosa. Un comando `magnetic_pulse` non deve mai essere ambiguo. Deve specificare quale uscita o coil usare, con quale intensità, per quanto tempo, con quali limiti e con quale politica di sicurezza. Deve inoltre essere soggetto a *validation* severa.

Un esempio:

```
{
  "command": "magnetic_pulse",
  "payload": {
    "coil_id": "main",
    "duty_cycle": 0.30,
    "duration_ms": 120,
    "direction": "attract",
    "max_thermal_load": 0.70
  },
  "requires_ack": true
}
```

Il nodo deve controllare che `coil_id` esista, che `coil_pwm` sia tra le *capabilities*, che `duty_cycle` sia entro il limite locale, che `duration_ms` non superi la durata massima, che `thermal_load` sia sotto soglia, che la batteria sia sufficiente e che lo stato del nodo consenta attuazione. Se anche una sola condizione critica fallisce, il comando deve essere rifiutato. Questo è un punto fondamentale: i comandi magnetici non devono essere “provati” se non validi; devono essere bloccati prima dell’esecuzione.

Il comando `set_coil_pwm` può essere ancora più rischioso se inteso come impostazione persistente di PWM. In generale, per sicurezza, è preferibile evitare comandi persistenti su bobine nelle prime versioni. Meglio usare impulsi temporizzati, perché terminano automaticamente. Se esiste `set_coil_pwm`, deve includere `duration_ms` o essere permesso solo in modalità diagnostica controllata. Un comando che imposta `duty_cycle` senza durata potrebbe lasciare la bobina attiva se la rete cade o il controller si blocca. Questo deve essere evitato.

Una regola corretta è:

ogni comando che attiva una coil deve avere durata massima locale obbligatoria.

Questa durata deve essere applicata dal firmware, non dal controller. Il controller può inviare stop, ma il nodo deve spegnere comunque l'uscita quando il timer locale scade. Questa è una garanzia di sicurezza indipendente dalla rete.

Il comando `disable_outputs` è un comando di sicurezza operativa. Esso disattiva LED non essenziali, PWM, coil o altre uscite attive. Può essere usato in caso di stato incerto, fine test o transizione a idle. È meno estremo di `emergency_stop`, ma deve avere priorità alta. Il nodo che riceve `disable_outputs` deve spegnere le uscite attive e confermare. Può mantenere comunicazione e heartbeat.

Il comando `emergency_stop` è il comando più critico. Deve avere priorità massima, payload minimo e riconoscimento rapido. Il nodo non deve perdere tempo a processare dettagli inutili. Appena riconosciuto un `emergency_stop` valido, deve disattivare tutte le uscite attive, cancellare comandi pendenti pericolosi, entrare in safety o emergency state e inviare ack se possibile. L'ordine è importante: prima sicurezza, poi conferma.

La logica deve essere:

ricevi `emergency_stop`;

verifica minima del protocollo e target;

disattiva immediatamente uscite attive;

annulla azioni temporizzate;

entra in stato safety/emergency;

invia ack o evento.

Questo comando deve essere broadcast-capable, cioè inviabile a tutti i nodi. Dato che UDP non garantisce consegna, il controller può ripeterlo più volte per breve tempo. Ma il nodo non deve dipendere dalla ricezione di più pacchetti: un solo `emergency_stop` valido deve bastare.

Il comando deve essere collegato alla priorità. Ogni comando può avere una priorità esplicita o implicita. `emergency_stop` ha priorità massima; `disable_outputs` alta; `magnetic_pulse` media-alta; `set_led` normale; `telemetry_request` bassa; `diagnostic_trace` bassa o controllata. La priorità aiuta il nodo a ordinare la coda interna. Se ci sono più comandi pendenti, un comando di sicurezza deve scavalcare gli altri.

Una gerarchia corretta è:

`emergency_stop`;

`disable_outputs` / safety commands;

sync commands urgenti;

comandi attuativi temporizzati;

comandi di stato;

comandi visuali;

diagnostica;

telemetria non urgente.

Questa gerarchia mantiene la reattività del nodo verso ciò che conta davvero. Un comando LED non deve ritardare uno stop. Un comando debug non deve ritardare un sync imminente. La gestione della priorità è parte della qualità del sistema.

Un comando può essere immediato, temporizzato o programmato. Un comando immediato viene eseguito appena ricevuto e validato. Un comando temporizzato contiene `execute_at` e viene eseguito in un momento futuro. Un comando programmato può essere inserito in una sequenza, con step e durata. Questa distinzione è centrale per il coordinamento dello swarm.

Un comando immediato è semplice:

```
"execute": "now"
```

Un comando temporizzato può essere:

```
"execute_at": 123460.00
```

Un comando sequenziale può includere:

```
"step_id": "phase_02",  
"sequence_id": "pattern_run_001"
```

La prima versione del sistema può usare comandi immediati e qualche comando temporizzato. Le sequenze complesse possono essere gestite dal controller. Nelle versioni successive, i nodi potranno eseguire sequenze locali predefinite. Anche qui, l'evoluzione deve essere progressiva.

Il comando deve avere una durata quando produce un effetto temporaneo. `set_led` può avere `duration_ms`; `magnetic_pulse` deve averla; `identify_node` deve averla; `diagnostic_blink` deve averla. La durata rende l'azione controllabile e impedisce stati indefiniti. Per comandi persistenti, come `set_mode`, la persistenza è esplicita perché cambia uno stato fino a nuovo comando. Per uscite fisiche energetiche, la persistenza è pericolosa e deve essere evitata o fortemente controllata.

Quindi:

stati logici possono essere persistenti;

uscite energetiche devono essere temporizzate;

diagnostica deve essere limitata;

emergenza deve interrompere tutto.

Questa regola rende il firmware sicuro.

Ogni comando deve produrre un esito. L'esito può essere `accepted`, `rejected`, `scheduled`, `executed`, `failed`, `expired`, `ignored` o `duplicate`. `accepted` significa che il nodo lo ha validato; `scheduled` significa che lo ha messo in coda per esecuzione futura; `executed` significa che lo ha eseguito; `rejected` significa che non lo eseguirà; `failed` significa che ha tentato ma non è riuscito; `expired` significa che è arrivato troppo tardi; `ignored` significa che non era destinato al nodo o non era rilevante; `duplicate` significa che era già stato processato. Questa distinzione è molto utile per il controller.

Un ack può quindi indicare:

```
{  
  "ref_message_id": "CMD-000245",  
  "result": "rejected",  
  "reason": "thermal_lock"  
}
```


Oppure:

```
{
  "ref_message_id": "CMD-000246",
  "result": "scheduled",
  "scheduled_for": 123460.00
}
```

Oppure:

```
{
  "ref_message_id": "CMD-000247",
  "result": "executed",
  "executed_at": 123460.01
}
```

Questi esiti permettono alla dashboard di mostrare lo stato reale del comando. Non basta sapere che un pacchetto è stato inviato; bisogna sapere cosa ne ha fatto il nodo.

Un comando può anche essere rifiutato per incompatibilità con capabilities. Se un nodo non supporta coil_pwm e riceve magnetic_pulse, deve rispondere unsupported_command oppure unsupported_capability. Questo è importante perché MicroBot avrà versioni diverse. Alcuni nodi saranno solo simulati, altri LED-only, altri con coil. Il controller deve inviare comandi coerenti, ma il nodo deve comunque proteggersi.

Il comando può essere rifiutato anche per stato operativo. Per esempio, un nodo in low_energy può rifiutare magnetic_pulse ma accettare heartbeat e set_led a bassa intensità. Un nodo in safety può rifiutare quasi tutto tranne request_status, disable_outputs, emergency_stop e comandi di reset autorizzati. Un nodo in diagnostic può accettare test che in active non sarebbero permessi, ma sempre entro limiti. La validazione deve quindi considerare command + current_state.

Una regola generale è:

$\text{valid_command} = \text{command_supported} \wedge \text{payload_valid} \wedge \text{state_allows} \wedge \text{safety_ok} \wedge \text{timing_valid}$

Se questa formula è falsa, il comando non deve essere eseguito. Il motivo del rifiuto deve essere registrato.

Il comando è anche il punto in cui si collega comunicazione e modello matematico. Un comando può modificare parametri dello swarm, come soglie di attacco, pesi di coesione, modalità di allineamento, target o ruoli. In simulazione, questo può avere effetto immediato sulla dinamica dei nodi. In hardware, può modificare stato o comportamento locale. In entrambi i casi, il comando è la leva attraverso cui il controller influenza l'evoluzione del sistema.

La catena può essere:

comando → variazione dello stato locale → variazione dell'interazione → variazione della forma

Per esempio, start_pattern cambia il ruolo e i target dei nodi; questo modifica le forze virtuali o le regole locali; i nodi si muovono o cambiano stato; le connessioni cambiano; la forma globale evolve. Il comando non è solo un messaggio: è l'innescò di una trasformazione del sistema.

Il comando deve essere progettato per non essere troppo basso livello quando non serve. Se il controller invia continuamente dettagli minimi, il sistema diventa fragile e dipendente dalla rete. Se invece invia comandi più semantici, come start_pattern o set_mode, il nodo può eseguire localmente parti del comportamento. Nelle prime versioni, un controllo più centralizzato è più semplice; nelle versioni avanzate, sarà utile spostare più logica nei nodi. Il protocollo deve poter supportare entrambi.

Si può distinguere:

comandi low-level: set_led, set_coil_pwm, magnetic_pulse;

comandi mid-level: set_mode, assign_role, set_behavior_parameters;

comandi high-level: start_pattern, stop_pattern, execute_phase.

I comandi low-level controllano direttamente funzioni hardware o immediate. I comandi mid-level modificano stato e comportamento. I comandi high-level avviano processi più complessi. Uno swarm maturo dovrebbe usare comandi high-level quando possibile e low-level per test, calibrazione e diagnostica. Questo riduce traffico e migliora autonomia locale.

Nel prototipo iniziale, però, i comandi low-level sono utilissimi perché permettono di testare componenti singoli. Prima si testa set_led. Poi si testa heartbeat. Poi set_mode. Poi, quando l'hardware è sicuro, magnetic_pulse. Poi pattern. Questa progressione è coerente con la metodologia del progetto MicroBot: prima elementi semplici, poi integrazione, poi comportamento collettivo.

Il comando deve essere compatibile con logging e replay. Se ogni comando viene registrato con message_id, timestamp, target, command, payload e result, allora una sequenza può essere analizzata dopo l'esecuzione. In alcuni casi, può essere anche riprodotta in simulazione. Questo è molto potente: una run reale può generare log, e quei log possono essere usati per replay virtuale o debug. Per ottenere questo, i comandi devono essere strutturati e non ambigui.

Un log comando può contenere:

message_id;

timestamp;

source;

target;

command;

payload_summary;

result;

reason;

latency;

correlation_id.

Questa struttura permette di ricostruire cosa è stato chiesto, a chi, quando, con quali parametri e con quale esito. È essenziale per una documentazione tecnica seria.

Il comando può essere associato a una correlation_id quando fa parte di un'operazione più ampia. Per esempio, tutti i comandi di una formazione lineare possono condividere correlation_id = line_run_001. Se MB-003 fallisce, il log mostra che il fallimento apparteneva a line_run_001. Questo permette analisi di run, pattern e test. Senza correlation_id, i comandi sarebbero isolati.

Il comando deve essere collegato anche al timestamp, che sarà approfondito nel punto successivo. Per ora è importante anticipare che ogni comando deve avere tempo di generazione e, quando necessario, tempo di esecuzione. Questo impedisce comandi vecchi, permette sincronizzazione e consente misurare latenza. Un comando senza tempo è più difficile da validare. In uno swarm distribuito, il tempo è parte del comando.

Un comando temporale minimo contiene:

timestamp = quando è stato generato;

execute_at = quando deve essere eseguito, se programmato;

expires_at o ttl = fino a quando è valido.

Questi campi evitano ambiguità. Un comando arrivato tardi può essere scartato. Un comando programmato può essere eseguito insieme agli altri. Un comando vecchio non può modificare lo stato dopo che il sistema è andato avanti.

Dal punto di vista del firmware, il comando deve passare attraverso un command dispatcher. Il dispatcher riceve un pacchetto già parsato e validato nei campi generali, legge command e invia il payload al modulo competente. Per esempio:

set_led → LED manager;

set_mode → state manager;

magnetic_pulse → magnetic actuator manager;

request_status → telemetry manager;

emergency_stop → safety manager;

assign_role → behavior manager.

Questa separazione rende il firmware modulare. Il parser non deve sapere come attivare una bobina; il magnetic manager non deve sapere come parsare JSON; il safety manager deve poter bloccare tutto. Ogni modulo ha responsabilità chiara. Questo è coerente con l'architettura generale del sistema.

Il command dispatcher deve ricevere anche l'esito dal modulo esecutore. Se il LED manager esegue set_led, restituisce executed. Se il magnetic manager rifiuta magnetic_pulse per thermal_lock, restituisce rejected. Il dispatcher costruisce ack o error. Questo crea una pipeline pulita:

packet receiver → parser → validator → dispatcher → module handler → result → ack/error → telemetry/log.

Questa pipeline è molto importante perché rende il comando osservabile in ogni fase. Se qualcosa fallisce, si sa dove.

I comandi devono essere testabili singolarmente. Ogni comando del protocollo dovrebbe avere casi di test: payload valido, payload mancante, valori fuori range, target sbagliato, comando duplicato, comando scaduto, nodo in safety, capability assente, ack corretto. Questo può sembrare eccessivo, ma per un sistema distribuito è molto utile. I comandi sono l'interfaccia critica del sistema; se sono fragili, tutto lo swarm diventa fragile.

Una tabella di test per set_led può includere:

set_led valido;

set_led con colore fuori range;

set_led senza duration_ms;

set_led con target errato;

set_led duplicato;

set_led durante safety.

Una tabella per magnetic_pulse deve essere ancora più rigorosa:

coil_id valido;

coil_id inesistente;

duty_cycle troppo alto;

duration_ms troppo lungo;

thermal_load alto;

battery low;

command expired;

duplicate message;

emergency_stop durante impulso.

Questo livello di test rende il progetto molto più credibile e pronto a evolvere.

Il comando deve essere progettato anche per essere leggibile nella documentazione. Ogni comando dovrebbe avere una sezione o tabella dedicata con nome, descrizione, direzione, payload, risposta, safety, note. Per esempio:

Nome comando: magnetic_pulse.

Direzione: controller verso nodo.

Funzione: attiva una bobina per un impulso temporizzato.

Payload obbligatorio: coil_id, duty_cycle, duration_ms.

Ack: obbligatorio.

Safety: controllo duty, durata, temperatura, batteria.

Errori possibili: unsupported_command, invalid_payload, thermal_lock, low_energy, command_expired.

Note: non deve mai essere persistente senza durata.

Questa forma rende il protocollo implementabile da chiunque legga la documentazione.

Il comando è anche il modo in cui si mantiene coerenza tra più repository o componenti. Se il simulatore, il firmware e la dashboard usano gli stessi nomi di comando, il sistema resta unificato. Se invece uno usa setLed, uno LED_SET, uno set_led_rgb e uno ledCommand, nasce frammentazione. È quindi importante definire una convenzione di naming stabile. Una convenzione consigliata è snake_case per i nomi comando:

set_led;

set_mode;

request_status;

assign_role;

start_pattern;

magnetic_pulse;

emergency_stop.

Questa scelta è leggibile, coerente con JSON e facilmente gestibile in Python e JavaScript. L'importante è essere costanti.

I comandi devono essere versionati quando cambiano significato. Se set_led in versione 0.1 accetta r, g, b e duration_ms, e in versione 0.2 cambia struttura, il campo protocol version deve permettere al nodo di interpretare correttamente. Meglio però evitare cambiamenti incompatibili nei comandi base. Si possono aggiungere campi opzionali senza rompere il protocollo. Per esempio, set_led può aggiungere brightness come campo opzionale. Un nodo vecchio può ignorarlo o rifiutarlo secondo policy. La compatibilità è importante.

Il comando deve essere progettato per gestire nodi offline. Se il controller invia un comando a un nodo non online, può metterlo in pending, rifiutare l'invio o segnalarlo come target_unavailable. Per comandi realtime, metterli in pending può essere sbagliato, perché quando il nodo torna online potrebbero essere vecchi. Per comandi di configurazione, può essere accettabile. Il controller deve quindi distinguere:

comandi realtime → non conservare oltre scadenza;

comandi di configurazione → possono essere applicati quando il nodo torna online, se ancora validi;

comandi di safety → inviare solo a nodi raggiungibili, ma mantenere safety locale indipendente.

Questa gestione evita che un nodo appena riconnesso esegua comandi vecchi e non più coerenti.

Il comando deve essere pensato anche rispetto ai gruppi. Un comando broadcast target all può essere eseguito da tutti. Un comando group target cluster_A deve essere eseguito solo dai nodi appartenenti a quel gruppo. Ogni nodo deve sapere il proprio group_id oppure riceverlo nei comandi precedenti. Se non appartiene al gruppo, ignora. Questo permette coordinamento di sottostrutture. Nei pattern complessi, i comandi di gruppo saranno più efficienti dei comandi individuali.

Un comando di gruppo può essere:

```
{
  "target": "cluster_A",
  "command": "set_mode",
  "payload": {
    "mode": "stabilizing"
  }
}
```

Il nodo verifica se group_id == cluster_A. Se sì, processa; altrimenti ignora. Questa logica è semplice ma potente.

Il comando deve prevedere anche il caso simulato. Un simulated_node può supportare comandi che il nodo fisico non supporta ancora, come set_virtual_position, set_behavior_parameters, inject_fault, simulate_connection. Questi comandi devono essere chiaramente marcati come simulation-only. Il controller deve evitare di inviarli a nodi fisici, salvo che siano ignorati. La distinzione tra physical command e simulation command evita confusione documentale.

Una capability può indicare:

simulation_only = true

oppure il comando può essere riservato al simulation_engine. Per esempio, set_virtual_position non dovrebbe essere accettato da un ESP32 fisico, perché non può teletrasportarsi. Nel simulatore invece è utile per test. Questa distinzione mantiene rigore tra realtà e modello.

Dal punto di vista teorico, il comando è l'unità minima di intenzionalità del sistema. Il modello matematico descrive come lo swarm evolve; il comando è il mezzo con cui il controller modifica le condizioni di quella evoluzione. Un comando può cambiare stato, parametri, ruoli, soglie, attuazione o feedback. In questo senso, il comando è il collegamento tra progetto umano e comportamento distribuito. L'utente o il controller non manipola direttamente ogni relazione fisica; invia comandi che modificano il comportamento dei nodi.

Nel paradigma MicroBot, questo è molto importante: la forma non viene solo disegnata, ma comandata attraverso regole. Un comando `start_pattern` non sposta magicamente tutti i nodi nella forma finale; avvia una sequenza di stati, target, interazioni e verifiche che portano alla forma. Il comando è quindi un innesco, non una garanzia automatica. La garanzia deriva dall'interazione tra comando, firmware, modello matematico, fisica e safety.

Un comando ben progettato deve essere chiaro, limitato, validabile, tracciabile e reversibile quando possibile. Chiaro significa che il suo significato non è ambiguo. Limitato significa che non può produrre effetti indefiniti o fuori range. Validabile significa che il nodo può controllarlo prima di eseguirlo. Tracciabile significa che può essere collegato a ID, ack, log ed eventi. Reversibile significa che, quando possibile, esiste un comando o una procedura per tornare a uno stato sicuro o precedente. Questi criteri dovrebbero guidare tutto il protocollo.

In conclusione, il comando è il nucleo operativo dei pacchetti MicroBot. Esso trasforma la comunicazione in azione, permettendo al controller di richiedere cambi di stato, feedback LED, diagnostica, assegnazione di ruoli, avvio di pattern, controllo magnetico e procedure di sicurezza. Tuttavia, il comando non deve essere eseguito ciecamente: deve attraversare validazione di protocollo, target, payload, capabilities, stato locale, timestamp e safety layer. I comandi devono essere classificati per rischio e priorità, temporizzati quando controllano uscite fisiche, confermati quando sono critici, registrati nei log e progettati in modo coerente tra simulazione, firmware e dashboard. In questo modo, il comando diventa una interfaccia rigorosa tra intenzione digitale e comportamento fisico distribuito. Il punto successivo, dedicato al timestamp, approfondirà il ruolo del tempo nei pacchetti, mostrando come ogni comando e ogni stato debbano essere collocati temporalmente per garantire sincronizzazione, validità, tracciabilità e coordinamento dello swarm.

8.8 Timestamp

Il timestamp rappresenta il riferimento temporale associato ai messaggi, agli stati, ai comandi e agli eventi del sistema MicroBot. Dopo aver definito identificatore e comando, diventa necessario introdurre il tempo come componente strutturale della comunicazione, perché in uno swarm distribuito non basta sapere chi ha inviato un messaggio e quale azione viene richiesta: bisogna sapere anche quando il messaggio è stato generato, quando è stato ricevuto, quando deve essere eseguito, per quanto tempo resta valido e come si colloca rispetto agli altri eventi del sistema. Il timestamp è quindi il campo che permette di trasformare la comunicazione da semplice scambio di dati a sequenza ordinata e analizzabile di eventi nel tempo.

Nel progetto MicroBot, il timestamp è fondamentale perché il sistema è dinamico. I nodi cambiano stato, le connessioni si formano e si rompono, i comandi vengono inviati, i pattern evolvono, la telemetria viene raccolta, la dashboard aggiorna la visualizzazione e il controller deve prendere decisioni sulla base di informazioni recenti. Un dato senza tempo è incompleto. Sapere che un nodo ha batteria bassa è utile, ma bisogna sapere quando quel dato è stato misurato. Sapere che un comando è stato ricevuto è utile, ma bisogna sapere se è stato ricevuto in tempo. Sapere che una connessione è avvenuta è utile, ma bisogna sapere in quale fase del pattern. Il timestamp fornisce questo contesto.

In forma generale, un pacchetto MicroBot dovrebbe contenere almeno un campo temporale:

`timestamp` = tempo di generazione del messaggio

Questo valore indica il momento in cui il messaggio è stato creato dalla sorgente. Se il controller invia un comando, il timestamp indica quando il controller ha generato quel comando. Se un nodo invia heartbeat, il timestamp indica quando il nodo ha prodotto quel messaggio. Se il simulatore registra un evento, il timestamp indica quando l'evento è avvenuto nel tempo della simulazione o nel tempo di sistema. Questo campo permette di ordinare i messaggi, calcolare ritardi, scartare pacchetti vecchi e ricostruire la storia dello swarm.

Tuttavia, nel sistema MicroBot è utile distinguere più tipi di tempo. Il primo è il tempo di generazione, cioè il momento in cui il pacchetto viene creato. Il secondo è il tempo di ricezione, cioè il momento in cui il destinatario riceve il pacchetto. Il terzo è il tempo di esecuzione, cioè il momento in cui il comando deve

produrre effetto. Il quarto è il tempo di scadenza, cioè il limite oltre il quale il pacchetto non deve più essere considerato valido. Il quinto è il tempo di evento, cioè il momento in cui una transizione fisica, logica o simulativa è avvenuta. Queste distinzioni sono importanti perché evitano ambiguità.

Una struttura temporale più completa può includere:

timestamp: momento di generazione del pacchetto;

received_at: momento di ricezione, misurato localmente dal destinatario;

execute_at: momento previsto di esecuzione;

expires_at: momento oltre il quale il comando non è più valido;

event_time: momento in cui un evento è avvenuto;

duration_ms: durata dell'azione richiesta.

Nelle prime versioni del protocollo, non è necessario usare sempre tutti questi campi. Tuttavia, la documentazione deve chiarire il loro significato, perché il sistema crescerà verso comandi sincronizzati, pattern temporizzati, azioni magnetiche controllate e telemetria storica. Ogni campo temporale ha una funzione diversa e deve essere usato in modo coerente.

Il timestamp di generazione è utile per valutare la freschezza del messaggio. Quando un nodo riceve un comando, può confrontare il timestamp con il proprio tempo locale o con il tempo stimato del controller. Se il messaggio è troppo vecchio, il nodo può rifiutarlo. Questo è particolarmente importante con UDP, perché un pacchetto può arrivare in ritardo oppure essere processato dopo che lo stato del sistema è cambiato. Un comando vecchio non deve poter modificare il comportamento corrente se non è più coerente con la situazione attuale.

Una regola possibile è:

se $\text{tempo_corrente} - \text{timestamp} > \text{max_message_age}$, allora rifiutare il comando.

Il valore `max_message_age` dipende dal tipo di comando. Un comando diagnostico può tollerare un'età maggiore. Un comando LED può tollerare un ritardo moderato. Un comando magnetico o di pattern temporizzato deve avere una finestra molto più stretta. Un `emergency_stop`, invece, rappresenta un caso particolare: anche se il timestamp è vecchio, il nodo potrebbe comunque trattarlo con priorità se il pacchetto è valido, perché la sicurezza può prevalere sulla logica temporale ordinaria. Tuttavia, anche per i comandi di emergenza è utile evitare messaggi obsoleti generati da sessioni precedenti. Questo richiede una politica precisa.

Il campo `execute_at` è ancora più importante per lo swarm. In un sistema distribuito, non è realistico assumere che tutti i nodi ricevano un comando nello stesso istante. Anche se UDP è rapido, la rete Wi-Fi può introdurre ritardi diversi. Se il controller inviasse un comando broadcast dicendo semplicemente "esegui ora", ogni MicroBot lo eseguirebbe nel momento in cui lo riceve e lo processa. Il risultato potrebbe essere una esecuzione non simultanea. Con `execute_at`, invece, il controller può inviare il comando in anticipo e indicare un istante futuro comune. Ogni nodo lo conserva e lo esegue quando il proprio clock raggiunge quel tempo.

Una struttura di comando temporizzato può essere:

```
{
  "command": "set_led",
  "timestamp": 123456.00,
  "execute_at": 123457.00,
  "payload": {
    "r": 0,
    "g": 255,
```

```
"b": 120,
"duration_ms": 500
}
}
```

In questo esempio, il comando viene generato al tempo 123456.00 ma deve essere eseguito al tempo 123457.00. Il controller invia il pacchetto con un secondo di anticipo. Se i nodi sono sufficientemente sincronizzati, possono eseguire l'azione nello stesso momento anche se ricevono il pacchetto con piccoli ritardi diversi. Questo principio è essenziale per pattern visivi, sequenze LED, impulsi magnetici coordinati, slot temporali e azioni distribuite.

Il campo `duration_ms` definisce quanto deve durare l'azione. Questo è fondamentale per ogni comando temporaneo. Un LED può accendersi per 500 ms; una bobina può generare un impulso per 100 ms; una modalità diagnostica può restare attiva per 10 secondi; una finestra di ascolto può durare un certo intervallo. La durata deve essere gestita localmente dal nodo, non dal controller. Il controller può inviare un comando di stop, ma il nodo deve avere un timer locale che termina automaticamente l'azione. Questo principio è soprattutto importante per le uscite energetiche, come bobine e PWM.

Una regola fondamentale è:

un comando fisico temporaneo deve contenere una durata e il nodo deve essere capace di terminarlo autonomamente.

Questo evita dipendenza dalla rete. Se il controller invia `magnetic_pulse` con `duration_ms = 120` e poi la comunicazione cade, il nodo deve comunque spegnere la bobina dopo 120 ms. Il timestamp e la durata permettono quindi di costruire sicurezza locale. Il tempo non serve solo a sincronizzare, ma anche a limitare il rischio.

Il campo `expires_at` o `tll` serve a definire la validità del comando. Un comando può avere senso solo entro una certa finestra temporale. Se arriva troppo tardi, eseguirlo potrebbe essere sbagliato. Per esempio, un comando `start_pattern` relativo a una fase già conclusa non deve essere eseguito in ritardo. Un comando di allineamento previsto per una finestra di aggancio non deve attivarsi quando i nodi si sono già separati. Il campo `expires_at` permette al nodo di rifiutare comandi scaduti.

Una struttura possibile è:

```
{
"timestamp": 123456.00,
"execute_at": 123457.00,
"expires_at": 123457.20
}
```

In questo caso, il comando deve essere eseguito al tempo 123457.00, ma se viene ricevuto o processato dopo 123457.20 non deve più essere considerato valido. Questo introduce una tolleranza di 200 ms. La tolleranza deve dipendere dal tipo di comando. Per una sequenza LED, può essere abbastanza ampia; per un impulso magnetico coordinato, deve essere più stretta; per una richiesta di stato, può essere meno critica.

Il timestamp è anche indispensabile per gli ack. Quando un nodo riceve un comando, può rispondere con un ack contenente il momento in cui il comando è stato ricevuto, accettato, programmato o eseguito. Questo permette al controller di misurare ritardi e verificare se lo swarm sta rispettando i tempi previsti. Un ack semplice può indicare solo `accepted`; un ack più ricco può includere `received_at`, `scheduled_for`, `executed_at` ed `execution_error_ms`.

Un ack temporale può essere:

```
{
"type": "ack",
"ref_message_id": "CMD-000245",
"result": "executed",
```



```
"received_at": 123456.21,  
"executed_at": 123457.01,  
"execution_error_ms": 10  
}
```

Questo messaggio dice che il nodo ha ricevuto il comando, lo ha eseguito e l'esecuzione è avvenuta 10 ms dopo il tempo previsto. Questa informazione è preziosissima per la sincronizzazione e per il debug. Senza timestamp, il controller saprebbe solo che il comando è stato eseguito; con timestamp, può sapere anche quanto bene è stato eseguito rispetto al tempo atteso.

Il timestamp è centrale anche nella telemetria. Ogni valore di telemetria deve essere associato al momento in cui è stato misurato o generato. Se un nodo invia batteria, temperatura stimata, stato LED, stato coil o rete, il controller deve sapere a quale istante si riferisce quel dato. In una rete con latenza variabile, il momento di arrivo del pacchetto non coincide necessariamente con il momento della misura. Per questo motivo, la telemetria deve includere timestamp locale del nodo o timestamp di generazione.

Un pacchetto telemetry può contenere:

```
{  
  "type": "telemetry",  
  "source": "MB-001",  
  "timestamp": 123460.50,  
  "payload": {  
    "battery": 0.82,  
    "temperature_estimate": 34.2,  
    "mode": "active"  
  }  
}
```

Il controller riceve questo pacchetto magari al tempo 123460.57, ma sa che il dato è stato generato a 123460.50. La differenza aiuta a stimare ritardo. Inoltre, se arrivano pacchetti fuori ordine, il controller può mantenere il dato più recente e ignorare quello più vecchio. Questo è essenziale per evitare che la dashboard mostri stati superati.

Una regola utile è:

per ogni nodo e per ogni tipo di telemetria, conservare il valore con timestamp più recente.

Se arriva un pacchetto più vecchio di uno già registrato, il controller può ignorarlo o archivarlo solo nei log. Questa logica mantiene lo stato corrente coerente.

Il timestamp è anche fondamentale per gli eventi. Un evento rappresenta una transizione significativa: un nodo entra online, una connessione viene formata, una soglia viene superata, un distacco avviene, un comando viene rifiutato, un safety trigger si attiva. Ogni evento deve avere event_time. In alcuni casi, event_time coincide con timestamp del pacchetto; in altri, può essere il momento reale della transizione, registrato localmente prima dell'invio. Questo è importante perché l'evento può essere trasmesso in ritardo, ma il sistema deve sapere quando è avvenuto.

Un evento di distacco può essere:

```
{  
  "type": "event",  
  "event": "detach_event",  
  "event_time": 123470.33,  
  "timestamp": 123470.35,  
  "source": "MB-002",  
  "payload": {  
    "other_node": "MB-003",  
    "cause": "distance_exceeded"  
  }  
}
```

Qui `event_time` indica quando il distacco è stato rilevato; `timestamp` indica quando il messaggio è stato generato. In una prima versione possono coincidere, ma distinguerli rende il modello più rigoroso. Nei sistemi distribuiti, la distinzione tra tempo dell'evento e tempo del messaggio è molto importante.

Il timestamp permette anche la ricostruzione della timeline globale del sistema. Quando tutti i pacchetti, comandi, ack, errori, heartbeat, telemetrie ed eventi possiedono riferimenti temporali, è possibile ordinare ciò che è accaduto. Questo è fondamentale per debugging. Se una formazione fallisce, bisogna capire la sequenza: il comando è stato inviato, i nodi lo hanno ricevuto, uno di essi lo ha rifiutato, una connessione non si è formata, il safety layer si è attivato, il pattern è stato interrotto. Senza timestamp, questi eventi sarebbero difficili da ordinare; con timestamp, diventano una storia analizzabile.

Una timeline può apparire così:

123456.00 — controller invia `start_pattern`;

123456.12 — MB-001 riceve e accetta;

123456.14 — MB-002 riceve e accetta;

123456.20 — MB-003 rifiuta per `low_energy`;

123457.00 — MB-001 e MB-002 eseguono fase 1;

123457.30 — MB-002 segnala `attach_event`;

123457.40 — controller rileva pattern incompleto;

123457.50 — controller invia `stop_pattern`.

Questa sequenza permette di capire il comportamento del sistema. Il timestamp trasforma una massa di messaggi in una cronologia tecnica.

Il problema principale è che, in un sistema distribuito, ogni dispositivo può avere un clock diverso. Il controller ha un proprio tempo; ogni ESP32 ha un proprio contatore locale; il simulatore può usare tempo reale o tempo simulato; la dashboard può usare il tempo del browser o del server. Se questi tempi non sono allineati, i timestamp non sono direttamente confrontabili. Per questo motivo, la comunicazione temporale deve distinguere tra tempo locale e tempo globale di sistema.

Il tempo locale è il tempo misurato da un dispositivo. Il tempo globale di sistema è un riferimento comune, stimato o imposto dal controller. Nelle prime versioni, il controller può fungere da riferimento temporale principale. I nodi possono ricevere messaggi di sincronizzazione e stimare un offset tra il proprio clock locale e il tempo del controller. Questo argomento sarà sviluppato nella Sezione 9, ma il timestamp nella Sezione 8 deve già essere progettato per supportarlo.

Una relazione concettuale può essere:

$$\text{tempo_sistema} \approx \text{tempo_locale_nodo} + \text{offset_nodo}$$

Il controller può stimare `offset_nodo` attraverso messaggi di sync. Se l'offset è noto, il nodo può interpretare `execute_at` nel tempo di sistema e convertirlo nel proprio tempo locale. Questo consente esecuzioni coordinate. Senza questa conversione, `execute_at` avrebbe significati diversi per ogni nodo.

Nelle prime versioni, se la sincronizzazione non è ancora implementata completamente, si può usare una semplificazione: i comandi temporizzati possono essere relativi, non assoluti. Invece di dire `execute_at = tempo globale T`, il controller può dire `execute_after_ms = 500`. Ogni nodo esegue 500 ms dopo la ricezione. Questa soluzione è più semplice, ma meno sincronizzata, perché i nodi ricevono il pacchetto in tempi diversi. È utile per demo semplici, ma non per azioni realmente coordinate. La progressione naturale è quindi:

fase iniziale: comandi immediati o relativi alla ricezione;

fase intermedia: comandi con ritardo relativo;

fase avanzata: comandi con `execute_at` basato su clock sincronizzato.

Questa evoluzione è coerente con lo sviluppo incrementale MicroBot. Non serve pretendere sincronizzazione perfetta nelle prime versioni, ma il protocollo deve essere progettato per supportarla.

Il timestamp è anche utile per rilevare nodi offline o comunicazione degradata. Ogni heartbeat contiene un timestamp e il controller registra il momento di arrivo. Se passa troppo tempo dall'ultimo heartbeat, il nodo viene marcato stale o offline. La soglia dipende dalla frequenza attesa degli heartbeat. Se un nodo dovrebbe inviare heartbeat ogni secondo e non invia nulla per cinque secondi, il controller può considerarlo non affidabile. Questo non richiede sincronizzazione perfetta; basta il tempo del controller per misurare l'intervallo dall'ultimo messaggio ricevuto.

Una regola semplice è:

se `tempo_controller_corrente - last_seen > timeout_offline`, allora nodo offline.

`last_seen` può essere il tempo di arrivo del pacchetto al controller, mentre timestamp nel payload indica quando il nodo lo ha generato. Entrambi sono utili. Il tempo di arrivo è affidabile per il controller; il timestamp del nodo è utile per stimare ritardo e coerenza temporale. In una prima versione, `last_seen` può bastare per presenza; in una versione avanzata, si può usare anche il timestamp del nodo.

Il timestamp aiuta a calcolare latenza. Se il controller invia un ping con timestamp `T_send` e riceve un pong al tempo `T_receive`, può calcolare il round-trip time:

$$RTT = T_{receive} - T_{send}$$

Se il pong contiene anche il tempo locale del nodo, il controller può stimare meglio la distribuzione del ritardo, anche se la stima precisa richiede sincronizzazione o assunzioni. Questa misura è utile per classificare la rete: buona, degradata, instabile. Il timestamp diventa quindi base per metriche di comunicazione.

Una dashboard può mostrare:

RTT medio;

RTT massimo;

jitter;

ultimo timestamp ricevuto;

tempo dall'ultimo heartbeat;

`execution_error`;

comandi scaduti.

Queste metriche rendono visibile la qualità temporale della comunicazione. Uno swarm non deve essere solo connesso; deve essere connesso in tempi coerenti.

Il timestamp è importante anche per evitare replay accidentali di messaggi vecchi. Se un vecchio comando viene ritrasmesso, duplicato o recuperato da una coda non aggiornata, il nodo può rifiutarlo perché il timestamp è troppo vecchio o perché `expires_at` è passato. Questo è un livello di protezione. In una versione

più avanzata, il timestamp può contribuire anche alla sicurezza contro replay malevoli, se combinato con autenticazione. Per ora, il suo ruolo principale è evitare effetti obsoleti e mantenere coerenza operativa.

Una regola è:

un comando operativo deve essere eseguito solo se appartiene alla finestra temporale corrente.

Questa finestra può essere definita da timestamp, ttl, expires_at o execute_at. Il concetto è lo stesso: il comando deve essere temporalmente valido.

Il timestamp entra anche nella gestione delle code. Un nodo può ricevere comandi programmati per il futuro e inserirli in una coda ordinata per execute_at. Quando il tempo locale raggiunge l'istante previsto, il nodo esegue il comando. Se riceve un comando con execute_at precedente al tempo corrente, può rifiutarlo come expired. Se riceve molti comandi futuri, deve avere un limite di coda per evitare memoria eccessiva. Ogni comando in coda deve essere associato a message_id, execute_at, expires_at e priorità.

Una coda temporale può contenere:

message_id;

command;

payload;

execute_at;

expires_at;

priority;

state.

Il nodo processa la coda in ordine temporale e di priorità. Questo permette sequenze coordinate senza bloccare il loop principale. Il nodo non aspetta in modo bloccante; controlla periodicamente la coda e attiva i comandi quando il tempo è raggiunto. Questo è fondamentale per firmware non bloccante.

Il timestamp è anche utile per i pattern. Un pattern può essere diviso in fasi temporali: fase di preparazione, fase di avvicinamento, fase di aggancio, fase di stabilizzazione, fase di completamento. Ogni fase può avere un tempo di inizio e fine. I comandi possono riferirsi a phase_id e execute_at. Questo permette di coordinare nodi diversi all'interno della stessa run. Il tempo diventa struttura del comportamento collettivo.

Una sequenza di pattern può essere:

phase_01_prepare: T0 → T1;

phase_02_align: T1 → T2;

phase_03_attach: T2 → T3;

phase_04_stabilize: T3 → T4.

Ogni nodo può ricevere comandi relativi a queste fasi. La dashboard può mostrare il progresso temporale del pattern. I log possono associare eventi alla fase corretta. Se un attach_event avviene fuori dalla finestra prevista, il controller può segnalarlo come anomalia. Senza timestamp, questa analisi sarebbe impossibile.

Nel modello matematico, il timestamp collega la comunicazione alla variabile t . Nella Sezione 7, molte grandezze sono state espresse come funzioni del tempo: $p_i(t)$, $v_i(t)$, $d_{ij}(t)$, $A_{ij}(t)$, $S_{\text{global}}(t)$. Il timestamp è il modo informatico con cui il sistema reale associa dati e messaggi a quel tempo t . La posizione di un nodo non è solo p_i ; è p_i al tempo t . La connessione non è solo A_{ij} ; è A_{ij} al tempo t . La telemetria non è solo un valore; è un valore al tempo t . Questo collegamento rende coerenti matematica, software e log.

Uno snapshot dello swarm deve quindi includere un timestamp globale o simulativo:

`snapshot_time = t`

e poi una lista di stati nodo:

`node_state_i(t)`

Questo permette di confrontare snapshot diversi, calcolare velocità, ricostruire traiettorie e analizzare evoluzione. Senza timestamp, uno snapshot è una fotografia isolata; con timestamp, diventa parte di una sequenza temporale.

Il timestamp è importante anche per distinguere tempo reale e tempo simulato. Nelle simulazioni, il tempo può avanzare più velocemente o più lentamente del tempo reale, può essere messo in pausa, riavvolto o eseguito a step. In hardware, invece, il tempo reale continua. Il sistema deve quindi dichiarare quale timebase viene usata. Un pacchetto può contenere `timebase_id`, ad esempio `real_time`, `simulation_time` o `controller_time`. Questo evita confusione quando si integrano simulazione e dashboard.

Una struttura possibile è:

`"timebase": "controller_time"`

oppure:

`"timebase": "simulation_time"`

Se un messaggio proviene dal simulatore, timestamp può riferirsi al tempo simulato. Se proviene da un ESP32, può riferirsi al tempo locale o al tempo sincronizzato con il controller. Questa distinzione è importante soprattutto nei replay. Un replay può riprodurre eventi secondo `simulation_time`, indipendentemente dal tempo reale di esecuzione.

Nel contesto dei file di log, il timestamp deve essere scelto in formato coerente. Per la comunicazione interna può essere un numero monotono, espresso in secondi o millisecondi. Per log leggibili dall'uomo può essere utile anche una data ISO 8601. Una buona strategia è usare entrambi nei sistemi più maturi: un tempo monotono per calcoli e ordinamento preciso, una data leggibile per documentazione. Il tempo monotono evita problemi dovuti a cambi dell'orologio di sistema; la data leggibile aiuta l'analisi umana.

Un log può contenere:

`timestamp_ms = 123456789;`

`datetime = "2026-04-30T21:45:12.345+02:00"`

Il primo è utile per calcolare differenze temporali; il secondo è utile per leggere quando è avvenuto l'evento. Nei pacchetti embedded, si può usare il formato più compatto; nei log salvati dal controller, si può arricchire con data leggibile.

Il timestamp deve essere monotono per le sequenze operative. Un clock di sistema che può tornare indietro, ad esempio perché sincronizzato con l'orologio del computer, può creare problemi. Per misurare durate e timeout è preferibile usare un clock monotono, cioè un contatore che cresce sempre. Gli ESP32 e i microcontrollori possono usare contatori millisecondi o microsecondi dall'avvio. Il controller può usare un

tempo monotono interno per RTT e timeout. Le date assolute sono utili per log, ma non devono essere l'unica base per misurare intervalli critici.

Questa distinzione può essere riassunta così:

clock monotono: misurare durate, timeout, intervalli, esecuzioni;

clock assoluto: datare log, sessioni, esperimenti, file.

MicroBot dovrebbe usare entrambi quando il sistema diventa più maturo.

Il timestamp ha un ruolo nella gestione del `boot_id`. Un nodo appena avviato può avere il proprio contatore locale a zero. Se invia timestamp locale senza indicare `boot_id`, il controller potrebbe confondere due messaggi di sessioni diverse. Per questo motivo, timestamp locale e `boot_id` devono essere combinati. Il timestamp locale è valido dentro una sessione di boot; `boot_id` distingue le sessioni. Se il nodo si resetta, il suo tempo locale riparte, ma `boot_id` cambia. Il controller può riconoscere la nuova sessione.

Un messaggio può includere:

`node_id = MB-001;`

`boot_id = boot_017;`

`timestamp_local_ms = 2500.`

Questo significa: il nodo MB-001, nella sessione `boot_017`, ha generato il messaggio 2500 ms dopo l'avvio. Il controller può mappare questo tempo locale su tempo di sistema se ha dati di sincronizzazione.

Il timestamp è fondamentale anche per l'hysteresis temporale. Nella Sezione 7 è stato introdotto il concetto che alcune condizioni devono persistere per più cicli o per un intervallo prima di generare una transizione. Per esempio, un distacco può essere confermato solo se la distanza supera la soglia per K passi o per un certo tempo. Il timestamp permette di misurare questa durata. Invece di contare solo cicli, si può dire:

distacco confermato se $d_{ij} > d_{\text{detach}}$ per più di ΔT_{detach} .

Questa forma è più robusta quando la frequenza di aggiornamento varia. Se il loop non ha sempre lo stesso periodo, contare cicli può essere meno preciso; misurare tempo reale o simulato è più corretto. Il timestamp rende possibile questa logica.

Analogamente, un aggancio può essere confermato se le condizioni restano valide per un certo intervallo:

attacco confermato se $d_{ij} < d_{\text{attach}}$ e orientamento compatibile per almeno ΔT_{attach} .

Questo evita agganci momentanei dovuti a passaggi rapidi. Il tempo diventa parte della condizione di stabilità. Nel sistema MicroBot, questa è una scelta molto importante perché connessioni fisiche e magnetiche non devono nascere o morire per fluttuazioni istantanee.

Il timestamp ha anche un ruolo nella safety. Un nodo può attivare una protezione se una condizione rischiosa persiste troppo a lungo. Per esempio, una bobina può avere un limite di attivazione cumulativa. Il firmware può registrare quando una bobina viene accesa e quando viene spenta, calcolare durata e carico termico stimato. Senza timestamp, non sarebbe possibile stimare l'esposizione temporale. La protezione termica è una funzione temporale.

Una logica possibile è:

`coil_on_time += durata_attivazione;`

`thermal_load = funzione(coil_on_time, duty_cycle, intervallo_di_raffreddamento);`

se `thermal_load > soglia`, entra in `thermal_lock`.

Qui il tempo è essenziale. Il safety layer non controlla solo valori istantanei, ma anche durata e accumulo. Il timestamp permette di calcolare questi valori.

Il timestamp permette anche di implementare cooldown. Se una bobina viene usata, il nodo può impedire un nuovo impulso fino a quando non è passato un tempo minimo di raffreddamento. Questo può essere espresso così:

se `tempo_corrente - last_coil_off_time < cooldown_ms`, rifiuta `magnetic_pulse`.

Questo protegge l'hardware. La comunicazione può chiedere un comando, ma il nodo decide se è temporalmente sicuro eseguirlo. Il timestamp e i timer locali rendono possibile questa decisione.

Il timestamp è utile anche per gestire comandi ripetuti. Un nodo può ricevere molti comandi simili in poco tempo. Alcuni possono essere accettati, altri limitati. Per esempio, `set_led` può essere aggiornato frequentemente senza problemi seri, ma `magnetic_pulse` deve avere limiti. Il nodo può applicare rate limiting, cioè limitare la frequenza di certi comandi. Anche qui serve il tempo:

se `numero_comandi_magnetic_pulse` negli ultimi 10 secondi `> limite`, rifiuta.

Questa regola protegge il sistema da sovraccarico e abuso accidentale.

Dal punto di vista della dashboard, il timestamp permette di mostrare non solo lo stato corrente, ma anche lo stato storico. Si possono visualizzare grafici nel tempo: batteria, temperatura stimata, latenza, numero di pacchetti, connessioni, errore pattern, eventi safety. Ogni punto del grafico richiede timestamp. La telemetria senza timestamp non può generare serie temporali affidabili. Questo è importante perché MicroBot punta a un ecosistema osservabile, non a una demo cieca.

Una dashboard avanzata può mostrare:

timeline comandi;

timeline eventi;

grafico heartbeat;

grafico RTT;

grafico `thermal_load`;

durata connessioni;

tempo di convergenza del pattern.

Tutte queste visualizzazioni dipendono dal timestamp.

Il timestamp consente anche replay e analisi offline. Se una run viene registrata con timestamp, può essere riprodotta rispettando i tempi originali oppure accelerata/rallentata. Questo è estremamente utile per documentazione, debug e presentazione. Si può mostrare come lo swarm ha evoluto la propria forma, quando i nodi hanno ricevuto comandi, quando sono avvenuti gli agganci e dove si è generato un errore. Un replay senza timestamp sarebbe solo una lista di stati senza ritmo temporale.

Un file di replay può contenere:

run_id;

timebase;

events ordinati per timestamp;

snapshots ordinati per timestamp;

commands ordinati per timestamp;

telemetry ordinata per timestamp.

Questa struttura trasforma una demo in un esperimento analizzabile. Nel tuo progetto, dove la documentazione e i log sono centrali, questa è una direzione molto forte.

Il timestamp deve essere usato anche nei confronti tra versioni. Se si testa lo stesso pattern in versioni diverse del simulatore, si può confrontare tempo di convergenza, numero di distacchi per unità di tempo, latenza media, durata delle connessioni e frequenza di errori. Queste metriche sono temporali. Senza timestamp, si potrebbero contare eventi, ma non misurare quando e con quale ritmo avvengono. Il tempo permette una valutazione molto più tecnica.

Per esempio:

versione A converge in 12.5 secondi;

versione B converge in 8.1 secondi;

versione A ha 15 detach_event in 30 secondi;

versione B ne ha 4;

versione B è quindi più stabile o più adatta al pattern testato.

Queste conclusioni richiedono dati temporizzati.

Il timestamp può anche aiutare a distinguere causa ed effetto. In sistemi complessi, molti eventi avvengono rapidamente. Se un nodo entra in safety dopo un comando, bisogna capire se il safety è stato causato da quel comando oppure se era già attivo prima. Ordinare temporalmente gli eventi aiuta a ricostruire causalità. Naturalmente, il timestamp da solo non prova causalità, ma fornisce una base per analizzarla. Senza tempo, causa ed effetto si confondono.

Un esempio:

123.00 — comando magnetic_pulse inviato;

123.04 — nodo accetta;

123.05 — coil attiva;

123.12 — thermal_load warning;

123.14 — safety_triggered;

123.15 — coil disattivata.

Questa sequenza mostra una possibile relazione tra impulso e safety. Se invece `safety_triggered` fosse avvenuto a 122.90, allora il comando sarebbe stato successivo a una condizione già critica. Il timestamp chiarisce l'ordine.

Il timestamp deve essere gestito con precisione anche nel caso di messaggi broadcast. Un comando broadcast può essere generato una sola volta con un timestamp comune. I nodi lo ricevono in tempi diversi, ma il messaggio contiene lo stesso riferimento. Questo permette al controller di confrontare le risposte dei nodi rispetto alla stessa origine. Se MB-001 risponde dopo 20 ms e MB-002 dopo 80 ms, il controller può vedere differenze di latenza. Se il comando include `execute_at`, può vedere chi lo ha eseguito in tempo.

Per comandi broadcast critici, ogni ack dovrebbe includere:

`ref_message_id`;

`source_node_id`;

`received_at`;

`executed_at` se applicabile;

`result`.

Così il controller può costruire una tabella di esecuzione:

MB-001: received 20 ms, executed +5 ms;

MB-002: received 65 ms, executed +7 ms;

MB-003: expired;

MB-004: no ack.

Questa tabella è molto utile per valutare lo stato dello swarm.

Il timestamp è importante anche per la modalità unicast. Quando il controller invia comandi individuali a molti nodi in sequenza, ogni comando può avere timestamp diverso. Se l'obiettivo è esecuzione simultanea, non bisogna affidarsi al tempo di invio, perché i comandi vengono spediti in momenti leggermente diversi. Bisogna usare `execute_at` comune. Questo è un principio fondamentale:

per esecuzione simultanea, usare `execute_at` comune, non "invio rapido in sequenza".

Se il controller invia cinquanta comandi uno dopo l'altro con "execute now", il primo nodo può agire prima dell'ultimo. Se tutti i comandi contengono `execute_at = T`, i nodi possono eseguire insieme. Questa è una differenza cruciale tra invio e coordinamento.

Il timestamp può includere anche precisione. Non tutti i campi temporali richiedono la stessa risoluzione. Per heartbeat può bastare millisecondi o anche decimi di secondo. Per impulsi magnetici brevi, possono servire millisecondi. Per analisi di performance, millisecondi sono spesso sufficienti nelle prime versioni. Microsecondi possono essere utili in firmware, ma non sempre necessari nel protocollo JSON. La risoluzione deve essere coerente con le capacità reali del sistema.

Una scelta pratica iniziale è usare millisecondi come unità principale per durata e timing operativo. Per timestamp assoluti, si può usare secondi con decimali o millisecondi interi. L'importante è documentare l'unità. Uno degli errori più pericolosi nei sistemi temporali è confondere secondi e millisecondi. Ogni campo deve avere nome chiaro: `duration_ms`, `timeout_ms`, `cooldown_ms`. Questo elimina ambiguità.

Per esempio:

`duration_ms = 120` indica 120 millisecondi;

`timestamp_ms = 123456` indica tempo in millisecondi;

`timestamp = 123.456` può indicare secondi con decimali, ma deve essere dichiarato.

Per evitare confusione, nei payload è spesso meglio usare suffissi espliciti come `_ms`.

Il timestamp deve essere trattato con attenzione nei formati JSON, perché numeri molto grandi possono avere problemi di precisione in alcuni ambienti JavaScript se superano certi limiti. Per timestamp molto grandi in millisecondi assoluti, può essere utile usare stringhe oppure separare tempo relativo e data leggibile. Nelle prime versioni locali, questo problema può essere marginale, ma è bene essere consapevoli. Usare tempo monotono relativo alla sessione riduce il rischio.

Una soluzione robusta può essere:

`session_start_datetime` = data assoluta salvata nei log;

`timestamp_ms` = millisecondi dall'inizio della sessione.

Così i pacchetti interni usano numeri più piccoli e i log possono ricostruire la data assoluta sommando l'origine. Questa struttura è molto adatta a esperimenti e replay.

Il timestamp è inoltre legato alla nozione di sessione. Una run di MicroBot può avere `run_id` e `start_time`. Tutti gli eventi della run possono usare timestamp relativo a `start_time`. Questo rende facile confrontare run diverse. Per esempio, in ogni run, `t = 0` è l'avvio del pattern. Poi si misura cosa succede a `t = 1 s`, `t = 2 s`, ecc. Questo è molto più utile per analisi sperimentale rispetto a date assolute.

Una run può essere descritta così:

`run_id = "line_run_001";`

`start_time = "2026-04-30T21:00:00+02:00";`

`timebase = "run_time_ms";`

events con `timestamp_ms` relativo.

Questo formato permette di analizzare la dinamica del pattern indipendentemente dall'orario reale.

Il timestamp deve essere coerente con il controllo di validità. Un comando può avere sia timestamp sia `sequence_number`. Questi due campi svolgono funzioni diverse. Il timestamp indica tempo; il `sequence_number` indica ordine logico. In molti casi lavorano insieme. Se due comandi hanno sequence diverso ma timestamp simile, il sequence aiuta a scegliere l'ultimo. Se un comando ha sequence alto ma timestamp vecchio, bisogna applicare la policy del sistema. In generale, per comandi di stato, `sequence_number` protegge dall'ordine fuori sequenza; timestamp protegge dalla vecchiaia. Entrambi sono utili.

Una regola può essere:

rifiuta se timestamp troppo vecchio;

rifiuta se `sequence_number` inferiore all'ultimo accettato per quel tipo di comando;

accetta solo se entrambi sono validi.

Questo rende il protocollo più robusto sopra UDP.

Il timestamp deve essere usato anche per comandi duplicati. Se due messaggi hanno stesso message_id, il nodo li tratta come duplicati. Se hanno ID diversi ma stesso contenuto e timestamp molto vicino, possono essere comandi separati oppure ripetizioni non riconosciute. Per evitare ambiguità, le ritrasmissioni devono mantenere lo stesso message_id o usare retry_of. Il timestamp aiuta, ma l'identificatore resta la base principale della deduplicazione. Il tempo da solo non basta.

Il timestamp ha un ruolo anche nella gestione di stati "stale". Un dato può essere tecnicamente presente, ma vecchio. Per esempio, la dashboard può avere l'ultimo valore di batteria di MB-005, ma se è stato ricevuto trenta minuti fa non rappresenta più lo stato attuale. Il sistema deve quindi distinguere valore valido e valore stale. Ogni stato derivato dalla telemetria dovrebbe avere timestamp. Se il valore supera una soglia di età, viene marcato come stale.

Una regola:

se $\text{current_time} - \text{telemetry_timestamp} > \text{telemetry_stale_timeout}$, marcare il dato come stale.

Questo aiuta la dashboard a non mostrare dati vecchi come se fossero attuali. Può mostrare "last update: 35 s ago" o cambiare stato visivo del nodo.

Il timestamp può essere usato anche per heartbeat jitter. Se gli heartbeat arrivano a intervalli irregolari, il controller può misurare la variazione. Un nodo che invia heartbeat ogni secondo ma arriva con intervalli 1.0, 1.1, 0.9, 1.0 è stabile; uno che arriva con 0.5, 2.8, 0.3, 4.0 è instabile. Questa informazione può influenzare il ruolo del nodo. Un nodo con comunicazione instabile non dovrebbe essere leader di una sequenza critica.

Il timestamp quindi contribuisce all'adattamento del sistema. Non è solo log; è input per decisioni.

Dal punto di vista della comunicazione a bassa latenza, il timestamp permette di separare ritardo di rete e ritardo di processing. Se il nodo include received_at e processed_at, si può capire se il ritardo è avvenuto sulla rete o nel firmware. Per esempio, se il pacchetto arriva rapidamente ma viene processato molto dopo, il problema è nel loop del nodo. Se arriva già tardi, il problema è rete o invio. Questa distinzione è molto utile per debugging.

Un ack avanzato può includere:

received_at_local;

processed_at_local;

executed_at_local.

Il controller può analizzare i tempi relativi. Anche se la sincronizzazione non è perfetta, il nodo può misurare intervalli interni: $\text{processing_delay} = \text{processed_at} - \text{received_at}$. Questo valore non richiede clock globale. È utilissimo per capire se il firmware è reattivo.

Il timestamp deve essere integrato con eventuali watchdog. Se il nodo non aggiorna certe funzioni entro tempi previsti, il watchdog può intervenire. Il firmware può usare timestamp locali per verificare che il loop principale sia vivo, che i comandi temporizzati vengano chiusi, che heartbeat vengano inviati e che uscite attive non superino durata massima. Anche qui il tempo è fondamento della sicurezza.

Un controllo può essere:

se output_active e $\text{current_time} > \text{output_end_time}$, spegni output.

Questo controllo deve essere eseguito anche se non arrivano messaggi. È indipendente dalla rete. Il timestamp locale sostiene l'autonomia del nodo.

Nel sistema MicroBot, il timestamp deve essere trattato come parte della struttura del pacchetto, ma anche come parte dello stato interno. Ogni nodo deve conservare tempi rilevanti: ultimo comando ricevuto, ultimo heartbeat inviato, ultimo pacchetto valido, ultima attivazione coil, ultimo errore, ultimo cambio modalità, ultimo sync, ultimo evento. Questi valori permettono comportamento robusto e telemetria significativa.

Uno stato interno temporale può includere:

`last_command_time;`

`last_heartbeat_time;`

`last_sync_time;`

`last_error_time;`

`last_mode_change_time;`

`last_output_activation_time;`

`last_safety_trigger_time.`

Questi campi non devono necessariamente essere tutti inviati sempre, ma il firmware può usarli per decisioni locali e inviarli in diagnostica.

Il timestamp ha anche un valore metodologico nella documentazione. Un progetto tecnico serio non descrive solo cosa il sistema fa, ma quando lo fa. Le sequenze temporali mostrano maturità ingegneristica. Per esempio, dire “il nodo riceve un comando e attiva il LED” è generico. Dire “il nodo riceve il comando, lo valida entro un ciclo, lo programma per `execute_at`, lo esegue per `duration_ms` e invia `ack` con `executed_at`” è molto più rigoroso. Il timestamp rende questa precisione possibile.

Nel caso di MicroBot, dove la documentazione deve crescere verso una forma da tesi o relazione tecnica completa, il timestamp permette di collegare comunicazione, sincronizzazione, telemetria e controllo. È un campo piccolo nel pacchetto, ma ha conseguenze enormi sull'intero sistema.

Dal punto di vista teorico, il timestamp è ciò che permette al sistema di essere interpretato come processo. Tutta la visione MicroBot è dinamica: la forma cambia, le connessioni cambiano, i nodi cambiano stato, il comportamento emerge nel tempo. Senza tempo, si avrebbe soltanto una collezione di stati statici. Con il timestamp, si ha una traiettoria. Lo swarm non è solo una struttura; è una sequenza ordinata di trasformazioni. Il timestamp è il filo che tiene insieme questa sequenza.

In conclusione, il timestamp è un elemento essenziale della comunicazione MicroBot. Esso permette di datare pacchetti, comandi, heartbeat, telemetria, `ack`, errori ed eventi; consente di verificare la freschezza dei messaggi, scartare comandi scaduti, misurare latenza, gestire `execute_at`, applicare `duration_ms`, costruire timeline, registrare log, sincronizzare nodi e controllare la sicurezza locale. Il timestamp collega il protocollo alla dinamica dello swarm, trasformando i messaggi in eventi ordinati nel tempo. Senza timestamp, il sistema sarebbe comunicativo ma non realmente coordinabile; con timestamp, diventa possibile costruire comportamento distribuito, sequenze temporizzate, pattern sincronizzati e analisi tecnica delle run. Il punto successivo, dedicato alla modalità broadcast, analizzerà come un singolo messaggio possa essere inviato contemporaneamente a tutti i MicroBot, permettendo comandi globali, sincronizzazione dello swarm e controllo collettivo efficiente.

8.9 Modalità broadcast

La modalità broadcast rappresenta il meccanismo attraverso cui il controller può inviare un singolo messaggio destinato a tutti i MicroBot presenti nella rete, senza dover generare un comando separato per ciascun nodo. Dopo aver definito identificatore, comando e timestamp, è necessario analizzare il broadcast perché esso costituisce una delle modalità più importanti per coordinare uno

swarm distribuito. In un sistema composto da molte unità, infatti, alcune istruzioni non riguardano un singolo MicroBot, ma l'intero gruppo: avvio di una modalità comune, sincronizzazione temporale, stop globale, aggiornamento di stato, cambio di fase, inizio di un pattern o comando di sicurezza. In questi casi, il broadcast permette di comunicare in modo più efficiente e coerente.

Nel contesto MicroBot, il broadcast può essere interpretato come una comunicazione uno-a-molti. Il controller genera un pacchetto, lo invia sulla rete con destinazione generale, e tutti i nodi che ascoltano sulla porta prevista possono riceverlo. A livello logico, il pacchetto contiene un campo target impostato a un valore globale, ad esempio:

```
target = "all"
```

Questo valore indica che il messaggio non è destinato a un nodo specifico, ma a tutti i MicroBot compatibili. Ogni nodo che riceve il pacchetto deve comunque validarlo: deve controllare protocollo, versione, sorgente, timestamp, tipo di comando, payload, safety e stato locale. Il fatto che un messaggio sia broadcast non significa che debba essere eseguito ciecamente da tutti. Significa soltanto che è stato distribuito a tutti. L'esecuzione resta responsabilità del nodo.

La modalità broadcast è particolarmente utile per i comandi globali. Un comando come `enter_idle`, per esempio, può essere inviato a tutti i nodi quando il sistema deve passare a una condizione stabile. Invece di inviare dieci, venti o cento pacchetti unicast, il controller invia un solo pacchetto broadcast. Ogni MicroBot riceve il comando, verifica che sia valido e, se lo stato locale lo consente, entra in idle. Questo riduce il traffico e rende più uniforme la distribuzione del comando. La stessa logica vale per comandi come `stop_pattern`, `sync_time`, `set_global_mode` o `emergency_stop`.

Un esempio di pacchetto broadcast può essere:

```
{
  "protocol": "microbot_udp",
  "version": "0.1",
  "message_id": "CMD-BCAST-000001",
  "sequence": 102,
  "timestamp": 123456.00,
  "source": "CTRL-001",
  "target": "all",
  "type": "command",
  "command": "enter_idle",
  "payload": {
    "reason": "manual_stop"
  },
  "requires_ack": true
}
```

In questo esempio, il comando è destinato a tutti i nodi. Ogni MicroBot può rispondere con un ack contenente il proprio `node_id` e il riferimento al `message_id` originale. In questo modo, il controller può sapere quali nodi hanno ricevuto e accettato il comando. Il broadcast riduce l'invio iniziale, ma non elimina la necessità di osservare le risposte, soprattutto quando il comando è importante.

Una caratteristica fondamentale del broadcast è che non garantisce automaticamente che tutti i nodi ricevano il pacchetto. Questo punto è essenziale. Su una rete UDP/Wi-Fi, un messaggio broadcast può essere perso da uno o più dispositivi. La perdita può dipendere da congestione, interferenza, distanza, buffer pieni, ritardi del firmware o condizioni temporanee della rete. Per questo motivo, il

sistema MicroBot non deve assumere che “broadcast inviato” significhi “comando ricevuto da tutti”. Deve distinguere tra invio del comando e conferma effettiva da parte dei nodi.

Questa distinzione porta a una regola importante:

broadcast = distribuzione efficiente del comando;
ack individuali = verifica della ricezione e dell'accettazione.

Per comandi non critici, il controller può anche non richiedere ack da tutti. Per esempio, un cambio di colore LED puramente dimostrativo può essere inviato in broadcast senza conferma obbligatoria. Se un nodo perde il pacchetto, il problema è limitato. Per comandi importanti, come `start_pattern`, `sync_time` o `enter_idle`, l'ack è consigliato. Per comandi critici, come `emergency_stop` o `disable_outputs`, il comportamento deve essere ancora più prudente: il controller può inviare il broadcast più volte, i nodi devono eseguire immediatamente quando lo ricevono e, se possibile, inviare conferma.

Il broadcast è quindi utile, ma deve essere gestito con politiche diverse in base alla criticità del comando. Una classificazione può essere:

broadcast informativo: non richiede ack obbligatorio;

broadcast operativo: richiede ack consigliato o parziale;

broadcast sincronizzato: richiede timestamp, `execute_at` e ack dai nodi coinvolti;

broadcast di sicurezza: deve avere priorità massima, può essere ripetuto, e deve produrre disattivazione locale immediata.

Questa classificazione evita di trattare tutti i broadcast nello stesso modo. Un messaggio di debug globale non ha lo stesso peso di un comando di emergenza. Il protocollo deve riflettere questa differenza.

La modalità broadcast è molto importante per la sincronizzazione. Se il controller deve aggiornare il riferimento temporale dei nodi, può inviare un pacchetto `sync` in broadcast. Ogni nodo lo riceve, registra il tempo di ricezione e risponde eventualmente con un `sync_response`. Anche se il broadcast non fornisce sincronizzazione perfetta da solo, è un modo efficiente per distribuire un riferimento comune. La Sezione 9 approfondirà la sincronizzazione, ma già qui è importante osservare che il broadcast è il canale naturale per inviare segnali temporali globali.

Un pacchetto broadcast di sincronizzazione può contenere:

```
{  
  "type": "sync",  
  "message_id": "SYNC-000042",  
  "source": "CTRL-001",  
  "target": "all",  
  "timestamp": 123500.00,  
  "payload": {  
    "controller_time": 123500.00,  
    "sync_round": 42  
  }  
}
```

Ogni nodo può usare questo messaggio per aggiornare la propria stima del tempo del controller. Tuttavia, dato che ogni nodo può riceverlo con latenza diversa, la sincronizzazione precisa richiede ulteriori misure, come round-trip time, offset stimati o comandi temporizzati. Il broadcast distribuisce il riferimento, ma non elimina il problema della latenza.

Il broadcast è anche molto utile per comandi con `execute_at`. In questo caso, il controller invia lo stesso comando a tutti i nodi, ma l'esecuzione non avviene al momento della ricezione. Avviene a un tempo futuro comune. Questo riduce l'effetto della latenza variabile tra nodi. Per esempio, se si vuole che tutti i LED lampeggino insieme, il controller può inviare:

```
execute_at = 123510.00
```

I nodi ricevono il comando in momenti leggermente diversi, ma lo eseguono quando il proprio clock raggiunge il tempo indicato. Questo trasforma il broadcast da semplice comando immediato a strumento di coordinamento temporale.

Una struttura può essere:

```
{
  "command": "set_led_pattern",
  "target": "all",
  "timestamp": 123509.00,
  "execute_at": 123510.00,
  "payload": {
    "pattern": "sync_flash",
    "color": "cyan",
    "duration_ms": 300
  }
}
```

In questo caso, il controller invia il comando con anticipo. I nodi lo programmano. Se un nodo lo riceve dopo `execute_at` più tolleranza, lo rifiuta come expired. Questo comportamento è molto più robusto rispetto a un broadcast immediato.

La modalità broadcast è particolarmente adatta ai comandi di fase. Uno swarm può attraversare fasi operative comuni: idle, prepare, align, attach, stabilize, release, diagnostic. Quando il sistema passa da una fase all'altra, può essere efficiente inviare un messaggio broadcast che aggiorna tutti i nodi. Per esempio:

```
phase = "align"
```

oppure:

```
phase = "stabilize"
```

Ogni nodo può interpretare la fase in base al proprio ruolo. Un nodo leader, un follower e un nodo di bordo possono ricevere lo stesso messaggio broadcast, ma reagire in modo diverso. Questo è un punto molto importante: il broadcast può distribuire un'informazione globale comune, mentre l'azione concreta resta differenziata localmente. In questo modo, si combina controllo globale e autonomia multi-agente.

Un comando broadcast di fase può essere:

```
{
"command": "set_swarm_phase",
"target": "all",
"payload": {
"phase": "align",
"pattern_run_id": "line_run_001"
}
}
```

Tutti i nodi ricevono la stessa fase. Il nodo con ruolo leader può iniziare una funzione di orientamento principale; i follower possono allinearsi; i nodi in safety possono rifiutare e comunicare il proprio stato; i nodi offline ovviamente non rispondono. Il broadcast non elimina la diversità di stato tra nodi, ma fornisce un riferimento comune.

La modalità broadcast è utile anche per l'avvio di pattern collettivi. Se il controller vuole avviare una formazione lineare, circolare o di aggregazione, può inviare un comando `start_pattern` con target `all` o con target di gruppo. Il messaggio contiene `pattern_id`, `pattern_run_id` e tempo di esecuzione. I nodi che partecipano al pattern lo accettano, quelli non compatibili o non disponibili lo rifiutano. Il controller raccoglie gli `ack` e decide se procedere, ritardare o abortire la run.

Un possibile flusso è:

controller invia `start_pattern broadcast`;

i nodi rispondono `accepted`, `rejected` o `unavailable`;

controller valuta quanti nodi sono pronti;

se il numero è sufficiente, invia `confirm_start` o prosegue con `execute_at`;

se troppi nodi falliscono, abortisce o modifica il pattern.

Questa logica rende il broadcast parte di un protocollo più ampio. Non basta inviare “start”; bisogna sapere se lo swarm è realmente pronto. Il broadcast è la fase di distribuzione, gli `ack` sono la fase di verifica.

Nel caso di comandi critici di sicurezza, il broadcast ha una funzione ancora più importante. Un comando `emergency_stop` deve raggiungere tutti i nodi nel modo più rapido possibile. Inviare comandi unicast uno per uno potrebbe introdurre ritardi tra il primo e l'ultimo nodo. Un broadcast permette di diffondere immediatamente il messaggio. Tuttavia, poiché UDP broadcast non garantisce consegna, il controller può ripetere il messaggio per un breve intervallo e i nodi devono avere anche timeout locali indipendenti. La sicurezza non può dipendere da un singolo pacchetto.

Un `emergency_stop broadcast` deve avere proprietà precise:

payload minimo;

priorità massima;

validazione rapida;

esecuzione immediata;

disattivazione locale delle uscite;

annullamento dei comandi pendenti;

ack se possibile, ma non prima della messa in sicurezza.

L'ordine è fondamentale. Il nodo non deve attendere di inviare ack prima di spegnere una bobina o disattivare un'uscita. La risposta viene dopo la messa in sicurezza. Il broadcast di emergenza è quindi un comando globale orientato alla protezione, non alla normale coordinazione.

La modalità broadcast può generare un problema chiamato ack storm, cioè un picco di risposte quando molti nodi rispondono contemporaneamente allo stesso messaggio. Se il controller invia un broadcast con `requires_ack = true` a molti MicroBot, tutti potrebbero rispondere nello stesso istante. Questo può congestionare la rete, aumentare collisioni e causare perdita di pacchetti. Con pochi nodi il problema è ridotto; con swarm più grandi diventa importante. Per questo motivo, il protocollo deve prevedere strategie per distribuire le risposte.

Una strategia semplice è introdurre un piccolo ritardo casuale controllato negli ack non critici. Per esempio, ogni nodo aspetta un tempo casuale tra 0 e 100 ms prima di inviare l'ack. Questo distribuisce le risposte. Un'altra strategia è usare slot di risposta basati su `node_id`. Per esempio, MB-001 risponde nel primo slot, MB-002 nel secondo, e così via. Una terza strategia è richiedere ack solo a nodi selezionati in modalità normale e a tutti solo in modalità diagnostica. La scelta dipende dal numero di nodi e dalla criticità.

Una logica slot-based può essere:

$$\text{ack_delay_ms} = \text{node_index} \times \text{slot_duration_ms}$$

Se `slot_duration_ms` è 10 ms, MB-001 risponde dopo 10 ms, MB-002 dopo 20 ms, MB-003 dopo 30 ms. Questa soluzione è ordinata ma richiede che ogni nodo conosca o possa derivare il proprio indice. Una soluzione con jitter casuale è più semplice ma meno deterministica. Per le prime versioni, un ritardo casuale leggero può bastare. Per versioni più mature, gli slot sono più eleganti.

Per i comandi di emergenza, però, il discorso cambia. L'azione deve essere immediata; soltanto l'ack può essere ritardato. Questo significa:

esecuzione `emergency_stop`: immediata;

ack `emergency_stop`: eventualmente distribuito per evitare congestione.

Questa distinzione mantiene sicurezza e gestione della rete.

Il broadcast deve essere associato a un identificatore di messaggio comune. Quando il controller invia un broadcast, tutti i nodi ricevono lo stesso `message_id`. Gli ack devono fare riferimento a quel `message_id`. Questo permette al controller di costruire una tabella di risposta. Per esempio:

CMD-BCAST-000010 inviato a all;

MB-001 accepted;

MB-002 accepted;

MB-003 rejected low_energy;

MB-004 no_response;

MB-005 accepted.

Questa tabella è essenziale per sapere lo stato dello swarm. Il broadcast non produce automaticamente certezza; la certezza viene dalla raccolta degli esiti.

Il controller può mantenere per ogni broadcast una `broadcast_response_table`. Essa può contenere:

`message_id`;
`command`;
`target`;
`sent_at`;
`expected_nodes`;
`responses_received`;
`missing_nodes`;
`rejected_nodes`;
`timeout_at`;
`overall_status`.

`overall_status` può essere `completed`, `partial`, `failed`, `timed_out` o `unsafe`. Se il comando era `start_pattern` e alcuni nodi non hanno risposto, il controller può decidere se continuare con un pattern ridotto o fermarsi. Se il comando era `emergency_stop`, il controller può segnalare nodi senza `ack` ma non deve necessariamente assumere che non abbiano eseguito: potrebbero aver eseguito ma non risposto. La telemetria successiva aiuterà a chiarire.

Il broadcast deve considerare anche i nodi non registrati. Un nodo appena acceso può ricevere un broadcast prima di aver completato `discovery`. Se il comando è generale ma il nodo non è ancora registrato, deve decidere cosa fare. Una politica prudente è accettare solo comandi di sicurezza e `discovery` finché la registrazione non è completa. Per esempio, `emergency_stop` può essere sempre valido; `start_pattern` richiede registrazione e sincronizzazione; `set_mode` può richiedere che il nodo sia `known` dal controller. Questo evita che nodi appena entrati eseguano comandi senza contesto.

Una regola può essere:

se nodo non registrato: accetta `discovery`, `sync base`, `emergency_stop`; rifiuta `pattern` e comandi attuativi.

Questa regola rende il sistema più sicuro. Il nodo deve prima essere riconosciuto, sincronizzato e configurato prima di partecipare a comandi complessi.

Il broadcast può essere usato per `discovery`. Il controller invia `discovery_request` in broadcast; tutti i MicroBot presenti rispondono con `discovery_response`. Questo è uno dei casi più naturali di broadcast, perché il controller non conosce ancora gli indirizzi dei nodi. Il broadcast permette di trovare chi è presente. Ogni nodo risponde con `node_id`, `firmware_version`, `capabilities`, `boot_id` e stato. Il controller aggiorna la tabella dei nodi.

Una sequenza di discovery è:

controller → broadcast discovery_request;

nodi → risposte individuali discovery_response;

controller → registra o aggiorna nodi;

dashboard → mostra swarm disponibile.

Anche qui, per evitare ack storm, le risposte discovery possono essere ritardate leggermente in modo casuale o slot-based. Con pochi nodi non è un problema, ma è bene progettare già la logica.

Il broadcast può essere usato anche per aggiornare parametri globali. Per esempio, il controller può inviare un nuovo valore di telemetry_level a tutti i nodi, riducendo o aumentando il traffico. Può inviare un nuovo global_safety_policy, un nuovo pattern_timebase o una nuova modalità di logging. Tuttavia, i parametri globali devono essere validati. Un nodo può rifiutare parametri incompatibili con la propria versione firmware o capabilities. Anche in broadcast, la risposta resta individuale.

Un comando globale di telemetria può essere:

```
{  
  "command": "set_telemetry_level",  
  "target": "all",  
  "payload": {  
    "level": "normal",  
    "interval_ms": 1000  
  }  
}
```

Ogni nodo aggiorna la propria frequenza di telemetria se il valore è valido. Se un nodo è in low_energy, potrebbe decidere di usare un intervallo più ampio e rispondere con accepted_modified o rejected_low_energy. Questo dimostra che il broadcast non elimina l'autonomia locale.

Nel sistema MicroBot, il broadcast deve distinguere tra all e all_available. target = all può indicare tutti i nodi che ricevono il messaggio. target = all_available, se introdotto, potrebbe significare tutti i nodi registrati e operativi. A livello di rete, entrambi possono essere broadcast; a livello logico, il nodo decide se appartiene alla categoria. Per le prime versioni, target = all è sufficiente. In futuro, si possono introdurre target più specifici.

Un'estensione naturale è il broadcast di gruppo, o groupcast logico. Tecnicamente il controller può inviare il pacchetto in broadcast sulla rete, ma il campo target indica group_id = cluster_A. Tutti i nodi ricevono il pacchetto, ma solo quelli appartenenti a cluster_A lo eseguono. Questo è molto utile perché evita di inviare molti unicast e permette di coordinare sottogruppi. Anche se l'indice distingue solo broadcast e unicast, il groupcast può essere documentato come estensione del broadcast filtrato logicamente.

Un esempio:

```
target = "group:cluster_A"
```

Il nodo verifica se appartiene a cluster_A. Se sì, processa. Se no, ignora. Questa soluzione è semplice e potente. Permette di controllare cluster, segmenti di pattern, sottostrutture o gruppi diagnostici.

La modalità broadcast deve essere collegata alla sicurezza dei comandi. Un comando broadcast sbagliato può influenzare molti nodi contemporaneamente. Questo lo rende potente ma anche rischioso. Per questo motivo, i comandi broadcast attuativi devono essere ancora più controllati. Un comando magnetic_pulse broadcast a tutti i nodi, per esempio, potrebbe attivare molte bobine simultaneamente, con consumo energetico e rischio termico elevati. Tale comando deve essere permesso solo in condizioni molto chiare, con limiti e safety locale. In molti casi, è meglio evitare broadcast attuativi diretti e usare comandi di pattern più controllati.

Una regola prudente è:

broadcast consentito liberamente per stato, sync, LED, discovery, stop e telemetria;
broadcast attuativo consentito solo con durata, limiti, execute_at, safety locale e ack;
broadcast magnetico globale da usare con cautela e preferibilmente in modalità test o pattern controllato.

Questa regola mostra consapevolezza ingegneristica. Non tutto ciò che è tecnicamente possibile è sempre progettualmente corretto.

Il broadcast può essere usato per stop_all. stop_all è diverso da emergency_stop. stop_all può indicare interruzione ordinata del comportamento corrente, spegnimento di pattern e ritorno a idle. emergency_stop indica condizione di sicurezza massima. Entrambi possono essere broadcast, ma hanno semantica diversa. stop_all può essere graceful; emergency_stop deve essere immediato e prioritario. La documentazione deve mantenere questa distinzione.

Un comando stop_all può avere:

```
reason = "end_demo";
```

```
mode_after_stop = "idle";
```

```
preserve_logs = true.
```

Un emergency_stop può invece avere payload minimo o nullo. Più è semplice, meglio è. Il nodo deve riconoscerlo anche in condizioni di stress.

Il broadcast è utile anche per la modalità demo. Se si vuole mostrare lo swarm a un osservatore, il controller può inviare comandi broadcast per far lampeggiare tutti i nodi, avviare una sequenza visiva, sincronizzare LED, entrare in modalità pattern o mostrare stato. Questa parte non è secondaria: un sistema distribuito deve essere comunicabile visivamente. I broadcast LED permettono di dimostrare subito che tutti i nodi sono connessi e rispondono. Prima ancora di avere attuazione fisica completa, una demo broadcast con LED sincronizzati può mostrare rete, identificazione, timestamp e coordinamento.

Una demo iniziale può essere:

```
discovery broadcast;
```

```
sync broadcast;
```

```
set_led_pattern broadcast con execute_at;
```

nodi lampeggiano insieme;

dashboard mostra ack, latenza e stato.

Questa demo è semplice ma tecnicamente significativa: dimostra rete, protocollo, broadcast, timestamp, ack e dashboard. È una base molto forte prima di introdurre comandi magnetici.

Il broadcast deve essere progettato per non saturare la rete. Inviare broadcast continui può disturbare la comunicazione, specialmente se ogni messaggio richiede risposta. Il controller deve usare broadcast in modo intenzionale, non come sostituto di ogni comunicazione. Per aggiornamenti frequenti individuali, l'unicast o telemetria specifica può essere più adatta. Per segnali globali, broadcast è perfetto. La scelta deve dipendere dal contenuto.

Una regola pratica:

- usa broadcast quando molti nodi devono ricevere lo stesso messaggio;
- usa unicast quando il messaggio riguarda un nodo specifico;
- usa groupcast logico quando il messaggio riguarda un sottoinsieme;
- evita broadcast ripetuti ad alta frequenza se non necessari.

Questa regola mantiene efficiente il sistema.

Il broadcast deve avere gestione dei timeout. Dopo aver inviato un broadcast con ack richiesto, il controller non può aspettare indefinitamente. Deve definire una finestra di risposta. Alla scadenza, marca i nodi mancanti come `no_response` per quel comando. Questo non significa necessariamente che siano offline, ma significa che non hanno confermato entro la finestra. Il controller può poi decidere retry, fallback o errore. Il timeout deve dipendere dal comando e dalla dimensione dello swarm.

Una struttura può essere:

`broadcast_timeout_ms` = valore secondo criticità e numero nodi.

Per pochi nodi, può essere breve. Per molti nodi con ack distribuiti, deve essere più ampio. Per comandi temporizzati, il timeout deve essere compatibile con `execute_at`: bisogna raccogliere le risposte prima che sia troppo tardi per intervenire.

Il controller può anche usare retry del broadcast. Se mancano alcuni ack, può inviare di nuovo lo stesso `message_id` o un messaggio `retry_of`. La gestione deve evitare duplicati. Se un nodo ha già eseguito il comando, non deve rieseguirlo se riceve lo stesso broadcast. Questo richiede deduplicazione tramite `message_id`, come già discusso. Il broadcast rende la deduplicazione ancora più importante, perché lo stesso pacchetto può essere ripetuto per aumentare la probabilità di ricezione.

Una regola:

ritrasmissione dello stesso broadcast critico deve mantenere stesso `message_id` o dichiarare `retry_of`, così i nodi non ripetono effetti non idempotenti.

Per comandi idempotenti come `enter_idle`, il rischio è minore. Per `magnetic_pulse`, è essenziale.

Il broadcast deve essere collegato alla modalità low-energy. Un nodo con batteria bassa può ricevere broadcast, ma può rispondere meno frequentemente o rifiutare comandi non essenziali. Per esempio, può accettare `emergency_stop`, `request_status` e `set_telemetry_level` ridotto, ma rifiutare

start_pattern o magnetic_pulse. Anche se il messaggio è globale, la risposta è locale. Questo è coerente con il modello multi-agente.

Una risposta può essere:

```
result = "rejected";
```

```
reason = "low_energy";
```

```
ref_message_id = "CMD-BCAST-000020".
```

Il controller può usare questi rifiuti per adattare il pattern. Se troppi nodi sono low_energy, può fermare la demo o passare a una modalità visuale.

La modalità broadcast deve essere osservabile nella dashboard. Quando il controller invia un broadcast, la dashboard dovrebbe mostrare il comando globale, il numero di nodi attesi, il numero di ack ricevuti, i nodi mancanti, i rifiuti e il tempo di completamento. Questo rende la comunicazione globale trasparente. L'utente può vedere se il comando ha raggiunto tutto lo swarm o solo una parte. In una demo, questo è molto utile: mostra che il sistema non è una scatola nera, ma una rete monitorata.

Un pannello broadcast può mostrare:

```
message_id;
```

```
command;
```

```
target;
```

```
sent_at;
```

```
ack_received / expected;
```

```
accepted_nodes;
```

```
rejected_nodes;
```

```
missing_nodes;
```

```
timeout;
```

```
overall_result.
```

Questi dati derivano direttamente dagli identificatori e timestamp già definiti. La dashboard diventa così una rappresentazione operativa del protocollo.

Il broadcast è anche utile per resettare stati logici globali. Per esempio, prima di una nuova run, il controller può inviare reset_runtime_state a tutti i nodi. Ogni nodo cancella comandi pendenti non critici, resetta contatori temporanei e torna a uno stato noto. Questo è utile per evitare che una run precedente lasci residui logici. Tuttavia, reset_runtime_state deve essere distinto da reset hardware. Un reset logico è più sicuro e controllato; un reboot del microcontrollore è più invasivo.

Un comando di reset logico broadcast può contenere:

```
clear_pending_commands = true;
```

```
clear_pattern_state = true;
```

```
preserve_identity = true;
```

```
preserve_calibration = true.
```

Questo comando può preparare lo swarm a un nuovo test. Anche qui, ogni nodo deve rispondere con ack o errore.

Il broadcast può essere usato anche per aggiornare la modalità di logging. Durante test, il controller può inviare set_log_level debug a tutti. Durante funzionamento normale, può inviare set_log_level normal. Questo consente di aumentare o ridurre la quantità di dati senza riconfigurare manualmente ogni nodo. Tuttavia, log_level troppo alto su molti nodi può aumentare traffico e memoria. Il controller deve usarlo con attenzione.

Il broadcast è importante anche nel rapporto tra nodi fisici e simulati. In un ambiente misto, il controller può inviare un broadcast che raggiunge nodi simulati e fisici. Ogni nodo risponde secondo capabilities. Questo permette demo ibride. Per esempio, dieci nodi simulati e due ESP32 reali possono ricevere lo stesso comando set_led_pattern. La dashboard mostra tutti; i nodi reali lampeggiano fisicamente; quelli simulati lampeggiano sullo schermo. Questa coerenza rafforza l'ecosistema MicroBot.

Per evitare confusione, il campo target o payload può specificare:

```
include_simulated = true/false;
```

```
include_physical = true/false.
```

Oppure si può usare group_id separati: physical_nodes, simulated_nodes. Questo permette al controller di scegliere. In una fase di test, può inviare solo ai simulati; in una demo, a tutti.

Il broadcast deve essere documentato anche come parte del controllo indiretto della forma. Un comando broadcast non assegna necessariamente una traiettoria individuale. Può modificare un parametro globale che tutti i nodi usano localmente. Per esempio, aumentare cohesion_weight in simulazione, entrare in phase align, impostare pattern_id o aggiornare safety_policy. In questo modo, il controller agisce sullo swarm come insieme, non solo su nodi singoli. Questo è coerente con la visione di materia programmabile: modificare regole globali e lasciare che le interazioni locali producano la forma.

Un esempio:

```
controller broadcast set_behavior_parameters;
```

tutti i nodi o il simulatore aggiornano pesi locali;

le forze dello swarm cambiano;

la configurazione evolve.

Questa catena mostra come il broadcast possa essere un comando sistemico, non solo un messaggio di rete.

Dal punto di vista dei limiti, il broadcast non è adatto a messaggi personalizzati per ogni nodo. Se ogni nodo deve ricevere un target_position diversa, un singolo broadcast può funzionare solo se il

payload contiene una mappa di target per tutti i node_id. Tuttavia, questo può rendere il pacchetto grande e poco efficiente. In alternativa, il controller può inviare comandi unicast a ciascun nodo o usare un pattern_id noto localmente da cui ogni nodo ricava il proprio target. La scelta dipende dalla dimensione dello swarm e dalla complessità del pattern.

Una soluzione intermedia è:

broadcast pattern_id e role assignment già noto;

ogni nodo calcola o recupera il proprio target localmente.

Questo riduce traffico. In una prima versione, però, può essere più semplice inviare target individuali. Il documento deve riconoscere entrambe le possibilità.

Il broadcast può essere associato anche a mappe compatte. Per piccoli swarm, un broadcast con payload contenente target per tutti può essere accettabile:

```
targets = {  
  "MB-001": {...},  
  "MB-002": {...},  
  "MB-003": {...}  
}
```

Ogni nodo legge solo la propria parte. Questo è semplice ma non scala bene se i nodi diventano molti. Per pochi prototipi, è utile. Per swarm grandi, meglio comandi di pattern e calcolo locale.

La modalità broadcast deve essere coerente con il safety layer. Un nodo in safety può ricevere broadcast, ma non deve uscire da safety solo perché un comando globale lo richiede. Per esempio, se arriva start_pattern broadcast e il nodo è in thermal_lock, deve rifiutare. Se arriva enter_idle, può accettare. Se arriva emergency_stop, lo esegue. Questo significa che ogni comando broadcast deve passare attraverso la macchina a stati locale. Il broadcast non ha autorità assoluta sulle protezioni.

Una regola:

safety locale prevale su broadcast operativo.

Questa regola deve essere esplicita. È una delle basi della robustezza del sistema.

Il broadcast può aiutare anche a mantenere consistenza dello stato globale. Se il controller rileva che alcuni nodi hanno modalità incoerenti, può inviare un broadcast di reset o set_mode per riallineare lo swarm. Tuttavia, il controller deve poi verificare le risposte. Se un nodo non si riallinea, deve essere marcato come out_of_sync. Questo stato può essere mostrato nella dashboard e può impedire l'avvio di pattern complessi.

Un nodo out_of_sync è un nodo che è online ma non coerente con la fase globale. Può accadere se ha perso un broadcast precedente, se è stato riavviato o se ha rifiutato un comando. Il broadcast successivo può correggerlo, ma solo se il nodo è disponibile. La comunicazione globale deve quindi prevedere riconciliazione.

Una procedura di riconciliazione può essere:

controller invia state_sync broadcast;

nodi rispondono con stato locale;

controller confronta con stato atteso;

controller invia correzioni unicast ai nodi divergenti.

Questo mostra come broadcast e unicast possano lavorare insieme. Il broadcast raccoglie o imposta stato globale; l'unicast corregge casi specifici.

Il broadcast ha valore anche nella procedura di shutdown. Prima di spegnere il sistema o terminare una demo, il controller può inviare un broadcast di shutdown_prepare o enter_idle. I nodi disattivano uscite, salvano eventuale stato minimo, riducono telemetria e segnalano pronto. Poi il sistema può essere spento fisicamente. Questo è più ordinato che interrompere semplicemente alimentazione o rete.

Un comando shutdown_prepare può contenere:

```
disable_outputs = true;
```

```
set_led = "idle";
```

```
flush_logs = true se supportato;
```

```
telemetry_level = "heartbeat_only".
```

Nelle prime versioni, può bastare enter_idle. In versioni più avanzate, una procedura ordinata sarà utile.

Dal punto di vista teorico, il broadcast rappresenta la voce globale del controller. Mentre le interazioni locali producono comportamento emergente, il broadcast permette al sistema centrale di modificare simultaneamente il contesto in cui tali interazioni avvengono. È una forma di controllo macroscopico. Il controller non deve necessariamente dire a ogni nodo cosa fare nel dettaglio; può dichiarare una fase, una modalità, un parametro o un obiettivo comune. I nodi applicano localmente quella informazione. Questa è una forma elegante di coordinamento distribuito.

Il broadcast è quindi coerente con la natura ibrida di MicroBot: centralizzato nella definizione delle condizioni globali, distribuito nell'esecuzione e nella validazione locale. Il controller invia un comando globale; ogni nodo lo interpreta in base al proprio stato, ruolo e safety. Il risultato è un comportamento collettivo che non è né completamente centralizzato né completamente anarchico. È coordinato.

In conclusione, la modalità broadcast è una componente essenziale della comunicazione MicroBot. Essa permette al controller di inviare comandi globali, sincronizzazioni, cambi di fase, discovery, stop, aggiornamenti di stato e segnali di sicurezza a tutti i nodi in modo efficiente. Tuttavia, il broadcast non garantisce automaticamente ricezione universale: deve essere accompagnato da identificatori, timestamp, ack selettivi, timeout, retry controllati, deduplicazione e validazione locale. Il broadcast è potente perché consente controllo collettivo, ma proprio per questo deve essere usato con disciplina, soprattutto per comandi attuativi o critici. Nel sistema MicroBot, esso rappresenta il canale attraverso cui il controller parla allo swarm come insieme, mentre ogni nodo mantiene autonomia decisionale e sicurezza locale. Il punto successivo, dedicato alla modalità unicast, completerà la Sezione 8 analizzando il caso opposto: la comunicazione diretta verso un singolo MicroBot, necessaria per controllo individuale, diagnostica mirata, assegnazione di ruoli specifici e gestione precisa dei nodi all'interno dello swarm.

8.10 Modalità unicast

La modalità unicast rappresenta il meccanismo attraverso cui il controller comunica direttamente con un singolo MicroBot, inviando un messaggio destinato a un nodo specifico identificato dal proprio `node_id` e raggiunto, a livello di rete, tramite il suo indirizzo corrente. Dopo aver analizzato la modalità broadcast, cioè la comunicazione globale verso tutto lo swarm, è necessario completare la Sezione 8 descrivendo il caso opposto: la comunicazione mirata. Se il broadcast è utile per comandi collettivi, sincronizzazione, discovery e stop globali, l'unicast è indispensabile quando il sistema deve controllare, interrogare, configurare o diagnosticare una singola unità. In uno swarm maturo, infatti, non tutte le informazioni e non tutti i comandi sono globali. Ogni MicroBot può avere stato, ruolo, capacità, posizione, livello energetico, errori e compiti differenti. La comunicazione unicast permette di trattare ogni nodo come entità individuale all'interno del sistema distribuito.

Nel progetto MicroBot, l'unicast è fondamentale perché consente precisione operativa. Il controller può inviare un comando a MB-001 senza influenzare MB-002, MB-003 o altri nodi presenti nella rete. Questo è necessario per funzioni come identificazione fisica del nodo, calibrazione, assegnazione di ruolo, richiesta di stato dettagliato, test di un singolo LED, prova controllata di una bobina, aggiornamento di parametri locali, diagnostica di errore o correzione di uno stato incoerente. Mentre il broadcast parla allo swarm come insieme, l'unicast parla a un MicroBot specifico. Questa distinzione è essenziale perché MicroBot è sia sistema collettivo sia insieme di unità autonome.

Dal punto di vista logico, un pacchetto unicast contiene un campo `target` impostato al `node_id` del destinatario:

```
target = "MB-001"
```

Questo campo indica che il messaggio deve essere processato soltanto dal nodo MB-001. Anche se per qualche ragione il pacchetto fosse ricevuto da altri nodi, essi dovrebbero ignorarlo perché il `target` non corrisponde alla loro identità. A livello di rete, il controller può inviare il datagramma UDP direttamente all'indirizzo IP associato a MB-001 nella propria tabella dei nodi. A livello applicativo, però, l'identificazione finale resta il `node_id`. Questa separazione tra indirizzo IP e identità logica è importante perché l'indirizzo può cambiare, mentre il `node_id` deve restare stabile.

Una struttura di comando unicast può essere:

```
{
  "protocol": "microbot_udp",
  "version": "0.1",
  "message_id": "CMD-UNICAST-000245",
  "sequence": 245,
  "timestamp": 123456.00,
  "source": "CTRL-001",
  "target": "MB-001",
  "type": "command",
  "command": "identify_node",
  "payload": {
    "color": "cyan",
    "pattern": "blink",
    "duration_ms": 3000
  },
}
```

```
"requires_ack": true
}
```

Questo messaggio è chiaramente destinato a MB-001. Il nodo ricevente controlla protocollo, versione, message_id, timestamp, source, target, command e payload. Se tutto è valido e lo stato locale lo consente, esegue il comando e risponde con un ack. Gli altri nodi non devono eseguire nulla. Questo comportamento permette di localizzare fisicamente un modulo, testarlo o modificarlo senza alterare l'intero swarm.

La modalità unicast è particolarmente utile durante la fase di sviluppo hardware. Quando si lavora con ESP32, LED, MOSFET, coil, batterie e sensori, è necessario testare un componente alla volta. Prima di comandare uno swarm intero, bisogna verificare che ogni nodo funzioni correttamente. L'unicast permette di fare proprio questo: si seleziona un MicroBot, si invia un comando specifico, si osserva la risposta, si controlla la telemetria e si registra l'esito. Questo approccio riduce il rischio, perché evita di attivare contemporaneamente più nodi quando si sta ancora validando il comportamento di uno solo.

Per esempio, se un nodo supporta una bobina controllata in PWM, non è prudente iniziare con un comando broadcast verso tutti. È più corretto inviare un comando unicast a un solo nodo, con duty cycle basso e durata limitata, verificando temperatura, consumo, risposta del firmware e comportamento del MOSFET. Solo dopo che il singolo nodo è stato validato si può pensare a comandi di gruppo o broadcast. L'unicast diventa quindi strumento di calibrazione e sicurezza sperimentale.

Un comando unicast per test magnetico può essere:

```
{
"command": "magnetic_pulse",
"target": "MB-002",
"payload": {
"coil_id": "main",
"duty_cycle": 0.20,
"duration_ms": 80,
"direction": "attract"
},
"requires_ack": true
}
```

Questo comando deve essere accettato solo se MB-002 dichiara capability coil_pwm, se il duty cycle è entro i limiti, se la durata è sicura, se la temperatura stimata è sotto soglia e se il nodo non è in safety. L'unicast non elimina la necessità del safety layer; anzi, rende possibile testare la sicurezza in modo controllato. Il nodo deve sempre essere l'autorità finale sull'esecuzione fisica.

La modalità unicast è anche fondamentale per la diagnostica mirata. Se la dashboard mostra che MB-004 ha heartbeat irregolari, errori di parsing, bassa batteria o stato safety, il controller può interrogare solo quel nodo. Un comando request_status unicast può chiedere telemetria dettagliata senza aumentare il traffico di tutti gli altri MicroBot. Questo è molto più efficiente rispetto a un broadcast diagnostico globale. La diagnostica deve essere proporzionata: se il problema riguarda un nodo, si interroga quel nodo; se riguarda tutto lo swarm, si usa broadcast o una richiesta globale.

Un request_status unicast può avere payload:

```
{  
  "level": "debug"  
}
```

Il nodo risponde con telemetria dettagliata: stato rete, contatori pacchetti, ultimo comando ricevuto, errori recenti, safety state, batteria, eventuale thermal_load, firmware_version e boot_id. Questi dati aiutano a capire se il problema è comunicativo, energetico, firmware o operativo. In una dashboard matura, l'utente potrebbe cliccare su MB-004 e premere “diagnostic”, generando proprio un comando unicast di questo tipo.

L'unicast è indispensabile anche per l'assegnazione individuale di ruoli. In uno swarm, alcuni nodi possono assumere funzioni diverse: leader, follower, connector, boundary, bridge, diagnostic, idle. Il broadcast può assegnare una fase globale, ma spesso il ruolo è specifico del nodo. Il controller può quindi inviare comandi assign_role unicast. Per esempio, MB-001 può essere leader, MB-002 e MB-003 follower, MB-004 connector. Questa assegnazione permette di costruire pattern più complessi rispetto a uno swarm totalmente omogeneo.

Un comando assign_role può essere:

```
{  
  "command": "assign_role",  
  "target": "MB-001",  
  "payload": {  
    "role": "leader",  
    "group_id": "line_segment_01",  
    "pattern_run_id": "line_run_001"  
  }  
}
```

Questo comando non modifica direttamente la posizione del nodo, ma modifica la sua funzione nel comportamento collettivo. Il nodo leader può interpretare fasi broadcast in modo diverso rispetto ai follower. Questo mostra come broadcast e unicast possano cooperare: l'unicast definisce ruoli individuali, il broadcast definisce fasi comuni. Il risultato è un controllo collettivo ma strutturato.

La modalità unicast è utile anche per assegnare target individuali. In una formazione geometrica, ogni MicroBot può avere una posizione obiettivo diversa. Se il controller calcola centralmente il pattern, può inviare a ogni nodo il proprio target tramite unicast. Per esempio, MB-001 riceve target A, MB-002 target B, MB-003 target C. Questo approccio è semplice da implementare e molto chiaro, anche se può diventare meno scalabile con molti nodi. Per i primi prototipi e simulazioni, però, è una soluzione efficace.

Un comando set_target_position può essere concettualmente:

```
{  
  "command": "set_target_position",  
  "target": "MB-003",  
  "payload": {  
    "x": 120.0,  
    "y": 80.0,  
    "z": 0.0,  
    "frame": "simulation_plane",  
    "pattern_run_id": "circle_run_001"  
  }  
}
```

```
}  
}
```

Questo comando è più naturale nei nodi simulati o in un controller che mantiene stato virtuale. Nel nodo fisico reale, senza sistema di locomozione autonomo o localizzazione completa, il target può essere interpretato come informazione di alto livello o come dato della simulazione. La documentazione deve mantenere chiara questa distinzione: alcuni comandi unicast sono pienamente fisici, altri sono simulativi o preparatori per versioni future.

L'unicast è molto utile per la calibrazione dei singoli nodi. Ogni MicroBot può avere differenze hardware: LED con intensità leggermente diversa, bobina con resistenza diversa, batteria con comportamento diverso, magneti con forza leggermente differente, sensore con offset. Il controller può inviare parametri di calibrazione a un singolo nodo. Questi parametri possono essere salvati localmente o mantenuti nella configurazione del controller. La calibrazione individuale è uno dei motivi principali per cui l'identità dei nodi è così importante.

Un comando di calibrazione potrebbe essere:

```
{  
  "command": "set_calibration",  
  "target": "MB-002",  
  "payload": {  
    "led_brightness_scale": 0.85,  
    "max_safe_duty": 0.32,  
    "temperature_offset": 1.5  
  }  
}
```

Questo comando deve essere protetto e validato, perché modifica limiti o correzioni del nodo. In una prima versione, la calibrazione può essere gestita solo dal controller e non scritta nel firmware. In una versione avanzata, può essere salvata in memoria non volatile. In entrambi i casi, l'unicast è il canale naturale per applicare parametri specifici a un nodo specifico.

La modalità unicast è necessaria anche per correggere stati incoerenti. Può accadere che uno o più nodi perdano un broadcast, vadano fuori fase o si riavviino. Il controller può rilevare che MB-005 è out_of_sync rispetto allo swarm. Invece di riavviare tutto il sistema, può inviare comandi unicast solo a quel nodo per riallinearlo: set_mode, sync_request, request_status, assign_role, reset_runtime_state. Questo rende il sistema più efficiente e robusto. Non serve disturbare tutto lo swarm per correggere un singolo elemento.

Una procedura di riconciliazione può essere:

il controller rileva MB-005 out_of_sync;

invia request_status unicast;

confronta stato reale con stato atteso;

invia set_mode o reset_runtime_state unicast;

attende ack;

reinserisce MB-005 nel pattern se compatibile.

Questa procedura mostra l'importanza dell'unicast nella manutenzione dinamica dello swarm. Lo swarm non è sempre perfettamente uniforme; alcuni nodi possono divergere. L'unicast permette interventi chirurgici.

Dal punto di vista del protocollo UDP, l'unicast può essere più controllabile del broadcast. Il controller invia il pacchetto all'indirizzo IP specifico del nodo e attende ack da quel nodo. Se non riceve risposta, può ritentare, marcare il nodo come non responsivo o passare a diagnostica. La gestione è più semplice rispetto al broadcast, perché esiste un solo destinatario atteso. Tuttavia, anche l'unicast non garantisce consegna se usa UDP. Servono comunque message_id, ack, timeout, retry, timestamp e deduplicazione.

Una tabella comandi unicast pendenti può contenere:

message_id;

target_node;

target_ip;

command;

sent_at;

timeout_at;

retry_count;

status;

last_error.

Se il comando riceve ack, status diventa accepted, scheduled o executed. Se scade, diventa timeout. Se viene rifiutato, status diventa rejected con reason. Questa struttura permette al controller di gestire ogni nodo in modo preciso.

Il retry unicast deve essere controllato. Se un comando non riceve ack, il controller può ritrasmettere. Ma se il comando non è idempotente, la ritrasmissione deve mantenere lo stesso message_id o indicare retry_of. Il nodo, riconoscendo un duplicato, non deve rieseguire l'azione se l'ha già accettata o eseguita. Questo vale soprattutto per impulsi, attivazioni temporizzate e comandi fisici. L'unicast non elimina il rischio di duplicazione: lo rende più gestibile perché riguarda un solo nodo.

Una regola importante è:

un retry unicast di un comando non idempotente deve essere deduplicabile dal nodo.

Se il comando è set_mode idle, ripeterlo è relativamente sicuro. Se è magnetic_pulse, ripeterlo senza controllo può produrre un secondo impulso. Quindi message_id e cache dei comandi processati restano fondamentali.

La modalità unicast è anche utile per comandi che richiedono risposte grandi. Se si vuole ottenere telemetria dettagliata, dump diagnostico o stato completo, farlo in broadcast da tutti i nodi può saturare la rete. Meglio interrogare un nodo alla volta, oppure pochi nodi selezionati. L'unicast

permette di distribuire il carico diagnostico nel tempo. Questo è particolarmente importante quando il numero di MicroBot cresce.

Per esempio, il controller può eseguire una scansione diagnostica sequenziale:

request_debug_status a MB-001;

attendi risposta;

request_debug_status a MB-002;

attendi risposta;

continua.

Oppure può interrogare gruppi piccoli. Questa strategia evita picchi di traffico e rende la diagnosi più ordinata.

L'unicast può essere usato anche per aggiornare firmware o configurazioni, ma bisogna distinguere bene i livelli. L'aggiornamento firmware via rete è un tema molto più complesso e non deve essere confuso con i normali comandi UDP realtime. Per le prime versioni, l'unicast può servire per configurazioni leggere, non per firmware update completo. Un eventuale aggiornamento firmware richiederebbe protocollo affidabile, verifica integrità, gestione errori e sicurezza. È corretto menzionarlo come possibilità futura, ma non va mescolato al protocollo UDP operativo.

Per configurazioni leggere, invece, l'unicast è adatto. Per esempio, impostare node_label, telemetry_level, max_safe_duty, LED brightness o gruppo di appartenenza può essere fatto tramite comandi mirati. Ogni configurazione deve avere ack e, se persistente, conferma di salvataggio.

Un comando set_node_config può avere:

```
{
"command": "set_node_config",
"target": "MB-003",
"payload": {
"telemetry_level": "normal",
"heartbeat_interval_ms": 1000
}
}
```

Il nodo valida i valori e risponde. Se salva in memoria persistente, può rispondere config_saved; se applica solo per la sessione corrente, può rispondere config_applied_runtime. Questa distinzione evita ambiguità.

La modalità unicast è importante anche per la sicurezza operativa, perché permette di isolare un nodo. Se MB-006 mostra comportamento anomalo, il controller può inviare a quel nodo un comando enter_safety o disable_outputs senza fermare tutto lo swarm. Questo è utile quando il problema è locale. Naturalmente, se il problema è sistemico o incerto, può essere necessario un broadcast stop_all o emergency_stop. Ma quando il problema è identificato, l'unicast permette un intervento mirato.

Un comando isolate_node può essere concettualmente:

```
{  
  "command": "enter_safety",  
  "target": "MB-006",  
  "payload": {  
    "reason": "thermal_warning"  
  }  
}
```

Il nodo entra in safety, disattiva uscite attive e continua a inviare heartbeat o telemetria minima. Gli altri nodi possono continuare, se il pattern lo consente. Questo è molto coerente con un sistema multi-agente robusto: un agente problematico può essere isolato senza spegnere l'intero sistema.

L'unicast è anche utile per reintegrare un nodo dopo una condizione di errore. Dopo diagnostica, cooldown o reset, il controller può inviare comandi specifici per riportare il nodo in idle, sincronizzarlo, assegnargli un ruolo e poi reinserirlo nel comportamento collettivo. Questo processo deve essere controllato; un nodo non dovrebbe rientrare automaticamente in un pattern complesso senza verifica. L'unicast permette una procedura graduale.

Una sequenza di reintegrazione può essere:

request_status;

sync_node;

set_mode idle;

assign_role;

set_telemetry_level normal;

mark_available;

partecipazione al prossimo pattern.

Questa procedura è molto più sicura di un semplice ritorno automatico dopo reset.

Dal punto di vista della dashboard, l'unicast corrisponde alle azioni sul singolo nodo. L'utente può selezionare MB-002 e inviare comandi: identify, request status, set LED, enter idle, run diagnostic, disable outputs. La dashboard dovrebbe mostrare chiaramente che il comando è individuale. Questo evita confusione tra azioni globali e azioni locali. Un pulsante "Identify Node" è unicast; un pulsante "Stop All" è broadcast; un pulsante "Start Pattern" può essere broadcast o misto.

Una scheda nodo nella dashboard può quindi generare comandi unicast. La scheda mostra stato, ultimo heartbeat, IP, capabilities, ruolo, errori e pulsanti di azione. Ogni azione produce un message_id, un ack atteso e un risultato visibile. Questo rende la comunicazione interattiva e tracciabile.

L'unicast permette anche di associare dati di rete al singolo nodo. Il controller può misurare RTT, packet loss, heartbeat delay e jitter per ogni MicroBot. Questi valori possono influenzare decisioni operative. Un nodo con latenza alta può non essere scelto per ruoli critici. Un nodo con packet loss elevato può essere escluso da azioni sincronizzate. Un nodo con comunicazione stabile può essere usato come riferimento o leader. Senza unicast e identificazione individuale, questa valutazione sarebbe molto più difficile.

Un profilo comunicativo per nodo può includere:

RTT medio;

RTT massimo;

jitter;

packet_loss stimato;

missed_heartbeat_count;

last_seen;

network_status.

Questi dati vengono costruiti attraverso ping, heartbeat, ack e messaggi individuali. Il controller può usare il profilo per adattare il comportamento dello swarm.

La modalità unicast è importante anche nella gestione dei nodi simulati. In una simulazione, il controller può inviare comandi a SIM-001, SIM-002, SIM-003 come se fossero nodi reali. Questo permette di testare lo stesso protocollo. Un comando unicast a SIM-001 può modificare il suo stato virtuale, ruolo o target. Un comando unicast a MB-001 modifica invece un dispositivo fisico. La differenza è nell'implementazione del nodo, non nella logica del messaggio. Questo è un punto molto forte dell'architettura MicroBot.

La distinzione tra physical_node e simulated_node può essere dichiarata nelle capabilities o nel profilo nodo. Il controller può decidere se inviare certi comandi solo ai fisici, solo ai simulati o a entrambi. Per esempio, set_virtual_position ha senso per SIM-001 ma non per MB-001. magnetic_pulse ha senso per MB-002 se supporta coil_pwm, ma può avere un equivalente virtuale nel simulatore. L'unicast permette di testare queste differenze in modo preciso.

L'unicast può essere usato anche per fault injection nella simulazione. Per testare robustezza, il controller può inviare a un nodo simulato un comando inject_fault, come perdita di comunicazione, batteria bassa, ritardo o errore di connessione. Questo non deve essere inviato a nodi fisici. Ma in simulazione è molto utile per verificare se lo swarm reagisce correttamente. Questo mostra ancora una volta la necessità di capabilities e target precisi.

Un comando simulation-only può essere:

```
{
  "command": "inject_fault",
  "target": "SIM-004",
  "payload": {
    "fault_type": "packet_loss",
    "severity": 0.30,
    "duration_ms": 5000
  }
}
```

Il controller non deve inviarlo a MB-004 fisico. La distinzione tra ID fisici e simulati, come MB-* e SIM-*, aiuta a evitare errori.

La modalità unicast è utile anche nel processo di provisioning iniziale. Quando un nuovo nodo viene aggiunto al sistema, dopo discovery il controller può configurarlo con parametri specifici: node_label, gruppo, modalità di telemetria, limiti di sicurezza, ruolo iniziale. Alcuni di questi dati possono essere inviati tramite unicast. Il provisioning è la fase in cui un dispositivo generico diventa parte coerente dello swarm MicroBot.

Una procedura di provisioning può essere:

nodo invia discovery_response;

controller riconosce node_id o lo assegna;

controller invia set_node_config;

controller invia sync_node;

controller richiede status;

dashboard marca nodo come ready.

Questa procedura permette di integrare nuovi nodi senza modificare manualmente tutto il sistema.

Dal punto di vista della sicurezza dei comandi, l'unicast riduce l'impatto di un errore rispetto al broadcast, ma non lo annulla. Un comando sbagliato inviato a un nodo può comunque danneggiare quel nodo o produrre comportamento indesiderato. Per questo motivo, la validazione locale resta obbligatoria. Inoltre, comandi sensibili unicast possono richiedere sorgente autorizzata, modalità diagnostica o conferma manuale nella dashboard. Questo vale soprattutto per calibrazione persistente, attuatori e reset.

Una policy può essere:

comandi LED: permessi in active e diagnostic;

request_status: permesso quasi sempre;

magnetic_pulse: permesso solo se node_state active/diagnostic e safety_ok;

set_calibration: permesso solo in diagnostic e da source autorizzata;

reset_node: permesso solo con conferma e non durante azioni critiche;

emergency_stop: permesso sempre se valido.

Questa tabella di policy rende il comportamento prevedibile.

L'unicast può essere combinato con timestamp ed execute_at. Un comando individuale può essere programmato per un tempo futuro, esattamente come un broadcast. Questo è utile quando ogni nodo deve eseguire un'azione diversa ma coordinata nel tempo. Il controller può inviare comandi unicast diversi a vari nodi, tutti con lo stesso execute_at. Anche se i pacchetti vengono inviati in sequenza, l'esecuzione può avvenire simultaneamente.

Per esempio:

MB-001 riceve set_led rosso execute_at T;

MB-002 riceve set_led blu execute_at T;

MB-003 riceve set_led verde execute_at T.

Ogni nodo esegue al tempo T. Questa è una tecnica molto utile per pattern differenziati. L'unicast fornisce contenuto individuale; execute_at fornisce sincronizzazione comune.

Per pattern più complessi, il controller può inviare a ogni nodo un piccolo programma di fase o un target individuale con execute_at. Questo riduce dipendenza dal momento di ricezione. Tuttavia, richiede clock sincronizzato, che verrà sviluppato nella Sezione 9. La modalità unicast è quindi strettamente collegata alla sincronizzazione.

L'unicast deve gestire il caso del nodo offline. Se il controller vuole inviare un comando a MB-007 ma il nodo è offline, deve decidere cosa fare. Per comandi realtime, normalmente non deve accodarli indefinitamente. Se MB-007 torna online dopo un minuto, un comando vecchio potrebbe essere pericoloso o inutile. Per configurazioni persistenti, invece, si può mantenere una pending_config da applicare alla riconnessione. Il controller deve distinguere tra comandi scadibili e configurazioni.

Una regola:

comandi operativi hanno expires_at e non vengono applicati dopo scadenza;

configurazioni possono restare pending se marcate come persistenti;

comandi di sicurezza non sono messi in coda per nodi offline, ma safety locale e stato offline vengono segnalati.

Questo evita effetti tardivi non desiderati.

L'unicast è utile anche per richiedere conferma forte prima di operazioni critiche. Per esempio, prima di attivare una coil su MB-002, il controller può inviare prepare_magnetic_pulse. Il nodo verifica stato, risponde ready o not_ready. Solo se ready, il controller invia magnetic_pulse con execute_at. Questa procedura a due fasi riduce il rischio. È più lenta, ma per test fisici è molto prudente.

Una sequenza sicura può essere:

request_ready_for_pulse;

nodo risponde ready;

controller invia magnetic_pulse temporizzato;

nodo esegue;

nodo invia executed;

controller registra esito.

Questo tipo di handshake è più adatto all'unicast che al broadcast, almeno nelle prime fasi. Permette test controllati e tracciabili.

La modalità unicast è anche utile per leggere dati specifici, come calibrazione o firmware version. Un comando `get_node_profile` può chiedere al nodo di inviare il proprio profilo completo. Questo profilo può includere `hardware_version`, `firmware_version`, `protocol_versions`, `capabilities`, `boot_id`, `uptime`, `safety_limits` e configurazione corrente. È uno strumento fondamentale per mantenere coerenza tra documentazione e sistema reale.

Un `get_node_profile` può essere inviato durante discovery o su richiesta. La risposta permette al controller di sapere se il nodo è compatibile con la versione del protocollo e con i comandi previsti. Se non lo è, il nodo può essere marcato come incompatible o legacy. Questo evita comandi sbagliati.

L'unicast permette anche operazioni di debug su parser e protocollo. Il controller può inviare messaggi di test a un nodo specifico e osservare errori. Per esempio, può testare `unsupported_command`, `invalid_payload`, `expired_command`. Questo aiuta a verificare che il firmware rifiuti correttamente messaggi non validi. È molto meglio testare questi casi su un nodo singolo che su tutto lo swarm. La robustezza del protocollo nasce da questi test.

Una suite di test unicast può includere:

ping valido;

target sbagliato;

payload mancante;

command sconosciuto;

timestamp scaduto;

message_id duplicato;

duty_cycle fuori range;

comando non supportato dalle capabilities;

comando rifiutato per safety.

Questa suite può diventare parte della validazione del firmware MicroBot.

Dal punto di vista teorico, l'unicast rappresenta la capacità del sistema di mantenere individualità dentro il comportamento collettivo. Lo swarm non è una massa indistinta. Ogni MicroBot ha identità, stato, capacità e storia. Il broadcast costruisce coerenza globale; l'unicast preserva precisione locale. Senza unicast, il controller potrebbe solo inviare comandi generali e perderebbe capacità di intervenire su singoli nodi. Senza broadcast, dovrebbe comandare tutto individualmente e perderebbe efficienza. La comunicazione MicroBot ha bisogno di entrambe.

Questa dualità è molto importante:

broadcast = controllo del gruppo;

unicast = controllo dell'individuo.

La forza del sistema sta nel combinare i due livelli. Un pattern può iniziare con broadcast, assegnare ruoli con unicast, sincronizzare con broadcast, correggere errori con unicast, fermare tutto con broadcast. Questa composizione rende il sistema flessibile.

Un esempio completo di sequenza può essere:

discovery broadcast;

get_node_profile unicast per nodi nuovi;

assign_role unicast;

sync broadcast;

start_pattern broadcast con execute_at;

correzioni target unicast;

telemetria heartbeat da tutti;

diagnostica unicast su nodo problematico;

stop_all broadcast.

Questa sequenza mostra come broadcast e unicast non siano alternative rigide, ma strumenti complementari.

Dal punto di vista della scalabilità, l'unicast può diventare costoso se usato per tutto. Inviare comandi individuali a centinaia di nodi può aumentare traffico e latenza. Per questo motivo, con swarm grandi bisogna usare unicast solo quando serve: configurazione, diagnostica, ruoli, comandi specifici, correzioni locali. Per le operazioni comuni, broadcast o groupcast sono più efficienti. Nei prototipi piccoli, l'unicast è semplice e gestibile; nei sistemi grandi, deve essere usato con strategia.

Una regola scalabile è:

usa broadcast per informazioni comuni;

usa groupcast per sottostrutture;

usa unicast per differenze individuali.

Questa regola mantiene equilibrio tra efficienza e precisione.

L'unicast deve essere documentato anche come parte della tracciabilità. Ogni comando unicast ha destinatario chiaro, ack atteso, risultato specifico. Nei log è più facile analizzare unicast rispetto a broadcast perché riguarda un nodo. Tuttavia, bisogna comunque registrare message_id, target, command, timestamp, payload_summary, result e reason. Questo permette di ricostruire la storia del nodo.

Un log unicast può mostrare:

123.00 CMD-UNICAST-0001 CTRL-001 → MB-002 identify_node accepted;

124.50 CMD-UNICAST-0002 CTRL-001 → MB-002 request_status executed;

125.00 CMD-UNICAST-0003 CTRL-001 → MB-002 magnetic_pulse rejected thermal_lock.

Questa timeline è chiara e utile. Permette di capire che MB-002 risponde, ma rifiuta magnetic_pulse per safety. Non c'è ambiguità.

L'unicast può essere importante anche per documentare esperimenti. Se si testano sei prototipi, ogni prova può essere associata al node_id e ai comandi unicast inviati. Questo permette di costruire una storia sperimentale per ogni modulo. Per esempio, MB-001 ha superato test LED, heartbeat e telemetria; MB-002 ha superato PWM a duty 20% ma ha mostrato thermal warning a 35%; MB-003 ha packet loss elevato. Questa tracciabilità è estremamente utile per sviluppo fisico e documentazione tecnica.

Nel progetto MicroBot, dove la documentazione è già molto estesa, collegare ogni prototipo a un ID e a una cronologia di comandi/test sarebbe un salto di qualità enorme. L'unicast è il canale naturale per generare questa cronologia.

La modalità unicast deve essere coerente con la gestione degli errori. Se il nodo rifiuta un comando, deve indicare il motivo. Se non risponde, il controller deve distinguere timeout da rejected. Se risponde con errore di protocollo, il controller deve registrarlo. Se l'IP associato al node_id non è più valido, il controller deve richiedere discovery o aggiornare la tabella. Ogni fallimento unicast deve avere una categoria.

Categorie possibili:

no_route;

timeout;

wrong_node_response;

unsupported_command;

invalid_payload;

safety_rejected;

low_energy;

busy;

command_expired;

duplicate_message;

executed;

accepted.

Queste categorie permettono al controller di reagire in modo appropriato. timeout può portare a retry; unsupported_command può aggiornare capabilities; safety_rejected può mostrare warning; low_energy può escludere il nodo dal pattern; busy può ritentare più tardi.

Il caso wrong_node_response è interessante: se il controller invia a IP associato a MB-001 ma riceve risposta da MB-009, significa che la tabella indirizzi è errata o la rete ha cambiato

assegnazioni. Il controller deve correggere la mappa node_id → IP tramite discovery. Questo dimostra perché il node_id nel pacchetto è più importante dell'IP. L'IP serve a raggiungere; il node_id serve a confermare identità.

L'unicast deve prevedere anche eventuali cambi di indirizzo IP. Se un nodo si riconnette e riceve nuovo IP, il controller deve aggiornare la sua tabella attraverso discovery_response, heartbeat o messaggio di registrazione. Finché non è aggiornato, i comandi unicast possono fallire. Per questo motivo, heartbeat e discovery devono includere node_id e indirizzo osservato. La dashboard può mostrare l'IP corrente ma non deve usarlo come identità principale.

Una struttura tabellare nel controller può essere:

```
node_id;  
  
current_ip;  
  
udp_port;  
  
last_seen;  
  
boot_id;  
  
capabilities;  
  
network_status.
```

L'unicast usa current_ip e udp_port, ma verifica node_id nella risposta.

L'unicast può anche essere usato per richiedere ritrasmissione di dati o chiarimenti. Se il controller riceve telemetria incompleta da MB-003, può inviare request_telemetry_detail solo a quel nodo. Se riceve un errore generico, può chiedere last_error_detail. Questo evita di appesantire tutti gli altri nodi. La comunicazione diventa adattiva: si inviano dettagli solo dove servono.

Un comando last_error_detail può restituire:

```
ultimo error_code;  
  
ref_message_id;  
  
timestamp;  
  
stato locale;  
  
payload diagnostico minimo.
```

Questo è molto utile per debugging.

Dal punto di vista della priorità, un comando unicast può essere high, normal o low. Un emergency_stop è più spesso broadcast, ma può anche essere unicast se si vuole fermare solo un nodo. disable_outputs può essere unicast. request_status è normale. debug_trace è basso. Il nodo deve processare le priorità anche per messaggi unicast. Se riceve un disable_outputs unicast mentre ha comandi LED o diagnostici pendenti, deve eseguire prima la disattivazione.

L'unicast può essere usato per attivare una modalità di “quarantena” del nodo. Se un nodo genera errori frequenti, il controller può metterlo in quarantena: il nodo continua a comunicare ma non partecipa ai pattern e rifiuta comandi attuativi. Questo è molto utile per mantenere lo swarm operativo anche con moduli difettosi. La quarantena può essere impostata via unicast e mostrata in dashboard.

Un comando può essere:

```
{
"command": "set_mode",
"target": "MB-006",
"payload": {
"mode": "quarantine",
"reason": "repeated_packet_errors"
}
}
```

Il nodo entra in modalità limitata. Questo è un esempio di gestione robusta degli agenti.

La modalità unicast è anche utile per attivare o disattivare telemetria di dettaglio su un singolo nodo. Durante una demo normale, tutti i nodi possono inviare heartbeat leggero. Se MB-003 mostra comportamento strano, il controller può aumentare solo la sua telemetria a debug. Questo evita di saturare la rete. Dopo il test, può riportarlo a normal. Questa strategia è molto efficiente.

Un comando:

```
{
"command": "set_telemetry_level",
"target": "MB-003",
"payload": {
"level": "debug",
"duration_ms": 10000
}
}
```

La durata impedisce che il nodo resti in debug indefinitamente. Dopo 10 secondi torna a normal. Anche qui il tempo è usato per sicurezza e controllo del traffico.

Dal punto di vista della simulazione, l'unicast permette di manipolare uno specifico agente per osservare il comportamento emergente. Si può spostare virtualmente SIM-005, cambiare il suo ruolo, simulare batteria bassa, rimuovere una connessione, aumentare latenza, cambiare peso di coesione locale. Questo consente esperimenti molto interessanti sulla robustezza dello swarm. Per esempio, cosa succede se un nodo leader fallisce? Cosa succede se un nodo ponte viene isolato? Cosa succede se un nodo ha latenza alta? L'unicast permette di creare perturbazioni controllate.

Questi esperimenti sono importanti perché uno swarm non deve funzionare solo in condizioni ideali. Deve tollerare differenze, errori e fallimenti. La comunicazione unicast offre lo strumento per testare questi casi.

In una documentazione avanzata, si potrebbe definire una tabella “Unicast diagnostic scenarios” con scenari come:

identificazione nodo;

richiesta stato dettagliato;

test LED;

test PWM sicuro;

assegnazione ruolo;

isolamento nodo;

reintegrazione nodo;

simulazione fault;

calibrazione locale;

rilevamento timeout.

Questa tabella mostrerebbe la maturità del sistema.

L'unicast deve essere visto anche come strumento di precisione nella relazione tra controller e grafo dello swarm. Se una connessione tra MB-001 e MB-002 è instabile, il controller può interrogare solo quei due nodi, chiedere distanza stimata, stato magnetico, ultimo attach_event, margine di distacco o errori locali. Non serve interrogare tutto lo swarm. La gestione delle connessioni può quindi essere locale e mirata.

Un comando request_connection_status può essere:

```
{  
  "command": "request_connection_status",  
  "target": "MB-001",  
  "payload": {  
    "other_node": "MB-002"  
  }  
}
```

Il nodo risponde con stato della connessione, se supportato. Questo è più avanzato, ma coerente con il modello del grafo dinamico.

Dal punto di vista fisico, l'unicast aiuta a evitare attivazioni simultanee non necessarie. Se si sta testando consumo o calore, si attiva un nodo alla volta. Se si sta testando una sequenza di aggancio, si può comandare prima un nodo, poi l'altro, osservando le risposte. Questo approccio è molto più sicuro rispetto a comandi globali. Nella fase prototipale, l'unicast sarà probabilmente la modalità principale per test hardware, mentre il broadcast sarà usato per discovery, LED globali e stop.

Una progressione sana può essere:

prima: unicast ping e LED;

poi: broadcast discovery e LED sync;

poi: unicast PWM test;

poi: unicast role assignment;

poi: broadcast pattern phase;

poi: combinazione broadcast/unicast per demo completa.

Questa progressione evita di saltare subito al comportamento collettivo fisico senza aver validato i nodi.

La modalità unicast deve essere collegata alla documentazione delle capabilities. Prima di inviare un comando individuale, il controller dovrebbe conoscere cosa il nodo supporta. Se la capability manca, il controller può evitare l'invio e mostrare errore lato dashboard. Se lo invia comunque, il nodo deve rifiutare. Questo doppio controllo è utile: prevenzione nel controller, protezione nel nodo.

Un flusso corretto:

controller consulta capabilities di MB-002;

vede coil_pwm supportato;

invia magnetic_pulse;

nodo valida di nuovo localmente;

esegue o rifiuta secondo safety.

Se controller sbaglia, il nodo protegge. Se nodo ha cambiato stato, la safety locale decide.

L'unicast può anche essere usato per comandi di reset. Un reset logico del nodo può cancellare stato runtime senza riavviare hardware. Un reboot fisico/software del microcontrollore è più invasivo. Entrambi devono essere gestiti con cautela e conferma. In una prima versione, reset_runtime_state è più sicuro. Reboot può essere diagnostico e richiedere modalità specifica.

Un reset_runtime_state unicast può includere:

clear_pending_commands;

clear_errors_noncritical;

return_mode;

preserve_identity;

preserve_calibration.

Il nodo esegue reset logico e risponde. Dopo il reset, il controller richiede status. Questo permette recupero da stati incoerenti.

L'unicast deve gestire anche busy state. Un nodo può essere occupato in una sequenza temporizzata e non accettare nuovi comandi operativi. Può però accettare emergency_stop o request_status. Se riceve un comando non urgente mentre è busy, può rispondere busy o scheduled_later, a seconda della policy. Questo evita conflitti tra comandi.

Una risposta può essere:

result = "rejected";

```
reason = "busy";
```

```
busy_until = 123470.00.
```

Il controller può ritentare dopo quel tempo. Questa logica rende il sistema più ordinato.

Dal punto di vista del modello multi-agente, l'unicast permette al controller di intervenire sull'agente senza modificare direttamente gli altri. Questo può sembrare più centralizzato, ma in realtà è necessario per mantenere ogni agente gestibile. Un sistema distribuito non significa assenza di controllo individuale; significa che ogni agente possiede stato e autonomia. L'unicast è il canale per comunicare con quell'agente quando serve.

La modalità unicast è quindi il livello più preciso del communication layer. Dove il broadcast definisce stati comuni, l'unicast definisce interventi individuali. Dove il broadcast sincronizza, l'unicast calibra. Dove il broadcast ferma tutti, l'unicast isola il nodo problematico. Dove il broadcast avvia una fase, l'unicast assegna ruoli e target. Questa complementarità completa l'architettura di comunicazione.

Dal punto di vista della Sezione 8 nel suo complesso, l'unicast chiude il quadro: rete Wi-Fi fornisce il mezzo; i MicroBot sono nodi di rete; UDP fornisce trasporto leggero; la bassa latenza definisce il requisito temporale; la struttura dei pacchetti definisce la grammatica; identificatore, comando e timestamp definiscono i campi fondamentali; broadcast e unicast definiscono le due modalità principali di distribuzione dei messaggi. Con questo, il communication layer diventa completo a livello concettuale.

In conclusione, la modalità unicast è essenziale per il sistema MicroBot perché permette comunicazione diretta, precisa e tracciabile con un singolo nodo. Essa supporta diagnostica mirata, calibrazione, assegnazione di ruoli, richiesta di stato, test hardware, correzione di errori, isolamento di nodi problematici, reintegrazione, comandi temporizzati individuali e gestione di differenze tra prototipi. L'unicast deve essere accompagnato da message_id, timestamp, ack, timeout, retry controllati, deduplicazione, controllo capabilities e safety locale. Il suo valore non è opposto al broadcast, ma complementare: broadcast coordina lo swarm come insieme, unicast gestisce il MicroBot come individuo. Insieme, queste due modalità permettono al sistema di essere sia collettivo sia preciso, sia emergente sia controllabile. Con questa sezione si chiude il livello comunicativo di base e si prepara il passaggio alla Sezione 9, dedicata alla sincronizzazione, dove verrà analizzato come controller e nodi possano condividere un riferimento temporale, coordinare azioni, gestire ritardi e organizzare lo swarm in slot temporali coerenti.

SEZIONE 9 — Sincronizzazione

La Sezione 9 introduce il livello temporale del sistema MicroBot, cioè l'insieme dei meccanismi attraverso cui controller, nodi distribuiti, simulazione, dashboard e log condividono un riferimento comune per ordinare eventi, eseguire comandi coordinati e mantenere coerenza tra azione digitale e comportamento fisico. Dopo aver definito la comunicazione nella Sezione 8, diventa necessario affrontare un problema ancora più delicato: non basta che i MicroBot ricevano messaggi; devono riceverli, interpretarli ed eventualmente eseguirli in un tempo coerente con lo stato dello swarm. La comunicazione definisce il “cosa” viene trasmesso; la sincronizzazione definisce il “quando” quel contenuto deve avere effetto.

Nel progetto MicroBot, la sincronizzazione è fondamentale perché il sistema è distribuito. Ogni MicroBot può avere un proprio microcontrollore, un proprio ciclo firmware, un proprio clock locale, una propria latenza di rete, un proprio stato energetico e una propria condizione operativa.

Se il controller invia un comando globale e ogni nodo lo esegue semplicemente appena lo riceve, il risultato può essere disallineato. Alcuni nodi possono agire prima, altri dopo, altri possono ricevere il pacchetto in ritardo, altri ancora possono rifiutare il comando perché scaduto o perché non sincronizzati. In uno swarm semplice questo può essere tollerabile; in uno swarm che deve formare pattern, gestire agganci magnetici, coordinare LED, applicare impulsi o mantenere una sequenza temporale, il disallineamento può compromettere il comportamento collettivo.

La sincronizzazione deve quindi essere vista come un livello architetturale autonomo. Essa non sostituisce la comunicazione, ma la rende temporalmente significativa. Un comando con identificatore, target e payload è completo dal punto di vista informativo, ma diventa veramente operativo solo quando possiede anche un riferimento temporale. Il campo timestamp, introdotto nella Sezione 8.8, è la base di questo processo; la Sezione 9 sviluppa il modo in cui quel riferimento temporale viene usato per costruire clock globale, slot temporali, coordinazione delle azioni, gestione del ritardo e allineamento tra dispositivi.

Nel sistema MicroBot, il tempo può essere considerato su più livelli. Esiste il tempo del controller, che rappresenta il riferimento principale usato per generare comandi, misurare latenze, coordinare pattern e registrare log. Esiste il tempo locale del singolo MicroBot, cioè il contatore interno del microcontrollore, spesso misurato in millisecondi dall'avvio. Esiste il tempo della simulazione, che può coincidere con il tempo reale oppure avanzare a velocità diversa, essere messo in pausa o essere riprodotto tramite replay. Esiste infine il tempo della dashboard, usato per visualizzare dati e sequenze. La sincronizzazione serve a collegare questi livelli, o almeno a dichiarare chiaramente quale tempo viene usato in ogni parte del sistema.

Un principio fondamentale è che la sincronizzazione perfetta non è realistica nelle prime versioni. MicroBot non deve partire dall'idea di ottenere immediatamente una precisione assoluta tra tutti i nodi. Deve invece costruire una sincronizzazione progressiva, sufficiente al livello di comportamento richiesto. Per una demo LED, può bastare una sincronizzazione approssimata nell'ordine di alcune decine di millisecondi. Per comandi magnetici temporizzati, potrebbe servire maggiore precisione e soprattutto maggiore prevedibilità. Per logging e telemetria, può essere sufficiente ordinare correttamente gli eventi. Per controllo fisico avanzato, serviranno misure più accurate. La sincronizzazione deve quindi essere progettata come sistema evolutivo, non come requisito monolitico.

La Sezione 9 seguirà una progressione precisa. Prima verrà descritto il controller come riferimento temporale principale. Poi sarà introdotto il concetto di clock globale del sistema, inteso non come orologio perfetto, ma come timebase condivisa o stimata. Successivamente verranno analizzati gli slot temporali, cioè finestre ordinate in cui i nodi possono eseguire azioni senza sovrapporsi o senza generare collisioni comunicative. Verrà poi sviluppata la coordinazione delle azioni, cioè il modo in cui più MicroBot possono eseguire comandi diversi o uguali mantenendo coerenza collettiva. Infine saranno trattate la gestione del ritardo e l'allineamento tra dispositivi, che rappresentano i due problemi pratici principali di ogni sistema distribuito: la rete non è istantanea e i clock locali non sono naturalmente identici.

9.1 Controller come riferimento temporale

Il controller come riferimento temporale rappresenta il primo elemento della sincronizzazione MicroBot. In un sistema distribuito composto da più nodi, è necessario scegliere quale componente debba fornire il riferimento principale per ordinare comandi, eventi, stati e sequenze operative. Nel progetto MicroBot, questo ruolo viene assegnato al controller centrale, perché esso è il componente che possiede la visione più completa dello swarm, genera i comandi globali, coordina la dashboard,

raccoglie telemetria, mantiene i log e può confrontare lo stato dei diversi nodi. Il controller non è necessariamente l'unico componente intelligente del sistema, ma è il punto più adatto per definire il tempo di riferimento comune.

Il controller può essere un programma Python, un server locale, una dashboard web con backend, un modulo runtime o un'applicazione dedicata. Indipendentemente dalla forma implementativa, il suo ruolo temporale consiste nel generare una timebase principale, cioè una linea temporale rispetto alla quale vengono marcati comandi, pacchetti, eventi, run sperimentali e pattern. Quando il controller invia un comando, associa un timestamp. Quando riceve un heartbeat, registra il tempo di arrivo. Quando avvia una formazione, definisce il tempo di inizio. Quando misura latenza, confronta tempo di invio e tempo di risposta. In questo modo, il controller diventa il punto centrale di ordinamento temporale.

Questa scelta è coerente con l'architettura MicroBot già definita. Il sistema non è completamente decentralizzato nelle prime versioni; possiede un controller centrale che governa le fasi principali. I MicroBot possono avere autonomia locale per validare comandi, applicare safety, gestire timer, spegnere uscite e rifiutare operazioni non sicure, ma la sequenza complessiva viene orchestrata dal controller. Per questo motivo, usare il controller come riferimento temporale è naturale. Esso rappresenta il livello che collega intenzione dell'utente, modello matematico, comunicazione, simulazione, logging e visualizzazione.

Il controller come riferimento temporale non significa che i nodi non abbiano un proprio clock. Ogni MicroBot deve comunque possedere un tempo locale, ad esempio un contatore in millisecondi dall'avvio. Questo tempo locale è indispensabile per gestire `duration_ms`, `timeout`, `cooldown`, `watchdog`, `heartbeat`, comandi temporizzati e `safety` locale. Tuttavia, il tempo locale del nodo non è automaticamente confrontabile con quello degli altri nodi. Due ESP32 possono accendersi in momenti diversi, avere contatori diversi e accumulare piccole derive nel tempo. Il controller serve proprio a fornire un riferimento esterno rispetto al quale ciascun nodo può stimare il proprio offset.

Una relazione concettuale può essere espressa così:

$$\text{tempo_controller} \approx \text{tempo_locale_nodo} + \text{offset_nodo}$$

In questa relazione, `offset_nodo` rappresenta la differenza stimata tra il clock locale del MicroBot e il tempo del controller. Se il nodo conosce o stima il proprio offset, può interpretare comandi basati su `execute_at` nel tempo del controller. Per esempio, se il controller invia "esegui al tempo 15000 ms del controller", il nodo deve convertire quel valore nel proprio tempo locale. Senza questa conversione, ogni nodo interpreterebbe `execute_at` in modo diverso e la sincronizzazione sarebbe impossibile.

Nelle prime versioni, il sistema può usare una sincronizzazione semplice. Il controller invia periodicamente messaggi di sync contenenti il proprio tempo corrente. Il nodo riceve il messaggio e calcola una stima approssimata dell'offset. Questa stima non sarà perfetta, perché il messaggio impiega tempo per attraversare la rete; tuttavia, può essere sufficiente per demo iniziali, LED sincronizzati e comandi non estremamente critici. In versioni successive, si potrà migliorare la stima usando round-trip time, più campioni, media filtrata, compensazione della latenza e scarto dei valori anomali.

Un messaggio di sincronizzazione semplice può contenere:

`controller_time_ms;`

`sync_id;`

timestamp di generazione;

valid_until;

timebase_id.

Il nodo riceve questo pacchetto, registra il proprio receive_time_local_ms e calcola una stima:

$$\text{offset_stimato} = \text{controller_time_ms} - \text{receive_time_local_ms}$$

Questa formula è semplice ma non tiene conto del ritardo di rete. Il messaggio è stato generato dal controller prima di arrivare al nodo, quindi quando il nodo lo riceve il tempo reale del controller è già avanzato. Se la latenza è bassa, l'errore può essere accettabile; se la latenza è alta o variabile, la stima peggiora. Per questo motivo, nelle versioni più accurate bisogna misurare il round-trip time.

Una procedura più robusta può essere:

il controller invia sync_request al tempo T1;

il nodo riceve al proprio tempo locale N2;

il nodo risponde con sync_response includendo N2 e il tempo di invio locale N3;

il controller riceve la risposta al tempo T4;

il controller calcola il round-trip time e stima offset e ritardo.

Questo modello è simile ai principi usati in protocolli di sincronizzazione più complessi, ma può essere semplificato per MicroBot. L'obiettivo non è costruire immediatamente un sistema di sincronizzazione professionale assoluto, ma ottenere una stima sufficiente a coordinare azioni distribuite con margini controllati. Il controller può mantenere per ogni nodo un profilo temporale individuale: offset stimato, RTT medio, jitter, ultimo sync valido, qualità della sincronizzazione.

Una tabella temporale nel controller può contenere:

node_id;

last_sync_time;

estimated_offset_ms;

average_rtt_ms;

jitter_ms;

sync_quality;

clock_status.

Il campo sync_quality può indicare se il nodo è sincronizzato bene, accettabile, degradato o non sincronizzato. Questa informazione è molto importante perché non tutti i nodi devono necessariamente essere trattati allo stesso modo. Un nodo con sincronizzazione buona può partecipare a sequenze temporizzate; un nodo con sincronizzazione degradata può ricevere solo comandi semplici o può essere escluso da azioni magnetiche coordinate. Il controller come

riferimento temporale non si limita quindi a inviare tempo; valuta anche quanto ogni nodo sia allineato a quel tempo.

Il concetto di controller come riferimento temporale è importante anche per i log. Tutti gli eventi raccolti dal sistema devono essere riportati, per quanto possibile, al tempo del controller o a una timebase comune. Se ogni nodo registra eventi solo con il proprio tempo locale, confrontare eventi tra nodi diventa difficile. Per esempio, MB-001 può dire che un evento è avvenuto a 2000 ms dal proprio boot, mentre MB-002 può dire che un altro evento è avvenuto a 8000 ms dal proprio boot. Senza sapere quando sono stati avviati e quale offset hanno, non si può stabilire quale evento sia avvenuto prima nel sistema globale. Il controller permette di mappare questi eventi su una timeline comune.

Un evento generato da un nodo può quindi essere ricevuto così:

`node_event_time_local` = tempo locale del nodo;

`estimated_controller_time` = `node_event_time_local` + `offset_nodo`;

`received_at_controller` = tempo di arrivo al controller.

Il controller può registrare sia il tempo stimato dell'evento sia il tempo di ricezione. Questa doppia informazione è utile. Il tempo stimato dell'evento indica quando il nodo pensa che la transizione sia avvenuta nel riferimento comune; il tempo di ricezione indica quando il controller ne è venuto a conoscenza. In sistemi distribuiti, queste due cose non coincidono. Un evento può avvenire prima ma essere ricevuto dopo. Conservare entrambi i dati rende i log più rigorosi.

Il controller come riferimento temporale ha anche un ruolo nella gestione delle run sperimentali. Quando viene avviato un test, una demo o un pattern, il controller può creare un `run_id` e un `run_start_time`. Da quel momento, tutti gli eventi possono essere espressi anche come tempo relativo alla run. Questo è molto utile per analisi e documentazione. Invece di confrontare solo date assolute, si può dire che al secondo 0.0 è iniziato il pattern, al secondo 1.2 i nodi hanno completato l'allineamento, al secondo 2.5 sono avvenuti gli agganci, al secondo 4.0 la struttura si è stabilizzata.

Una run può essere rappresentata così:

`run_id` = "line_run_001";

`controller_start_time_ms` = 500000;

`run_time_ms` = `controller_time_ms` - `controller_start_time_ms`.

Questa struttura permette di confrontare più esperimenti tra loro. Se `line_run_001` converge in 4.2 secondi e `line_run_002` converge in 3.8 secondi, il sistema può confrontare performance, stabilità, numero di errori e distacchi. Il controller è il punto naturale per definire `run_time`, perché gestisce l'avvio e la fine delle prove.

Il controller come riferimento temporale è essenziale anche per la dashboard. La dashboard deve mostrare stati e eventi in ordine coerente. Se riceve dati da nodi diversi con timestamp locali non allineati, rischia di mostrare una timeline confusa. Il controller può pre-processare i dati, convertirli in una timebase comune e inviarli alla dashboard già ordinati o almeno arricchiti con timestamp globale stimato. In questo modo, l'interfaccia non deve interpretare autonomamente clock multipli; può visualizzare una storia coerente.

La dashboard può mostrare:

tempo della run;

ultimo heartbeat per nodo;

latenza stimata;

sync quality;

eventi ordinati;

comandi programmati;

execution_error rispetto a execute_at.

Tutte queste informazioni dipendono dal controller come riferimento temporale. La dashboard non è il generatore principale del tempo; è il visualizzatore della timebase orchestrata dal controller.

Un aspetto molto importante è la distinzione tra tempo di sistema e tempo reale assoluto. Il controller può usare un clock monotono interno per misurare intervalli, timeout e comandi. Questo clock monotono è più adatto al controllo perché non torna indietro e non viene influenzato da modifiche dell'orologio del computer. Per i log umani, invece, può essere utile registrare anche la data reale. Il controller può quindi mantenere due riferimenti:

tempo monotono del controller, usato per controllo e sincronizzazione;

datetime assoluto, usato per documentazione, file e log leggibili.

Questa distinzione è molto sana. Se il sistema operativo modifica l'orario, il controllo non deve essere influenzato. I timeout, gli execute_at e le durate devono basarsi su tempo monotono. La data assoluta serve a sapere quando è avvenuto l'esperimento, non a controllare direttamente le uscite fisiche.

Un log del controller può quindi includere:

controller_time_ms;

run_time_ms;

datetime_iso;

event_type;

node_id;

message_id.

Questa struttura è estremamente utile perché permette sia analisi tecnica sia lettura umana. controller_time_ms ordina gli eventi; run_time_ms li colloca dentro l'esperimento; datetime_iso li collega al calendario reale.

Il controller come riferimento temporale deve anche gestire il concetto di validità dei comandi. Quando il controller genera un comando, può decidere execute_at ed expires_at in base alla propria timebase. Per esempio, se vuole che un comando venga eseguito tra 500 ms, calcola:

$\text{execute_at} = \text{controller_time_now} + 500 \text{ ms}$

Se vuole che il comando non sia più valido dopo 900 ms, calcola:

`expires_at = controller_time_now + 900 ms`

Il nodo riceve questi valori e li interpreta tramite il proprio offset. Questo permette al controller di pianificare azioni future. Il comando non è più legato al momento di arrivo del pacchetto, ma a un tempo globale stimato. Questo è il cuore della sincronizzazione operativa.

Per fare ciò, il nodo deve conoscere lo stato della propria sincronizzazione. Se il nodo non ha un offset valido o se l'ultimo sync è troppo vecchio, dovrebbe rifiutare comandi con `execute_at` assoluto oppure chiedere una nuova sincronizzazione. Eseguire comandi temporizzati senza sapere come convertire il tempo del controller sarebbe pericoloso o incoerente. La validazione locale deve quindi includere `sync_status`.

Una regola può essere:

se comando contiene `execute_at` basato su `controller_time` e `sync_status` non è valido, allora rifiutare o richiedere sync.

Questo è importante. Il nodo non deve fingere di essere sincronizzato quando non lo è. La dashboard può mostrare `sync_status` per ogni nodo, così l'utente capisce quali unità sono pronte per comandi temporizzati.

Il controller deve aggiornare periodicamente la sincronizzazione. I clock locali possono derivare nel tempo. Anche se un nodo è sincronizzato all'inizio, dopo minuti o ore può accumulare differenze. Per questo motivo, il controller deve inviare sync periodici o misurare offset a intervalli regolari. La frequenza di sincronizzazione dipende dalla precisione richiesta. Per demo semplici, può essere bassa; per sequenze temporizzate più precise, deve aumentare. Tuttavia, sync troppo frequenti generano traffico. Anche qui serve equilibrio.

Una politica può essere:

sync normale a intervalli regolari;

sync più frequente prima di pattern temporizzati;

sync immediato dopo reboot di un nodo;

sync richiesto se jitter o offset superano soglia;

sync ridotto in modalità idle o low-power.

Questa strategia mantiene il sistema sincronizzato senza saturare la rete. Il controller adatta la sincronizzazione al contesto operativo.

Il controller come riferimento temporale deve anche decidere quali nodi sono "ammissibili" per una sequenza. Prima di avviare un pattern sincronizzato, può controllare:

nodo online;

heartbeat recente;

`sync_quality` sufficiente;

RTT accettabile;

jitter sotto soglia;

stato safety ok;

capacità richieste presenti.

Solo i nodi che soddisfano questi criteri partecipano alla run. Gli altri possono essere esclusi, messi in idle o segnalati come non pronti. Questa validazione evita che uno swarm apparentemente connesso ma temporalmente disallineato avvii una sequenza fragile. Il controller diventa quindi arbitro della readiness temporale del sistema.

Una condizione di readiness può essere:

$$\text{ready_for_timed_action} = \text{online} \wedge \text{sync_valid} \wedge \text{network_stable} \wedge \text{safety_ok} \wedge \text{capabilities_ok}$$

Questa formula sintetizza bene il concetto. Per un comando temporizzato non basta essere online. Il nodo deve essere temporalmente allineato, comunicativamente stabile e sicuro.

Il controller come riferimento temporale ha un ruolo anche nella gestione dei ritardi. Se un nodo risponde lentamente, il controller può aumentare il margine tra invio ed execute_at. Per esempio, se il RTT medio dello swarm è 60 ms ma un nodo ha RTT di 180 ms, il controller può decidere di inviare comandi con anticipo maggiore oppure escludere quel nodo da azioni strette. In questo modo, la sincronizzazione non è statica, ma adattiva. Il controller misura la rete e modifica la pianificazione.

Una politica di anticipo può essere:

$$\text{lead_time_ms} = \text{max_rtt_atteso} + \text{jitter_margin} + \text{processing_margin}$$

Dove lead_time_ms è il tempo minimo tra invio del comando ed execute_at. Se il controller invia troppo tardi, alcuni nodi potrebbero ricevere il comando dopo la scadenza. Se invia con anticipo sufficiente, i nodi hanno tempo di validare e programmare l'esecuzione. Questo è un concetto molto importante per il comportamento distribuito.

Per esempio, se max_rtt_atteso è 120 ms, jitter_margin 50 ms e processing_margin 30 ms, il controller dovrebbe inviare almeno 200 ms prima di execute_at. Per comandi critici, può usare margini maggiori. Per comandi non critici, margini più piccoli. La scelta deve essere basata su misure reali, non su supposizioni.

Il controller deve anche gestire nodi che ricevono comandi troppo tardi. Se un nodo risponde command_expired, il controller deve decidere se abortire la sequenza, continuare senza quel nodo, ritentare con un nuovo execute_at o passare a modalità safe. Questa decisione dipende dal comando. Per un LED sync dimostrativo, si può continuare. Per un impulso magnetico coordinato, potrebbe essere meglio abortire o ripianificare. Il controller deve quindi collegare sincronizzazione e criticità dell'azione.

Una policy può essere:

se comando visuale e un nodo scade, registra warning;

se comando pattern e pochi nodi scadono, valuta pattern ridotto;

se comando magnetico coordinato e un nodo scade, abortisci fase o entra in safe;

se comando safety scade, esegui comunque azione locale appena ricevuta se semanticamente appropriato.

Questa policy deve essere raffinata nel tempo, ma la struttura concettuale è questa.

Il controller come riferimento temporale è utile anche quando i nodi hanno capacità diverse. Un nodo simulato può seguire il tempo della simulazione; un nodo fisico segue tempo reale. Se si costruisce una demo ibrida, il controller deve decidere quale timebase domina. In una demo fisica live, il controller_time reale deve dominare. In un replay o simulazione offline, simulation_time può dominare. Il controller deve dichiarare timebase_id nei messaggi per evitare confusione.

Un messaggio può indicare:

timebase_id = "controller_realtime"

oppure:

timebase_id = "simulation_run_time"

Il nodo fisico dovrebbe accettare solo timebase compatibili. Un simulated_node può accettare simulation_time. Questo impedisce che un comando di replay venga interpretato come comando fisico live. È un dettaglio importante per un ecosistema che combina simulazione e hardware.

Il controller come riferimento temporale deve avere anche una responsabilità di archiviazione. Ogni run dovrebbe registrare le impostazioni temporali: frequenza di sync, timebase usata, nodi sincronizzati, offset stimati, latenza media, jitter, margini execute_at, timeout. Questi dati permettono di valutare la qualità della run. Se una demo fallisce, sapere che la rete aveva jitter alto o che alcuni nodi erano sync_degraded aiuta a spiegare il risultato.

Un record di run può contenere:

run_id;

timebase_id;

controller_start_time;

sync_interval_ms;

lead_time_ms;

nodes_participating;

average_rtt_by_node;

sync_quality_by_node;

timing_errors;

expired_commands.

Questa struttura rende la sincronizzazione analizzabile e documentabile. Non resta una funzione nascosta.

Dal punto di vista del firmware, il nodo deve trattare il controller come sorgente temporale autorevole solo se il messaggio proviene da una sorgente autorizzata. Un nodo non dovrebbe accettare sync da qualunque source casuale. Anche nelle prime versioni, il pacchetto sync dovrebbe avere source = CTRL-001 o sorgente riconosciuta. In futuro, si potrà aggiungere autenticazione. Per ora, il controllo della source è già un primo livello di ordine. Il tempo di sistema non deve essere modificato da messaggi non validi.

Una regola:

accetta sync solo da source autorizzata e protocol_version supportata.

Se un messaggio sync è malformato, vecchio o proveniente da sorgente non riconosciuta, il nodo lo ignora o segnala errore. Questo protegge la coerenza temporale.

Il controller come riferimento temporale deve poter gestire anche il proprio riavvio. Se il controller si riavvia, la timebase può cambiare. I nodi devono riconoscere una nuova sessione del controller tramite controller_boot_id o session_id. Se ricevono comandi da CTRL-001 ma con controller_session diversa, devono eventualmente invalidare vecchi riferimenti temporali, cancellare comandi pendenti basati sulla sessione precedente e richiedere nuovo sync. Questo evita che comandi vecchi di una sessione precedente vengano eseguiti nella nuova.

Un messaggio può contenere:

controller_id = "CTRL-001";

controller_session_id = "CTRL-BOOT-004";

timestamp_ms = ...

Il nodo conserva l'ultima sessione valida. Se la sessione cambia, aggiorna sync_status e può entrare in modalità resync_required. Questa gestione è più avanzata, ma molto utile per robustezza.

Il controller come riferimento temporale è anche importante nel caso di emergency_stop. Un comando di emergenza non deve dipendere eccessivamente dalla sincronizzazione, perché deve essere eseguito subito quando ricevuto. Tuttavia, il controller deve comunque registrare quando lo ha inviato, quando i nodi hanno confermato e quali nodi non hanno risposto. Il tempo serve quindi per audit e diagnosi, non necessariamente per programmare l'esecuzione. Questo mostra che non tutti i comandi usano il tempo nello stesso modo.

Per emergency_stop:

execute_at non necessario;

timestamp utile;

ack con received_at/executed_at utile;

timeout utile per identificare nodi senza conferma.

Il nodo esegue immediatamente e registra il tempo locale. Il controller ricostruisce la timeline di emergenza. Questo è importante per sicurezza e debugging.

Il controller come riferimento temporale deve infine essere coerente con la filosofia generale del progetto MicroBot. Il sistema è distribuito, ma non privo di ordine. I nodi hanno autonomia locale,

ma il comportamento collettivo richiede una regia temporale. Il controller fornisce questa regia. Non impone necessariamente ogni micro-movimento, ma stabilisce quando iniziano le fasi, quando scadono i comandi, quando avvengono i pattern, quando viene richiesta telemetria, quando si considera un nodo offline e quando una run è completata. Il controller è quindi il metronomo del sistema.

Questa immagine del controller come metronomo è tecnicamente corretta: esso scandisce il tempo comune, mentre i MicroBot eseguono localmente in base al proprio stato e ai propri vincoli. Se il metronomo è instabile, lo swarm perde coordinamento. Se i nodi non ascoltano o non riescono ad allinearsi, il comportamento diventa incoerente. La sincronizzazione è il contratto temporale tra controller e nodi.

In conclusione, il controller come riferimento temporale è il fondamento della sincronizzazione MicroBot. Esso definisce la timebase principale, genera timestamp, pianifica `execute_at`, misura latenze, raccoglie ack, ordina eventi, costruisce timeline, avvia run sperimentali e valuta la qualità temporale dei nodi. I MicroBot mantengono clock locali per sicurezza e temporizzazioni interne, ma usano il controller come riferimento comune per azioni coordinate. Questa architettura permette di passare da messaggi isolati a sequenze distribuite coerenti. Il punto successivo, dedicato al clock globale del sistema, formalizzerà questa idea descrivendo come il tempo del controller possa diventare una base condivisa, stimata e validata per tutto lo swarm.

9.2 Clock globale del sistema

Il clock globale del sistema rappresenta il riferimento temporale condiviso attraverso cui lo swarm MicroBot può ordinare eventi, programmare comandi, coordinare azioni e mantenere coerenza tra nodi fisici, nodi simulati, controller, dashboard e log. Dopo aver definito il controller come riferimento temporale principale, è necessario formalizzare il concetto di clock globale, perché in un sistema distribuito il tempo non è automaticamente unico. Ogni MicroBot possiede un proprio clock locale, il controller possiede il proprio clock, la simulazione può avere un tempo interno e la dashboard può ricevere dati con ritardi diversi. Il clock globale non elimina completamente queste differenze, ma fornisce una base comune rispetto alla quale il sistema può interpretare il “prima”, il “dopo”, il “quando eseguire” e il “per quanto tempo”.

Nel progetto MicroBot, il clock globale non deve essere inteso come un orologio fisicamente perfetto presente identico in ogni nodo. Sarebbe irrealistico nelle prime versioni. Deve essere invece interpretato come una timebase logica condivisa, generata dal controller e stimata dai nodi attraverso messaggi di sincronizzazione. Ogni MicroBot mantiene il proprio tempo locale, ma cerca di costruire una relazione tra il proprio clock e il clock del controller. In questo modo, quando il controller invia un comando con `execute_at`, il nodo può tradurre quel tempo globale in un istante del proprio clock locale. Il clock globale è quindi una convenzione coordinata, non una proprietà assoluta naturale del sistema.

La relazione fondamentale può essere descritta così:

$$\text{tempo_globale} \approx \text{tempo_locale_nodo} + \text{offset_nodo}$$

In questa relazione, il tempo_globale coincide, nelle prime versioni, con il tempo del controller. Il tempo_locale_nodo è il contatore interno del MicroBot, spesso misurato in millisecondi dall'avvio. L'offset_nodo è la differenza stimata tra il tempo del controller e il tempo locale del nodo. Se questa stima è abbastanza buona, il nodo può interpretare correttamente i comandi temporizzati. Se invece l'offset è errato, il nodo può eseguire troppo presto o troppo tardi. Per questo motivo, la qualità

della sincronizzazione dipende dalla precisione con cui ogni nodo conosce il proprio offset rispetto al clock globale.

Il clock globale ha una funzione fondamentale: permette di separare il tempo di ricezione dal tempo di esecuzione. Senza clock globale, un comando viene normalmente eseguito appena ricevuto. Questo significa che due MicroBot che ricevono lo stesso pacchetto in momenti diversi eseguono in momenti diversi. Con clock globale, invece, il controller può inviare il comando in anticipo e specificare un istante futuro comune. I nodi possono ricevere il comando con ritardi diversi, ma eseguirlo nello stesso momento logico. Questa differenza è essenziale per costruire azioni coordinate.

Un esempio semplice è il lampeggio sincronizzato dei LED. Senza clock globale, il controller invia un comando broadcast e ogni nodo lampeggia appena riceve il pacchetto. Alcuni LED possono accendersi leggermente prima, altri leggermente dopo. Con clock globale, il controller invia il comando:

`execute_at = T`

Ogni nodo riceve il comando, lo valida, lo inserisce nella propria coda temporale e lo esegue quando il proprio clock locale, corretto tramite offset, corrisponde al tempo globale T. In questo modo, il lampeggio può apparire simultaneo anche se la rete non ha consegnato il pacchetto esattamente nello stesso istante a tutti i nodi. Questo stesso principio, applicato a comandi più complessi, consente sincronizzazione di pattern, fasi operative e impulsi controllati.

Il clock globale è ancora più importante quando il sistema passa da semplici LED a comportamenti magnetici. Un impulso magnetico coordinato tra più MicroBot può dipendere da finestre temporali precise. Se una bobina viene attivata troppo presto rispetto a un'altra, il campo risultante può non corrispondere alla configurazione prevista. Se una fase di aggancio inizia prima che i nodi siano allineati, il sistema può generare una connessione instabile. Se una fase di distacco viene eseguita in ritardo, la riconfigurazione può fallire. Il clock globale permette di definire sequenze ordinate: prima preparazione, poi allineamento, poi aggancio, poi stabilizzazione, poi eventuale distacco.

Una sequenza temporale può essere rappresentata così:

T0: preparazione dello swarm;
T1: avvio fase di allineamento;
T2: abilitazione aggancio;
T3: stabilizzazione connessioni;
T4: verifica telemetria;
T5: fine pattern o transizione successiva.

Ogni nodo può ricevere comandi associati a questi tempi. Anche se i nodi hanno ruoli diversi, tutti condividono la stessa struttura temporale. Il leader può eseguire una certa azione a T1, i follower un'altra a T1, i nodi di bordo un'altra ancora, ma il riferimento è comune. Il clock globale non obbliga tutti a fare la stessa cosa; permette a tutti di sapere quando fare ciò che devono fare.

Il clock globale è utile anche per ordinare gli eventi. In uno swarm, molti eventi possono avvenire quasi contemporaneamente: un nodo riceve un comando, un altro segnala heartbeat, una connessione viene formata, una soglia viene superata, un terzo nodo entra in safety. Senza un riferimento temporale comune, ricostruire l'ordine reale degli eventi diventa difficile. Il controller può ricevere prima un evento avvenuto dopo, semplicemente perché il pacchetto è arrivato più rapidamente. Il clock globale consente di distinguere il tempo dell'evento dal tempo di ricezione. Questo migliora la qualità dei log e permette analisi causale più rigorosa.

Per esempio, il controller può ricevere questi messaggi nell'ordine:

messaggio A ricevuto alle 10:00:02.300;
messaggio B ricevuto alle 10:00:02.320.

Tuttavia, se i timestamp globali stimati indicano:

evento A avvenuto a $T = 5200$ ms;
evento B avvenuto a $T = 5180$ ms;

allora B è avvenuto prima di A, anche se è stato ricevuto dopo. Questa distinzione è cruciale nei sistemi distribuiti. Il clock globale permette di ricostruire una timeline più vicina alla dinamica reale dello swarm, invece di basarsi soltanto sull'ordine di arrivo dei pacchetti.

Dal punto di vista della simulazione, il clock globale può coincidere con il tempo della simulazione oppure con il tempo reale del controller. Questa distinzione deve essere dichiarata. In una simulazione live collegata a nodi fisici, è naturale usare il tempo del controller reale come riferimento. In una simulazione offline o replay, invece, il tempo può essere controllato dal motore simulativo: può andare più veloce, più lento, fermarsi o essere riavvolto. In questo caso, il clock globale del sistema simulato è `simulation_time`, non necessariamente il tempo reale del computer. Per evitare ambiguità, i messaggi e gli snapshot dovrebbero indicare la timebase utilizzata.

Una struttura di messaggio può contenere:

`timebase_id = "controller_time"`

oppure:

`timebase_id = "simulation_time"`

Questa informazione chiarisce come interpretare `timestamp`, `execute_at`, `event_time` e `duration_ms`. Un nodo fisico reale dovrebbe normalmente accettare comandi basati su `controller_time`, mentre un nodo simulato può lavorare con `simulation_time`. In una demo ibrida, il controller deve decidere quale timebase domina e convertire, se necessario, tra tempo reale e tempo simulato. Questa distinzione è importante perché MicroBot non è solo hardware, ma anche ambiente di simulazione, dashboard, replay e documentazione sperimentale.

Il clock globale deve essere monotono dal punto di vista operativo. Per il controllo, è pericoloso usare un orologio che può tornare indietro o saltare in avanti a causa di correzioni del sistema operativo. Le durate, i timeout, gli `execute_at`, gli `expires_at` e le finestre temporali devono basarsi su un clock monotono, cioè un contatore che cresce sempre durante la sessione. Il tempo assoluto, come data e ora del calendario, è utile per i log umani, ma non dovrebbe essere la base primaria del controllo realtime.

Una scelta corretta è distinguere:

clock globale monotono: usato per controllo, sincronizzazione, timeout, `execute_at`;
datetime assoluto: usato per archiviazione, nome delle run, documentazione e lettura umana.

Per esempio, il controller può avviare una run alle 2026-04-30T21:00:00+02:00, ma internamente usare `run_time_ms = 0` come inizio dell'esperimento. Tutti i comandi e gli eventi della run possono essere riferiti a `run_time_ms`. Questo rende più semplice analizzare l'esperimento: al millisecondo 0 parte la run, al millisecondo 1200 avviene l'allineamento, al millisecondo 2500 vengono confermati

gli agganci, al millisecondo 4000 si completa la stabilizzazione. La data assoluta resta disponibile, ma il comportamento viene analizzato nel tempo relativo della run.

Nel sistema MicroBot, si possono quindi distinguere tre livelli temporali:

tempo di sessione del controller;
tempo relativo della run;
tempo locale del nodo.

Il tempo di sessione del controller parte quando il controller viene avviato. Il tempo relativo della run parte quando inizia un test, una demo o un pattern specifico. Il tempo locale del nodo parte quando il MicroBot si accende o si riavvia. Il clock globale operativo può essere il tempo di sessione del controller o il tempo della run, a seconda del contesto. L'importante è che il sistema dichiari quale riferimento viene usato e mantenga una mappatura coerente.

Una mappatura utile è:

`controller_time_ms` = tempo monotono del controller;
`run_time_ms` = `controller_time_ms` - `run_start_controller_time_ms`;
`node_local_time_ms` = tempo monotono interno del nodo;
`node_offset_ms` = `controller_time_ms` - `node_local_time_ms` stimato.

Questa struttura permette al controller di convertire eventi locali dei nodi in tempi globali e tempi relativi alla run. Per esempio, se MB-001 segnala un evento al proprio tempo locale 8120 ms e l'offset stimato è 15400 ms, l'evento può essere collocato nel tempo controller a circa 23520 ms. Se la run è iniziata a controller_time 20000 ms, l'evento appartiene alla run al tempo 3520 ms. Questa conversione permette analisi temporale precisa.

Il clock globale deve essere accompagnato da una misura di qualità. Non basta stimare un offset; bisogna sapere quanto quella stima è affidabile. La rete introduce latenza e jitter, il firmware può processare messaggi con piccoli ritardi, il clock locale può derivare. Per questo motivo, ogni nodo dovrebbe avere uno stato di sincronizzazione. Non tutti i nodi saranno sincronizzati allo stesso livello. Alcuni possono essere `sync_good`, altri `sync_degraded`, altri `sync_lost`. Il controller deve considerare questa qualità prima di inviare comandi temporizzati.

Una classificazione possibile è:

`sync_good`: offset recente, RTT basso, jitter basso;
`sync_ok`: offset valido ma con margine maggiore;
`sync_degraded`: offset vecchio o jitter elevato;
`sync_lost`: nessun riferimento temporale affidabile;
`sync_unknown`: nodo appena connesso o non ancora misurato.

Questa classificazione può essere mostrata nella dashboard e usata nelle decisioni operative. Un nodo `sync_good` può partecipare a impulsi coordinati; un nodo `sync_ok` può partecipare a pattern meno critici; un nodo `sync_degraded` può essere limitato a comandi immediati o non critici; un nodo `sync_lost` non dovrebbe eseguire `execute_at` basati sul clock globale. Questa logica protegge lo swarm da errori temporali.

Il clock globale deve inoltre considerare la deriva del clock locale. Anche se un nodo viene sincronizzato a un certo istante, il suo clock interno può non procedere esattamente alla stessa velocità del clock del controller. Nel tempo, la differenza può aumentare. Questa deriva può essere piccola ma non nulla. Per azioni brevi o demo di pochi secondi può essere trascurabile; per sistemi

più lunghi deve essere corretta con sync periodici. Il controller può aggiornare l'offset nel tempo e, se necessario, stimare anche una correzione di drift.

Una forma più avanzata della relazione temporale è:

$$\text{tempo_globale} \approx \text{scala_nodo} \cdot \text{tempo_locale_nodo} + \text{offset_nodo}$$

Dove `scala_nodo` rappresenta una piccola correzione della velocità del clock locale. Nelle prime versioni, si può assumere `scala_nodo = 1` e correggere solo offset. In versioni più evolute, si può stimare anche la deriva. È importante non complicare troppo la prima implementazione, ma documentare questa possibilità mostra consapevolezza del problema.

Il clock globale deve essere utilizzato anche per gli slot temporali, che verranno approfonditi nel punto successivo. Gli slot sono finestre definite sul clock globale. Per esempio, uno slot può andare da $T = 1000$ ms a $T = 1100$ ms. Durante quello slot, certi nodi possono inviare ack, eseguire una fase, attivare LED o trasmettere telemetria. Senza clock globale, gli slot non avrebbero significato condiviso. Ogni nodo avrebbe una finestra diversa. Con clock globale, invece, tutti i nodi possono riferirsi alla stessa griglia temporale, anche se la traducono localmente.

Una griglia temporale può essere:

slot_0: $T_0 \rightarrow T_0 + \Delta T$;

slot_1: $T_0 + \Delta T \rightarrow T_0 + 2\Delta T$;

slot_2: $T_0 + 2\Delta T \rightarrow T_0 + 3\Delta T$.

Il controller comunica T_0 e ΔT . Ogni nodo calcola quando iniziano gli slot nel proprio tempo locale. Questo permette di distribuire risposte, evitare collisioni di ack, organizzare fasi e coordinare azioni. Il clock globale è quindi la base matematica degli slot.

Il clock globale è anche essenziale per la gestione dei timeout. Quando il controller invia un comando e attende risposta, deve stabilire una scadenza. Questa scadenza è definita nel suo tempo globale. Se entro timeout non arriva ack, il comando è considerato non confermato. Analogamente, il nodo può ricevere un comando con `expires_at` e rifiutarlo se il tempo globale stimato è oltre la scadenza. Tutto questo richiede una timebase condivisa. Senza clock globale, i timeout esisterebbero solo localmente e non sarebbero confrontabili.

Un comando può contenere:

`execute_at = 5000 ms`;

`expires_at = 5300 ms`.

Il nodo converte questi valori nel proprio tempo locale. Se riceve il comando e il suo tempo globale stimato è già 5400 ms, lo rifiuta. Se è 4900 ms, lo programma. Se è 5200 ms, può decidere se eseguire ancora entro tolleranza. Questa logica rende il comportamento più robusto rispetto alla latenza.

Il clock globale deve essere collegato anche alla sicurezza. Alcune azioni devono avere durata massima e cooldown. Questi tempi possono essere gestiti localmente dal nodo, ma devono essere riportati al clock globale nei log. Per esempio, una bobina si attiva localmente per 120 ms; il nodo sa quando accenderla e spegnerla usando il suo clock locale. Poi invia al controller l'evento con `local_on_time`, `local_off_time` e, se possibile, `controller_time` stimato. Il controller può registrare durata e collocazione nella run. Così safety locale e logging globale restano coerenti.

Una struttura di evento può contenere:

```
coil_on_local_ms;  
coil_off_local_ms;  
coil_on_global_estimated_ms;  
coil_off_global_estimated_ms;  
duration_ms.
```

Nelle prime versioni si può semplificare, ma il principio è chiaro: il nodo protegge fisicamente usando il proprio tempo locale; il controller analizza usando il clock globale. I due livelli devono dialogare.

Il clock globale è utile anche per misurare la durata delle connessioni. Nel modello matematico, una connessione tra due MicroBot ha un tempo di creazione e un tempo di distacco. Se questi tempi vengono registrati in clock globale, si può calcolare `connection_lifetime`. Questo valore è importante per valutare stabilità dello swarm. Una connessione che dura 50 ms può essere un aggancio instabile; una che dura 10 secondi può indicare stabilità; una che non si distacca mai può indicare rigidità eccessiva. Il clock globale permette di misurare questi fenomeni in modo uniforme.

Una connessione può avere:

```
created_at_global_ms;  
detached_at_global_ms;  
lifetime_ms = detached_at - created_at.
```

Questi dati possono essere confrontati tra nodi, run e versioni del simulatore. Senza clock globale, ogni connessione avrebbe tempi locali difficili da confrontare.

Il clock globale deve anche gestire eventi simultanei o quasi simultanei. In uno swarm, due eventi possono avere timestamp molto vicini. La risoluzione del clock determina quanto il sistema può distinguerli. Per molte funzioni iniziali, una risoluzione in millisecondi è sufficiente. Per fenomeni molto rapidi, potrebbe servire maggiore precisione. Tuttavia, la precisione utile non deve superare troppo la precisione reale della rete e del firmware. Non ha senso dichiarare sincronizzazione al microsecondo se la latenza Wi-Fi varia di millisecondi o decine di millisecondi. La risoluzione deve essere coerente con il sistema.

Una scelta realistica iniziale è lavorare in millisecondi. Questo è comprensibile, leggibile, adatto a LED, heartbeat, timeout, pattern iniziali e impulsi controllati di durata non estrema. In futuro, parti firmware interne possono usare microsecondi per PWM o misure locali, ma il protocollo di alto livello può restare in millisecondi. Questa separazione evita complessità inutile.

Il clock globale deve essere robusto rispetto ai reboot. Se un nodo si riavvia, il suo tempo locale riparte. Il controller deve rilevarlo tramite `boot_id` e invalidare l'offset precedente. Un offset calcolato prima del reboot non è più valido perché il tempo locale ha nuova origine. Il nodo deve passare a `sync_unknown` o `sync_lost` finché non riceve nuova sincronizzazione. Questo è un dettaglio fondamentale. Senza questa regola, il controller potrebbe usare una conversione temporale sbagliata e inviare `execute_at` che il nodo interpreta male.

Una regola:

se `boot_id` cambia, invalidare `sync_offset` del nodo e richiedere nuova sincronizzazione.

Questa regola deve essere implementata nel controller e possibilmente anche nel nodo. Dopo reboot, il nodo deve annunciare nuova sessione e non accettare comandi temporizzati basati su vecchio sync.

Anche il reboot del controller deve essere gestito. Se il controller cambia session_id, i nodi devono riconoscere che il riferimento temporale può essere cambiato. I comandi pendenti basati sulla vecchia sessione devono essere scartati. Il controller deve inviare nuova discovery o sync. In uno sviluppo iniziale questo può essere gestito manualmente, ma in una documentazione rigorosa va previsto. Il clock globale dipende dalla sessione del controller; se la sessione cambia, la timebase deve essere rinegoziata.

Una struttura può includere:

controller_session_id;

timebase_id;

sync_epoch.

Il sync_epoch può aumentare quando il controller resetta la sincronizzazione. I nodi possono rifiutare comandi con sync_epoch vecchio. Questo evita l'esecuzione di messaggi appartenenti a una sessione temporale precedente.

Il clock globale deve essere integrato nel protocollo dei pacchetti. Ogni messaggio che usa tempo globale dovrebbe dichiararlo. Campi come timestamp, execute_at ed expires_at devono essere interpretati rispetto a una timebase specifica. In una versione semplice, si può assumere che tutti questi valori siano in controller_time_ms. In una versione più chiara, si può aggiungere timebase_id. Questo riduce ambiguità quando si introducono simulazione e replay.

Un pacchetto temporizzato può contenere:

```
"timebase": "controller_time_ms",  
"timestamp_ms": 120000,  
"execute_at_ms": 121000,  
"expires_at_ms": 121300
```

Questa forma è esplicita. I suffissi _ms eliminano ambiguità sulle unità. Il nodo sa che deve usare la conversione verso controller_time_ms. Se non ha sync valido per quella timebase, rifiuta.

Il clock globale deve essere osservabile nella dashboard. Una dashboard matura può mostrare il tempo della run, la timebase attiva, lo stato di sincronizzazione dei nodi, l'offset stimato, il jitter, il tempo dall'ultimo sync e gli errori di esecuzione. Questo rende la sincronizzazione visibile e non nascosta. In uno swarm, molti problemi possono sembrare errori di comportamento ma derivare da tempo disallineato. Mostrare sync_quality aiuta a capire se il problema è nelle regole, nella rete o nella sincronizzazione.

Una scheda nodo può mostrare:

```
sync_status;  
estimated_offset_ms;  
last_sync_age_ms;  
average_rtt_ms;
```

jitter_ms;
last_execution_error_ms.

Questi dati permettono di capire subito se un nodo è temporalmente affidabile. Se MB-004 ha sync_lost, non bisogna aspettarsi che partecipi correttamente a un pattern temporizzato. Se MB-005 ha execution_error elevato, può esserci un problema firmware o rete.

Il clock globale permette anche di definire metriche di performance temporale. Alcune metriche utili sono:

sync_error stimato;
execution_error;
command_latency;
ack_delay;
event_delay;
heartbeat_interval_error;
pattern_phase_delay.

Queste metriche consentono di confrontare versioni del sistema. Per esempio, una nuova versione del firmware può ridurre execution_error medio; una nuova strategia di sync può ridurre jitter; una nuova rete può migliorare RTT. Senza clock globale, questi miglioramenti sarebbero difficili da misurare.

Il clock globale deve essere collegato anche al modello di controllo. Il controller può decidere di inviare comandi con lead_time maggiore se la qualità temporale è scarsa. Può anche decidere di ridurre la complessità del pattern. Per esempio, se lo swarm ha sync_good, può eseguire una sequenza LED rapida. Se ha sync_degraded, può eseguire una sequenza più lenta con finestre più ampie. Se ha sync_lost, può evitare sequenze temporizzate. Questo è adattamento basato sul clock.

Una logica può essere:

sync_good → pattern rapido e temporizzato;
sync_ok → pattern normale con margini;
sync_degraded → pattern lento o solo visuale;
sync_lost → comandi immediati semplici o idle.

Questa capacità rende il sistema più robusto. Non tratta il tempo come dato fisso, ma come risorsa misurata.

Dal punto di vista teorico, il clock globale rappresenta la dimensione temporale condivisa dello swarm. Nelle sezioni precedenti, il sistema è stato definito come insieme di posizioni, stati, connessioni e forze. Tutte queste grandezze sono funzioni del tempo. Il clock globale è il riferimento che permette di dire quale configurazione esiste in un certo istante, quale transizione avviene dopo un'altra e quale fase appartiene a quale intervallo. Senza clock globale, lo swarm sarebbe una collezione di eventi locali non facilmente componibili. Con clock globale, diventa un processo collettivo ordinato.

Questa idea è molto coerente con il concetto di forma come evento dinamico. La forma MicroBot non è un oggetto statico; è una configurazione che si mantiene e si trasforma nel tempo. Il clock globale è ciò che permette di descrivere questa trasformazione. Una linea non è solo una disposizione di nodi, ma una sequenza: i nodi si preparano, si avvicinano, si allineano, si agganciano, si stabilizzano e poi eventualmente si riconfigurano. Ogni fase ha un tempo. Il clock globale è la struttura che dà ordine a questa evoluzione.

Nel concetto di materia programmabile, il tempo è tanto importante quanto la geometria. Programmare la materia non significa solo definire una forma finale, ma definire il processo attraverso cui quella forma nasce, resta stabile e cambia. Il clock globale permette di programmare il processo. Un comando non dice solo “fai questo”, ma “fai questo a questo istante, per questa durata, entro questa finestra, in questa fase”. Questa è una differenza enorme dal punto di vista ingegneristico.

Il clock globale deve però restare proporzionato allo stato reale del progetto. Nelle prime versioni MicroBot, non serve inseguire una precisione estrema. Serve costruire una base chiara: timestamp coerenti, tempo monotono del controller, offset stimato, sync periodico, `execute_at`, `expires_at`, `duration_ms`, log ordinati e dashboard che mostra `sync_status`. Questa base è già sufficiente per trasformare il sistema da semplice scambio di messaggi a coordinamento temporale. Le ottimizzazioni avanzate possono arrivare dopo.

Una roadmap temporale può essere:

- v0.1: timestamp nei pacchetti e log ordinati;
- v0.2: `controller_time` monotono e `run_time`;
- v0.3: `execute_at` e `duration_ms` per comandi temporizzati;
- v0.4: sync periodico e offset per nodo;
- v0.5: `sync_quality`, RTT, jitter e dashboard temporale;
- v0.6: slot temporali e ack distribuiti;
- v0.7: compensazione drift e sincronizzazione più robusta.

Questa progressione è coerente con lo sviluppo incrementale del sistema. Ogni versione aggiunge controllo temporale senza rendere subito il progetto ingestibile.

In conclusione, il clock globale del sistema è la timebase logica che permette a MicroBot di trasformare messaggi distribuiti in azioni coordinate. Esso nasce dal controller come riferimento principale, viene stimato dai nodi attraverso offset e sincronizzazione, viene usato per timestamp, `execute_at`, `expires_at`, `duration_ms`, log, run, eventi, pattern e safety. Il clock globale non richiede perfezione assoluta nelle prime versioni, ma richiede coerenza, monotonicità, qualità misurabile e gestione chiara dei casi di errore. Senza clock globale, i nodi possono comunicare ma non coordinarsi temporalmente; con clock globale, lo swarm può eseguire fasi comuni, ordinare eventi, programmare azioni e costruire forme dinamiche in modo molto più rigoroso. Il punto successivo, dedicato agli slot temporali, userà questo riferimento condiviso per descrivere come il tempo possa essere diviso in finestre operative, utili per coordinare azioni, distribuire risposte, ridurre collisioni comunicative e organizzare sequenze dello swarm.

9.3 Slot temporali

Gli slot temporali rappresentano la suddivisione ordinata del clock globale in finestre operative, ciascuna destinata a una determinata azione, fase, risposta o trasmissione del sistema MicroBot. Dopo aver definito il clock globale come riferimento temporale condiviso, diventa necessario spiegare come questo tempo possa essere organizzato in intervalli discreti e controllabili. In uno swarm distribuito, infatti, non basta avere un tempo comune: bisogna anche stabilire quando i nodi devono parlare, quando devono agire, quando devono attendere, quando devono rispondere e quando devono evitare di sovrapporsi. Gli slot temporali servono proprio a trasformare il tempo condiviso in una struttura organizzata.

Nel progetto MicroBot, uno slot temporale può essere definito come una finestra temporale compresa tra un istante di inizio e un istante di fine, misurata rispetto al clock globale del sistema.

Se il clock globale è fornito dal controller, ogni slot viene definito nel tempo del controller e poi tradotto localmente dai nodi attraverso l'offset stimato. In forma generale, uno slot può essere rappresentato come:

$$\text{slot_k} = [T_k, T_k + \Delta T)$$

dove T_k rappresenta l'istante di inizio dello slot k-esimo e ΔT rappresenta la durata dello slot. Tutti i nodi che partecipano alla sequenza devono interpretare questo intervallo rispetto alla stessa timebase. Se un nodo è sincronizzato correttamente, può sapere quando uno slot inizia e finisce nel proprio tempo locale. Questo permette di organizzare azioni coordinate anche in presenza di comunicazione non perfettamente simultanea.

L'idea di slot temporale nasce dall'esigenza di evitare caos operativo. Se tutti i MicroBot inviano messaggi, ack, heartbeat, telemetria o eventi nello stesso istante, la rete può congestionarsi. Se tutti tentano di attivare comportamenti fisici senza una sequenza ordinata, il sistema può generare instabilità. Se ogni nodo risponde a un broadcast immediatamente, il controller può ricevere un picco di pacchetti difficile da gestire. Lo slot temporale fornisce una soluzione: invece di lasciare che tutti agiscano appena possibile, il sistema assegna finestre temporali in cui determinate azioni sono consentite o previste. Il tempo diventa quindi una risorsa condivisa da pianificare.

Nel livello comunicativo, gli slot temporali sono utili soprattutto per distribuire le risposte dei nodi. Dopo un comando broadcast, per esempio, tutti i MicroBot potrebbero dover inviare un ack. Se lo fanno contemporaneamente, si crea il problema già discusso dell'ack storm. Una soluzione consiste nell'assegnare a ciascun nodo uno slot di risposta. Il controller invia un comando globale e specifica una finestra di risposta divisa in sotto-slot. Ogni nodo calcola il proprio sotto-slot in base al `node_id`, a un indice assegnato o a una funzione deterministica. In questo modo, le risposte vengono distribuite nel tempo.

Un esempio concettuale può essere:

$$\text{finestra_ack} = [T_ack_start, T_ack_end]$$

$$\text{slot_ack_MB-001} = [T_ack_start, T_ack_start + \Delta T_ack)$$

$$\text{slot_ack_MB-002} = [T_ack_start + \Delta T_ack, T_ack_start + 2\Delta T_ack)$$

$$\text{slot_ack_MB-003} = [T_ack_start + 2\Delta T_ack, T_ack_start + 3\Delta T_ack)$$

In questa struttura, ogni nodo sa quando deve rispondere. Il controller non riceve tutti gli ack nello stesso istante, ma una sequenza ordinata. Questo riduce congestione, collisioni e perdita di pacchetti. Nei primi prototipi con pochi nodi può sembrare eccessivo, ma diventa molto importante quando il numero di MicroBot cresce. Progettare gli slot fin dall'inizio rende il protocollo più scalabile.

Gli slot temporali possono essere assegnati in modo statico o dinamico. Nell'assegnazione statica, ogni nodo ha sempre lo stesso slot relativo, determinato dal proprio `node_id` o da un indice. Per esempio, MB-001 risponde sempre prima di MB-002, MB-002 prima di MB-003, e così via. Questa soluzione è semplice, prevedibile e facile da debuggare. Nell'assegnazione dinamica, invece, il controller può assegnare slot diversi a seconda del pattern, dello stato dello swarm, della priorità dei nodi o della qualità della rete. Questa soluzione è più flessibile, ma richiede più coordinamento. Nelle prime versioni MicroBot, una logica statica o semi-statica è probabilmente la più adatta.

Una formula semplice per calcolare lo slot di risposta può essere:

$\text{slot_index} = \text{indice_del_nodo}$

$\text{ack_time} = T_{\text{ack_start}} + \text{slot_index} \cdot \Delta T_{\text{ack}}$

Dove indice_del_nodo può essere derivato da una tabella del controller o dal suffisso numerico del node_id . Per esempio, MB-001 può avere indice 0, MB-002 indice 1, MB-003 indice 2. Questa soluzione funziona bene se gli ID sono ordinati e non ci sono conflitti. In versioni più avanzate, il controller può inviare direttamente slot_index nel comando, evitando che ogni nodo debba derivarlo autonomamente.

Un pacchetto broadcast potrebbe quindi contenere:

```
{
  "command": "start_pattern",
  "target": "all",
  "execute_at_ms": 5000,
  "payload": {
    "pattern_id": "line_basic_v0.1",
    "ack_window_start_ms": 5200,
    "ack_slot_duration_ms": 20
  }
}
```

Ogni nodo riceve il comando, programma l'esecuzione del pattern a 5000 ms e, se deve inviare ack, lo fa nel proprio slot a partire da 5200 ms. In questo modo, esecuzione e conferma sono entrambe organizzate temporalmente. Il comando non si limita a dire cosa fare, ma definisce anche quando eseguire e quando rispondere.

Gli slot temporali non servono soltanto alla comunicazione, ma anche alla coordinazione delle azioni. In uno swarm, alcune azioni devono essere simultanee, mentre altre devono essere sequenziali. Per esempio, in una fase di preparazione tutti i nodi possono entrare nello stesso stato contemporaneamente. In una fase di aggancio, invece, potrebbe essere utile attivare gruppi diversi in momenti diversi, per evitare instabilità fisica o eccessivo consumo energetico. Gli slot permettono di pianificare queste azioni in modo ordinato. Un gruppo di nodi può agire nello slot 1, un altro nello slot 2, un altro ancora nello slot 3. Il controller non deve inviare comandi in tempo reale per ogni passaggio; può inviare una sequenza programmata.

Una sequenza di slot comportamentali può essere:

slot_0: preparazione e validazione;

slot_1: allineamento dei nodi leader;

slot_2: allineamento dei follower;

slot_3: abilitazione magnetica controllata;

slot_4: verifica connessioni;

slot_5: stabilizzazione;

slot_6: telemetria finale.

Questa struttura rende il comportamento dello swarm più prevedibile. Invece di avviare tutto insieme, il sistema attraversa fasi temporali. Ogni fase ha un ruolo, una durata e condizioni di successo. Se una fase fallisce, il controller può interrompere o modificare la sequenza prima di procedere. Questo è molto importante per le versioni fisiche, dove attivare troppe cose contemporaneamente può essere rischioso.

Nel caso di comandi magnetici, gli slot temporali assumono un valore ancora più delicato. Le bobine consumano energia e possono generare calore. Se molti MicroBot attivano coil nello stesso istante, il consumo totale può aumentare rapidamente, la batteria può subire carico elevato e il sistema può diventare difficile da controllare. Gli slot permettono di distribuire l'attuazione nel tempo. Invece di attivare tutte le bobine insieme, il controller può attivare un sottoinsieme per volta. Questo riduce picchi energetici e permette al sistema di osservare l'effetto di ogni fase.

Una logica possibile è:

slot_magnetico_1: attiva solo nodi leader;

slot_magnetico_2: attiva nodi connector;

slot_magnetico_3: attiva nodi follower;

slot_magnetico_4: spegni attuazioni e verifica stabilità.

Questa sequenza è più controllata di un comando globale indiscriminato. Ogni slot può avere duty cycle, durata e limiti specifici. Il safety layer locale resta comunque attivo, ma lo slot temporale aiuta a ridurre il rischio sistemico. È una forma di sicurezza a livello di orchestrazione.

Gli slot temporali sono utili anche per separare comunicazione e attuazione. Un nodo non dovrebbe essere costretto a ricevere, processare, attuare e rispondere tutto nello stesso istante. Una sequenza più robusta può prevedere uno slot per ricevere e validare il comando, uno slot per eseguire, uno slot per inviare ack o telemetria, uno slot per stabilizzare. Questo riduce sovraccarico del firmware e rende più chiaro il comportamento del sistema.

Una sequenza ordinata può essere:

slot_receive: i nodi ricevono e validano i comandi;

slot_execute: i nodi eseguono le azioni programmate;

slot_measure: i nodi aggiornano stato e sensori;

slot_report: i nodi inviano ack o telemetria;

slot_decide: il controller valuta la fase successiva.

Questa struttura è molto vicina a un ciclo di controllo distribuito. Non tutte le fasi devono essere sempre esplicite nelle prime versioni, ma il concetto è utile. Il sistema diventa più leggibile perché ogni attività ha un tempo dedicato. Se qualcosa fallisce, si può capire in quale slot è avvenuto il problema.

Gli slot temporali possono essere assoluti o relativi. Uno slot assoluto è definito rispetto a un tempo globale specifico, per esempio da $T = 10000$ ms a $T = 10100$ ms. Uno slot relativo è definito rispetto all'inizio di una run o di una fase, per esempio "200 ms dopo l'inizio del pattern". Gli slot relativi sono spesso più comodi per descrivere pattern e sequenze. Il controller può tradurre gli slot relativi

in tempi assoluti al momento dell'avvio della run. Per esempio, se la run inizia a $T_0 = 5000$ ms e uno slot relativo inizia a +300 ms, allora lo slot assoluto inizia a 5300 ms.

Questa distinzione permette di definire pattern riutilizzabili. Un pattern può essere scritto come sequenza relativa:

phase_prepare: 0–500 ms;

phase_align: 500–1500 ms;

phase_attach: 1500–2200 ms;

phase_stabilize: 2200–4000 ms.

Quando il controller avvia la run, assegna un T_0 e calcola tutti gli slot. In questo modo, lo stesso pattern può essere eseguito in momenti diversi senza riscrivere la sequenza. Questo è molto utile per il file di configurazione dei pattern e per la simulazione.

Gli slot temporali possono essere collegati a `pattern_id` e `pattern_run_id`. Il `pattern_id` descrive la struttura del comportamento; il `pattern_run_id` identifica una specifica esecuzione. Gli slot appartengono alla run. Per esempio, `line_basic_v0.1` può avere una sequenza di slot standard, ma `line_run_003` è una esecuzione particolare con T_0 , nodi partecipanti, offset, ack e risultati specifici. Questo permette di confrontare run diverse dello stesso pattern.

Una struttura dati di run potrebbe includere:

`pattern_id`;

`pattern_run_id`;

`run_start_time_ms`;

`slot_plan`;

`nodes_participating`;

`slot_results`.

Lo `slot_plan` definisce cosa dovrebbe accadere. Gli `slot_results` registrano cosa è realmente accaduto. Questo confronto tra piano e risultato è molto importante per la validazione tecnica. Se uno slot di allineamento doveva terminare a $T = 1500$ ms ma alcuni nodi hanno risposto solo a $T = 1800$ ms, il sistema può registrare un ritardo di fase. Se lo slot di aggancio non produce connessioni attese, il pattern può essere considerato fallito o parziale.

Gli slot temporali devono essere abbastanza lunghi da tollerare latenza, jitter e tempo di elaborazione. Se uno slot è troppo stretto rispetto alla qualità reale della rete, i nodi possono non riuscire a eseguire o rispondere in tempo. Per esempio, se lo slot dura 5 ms ma il jitter della rete è 30 ms, la pianificazione è irrealistica. La durata dello slot deve essere scelta sulla base di misure concrete: RTT medio, jitter, tempo di parsing, frequenza del loop firmware, numero di nodi e criticità dell'azione. Questo è un punto metodologico fondamentale.

Una regola pratica è:

$$\Delta T_{\text{slot}} > \text{jitter_atteso} + \text{tempo_elaborazione} + \text{margine_sicurezza}$$

Per gli ack, bisogna considerare anche il numero di nodi. Se ogni nodo ha uno slot di risposta di 20 ms e ci sono 10 nodi, la finestra totale è 200 ms. Se ci sono 100 nodi, diventa 2000 ms. Con swarm grandi, slot individuali possono diventare costosi. In quel caso, si possono usare slot di gruppo, ack aggregati, risposte casualizzate o telemetria compressa. Per i primi prototipi, slot individuali sono perfettamente gestibili e molto chiari.

Gli slot temporali possono essere assegnati per nodo, per gruppo o per ruolo. Gli slot per nodo sono precisi ma meno scalabili. Gli slot per gruppo permettono a un insieme di nodi di agire insieme. Gli slot per ruolo permettono di coordinare leader, follower, connector, boundary e nodi diagnostici. Per esempio, tutti i leader possono agire nello slot 1, i connector nello slot 2, i follower nello slot 3. Questa struttura è molto coerente con il sistema multi-agente definito nella Sezione 7.

Una sequenza per ruolo può essere:

slot_leader: nodi con ruolo leader impostano orientamento;

slot_connector: nodi connector preparano aggancio;

slot_follower: nodi follower si allineano;

slot_boundary: nodi di bordo verificano stabilità.

Questa logica non richiede che ogni nodo abbia un comando completamente diverso. Il controller può inviare un piano di slot comune, e ogni nodo interpreta il proprio comportamento in base al ruolo. Questo riduce traffico e aumenta autonomia locale. In una prima versione, il controller può essere più esplicito; in una versione avanzata, i nodi possono interpretare piani più generali.

Gli slot temporali possono essere usati anche per telemetria. Se tutti i nodi inviano telemetria nello stesso momento, si rischia congestione. Il sistema può definire finestre di telemetria distribuite. Per esempio, ogni nodo invia heartbeat in un intervallo assegnato. Oppure la telemetria dettagliata viene richiesta a gruppi in slot successivi. Questo rende la raccolta dati più ordinata.

Una strategia è:

heartbeat leggero con jitter controllato;

telemetria normale distribuita per node_id;

telemetria debug solo su richiesta unicast;

telemetria di pattern in slot report dopo ogni fase.

Questa organizzazione evita traffico non necessario. È particolarmente importante quando la dashboard deve restare fluida e il controller deve ricevere dati senza ritardi eccessivi.

Gli slot temporali possono ridurre la probabilità di collisioni comunicative, ma non eliminano completamente problemi di rete. Wi-Fi e UDP non garantiscono determinismo stretto. Un nodo può perdere lo slot per ritardo, carico firmware o perdita del pacchetto. Per questo motivo, ogni slot deve avere una politica di errore. Se un nodo non risponde nel proprio slot, il controller può registrare missed_slot. Se risponde fuori slot, può registrare late_response. Se l'azione non viene eseguita entro la finestra, può registrare execution_timeout. Questi eventi sono importanti per la diagnosi.

Una risposta fuori slot non deve essere automaticamente ignorata in tutti i casi. Dipende dal tipo di messaggio. Un ack tardivo può ancora essere utile per sapere che il nodo ha ricevuto il comando, ma può indicare che non è stato sufficientemente sincronizzato. Una telemetria tardiva può essere registrata come stale. Un comando attuativo tardivo non deve essere eseguito se la finestra è scaduta. La policy deve quindi distinguere tra dati informativi e azioni fisiche.

Una regola utile è:

informazioni tardive possono essere registrate con warning;

azioni tardive devono essere rifiutate se fuori finestra;

ack tardivi aggiornano diagnosi ma non devono necessariamente cambiare decisioni già prese.

Questa distinzione protegge il sistema. Il tempo non serve solo a ordinare; serve anche a decidere se qualcosa è ancora valido.

Gli slot temporali devono essere integrati con il campo `expires_at`. Ogni comando associato a uno slot può avere una scadenza coincidente con la fine dello slot o con una tolleranza successiva. Se il nodo non riesce a eseguire entro lo slot, rifiuta il comando come expired. Questo impedisce azioni fuori tempo. Per esempio, un impulso magnetico previsto nello `slot_attach` non deve essere eseguito durante lo `slot_stabilize`, perché potrebbe disturbare la fase successiva.

Un comando può quindi contenere:

`execute_at_ms` = inizio dello slot;

`expires_at_ms` = fine dello slot;

`duration_ms` = durata dell'azione dentro lo slot.

Il nodo deve verificare che `duration_ms` sia compatibile con la finestra. Se `duration_ms` supera la durata dello slot, il comando è malformato o deve essere rifiutato. Questo evita che un'azione invada lo slot successivo. Per comandi fisici, questa verifica è fondamentale.

Gli slot temporali devono essere coerenti con la priorità dei comandi. Un comando di emergenza può interrompere qualunque slot. Se il sistema è nello `slot_attach` e arriva `emergency_stop`, il nodo non deve aspettare la fine dello slot. Deve interrompere subito. Gli slot organizzano il comportamento normale, ma la safety ha priorità superiore. Questa gerarchia deve essere chiara.

La regola è:

safety ed `emergency_stop` prevalgono sugli slot temporali.

Questo significa che uno slot non è una gabbia rigida. È una pianificazione operativa. In condizioni normali, i nodi la rispettano; in condizioni di sicurezza, viene interrotta. Il controller deve registrare che la run è stata abortita per safety e non considerare il mancato completamento degli slot come errore ordinario.

Gli slot temporali possono essere utilizzati per costruire una macchina a stati temporizzata. Ogni fase dello swarm può essere associata a uno slot o a una sequenza di slot. La macchina a stati globale può passare da `prepare` ad `align`, da `align` ad `attach`, da `attach` a `stabilize`, da `stabilize` a `complete`. Ogni transizione può avvenire per tempo o per condizione. Per esempio, lo slot `align` dura

al massimo 1000 ms, ma può terminare prima se tutti i nodi segnalano alignment_ok. Questo introduce una combinazione di controllo temporale e controllo event-based.

Una fase può quindi avere:

start_time;

max_duration;

success_condition;

failure_condition;

next_phase_on_success;

next_phase_on_timeout.

Questa struttura è molto potente. Non si aspetta sempre passivamente la fine dello slot; si può avanzare se il sistema è pronto. Ma se il sistema non è pronto entro il tempo massimo, si attiva una policy. Questo rende lo swarm più efficiente e robusto.

Per esempio:

fase align inizia a T1;

durata massima 1500 ms;

successo se almeno 90% nodi invia alignment_ok;

fallimento se più del 20% nodi è offline o safety;

timeout se $T > T1 + 1500 \text{ ms}$.

Questa logica trasforma gli slot temporali in fasi intelligenti, non semplici intervalli fissi. Nelle prime versioni, si può usare solo durata fissa; nelle versioni successive, si possono aggiungere condizioni.

Gli slot temporali sono utili anche nella simulazione. Il motore simulativo può avanzare a step discreti, e ogni step può corrispondere a uno slot o a una frazione di slot. Per esempio, la simulazione può aggiornare le forze ogni Δt , ma il controller può inviare comandi solo a intervalli più ampi. Questa separazione tra step fisico-simulativo e slot di comunicazione è importante. Il simulatore può avere frequenza interna alta, mentre la comunicazione lavora a frequenza più bassa. Non tutto deve avvenire allo stesso ritmo.

Si può distinguere:

simulation_step: aggiornamento matematico delle posizioni e forze;

communication_slot: finestra di invio/ricezione messaggi;

control_slot: finestra di decisione del controller;

telemetry_slot: finestra di report.

Questa distinzione rende l'architettura più pulita. Il simulatore può aggiornare continuamente il modello, ma comunicare snapshot a frequenza controllata. I nodi fisici possono inviare heartbeat a intervalli diversi dai comandi. Il controller può prendere decisioni a ogni fase, non a ogni micro-step.

Gli slot temporali possono avere durata diversa a seconda della funzione. Non tutti devono essere uguali. Uno slot di ack può durare 10–50 ms per nodo, in base alla rete. Uno slot di LED può durare 300 ms. Uno slot di allineamento può durare 1 secondo. Uno slot di stabilizzazione può durare più a lungo. Gli slot devono essere dimensionati rispetto al fenomeno che rappresentano. Immaginare un'unica durata per tutto sarebbe troppo rigido.

Una possibile classificazione è:

slot comunicativo breve: ack, ping, risposta status;

slot operativo medio: LED, cambio modalità, sincronizzazione fase;

slot fisico controllato: impulso magnetico, test PWM, attuazione breve;

slot comportamentale lungo: allineamento, aggregazione, stabilizzazione;

slot diagnostico variabile: raccolta telemetria e debug.

Questa classificazione aiuta a progettare sequenze realistiche. Le azioni fisiche e comportamentali hanno tempi diversi da quelle puramente comunicative.

Gli slot temporali devono essere visibili nella dashboard. Una dashboard avanzata potrebbe mostrare la timeline della run, indicando slot correnti e successivi: prepare, align, attach, stabilize, report. Potrebbe evidenziare se lo slot è in corso, completato, fallito o in timeout. Potrebbe mostrare quali nodi hanno risposto nel proprio slot e quali no. Questo rende il comportamento dello swarm molto più comprensibile. L'utente non vede solo nodi che cambiano stato, ma una sequenza temporale organizzata.

Una timeline dashboard può mostrare:

slot 0 — prepare — completed;

slot 1 — align — completed with warning;

slot 2 — attach — in progress;

slot 3 — stabilize — pending;

slot 4 — report — pending.

Sotto ogni slot, la dashboard può mostrare nodi partecipanti, ack ricevuti, errori, latenze e metriche. Questo sarebbe estremamente utile per presentare il progetto in modo tecnico e professionale.

Gli slot temporali possono essere registrati nei log. Ogni slot dovrebbe avere un ID, un tempo di inizio, un tempo di fine previsto, un tempo di fine reale, una fase, nodi coinvolti, risultati e anomalie. Questo rende le run analizzabili. Se una sequenza fallisce, si può capire in quale slot è avvenuto il problema. Non si analizza un flusso caotico di eventi, ma una struttura ordinata.

Un record slot può contenere:

slot_id;
pattern_run_id;
phase_name;
start_time_ms;
end_time_ms;
actual_end_time_ms;
expected_nodes;
responded_nodes;
missing_nodes;
late_nodes;
result;
notes.

Questo record è molto coerente con l'approccio documentale del progetto MicroBot. Permette di trasformare demo e test in dati analizzabili.

Gli slot temporali devono essere collegati anche alla gestione del ritardo. Se il controller sa che un nodo ha RTT alto o jitter elevato, può assegnargli slot più ampi o margini maggiori. Oppure può posizionarlo in uno slot meno critico. Per esempio, un nodo con rete instabile può essere escluso dallo slot magnetico critico ma incluso nello slot telemetria. In questo modo, la pianificazione temporale diventa adattiva.

Una strategia può essere:

nodi sync_good → slot stretti e azioni critiche;

nodi sync_ok → slot normali;

nodi sync_degraded → slot larghi o solo diagnostica;

nodi sync_lost → esclusi da sequenze temporizzate.

Questa logica lega slot temporali e qualità della sincronizzazione. Non tutti i nodi devono essere trattati allo stesso modo se la loro qualità temporale è diversa.

Gli slot temporali possono essere usati anche per evitare conflitti tra comandi locali. Un nodo può ricevere più comandi programmati. Se due comandi richiedono la stessa risorsa nello stesso slot, il firmware deve decidere cosa fare. Per esempio, non si dovrebbe attivare una bobina in due modalità diverse nello stesso intervallo. Non si dovrebbe eseguire un pattern LED diagnostico se il LED è già usato per segnalare safety. Gli slot aiutano a rilevare conflitti, perché ogni comando occupa una finestra temporale e una risorsa.

Una struttura di scheduling locale può includere:

resource_id;

start_time;

end_time;

command_id;

priority.

Se un nuovo comando occupa la stessa risorsa in un intervallo sovrapposto, il nodo confronta priorità e policy. Un comando safety può interrompere tutto. Un comando LED decorativo può essere rifiutato se il LED è usato per errore. Un comando coil non può sovrapporsi a un altro comando coil incompatibile. Questa logica trasforma il MicroBot in un piccolo scheduler embedded.

Gli slot temporali sono quindi collegati alla gestione delle risorse. Il tempo non è l'unica risorsa; ci sono anche LED, coil, comunicazione, batteria, termica, CPU. Uno slot definisce quando una risorsa viene usata. Lo scheduling impedisce sovrapposizioni non desiderate. Questo è particolarmente importante quando il nodo evolve da prototipo LED a modulo con più funzioni.

Un esempio:

slot 1000–1200 ms: LED usato per identify_node;

slot 1100–1300 ms: LED richiesto per pattern_status.

C'è sovrapposizione. Il nodo deve decidere se rifiutare il secondo comando, accodarlo, sostituire il primo o applicare priorità. Se identify_node è manuale e pattern_status è operativo, la policy può decidere. Senza uno scheduler temporale, questi conflitti diventano caotici.

Gli slot temporali possono essere usati anche per coordinare aggiornamento dei parametri. Se il controller modifica pesi di coesione o soglie di attacco durante una simulazione, può specificare che il nuovo parametro diventi attivo all'inizio di uno slot. Questo evita che alcuni nodi usino parametri vecchi e altri nuovi nello stesso intervallo. Il comando può essere ricevuto in anticipo ma applicato a T_update. Così il cambio di comportamento è coerente.

Un comando può essere:

set_behavior_parameters con apply_at_ms = T_next_phase

I nodi ricevono, validano e applicano solo quando il clock globale raggiunge T_next_phase. Questo è molto utile nelle sequenze multi-fase. Il sistema evita transizioni disordinate.

Gli slot temporali possono essere interpretati anche dal punto di vista matematico come discretizzazione del tempo. Nella Sezione 7, il modello discreto del moto aggiorna posizioni e stati a intervalli Δt . Gli slot temporali estendono questa idea al livello comunicativo e operativo. Il tempo continuo viene diviso in passi o finestre. Ogni finestra rappresenta un'unità di coordinamento. Questo rende il sistema più semplice da simulare, implementare e analizzare.

Nel modello, si può scrivere:

$$t_k = t_0 + k \cdot \Delta t$$

e ogni aggiornamento avviene a t_k . Negli slot operativi, invece, ogni finestra può contenere più aggiornamenti interni. Per esempio, durante lo slot align, il simulatore può fare molti step di movimento; durante lo slot report, i nodi inviano dati. Il concetto è lo stesso: organizzare il tempo in unità discrete.

Gli slot temporali devono essere progettati per non rendere il sistema troppo rigido. Se ogni azione deve avvenire in slot strettissimi e fissi, lo swarm può diventare fragile. Una rete reale ha variabilità; un sistema fisico ha ritardi; i nodi possono avere condizioni diverse. Per questo motivo, gli slot devono avere tolleranze. Alcune fasi possono essere event-driven: terminano quando la condizione è raggiunta, entro un massimo. Altre possono essere time-driven: durano un intervallo preciso. La combinazione è la più efficace.

Si può distinguere:

slot rigido: usato per azioni temporizzate precise;

slot flessibile: usato per fasi comportamentali con durata massima;

slot condizionale: termina quando si verifica una condizione;

slot di fallback: attivato se una fase fallisce o scade.

Questa classificazione permette di progettare sequenze robuste. Non tutto deve essere una griglia rigida; il tempo deve guidare, ma il sistema deve reagire agli eventi.

Un esempio di slot condizionale:

fase stabilizzazione inizia a T_3 ;

durata massima 2000 ms;

termina prima se `connection_stability_score` > soglia per 500 ms.

Questa logica è molto adatta a MicroBot. Una formazione non deve aspettare sempre esattamente due secondi se è già stabile, ma non deve nemmeno restare indefinitamente in attesa. Lo slot dà limite e struttura.

Gli slot temporali possono essere usati per implementare hysteresis temporale. Nelle connessioni, una condizione può dover persistere per un certo numero di slot o per una certa durata. Per esempio, un nodo può considerare una connessione stabile solo se resta valida per almeno tre slot di controllo consecutivi. Oppure un distacco può essere confermato solo se la condizione di distanza eccessiva persiste per due slot. Questo rende il sistema meno sensibile a rumore e oscillazioni.

Una regola può essere:

`attached_confirmed` se `condizione_attacco` vera per N slot consecutivi;

`detached_confirmed` se `condizione_distacco` vera per M slot consecutivi.

Questa forma è particolarmente semplice da implementare in un sistema a slot, perché si contano finestre discrete. In alternativa, si misura durata continua tramite timestamp. Entrambe le soluzioni sono coerenti; gli slot rendono la logica più organizzata.

Gli slot temporali possono anche supportare modalità di risparmio energetico. Un nodo non deve necessariamente trasmettere continuamente. Può svegliarsi o attivare alcune funzioni in slot definiti, soprattutto in versioni future a batteria. Per esempio, telemetria dettagliata ogni N slot, heartbeat leggero ogni M slot, radio sempre attiva nelle prime versioni ma ottimizzabile dopo. Anche se nelle prime fasi ESP32 resterà probabilmente con Wi-Fi attivo, la struttura a slot prepara ottimizzazioni future.

Una logica futura può essere:

slot ascolto;

slot esecuzione;

slot telemetria;

slot low-power.

Questo è più avanzato, ma coerente con l'evoluzione verso nodi energeticamente efficienti. La batteria è un vincolo importante e il tempo può aiutare a gestirla.

Gli slot temporali devono essere collegati alle modalità broadcast e unicast. Un broadcast può definire un piano di slot comune. Un unicast può assegnare uno slot specifico a un nodo. Per esempio, il controller può inviare in broadcast il pattern temporale generale e poi inviare unicast a MB-003 una correzione del suo slot o un target specifico. Questa combinazione permette flessibilità. Il broadcast dà struttura globale; l'unicast corregge dettagli individuali.

Una sequenza può essere:

broadcast slot_plan per pattern_run;

unicast role/slot override per nodi specifici;

broadcast start_at;

nodi eseguono secondo slot_plan;

ack distribuiti per slot.

Questa architettura è molto potente e scalabile. I nodi non ricevono tutto in tempo reale, ma un piano temporale da seguire.

Uno slot_plan può essere rappresentato in forma compatta:

```
[
{ "slot_id": "prepare", "start_ms": 0, "duration_ms": 500 },
{ "slot_id": "align", "start_ms": 500, "duration_ms": 1000 },
{ "slot_id": "attach", "start_ms": 1500, "duration_ms": 600 },
{ "slot_id": "stabilize", "start_ms": 2100, "duration_ms": 1500 }
]
```

Questo piano è relativo all'inizio della run. Il controller invia run_start_time_ms, e i nodi calcolano gli istanti assoluti. In una prima versione, questo piano può essere gestito solo dal controller e non inviato completo ai nodi. Ma dal punto di vista architetturale, lo slot_plan è una struttura molto utile.

Gli slot temporali devono essere compatibili con nodi che entrano tardi nella rete. Se un nodo si collega durante una run già avviata, potrebbe non sapere in quale slot si trova il sistema. Il controller deve decidere se integrarlo subito, tenerlo in idle o aspettare la prossima run. In generale, per sicurezza, un nodo che entra tardi non dovrebbe partecipare a una sequenza temporizzata già in corso, a meno che il controller non lo sincronizzi e lo assegni esplicitamente. Può restare in idle e inviare heartbeat.

Una regola:

nodi `late_join` non partecipano automaticamente agli slot correnti; richiedono sync e assegnazione esplicita.

Questo evita che un nodo appena connesso esegua comandi fuori contesto. La sincronizzazione e lo slot corrente devono essere noti prima di agire.

Gli slot temporali devono gestire anche nodi che saltano uno slot. Se un nodo perde un comando o non risponde nello slot previsto, il controller può marcarlo come `missed_slot`. A seconda del tipo di slot, può continuare o abortire. Se si tratta di uno slot di telemetria, il sistema può continuare. Se si tratta di uno slot di safety, il problema è più serio. Se si tratta di uno slot di attuazione magnetica coordinata, il controller può fermare la fase. Ogni slot deve avere criticità.

Una struttura slot può includere:

`criticality = low / medium / high / safety`

Se uno slot `low` fallisce, si registra warning. Se uno slot `high` fallisce, si può abortire la run. Se uno slot `safety` fallisce, si entra in modalità sicura. Questa classificazione rende la risposta agli errori più sistematica.

Gli slot temporali hanno anche un ruolo nella progettazione del protocollo di risposta. Un ack può indicare non solo che il comando è stato ricevuto, ma anche in quale slot è stato processato. Per esempio:

`ref_message_id;`

`slot_id;`

`result;`

`received_at;`

`executed_at;`

`slot_status.`

Questo permette di capire se il nodo ha rispettato lo slot. Un risultato può essere `executed_in_slot`, `executed_late`, `missed_slot`, `skipped_due_to_safety`, `rejected_expired`. Questi dettagli rendono la telemetria molto più utile.

Un ack può essere:

```
{  
  "ref_message_id": "CMD-BCAST-000120",  
  "slot_id": "attach",  
}
```

```
"result": "executed_in_slot",  
"executed_at_ms": 1520  
}
```

Oppure:

```
{  
  "ref_message_id": "CMD-BCAST-000120",  
  "slot_id": "attach",  
  "result": "rejected_expired",  
  "reason": "received_after_slot_end"  
}
```

Questo livello di dettaglio è fondamentale per capire se un problema è di rete, firmware, sincronizzazione o logica di pattern.

Gli slot temporali possono essere usati per coordinare la raccolta di snapshot. In simulazione o dashboard, il controller può decidere di acquisire snapshot dello stato globale a intervalli fissi, per esempio ogni 100 ms. Ogni snapshot appartiene a uno slot di osservazione. Questo rende i dati più confrontabili. Se la telemetria arriva da nodi diversi in tempi diversi, il controller può aggregarla in finestre temporali, costruendo uno stato approssimato dello swarm per ogni slot. Questa tecnica è utile quando i dati non arrivano perfettamente simultanei.

Uno snapshot slot-based può essere:

snapshot_slot_001: dati ricevuti tra T e $T + \Delta T$;

snapshot_slot_002: dati ricevuti tra $T + \Delta T$ e $T + 2\Delta T$.

Il controller può usare l'ultimo dato valido di ogni nodo entro la finestra. Questo permette di visualizzare lo swarm in modo ordinato anche se la telemetria è asincrona. È un concetto molto utile per dashboard e replay.

Gli slot temporali devono essere adattati alla scala dello swarm. Con pochi nodi, gli slot possono essere larghi e semplici. Con molti nodi, bisogna ridurre traffico, usare gruppi, aggregare risposte, aumentare efficienza. La scalabilità temporale è parte della scalabilità generale. Non basta aggiungere nodi; bisogna organizzare il tempo in modo da permettere a più nodi di comunicare e agire senza saturare il sistema.

Per piccoli swarm:

slot individuali semplici;

ack da tutti;

telemetria frequente;

debug dettagliato.

Per swarm medi:

slot per gruppo;

ack selettivi;

telemetria normale;

debug mirato.

Per swarm grandi:

slot aggregati;

telemetria compressa;

eventi prioritari;

campionamento;

controllo per ruoli o cluster.

Questa evoluzione mostra come gli slot temporali possano crescere con il sistema. La prima implementazione può essere semplice ma deve avere una direzione chiara.

Dal punto di vista della documentazione, gli slot temporali dovrebbero essere descritti nei file di configurazione dei pattern e nei log delle run. Ogni pattern può avere una sequenza di slot. Ogni run può registrare l'esito degli slot. Ogni errore può essere associato a uno slot. Questa struttura rende la documentazione molto più precisa. Invece di dire "durante la demo alcuni nodi non hanno risposto", si può dire "durante lo slot attach della run line_run_001, MB-004 e MB-006 non hanno inviato ack entro la finestra prevista". Questa è una differenza enorme in termini di rigore.

Una documentazione di test può riportare:

slot prepare completato con 6/6 nodi;

slot align completato con 5/6 nodi, MB-004 late;

slot attach abortito per safety su MB-002;

slot report completato con telemetria parziale.

Questa forma rende il progetto molto più professionale e analizzabile.

Gli slot temporali possono anche essere integrati nella narrativa del sistema MicroBot come forma temporale della materia programmabile. Se la materia programmabile è una struttura che cambia forma nel tempo, gli slot sono le unità temporali con cui la trasformazione viene orchestrata. La forma non appare tutta insieme; emerge attraverso fasi. Ogni slot rappresenta un momento del processo: preparare, avvicinare, orientare, agganciare, stabilizzare, verificare, riconfigurare. Il tempo diventa parte della programmazione della forma.

Questa idea è molto forte dal punto di vista teorico. MicroBot non programma solo posizioni e connessioni, ma anche sequenze. La forma finale dipende dal percorso temporale. Due sistemi possono avere lo stesso target finale ma arrivarci con sequenze diverse, producendo stabilità diversa. Gli slot temporali permettono di controllare questo percorso. Programmare lo swarm significa quindi definire regole spaziali, relazioni e tempi.

Dal punto di vista pratico, nelle prime versioni del progetto si può iniziare con slot molto semplici. Per esempio, dopo un broadcast, ogni nodo risponde con un ritardo basato sul proprio ID. Questo è già uno slot di risposta. Poi si può introdurre execute_at per LED sincronizzati. Poi si può

introdurre slot di pattern: prepare, align, report. Poi slot magnetici controllati. Questa progressione evita di costruire subito un sistema complesso, ma mantiene coerenza con l'architettura finale.

Una roadmap degli slot può essere:

v0.1: ack delay basato su node_id;

v0.2: execute_at per comandi visuali;

v0.3: finestre di risposta dopo broadcast;

v0.4: slot di fase per pattern simulati;

v0.5: slot per telemetria distribuita;

v0.6: slot per attuazione magnetica sicura;

v0.7: slot adattivi basati su sync_quality e rete.

Questa roadmap è coerente con l'evoluzione del sistema e con la logica di sviluppo incrementale già usata in MicroBot.

Gli slot temporali devono infine essere considerati come strumento di controllo, non come vincolo assoluto. Il sistema deve poter deviare dagli slot quando la sicurezza lo richiede, quando un evento anticipa la fase successiva, quando un nodo fallisce o quando il controller decide di abortire. Gli slot danno ordine, ma non devono impedire adattamento. Uno swarm intelligente deve essere temporizzato ma anche reattivo.

In conclusione, gli slot temporali costituiscono il meccanismo attraverso cui il clock globale MicroBot viene trasformato in finestre operative utilizzabili. Essi permettono di distribuire ack, evitare congestione, coordinare fasi, organizzare pattern, pianificare attuazioni, raccogliere telemetria, gestire risorse, ridurre conflitti e rendere le run analizzabili. Gli slot possono essere assegnati per nodo, gruppo, ruolo o fase; possono essere statici, dinamici, rigidi, flessibili o condizionali; possono servire alla comunicazione, alla simulazione, alla safety e al comportamento fisico. Il loro valore è trasformare il tempo condiviso in una struttura eseguibile. Il punto successivo, dedicato alla coordinazione delle azioni, utilizzerà gli slot temporali e il clock globale per descrivere come più MicroBot possano eseguire comportamenti distribuiti in modo coerente, evitando conflitti e costruendo sequenze collettive controllate.

9.4 Coordinazione delle azioni

La coordinazione delle azioni rappresenta il livello in cui il clock globale, gli slot temporali, i comandi, gli identificatori e lo stato dei nodi vengono combinati per produrre comportamenti collettivi coerenti. Dopo aver definito il controller come riferimento temporale, il clock globale del sistema e gli slot temporali, diventa necessario spiegare come questi strumenti vengano usati concretamente per far agire più MicroBot come parti di uno stesso sistema. La coordinazione non consiste semplicemente nell'inviare lo stesso comando a tutti i nodi, ma nel fare in modo che ogni MicroBot esegua l'azione corretta, nel momento corretto, con il ruolo corretto, rispettando vincoli di sicurezza, stato locale, capacità hardware, comunicazione e obiettivo globale dello swarm.

Nel progetto MicroBot, la coordinazione delle azioni è fondamentale perché il sistema non è composto da attuatori passivi comandati individualmente, ma da nodi distribuiti che possono avere identità, stato, ruolo, capacità e condizioni operative diverse. Alcuni nodi possono essere leader,

altri follower, altri connector, altri boundary, altri ancora possono essere in idle, low energy, diagnostic o safety. Coordinare lo swarm significa quindi orchestrare un insieme eterogeneo di agenti, non soltanto inviare un impulso generico a una massa uniforme. Questa distinzione è importante perché rende MicroBot più vicino a un sistema multi-agente reale che a una semplice sequenza di LED o motori comandati in parallelo.

La coordinazione delle azioni può essere definita come il processo attraverso cui il controller assegna o comunica una sequenza temporale di comportamenti, e ogni nodo la interpreta localmente in base al proprio stato. In forma generale, si può rappresentare così:

$$\text{azione_i}(t) = \text{funzione}(\text{comando_globale}, \text{ruolo_i}, \text{stato_i}(t), \text{slot_corrente}, \text{vincoli_i}, \text{safety_i})$$

Questa formulazione è molto importante perché mostra che l'azione del nodo i-esimo non dipende soltanto dal comando ricevuto. Dipende anche dal ruolo del nodo, dal suo stato corrente, dallo slot temporale in cui si trova il sistema, dai vincoli hardware e dal safety layer. Due MicroBot possono ricevere lo stesso comando globale ma produrre azioni diverse. Questo non è un errore: è il comportamento corretto di un sistema distribuito organizzato.

Per esempio, durante una fase di allineamento, il controller può inviare un comando broadcast che indica:

phase = "align"

Tutti i nodi ricevono la stessa fase, ma un nodo leader può orientarsi verso un asse globale, un follower può orientarsi verso il vicino principale, un connector può preparare una porta magnetica, un boundary può mantenere distanza e stabilità. Il comando globale fornisce il contesto comune; il ruolo locale determina l'azione specifica. Questa logica permette di ridurre il traffico di rete, perché il controller non deve necessariamente inviare a ogni nodo ogni dettaglio operativo. Può inviare una fase comune e lasciare che le regole locali facciano il resto.

La coordinazione può avvenire su tre livelli principali: coordinazione di stato, coordinazione temporale e coordinazione fisica. La coordinazione di stato serve a garantire che i nodi si trovino nella modalità corretta prima di iniziare una fase. La coordinazione temporale serve a garantire che le azioni avvengano negli slot o negli istanti previsti. La coordinazione fisica serve a garantire che il risultato delle azioni sia compatibile con il comportamento dello swarm, evitando collisioni, sovrapposizioni, agganci errati, attuazioni simultanee pericolose o instabilità. Questi tre livelli devono lavorare insieme.

La coordinazione di stato è il primo livello. Prima di avviare un pattern o una sequenza, il controller deve sapere quali nodi sono online, quali sono sincronizzati, quali hanno le capabilities richieste, quali sono in safety, quali hanno batteria sufficiente e quali appartengono al gruppo operativo. Avviare una sequenza senza questa verifica può generare comportamenti incoerenti. Per esempio, se un nodo non è sincronizzato e riceve un comando con execute_at, potrebbe non eseguirlo correttamente. Se un nodo è in low_energy, potrebbe rifiutare comandi magnetici. Se un nodo non supporta coil_pwm, non deve partecipare a una fase attuativa. La coordinazione inizia quindi con la readiness.

Una condizione di readiness può essere espressa come:

$$\text{ready_i} = \text{online_i} \wedge \text{sync_valid_i} \wedge \text{safety_ok_i} \wedge \text{capability_ok_i} \wedge \text{role_assigned_i}$$

Il nodo i può partecipare alla sequenza soltanto se questa condizione è vera. In pratica, il controller deve costruire una lista dei nodi disponibili e compatibili. Se uno o più nodi non sono pronti, il

sistema può scegliere tra più strategie: abortire la run, eseguire una versione ridotta del pattern, sostituire il ruolo con un altro nodo, mettere il nodo problematico in idle o passare a modalità diagnostica. Questa scelta dipende dal tipo di pattern e dalla criticità dei nodi mancanti.

La coordinazione temporale è il secondo livello. Una volta scelti i nodi partecipanti, il controller deve definire quando ogni azione deve avvenire. Questo può essere fatto tramite `execute_at`, slot temporali, `phase_start`, `phase_end` e `duration_ms`. Ogni comando operativo deve essere collocato in una finestra temporale chiara. Se l'azione è globale, può essere inviata in broadcast con un `execute_at` comune. Se l'azione è individuale, può essere inviata in unicast con lo stesso `execute_at` o con slot differenziati. Il punto fondamentale è che il momento di invio non deve essere confuso con il momento di esecuzione.

Per esempio, il controller può inviare comandi individuali ai nodi tra $T = 1000$ ms e $T = 1300$ ms, ma programmare tutti per eseguire a $T = 2000$ ms. Questo concede tempo alla rete per consegnare i pacchetti e ai nodi per validarli. La coordinazione temporale si basa quindi su una pianificazione anticipata. Un sistema che invia comandi all'ultimo istante è fragile; un sistema che invia comandi con margine e usa `execute_at` è molto più robusto. La latenza non viene eliminata, ma viene assorbita dalla pianificazione.

La coordinazione fisica è il terzo livello. Anche se tutti i nodi sono pronti e sincronizzati, il sistema deve garantire che le azioni fisiche siano compatibili. Se più MicroBot si muovono o simulano movimento verso lo stesso punto, la separazione deve impedire collisioni. Se più bobine vengono attivate, i limiti energetici e termici devono essere rispettati. Se più connessioni magnetiche vengono tentate contemporaneamente, bisogna evitare agganci indesiderati. Se una struttura deve stabilizzarsi, non bisogna cambiare fase troppo presto. La coordinazione fisica richiede quindi feedback, safety e criteri di completamento.

Una sequenza coordinata non dovrebbe essere solo temporale, ma anche condizionale. Per esempio, il controller può dire: “passa dalla fase align alla fase attach solo se almeno una certa percentuale di nodi ha segnalato `alignment_ok`”. Oppure: “passa da attach a stabilize solo se le connessioni previste sono state formate o se il tempo massimo è scaduto”. Questo evita di procedere meccanicamente anche quando lo stato fisico non è pronto. La coordinazione delle azioni deve quindi combinare time-driven control e event-driven control.

Un modello generale di fase può essere:

```
fase = {inizio, durata_massima, azioni, condizioni_successo, condizioni_fallimento, fase_successiva}
```

Questa struttura permette di organizzare il comportamento dello swarm come una macchina a stati temporizzata. Ogni fase ha un tempo massimo, ma può terminare prima se la condizione di successo è raggiunta. Può anche abortire se si verifica una condizione di fallimento, come `safety_triggered`, nodo critico offline, `thermal_lock` o troppi comandi scaduti. Questa logica è molto più robusta di una sequenza fissa senza feedback.

Nel caso di una formazione lineare, una coordinazione possibile può essere descritta così. Prima il controller esegue discovery e verifica i nodi disponibili. Poi assegna ruoli: un leader, alcuni follower e forse due nodi terminali. Successivamente invia un comando di preparazione con `execute_at` comune. Nella fase align, i nodi orientano o simulano il proprio orientamento secondo la linea. Nella fase spacing, i nodi regolano distanza e separazione. Nella fase attach, vengono abilitate connessioni o stati magnetici compatibili. Nella fase stabilize, il sistema riduce movimenti e verifica connessioni. Nella fase report, i nodi inviano telemetria e il controller valuta il risultato. Questa non è una semplice animazione: è una sequenza coordinata di stati.

In una formazione circolare, la coordinazione cambia. I nodi non devono allinearsi tutti lungo lo stesso asse, ma distribuirsi intorno a una curvatura. Ogni nodo può avere un target angolare o una posizione relativa. I ruoli possono essere meno gerarchici e più geometrici. Alcuni nodi possono essere assegnati a settori del cerchio. La fase di separazione è fondamentale per evitare che più nodi occupino la stessa zona. La fase di allineamento può essere radiale o tangenziale. La fase di stabilizzazione deve verificare continuità della forma. Questo dimostra che la coordinazione dipende dal pattern, non è identica per tutte le configurazioni.

La coordinazione delle azioni deve tenere conto del fatto che i nodi possono avere capacità diverse. Un nodo LED-only può partecipare a una demo visuale ma non a una fase magnetica. Un nodo simulato può avere posizione virtuale ma non attuazione fisica. Un nodo con coil può eseguire magnetic_pulse. Un nodo in low_energy può essere escluso da compiti attivi. Il controller deve quindi costruire una mappa delle capabilities e assegnare azioni compatibili. Questo è essenziale per evitare comandi inutili o rifiutati.

Una mappa di coordinazione può contenere:

node_id;
role;
capabilities;
current_state;
sync_status;
assigned_action;
assigned_slot;
fallback_action.

Questa struttura permette al controller di sapere cosa ogni MicroBot deve fare, quando deve farlo e cosa fare se non può farlo. Il fallback è importante: se un nodo non può eseguire l'azione principale, può entrare in idle, segnalare errore, mantenere stato passivo o essere sostituito da un altro nodo. Senza fallback, la coordinazione sarebbe fragile.

La coordinazione può essere centralizzata, distribuita o ibrida. Nella coordinazione centralizzata, il controller calcola tutto e invia comandi specifici ai nodi. Questo è più semplice da controllare e debug, ma può scalare peggio. Nella coordinazione distribuita, i nodi ricevono regole generali e decidono localmente in base ai vicini. Questo è più scalabile e più vicino allo swarm robotics, ma più difficile da prevedere. Nel progetto MicroBot, la strategia più coerente è ibrida: il controller definisce fasi, ruoli, parametri e obiettivi; i nodi applicano localmente validazione, safety, timer e regole compatibili con il proprio stato.

Questa architettura ibrida può essere rappresentata così:

controller = pianificazione globale, timebase, pattern, fasi, logica di alto livello;

nodi = esecuzione locale, safety, validazione, timer, feedback, micro-decisioni;

dashboard = osservazione, comando umano, diagnosi, timeline;

simulatore = validazione del comportamento, replay, test di scenari.

La coordinazione delle azioni nasce dall'interazione tra questi componenti. Nessuno dei livelli, da solo, è sufficiente. Il controller senza nodi intelligenti diventa fragile; i nodi senza controller non hanno direzione globale; la dashboard senza telemetria non osserva; il simulatore senza protocollo comune resta separato dall'hardware. La coordinazione è il punto in cui l'ecosistema si unifica.

Un elemento fondamentale della coordinazione è la conferma dell'azione. In un sistema distribuito, il controller non deve assumere che un comando sia stato eseguito solo perché è stato inviato. Deve ricevere ack, event o telemetry. Per ogni azione coordinata, può essere necessario distinguere tra `command_sent`, `command_received`, `command_accepted`, `command_scheduled`, `command_executed` e `action_verified`. Questi stati rappresentano livelli diversi di certezza.

Per esempio:

`command_sent` significa che il controller ha inviato il pacchetto;

`command_received` significa che il nodo lo ha ricevuto;

`command_accepted` significa che il nodo lo ha validato;

`command_scheduled` significa che lo ha messo in coda per `execute_at`;

`command_executed` significa che ha eseguito l'azione;

`action_verified` significa che il risultato è stato osservato o confermato da telemetria.

Questa distinzione è molto importante. Un nodo può accettare un comando ma non eseguirlo perché entra in safety prima di `execute_at`. Può eseguirlo ma il risultato fisico può non essere quello previsto. Può inviare ack ma poi perdere connessione. La coordinazione robusta non si ferma al primo livello di conferma.

Nel caso di azioni visuali, `action_verified` può essere meno critica. Se un LED lampeggia e il nodo invia `executed`, può bastare. Nel caso di azioni magnetiche, invece, `executed` non garantisce che l'aggancio sia avvenuto. Serve un evento `attach_event`, una misura di stato, una telemetria o un feedback esterno. Questo è fondamentale: attivare una bobina non equivale a formare una connessione. Il comando controlla un'azione; il risultato fisico deve essere verificato separatamente quando possibile.

La coordinazione delle azioni deve quindi distinguere tra comando e obiettivo. Un comando è "attiva questa uscita per questa durata". Un obiettivo è "forma questa connessione" oppure "raggiungi questa formazione". Il controller può inviare comandi, ma deve valutare se l'obiettivo è stato raggiunto. Questa distinzione evita una documentazione troppo ingenua. Nel sistema MicroBot, il comportamento finale non è garantito da un singolo messaggio; emerge dalla combinazione di comandi, fisica, feedback e regole.

Una coordinazione avanzata può essere descritta come ciclo chiuso:

pianificazione → invio comandi → esecuzione locale → feedback → valutazione → correzione

Questo ciclo è molto più potente di un controllo open-loop, dove il controller invia comandi senza osservare il risultato. Nelle prime versioni, il sistema può essere parzialmente open-loop, soprattutto con LED e simulazione. Ma la direzione corretta è il closed-loop: ogni fase produce feedback e il

controller decide se continuare, correggere o fermare. La telemetria e gli eventi sono quindi parte integrante della coordinazione.

Un esempio di coordinazione closed-loop può essere:

il controller invia `phase_align`;

i nodi rispondono `alignment_progress`;

il controller aspetta `alignment_ok` da almeno N nodi;

se la condizione è raggiunta, invia `phase_attach`;

se scade il tempo, invia `correction` o `abort`.

Questa sequenza mostra come il tempo e gli eventi lavorino insieme. Non si procede solo perché il timer è finito; si procede perché lo stato ha raggiunto la condizione richiesta o perché è scaduto un limite massimo.

La coordinazione delle azioni deve anche gestire conflitti tra obiettivi. Un nodo può ricevere un comando di pattern e contemporaneamente trovarsi vicino a una violazione di separazione. Può essere assegnato a una fase di attuazione ma avere `thermal_lock`. Può dover inviare telemetria ma essere nello slot di esecuzione. Può dover mantenere un aggancio ma ricevere un comando di distacco. Il sistema deve definire priorità. In generale, safety e vincoli fisici devono prevalere su pattern e obiettivi estetici.

Una gerarchia utile è:

emergenza e safety;

protezione energetica e termica;

validità temporale del comando;

vincoli fisici locali;

mantenimento connessioni critiche;

azione di fase corrente;

telemetria e diagnostica;

azioni visuali secondarie.

Questa gerarchia permette al nodo di risolvere conflitti localmente. Il controller può pianificare, ma il nodo deve poter rifiutare o ritardare un'azione se viola priorità superiori. Questo è essenziale per evitare che la coordinazione globale produca danni locali.

La coordinazione deve prevedere anche il caso di nodi mancanti. In uno swarm reale, alcuni nodi possono essere offline, non sincronizzati, in errore o non disponibili. Il controller deve decidere se il pattern può funzionare comunque. Alcune formazioni richiedono tutti i nodi; altre possono adattarsi. Per esempio, una linea di cinque nodi può diventare una linea di quattro se uno manca; una struttura con un nodo ponte critico potrebbe non essere possibile senza quel nodo. La coordinazione deve quindi conoscere la dipendenza del pattern dai nodi.

Un pattern può dichiarare:

minimum_nodes;

required_roles;

optional_roles;

critical_nodes;

degradable = true/false.

Se degradable è true, il controller può adattare la formazione. Se è false, abortisce se manca un nodo critico. Questo rende il sistema più flessibile e rigoroso. Non tutti i pattern hanno la stessa tolleranza.

La coordinazione può essere gestita attraverso un piano di azione. Un action_plan è una struttura che descrive le fasi, gli slot, i nodi coinvolti, i comandi, le condizioni e i fallback. Questo piano può essere generato dal controller prima della run e poi eseguito progressivamente. Nelle prime versioni può essere implicito nel codice; nelle versioni più mature può essere definito in un file JSON o YAML. Questa sarebbe una evoluzione molto coerente con l'organizzazione dei file MicroBot.

Un action_plan può contenere:

plan_id;

pattern_id;

pattern_run_id;

timebase;

participants;

roles;

phases;

commands;

slot_plan;

success_conditions;

failure_conditions;

fallbacks.

Questa struttura trasforma la coordinazione in un oggetto documentabile e ripetibile. Il sistema non agisce in modo improvvisato; segue un piano. Dopo la run, il controller può confrontare action_plan e actual_run_log. Questo permette analisi tecnica molto precisa.

Una fase del piano potrebbe essere:

phase_id = "align";

```
start_offset_ms = 500;

max_duration_ms = 1500;

actions = "set alignment mode for participants";

success_condition = "alignment_ok_count >= 5";

failure_condition = "safety_count > 0 or timeout";

on_success = "attach";

on_failure = "abort_or_retry".
```

Questa forma è molto vicina a una macchina a stati temporizzata. È una base forte per evolvere il sistema verso controllo più professionale.

La coordinazione delle azioni deve essere coerente con la simulazione. Prima di applicare una sequenza a nodi fisici, il controller può testarla su nodi simulati. La stessa action_plan può essere eseguita nel simulation_engine, producendo log, metriche e risultati virtuali. Se la sequenza è instabile in simulazione, non ha senso provarla subito sull'hardware. Se funziona in simulazione, può essere trasferita gradualmente ai nodi fisici, iniziando da funzioni sicure come LED e stati. Questo crea un ponte tra teoria, simulazione e hardware.

Una procedura corretta può essere:

```
test action_plan in simulazione pura;

test con nodi simulati e protocollo reale;

test con un nodo fisico LED-only;

test con più nodi fisici senza attuazione magnetica;

test con attuazione singola in unicast;

test con coordinazione fisica controllata.
```

Questa progressione riduce rischio e mantiene coerenza. La coordinazione non deve essere introdotta direttamente sul comportamento più pericoloso; deve essere validata a livelli crescenti.

La coordinazione delle azioni deve inoltre essere osservabile. Ogni fase deve generare log. Ogni comando deve avere message_id. Ogni ack deve indicare result. Ogni evento deve essere timestampato. Ogni slot deve avere stato. La dashboard deve poter mostrare cosa è in corso: fase corrente, nodi partecipanti, nodi pronti, nodi mancanti, ritardi, errori, condizioni di successo. Senza osservabilità, una sequenza coordinata può fallire senza spiegazione. Con osservabilità, il sistema diventa debuggabile.

Un pannello di coordinazione potrebbe mostrare:

```
pattern_run_id;

fase corrente;

tempo trascorso;
```

slot corrente;
nodi attesi;
nodi pronti;
nodi in ritardo;
nodi in safety;
condizione di successo;
prossima fase;
eventi recenti.

Questa visualizzazione sarebbe estremamente utile per presentare MicroBot in modo tecnico. Mostra che il sistema non è una demo casuale, ma un runtime coordinato con stato e timeline.

La coordinazione delle azioni deve considerare anche la frequenza di aggiornamento. Non tutte le decisioni devono essere prese a ogni millisecondo. Il controller può operare a livello di fasi e slot, mentre i nodi gestiscono timer e safety localmente. Questa separazione riduce carico e rende il sistema più stabile. Se il controller tentasse di controllare ogni micro-azione in tempo reale su Wi-Fi, il sistema diventerebbe fragile. È meglio far gestire al nodo le azioni temporizzate brevi e al controller le decisioni di alto livello.

Questa separazione può essere espressa così:

azioni rapide e safety → gestione locale del nodo;
sequenze e fasi → gestione del controller;
visualizzazione e diagnosi → dashboard;
validazione dinamica → simulatore e log.

Per esempio, un impulso di 100 ms deve essere temporizzato dal nodo, non dal controller. Il controller decide che l'impulso deve avvenire a T; il nodo lo esegue e lo termina localmente. Il controller non deve inviare “accendi” e poi “spegni” sperando che il secondo pacchetto arrivi in tempo. Questo è un principio fondamentale di coordinazione sicura.

La coordinazione deve distinguere tra azioni simultanee e azioni sequenziali. Alcune azioni devono avvenire contemporaneamente: lampeggio LED, cambio fase, inizio pattern, sincronizzazione. Altre devono essere sequenziali: risposte ack, attuazioni magnetiche per gruppi, diagnostica di nodi, calibrazione, raccolta dati dettagliata. La scelta dipende da traffico, sicurezza, fisica e obiettivo. Un sistema maturo sa quando sincronizzare e quando distribuire nel tempo.

Una tabella concettuale può essere:

azioni simultanee: sync flash, enter phase, start run, emergency stop;
azioni slot-based: ack, telemetry report, group activation;
azioni unicast sequenziali: calibration, debug, node-specific tests;

azioni condizionali: attach confirmation, stabilization, recovery.

Questa classificazione aiuta a progettare il runtime. Non tutto deve essere broadcast e simultaneo; non tutto deve essere unicast e sequenziale. Ogni azione ha la propria natura.

La coordinazione delle azioni deve gestire anche la comunicazione di ritorno. Dopo una fase, i nodi devono comunicare risultato. Se tutti inviano telemetria completa, la rete può congestionarsi. Il controller può richiedere solo eventi significativi o telemetria sintetica. Per esempio, dopo una fase align, ogni nodo può inviare alignment_ok true/false e un errore angolare stimato. Dopo attach, può inviare connection_state. Dopo stabilize, può inviare stability_score. Questo rende il feedback mirato.

Un feedback di fase può essere:

phase_id;

node_id;

result;

metric;

reason;

timestamp.

Per esempio:

phase_id = "align";

result = "ok";

metric = alignment_error_deg: 4.2.

Oppure:

phase_id = "attach";

result = "failed";

reason = "distance_threshold_not_met".

Questi messaggi permettono al controller di decidere la fase successiva. La coordinazione non è cieca; è guidata da feedback.

La coordinazione deve prevedere recupero dagli errori. Se un nodo fallisce una fase, il controller può ritentare, escludere il nodo, modificare il pattern, aumentare durata dello slot, inviare un comando correttivo o abortire. La scelta dipende dal tipo di errore. Un timeout comunicativo può richiedere retry o esclusione. Un errore di safety richiede stop o isolamento. Un errore di allineamento può richiedere una fase di correzione. Un errore di capability indica configurazione sbagliata. Ogni errore deve avere una policy.

Una matrice semplice può essere:

timeout ack → retry limitato;

command_expired → ripianifica con execute_at futuro;

safety_rejected → escludi nodo o abortisci fase;

low_energy → riduci ruolo o pattern;

unsupported_command → correggi capabilities o non usare nodo;

alignment_failed → estendi fase o invia correction;

connection_failed → ritenta attach o passa a fallback.

Questa matrice rende il comportamento del controller prevedibile. Senza policy, ogni errore richiederebbe intervento manuale.

La coordinazione delle azioni deve anche evitare comandi contraddittori. Il controller non dovrebbe inviare a un nodo due comandi incompatibili nello stesso intervallo. Per esempio, non deve inviare set_mode active e enter_safety nello stesso slot, salvo che enter_safety abbia priorità superiore e annulli il primo. Non deve assegnare due ruoli incompatibili nello stesso pattern. Non deve attivare e disattivare la stessa risorsa senza ordine chiaro. Questo richiede validazione del piano prima dell'invio.

Il controller può eseguire un controllo di coerenza dell'action_plan:

nessun nodo ha due ruoli esclusivi contemporanei;

nessuna risorsa è usata da due comandi incompatibili nello stesso slot;

tutti i comandi hanno capabilities richieste;

tutti i comandi temporizzati hanno execute_at valido;

tutte le fasi hanno timeout;

tutti i fallback critici sono definiti.

Questo controllo evita errori prima che arrivino ai nodi. La safety locale resta comunque attiva, ma è meglio prevenire a livello di piano.

La coordinazione deve considerare anche i gruppi. In uno swarm più grande, il controller può coordinare cluster invece di singoli nodi. Un gruppo può ricevere una fase comune, mentre i nodi interni applicano regole locali. Questo riduce complessità. Per esempio, cluster_A può entrare in align mentre cluster_B resta idle; poi cluster_B entra in attach; poi entrambi stabilizzano. Questa coordinazione gerarchica è una naturale evoluzione del sistema.

Una struttura gerarchica può essere:

swarm;

cluster;

nodo;

risorsa locale.

Il controller può inviare comandi a livello swarm, cluster o nodo. Le azioni locali vengono gestite sotto. Questo rende lo swarm scalabile. Nei primi prototipi, il livello cluster può essere simulato; in futuro diventerà importante.

La coordinazione delle azioni è anche collegata alla topologia del grafo. Se due nodi devono agganciarsi, devono essere coordinati come coppia o relazione. Un comando di connessione non riguarda solo un nodo; riguarda almeno due nodi e il loro stato relativo. Il controller deve quindi coordinare coppie, archi o sottografi. Per esempio, per creare una connessione tra MB-001 e MB-002, entrambi devono essere disponibili, compatibili, sincronizzati, vicini e nello stato corretto. Se uno dei due è in safety, la connessione non deve essere tentata.

Una relazione coordinata può essere descritta così:

```
connection_request = {node_a, node_b, target_state, execute_at, constraints}
```

Il controller invia comandi ai due nodi o al gruppo, e verifica l'esito. La connessione viene considerata riuscita solo se entrambi confermano o se il sistema osserva un attach_event. Questo è più rigoroso di un comando unilaterale.

Nel modello del grafo, la coordinazione delle azioni può essere vista come controllo degli archi. I nodi sono agenti; le azioni coordinano la creazione, il mantenimento o la rimozione di archi tra essi. Una formazione non è solo un insieme di posizioni, ma un grafo di relazioni attive. Coordinare una formazione significa coordinare quali archi devono esistere, quando devono essere creati, quanto devono durare e quando devono essere rimossi. Questo collega direttamente la Sezione 9 alla Sezione 7.

Per esempio, una linea di quattro nodi può richiedere:

CONN MB-001 — MB-002;

CONN MB-002 — MB-003;

CONN MB-003 — MB-004;

nessuna connessione MB-001 — MB-004.

La coordinazione deve favorire gli archi desiderati e impedire quelli indesiderati. Questo può richiedere timing, orientamento, separazione e comandi selettivi. Non basta attrarre tutto verso tutto. La coordinazione è ciò che trasforma attrazione e vicinanza in struttura specifica.

La coordinazione delle azioni deve gestire anche riconfigurazione. Passare da una linea a un cerchio, o da un cluster a una catena, richiede rompere alcune connessioni e crearne altre. Questo deve avvenire in modo ordinato. Se tutte le connessioni vengono sciolte insieme, lo swarm può disperdersi. Se tutte le nuove connessioni vengono tentate insieme, possono formarsi agganci errati. Una riconfigurazione efficace usa fasi: stabilizzare, selezionare connessioni da rilasciare, rilasciare, riallineare, avvicinare, agganciare, verificare. Ogni fase richiede coordinazione.

Una sequenza di riconfigurazione può essere:

phase_freeze: ridurre movimento e registrare stato;

phase_release_selected: disattivare o indebolire connessioni specifiche;

phase_separate: applicare separazione controllata;

phase_reassign_roles: assegnare nuovi ruoli;

phase_align_new_pattern: orientare verso nuova forma;

phase_attach_new_edges: creare nuove connessioni;

phase_stabilize: confermare struttura finale.

Questa sequenza è molto coerente con il concetto di materia programmabile. La forma cambia attraverso una grammatica temporale di azioni. La coordinazione è il linguaggio operativo di questa trasformazione.

Dal punto di vista energetico, la coordinazione può ridurre consumo. Se il sistema attiva solo ciò che serve, quando serve, evita sprechi. Se distribuisce attuazioni in slot, riduce picchi. Se verifica condizioni prima di attivare bobine, evita impulsi inutili. Se usa magneti permanenti o coesione passiva dove possibile, riduce carico attivo. La coordinazione non è solo ordine comportamentale; è anche gestione efficiente dell'energia.

Una strategia energetica coordinata può essere:

prima usare allineamento passivo o simulativo;

attivare coil solo se distanza e orientamento sono nella finestra utile;

limitare durata;

verificare risultato;

non ritentare indefinitamente;

applicare cooldown.

Questa logica protegge batteria e componenti. Un sistema non coordinato potrebbe attivare attuatori in momenti sbagliati e consumare senza produrre risultato.

La coordinazione deve considerare anche il carico termico. Se un nodo ha già usato la coil in una fase precedente, potrebbe dover saltare una fase successiva o avere duty cycle ridotto. Il controller può ricevere thermal_load e pianificare di conseguenza. Per esempio, può alternare nodi o gruppi per permettere raffreddamento. Gli slot temporali rendono questa alternanza possibile. La safety locale blocca comandi pericolosi, ma il controller può coordinare per evitare di arrivare al blocco.

Una logica può essere:

se thermal_load_i alto, assegnare nodo i a slot passivo;

se thermal_load_i basso, nodo può partecipare ad attuazione;

se molti nodi sono thermal_high, prolungare fase di cooldown.

Questo è un esempio di coordinazione adattiva basata su telemetria. Il sistema non segue solo un piano fisso; lo modifica in base allo stato reale.

La coordinazione delle azioni deve distinguere tra demo, simulazione e hardware reale. In una demo visuale, la coordinazione può essere più permissiva. Se un LED lampeggia in ritardo, è un problema

estetico o di comunicazione, ma non fisicamente pericoloso. In simulazione, si possono provare sequenze rischiose senza danno reale. In hardware con coil e batterie, invece, la coordinazione deve essere più prudente. La stessa architettura può supportare tutte le modalità, ma le policy devono cambiare.

Una modalità demo può permettere:

retry più semplici;

LED pattern rapidi;

tolleranza maggiore agli errori.

Una modalità hardware_safe può richiedere:

readiness completa;

sync valida;

limiti stretti;

ack obbligatori;

cooldown;

abort automatico su safety.

Questa distinzione è importante per non trattare prototipi fisici come se fossero simulazione. La documentazione deve sempre mantenere chiaro il livello reale dell'azione.

La coordinazione delle azioni è anche ciò che permette di presentare MicroBot come ecosistema, non come insieme di esperimenti separati. Quando comunicazione, tempo, simulazione, dashboard, firmware e safety lavorano insieme, il progetto assume forma di sistema. Il controller non invia comandi casuali; esegue piani. I nodi non lampeggiano soltanto; partecipano a fasi. La dashboard non mostra dati isolati; mostra una run. La simulazione non è un giocattolo; è ambiente di validazione. La coordinazione è quindi il punto in cui tutti i file, moduli e prototipi convergono in una logica comune.

Dal punto di vista della documentazione tecnica, questa sezione è molto importante perché dimostra maturità architetturale. Un progetto di swarm robotics non può limitarsi a dire che i nodi comunicano: deve spiegare come vengono coordinati. Deve mostrare come si passa da messaggio a fase, da fase a comportamento, da comportamento a forma, da forma a verifica. La coordinazione delle azioni è il ponte tra protocollo e comportamento emergente. È il punto in cui la Sezione 8 e la Sezione 9 si collegano alla Sezione 7.

Una sintesi concettuale può essere:

Sezione 7: definisce le regole del comportamento locale e collettivo;

Sezione 8: definisce come i messaggi viaggiano tra controller e nodi;

Sezione 9: definisce quando quei messaggi producono azioni coordinate;

Sezione 10: definirà quali pattern e comportamenti concreti emergono da questa organizzazione.

Questa relazione è importante perché mantiene coerenza nell'indice. La coordinazione non è un tema isolato; prepara direttamente la sezione successiva sui pattern.

La coordinazione delle azioni deve prevedere anche una forma di validazione post-run. Alla fine di una sequenza, il controller deve valutare se l'obiettivo è stato raggiunto. Per una demo LED, può verificare `ack` ed `execution_error`. Per una simulazione, può calcolare errore rispetto al pattern, coesione, separazione, allineamento e stabilità. Per hardware fisico, può verificare telemetria, stati, eventuali eventi e osservazioni manuali. La run deve produrre un risultato classificabile: `success`, `partial_success`, `failed`, `aborted`, `safety_abort`.

Una classificazione finale può essere:

`success`: tutte le condizioni principali soddisfatte;

`partial_success`: comportamento riuscito ma con nodi mancanti o warning;

`failed`: obiettivo non raggiunto;

`aborted`: sequenza interrotta dal controller;

`safety_abort`: interrotta per safety;

`invalid_run`: condizioni iniziali non valide.

Questa classificazione rende il processo sperimentale tracciabile. Non ogni demo è semplicemente “andata” o “non andata”; esistono stati intermedi utili.

Il risultato della run può essere salvato con metriche:

tempo di completamento;

nodi partecipanti;

nodi mancanti;

numero di retry;

numero di comandi scaduti;

`execution_error` medio;

eventi safety;

stabilità finale;

errore pattern finale.

Queste metriche permettono confronto tra versioni. Se una versione del protocollo riduce comandi scaduti, è un miglioramento. Se una nuova strategia di slot riduce ack storm, è un miglioramento. Se una nuova coordinazione di pattern riduce collisioni simulate, è un miglioramento. La coordinazione diventa quindi misurabile.

La coordinazione delle azioni può essere vista, in termini di controllo, come un livello di orchestrazione sopra gli agenti. Non sostituisce le regole locali, ma le organizza. Ogni MicroBot continua ad avere il proprio stato e la propria safety. Il controller non forza un nodo a eseguire ciò

che non può. Il sistema lavora attraverso contratti: il controller propone azioni; il nodo valida; il nodo esegue se possibile; il controller osserva; il piano evolve. Questa è una forma molto più robusta di controllo rispetto a una catena rigida di ordini.

Questo approccio è anche più realistico rispetto all'hardware. I nodi fisici avranno limiti, rumore, ritardi, differenze e guasti. Una coordinazione che assume perfezione fallirà. Una coordinazione che assume imperfezione e la gestisce sarà molto più forte. MicroBot deve quindi essere progettato non solo per funzionare quando tutto va bene, ma per restare comprensibile quando qualcosa va storto. La coordinazione delle azioni deve includere errore, ritardo, rifiuto, fallback e recupero.

Dal punto di vista teorico avanzato, la coordinazione è ciò che permette allo swarm di diventare un organismo distribuito. In un organismo, le parti non fanno tutte la stessa cosa; fanno cose diverse in tempi coerenti. Allo stesso modo, i MicroBot non devono necessariamente eseguire lo stesso comando. Devono contribuire a una trasformazione comune. Alcuni stabilizzano, alcuni si allineano, alcuni connettono, alcuni osservano, alcuni restano passivi. La coordinazione trasforma la pluralità in funzione collettiva.

Questo è molto vicino alla visione MicroBot come materia programmabile: una materia non cambia forma perché ogni particella riceve lo stesso ordine, ma perché ogni unità locale segue regole e tempi compatibili con una trasformazione globale. La coordinazione delle azioni è la regia di questa trasformazione. Il tempo, i ruoli, le connessioni e i vincoli diventano parti dello stesso linguaggio.

In conclusione, la coordinazione delle azioni è il processo attraverso cui MicroBot trasforma comunicazione e sincronizzazione in comportamento collettivo. Essa richiede readiness dei nodi, assegnazione di ruoli, pianificazione temporale, slot, `execute_at`, feedback, safety, fallback, log e verifica dei risultati. La coordinazione può essere centralizzata, distribuita o ibrida, ma nel progetto MicroBot la forma più coerente è ibrida: il controller organizza il comportamento globale, mentre i nodi mantengono autonomia locale per validazione, timer e sicurezza. Questo permette allo swarm di eseguire fasi, pattern, riconfigurazioni, test e procedure diagnostiche in modo ordinato. Il punto successivo, dedicato alla gestione del ritardo, analizzerà uno dei problemi principali di questa coordinazione: la rete e i dispositivi non reagiscono mai istantaneamente, quindi il sistema deve misurare, compensare e tollerare ritardi senza perdere stabilità o sicurezza.

9.5 Gestione del ritardo

La gestione del ritardo rappresenta uno dei problemi centrali della sincronizzazione MicroBot, perché nessun sistema distribuito reale può assumere che i messaggi vengano trasmessi, ricevuti, interpretati ed eseguiti istantaneamente. Dopo aver definito il controller come riferimento temporale, il clock globale, gli slot temporali e la coordinazione delle azioni, è necessario analizzare il ritardo come variabile strutturale del sistema. In uno swarm composto da più MicroBot, ogni comando attraversa una catena tecnica: viene generato dal controller, serializzato in un pacchetto, trasmesso sulla rete Wi-Fi, ricevuto dal nodo, parsato dal firmware, validato, inserito eventualmente in una coda, eseguito nello slot corretto e confermato tramite ack o telemetria. Ognuna di queste fasi introduce tempo. La gestione del ritardo consiste quindi nel misurare, prevedere, assorbire e compensare questi tempi, evitando che producano disallineamento, instabilità o comportamenti non sicuri.

Nel progetto MicroBot, il ritardo non deve essere considerato soltanto come un difetto della rete, ma come una proprietà inevitabile dell'architettura distribuita. Ogni componente del sistema ha un proprio ritmo: il controller genera comandi a una certa frequenza, la rete Wi-Fi consegna pacchetti con latenza variabile, l'ESP32 esegue il proprio loop firmware, la dashboard aggiorna la visualizzazione con un intervallo separato, il simulatore può avanzare a step discreti, e il safety

layer locale può interrompere azioni in modo indipendente. La gestione del ritardo serve a far convivere questi ritmi diversi dentro una sequenza coerente. Senza questa gestione, il sistema rischierebbe di confondere “comando inviato” con “comando eseguito”, “messaggio ricevuto” con “azione completata”, “stato visualizzato” con “stato reale”.

Il primo tipo di ritardo è il ritardo di comunicazione. Esso rappresenta il tempo che intercorre tra l’invio di un pacchetto da parte del controller e la sua ricezione da parte del nodo, oppure tra l’invio di una risposta da parte del nodo e la ricezione da parte del controller. Nel caso di UDP su Wi-Fi, questo ritardo può essere basso, ma non è garantito in modo deterministico. Può variare a causa di congestione, interferenze, distanza dal router o access point, qualità del segnale, carico del microcontrollore e numero di dispositivi presenti sulla rete. Per questo motivo, il controller deve misurare il ritardo invece di assumerlo come costante.

Una misura utile è il round-trip time, cioè il tempo andata e ritorno tra controller e nodo. Il controller invia un ping o un messaggio di sincronizzazione al tempo $T1$, il nodo risponde, e il controller riceve la risposta al tempo $T4$. Il round-trip time può essere espresso come:

$$RTT = T4 - T1$$

Questa misura non fornisce automaticamente la latenza unidirezionale esatta, perché il tempo di andata e quello di ritorno possono essere diversi, ma offre comunque una stima pratica della qualità del collegamento. Se il RTT è basso e stabile, il nodo è comunicativamente affidabile. Se il RTT è alto o molto variabile, il nodo deve essere trattato con margini maggiori o escluso da azioni temporizzate critiche. Nel sistema MicroBot, il RTT dovrebbe essere registrato per ogni nodo, non solo come valore globale della rete.

Il secondo tipo di ritardo è il jitter. Il jitter indica la variazione del ritardo nel tempo. Due nodi possono avere la stessa latenza media, ma comportamenti molto diversi se uno ha ritardo stabile e l’altro ritardo irregolare. In uno swarm, il jitter è particolarmente pericoloso perché rende difficile prevedere quando un messaggio arriverà. Un sistema può compensare un ritardo costante inviando comandi con anticipo; è molto più difficile compensare un ritardo imprevedibile. Per questo motivo, il controller deve osservare non solo RTT medio, ma anche RTT massimo, minimo e variazione tra misure consecutive.

Un nodo con RTT medio di 40 ms e jitter di 5 ms è più affidabile di un nodo con RTT medio di 40 ms ma jitter di 80 ms. Nel primo caso, il controller può pianificare comandi temporizzati con margini relativamente piccoli. Nel secondo caso, deve usare finestre più larghe, `execute_at` più lontani nel futuro o evitare azioni che richiedono precisione. La gestione del ritardo non consiste quindi soltanto nel ridurre la latenza, ma nel progettare il sistema in modo da tollerare la variabilità.

Il terzo tipo di ritardo è il ritardo di elaborazione locale. Anche dopo che un pacchetto è arrivato al MicroBot, non viene necessariamente eseguito immediatamente. Il firmware deve leggerlo dal buffer UDP, copiarlo, parsarlo, verificare il formato, controllare `target`, `message_id`, `timestamp`, `comando`, `payload`, `capabilities` e `safety`. Se il loop firmware è occupato in altre operazioni, il pacchetto può restare in attesa. Se il codice usa operazioni bloccanti, come `delay` lunghi per LED o test diagnostici, il ritardo può aumentare molto. Per questo motivo, la gestione del ritardo richiede firmware non bloccante e separazione chiara tra ricezione, validazione ed esecuzione.

Nel MicroBot, un comando non dovrebbe mai essere implementato in modo da bloccare il nodo per tutta la durata dell’azione. Un `set_led` di 500 ms non deve fermare il loop per 500 ms. Un `magnetic_pulse` di 100 ms non deve impedire al nodo di ricevere `emergency_stop`, aggiornare `safety` o spegnere l’uscita a tempo. Il nodo deve invece impostare uno stato temporizzato e continuare a

girare nel loop principale. Questa scelta riduce il ritardo di risposta e mantiene il sistema reattivo. La gestione del ritardo è quindi anche una disciplina di progettazione firmware.

Il quarto tipo di ritardo è il ritardo di esecuzione. Anche quando il comando è stato ricevuto e validato, l'azione può essere programmata per un tempo futuro tramite `execute_at`. In questo caso, il ritardo tra ricezione ed esecuzione non è un problema, ma una scelta progettuale. Il nodo riceve il comando in anticipo e lo esegue nel momento stabilito dal clock globale. Questo meccanismo è uno dei modi più importanti per gestire il ritardo di rete: invece di cercare di far arrivare tutti i messaggi nello stesso istante, il controller li invia prima e lascia che i nodi li eseguano in modo sincronizzato. La latenza viene assorbita dal margine tra invio ed `execute_at`.

Questo margine può essere chiamato *lead time*. Il *lead time* è il tempo tra la generazione o trasmissione del comando e il suo istante di esecuzione previsto. Deve essere abbastanza grande da permettere al messaggio di raggiungere tutti i nodi coinvolti, essere validato e programmato. Una formula concettuale può essere:

$$\text{lead_time} \geq \text{latenza_massima_attesa} + \text{jitter_atteso} + \text{tempo_elaborazione} + \text{margine_sicurezza}$$

Se il *lead time* è troppo breve, alcuni nodi riceveranno il comando tardi e lo rifiuteranno come `expired`, oppure lo eseguiranno fuori fase. Se il *lead time* è troppo lungo, il sistema diventa meno reattivo, perché ogni azione deve essere pianificata molto in anticipo. La scelta corretta dipende dal tipo di azione. Un LED sync può tollerare *lead time* moderato. Un comando magnetico coordinato deve avere margini più prudenti. Un `emergency_stop`, invece, non deve aspettare `execute_at`: deve essere eseguito subito quando ricevuto.

La gestione del ritardo deve quindi distinguere tra comandi immediati, comandi temporizzati e comandi di emergenza. I comandi immediati vengono eseguiti appena ricevuti e validati, ma sono sensibili alla latenza. Sono adatti ad azioni non critiche o individuali. I comandi temporizzati usano `execute_at` e sono adatti alla coordinazione. I comandi di emergenza ignorano la normale pianificazione e prevalgono sugli slot, perché la sicurezza richiede risposta immediata. Questa classificazione permette di usare strategie diverse in base alla funzione.

Per i comandi immediati, il ritardo viene accettato e misurato. Per i comandi temporizzati, il ritardo viene compensato inviando in anticipo. Per i comandi di emergenza, il ritardo viene ridotto al minimo tramite priorità, payload semplice, broadcast ripetuto e validazione rapida. Questa tripartizione è molto importante perché evita una soluzione unica per problemi diversi. Non tutti i comandi devono essere trattati con la stessa logica temporale.

La gestione del ritardo deve anche considerare il ritardo di feedback. Dopo aver eseguito un'azione, il nodo invia un `ack`, un event o una telemetria. Anche questa risposta arriva con ritardo. Il controller non deve assumere che l'assenza immediata di risposta significhi fallimento. Deve aspettare entro una finestra di *timeout* coerente con il tipo di comando, con la qualità della rete e con il numero di nodi coinvolti. Se il *timeout* è troppo breve, il controller dichiarerà fallimenti falsi. Se è troppo lungo, il sistema reagirà lentamente ai problemi. Anche il *timeout* deve quindi essere progettato sulla base delle misure di rete.

Una politica corretta può essere:

$$\text{timeout_ack} = \text{RTT_medio} + \text{margine_jitter} + \text{margine_elaborazione}$$

Per comandi broadcast con molti nodi e slot di risposta, il *timeout* deve includere anche la durata della finestra di `ack`. Per esempio, se ogni nodo ha uno slot di risposta e la finestra totale dura 500 ms, il controller non deve dichiarare *timeout* dopo 100 ms. Deve attendere la fine della finestra più

un margine. La gestione del ritardo è quindi strettamente collegata agli slot temporali. Gli slot non solo organizzano le risposte, ma definiscono anche quando una risposta può essere considerata in ritardo.

Il ritardo può essere gestito tramite finestre di validità. Un comando può contenere `expires_at`, cioè un tempo oltre il quale non deve più essere eseguito. Questo impedisce che pacchetti vecchi producano azioni fuori contesto. Se un nodo riceve un comando dopo la fine della finestra, deve rifiutarlo come `command_expired`. Questo comportamento è essenziale nei pattern temporizzati. Un comando previsto per la fase align non deve essere eseguito durante la fase attach; un impulso previsto nello slot magnetico non deve attivarsi durante la stabilizzazione; una correzione prevista per una run precedente non deve agire sulla run successiva.

Una regola generale è:

se `tempo_globale_stimato > expires_at`, il comando viene rifiutato.

Per comandi informativi, il dato può essere registrato come stale. Per comandi fisici, deve essere bloccato. Questa distinzione protegge il sistema da ritardi non controllati. Un dato vecchio può essere utile per analisi storica; un comando fisico vecchio può essere pericoloso. La gestione del ritardo deve quindi distinguere tra informazione e azione.

Nel sistema MicroBot, è utile classificare i messaggi in base alla tolleranza al ritardo. Alcuni messaggi sono delay-tolerant, cioè possono arrivare in ritardo senza gravi conseguenze. Esempi sono log, diagnostica non urgente, telemetria storica. Altri sono delay-sensitive, cioè perdono valore se arrivano troppo tardi, come telemetria live, ack di fase, progress update. Altri ancora sono delay-critical, cioè non devono essere eseguiti se fuori finestra, come impulsi magnetici, attivazioni coordinate, fase di aggancio. Infine ci sono i messaggi safety-critical, come `emergency_stop`, che devono essere processati immediatamente quando ricevuti.

Questa classificazione può essere descritta così:

delay-tolerant: può essere registrato anche se tardivo;

delay-sensitive: utile solo se recente;

delay-critical: valido solo entro una finestra temporale;

safety-critical: priorità massima, esecuzione immediata se valido.

Ogni classe richiede una policy diversa. Questo rende il sistema molto più rigoroso rispetto a una gestione generica dei pacchetti.

La gestione del ritardo richiede anche una coda dei comandi. Quando un nodo riceve comandi temporizzati, non deve eseguirli subito. Deve inserirli in una coda ordinata per `execute_at` e priorità. Il loop firmware controlla periodicamente la coda e attiva i comandi quando arriva il momento. Se un comando scade prima dell'esecuzione, viene rifiutato o rimosso. Se arriva un comando di priorità superiore, come `emergency_stop`, può cancellare o sospendere i comandi pendenti. Questa coda è un piccolo scheduler locale.

Una coda locale può contenere:

`message_id`;

`command`;

payload;

execute_at;

expires_at;

priority;

resource_id;

status.

Il campo `resource_id` è utile perché alcune risorse non possono essere usate da più comandi nello stesso tempo. Per esempio, una bobina non può ricevere due impulsi sovrapposti, e un LED potrebbe dover scegliere tra segnalazione di errore e pattern decorativo. La coda permette di rilevare conflitti temporali e risolverli prima che producano comportamento ambiguo. Anche questo è parte della gestione del ritardo: non basta sapere quando eseguire, bisogna sapere cosa è già previsto in quel momento.

Il ritardo deve essere gestito anche dal controller attraverso una coda di comandi pendenti. Quando invia un comando che richiede `ack`, il controller lo registra come `pending`. Se riceve `ack` entro `timeout`, aggiorna lo stato. Se non lo riceve, decide se ritrasmettere, dichiarare errore o applicare `fallback`. Questa logica deve essere diversa per comando idempotente e non idempotente. Un comando idempotente, come `set_mode idle`, può essere ritrasmesso con rischio basso. Un comando non idempotente, come `magnetic_pulse`, richiede deduplicazione rigorosa. Il controller deve ritrasmettere con lo stesso `message_id` o con `retry_of`, in modo che il nodo non esegua due volte la stessa azione.

La gestione del ritardo è strettamente collegata alla deduplicazione. Un pacchetto può arrivare tardi dopo che il controller ha già ritrasmesso il comando. Il nodo potrebbe ricevere sia l'originale sia il `retry`. Senza `message_id`, potrebbe eseguire due volte. Con `message_id`, riconosce il duplicato. Questo è fondamentale per la sicurezza. Il ritardo e la ritrasmissione sono collegati: ogni volta che si ritrasmette per compensare un possibile pacchetto perso, si introduce la possibilità che arrivino più copie dello stesso comando. La deduplicazione è la protezione contro questo effetto.

Una regola robusta è:

un comando non idempotente viene eseguito al massimo una volta per `command_id`.

Il `message_id` o `command_id` deve essere conservato in una cache temporanea. Se il nodo riceve di nuovo lo stesso comando, reinvia l'`ack` o segnala `duplicate`, ma non riesegue l'azione. Questa regola è indispensabile per qualunque comando attuativo.

Il ritardo deve essere gestito anche a livello di dashboard. La dashboard mostra uno stato che può essere leggermente vecchio rispetto al sistema reale. Se la dashboard riceve telemetria ogni secondo, ciò che mostra potrebbe avere centinaia di millisecondi o più di ritardo. Questo non è necessariamente un problema, ma deve essere indicato. La dashboard dovrebbe mostrare `last_seen`, età della telemetria, stato stale e magari qualità della connessione. Così l'utente non interpreta dati vecchi come se fossero istantanei.

Un valore di telemetria dovrebbe quindi essere accompagnato da:

timestamp della misura;

tempo di ricezione;

age_ms;

stato validità.

Se age_ms supera una soglia, il dato viene marcato stale. Questo è molto importante per evitare decisioni sbagliate basate su informazioni vecchie. Un nodo può apparire online nella dashboard, ma se l'ultimo heartbeat è di 20 secondi fa, in realtà non è affidabile. La gestione del ritardo diventa quindi anche gestione della freschezza dell'informazione.

Il ritardo può essere compensato in vari modi. Il primo modo è il margine temporale, cioè inviare comandi con anticipo sufficiente. Il secondo è l'esecuzione programmata tramite execute_at. Il terzo è l'uso di slot temporali più ampi. Il quarto è il filtraggio dei dati, per evitare di reagire a misure isolate o ritardate. Il quinto è l'adattamento dinamico, cioè modificare il piano in base alla qualità della rete. Il sesto è il fallback, cioè scegliere un comportamento più sicuro quando il ritardo supera soglie accettabili. MicroBot deve usare una combinazione di queste strategie.

Una politica adattiva può essere:

se RTT e jitter sono bassi, usare slot normali e lead time ridotto;

se RTT aumenta, aumentare lead time;

se jitter aumenta, aumentare finestra di validità o ridurre precisione richiesta;

se molti comandi scadono, rallentare il pattern;

se il nodo è instabile, escluderlo da azioni critiche;

se il ritardo supera soglia safety, entrare in modalità sicura o idle.

Questa logica è molto importante perché rende il sistema robusto. Non si assume che la rete sia sempre perfetta. Il sistema osserva la rete e adatta il comportamento.

Nel caso di pattern, il ritardo può causare disallineamento tra fasi. Per esempio, un nodo può essere ancora nella fase align mentre il controller ha già iniziato attach. Questo può accadere se il nodo ha perso un comando, lo ha ricevuto tardi o non ha aggiornato stato. Per evitare questa incoerenza, ogni comando di fase deve includere phase_id, pattern_run_id e tempo di validità. Il nodo deve sapere a quale run e fase appartiene il comando. Se riceve un comando di una fase vecchia, deve rifiutarlo. Se riceve un comando di una run non corrente, deve ignorarlo o richiedere resync.

Una regola è:

accetta comandi di fase solo se pattern_run_id corrisponde alla run corrente e phase_id è coerente con lo stato locale.

Questa regola impedisce che un messaggio ritardato di una fase precedente interferisca con la fase attuale. È particolarmente importante con UDP e con retry. Il tempo non basta; serve anche identificazione della run e della fase.

Il ritardo può influire anche sulle connessioni. Nel modello matematico, una connessione può formarsi o rompersi in base a distanza, orientamento, soglie e hysteresis. Se i messaggi che descrivono queste condizioni arrivano in ritardo, il controller può avere una visione non aggiornata

dello stato fisico. Per esempio, può credere che due nodi siano vicini mentre si sono già allontanati, oppure può credere che una connessione sia stabile mentre è appena caduta. Per questo motivo, lo stato fisico critico deve essere gestito localmente quando possibile. Il nodo non deve aspettare il controller per spegnere una coil dopo `duration_ms`, applicare cooldown o entrare in safety. Il controller coordina, ma il nodo protegge.

Questa è una distinzione fondamentale:

controllo globale tollera ritardo tramite pianificazione;

safety locale non deve dipendere dal ritardo di rete.

Se una condizione è pericolosa, il nodo deve intervenire localmente. Se una bobina supera il tempo massimo, il nodo la spegne. Se un comando è scaduto, il nodo lo rifiuta. Se perde sincronizzazione, non esegue comandi temporizzati critici. La gestione del ritardo deve quindi distribuire responsabilità tra controller e firmware.

Il ritardo può essere misurato e registrato nei log. Ogni comando può avere timestamp di invio, timestamp di ricezione stimato o dichiarato dal nodo, timestamp di esecuzione e timestamp di ack. Da questi valori si possono calcolare diverse metriche:

`command_delivery_delay`;

`processing_delay`;

`execution_delay`;

`ack_delay`;

`total_command_roundtrip`;

`execution_error`.

Queste metriche permettono di capire dove si accumula ritardo. Se `delivery_delay` è alto, il problema è rete. Se `processing_delay` è alto, il problema è firmware. Se `execution_error` è alto nonostante ricezione puntuale, il problema può essere scheduling locale o clock. Se `ack_delay` è alto, può esserci congestione in risposta. Senza questi dati, sarebbe difficile diagnosticare il sistema.

Un ack avanzato potrebbe indicare:

`received_at_local`;

`processed_at_local`;

`scheduled_for`;

`executed_at_local`;

`ack_sent_at_local`.

Il controller può usare questi valori insieme all'offset stimato per ricostruire la pipeline. Anche se nelle prime versioni si può registrare meno, la struttura concettuale è importante. Il ritardo non è un numero unico; è una composizione di ritardi.

La gestione del ritardo deve considerare anche la simulazione. Il simulatore può introdurre ritardi artificiali per testare la robustezza del protocollo. Questo è molto utile. Se il sistema funziona solo con latenza zero, non è realistico. La simulazione dovrebbe poter applicare delay medio, jitter, perdita pacchetti, duplicazione e riordinamento dei messaggi. In questo modo si possono testare timeout, retry, deduplicazione, expires_at, slot temporali e fallback senza rischiare hardware reale.

Una simulazione di rete può includere:

delay fisso;

delay casuale;

jitter variabile;

packet_loss;

duplicate_packet;

out_of_order_delivery;

node_timeout.

Questi scenari aiutano a validare il communication layer e la sincronizzazione. Per MicroBot, che possiede già un ecosistema di simulazione e dashboard, integrare test di ritardo sarebbe molto coerente. Permetterebbe di dimostrare che il sistema non è progettato solo per condizioni ideali.

Il ritardo può essere gestito anche attraverso filtri. Per esempio, una singola misura di RTT molto alta può essere un outlier. Il controller non dovrebbe necessariamente modificare tutto il piano per un singolo campione anomalo. Può usare medie mobili, mediane, finestre temporali o soglie. Allo stesso tempo, non deve ignorare ritardi persistenti. La gestione del ritardo richiede quindi una forma di stima robusta.

Una strategia semplice è mantenere per ogni nodo:

RTT medio recente;

RTT massimo recente;

jitter recente;

numero di timeout recenti;

missed heartbeat consecutivi.

Se un singolo RTT è alto ma gli altri sono normali, si registra warning. Se molti RTT sono alti o ci sono timeout consecutivi, si degrada sync_status o network_status. Questo evita reazioni eccessive a rumore isolato e permette reazioni corrette a problemi reali.

Il ritardo deve essere integrato nella classificazione dello stato del nodo. Un nodo non è solo online o offline. Può essere online ma delayed, online ma unstable, online ma sync_degraded. Questi stati sono importanti. Un nodo delayed può ancora inviare heartbeat, ma non essere adatto a pattern temporizzati. Un nodo unstable può essere escluso da attuazioni fisiche. Un nodo sync_degraded può ricevere solo comandi immediati non critici. Questa granularità rende lo swarm più robusto.

Una classificazione può essere:

online_stable;

online_delayed;

online_unstable;

sync_degraded;

stale;

offline.

Questa classificazione può essere mostrata nella dashboard e usata dal controller nella readiness. In una demo, vedere che un nodo è online_delayed aiuta a spiegare perché non partecipa a un certo pattern.

La gestione del ritardo deve anche affrontare il caso del ritardo asimmetrico. Il tempo di andata controller → nodo può essere diverso dal tempo di ritorno nodo → controller. Il RTT misura la somma, ma non garantisce simmetria. Nei sistemi iniziali si può assumere approssimativamente che la latenza sia simmetrica, ma bisogna sapere che è una semplificazione. Per comandi con execute_at e margini ampi, l'errore può essere tollerabile. Per sincronizzazione più precisa, servono metodi più avanzati o hardware/protocolli più deterministici. Nella documentazione è corretto riconoscere questa approssimazione.

Una stima semplice può usare:

latenza_unidirezionale $\approx$ RTT / 2

ma questa è una stima, non una certezza. Il sistema deve usare margini per compensare l'incertezza. Se il RTT è 80 ms, assumere 40 ms di andata può essere ragionevole per demo non critiche, ma non sufficiente per precisioni strette. Ancora una volta, la risposta è progettare finestre e tolleranze coerenti con la precisione reale.

Il ritardo può anche essere causato da congestione prodotta dal sistema stesso. Se il controller invia troppi pacchetti, se tutti i nodi rispondono insieme, se la telemetria debug è troppo frequente o se la dashboard richiede dati eccessivi, il sistema crea il proprio ritardo. Per questo motivo, la gestione del ritardo non è solo compensazione, ma prevenzione. Bisogna ridurre traffico inutile, limitare telemetria, distribuire ack e usare broadcast o unicast in modo appropriato.

Una strategia preventiva è:

messaggi compatti;

telemetria graduata;

ack selettivi;

slot di risposta;

debug solo su richiesta;

broadcast per comandi comuni;

unicast per dettagli individuali.

Questa strategia riduce il rischio di saturazione. Un sistema ben progettato non combatte continuamente il ritardo; lo evita quando possibile.

Il ritardo deve essere considerato anche nell'interazione tra controller e dashboard. La dashboard non deve rallentare il controller. Se l'interfaccia grafica aggiorna troppi elementi o richiede dati troppo frequentemente, può influenzare il runtime. È utile separare il loop di controllo dal loop di visualizzazione. Il controller può ricevere e processare messaggi a una frequenza più alta, mentre la dashboard visualizza snapshot aggregati a frequenza più bassa. Questo mantiene il sistema reattivo.

Una struttura sana è:

runtime controller: gestisce comandi, sync, ack, safety logic;

telemetry aggregator: prepara stato sintetico;

dashboard: visualizza snapshot e timeline.

Questa separazione evita che la visualizzazione diventi collo di bottiglia. Nel progetto MicroBot, dove la dashboard ha forte valore dimostrativo, è importante mantenere questa distinzione: l'interfaccia deve mostrare il sistema, non rallentarlo.

La gestione del ritardo è importante anche per il logging. Scrivere log su disco può introdurre ritardo se fatto in modo sincrono e frequente. Nei sistemi più maturi, il logging dovrebbe essere bufferizzato o gestito in modo da non bloccare il loop principale. Nei primi prototipi Python questo problema può essere minore, ma va considerato. Se ogni pacchetto genera una scrittura pesante immediata, il controller può rallentare. Meglio registrare eventi essenziali, aggregare dati e scrivere in modo controllato. Anche il log deve rispettare la logica della bassa latenza.

Il ritardo può essere gestito tramite fallback temporali. Se una fase non riceve conferme entro il tempo massimo, il sistema deve decidere cosa fare. Non può aspettare indefinitamente. Ogni fase deve avere un timeout. Al timeout, il controller può scegliere retry, estensione, degradazione, abort o safety. La scelta dipende dalla criticità. Questo impedisce che lo swarm resti bloccato in uno stato intermedio.

Una fase può avere:

max_duration_ms;

timeout_policy;

retry_limit;

fallback_phase.

Per esempio, una fase align può essere estesa una volta se pochi nodi sono in ritardo. Una fase attach può essere ritentata con parametri più conservativi. Una fase magnetica può essere abortita se un nodo critico non risponde. Una fase safety non deve avere retry prima dell'azione locale: deve agire subito. Ogni fase deve avere una politica temporale.

Il ritardo deve essere valutato anche rispetto alla fisica. Un ritardo di 100 ms può essere irrilevante per una visualizzazione lenta, ma significativo per un impulso di 80 ms. Il valore del ritardo va sempre confrontato con la durata dell'azione. Se l'azione dura 5 secondi, un ritardo di 50 ms è

piccolo. Se l'azione dura 50 ms, è enorme. Per questo motivo, il sistema deve definire tolleranze relative al tipo di comportamento.

Una regola concettuale è:

$\text{ritardo_accettabile} < \text{frazione della durata o della finestra dell'azione.}$

Per LED decorativi, la tolleranza può essere ampia. Per sincronizzazione visiva precisa, deve essere più stretta. Per azioni magnetiche brevi, deve essere molto controllata. Questa relazione tra ritardo e scala temporale dell'azione è essenziale.

Nel MicroBot, molte prime demo possono essere progettate con scale temporali più larghe proprio per tollerare la rete. Invece di sequenze rapidissime, si possono usare fasi di 500 ms, 1 secondo o più. Questo permette di validare architettura, protocollo e dashboard senza pretendere precisione estrema. Poi, quando il sistema è stabile, si può ridurre la durata degli slot e aumentare precisione. È una scelta metodologica intelligente: prima robustezza, poi performance.

Il ritardo deve essere gestito anche nei comandi di gruppo. Se un gruppo contiene nodi con qualità di rete diversa, il controller può scegliere il lead time in base al nodo peggiore o escludere il nodo instabile. Scegliere sempre il nodo peggiore rende il sistema più lento; escludere troppo facilmente può ridurre partecipazione. Serve una policy. Per pattern critici, meglio escludere nodi instabili. Per pattern visuali o non critici, si può includerli con margini maggiori.

Una strategia può essere:

azioni critiche: pianificazione basata su nodi affidabili;

azioni non critiche: inclusione più ampia con tolleranza;

azioni diagnostiche: includere nodi instabili per analisi;

azioni safety: inviare a tutti, indipendentemente dallo stato.

Questa flessibilità evita decisioni rigide.

Il ritardo può essere anche intenzionale. Gli slot temporali introducono ritardi pianificati per distribuire risposte o azioni. Questo tipo di ritardo non è un problema; è uno strumento. È importante distinguere delay non controllato da delay pianificato. Un ack ritardato perché assegnato a uno slot è corretto. Un ack ritardato perché il nodo è bloccato è un problema. Il controller deve interpretare il ritardo rispetto al piano.

Per fare questo, ogni risposta dovrebbe indicare slot_id o scheduled_time. Se una risposta arriva dopo il suo slot, è late. Se arriva nel proprio slot, anche se non immediata, è corretta. Il contesto temporale definisce il significato del ritardo. Questo è un punto importante: il tempo non va giudicato assolutamente, ma rispetto alla pianificazione.

La gestione del ritardo deve considerare i messaggi fuori ordine. UDP può consegnare pacchetti in ordine diverso da quello di invio. Inoltre, un retry può arrivare prima dell'originale o viceversa. Per evitare problemi, i pacchetti devono avere sequence_number, timestamp, message_id e phase_id quando necessario. Il nodo deve rifiutare comandi obsoleti e deduplicare comandi ripetuti. Il controller deve ignorare telemetria più vecchia se ne ha già una più recente. Questa è una forma di gestione del ritardo perché l'arrivo fuori ordine è spesso conseguenza di ritardi variabili.

Una regola per stati è:

applica solo aggiornamenti con `sequence_number` maggiore o `timestamp` più recente.

Una regola per comandi è:

esegui solo se `message_id` non processato, `timestamp` valido e `phase_id` coerente.

Queste regole impediscono che messaggi tardivi riportino il sistema indietro.

Il ritardo deve essere integrato nella progettazione dei comandi magnetici. Un comando magnetico non deve dipendere da un futuro comando di stop inviato dalla rete. Deve includere `duration_ms` e il nodo deve spegnere localmente. Questo è già stato definito nelle sezioni precedenti, ma qui diventa centrale: il ritardo della rete rende pericoloso affidarsi a comandi separati “on” e “off”. Un pacchetto “off” potrebbe arrivare tardi o perdersi. Il comando corretto è un impulso temporizzato autonomo.

Quindi, per attuazione fisica:

evitare comando “coil_on” persistente senza durata;

preferire “magnetic_pulse” con `duration_ms`;

applicare `max_duration` locale;

applicare cooldown;

bloccare se `sync` o `timing` non validi.

Questa logica rende il sistema sicuro anche in presenza di ritardi.

La gestione del ritardo è collegata anche alla perdita di comunicazione. Se un nodo smette di ricevere messaggi dal controller, non deve restare indefinitamente in uno stato attivo. Deve avere timeout locali. Per esempio, se non riceve heartbeat o comandi validi dal controller per un certo tempo, può entrare in safe idle, spegnere uscite attive e segnalare riconnessione. Questo protegge il sistema da ritardi estremi, cioè assenza prolungata di comunicazione.

Una regola locale può essere:

se `controller_seen_time` supera `timeout_controller_lost`, disattiva uscite non essenziali e passa a `safe_idle`.

Questo comportamento deve essere calibrato. Non deve attivarsi per un singolo pacchetto perso, ma per perdita prolungata. Il nodo deve distinguere jitter normale da disconnessione. Heartbeat e `last_seen` servono a questo.

Il ritardo può essere rappresentato nella dashboard come stato visuale. Un nodo può essere verde se stabile, giallo se delayed, arancione se unstable, rosso se offline o safety. La dashboard può mostrare “last seen 120 ms ago”, “RTT 45 ms”, “jitter 8 ms”, “sync degraded”. Questo rende visibile una proprietà che altrimenti resterebbe nascosta. Per l’utente o per chi osserva il progetto, vedere la qualità temporale dei nodi dimostra maturità tecnica.

Una dashboard MicroBot ben progettata dovrebbe mostrare almeno:

`last_seen`;

RTT medio;

jitter;

sync_status;

command_pending;

expired_commands;

late_ack_count.

Questi valori permettono di capire se i problemi dello swarm derivano da ritardo comunicativo o da logica comportamentale.

Il ritardo deve essere documentato nei test. Ogni esperimento dovrebbe registrare numero di nodi, modalità di rete, tipo di comando, dimensione pacchetti, RTT medio, jitter, packet loss, comandi scaduti, late ack e risultato della run. Questo permette di costruire una base sperimentale. Non basta dire che la comunicazione funziona; bisogna sapere con quali ritardi e con quali margini. Questa documentazione sarà molto utile per evolvere il sistema e per presentarlo in modo credibile.

Una tabella di test del ritardo potrebbe includere:

test_id;

numero_nodi;

rete usata;

tipo messaggio;

frequenza messaggi;

RTT medio;

RTT massimo;

jitter;

timeout;

expired_commands;

note sul comportamento.

Questi dati permettono di confrontare versioni del protocollo e del firmware. Se dopo una modifica il jitter diminuisce o i timeout calano, c'è un miglioramento misurabile.

La gestione del ritardo deve essere vista come parte della scalabilità. Con pochi nodi, il ritardo può essere basso e facilmente gestibile. Con molti nodi, traffico, ack, telemetria e collisioni aumentano. La scalabilità non riguarda solo calcolo e comunicazione, ma anche tempo. Se lo swarm cresce, bisogna progettare slot, gruppi, ack selettivi, telemetria compressa e comandi ad alto livello. Altrimenti il sistema diventa lento e instabile.

Una strategia scalabile è:

pochi nodi: ack da tutti, debug dettagliato, comandi diretti;

nodi medi: slot di gruppo, telemetria normale, debug mirato;

molti nodi: controllo per cluster, ack aggregati, eventi prioritari, riduzione traffico.

Questa progressione permette al communication layer di crescere senza collassare sotto il proprio traffico.

Il ritardo deve essere collegato anche alla qualità del comportamento emergente. Se i nodi reagiscono con ritardi troppo diversi, il comportamento collettivo può apparire disordinato. La coesione può essere calcolata su stati vecchi, l'allineamento può arrivare in ritardo, la separazione può correggere troppo tardi, l'aggancio può avvenire fuori fase. Nel simulatore, il ritardo può essere modellato esplicitamente per capire quanto il comportamento è robusto. Uno swarm robusto deve tollerare piccoli ritardi senza collassare. Se piccole latenze producono grandi errori, le regole devono essere stabilizzate.

Questo collega la gestione del ritardo al modello matematico della Sezione 7. Le regole locali non devono essere troppo aggressive se la comunicazione è lenta. Se il controller aggiorna target o parametri con ritardo, i nodi devono avere smoothing, hysteresis e limiti di accelerazione. Altrimenti il sistema può oscillare. Il ritardo in un ciclo di controllo può generare instabilità. MicroBot deve quindi combinare comunicazione temporizzata, hysteresis e smorzamento.

Un principio generale è:

più il ritardo è alto, più il controllo deve essere prudente, filtrato e locale.

Se la rete è lenta o instabile, non bisogna comandare micro-correzioni rapide dal controller. Meglio inviare obiettivi più lenti, regole locali e fasi ampie. Se la rete è stabile, si possono usare sequenze più precise. Questo è un principio importante per evitare instabilità da controllo remoto.

La gestione del ritardo deve includere anche una politica di degradazione. Quando la qualità temporale peggiora, il sistema non deve necessariamente fallire subito. Può passare a modalità più semplici. Per esempio, da pattern magnetico a pattern LED, da controllo temporizzato a idle, da telemetria debug a heartbeat, da swarm completo a gruppo ridotto. Questa capacità di degradare in modo controllato è molto importante per robustezza.

Una possibile scala di degradazione è:

modalità completa: pattern coordinato e telemetria normale;

modalità conservativa: pattern più lento e slot più larghi;

modalità visuale: solo LED e simulazione, niente attuazione magnetica;

modalità idle: nodi online ma fermi;

modalità safety: uscite disattivate e diagnostica.

Questa scala consente al sistema di restare utile anche quando il ritardo impedisce funzioni avanzate. Non tutto deve spegnersi immediatamente per ogni problema; ma le funzioni rischiose devono essere ridotte.

Il ritardo deve essere considerato anche nella relazione tra nodi fisici e simulati. Un nodo simulato può reagire quasi istantaneamente dentro lo stesso processo software, mentre un nodo fisico ha rete, firmware e hardware. In una demo ibrida, questa differenza può creare disallineamento. Il controller può introdurre ritardi artificiali nei nodi simulati per allinearli ai fisici, oppure può usare `execute_at` comune. Questo è importante se si vuole visualizzare insieme hardware reale e swarm virtuale. Il simulatore non deve sempre essere “troppo perfetto”, altrimenti non rappresenta bene il sistema fisico.

Una strategia è:

nodi simulati seguono `controller_time` e rispettano gli stessi `execute_at`;

la dashboard visualizza tutti rispetto alla stessa timebase;

eventuali differenze di latenza vengono registrate ma non nascoste.

Questo mantiene coerenza tra mondi.

Il ritardo deve essere gestito anche nelle operazioni manuali dell'utente. Quando l'utente preme un pulsante nella dashboard, il comando non arriva istantaneamente al nodo. La dashboard dovrebbe mostrare lo stato `pending`, poi `accepted`, poi `executed` o `rejected`. Questo evita l'illusione che il click sia già azione fisica. Per comandi critici, può essere richiesta conferma o mostrato chiaramente l'esito. La gestione del ritardo migliora anche l'esperienza utente perché rende visibile la pipeline.

Una sequenza dashboard corretta è:

utente invia comando;

stato comando = `sent`;

ack ricevuto → `accepted`;

esecuzione confermata → `executed`;

nessuna risposta → `timeout`;

rifiuto → `rejected` con reason.

Questa trasparenza è molto importante per un sistema tecnico.

La gestione del ritardo deve essere integrata anche nella progettazione dei file di log e replay. Quando si rigioca una run, bisogna decidere se riprodurre i ritardi originali o una sequenza normalizzata. Per debug, è utile riprodurre i ritardi originali, perché possono aver causato il comportamento osservato. Per presentazione, si può mostrare una versione normalizzata o accelerata. In ogni caso, i timestamp e i delay devono essere registrati. Il replay non deve perdere la dimensione temporale.

Un replay può usare:

`event_time` per la dinamica reale;

`receive_time` per vedere quando il controller ha saputo l'evento;

`delay = receive_time - event_time` per analisi della rete.

Questo permette di capire non solo cosa è successo, ma anche quando il sistema ne è venuto a conoscenza.

Dal punto di vista teorico, il ritardo introduce una distanza tra intenzione e realtà. Il controller genera un'intenzione sotto forma di comando; il nodo la riceve dopo un certo tempo; l'azione fisica avviene ancora dopo; il feedback torna dopo ulteriore ritardo. Il sistema reale è quindi sempre leggermente indietro rispetto al modello ideale del controller. La gestione del ritardo serve a ridurre o compensare questa distanza. Non può eliminarla completamente, ma può renderla prevedibile, misurabile e sicura.

Questa idea è fondamentale per MicroBot: il comportamento collettivo emerge nel tempo, ma il tempo è imperfetto. I nodi non condividono istantaneamente lo stesso stato. La rete non è un mezzo magico. Il controller deve lavorare con informazione ritardata. Per questo motivo, il sistema deve essere progettato con margini, feedback, hysteresis, safety locale e adattamento. Un progetto che ignora il ritardo è fragile; un progetto che lo misura e lo gestisce è molto più maturo.

La gestione del ritardo prepara direttamente il punto successivo, l'allineamento tra dispositivi. Infatti, misurare e compensare i ritardi è il prerequisito per mantenere i clock locali coerenti con il clock globale. Se il controller non conosce almeno approssimativamente i ritardi di comunicazione, non può stimare bene l'offset dei nodi. Se i nodi non sanno quanto è affidabile il loro sync, non possono eseguire comandi temporizzati in modo sicuro. La gestione del ritardo è quindi il ponte tra comunicazione e allineamento temporale.

In conclusione, la gestione del ritardo è una componente essenziale della sincronizzazione MicroBot. Essa riconosce che ogni messaggio, comando, ack, telemetria ed evento attraversa una pipeline temporale non istantanea. Il sistema deve misurare RTT e jitter, usare lead time, `execute_at`, `expires_at`, slot temporali, timeout, retry controllati, deduplicazione, code locali, feedback e fallback. Deve distinguere messaggi tolleranti al ritardo, sensibili al ritardo, critici nel tempo e critici per la sicurezza. Deve mantenere safety locale indipendente dalla rete e adattare i pattern alla qualità temporale reale. In questo modo, MicroBot non assume una rete perfetta, ma costruisce un runtime capace di funzionare anche con ritardi, variazioni e imperfezioni. Il punto successivo, dedicato all'allineamento tra dispositivi, completerà la Sezione 9 descrivendo come i diversi clock locali dei MicroBot possano essere mantenuti coerenti rispetto al riferimento globale, permettendo allo swarm di agire come sistema temporale unificato.

9.6 Allineamento tra dispositivi

L'allineamento tra dispositivi rappresenta il punto conclusivo della Sezione 9 e costituisce la sintesi operativa di tutto ciò che è stato definito nella sincronizzazione: controller come riferimento temporale, clock globale, slot temporali, coordinazione delle azioni e gestione del ritardo. In un sistema distribuito come MicroBot, ogni unità possiede il proprio clock locale, il proprio ciclo firmware, il proprio stato di rete, la propria latenza, la propria condizione energetica e la propria disponibilità operativa. L'allineamento tra dispositivi è il processo attraverso cui queste differenze vengono ridotte, misurate o compensate, così che i MicroBot possano agire come parti di un sistema temporale comune invece che come dispositivi isolati e disordinati.

Nel progetto MicroBot, l'allineamento tra dispositivi non deve essere inteso soltanto come sincronizzazione degli orologi. È un concetto più ampio. Significa che i nodi devono condividere una stessa interpretazione della fase corrente, della run attiva, del clock globale, della modalità operativa, dei comandi validi, degli slot previsti e delle condizioni di sicurezza. Due MicroBot possono avere un clock abbastanza sincronizzato, ma essere comunque disallineati se uno crede di essere nella fase align e l'altro nella fase attach. Allo stesso modo, due nodi possono essere nella

stessa fase ma avere offset temporali troppo diversi per eseguire azioni simultanee. L'allineamento corretto richiede quindi coerenza temporale, coerenza di stato e coerenza operativa.

Il primo livello di allineamento è l'allineamento temporale. Ogni MicroBot deve stimare il rapporto tra il proprio clock locale e il clock globale del controller. Come definito nei punti precedenti, questa relazione può essere approssimata come:

$$\text{tempo_globale} \approx \text{tempo_locale_nodo} + \text{offset_nodo}$$

Il controller invia messaggi di sincronizzazione, il nodo registra il proprio tempo locale di ricezione, calcola o aggiorna l'offset stimato e usa tale offset per interpretare timestamp, execute_at, expires_at e slot temporali. Questo permette al nodo di tradurre un tempo globale in tempo locale. Se il controller dice “esegui a $T = 12000$ ms”, il nodo deve sapere a quale valore del proprio contatore locale corrisponde quell'istante. Senza questa traduzione, i comandi temporizzati non avrebbero significato comune.

Il secondo livello è l'allineamento di stato. Ogni nodo deve sapere quale sia la modalità globale del sistema e quale sia la propria modalità locale. Il controller può dichiarare che la run corrente è line_run_001, fase align, slot align_02, timebase controller_time_ms. Un nodo correttamente allineato deve riconoscere la stessa run, la stessa fase e la stessa timebase. Se un nodo è rimasto a una fase precedente, ha perso un broadcast, si è riavviato o non ha ricevuto un comando di transizione, allora è temporalmente o logicamente disallineato. In questo caso non deve partecipare ciecamente alla fase successiva; deve richiedere resync, entrare in idle oppure essere corretto dal controller tramite unicast.

Il terzo livello è l'allineamento operativo. Anche se il nodo conosce il tempo e la fase corretta, deve essere in grado di eseguire le azioni richieste. Deve avere le capabilities necessarie, non deve essere in safety, non deve essere in low energy, non deve avere risorse occupate, non deve avere comandi pendenti incompatibili e deve rispettare i vincoli locali. Un nodo può essere temporalmente allineato ma operativamente non disponibile. In quel caso, deve rispondere con un rifiuto motivato, ad esempio safety_rejected, low_energy, busy, unsupported_command o sync_degraded. L'allineamento, quindi, non è solo “sapere che ore sono”, ma essere pronto e coerente con il ruolo richiesto.

Una condizione completa di allineamento può essere espressa così:

$$\text{aligned}_i = \text{sync_valid}_i \wedge \text{phase_match}_i \wedge \text{run_match}_i \wedge \text{state_compatible}_i \wedge \text{safety_ok}_i \wedge \text{capability_ok}_i$$

Il nodo i -esimo può essere considerato allineato con il sistema solo se tutte queste condizioni sono soddisfatte. Questa formula sintetizza bene la natura multilivello del problema. sync_valid indica che il riferimento temporale è valido. phase_match indica che il nodo conosce la fase corrente. run_match indica che il nodo sta partecipando alla run corretta. state_compatible indica che il suo stato locale è coerente con l'azione richiesta. safety_ok indica che non esistono blocchi di sicurezza. capability_ok indica che il nodo può effettivamente svolgere quel compito.

L'allineamento tra dispositivi deve essere mantenuto nel tempo, non soltanto stabilito all'avvio. Un nodo può essere perfettamente allineato all'inizio di una run e poi perdere pacchetti, accumulare drift, entrare in low energy, riavviarsi, perdere sync o ricevere in ritardo una transizione di fase. Per questo motivo, il controller deve monitorare continuamente lo stato di allineamento. Ogni heartbeat, ack, telemetry, sync_response ed event può contribuire ad aggiornare la valutazione. L'allineamento non è uno stato fisso; è una condizione dinamica da verificare periodicamente.

Una tabella di allineamento nel controller può contenere:

node_id;
sync_status;
estimated_offset_ms;
last_sync_age_ms;
current_run_id;
current_phase_id;
last_ack_phase;
network_status;
operational_state;
safety_state;
alignment_status.

Il campo alignment_status può assumere valori come aligned, partially_aligned, resync_required, out_of_phase, sync_lost, safety_blocked, offline. Questa classificazione permette al controller e alla dashboard di capire immediatamente quali nodi possono partecipare alla sequenza corrente e quali richiedono intervento. In un sistema distribuito, avere una lista di nodi “online” non basta; bisogna sapere se sono allineati alla logica temporale e operativa della run.

L’allineamento deve iniziare con una procedura di sincronizzazione iniziale. Quando il sistema viene avviato, il controller esegue discovery, identifica i nodi, raccoglie capabilities, misura RTT, invia messaggi di sync e assegna uno stato iniziale. Soltanto dopo questa fase i nodi possono essere marcati come ready. Questo processo evita che un nodo appena collegato partecipi a una run senza conoscere timebase, fase o parametri. La procedura iniziale può essere strutturata così:

discovery dei nodi presenti;
registrazione di node_id, boot_id, firmware_version e capabilities;
misura iniziale di RTT e jitter;
invio di messaggi sync;
calcolo di offset stimato;
assegnazione di modalità idle o ready;
verifica tramite ack;
abilitazione alla partecipazione ai pattern.

Questa sequenza è molto importante perché crea una base ordinata. In un prototipo semplice, alcune di queste fasi possono essere manuali o semplificate. Tuttavia, dal punto di vista architetturale, esse definiscono la procedura corretta per portare uno swarm da stato sconosciuto a stato pronto.

L'allineamento deve gestire il `boot_id` dei nodi. Se un MicroBot si riavvia, il suo clock locale riparte e il suo stato interno può essere perso. Anche se mantiene lo stesso `node_id`, non è più allineato come prima. Il controller deve rilevare il cambio di `boot_id` e invalidare `sync_offset`, comandi pendenti, `phase state` e eventuali informazioni runtime associate alla sessione precedente. Il nodo deve essere riportato in una fase di `resync`. Questo evita che un nodo appena riavviato esegua comandi vecchi o si consideri parte di una run che non conosce più.

Una regola rigorosa è:

se `boot_id` cambia, allora `alignment_status = resync_required`.

Da quel momento, il nodo non deve partecipare a comandi temporizzati o attuativi finché non ha completato di nuovo `discovery`, `sync` e verifica di stato. Può continuare a rispondere a `request_status`, `sync_request`, `enter_idle`, `emergency_stop` e comandi diagnostici sicuri. Questo comportamento protegge il sistema da disallineamenti causati da reset improvvisi.

L'allineamento deve gestire anche il `controller_session_id`. Se il controller si riavvia, cambia la timebase principale. I nodi possono ancora essere accesi, ma il riferimento temporale precedente potrebbe non essere più valido. Per evitare ambiguità, i messaggi del controller dovrebbero includere un identificatore di sessione. Se i nodi ricevono una nuova sessione, devono invalidare comandi temporizzati appartenenti alla sessione precedente e attendere nuovo `sync`. Questo vale soprattutto per sistemi in cui i nodi restano accesi mentre il software controller viene riavviato.

Una regola simmetrica è:

se `controller_session_id` cambia, allora invalidare comandi temporizzati pendenti e richiedere nuovo `sync`.

Questa regola impedisce che un comando programmato da una vecchia esecuzione venga interpretato dentro una nuova. In un sistema sperimentale con molte prove, riavvii e modifiche, questa distinzione è molto utile.

L'allineamento tra dispositivi deve considerare anche il rapporto tra nodi fisici e nodi simulati. Un nodo simulato può essere allineato in modo quasi perfetto al tempo del controller o al tempo della simulazione, perché vive nello stesso ambiente software. Un nodo fisico, invece, passa attraverso Wi-Fi, firmware e clock locale. Se una demo combina nodi simulati e fisici, il controller deve garantire che entrambi seguano la stessa timebase o che le differenze siano dichiarate. Altrimenti la dashboard potrebbe mostrare una coerenza apparente che non corrisponde al comportamento fisico reale.

In una demo ibrida, la scelta più ordinata è usare il `controller_time` come timebase comune. I nodi simulati vengono aggiornati rispetto a `controller_time`; i nodi fisici ricevono `sync` verso `controller_time`; la dashboard visualizza entrambi rispetto allo stesso riferimento. Se invece si usa `simulation_time`, bisogna evitare che comandi di replay o simulazione vengano inviati ai nodi fisici reali senza protezione. La presenza di `timebase_id` nei messaggi aiuta proprio a evitare questa confusione.

L'allineamento tra dispositivi deve essere visibile nella dashboard. Non basta che il controller calcoli internamente `offset` e `sync_quality`. L'utente deve poter vedere quali nodi sono allineati, quali sono in ritardo, quali sono fuori fase e quali richiedono sincronizzazione. Questo è fondamentale per debug e presentazione. Una dashboard MicroBot seria dovrebbe mostrare una vista di sincronizzazione con indicatori per nodo: `sync_good`, `sync_ok`, `sync_degraded`, `sync_lost`,

out_of_phase, offline, safety_blocked. Questa vista aiuta a distinguere problemi di rete, problemi di firmware e problemi di coordinazione.

Una scheda nodo può mostrare:

Node ID: MB-003;

Clock offset: +18 ms;

RTT medio: 42 ms;

Jitter: 7 ms;

Last sync: 850 ms fa;

Run: line_run_001;

Phase: align;

Alignment status: aligned.

Oppure:

Node ID: MB-005;

Clock offset: non valido;

RTT medio: 180 ms;

Jitter: 95 ms;

Last sync: 12 s fa;

Run: unknown;

Phase: unknown;

Alignment status: resync_required.

Queste informazioni rendono immediatamente comprensibile perché un nodo possa o non possa partecipare a un pattern. La dashboard diventa non solo estetica, ma diagnostica.

L'allineamento deve essere verificato prima delle azioni critiche. Prima di inviare una fase magnetica, un pattern temporizzato o una sequenza di aggancio, il controller deve eseguire una readiness check. Questa verifica controlla che i nodi coinvolti siano online, sincronizzati, allineati alla run corrente, nella fase corretta, con safety ok e capabilities adeguate. Se la verifica fallisce, il controller può fermare la sequenza, eseguire resync, degradare il pattern o escludere nodi. Questa verifica deve avvenire prima dell'attuazione, non dopo.

Una readiness check per una fase critica può essere:

tutti i nodi richiesti online;

nessun nodo in safety;

sync_status almeno sync_ok;

alignment_status = aligned;

RTT e jitter sotto soglia;

phase_id coerente;

capabilities richieste presenti;

nessun comando pendente incompatibile.

Se una di queste condizioni non è soddisfatta, il sistema non dovrebbe procedere automaticamente. Per una demo visuale si può essere più permissivi; per attuazione fisica, bisogna essere più rigidi. L'allineamento tra dispositivi diventa quindi un requisito di sicurezza.

L'allineamento può essere mantenuto tramite messaggi periodici di sync. Il controller può inviare sync broadcast o unicast a intervalli regolari. I nodi rispondono con sync_response. Il controller aggiorna offset e jitter. Tuttavia, non tutti i nodi devono necessariamente ricevere sync alla stessa frequenza in ogni momento. Un nodo coinvolto in una fase critica può essere sincronizzato più spesso; un nodo idle può ricevere sync meno frequente. Questa gestione adattiva riduce traffico e mantiene qualità dove serve.

Una politica può essere:

sync frequente prima e durante pattern temporizzati;

sync normale durante idle;

sync immediato dopo reboot o perdita di heartbeat;

sync mirato unicast per nodi con jitter alto;

sync broadcast per riallineamento globale.

Questa politica mantiene il sistema efficiente. Non serve bombardare la rete di sync continui quando lo swarm è inattivo, ma è necessario rafforzare la sincronizzazione prima di azioni coordinate.

L'allineamento deve gestire anche i nodi out_of_phase. Un nodo out_of_phase è online e magari sincronizzato, ma non si trova nella fase corretta della run. Può aver perso un comando broadcast, ricevuto un pacchetto in ritardo o rifiutato una transizione. Questo stato è diverso da sync_lost. Un nodo sync_lost non ha tempo affidabile; un nodo out_of_phase ha tempo forse valido ma stato logico incoerente. La correzione è diversa. Per sync_lost serve risincronizzazione temporale. Per out_of_phase serve riallineamento di stato.

Una procedura per out_of_phase può essere:

controller rileva fase nodo diversa dalla fase globale;

invia request_status unicast;

verifica run_id e phase_id locali;

se il nodo può recuperare, invia set_phase o state_sync;

se la fase corrente è critica, esclude il nodo fino alla prossima run;

se il nodo è instabile, lo mette in idle o diagnostic.

Questa procedura evita che un nodo fuori fase entri improvvisamente in una sequenza non coerente. In uno swarm fisico, questo è importante per evitare azioni indesiderate.

L'allineamento può anche essere parziale. Non sempre serve che tutti i nodi siano perfettamente allineati per ogni funzione. Per una visualizzazione LED lenta, può bastare sync_ok. Per telemetria periodica, può bastare che il nodo sia online. Per attuazione magnetica, serve sync_good e phase_match. Per emergency_stop, invece, non serve allineamento temporale: il comando deve essere eseguito immediatamente se valido. Il sistema deve quindi definire livelli di allineamento richiesti per classe di comando.

Una tabella concettuale può essere:

request_status: richiede solo nodo online;

set_led semplice: richiede online e command validity;

set_led sincronizzato: richiede sync_ok;

start_pattern: richiede aligned o partially_aligned secondo criticità;

magnetic_pulse coordinato: richiede aligned e sync_good;

emergency_stop: richiede solo riconoscimento valido del comando.

Questa distinzione evita di bloccare funzioni semplici per requisiti inutilmente severi, ma protegge le funzioni critiche.

L'allineamento tra dispositivi deve includere anche l'allineamento delle configurazioni. Un nodo potrebbe avere firmware vecchio, protocol_version diversa, capabilities non aggiornate o parametri locali differenti rispetto a ciò che il controller si aspetta. Anche se il tempo è sincronizzato, una configurazione divergente può causare comportamento incoerente. Per esempio, se MB-001 interpreta set_led_pattern in un modo e MB-002 in un altro perché hanno firmware diversi, una sequenza visuale può risultare disallineata. Per questo motivo, discovery e get_node_profile devono raccogliere versioni e capabilities.

Una condizione di allineamento configurativo può includere:

protocol_version compatibile;

firmware_version accettata;

capabilities coerenti;

configuration_epoch aggiornato;

calibration_profile noto;

safety_limits dichiarati.

Il controller può marcare un nodo come `config_mismatch` se non rispetta questi criteri. Un nodo `config_mismatch` può essere escluso da pattern complessi, pur restando disponibile per diagnostica. Questa è una forma di allineamento spesso trascurata, ma molto importante.

L'allineamento delle configurazioni può essere gestito tramite `configuration_epoch`. Ogni volta che il controller invia una configurazione globale o di run, assegna un numero di versione. I nodi rispondono con `config_epoch` applicato. Prima di avviare una fase, il controller verifica che tutti i nodi coinvolti abbiano la stessa configurazione. Questo evita che alcuni usino parametri vecchi e altri nuovi.

Una struttura può essere:

```
config_epoch = 17;
```

```
pattern_config_id = "line_basic_v0.1";
```

```
safety_policy_id = "safe_policy_v0.2";
```

```
timebase_id = "controller_time_ms".
```

Un nodo allineato deve riportare gli stessi ID. Se non li riporta, serve update o esclusione. Questa logica è molto utile nelle simulazioni e nelle demo complesse.

L'allineamento tra dispositivi deve considerare anche l'allineamento dei ruoli. Il controller può assegnare ruoli tramite unicast o broadcast filtrato. Ogni nodo deve sapere il proprio ruolo corrente e il gruppo di appartenenza. Se un nodo crede di essere follower ma il controller lo considera connector, il comportamento può essere incoerente. Per questo motivo, gli ack di `assign_role` e la telemetria devono riportare `role_current`. Prima di una fase, il controller può verificare che ogni nodo abbia il ruolo previsto.

Una tabella ruoli può contenere:

```
node_id;
```

```
expected_role;
```

```
reported_role;
```

```
role_status.
```

Se `expected_role` e `reported_role` divergono, il nodo è `role_mismatch`. Il controller può inviare di nuovo `assign_role` o escludere il nodo. Questo è importante nei pattern, perché il ruolo determina l'azione locale durante le fasi broadcast.

L'allineamento tra dispositivi deve includere anche la gestione dei gruppi. Un nodo deve sapere se appartiene a `cluster_A`, `line_segment_01`, `physical_nodes`, `simulated_nodes` o `diagnostic_group`. Se riceve un comando groupcast ma non ha `group_id` aggiornato, può ignorare erroneamente il comando o eseguirlo quando non dovrebbe. Per evitare questo, `group_id` deve essere parte della configurazione allineata. Ogni nodo deve riportare il proprio gruppo nei heartbeat estesi o nella telemetria.

Una condizione di allineamento di gruppo è:

```
reported_group_id = expected_group_id
```

Se non coincide, il controller deve correggere. Questo è particolarmente importante quando si passa da una run all'altra e i gruppi cambiano.

L'allineamento deve essere verificato anche nei comandi broadcast. Dopo un broadcast globale, non è sufficiente ricevere ack generici. Il controller dovrebbe sapere se i nodi hanno effettivamente applicato la transizione. Un ack può indicare accepted, scheduled o executed, ma per l'allineamento di stato serve anche il nuovo phase_id o mode riportato. Per esempio, dopo set_phase align, il nodo può rispondere:

```
result = executed;
```

```
current_phase = align;
```

```
pattern_run_id = line_run_001.
```

Questo conferma che il nodo non solo ha ricevuto il comando, ma si considera nella fase corretta. In caso contrario, il controller può rilevare disallineamento.

L'allineamento tra dispositivi può essere migliorato tramite state_sync. Il comando state_sync serve a riallineare un nodo o tutto lo swarm allo stato globale corrente. Può includere run_id, phase_id, timebase, config_epoch, role, group, telemetry_level e safety_policy. È un comando più strutturato rispetto a set_mode, perché non cambia un solo campo: ricostruisce il contesto operativo del nodo. Può essere inviato in broadcast per riallineamento globale o in unicast per correggere un nodo specifico.

Un state_sync può contenere:

```
current_run_id;
```

```
current_phase_id;
```

```
timebase_id;
```

```
controller_time_ms;
```

```
config_epoch;
```

```
expected_mode;
```

```
role;
```

```
group_id;
```

```
slot_plan_version;
```

```
valid_until.
```

Il nodo riceve, valida e aggiorna il proprio contesto. Se alcuni campi non sono compatibili con firmware o stato locale, risponde con errore. Questo comando può diventare uno strumento centrale per mantenere coerenza in sistemi complessi.

L'allineamento deve gestire anche lo stato stale. Un nodo può non essere completamente offline, ma può avere informazioni vecchie. Se l'ultimo heartbeat è troppo vecchio, se l'ultimo sync è scaduto, se la telemetria non è aggiornata o se l'ultima fase confermata è precedente, il nodo deve essere

considerato parzialmente disallineato. Il controller non deve trattarlo come pienamente affidabile. Lo stato stale è uno stato intermedio molto utile.

Una regola può essere:

se last_seen supera soglia breve, status = stale;

se last_sync_age supera soglia, sync_status = degraded;

se phase_ack mancante, alignment_status = uncertain;

se timeout prolungato, status = offline.

Questa gradualità evita decisioni troppo brusche. Un nodo non passa immediatamente da perfetto a offline; attraversa stati di incertezza. Il controller può reagire con request_status, sync o resync prima di escluderlo.

L'allineamento tra dispositivi deve considerare la velocità con cui il sistema cambia fase. Se le fasi sono troppo rapide rispetto a rete e firmware, i nodi possono restare indietro. Un sistema con nodi allineati richiede che la velocità del piano sia compatibile con la qualità temporale. Se il controller passa da prepare ad align ad attach troppo rapidamente, alcuni nodi potrebbero non confermare una fase prima della successiva. Questo genera out_of_phase. Per evitare ciò, ogni fase deve avere una durata minima e condizioni di completamento.

Una regola è:

non avanzare a una fase critica finché i nodi richiesti non hanno confermato la fase precedente o finché non è stata applicata una policy di fallback.

Questo rende la sequenza robusta. Il controller non deve procedere solo perché il proprio timer è avanzato se i nodi non sono pronti, salvo che il piano preveda esplicitamente un timeout e una degradazione.

L'allineamento tra dispositivi può essere supportato da heartbeat arricchiti. Un heartbeat minimale dice solo che il nodo è vivo. Un heartbeat arricchito può includere mode, phase_id, run_id, sync_status, safety_state, config_epoch e role. Questo permette al controller di monitorare l'allineamento senza inviare continue richieste separate. Tuttavia, heartbeat troppo ricchi aumentano traffico. Una soluzione è usare heartbeat basic in condizioni normali e heartbeat extended durante pattern o debug.

Un heartbeat extended può contenere:

node_id;

boot_id;

mode;

run_id;

phase_id;

sync_status;

config_epoch;

role;

safety_state;

timestamp.

Questo messaggio fornisce al controller una vista compatta dello stato del nodo. Se qualcosa diverge, il controller può intervenire.

L'allineamento deve essere robusto ai pacchetti persi. Se un nodo perde un broadcast di cambio fase, può restare nella fase precedente. Per ridurre questo rischio, il controller può includere phase_id nei comandi successivi e negli heartbeat attesi. Il nodo può accorgersi che un comando appartiene a una fase che non conosce e chiedere state_sync. Oppure il controller può rilevare che il nodo riporta phase_id vecchio e correggerlo. Questo evita che un singolo pacchetto perso produca disallineamento permanente.

Una regola utile è:

ogni comando di fase deve includere pattern_run_id e phase_id.

Così, anche se il nodo perde un messaggio, può capire il contesto dei messaggi successivi. Se il contesto non coincide, non esegue ciecamente; richiede riallineamento.

L'allineamento tra dispositivi deve anche gestire messaggi ritardati. Un nodo può ricevere un comando vecchio appartenente a una fase precedente. Se controlla phase_id ed expires_at, lo rifiuta. Se non li controlla, potrebbe eseguire un'azione fuori fase. Per questo motivo, l'allineamento logico e temporale sono collegati. Il comando è valido solo se run_id, phase_id e finestra temporale corrispondono allo stato corrente del nodo.

Una validazione completa può essere:

protocol valido;

source autorizzata;

target corretto;

message_id non duplicato;

timestamp valido;

run_id coerente;

phase_id coerente;

config_epoch coerente;

sync_status sufficiente;

safety_ok.

Solo dopo questi controlli il comando può essere eseguito. Per comandi semplici si può usare una validazione più leggera; per comandi critici deve essere completa.

L'allineamento tra dispositivi è importante anche per la raccolta dati. Se la telemetria di nodi diversi non è allineata temporalmente, il controller deve aggregarla con attenzione. Per esempio, una dashboard può mostrare uno snapshot dello swarm a un certo tempo. Ma i dati dei nodi possono arrivare in momenti diversi. Il controller deve decidere se usare l'ultimo dato disponibile, interpolare, attendere una finestra di raccolta o marcare dati mancanti. Questo è un problema di allineamento dei dati, non solo dei dispositivi.

Una strategia semplice è usare finestre temporali:

snapshot_time = T;

raccogli dati generati entro $[T - \Delta, T + \Delta]$;

se un nodo non ha dato recente, marcarlo stale nello snapshot.

Questa tecnica permette di costruire uno stato globale approssimato. In simulazione, i dati possono essere perfettamente allineati; in hardware, no. La dashboard deve tenere conto di questa differenza.

L'allineamento dei dati è fondamentale per calcolare metriche come coesione, distanza media, stabilità, errore pattern e connettività. Se le posizioni o stati dei nodi appartengono a tempi diversi, la metrica può essere distorta. In una prima versione, con nodi simulati, questo problema è minore. In versioni fisiche con sensori e telemetria, diventerà più importante. Il timestamp e il clock globale servono a ridurre questa distorsione.

L'allineamento tra dispositivi deve essere pensato anche per le fasi di recovery. Quando un nodo perde allineamento, il controller può tentare un recupero. Il recupero deve essere graduale e sicuro. Non si deve reinserire immediatamente il nodo in una fase critica. Prima bisogna verificare presenza, sync, stato, safety e configurazione. Poi si può assegnare una modalità idle, aggiornare fase, eventualmente assegnare ruolo e infine renderlo available per la prossima fase o run.

Una procedura di recovery può essere:

mark node as resync_required;

send request_status;

send sync_request;

update offset and sync_quality;

send state_sync;

verify ack and reported state;

if compatible, mark aligned;

if not compatible, keep idle or diagnostic.

Questa procedura evita rientri caotici. Un nodo recuperato deve passare da uno stato controllato.

L'allineamento deve considerare anche le condizioni in cui il recupero non è possibile. Se un nodo ha safety attiva, firmware incompatibile, batteria bassa, jitter eccessivo o capability mancante, non deve essere forzato a rientrare. Il controller deve saper accettare che un nodo resti escluso. Questo è

importante per la robustezza del sistema: l'obiettivo non è obbligare tutti i nodi a partecipare sempre, ma mantenere il comportamento complessivo sicuro e coerente.

Una policy può essere:

se sync_lost ma safety_ok → tentare resync;

se out_of_phase ma online → state_sync;

se low_energy → idle o ruolo passivo;

se safety_blocked → non reintegrare fino a reset/diagnostica;

se firmware_incompatible → escludere da pattern;

se network_unstable → usare solo funzioni non critiche.

Questa policy rende la gestione dell'allineamento realistica.

L'allineamento tra dispositivi ha un ruolo fondamentale nella scalabilità. Con pochi nodi, si può correggere manualmente un disallineamento. Con molti nodi, serve automazione. Il controller deve monitorare e classificare automaticamente lo stato di allineamento. Deve poter inviare sync, state_sync, correzioni e fallback senza richiedere intervento manuale per ogni nodo. Questo è uno dei passaggi che trasformano MicroBot da prototipo a sistema swarm più maturo.

Per swarm piccoli:

controllo manuale e dashboard dettagliata;

resync unicast su nodi specifici;

debug visibile.

Per swarm medi:

sync periodico;

classificazione automatica;

state_sync broadcast;

correzione di gruppi.

Per swarm grandi:

allineamento per cluster;

leader locali o gateway;

telemetria aggregata;

fallback gerarchici.

Questa evoluzione mostra che l'allineamento deve crescere con il numero di nodi. Non basta aggiungere più MicroBot; bisogna aggiungere metodi per mantenerli coerenti.

L'allineamento può essere organizzato anche per cluster. In uno swarm grande, il controller potrebbe non allineare ogni nodo direttamente in ogni momento. Può allineare cluster, gruppi o leader locali. Un leader locale può mantenere coerenza di un sottogruppo. Questa è una direzione futura più avanzata. Per le prime versioni, il controller centrale gestisce direttamente tutti i nodi. Tuttavia, documentare la possibilità di allineamento gerarchico è utile per la scalabilità.

Una struttura gerarchica può essere:

controller globale;

cluster controller o leader locale;

nodi del cluster.

Il controller globale sincronizza i leader; i leader sincronizzano i nodi locali. Questa architettura riduce traffico verso il controller centrale e può migliorare scalabilità. Non è necessaria subito, ma è coerente con swarm robotics avanzata.

L'allineamento tra dispositivi deve essere collegato al safety layer. Un nodo non allineato non deve eseguire azioni rischiose. Se `sync_status` è perso, se `phase_id` è incerto o se `config_epoch` non coincide, il nodo può accettare solo comandi sicuri. Questa regola è fondamentale. Il disallineamento non deve essere trattato solo come problema di performance, ma come potenziale rischio operativo. Un nodo fuori fase potrebbe attivare una bobina nel momento sbagliato o mantenere uno stato errato. Per questo motivo, la perdita di allineamento deve attivare limitazioni.

Una regola generale è:

se `alignment_status` non è `aligned`, bloccare comandi attuativi critici e consentire solo diagnostica, `sync`, `idle`, `disable_outputs` ed `emergency_stop`.

Questa regola protegge il sistema. Un nodo disallineato deve prima essere riallineato o isolato.

L'allineamento deve essere verificabile tramite test. Una suite di test può includere casi come: nodo avviato correttamente, nodo con clock offset, nodo con jitter alto, nodo che perde un broadcast, nodo che si riavvia durante una run, nodo che riceve comando di fase vecchio, nodo con `config_epoch` errato, nodo in safety durante `state_sync`. Ogni caso deve produrre una risposta prevista. Questo rende il sistema robusto non solo in teoria, ma anche in implementazione.

Una tabella di test può includere:

`test_sync_initial`;

`test_sync_drift`;

`test_boot_id_change`;

`test_out_of_phase_detection`;

`test_state_sync_recovery`;

`test_stale_node_detection`;


```
test_command_rejected_when_alignment_lost;  
  
test_late_join_node_idle;  
  
test_config_epoch_mismatch;  
  
test_role_mismatch_correction.
```

Questi test sarebbero molto utili per validare il runtime MicroBot. Anche se non tutti vengono implementati subito, documentarli mostra una direzione tecnica chiara.

L'allineamento tra dispositivi è anche essenziale per il replay. Se una run viene registrata, ogni evento deve essere collocato nella timeline comune. Per ricostruire correttamente cosa è successo, il sistema deve sapere quale offset era valido per ogni nodo in ogni momento. Se un nodo ha perso sync durante la run, gli eventi successivi devono essere marcati con minore affidabilità temporale. Questo rende il replay più onesto. Non tutti i dati hanno la stessa qualità temporale.

Un evento nel log può includere:

```
event_time_global_estimated;  
  
event_time_local;  
  
sync_quality_at_event;  
  
node_id;  
  
phase_id;  
  
run_id.
```

Questo permette di capire se un evento è stato collocato con alta o bassa fiducia. In analisi sperimentale, questa distinzione è molto importante.

L'allineamento tra dispositivi deve essere collegato anche alle metriche finali della run. Una run può fallire non perché il pattern era sbagliato, ma perché i nodi non erano allineati. Per questo motivo, il report finale dovrebbe includere metriche di allineamento: percentuale di nodi aligned, numero di resync, numero di out_of_phase, comandi rifiutati per sync_degraded, execution_error medio, latenze e jitter. Questi dati aiutano a interpretare il risultato.

Un report di run può dire:

```
nodi partecipanti: 8;  
  
nodi aligned all'inizio: 8/8;  
  
nodi aligned alla fine: 7/8;  
  
resync eseguiti: 1;  
  
out_of_phase events: 2;  
  
comandi scaduti: 3;  
  
execution_error medio: 14 ms;
```

risultato: partial_success.

Questa descrizione è molto più tecnica di un semplice “la run è riuscita parzialmente”. Permette di capire dove migliorare.

L’allineamento tra dispositivi deve essere progettato in modo incrementale. Nelle prime versioni, può bastare una sincronizzazione semplice e una verifica manuale della fase. Poi si può aggiungere sync_status, offset stimato, heartbeat extended, state_sync, config_epoch, dashboard temporale e recovery. Non serve implementare subito un sistema perfetto. Però il documento deve definire la direzione. La cosa importante è che ogni nuova versione renda più esplicita e verificabile la coerenza tra nodi.

Una roadmap possibile è:

v0.1: timestamp e heartbeat;

v0.2: node_id, boot_id e last_seen;

v0.3: sync semplice con controller_time;

v0.4: offset stimato e sync_status;

v0.5: run_id e phase_id nei pacchetti;

v0.6: state_sync e rilevamento out_of_phase;

v0.7: config_epoch e role verification;

v0.8: dashboard di allineamento;

v0.9: recovery automatico e policy di degradazione.

Questa progressione è realistica e coerente con lo sviluppo incrementale del progetto. Ogni passaggio aggiunge controllo senza rendere subito il sistema troppo complesso.

Dal punto di vista teorico, l’allineamento tra dispositivi rappresenta il passaggio da pluralità a unità operativa. Uno swarm non è soltanto un insieme di nodi connessi; è un insieme di nodi che condividono un riferimento sufficiente di tempo, stato e intenzione. Senza allineamento, i MicroBot possono comunicare ma non agire veramente insieme. Con allineamento, invece, diventano parti di una stessa trasformazione. Ogni nodo mantiene individualità, ma partecipa a una timeline comune, a una run comune e a una fase comune. Questa è la base del comportamento collettivo controllato.

Questa idea si collega direttamente alla visione di MicroBot come materia programmabile. Una materia programmabile richiede che le sue unità non siano semplicemente presenti nello spazio, ma coordinate nel tempo. La forma non è soltanto configurazione geometrica; è configurazione mantenuta da relazioni temporali e operative. Se alcuni nodi cambiano stato prima, altri dopo, altri in una fase diversa, la forma si degrada. Se invece i nodi sono allineati, la trasformazione può essere coerente. L’allineamento è quindi una condizione della forma dinamica.

In termini di sistema, si può dire che:

la comunicazione permette ai nodi di scambiare informazioni;

la sincronizzazione permette ai nodi di condividere il tempo;

l'allineamento permette ai nodi di condividere contesto operativo;

la coordinazione permette ai nodi di trasformare contesto e tempo in azione collettiva.

Questa catena chiude la Sezione 9. La sincronizzazione non è un dettaglio tecnico secondario, ma il fondamento temporale dello swarm. Senza sincronizzazione, i pattern sono fragili; senza allineamento, i nodi sono incoerenti; senza gestione del ritardo, il controller agisce su informazioni vecchie; senza clock globale, non esiste una timeline comune. Tutti questi elementi servono a costruire un sistema distribuito capace di comportarsi come unità.

L'allineamento tra dispositivi deve essere visto anche come contratto operativo. Il controller garantisce una timebase, fasi e comandi coerenti. I nodi garantiscono di dichiarare il proprio stato, rifiutare comandi non validi, segnalare disallineamento e proteggere le proprie risorse. La dashboard garantisce osservabilità. Il simulatore permette test. Questo contratto rende MicroBot più robusto perché ogni componente ha responsabilità chiara.

In conclusione, l'allineamento tra dispositivi è il processo che consente ai MicroBot di condividere tempo, fase, run, configurazione, ruolo e stato operativo in modo sufficientemente coerente da partecipare a comportamenti collettivi controllati. Esso si basa su clock globale, offset per nodo, sync periodico, heartbeat, run_id, phase_id, config_epoch, state_sync, readiness check, recovery e dashboard diagnostica. Un nodo allineato non è soltanto online: è temporalmente sincronizzato, logicamente coerente, configurato correttamente, operativo e sicuro. Questo punto completa la Sezione 9 perché mostra come la sincronizzazione non sia solo una questione di orologi, ma una condizione generale di coerenza distribuita. Con questa base, il sistema MicroBot può passare alla Sezione 10, dedicata ai pattern e ai comportamenti, dove il tempo condiviso, la comunicazione e il modello multi-agente vengono usati per costruire formazioni geometriche, linee, cerchi, strutture complesse, assegnazione dei ruoli, adattamento dinamico e correzione degli errori.

SEZIONE 10 — Pattern e comportamenti

La Sezione 10 introduce il livello dei pattern e dei comportamenti del sistema MicroBot, cioè il punto in cui il modello matematico, la comunicazione e la sincronizzazione vengono trasformati in configurazioni osservabili dello swarm. Dopo aver definito le coordinate, le interazioni, le soglie, le forze, la comunicazione UDP, i pacchetti, gli identificatori, il clock globale, gli slot temporali e l'allineamento tra dispositivi, diventa necessario descrivere che cosa il sistema deve effettivamente produrre. I pattern rappresentano le forme, le disposizioni e le strutture collettive che i MicroBot devono essere in grado di generare; i comportamenti rappresentano invece le strategie dinamiche attraverso cui tali forme vengono raggiunte, mantenute, corrette o trasformate.

Nel progetto MicroBot, un pattern non deve essere inteso come una semplice immagine finale o come una disposizione statica di punti. Un pattern è una configurazione obiettivo, ma anche una procedura dinamica. Per esempio, una linea non è soltanto “nodi disposti lungo un asse”; è anche il processo attraverso cui i nodi vengono identificati, ordinati, distanziati, allineati, eventualmente agganciati e stabilizzati. Allo stesso modo, un cerchio non è soltanto una forma geometrica; è una distribuzione di unità intorno a un centro, con angoli, distanze, ruoli e connessioni coerenti. Una struttura complessa non è solo un insieme di posizioni target; è una rete di relazioni, vincoli, ruoli e fasi temporali.

Questa sezione è quindi fondamentale perché collega la teoria alla dimostrazione concreta del sistema. Le sezioni precedenti hanno costruito gli strumenti: la Sezione 7 ha definito come modellare matematicamente lo swarm; la Sezione 8 ha definito come far comunicare controller e nodi; la Sezione 9 ha definito come allinearli nel tempo. La Sezione 10 usa questi strumenti per

spiegare come MicroBot possa passare da un insieme di unità distribuite a forme organizzate. In altre parole, qui il sistema smette di essere soltanto infrastruttura e diventa comportamento.

I pattern possono essere classificati in più livelli. Il primo livello comprende pattern semplici, come linea, griglia, cerchio, catena, cluster e disposizione a stella. Questi pattern sono utili nelle prime versioni perché sono facili da visualizzare, misurare e verificare. Il secondo livello comprende pattern strutturali, dove non conta solo la posizione dei nodi, ma anche la connessione tra essi: catene magnetiche, moduli aggregati, segmenti articolati, sottogruppi e strutture con nodi connector. Il terzo livello comprende pattern dinamici, cioè configurazioni che cambiano nel tempo: espansione, compressione, rotazione, riconfigurazione, separazione, fusione di cluster e transizione da una forma all'altra. Il quarto livello comprende pattern adattivi, in cui il sistema modifica la propria strategia in base a errori, nodi mancanti, ritardi, bassa energia o instabilità.

Nel contesto MicroBot, questa progressione è molto importante perché consente sviluppo incrementale. Non è necessario partire immediatamente da strutture complesse. È più rigoroso iniziare con pattern semplici, validarli in simulazione, rappresentarli nella dashboard, associarli a file di configurazione, poi trasferirli a LED e nodi fisici. Una linea sincronizzata con LED può essere il primo pattern. Una catena simulata può essere il secondo. Un cluster con coesione e separazione può essere il terzo. Una struttura con ruoli differenziati può essere il quarto. Una riconfigurazione tra due forme può essere il quinto. Questo approccio permette di costruire il sistema senza saltare passaggi tecnici.

Dal punto di vista architetturale, un pattern MicroBot dovrebbe contenere almeno cinque elementi: una definizione geometrica, una definizione topologica, una definizione comportamentale, una definizione temporale e una definizione di verifica. La definizione geometrica indica dove dovrebbero trovarsi i nodi o quale forma dovrebbero approssimare. La definizione topologica indica quali connessioni o relazioni devono esistere tra i nodi. La definizione comportamentale indica quali regole locali devono essere attivate per raggiungere la forma. La definizione temporale indica in quali fasi e slot il pattern deve essere eseguito. La definizione di verifica indica come misurare se il pattern è stato raggiunto.

In forma sintetica:

pattern = geometria + topologia + comportamento + tempo + verifica

Questa formula è molto importante perché impedisce di ridurre il pattern a una sola immagine. Un pattern valido per MicroBot non è soltanto un disegno; è un oggetto tecnico completo. Deve dire che forma si vuole, quali nodi partecipano, quali ruoli hanno, quali connessioni sono desiderate, quali fasi devono essere eseguite, quali condizioni definiscono successo e quali fallback attivare in caso di errore.

10.1 Definizione di pattern

La definizione di pattern costituisce il primo punto della Sezione 10 e serve a stabilire con precisione che cosa si intende per pattern nel sistema MicroBot. Un pattern è una configurazione organizzata dello swarm, descritta attraverso posizioni, relazioni, ruoli, stati, vincoli e fasi temporali. Esso rappresenta il ponte tra l'idea astratta di forma e la sua realizzazione operativa tramite nodi distribuiti. Nel linguaggio del progetto, un pattern non è soltanto il risultato finale che si vuole vedere, ma l'intero modello che permette al sistema di costruire, mantenere e valutare quel risultato.

In un sistema tradizionale, una forma potrebbe essere descritta semplicemente attraverso coordinate finali. Per esempio, si potrebbe dire che il nodo 1 deve trovarsi in una certa posizione, il nodo 2 in un'altra, il nodo 3 in un'altra ancora. Nel MicroBot, questa descrizione è utile ma non sufficiente. I nodi non sono punti matematici privi di vincoli; sono unità fisiche o simulate dotate di stato, comunicazione, limiti, tempi di risposta, possibili agganci, energia, sicurezza e ruoli. Per questo motivo, il pattern deve includere anche il modo in cui la forma viene raggiunta e mantenuta. La posizione finale è solo una parte del problema.

Un pattern MicroBot può essere formalmente descritto come una struttura composta da un insieme di nodi partecipanti, una geometria obiettivo, una topologia desiderata, un insieme di ruoli, una sequenza di fasi e un insieme di metriche di verifica. In forma concettuale:

$$P = (V_P, G_P, R_P, T_P, B_P, M_P)$$

dove V_P rappresenta i nodi partecipanti al pattern, G_P rappresenta la geometria obiettivo, R_P rappresenta le relazioni o connessioni desiderate, T_P rappresenta la sequenza temporale, B_P rappresenta i comportamenti locali attivati e M_P rappresenta le metriche di valutazione. Questa formulazione rende il pattern un oggetto completo, non un semplice target visivo.

Il primo elemento è V_P , cioè l'insieme dei nodi partecipanti. Non tutti i MicroBot presenti nello swarm devono necessariamente partecipare a ogni pattern. Alcuni possono essere esclusi perché offline, in safety, con batteria bassa, non sincronizzati, privi delle capabilities richieste o semplicemente non necessari alla forma. Il pattern deve quindi dichiarare quali nodi sono coinvolti, oppure quali criteri devono essere usati per selezionarli. Per esempio, una linea di cinque nodi richiede almeno cinque unità disponibili; un pattern LED globale può coinvolgere tutti i nodi online; una struttura magnetica può coinvolgere solo nodi con capability coil_pwm o magnetic_state.

La selezione dei partecipanti può essere esplicita o automatica. In una selezione esplicita, il pattern indica direttamente:

```
participants = [MB-001, MB-002, MB-003, MB-004]
```

In una selezione automatica, il controller sceglie i nodi in base a criteri:

seleziona tutti i nodi online;

seleziona i nodi con sync_good;

seleziona i nodi con capability led_rgb;

seleziona i nodi appartenenti al group_id cluster_A;

seleziona i primi N nodi disponibili.

La selezione automatica è più flessibile, ma richiede più logica nel controller. Per le prime versioni, una selezione esplicita o semi-automatica è più semplice e più facile da debuggare. Per versioni successive, la selezione automatica diventa importante perché consente al pattern di adattarsi a nodi mancanti o differenti.

Il secondo elemento è G_P , cioè la geometria del pattern. Essa descrive la forma spaziale desiderata. Può essere una linea, un cerchio, una griglia, un cluster, una catena, una struttura radiale, una superficie approssimata o una forma più complessa. La geometria può essere definita attraverso coordinate target, regole geometriche o funzioni generative. Per esempio, una linea può essere

definita da un punto iniziale, una direzione e una distanza tra nodi. Un cerchio può essere definito da centro, raggio e numero di nodi. Una griglia può essere definita da righe, colonne e passo. Una forma complessa può essere definita da una lista di punti o da un modello.

Per una linea, la posizione target del nodo i può essere descritta concettualmente come:

$$p_i_target = p_0 + i \cdot d \cdot u$$

dove p_0 è il punto iniziale, d è la distanza tra nodi e u è il vettore direzione. Questa formula permette di generare una catena ordinata. Non è necessario salvare manualmente ogni posizione se il pattern può essere generato da parametri. Questo rende il sistema più compatto e più flessibile. Cambiando d o u , si ottiene una linea più lunga, più corta o orientata diversamente.

Per un cerchio, si può usare:

$$p_i_target = c + r \cdot (\cos \theta_i, \sin \theta_i)$$

dove c è il centro, r è il raggio e θ_i è l'angolo assegnato al nodo i . Se i nodi sono distribuiti uniformemente, si può porre:

$$\theta_i = 2\pi i / N$$

dove N è il numero di nodi. Anche qui, il pattern viene generato da pochi parametri. Questa impostazione è molto utile nella simulazione e nella dashboard, perché permette di visualizzare pattern scalabili senza scrivere manualmente ogni target.

Il terzo elemento è R_P , cioè la topologia desiderata. La geometria dice dove dovrebbero stare i nodi; la topologia dice quali relazioni dovrebbero esistere. In uno swarm MicroBot, questo è fondamentale. Due pattern possono avere geometria simile ma topologia diversa. Una linea può essere solo una disposizione visiva di punti, oppure può essere una catena in cui ogni nodo è connesso al precedente e al successivo. Un cluster può essere compatto ma privo di connessioni definite, oppure può avere una rete di agganci specifici. La topologia trasforma la forma da immagine a struttura.

La topologia può essere rappresentata come un grafo:

$$G_pattern = (V_P, E_P)$$

dove V_P sono i nodi partecipanti ed E_P sono gli archi desiderati. Per una linea di quattro nodi, gli archi desiderati possono essere:

$$E_P = \{(MB-001, MB-002), (MB-002, MB-003), (MB-003, MB-004)\}$$

Per un cerchio di quattro nodi, gli archi possono essere:

$$E_P = \{(MB-001, MB-002), (MB-002, MB-003), (MB-003, MB-004), (MB-004, MB-001)\}$$

Questa differenza è importante. Una linea e un cerchio con quattro nodi possono avere distanze simili, ma la connessione tra ultimo e primo cambia completamente la topologia. Il pattern deve quindi specificare non solo la forma spaziale, ma anche quali connessioni sono desiderate.

Nel contesto fisico, una connessione desiderata può corrispondere a un aggancio magnetico, a una vicinanza controllata, a una relazione di comunicazione, a un vincolo logico o a una relazione di simulazione. Non tutte le connessioni devono essere fisicamente implementate nelle prime versioni.

Nei prototipi LED e simulativi, una connessione può essere rappresentata virtualmente. In versioni con magneti o coil, la connessione può diventare fisica. La struttura del pattern deve essere la stessa, anche se l'implementazione cambia.

Il quarto elemento è il ruolo dei nodi. Un pattern spesso richiede che i nodi non siano tutti equivalenti. In una linea, ci possono essere nodi terminali, nodi intermedi e forse un leader. In un cerchio, ogni nodo può avere ruolo simile, ma alcuni possono fungere da anchor o reference. In una struttura complessa, possono esserci connector, boundary, core, bridge, sensor, actuator o diagnostic nodes. Il ruolo indica come il nodo interpreta comandi globali e fasi temporali.

Un pattern può assegnare ruoli così:

MB-001 → leader;

MB-002 → follower;

MB-003 → connector;

MB-004 → boundary.

Il ruolo non deve essere confuso con l'identità. MB-001 resta MB-001, ma durante una certa run può assumere ruolo leader. In una run successiva, potrebbe essere follower. Questa distinzione è già stata introdotta nelle sezioni precedenti ed è essenziale per mantenere tracciabilità. Il `node_id` è stabile; il ruolo è operativo e temporaneo.

I ruoli permettono di rendere i comandi più compatti. Invece di inviare a ogni nodo un comportamento completamente diverso, il controller può inviare una fase globale e ogni nodo agisce secondo il proprio ruolo. Per esempio, durante `phase_align`, il leader può orientarsi verso l'asse principale, i follower possono allinearsi al vicino precedente, i connector possono preparare l'aggancio, i boundary possono mantenere distanza. Questo riduce traffico e rende il sistema più vicino a un modello multi-agente.

Il quinto elemento è `T_P`, cioè la sequenza temporale del pattern. Un pattern deve essere eseguito nel tempo. Anche una forma apparentemente semplice richiede fasi: preparazione, assegnazione ruoli, allineamento, movimento o avvicinamento, aggancio, stabilizzazione e verifica. La sequenza temporale può essere definita tramite `slot`, `phase_id`, `execute_at`, `duration_ms` e condizioni di avanzamento. Questo collega direttamente la Sezione 10 alla Sezione 9. Un pattern non è solo geometria; è geometria temporizzata.

Una sequenza generica può essere:

`phase_prepare;`

`phase_assign_roles;`

`phase_align;`

`phase_move_or_adjust;`

`phase_attach;`

`phase_stabilize;`

`phase_report.`

Ogni fase può avere durata massima, condizioni di successo, condizioni di fallimento e fallback. Per esempio, `phase_align` può terminare quando tutti i nodi hanno `alignment_error` sotto soglia oppure quando scade un timeout. `phase_attach` può terminare quando le connessioni desiderate sono confermate oppure quando falliscono troppi tentativi. `phase_stabilize` può durare finché la struttura resta stabile per un certo intervallo. Questa logica rende il pattern robusto e verificabile.

Il sesto elemento è `B_P`, cioè l'insieme dei comportamenti locali attivati dal pattern. La geometria dice dove andare; la topologia dice con chi connettersi; il comportamento dice come reagire localmente. I comportamenti possono includere coesione, separazione, allineamento, attrazione verso target, mantenimento distanza, ricerca vicino, aggancio, distacco, stabilizzazione, avoidance, fallback e idle. Il pattern può specificare quali pesi o regole devono essere attivi.

Per esempio, durante la formazione di un cluster, il comportamento di coesione può avere peso alto, mentre l'allineamento può essere meno importante. Durante una linea, l'allineamento e la separazione sono più importanti. Durante un cerchio, la distribuzione angolare e la distanza dal centro diventano rilevanti. Durante la stabilizzazione, i movimenti devono essere ridotti e l'hysteresis deve impedire distacchi oscillanti. Quindi ogni pattern può modificare i pesi comportamentali.

Un set di parametri può essere:

`cohesion_weight;`

`separation_weight;`

`alignment_weight;`

`target_weight;`

`magnetic_weight;`

`stabilization_weight.`

Questi parametri possono essere usati nel simulatore e, in futuro, nei nodi fisici o nel controller. In una prima versione, possono agire solo nella simulazione. In una versione più avanzata, possono influenzare il comportamento locale dei MicroBot. La documentazione deve mantenere questa distinzione tra modello simulato e attuazione fisica.

Il settimo elemento è `M_P`, cioè l'insieme delle metriche di verifica. Un pattern deve poter essere valutato. Non basta dire che una linea “sembra” una linea; bisogna misurare quanto lo è. Per una linea, si può misurare errore di distanza rispetto alle posizioni target, errore di allineamento, deviazione laterale, distanza media tra nodi e numero di connessioni corrette. Per un cerchio, si può misurare errore radiale, distribuzione angolare, chiusura della topologia, distanza tra vicini e stabilità. Per un cluster, si può misurare compattezza, collisioni evitate, connettività e dispersione.

Una metrica generale può essere:

```
pattern_error = funzione(errore_geometrico, errore_topologico, instabilità, collisioni,
nodi_mancanti)
```

Questa funzione può essere semplice nelle prime versioni e più raffinata dopo. L'importante è che il pattern abbia criteri di successo. Senza metriche, la valutazione resta soggettiva. Con metriche, il sistema può classificare una run come success, partial_success, failed o aborted. Questo è fondamentale per la documentazione e per confrontare versioni.

Per esempio, una linea può essere considerata riuscita se:

almeno N nodi partecipano;

l'errore medio rispetto alla linea è sotto soglia;

la distanza tra vicini è entro intervallo;

le connessioni previste sono presenti;

nessun nodo è in safety;

la struttura resta stabile per almeno ΔT_{stable} .

Questa definizione è molto più rigorosa di “i nodi sono più o meno in fila”. Permette di testare il pattern in simulazione e in demo.

Un pattern può essere statico o dinamico. Un pattern statico ha un obiettivo finale relativamente stabile: linea, cerchio, griglia, cluster. Un pattern dinamico descrive una trasformazione nel tempo: espansione, contrazione, rotazione, oscillazione, transizione da linea a cerchio, fusione di cluster, separazione in gruppi. Nei pattern dinamici, la verifica non riguarda solo la forma finale, ma anche il percorso. Per esempio, una transizione da linea a cerchio può essere valutata in base a quanto evita collisioni, quanto mantiene connessioni critiche, quanto tempo impiega e quanto stabilmente raggiunge la forma finale.

Un pattern dinamico può essere descritto come sequenza di pattern:

$P_{\text{dynamic}} = P_1 \rightarrow P_2 \rightarrow P_3 \rightarrow \dots \rightarrow P_k$

Per esempio:

linea $\rightarrow$ arco $\rightarrow$ cerchio

Oppure:

cluster compatto $\rightarrow$ separazione in due cluster $\rightarrow$ riconnessione tramite ponte

Questa visione è molto coerente con MicroBot come materia programmabile. La materia non deve solo assumere una forma; deve poter trasformarsi tra forme. La Sezione 10 dovrà quindi distinguere pattern statici e pattern di transizione.

La definizione di pattern deve includere anche fallback. Un pattern reale può fallire: un nodo può non rispondere, una connessione può non formarsi, un comando può scadere, un nodo può entrare in safety, la rete può degradarsi. Il pattern deve specificare cosa fare in questi casi. Il fallback può essere: ritentare, allungare una fase, escludere un nodo, passare a pattern ridotto, tornare a idle, entrare in safety o abortire. Questa parte è essenziale per non progettare pattern validi solo in condizioni ideali.

Un pattern può avere una policy:

if node_missing and degradable = true $\rightarrow$ adattare pattern;

if critical_node_missing $\rightarrow$ abort;

if safety_triggered → stop fase e isolate node;

if too_many_timeouts → slow_down or abort;

if topology_error persists → retry attach or report partial_success.

Queste regole rendono il sistema più robusto. Un pattern che non prevede errore è incompleto. Nel mondo fisico, errore e variabilità sono normali.

La definizione di pattern deve essere compatibile con i file di configurazione. In un ecosistema MicroBot ben organizzato, i pattern non dovrebbero essere solo hardcoded nel codice. Dovrebbero poter essere descritti in file JSON o YAML, almeno nelle versioni avanzate. Un file di pattern può definire nome, versione, descrizione, numero minimo di nodi, geometria, ruoli, fasi, parametri, metriche e fallback. Questo rende i pattern documentabili, modificabili e riutilizzabili.

Una struttura concettuale di file potrebbe essere:

pattern_id;

version;

description;

minimum_nodes;

geometry_type;

topology_type;

roles;

phases;

behavior_parameters;

success_metrics;

fallback_policy.

Questa struttura sarebbe molto coerente con l'approccio già usato nei file MicroBot come config, dataset, maps e runtime state. Permetterebbe di avere pattern_line_basic.json, pattern_circle_basic.json, pattern_cluster_basic.json e così via. La dashboard potrebbe caricare questi pattern e mostrarli. Il simulatore potrebbe eseguirli. Il controller potrebbe tradurli in comandi. Questa è una direzione molto forte per l'ecosistema.

Un pattern deve avere anche una versione. Come il protocollo, anche i pattern evolveranno. Una line_basic_v0.1 può essere una semplice disposizione visuale; line_basic_v0.2 può includere ruoli; line_basic_v0.3 può includere topologia; line_basic_v0.4 può includere fallback. Versionare i pattern permette di confrontare evoluzioni e non confondere risultati. Se una run è stata eseguita con line_basic_v0.2, il log deve indicarlo. Questo rende l'analisi storica più rigorosa.

Un pattern_run_id distingue invece una esecuzione specifica. Il pattern_id indica il modello; pattern_run_id indica la prova. Per esempio:

```
pattern_id = "line_basic_v0.3"
```

```
pattern_run_id = "line_run_2026_04_30_001"
```

Questo permette di eseguire lo stesso pattern più volte e confrontare risultati. Ogni run può avere nodi diversi, condizioni diverse, latenze diverse e risultati diversi. La distinzione tra pattern e run è fondamentale per una documentazione sperimentale seria.

La definizione di pattern deve essere indipendente, per quanto possibile, dall'hardware specifico. Un pattern come "linea" può essere eseguito in simulazione, su LED, su dashboard o in futuro su nodi magnetici fisici. La geometria e la logica di alto livello restano simili, ma l'implementazione cambia. Questo è un vantaggio enorme. Se il pattern è definito in modo astratto, il sistema può usarlo in più modalità. Se invece il pattern è legato direttamente a un dettaglio hardware, diventa meno riutilizzabile.

Si può quindi distinguere:

pattern astratto: forma, ruoli, topologia, fasi;

implementation profile: simulazione, LED, magnetico, diagnostico, ibrido.

Per esempio, line_basic può avere un implementation_profile "visual_led", in cui i nodi mostrano la linea tramite colori e dashboard, e un implementation_profile "simulation_motion", in cui i nodi virtuali si muovono verso target. In futuro potrebbe avere "physical_magnetic", in cui nodi reali usano agganci. Questa separazione è molto importante per lo sviluppo progressivo.

La definizione di pattern deve anche considerare i vincoli fisici. Una forma può essere matematicamente valida ma fisicamente difficile o impossibile per i prototipi attuali. Per esempio, una griglia tridimensionale può essere interessante, ma se i nodi fisici sono su un piano e non hanno locomozione autonoma completa, deve essere trattata come pattern simulativo o futuro. La documentazione deve distinguere ciò che è implementabile ora da ciò che è obiettivo futuro. Questo mantiene rigore e impedisce allucinazioni progettuali.

Un pattern può quindi avere un campo:

```
implementation_status = simulated_only / visual_demo / hardware_partial / hardware_ready /  
future_concept
```

Questa classificazione è utilissima. Permette di dire chiaramente che una linea LED è hardware visual demo, una aggregazione nel simulatore è simulated_only, un aggancio magnetico con coil può essere hardware_partial, una struttura tridimensionale può essere future_concept. Così il progetto resta ambizioso ma onesto.

Nel MicroBot, i primi pattern realmente utili potrebbero essere visuali e simulativi. Per esempio, una linea sincronizzata di LED dimostra comunicazione, identificazione, broadcast, unicast, timestamp e sync. Un cerchio nella dashboard dimostra modello geometrico e assegnazione target. Un cluster simulato dimostra coesione e separazione. Una catena virtuale con connessioni dimostra topologia. Una demo hardware con due nodi e LED o magneti dimostra il passaggio fisico. Questa progressione rende il sistema credibile.

La definizione di pattern deve anche includere la scala. La stessa forma può essere definita per 3 nodi, 10 nodi o 100 nodi, ma le regole possono cambiare. Una linea di 3 nodi è semplice; una linea di 100 nodi richiede gestione di gruppi, ruoli, segmenti e stabilità. Un cerchio di 4 nodi è quasi un

quadrato; un cerchio di 40 nodi approssima una curva. Il pattern deve quindi essere parametrico rispetto al numero di nodi. Dove possibile, meglio definire pattern scalabili.

Un pattern scalabile usa parametri:

N = numero nodi;

spacing = distanza desiderata;

radius = funzione(N , spacing);

roles = assegnati in base a indice;

topology = generata automaticamente.

Questo permette al pattern di adattarsi. Un pattern non scalabile, invece, definisce manualmente ogni nodo e funziona solo con quel numero. Per i primi test, un pattern non scalabile può andare bene; per la visione MicroBot, la scalabilità è importante.

La definizione di pattern deve anche collegarsi alle metriche di emergenza. Un comportamento emergente deve produrre pattern riconoscibili. Quindi il pattern è anche il criterio con cui si valuta l'emergenza. Se lo swarm usa regole locali e produce una linea, la metrica di linea misura quanto l'emergenza è andata nella direzione desiderata. Se produce un cluster ma con collisioni, il pattern è parzialmente riuscito ma la separazione è insufficiente. I pattern rendono misurabile il comportamento collettivo.

Questo punto collega direttamente la Sezione 10 alla chiusura della Sezione 7. L'emergenza non resta un concetto astratto: viene valutata rispetto ai pattern. Un pattern riuscito è un'emergenza orientata e controllata. Un pattern fallito è un'emergenza non coerente con l'obiettivo o una coordinazione insufficiente. La definizione di pattern è quindi strumento di valutazione scientifica del comportamento dello swarm.

La definizione di pattern deve includere anche il rapporto con la correzione degli errori. Se il pattern osservato differisce dal pattern target, il sistema può calcolare un errore e inviare correzioni. Per esempio, se un nodo è troppo lontano dalla linea, il controller può aumentare target_weight o assegnare un comando correttivo. Se una connessione manca, può ritentare attach. Se un nodo è fuori fase, può inviare state_sync. Se un nodo è offline, può ridistribuire il pattern. Questo sarà approfondito nei punti successivi, ma già qui è chiaro che un pattern deve permettere confronto tra desiderato e reale.

Il confronto può essere:

$\text{errore_pattern}(t) = \text{distanza tra stato_attuale}(t) \text{ e pattern_target}(t)$

Dove stato_attuale include posizioni, connessioni, ruoli e stati; pattern_target include geometria e topologia desiderate. Questa metrica può guidare il controller. Più l'errore è alto, più il sistema deve correggere, rallentare o abortire. Più l'errore è basso e stabile, più il pattern è riuscito.

La definizione di pattern deve essere compatibile con la presenza di nodi mancanti. Un pattern può essere rigidamente definito per N nodi oppure adattabile. Un pattern adattabile può ricalcolare geometria e topologia se manca un nodo. Per esempio, una linea di cinque nodi può diventare linea di quattro; un cerchio può ridistribuire angoli; un cluster può ridurre densità. Tuttavia, non tutti i pattern sono adattabili. Se un nodo ha ruolo critico, la mancanza può rendere impossibile la forma. Il pattern deve dichiarare questa proprietà.

Un campo utile è:

degradable = true / false

E anche:

minimum_nodes;

critical_roles;

fallback_geometry.

Se degradable è true e un nodo manca, il controller può generare una versione ridotta. Se degradable è false, la run viene abortita. Questo rende il sistema più robusto e più chiaro.

La definizione di pattern deve anche includere eventuali priorità. Alcuni pattern privilegiano precisione geometrica; altri privilegiano stabilità; altri privilegiano rapidità; altri privilegiano sicurezza; altri privilegiano consumo energetico basso. Queste priorità influenzano il comportamento. Per esempio, un pattern “fast_line” può accettare maggiore errore ma convergere rapidamente; un pattern “stable_line” può essere più lento ma ridurre oscillazioni; un pattern “safe_demo” può evitare attuazioni magnetiche e usare solo LED/simulazione. Questi profili permettono di adattare il sistema al contesto.

Un campo può essere:

optimization_goal = "stability"

oppure:

optimization_goal = "speed"

oppure:

optimization_goal = "energy_saving"

Questo aiuta il controller a scegliere parametri e fallback. Anche se nelle prime versioni può essere solo documentale, è utile per la struttura futura.

La definizione di pattern deve tenere conto della dimensione didattica e dimostrativa. MicroBot non è solo un sistema tecnico interno; deve essere mostrabile. I pattern semplici come linea, cerchio, griglia e cluster sono ottimi perché intuitivi. Chi osserva capisce subito se il sistema si sta organizzando. Pattern troppo astratti possono essere tecnicamente interessanti ma meno comunicativi. Per presentazioni, pitch e demo ad Andrea o ad altri, è utile scegliere pattern visivamente chiari e spiegabili.

Una sequenza dimostrativa efficace potrebbe essere:

discovery dei nodi;

lampeggio sincronizzato;

formazione linea nella dashboard;

passaggio linea → cerchio;

cluster aggregation simulata;
evidenza di ruoli e connessioni;
log finale con metriche.

Questa demo mostrerebbe comunicazione, sincronizzazione, pattern, emergenza e osservabilità. Anche senza hardware magnetico completo, avrebbe valore tecnico perché dimostra architettura.

La definizione di pattern deve infine essere documentata con precisione. Ogni pattern dovrebbe avere una scheda tecnica. La scheda dovrebbe contenere nome, versione, scopo, numero di nodi richiesto, stato implementativo, geometria, topologia, ruoli, fasi, comandi usati, metriche, fallback, limiti e note. Questo renderebbe i pattern parte stabile dell'ecosistema MicroBot, non semplici idee sparse.

Una scheda pattern può avere:

Pattern ID;

Nome;

Versione;

Descrizione;

Tipo geometrico;

Tipo topologico;

Numero minimo di nodi;

Capabilities richieste;

Modalità supportate;

Ruoli;

Fasi;

Parametri;

Metriche di successo;

Fallback;

Limiti;

Stato implementativo.

Questa struttura sarà molto utile nei documenti, nei file README e nei repository. Ogni pattern diventa un componente documentato del sistema.

In conclusione, la definizione di pattern è il fondamento della Sezione 10. Un pattern MicroBot è una configurazione organizzata dello swarm descritta non solo da una forma finale, ma da geometria, topologia, ruoli, comportamenti, fasi temporali, metriche e fallback. Esso permette di

trasformare il modello matematico in obiettivi concreti, la comunicazione in comandi orientati, la sincronizzazione in sequenze operative e la dashboard in osservazione del comportamento collettivo. Definire bene i pattern significa rendere MicroBot verificabile, dimostrabile e scalabile. Il punto successivo potrà quindi entrare nei pattern geometrici semplici, come linea, cerchio, griglia e cluster, mostrando come le forme di base possano essere generate, misurate e usate come primi comportamenti fondamentali dello swarm.

10.2 Pattern geometrici semplici

I pattern geometrici semplici rappresentano il primo livello operativo del sistema MicroBot, perché permettono di tradurre la definizione generale di pattern in forme concrete, leggibili e verificabili. Dopo aver stabilito che un pattern non è soltanto una figura finale ma una struttura composta da geometria, topologia, ruoli, comportamenti, fasi temporali, metriche e fallback, è necessario partire dalle forme di base: linea, cerchio, griglia e cluster. Questi pattern sono fondamentali perché costituiscono il vocabolario geometrico iniziale dello swarm. Prima di progettare strutture complesse, riconfigurazioni dinamiche o comportamenti adattivi, il sistema deve saper generare, rappresentare e valutare forme semplici.

Nel progetto MicroBot, i pattern geometrici semplici hanno un valore tecnico e dimostrativo molto alto. Dal punto di vista tecnico, permettono di testare identificazione dei nodi, assegnazione dei ruoli, calcolo di target, sincronizzazione temporale, comunicazione broadcast/unicast, telemetria, misurazione dell'errore e dashboard. Dal punto di vista dimostrativo, sono immediatamente comprensibili: una linea si vede, un cerchio si vede, una griglia si vede, un cluster si vede. Questo è importante perché una demo efficace deve rendere percepibile il comportamento collettivo senza richiedere spiegazioni troppo astratte. Se i MicroBot virtuali o fisici passano da disposizione casuale a linea, poi a cerchio, poi a cluster, l'osservatore capisce subito che il sistema sta coordinando unità distribuite.

Il primo obiettivo dei pattern geometrici semplici non è ottenere subito movimento fisico autonomo completo. Nelle prime versioni, questi pattern possono vivere nella simulazione, nella dashboard e nei LED. Per esempio, una linea può essere rappresentata da nodi virtuali disposti nello spazio simulato e da MicroBot fisici che mostrano il proprio ruolo tramite LED. Un cerchio può essere visualizzato nella dashboard come pattern target e come stato corrente dello swarm simulato. Un cluster può essere generato dal simulation engine tramite regole di coesione e separazione. Una griglia può essere usata come pattern statico per testare assegnazione di posizioni e metriche. Questa distinzione è importante: il pattern può essere definito in modo astratto anche quando l'hardware fisico non implementa ancora tutti i movimenti necessari.

Un pattern geometrico semplice può quindi essere usato in tre modalità principali. La prima modalità è "simulation-only", dove i nodi sono agenti virtuali e il pattern viene raggiunto tramite aggiornamento delle posizioni simulate. La seconda modalità è "visual-demo", dove i nodi fisici non si spostano realmente ma rappresentano lo stato del pattern tramite LED, dashboard e telemetria. La terza modalità è "hardware-assisted", dove alcune funzioni fisiche, come aggancio, orientamento, feedback o attuazione magnetica limitata, vengono effettivamente coinvolte. Questa classificazione permette di mantenere rigore: non si dichiara che un prototipo fisico esegue una formazione completa se in realtà la formazione è simulata; allo stesso tempo si riconosce che la stessa architettura del pattern prepara la futura implementazione fisica.

La linea è il pattern geometrico più semplice e probabilmente il primo da implementare. Una linea è una disposizione ordinata di nodi lungo un asse. Dal punto di vista geometrico, richiede un punto iniziale, una direzione, una distanza tra nodi e un ordinamento. Se si hanno N nodi, la posizione

target del nodo i-esimo può essere generata a partire da un punto base p_0 , da un vettore direzione unitario u e da una distanza d . La posizione target può essere descritta come:

$$p_{i_target} = p_0 + i \cdot d \cdot u$$

Questa formula è semplice ma molto potente. Permette di generare linee di lunghezza variabile senza scrivere manualmente ogni posizione. Cambiando N , si cambia il numero di nodi. Cambiando d , si cambia la distanza tra essi. Cambiando u , si cambia l'orientamento della linea. In una simulazione bidimensionale, u può essere un vettore nel piano. In una rappresentazione tridimensionale futura, u può essere un vettore nello spazio. La linea diventa quindi un pattern parametrico.

Dal punto di vista topologico, una linea può essere interpretata in due modi. Nel primo caso è una semplice disposizione geometrica, quindi conta solo che i nodi siano approssimativamente allineati e distanziati. Nel secondo caso è una catena, quindi ogni nodo deve essere connesso al precedente e al successivo. Questa distinzione è fondamentale. Una linea visuale può essere raggiunta anche senza connessioni fisiche; una catena strutturale richiede invece archi specifici nel grafo. Per N nodi ordinati, la topologia lineare desiderata è:

$$E_{line} = \{(n_0, n_1), (n_1, n_2), \dots, (n_{n-2}, n_{n-1})\}$$

Questa topologia dice che ogni nodo interno ha due vicini e i due nodi estremi hanno un solo vicino. Nei primi prototipi, questa connessione può essere simulata. In futuro, può corrispondere a un aggancio magnetico o a una relazione fisica. La differenza tra linea geometrica e catena topologica deve essere mantenuta chiara nella documentazione.

I ruoli nella linea sono facili da definire. Il primo nodo può essere leader o anchor. L'ultimo può essere terminal. I nodi centrali possono essere follower o connector. Questa assegnazione è utile perché non tutti i nodi hanno la stessa funzione. Il leader può definire l'origine o l'orientamento della linea. I follower possono mantenere distanza dal nodo precedente. I connector possono mantenere connessioni tra due vicini. I terminal possono evitare connessioni oltre l'estremità. In una demo LED, il leader può essere mostrato con un colore diverso, i connector con un altro, i terminal con un altro ancora.

Una possibile assegnazione dei ruoli per una linea di cinque nodi è:

MB-001: leader;

MB-002: connector;

MB-003: connector;

MB-004: connector;

MB-005: terminal.

In una versione più simmetrica, si può avere un anchor centrale e due rami laterali. Ma per una prima implementazione, una linea ordinata da un nodo iniziale a un nodo finale è più semplice. La semplicità è un vantaggio: permette di testare tutta la pipeline senza introdurre geometrie troppo difficili.

La sequenza temporale della linea può essere organizzata in fasi. La prima fase è prepare, in cui il controller seleziona i nodi, verifica readiness, assegna ruoli e calcola target. La seconda fase è align,

in cui i nodi o la simulazione orientano la disposizione verso la direzione della linea. La terza fase è spacing, in cui si corregge la distanza tra nodi. La quarta fase è attach, se si vuole una catena topologica. La quinta fase è stabilize, in cui il sistema verifica che la linea rimanga stabile per un certo intervallo. La sesta fase è report, in cui vengono raccolte metriche e risultati.

Una sequenza possibile è:

phase_prepare;

phase_assign_roles;

phase_align;

phase_spacing;

phase_attach;

phase_stabilize;

phase_report.

Nelle prime versioni, phase_attach può essere puramente virtuale oppure assente. Se il pattern è solo geometrico, basta verificare allineamento e distanza. Se il pattern è topologico, bisogna verificare anche le connessioni. Questa modularità permette di usare lo stesso pattern linea in versioni diverse.

Le metriche della linea devono misurare quanto i nodi sono vicini alla forma desiderata. Una metrica fondamentale è l'errore laterale rispetto all'asse della linea. Se un nodo target dovrebbe stare sulla linea, la sua distanza perpendicolare dall'asse misura quanto è fuori allineamento. Un'altra metrica è l'errore di spacing, cioè la differenza tra distanza effettiva tra nodi vicini e distanza desiderata d . Una terza metrica è l'errore d'ordine, cioè se i nodi rispettano l'ordine previsto lungo l'asse. Una quarta metrica è la topologia, cioè se le connessioni desiderate sono presenti e quelle indesiderate assenti.

Per una linea si possono quindi usare:

errore_laterale_medio;

errore_spacing_medio;

errore_ordine;

connessioni_corrette / connessioni_attese;

stabilità_temporale.

Una linea può essere considerata riuscita se l'errore laterale medio è sotto una soglia, l'errore di spacing è accettabile, i nodi rispettano l'ordine, le connessioni attese sono presenti se richieste e la struttura resta stabile per una durata minima. Questa definizione trasforma una forma visiva in una condizione misurabile.

Il secondo pattern geometrico semplice è il cerchio. Il cerchio è più complesso della linea perché richiede distribuzione angolare intorno a un centro. Dal punto di vista geometrico, il pattern è

definito da un centro c , un raggio r e un numero N di nodi. Se la distribuzione è uniforme, ogni nodo riceve un angolo θ_i . La posizione target può essere definita come:

$$p_{i_target} = c + r \cdot (\cos \theta_i, \sin \theta_i)$$

con:

$$\theta_i = 2\pi i / N$$

Questa formula genera punti equidistanti su una circonferenza. Il cerchio è molto utile nella dashboard perché mostra chiaramente coordinazione e distribuzione. È anche utile per testare assegnazione angolare, separazione e chiusura topologica. A differenza della linea, il cerchio non ha estremità. Ogni nodo può avere due vicini se la topologia è una catena chiusa.

Dal punto di vista topologico, il cerchio può essere definito come ciclo. Per N nodi ordinati, le connessioni desiderate sono:

$$E_{circle} = \{(n_0, n_1), (n_1, n_2), \dots, (n_{n-2}, n_{n-1}), (n_{n-1}, n_0)\}$$

La connessione finale tra ultimo e primo distingue il cerchio dalla linea. In una demo simulativa, questo può essere mostrato con archi grafici nella dashboard. In una versione fisica futura, potrebbe rappresentare una struttura chiusa o un aggancio finale. Il cerchio è quindi un ottimo pattern per dimostrare topologia, non solo geometria.

I ruoli nel cerchio possono essere più uniformi. Tutti i nodi possono essere `ring_node`. Tuttavia, può essere utile assegnare un `anchor` o `reference_node` per definire l'orientamento iniziale. Senza un nodo di riferimento, il cerchio può ruotare senza cambiare forma: geometricamente resta corretto, ma per la dashboard o per una run ripetibile può essere utile fissare un punto. Un nodo può quindi avere ruolo `anchor` e gli altri ruolo `ring_member`. In una versione più avanzata, si possono avere `connector` o `sector_leader` per suddividere il cerchio in archi.

Una possibile assegnazione è:

MB-001: `anchor`;

MB-002—MB-N: `ring_member`.

Il nodo `anchor` definisce $\theta = 0$ o il punto iniziale del cerchio. Gli altri vengono distribuiti in ordine. Questo rende il pattern riproducibile. Se invece non interessa l'orientamento assoluto, il cerchio può essere valutato rispetto al centro e alla distanza angolare relativa.

La sequenza temporale del cerchio può includere `prepare`, `distribute_angles`, `radial_align`, `angular_spacing`, `close_loop`, `stabilize` e `report`. La fase `radial_align` porta i nodi verso il raggio desiderato. La fase `angular_spacing` corregge la distribuzione angolare. La fase `close_loop` verifica la connessione tra ultimo e primo, se la topologia è richiesta. La fase `stabilize` misura se il cerchio resta stabile nel tempo.

Le metriche del cerchio includono errore radiale, errore angolare, uniformità della distribuzione, chiusura topologica e stabilità. L'errore radiale misura quanto ogni nodo è distante dal raggio r . L'errore angolare misura quanto gli angoli tra nodi vicini sono vicini a $2\pi/N$. La chiusura topologica misura se il ciclo è completo. La stabilità misura se la configurazione resta coerente per una durata minima.

Per il cerchio si possono usare:

errore_radiale_medio;

varianza_angolare;

distanza_media_tra_vicini;

connessione_finale_presente;

numero_archi_corretti;

stabilità_temporale.

Un cerchio può essere considerato riuscito se i nodi sono vicini al raggio desiderato, gli angoli sono distribuiti in modo abbastanza uniforme, non ci sono sovrapposizioni, le connessioni desiderate sono presenti e la forma resta stabile. In una demo visuale, la dashboard può mostrare target circle, current nodes e pattern_error in tempo reale.

Il terzo pattern geometrico semplice è la griglia. La griglia rappresenta una disposizione regolare su righe e colonne. È molto utile perché introduce struttura bidimensionale ordinata. Una griglia può essere definita da numero di righe R , numero di colonne C , passo orizzontale dx , passo verticale dy e origine p_0 . Per un nodo assegnato alla riga a e alla colonna b , la posizione target è:

$$p_{ab_target} = p_0 + b \cdot dx \cdot e_x + a \cdot dy \cdot e_y$$

dove e_x ed e_y sono i vettori base del piano. Se $N = R \cdot C$, ogni nodo occupa una cella. Se N è minore, la griglia può essere parziale. Se N è maggiore, si possono aggiungere livelli o cluster. La griglia è più complessa di linea e cerchio perché richiede assegnazione bidimensionale e vicinato multiplo.

Dal punto di vista topologico, una griglia può avere connessioni orizzontali, verticali o entrambe. Una griglia 2D completa può essere rappresentata come grafo in cui ogni nodo interno ha fino a quattro vicini: sopra, sotto, sinistra e destra. I nodi di bordo hanno meno vicini. Questa topologia è molto utile per simulare strutture reticolari, superfici, materiali modulari e aggregazioni più robuste.

Per una griglia, gli archi desiderati possono essere:

connessione orizzontale tra celle adiacenti nella stessa riga;

connessione verticale tra celle adiacenti nella stessa colonna.

Questa topologia è più ricca di quella della linea e del cerchio. Permette di introdurre ruoli come corner, edge, internal e connector. I nodi corner hanno due vicini, i nodi edge ne hanno tre, i nodi internal ne hanno quattro. Questa distinzione è molto interessante perché collega geometria e ruolo in modo naturale.

Una possibile classificazione dei ruoli nella griglia è:

corner_node;

edge_node;

internal_node;

grid_anchor.

Il `grid_anchor` può definire l'origine della griglia. I `corner_node` definiscono gli angoli. Gli `edge_node` formano i bordi. Gli `internal_node` stabilizzano la struttura. In una simulazione, questi ruoli possono essere visualizzati con colori diversi. In futuro, potrebbero corrispondere a funzioni fisiche differenti.

La sequenza temporale della griglia può essere più articolata. Una strategia semplice è costruire prima una riga base, poi aggiungere righe successive. Questa costruzione incrementale è più controllabile rispetto al tentativo di formare tutta la griglia insieme. Per esempio:

```
phase_prepare;  
  
phase_build_base_line;  
  
phase_add_second_row;  
  
phase_align_columns;  
  
phase_attach_horizontal_edges;  
  
phase_attach_vertical_edges;  
  
phase_stabilize_grid;  
  
phase_report.
```

Questa sequenza mostra come la griglia possa essere costruita da pattern più semplici. Prima si usa la linea, poi si estende in due dimensioni. Questo è molto coerente con un approccio incrementale: pattern complessi vengono costruiti combinando pattern semplici. La griglia non è un punto di partenza, ma una composizione.

Le metriche della griglia includono errore rispetto alle celle target, errore di righe, errore di colonne, regolarità del passo, correttezza della topologia, angoli della struttura e stabilità. Un nodo può essere vicino alla posizione target ma la griglia può essere globalmente ruotata o deformata. Bisogna quindi misurare sia errore locale sia regolarità globale. In una prima versione, basta misurare distanza dai target; in una versione più matura, si può valutare la qualità reticolare.

Metriche utili:

```
errore_posizione_medio;  
  
errore_dx_medio;  
  
errore_dy_medio;  
  
deformazione_angolare;  
  
percentuale_archi_orizzontali_presenti;  
  
percentuale_archi_verticali_presenti;  
  
nodi_fuori_cella;  
  
stabilità_temporale.
```

La griglia è particolarmente utile per mostrare il concetto di programmable matter, perché assomiglia a una struttura materiale modulare. Una linea mostra coordinazione monodimensionale; un cerchio mostra chiusura; una griglia mostra superficie e regolarità. È quindi un pattern importante per la visione futura, anche se l'implementazione fisica completa richiederà più sviluppo.

Il quarto pattern geometrico semplice è il cluster. A differenza di linea, cerchio e griglia, il cluster non richiede una posizione specifica per ogni nodo, ma una compattezza collettiva. Il suo obiettivo è portare i nodi in una regione comune mantenendo separazione minima e evitando collisioni o sovrapposizioni. È un pattern molto naturale per sistemi swarm, perché può emergere da regole locali di coesione e separazione. Il cluster è quindi più comportamentale e meno rigidamente geometrico rispetto agli altri.

Un cluster può essere definito da centro c , raggio massimo R_{cluster} , distanza minima d_{min} tra nodi e densità desiderata. L'obiettivo è che tutti i nodi si trovino entro una certa regione intorno al centro, senza essere troppo vicini tra loro. Non serve assegnare a ogni nodo una posizione esatta; basta che la distribuzione globale sia compatta e stabile. Questo rende il cluster più adattabile a nodi mancanti o aggiunti.

Una condizione geometrica può essere:

$$\|p_i - c\| \leq R_{\text{cluster}} \text{ per tutti i nodi partecipanti}$$

e una condizione di separazione:

$$\|p_i - p_j\| \geq d_{\text{min}} \text{ per ogni coppia di nodi distinti}$$

Queste due condizioni bilanciano coesione e separazione. La coesione porta i nodi verso il centro; la separazione impedisce collisioni. Questo pattern è molto coerente con le regole boids e con il modello multi-agente della Sezione 7.

Dal punto di vista topologico, il cluster può avere una topologia meno rigida. Può richiedere che il grafo dei nodi sia connesso, senza specificare esattamente tutti gli archi. Oppure può richiedere un certo numero minimo di vicini per nodo. Per esempio, il cluster può essere considerato riuscito se il grafo di prossimità è connesso e ogni nodo ha almeno un vicino entro raggio di connessione. Questa topologia è più flessibile di linea e cerchio.

Una condizione topologica del cluster può essere:

il grafo di prossimità G_{cluster} è connesso;

ogni nodo ha $\text{degree} \geq 1$;

nessun nodo è isolato.

In una versione più densa, si può richiedere degree medio maggiore. In una versione più leggera, basta evitare isolamento. Questo rende il cluster un pattern adattabile e robusto.

I ruoli nel cluster possono includere core_node , boundary_node e free_node . I core_node sono vicini al centro. I boundary_node si trovano sul bordo e aiutano a mantenere forma e dimensione. I free_node sono nodi ancora non integrati o in avvicinamento. Questa classificazione è utile per la dashboard e per comportamenti adattivi. Un boundary_node può avere comportamento di contenimento; un core_node può stabilizzare; un free_node può cercare connessione.

La sequenza temporale del cluster può essere:

```
phase_prepare;  
phase_converge;  
phase_separate;  
phase_connect;  
phase_stabilize;  
phase_report.
```

La fase converge attiva coesione verso il centro o verso il gruppo. La fase separate impedisce sovrapposizione. La fase connect verifica che non ci siano nodi isolati. La fase stabilize riduce oscillazioni. Il cluster è un ottimo pattern per testare equilibrio tra attrazione e repulsione. Se la coesione è troppo alta, i nodi collassano. Se la separazione è troppo alta, il cluster si disperde. Se i pesi sono bilanciati, emerge una struttura compatta.

Le metriche del cluster includono raggio effettivo, distanza media dal centro, distanza minima tra nodi, numero di collisioni o violazioni di separazione, connettività del grafo e stabilità. Un cluster riuscito non è necessariamente perfettamente simmetrico; deve essere compatto, connesso e sicuro.

Metriche utili:

```
cluster_radius_effettivo;  
distanza_media_dal_centro;  
min_pairwise_distance;  
violazioni_d_min;  
numero_nodi_isolati;  
connettività_grafo;  
stabilità_temporale.
```

Il cluster è particolarmente interessante perché può essere il primo pattern davvero emergente. Mentre linea, cerchio e griglia possono essere target-based, il cluster può nascere da regole locali. Ogni nodo segue coesione e separazione; la forma globale emerge. Questo lo rende molto importante per dimostrare la logica swarm di MicroBot.

Oltre a linea, cerchio, griglia e cluster, si possono considerare anche pattern geometrici intermedi come catena, stella e anello aperto. La catena è simile alla linea ma enfatizza la topologia di connessione. La stella ha un nodo centrale e nodi radiali. L'anello aperto è una transizione tra linea curva e cerchio. Questi pattern possono essere introdotti dopo i quattro fondamentali. La cosa importante è non disperdere subito la documentazione: prima si consolidano le forme base, poi si espande.

La catena può essere considerata una linea topologica, dove l'obiettivo principale non è l'allineamento perfetto ma la sequenza di connessioni. Una catena può essere curva, piegata o

flessibile. Questo è molto importante per MicroBot se si immagina una struttura modulare che può adattarsi. La linea è rigida e geometrica; la catena è relazionale. Una catena riuscita richiede che ogni nodo sia connesso al successivo, anche se la forma non è perfettamente dritta.

La stella può essere definita da un nodo centrale e K nodi periferici. La geometria target è radiale; la topologia è un grafo star, con archi dal centro a ogni periferico. È un pattern utile per mostrare gerarchia e centralità. Può rappresentare un controller locale, un nodo hub o una struttura di aggancio radiale. In una dashboard, è molto chiaro visivamente.

Un pattern stella ha topologia:

$$E_star = \{(center, n_1), (center, n_2), ..., (center, n_k)\}$$

La metrica principale è distanza dei nodi periferici dal centro, distribuzione angolare e presenza degli archi radiali. Può essere un buon pattern successivo al cerchio perché condivide la distribuzione angolare ma cambia topologia.

L'anello aperto o arco può essere definito come porzione di cerchio. È utile come transizione da linea a cerchio. Invece di passare direttamente da linea a cerchio chiuso, il sistema può formare un arco e poi chiuderlo. Questo introduce riconfigurazione graduale. Un arco ha centro, raggio, angolo iniziale e angolo finale. La distribuzione angolare avviene solo su un intervallo, non su 2π .

Questi pattern intermedi dimostrano che il vocabolario geometrico può crescere per composizione. Linea, cerchio, griglia e cluster sono le basi; catena, stella e arco sono estensioni; strutture complesse saranno combinazioni di queste.

I pattern geometrici semplici devono essere rappresentati chiaramente nei file e nella dashboard. Ogni pattern dovrebbe avere un'anteprima target, una vista dello stato corrente, una metrica di errore e un log delle fasi. Per la linea, la dashboard può mostrare asse target e nodi correnti. Per il cerchio, centro e raggio. Per la griglia, celle target. Per il cluster, area target e raggio effettivo. Questa visualizzazione è essenziale perché rende il pattern misurabile e comprensibile.

Una dashboard pattern dovrebbe mostrare:

pattern_id;

pattern_run_id;

forma target;

nodi partecipanti;

ruoli;

fase corrente;

errore geometrico;

errore topologico;

stato di stabilità;

risultato finale.

Questi dati permettono di capire se il pattern sta funzionando. In una demo, l'osservatore può vedere la forma desiderata e la forma reale convergere.

I pattern geometrici semplici devono essere compatibili con il controller e il simulatore. Il controller deve poter generare target e comandi. Il simulatore deve poter usare target e regole per aggiornare i nodi virtuali. La dashboard deve poter visualizzare. I nodi fisici devono poter ricevere stati o ruoli coerenti. Questa compatibilità richiede una rappresentazione comune del pattern. Per esempio, un file JSON del pattern linea può essere letto dal controller, dal simulatore e dalla dashboard. Questo evita duplicazioni.

Una struttura dati minima per un pattern linea potrebbe contenere:

pattern_id: line_basic_v0.1;

geometry_type: line;

node_count: 5;

spacing: 0.10;

direction: [1,0,0];

topology_type: chain;

roles: leader, connector, terminal;

phases: prepare, align, spacing, stabilize;

success_metrics: lateral_error, spacing_error, topology_score.

Una struttura per cerchio potrebbe contenere:

pattern_id: circle_basic_v0.1;

geometry_type: circle;

radius: 0.20;

node_count: 8;

topology_type: cycle;

roles: anchor, ring_member;

success_metrics: radial_error, angular_variance, closure_score.

Questi file non devono necessariamente essere implementati subito, ma la loro struttura concettuale è utile per mantenere coerenza nel progetto.

I pattern geometrici semplici devono anche essere usati come test di regressione. Se il sistema cambia codice, protocollo o simulatore, si può verificare che linea, cerchio, griglia e cluster funzionino ancora. Questi pattern diventano test base. Se una nuova modifica rompe il pattern linea, significa che qualcosa nella coordinazione, nel calcolo target o nella comunicazione non va. Questo rende i pattern strumenti di validazione, non solo demo.

Una suite di test pattern può includere:

line_basic con 3 nodi;

line_basic con 5 nodi;

circle_basic con 6 nodi;

grid_basic 2x3;

cluster_basic con 10 nodi;

pattern con nodo mancante;

pattern con nodo delayed;

pattern con nodo in safety.

Questi test permettono di verificare sia comportamento ideale sia robustezza. Sono molto coerenti con la visione di un sistema documentato e ingegnerizzato.

I pattern geometrici semplici devono essere collegati alla correzione degli errori. Se una linea ha errore laterale alto, il controller o il simulatore può aumentare il peso verso la linea. Se un cerchio ha distribuzione angolare irregolare, può aumentare separazione tangenziale. Se una griglia è deformata, può correggere righe e colonne. Se un cluster è troppo disperso, può aumentare coesione. Quindi ogni pattern deve suggerire anche strategie di correzione.

Per la linea:

errore laterale alto → aumentare attrazione verso asse;

spacing irregolare → regolare distanza tra vicini;

ordine errato → riassegnare target o ruoli;

connessione mancante → ritentare attach tra coppie previste.

Per il cerchio:

errore radiale alto → correggere raggio;

angoli irregolari → aumentare distribuzione angolare;

chiusura mancante → correggere coppia ultimo-primo;

collisioni → aumentare separazione.

Per la griglia:

celle vuote → riassegnare nodo;

righe deformate → correggere allineamento orizzontale;

colonne deformate → correggere allineamento verticale;

topologia incompleta → ritentare connessioni locali.

Per il cluster:

dispersione alta → aumentare coesione;

collisioni → aumentare separazione;

nodi isolati → attrazione verso nearest cluster;

oscillazioni → aumentare smorzamento.

Queste strategie saranno approfondite nella correzione degli errori, ma già nei pattern semplici si vede il collegamento tra metrica e azione correttiva.

Dal punto di vista teorico, i pattern geometrici semplici sono i mattoni base della forma programmabile. Una materia programmabile complessa non nasce direttamente da forme arbitrarie, ma da primitive: linee, superfici, cluster, anelli, connessioni, reticoli. MicroBot deve quindi possedere un vocabolario di primitive geometriche. Ogni primitiva deve essere definita, eseguita, misurata e corretta. Quando queste primitive sono stabili, si possono combinare per costruire strutture più complesse.

Una struttura complessa può essere vista come composizione:

struttura = linea + arco + cluster + connessioni

oppure:

superficie = molte linee parallele + connessioni verticali

oppure:

reticolo = griglia + stabilizzazione + topologia

Questa composizione è il passaggio naturale verso il punto successivo, dedicato alle strutture complesse. Senza primitive semplici affidabili, le strutture complesse sarebbero fragili e difficili da debuggare.

I pattern geometrici semplici devono essere introdotti anche nella narrazione del progetto come milestone. Una roadmap efficace potrebbe dichiarare che MicroBot deve prima dimostrare pattern base in simulazione, poi pattern base su dashboard, poi pattern base con feedback LED, poi pattern base con piccoli prototipi fisici, poi strutture più complesse. Questo rende lo sviluppo leggibile a un osservatore esterno. Non sembra un progetto disperso, ma una progressione ordinata.

Una possibile progressione è:

linea simulata;

cerchio simulato;

cluster simulato;

griglia simulata;

linea visuale con LED fisici;
cerchio visuale in dashboard più nodi reali come indicatori;
catena topologica simulata;
primi test fisici di connessione tra due nodi;
struttura mista simulata-fisica.

Questa sequenza è realistica e coerente con ciò che è stato costruito finora nel progetto MicroBot.

In conclusione, i pattern geometrici semplici costituiscono il primo vocabolario operativo dello swarm MicroBot. Linea, cerchio, griglia e cluster permettono di testare geometria, topologia, ruoli, sincronizzazione, comunicazione, dashboard, metriche e correzione degli errori in forme comprensibili e misurabili. La linea introduce ordine monodimensionale e catena; il cerchio introduce chiusura e distribuzione angolare; la griglia introduce superficie e regolarità bidimensionale; il cluster introduce coesione emergente e separazione. Questi pattern non sono semplici figure, ma primitive tecniche complete, ciascuna con target, ruoli, fasi, metriche, vincoli e fallback. Una volta consolidati, essi diventano la base per strutture complesse, riconfigurazioni dinamiche e comportamenti adattivi. Il punto successivo potrà quindi analizzare le strutture complesse, cioè il modo in cui più pattern semplici possono essere combinati per generare forme più ricche, modulari e vicine alla visione di MicroBot come materia programmabile distribuita.

10.3 Strutture complesse

Le strutture complesse rappresentano il livello successivo rispetto ai pattern geometrici semplici, perché nascono dalla combinazione di più primitive come linee, cerchi, griglie, cluster, catene, archi e connessioni topologiche. Dopo aver definito le forme fondamentali dello swarm, è necessario descrivere come MicroBot possa usarle come mattoni per costruire configurazioni più ricche, modulari e vicine al concetto di materia programmabile. Una struttura complessa non è semplicemente una forma più grande: è una configurazione composta da sottostrutture, ruoli differenziati, connessioni multiple, fasi temporali coordinate, vincoli fisici e metriche di stabilità più articolate.

Nel progetto MicroBot, una struttura complessa può essere vista come un pattern composto. Se una linea è una primitiva monodimensionale, una griglia può essere vista come molte linee parallele collegate tra loro. Se un cerchio è una primitiva chiusa, una struttura ad anello rinforzato può essere vista come più cerchi concentrici o come un ciclo con nodi connector. Se un cluster è una massa compatta, una struttura modulare può essere vista come un insieme di cluster collegati da catene o ponti. Questa logica compositiva è fondamentale perché permette di non progettare ogni forma da zero. Il sistema può costruire forme complesse combinando primitive già definite, testate e misurate.

Una struttura complessa può essere formalmente descritta come un insieme di sub-pattern collegati tra loro. In forma concettuale:

$$S = \{P_1, P_2, P_3, \dots, P_k, C\}$$

dove P_1, P_2, P_3 fino a P_k rappresentano i pattern componenti, mentre C rappresenta l'insieme delle connessioni, vincoli e relazioni tra tali pattern. Per esempio, una struttura “ponte” può essere composta da due cluster laterali e una catena centrale. Una struttura “reticolo” può essere composta da linee orizzontali e verticali. Una struttura “mano” o “arto articolato” può essere composta da una

base, segmenti lineari e giunti. Una struttura “scudo” o “superficie” può essere composta da una griglia più un bordo rinforzato.

Questa idea è molto importante per MicroBot perché avvicina il progetto alla visione di programmable matter. Una materia programmabile non si limita a formare figure elementari; deve poter costruire strutture con parti, funzioni, bordi, interni, giunti, connessioni e zone differenziate. In una struttura complessa, ogni area può avere un ruolo diverso. Alcuni nodi possono formare il nucleo, altri il bordo, altri connessioni, altri stabilizzatori, altri sensori, altri indicatori o nodi di controllo. Questo porta il sistema oltre il semplice pattern geometrico e lo avvicina a un organismo modulare.

Il primo esempio di struttura complessa è il reticolo. Un reticolo può essere considerato come una griglia con topologia esplicita e vincoli di stabilità. Una semplice griglia geometrica richiede che i nodi occupino posizioni regolari; un reticolo richiede anche che i nodi siano collegati secondo archi orizzontali, verticali o diagonali. Questo significa che la struttura non viene valutata soltanto per la sua forma, ma anche per la sua connettività. Un reticolo può avere nodi interni, nodi di bordo, nodi angolari e nodi di rinforzo. Ogni categoria ha ruolo diverso e grado topologico diverso.

In un reticolo quadrato, i nodi interni possono avere quattro connessioni principali: sopra, sotto, sinistra e destra. I nodi di bordo ne hanno tre o due. I nodi angolari ne hanno due. In una versione rinforzata, possono esistere anche connessioni diagonali. Questo rende il grafo più resistente a distacchi o errori. Dal punto di vista del pattern, il reticolo può essere definito da righe, colonne, passo, topologia desiderata e regole di connessione. Dal punto di vista comportamentale, deve mantenere distanze regolari e impedire deformazioni eccessive.

Le metriche di un reticolo sono più complesse di quelle della griglia semplice. Non basta misurare la distanza dei nodi dai target; bisogna misurare anche regolarità delle celle, deformazione angolare, connettività, presenza di archi, stabilità dei bordi e capacità di mantenere la forma dopo piccoli disturbi. Una struttura reticolare può essere considerata riuscita se mantiene la geometria globale, conserva le connessioni previste e non sviluppa zone isolate o collassate. In questo senso, il reticolo è una struttura utile per dimostrare stabilità, non solo disposizione.

Un secondo esempio è la struttura a ponte. Essa può essere composta da due cluster laterali collegati da una catena centrale. I cluster rappresentano basi o supporti; la catena rappresenta il collegamento. Questa struttura è interessante perché introduce il concetto di nodo critico. Se un nodo centrale della catena fallisce, il ponte può rompersi. Se un nodo interno a un cluster fallisce, la struttura può restare parzialmente stabile. Quindi non tutti i nodi hanno la stessa importanza. Questo porta al concetto di criticità topologica.

Una struttura a ponte può essere definita così:

cluster_A = gruppo sinistro;

cluster_B = gruppo destro;

bridge_chain = sequenza di nodi connector;

edges = connessioni interne ai cluster + connessioni della catena + connessioni tra catena e cluster.

Il controller deve coordinare prima la formazione dei cluster, poi la catena, poi l’aggancio tra catena e cluster. Oppure può fare il contrario: formare prima una linea centrale e poi aggregare i cluster ai lati. Queste due strategie possono produrre risultati diversi. La struttura complessa non è solo “che forma finale voglio”, ma “in quale ordine conviene costruirla”. Questo è un punto fondamentale.

Il terzo esempio è la struttura ad anello rinforzato. Essa deriva dal cerchio, ma aggiunge connessioni supplementari o nodi interni. Un cerchio semplice è un ciclo; se un arco si rompe, l'anello perde chiusura. Un anello rinforzato può avere nodi interni che collegano parti opposte, oppure un secondo anello concentrico, oppure connessioni diagonali. Questa struttura introduce ridondanza. La ridondanza è molto importante nei sistemi distribuiti perché permette alla struttura di tollerare il fallimento di alcuni nodi o connessioni.

Un anello rinforzato può avere:

outer_ring: nodi sul bordo esterno;

inner_ring: nodi interni o secondario;

radial_edges: connessioni tra anello interno ed esterno;

stabilizer_nodes: nodi che riducono deformazione.

Questa struttura può essere utile per simulare superfici curve, bordi strutturali, contenitori o moduli chiusi. Dal punto di vista visivo, è molto efficace nella dashboard perché mostra chiaramente il passaggio da pattern semplice a struttura più robusta. Dal punto di vista tecnico, permette di testare topologia, ruoli e stabilità in modo più avanzato.

Il quarto esempio è la struttura modulare a cluster collegati. In questo caso, non si cerca una singola forma continua, ma un insieme di gruppi connessi da relazioni. Ogni cluster può avere forma interna compatta, mentre le connessioni tra cluster possono essere catene, ponti o archi. Questa struttura è molto importante perché introduce una gerarchia. I nodi formano cluster; i cluster formano una struttura superiore. Il comportamento può quindi essere coordinato su due livelli: locale dentro il cluster e globale tra cluster.

Una struttura gerarchica può essere rappresentata così:

livello nodo: ogni MicroBot ha stato, ruolo e vicini;

livello cluster: gruppi di MicroBot formano sottostrutture;

livello struttura: i cluster vengono collegati in una forma più grande.

Questa gerarchia è molto vicina a come potrebbero funzionare sistemi di swarm più grandi. Non si può controllare ogni nodo singolarmente in strutture enormi; bisogna controllare gruppi. Il concetto di cluster collegati prepara quindi la scalabilità del progetto. Una dashboard potrebbe mostrare sia i singoli nodi sia i cluster come unità aggregate.

Il quinto esempio è la struttura articolata. Una struttura articolata è composta da segmenti e giunti. Un segmento può essere una catena o una linea rinforzata; un giunto può essere un cluster piccolo o una connessione flessibile tra segmenti. Questo tipo di struttura è interessante perché introduce movimento relativo tra parti. Non tutto deve essere rigido. Una struttura articolata può piegarsi, ruotare, aprirsi o chiudersi. Anche se nelle prime versioni questo sarà soprattutto simulativo, è molto coerente con la visione futura di MicroBot.

Una struttura articolata può avere:

segment_A;

joint_01;

segment_B;

joint_02;

segment_C.

Ogni segmento ha stabilità interna; ogni giunto permette variazione angolare. Le metriche non misurano solo posizione dei nodi, ma anche angoli tra segmenti, rigidità relativa, libertà del giunto e stabilità della connessione. Questo apre la strada a comportamenti più complessi come bracci modulari, superfici pieghevoli, strutture che cambiano forma e configurazioni adattive. È un livello più avanzato, ma importante nella documentazione strategica.

Il sesto esempio è la superficie modulare. Una superficie può essere costruita come griglia o reticolo bidimensionale, con bordi e interno. I nodi del bordo mantengono forma esterna; i nodi interni stabilizzano densità e connessioni. Questa struttura è particolarmente legata al concetto di materia programmabile, perché una superficie modulare può rappresentare un materiale che cambia profilo. In simulazione, può essere una membrana di nodi. In hardware futuro, potrebbe essere una rete di moduli che si agganciano magneticamente o meccanicamente.

Una superficie modulare richiede metriche avanzate:

errore di bordo;

uniformità interna;

connettività locale;

assenza di buchi indesiderati;

stabilità della superficie;

deformazione rispetto alla forma target.

Rispetto a linea e cerchio, una superficie introduce molti più vincoli. Per questo non deve essere il primo pattern fisico, ma è una direzione naturale dopo griglie e reticoli simulati.

Le strutture complesse richiedono una gestione esplicita dei ruoli. Nei pattern semplici, i ruoli possono essere pochi: leader, follower, anchor, terminal. Nelle strutture complesse, servono ruoli più specifici: core_node, boundary_node, connector_node, bridge_node, joint_node, stabilizer_node, sensor_node, actuator_node, passive_node. Ogni ruolo definisce un comportamento locale diverso e una diversa importanza nella struttura. Un connector_node può essere critico per mantenere due sottostrutture collegate. Un boundary_node può essere importante per mantenere la forma esterna. Un core_node può contribuire alla stabilità interna. Un passive_node può partecipare alla forma senza attuazione.

Questa classificazione permette al controller di ragionare in modo più intelligente. Se un core_node fallisce, il sistema può tollerarlo forse meglio che se fallisce un bridge_node. Se un boundary_node è fuori posizione, la forma esterna si degrada. Se un connector_node è in safety, due cluster possono separarsi. Quindi il ruolo influenza il fallback. Le strutture complesse rendono necessaria una mappa delle criticità.

Una mappa di criticità può classificare i nodi così:

critical: essenziale per la struttura;

important: utile ma sostituibile;

optional: migliora forma ma non essenziale;

passive: partecipa senza azioni attive;

diagnostic: osserva o misura.

Questa informazione è molto utile per decidere cosa fare in caso di errore. Se un nodo critical fallisce, la run può essere abortita. Se un nodo optional fallisce, la struttura può degradare ma continuare. Questo rende il sistema più robusto e realistico.

Le strutture complesse richiedono anche topologie più articolate. Nei pattern semplici, la topologia è lineare, ciclica, griglia o cluster. Nelle strutture complesse, la topologia può essere gerarchica, ridondante, modulare o mista. Per esempio, una struttura può avere un nucleo altamente connesso e bracci lineari poco connessi. Oppure può avere un bordo ciclico e un interno reticolare. Oppure può avere più cluster collegati da ponti. Queste topologie devono essere descritte come grafi.

Un grafo strutturale complesso può contenere:

nodi;

archi fisici o virtuali;

archi desiderati;

archi vietati;

peso degli archi;

criticità degli archi;

tipo di connessione.

Gli archi non sono tutti uguali. Una connessione di bordo può avere funzione diversa da una connessione interna. Una connessione temporanea di fase può essere diversa da una connessione strutturale permanente. Una connessione di comunicazione può essere diversa da una connessione magnetica. Il pattern deve quindi specificare il tipo di arco, non solo la coppia di nodi.

Una struttura complessa può avere archi di tipo:

structural_edge;

temporary_edge;

alignment_edge;

magnetic_edge;

visual_edge;

communication_edge;

forbidden_edge.

Il `forbidden_edge` è particolarmente interessante: indica che due nodi non dovrebbero agganciarsi o avvicinarsi troppo. Nei sistemi fisici, evitare connessioni indesiderate è importante quanto creare connessioni desiderate. Se lo swarm forma archi sbagliati, la struttura finale può essere topologicamente errata anche se la geometria sembra accettabile. Questo è un problema molto reale nella materia programmabile.

Le strutture complesse richiedono quindi metriche topologiche più precise. Non basta misurare quanti archi desiderati sono presenti; bisogna misurare anche quanti archi indesiderati sono presenti. Una struttura può avere tutte le connessioni richieste ma anche connessioni extra che la rendono rigida o sbagliata. Per esempio, in una catena, una connessione tra nodo 1 e nodo 4 può creare un ciclo indesiderato. In un braccio articolato, una connessione extra può impedire il movimento del giunto. Quindi la metrica deve considerare archi mancanti ed archi extra.

Una metrica topologica può essere:

$$\text{topology_score} = \text{archi_corretti} - \text{penalità_archi_mancanti} - \text{penalità_archi_extra}$$

Questa formula può essere raffinata, ma il principio è chiaro. La qualità topologica dipende sia dalla presenza del necessario sia dall'assenza dell'indesiderato. Questo è fondamentale per strutture complesse.

Le strutture complesse richiedono una costruzione per fasi. Non è realistico tentare di formare tutta la struttura in un singolo comando. Il controller deve decomporre la struttura in sottostrutture e fasi. Prima si costruiscono primitive stabili, poi si collegano, poi si stabilizzano, poi eventualmente si rinforzano. Questa decomposizione rende il comportamento più controllabile e debuggabile. Se qualcosa fallisce, si sa in quale sottostruttura o fase è avvenuto il problema.

Una strategia generale può essere:

fase 1: selezione dei nodi e verifica readiness;

fase 2: assegnazione di gruppi e ruoli;

fase 3: costruzione delle primitive locali;

fase 4: collegamento tra primitive;

fase 5: stabilizzazione della topologia globale;

fase 6: verifica geometrica e topologica;

fase 7: correzione o fallback;

fase 8: report finale.

Questa struttura è applicabile a molti pattern complessi. Per un ponte, le primitive sono cluster e catena. Per un reticolo, le primitive sono linee. Per una superficie, le primitive sono righe, colonne e bordo. Per una struttura articolata, le primitive sono segmenti e giunti. Il principio di decomposizione resta lo stesso.

Le strutture complesse possono essere rappresentate tramite una gerarchia di pattern. Invece di avere un unico file gigantesco che definisce tutto, si può avere un pattern principale che richiama sub-pattern. Per esempio:


```
structure_bridge_v0.1

include cluster_left_basic;

include cluster_right_basic;

include chain_connector_basic;

define interconnections.
```

Questa organizzazione è molto utile per mantenere la documentazione pulita. I sub-pattern possono essere testati separatamente. Se cluster_basic migliora, tutte le strutture che lo usano possono beneficiarne. Questo è simile alla programmazione modulare: i pattern diventano componenti riutilizzabili.

Una struttura dati potrebbe essere:

```
structure_id;

subpatterns;

subpattern_roles;

interconnections;

global_phases;

global_metrics;

fallback_policy.
```

Questa struttura permette di definire pattern complessi in modo ordinato. È molto coerente con l'approccio da repository tecnico: ogni pattern ha file, descrizione, test, metriche e versionamento.

Le strutture complesse devono essere versionate ancora più attentamente dei pattern semplici. Una piccola modifica nella topologia può cambiare completamente il comportamento. Per esempio, aggiungere una connessione diagonale a un reticolo può aumentare stabilità ma ridurre flessibilità. Cambiare un nodo da optional a critical può modificare la policy di fallback. Cambiare l'ordine delle fasi può migliorare o peggiorare la convergenza. Ogni versione deve quindi essere documentata.

Una struttura può avere:

```
structure_id = "bridge_cluster_chain_v0.1";

version = "0.1";

status = "simulation_only";

notes = "composed of two clusters and one connector chain";

required_nodes = 10;

degradable = true.
```

Nelle run, bisogna sempre salvare lo `structure_id` e la versione. Questo permette di confrontare esperimenti e non confondere risultati di versioni diverse.

Le strutture complesse richiedono anche una distinzione tra forma globale e qualità locale. Una struttura può apparire globalmente corretta ma avere errori locali. Per esempio, una griglia può sembrare rettangolare ma avere una cella deformata. Un ponte può collegare due cluster ma avere un nodo centrale instabile. Un anello può essere chiuso ma avere un tratto troppo compresso. Quindi le metriche devono essere sia globali sia locali.

Metriche globali possono essere:

errore rispetto alla forma complessiva;

connettività generale;

stabilità della struttura;

numero di nodi partecipanti;

tempo di completamento.

Metriche locali possono essere:

errore per nodo;

errore per arco;

stabilità per connessione;

stress locale;

violazioni di distanza minima;

nodi con ruolo non rispettato.

Questa doppia analisi è molto utile nella dashboard. La dashboard può mostrare `pattern_error` globale, ma anche evidenziare nodi o connessioni problematiche. Così il debug diventa più preciso.

Le strutture complesse devono gestire la ridondanza. Una struttura ridondante può tollerare la perdita di un nodo o di un arco senza collassare. Una struttura non ridondante può essere più leggera ma fragile. Il progetto MicroBot dovrebbe distinguere tra strutture minime e strutture robuste. Una linea semplice è fragile: se un nodo centrale fallisce, la catena si divide. Una linea con connessioni di backup o un reticolo è più robusta. Una struttura con cluster e ponti multipli può tollerare più errori. Questa distinzione è importante per progettare comportamenti adattivi.

Un parametro utile può essere:

`redundancy_level` = low / medium / high

Una struttura con `redundancy_level` alto richiede più nodi e più connessioni, ma può essere più stabile. Una struttura con `redundancy_level` basso è più semplice e meno costosa, ma meno tollerante. Il controller può scegliere in base al contesto: demo rapida, simulazione, hardware limitato, missione sicura, test di robustezza.

Le strutture complesse richiedono anche una gestione delle risorse. Non tutti i nodi possono attivare tutte le funzioni nello stesso momento. Alcuni possono avere coil, altri solo LED. Alcuni possono avere batteria migliore. Alcuni possono essere più affidabili in rete. Il controller deve assegnare ruoli tenendo conto di capabilities e qualità. Un nodo con rete stabile può essere scelto come anchor. Un nodo con batteria bassa può essere ruolo passivo. Un nodo con coil può essere connector attivo. Questa assegnazione rende la struttura più realistica.

Una funzione di assegnazione può considerare:

capabilities;
sync_quality;
battery_level;
thermal_state;
network_status;
previous_errors;
required_role.

Il ruolo non deve essere assegnato casualmente se ci sono vincoli. Questo è importante soprattutto per strutture complesse, dove alcuni ruoli sono critici. Un bridge_node deve essere affidabile; un stabilizer_node deve avere capacità e stato adatti; un sensor_node deve avere telemetria valida.

Le strutture complesse devono essere compatibili con la modalità ibrida simulazione-hardware. Per esempio, una struttura complessa può essere simulata interamente nella dashboard, mentre solo alcuni nodi fisici rappresentano parti chiave tramite LED. Oppure due nodi fisici possono rappresentare connector reali e il resto simulato. Questa modalità ibrida è molto utile durante lo sviluppo perché permette di dimostrare l'architettura anche prima di avere molti moduli fisici. La struttura complessa esiste nel modello e nel runtime; l'hardware fisico può essere introdotto gradualmente.

Una demo ibrida potrebbe essere:

simulazione di 30 nodi che formano un reticolo;
3 ESP32 fisici rappresentano nodi anchor/connector;
dashboard mostra struttura completa;
LED fisici indicano fase e ruolo;
telemetria fisica viene integrata nella vista.

Questa demo sarebbe potente perché mostra scalabilità concettuale e collegamento al mondo reale senza richiedere subito 30 moduli fisici. È una strategia molto intelligente per presentare MicroBot.

Le strutture complesse devono prevedere fallback strutturali. Se una sottostruttura fallisce, il sistema può adattarsi. Per esempio, se un ponte non riesce a formare una catena completa, può formare un ponte più corto, un collegamento alternativo o dichiarare partial_success. Se un reticolo perde una riga, può diventare reticolo ridotto. Se un anello non chiude, può diventare arco stabile.

Se un cluster non si collega a un altro, può restare cluster separato. Questi fallback devono essere definiti nel pattern, non improvvisati.

Esempi di fallback:

circle → arc se chiusura fallisce;

grid 3x3 → grid 2x3 se una riga fallisce;

bridge → two clusters unconnected se catena fallisce;

reticolo rinforzato → reticolo base se diagonali falliscono;

struttura articolata → segmenti separati in idle se giunto fallisce.

Questi fallback rendono il sistema più comunicabile. Una run non deve essere solo successo/fallimento; può avere una forma degradata ma documentata. Questo è molto importante per la maturità sperimentale.

Le strutture complesse richiedono anche correzione multi-livello. Se un errore avviene in un nodo, si può correggere localmente. Se avviene in una connessione, si può correggere l'arco. Se avviene in un sub-pattern, si può correggere il gruppo. Se avviene nella struttura globale, si può cambiare fase o fallback. Questa gerarchia di correzione è fondamentale.

Un errore locale può essere:

MB-004 fuori posizione;

connessione MB-004—MB-005 mancante;

cluster_A troppo disperso;

ponte non connesso;

reticolo deformato globalmente.

Ogni livello richiede una correzione diversa. Non si deve sempre correggere tutto lo swarm per un errore locale. La struttura complessa permette correzioni mirate se la gerarchia è definita bene.

Le strutture complesse devono essere registrate nei log con una struttura gerarchica. Un log piatto di migliaia di eventi diventa difficile da interpretare. È meglio collegare eventi a structure_run_id, subpattern_id, phase_id, node_id e edge_id. Così si può filtrare: mostra tutti gli eventi del bridge_chain; mostra errori del cluster_A; mostra connessioni mancanti nel reticolo; mostra safety nei boundary_node. Questo rende il debug molto più efficace.

Un evento può contenere:

structure_run_id;

subpattern_id;

phase_id;

node_id opzionale;

edge_id opzionale;

event_type;

timestamp;

severity;

result.

Questa organizzazione è molto coerente con la visione di documentazione tecnica estesa. Permette di analizzare strutture complesse senza perdersi.

Le strutture complesse devono essere spiegabili nella dashboard. La visualizzazione deve mostrare non solo nodi, ma anche sub-pattern, ruoli e connessioni. Per esempio, in un ponte, la dashboard può colorare i due cluster e la catena centrale. In un reticolo, può evidenziare righe, colonne e celle deformate. In un anello rinforzato, può mostrare outer_ring, inner_ring e radial_edges. Questa visualizzazione aiuta a capire il comportamento. Senza gerarchia visiva, una struttura complessa può sembrare solo un ammasso di punti.

Una dashboard strutturale può includere:

vista globale;

vista sub-pattern;

vista topologica;

vista ruoli;

vista errori locali;

vista timeline delle fasi;

metriche globali e locali.

Questa dashboard sarebbe molto utile per dimostrare MicroBot come ecosistema serio. Non è solo grafica bella; è strumento di analisi.

Le strutture complesse introducono anche il concetto di vincoli tra sub-pattern. Per esempio, due cluster collegati da un ponte devono mantenere una distanza compatibile con la lunghezza del ponte. Un anello interno deve restare concentrico con quello esterno. Un giunto deve mantenere un angolo entro certi limiti. Una superficie deve mantenere continuità tra celle. Questi vincoli sono diversi dalle forze locali; sono relazioni tra parti.

Un vincolo tra sub-pattern può essere:

$\text{distance_between}(\text{cluster_A}, \text{cluster_B}) \approx L_{\text{bridge}};$

$\text{angle_between}(\text{segment_A}, \text{segment_B}) \in [\theta_{\text{min}}, \theta_{\text{max}}];$

$\text{center}(\text{inner_ring}) \approx \text{center}(\text{outer_ring});$

$\text{boundary}(\text{surface}) \text{ follows } \text{target_shape}.$

Questi vincoli permettono di valutare la struttura a livello superiore. Una struttura complessa può avere sub-pattern corretti individualmente ma vincoli globali errati. Per esempio, due cluster possono essere compatti ma troppo lontani per il ponte. Quindi serve valutazione multilivello.

Le strutture complesse possono essere statiche o dinamiche. Una struttura statica viene costruita e mantenuta. Una struttura dinamica cambia forma mantenendo alcune relazioni. Per esempio, una catena può piegarsi; un reticolo può deformarsi; un anello può espandersi; una superficie può cambiare curvatura; due cluster possono separarsi e riunirsi. Questo livello prepara la riconfigurazione dinamica, che sarà un punto importante successivo. La struttura complessa non è necessariamente finale: può essere uno stato intermedio di una trasformazione.

Una struttura dinamica può essere definita come:

$S(t)$ = insieme di sub-pattern e connessioni che variano nel tempo

Alcuni vincoli restano costanti, altri cambiano. Per esempio, una catena può mantenere topologia ma cambiare geometria. Un anello può mantenere chiusura ma cambiare raggio. Un reticolo può mantenere connessioni ma deformarsi. Questa distinzione tra topologia stabile e geometria variabile è molto importante. Una struttura può cambiare forma senza cambiare connessioni, oppure può cambiare connessioni per riconfigurarsi.

Le strutture complesse devono considerare anche il problema della rigidità. Troppe connessioni possono rendere una struttura stabile ma difficile da riconfigurare. Poche connessioni la rendono flessibile ma fragile. Il sistema deve bilanciare rigidità e adattabilità. Questo è un tema centrale per la materia programmabile. Una struttura deve essere abbastanza stabile da mantenere forma, ma abbastanza flessibile da poter cambiare. I pattern complessi devono quindi dichiarare il livello di rigidità desiderato.

Un parametro può essere:

`rigidity_level = flexible / semi_rigid / rigid`

Una catena può essere flexible. Un reticolo rinforzato può essere rigid. Una superficie adattiva può essere semi_rigid. Questo parametro può influenzare topologia, numero di connessioni, peso di stabilizzazione e fallback. È un concetto molto utile per spiegare la differenza tra strutture.

Le strutture complesse richiedono una distinzione tra connessioni permanenti e temporanee. Una connessione permanente è parte della struttura finale. Una connessione temporanea può servire durante la costruzione, ma poi essere rilasciata. Per esempio, durante la formazione di un ponte, alcuni nodi possono usare connessioni temporanee di allineamento che poi scompaiono. Questo è simile ai supporti temporanei in costruzione. Dal punto di vista dello swarm, è molto interessante perché permette di costruire forme difficili con fasi intermedie.

Un pattern complesso può quindi definire:

`final_edges;`

`temporary_edges;`

`forbidden_edges;`

`transition_edges.`

Le `temporary_edges` sono valide solo in alcune fasi. Le `final_edges` devono restare alla fine. Le `forbidden_edges` non devono formarsi. Le `transition_edges` possono essere usate durante riconfigurazione. Questa classificazione rende la topologia molto più espressiva.

Le strutture complesse possono essere usate anche per testare robustezza. Un test può rimuovere virtualmente un nodo e vedere se la struttura resta connessa. Può introdurre delay in un connector e vedere se il ponte fallisce. Può aumentare jitter in un cluster e vedere se si disallinea. Può simulare `thermal_lock` su un nodo critico. Questi test mostrano quanto la struttura sia tollerante. È un passo molto importante verso sistemi affidabili.

Test possibili:

rimozione di un nodo optional;

rimozione di un nodo critical;

perdita di una connessione interna;

perdita di un bridge edge;

nodo delayed durante fase attach;

nodo safety durante stabilizzazione;

configurazione con nodi insufficienti;

fallback da struttura completa a struttura ridotta.

Questi test possono essere eseguiti prima in simulazione. I risultati possono diventare report tecnici. Questo rafforza molto il valore del progetto.

Le strutture complesse devono essere introdotte gradualmente nella roadmap. Prima si consolidano pattern semplici. Poi si costruisce una struttura composta da due linee. Poi una griglia. Poi un reticolo. Poi cluster collegati. Poi strutture articolate. Poi superfici. Questo ordine è importante perché ogni livello dipende dal precedente. Saltare direttamente a strutture avanzate rischia di creare confusione e fragilità.

Una roadmap possibile è:

v0.1: linea e catena;

v0.2: cerchio e ciclo;

v0.3: griglia semplice;

v0.4: reticolo con topologia;

v0.5: cluster collegati;

v0.6: ponte cluster-catena-cluster;

v0.7: anello rinforzato;

v0.8: struttura articolata;

v0.9: superficie modulare simulata;

v1.0: riconfigurazione tra strutture.

Questa progressione è chiara, dimostrabile e tecnicamente sensata.

Dal punto di vista del repository, le strutture complesse possono avere una cartella dedicata. Per esempio, dentro una repo di simulazione o pattern si potrebbe avere:

patterns/simple/line_basic.json;

patterns/simple/circle_basic.json;

patterns/simple/grid_basic.json;

patterns/simple/cluster_basic.json;

patterns/complex/bridge_cluster_chain.json;

patterns/complex/reinforced_ring.json;

patterns/complex/lattice_surface.json;

patterns/complex/articulated_segment.json.

Ogni file dovrebbe avere documentazione associata, metriche e test. Questo renderebbe il progetto ordinato e facilmente presentabile. Non è solo codice: è un catalogo di comportamenti.

Le strutture complesse devono anche essere collegate alla narrativa tecnica del progetto. Quando si presenta MicroBot, si può spiegare che il sistema parte da primitive geometriche e poi le combina. Questa è una narrazione forte perché mostra metodo. Non si sta promettendo una forma impossibile dal nulla; si sta costruendo un linguaggio progressivo. Prima il nodo. Poi la comunicazione. Poi la sincronizzazione. Poi la linea. Poi il cerchio. Poi la griglia. Poi il reticolo. Poi strutture modulari. Questo percorso è credibile e potente.

Una frase concettuale efficace è: MicroBot non costruisce forme complesse come immagini isolate, ma come composizioni di primitive geometriche e topologiche, coordinate nel tempo da un controller e validate da metriche di stabilità.

Questa frase sintetizza bene il senso della sezione.

Le strutture complesse devono restare compatibili con i limiti attuali. È importante distinguere tra strutture implementate, simulate, visualizzate e future. Un reticolo nella dashboard non equivale a un reticolo fisico. Una struttura magnetica simulata non equivale a un hardware già pronto. Però definire queste strutture nel modello è comunque utile perché prepara il sistema. La documentazione deve quindi essere precisa:

simulation_ready: eseguibile nel simulatore;

dashboard_ready: visualizzabile;

protocol_ready: descrivibile nei pacchetti;

hardware_partial: testabile in parte;

hardware_future: richiede sviluppo fisico.

Questa classificazione mantiene onestà tecnica e allo stesso tempo valorizza il lavoro fatto. Non si deve nascondere che alcune parti sono future, ma si deve mostrare che sono già previste nell'architettura.

In conclusione, le strutture complesse rappresentano il passaggio dai pattern geometrici semplici alla materia programmabile modulare. Esse nascono dalla composizione di primitive come linee, cerchi, griglie e cluster, ma aggiungono gerarchia, ruoli, topologie articolate, vincoli tra sottostrutture, connessioni temporanee, ridondanza, rigidità controllata, fallback e metriche multilivello. Una struttura complessa non è solo più grande: è più organizzata. Essa possiede parti, relazioni, priorità e strategie di costruzione. Nel sistema MicroBot, le strutture complesse permettono di passare da demo semplici a configurazioni più realistiche, come reticoli, ponti, anelli rinforzati, cluster collegati, segmenti articolati e superfici modulari. Il punto successivo potrà quindi approfondire l'assegnazione dei ruoli, cioè il modo in cui i singoli MicroBot vengono specializzati all'interno di pattern e strutture per rendere possibile una coordinazione più efficiente, robusta e adattiva.

10.4 Assegnazione dei ruoli

L'assegnazione dei ruoli rappresenta il meccanismo attraverso cui i MicroBot vengono differenziati all'interno di un pattern o di una struttura, così che ogni nodo non sia soltanto una particella anonima dello swarm, ma una unità con una funzione specifica. Dopo aver definito i pattern geometrici semplici e le strutture complesse, diventa necessario spiegare come il sistema decida quale MicroBot deve comportarsi da leader, follower, connector, boundary, anchor, core, bridge, terminal, stabilizer o nodo diagnostico. In uno swarm realmente utile, infatti, non tutti i nodi devono fare la stessa cosa. Alcuni devono guidare la formazione, altri devono mantenere distanza, altri devono chiudere connessioni, altri devono stabilizzare bordi, altri devono restare passivi o monitorare lo stato del sistema. L'assegnazione dei ruoli è quindi il livello che trasforma un gruppo uniforme di nodi in una struttura organizzata.

Nel progetto MicroBot, il ruolo deve essere distinto in modo rigoroso dall'identità del nodo. Il `node_id` identifica stabilmente il MicroBot, per esempio MB-001, MB-002 o SIM-004. Il ruolo, invece, indica la funzione temporanea che quel nodo assume durante una run, un pattern o una fase. MB-001 può essere leader in una linea, anchor in un cerchio, connector in un ponte, oppure nodo passivo in una griglia. L'identità resta la stessa; il ruolo cambia in base al contesto. Questa distinzione è fondamentale perché permette di mantenere tracciabilità storica del nodo senza bloccarlo sempre nella stessa funzione.

In forma sintetica:

`node_id` = identità stabile del dispositivo;

`role` = funzione operativa assegnata nella run corrente.

Questa separazione è importantissima per evitare confusione nei log, nella dashboard e nella documentazione. Se un nodo viene chiamato "leader" come se fosse la sua identità permanente, diventa difficile capire cosa accade quando quel ruolo cambia. Invece, usando `node_id` stabile e ruolo temporaneo, il sistema può dire con precisione: "MB-003 ha assunto ruolo connector durante `bridge_run_002`", oppure "MB-001 era anchor nella run `circle_run_001` ma follower nella run `line_run_003`". Questa precisione rende il sistema molto più professionale.

L'assegnazione dei ruoli può avvenire in modo statico, dinamico o ibrido. Nell'assegnazione statica, il controller definisce manualmente o tramite configurazione quale ruolo appartiene a ogni nodo. Per esempio, in una linea di cinque nodi, MB-001 è leader, MB-002, MB-003 e MB-004 sono connector, MB-005 è terminal. Questo approccio è semplice, leggibile e adatto alle prime versioni. Nell'assegnazione dinamica, invece, il controller sceglie i ruoli in base allo stato reale dei nodi: posizione, batteria, capacità, qualità di rete, sincronizzazione, vicinato, errori precedenti o disponibilità. Nell'assegnazione ibrida, una struttura di ruoli viene proposta dal pattern, ma il controller può modificarla in base alle condizioni effettive.

Per le prime versioni MicroBot, l'approccio più sensato è ibrido ma controllato. Il pattern può dichiarare i ruoli necessari, mentre il controller li assegna ai nodi disponibili in base a criteri semplici e verificabili. Per esempio, il pattern "line_basic_v0.1" richiede un leader, alcuni connector e un terminal. Il controller seleziona tra i nodi online quelli sincronizzati, sicuri e compatibili. Se il ruolo richiede una capability specifica, come coil_pwm o led_rgb, il controller sceglie soltanto nodi che la possiedono. Questo consente al sistema di essere flessibile senza diventare subito troppo complesso.

Una struttura generale può essere:

pattern richiede ruoli;

controller seleziona nodi compatibili;

controller assegna ruoli;

nodi confermano ruolo;

dashboard visualizza ruolo;

log registra assegnazione.

Questa pipeline è fondamentale. Il ruolo non deve esistere solo nella testa del controller. Deve essere comunicato al nodo, confermato tramite ack e visibile nella dashboard. Se il controller crede che MB-002 sia connector ma il nodo non ha ricevuto il comando, si crea disallineamento. L'assegnazione dei ruoli deve quindi essere parte del protocollo e della sincronizzazione, non un'informazione nascosta.

Il comando principale per assegnare un ruolo può essere assign_role. Questo comando viene inviato in unicast quando il ruolo è specifico del nodo, oppure in broadcast/groupcast quando un insieme di nodi deve assumere lo stesso ruolo. Nella maggior parte dei casi, l'assegnazione individuale tramite unicast è più chiara e controllabile. Il pacchetto può contenere node_id come target, role, pattern_run_id, group_id, phase_id opzionale e valid_until. Il nodo riceve il comando, verifica che il ruolo sia supportato, aggiorna il proprio stato operativo e risponde con ack.

Un esempio concettuale è:

```
{
  "command": "assign_role",
  "target": "MB-002",
  "payload": {
    "role": "connector",
    "pattern_run_id": "bridge_run_001",
    "group_id": "bridge_chain",
```

```
"valid_until": 125000
}
}
```

Questo comando assegna a MB-002 il ruolo connector dentro una run specifica. Il campo `pattern_run_id` è essenziale perché il ruolo non deve essere applicato a tutte le run future. Il campo `group_id` indica l'appartenenza a una sottostruttura. Il campo `valid_until` può essere utile se il ruolo è temporaneo. Nelle prime versioni, `valid_until` può essere omesso, ma `pattern_run_id` dovrebbe essere presente appena si gestiscono più pattern.

I ruoli possono essere classificati in base alla loro funzione. La prima categoria comprende ruoli di riferimento, come `leader`, `anchor` e `reference_node`. Questi nodi definiscono origine, orientamento, fase o punto di partenza del pattern. La seconda categoria comprende ruoli strutturali, come `connector`, `bridge_node`, `terminal`, `boundary_node`, `internal_node` e `core_node`. Questi nodi mantengono connessioni, bordi, segmenti o nucleo della struttura. La terza categoria comprende ruoli comportamentali, come `follower`, `seeker`, `stabilizer`, `aligner` o `separator`. Questi nodi applicano regole locali specifiche. La quarta categoria comprende ruoli di sistema, come `diagnostic_node`, `passive_node`, `observer_node` o `safety_monitor`. Questi nodi servono a osservare, segnalare o limitare il comportamento.

Una classificazione ordinata può essere:

ruoli di riferimento: `leader`, `anchor`, `reference_node`;

ruoli strutturali: `connector`, `bridge_node`, `boundary_node`, `terminal_node`, `core_node`;

ruoli comportamentali: `follower`, `seeker`, `stabilizer`, `aligner`, `separator`;

ruoli operativi: `actuator_node`, `sensor_node`, `diagnostic_node`, `passive_node`;

ruoli di sicurezza: `safety_monitor`, `isolated_node`, `quarantine_node`.

Questa classificazione rende il sistema più chiaro. Quando si legge un ruolo, si capisce immediatamente se serve a definire geometria, mantenere topologia, eseguire azione, osservare o proteggere.

Il ruolo `leader` è uno dei più intuitivi. In una linea o in una catena, il `leader` può definire l'inizio della struttura, l'orientamento o la direzione del movimento. In una formazione dinamica, può essere il nodo che riceve comandi principali o che stabilisce un riferimento locale. Tuttavia, nel progetto `MicroBot` è importante non rendere il `leader` un punto unico di fallimento troppo rigido. Se il `leader` fallisce, la struttura non dovrebbe necessariamente collassare in tutti i pattern. Per questo motivo, il ruolo `leader` deve essere usato con attenzione e, nelle versioni più avanzate, può essere sostituito da `anchor` multipli o `leader` di cluster.

Il ruolo `anchor` è simile al `leader` ma più geometrico. Un `anchor` non deve necessariamente guidare il comportamento, ma definisce un riferimento spaziale o topologico. In un cerchio, l'`anchor` può indicare l'angolo zero. In una griglia, può definire l'origine. In una superficie, può definire un bordo fisso. L'`anchor` aiuta a rendere il pattern ripetibile. Senza `anchor`, una forma può essere corretta ma ruotata o traslata rispetto alla dashboard. Con `anchor`, la forma ha orientamento e posizione più controllati.

Il ruolo `follower` indica un nodo che segue una regola rispetto a un riferimento, a un vicino o a un target. In una linea, un `follower` può mantenere distanza dal nodo precedente. In una catena, può

seguire il connector davanti. In un pattern dinamico, può aggiornare il proprio target in base al leader o al gruppo. Il follower è un ruolo utile perché permette di distribuire la formazione: non tutti i nodi devono conoscere tutta la struttura; alcuni possono seguire relazioni locali. Questo è più vicino alla logica swarm.

Il ruolo connector è uno dei più importanti per MicroBot. Un connector è un nodo incaricato di mantenere o creare una connessione tra parti della struttura. In una catena, i nodi intermedi sono connector perché collegano predecessore e successore. In un ponte, i connector formano la catena centrale. In una griglia, i connector mantengono archi orizzontali e verticali. In una struttura complessa, un connector può essere critico per unire cluster o segmenti. Questo ruolo è particolarmente rilevante quando si introducono agganci magnetici o connessioni topologiche simulate.

Un connector deve avere comportamenti specifici: mantenere vicini corretti, evitare connessioni indesiderate, rispettare limiti di distanza, segnalare detach_event se perde una connessione, e rispondere a comandi di attach o stabilize. Se il nodo possiede hardware attuativo, come coil_pwm, può partecipare a impulsi magnetici. Se non lo possiede, può comunque essere connector virtuale o visuale. La capability del nodo determina quanto il ruolo può essere fisicamente eseguito.

Il ruolo boundary_node è fondamentale nelle strutture bidimensionali e nei cluster. I nodi di bordo definiscono il contorno della forma. In una griglia, i boundary_node sono quelli sui lati esterni. In un cluster, sono i nodi periferici. In una superficie, definiscono il bordo del materiale. Questi nodi sono importanti perché il bordo spesso determina la forma percepita. Un errore nel bordo può deformare l'intera struttura. I boundary_node possono avere regole diverse dai core_node: devono mantenere forma esterna, evitare dispersione e forse reagire più fortemente a target geometrici.

Il ruolo core_node indica un nodo interno a una struttura. In un cluster, i core_node stabilizzano la massa centrale. In una griglia o superficie, occupano celle interne. In un reticolo, mantengono connettività interna. I core_node possono essere meno visibili ma molto importanti per stabilità. Se un core_node manca, la struttura può avere un buco interno o perdere densità. Tuttavia, in alcune strutture il core può essere ridondante, quindi la perdita di un core_node può essere meno critica della perdita di un bridge_node.

Il ruolo bridge_node è specifico per strutture che collegano sottoparti. Un ponte tra cluster, una catena centrale, un collegamento tra segmenti o un raccordo tra due reticoli può usare bridge_node. Questo ruolo è spesso critico, perché se fallisce può separare la struttura in componenti scollegati. Il controller dovrebbe assegnare bridge_node a nodi con buona sincronizzazione, buona rete, batteria sufficiente e capability adeguate. Non è un ruolo da assegnare casualmente se esistono alternative.

Il ruolo terminal_node indica un nodo che sta all'estremità di una linea, catena, arco o segmento. I terminal_node hanno meno vicini previsti e devono evitare connessioni oltre l'estremità. In una catena, un terminal_node ha un solo vicino strutturale. Questo ruolo è utile per definire estremità e direzione. Nella dashboard può essere evidenziato perché mostra dove inizia e finisce una struttura.

Il ruolo stabilizer indica un nodo o gruppo di nodi incaricato di mantenere stabilità dopo la formazione. Può ridurre oscillazioni, mantenere distanza, evitare distacchi, segnalare instabilità o correggere piccoli errori. In simulazione, uno stabilizer può avere peso maggiore su stabilization_weight. In hardware futuro, potrebbe attivare feedback o mantenere una connessione passiva. Questo ruolo è importante perché formare una struttura non basta: bisogna mantenerla. La stabilità è parte del pattern.

Il ruolo seeker può essere usato per nodi che devono cercare una connessione, un gruppo o una posizione. In un cluster, un nodo isolato può diventare seeker e muoversi verso il gruppo. In una

struttura composta, un nodo può cercare il connector più vicino. In hardware fisico senza locomozione autonoma completa, questo ruolo può restare simulativo o visuale. Ma dal punto di vista del modello swarm è utile perché descrive nodi in fase di integrazione.

Il ruolo `passive_node` indica un nodo che partecipa alla struttura senza attuazioni attive. Può ricevere comandi, inviare heartbeat, mostrare stato tramite LED, ma non eseguire azioni energetiche. Questo ruolo è utile per nodi con batteria bassa, capability limitata o funzione decorativa. In una demo, alcuni nodi fisici possono essere `passive_node` mentre la simulazione mostra comportamento completo. Questo permette integrazione graduale dei prototipi.

Il ruolo `diagnostic_node` serve a osservare e riportare dati. Un nodo diagnostico può avere telemetria più dettagliata, registrare eventi, segnalare qualità rete, oppure rappresentare un punto di monitoraggio nella simulazione. In una struttura complessa, un `diagnostic_node` può essere assegnato a un punto critico per misurare stabilità. Nei primi prototipi, questo ruolo può essere implementato semplicemente aumentando il `telemetry_level` su un nodo specifico.

Il ruolo `actuator_node` indica un nodo autorizzato o capace di eseguire attuazioni fisiche, come coil, PWM, vibrazione, motore o altro. Non tutti i nodi devono essere actuator. Alcuni possono essere solo indicatori. Questo ruolo deve essere collegato strettamente alle capabilities e alla safety. Un nodo può essere `actuator_node` solo se hardware e firmware lo supportano. Il controller deve evitare di assegnare ruoli attuativi a nodi LED-only.

Il ruolo `sensor_node` indica un nodo che fornisce dati ambientali o locali. Nelle prime versioni può essere simulato o limitato alla telemetria base, come batteria stimata e temperatura stimata. In futuro può includere sensori di prossimità, IMU, campo magnetico, corrente, temperatura reale o altro. Un `sensor_node` può aiutare il controller a valutare lo stato della struttura. Per esempio, in una catena, i `sensor_node` possono essere ai giunti o ai punti critici.

Il ruolo `safety_monitor` è un ruolo orientato alla protezione. Un nodo o modulo può monitorare condizioni critiche e segnalare problemi. Nelle prime versioni, ogni nodo deve avere safety locale; quindi il ruolo `safety_monitor` può essere più una funzione logica del firmware che un ruolo esclusivo. Tuttavia, in strutture complesse, alcuni nodi possono avere telemetria più ricca o essere incaricati di monitorare sottogruppi. Questo prepara futuri sviluppi.

L'assegnazione dei ruoli deve essere guidata da criteri. Il controller non dovrebbe assegnare ruoli importanti senza considerare lo stato dei nodi. I criteri principali sono: capabilities, `sync_status`, `network_status`, `battery_level`, `thermal_state`, `current_mode`, `firmware_version`, posizione iniziale o simulata, storico errori e disponibilità. Un nodo con `sync_good` e rete stabile è candidato migliore per ruoli temporizzati. Un nodo con `coil_pwm` è candidato per connector attivo. Un nodo con `low_energy` può diventare `passive_node`. Un nodo con molti errori recenti non dovrebbe essere `bridge_node`.

Una funzione di assegnazione può concettualmente usare uno score:

$$\text{role_score}(\text{node}, \text{role}) = \text{compatibilità_capability} + \text{qualità_sync} + \text{qualità_rete} + \text{energia} + \text{affidabilità} - \text{penalità_errori}$$

Il controller sceglie il nodo con score più alto per i ruoli critici. Questa formula non deve essere implementata subito in modo complesso. Nelle prime versioni, può bastare una tabella manuale. Ma documentare lo scoring mostra come il sistema potrà diventare più adattivo.

Per esempio, per assegnare un `bridge_node`, i criteri possono essere:

capability connector o coil_pwm se richiesta;

sync_status almeno sync_ok;

network_status stabile;

battery non bassa;

nessun safety warning;

storico errori basso.

Per assegnare un passive_node, invece, i criteri possono essere meno severi. Per assegnare un diagnostic_node, può essere utile scegliere un nodo con buona rete e capacità telemetria. Ogni ruolo ha criteri diversi.

L'assegnazione dei ruoli deve essere compatibile con la topologia del pattern. In una linea, i terminal_node devono stare alle estremità, non al centro. In una griglia, i corner_node devono corrispondere alle celle angolari. In un cerchio, l'anchor può essere uno specifico punto angolare. In un ponte, i bridge_node devono essere nella catena di connessione. Quindi assegnare un ruolo significa anche assegnare una posizione logica nella struttura. Il ruolo è legato al grafo del pattern.

Una struttura può indicare:

logical_position;

role;

expected_neighbors.

Per esempio:

MB-001: role leader, logical_position line_index_0, expected_neighbors [MB-002];

MB-002: role connector, logical_position line_index_1, expected_neighbors [MB-001, MB-003];

MB-005: role terminal, logical_position line_index_4, expected_neighbors [MB-004].

Questa descrizione collega ruolo, posizione e topologia. È molto utile per verificare se il nodo è nel posto corretto e se ha le connessioni previste.

I ruoli devono essere comunicati ai nodi in modo tracciabile. Il controller invia assign_role e riceve ack. L'ack dovrebbe includere role_applied, pattern_run_id, group_id e current_mode. Se il nodo non supporta il ruolo, deve rispondere unsupported_role. Se il ruolo richiede capability mancante, può rispondere capability_missing. Se il nodo è in safety, può rispondere safety_rejected. Questo rende l'assegnazione verificabile.

Un ack di ruolo può contenere:

ref_message_id;

result = role_assigned;

role = connector;

pattern_run_id;

current_phase;

timestamp.

Oppure:

result = rejected;

reason = unsupported_role.

Questa risposta permette al controller di correggere la mappa ruoli. Non bisogna assumere che il ruolo sia stato applicato finché non arriva conferma.

L'assegnazione dei ruoli deve essere mostrata nella dashboard. Ogni nodo deve avere un'etichetta o un colore che indica il ruolo corrente. Per esempio, leader, connector, boundary, core, terminal, diagnostic. Questo rende i pattern comprensibili. Quando si mostra una linea, l'osservatore può vedere chi è leader e chi è terminal. Quando si mostra un ponte, può vedere quali nodi sono cluster e quali bridge. Quando si mostra una griglia, può vedere corner, edge e internal. La visualizzazione dei ruoli trasforma una nuvola di punti in una struttura leggibile.

Una dashboard può mostrare:

node_id;

role;

group_id;

phase;

capabilities;

role_status;

expected_neighbors.

Questo pannello è molto utile per debug. Se un nodo ha ruolo sbagliato o non confermato, il problema si vede subito. Senza questa vista, errori di ruolo possono restare nascosti e apparire solo come pattern fallito.

I ruoli devono essere registrati nei log. Ogni run dovrebbe salvare quale nodo ha avuto quale ruolo e quando. Se un ruolo cambia durante la run, anche il cambio deve essere registrato. Questo è importante perché il comportamento successivo dipende dal ruolo. Se un nodo ha generato un errore come connector, non è la stessa cosa che generarlo come passive_node. Il ruolo dà contesto agli eventi.

Un log di assegnazione può includere:

timestamp;

pattern_run_id;

node_id;

previous_role;

new_role;

reason;

assigned_by;

result.

Per esempio:

T=1200 ms, bridge_run_001, MB-004, previous_role=idle, new_role=bridge_node,
reason=structure_assignment, result=accepted.

Questa registrazione rende la run analizzabile. Se il ponte fallisce, si può vedere quali nodi erano bridge_node e se hanno confermato il ruolo.

L'assegnazione dei ruoli può essere modificata durante una run. Non sempre il ruolo resta fisso. Un nodo può iniziare come seeker, poi diventare connector quando trova il gruppo. Un nodo può essere follower durante align e stabilizer durante stabilize. Un nodo può essere active_connector durante attach e passive_connector durante mantenimento. Questa evoluzione è molto utile perché un pattern è composto da fasi diverse. Il ruolo può essere globale alla run o specifico della fase.

Si può distinguere:

run_role = ruolo principale nella run;

phase_role = ruolo specifico nella fase corrente.

Per esempio, MB-003 può avere run_role = connector, ma phase_role = aligner durante phase_align e phase_role = stabilizer durante phase_stabilize. Questa distinzione permette comportamenti più precisi. Nelle prime versioni si può usare un solo role field; nelle versioni avanzate, run_role e phase_role sono più rigorosi.

Una struttura di ruolo avanzata può essere:

```
{  
  "run_role": "connector",  
  "phase_role": "stabilizer",  
  "group_id": "bridge_chain"  
}
```

Il nodo può interpretare entrambi. Il controller può aggiornare phase_role a ogni fase. Questo rende il comportamento più modulare.

I ruoli devono essere compatibili con la macchina a stati. Un nodo in safety non può assumere ruoli attivi. Un nodo in low_energy può assumere solo ruoli passivi o diagnostici. Un nodo offline non ha ruolo operativo. Un nodo in diagnostic può assumere diagnostic_node ma non connector attivo, salvo test. Il controller deve verificare lo stato prima di assegnare ruoli, e il nodo deve verificare localmente prima di accettare. Questo doppio controllo è essenziale.

Una regola:

$\text{role_assignment_valid} = \text{role_supported} \wedge \text{state_allows_role} \wedge \text{capabilities_ok} \wedge \text{safety_ok}$

Se questa condizione è falsa, il nodo rifiuta. In questo modo, un errore del controller non produce comportamento incoerente.

I ruoli devono essere collegati alle capabilities. Un ruolo può richiedere capacità specifiche. Per esempio:

leader: richiede sync_good e comunicazione stabile;

connector: richiede capacità di mantenere connessioni o almeno tracking di vicini;

active_connector: richiede coil_pwm o attuatore equivalente;

boundary: richiede capacità di stimare o ricevere posizione;

diagnostic: richiede telemetry_debug;

passive: richiede solo heartbeat e stato base.

Questa mappa $\text{role} \rightarrow \text{required_capabilities}$ deve essere documentata. Senza questa mappa, il controller potrebbe assegnare ruoli impossibili. La mappa può essere parte del pattern o del runtime.

Una tabella concettuale può essere:

leader: heartbeat, sync, command_ack;

connector: heartbeat, sync, connection_state;

active_connector: connector + coil_pwm + safety_lock;

boundary_node: position_estimate o simulation_position;

diagnostic_node: telemetry_debug;

passive_node: heartbeat, led_status opzionale.

Questa tabella può evolvere con l'hardware. Nei primi prototipi, molti ruoli saranno simulation-only o visual-demo. In futuro, diventeranno hardware-ready.

L'assegnazione dei ruoli deve essere differente tra simulazione e hardware reale. In simulazione, si può assegnare qualunque ruolo logico a qualunque nodo virtuale, purché il modello lo supporti. In hardware, bisogna rispettare limiti reali. Per esempio, un nodo simulato può diventare active_connector senza problemi, ma un ESP32 LED-only non può eseguire attuazioni magnetiche. Quindi il role_assignment deve considerare implementation_profile.

Un ruolo può avere status:

simulation_supported;

visual_supported;

hardware_supported;

hardware_future.

Per esempio, `boundary_node` può essere `simulation_supported` e `visual_supported`, ma `hardware_future` se non esistono ancora sensori di posizione. `connector` può essere `simulation_supported` e `hardware_partial` se esistono solo magneti passivi o LED indicatori. `active_connector` può essere `hardware_partial` solo quando `coil` e `safety` sono implementati. Questa classificazione mantiene il documento onesto e ordinato.

L'assegnazione dei ruoli può essere manuale nelle prime demo. Il controller o un file config può indicare esattamente chi fa cosa. Questo è utile perché riduce complessità. Per una demo ad Andrea, per esempio, si può preparare una run con sei nodi simulati o fisici e ruoli già definiti. La dashboard mostra chiaramente la sequenza. Successivamente, si potrà introdurre assegnazione automatica basata su stato e score. Non bisogna partire subito da algoritmi complessi se il sistema base non è ancora consolidato.

Una progressione realistica è:

v0.1: ruoli manuali in file di configurazione;

v0.2: comando `assign_role` e `ack`;

v0.3: ruoli visualizzati in dashboard;

v0.4: ruoli legati a `pattern_run_id`;

v0.5: validazione capabilities;

v0.6: assegnazione automatica semplice;

v0.7: riassegnazione dinamica in caso di errore;

v0.8: ruoli gerarchici per cluster;

v0.9: role scoring adattivo.

Questa roadmap è coerente con lo sviluppo incrementale MicroBot. Ogni passo aggiunge maturità senza rompere l'architettura.

L'assegnazione dinamica dei ruoli richiede criteri di scelta. In una linea, il leader potrebbe essere il nodo più vicino al punto iniziale target, o il nodo con migliore rete, o quello definito manualmente. In un cluster, i `core_node` possono essere i nodi più vicini al centro; i `boundary_node` quelli più esterni. In un ponte, i `bridge_node` possono essere scelti tra nodi tra due cluster. In una griglia, `corner_node` e `edge_node` possono essere scelti in base alle posizioni target. Se il sistema lavora solo in simulazione, queste scelte possono basarsi sulle coordinate. Se lavora con hardware reale senza localizzazione, la scelta può basarsi su configurazione manuale o disposizione nota.

Questa distinzione è importante: l'assegnazione dinamica richiede informazioni. Se il sistema non conosce le posizioni fisiche reali, non può assegnare automaticamente ruoli geometrici basati sulla posizione, salvo che l'utente glielo fornisca o la dashboard lavori in simulazione. Quindi, per l'hardware iniziale, ruoli geometrici fisici possono essere manuali. Per la simulazione, possono essere automatici. Questo mantiene rigore.

L'assegnazione dei ruoli deve considerare anche nodi mancanti. Se un nodo assegnato a un ruolo critico è offline, il controller deve decidere se sostituirlo. La sostituzione richiede che esista un nodo compatibile. Se un `terminal_node` manca in una linea, un altro nodo può diventare terminal. Se un

bridge_node manca e non esistono alternative, la struttura può fallire. Se un diagnostic_node manca, la run può continuare con meno osservabilità. Ogni ruolo deve avere una policy di sostituzione.

Una policy può includere:

replaceable = true / false;

replacement_candidates;

minimum_capabilities;

impact_if_missing;

fallback_role.

Per esempio, bridge_node può essere replaceable solo se esiste un altro connector disponibile.

Anchor può essere sostituibile ma può cambiare orientamento del pattern. Diagnostic_node può essere optional. Active_connector può essere non sostituibile se solo un nodo ha coil_pwm. Questa informazione è molto utile nei pattern complessi.

I ruoli possono avere priorità. Se due ruoli competono per lo stesso nodo, il controller deve scegliere. Per esempio, un nodo può essere adatto sia come connector sia come diagnostic_node. Se il pattern richiede un connector critico, quel ruolo ha priorità. La diagnostica può essere assegnata a un altro nodo. Ogni ruolo può avere priority o criticality.

Una classificazione può essere:

critical_role;

structural_role;

support_role;

optional_role.

I ruoli critici devono essere assegnati prima. I ruoli opzionali vengono assegnati dopo, se restano nodi disponibili. Questa strategia evita di consumare un nodo affidabile per un ruolo secondario mentre manca un ruolo centrale.

L'assegnazione dei ruoli deve evitare conflitti. Un nodo non dovrebbe avere due ruoli incompatibili nello stesso tempo. Per esempio, non può essere passive_node e active_connector contemporaneamente nella stessa fase. Non può essere terminal e internal_node nello stesso segmento. Non può essere safety_monitor attivo e nodo in quarantine se la policy lo impedisce. Alcuni ruoli invece possono essere combinabili: connector e stabilizer possono coesistere; boundary_node e sensor_node possono coesistere; leader e anchor possono coincidere. Serve quindi una matrice di compatibilità dei ruoli.

Una regola semplice è:

un nodo può avere un ruolo primario e ruoli secondari compatibili.

Per esempio:

primary_role = connector;

`secondary_roles = [stabilizer, diagnostic]`

Oppure:

`primary_role = anchor;`

`secondary_roles = [leader]`

Questa struttura è più flessibile di un solo ruolo, ma richiede più gestione. Nelle prime versioni, un solo ruolo primario può bastare. In versioni avanzate, ruoli secondari saranno utili.

L'assegnazione dei ruoli deve essere sincronizzata nel tempo. Se il controller assegna ruoli a nodi diversi, tutti devono applicarli prima dell'inizio della fase che li usa. Per evitare disallineamento, `assign_role` può includere `apply_at` o `phase_id`. Per esempio, i ruoli diventano attivi all'inizio della run o della prossima fase. Se un nodo riceve il ruolo in ritardo, può rifiutarlo come `expired` o chiedere `resync`. Questo è particolarmente importante quando i ruoli cambiano durante una run.

Un comando può dire:

`role = connector;`

`apply_at_phase = attach;`

`pattern_run_id = bridge_run_001.`

Il nodo può memorizzare il ruolo e attivarlo quando arriva `phase_attach`. Questo permette transizioni ordinate. In una versione semplice, i ruoli sono applicati immediatamente prima della run. In versioni più complesse, i ruoli possono essere programmati per fasi future.

L'assegnazione dei ruoli deve essere verificata tramite `heartbeat` o `status`. Dopo l'assegnazione, il nodo dovrebbe riportare `current_role` nei messaggi di stato. Questo permette al controller di rilevare `role_mismatch`. Se il controller si aspetta `connector` ma il nodo riporta `idle` o `follower`, qualcosa non va. Il controller può inviare di nuovo `assign_role`, `state_sync` o escludere il nodo. La dashboard deve mostrare questi mismatch.

Una regola:

`se reported_role ≠ expected_role, alignment_status = role_mismatch.`

Questo mostra che l'allineamento operativo comprende anche i ruoli. Un nodo può essere sincronizzato nel tempo ma avere ruolo sbagliato; quindi non è pienamente allineato al pattern.

I ruoli devono essere collegati alle metriche. Quando si valuta un pattern, l'errore di un nodo va interpretato in base al ruolo. Un `boundary_node` fuori posizione può essere più grave di un `core_node` leggermente fuori posizione se si valuta il bordo. Un `bridge_node` mancante può essere più grave di un `optional_node` mancante. Un `leader` con jitter alto può essere più problematico di un `follower` con jitter alto. Quindi il `pattern_error` può pesare diversamente i nodi in base ai ruoli.

Una metrica pesata può essere:

`pattern_error = somma_i peso_ruolo_i · errore_i`

Dove `peso_ruolo_i` è più alto per ruoli critici. Questo permette di valutare la qualità della struttura in modo più realistico. Non tutti gli errori hanno lo stesso impatto.

Le connessioni possono essere pesate allo stesso modo. Un edge tra due `bridge_node` può essere critico; un edge opzionale di rinforzo può essere meno importante. La `topologia_score` può quindi usare pesi sugli archi. Questo è particolarmente utile nelle strutture complesse. I ruoli dei nodi influenzano la criticità degli archi.

Un arco può avere `criticality` derivata dai ruoli:

`bridge_node—bridge_node`: alta;

`core_node—core_node`: media;

`optional_node—optional_node`: bassa;

`boundary_node—boundary_node`: media-alta se definisce bordo.

Queste informazioni aiutano il controller a decidere fallback. Se manca un arco critico, abort. Se manca un arco opzionale, `partial_success`.

L'assegnazione dei ruoli deve anche supportare la riconfigurazione. Quando il sistema passa da una struttura a un'altra, i ruoli possono cambiare. Un nodo che era `terminal` in una linea può diventare `ring_member` in un cerchio. Un `connector` in un ponte può diventare `core_node` in un cluster. Il controller deve gestire questa transizione senza confondere stati. Ogni cambio ruolo deve essere associato a una fase e a un `pattern_run_id`. I ruoli vecchi devono essere invalidati quando la run finisce o quando inizia una nuova configurazione.

Una transizione può essere:

`line_run_001` termina;

controller invia `clear_roles` o `new assign_role` per `circle_run_002`;

nodi confermano nuovi ruoli;

`circle_run_002` inizia.

Questo evita che un nodo mantenga ruolo vecchio dentro pattern nuovo. Il comando `clear_roles` può essere utile per riportare tutti i nodi a `idle_role` prima di una nuova run.

Un comando `clear_roles` può avere:

`target = all`;

`pattern_run_id = previous`;

`mode_after_clear = idle`.

Dopo `clear_roles`, il controller assegna nuovi ruoli. In una versione semplice, `set_mode idle` può già implicare reset dei ruoli, ma documentare `clear_roles` rende il comportamento più esplicito.

I ruoli devono essere gestiti anche in caso di abort. Se una run viene abortita per safety o errore, i ruoli della run devono essere invalidati o marcati come `inactive`. Un nodo non deve continuare a comportarsi da `connector` dopo che il pattern è stato interrotto. Il controller può inviare `stop_pattern` o `enter_idle`, e i nodi devono pulire il contesto di ruolo. Questo è importante per evitare stati residui.

Una regola:

alla fine o abort di una run, tutti i `phase_role` diventano inactive e i `run_role` vengono cancellati o archiviati.

Il log conserva ciò che erano, ma il nodo non li usa più. Questa distinzione tra storico e stato attivo è importante.

L'assegnazione dei ruoli può essere rappresentata nei file pattern. Un pattern può dichiarare ruoli richiesti e criteri. Per esempio, `line_basic` richiede 1 leader, N-2 connector e 1 terminal. `Circle_basic` richiede 1 anchor e N-1 ring_member. `Grid_basic` richiede corner, edge e internal. `Bridge_complex` richiede cluster_core, bridge_node e connector. Questo rende il pattern auto-descrittivo.

Una definizione di ruoli per `line_basic` può essere:

`roles_required:`

`leader: 1;`

`connector: N-2;`

`terminal: 1.`

Per `grid_basic`:

`corner_node: 4;`

`edge_node: 2R + 2C - 8;`

`internal_node: (R-2)(C-2).`

Queste formule permettono di generare ruoli in base alle dimensioni del pattern. È una soluzione pulita e scalabile.

Nel caso della griglia, l'assegnazione dei ruoli può essere automatica dalla posizione logica. Le celle agli angoli sono `corner_node`. Le celle sui bordi sono `edge_node`. Le celle interne sono `internal_node`. Questo mostra come la geometria generi ruoli. Nel cerchio, l'anchor può essere scelto manualmente o come nodo con indice 0. Nel cluster, i `core_node` e `boundary_node` possono essere determinati dinamicamente in base alla distanza dal centro o dalla densità locale. Ogni pattern ha quindi una logica specifica di role assignment.

Per il cluster, una regola può essere:

se distanza dal centro < soglia_core → `core_node`;

se distanza dal centro ≈ raggio_cluster → `boundary_node`;

se nodo isolato → `seeker`;

se nodo stabile e vicino a molti vicini → `stabilizer`.

Questa assegnazione può cambiare nel tempo, perché i nodi si muovono o il cluster evolve. È un esempio di ruolo dinamico basato sullo stato.

I ruoli dinamici devono essere gestiti con hysteresis per evitare oscillazioni. Se un nodo passa continuamente da `core_node` a `boundary_node` perché si trova vicino alla soglia, il comportamento può diventare instabile. Bisogna usare soglie con margine o richiedere che una condizione persista per un certo tempo prima di cambiare ruolo. Questo concetto è analogo all'hysteresis delle connessioni.

Una regola può essere:

un nodo diventa `boundary_node` solo se resta nella regione di bordo per almeno ΔT_{role} ;

un nodo torna `core_node` solo se rientra sotto una soglia più interna per almeno ΔT_{role} .

Questo evita cambi ruolo troppo frequenti. I ruoli devono essere stabili abbastanza da guidare il comportamento.

L'assegnazione dei ruoli deve considerare anche la possibilità di ruoli locali determinati dal vicinato. Un nodo può diventare `connector` perché ha due vicini importanti, o `stabilizer` perché si trova in una zona instabile, o `bridge_node` perché collega due cluster. Queste decisioni possono essere calcolate dal controller usando il grafo dello swarm. Per esempio, un nodo con alta *betweenness centrality* nel grafo può essere un `bridge_node` perché collega componenti diverse. Questa è una direzione più avanzata, ma molto interessante.

Dal punto di vista del grafo:

un nodo con grado alto può essere `hub`;

un nodo con grado basso e posizione estrema può essere `terminal`;

un nodo che collega due componenti può essere `bridge`;

un nodo in bordo geometrico può essere `boundary`;

un nodo centrale e stabile può essere `core`.

Questa logica collega teoria dei grafi e *role assignment*. È molto coerente con MicroBot come swarm intelligente.

L'assegnazione dei ruoli può anche essere usata per ridurre complessità comunicativa. Invece di inviare comandi a tutti i nodi, il controller può inviare comandi a ruoli. Per esempio:

```
target_role = connector;
```

```
command = stabilize_connections.
```

Tutti i nodi con ruolo `connector` processano il comando. Questa è una forma di *groupcast* logico basato sui ruoli. Può essere implementata con *broadcast*: tutti ricevono il messaggio, ma solo i `connector` lo eseguono. Questo riduce la necessità di elencare `node_id`. È molto utile per pattern complessi.

Un pacchetto può contenere:

```
target = "role:connector"
```

oppure:

```
target_filter = { "role": "boundary_node", "group_id": "cluster_A" }
```

In una prima versione può essere più semplice usare target espliciti, ma target per ruolo è una potente estensione. Richiede che i nodi conoscano il proprio ruolo e che il controller mantenga coerenza.

I ruoli possono essere usati anche per definire colori LED. Questo è molto utile nelle demo fisiche. Leader può essere blu, connector verde, terminal giallo, safety rosso, diagnostic viola, boundary arancione. Così un osservatore può vedere il ruolo del nodo reale. Anche se i MicroBot non si muovono ancora fisicamente, i LED mostrano la logica dello swarm. Questa è una strategia dimostrativa molto forte.

Una mappatura visuale può essere:

leader → blu;

connector → verde;

terminal → giallo;

boundary → arancione;

core → bianco;

diagnostic → viola;

safety/quarantine → rosso.

Questa mappatura deve essere coerente nella dashboard e nell'hardware. Così il colore ha significato stabile. Nelle documentazioni e video, si può spiegare che i colori mostrano il ruolo assegnato dal controller.

L'assegnazione dei ruoli deve essere integrata con i pattern geometrici semplici. Nella linea, i ruoli sono leader, connector e terminal. Nel cerchio, anchor e ring_member. Nella griglia, corner, edge e internal. Nel cluster, core, boundary, seeker e stabilizer. Questa mappa dovrebbe essere descritta per ogni pattern. Così quando si introduce un pattern, si sa subito quali ruoli richiede.

Una tabella descrittiva può essere:

linea: leader, connector, terminal;

cerchio: anchor, ring_member;

griglia: grid_anchor, corner_node, edge_node, internal_node;

cluster: core_node, boundary_node, seeker, stabilizer;

ponte: cluster_core, bridge_node, connector, anchor;

reticolo: corner_node, edge_node, internal_node, reinforcement_node.

Questa tabella rende il sistema più leggibile e prepara la Sezione sui comportamenti adattivi.

L'assegnazione dei ruoli è anche cruciale per la correzione degli errori. Se un nodo con ruolo connector fallisce, il controller deve cercare un replacement_connector. Se un boundary_node è fuori posizione, il sistema deve correggere bordo. Se un core_node è instabile, può aumentare stabilizzazione locale. Se un leader fallisce, può promuovere un follower a leader. La correzione dipende dal ruolo. Senza ruoli, tutti gli errori sembrano uguali; con ruoli, il controller sa che cosa significa un errore.

Una policy di correzione può essere:

leader_failed → eleggere nuovo leader o abortire se non possibile;

connector_failed → cercare nodo sostitutivo o ridurre topologia;

boundary_error → correggere forma esterna;

core_error → correggere densità interna;

diagnostic_failed → continuare con ridotta osservabilità;

passive_failed → registrare warning se non critico.

Queste regole rendono la risposta agli errori più intelligente.

L'assegnazione dei ruoli deve considerare l'adattamento dinamico. In uno swarm avanzato, i ruoli possono cambiare per compensare problemi. Se un nodo bridge ha batteria bassa, un altro può diventare bridge. Se un boundary_node perde connessione, un core_node vicino può spostarsi a boundary. Se un leader ha jitter alto, un nodo più stabile può essere promosso. Questo è uno dei passaggi verso comportamento adattivo. La role assignment non è solo iniziale; può diventare una strategia di resilienza.

Tuttavia, la riassegnazione dinamica deve essere controllata. Cambiare ruoli troppo spesso può creare instabilità. Ogni cambio ruolo deve avere motivo, timestamp, fase, ack e possibile cooldown. Un nodo appena promosso deve confermare. Il controller deve aggiornare dashboard e log. La riassegnazione non deve essere caotica.

Una regola può essere:

role_change_allowed solo se nuova assegnazione migliora una condizione critica e se il nodo candidato è ready.

Inoltre:

non cambiare ruolo durante azione critica salvo safety o recovery.

Questo evita riassegnazioni in momenti sbagliati.

L'assegnazione dei ruoli deve supportare il concetto di cluster. In strutture grandi, i ruoli possono essere locali al cluster. Si può avere leader di cluster, connector di cluster, boundary di cluster. Il controller globale può assegnare ruoli ai cluster, mentre un livello locale assegna ruoli ai nodi. Questa è una direzione futura per scalabilità. Per ora, il controller centrale può gestire tutto, ma la documentazione deve preparare la gerarchia.

Una struttura gerarchica dei ruoli può essere:

global_role: funzione nello swarm completo;

cluster_role: funzione nel cluster;

local_role: funzione rispetto ai vicini.

Per esempio, un nodo può essere global_role = bridge_node, cluster_role = boundary_node, local_role = connector. Questo è avanzato, ma molto potente per strutture complesse. Nelle prime versioni si può usare un solo role, ma il modello concettuale può prevedere gerarchia.

L'assegnazione dei ruoli deve essere compatibile con la sicurezza. Un nodo in quarantene non può essere assegnato a ruolo attivo. Un nodo in safety può essere solo safety_node o diagnostic, finché non viene ripristinato. Un nodo con thermal_lock non può essere active_connector. Un nodo con low_energy può essere passive o observer. Il safety layer deve avere priorità sui ruoli. Il ruolo è una richiesta operativa, non un'autorizzazione assoluta.

Una regola fondamentale è:

safety_state prevale su assigned_role.

Se il nodo entra in safety, il suo ruolo attivo viene sospeso. Il controller riceve evento e aggiorna mappa ruoli. Questo impedisce che un nodo continui a essere considerato connector attivo quando in realtà ha disattivato le uscite.

Il ruolo può anche influenzare la safety. Un active_connector può avere limiti più stretti di un passive_node. Un bridge_node critico può richiedere telemetria più frequente. Un diagnostic_node può inviare più dati. Un actuator_node deve avere cooldown e max_duration. Quindi role e safety interagiscono. Il ruolo definisce quali controlli sono più importanti.

Per esempio:

active_connector → controllare thermal_load e duty_cycle;

leader → controllare sync_quality e network_status;

boundary_node → controllare posizione/errore bordo;

diagnostic_node → controllare traffico telemetria;

passive_node → controllare solo heartbeat e stato base.

Questa specializzazione rende il sistema più efficiente.

L'assegnazione dei ruoli deve essere documentata come contratto tra pattern e nodo. Il pattern dichiara cosa serve. Il controller assegna. Il nodo accetta se compatibile. La dashboard mostra. Il log registra. Il safety layer può sospendere. Questo ciclo rende il ruolo una parte formale del sistema, non un'etichetta estetica.

In conclusione, l'assegnazione dei ruoli è il meccanismo che permette a MicroBot di passare da uno swarm uniforme a una struttura funzionalmente organizzata. Il ruolo definisce cosa un nodo deve fare dentro una run, una fase, un pattern o una sottostruttura. Esso è distinto dall'identità stabile del nodo, viene assegnato dal controller, validato dal nodo, mostrato nella dashboard, registrato nei log e usato per guidare comandi, comportamenti, metriche e fallback. I ruoli possono essere di

riferimento, strutturali, comportamentali, operativi o di sicurezza; possono essere statici, dinamici o gerarchici; possono dipendere da capabilities, rete, energia, posizione, topologia e stato locale. Senza ruoli, i pattern restano forme anonime; con i ruoli, diventano sistemi organizzati in cui ogni MicroBot contribuisce alla struttura collettiva. Il punto successivo potrà quindi approfondire l'adattamento dinamico, cioè il modo in cui lo swarm modifica ruoli, parametri, fasi e strategie quando l'ambiente, la rete o lo stato dei nodi cambiano durante l'esecuzione.

10.5 Adattamento dinamico

L'adattamento dinamico rappresenta la capacità del sistema MicroBot di modificare il proprio comportamento durante l'esecuzione, in risposta a cambiamenti dello stato dei nodi, della rete, della sincronizzazione, della geometria, della topologia o delle condizioni di sicurezza. Dopo aver definito pattern, strutture complesse e assegnazione dei ruoli, è necessario introdurre il concetto di adattamento perché uno swarm reale non può essere progettato come una sequenza rigida valida soltanto in condizioni perfette. I MicroBot possono perdere pacchetti, entrare in safety, avere batteria bassa, ricevere comandi in ritardo, cambiare fase, perdere una connessione, non raggiungere una posizione target o presentare capacità diverse rispetto a quelle attese. L'adattamento dinamico è il livello che permette al sistema di reagire senza collassare immediatamente.

Nel progetto MicroBot, l'adattamento dinamico non deve essere inteso come improvvisazione casuale. Deve essere una risposta controllata, documentata e misurabile. Il sistema non cambia comportamento "a caso"; modifica parametri, ruoli, fasi, slot, gruppi o strategie sulla base di condizioni osservabili. Questo è un punto fondamentale. Un sistema adattivo maturo non è semplicemente flessibile, ma possiede regole precise per decidere quando adattarsi, come adattarsi, entro quali limiti e con quali conseguenze. L'adattamento deve quindi essere progettato come parte del runtime, non come eccezione manuale.

La forma generale dell'adattamento può essere espressa così:

stato osservato → rilevazione deviazione → scelta strategia → modifica controllata → verifica risultato

Questa catena descrive l'intero processo. Prima il controller o il nodo osserva lo stato reale. Poi confronta questo stato con il comportamento atteso. Se esiste una deviazione significativa, sceglie una strategia di correzione. Poi applica la modifica, che può riguardare ruoli, parametri, fasi, comandi o fallback. Infine verifica se la modifica ha migliorato la situazione. L'adattamento dinamico, quindi, è un ciclo chiuso. Non basta cambiare qualcosa; bisogna misurare se il cambiamento produce un effetto utile.

Il primo livello di adattamento riguarda i parametri comportamentali. In simulazione o in un controller avanzato, il comportamento dello swarm può dipendere da pesi come coesione, separazione, allineamento, attrazione verso target, stabilizzazione, repulsione da ostacoli e forza di connessione. Se il pattern non converge correttamente, il controller può modificare questi pesi. Per esempio, se un cluster è troppo disperso, può aumentare cohesion_weight. Se i nodi sono troppo vicini o si sovrappongono, può aumentare separation_weight. Se una linea è disordinata, può aumentare alignment_weight. Se una struttura oscilla, può aumentare stabilization_weight o ridurre la forza di correzione.

Una regola semplice può essere:

se dispersione alta → aumentare coesione;

se collisioni o distanza minima violata → aumentare separazione;

se forma disallineata → aumentare allineamento;

se oscillazioni elevate → aumentare stabilizzazione e ridurre aggressività;

se convergenza lenta → aumentare attrazione verso target entro limiti sicuri.

Queste regole collegano metriche e parametri. L'adattamento non nasce da una decisione arbitraria, ma dalla relazione tra errore misurato e variabile di controllo. In una prima versione, queste modifiche possono essere manuali o semi-automatiche. In una versione più avanzata, il controller può applicarle automaticamente entro soglie definite.

Il secondo livello di adattamento riguarda i ruoli. Se un nodo assegnato a un ruolo critico fallisce, il sistema può tentare di riassegnare quel ruolo a un altro nodo. Per esempio, se un `bridge_node` entra in `safety`, il controller può cercare un altro nodo compatibile per sostituirlo. Se un leader ha jitter elevato o perde sincronizzazione, un follower più stabile può essere promosso a leader. Se un `boundary_node` si trova troppo lontano dal bordo previsto, un `core_node` vicino può diventare `boundary_node`. Questo tipo di adattamento è molto potente perché permette alla struttura di recuperare da errori locali.

Tuttavia, la riassegnazione dei ruoli deve essere prudente. Cambiare ruoli troppo spesso può generare instabilità. Se un nodo passa continuamente da follower a connector e poi a stabilizer, il sistema diventa difficile da controllare e da interpretare. Per questo motivo, ogni cambio ruolo deve avere una motivazione chiara, una soglia minima, una durata minima e una registrazione nei log. Un ruolo non dovrebbe cambiare per una fluttuazione istantanea; dovrebbe cambiare solo se una condizione persiste o se un evento importante lo richiede.

Una regola robusta può essere:

cambiare ruolo solo se la condizione che richiede il cambio persiste per almeno ΔT_{role} oppure se si verifica un evento critico come `safety`, `offline` o perdita di connessione strutturale.

Questo introduce hysteresis anche nella gestione dei ruoli. L'hysteresis impedisce oscillazioni decisionali. Un nodo diventa `boundary_node` solo se resta effettivamente in zona di bordo per un certo tempo. Un leader viene sostituito solo se la sua qualità temporale resta sotto soglia o se non risponde. Un connector viene sostituito solo se la connessione critica risulta fallita o non disponibile. Questo rende l'adattamento stabile.

Il terzo livello di adattamento riguarda la geometria del pattern. Se un pattern non può essere raggiunto nella forma prevista, il sistema può generare una versione modificata. Per esempio, una linea di cinque nodi può diventare una linea di quattro se un nodo manca. Un cerchio può ridistribuire gli angoli se il numero di nodi cambia. Una griglia 3x3 può degradare in una griglia 2x3. Un anello non chiuso può diventare arco stabile. Questo tipo di adattamento è particolarmente importante perché permette al sistema di produrre comunque una forma coerente anche in condizioni imperfette.

La geometria adattiva deve essere definita nel pattern tramite parametri come `minimum_nodes`, `degradable` e `fallback_geometry`. Se `degradable = true`, il controller può ricalcolare la geometria con i nodi disponibili. Se `degradable = false`, il pattern viene abortito quando mancano nodi critici. Questa distinzione è essenziale. Non tutte le forme sono adattabili. Alcune strutture richiedono un numero preciso di nodi o ruoli. Altre possono scalare.

Un esempio:

pattern line_basic richiede minimum_nodes = 2 e degradable = true;

pattern circle_closed richiede minimum_nodes = 4 e degradable = true, ma se manca chiusura può diventare arc_basic;

pattern bridge_cluster_chain richiede almeno due cluster e una catena connector; se manca un bridge_node critico può fallire;

pattern grid_3x3 può degradare a grid_2x3 o partial_grid.

Queste regole rendono l'adattamento prevedibile. Il sistema non inventa una forma qualsiasi; sceglie tra fallback documentati.

Il quarto livello di adattamento riguarda la topologia. Se una connessione desiderata non si forma, il sistema può tentare un arco alternativo, una connessione ridondante o una struttura parziale. Per esempio, in un reticolo, se un edge manca ma esiste una connessione diagonale di rinforzo, la struttura può restare connessa. In un ponte, se la catena centrale fallisce, il sistema può cercare un percorso alternativo tra i cluster. In un cluster, se un nodo resta isolato, può essere riassegnato a seeker o escluso temporaneamente. L'adattamento topologico è più avanzato dell'adattamento geometrico perché lavora sul grafo delle relazioni.

Una strategia topologica può essere:

se arco desiderato mancante → ritentare connessione;

se arco ancora mancante → cercare arco alternativo;

se arco critico mancante → abortire o degradare struttura;

se arco opzionale mancante → registrare warning e continuare;

se arco indesiderato presente → fase di separazione o correzione.

Questa logica richiede che il sistema distingua archi critici, importanti, opzionali e vietati. Senza questa classificazione, il controller non sa quanto sia grave un errore topologico. Una connessione mancante tra due nodi centrali di un ponte può essere gravissima; una connessione opzionale di rinforzo può essere meno importante. L'adattamento deve quindi essere sensibile alla criticità.

Il quinto livello di adattamento riguarda il tempo. Se la rete è lenta, se gli ack arrivano in ritardo, se molti comandi scadono o se i nodi hanno sync_degraded, il controller può modificare la sequenza temporale. Può aumentare il lead_time, allargare gli slot, rallentare le fasi, ridurre la frequenza della telemetria, inviare sync più frequenti o passare a pattern meno rapidi. Questo tipo di adattamento è direttamente collegato alla Sezione 9. La sincronizzazione non è statica: il sistema può adattare i tempi alla qualità reale della comunicazione.

Una politica temporale può essere:

se RTT medio aumenta → aumentare lead_time;

se jitter aumenta → allargare slot e finestre di validità;

se molti ack sono late → aumentare ack_window;

se sync_status degrada → inviare resync o ridurre complessità;

se molti command_expired → rallentare pattern o abortire fase;

se rete instabile → passare a modalità visuale o idle.

Questo è un adattamento fondamentale. Il sistema non deve continuare a eseguire pattern rapidi quando la rete non li supporta. Deve rallentare, semplificare o fermarsi. Questa capacità rende MicroBot molto più robusto e credibile.

Il sesto livello di adattamento riguarda la modalità operativa. Il sistema può passare da una modalità completa a una modalità conservativa, da hardware-assisted a visual-demo, da pattern dinamico a pattern statico, da attuazione magnetica a solo LED, da controllo completo a diagnostica. Questa è una forma di degradazione controllata. Invece di fallire bruscamente, MicroBot può ridurre le proprie funzioni mantenendo osservabilità e sicurezza.

Una scala di degradazione può essere:

full_mode: pattern completo con simulazione, dashboard, telemetria e attuazione supportata;

conservative_mode: pattern più lento, slot più larghi, comandi meno aggressivi;

visual_mode: nessuna attuazione critica, solo LED/dashboard/simulazione;

diagnostic_mode: raccolta dati e verifica stato;

idle_mode: nodi online ma fermi;

safety_mode: uscite disattivate, comandi critici bloccati.

Questa scala è estremamente utile perché permette al progetto di restare operativo anche quando una parte non è pronta. Se l'hardware magnetico non è sicuro, si può usare visual_mode. Se la rete è instabile, si può passare a diagnostic_mode. Se un nodo entra in safety, si può isolare. L'adattamento non significa sempre "fare di più"; spesso significa sapere quando fare di meno.

Il settimo livello riguarda l'adattamento energetico e termico. Se un nodo ha batteria bassa, thermal_load alto o cooldown attivo, il controller deve modificare il suo ruolo o i comandi che gli invia. Un nodo low_energy può diventare passive_node. Un active_connector con thermal_load alto può essere sostituito da un altro connector o assegnato a una fase di riposo. Un gruppo di nodi con carico elevato può ricevere una fase di cooldown. Questo protegge l'hardware e rende il sistema più sostenibile.

Una politica energetica può essere:

se battery_level basso → ruolo passivo o esclusione da attuazione;

se thermal_load alto → blocco temporaneo di coil o PWM;

se cooldown attivo → rifiuto comandi magnetici;

se molti nodi thermal_high → inserire phase_cooldown;

se energia globale insufficiente → pattern ridotto o visual_mode.

Questo livello è fondamentale quando si passa dal simulatore all'hardware reale. In simulazione si può eseguire qualunque azione senza consumo reale; in hardware ogni attuazione ha costo e rischio. L'adattamento energetico è quindi parte della maturità fisica del progetto.

L'ottavo livello riguarda l'adattamento alla qualità della telemetria. Il controller prende decisioni sulla base dei dati ricevuti. Se la telemetria è incompleta, vecchia o incoerente, l'adattamento deve essere prudente. Non si può correggere aggressivamente un nodo se non si è sicuri del suo stato. Se un dato è stale, il controller può chiedere refresh. Se la telemetria è rumorosa, può usare filtri. Se i dati sono insufficienti, può passare a diagnostica. Questo impedisce correzioni basate su informazioni sbagliate.

Una regola generale può essere:

se qualità_dato bassa → non applicare correzioni critiche; richiedere nuova telemetria o degradare modalità.

Questo principio è importante perché l'adattamento basato su dati errati può peggiorare la situazione. Meglio rallentare e osservare che correggere in modo cieco. In uno swarm, le informazioni sono sempre parziali e ritardate; il controller deve tenerne conto.

L'adattamento dinamico deve essere basato su soglie. Ogni metrica deve avere soglie che determinano quando intervenire. Per esempio, errore geometrico sotto soglia = nessuna correzione. Errore medio = correzione leggera. Errore alto = fase di recovery. Errore critico = abort. Questa classificazione evita reazioni eccessive a piccoli errori. Un sistema che corregge continuamente anche minime deviazioni può oscillare. Un sistema che non corregge mai può divergere. Serve equilibrio.

Una scala di errore può essere:

normal: errore entro tolleranza;

warning: errore sopra tolleranza ma recuperabile;

degraded: errore persistente o multiplo;

critical: errore che compromette pattern o sicurezza.

Per ogni livello, esiste una risposta:

normal → continua;

warning → correzione leggera;

degraded → rallenta, resync o fallback;

critical → abort, idle o safety.

Questa struttura rende l'adattamento prevedibile e documentabile.

L'adattamento deve usare hysteresis anche sulle soglie. Se una metrica oscilla intorno alla soglia, il sistema non deve passare continuamente da uno stato all'altro. Per esempio, se sync_quality oscilla tra ok e degraded, il controller non deve cambiare modalità ogni ciclo. Può richiedere che la

condizione degraded persista per un certo tempo prima di attivare fallback, e che la condizione ok persista prima di tornare normale. Questo evita flapping.

Una regola può essere:

entra in degraded solo se la metrica resta oltre soglia per ΔT_{enter} ;

torna in normal solo se resta sotto soglia per ΔT_{exit} .

Questa logica è essenziale nei sistemi reali, dove misure e rete sono rumorose. L'hysteresis rende l'adattamento stabile.

L'adattamento dinamico può essere centralizzato, locale o ibrido. L'adattamento centralizzato avviene nel controller: il controller osserva lo stato globale, decide modifiche e invia nuovi comandi. L'adattamento locale avviene nel nodo: il nodo rifiuta comandi, applica safety, gestisce cooldown, modifica comportamento locale o passa a idle. L'adattamento ibrido combina entrambi. Nel progetto MicroBot, l'approccio migliore è ibrido. Il controller ha visione globale, ma il nodo deve poter proteggersi e adattarsi localmente senza aspettare la rete.

Esempi di adattamento locale:

rifiutare magnetic_pulse se thermal_load alto;

spegnere output dopo duration_ms;

entrare in safe_idle se controller perso;

ridurre telemetria se low_energy;

ignorare comando scaduto;

sospendere ruolo attivo se safety attiva.

Esempi di adattamento centrale:

ricalcolare pattern con nodi disponibili;

riassegnare ruoli;

aumentare lead_time;

scegliere fallback_geometry;

escludere nodi instabili;

modificare parametri del comportamento.

Questa divisione è molto importante. Il controller gestisce strategia; il nodo gestisce integrità locale. Nessuno dei due livelli deve sostituire completamente l'altro.

L'adattamento dinamico deve essere integrato con i log. Ogni adattamento deve essere registrato con motivo, tempo, metrica scatenante, modifica applicata e risultato. Senza log, il comportamento adattivo diventa difficile da spiegare. Per esempio, se una run passa da circle a arc, bisogna sapere perché: nodo mancante, chiusura fallita, timeout, safety, topology_error. Se un leader viene sostituito, bisogna sapere quale metrica lo ha causato. Questo rende il sistema trasparente.

Un log di adattamento può contenere:

adaptation_id;
timestamp;
pattern_run_id;
trigger_metric;
trigger_value;
threshold;
action_taken;
affected_nodes;
previous_state;
new_state;
result.

Per esempio:

adaptation_id = ADAPT-00017;
trigger = sync_degraded;
node = MB-004;
action = exclude_from_timed_phase;
result = partial_success.

Questo tipo di log è molto utile per debug, documentazione e presentazioni. Mostra che il sistema non si limita a fallire, ma prende decisioni ragionate.

L'adattamento deve essere visibile nella dashboard. Quando il sistema cambia strategia, l'utente deve vederlo. Se un pattern viene degradato, la dashboard dovrebbe mostrare "pattern degraded: node MB-004 unavailable". Se un ruolo viene riassegnato, dovrebbe mostrare il cambio. Se lo slot viene allargato per jitter alto, dovrebbe comparire un warning. Questo rende il comportamento comprensibile. Un sistema adattivo invisibile può sembrare imprevedibile; un sistema adattivo visibile sembra intelligente e controllato.

La dashboard potrebbe mostrare:

adaptation status;
trigger;
affected nodes;
new plan;

fallback active;

confidence;

remaining limitations.

Questo è particolarmente importante per presentazioni tecniche. Chi osserva può capire che MicroBot sta reagendo a condizioni reali, non semplicemente eseguendo uno script.

L'adattamento dinamico deve distinguere tra correzione e fallback. La correzione cerca di riportare il pattern originale alla condizione prevista. Il fallback accetta che il pattern originale non sia raggiungibile e passa a una versione alternativa. Per esempio, se una linea ha spacing errato, il sistema può correggere distanze. Se un nodo necessario manca, può passare a linea più corta. La correzione mantiene l'obiettivo; il fallback cambia l'obiettivo entro limiti documentati.

Questa distinzione è importante:

correction = modifica del comportamento per raggiungere ancora il target originale;

fallback = sostituzione del target con un target alternativo accettabile.

Un sistema maturo deve sapere quando correggere e quando smettere di insistere. Ritentare all'infinito una connessione impossibile è sbagliato. Passare a fallback dopo un certo numero di tentativi è più robusto. Le policy devono definire `retry_limit`, `timeout` e condizioni di fallback.

Un esempio:

se `attach_edge` fallisce una volta → `retry`;

se fallisce tre volte → `fallback topology`;

se il fallback non è possibile → `abort pattern`.

Questo evita loop infiniti e rende il comportamento prevedibile.

L'adattamento dinamico deve considerare il rischio di instabilità. Ogni modifica può avere effetti collaterali. Aumentare coesione può ridurre dispersione ma aumentare collisioni. Aumentare separazione può evitare collisioni ma rompere cluster. Riassegnare un leader può migliorare rete ma cambiare orientamento del pattern. Allargare slot migliora affidabilità ma rallenta la run. Ogni adattamento ha un costo. Il controller deve quindi valutare trade-off, non solo applicare regole isolate.

Una logica corretta è:

ogni adattamento deve avere beneficio atteso e costo accettabile.

Per esempio, se una linea ha piccolo errore laterale, non serve riassegnare ruoli. Una correzione leggera è sufficiente. Se una struttura critica perde connettività, un adattamento più radicale è giustificato. Questa proporzionalità evita eccessi.

L'adattamento può essere classificato per intensità. Un adattamento leggero modifica parametri. Un adattamento medio modifica ruoli o slot. Un adattamento forte modifica geometria o topologia. Un adattamento estremo abortisce la run o entra in safety. Questa scala aiuta il controller a scegliere la risposta minima efficace.

Una scala possibile è:

level_0: nessun adattamento;

level_1: correzione parametri;

level_2: estensione fase o slot;

level_3: riassegnazione ruolo;

level_4: fallback geometrico/topologico;

level_5: abort run;

level_6: safety stop.

Il sistema dovrebbe preferire il livello più basso che può risolvere il problema, salvo emergenze. Questo principio è molto sano: non si usa una risposta drastica per un problema lieve.

L'adattamento dinamico deve essere compatibile con la sicurezza. Il safety layer ha priorità. Nessun adattamento deve aggirare i limiti locali. Se un nodo rifiuta un comando per thermal_lock, il controller non deve continuare a forzare lo stesso comando. Può aspettare cooldown, assegnare ruolo passivo, sostituire il nodo o degradare pattern. Ma non deve bypassare safety. Questo è fondamentale. L'adattamento non è un modo per "insistere"; è un modo per rispettare i vincoli e trovare alternative.

Una regola fondamentale è:

l'adattamento può modificare il piano, ma non può violare safety, limiti hardware o comandi rifiutati per protezione.

Questo deve essere esplicito nella documentazione. Rende il sistema responsabile e ingegneristicamente corretto.

L'adattamento dinamico deve considerare i nodi simulati in modo diverso dai nodi fisici. In simulazione, si possono provare adattamenti aggressivi, cambiare parametri rapidamente, testare fallback complessi e introdurre fault artificiali. Nell'hardware, gli adattamenti devono essere più conservativi. Per esempio, aumentare "magnetic_weight" in simulazione non ha conseguenze fisiche; aumentare duty_cycle su una coil reale può avere conseguenze termiche. Quindi le policy devono dipendere dall'implementation_profile.

Una modalità simulation può permettere:

parametri variabili più ampi;

fault injection;

retry più aggressivi;

test di topologie alternative.

Una modalità hardware_safe deve imporre:

limiti stretti;

cooldown;

conferme;

fallback conservativi;

stop su safety.

Questa distinzione permette di usare la simulazione come laboratorio senza trasferire automaticamente ogni strategia all'hardware.

L'adattamento dinamico deve essere testato attraverso scenari controllati. Non basta implementarlo e sperare che funzioni. Bisogna creare test in cui un nodo manca, un nodo è in ritardo, un nodo entra in safety, una connessione fallisce, la rete peggiora, un pattern non converge, la batteria scende, il clock perde sync. Ogni scenario deve avere risposta prevista. Questo rende il sistema validabile.

Scenari di test utili sono:

node_missing_during_line;

leader_timeout_during_circle;

connector_failure_during_bridge;

cluster_dispersion_high;

grid_cell_missing;

sync_degraded_mid_run;

thermal_lock_active_connector;

late_ack_broadcast;

stale_telemetry_correction;

fallback_circle_to_arc.

Questi test trasformano l'adattamento da idea teorica a funzione reale. Nei file di progetto potrebbero diventare scenari JSON o script di simulazione.

L'adattamento dinamico può essere collegato alla machine learning solo in una fase futura. È importante dirlo con rigore: nelle prime versioni, l'adattamento dovrebbe essere rule-based, cioè basato su regole esplicite, soglie, metriche e fallback. Questo è più semplice, spiegabile e controllabile. In futuro, si potranno usare algoritmi di ottimizzazione, apprendimento rinforzato o modelli predittivi per scegliere parametri e strategie. Ma prima serve una base deterministica e documentata. Saltare subito al machine learning renderebbe il sistema più difficile da verificare.

Una progressione sensata è:

prima: adattamento manuale;

poi: regole esplicite;

poi: soglie e hysteresis;

poi: scoring e fallback automatici;

poi: ottimizzazione parametrica;

solo dopo: apprendimento automatico o predizione.

Questa progressione è molto coerente con l'approccio ingegneristico. Prima si costruisce un sistema comprensibile; poi lo si rende intelligente.

L'adattamento dinamico deve essere legato alla nozione di "confidence". Quando il controller decide una modifica, può assegnare un livello di fiducia alla decisione. Se i dati sono recenti, coerenti e completi, la confidence è alta. Se i dati sono vecchi, parziali o rumorosi, la confidence è bassa. Gli adattamenti critici dovrebbero richiedere confidence alta. Se la confidence è bassa, meglio richiedere telemetria o passare a modalità conservativa.

Una regola può essere:

se confidence alta → applicare correzione;

se confidence media → applicare correzione conservativa;

se confidence bassa → richiedere dati o sospendere fase.

Questo è utile perché il controller spesso lavora con informazioni ritardate. Non tutte le decisioni hanno lo stesso livello di certezza.

L'adattamento dinamico deve includere una memoria dello stato precedente. Se il sistema cambia parametri, deve poter sapere quali erano i parametri prima. Se il fallback fallisce, può tornare a idle o a una configurazione sicura. Se una riassegnazione non funziona, può registrare il fallimento e non ripetere la stessa scelta. Questa memoria evita cicli stupidi. Il controller deve sapere quali adattamenti sono già stati tentati.

Un record può contenere:

adaptation_attempts;

last_failed_strategy;

retry_count;

cooldown_before_retry;

previous_parameters;

previous_roles.

Per esempio, se un connector è stato sostituito da MB-006 e anche MB-006 fallisce, il controller non deve continuare a riassegnare lo stesso ruolo senza criterio. Può dichiarare topology_failure o fallback. Questo rende il comportamento più robusto.

L'adattamento dinamico deve considerare anche l'utente. In alcune modalità, il sistema può adattarsi automaticamente. In altre, può proporre adattamento e aspettare conferma umana. Per esempio, in simulazione può cambiare pattern automaticamente. In hardware reale, se il fallback implica attuazione fisica diversa, potrebbe richiedere conferma. Questo dipende dal livello di

sicurezza e autonomia desiderato. Nella documentazione, si può distinguere tra `autonomous_adaptation` e `assisted_adaptation`.

`Autonomous_adaptation` può essere permesso per:

telemetria;

slot;

parametri simulativi;

ruoli non critici;

fallback visuali.

`Assisted_adaptation` può essere richiesto per:

attuazioni fisiche nuove;

cambi di `safety policy`;

comandi magnetici fuori routine;

reset hardware;

calibrazioni persistenti.

Questa distinzione protegge l'hardware e mantiene controllo umano dove serve.

L'adattamento dinamico è molto importante anche per la presentazione del progetto. Mostrare un sistema che reagisce a un nodo mancante o a un ritardo comunica maturità. Per esempio, una demo potrebbe simulare la perdita di un nodo durante una linea. Il sistema rileva il nodo mancante, ricalcola la linea con i nodi disponibili, aggiorna la dashboard e mostra `partial_success`. Questa demo sarebbe molto più interessante di una semplice animazione perfetta, perché mostra resilienza. MicroBot non deve sembrare un video preprogrammato; deve sembrare un runtime che osserva e reagisce.

Una demo adattiva efficace potrebbe essere:

avvio linea con sei nodi;

uno dei nodi viene marcato offline;

dashboard mostra `node_missing`;

controller ricalcola linea con cinque nodi;

ruoli vengono riassegnati;

pattern continua come `degraded_line`;

log finale mostra `partial_success` e causa.

Questa sequenza racconta molto bene la differenza tra animazione e sistema. L'adattamento diventa parte della narrativa tecnica.

L'adattamento dinamico può anche essere usato per migliorare progressivamente i pattern. Ogni run produce metriche. Se un pattern fallisce spesso per lo stesso motivo, il controller o l'utente può modificare parametri, fasi o fallback. Nel tempo, i pattern diventano più robusti. Anche senza machine learning, il sistema può avere una forma di miglioramento sperimentale: analisi dei log, identificazione degli errori ricorrenti, aggiornamento dei parametri e nuova run. Questo è molto coerente con una metodologia di ricerca/prototipazione.

Un ciclo sperimentale può essere:

esegui pattern;

raccogli metriche;

identifica errore dominante;

modifica parametri o fasi;

riesegui;

confronta risultati.

Questo è un adattamento tra run, diverso dall'adattamento durante la run. Entrambi sono importanti. L'adattamento intra-run reagisce in tempo reale. L'adattamento inter-run migliora il progetto nel tempo.

Si può quindi distinguere:

adattamento realtime: avviene durante la run;

adattamento sperimentale: avviene tra una run e la successiva;

adattamento progettuale: modifica pattern, firmware o architettura.

Questa distinzione è molto utile per la documentazione. Non tutto deve essere automatico e immediato. Molte migliorie nascono dall'analisi dopo l'esperimento.

L'adattamento dinamico deve essere collegato alla correzione degli errori, ma non coincidere completamente con essa. La correzione degli errori si concentra sul riportare il sistema verso il comportamento atteso. L'adattamento è più ampio: può anche cambiare l'obiettivo, ridurre la modalità, riassegnare ruoli, modificare tempi o degradare pattern. Quindi la correzione è una forma specifica di adattamento, ma l'adattamento include anche fallback e riconfigurazione.

In formula concettuale:

correzione $\subset$ adattamento

La correzione dice: "torna al target". Il fallback dice: "usa un target alternativo". La degradazione dice: "riduci funzionalità". La recovery dice: "ripristina coerenza". Tutti sono adattamenti. Questa distinzione prepara il punto successivo sulla correzione degli errori, che sarà più specifico e operativo.

L'adattamento dinamico deve rispettare l'architettura complessiva del sistema. Il controller decide adattamenti globali; i nodi applicano safety e validazione locale; la dashboard mostra stato e decisioni; il simulatore permette test di scenari; i log registrano motivi e risultati. Se uno di questi

livelli manca, l'adattamento diventa meno affidabile. Per esempio, senza dashboard l'utente non capisce cosa è successo. Senza log non si può analizzare. Senza safety locale l'adattamento può essere pericoloso. Senza simulazione è difficile testare. L'adattamento è quindi un comportamento dell'intero ecosistema, non di un solo modulo.

Una pipeline completa può essere:

telemetria/eventi → controller state estimator → adaptation policy → new action_plan or correction command → node validation → execution → feedback → log/dashboard.

Questa pipeline mostra come l'adattamento sia una funzione del runtime. Non è un elemento decorativo; è parte del ciclo di controllo.

Dal punto di vista matematico, l'adattamento può essere visto come una modifica dei parametri del sistema in funzione dello stato. Se $x(t)$ rappresenta lo stato dello swarm e P rappresenta il pattern target, l'errore $e(t)$ misura la distanza tra stato attuale e target. L'adattamento modifica parametri $\theta(t)$, ruoli $r(t)$ o target P' in base a $e(t)$. In forma concettuale:

$$\theta(t+1) = f(\theta(t), e(t), \text{stato_rete}, \text{safety}, \text{vincoli})$$

Questa formula mostra che i parametri non sono fissi. Possono evolvere. Tuttavia, la funzione f deve essere limitata e controllata, altrimenti il sistema può diventare instabile. L'adattamento è quindi un controllo parametrico con vincoli.

Per evitare instabilità, si possono definire limiti:

`cohesion_weight` ∈ [min, max];

`separation_weight` ∈ [min, max];

`max_role_changes_per_run`;

`max_retries_per_phase`;

`min_phase_duration`;

`max_pattern_degradation_level`.

Questi limiti impediscono al sistema di adattarsi troppo aggressivamente. Un adattamento senza limiti può diventare esso stesso una fonte di errore.

L'adattamento dinamico deve tenere conto anche della leggibilità scientifica. Se il sistema cambia troppi parametri contemporaneamente, diventa difficile capire quale modifica ha prodotto quale effetto. Per le prime versioni, è meglio adattare una cosa alla volta o registrare chiaramente le modifiche. Questo rende i test interpretabili. In una tesi o relazione tecnica, poter spiegare “abbiamo aumentato `separation_weight` perché il numero di violazioni di distanza minima era alto” è molto meglio che dire “il sistema ha cambiato vari parametri”.

Una buona pratica è:

adattamenti semplici, tracciati e motivati.

Questo principio mantiene il progetto rigoroso e presentabile.

L'adattamento dinamico può essere collegato ai file di configurazione tramite `adaptation_policy`. Ogni pattern può indicare quali adattamenti sono consentiti. Per esempio, `line_basic` può consentire riduzione del numero di nodi e riassegnazione terminal, ma non cambiare direzione. `Circle_basic` può consentire fallback ad arco. `Bridge_complex` può consentire sostituzione `bridge_node` ma non rimozione di entrambi i cluster. Questa policy rende l'adattamento specifico del pattern.

Una `adaptation_policy` può contenere:

`allowed_parameter_changes;`

`allowed_role_reassignments;`

`allowed_fallbacks;`

`retry_limits;`

`degradation_rules;`

`abort_conditions;`

`safety_conditions.`

Questa struttura è molto utile perché evita adattamenti generici inadatti a certi pattern. Ogni pattern ha le proprie regole.

Per esempio, in `line_basic`:

`allowed_fallbacks = line_shorter;`

`max_missing_nodes = N - minimum_nodes;`

`role_reassignment = true;`

`change_spacing = true` entro limiti.

In `circle_closed`:

`allowed_fallbacks = arc_open;`

`role_reassignment = true;`

`closure_retry_limit = 2.`

In `bridge_complex`:

`allowed_fallbacks = disconnected_clusters, shorter_bridge;`

`bridge_node_replacement = true;`

`critical_cluster_missing = abort.`

Questi esempi mostrano come l'adattamento diventi parte della definizione del pattern.

L'adattamento dinamico deve essere compatibile con la versione del pattern. Se una policy cambia, cambia anche il comportamento della run. Quindi `adaptation_policy` deve essere versionata insieme

al pattern. Un line_basic_v0.1 senza fallback non è lo stesso di line_basic_v0.2 con fallback. Nei log bisogna salvare la versione. Questo rende le run confrontabili.

Un log deve indicare:

pattern_id;

pattern_version;

adaptation_policy_version;

runtime_version;

firmware_versions.

Così, se una run ha successo e un'altra no, si può capire se è cambiato il pattern, la policy o il runtime. Questa precisione è importante in un progetto grande.

L'adattamento dinamico deve anche considerare le limitazioni etiche e fisiche del sistema. Se un adattamento aumentasse l'attuazione oltre limiti sicuri, non deve essere permesso. Se una strategia genera calore eccessivo, va bloccata. Se una sequenza potrebbe danneggiare componenti, deve essere simulata prima. L'adattamento non deve sacrificare sicurezza per completare un pattern. Questo è un principio fondamentale.

Una formulazione chiara è:

il pattern è subordinato alla sicurezza; l'adattamento deve cercare soluzioni dentro i vincoli, non oltre i vincoli.

Questo rende il sistema affidabile.

L'adattamento dinamico può essere visto come la capacità dello swarm di mantenere coerenza sotto perturbazione. Una perturbazione può essere un nodo mancante, un ritardo, un errore, un disturbo fisico, un cambiamento di target o una variazione energetica. Il sistema adattivo non elimina la perturbazione, ma modifica il proprio comportamento per restare entro una regione accettabile. Questa è una proprietà molto importante per qualunque sistema complesso.

In termini concettuali:

robustezza = capacità di mantenere funzione nonostante perturbazioni;

adattamento = meccanismo attivo che modifica comportamento per preservare funzione.

MicroBot deve puntare a entrambi. Alcune funzioni devono essere robuste per design, come safety locale e comandi temporizzati. Altre possono essere adattive, come pattern, ruoli e parametri. La combinazione produce resilienza.

L'adattamento dinamico prepara anche il concetto di comportamento emergente controllato. Un comportamento emergente puro può essere difficile da prevedere. Un comportamento completamente centralizzato può essere rigido. L'adattamento dinamico permette una via intermedia: il controller osserva l'emergenza e modifica parametri o ruoli per orientarla. Lo swarm mantiene regole locali, ma il sistema globale corregge traiettoria e struttura. Questo è molto coerente con la filosofia MicroBot.

In una simulazione, questo può apparire così:

i nodi tendono a clusterizzare;

il controller osserva eccessiva densità;

aumenta separazione;

il cluster si stabilizza;

il controller osserva nodi isolati;

aumenta attrazione per isolati o assegna seeker;

il cluster diventa connesso.

Questo è comportamento emergente guidato. Non ogni posizione è comandata manualmente, ma l'emergenza è regolata.

L'adattamento dinamico deve infine essere descritto come uno dei passaggi chiave verso versioni più intelligenti di MicroBot. Una prima versione può eseguire pattern fissi. Una seconda può correggere errori. Una terza può degradare pattern. Una quarta può riassegnare ruoli. Una quinta può ottimizzare parametri. Una sesta può apprendere strategie. Questa progressione permette di comunicare una roadmap chiara e credibile.

Una roadmap adattiva può essere:

v0.1: pattern fissi senza adattamento;

v0.2: metriche di errore e warning;

v0.3: correzioni manuali;

v0.4: fallback semplici;

v0.5: adattamento temporale a ritardi;

v0.6: riassegnazione ruoli;

v0.7: adattamento geometrico e topologico;

v0.8: policy adattive per pattern;

v0.9: ottimizzazione automatica dei parametri;

v1.0: adattamento predittivo e apprendimento controllato.

Questa roadmap è ordinata, realistica e allineata con l'evoluzione dell'ecosistema MicroBot.

In conclusione, l'adattamento dinamico è la capacità del sistema MicroBot di modificare comportamento, parametri, ruoli, tempi, geometria, topologia o modalità operativa in risposta a condizioni reali osservate durante una run. Esso permette allo swarm di non dipendere da condizioni perfette e di reagire a nodi mancanti, ritardi, errori, bassa energia, instabilità, connessioni fallite o telemetria degradata. L'adattamento deve essere rule-based nelle prime versioni, tracciato nei log, visibile nella dashboard, vincolato dalla safety e definito nelle policy dei pattern. Esso non

sostituisce la progettazione rigorosa, ma la rafforza: un pattern ben definito dice che cosa si vuole ottenere; l'adattamento dice come modificare il piano quando la realtà non coincide con l'ideale. Il punto successivo, dedicato alla correzione degli errori, entrerà più nello specifico delle strategie con cui il sistema rileva deviazioni rispetto al pattern target e applica interventi correttivi per riportare lo swarm verso una configurazione valida, stabile e sicura.

10.6 Correzione degli errori

La correzione degli errori rappresenta il meccanismo attraverso cui il sistema MicroBot rileva una deviazione rispetto al pattern atteso e applica un intervento mirato per riportare lo swarm verso una configurazione valida, stabile e sicura. Dopo aver definito l'adattamento dinamico come capacità generale di modificare strategia, ruoli, parametri, tempi o fallback, è necessario isolare la correzione degli errori come processo più specifico. L'adattamento può cambiare l'obiettivo o degradare il comportamento; la correzione, invece, mantiene l'obiettivo originale e cerca di ridurre la distanza tra stato reale e stato desiderato.

Nel progetto MicroBot, un errore non deve essere interpretato soltanto come un guasto grave. Esistono errori piccoli, errori temporanei, errori geometrici, errori topologici, errori temporali, errori di comunicazione, errori di ruolo, errori di configurazione, errori di safety e errori di esecuzione. Una linea leggermente storta è un errore geometrico. Una connessione mancante in una catena è un errore topologico. Un nodo che risponde fuori slot è un errore temporale. Un nodo che ha ruolo sbagliato è un errore operativo. Un comando rifiutato per `thermal_lock` è un errore di esecuzione legato alla sicurezza. La correzione degli errori deve quindi partire da una classificazione precisa, perché ogni tipo di errore richiede una risposta diversa.

Il primo principio è che non si può correggere ciò che non si misura. Per correggere un pattern, il sistema deve confrontare lo stato attuale dello swarm con lo stato target. Lo stato attuale può includere posizioni simulate, ruoli, connessioni, fasi, telemetria, `sync_status`, `ack`, eventi e `safety_state`. Lo stato target è definito dal pattern: geometria desiderata, topologia desiderata, ruoli attesi, fase corrente, slot previsti e metriche di successo. L'errore nasce dalla differenza tra questi due livelli.

In forma concettuale:

$$\text{errore}(t) = \text{stato_atteso}(t) - \text{stato_osservato}(t)$$

Questa formula non deve essere intesa solo in senso numerico. Per la geometria, l'errore può essere una distanza. Per la topologia, può essere un arco mancante. Per i ruoli, può essere una differenza tra `expected_role` e `reported_role`. Per il tempo, può essere una risposta tardiva. Per la safety, può essere un comando rifiutato. La correzione degli errori richiede quindi metriche diverse, ma tutte seguono la stessa logica: identificare una deviazione e ridurla.

Il secondo principio è che la correzione deve essere proporzionata. Non ogni errore richiede una risposta drastica. Se un nodo è leggermente fuori linea, può bastare una correzione dei parametri o un piccolo comando di riallineamento. Se una connessione opzionale manca, può bastare registrare un warning. Se un nodo critico entra in safety, invece, può essere necessario interrompere la fase. La risposta deve essere coerente con gravità, persistenza e criticità dell'errore.

Una scala utile può essere:

errore lieve: correzione locale o nessun intervento immediato;

errore medio: correzione di parametro, ruolo o slot;

errore persistente: fase di recovery o retry;

errore strutturale: fallback o ricalcolo del pattern;

errore critico: abort, idle o safety.

Questa scala impedisce al sistema di reagire troppo a piccoli disturbi o troppo poco a problemi gravi. La correzione deve essere stabile, non isterica.

Il terzo principio è che la correzione deve rispettare la safety locale. Nessun intervento correttivo deve forzare un nodo a superare limiti energetici, termici, temporali o hardware. Se un nodo rifiuta un comando perché è in thermal_lock, la correzione non deve insistere aumentando duty cycle o ripetendo impulsi. Deve scegliere una strategia sicura: attendere cooldown, riassegnare ruolo, escludere il nodo, degradare il pattern o entrare in idle. Questo principio è fondamentale perché la correzione degli errori non deve diventare una causa di rischio.

Una regola centrale è:

un errore di pattern non giustifica la violazione dei vincoli di sicurezza.

Il pattern è un obiettivo; la sicurezza è un limite superiore. Il sistema può correggere solo dentro i vincoli ammessi.

Il primo tipo di errore da analizzare è l'errore geometrico. Esso indica che i nodi non si trovano nella posizione, regione o forma prevista. Nella simulazione, questo errore può essere calcolato direttamente confrontando posizione attuale e posizione target. In una linea, l'errore geometrico può essere la distanza perpendicolare dall'asse. In un cerchio, l'errore radiale rispetto al raggio. In una griglia, la distanza dalla cella assegnata. In un cluster, la dispersione rispetto al centro o al raggio massimo. L'errore geometrico è il più intuitivo perché riguarda la forma visibile.

Per una linea, si può misurare:

errore_laterale;

errore_di_spacing;

errore_di_ordine.

Se l'errore laterale è alto, significa che i nodi non sono allineati. Se l'errore di spacing è alto, significa che le distanze tra vicini non rispettano il passo desiderato. Se l'errore di ordine è alto, significa che i nodi sono scambiati o disposti in sequenza sbagliata. Ogni errore richiede una correzione diversa. L'allineamento si corregge aumentando l'attrazione verso l'asse. Lo spacing si corregge agendo sulle distanze tra vicini. L'ordine si corregge riassegnando target o ruoli.

Per un cerchio, gli errori principali sono:

errore_radiale;

errore_angolare;

errore_di_chiusura.

L'errore radiale indica che i nodi sono troppo vicini o troppo lontani dal centro. L'errore angolare indica che non sono distribuiti uniformemente. L'errore di chiusura indica che l'anello non è

completo o che l'ultimo nodo non si collega correttamente al primo. Le correzioni possono essere: aumentare attrazione radiale, aumentare separazione tangenziale, riassegnare angoli o ritentare la chiusura topologica.

Per una griglia, gli errori geometrici includono:

celle vuote;

nodi fuori cella;

righe non allineate;

colonne deformate;

passo non uniforme.

Le correzioni possono essere locali, agendo su singole celle, oppure globali, correggendo orientamento e scala della griglia. Se molte celle sono deformate nello stesso modo, potrebbe essere un errore globale di trasformazione. Se una sola cella è sbagliata, è un errore locale. Questa distinzione è importante: correggere globalmente un errore locale può peggiorare tutto il pattern.

Per un cluster, gli errori geometrici sono diversi perché non esiste una posizione target rigida per ogni nodo. Gli errori principali sono dispersione eccessiva, collisioni o nodi isolati. Se il cluster è troppo disperso, si aumenta coesione. Se i nodi sono troppo vicini, si aumenta separazione. Se un nodo è isolato, può diventare seeker o ricevere un target verso il centro del cluster. Il cluster è quindi un buon esempio di correzione basata su parametri comportamentali invece che su target individuali.

Il secondo tipo di errore è l'errore topologico. Esso riguarda le connessioni tra nodi. Una struttura può avere geometria quasi corretta ma topologia sbagliata. Per esempio, quattro nodi possono formare visivamente un cerchio, ma se la connessione tra ultimo e primo manca, il ciclo non è completo. Una linea può sembrare allineata, ma se un nodo centrale non è connesso ai vicini, la catena è spezzata. Un reticolo può apparire regolare, ma se mancano archi interni, la struttura è fragile. La topologia misura le relazioni, non solo le posizioni.

Gli errori topologici principali sono:

arco desiderato mancante;

arco indesiderato presente;

arco critico instabile;

nodo isolato;

grafo disconnesso;

connessione duplicata o ambigua.

Un arco desiderato mancante può richiedere una fase di attach, un riallineamento dei nodi coinvolti o una sostituzione del nodo. Un arco indesiderato presente può richiedere separazione, rimozione del vincolo o fase di detach. Un grafo disconnesso può richiedere la costruzione di un ponte o fallback. Un nodo isolato può diventare seeker o essere escluso. L'errore topologico è spesso più

importante dell'errore geometrico nelle strutture complesse, perché la funzione della struttura dipende dalle relazioni.

Una metrica topologica può distinguere:

missing_edges;

extra_edges;

wrong_edges;

unstable_edges;

critical_edge_failures.

Non tutti gli archi hanno lo stesso peso. La mancanza di un arco opzionale può essere tollerabile; la mancanza di un bridge_edge critico può interrompere la struttura. La correzione deve quindi considerare la criticità dell'arco. Se manca un edge critico, il sistema può ritentare immediatamente o abortire. Se manca un edge opzionale, può continuare con warning. Questa distinzione evita di trattare ogni errore topologico allo stesso modo.

Il terzo tipo di errore è l'errore temporale. Esso riguarda la mancata esecuzione nei tempi previsti. Un nodo può rispondere fuori slot, ricevere un comando dopo expires_at, eseguire con execution_error troppo alto, perdere un sync o restare in una fase precedente. Gli errori temporali sono particolarmente importanti perché possono causare errori geometrici e topologici. Se un nodo esegue in ritardo una fase di attach, può mancare una connessione. Se risponde fuori slot, il controller può prendere decisioni con dati incompleti. Se perde sync, non può partecipare a comandi temporizzati.

Gli errori temporali principali sono:

late_ack;

missed_slot;

command_expired;

execution_error alto;

sync_degraded;

sync_lost;

out_of_phase.

La correzione può includere resync, aumento del lead_time, ampliamento degli slot, riduzione della velocità del pattern, esclusione del nodo da fasi critiche o passaggio a modalità conservativa. Se l'errore temporale è isolato, può bastare un warning. Se è persistente, indica che il nodo o la rete non sono adatti a quella sequenza. In quel caso bisogna adattare tempi o ridurre complessità.

Una regola utile è:

se un errore temporale si ripete per lo stesso nodo, degradare il suo ruolo o escluderlo da azioni temporizzate critiche.

Questo impedisce che un nodo con rete instabile continui a compromettere il pattern. Il nodo può restare online e inviare telemetria, ma non deve essere `bridge_node` o `active_connector` in fasi precise.

Il quarto tipo di errore è l'errore di ruolo. Esso avviene quando un nodo non possiede il ruolo atteso, non lo ha applicato, lo ha perso, lo riporta in modo incoerente o non può eseguirlo. Per esempio, il controller si aspetta che MB-003 sia connector, ma il nodo riporta `role = idle`. Oppure MB-004 è assegnato a leader ma ha `sync_degraded`. Oppure un nodo con ruolo `active_connector` non possiede `capability coil_pwm`. Questi errori impediscono una coordinazione corretta.

Gli errori di ruolo possono essere:

`role_mismatch;`

`unsupported_role;`

`capability_missing_for_role;`

`role_not_confirmed;`

`role_conflict;`

`role_expired;`

`role_suspended_by_safety.`

La correzione può essere inviare nuovamente `assign_role`, eseguire `state_sync`, riassegnare il ruolo a un altro nodo, cambiare il ruolo in uno compatibile o abortire se il ruolo è critico. Se un ruolo è opzionale, il pattern può continuare. Se è critico, bisogna intervenire. La dashboard deve mostrare chiaramente `expected_role` e `reported_role` per evitare ambiguità.

Il quinto tipo di errore è l'errore di comunicazione. Esso include pacchetti persi, ack mancanti, timeout, duplicati, pacchetti fuori ordine, target errato, source non autorizzata o messaggi malformati. Questi errori appartengono alla Sezione 8, ma influenzano direttamente i pattern. Un pattern può fallire non perché la geometria sia sbagliata, ma perché i comandi non sono arrivati o le risposte non sono state ricevute. La correzione deve distinguere tra errore di pattern ed errore di comunicazione, altrimenti si rischia di correggere la forma quando il problema è la rete.

Gli errori di comunicazione possono essere:

`no_ack;`

`timeout;`

`duplicate_message;`

`invalid_payload;`

`unsupported_command;`

`wrong_target;`

`packet_out_of_order;`

source_rejected.

Le correzioni includono retry controllato, deduplicazione, request_status, discovery, aggiornamento della tabella node_id → IP, riduzione del traffico, slot di ack più ampi o debug di rete. Se un comando è idempotente, può essere ritrasmesso. Se non è idempotente, bisogna usare lo stesso message_id o retry_of. Questo evita esecuzioni duplicate.

Il sesto tipo di errore è l'errore di configurazione. Esso avviene quando controller, nodo, pattern o simulatore non condividono la stessa versione di protocollo, firmware, config_epoch, pattern_version, safety_policy o role mapping. Questo tipo di errore può essere sottile. Il nodo può rispondere, ma interpretare un comando in modo diverso. Oppure può avere capabilities non aggiornate. Oppure può usare una vecchia configurazione del pattern. La correzione richiede state_sync, config update, esclusione del nodo incompatibile o blocco della run.

Gli errori di configurazione includono:

protocol_version_mismatch;

firmware_version_unsupported;

config_epoch_mismatch;

pattern_version_mismatch;

safety_policy_mismatch;

capability_table_stale.

Questi errori sono molto importanti nei progetti grandi, perché file, firmware e dashboard possono evolvere separatamente. Il controller deve verificare compatibilità prima di avviare pattern complessi. Se la configurazione non coincide, la correzione più sicura è fermare la run e riallineare, non tentare di procedere.

Il settimo tipo di errore è l'errore di safety o di esecuzione fisica. Questo include thermal_lock, low_energy, output_blocked, cooldown_active, hardware_fault, overcurrent, battery_low, invalid_duty_cycle o duration_too_long. Questi errori non vanno "corretti" forzando il nodo, ma gestiti rispettando il limite. La correzione può consistere in riassegnazione di ruolo, esclusione, cooldown, riduzione del pattern, modalità visuale o safety_stop. In un sistema fisico, questi errori sono prioritari.

Un errore safety può generare:

isola nodo;

disabilita output;

riduci telemetria;

richiedi diagnostica;

passa a idle;

abortisci fase critica.

La correzione degli errori deve sempre lasciare al nodo il diritto di rifiutare comandi non sicuri. Il controller può proporre correzioni, ma il nodo decide localmente se sono ammissibili.

La pipeline generale di correzione può essere descritta in più fasi. La prima fase è detection, cioè rilevamento dell'errore. La seconda è classification, cioè determinare il tipo di errore. La terza è severity assessment, cioè valutare gravità e criticità. La quarta è strategy selection, cioè scegliere la risposta correttiva. La quinta è execution, cioè applicare la correzione. La sesta è verification, cioè controllare se l'errore è stato ridotto. La settima è logging, cioè registrare causa, azione e risultato.

La sequenza è:

detect → classify → evaluate → correct → verify → log

Questa pipeline è essenziale. Se il sistema rileva un errore ma non lo classifica, può applicare correzione sbagliata. Se lo classifica ma non valuta gravità, può reagire male. Se corregge ma non verifica, non sa se ha funzionato. Se non registra, non si può analizzare dopo. La correzione degli errori deve essere un processo completo.

La fase di detection può avvenire in più modi. Il controller può rilevare errori confrontando pattern target e stato osservato. Il nodo può rilevare errori locali, come comando scaduto o safety. La dashboard può mostrare anomalie ma non dovrebbe essere il motore primario di detection critica. Il simulatore può generare eventi di errore durante test. L'origine dell'errore deve essere registrata, perché un errore rilevato localmente dal nodo è diverso da uno stimato dal controller.

Le fonti di detection possono essere:

controller metrics;

node safety layer;

telemetry events;

ack results;

simulation engine;

dashboard diagnostics;

manual operator observation.

Una buona architettura non dipende da una sola fonte. Per esempio, un attach_event può essere confermato dal nodo, ma la dashboard può anche visualizzare la connessione. Una geometria può essere misurata dal simulatore. Un errore fisico può essere segnalato dal firmware. Questo rende la detection più robusta.

La fase di classification deve trasformare un evento grezzo in una categoria. Per esempio, "MB-004 non ha risposto" può essere timeout, offline, delayed, wrong_ip, firmware crash o rete instabile. All'inizio, il controller può classificare semplicemente come no_ack. Poi può approfondire con request_status, ping, discovery o last_seen. La classificazione può quindi essere progressiva: prima generica, poi più precisa.

Un esempio:

evento iniziale: no_ack da MB-004;

controllo last_seen: heartbeat recente presente;

classificazione aggiornata: command_ack_timeout ma nodo online;

azione: retry o request_status;

se retry fallisce e heartbeat continua: comando rifiutato/non processato;

se heartbeat sparisce: nodo offline.

Questa logica evita conclusioni affrettate. Un no_ack non significa necessariamente nodo morto. Può significare ritardo, perdita del pacchetto, ack perso o comando scaduto. La correzione deve essere basata su diagnosi.

La fase di severity assessment valuta quanto l'errore compromette il pattern. Un errore geometrico lieve su un nodo optional può essere warning. Un errore topologico su un bridge critical può essere critical. Un sync_degraded su un diagnostic_node può essere tollerabile; su leader temporizzato può essere grave. La gravità dipende da tipo di errore, ruolo del nodo, fase corrente, criticità della struttura e possibilità di fallback.

Una funzione di severità può considerare:

error_type;

node_role;

edge_criticality;

phase_criticality;

persistence_duration;

safety_involvement;

number_of_nodes_affected.

Questo è importante perché lo stesso errore può avere gravità diversa. Un late_ack durante report è meno grave di un late_ack durante attach. Un nodo offline prima della run può essere gestito; un nodo offline durante una fase magnetica può essere critico. La correzione deve essere sensibile al contesto.

La fase di strategy selection sceglie il tipo di intervento. Le strategie principali sono: retry, resync, state_sync, parameter_adjustment, role_reassignment, local_correction, topology_repair, phase_extension, fallback, isolation, abort e safety_stop. Ognuna ha un campo di applicazione.

Retry serve quando si sospetta perdita di pacchetto o risposta. Resync serve quando il problema è temporale. State_sync serve quando il nodo è fuori fase o configurazione. Parameter_adjustment serve per errori geometrici o comportamentali. Role_reassignment serve se un ruolo critico è perso. Topology_repair serve per connessioni mancanti o extra. Phase_extension serve se la fase richiede più tempo. Fallback serve se il pattern originale non è più raggiungibile. Isolation serve per nodi problematici. Abort serve per interrompere la run. Safety_stop serve per protezione immediata.

Questa mappa può essere espressa così:

errore comunicativo → retry / request_status / discovery;
errore temporale → resync / lead_time maggiore / slot più ampio;
errore geometrico → parameter_adjustment / target correction;
errore topologico → topology_repair / attach retry / detach;
errore di ruolo → assign_role / role_reassignment;
errore configurazione → state_sync / config update / abort;
errore safety → isolation / cooldown / abort / safety_stop.

Questa tabella è una base molto utile per implementare il controller.

La fase di execution applica la correzione. Deve essere tracciata come qualsiasi altro comando. Se il controller invia una correzione, deve generare message_id, timestamp, target, command e reason. Il nodo deve rispondere. La correzione non deve essere invisibile. Per esempio, se il controller riassegna un ruolo, deve loggare perché. Se aumenta lead_time, deve loggare il trigger. Se ritenta una connessione, deve associare il retry al missing_edge. Questo rende il comportamento interpretabile.

Un comando correttivo può avere:

correction_id;
reason;
ref_error_id;
target;
command;
payload;
expected_effect.

Per esempio:

correction_id = CORR-00031;
reason = missing_edge MB-002—MB-003;
command = retry_attach;
target = group edge_pair_002_003.

Questo livello di dettaglio è molto utile per debug e report.

La fase di verification controlla se la correzione ha funzionato. Se l'errore geometrico diminuisce, la correzione è efficace. Se l'arco mancante appare, topology_repair riuscito. Se il nodo torna sync_good, resync riuscito. Se l'errore persiste, il sistema può provare un'altra strategia o passare a

fallback. Senza verification, il controller potrebbe continuare a credere che la correzione abbia avuto effetto senza prove.

Una verifica può essere:

misura errore prima;

applica correzione;

attendi finestra di osservazione;

misura errore dopo;

se errore diminuisce sotto soglia → success;

se diminuisce ma resta alto → continue correction;

se non cambia → strategy_failed;

se peggiora → rollback o fallback.

Questa logica è molto simile al controllo sperimentale. La correzione deve produrre un miglioramento misurabile.

La correzione degli errori deve avere limiti di retry. Se un attach fallisce cinque volte, continuare a ritentare può essere inutile o rischioso. Se un nodo non risponde dopo tre retry, meglio marcarlo problematico. Se un resync fallisce ripetutamente, il nodo deve essere escluso da fasi temporizzate. I retry infiniti sono un errore progettuale. Ogni strategia correttiva deve avere retry_limit e timeout.

Una policy può essere:

retry comunicativo massimo 3 volte;

retry attach massimo 2 volte per fase;

resync massimo 3 tentativi prima di sync_lost;

role_reassignment massimo 1 o 2 volte per ruolo critico;

topology_repair massimo secondo criticality;

dopo retry_limit → fallback o abort.

Questa policy mantiene il sistema controllato. Evita loop infiniti e protegge hardware e rete.

La correzione degli errori deve distinguere tra correzione locale e globale. Una correzione locale coinvolge un nodo, una connessione o un piccolo gruppo. Una correzione globale modifica parametri dell'intero swarm, fase o pattern. In generale, è meglio iniziare con correzioni locali quando il problema è locale. Se un solo nodo è fuori posizione, non ha senso modificare tutta la linea. Se tutto il cluster è disperso, allora serve correzione globale. Questa distinzione riduce effetti collaterali.

Esempi di correzione locale:

request_status su un nodo;

resync unicast;

riassegnazione ruolo di un nodo;

retry attach tra due nodi;

correzione target di un nodo;

isolamento di un nodo in safety.

Esempi di correzione globale:

aumento cohesion_weight;

allargamento slot per tutto lo swarm;

fallback da cerchio ad arco;

restart della fase;

abort del pattern;

state_sync broadcast.

La scelta dipende dalla distribuzione dell'errore. Se l'errore è concentrato, correzione locale. Se è sistemico, correzione globale.

La correzione degli errori deve essere integrata con i pattern semplici. Per la linea, le correzioni principali sono riallineamento, spacing correction, reorder e reconnect. Il riallineamento riduce errore laterale. Lo spacing correction regola distanze. Il reorder corregge nodi scambiati. Il reconnect ripara archi mancanti nella catena. Queste correzioni possono essere implementate nel simulatore e mostrate nella dashboard.

Per il cerchio, le correzioni sono radial correction, angular redistribution, closure repair e collision separation. La radial correction porta i nodi verso il raggio target. Angular redistribution uniforma gli angoli. Closure repair tenta connessione ultimo-primo. Collision separation aumenta distanza tra vicini troppo vicini. Se closure repair fallisce, fallback ad arco.

Per la griglia, le correzioni sono cell reassignment, row alignment, column alignment, edge repair e partial_grid fallback. Se una cella è vuota e un nodo è fuori cella, si può riassegnare. Se una riga è deformata, si corregge la riga. Se una colonna è deformata, si corregge la colonna. Se mancano archi, si tenta repair. Se troppe celle mancano, si riduce la griglia.

Per il cluster, le correzioni sono cohesion increase, separation increase, isolated_node_recovery, density balancing e stabilization. Se un nodo è isolato, diventa seeker. Se il cluster è troppo compatto, aumenta separazione. Se è troppo disperso, aumenta coesione. Se oscilla, aumenta smorzamento. Il cluster è un buon caso di correzione parametrica.

La correzione degli errori nelle strutture complesse deve essere gerarchica. Se un sub-pattern fallisce, si può correggere il sub-pattern senza modificare tutta la struttura. Se il sub-pattern non può essere recuperato, si modifica la struttura globale. Per esempio, in un ponte, se la catena centrale ha un nodo fuori ruolo, si corregge la catena. Se la catena intera fallisce, il ponte può degradare a due

cluster separati. In un reticolo, se una cella è deformata, si corregge localmente; se molte celle sono deformate, si ricalcola la griglia o si passa a partial_lattice.

Una strategia gerarchica è:

correggi nodo;

se non basta, correggi edge;

se non basta, correggi sub-pattern;

se non basta, correggi struttura globale;

se non basta, fallback o abort.

Questa gerarchia evita interventi troppo grandi su problemi piccoli e fornisce una scala chiara di escalation.

La correzione degli errori deve essere collegata all'adattamento dinamico ma mantenere obiettivo. Se la correzione riesce, il pattern originale viene preservato. Se fallisce, il sistema può passare ad adattamento più ampio o fallback. Quindi, correzione e adattamento lavorano in sequenza. Prima si prova a correggere. Poi, se l'errore è persistente o non correggibile, si adatta l'obiettivo.

Una policy può essere:

errore rilevato;

tentativo di correzione entro retry_limit;

se successo → continua pattern originale;

se fallimento ma fallback disponibile → adatta pattern;

se nessun fallback → abort o safety secondo criticità.

Questa sequenza è ordinata e leggibile. Permette di spiegare chiaramente perché una run è diventata partial_success invece di success.

La correzione degli errori deve essere visibile nella dashboard. Quando viene rilevato un errore, la dashboard dovrebbe evidenziare nodo, connessione, fase o metrica coinvolta. Quando viene applicata una correzione, dovrebbe mostrare lo stato correction_pending, correction_applied, correction_success o correction_failed. Questo è molto importante per demo e debugging. Se la dashboard mostra solo il risultato finale, si perde la parte più intelligente del sistema. La correzione degli errori è una funzione da far vedere.

Una dashboard potrebbe mostrare:

error_id;

type;

severity;

affected_node o affected_edge;

correction_strategy;

attempt_count;

current_result;

next_action.

Per esempio:

ERROR-021: missing_edge MB-002—MB-003;

severity: medium;

correction: retry_attach;

attempt: 1/2;

status: pending.

Questa visualizzazione rende MicroBot molto più professionale e controllabile.

La correzione degli errori deve essere registrata nei log finali della run. Ogni errore e correzione devono essere parte del report. Questo permette di capire se una run è stata pulita o se è riuscita solo grazie a molte correzioni. Due run possono avere entrambe success, ma una con zero correzioni e una con dieci correzioni. La prima è più stabile. Il report deve quindi includere numero di errori, correzioni applicate, correzioni riuscite, correzioni fallite e fallback.

Metriche di correzione utili sono:

total_errors;

errors_by_type;

corrections_attempted;

corrections_successful;

corrections_failed;

average_correction_time;

retry_count;

fallback_triggered;

safety_abort_count.

Questi dati trasformano la correzione in metrica di qualità. Un sistema che richiede meno correzioni è più stabile. Un sistema che corregge bene è più robusto. Un sistema che corregge troppo spesso può essere fragile ma resiliente. Queste sfumature sono importanti.

La correzione degli errori deve essere testata con scenari specifici. Ogni categoria di errore dovrebbe avere test. Per esempio, un test in cui un nodo è fuori linea, uno in cui una connessione manca, uno in cui un ack arriva tardi, uno in cui un nodo ha ruolo sbagliato, uno in cui

config_epoch non coincide, uno in cui thermal_lock blocca un comando. Il sistema deve produrre risposta prevista.

Test possibili:

line_lateral_error_correction;

line_spacing_error_correction;

circle_radial_error_correction;

circle_closure_failure_fallback;

grid_missing_cell_recovery;

cluster_isolated_node_recovery;

bridge_missing_edge_repair;

role_mismatch_state_sync;

sync_lost_resync;

thermal_lock_role_degradation.

Questi test possono essere simulati prima. In hardware, si possono testare versioni sicure: LED, ruoli, timeout, telemetria. La parte magnetica deve essere testata solo con limiti severi.

La correzione degli errori deve essere implementata inizialmente in modo semplice. Non serve costruire subito un sistema automatico complesso. Una prima versione può solo rilevare e mostrare errori. Una seconda può suggerire correzioni. Una terza può applicare correzioni manuali. Una quarta può automatizzare correzioni sicure. Una quinta può correggere dinamicamente pattern e ruoli. Questa progressione è più controllabile.

Una roadmap può essere:

v0.1: metriche di errore e dashboard warning;

v0.2: classificazione errori;

v0.3: log errori e report finale;

v0.4: correzioni manuali tramite dashboard;

v0.5: correzioni automatiche in simulazione;

v0.6: correzioni automatiche per LED/visual demo;

v0.7: correzioni conservative hardware-safe;

v0.8: topology repair e role reassignment;

v0.9: fallback automatici strutturati;

v1.0: correction policy completa per pattern.

Questa roadmap mantiene coerenza con lo sviluppo incrementale del progetto.

Dal punto di vista fisico, la correzione degli errori deve essere più prudente rispetto alla simulazione. In simulazione, si può correggere posizione, parametri e topologia liberamente. Nell'hardware reale, il sistema può non avere controllo diretto sulla posizione. Può correggere tramite LED, ruoli, comandi magnetici limitati, feedback e istruzioni, ma non deve dichiarare capacità non presenti. Quindi la documentazione deve distinguere tra `correction_simulation`, `correction_visual` e `correction_physical`.

`Correction_simulation` può modificare posizioni virtuali, forze e target. `Correction_visual` può cambiare LED, ruoli e dashboard. `Correction_physical` può inviare comandi reali solo se hardware e safety lo supportano. Questa distinzione è fondamentale per non confondere modello e prototipo.

Per esempio:

`line_spacing_error` in simulazione → modifica target/forze;

`line_spacing_error` in visual demo → dashboard evidenzia errore e LED segnala nodo;

`line_spacing_error` in hardware futuro → attuazione fisica solo se locomozione/aggancio supportati.

Questa chiarezza mantiene il documento tecnicamente onesto.

La correzione degli errori può anche essere assistita dall'operatore umano nelle prime versioni. La dashboard può rilevare errore e suggerire: “MB-003 fuori ruolo, inviare `state_sync`?” oppure “Nodo MB-004 non risponde, eseguire `discovery`?” oppure “Chiusura cerchio fallita, passare ad arco?” Questo approccio `assisted correction` è molto utile quando l'automazione completa non è ancora sicura. L'utente resta nel controllo, ma il sistema fornisce diagnosi e azioni.

In futuro, le correzioni più sicure e frequenti potranno diventare automatiche. Per esempio, `resync` di un nodo, richiesta status, `retry` di un comando idempotente, `update` di role visuale o `fallback` simulativo possono essere automatici. Attuazioni fisiche critiche possono restare assistite. Questa distinzione è molto sana.

La correzione degli errori deve considerare anche la causa radice. Correggere l'effetto senza capire la causa può portare a ripetizioni. Se una linea è sempre storta perché il target è calcolato male, aumentare allineamento non risolve davvero. Se un nodo perde sempre sync perché la rete è instabile, ritentare comandi non basta. Se una connessione fallisce perché i ruoli sono sbagliati, ritentare attach non risolve. Il controller dovrebbe registrare errori ricorrenti e permettere analisi `root cause`.

Una diagnosi può distinguere:

`symptom`: cosa si osserva;

`cause_candidate`: possibile causa;

`correction`: intervento applicato;

`outcome`: risultato.

Per esempio:

`symptom` = `missing_edge`;

```
cause_candidate = node_out_of_phase;
```

```
correction = state_sync;
```

```
outcome = edge repaired.
```

Questa struttura aiuta a migliorare il sistema nel tempo. La correzione non è solo reazione immediata; è anche apprendimento progettuale.

La correzione degli errori deve essere collegata ai file di pattern tramite `correction_policy`. Ogni pattern può specificare quali errori riconosce e quali correzioni sono consentite. Per `line_basic`, si definiscono `spacing_error` e `lateral_error`. Per `circle_basic`, `radial_error` e `closure_error`. Per `bridge_complex`, `missing_bridge_edge` e `cluster_disconnect`. Questa policy evita correzioni generiche inadatte. Un pattern sa quali errori sono rilevanti per sé.

Una `correction_policy` può includere:

```
error_types;
```

```
detection_metrics;
```

```
thresholds;
```

```
severity_rules;
```

```
allowed_corrections;
```

```
retry_limits;
```

```
fallback_after_failure.
```

Questa struttura è molto utile per trasformare la correzione da codice sparso a parte formale del pattern. Ogni pattern diventa auto-diagnostico.

Un esempio per `circle_basic`:

```
radial_error_threshold;
```

```
angular_variance_threshold;
```

```
closure_required = true;
```

```
closure_retry_limit = 2;
```

```
fallback_if_closure_failed = arc_open.
```

Un esempio per `cluster_basic`:

```
max_cluster_radius;
```

```
min_pairwise_distance;
```

```
isolated_node_threshold;
```

```
correction_if_dispersion = increase_cohesion;
```

`correction_if_collision = increase_separation.`

Queste regole sono semplici ma potenti.

La correzione degli errori deve essere compatibile con la versione del runtime. Se una vecchia versione del firmware non supporta un comando correttivo, il controller deve saperlo. Per esempio, `retry_attach` può essere supportato solo da nodi con `connection_manager`. `state_sync` può richiedere firmware v0.3. `correction_led_signal` può essere disponibile su nodi LED. Quindi la `correction_policy` deve consultare `capabilities` e `protocol_version`. Altrimenti la correzione stessa può fallire per `unsupported_command`.

Una regola:

prima di applicare una correzione, verificare che tutti i nodi coinvolti supportino i comandi necessari.

Se non li supportano, scegliere correzione alternativa o fallback. Questo è particolarmente importante in un ecosistema con nodi simulati e fisici diversi.

La correzione degli errori può anche influenzare il punteggio finale della run. Una run che ha raggiunto il pattern dopo molte correzioni può essere `success_with_corrections`. Una run che ha richiesto fallback è `partial_success`. Una run che ha fallito correzione è `failed`. Questa classificazione è più precisa.

Possibili risultati finali:

`success_clean;`

`success_with_corrections;`

`partial_success_with_fallback;`

`failed_after_corrections;`

`aborted_by_safety;`

`invalid_due_to_configuration.`

Questi stati rendono i report molto più informativi. “Success” da solo non dice abbastanza. Un successo pulito e un successo dopo tre recovery sono diversi.

La correzione degli errori deve essere collegata alla narrativa del progetto come dimostrazione di intelligenza sistemica. Un sistema che riesce solo in condizioni perfette sembra fragile. Un sistema che rileva errori, li classifica, li corregge o degrada in modo sicuro mostra maturità. Per MicroBot, la correzione degli errori è una delle funzioni che distinguono un semplice simulatore da un runtime di swarm. Non si tratta solo di far muovere punti sullo schermo; si tratta di gestire deviazioni, ritardi, nodi mancanti e strutture incomplete.

Una demo forte potrebbe mostrare volontariamente un errore: un nodo simulato viene spostato fuori linea, il sistema rileva `lateral_error`, applica correzione, la linea torna stabile. Oppure un nodo viene marcato offline, il sistema ricalcola la linea con un nodo in meno e registra `partial_success`. Oppure un comando arriva tardi, il sistema lo rifiuta come `expired` e non esegue un'azione fuori fase. Queste demo raccontano molto più di una sequenza perfetta.

Dal punto di vista teorico, la correzione degli errori è il meccanismo con cui il sistema riduce l'entropia comportamentale dello swarm. Le unità locali possono deviare, la rete può introdurre ritardi, la topologia può essere incompleta, ma il controller e i nodi usano feedback per riportare il sistema verso una regione ordinata. Questa regione ordinata è definita dal pattern. Il pattern è l'obiettivo; l'errore misura la distanza; la correzione è la forza informativa che riduce tale distanza. In questo senso, la correzione è un elemento centrale della materia programmabile: senza feedback e correzione, la forma non è robusta.

La correzione degli errori deve però evitare overcorrection. Correggere troppo può creare oscillazioni. Se un nodo viene spinto continuamente verso il target con forza eccessiva, può superarlo e oscillare. Se il controller cambia ruoli troppo rapidamente, la struttura diventa instabile. Se aumenta coesione e poi separazione alternativamente, il cluster può vibrare. Per questo servono smorzamento, soglie, hysteresis e limiti di correzione.

Una regola generale:

correzione graduale, limitata e verificata.

Meglio applicare piccole correzioni misurabili che interventi drastici, salvo casi critici. Questo principio vale soprattutto per sistemi fisici.

La correzione degli errori può essere vista come parte di un controllo a retroazione. Il sistema misura errore, calcola correzione, applica, misura di nuovo. In forma semplice:

errore → correzione → nuovo stato → nuovo errore

Se il nuovo errore diminuisce, la correzione è utile. Se aumenta, bisogna ridurre o cambiare strategia. Questa è la base di qualunque controllo robusto. Nel MicroBot, questo controllo può essere implementato inizialmente nel simulatore, poi nella dashboard/controller, poi in parte nei nodi fisici.

In conclusione, la correzione degli errori è il processo specifico con cui MicroBot rileva deviazioni rispetto a pattern, ruoli, tempi, connessioni, configurazioni o safety e applica interventi mirati per ridurre tali deviazioni. Essa richiede metriche, classificazione, soglie, severità, strategie, verifica e logging. Gli errori possono essere geometrici, topologici, temporali, comunicativi, di ruolo, di configurazione o di safety; ognuno richiede una correzione diversa. La correzione deve essere proporzionata, tracciabile, compatibile con capabilities, visibile nella dashboard e subordinata alla sicurezza. Se funziona, preserva il pattern originale; se fallisce, apre la strada a fallback o adattamento dinamico. Con questo punto si completa la logica fondamentale dei pattern e comportamenti: MicroBot non solo genera forme, ma le osserva, le valuta e tenta di mantenerle coerenti. La sezione successiva potrà quindi passare al livello fisico e hardware, analizzando come queste logiche astratte vengano progressivamente collegate a componenti reali, microcontrollori, LED, alimentazione, magneti, sensori, attuatori e prototipi sperimentali.

SEZIONE 11 — Safety Layer

La Sezione 11 introduce il Safety Layer del sistema MicroBot, cioè il livello di protezione progettato per impedire che i comandi, gli attuatori, le bobine, l'alimentazione, la batteria, il firmware o il comportamento dello swarm possano produrre condizioni instabili, pericolose o non controllate. Dopo aver definito i pattern, i comportamenti, l'assegnazione dei ruoli, l'adattamento dinamico e la correzione degli errori, diventa necessario introdurre un principio fondamentale:

nessuna azione del sistema deve essere eseguita se viola i limiti fisici, energetici, termici o operativi dei MicroBot.

Il Safety Layer non è un'aggiunta secondaria, ma una componente centrale dell'architettura. In un sistema distribuito, infatti, non basta che il controller sia in grado di inviare comandi corretti. Ogni nodo deve anche essere in grado di proteggere sé stesso. Questo punto è essenziale: se il controller invia per errore un comando troppo lungo, un duty cycle troppo alto, una richiesta di attivazione incompatibile con lo stato locale o un comando ricevuto fuori tempo massimo, il MicroBot non deve eseguire ciecamente. Deve validare, filtrare, limitare, rifiutare o interrompere l'azione. Il Safety Layer è quindi la barriera tra la logica del sistema e il rischio fisico.

Nel progetto MicroBot, la sicurezza riguarda diversi livelli. Il primo è il controllo della corrente, perché ogni componente elettronico ha limiti di assorbimento e ogni attuatore deve essere pilotato entro soglie sicure. Il secondo è la protezione delle bobine, soprattutto se il sistema usa elettromagneti, coil, PWM o attuatori magnetici. Il terzo è il monitoraggio della temperatura, perché corrente, bobine, driver e microcontrollori possono scaldarsi. Il quarto è la prevenzione del surriscaldamento, cioè l'insieme delle regole che impediscono al sistema di continuare ad attivare componenti quando il carico termico è troppo alto. Il quinto è la gestione della batteria, necessaria quando i nodi diventano autonomi. Il sesto è la protezione energetica, cioè la capacità di ridurre consumi, evitare picchi e passare a modalità conservative. Il settimo è il watchdog timer, che protegge il firmware da blocchi o loop non controllati. L'ottavo è il ripristino automatico, cioè la capacità del nodo o del controller di tornare a uno stato sicuro dopo errore, timeout o reset.

Il principio generale del Safety Layer può essere espresso così:

nessun comando deve avere autorità superiore alla sicurezza locale del nodo.

Questo significa che il controller può proporre un'azione, ma il nodo deve sempre verificare se quell'azione è permessa. Un comando `magnetic_pulse`, per esempio, può essere valido dal punto di vista del protocollo, ma deve essere rifiutato se la bobina è in cooldown, se la temperatura è troppo alta, se il duty cycle supera la soglia, se la batteria è troppo bassa, se il nodo è in safety mode o se il comando è arrivato oltre `expires_at`. In questo senso, la sicurezza non è solo centralizzata. È distribuita. Ogni MicroBot possiede un piccolo livello decisionale locale che impedisce esecuzioni pericolose.

Il Safety Layer deve anche essere coerente con la simulazione. Nelle prime versioni, molte variabili possono essere simulate: batteria stimata, temperatura stimata, carico termico virtuale, stato della bobina, energia residua e consumo teorico. Anche se questi valori non provengono ancora da sensori reali, sono utili per progettare la logica del sistema. Un nodo simulato può entrare in `thermal_warning`, `low_energy` o `cooldown_active`. Questo permette di testare il comportamento del controller prima di collegare hardware reale. Quando poi si aggiungono sensori fisici, la stessa logica può essere alimentata da dati misurati invece che stimati.

Il Safety Layer deve quindi essere pensato come un contratto tra firmware, controller, simulatore e dashboard. Il firmware protegge il nodo localmente. Il controller evita di inviare comandi incompatibili. Il simulatore permette di testare scenari di rischio senza danni reali. La dashboard mostra lo stato di sicurezza in modo chiaro. I log registrano ogni rifiuto, ogni warning, ogni blocco e ogni ripristino. Questo rende la sicurezza osservabile, non nascosta.

11.1 Controllo della corrente

Il controllo della corrente rappresenta il primo elemento del Safety Layer, perché ogni MicroBot fisico dipende da circuiti elettronici reali, e ogni circuito ha limiti di assorbimento che non possono essere ignorati. Nel modello astratto, un comando può sembrare soltanto un messaggio.

Nell'hardware, invece, quel comando può trasformarsi in corrente che attraversa LED, resistenze, driver, bobine, MOSFET, sensori, regolatori o piste conduttive. Se la corrente supera i limiti previsti, il sistema può scaldarsi, diventare instabile, resettarsi, danneggiare componenti o produrre comportamenti imprevedibili. Per questo motivo, la corrente deve essere controllata sia a livello progettuale sia a livello firmware.

Nel progetto MicroBot, il controllo della corrente deve essere affrontato fin dai prototipi più semplici. Anche un LED richiede una resistenza adeguata. Anche un ESP32 alimentato via USB ha limiti di corrente. Anche una breadboard ha limiti pratici. Quando poi si introducono bobine, magneti attivi o uscite PWM, il problema diventa molto più importante. Una bobina, a differenza di un LED, può assorbire corrente significativa e generare calore rapidamente. Se viene attivata troppo a lungo o con duty cycle troppo alto, può surriscaldarsi. Se il driver non è adatto, può danneggiarsi. Se l'alimentazione non è stabile, il microcontrollore può resettarsi.

Il controllo della corrente deve quindi partire da un principio semplice: ogni uscita fisica deve avere un limite massimo definito. Questo limite può essere hardware, firmware o entrambi. Il limite hardware deriva dalla scelta dei componenti: resistenze, driver, transistor, MOSFET, regolatori, alimentazione e protezioni. Il limite firmware deriva dalle regole che impediscono comandi troppo intensi o troppo lunghi. I due livelli devono lavorare insieme. Il firmware non deve essere l'unica protezione, perché un errore software può sempre accadere. L'hardware non deve essere ignorato, perché un circuito mal dimensionato resta pericoloso anche con codice corretto.

Una regola di base è:

corrente richiesta dal carico $\leq$ corrente massima sicura del circuito

Questa relazione deve essere verificata per ogni componente. Se un LED richiede pochi milliampere, il problema è relativamente semplice. Se una bobina richiede centinaia di milliampere o più, bisogna dimensionare correttamente driver, alimentazione e dissipazione. Il sistema MicroBot deve quindi distinguere tra uscite leggere e uscite ad alto assorbimento. Le prime possono essere gestite direttamente o quasi direttamente dal microcontrollore, nei limiti consentiti. Le seconde devono essere pilotate tramite circuiti dedicati.

Nel caso dei LED, il controllo della corrente avviene principalmente tramite resistenza in serie e scelta del GPIO. Un LED collegato senza resistenza può assorbire troppa corrente e danneggiare il pin o il LED stesso. Una resistenza adeguata limita la corrente. Questo è il primo esempio concreto di safety hardware. Anche se il comando software dice "accendi LED", la resistenza impedisce che il LED assorba corrente eccessiva. Nel contesto MicroBot, questo è importante perché i primi prototipi LED devono essere già costruiti con logica sicura, non solo funzionale.

Nel caso delle bobine, il controllo della corrente è molto più delicato. Una bobina può essere pilotata tramite PWM, ma il duty cycle deve essere limitato. Un duty cycle alto aumenta il tempo medio di alimentazione e quindi la corrente media e il calore. Se la bobina resta attiva troppo a lungo, anche con duty cycle moderato, può scaldarsi. Per questo motivo, ogni comando legato a una bobina deve includere durata massima, duty cycle massimo, cooldown e stato termico. Un comando generico "coil_on" senza durata è pericoloso. È molto più sicuro usare comandi del tipo "magnetic_pulse" con durata limitata.

Una struttura corretta di comando può essere:

command = magnetic_pulse
duty_cycle = valore limitato
duration_ms = durata limitata
cooldown_required = true

Il nodo deve verificare che duty_cycle e duration_ms siano entro i limiti. Se non lo sono, deve rifiutare il comando. Inoltre, al termine della durata, il firmware deve spegnere automaticamente l'uscita. Non deve aspettare un secondo comando "off" dal controller, perché quel comando potrebbe arrivare in ritardo o perdersi. Questo punto è fondamentale: tutte le attuazioni ad alto consumo devono essere auto-terminate localmente dal nodo.

Il controllo della corrente deve quindi distinguere tra comandi persistenti e comandi impulsivi. I comandi persistenti mantengono un'uscita attiva finché non arriva un altro comando. Sono rischiosi per attuatori ad alto consumo. I comandi impulsivi hanno una durata incorporata e terminano automaticamente. Per bobine e attuatori magnetici, la strategia corretta è usare comandi impulsivi. Per LED e indicatori a basso consumo, si possono usare anche stati persistenti, ma comunque con logica non bloccante e possibilità di override safety.

Una regola progettuale può essere:

per carichi ad alto assorbimento, evitare stati attivi indefiniti; usare solo impulsi temporizzati e limitati.

Questa regola protegge il nodo anche in caso di perdita di comunicazione. Se il controller sparisce, la bobina non resta accesa per sempre. Il nodo la spegne da solo quando scade duration_ms o quando interviene il safety timeout.

Il controllo della corrente richiede anche una stima del carico totale. Un singolo MicroBot può avere più uscite: LED, sensori, bobina, eventuale driver, modulo Wi-Fi. Anche se ogni componente è entro il proprio limite, il totale può essere eccessivo per l'alimentazione. Per esempio, attivare più bobine contemporaneamente in uno swarm può generare picchi energetici. Il controller deve quindi ragionare non solo sul singolo nodo, ma anche sul gruppo. Se dieci nodi attivano coil nello stesso istante, il consumo totale può essere molto più alto rispetto a una sequenza distribuita in slot.

Questo collega il controllo della corrente alla sincronizzazione e agli slot temporali. Invece di attivare tutti i nodi contemporaneamente, il sistema può distribuire le attuazioni in finestre diverse. Questa strategia riduce picchi di corrente e rende il comportamento più controllabile. Il Safety Layer quindi non lavora solo nel firmware del singolo nodo, ma anche nel piano temporale globale del controller. Gli slot temporali non servono soltanto alla comunicazione; servono anche a distribuire il carico energetico.

Una politica può essere:

non attivare più di K nodi ad alto assorbimento nello stesso slot;

alternare gruppi di attuazione;

inserire cooldown tra impulsi;

ridurre duty cycle se molti nodi sono attivi;

abortire la fase se il carico stimato supera soglia.

Questa logica è molto importante per versioni future con più MicroBot fisici. Anche se nei primi test si lavora con uno o due moduli, l'architettura deve prevedere il problema dello swarm intero.

Il controllo della corrente può essere implementato attraverso una tabella di limiti. Ogni nodo può dichiarare limiti locali, come `max_pwm_duty`, `max_pulse_duration_ms`, `min_cooldown_ms`, `max_pulses_per_minute`, `max_active_outputs` e `current_profile`. Il controller usa questi dati per pianificare. Il nodo li usa per validare. Se il controller invia un comando fuori limite, il nodo rifiuta. Se il controller non conosce i limiti aggiornati, può richiederli tramite `request_status` o `get_node_profile`.

Una tabella di limiti può contenere:

`max_safe_duty_cycle`;

`max_pulse_duration_ms`;

`min_cooldown_ms`;

`max_repetition_count`;

`estimated_current_ma`;

`output_type`;

`thermal_class`.

Questi valori possono essere inizialmente conservativi e manuali. In futuro possono essere calibrati sperimentalmente. L'importante è che esistano. Un attuatore senza limiti documentati non dovrebbe essere usato in pattern automatici.

Il controllo della corrente deve essere collegato alla modalità `low_energy`. Se la batteria è bassa o se l'alimentazione è debole, il nodo deve ridurre o rifiutare comandi ad alto consumo. Anche se il comando sarebbe valido in condizioni normali, può diventare non sicuro quando l'energia disponibile è bassa. Un nodo `low_energy` può continuare a inviare heartbeat, ricevere comandi, mostrare LED ridotti o restare in idle, ma dovrebbe evitare coil, PWM intenso o attuazioni ripetute.

Una regola può essere:

se `energy_state = low_energy`, bloccare comandi ad alto assorbimento e consentire solo funzioni essenziali.

Questo permette al sistema di degradare in modo sicuro. Il controller può vedere che il nodo è `low_energy` e assegnargli ruolo passivo invece di `active_connector`. La gestione della corrente diventa quindi parte dell'assegnazione dei ruoli.

Il controllo della corrente deve generare errori strutturati. Se un comando viene rifiutato perché richiede troppa corrente, il nodo deve comunicare una ragione precisa. Non basta dire `failed`. Deve dire `current_limit_exceeded`, `duty_cycle_too_high`, `duration_too_long`, `cooldown_active` o `low_energy`. Questo permette al controller di correggere il piano. Se il problema è `duty_cycle`, può ridurlo. Se il problema è `cooldown`, può aspettare. Se il problema è `low_energy`, può cambiare ruolo. La precisione dell'errore permette adattamento intelligente.

Un errore può essere:

```
result = rejected
reason = duty_cycle_too_high
max_allowed = 0.25
requested = 0.40
```

Oppure:

```
result = rejected
reason = cooldown_active
cooldown_remaining_ms = 3500
```

Questi dati sono molto utili nella dashboard e nei log. Rendono il Safety Layer osservabile.

Il controllo della corrente deve anche prevedere uno stato di uscita sicuro. Ogni output ad alto consumo deve avere default OFF. All'avvio, al reset, alla perdita del controller, all'emergency_stop e alla safety_mode, l'uscita deve andare a OFF. Il firmware deve impostare i pin in modo sicuro il prima possibile durante il boot. Questo è un principio fondamentale. Non si deve aspettare che il controller dica di spegnere. Il nodo deve sapere che, in assenza di controllo valido, lo stato sicuro è disattivazione.

Una regola può essere:

in condizioni incerte, spegnere uscite ad alto consumo.

Le condizioni incerte includono boot, reset, sync_lost, controller_lost, invalid_command, safety_triggered, watchdog_reset e low_energy severo. Questa regola riduce il rischio di comportamenti incontrollati.

Il controllo della corrente deve essere integrato con il watchdog timer. Se il firmware si blocca mentre un'uscita è attiva, il watchdog deve resettare il nodo. Ma prima ancora, la logica firmware dovrebbe evitare blocchi. Per attuatori critici, è utile avere una variabile output_expire_time controllata continuamente nel loop. Se il tempo corrente supera output_expire_time, l'uscita viene spenta. Anche se il controller non risponde, il nodo mantiene il limite locale. Il watchdog è una protezione ulteriore contro blocchi più gravi.

Una logica minima è:

quando attivi una bobina, salva off_time = now + duration_ms;

a ogni ciclo, se now >= off_time, spegni bobina;

se entra emergency_stop, spegni subito;

se entra safety, spegni subito;

se firmware si blocca, watchdog reset.

Questa catena protegge su più livelli. Non si affida a una sola barriera.

Il controllo della corrente deve essere testato in modo incrementale. Prima si testano LED con resistenza. Poi si testa PWM su carichi sicuri. Poi si testa un driver senza carico critico. Poi si testa una bobina con durata breve e duty basso. Poi si osserva temperatura, alimentazione e comportamento. Non si deve partire da duty alto o durata lunga. Ogni test deve essere documentato con valore di duty, durata, alimentazione, comportamento e temperatura stimata o misurata.

Una sequenza di test prudente può essere:

test output digitale spento/acceso su LED;

test PWM su LED;

test driver con carico leggero;

test bobina con impulso brevissimo;

test bobina con duty basso;

test cooldown;

test rifiuto comando fuori limite;

test emergency_stop durante attuazione;

test perdita controller durante attuazione.

Questi test verificano che il Safety Layer funzioni davvero, non solo sulla carta.

Il controllo della corrente deve essere rappresentato nella dashboard. La dashboard può mostrare output_state, duty_cycle corrente, cooldown, current_limit_status, energy_state, last_rejection_reason e active_pulse_remaining_ms. Anche se la corrente reale non viene ancora misurata, si può mostrare una stima o stato logico. In futuro, con sensori reali, si potrà mostrare corrente misurata. Questa visualizzazione è utile perché permette di capire perché un nodo rifiuta comandi o entra in modalità conservativa.

Una scheda nodo può mostrare:

output attivi;

duty cycle;

durata residua impulso;

cooldown residuo;

limite massimo;

stato energetico;

ultimo errore di corrente.

Questi dati sono fondamentali per debug hardware.

Il controllo della corrente deve essere documentato nei log. Ogni attivazione ad alto consumo deve essere registrata: comando, duty, durata, nodo, timestamp, esito, eventuale rifiuto, cooldown e stato safety. Questo permette di ricostruire cosa è successo in caso di problema. Se una bobina si scalda troppo, il log deve mostrare quante volte è stata attivata, con quali parametri e con quali intervalli. Senza log, sarebbe difficile capire la causa.

Un log di attuazione può contenere:

node_id;
command_id;
output_id;
duty_cycle;
duration_ms;
start_time;
end_time;
result;
cooldown_started;
safety_flags.

Questa struttura è molto importante nelle versioni hardware.

Il controllo della corrente deve essere collegato alla prevenzione del surriscaldamento. Corrente e temperatura sono legate. Un componente che assorbe corrente genera calore. Anche senza sensore termico, il sistema può stimare thermal_load in base agli impulsi recenti. Per esempio, ogni magnetic_pulse aumenta thermal_load; il tempo di riposo lo riduce. Se thermal_load supera una soglia, il nodo entra in thermal_warning o thermal_lock. Questo è un modo per implementare protezione termica anche prima di avere un sensore reale.

Una stima semplice può essere:

thermal_load aumenta con $\text{duty_cycle} \times \text{duration_ms}$;
thermal_load diminuisce nel tempo durante cooldown;
se $\text{thermal_load} > \text{soglia_warning}$, ridurre comandi;
se $\text{thermal_load} > \text{soglia_lock}$, bloccare attuatori.

Questa logica sarà approfondita nel monitoraggio della temperatura, ma nasce già dal controllo della corrente. La corrente non è solo un valore elettrico; è una causa di calore.

Il controllo della corrente deve anche influenzare la pianificazione dei pattern. Se un pattern richiede troppe attuazioni, il controller deve ridurle o distribuirle. Una struttura complessa con molti connector attivi può essere troppo pesante per l'alimentazione. Il controller può scegliere una versione visuale, una versione simulata, una versione a gruppi o una versione più lenta. Questo collega Safety Layer e pattern. Il pattern non deve ignorare l'energia.

Una policy di pattern può includere:

max_active_actuators_per_phase;
max_total_estimated_current;
required_cooldown_between_phases;

`fallback_to_visual_if_energy_low;`

`abort_if_current_limit_exceeded.`

Questa policy rende i pattern hardware-safe. Non tutti i pattern simulati sono automaticamente eseguibili fisicamente.

Il controllo della corrente deve distinguere tra corrente stimata e corrente misurata. Nelle prime versioni, il sistema può stimare la corrente in base al tipo di carico e al comando. Per esempio, se una bobina è nota per assorbire una certa corrente a un certo duty, il sistema stima. In versioni più avanzate, si possono aggiungere sensori di corrente o misure indirette. La documentazione deve indicare la fonte del dato: `estimated` o `measured`. Questo evita confusione.

Un campo può essere:

`current_source = estimated`

oppure:

`current_source = measured`

Se il dato è stimato, la dashboard deve trattarlo come approssimazione. Se è misurato, può essere usato con maggiore fiducia. In ogni caso, per sicurezza, i limiti dovrebbero essere conservativi.

Il controllo della corrente deve essere considerato anche nella scelta dei componenti. Un GPIO del microcontrollore non deve pilotare direttamente carichi pesanti. Per bobine o magneti attivi servono driver appropriati. L'alimentazione deve essere dimensionata. Le masse devono essere gestite correttamente. I componenti devono avere margini. Anche se questa sezione non è una guida di elettronica dettagliata, deve fissare il principio: il Safety Layer software non sostituisce il corretto dimensionamento hardware.

Una formulazione corretta è:

il firmware limita, ma l'hardware deve essere dimensionato per sopportare il caso peggiore previsto.

Se l'hardware non è adatto, il software non basta. Questo principio è importantissimo quando si passerà da LED a bobine.

Il controllo della corrente deve supportare `emergency_stop`. In caso di `emergency_stop`, tutte le uscite ad alto consumo devono essere disattivate immediatamente. Il nodo non deve aspettare slot, phase o ack. Deve spegnere prima, poi eventualmente rispondere. Questo ordine è fondamentale. La corrente deve essere interrotta il prima possibile.

La sequenza corretta è:

ricevi `emergency_stop`;

disattiva output critici;

cancella comandi pendenti;

entra in `safety` o `safe_idle`;

invia ack se possibile;

registra evento.

Non deve essere:

ricevi emergency_stop;

invia ack;

poi spegni.

La protezione fisica viene prima della comunicazione.

Il controllo della corrente deve infine essere visto come una condizione di credibilità del progetto. Un sistema che parla di bobine, magneti e microbot deve dimostrare di sapere che ogni attuazione ha costi e limiti. Inserire controllo della corrente, duty cycle, durata, cooldown, rifiuto dei comandi fuori range e logging non è un dettaglio: è ciò che rende il progetto ingegneristicamente serio. Anche se alcune parti sono ancora simulate, il fatto che siano già previste nel Safety Layer mostra che l'architettura è pensata per diventare fisica senza essere irresponsabile.

In conclusione, il controllo della corrente è il primo pilastro del Safety Layer MicroBot. Esso garantisce che LED, uscite PWM, driver, bobine e futuri attuatori vengano usati entro limiti sicuri. Questo richiede limiti hardware, validazione firmware, durata massima, duty cycle massimo, cooldown, spegnimento automatico, rifiuto dei comandi fuori soglia, log strutturati, dashboard diagnostica e pianificazione temporale del carico. Il controller può proporre un'azione, ma il nodo deve decidere se la corrente richiesta è compatibile con il proprio stato. Se non lo è, deve rifiutare. Questo principio prepara direttamente il punto successivo, dedicato alla protezione delle bobine, dove il controllo della corrente diventa ancora più importante perché gli attuatori magnetici rappresentano uno dei componenti più delicati e caratterizzanti del sistema MicroBot.

11.2 Protezione delle bobine

La protezione delle bobine rappresenta uno dei punti più delicati del Safety Layer MicroBot, perché le bobine sono tra i componenti più importanti per trasformare il sistema da semplice rete di nodi visuali a piattaforma fisica capace di interazione magnetica, aggancio controllato, attrazione, rilascio e comportamento strutturale. Dopo aver definito il controllo della corrente, è necessario approfondire il caso specifico delle bobine, perché esse introducono rischi e vincoli più severi rispetto a LED, segnali logici o telemetria. Una bobina non è un indicatore passivo: è un carico elettrico che può assorbire corrente significativa, generare campo magnetico, produrre calore, indurre tensioni di ritorno, sollecitare driver e alimentazione, e influenzare direttamente il comportamento fisico dello swarm.

Nel progetto MicroBot, le bobine devono essere considerate componenti attuativi critici. La loro funzione potenziale è permettere ai MicroBot di generare interazione magnetica controllata, avvicinarsi, agganciarsi, stabilizzare una connessione, modificare una forza locale o partecipare a una struttura riconfigurabile. Tuttavia, proprio perché una bobina trasforma energia elettrica in campo magnetico e calore, non può essere comandata con la stessa leggerezza con cui si accende un LED. Ogni attivazione deve essere limitata, temporizzata, verificata, registrata e subordinata allo stato di sicurezza del nodo.

Il principio fondamentale è che una bobina non deve mai restare attiva indefinitamente per effetto di un comando persistente. Un comando come coil_on senza durata massima è pericoloso, perché se il controller perde comunicazione, se il pacchetto di spegnimento non arriva, se il firmware si blocca o se la rete cade, la bobina potrebbe restare alimentata troppo a lungo. Per questo motivo, il comando

corretto non deve essere “accendi la bobina”, ma “esegui un impulso magnetico limitato”, cioè un `magnetic_pulse` con `duty cycle`, durata, direzione logica, cooldown e limiti locali.

Una struttura sicura del comando può essere espressa così:

```
command = magnetic_pulse
coil_id = main
duty_cycle = valore compreso entro soglia sicura
duration_ms = durata limitata
direction = attract / release / hold se supportato
execute_at_ms = opzionale, se comando sincronizzato
expires_at_ms = obbligatorio per evitare esecuzione tardiva
```

Questa forma è molto più sicura perché ogni comando contiene già la propria condizione di spegnimento. Il nodo non aspetta un comando successivo per disattivare la bobina; imposta internamente un timer e spegne l'uscita quando la durata termina. Se arriva `emergency_stop`, la spegne prima. Se entra in `safety`, la spegne prima. Se il comando è scaduto, non lo esegue. Se il `duty cycle` è troppo alto, lo rifiuta. Se il cooldown non è terminato, lo rifiuta. Questa logica rende la bobina controllabile e protegge il sistema da errori di comunicazione.

La protezione delle bobine deve essere costruita su più livelli. Il primo livello è hardware: la bobina deve essere pilotata attraverso un circuito adeguato, non direttamente da un pin del microcontrollore se richiede corrente superiore ai limiti del GPIO. Serve un driver, un transistor o un MOSFET adeguato, dimensionato rispetto alla corrente richiesta, alla tensione, alla dissipazione e alla frequenza PWM. Inoltre, una bobina è un carico induttivo, quindi può generare tensioni inverse o transitori quando viene disattivata. Per questo motivo, il circuito deve prevedere protezioni adeguate, come diodo di flyback o soluzioni equivalenti in base al tipo di pilotaggio. Il documento non deve trasformarsi in uno schema elettrico completo, ma deve fissare chiaramente il principio: una bobina non è un LED e non deve essere pilotata come se lo fosse.

Il secondo livello è firmware. Il firmware deve impedire attivazioni fuori limite. Anche se il controller invia un comando formalmente corretto, il nodo deve validare i parametri. Deve controllare che `coil_id` esista, che il `duty cycle` sia inferiore al massimo consentito, che `duration_ms` sia sotto la durata massima, che il cooldown sia terminato, che il nodo non sia in `thermal_lock`, che la batteria o l'alimentazione siano sufficienti, che la modalità corrente permetta attuazione e che il comando sia temporalmente valido. Se uno di questi controlli fallisce, il comando viene rifiutato e il nodo invia un errore strutturato.

Una validazione minima per bobina può essere:

```
coil_id riconosciuto;
capability coil_pwm presente;
mode compatibile con attuazione;
safety_state = ok;
thermal_state non in lock;
energy_state sufficiente;
duty_cycle ≤ max_safe_duty_cycle;
```

`duration_ms ≤ max_pulse_duration_ms;`

`cooldown_remaining_ms = 0;`

`timestamp/expires_at` validi;

`message_id` non duplicato.

Questa lista è lunga perché la bobina è un componente critico. Non basta che il comando arrivi al nodo. Deve essere compatibile con tutto lo stato fisico e logico del MicroBot.

Il terzo livello è controller. Il controller deve evitare di inviare comandi che il nodo rifiuterà. Questo significa che deve conoscere capabilities, limiti, cooldown, stato energetico e stato termico dei nodi. Se MB-004 è in cooldown, il controller non dovrebbe assegnargli un nuovo magnetic_pulse. Se MB-002 non ha coil_pwm, non deve essere scelto come active_connector. Se un pattern richiede troppe attivazioni contemporanee, il controller deve distribuirle in slot o ridurre la fase. Il nodo è l'ultima barriera; il controller deve essere la prima linea di pianificazione sicura.

Il quarto livello è dashboard e log. La protezione delle bobine deve essere osservabile. L'utente deve poter vedere se una bobina è attiva, quale duty cycle è stato richiesto, quanto dura l'impulso, quanto cooldown resta, se c'è stato un rifiuto, se il nodo è entrato in thermal_warning o thermal_lock. Ogni attivazione deve essere registrata. Questo è fondamentale perché, in caso di comportamento anomalo, il log deve permettere di ricostruire la sequenza: quando la bobina è stata attivata, per quanto tempo, con quale duty cycle, da quale comando, con quale risultato e con quale temperatura stimata o misurata.

La protezione delle bobine deve partire dalla definizione di limiti conservativi. Nelle prime prove non si devono usare valori aggressivi. Il duty cycle deve essere basso, la durata breve, il cooldown ampio. L'obiettivo iniziale non è ottenere la massima forza magnetica, ma verificare che la catena hardware-firmware-controller funzioni senza surriscaldamento e senza instabilità. Solo dopo test documentati si possono aumentare gradualmente i parametri. Questo approccio è essenziale: prima si valida il controllo, poi si cerca la performance.

Una politica iniziale prudente può essere:

duty cycle massimo basso;

durata massima breve;

cooldown obbligatorio dopo ogni impulso;

numero massimo di impulsi per finestra temporale;

blocco automatico in caso di comandi ripetuti troppo ravvicinati;

nessun comando persistente di bobina attiva;

emergency_stop sempre prioritario.

Questa politica può sembrare conservativa, ma è esattamente ciò che serve nelle prime fasi hardware. Una bobina che funziona ma si scalda troppo non è una soluzione affidabile. Una bobina controllata entro limiti chiari, anche se debole, è una base sperimentale molto migliore.

La protezione delle bobine richiede la definizione di uno stato specifico della bobina. Il nodo non deve sapere solo se la bobina è accesa o spenta. Deve sapere se è idle, active, cooldown, disabled, fault, thermal_lock o unavailable. Questo stato permette al firmware e al controller di capire cosa sta succedendo. Una bobina in idle è pronta. Una bobina in active sta eseguendo un impulso. Una bobina in cooldown ha appena eseguito un impulso e non può essere riattivata. Una bobina in disabled è disattivata per scelta o configurazione. Una bobina in fault ha rilevato errore. Una bobina in thermal_lock è bloccata per rischio termico. Una bobina unavailable non è presente o non è supportata dal nodo.

Una macchina a stati semplice può essere:

idle → active → cooldown → idle

Con transizioni di sicurezza:

active → safety_off

cooldown → thermal_lock se carico stimato troppo alto

idle → disabled se capability disabilitata

qualsiasi stato → safety_off in caso di emergency_stop

Questa macchina a stati è molto importante perché impedisce transizioni casuali. La bobina non può passare da active ad active senza cooldown, salvo che la policy lo permetta in modo controllato. Non può attivarsi se è disabled o thermal_lock. Non può restare active oltre duration_ms. Ogni transizione deve essere motivata da comando, timer o safety event.

Il comando magnetic_pulse deve quindi interagire con questa macchina a stati. Se la bobina è idle e il comando è valido, passa ad active. Il firmware salva start_time, end_time, duty_cycle e command_id. Quando il tempo termina, spegne l'uscita e passa a cooldown. Quando il cooldown termina, torna idle. Se durante active arriva emergency_stop, spegne e passa a safety_off o safety. Se durante cooldown arriva un nuovo comando, lo rifiuta con cooldown_active. Se thermal_load supera soglia, passa a thermal_lock.

Questo comportamento può essere descritto così:

ricezione comando valido;

attivazione bobina;

timer locale di durata;

spegnimento automatico;

avvio cooldown;

ritorno a idle solo dopo cooldown;

rifiuto di nuovi impulsi se cooldown non terminato.

La protezione delle bobine deve quindi dipendere dal tempo locale del nodo, non soltanto dal controller. Il controller può pianificare, ma il nodo deve essere responsabile dello spegnimento. Questo è uno dei principi più importanti dell'intero Safety Layer.

La gestione del duty cycle è un altro elemento centrale. Il duty cycle determina la percentuale del periodo PWM durante cui il segnale resta attivo. Un duty cycle alto produce maggiore corrente media e quindi più calore. Il sistema deve avere un `max_safe_duty_cycle` definito per ciascuna bobina o classe hardware. Questo valore non deve essere scelto a caso. Nelle prime fasi, deve essere conservativo. In seguito, può essere raffinato con misure sperimentali. Il firmware deve rifiutare qualunque valore superiore.

Se il controller richiede `duty_cycle = 0.60` ma il limite locale è 0.25, il nodo deve rifiutare. Non deve ridurre automaticamente senza comunicarlo, a meno che la policy preveda `accepted_modified`. È meglio essere espliciti. Se il nodo modifica il comando, deve dirlo. Per esempio, può rispondere:

```
result = accepted_modified
requested_duty = 0.60
applied_duty = 0.25
reason = clamped_to_safe_limit
```

Oppure può rifiutare:

```
result = rejected
reason = duty_cycle_too_high
max_allowed = 0.25
requested = 0.60
```

La scelta tra rifiuto e clipping dipende dalla policy. Per comandi fisici critici, il rifiuto è spesso più chiaro. Per demo conservative, `accepted_modified` può essere utile. In ogni caso, la modifica deve essere registrata.

La durata dell'impulso è altrettanto importante. Anche un duty cycle moderato può essere rischioso se la bobina resta attiva troppo a lungo. Ogni bobina deve avere `max_pulse_duration_ms`. Il nodo deve confrontare `duration_ms` con questo limite. Se `duration_ms` è troppo alto, il comando viene rifiutato o modificato secondo policy. Inoltre, il firmware deve implementare uno spegnimento automatico basato su timer locale. Questo timer deve essere indipendente dalla ricezione di ulteriori pacchetti.

Una regola fondamentale è:

`duration_ms` è un limite massimo operativo, non un suggerimento.

Quando `duration_ms` termina, la bobina deve spegnersi. Anche se il pattern non è finito. Anche se il controller non ha ancora inviato `ack`. Anche se la dashboard è bloccata. La protezione locale prevale.

Il cooldown serve a impedire impulsi troppo ravvicinati. Una bobina può non surriscaldarsi con un singolo impulso breve, ma può accumulare calore se gli impulsi vengono ripetuti rapidamente. Il cooldown è un intervallo minimo di riposo tra un impulso e il successivo. Durante cooldown, il nodo rifiuta nuovi comandi per quella bobina. Il cooldown può essere fisso o dipendere da duty e durata. In una prima versione, un cooldown fisso è più semplice. In una versione più avanzata, il cooldown può essere proporzionale al carico termico stimato.

Una formula concettuale può essere:

$$\text{cooldown_ms} = \text{base_cooldown} + k \cdot \text{duty_cycle} \cdot \text{duration_ms}$$

Questo significa che impulsi più intensi o lunghi richiedono più tempo di riposo. Anche se non viene implementata subito, questa logica è coerente con una protezione termica realistica.

La protezione delle bobine deve includere un modello di `thermal_load`. Anche senza sensore reale, il nodo può stimare il carico termico accumulato. Ogni impulso aumenta `thermal_load` in base a `duty_cycle` e `duration_ms`. Nel tempo, `thermal_load` diminuisce gradualmente. Se `thermal_load` supera una soglia di `warning`, il nodo può ridurre o rifiutare impulsi intensi. Se supera una soglia di `lock`, entra in `thermal_lock` e blocca la bobina finché non si raffredda. Questo modello è una protezione software importante nelle prime versioni.

Una logica semplice può essere:

`thermal_load += duty_cycle × duration_ms × coefficiente_bobina`

`thermal_load -= cooling_rate × delta_time`

se `thermal_load > warning_threshold` → `thermal_warning`

se `thermal_load > lock_threshold` → `thermal_lock`

Il valore non è una temperatura reale, ma un indicatore di rischio termico. Deve essere dichiarato come `estimated_thermal_load`. Quando verrà aggiunto un sensore di temperatura, il sistema potrà combinare stima e misura. Fino ad allora, è meglio avere una stima conservativa che nessun controllo.

La protezione delle bobine deve distinguere tra `thermal_warning` e `thermal_lock`. `thermal_warning` indica che il carico termico sta salendo e che il sistema deve diventare prudente. In questa fase, il nodo può accettare solo impulsi più deboli, aumentare cooldown o segnalare `warning`. `thermal_lock` indica invece che la bobina è bloccata per sicurezza e non deve essere attivata. La differenza è importante: `warning` è uno stato degradato; `lock` è uno stato di blocco.

Una politica può essere:

`thermal_ok`: comandi entro limiti normali;

`thermal_warning`: riduzione duty massimo e cooldown più lungo;

`thermal_lock`: rifiuto di tutti i comandi bobina;

`thermal_recovery`: attesa fino a ritorno sotto soglia;

`thermal_ok` ripristinato solo dopo margine di sicurezza.

Il ritorno da `thermal_lock` non deve avvenire appena il valore scende di pochissimo sotto soglia. Serve hysteresis. Se la soglia di `lock` è 100, il sistema può rientrare solo quando `thermal_load` scende sotto 70. Questo impedisce oscillazioni tra `lock` e `unlock`.

La protezione delle bobine deve considerare anche la direzione logica del comando magnetico. In alcune configurazioni future, il sistema potrebbe distinguere `attract`, `release`, `hold` o `polarity` se l'hardware lo permette. Nelle prime versioni, però, una bobina semplice potrebbe non supportare vera inversione di polarità. La documentazione deve essere precisa: `direction` può essere un campo logico o simulativo se l'hardware non supporta polarità reale. Il firmware deve dichiarare quali modalità sono supportate. Se riceve `direction = release` ma non supporta `release` attivo, deve rifiutare o interpretare come `coil_off` solo se previsto.

Una mappa di capability può essere:

coil_pwm: supporta intensità PWM;

coil_polarity_control: supporta inversione di polarità;

coil_release_active: supporta rilascio attivo;

coil_hold_mode: supporta mantenimento a duty ridotto;

coil_temperature_feedback: supporta temperatura reale.

Questa distinzione evita confusione tra modello e hardware. Non bisogna dichiarare funzioni magnetiche che il circuito reale non può eseguire. Il sistema può simulare attract/release, ma l'hardware deve essere onesto sulle proprie capabilities.

La protezione delle bobine deve includere il controllo dei comandi sincronizzati. Se più nodi devono attivare bobine nello stesso slot, il controller deve verificare il carico complessivo. Un singolo nodo può essere sicuro, ma dieci nodi attivi contemporaneamente possono creare picco di consumo. Il controller deve quindi applicare una policy di swarm-level current protection. Può limitare il numero massimo di bobine attive per slot, distribuire i comandi in gruppi o usare duty più basso quando il numero di attuatori attivi è alto.

Una policy globale può essere:

$\text{max_active_coils_per_slot} = K$

Se il pattern richiede più di K bobine, il controller divide l'azione in più slot. Questo è un modo elegante di collegare Safety Layer e slot temporali. Il pattern resta lo stesso, ma l'attuazione fisica viene pianificata in modo sicuro.

La protezione delle bobine deve anche prevenire attivazioni contrastanti. In una struttura complessa, due bobine vicine potrebbero generare effetti indesiderati se attivate insieme o in direzioni incompatibili. Anche senza modellare perfettamente il campo magnetico, il sistema può applicare regole conservative: non attivare contemporaneamente bobine in coppie vicine se non previste dal pattern; non attivare coil di release e attract sullo stesso edge nello stesso slot; non attivare nodi in stato out_of_phase. Questa protezione topologica è importante per evitare comportamenti fisici confusi.

Una regola può essere:

un comando bobina è valido solo se phase_id, edge_id e role sono coerenti con il pattern corrente.

Per esempio, un nodo connector può attivare la bobina solo per un edge previsto nella fase attach. Se riceve un magnetic_pulse generico fuori fase, lo rifiuta. Questo impedisce attivazioni magnetiche fuori contesto.

La protezione delle bobine richiede quindi un collegamento con i ruoli. Non tutti i nodi devono poter attivare bobine in ogni momento. Un active_connector può essere autorizzato. Un passive_node no. Un diagnostic_node può eseguire test limitati solo in diagnostic mode. Un boundary_node può essere attivo solo se il pattern lo richiede. Il firmware può controllare assigned_role prima di accettare un comando. Il controller deve assegnare ruoli compatibili prima di inviare comandi magnetici.

Una validazione role-based può essere:

command magnetic_pulse permesso se $\text{role} \in \{\text{active_connector}, \text{actuator_node}, \text{test_node}\}$ e mode compatibile.

Se $\text{role} = \text{passive_node}$, il comando viene rifiutato con $\text{reason} = \text{role_not_authorized}$. Questa regola aumenta la sicurezza perché impedisce attuazioni accidentali su nodi che non dovrebbero essere attivi.

La protezione delle bobine deve distinguere tra modalità test e modalità pattern. In modalità test, l'utente può voler provare una bobina singola con duty basso e durata breve. In modalità pattern, la bobina viene usata come parte di una sequenza coordinata. Le policy devono essere diverse. In test, il sistema deve essere molto conservativo, con un solo nodo alla volta e parametri limitati. In pattern, il controller può coordinare più nodi, ma solo se readiness, sync, ruoli e safety sono validi.

Modalità test:

un solo nodo;

duty basso;

durata breve;

cooldown lungo;

supervisione dashboard;

nessun pattern automatico complesso.

Modalità pattern:

nodi selezionati;

slot temporali;

ruoli validi;

sync valida;

policy globale di corrente;

log completo.

Questa distinzione è molto importante durante lo sviluppo hardware. I test manuali non devono essere confusi con attuazione swarm automatica.

La protezione delle bobine deve includere un blocco in caso di comunicazione persa. Se il nodo non vede più il controller per un certo tempo, non deve accettare o mantenere attuazioni. Se una bobina è attiva e il controller scompare, la durata locale la spegne comunque. Se la bobina è in idle, il nodo entra in `safe_idle` o blocca nuovi comandi finché la comunicazione non torna. Questo evita condizioni non supervisionate.

Una regola può essere:

se controller_lost = true, coil_commands_enabled = false.

Per riabilitare, serve riconnessione, sync e state_sync. In modalità hardware reale, questo comportamento è essenziale.

La protezione delle bobine deve anche supportare emergency_stop. Come già detto, emergency_stop deve spegnere immediatamente tutte le bobine. Deve cancellare i comandi pendenti e impedire nuove attivazioni finché il sistema non viene ripristinato. Il nodo deve eseguire prima lo spegnimento, poi inviare ack. Il controller deve inviare emergency_stop in broadcast e, se necessario, ripeterlo per un breve periodo. Ma il nodo non deve dipendere da ripetizioni: alla prima ricezione valida spegne.

La sequenza corretta è:

ricezione emergency_stop;

coil_output = OFF;

PWM duty = 0;

pending coil commands cancellati;

coil_state = safety_off;

mode = safety o safe_idle;

ack inviato se rete disponibile.

Questa logica deve essere implementata in firmware con priorità massima.

La protezione delle bobine deve gestire anche comandi duplicati. Se un magnetic_pulse viene ritrasmesso per errore, il nodo non deve eseguire due impulsi. Deve riconoscere message_id già processato. Questo è particolarmente importante per attuatori. Un duplicate_message può essere innocuo per set_mode idle, ma non per magnetic_pulse. Il nodo deve mantenere una cache dei comandi attuativi recenti. Se riceve lo stesso message_id, può reinviare ack ma non ripetere l'impulso.

Una regola fondamentale è:

ogni magnetic_pulse con lo stesso message_id può essere eseguito al massimo una volta.

Se il controller vuole davvero un secondo impulso, deve inviare un nuovo command_id, rispettando cooldown e limiti. Questo protegge da retry e pacchetti duplicati.

La protezione delle bobine deve considerare anche il caso di comando arrivato tardi. Un magnetic_pulse associato a una fase attach non deve essere eseguito durante stabilize o dopo la fine della run. Per questo deve avere expires_at e pattern_run_id. Il nodo verifica che il comando appartenga alla run corrente e alla fase corretta. Se è vecchio, lo rifiuta come command_expired o phase_mismatch.

Una validazione temporale minima è:

pattern_run_id corrisponde;

phase_id compatibile;

timebase valida;

execute_at non nel passato oltre tolleranza;

expires_at non superato.

Se una di queste condizioni fallisce, non si attiva la bobina. Questo protegge da ritardi e pacchetti fuori ordine.

La protezione delle bobine deve prevedere errori specifici. Alcuni errori possibili sono: coil_not_available, coil_disabled, duty_cycle_too_high, duration_too_long, cooldown_active, thermal_warning_limit, thermal_lock, energy_low, role_not_authorized, phase_mismatch, command_expired, duplicate_pulse, driver_fault se rilevabile. Questi errori devono essere codificati in modo leggibile. Il controller deve poterli usare per adattamento e correzione.

Una risposta può essere:

```
type = ack
result = rejected
reason = thermal_lock
coil_id = main
thermal_load = 0.92
cooldown_remaining_ms = 12000
```

Oppure:

```
type = ack
result = rejected
reason = role_not_authorized
role = passive_node
required_role = active_connector
```

Queste risposte rendono il Safety Layer operativo. Non è solo blocco; è comunicazione del motivo.

La protezione delle bobine deve essere visualizzata nella dashboard con uno stato chiaro. Ogni nodo con bobina dovrebbe mostrare coil_state, duty attuale, pulse_remaining_ms, cooldown_remaining_ms, thermal_load, last_pulse, last_rejection_reason e safety_state. Durante una demo hardware, questa vista permette di capire subito se il nodo è pronto, in cooldown o bloccato. Senza dashboard, l'utente dovrebbe intuire dal comportamento fisico, che è molto meno sicuro.

Una scheda bobina può mostrare:

coil_id;

coil_state;

capability;

last_command_id;

current_duty;

remaining_pulse_time;

cooldown_remaining;

thermal_load;

thermal_state;

last_error.

Questo pannello sarebbe molto utile per sviluppo e presentazione. Mostra che l'attuazione magnetica è controllata, non improvvisata.

La protezione delle bobine deve essere documentata nei test. Ogni test deve indicare configurazione, duty, durata, cooldown, alimentazione, temperatura stimata o misurata, comportamento osservato e risultato. I test non devono limitarsi a “funziona/non funziona”. Devono registrare se la bobina si scalda, se il driver resta stabile, se l'ESP32 si resetta, se il comando viene rifiutato correttamente, se emergency_stop funziona, se il cooldown viene rispettato.

Una scheda test può contenere:

test_id;

node_id;

coil_id;

hardware_version;

firmware_version;

duty_cycle;

duration_ms;

cooldown_ms;

repetition_count;

power_source;

thermal_observation;

result;

notes.

Questi dati sono fondamentali per costruire fiducia. Se in futuro si aumenta la potenza, serve sapere da quali valori si è partiti.

La protezione delle bobine deve essere sviluppata per fasi. Nella prima fase, la bobina può essere solo simulata nel firmware e nella dashboard. Il nodo riceve magnetic_pulse ma non pilota un carico reale; aggiorna coil_state virtuale. Questo serve a testare protocollo, timing, cooldown e log. Nella seconda fase, si usa un carico sicuro come LED o piccolo carico di prova per verificare PWM. Nella

terza, si usa driver senza carico critico. Nella quarta, si collega una bobina reale con parametri molto conservativi. Nella quinta, si integra nel pattern.

Una progressione sicura è:

coil_state simulato;

PWM su LED;

PWM su carico test sicuro;

driver testato;

bobina reale con impulso breve;

cooldown e thermal_load;

emergency_stop;

pattern con singolo nodo;

pattern con più nodi in slot separati.

Questa sequenza evita di mettere subito a rischio componenti e permette di verificare ogni livello.

La protezione delle bobine deve mantenere distinzione tra hardware reale e simulazione. In simulazione, un magnetic_pulse può modificare una forza virtuale. Nell'hardware reale, modifica una corrente e un campo. Il comando può avere lo stesso nome, ma l'effetto e i rischi sono diversi. Per questo, il nodo deve dichiarare implementation_type. Un simulated_node può accettare impulsi virtuali. Un hardware_visual_node li rifiuta. Un hardware_coil_node li accetta solo entro limiti. Questo evita che il controller tratti tutti i nodi come se fossero uguali.

Una capability può essere:

magnetic_actuation = simulated / none / hardware_partial / hardware_ready

Se magnetic_actuation = none, magnetic_pulse non viene inviato. Se simulated, agisce solo nel simulatore. Se hardware_partial, richiede modalità test e limiti severi. Se hardware_ready, può partecipare a pattern hardware-safe. Questa classificazione è molto importante per evitare allucinazioni tecniche e mantenere coerenza.

La protezione delle bobine deve considerare la polarità e l'inversione solo se l'hardware lo supporta. Un semplice MOSFET low-side può accendere e spegnere una bobina, ma non invertire la corrente. Per invertire polarità servono circuiti più complessi, come ponte H o configurazioni dedicate. Quindi, nel documento, quando si parla di attract e release, bisogna distinguere tra release passivo, cioè spegnimento della bobina, e release attivo, cioè campo opposto o altra azione fisica. Nelle prime versioni, release potrebbe significare solo coil_off o riduzione del campo.

Questa distinzione è fondamentale. Se l'hardware non supporta polarità inversa, non si deve dichiarare rilascio attivo reale. Si può dire:

release passivo = disattivazione bobina;

release attivo = funzione futura che richiede hardware dedicato.

Questa precisione mantiene il documento tecnicamente serio.

La protezione delle bobine deve essere legata alla gestione della batteria. Se il nodo è alimentato a batteria, la bobina può causare cali di tensione o consumare rapidamente energia. Il nodo deve evitare impulsi quando `battery_level` è troppo basso o quando la tensione stimata scende sotto soglia. Anche se il firmware non misura ancora la batteria, il modello deve prevedere `energy_state`. In futuro, una misura reale potrà alimentare questa logica. In ogni caso, le bobine sono tra i principali consumatori energetici, quindi devono essere disabilitate in `low_energy`.

Una regola è:

se `energy_state` $\in$ {`low_energy`, `critical_energy`}, rifiutare `magnetic_pulse`.

In `critical_energy`, il nodo dovrebbe spegnere uscite non essenziali e restare in heartbeat minimo o `safety`. Questo sarà approfondito nella gestione della batteria, ma per le bobine va già dichiarato.

La protezione delle bobine deve anche considerare i limiti meccanici e magnetici. Un campo magnetico può produrre attrazione indesiderata, movimento improvviso, aggancio non previsto o interferenza con altri componenti. Anche se nei primi prototipi la forza può essere piccola, il principio deve essere presente: l'attuazione magnetica deve essere contestuale. Non si attiva una bobina senza sapere quale edge, fase o obiettivo sta servendo. Questo collega la bobina alla topologia del pattern. Una bobina non deve essere solo "accesa"; deve essere attivata per un motivo strutturale.

Un comando ben formato può includere:

`edge_id`;

`target_neighbor`;

`pattern_run_id`;

`phase_id`;

`role_required`.

Per esempio, un `active_connector` può attivare una bobina per tentare connessione con MB-003 durante `phase_attach` della `run_bridge_run_001`. Questo è molto più controllato di un comando generico. Nelle prime versioni, `edge_id` può essere opzionale; nelle versioni hardware più avanzate, diventa importante.

La protezione delle bobine deve impedire azioni fuori fase. Se il pattern è in `phase_prepare`, non si devono attivare bobine. Se è in `phase_align`, forse solo LED o preparazione. Se è in `phase_attach`, alcune bobine possono essere abilitate. Se è in `phase_stabilize`, magari si riduce o disattiva. Se è in `phase_report`, nessuna attuazione. Questa disciplina temporale evita attivazioni caotiche.

Una policy per fase può essere:

`prepare`: bobine disabilitate;

`align`: bobine disabilitate o solo test leggerissimo se previsto;

`attach`: bobine abilitate secondo ruoli e slot;

stabilize: bobine limitate o hold mode se supportato;

report: bobine disabilitate;

safety: bobine disabilitate.

Questa tabella rende il comportamento prevedibile.

La protezione delle bobine deve prevedere modalità hold solo con cautela. Un hold mode potrebbe mantenere una connessione con duty ridotto. Tuttavia, mantenere una bobina attiva a lungo, anche a duty basso, può generare calore. Nelle prime versioni è meglio evitare hold persistente reale e trattarlo come funzione futura o simulata, salvo test molto controllati. Se implementato, hold deve avere durata massima, duty molto basso, monitoraggio termico e timeout. Non deve mai essere indefinito.

Una regola è:

hold mode, se supportato, deve avere max_hold_duration_ms e thermal monitoring.

Questo evita che “hold” diventi una bobina sempre accesa.

La protezione delle bobine deve infine essere parte della definizione dei pattern hardware. Un pattern che richiede agganci magnetici deve dichiarare il profilo di attuazione: quali nodi attivano, quando, con quale duty massimo, per quanto tempo, quanti retry sono ammessi, quale cooldown serve e quale fallback scatta se la connessione non si forma. Il pattern non può limitarsi a dire “attach”. Deve dire come l’attach viene tentato in modo sicuro.

Una fase attach hardware-safe può contenere:

active_nodes;

target_edges;

max_duty_cycle;

pulse_duration_ms;

cooldown_ms;

max_retries;

required_sync_status;

abort_conditions;

fallback_if_failed.

Questa struttura collega Safety Layer e pattern. La forma desiderata deve essere compatibile con i limiti fisici.

In conclusione, la protezione delle bobine è uno dei componenti più importanti del Safety Layer MicroBot perché riguarda il passaggio dalla simulazione e visualizzazione all’attuazione magnetica reale. Una bobina deve essere pilotata con hardware adeguato, firmware non bloccante, comandi temporizzati, duty cycle limitato, durata massima, cooldown, thermal_load, emergency_stop,

deduplicazione, controllo di fase, controllo di ruolo, controllo energetico, dashboard diagnostica e log completo. Il controller può pianificare l'azione, ma il nodo deve sempre proteggere localmente la bobina. La bobina non deve mai restare attiva indefinitamente, non deve essere comandata fuori fase, non deve ignorare cooldown o temperatura, e non deve essere usata se il nodo non è nello stato corretto. Questo punto prepara direttamente il successivo, dedicato al monitoraggio della temperatura, perché ogni protezione magnetica reale dipende dalla capacità di stimare o misurare il calore generato dagli attuatori e prevenire condizioni di surriscaldamento.

11.3 Monitoraggio della temperatura

Il monitoraggio della temperatura rappresenta il terzo elemento del Safety Layer MicroBot e costituisce il collegamento diretto tra controllo della corrente, protezione delle bobine e prevenzione del surriscaldamento. Ogni volta che un componente elettronico assorbe corrente, una parte dell'energia viene trasformata in calore. Questo fenomeno è normale, ma diventa critico quando il calore accumulato supera la capacità del componente, della scheda, del contenitore o dell'ambiente di dissiparlo. Nel progetto MicroBot, dove si prevede l'uso progressivo di LED, microcontrollori, driver, PWM, bobine, sensori e alimentazioni autonome, la temperatura deve essere considerata una variabile di sicurezza fondamentale, non un dettaglio secondario.

Il monitoraggio della temperatura ha un obiettivo preciso: impedire che un MicroBot continui a eseguire azioni quando il suo stato termico non è più compatibile con il funzionamento sicuro. Questo significa che il sistema deve sapere, o almeno stimare, se un nodo sta accumulando troppo calore. Nei primi prototipi, la temperatura può essere gestita tramite un modello stimato, cioè calcolando un `thermal_load` virtuale in base a duty cycle, durata degli impulsi, frequenza di attivazione e cooldown. Nelle versioni successive, invece, il sistema può integrare sensori reali di temperatura o usare dati disponibili da componenti specifici. La distinzione tra temperatura stimata e temperatura misurata è essenziale per mantenere rigore tecnico.

Un MicroBot iniziale potrebbe non avere un sensore termico fisico. Questo non significa che il Safety Layer debba ignorare la temperatura. Al contrario, proprio perché non esiste ancora una misura reale, bisogna adottare una politica conservativa. Se una bobina viene attivata più volte, il firmware può aumentare una variabile `thermal_load`. Se passa tempo senza attivazioni, la variabile diminuisce gradualmente. Se `thermal_load` supera una soglia, il nodo entra in `thermal_warning`. Se supera una soglia più alta, entra in `thermal_lock`. Questo modello non produce una temperatura in gradi Celsius, ma fornisce un indicatore interno di rischio termico.

Una formulazione concettuale può essere:

`thermal_load` = carico termico stimato del nodo o della bobina

`thermal_load` aumenta quando vengono attivati carichi ad alto consumo;

`thermal_load` diminuisce durante i periodi di riposo;

`thermal_warning` si attiva quando il carico supera una prima soglia;

`thermal_lock` si attiva quando il carico supera una soglia critica.

Questa struttura è molto utile perché permette di implementare protezione termica anche prima dell'hardware definitivo. Il sistema può già testare logiche di cooldown, rifiuto dei comandi, riduzione del duty cycle e fallback del pattern. In simulazione, si può generare un `thermal_load` artificiale per verificare come reagisce il controller. Nei prototipi fisici, si può iniziare con parametri

estremamente conservativi. Quando poi saranno disponibili misure reali, il modello potrà essere calibrato.

La temperatura deve essere monitorata su più livelli. Il primo livello riguarda la bobina, perché è il componente più soggetto ad accumulo termico quando viene pilotata con corrente. Il secondo livello riguarda il driver o MOSFET, che può scaldarsi durante il pilotaggio. Il terzo livello riguarda il microcontrollore, specialmente se lavora in Wi-Fi, invia telemetria, processa pacchetti e controlla uscite. Il quarto livello riguarda la batteria, che può essere sensibile a scariche intense o condizioni non sicure. Il quinto livello riguarda l'ambiente fisico del nodo, cioè la capacità complessiva della struttura di dissipare calore.

Nel progetto MicroBot, non tutti questi livelli saranno misurabili subito. Tuttavia, devono essere previsti. Una bobina può avere `thermal_load_coil`. Il driver può avere `thermal_load_driver` o almeno uno stato stimato. La scheda può avere `board_temperature` se un sensore lo permette. La batteria può avere `battery_temperature` in versioni avanzate. La dashboard può mostrare quali dati sono misurati e quali sono stimati. Questa distinzione deve essere sempre esplicita.

Una struttura di telemetria termica può contenere:

`temperature_source`;

`coil_temperature_c` se misurata;

`board_temperature_c` se misurata;

`battery_temperature_c` se misurata;

`thermal_load_estimated`;

`thermal_state`;

`thermal_warning_active`;

`thermal_lock_active`;

`cooldown_remaining_ms`.

Il campo `temperature_source` è fondamentale. Può assumere valori come `simulated`, `estimated`, `measured` o `unavailable`. Se il dato è `estimated`, la dashboard e il controller devono sapere che non è una temperatura reale. Se è `measured`, può essere usato con maggiore fiducia. Se è `unavailable`, il sistema deve applicare policy conservative. In assenza di informazione termica, non si deve aumentare aggressivamente la potenza.

Il monitoraggio della temperatura deve introdurre stati termici chiari. Uno stato termico minimo può includere `thermal_ok`, `thermal_warning`, `thermal_lock` e `thermal_recovery`. `thermal_ok` indica che il nodo può accettare comandi entro i limiti normali. `thermal_warning` indica che il carico termico è aumentato e che il sistema deve ridurre intensità, frequenza o durata. `thermal_lock` indica che gli attuatori termicamente critici sono bloccati. `thermal_recovery` indica che il nodo sta raffreddando e non è ancora tornato pienamente operativo. Questa macchina a stati impedisce interpretazioni ambigue.

Una progressione possibile è:

`thermal_ok` → `thermal_warning` → `thermal_lock` → `thermal_recovery` → `thermal_ok`

Il passaggio da `thermal_ok` a `thermal_warning` avviene quando il carico supera una prima soglia. Il passaggio da `thermal_warning` a `thermal_lock` avviene quando il carico supera una soglia critica. Il passaggio da `thermal_lock` a `thermal_recovery` avviene quando il carico scende sotto la soglia critica, ma non ancora sotto il livello sicuro. Il ritorno a `thermal_ok` avviene solo quando il carico è sceso sotto una soglia di sicurezza più bassa. Questo introduce hysteresis, cioè una differenza tra soglia di ingresso e soglia di uscita. L'hysteresis è fondamentale per evitare che il nodo passi continuamente da lock a ok se il valore oscilla vicino alla soglia.

Una regola corretta è:

entrare in `thermal_lock` quando `thermal_load` supera la soglia alta;

uscire da `thermal_lock` solo quando `thermal_load` scende sotto una soglia più bassa.

Questo evita instabilità decisionale. Senza hysteresis, il nodo potrebbe sbloccarsi e ribloccarsi continuamente, generando comportamento confuso. Con hysteresis, il rientro è più stabile e sicuro.

Il monitoraggio della temperatura deve essere integrato con i comandi. Prima di eseguire un comando ad alto consumo, il nodo deve controllare `thermal_state`. Se `thermal_state = thermal_ok`, può accettare il comando se tutti gli altri controlli sono validi. Se `thermal_state = thermal_warning`, può accettare solo comandi ridotti o può richiedere duty più basso e durata più breve. Se `thermal_state = thermal_lock`, deve rifiutare comandi ad alto consumo. Se `thermal_state = thermal_recovery`, può continuare a rifiutare finché non torna `thermal_ok`. Questa logica rende la temperatura un vincolo operativo.

Una policy può essere:

`thermal_ok`: comandi attuativi consentiti entro limiti normali;

`thermal_warning`: comandi consentiti solo con limiti ridotti o cooldown esteso;

`thermal_lock`: comandi attuativi rifiutati;

`thermal_recovery`: comandi attuativi sospesi finché il recupero non è completo.

Questa policy deve essere applicata localmente dal firmware. Il controller può sapere che un nodo è in warning o lock, ma il nodo non deve affidarsi solo al controller. Se il controller sbaglia, il nodo protegge comunque sé stesso.

Il monitoraggio della temperatura deve essere collegato alla protezione delle bobine. Ogni impulso magnetico aumenta il carico termico. La quantità di incremento può dipendere da `duty_cycle`, `duration_ms` e caratteristiche della bobina. In una forma semplice:

$$\text{thermal_load_increment} = \text{duty_cycle} \times \text{duration_ms} \times \text{coil_factor}$$

Dove `coil_factor` rappresenta quanto quella bobina contribuisce al riscaldamento. Nelle prime versioni, `coil_factor` può essere una costante prudente. In futuro, può essere calibrato sperimentalmente. Questa formula non pretende di essere un modello fisico perfetto, ma è sufficiente per costruire una logica di protezione. Se un nodo riceve molti impulsi ravvicinati, `thermal_load` cresce e il sistema entra in warning o lock.

Il raffreddamento può essere modellato come decremento nel tempo:

$$\text{thermal_load} = \text{thermal_load} - \text{cooling_rate} \times \text{delta_time}$$

Anche `cooling_rate` può essere una costante prudente. In un ambiente reale, il raffreddamento dipende da materiali, ventilazione, temperatura ambiente, dimensione della bobina, contenitore e supporto. Nelle prime versioni, meglio sottostimare il raffreddamento, cioè assumere che il componente si raffreddi lentamente. Questo rende la protezione conservativa. Quando saranno disponibili dati sperimentali, si potrà migliorare il modello.

Il monitoraggio della temperatura deve anche considerare il carico cumulativo del nodo. Non è solo la bobina a generare calore. Anche driver, regolatori, microcontrollore e altri componenti contribuiscono. Un nodo che invia molta telemetria Wi-Fi e pilota una bobina può scaldare più di un nodo idle. Per una prima versione, si può monitorare separatamente `thermal_load_coil` e `thermal_load_node`. Il primo rappresenta il rischio della bobina; il secondo rappresenta il rischio complessivo. In futuro, si possono aggiungere sensori reali.

Una struttura può essere:

`thermal_load_coil;`

`thermal_load_driver;`

`thermal_load_node;`

`thermal_state_global.`

Se `thermal_load_coil` è alto, si bloccano comandi bobina. Se `thermal_load_node` è alto, si può ridurre anche telemetria, LED intensi o altre funzioni non essenziali. Questo permette protezione più raffinata.

Il monitoraggio della temperatura deve influenzare l'assegnazione dei ruoli. Un nodo in `thermal_warning` non dovrebbe essere scelto come `active_connector` se esistono alternative. Un nodo in `thermal_lock` non deve ricevere ruoli attuativi. Può essere `passive_node`, `diagnostic_node` o `idle`. Se un `bridge_node` entra in `thermal_lock`, il controller può cercare un `replacement_connector` o passare a fallback. La temperatura diventa quindi una variabile del role assignment e dell'adattamento dinamico.

Una regola può essere:

se `thermal_state` $\neq$ `thermal_ok`, ridurre priorità del nodo per ruoli attivi.

Per ruoli critici, il controller deve richiedere `thermal_ok`. Per ruoli passivi, può accettare `thermal_warning` se non comporta attuazioni. Questo rende l'assegnazione più intelligente.

Il monitoraggio della temperatura deve influenzare i pattern. Un pattern che richiede molte attuazioni magnetiche può essere ridotto, rallentato o trasformato in modalità visuale se troppi nodi sono in `thermal_warning`. Il controller può inserire una `phase_cooldown` tra `attach` e `stabilize`, oppure distribuire impulsi in più slot. Se il sistema rileva `thermal_lock` su un nodo critico, può abortire la fase o eseguire fallback. Questo collega il Safety Layer direttamente alla Sezione 10.

Una policy di pattern può essere:

se un nodo non critico è in `thermal_warning`, continuare con duty ridotto;

se un nodo critico è in `thermal_warning`, estendere cooldown o sostituire ruolo;

se un nodo critico è in `thermal_lock`, abortire fase o fallback;

se più nodi sono in `thermal_warning`, inserire cooldown globale;

se molti nodi sono in `thermal_lock`, passare a `visual_mode` o `safety`.

Questa logica rende lo swarm capace di proteggersi collettivamente, non solo nodo per nodo.

Il monitoraggio della temperatura deve essere visibile nella dashboard. Ogni nodo deve mostrare `thermal_state` in modo chiaro. Se il dato è stimato, deve essere indicato. Se è misurato, deve essere indicato. La dashboard deve mostrare warning, lock, cooldown e carico termico. Questo serve sia per debug sia per presentazione. Quando un comando viene rifiutato per `thermal_lock`, l'utente deve vedere immediatamente perché. Questo evita interpretazioni sbagliate, per esempio pensare che il nodo non funzioni quando in realtà si sta proteggendo.

Una scheda termica può mostrare:

`thermal_state`;

`temperature_source`;

`estimated_thermal_load`;

`measured_temperature_c`, se disponibile;

`warning_threshold`;

`lock_threshold`;

`cooldown_remaining_ms`;

`last_heating_event`;

`last_thermal_rejection`.

Questi dati rendono la sicurezza trasparente. Il Safety Layer non è una scatola nera. L'utente vede il motivo delle decisioni.

Il monitoraggio della temperatura deve generare eventi nei log. Ogni passaggio di stato termico deve essere registrato. Quando un nodo entra in `thermal_warning`, il log deve registrare tempo, valore stimato o misurato, ultima attuazione e stato del nodo. Quando entra in `thermal_lock`, deve registrare quali comandi sono stati bloccati. Quando torna in `thermal_ok`, deve registrare quanto tempo è durato il raffreddamento. Questi dati sono fondamentali per analizzare i test hardware.

Un evento termico può contenere:

`event_type = thermal_warning / thermal_lock / thermal_recovery / thermal_ok`;

`node_id`;

`coil_id` se applicabile;

`thermal_load`;

`measured_temperature_c` se disponibile;

`source`;

trigger_command_id;

timestamp;

cooldown_remaining_ms.

Questa struttura permette di ricostruire la storia termica del nodo. Se un test causa ripetuti thermal_warning, bisogna ridurre duty, durata o frequenza.

Il monitoraggio della temperatura deve essere collegato al concetto di cooldown. Il cooldown non è soltanto una pausa arbitraria dopo un impulso. È una strategia per permettere al carico termico di scendere. Se thermal_load è alto, il cooldown deve essere più lungo. Se thermal_load è basso, può essere più breve. Una politica avanzata può calcolare cooldown dinamico. Una politica iniziale può usare cooldown fisso ma conservativo. In entrambi i casi, il cooldown deve essere rispettato dal firmware.

Una regola è:

durante cooldown, comandi termicamente critici vengono rifiutati.

Il nodo può continuare a eseguire funzioni leggere, come heartbeat o LED a bassa intensità, ma non deve riattivare la bobina. Se il controller tenta di farlo, il nodo risponde cooldown_active. Questo è un comportamento corretto, non un errore del nodo.

Il monitoraggio della temperatura deve anche prevenire l'effetto di accumulo da retry. Se un comando magnetico fallisce, il controller potrebbe voler ritentare. Ma ogni retry genera nuovo calore. Quindi retry e thermal_load devono essere collegati. Un pattern non deve ritentare attach molte volte senza considerare temperatura. Il retry_limit deve dipendere anche dallo stato termico. Se thermal_warning è attivo, i retry devono ridursi o fermarsi.

Una policy può essere:

se thermal_ok, retry entro limite normale;

se thermal_warning, ridurre retry o aumentare cooldown;

se thermal_lock, nessun retry;

se retry fallisce e thermal_load cresce, passare a fallback.

Questa logica impedisce che la correzione degli errori peggiori la sicurezza termica. È un punto molto importante: il sistema non deve insistere fisicamente solo perché il pattern non è riuscito.

Il monitoraggio della temperatura deve considerare anche la temperatura ambiente. In versioni avanzate, se il nodo lavora in un ambiente caldo, i margini devono essere più conservativi. Se la temperatura ambiente è alta, il raffreddamento è meno efficace. Se la temperatura ambiente è bassa, il raffreddamento può essere migliore. Nei primi prototipi, questa misura probabilmente non sarà disponibile. Ma il modello deve prevedere che le soglie termiche reali dipendono dal contesto. Per questo motivo, i test devono registrare condizioni ambientali approssimative quando si lavora con attuatori.

Un report di test può indicare:

temperatura ambiente stimata;

durata del test;

numero di impulsi;

duty cycle;

thermal_load finale;

eventuali warning;

tempo di recupero.

Questi dati aiutano a capire se un comportamento è sicuro solo in certe condizioni. Per una tesi o documentazione tecnica, questo livello di rigore è molto utile.

Il monitoraggio della temperatura deve distinguere tra dati affidabili e dati incerti. Una temperatura misurata da un sensore mal posizionato può non rappresentare la bobina. Una stima può essere troppo conservativa o troppo ottimistica. Un sensore può avere ritardo termico: misura l'aumento con qualche secondo di ritardo. Quindi anche la temperatura misurata non è perfetta. Il sistema deve trattare la termica con margini. Non bisogna aspettare di leggere un valore pericoloso prima di fermarsi; meglio fermarsi prima con soglie conservative.

Una regola di progetto è:

le soglie di blocco devono essere conservative e includere margine rispetto al limite reale del componente.

Se un componente ha una temperatura massima dichiarata, non bisogna usare quella come soglia operativa. Bisogna usare un valore più basso, tenendo conto di incertezza, ritardi di misura e variabilità. Questo è un principio generale di sicurezza hardware.

Il monitoraggio della temperatura deve essere integrato con la modalità diagnostic. In diagnostic mode, il sistema può permettere test termici controllati: un impulso breve, lettura del carico stimato, osservazione del raffreddamento, verifica del cooldown. Questi test devono essere limitati e supervisionati. La modalità diagnostic non deve bypassare i limiti, ma può fornire più informazioni. Per esempio, può mostrare thermal_load in tempo reale o registrare curve di raffreddamento.

Una procedura diagnostica può essere:

verifica nodo in idle e safety_ok;

applica impulso di test a duty basso;

registra thermal_load prima e dopo;

attende cooldown;

verifica ritorno verso thermal_ok;

salva log del test.

Questa procedura aiuta a calibrare il modello termico. Nelle prime fasi, si può fare manualmente osservando il comportamento e toccando con prudenza i componenti solo se sicuro; in versioni più mature, si useranno sensori.

Il monitoraggio della temperatura deve essere legato al watchdog e al ripristino automatico. Se il firmware si resetta a causa di watchdog o brownout mentre thermal_load era alto, il controller deve essere prudente. Dopo un reset, il nodo potrebbe aver perso la stima termica locale. Se il componente era caldo prima del reset, il firmware nuovo potrebbe non saperlo. Per questo motivo, dopo reset o boot, un nodo con bobina dovrebbe entrare in una fase di cooldown iniziale o dichiarare thermal_state = unknown/recovery finché non passa un tempo di sicurezza o finché una misura reale non conferma temperatura ok.

Questa è una considerazione importante. Se thermal_load è solo in RAM, un reset lo cancella. Ma il calore fisico non sparisce con il reset. Quindi un nodo non dovrebbe tornare immediatamente a piena attuazione dopo reboot. Una policy conservativa può essere:

dopo reboot, coil_commands_disabled per un cooldown iniziale;

thermal_state = thermal_recovery;

solo dopo recovery_time, thermal_state = thermal_ok se non ci sono warning.

Questa regola protegge da reset durante surriscaldamento. È un dettaglio molto maturo.

Il monitoraggio della temperatura deve essere coordinato con la gestione della batteria. Batterie e celle possono essere sensibili alla temperatura. In versioni autonome, un aumento termico può riguardare non solo bobina e driver, ma anche batteria o regolatore. Se battery_temperature è alta, il sistema deve ridurre carichi e impedire attuazioni intense. Se non esiste misura reale, bisogna evitare profili di consumo aggressivi. La sezione sulla batteria approfondirà questo aspetto, ma qui va stabilito che temperatura ed energia sono collegate.

Una regola generale è:

thermal_warning o battery_warning riducono il profilo energetico del nodo.

Il nodo può passare da active a conservative, da actuator a passive, da pattern a diagnostic o idle. Questo protegge sia componenti sia batteria.

Il monitoraggio della temperatura deve essere integrato nella telemetria del sistema. La Sezione 13 tratterà la telemetria in modo specifico, ma già qui si deve dire che thermal_state è una delle informazioni più importanti da inviare al controller. Un heartbeat minimo può indicare solo mode, ma un heartbeat esteso durante pattern hardware deve includere almeno thermal_state e possibilmente thermal_load. Se un nodo entra in warning o lock, deve inviare un evento immediato, non aspettare il prossimo heartbeat lungo. Gli eventi termici sono prioritari.

Un evento prioritario può essere:

thermal_lock_triggered;

thermal_warning_triggered;

thermal_recovery_started;

thermal_ok_restored.

Il controller deve reagire a questi eventi aggiornando ruoli, pattern e dashboard. Per esempio, se un active_connector entra in thermal_lock durante phase_attach, il controller deve bloccare nuovi impulsi e decidere se fallback o abort.

Il monitoraggio della temperatura deve anche permettere analisi post-run. Alla fine di una run, il report dovrebbe indicare carico termico massimo, numero di warning, numero di lock, tempo totale in cooldown, nodi più sollecitati e comandi rifiutati per ragioni termiche. Questi dati sono fondamentali per migliorare pattern e hardware. Se una certa fase produce sempre `thermal_warning`, bisogna ridurre duty, aumentare slot, modificare topologia o cambiare hardware. Senza report termico, il sistema potrebbe sembrare funzionare ma stressare troppo i componenti.

Un report termico può includere:

`max_thermal_load_by_node;`

`thermal_warning_count;`

`thermal_lock_count;`

`average_cooldown_time;`

`coil_activation_count;`

`rejected_due_to_thermal;`

`phase_with_highest_thermal_load;`

`recommendations.`

Questo tipo di report renderebbe la documentazione hardware molto più forte.

Il monitoraggio della temperatura deve essere considerato anche nelle demo. Durante una presentazione, il sistema deve evitare di spingere l'hardware al limite. Una demo deve essere ripetibile e sicura. Se una sequenza richiede molte attivazioni, deve avere pause. Se la dashboard mostra `thermal_load`, si può spiegare che il sistema non esegue comandi fuori soglia. Questo è un punto positivo, non un limite: dimostra che MicroBot non è un giocattolo incontrollato ma una piattaforma con protezione integrata.

Una demo sicura può mostrare:

impulso breve;

`thermal_load` che aumenta;

cooldown che parte;

nuovo comando rifiutato durante cooldown;

ritorno a `thermal_ok`;

secondo impulso accettato.

Questa mini-demo del Safety Layer sarebbe molto efficace perché mostra la logica di protezione in modo visibile e comprensibile.

Il monitoraggio della temperatura deve avere una roadmap incrementale. Nella prima versione, `thermal_load` può essere simulato. Nella seconda, può essere stimato nel firmware in base agli impulsi. Nella terza, può essere mostrato in dashboard. Nella quarta, si possono aggiungere sensori

reali. Nella quinta, si può calibrare il modello stimato confrontandolo con misure. Nella sesta, si possono usare policy adattive basate su temperatura reale.

Una roadmap termica può essere:

v0.1: thermal_state simulato nel software;

v0.2: thermal_load stimato nel firmware;

v0.3: cooldown e thermal_lock automatici;

v0.4: dashboard termica;

v0.5: logging termico per run;

v0.6: sensore temperatura su nodo o bobina;

v0.7: calibrazione tra stima e misura;

v0.8: adattamento automatico dei pattern in base allo stato termico.

Questa progressione è coerente con lo sviluppo incrementale del progetto. Non richiede subito sensori, ma prepara il sistema a integrarli.

Il monitoraggio della temperatura deve essere trattato come parte della credibilità scientifica del sistema. Qualunque progetto che usa bobine, attuatori, batterie o microcontrollori deve affrontare il calore. Ignorarlo renderebbe la documentazione fragile. Includerlo invece mostra consapevolezza dei limiti fisici. Per MicroBot, che vuole collegare teoria, simulazione, hardware e visione futura, la temperatura è uno dei punti in cui si vede il passaggio dalla matematica alla realtà. Il modello può dire che un nodo deve attrarre un altro nodo, ma l'hardware può farlo solo entro vincoli termici.

In conclusione, il monitoraggio della temperatura è il sistema attraverso cui MicroBot osserva, stima o misura il carico termico dei nodi, delle bobine, dei driver e degli altri componenti critici. Esso introduce stati come thermal_ok, thermal_warning, thermal_lock e thermal_recovery; usa soglie conservative, cooldown, hysteresis, logging, dashboard e rifiuto dei comandi fuori limite. Nelle prime versioni può basarsi su thermal_load stimato; nelle versioni avanzate può integrare sensori reali. Il suo scopo non è soltanto evitare danni, ma rendere il comportamento fisico del sistema compatibile con la sicurezza. La temperatura diventa quindi una variabile operativa: influenza comandi, ruoli, pattern, fallback, telemetria e report. Il punto successivo, dedicato alla prevenzione del surriscaldamento, svilupperà ulteriormente questa logica, spiegando come il sistema possa impedire in anticipo l'accumulo eccessivo di calore attraverso limiti, cooldown, degradazione, scheduling e strategie conservative.

11.4 Prevenzione del surriscaldamento

La prevenzione del surriscaldamento rappresenta il passaggio successivo rispetto al semplice monitoraggio della temperatura. Monitorare significa osservare, stimare o misurare lo stato termico del sistema; prevenire significa invece impedire che il sistema arrivi in condizioni termiche critiche. Questa distinzione è fondamentale. Un Safety Layer maturo non deve limitarsi ad accorgersi che un componente è già troppo caldo, ma deve progettare il comportamento dei MicroBot in modo da ridurre la probabilità stessa che il surriscaldamento avvenga. Nel progetto MicroBot, la prevenzione del surriscaldamento deve quindi essere vista come una strategia attiva di pianificazione, limitazione, distribuzione del carico, raffreddamento e degradazione controllata.

Il surriscaldamento può derivare da più cause. La prima è l'attivazione prolungata di un carico ad alto assorbimento, come una bobina o un attuatore. La seconda è la ripetizione troppo ravvicinata di impulsi, anche se ogni impulso singolo è entro i limiti. La terza è la concentrazione di molti comandi energetici nello stesso intervallo temporale. La quarta è una dissipazione insufficiente, causata da componenti piccoli, supporti chiusi, mancanza di ventilazione o materiali poco adatti. La quinta è una condizione ambientale sfavorevole, per esempio temperatura esterna alta. La sesta è un firmware non corretto, che mantiene uscite attive più del previsto o non applica cooldown. La settima è una pianificazione del pattern troppo aggressiva, che richiede al nodo più energia di quanta possa sostenere.

Per questo motivo, la prevenzione del surriscaldamento non può essere affidata a un solo meccanismo. Deve essere distribuita su più livelli: hardware, firmware, controller, pattern, simulazione, dashboard e log. L'hardware deve essere dimensionato con margine. Il firmware deve imporre limiti locali. Il controller deve pianificare comandi compatibili con lo stato termico. I pattern devono dichiarare quante attuazioni richiedono e quali fallback usare. La simulazione deve poter testare scenari di `thermal_load`. La dashboard deve rendere visibile il rischio termico. I log devono permettere analisi dopo ogni run. Solo l'insieme di questi livelli rende la prevenzione credibile.

Il principio generale può essere espresso così:

il sistema deve ridurre il carico termico prima che la temperatura raggiunga una condizione critica.

Questa frase è centrale. Significa che `thermal_lock` non deve essere la risposta normale. `Thermal_lock` è una protezione di ultima linea. Se il sistema entra spesso in `thermal_lock`, significa che la pianificazione è troppo aggressiva, il cooldown è troppo corto, il duty cycle è troppo alto, la durata degli impulsi è eccessiva o l'hardware non è adeguato. Una buona prevenzione dovrebbe mantenere il nodo prevalentemente in `thermal_ok` e, al massimo, occasionalmente in `thermal_warning`. Il `thermal_lock` deve essere raro e trattato come segnale di revisione del comportamento.

La prima strategia di prevenzione è la limitazione della durata. Ogni attuatore termicamente critico deve avere una durata massima di attivazione. Questo vale soprattutto per bobine, driver e uscite PWM collegate a carichi significativi. Una bobina non deve restare attiva indefinitamente. Ogni comando deve avere `duration_ms`, e il firmware deve spegnere automaticamente l'uscita quando la durata scade. La durata massima deve essere conservativa nelle prime versioni e aumentata solo dopo test documentati.

La durata massima non deve essere scelta soltanto in base al fatto che la bobina "sembra reggere". Deve essere scelta considerando margine, ripetibilità, temperatura ambiente, alimentazione, driver, duty cycle e numero di ripetizioni. Un impulso singolo di 200 ms può essere sicuro; dieci impulsi consecutivi di 200 ms possono non esserlo. Per questo motivo, `max_pulse_duration_ms` deve essere collegato a cooldown e `thermal_load`. La durata massima è il primo argine, ma non basta da sola.

La seconda strategia è la limitazione del duty cycle. Il duty cycle determina la potenza media erogata in un segnale PWM. Ridurre il duty cycle riduce corrente media e calore generato. Per prevenire il surriscaldamento, ogni bobina o attuatore deve avere un `max_safe_duty_cycle`. In `thermal_ok` può essere usato il limite normale. In `thermal_warning`, il limite deve essere ridotto. In `thermal_lock`, il duty cycle deve essere zero perché l'attuatore non deve essere attivato. Questa logica rende la prevenzione adattiva.

Una politica può essere descritta così:

thermal_ok: duty massimo normale;

thermal_warning: duty massimo ridotto;

thermal_recovery: duty bloccato o fortemente limitato;

thermal_lock: duty pari a zero.

Questa progressione impedisce al sistema di continuare a spingere l'attuatore quando il rischio termico è già salito. È una prevenzione attiva, perché riduce la potenza prima del blocco completo.

La terza strategia è il cooldown obbligatorio. Il cooldown è il tempo minimo di riposo dopo un'attuazione. Senza cooldown, impulsi singolarmente sicuri possono accumulare calore. Il firmware deve impedire nuove attivazioni finché il cooldown non è terminato. Il controller deve leggere o stimare `cooldown_remaining_ms` e pianificare di conseguenza. Se un pattern richiede un nuovo impulso prima della fine del cooldown, il controller deve ritardare la fase, scegliere un altro nodo, ridurre il pattern o passare a fallback.

Il cooldown può essere fisso o dinamico. Un cooldown fisso è più semplice: dopo ogni impulso, si aspetta sempre lo stesso tempo. Un cooldown dinamico è più intelligente: impulsi più lunghi o più intensi richiedono più riposo. Nelle prime versioni MicroBot, un cooldown fisso e conservativo è probabilmente la scelta più sicura. In versioni successive, si può calcolare:

$$\text{cooldown_ms} = \text{base_cooldown} + \text{coefficiente} \times \text{duty_cycle} \times \text{duration_ms}$$

Questa formula permette di collegare il riposo al carico reale generato dal comando. Anche se approssimata, migliora la protezione rispetto a un valore unico.

La quarta strategia è la distribuzione temporale del carico. Se più MicroBot attivano bobine nello stesso momento, il carico energetico e termico complessivo può aumentare rapidamente. Anche se ogni nodo è protetto localmente, lo swarm nel suo insieme può creare picchi di consumo e stress. Per prevenire questo, il controller deve usare slot temporali per distribuire le attuazioni. Invece di attivare dieci nodi insieme, può attivarne due per volta, poi attendere cooldown, poi attivare un altro gruppo. Questo rende il pattern più lento, ma molto più sicuro.

Una regola di swarm-level prevention può essere:

non più di K attuatori termicamente critici attivi nello stesso slot.

Il valore K dipende dall'alimentazione, dal tipo di attuatore, dal pattern e dal livello di sicurezza richiesto. Nelle prime demo, K dovrebbe essere basso. In simulazione si possono testare valori più alti, ma nell'hardware reale bisogna restare conservativi. Questa strategia mostra come la prevenzione del surriscaldamento non sia solo locale, ma anche sistemica.

La quinta strategia è la riduzione della frequenza dei comandi. Anche se duty e durata sono limitati, inviare troppi comandi ravvicinati può aumentare calore. Il controller deve quindi controllare quante attuazioni vengono richieste in una finestra temporale. Un nodo può avere `max_pulses_per_minute` o `max_energy_per_window`. Se supera questo limite, entra in modalità conservativa o rifiuta nuovi comandi. Questo è particolarmente importante nei pattern con retry. Un sistema che fallisce un aggancio non deve ritentare all'infinito, perché ogni retry produce nuovo calore.

Una politica può essere:

se il numero di impulsi recenti supera soglia, bloccare nuovi impulsi o allungare cooldown.

Questa logica impedisce accumulo progressivo. È diversa dal cooldown singolo: il cooldown protegge tra due impulsi; il limite per finestra protegge contro molte ripetizioni nel tempo.

La sesta strategia è l'inserimento di fasi di raffreddamento nei pattern. Un pattern hardware-safe non deve essere solo una sequenza di attuazioni. Deve includere pause quando necessario. Per esempio, una fase attach può essere seguita da phase_cooldown prima di stabilize o prima di un nuovo retry. Durante phase_cooldown, i nodi mantengono stato sicuro, inviano telemetria, aggiornano thermal_load e aspettano che i carichi scendano. Questo rende il pattern più lento, ma molto più adatto all'hardware reale.

Una sequenza prudente può essere:

prepare;

align;

attach_pulse_group_1;

cooldown_group_1;

attach_pulse_group_2;

cooldown_group_2;

stabilize;

report.

Questa struttura dimostra che il tempo non serve solo a coordinare la forma, ma anche a proteggere i componenti. Nella materia programmabile reale, la forma deve emergere entro vincoli energetici e termici. Il pattern deve rispettare questi vincoli.

La settima strategia è la degradazione automatica della modalità operativa. Se il sistema rileva thermal_warning su uno o più nodi, può passare da full_mode a conservative_mode. In conservative_mode, riduce duty, aumenta cooldown, allarga gli slot e limita le attuazioni. Se il rischio cresce, può passare a visual_mode, cioè mantenere dashboard e LED ma disabilitare bobine. Se il rischio diventa critico, entra in safety_mode. Questa scala di degradazione impedisce il passaggio brusco da comportamento normale a blocco completo.

Una scala efficace è:

full_mode: attuazione consentita entro limiti normali;

conservative_mode: attuazione ridotta e più lenta;

visual_mode: bobine disabilite, solo LED/dashboard/simulazione;

diagnostic_mode: raccolta dati e test limitati;

safety_mode: uscite critiche spente e comandi attuativi bloccati.

Questa progressione è molto importante per la presentazione del sistema. Mostra che MicroBot non fallisce semplicemente quando si scalda; modifica la propria modalità per proteggersi. È una forma di adattamento dinamico applicata alla sicurezza termica.

L'ottava strategia è la riassegnazione dei ruoli. Se un nodo inizia a scaldarsi, il controller può rimuoverlo da ruoli attivi. Un `active_connector` in `thermal_warning` può diventare `passive_connector` o `diagnostic_node`. Un altro nodo, se disponibile e termicamente sicuro, può assumere il ruolo attivo. Questo permette al pattern di continuare senza forzare sempre lo stesso nodo. In strutture complesse, la distribuzione del carico tra nodi è essenziale. Se lo stesso `bridge_node` esegue sempre tutte le attuazioni, si scalderà più degli altri. Il controller può alternare ruoli o pianificare turni.

Una politica può essere:

se `thermal_warning` su un nodo attivo, ridurre il suo ruolo;

se esiste nodo compatibile `thermal_ok`, riassegnare;

se non esiste alternativa, rallentare pattern o fallback;

se `thermal_lock` su nodo critico, abortire fase o passare a struttura degradata.

Questo collega prevenzione termica, assegnazione dei ruoli e adattamento dinamico. Il ruolo non è solo funzione geometrica; può essere anche distribuzione del carico fisico.

La nona strategia è il controllo dei retry. La correzione degli errori può generare retry, ma ogni retry fisico può aumentare il carico termico. Per prevenire surriscaldamento, `retry_limit` deve dipendere dallo stato termico. Se `thermal_ok`, il pattern può tentare il numero standard di retry. Se `thermal_warning`, il numero di retry deve essere ridotto o accompagnato da cooldown più lungo. Se `thermal_lock`, nessun retry è permesso. Questo evita che il sistema tenti di correggere una connessione mancante fino a scaldare eccessivamente l'attuatore.

Una regola è:

la correzione degli errori non deve aumentare il rischio termico oltre i limiti del Safety Layer.

Questa frase è importante perché collega due funzioni fondamentali. Correggere un errore di pattern è utile, ma non deve avvenire a costo di danneggiare componenti. Se una connessione non si forma dopo i tentativi sicuri, il sistema deve passare a fallback o dichiarare `partial_success`, non insistere.

La decima strategia è la previsione termica prima dell'esecuzione. Il controller può stimare il carico termico previsto di una fase prima di avviarla. Se una fase `attach` richiede dieci impulsi su cinque nodi, può stimare se i nodi resteranno entro soglia. Se la previsione mostra rischio di `thermal_warning` o `lock`, il controller può modificare il piano prima di iniziare. Questo è un livello più maturo di prevenzione: non si aspetta l'errore; lo si prevede.

Una stima può considerare:

`thermal_load` corrente dei nodi;

numero di impulsi previsti;

duty cycle previsto;

duration_ms previsto;

cooldown disponibile;

tempo di raffreddamento tra slot.

Se la stima supera una soglia, il controller può distribuire le attuazioni, ridurre duty, aumentare cooldown o passare a visual_mode. Questa è prevenzione vera: il piano viene adattato prima di generare calore eccessivo.

La prevenzione del surriscaldamento deve essere applicata anche ai LED, anche se i LED sono meno critici delle bobine. Se si usano molti LED ad alta luminosità o LED RGB, il consumo può aumentare. Nei prototipi iniziali questo rischio è minore, ma il principio resta valido: la luminosità può essere limitata, i pattern LED possono usare duty ridotto, e in low_energy o thermal_warning si può ridurre intensità. Questo rende il sistema coerente. Non si protegge solo il componente più pericoloso; si applica una filosofia generale di risparmio e sicurezza.

Una policy LED può essere:

normal_mode: luminosità normale;

low_energy: luminosità ridotta;

thermal_warning: evitare pattern intensi non necessari;

safety_mode: LED rosso o pattern di errore a basso duty.

Questa logica è utile soprattutto per demo fisiche. Un LED può segnalare lo stato senza consumare eccessivamente.

La prevenzione del surriscaldamento deve essere integrata con la gestione della batteria. Una batteria sotto stress può scaldarsi o subire cali di tensione. Se il sistema richiede troppa corrente, il nodo può resettarsi o entrare in stato instabile. Per questo motivo, thermal_state ed energy_state devono dialogare. Se battery_state è low, il sistema deve ridurre attuazioni. Se thermal_state è warning, deve ridurre consumo. Se entrambi sono degradati, deve entrare in modalità conservativa o safety. La prevenzione termica e la protezione energetica sono quindi due lati dello stesso problema.

Una condizione combinata può essere:

se thermal_warning oppure low_energy, ridurre carico;

se thermal_lock oppure critical_energy, bloccare attuazioni;

se thermal_warning e low_energy insieme, passare direttamente a conservative_mode o visual_mode.

Questa logica rende il nodo più prudente quando più segnali di rischio si sommano.

La prevenzione del surriscaldamento deve considerare anche il firmware. Un firmware scritto male può causare surriscaldamento anche con hardware corretto. Per esempio, un loop bloccante può impedire lo spegnimento tempestivo di una bobina. Un errore nella gestione del timer può mantenere PWM attivo. Un bug nella deduplicazione può eseguire due volte lo stesso impulso. Una mancata gestione del cooldown può accettare comandi troppo ravvicinati. Per questo motivo, il firmware deve essere progettato con logica fail-safe e test specifici.

Alcuni requisiti firmware sono:

nessun delay bloccante durante attuazioni critiche;

timer locale per ogni uscita critica;

spegnimento automatico al termine della durata;

cooldown controllato nel loop principale;

emergency_stop prioritario;

watchdog timer attivo;

stato sicuro all'avvio;

rifiuto di comandi duplicati;

rifiuto di comandi scaduti.

Questi requisiti non sono opzionali quando si usano bobine. Sono parte della prevenzione del surriscaldamento.

La prevenzione deve essere verificata con test negativi, non solo con test positivi. Un test positivo verifica che un impulso valido funzioni. Un test negativo verifica che un impulso non valido venga rifiutato. Per il Safety Layer, i test negativi sono fondamentali. Bisogna verificare che il nodo rifiuti duration_ms troppo lungo, duty troppo alto, comando durante cooldown, comando in thermal_lock, comando duplicato, comando scaduto e comando fuori fase. Solo così si può dire che la prevenzione funziona davvero.

Una suite di test può includere:

comando valido entro limiti;

comando con duty troppo alto;

comando con durata troppo lunga;

comando durante cooldown;

comando ripetuto con stesso message_id;

comando scaduto;

comando durante thermal_warning;

comando durante thermal_lock;

emergency_stop durante impulso;

reboot dopo thermal_load alto;

controller_lost durante impulso.

Questi test devono essere eseguiti prima in simulazione, poi su carichi sicuri, poi eventualmente su bobine reali. Questo ordine è importante per ridurre rischio.

La prevenzione del surriscaldamento deve essere visibile nella dashboard. Non basta mostrare che il nodo è in thermal_lock. La dashboard dovrebbe mostrare anche azioni preventive: duty ridotto, cooldown attivo, modalità conservativa, slot allargati, attuazioni distribuite, ruolo riassegnato. Questo permette all'utente di capire che il sistema sta prevenendo, non solo reagendo. Una dashboard matura può mostrare "thermal prevention active" con motivo e strategia.

Un pannello potrebbe mostrare:

thermal_state;

thermal_prevention_mode;

current_limits_applied;

cooldown_remaining;

max_allowed_duty_now;

reason_for_reduction;

next_allowed_activation_time;

active_fallback.

Questi dati aiutano a diagnosticare. Se l'utente prova a inviare un comando e viene rifiutato, vede subito che il nodo è in cooldown o che il duty massimo è stato ridotto per warning termico.

La prevenzione deve essere registrata nei log non solo quando avviene un blocco, ma anche quando il sistema evita una condizione rischiosa. Per esempio, se il controller decide di allargare gli slot perché thermal_load previsto è alto, deve registrarlo. Se riduce duty, deve registrarlo. Se cambia ruolo a un nodo per evitare surriscaldamento, deve registrarlo. Questi eventi dimostrano che il sistema sta prevenendo attivamente.

Un evento di prevenzione può contenere:

event_type = thermal_prevention;

trigger = predicted_thermal_load_high;

action = reduce_duty / extend_cooldown / reschedule_slot / reassign_role / fallback_visual;

affected_nodes;

previous_value;

new_value;

expected_effect;

timestamp.

Questi log sono molto utili nella documentazione. Permettono di mostrare che il runtime prende decisioni conservative motivate da dati.

La prevenzione del surriscaldamento deve essere considerata anche nella progettazione meccanica futura. Un MicroBot fisico piccolo può avere poca capacità di dissipazione. Se si chiude una bobina in un contenitore senza ventilazione o senza materiale adatto, il calore si accumula. Se il componente è troppo vicino a batteria o sensori sensibili, può creare problemi. Anche se la documentazione attuale è architetturale, deve dichiarare che la struttura meccanica dovrà favorire dissipazione, separazione dei componenti critici e accesso termico. Il software può aiutare, ma non può compensare un design fisico completamente sbagliato.

Una regola progettuale è:

il design meccanico deve permettere dissipazione coerente con il profilo energetico previsto.

Questo sarà più importante quando il progetto passerà da breadboard a modulo compatto. Un prototipo su breadboard dissipa in modo diverso da un modulo chiuso. I test fatti su breadboard non valgono automaticamente per una scocca chiusa. Anche questo va tenuto presente.

La prevenzione deve tenere conto della miniaturizzazione. Più un MicroBot diventa piccolo, più diventa difficile dissipare calore. Una bobina piccola può scaldarsi rapidamente. Una batteria piccola può fornire meno corrente. Un driver compatto può avere meno margine termico. Quindi, man mano che il progetto si avvicina a micro-unità reali, la prevenzione termica diventa più importante, non meno. La roadmap hardware deve includere test termici a ogni riduzione di scala.

Una considerazione importante è:

il sistema può funzionare a una scala prototipale ma richiedere riprogettazione termica per una scala più piccola.

Questo è normale e non è un fallimento. È un vincolo ingegneristico. La documentazione deve riconoscerlo.

La prevenzione del surriscaldamento deve anche distinguere tra dimostrazione e funzionamento continuo. Un impulso breve durante una demo può essere sicuro. Un funzionamento continuo per minuti o ore richiede analisi diversa. Se MicroBot deve diventare piattaforma reale, non basta sopravvivere a una singola prova. Deve poter ripetere cicli senza accumulare calore oltre soglia. Per questo, i test devono includere run ripetute e non solo attivazioni isolate.

Una prova più seria può essere:

eseguire pattern per una run;

attendere cooldown;

ripetere più volte;

registrare thermal_load massimo;

verificare se thermal_load torna a baseline;

controllare warning ricorrenti;

modificare limiti se necessario.

Questo tipo di test mostra se il sistema è ripetibile. Una demo che funziona una volta ma non può essere ripetuta non è ancora robusta.

La prevenzione del surriscaldamento deve collegarsi alla simulazione software. La Sezione 12 tratterà la simulazione, ma già qui è importante dire che la simulazione può includere un modello termico semplificato. Ogni attuazione virtuale aumenta `thermal_load`. Ogni periodo di riposo lo riduce. Il simulatore può mostrare quali pattern sono termicamente sostenibili e quali no. Questo permette di testare strategie prima dell'hardware. Per esempio, si può simulare la differenza tra attivare tutti i connector insieme o distribuirli in slot. Il risultato può essere confrontato in termini di `thermal_load`.

Una simulazione termica può rispondere a domande come:

quanti impulsi genera una fase attach;

quali nodi si scaldano di più;

quale slot plan riduce il picco termico;

quanto cooldown serve tra fasi;

quale pattern è troppo aggressivo per hardware reale.

Questa funzione sarebbe molto utile per trasformare la sicurezza in parte della progettazione, non solo del firmware.

La prevenzione del surriscaldamento deve essere parte dei criteri di validazione del comportamento. Un pattern non può essere considerato pienamente riuscito se raggiunge la forma ma porta molti nodi in `thermal_lock`. In quel caso, il pattern è geometricamente riuscito ma termicamente non sostenibile. La validazione deve includere metriche termiche. Questo è un punto fondamentale: la qualità del comportamento non riguarda solo forma e topologia, ma anche sicurezza ed energia.

Una run può essere classificata così:

`success_clean`: pattern riuscito senza warning termici;

`success_with_warnings`: pattern riuscito ma con warning;

`partial_success_thermal_limited`: pattern degradato per prevenzione termica;

`failed_thermal_lock`: pattern fallito per blocco termico;

`aborted_safety`: pattern interrotto per rischio.

Questa classificazione è molto più rigorosa di un semplice success/fail. Mostra che la sicurezza è parte del risultato.

La prevenzione del surriscaldamento deve anche produrre raccomandazioni. Se una run genera warning ripetuti, il report può suggerire di ridurre duty, aumentare cooldown, distribuire attuatori, modificare ruolo dei nodi, ridurre retry o migliorare dissipazione. Queste raccomandazioni possono essere automatiche o manuali. Anche una forma semplice di report è utile per evolvere il progetto.

Un report può dire:

il nodo MB-004 ha raggiunto thermal_warning tre volte durante phase_attach;

il duty cycle medio richiesto era alto rispetto al limite conservativo;

si consiglia di aumentare cooldown o distribuire gli impulsi su più slot;

si consiglia di evitare MB-004 come active_connector nella run successiva.

Questo tipo di analisi rende il sistema più intelligente e aiuta lo sviluppo sperimentale.

La prevenzione del surriscaldamento deve essere coerente con il principio di gradualità. Ogni nuova funzione hardware deve iniziare con limiti bassi. Ogni aumento di potenza deve essere accompagnato da test. Ogni nuova scocca o disposizione deve richiedere nuova validazione termica. Ogni nuovo driver o bobina deve avere profilo termico. Non si deve trasferire automaticamente un limite da un componente a un altro. Questo è importante perché componenti apparentemente simili possono comportarsi diversamente.

Una regola di metodo è:

ogni combinazione bobina-driver-alimentazione-supporto deve essere validata separatamente.

Questo evita errori per generalizzazione. Se una bobina funziona con un alimentatore e un driver, non significa che funzioni allo stesso modo con un altro setup.

La prevenzione del surriscaldamento deve essere progettata in modo che il fallimento sia sicuro. Se una misura termica diventa unavailable, se il sensore si guasta, se thermal_load non è affidabile, se il nodo si resetta, il sistema deve assumere una condizione conservativa. Non deve dire “non ho dati, quindi continuo”. Deve dire “non ho dati, quindi limite”. Questo è un principio fondamentale nei sistemi safety-oriented.

Una regola è:

assenza di informazione termica affidabile → modalità conservativa.

In pratica, se temperature_source = unavailable e il nodo ha attuatori critici, il controller deve limitare pattern hardware o richiedere diagnostic. Se il nodo ha solo LED, la restrizione può essere minima. La severità dipende dal tipo di carico.

La prevenzione del surriscaldamento deve essere separata dal recupero. Prevenire significa evitare di arrivare al problema. Recuperare significa tornare da uno stato di warning o lock. Entrambi sono necessari, ma sono diversi. La prevenzione usa limiti, scheduling e cooldown. Il recupero usa attesa, raffreddamento, rientro con hysteresis e riabilitazione graduale. La sezione sul ripristino automatico approfondirà il recupero, ma qui bisogna chiarire che il miglior recupero è quello che si rende raramente necessario grazie a prevenzione efficace.

Una frase chiave è:

la prevenzione riduce la frequenza degli stati critici; il recupero gestisce gli stati critici quando comunque avvengono.

Questa distinzione aiuta a progettare il Safety Layer in modo ordinato.

La prevenzione del surriscaldamento deve essere considerata anche nel potenziale imprenditoriale e scientifico del progetto. Un sistema che integra protezione termica, limiti energetici, scheduling e

log di sicurezza è molto più credibile di un sistema che mostra solo movimento o luci. In robotica e hardware, la sicurezza è parte del valore. Se MicroBot vuole essere presentato come piattaforma, deve dimostrare che i suoi comportamenti sono controllati entro vincoli fisici reali. La prevenzione del surriscaldamento è una prova di maturità progettuale.

In conclusione, la prevenzione del surriscaldamento è la strategia con cui MicroBot impedisce che bobine, driver, microcontrollori, LED, batterie e altri componenti raggiungano condizioni termiche critiche. Essa non si limita a leggere una temperatura, ma agisce prima del problema attraverso durata massima, duty cycle limitato, cooldown, slot temporali, riduzione dei retry, degradazione della modalità operativa, riassegnazione dei ruoli, previsione del carico, dashboard diagnostica, log e test conservativi. Il thermal_lock resta una protezione necessaria, ma non deve essere la condizione normale: un sistema ben progettato deve prevenire l'arrivo frequente al blocco. Questa logica prepara direttamente il punto successivo, dedicato alla gestione della batteria, perché calore, corrente, autonomia e sicurezza energetica sono strettamente collegati: un MicroBot fisico non deve soltanto evitare di scaldarsi, ma deve anche usare l'energia in modo controllato, misurabile e sostenibile.

11.5 Gestione della batteria

La gestione della batteria rappresenta uno dei punti centrali del Safety Layer MicroBot, perché un nodo fisico realmente autonomo non può dipendere soltanto dalla comunicazione, dal firmware e dal controllo degli attuatori: deve anche possedere una strategia chiara per usare, monitorare e proteggere la propria energia. Finché il prototipo è alimentato via USB, il problema energetico è parzialmente semplificato. Il nodo riceve energia stabile da una sorgente esterna e può concentrarsi su comunicazione, LED, firmware, telemetria e primi test. Quando però il MicroBot passa verso una forma più autonoma, alimentata da batteria, il sistema deve affrontare vincoli nuovi: autonomia limitata, cali di tensione, corrente massima disponibile, degrado della batteria, ricarica, protezione da scarica profonda, protezione da sovraccarico, sicurezza termica e priorità dei consumi.

Nel progetto MicroBot, la batteria non deve essere considerata semplicemente come un componente che “dà corrente”. Deve essere considerata come una risorsa dinamica del nodo. La batteria determina quanto a lungo il nodo può restare online, quali azioni può eseguire, quanto spesso può attivare componenti ad alto consumo, quando deve ridurre funzionalità e quando deve entrare in modalità di protezione. In uno swarm, la batteria influenza anche il comportamento collettivo: un nodo con energia bassa non dovrebbe essere assegnato a ruoli critici, non dovrebbe diventare active_connector, non dovrebbe eseguire impulsi magnetici ripetuti e non dovrebbe essere usato come nodo centrale di una struttura se esistono alternative più stabili.

Il primo principio della gestione della batteria è che l'energia disponibile deve influenzare il comportamento del nodo. Un MicroBot non può comportarsi allo stesso modo con batteria piena, batteria media, batteria bassa o batteria critica. Deve esistere uno stato energetico, che può essere espresso in modo semplice come energy_ok, energy_medium, low_energy e critical_energy. Questi stati devono condizionare i comandi accettati, i ruoli assegnabili, la frequenza della telemetria, la luminosità dei LED, l'uso di PWM, l'attivazione delle bobine e la partecipazione ai pattern. La batteria diventa quindi parte del processo decisionale.

Una classificazione iniziale può essere:

energy_ok: il nodo può eseguire funzioni normali entro i limiti di sicurezza;

energy_medium: il nodo funziona normalmente ma il controller può iniziare a preferire ruoli meno costosi;

low_energy: il nodo limita comandi ad alto consumo e preferisce ruoli passivi;

critical_energy: il nodo blocca attuazioni, riduce telemetria e prepara safe_idle o spegnimento controllato.

Questa classificazione è più utile di un semplice valore percentuale, perché collega direttamente l'energia al comportamento. Il controller non deve soltanto sapere che MB-004 ha il 22% di batteria; deve sapere che MB-004 è in low_energy e quindi non deve ricevere comandi magnetici o ruoli attivi. Il firmware deve applicare localmente questa logica anche se il controller non aggiorna subito il piano.

La gestione della batteria può basarsi, nelle prime versioni, su valori stimati. Se il nodo è alimentato via USB, il battery_level può essere simulated o unavailable. Se il nodo usa una batteria ma non ha circuito di misura, il valore può essere stimato manualmente o non disponibile. In versioni successive, il nodo può misurare la tensione della batteria tramite partitore resistivo e ingresso analogico, oppure usare un modulo dedicato di battery management. In ogni caso, la documentazione deve distinguere tra batteria misurata, batteria stimata e batteria simulata.

Una struttura di telemetria energetica può contenere:

battery_source;

battery_voltage_v;

battery_level_estimated_percent;

energy_state;

power_source;

low_energy_warning;

critical_energy_lock;

last_energy_event.

Il campo battery_source può assumere valori come simulated, estimated, measured o unavailable. Se il dato è simulated, serve per testare il comportamento del controller. Se è estimated, può guidare decisioni conservative ma con cautela. Se è measured, può essere usato con maggiore affidabilità. Se è unavailable, il sistema deve adottare una politica prudente, soprattutto se il nodo possiede attuatori ad alto consumo.

La batteria deve essere protetta dalla scarica eccessiva. Una scarica profonda può danneggiare alcune batterie e rendere il sistema instabile. Anche senza entrare nel dettaglio chimico di ogni tecnologia, il principio progettuale è chiaro: il nodo non deve continuare ad assorbire energia in modo aggressivo quando la batteria è sotto soglia. Se energy_state diventa critical_energy, il nodo deve spegnere uscite non essenziali, rifiutare comandi attuativi, ridurre telemetria e segnalare al controller la condizione critica. Se possibile, deve passare a safe_idle. Il controller deve escluderlo dai pattern o assegnargli un ruolo passivo.

Una regola fondamentale è:

se energy_state = critical_energy, il nodo non deve eseguire comandi ad alto assorbimento.

Questa regola deve essere locale. Il firmware non deve aspettare che il controller decida. Se riceve `magnetic_pulse`, `set_pwm_output` o altri comandi energeticamente costosi mentre è in `critical_energy`, deve rifiutarli con un errore strutturato. Il controller può poi adattare il pattern. Ma la protezione primaria è nel nodo.

La gestione della batteria deve distinguere tra carichi leggeri e carichi pesanti. Heartbeat, piccoli LED e comunicazione intermittente sono carichi relativamente leggeri, anche se il Wi-Fi può comunque consumare in modo significativo. Bobine, motori, attuatori e LED ad alta intensità sono carichi più pesanti. In `low_energy`, il nodo può continuare a comunicare ma deve limitare uscite pesanti. In `critical_energy`, può ridurre anche la comunicazione, inviando solo heartbeat essenziale o eventi critici. Questo permette al nodo di restare visibile al controller più a lungo senza consumare inutilmente.

Una politica energetica può essere:

`energy_ok`: tutte le funzioni consentite entro `safety`;

`energy_medium`: nessuna limitazione forte, ma preferenza per ruoli meno costosi;

`low_energy`: bobine e PWM critici bloccati o fortemente limitati;

`critical_energy`: solo heartbeat essenziale, `request_status`, `emergency_stop`, `safe_idle`;

`energy_unknown`: modalità conservativa se il nodo ha attuatori critici.

Il caso `energy_unknown` è importante. Se il sistema non conosce lo stato della batteria, non deve comportarsi come se fosse tutto perfetto. Può continuare in `visual_mode`, ma dovrebbe evitare pattern fisici aggressivi. L'assenza di informazione non deve essere interpretata come sicurezza.

La batteria deve influenzare l'assegnazione dei ruoli. Un nodo con energia alta può essere candidato per ruoli attivi. Un nodo con energia media può essere usato ma non sovraccaricato. Un nodo in `low_energy` deve essere preferibilmente `passive_node`, `observer_node` o `diagnostic_node` leggero. Un nodo in `critical_energy` deve essere escluso dai pattern e mantenuto in `safe_idle`. Questo principio rende lo swarm più robusto: non si affida una funzione critica a un nodo che potrebbe spegnersi o resettarsi durante la run.

Una regola di role assignment può essere:

un nodo in `low_energy` non può essere `active_connector`, `bridge_node` critico o `actuator_node`, salvo modalità di test esplicitamente autorizzata e con limiti severi.

Questa regola è coerente con le sezioni precedenti sui ruoli. Il ruolo non dipende solo dalla geometria, ma anche dallo stato fisico. Un nodo può essere perfetto come posizione, ma inadatto come energia. Il controller deve scegliere non solo "dove sta il nodo", ma anche "quanto è affidabile quel nodo".

La gestione della batteria deve essere collegata ai pattern. Un pattern può richiedere un certo profilo energetico. Una linea LED ha costo basso. Un pattern con bobine ha costo alto. Una struttura complessa con molti connector attivi ha costo ancora maggiore. Prima di avviare una run, il controller dovrebbe eseguire una `energy readiness check`. Questa verifica controlla che i nodi coinvolti abbiano energia sufficiente per il pattern richiesto. Se alcuni nodi non sono pronti, il controller può scegliere una versione ridotta, una modalità visuale o un fallback.

Una `readiness check` energetica può includere:

energy_state dei nodi partecipanti;

numero di nodi in low_energy;

numero di nodi in critical_energy;

carico stimato della run;

attuazioni previste;

cooldown energetici e termici;

possibilità di fallback visuale.

Se il pattern richiede attuazioni pesanti e molti nodi sono in low_energy, la run non dovrebbe partire in modalità fisica. Può partire in simulazione o visual_demo. Questo è un comportamento intelligente e sicuro.

La gestione della batteria deve anche considerare i picchi di corrente. Una batteria può avere energia residua sufficiente ma non essere in grado di sostenere un picco elevato. Questo è importante quando si usano bobine o motori. Anche se il livello percentuale è alto, un impulso troppo intenso può causare calo di tensione e reset del microcontrollore. Per prevenire questo, il sistema deve limitare il numero di carichi pesanti attivi contemporaneamente e ridurre duty cycle se l'alimentazione non è robusta. Questo collega gestione della batteria e protezione energetica.

Una regola può essere:

la capacità energetica residua non basta; bisogna considerare anche la corrente istantanea richiesta.

Per questo motivo, il controller deve evitare attivazioni simultanee e il nodo deve rifiutare comandi che superano il profilo energetico sicuro. Se non esiste misura di corrente reale, bisogna usare limiti conservativi. Se in futuro verrà aggiunto un sensore di corrente o un modulo di power management, il sistema potrà diventare più preciso.

La batteria deve essere collegata al monitoraggio della temperatura. Scariche intense possono produrre calore. Componenti caldi possono peggiorare efficienza e sicurezza. Una batteria in ambiente caldo può essere più delicata. Quindi energy_state e thermal_state devono essere valutati insieme. Un nodo in energy_ok ma thermal_warning deve limitare attuatori. Un nodo in low_energy ma thermal_ok deve comunque limitare attuatori. Un nodo in low_energy e thermal_warning deve essere fortemente degradato. Quando più segnali di rischio si sommano, la risposta deve essere più conservativa.

Una politica combinata può essere:

energy_ok + thermal_ok: operatività normale;

energy_ok + thermal_warning: riduzione termica;

low_energy + thermal_ok: riduzione energetica;

low_energy + thermal_warning: modalità conservativa severa;

critical_energy o thermal_lock: blocco attuazioni;

critical_energy + thermal_lock: safety/idle immediato.

Questa matrice rende il comportamento più robusto. Non si considera mai una sola variabile isolata.

La gestione della batteria deve prevedere eventi energetici. Quando il nodo passa da energy_ok a energy_medium, da energy_medium a low_energy o da low_energy a critical_energy, deve inviare un evento al controller. Questi eventi devono essere prioritari rispetto alla telemetria ordinaria, perché possono cambiare ruoli, pattern e safety. Il controller deve reagire aggiornando dashboard, log e action_plan. Se un nodo critico entra in low_energy durante una run, il sistema deve decidere se continuare, riassegnare ruolo, rallentare o abortire.

Un evento energetico può contenere:

event_type = energy_state_changed;

node_id;

previous_energy_state;

new_energy_state;

battery_level_estimated_percent;

battery_voltage_v se disponibile;

timestamp;

affected_capabilities.

Il campo affected_capabilities è utile perché low_energy può disabilitare coil_pwm, pwm_output o led_high_brightness. Il controller deve sapere quali funzioni restano disponibili.

La batteria deve essere visibile nella dashboard. Ogni nodo dovrebbe mostrare power_source, energy_state, battery_level se disponibile, battery_voltage se misurata, low_energy warning, critical_energy lock e funzioni disabilitate. Questo è essenziale per capire perché un nodo non partecipa a un pattern. Se MB-005 è in passive_node perché ha low_energy, la dashboard deve mostrarlo. Senza visibilità, l'utente potrebbe pensare che il sistema abbia assegnato male i ruoli.

Una scheda energetica può mostrare:

power_source;

battery_source;

battery_level;

battery_voltage;

energy_state;

disabled_outputs;

estimated_runtime se disponibile;

last_energy_event;

role_limitations.

Questo rende la batteria parte del runtime osservabile, non un dettaglio nascosto.

La gestione della batteria deve essere registrata nei log. Ogni cambio di stato energetico, ogni comando rifiutato per low_energy, ogni fallback causato da energia insufficiente e ogni modifica di ruolo dovuta alla batteria deve essere salvata. Questo permette analisi post-run. Se un pattern fallisce spesso perché i nodi entrano in low_energy, bisogna ridurre il carico, migliorare alimentazione o cambiare batteria. Senza log energetico, il problema può sembrare casuale.

Un log energetico può contenere:

node_id;

timestamp;

energy_state;

battery_level;

battery_voltage;

event_type;

command_rejected se presente;

role_change se presente;

pattern_impact.

Per esempio:

MB-004 entra in low_energy durante phase_attach;

controller rimuove MB-004 da active_connector;

pattern passa a partial_success.

Questo tipo di log collega energia e comportamento, che è esattamente ciò che serve in uno swarm fisico.

La gestione della batteria deve distinguere tra alimentazione via USB, alimentazione da power bank, batteria integrata e alimentazione da banco. Ogni sorgente ha caratteristiche diverse. USB da computer può essere comoda per debug ma può avere limiti. Un power bank può fornire più autonomia ma può avere comportamento automatico di spegnimento se il carico è basso. Una batteria integrata richiede protezioni di carica e scarica. Un alimentatore da banco è utile per test controllati ma non rappresenta autonomia reale. Il nodo deve possibilmente dichiarare power_source, o almeno il sistema deve documentare in quale modalità è stato testato.

Una classificazione può essere:

power_source = USB_DEBUG;

power_source = POWER_BANK;

power_source = BATTERY_INTERNAL;

power_source = BENCH_SUPPLY;

power_source = SIMULATED.

Questo campo è utile nei log e nei test. Una run fatta via USB_DEBUG non prova autonomia a batteria. Una run fatta con BATTERY_INTERNAL è più vicina al comportamento autonomo. La documentazione deve essere chiara.

La gestione della batteria deve anche considerare la ricarica, anche se non viene implementata subito. Un MicroBot autonomo dovrà avere una strategia per ricaricare o sostituire la batteria. Nelle prime versioni, si può ricaricare manualmente o usare moduli protetti già pronti. In versioni più avanzate, si potrebbe immaginare docking, ricarica induttiva o gestione centralizzata. Tuttavia, prima di arrivare a questo, il sistema deve almeno dichiarare che la ricarica è fuori dal comportamento operativo automatico iniziale e che durante la carica gli attuatori critici potrebbero essere disabilitati.

Una regola prudente può essere:

durante charging_state, bloccare attuazioni ad alto consumo salvo modalità test controllata.

Questo evita combinazioni non sicure tra ricarica e carico elevato. La ricarica introduce vincoli propri e non deve essere mescolata ingenuamente con pattern attivi.

La batteria deve influenzare la frequenza della telemetria. In energy_ok, il nodo può inviare telemetria normale. In low_energy, può ridurre la frequenza e inviare solo dati essenziali. In critical_energy, può inviare solo heartbeat minimo o evento critico. Questo riduce consumo. La telemetria è utile, ma non deve svuotare ulteriormente un nodo in stato critico. Il controller può adattare telemetry_level in base a energy_state.

Una politica può essere:

energy_ok: telemetry_normal;

energy_medium: telemetry_normal o reduced se swarm grande;

low_energy: telemetry_reduced;

critical_energy: telemetry_minimal;

diagnostic_on_request solo se energia sufficiente.

Questa logica verrà ripresa nella Sezione 13, ma è già parte della gestione batteria. La telemetria non è gratis: consuma energia, soprattutto con comunicazione wireless.

La gestione della batteria deve essere collegata alla modalità sleep o low-power futura. Nelle prime versioni ESP32, il nodo probabilmente resterà sempre attivo durante i test. In futuro, però, per aumentare autonomia, si potranno introdurre modalità low-power. Un nodo potrebbe ridurre frequenza di heartbeat, spegnere LED, sospendere sensori o entrare in sleep tra finestre temporali. Questa direzione è interessante ma deve essere introdotta gradualmente, perché il sonno del nodo può complicare comunicazione e sincronizzazione.

Una roadmap low-power può essere:

v0.1: nodo sempre attivo per debug;

v0.2: LED ridotti in low_energy;

v0.3: telemetria ridotta;

v0.4: disattivazione sensori non essenziali;

v0.5: finestre di ascolto programmate;

v0.6: modalità sleep controllata;

v0.7: wake-up per pattern o emergenze se supportato.

La modalità sleep deve essere compatibile con il Safety Layer. Un nodo che dorme non deve mantenere attuatori attivi. Prima di entrare in sleep, deve spegnere uscite critiche, salvare stato e informare il controller. Questo è fondamentale.

La gestione della batteria deve prevedere un safe shutdown logico. Se energy_state diventa critical_energy, il nodo può non spegnersi fisicamente subito, ma può entrare in safe_idle, disattivare uscite e ridurre comunicazione. Se la batteria continua a scendere, in futuro potrebbe spegnersi o richiedere intervento umano. Il safe shutdown deve essere ordinato: prima blocco attuatori, poi notifica, poi riduzione funzioni. Non deve semplicemente collassare con un reset improvviso, se possibile.

Una sequenza può essere:

critical_energy rilevato;

disattivazione uscite non essenziali;

cancellazione comandi pendenti;

evento energy_critical al controller;

mode = safe_idle;

telemetry_minimal;

attesa ricarica/sostituzione/ripristino.

Questa sequenza protegge sia hardware sia coerenza del sistema. Il controller sa che il nodo non è disponibile e può adattare i pattern.

La batteria deve anche essere considerata nei test di durata. Un test di pochi secondi dimostra comunicazione, ma non autonomia. Per valutare un nodo a batteria, bisogna misurare quanto resta attivo in idle, quanto consuma con LED, quanto con Wi-Fi, quanto con telemetria e quanto con attuatori. Anche se i primi valori sono approssimativi, documentarli aiuta moltissimo. Permette di capire quali funzioni consumano di più e dove ottimizzare.

Una tabella di test autonomia può includere:

test_id;

hardware_version;

battery_type;
power_source;
mode;
telemetry_frequency;
LED_state;
actuation_profile;
runtime_minutes;
energy_state_end;
notes.

Questo tipo di dati sarà importante se il progetto deve evolvere verso prototipi autonomi credibili.

La gestione della batteria deve considerare la differenza tra autonomia individuale e autonomia dello swarm. Un singolo nodo può avere batteria sufficiente, ma lo swarm nel suo insieme può essere limitato dal nodo più debole se quel nodo ha ruolo critico. Se una struttura dipende da un bridge_node con batteria bassa, tutta la run è fragile. Il controller deve quindi valutare non solo l'energia media dello swarm, ma anche l'energia dei nodi critici. Questo è molto importante nelle strutture complesse.

Una metrica utile è:

minimum_energy_of_critical_nodes

Se questa metrica è bassa, il pattern è a rischio anche se molti nodi hanno energia alta. Il controller può riassegnare ruoli prima di iniziare. Questo è un esempio di pianificazione energetica intelligente.

La gestione della batteria deve supportare fallback. Se un nodo va in low_energy durante una run, il pattern può degradare. Una linea può ridursi. Un cerchio può diventare arco. Un ponte può usare una catena più corta o dichiarare partial_success. Una struttura fisica può passare a visual_mode. Il fallback energetico deve essere definito come parte della adaptation_policy del pattern. Non deve essere improvvisato.

Esempi:

low_energy su nodo optional → continua con warning;

low_energy su nodo attivo → riassegna ruolo;

critical_energy su nodo critico → abort fase o fallback;

energy_unknown su nodi attuativi → visual_mode;

swarm_energy_low → riduci pattern o passa a simulazione.

Questa politica rende la batteria parte della logica dei pattern, non solo un dato di telemetria.

La gestione della batteria deve essere progettata con margini. Non bisogna aspettare che la batteria sia quasi completamente scarica per ridurre carico. Le soglie devono essere conservative. Una soglia `low_energy` deve attivarsi prima della zona pericolosa. Una soglia `critical_energy` deve lasciare ancora abbastanza energia per comunicare l'errore e spegnere in modo ordinato. Se il nodo aspetta troppo, potrebbe resettarsi prima di inviare eventi. Quindi le soglie devono essere pensate non solo per proteggere la batteria, ma anche per mantenere controllo.

Una regola è:

`critical_energy` deve essere rilevato abbastanza presto da permettere `safe_idle` e comunicazione finale.

Questo è un punto importante. Un sistema che rileva la criticità solo quando il nodo si spegne non ha davvero gestione batteria.

La gestione della batteria deve essere legata alla protezione energetica, che verrà approfondita nel punto successivo. La batteria descrive la risorsa; la protezione energetica descrive le politiche per usarla senza superare limiti. In altre parole, battery management osserva stato, tensione, livello e fonte di alimentazione; energy protection decide cosa bloccare, ridurre, distribuire o degradare. I due livelli sono collegati ma non identici.

Si può dire:

gestione della batteria = monitorare e interpretare la risorsa energetica del nodo;

protezione energetica = applicare limiti operativi in base allo stato energetico.

Questa distinzione aiuta a mantenere ordinata la Sezione 11.

La gestione della batteria deve essere coerente con la visione futura del MicroBot come piattaforma modulare. Se i nodi devono diventare piccoli, autonomi e coordinati, l'energia sarà uno dei vincoli principali. La comunicazione wireless, i sensori, gli attuatori e il calcolo consumano. Ogni capacità aggiunta aumenta il carico. Per questo, la progettazione futura dovrà bilanciare potenza, autonomia, dimensione, peso e sicurezza. Non basta aggiungere componenti; bisogna sapere se la batteria può sostenerli.

Questo punto è importante anche per la scalabilità. Con pochi nodi, si possono usare alimentazioni esterne o batterie grandi. Con molti nodi, il costo energetico complessivo diventa più rilevante. Lo swarm deve saper funzionare con nodi a energia diversa, ricariche diverse e stati diversi. La gestione batteria è quindi una base della scalabilità fisica.

In conclusione, la gestione della batteria è il sistema attraverso cui MicroBot osserva, interpreta e protegge la risorsa energetica dei nodi fisici. Essa introduce stati come `energy_ok`, `energy_medium`, `low_energy` e `critical_energy`; distingue dati simulati, stimati e misurati; limita comandi ad alto consumo; influenza ruoli, pattern, telemetria, modalità operative e fallback; protegge da scarica eccessiva, picchi di corrente e reset improvvisi; rende visibile nella dashboard lo stato energetico e registra nei log ogni evento significativo. In un MicroBot autonomo, la batteria non è un dettaglio tecnico, ma una variabile di controllo. Un nodo con energia bassa non deve essere trattato come un nodo pienamente operativo. Questo principio prepara direttamente il punto successivo, dedicato alla protezione energetica, dove la gestione della batteria viene trasformata in regole operative per limitare consumi, distribuire carichi, prevenire picchi e mantenere lo swarm funzionante in modo sicuro e sostenibile.

11.6 Protezione energetica

La protezione energetica rappresenta il livello del Safety Layer che trasforma la gestione della batteria e il controllo della corrente in regole operative concrete. Se la gestione della batteria serve a osservare lo stato energetico del nodo, la protezione energetica serve a decidere che cosa il nodo può o non può fare in base a quello stato. Questa distinzione è fondamentale: sapere che un MicroBot ha poca energia non basta; il sistema deve anche modificare comportamento, ruoli, pattern, telemetria, attuazioni e priorità in modo coerente. La protezione energetica è quindi il meccanismo che impedisce al sistema di consumare più energia di quanta sia sicuro consumare.

Nel progetto MicroBot, la protezione energetica deve essere considerata una funzione distribuita. Il controller può pianificare il consumo dello swarm, ma ogni nodo deve proteggere sé stesso localmente. Questo significa che il controller non dovrebbe assegnare un ruolo attivo a un nodo in `low_energy`, ma anche che il nodo deve rifiutare il comando se il controller sbaglia. La protezione energetica non può dipendere da un solo punto del sistema. Deve esistere nel firmware del nodo, nel runtime del controller, nei pattern, nella dashboard, nella telemetria e nei log. Solo così il sistema può restare sicuro anche in presenza di errori, ritardi, pacchetti persi o condizioni impreviste.

Il principio generale della protezione energetica può essere espresso così:

ogni azione deve essere compatibile con l'energia disponibile, con il carico richiesto e con il margine di sicurezza del nodo.

Questa frase contiene tre elementi. Il primo è l'energia disponibile, cioè lo stato della batteria o della sorgente di alimentazione. Il secondo è il carico richiesto, cioè quanto consuma l'azione proposta. Il terzo è il margine di sicurezza, cioè la distanza tra il funzionamento attuale e una condizione critica. Un comando può essere accettabile quando il nodo è alimentato via USB, ma non quando è a batteria bassa. Può essere accettabile se eseguito una volta, ma non se ripetuto. Può essere accettabile per un nodo in idle, ma non per un nodo già caldo o impegnato in altri compiti. La protezione energetica deve valutare il contesto, non solo il comando isolato.

La prima funzione della protezione energetica è la classificazione dei consumi. Non tutti i comandi hanno lo stesso costo. Un `request_status` consuma poco. Un `heartbeat` leggero consuma poco. Un LED a bassa intensità consuma moderatamente. Una trasmissione Wi-Fi frequente consuma di più. Una bobina o un attuatore PWM può consumare molto. Il sistema deve quindi associare ogni comando a una classe energetica. Questa classificazione permette al firmware e al controller di decidere quali comandi consentire in base allo stato energetico.

Una classificazione semplice può essere:

classe energetica minima: `heartbeat`, `ping`, `request_status`, `emergency_stop`;

classe energetica bassa: LED leggero, `ack`, telemetria essenziale;

classe energetica media: LED pattern complesso, telemetria frequente, diagnostica estesa;

classe energetica alta: PWM, bobine, attuatori, sensori energivori, pattern fisici;

classe energetica critica: attuazioni ripetute, impulsi magnetici intensi, sequenze multi-nodo ad alto carico.

Questa classificazione deve essere usata per applicare policy. In `energy_ok`, il nodo può accettare molte classi, sempre entro limiti di sicurezza. In `low_energy`, deve accettare solo classi minime e

basse. In `critical_energy`, deve accettare soltanto comandi essenziali, come `emergency_stop`, `safe_idle`, `request_status` minimo e forse `heartbeat`. Questa logica protegge il nodo da consumi non necessari.

La seconda funzione è il blocco dei comandi energeticamente incompatibili. Se il nodo è in `low_energy`, un comando `magnetic_pulse` deve essere rifiutato. Se è in `critical_energy`, anche comandi LED intensi o telemetria debug possono essere rifiutati. Se `power_source` è incerta, i comandi ad alto consumo devono essere limitati. Questo blocco deve essere locale e immediato. Il nodo non deve eseguire il comando e poi segnalare di essere in difficoltà. Deve rifiutarlo prima dell'esecuzione.

Una validazione energetica minima può essere:

verifica `energy_state`;

verifica classe energetica del comando;

verifica carico stimato;

verifica limiti di batteria o alimentazione;

verifica eventuali cooldown energetici;

accetta solo se il comando rientra nella policy corrente.

Se la validazione fallisce, il nodo deve inviare un errore strutturato. Per esempio:

```
result = rejected
reason = energy_limit
energy_state = low_energy
command_class = high_energy
allowed_max_class = low_energy
```

Questa risposta permette al controller di adattare il piano. Può riassegnare il ruolo, ridurre il pattern o passare a `visual_mode`. Il rifiuto non è un malfunzionamento: è un comportamento corretto del Safety Layer.

La terza funzione è la riduzione automatica del profilo operativo. Quando l'energia diminuisce, il nodo deve ridurre gradualmente le funzioni non essenziali. Non deve passare immediatamente da pienamente attivo a spento, salvo casi critici. La degradazione energetica deve essere progressiva. Prima si riduce luminosità LED. Poi si riduce frequenza della telemetria. Poi si bloccano attuatori ad alto consumo. Poi si passa a ruoli passivi. Poi si entra in `safe_idle`. Questa progressione permette al sistema di continuare a osservare e controllare il nodo più a lungo.

Una scala possibile è:

`energy_ok`: piena operatività entro limiti standard;

`energy_medium`: riduzione preventiva di carichi non necessari;

`low_energy`: blocco attuatori ad alto consumo, LED ridotti, telemetria ridotta;

`critical_energy`: solo funzioni essenziali, `safe_idle`, no pattern attivi;

energy_shutdown_pending: preparazione a spegnimento o intervento manuale.

Questa scala è utile perché collega energia e modalità operativa. Il nodo non è semplicemente “acceso” o “spento”; può trovarsi in stati intermedi. La dashboard deve mostrare questi stati, e il controller deve usarli nella pianificazione.

La quarta funzione è la protezione contro i picchi. Una batteria o una sorgente di alimentazione può sostenere un consumo medio, ma non necessariamente un picco improvviso. I picchi possono causare cali di tensione, reset del microcontrollore, instabilità Wi-Fi, errori di sensori o comportamento non deterministico. Le bobine e gli attuatori sono i principali responsabili, ma anche l’attivazione simultanea di più moduli può generare problemi. La protezione energetica deve quindi impedire che troppi carichi elevati vengano attivati nello stesso momento.

A livello di singolo nodo, il firmware può imporre:

max_active_outputs;

max_instantaneous_load_class;

no_simultaneous_high_energy_outputs;

minimum_delay_between_high_load_actions.

A livello di swarm, il controller può imporre:

max_high_energy_nodes_per_slot;

max_total_estimated_current_per_phase;

actuation_groups;

staggered_activation;

energy-aware slot scheduling.

Questo significa che la protezione energetica non è soltanto locale. Un nodo può essere sicuro da solo, ma lo swarm può produrre picchi se molti nodi agiscono insieme. Il controller deve pianificare il carico globale. Questa è una funzione fondamentale quando si passa da uno o due prototipi a molti MicroBot.

La quinta funzione è la pianificazione energetica dei pattern. Un pattern non deve essere valutato solo in base alla geometria e alla topologia, ma anche in base al costo energetico previsto. Una linea visuale con LED ha un costo basso. Una catena magnetica con attach ripetuti ha un costo alto. Una struttura complessa con molti connector attivi ha un costo ancora più alto. Prima di avviare una run, il controller deve stimare se i nodi possono sostenerla. Se non possono, deve scegliere una variante più leggera.

Una pattern_energy_check può includere:

numero di nodi coinvolti;

numero di attuazioni previste;

classe energetica media della run;

nodì critici e loro energy_state;

carico stimato per fase;

eventuali retry previsti;

cooldown necessari;

possibilità di fallback.

Se la verifica fallisce, il controller può bloccare la run fisica e proporre una run simulata o visuale. Questo è un comportamento corretto: il sistema non deve forzare hardware non pronto. Una simulazione può continuare anche quando l'hardware fisico viene escluso. In questo modo, lo sviluppo non si ferma, ma resta sicuro.

La sesta funzione è la priorità energetica tra comandi. In condizioni di energia limitata, il nodo deve sapere quali comandi sono più importanti. Emergency_stop ha sempre priorità massima. Safe_idle è essenziale. Heartbeat minimo è importante per restare visibile al controller. Telemetria debug è sacrificabile. LED decorativi sono sacrificabili. Attuazioni fisiche sono sacrificabili, salvo casi speciali definiti e sicuri. Questa gerarchia permette al nodo di usare poca energia per restare controllabile.

Una gerarchia di priorità può essere:

priorità 0: emergency_stop, disattivazione uscite critiche;

priorità 1: safe_idle, safety_state update;

priorità 2: heartbeat minimo, energy event;

priorità 3: request_status essenziale;

priorità 4: LED di stato a basso consumo;

priorità 5: telemetria normale;

priorità 6: pattern visuali;

priorità 7: attuazioni ad alto consumo.

Quando energy_state scende, il nodo disabilita progressivamente le priorità più basse. Questo rende la degradazione prevedibile. Il controller può sapere che in critical_energy non deve aspettarsi telemetria dettagliata o pattern LED complessi, ma deve ancora ricevere, se possibile, un evento essenziale.

La settima funzione è la gestione della telemetria energeticamente efficiente. La telemetria è necessaria, ma consuma energia. In uno swarm grande, inviare dati troppo frequentemente può aumentare carico wireless e consumo. La protezione energetica deve regolare telemetry_level. In energy_ok, il nodo può inviare dati normali. In low_energy, deve inviare solo dati essenziali. In critical_energy, deve inviare evento critico e poi ridurre al minimo. Questo evita che la diagnostica stessa diventi una causa di consumo eccessivo.

Una politica può essere:

telemetry_full: usata solo in diagnostic o test;

telemetry_normal: usata in pattern standard;

telemetry_reduced: usata in low_energy o swarm numerosi;

telemetry_minimal: usata in critical_energy;

telemetry_on_request: usata per evitare invii periodici non necessari.

Questa logica sarà approfondita nella Sezione 13, ma fa parte della protezione energetica perché decide quanta informazione inviare in base al costo. Un nodo non deve consumare energia per inviare dati inutili quando sta cercando di sopravvivere.

L'ottava funzione è la gestione dei LED come carico energetico controllato. Nei primi prototipi, i LED sono fondamentali per visualizzare stato e ruoli. Tuttavia, se si usano molti LED o LED RGB, il consumo non è nullo. La protezione energetica può ridurre brightness, frequenza di lampeggio o numero di LED attivi in low_energy. In critical_energy, può lasciare solo un segnale minimo o spegnere tutto tranne un indicatore essenziale. Questo mantiene la visibilità senza sprechi.

Una policy LED può essere:

energy_ok: brightness normale;

energy_medium: brightness leggermente ridotto;

low_energy: brightness basso e pattern semplici;

critical_energy: solo segnale essenziale o LED spento;

safety: LED rosso a basso duty se energia sufficiente.

Questa scelta ha anche valore dimostrativo: il nodo mostra fisicamente che sta risparmiando energia. La dashboard può rispecchiare lo stesso stato.

La nona funzione è la protezione contro il brownout o reset da caduta di tensione. Se un nodo richiede troppa corrente, la tensione può scendere e il microcontrollore può resettarsi. Questo è pericoloso perché un reset improvviso può far perdere stato, sync, role, thermal_load stimato e comandi pendenti. La protezione energetica deve quindi evitare carichi che possano causare cadute di tensione. Se il firmware o l'hardware rilevano brownout o reset frequenti, il controller deve marcare il nodo come unstable e ridurre il suo ruolo.

Una policy può essere:

se un nodo mostra reboot ripetuti durante attuazioni, disabilitare attuazioni ad alto consumo per quel nodo;

se boot_id cambia durante una run, invalidare sync e ruoli;

se reboot avviene dopo comando ad alto carico, registrare possible_energy_fault;

se energy_fault ricorre, mettere nodo in diagnostic_mode.

Questo collega protezione energetica e allineamento tra dispositivi. Un nodo che si resetta non è più allineato, anche se torna online. Deve essere recuperato con `state_sync` e non reinserito immediatamente in pattern critici.

La decima funzione è la definizione di un energy budget per ogni run. Un energy budget è una stima del consumo massimo ammesso per una run o una fase. Per esempio, una fase attach può avere un massimo numero di impulsi, una durata massima totale di PWM e un massimo carico stimato. Se il pattern supera il budget, viene rallentato, degradato o interrotto. Questo impedisce che un pattern consumi energia indefinitamente nel tentativo di correggere errori.

Un energy budget può includere:

`max_high_energy_commands;`

`max_total_pulse_time_ms;`

`max_retry_count;`

`max_active_nodes_per_slot;`

`max_estimated_energy_cost;`

`max_phase_duration;`

`fallback_if_budget_exceeded.`

Quando il budget viene superato, il controller non deve continuare come se nulla fosse. Deve dichiarare `energy_budget_exceeded` e applicare fallback. Questo rende il comportamento prevedibile e sicuro.

L'undicesima funzione è l'integrazione della protezione energetica con la correzione degli errori. Come già visto, correggere errori può costare energia. Se una connessione non si forma, il controller può ritentare. Ma ogni tentativo consuma. La protezione energetica deve impedire che la correzione diventi una fonte di scarica eccessiva. Il retry deve avere un costo e un limite. Se il costo supera il budget, il sistema deve passare a fallback.

Una regola fondamentale è:

ogni retry fisico deve essere autorizzato dal budget energetico e dallo stato energetico del nodo.

Questo principio impedisce loop di correzione non sostenibili. Un sistema ben progettato sa quando smettere di insistere. Se una connessione non riesce entro i tentativi sicuri, la run può diventare `partial_success` o fallback, ma non deve consumare tutta la batteria.

La dodicesima funzione è la scelta energetica dei ruoli. Come già anticipato, il controller deve assegnare ruoli tenendo conto dell'energia. Un nodo con `energy_ok` può essere leader, connector o actuator. Un nodo con `energy_medium` può ancora partecipare, ma forse non dovrebbe essere il più caricato. Un nodo in `low_energy` deve diventare `passive_node`, `observer` o `diagnostic` leggero. Un nodo in `critical_energy` deve essere escluso dalla struttura attiva. Questo consente al sistema di distribuire il consumo.

Una strategia avanzata può usare uno score:

`energy_score = funzione(battery_level, energy_state, thermal_state, recent_load, power_source)`

Il controller può preferire nodi con `energy_score` alto per ruoli critici. Questo rende lo swarm più robusto. Se si assegna sempre lo stesso nodo come `bridge_node`, si consumerà prima. Se invece si ruotano i ruoli in base all'energia, il carico si distribuisce meglio.

La tredicesima funzione è la protezione energetica del controller o del sistema centrale, se applicabile. Nei primi prototipi, il controller sarà probabilmente un computer o server locale, quindi il problema è minore. Tuttavia, in versioni embedded o mobili, anche il controller potrebbe avere limiti energetici. La documentazione può limitarsi a notare che la protezione energetica è principalmente per i nodi, ma in architetture future potrà estendersi a gateway, controller locali o cluster leader. Questo sarà importante per la scalabilità.

La quattordicesima funzione è la distinzione tra energia reale e energia simulata. Nei nodi simulati, `energy_state` può essere generato artificialmente per testare policy. Questo è utile: si può simulare un nodo in `low_energy` e verificare se il controller riassegna ruoli. Si può simulare `energy_budget_exceeded` e vedere se il pattern degrada. Questi test possono essere fatti senza hardware reale. Quando si passa all'hardware, gli stessi stati possono essere alimentati da misure o stime reali. La logica resta uguale.

Una simulazione energetica può testare:

nodo che entra in `low_energy` durante una linea;

`active_connector` che va in `critical_energy` durante un ponte;

swarm con energia media bassa;

pattern che supera `energy_budget`;

fallback automatico a `visual_mode`;

telemetria ridotta in `low_energy`.

Questi scenari rendono il Safety Layer verificabile prima della parte fisica completa. È un vantaggio enorme.

La quindicesima funzione è la dashboard energetica. La protezione energetica deve essere visibile. Non basta sapere internamente che un comando è stato rifiutato. La dashboard deve mostrare `energy_state`, comandi bloccati, limiti correnti, budget della run, consumo stimato, `power_source`, nodi in `low_energy` e nodi esclusi. Questo permette all'utente di capire perché il sistema ha ridotto o modificato il pattern.

Una dashboard energetica può mostrare:

energia per nodo;

stato energetico dello swarm;

nodi critici con energia bassa;

budget della run;

comandi ad alto consumo previsti;

comandi bloccati;

modalità energetica corrente;

fallback energetico attivo.

Questa vista è molto utile per presentare il progetto in modo serio. Mostra che il sistema non invia semplicemente comandi, ma gestisce risorse.

La sedicesima funzione è il logging energetico. Ogni decisione energetica deve essere registrata. Se un comando viene rifiutato per `low_energy`, va registrato. Se il controller cambia ruolo per ridurre consumo, va registrato. Se un pattern passa a `visual_mode` per `energy_budget_exceeded`, va registrato. Se un nodo entra in `critical_energy`, va registrato. Questi log permettono di analizzare l'efficienza del sistema.

Un evento di protezione energetica può contenere:

`event_type`;

`node_id`;

`energy_state`;

`command_id` se presente;

`reason`;

`action_taken`;

`previous_mode`;

`new_mode`;

`pattern_run_id`;

`timestamp`.

Per esempio:

```
event_type = energy_protection_triggered;  
node_id = MB-004;  
reason = low_energy_blocks_high_energy_command;  
action_taken = reject_magnetic_pulse;  
pattern_run_id = bridge_run_003.
```

Questo tipo di log collega direttamente energia e comportamento.

La diciassettesima funzione è la generazione di report energetici post-run. Alla fine di una run, il sistema dovrebbe indicare se il pattern è stato sostenibile. Un pattern può essere geometricamente riuscito ma energeticamente inefficiente. Se ha consumato troppo, richiesto troppi retry o portato molti nodi in `low_energy`, va segnalato. Questo permette di migliorare pattern e hardware. La qualità di una run non è solo forma finale, ma anche costo.

Un report energetico può includere:

numero di comandi ad alto consumo;

nodi più sollecitati;

eventi low_energy;

eventi critical_energy;

comandi rifiutati per energia;

budget previsto;

budget usato;

fallback energetici;

raccomandazioni.

Questo report può diventare parte della documentazione sperimentale. Se un pattern è troppo costoso, si può ottimizzare.

La diciottesima funzione è la protezione energetica come criterio di successo. Un pattern non dovrebbe essere classificato come success_clean se ha portato nodi in critical_energy. Può essere success_with_energy_warnings o partial_success_energy_limited. Questo rende i risultati più onesti. La sicurezza energetica è parte della validazione, non un dettaglio separato.

Una classificazione possibile è:

success_clean: pattern riuscito senza warning energetici;

success_with_energy_warnings: pattern riuscito ma con low_energy o budget alto;

partial_success_energy_limited: pattern degradato per protezione energetica;

failed_energy_budget_exceeded: pattern fallito per consumo eccessivo;

aborted_energy_safety: run interrotta per critical_energy o rischio energetico.

Questa classificazione sarà molto utile nei report e nella dashboard.

La diciannovesima funzione è la progettazione conservativa dei limiti. Come per la temperatura, le soglie energetiche devono includere margine. Non bisogna aspettare che la batteria sia quasi morta o che la tensione sia già instabile. Le soglie low_energy e critical_energy devono attivarsi abbastanza presto da permettere al nodo di spegnere uscite e comunicare lo stato. Se il nodo si resetta prima di notificare, la protezione è arrivata troppo tardi.

Una regola è:

critical_energy deve lasciare energia sufficiente per safe_idle e comunicazione dell'evento.

Questo principio è importantissimo. La protezione energetica deve anticipare, non inseguire il collasso.

La ventesima funzione è la compatibilità con il watchdog timer. Se un nodo si blocca mentre consuma energia, il watchdog può resettarlo. Tuttavia, il reset non deve portare subito a piena operatività. Dopo un watchdog reset, il nodo deve dichiarare boot_id nuovo, entrare in stato conservativo e attendere state_sync. Se il reset è avvenuto durante una fase ad alto consumo, il

controller deve sospettare `energy_fault` o `thermal_fault`. Questo evita che il nodo venga reinserito subito in un ruolo critico.

Una policy post-reset può essere:

disattivare uscite all'avvio;

entrare in `safe_idle`;

inviare boot event;

attendere sync e status check;

non accettare comandi ad alto consumo finché il controller non completa recovery;

se reset ripetuti, passare a `diagnostic_mode`.

Questa policy collega protezione energetica, watchdog e ripristino automatico.

La ventunesima funzione è la protezione energetica in caso di comunicazione persa. Se il nodo non riceve più segnali dal controller, deve ridurre consumo. Non deve continuare pattern o attuazioni. Deve spegnere uscite ad alto consumo e restare in heartbeat minimo o `safe_idle`. Questo protegge la batteria e l'hardware. Il `controller_lost` non è solo un problema comunicativo; è anche un problema energetico, perché il nodo non deve consumare energia in un comportamento non supervisionato.

Una regola è:

se `controller_lost`, disattivare carichi non essenziali e passare a `safe_idle`.

Questa regola è semplice ma fondamentale. In uno swarm fisico, l'assenza del controller deve portare a conservazione, non a continuazione cieca.

La ventiduesima funzione è la differenza tra sviluppo e operatività. Durante lo sviluppo via USB, alcune limitazioni energetiche possono sembrare meno importanti. Tuttavia, devono essere implementate comunque in forma logica, perché preparano la versione a batteria. Anche se `battery_source = unavailable` o `USB_DEBUG`, il sistema può testare `energy_state` simulati. Questo permette di verificare policy prima dell'autonomia reale. Non bisogna aspettare la batteria fisica per progettare la protezione energetica.

Una modalità di sviluppo può usare:

`energy_state` simulato;

battery events generati manualmente;

dashboard energetica;

test di rifiuto comandi;

fallback visuale;

log energetici.

Questo rende la protezione energetica parte del software fin dall'inizio. Quando arriverà l'hardware, la logica sarà già pronta.

La ventitreesima funzione è la relazione con la scalabilità. Con pochi nodi, il consumo è relativamente facile da osservare. Con molti nodi, il consumo globale diventa più difficile. Lo swarm deve evitare che tutti i nodi consumino molto contemporaneamente. Deve bilanciare carico, ruoli e attuazioni. La protezione energetica diventa una funzione di coordinazione globale. Non basta sapere che ogni nodo è protetto; bisogna sapere che l'intero swarm è energeticamente sostenibile.

Una metrica globale può essere:

swarm_energy_state;

number_of_low_energy_nodes;

critical_nodes_energy_min;

estimated_total_load_per_phase;

energy_balance_score.

Queste metriche possono guidare il controller. Se troppi nodi sono in low_energy, il pattern deve essere ridotto. Se i nodi critici sono deboli, i ruoli vanno riassegnati. Se il carico totale previsto è alto, il pattern va rallentato.

La protezione energetica deve anche avere una roadmap incrementale. Nella prima versione, si possono simulare energy_state e rifiuti. Nella seconda, si integra la dashboard. Nella terza, si aggiunge battery_voltage se disponibile. Nella quarta, si implementano limiti reali su PWM e bobine. Nella quinta, si usa energy budget per pattern. Nella sesta, si ottimizza role assignment in base all'energia. Nella settima, si introducono modalità low-power vere.

Una roadmap può essere:

v0.1: energy_state simulato;

v0.2: comandi rifiutati in low_energy simulato;

v0.3: dashboard energetica;

v0.4: log energetici;

v0.5: battery_voltage misurata se hardware disponibile;

v0.6: limiti energetici su PWM e bobine;

v0.7: energy budget per run;

v0.8: role assignment energy-aware;

v0.9: telemetria adattiva;

v1.0: modalità low-power e scheduling energetico avanzato.

Questa progressione è coerente con il resto del progetto. Non richiede subito hardware complesso, ma prepara la struttura.

La protezione energetica deve infine essere vista come una forma di intelligenza operativa. Un sistema che sa quando ridurre, quando fermarsi, quando cambiare ruolo e quando passare a modalità conservativa è più maturo di un sistema che esegue sempre tutto finché si spegne. La protezione energetica non limita il progetto; lo rende più credibile. Nel contesto MicroBot, dove l'obiettivo finale è uno swarm modulare, estensibile e potenzialmente fisico, l'energia sarà sempre una risorsa critica. Gestirla bene significa rendere possibile la scalabilità.

In conclusione, la protezione energetica è il livello del Safety Layer che applica regole operative basate sull'energia disponibile, sul carico richiesto e sul margine di sicurezza. Essa classifica i comandi per costo energetico, blocca attuazioni incompatibili, riduce profili operativi, previene picchi, pianifica i pattern in base al budget energetico, regola telemetria e LED, influenza i ruoli, protegge da brownout, limita retry, registra eventi e produce report post-run. La batteria fornisce la risorsa; la protezione energetica decide come usarla senza mettere a rischio il nodo. Questo punto prepara direttamente il successivo, dedicato al watchdog timer, perché anche con limiti energetici e termici ben progettati il firmware può bloccarsi, resettarsi o entrare in stati non previsti: il watchdog serve a riportare il nodo verso una condizione controllata quando il software non risponde più correttamente.

11.7 Watchdog timer

Il watchdog timer rappresenta il livello del Safety Layer progettato per proteggere il nodo MicroBot da blocchi del firmware, loop non controllati, ritardi eccessivi, deadlock logici, mancata esecuzione del ciclo principale o condizioni in cui il software non riesce più a garantire il controllo sicuro delle uscite. Dopo aver definito controllo della corrente, protezione delle bobine, monitoraggio della temperatura, prevenzione del surriscaldamento, gestione della batteria e protezione energetica, diventa necessario introdurre un meccanismo capace di intervenire quando il firmware stesso smette di comportarsi correttamente. Il watchdog è quindi una protezione contro il fallimento del controllo software.

In un MicroBot fisico, il firmware ha responsabilità critiche: ricevere comandi, validare pacchetti, aggiornare lo stato locale, gestire LED, inviare heartbeat, controllare timer, spegnere uscite dopo `duration_ms`, rispettare cooldown, controllare `thermal_state`, verificare `energy_state` e rispondere a `emergency_stop`. Se il firmware si blocca, tutte queste funzioni possono interrompersi. Il problema diventa particolarmente grave quando il nodo sta controllando una bobina, un'uscita PWM, un driver o un altro componente ad alto consumo. In quel caso, un blocco software potrebbe impedire lo spegnimento tempestivo dell'uscita. Il watchdog timer serve a ridurre questo rischio riportando il microcontrollore a uno stato noto quando il ciclo firmware non risponde più.

Il principio generale può essere espresso così:

se il firmware non dimostra periodicamente di essere vivo e funzionante, il sistema deve forzare un reset o una transizione verso uno stato sicuro.

Questa frase è centrale. Il watchdog non controlla direttamente se il pattern è corretto, se la geometria è giusta o se il nodo ha il ruolo desiderato. Controlla una cosa più fondamentale: il firmware sta ancora girando? Il loop principale viene ancora eseguito? Il nodo riesce ancora a nutrire il watchdog entro il tempo previsto? Se la risposta è no, il sistema deve assumere che il controllo locale sia compromesso e deve intervenire.

Il watchdog timer può essere hardware, software o combinato. Un watchdog hardware è integrato nel microcontrollore o in un circuito esterno e forza un reset se non viene aggiornato entro un certo intervallo. Un watchdog software è una logica implementata nel firmware che controlla tempi, task,

heartbeat interni e stati. Nel contesto MicroBot, l'approccio più solido è usare entrambi i concetti: il microcontrollore deve avere un watchdog reale per recuperare da blocchi gravi, mentre il firmware deve avere controlli logici per rilevare condizioni anomale prima del reset.

Il watchdog hardware protegge da blocchi profondi, per esempio loop infiniti, deadlock, task bloccati, errori di memoria, funzioni che non ritornano o condizioni in cui il loop principale non viene più eseguito. Il watchdog software protegge da anomalie più sottili: un task che non aggiorna la telemetria, un comando rimasto pending troppo a lungo, una bobina che dovrebbe essere spenta ma risulta ancora active, una fase locale che supera `max_duration`, un heartbeat interno non aggiornato. I due livelli sono complementari. Il watchdog hardware è l'ultima barriera; il watchdog software è una diagnosi più raffinata.

Nel firmware MicroBot, il watchdog non deve essere “nutrito” in modo cieco. Se il codice chiama continuamente la funzione di reset del watchdog anche quando alcuni sottosistemi sono bloccati, il watchdog perde significato. La logica corretta è che il watchdog venga aggiornato solo se il ciclo principale ha completato controlli essenziali. Per esempio, il firmware dovrebbe verificare ricezione comandi, timer uscite, safety state, gestione cooldown e aggiornamento dello stato. Solo dopo questi controlli può indicare al watchdog che il nodo è ancora sano. Questo evita di mantenere vivo un firmware che in realtà non sta più controllando bene il nodo.

Una logica concettuale può essere:

esegui ciclo principale;

controlla pacchetti in ingresso;

aggiorna timer delle uscite;

verifica safety;

verifica energia e temperatura;

aggiorna heartbeat interno;

se tutti i controlli critici sono stati eseguiti, aggiorna il watchdog.

Questo principio è molto importante. Il watchdog non deve essere trattato come una formalità tecnica, ma come una prova periodica che il nodo ha ancora capacità di controllo. Se un sottosistema critico non viene aggiornato, il watchdog non dovrebbe essere alimentato normalmente, oppure il firmware dovrebbe entrare in `safe_idle` prima che il watchdog resettì il dispositivo.

Il watchdog timer deve essere progettato tenendo conto del tempo massimo accettabile di blocco. Se il timeout del watchdog è troppo lungo, il nodo può restare in uno stato pericoloso per troppo tempo. Se è troppo corto, può resettare il nodo anche durante operazioni normali leggermente lente. La scelta del timeout deve quindi bilanciare reattività e stabilità. Per un nodo che controlla solo LED, il timeout può essere meno critico. Per un nodo che controlla bobine o attuatori, il timeout deve essere abbastanza breve da ridurre il rischio, ma non così breve da causare reset continui durante Wi-Fi, parsing o operazioni normali.

Il watchdog timeout deve essere collegato ai limiti delle uscite. Per esempio, se una bobina ha `max_pulse_duration_ms` molto breve, il firmware deve essere in grado di spegnerla localmente prima che il watchdog intervenga. Il watchdog non deve essere il meccanismo normale di

spegnimento della bobina. La bobina deve spegnersi tramite timer firmware. Il watchdog serve se quel timer non viene più eseguito. Quindi il watchdog è una protezione di emergenza, non il controllo ordinario.

Una regola fondamentale è:

le uscite critiche devono essere auto-terminate dal firmware; il watchdog serve come protezione ulteriore se il firmware si blocca.

Questo evita una progettazione sbagliata. Non bisogna dire: “Se la bobina resta accesa, tanto il watchdog resetta”. No. Il firmware deve spegnerla correttamente. Il watchdog interviene solo se qualcosa va storto.

Il watchdog timer deve interagire con lo stato sicuro all’avvio. Dopo un reset causato dal watchdog, il nodo riparte. All’avvio, tutte le uscite critiche devono essere impostate immediatamente in stato OFF. Questo è essenziale perché un reset deve riportare il nodo in condizione sicura, non in uno stato ambiguo. Il setup del firmware deve quindi impostare GPIO, PWM e driver in configurazione sicura prima di connettersi al Wi-Fi, prima di avviare pattern e prima di accettare comandi.

La sequenza corretta dopo reset è:

- inizializzazione del microcontrollore;
- impostazione immediata delle uscite critiche su OFF;
- inizializzazione stato interno in `safe_idle` o `booting`;
- generazione o aggiornamento `boot_id`;
- connessione alla rete;
- invio `boot_event` o `heartbeat` al controller;
- attesa di `sync` e `state_sync`;
- riabilitazione graduale delle funzioni solo dopo verifica.

Questa sequenza impedisce che il nodo, dopo un reset, rientri automaticamente in un pattern attivo senza essere riallineato. Un watchdog reset significa che qualcosa è andato storto. Il controller deve saperlo e deve trattare il nodo come non allineato finché non viene verificato.

Il `boot_id` è quindi fondamentale. Se il watchdog resetta il nodo, il `boot_id` cambia. Il controller rileva che lo stesso `node_id` è tornato online con un nuovo `boot_id`. A quel punto deve invalidare `sync`, ruoli, comandi pendenti e stato precedente. Il nodo non deve essere considerato ancora parte della stessa fase operativa. Deve passare attraverso `discovery`, `request_status`, `sync` e `state_sync` prima di rientrare nella run, se il rientro è consentito.

Una regola precisa è:

se `boot_id` cambia durante una run, il nodo deve essere marcato come `resync_required` e non deve eseguire comandi temporizzati o attuativi finché non viene riallineato.

Questa regola collega watchdog, sincronizzazione e safety. Il reset non è invisibile. È un evento del sistema.

Il watchdog timer deve generare eventi diagnosticabili. Dopo un reset, il nodo dovrebbe poter indicare, se possibile, che la causa del riavvio è stata watchdog_reset. Molti microcontrollori permettono di leggere la causa del reset. Se disponibile, questa informazione va inviata al controller. Se non disponibile, il controller può comunque dedurre un reset osservando boot_id cambiato e uptime azzerato. In ogni caso, il log deve registrare che il nodo si è riavviato durante una certa run o fase.

Un evento di boot può contenere:

node_id;

boot_id;

reset_reason;

uptime_ms = 0 o valore iniziale;

firmware_version;

hardware_version;

previous_state_unknown;

safe_mode_enabled.

Il campo reset_reason può essere watchdog_reset, power_on_reset, brownout_reset, manual_reset, software_reset o unknown. Questa informazione è molto utile per distinguere problemi. Un watchdog_reset indica blocco software. Un brownout_reset può indicare problema energetico. Un power_on_reset può indicare accensione normale. Un reset unknown richiede cautela.

Il watchdog timer deve essere collegato alla protezione energetica. Un reset improvviso può essere causato non solo da un bug software, ma anche da un calo di tensione. Se il nodo si resetta durante attivazioni ad alto consumo, il controller deve sospettare un energy_fault o brownout. In quel caso, non basta riavviare e continuare. Bisogna ridurre carichi, bloccare attuatori, passare a diagnostic_mode o verificare alimentazione. Il watchdog e i reset sono quindi anche segnali indiretti di problemi energetici.

Una policy può essere:

se reset durante fase ad alto consumo, marcare possible_energy_fault;

se reset ripetuti nello stesso nodo, escludere da ruoli attivi;

se reset ripetuti nello swarm, ridurre carico globale o controllare alimentazione;

se reset dopo comando bobina, bloccare coil su quel nodo finché non viene diagnosticato.

Questa logica evita di ignorare reset ricorrenti. Un nodo che si resetta spesso non è affidabile per pattern critici.

Il watchdog timer deve essere collegato alla protezione termica. Se il nodo si resetta mentre thermal_load stimato era alto, bisogna essere prudenti. Il reset cancella la RAM, ma non cancella il calore fisico. Dopo watchdog_reset, il firmware potrebbe perdere la stima del thermal_load. Per questo motivo, dopo un reset, un nodo con bobine o attuatori dovrebbe entrare in thermal_recovery

conservativa per un tempo minimo, oppure richiedere verifica termica se esiste un sensore. Questo evita che una bobina calda venga riattivata subito dopo un reset.

Una regola prudente è:

dopo `watchdog_reset`, bloccare attuatori termicamente critici per un `recovery_time` iniziale.

Questa regola può sembrare severa, ma è sensata. Se il reset è avvenuto durante un comportamento anomalo, non si può assumere che l'hardware sia freddo e pronto. Serve una pausa di sicurezza.

Il watchdog timer deve proteggere anche da loop bloccanti. Uno degli errori più comuni nei firmware iniziali è usare delay lunghi per gestire LED, test o sequenze. Un delay lungo può impedire al nodo di ricevere `emergency_stop`, controllare timeout, inviare heartbeat o spegnere uscite. In un sistema MicroBot, questa pratica è incompatibile con il Safety Layer. Il firmware deve essere non bloccante, soprattutto se gestisce attuatori. Il watchdog può rilevare blocchi estremi, ma il design corretto è evitare funzioni che bloccano il loop.

Una regola di firmware è:

nessuna operazione ordinaria deve impedire il controllo periodico di safety, timer e comunicazione.

Questo include LED pattern, parsing, Wi-Fi, telemetria, diagnostica e attuazioni. Ogni operazione deve essere spezzata in step temporizzati. Il loop deve restare reattivo.

Il watchdog timer può essere integrato con un task monitor interno. Se il firmware è organizzato in più task o moduli logici, ciascuno può aggiornare un proprio timestamp interno. Il loop principale o un supervisor controlla che ogni task critico sia aggiornato. Per esempio: `communication_task`, `safety_task`, `output_task`, `telemetry_task`. Se `communication_task` non aggiorna da troppo tempo, il nodo può entrare in degraded. Se `output_task` non aggiorna, è più grave perché potrebbe non spegnere uscite. Se `safety_task` non aggiorna, il nodo deve andare in safe mode o lasciare scattare il watchdog.

Una tabella interna può essere:

`communication_task_last_ok;`

`safety_task_last_ok;`

`output_task_last_ok;`

`telemetry_task_last_ok;`

`sync_task_last_ok.`

Il watchdog hardware viene alimentato solo se i task critici sono ok. Questo è più maturo rispetto a chiamare semplicemente `watchdog_reset` a ogni loop. In una prima versione, si può implementare una logica più semplice; nelle versioni avanzate, questa struttura aumenta affidabilità.

Il watchdog timer deve essere coerente con l'`emergency_stop`. `Emergency_stop` deve avere priorità superiore a qualsiasi operazione. Se il firmware è vivo, deve eseguirlo immediatamente. Se il firmware è bloccato e non può riceverlo, il watchdog deve intervenire resettando. Dopo reset, le uscite devono essere OFF. Questo non sostituisce l'`emergency_stop`, ma crea una protezione aggiuntiva. L'obiettivo è che anche se il comando di emergenza non viene processato per blocco firmware, il sistema non resti indefinitamente in stato attivo.

La catena di sicurezza è quindi:

firmware vivo → emergency_stop spegne subito;

firmware bloccato → watchdog reset;

dopo reset → uscite OFF;

controller rileva boot_id cambiato → nodo in resync_required/safe_idle.

Questa catena deve essere chiarissima nella progettazione. L'emergenza non deve dipendere da un singolo livello.

Il watchdog timer deve essere considerato anche dal punto di vista del controller. Il controller può avere un proprio watchdog logico. Se il controller non riceve heartbeat da un nodo, marca il nodo stale o offline. Se non riceve risposte da più nodi, può sospendere pattern. Se il processo controller stesso si blocca, il sistema dovrebbe avere un modo per riavviare o quantomeno lasciare i nodi in safe_idle grazie ai loro controller_lost timeout locali. Questo significa che il Safety Layer non riguarda solo i nodi, ma anche il software centrale.

Nel contesto attuale, il watchdog del controller può essere inizialmente logico: heartbeat monitor, timeout delle run, controllo dei thread/processi, log di errore. In versioni più mature, se il controller gira su un dispositivo embedded, può avere un vero watchdog di sistema. Ma già ora si può definire il principio:

se il controller non aggiorna lo stato della run entro un certo tempo, i nodi devono degradare verso safe_idle.

Questo protegge lo swarm da un controller bloccato. I nodi non devono continuare attuazioni solo perché il controller non ha detto di fermarsi. Devono avere timeout locali.

Il watchdog timer deve quindi essere collegato al controller_lost timeout. Questo timeout non è identico al watchdog hardware. Il watchdog hardware controlla se il firmware locale è vivo. Il controller_lost controlla se il nodo vede ancora il controller. Se il controller non comunica per un tempo troppo lungo, il nodo non deve eseguire pattern attivi. Deve disattivare carichi critici e passare a safe_idle. Questo è un watchdog comunicativo.

Una regola è:

se $\text{now} - \text{last_controller_seen} > \text{controller_lost_timeout}$, entrare in safe_idle e bloccare attuazioni.

Questa regola protegge da crash del controller, rete persa o computer spento. È una componente fondamentale per hardware reale.

Il watchdog timer deve essere visibile nella dashboard. La dashboard dovrebbe mostrare per ogni nodo uptime, boot_id, reset_reason, watchdog_status, last_heartbeat, controller_lost_timer e numero di reset recenti. Se un nodo si resetta spesso, deve essere evidente. Un nodo che appare online ma cambia boot_id frequentemente è instabile. La dashboard deve segnalarlo come unstable, non trattarlo come sano.

Una scheda watchdog può mostrare:

uptime;

boot_id;

last_reset_reason;

watchdog_enabled;

last_watchdog_reset_time;

reset_count_recent;

controller_lost_timeout;

last_controller_seen;

node_stability_status.

Questi dati aiutano moltissimo nel debug. Se un nodo fallisce pattern perché si resetta, lo si vede subito. Senza questa visibilità, il problema potrebbe sembrare comunicazione o ruolo, mentre in realtà è reset firmware o energia.

Il watchdog timer deve essere registrato nei log. Ogni reset deve diventare un evento. Il log deve indicare quando il nodo è sparito, quando è tornato, quale boot_id aveva prima e dopo, quale fase era in corso, quali comandi erano pendenti e quale reset_reason è stato dichiarato. Questo permette analisi post-run. Se un reset avviene sempre durante phase_attach, il problema può essere carico energetico o bug in quella fase. Se avviene durante telemetria debug, può essere sovraccarico software. Se avviene casualmente, serve indagine più ampia.

Un evento di reset può contenere:

event_type = node_reset_detected;

node_id;

old_boot_id;

new_boot_id;

reset_reason;

pattern_run_id;

phase_id_at_last_seen;

last_command_sent;

time_offline_ms;

recovery_action.

Questo log collega watchdog e comportamento dello swarm. Un reset non è solo evento locale; può influenzare pattern, ruoli e safety.

Il watchdog timer deve influenzare il risultato della run. Se un nodo si resetta durante una run, il pattern non dovrebbe essere classificato come success_clean. Può essere partial_success, success_with_recovery o failed, a seconda del ruolo del nodo e dell'impatto. Se il nodo era optional

e il sistema ha recuperato, si può continuare. Se era critical, la run può fallire o passare a fallback. Questo rende i risultati più onesti.

Una classificazione possibile è:

success_clean: nessun reset o watchdog event;

success_with_recovery: reset gestito senza perdita del pattern;

partial_success_watchdog_recovery: pattern degradato dopo reset;

failed_watchdog_reset: reset di nodo critico non recuperabile;

aborted_safety_after_watchdog: run interrotta dopo reset in fase critica.

Questa classificazione aiuta a distinguere tra pattern riuscito in condizioni stabili e pattern riuscito nonostante problemi.

Il watchdog timer deve essere testato esplicitamente. Non basta abilitarlo. Bisogna verificare che funzioni. I test devono controllare che il nodo resettasse se il loop viene bloccato, che le uscite siano spente dopo reset, che il boot_id cambi, che il controller rilevi il reset, che il nodo non rientri automaticamente in un pattern attivo, che state_sync sia richiesto e che comandi attuativi siano bloccati durante recovery. Questi test sono fondamentali per validare il Safety Layer.

Una suite di test può includere:

firmware_loop_block_test;

watchdog_reset_detection_test;

outputs_off_after_reset_test;

boot_id_change_test;

controller_detects_reset_test;

no_auto_rejoin_after_reset_test;

thermal_recovery_after_watchdog_test;

energy_fault_after_repeated_reset_test;

controller_lost_safe_idle_test.

Questi test possono essere simulati inizialmente. Per esempio, il firmware può avere una modalità diagnostic che forza un loop bloccante controllato o simula watchdog_reset. In hardware reale, i test devono essere eseguiti senza carichi pericolosi o con carichi sicuri.

Il watchdog timer deve essere progettato in modo da non causare reset inutili. Un watchdog troppo aggressivo può creare instabilità. Se il nodo resettasse spesso durante normali operazioni Wi-Fi o parsing, il timeout è troppo breve o il firmware è troppo bloccante. La soluzione non è semplicemente disattivare il watchdog, ma correggere firmware e scegliere timeout corretto. Il watchdog deve essere abbastanza severo da rilevare blocchi reali, ma abbastanza tollerante da non interferire con operazioni normali.

Una buona pratica è:

misurare il tempo massimo normale del loop e impostare il watchdog con margine, mantenendo comunque un limite compatibile con la sicurezza delle uscite.

Se un'operazione richiede molto tempo, va spezzata. Non bisogna aumentare indefinitamente il timeout per compensare codice bloccante. Un firmware MicroBot deve essere progettato per restare reattivo.

Il watchdog timer deve anche evitare falsi segnali di salute. Se il firmware alimenta il watchdog ma non aggiorna output timers, il nodo può restare in uno stato non sicuro senza reset. Per questo, il watchdog deve essere alimentato dopo i controlli critici. Una variabile come `safety_loop_ok` può essere richiesta prima di resettare il watchdog. Se `safety_loop_ok` è falsa, il nodo deve entrare in `safe_idle` o lasciar scattare il reset.

Una logica più sicura è:

`safety_ok = output_timer_checked AND emergency_stop_checked AND thermal_checked AND energy_checked`

se `safety_ok` è true, alimenta watchdog;

se `safety_ok` è false, spegni uscite e non alimentare watchdog oppure entra in `safety`.

Questo rende il watchdog parte del Safety Layer reale, non solo un timer di sistema.

Il watchdog timer deve essere compatibile con il ripristino automatico, che sarà il punto successivo. Il watchdog individua o risolve un blocco tramite reset; il ripristino automatico definisce cosa succede dopo. Il reset da solo non basta. Dopo il reset bisogna ristabilire identità, safe state, comunicazione, sync, ruoli, capabilities e stato operativo. Se il nodo rientra senza processo di recovery, il watchdog può creare nuovi problemi. Per questo, watchdog e ripristino automatico devono essere progettati insieme.

La relazione è:

watchdog timer = rileva blocco e forza ritorno a boot/safe state;

ripristino automatico = ricostruisce in modo controllato lo stato del nodo dopo errore o reset.

Questa distinzione prepara la sezione successiva.

Il watchdog timer deve essere integrato con la documentazione hardware e firmware. Ogni versione del nodo dovrebbe indicare se il watchdog è abilitato, quale timeout usa, quali uscite vengono spente all'avvio, come viene gestito `reset_reason`, quali eventi vengono inviati al controller e quali funzioni sono bloccate durante recovery. Senza questa documentazione, sarebbe difficile valutare la sicurezza del firmware.

Una scheda firmware può includere:

`watchdog_enabled;`

`watchdog_timeout_ms;`

`reset_reason_supported;`

safe_outputs_on_boot;
controller_lost_timeout_ms;
post_reset_recovery_mode;
high_energy_outputs_blocked_after_reset;
diagnostic_watchdog_test_available.

Questi dati rendono il nodo verificabile.

Il watchdog timer deve essere considerato anche nella simulazione software. Il simulatore può simulare un nodo che si blocca, smette di inviare heartbeat, cambia boot_id o rientra dopo reset. Questo permette di testare il comportamento del controller senza hardware reale. La simulazione può generare eventi come simulated_watchdog_reset o node_firmware_hang. Il controller deve reagire come farebbe con un nodo reale: marcarlo offline, poi resync_required, poi recovery o esclusione.

Scenari simulati utili sono:

nodo smette di inviare heartbeat durante pattern;
nodo rientra con nuovo boot_id;
nodo rientra durante fase critica;
nodo resetta ripetutamente;
nodo resetta dopo comando high_energy;
controller perde tutti i nodi per crash simulato.

Questi scenari aiutano a validare la resilienza del runtime.

Il watchdog timer deve proteggere anche la coerenza dello swarm. Un nodo che si resetta perde contesto: ruolo, fase, comandi pendenti, sync, state machine, thermal_load stimato, energy state se non persistente. Se rientra come se nulla fosse, può creare disallineamento. Quindi il controller deve trattare ogni watchdog_reset come evento che rompe la continuità operativa del nodo. Il nodo può essere ancora fisicamente lo stesso, ma logicamente è una nuova sessione. Il boot_id serve proprio a esprimere questa frattura.

Una regola di coerenza è:

dopo reset, il nodo non appartiene più automaticamente alla run corrente.

Può essere reintegrato solo tramite recovery. Se la fase corrente è critica, il controller può decidere di non reintegrarlo fino alla run successiva. Questo è particolarmente importante per pattern con bobine o connessioni fisiche.

Il watchdog timer deve contribuire alla sicurezza ma non sostituire buone pratiche. Un sistema pieno di blocchi che viene continuamente resettato dal watchdog non è un sistema sano. Il watchdog è una barriera di protezione, non una soluzione a bug strutturali. Se reset avvengono spesso, bisogna

correggere firmware, alimentazione, timing, memoria, parsing o carico. I log di watchdog devono essere usati per migliorare il sistema, non ignorati.

Una regola metodologica è:

ogni watchdog_reset deve essere trattato come evento significativo da analizzare.

In fase di sviluppo, un reset può essere accettabile come test. In fase dimostrativa o operativa, reset frequenti indicano instabilità. Questo deve essere chiaro nella documentazione.

Il watchdog timer deve essere legato alla memoria degli errori. Se possibile, il nodo può conservare un piccolo contatore di reset o ultimo errore in memoria non volatile o inviarlo subito al controller. Anche se non viene implementato subito, il concetto è utile. Un nodo che si resetta dieci volte in un'ora deve essere marcato unreliable. Il controller può mantenere questa informazione anche se il nodo perde memoria locale. Questo permette di evitare che un nodo instabile venga continuamente reintegrato.

Una metrica può essere:

reset_count_recent;

watchdog_reset_count;

brownout_reset_count;

mean_time_between_resets;

stability_score.

Il role assignment può usare stability_score. Un nodo con reset frequenti non deve diventare leader o bridge_node critico. Questo collega watchdog e assegnazione dei ruoli.

Il watchdog timer deve infine essere visto come elemento di maturità del firmware embedded. Un progetto che controlla attuatori fisici deve prevedere cosa succede se il codice si blocca. Non basta scrivere firmware che funziona quando tutto va bene. Bisogna progettare il comportamento quando qualcosa va male. Il watchdog è uno dei meccanismi principali per questo. Nel contesto MicroBot, esso garantisce che ogni nodo abbia una possibilità di tornare a uno stato sicuro anche quando il software locale perde controllo.

In conclusione, il watchdog timer è il meccanismo del Safety Layer che protegge il MicroBot da blocchi del firmware, loop infiniti, task non aggiornati, mancata gestione dei timer e condizioni in cui il nodo non riesce più a garantire controllo sicuro delle uscite. Esso deve essere integrato con firmware non bloccante, timer locali, stato sicuro all'avvio, boot_id, reset_reason, controller_lost timeout, log, dashboard, recovery e riassegnazione dei ruoli. Il watchdog non deve sostituire la logica ordinaria di sicurezza, ma deve intervenire quando quella logica non viene più eseguita. Dopo un watchdog reset, il nodo non deve rientrare automaticamente in una run attiva: deve essere considerato da riallineare, verificare e reintegrare con prudenza. Il punto successivo, dedicato al ripristino automatico, completerà la Sezione 11 spiegando come il sistema deve recuperare dopo reset, safety event, perdita di comunicazione, thermal lock, energy fault o errore operativo, riportando MicroBot verso uno stato sicuro, tracciabile e coerente.

11.8 Ripristino automatico

Il ripristino automatico rappresenta il punto conclusivo della Sezione 11 e descrive il modo in cui il sistema MicroBot deve tornare a uno stato sicuro, coerente e controllabile dopo un errore, un reset, una perdita di comunicazione, un blocco termico, un problema energetico, un comando rifiutato, un watchdog reset o una condizione di safety. Dopo aver definito corrente, bobine, temperatura, surriscaldamento, batteria, protezione energetica e watchdog timer, è necessario chiudere il Safety Layer con una domanda fondamentale: cosa succede dopo che il sistema ha rilevato un problema?

Il ripristino automatico non significa che il sistema debba ripartire sempre da solo come se nulla fosse accaduto. Questo sarebbe pericoloso. Significa invece che MicroBot deve possedere una procedura ordinata per uscire da stati anomali, ricostruire il proprio contesto operativo, comunicare l'errore, riportarsi in una modalità sicura e, solo se le condizioni lo permettono, rientrare gradualmente nel comportamento normale. Il ripristino automatico non è quindi un “riavvio cieco”, ma una sequenza controllata di recovery.

Il principio generale può essere espresso così:

dopo un errore, il sistema deve prima tornare sicuro, poi tornare osservabile, poi tornare sincronizzato, e solo alla fine tornare operativo.

Questa sequenza è fondamentale. La priorità non è riprendere subito il pattern. La priorità è impedire danni, rendere il nodo nuovamente visibile al controller, ricostruire stato e sincronizzazione, verificare le condizioni fisiche, e soltanto dopo decidere se reintegrare il nodo nella run. Un MicroBot che si resetta, entra in `thermal_lock` o perde comunicazione non deve rientrare automaticamente come `active_connector`, `leader` o `bridge_node`. Deve prima passare attraverso una procedura di verifica.

Il ripristino automatico può essere necessario in diversi casi. Il primo caso è il watchdog reset, quando il firmware si è bloccato e il microcontrollore è stato riavviato. Il secondo è il brownout o reset energetico, quando il nodo si riavvia a causa di caduta di tensione o alimentazione instabile. Il terzo è la perdita del controller, quando il nodo non riceve più comandi o heartbeat dal sistema centrale. Il quarto è il `thermal_lock`, quando una bobina, un driver o un componente ha raggiunto una condizione termica non sicura. Il quinto è `critical_energy`, quando la batteria o l'alimentazione non permettono più operazioni normali. Il sesto è `safety_stop` o `emergency_stop`, quando il sistema viene interrotto per protezione. Il settimo è un errore di ruolo, configurazione o fase, quando il nodo risulta fuori contesto rispetto alla run corrente.

Tutti questi casi non devono essere trattati allo stesso modo. Un watchdog reset richiede ricostruzione del `boot_id`, dello stato e della sincronizzazione. Un `thermal_lock` richiede raffreddamento e blocco attuatori. Un `critical_energy` richiede riduzione drastica delle funzioni. Un `role_mismatch` richiede `state_sync` o riassegnazione. Una perdita del controller richiede `safe_idle` e attesa di riconnessione. Il ripristino automatico deve quindi essere contestuale: prima identifica la causa, poi applica la procedura corretta.

Una classificazione iniziale può essere:

`reset_recovery`: recupero dopo riavvio del nodo;

`communication_recovery`: recupero dopo perdita di controller o rete;

thermal_recovery: recupero dopo warning o lock termico;

energy_recovery: recupero dopo low_energy o critical_energy;

safety_recovery: recupero dopo emergency_stop o safety_stop;

state_recovery: recupero dopo fase, ruolo o configurazione incoerente;

pattern_recovery: recupero della run o fallback del pattern.

Questa classificazione è importante perché evita una risposta generica. “Ripristinare” non significa sempre fare la stessa cosa. A volte significa resync. A volte significa aspettare cooldown. A volte significa escludere il nodo. A volte significa abortire la run. A volte significa continuare in modalità degradata.

La prima fase del ripristino automatico è sempre la messa in sicurezza. Quando si verifica un errore critico, il nodo deve disattivare tutte le uscite ad alto rischio: bobine, PWM, driver, eventuali motori o attuatori. I LED possono restare accesi solo se servono a segnalare stato e se l’energia lo permette. I comandi pendenti devono essere cancellati o sospesi. La macchina a stati deve entrare in safe_idle, safety, thermal_recovery, energy_recovery o diagnostic_mode a seconda del caso. Questa fase deve avvenire localmente, senza aspettare il controller.

Una sequenza minima di messa in sicurezza può essere:

spegnere uscite critiche;

azzerare PWM;

cancellare magnetic_pulse pendenti;

bloccare nuovi comandi attuativi;

registrare stato di errore locale;

mantenere solo comunicazione essenziale;

inviare evento al controller se possibile.

Questa sequenza protegge il nodo anche se il controller non è disponibile. È coerente con tutta la filosofia del Safety Layer: la sicurezza locale ha priorità assoluta.

La seconda fase è il recupero dell’osservabilità. Dopo essersi messo in sicurezza, il nodo deve tornare visibile al controller. Questo può avvenire tramite heartbeat, boot_event, error_event o recovery_event. L’obiettivo è far sapere al controller: “sono vivo, sono in stato sicuro, ho avuto questo problema, queste funzioni sono disponibili, queste invece sono bloccate”. Senza questa comunicazione, il controller non può decidere come procedere.

Un recovery_event può contenere:

node_id;

boot_id;

recovery_type;

previous_error;

current_mode;

safety_state;

energy_state;

thermal_state;

available_capabilities;

blocked_capabilities;

timestamp;

requires_state_sync.

Per esempio, se un nodo rientra dopo watchdog reset, può inviare:

recovery_type = reset_recovery;
reset_reason = watchdog_reset;
mode = safe_idle;
requires_state_sync = true;
blocked_capabilities = coil_pwm fino a verifica.

Se invece il nodo esce da thermal_lock, può inviare:

recovery_type = thermal_recovery;
thermal_state = thermal_recovery;
coil_commands_enabled = false;
cooldown_remaining_ms = valore residuo.

Questi messaggi rendono il ripristino leggibile e tracciabile.

La terza fase è la ricostruzione dell'identità di sessione. Il node_id resta stabile, ma dopo un reset cambia il boot_id. Questo significa che il controller deve capire che lo stesso nodo fisico è tornato online in una nuova sessione firmware. Tutti i comandi pendenti associati al vecchio boot_id devono essere invalidati. Il sync precedente non è più valido. I ruoli precedenti non devono essere considerati automaticamente attivi. Le fasi precedenti devono essere riconfermate. Questa regola è essenziale per evitare che un nodo riavviato rientri in una run con uno stato vecchio o incompleto.

Una regola rigorosa è:

nuovo boot_id = nuova sessione operativa del nodo.

Da questa regola derivano conseguenze precise:

invalidare offset temporale precedente;

invalidare comandi temporizzati pendenti;

invalidare ruoli attivi non riconfermati;

richiedere request_status;

richiedere sync;

richiedere state_sync;

non autorizzare attuazioni critiche fino a recovery completata.

Questo rende il reset sicuro. Il nodo torna online, ma non viene immediatamente trattato come se fosse sempre rimasto allineato.

La quarta fase è il recupero della sincronizzazione. Dopo reset, perdita di comunicazione o stato out_of_phase, il nodo deve riallinearsi al controller. Deve ricevere sync, aggiornare offset, verificare timebase, recuperare run_id e phase_id se necessario. Fino a quando sync_status non torna almeno sync_ok, il nodo non deve eseguire comandi temporizzati critici. Può ricevere request_status, set_idle, diagnostic, clear_error o emergency_stop, ma non magnetic_pulse sincronizzati o azioni di pattern precise.

Una procedura di sync recovery può essere:

controller rileva nodo in recovery;

invia sync_request;

nodo risponde con tempo locale e boot_id;

controller aggiorna offset;

controller verifica RTT e jitter;

se qualità sufficiente, sync_status = sync_ok;

altrimenti sync_status = sync_degraded e nodo resta escluso da fasi temporizzate critiche.

Questa procedura collega ripristino automatico e Sezione 9. Il tempo condiviso deve essere ricostruito prima dell'azione condivisa.

La quinta fase è il recupero dello stato operativo. Un nodo deve sapere in quale modalità si trova: idle, diagnostic, safety, thermal_recovery, energy_recovery o active. Dopo un errore, la modalità iniziale deve essere conservativa. Il controller può poi inviare state_sync per riportare il nodo a uno stato coerente. State_sync può includere run_id, phase_id, config_epoch, role, group_id, timebase e policy. Tuttavia, il nodo deve accettare state_sync solo se le proprie condizioni locali lo permettono.

Una regola importante è:

state_sync riallinea il contesto, ma non annulla safety, thermal_lock o critical_energy.

Se il nodo è ancora in thermal_lock, anche un state_sync valido non deve riabilitare la bobina. Se è in critical_energy, non deve diventare active_connector. Se è in safety, deve restare protetto finché la condizione non viene risolta. Il controller può riallineare la logica, ma non può forzare la fisica.

La sesta fase è il recupero delle capabilities. Dopo un errore, alcune capacità possono essere temporaneamente disabilitate. Per esempio, dopo thermal_lock la capability coil_pwm può essere presente hardware-wise, ma operationally blocked. Dopo low_energy, PWM e LED ad alta luminosità possono essere disabilitati. Dopo watchdog reset, le bobine possono restare bloccate per

recovery_time. Quindi il nodo deve distinguere tra capabilities installate e capabilities attualmente disponibili.

Una distinzione utile è:

hardware_capabilities = capacità fisicamente presenti;

available_capabilities = capacità attualmente utilizzabili;

blocked_capabilities = capacità presenti ma temporaneamente bloccate.

Per esempio:

hardware_capabilities: coil_pwm, led_status, heartbeat;

available_capabilities: led_status, heartbeat;

blocked_capabilities: coil_pwm per thermal_recovery.

Questa distinzione è molto importante. Il controller non deve assegnare un ruolo attivo solo perché il nodo “ha” una bobina, se quella bobina è bloccata per sicurezza.

La settima fase è la decisione di reintegrazione. Dopo che il nodo è tornato osservabile, sincronizzato e coerente, il controller deve decidere se reintegrarlo nella run corrente, tenerlo in idle, assegnarlo a un ruolo passivo, spostarlo in diagnostic_mode o escluderlo fino alla prossima run. Questa decisione dipende dalla fase corrente, dal ruolo precedente, dalla criticità del pattern, dal tipo di errore e dallo stato fisico del nodo.

Una policy può essere:

se la run è in fase non critica e il nodo è recuperato, reintegro possibile;

se la run è in phase_attach o altra fase critica, reintegro solo se esplicitamente sicuro;

se il nodo era optional, può restare escluso;

se il nodo era critical, valutare fallback o abort;

se l'errore era watchdog_reset o energy_fault, preferire diagnostic prima di ruolo attivo;

se l'errore era thermal_lock, reintegro attivo solo dopo thermal_ok stabile.

Questa policy impedisce reintegri impulsivi. Un nodo recuperato non è automaticamente un nodo affidabile. Deve dimostrare stabilità.

Il ripristino automatico deve distinguere tra recovery locale e recovery globale. La recovery locale riguarda il singolo nodo. La recovery globale riguarda il pattern o lo swarm. Se un nodo optional recupera, il pattern può continuare senza modifiche. Se un nodo critical fallisce, il controller può dover ricalcolare pattern, cambiare ruoli, ridurre struttura o abortire. Il ripristino automatico quindi non è solo firmware: è anche gestione del runtime e del comportamento collettivo.

Una gerarchia di recovery può essere:

node_recovery: recupero del singolo MicroBot;

edge_recovery: ripristino di una connessione o relazione;

role_recovery: riassegnazione di una funzione;

pattern_recovery: correzione o fallback del pattern;

swarm_recovery: ritorno dell'intero sistema a stato sicuro.

Questa gerarchia è utile perché gli errori possono avere effetti locali o sistemici. Un reset di un LED-only node può essere locale. Un reset di un bridge_node può rompere una struttura. Un controller_lost può coinvolgere tutto lo swarm.

Il ripristino automatico dopo thermal_lock deve essere particolarmente prudente. Thermal_lock indica che il carico termico ha raggiunto una soglia critica. Il nodo non deve tornare immediatamente a thermal_ok appena il valore stimato scende leggermente. Deve passare per thermal_recovery. Durante thermal_recovery, attuatori termicamente critici restano bloccati. Solo quando thermal_load è sceso sotto una soglia di sicurezza e vi è stato un intervallo stabile, il nodo può tornare thermal_ok. Questo processo deve usare hysteresis.

Una sequenza corretta è:

thermal_lock attivo;

bobine disabilitate;

cooldown prolungato;

thermal_load decresce;

entra in thermal_recovery;

attende soglia sicura e tempo minimo;

thermal_ok ripristinato;

capabilities attuative riabilite gradualmente.

La parola “gradualmente” è importante. Dopo un lock, non si deve riprendere con duty massimo. Si può rientrare in conservative_mode, con duty ridotto e cooldown maggiore. Solo dopo stabilità si torna a full_mode, se necessario.

Il ripristino automatico dopo critical_energy deve essere ancora più restrittivo. Se il nodo ha energia critica, non può “recuperare” senza una ricarica, una sostituzione batteria o una sorgente stabile. Il sistema può passare da critical_energy a low_energy o energy_ok solo se la misura o lo stato energetico lo confermano. Se il valore è simulato, il controller può modificare lo stato per test. Se il valore è reale, il nodo deve rilevare tensione sufficiente. Fino ad allora, le funzioni ad alto consumo restano bloccate.

Una sequenza energetica può essere:

critical_energy;

safe_idle;

attuatori disabilitati;

telemetria minima;

attesa ricarica o sostituzione;

energy_state torna low_energy o energy_ok;

request_status;

state_sync;

eventuale reintegro passivo;

solo dopo stabilità, eventuale ruolo attivo.

Il nodo non deve passare da critical_energy a active direttamente. Serve transizione controllata.

Il ripristino automatico dopo perdita del controller richiede un comportamento specifico. Se il nodo non riceve comunicazioni dal controller per un tempo superiore a controller_lost_timeout, deve entrare in safe_idle. Quando il controller torna, il nodo non deve riprendere automaticamente il vecchio pattern. Deve inviare stato, ricevere sync e state_sync. Questo protegge il sistema da interruzioni temporanee di rete o riavvii del software centrale.

Una sequenza può essere:

last_controller_seen supera soglia;

nodo entra in safe_idle;

disattiva uscite critiche;

mantiene heartbeat o tenta reconnect;

controller torna online;

controller rileva nodo safe_idle;

esegue sync e request_status;

decide se reintegro o nuova run.

Questa procedura è fondamentale perché la perdita del controller può avvenire facilmente durante sviluppo: computer che si spegne, server che crasha, Wi-Fi instabile, script riavviato. Il nodo fisico deve restare sicuro.

Il ripristino automatico dopo emergency_stop deve essere controllato manualmente o semi-manualmente. Emergency_stop è una condizione grave. Dopo un emergency_stop, il sistema non dovrebbe ripartire automaticamente solo perché il comando è stato ricevuto. Deve restare in safety o safe_idle finché non viene eseguita una procedura di clear_safety. Questa procedura deve verificare che le cause dell'emergenza siano risolte. In una prima versione, clear_safety può essere manuale dalla dashboard o da comando esplicito. Non deve essere automatico immediato.

Una regola forte è:

emergency_stop richiede ripristino esplicito prima di nuove attuazioni.

Questo evita riavvii pericolosi. Se qualcuno preme stop, il sistema deve fermarsi e restare fermo finché non viene autorizzato il recupero. La ripartenza automatica dopo emergenza sarebbe una pessima scelta.

La procedura `clear_safety` può includere:

tutte le uscite critiche OFF;

nessun comando attuativo pendente;

`thermal_state` non in lock;

`energy_state` non critical;

controller sincronizzato;

utente o controller autorizza clear;

nodo passa da `safety` a `safe_idle`;

successivo `state_sync` se necessario.

Anche dopo `clear_safety`, il nodo non deve tornare subito active. Deve passare per idle o diagnostic.

Il ripristino automatico dopo `role_mismatch` o `phase_mismatch` è meno fisico ma comunque importante. Se il controller si aspetta che il nodo sia `connector` e il nodo riporta `passive_node`, o se il controller è in `phase_attach` mentre il nodo è in `phase_align`, il nodo è logicamente disallineato. In questo caso, `recovery` significa `state_sync` o riassegnazione ruolo. Non serve necessariamente `safety_stop`, salvo che l'errore riguardi attuazioni critiche. Il sistema deve correggere il contesto.

Una procedura può essere:

rileva mismatch;

blocca comandi critici per il nodo;

invia `request_status`;

confronta `run_id`, `phase_id`, `role`, `config_epoch`;

invia `state_sync` o `assign_role`;

attende ack;

verifica `reported_state`;

se coerente, reintegra;

se persistente, esclude nodo dalla run.

Questa `recovery` è essenziale per mantenere coerenza distribuita.

Il ripristino automatico deve avere limiti. Non deve tentare `recovery` infinite. Se un nodo continua a resettarsi, continua a entrare in `thermal_lock`, continua a perdere sync o continua a rifiutare `state_sync`, il controller deve marcarlo `unstable` o `failed` e smettere di reintegrarlo automaticamente.

Recovery infinita può consumare energia, generare calore, occupare rete e rendere il sistema confuso. Serve un `retry_limit`.

Una policy può essere:

massimo N tentativi di recovery per nodo per run;

massimo M reset recenti prima di `diagnostic_mode` obbligatoria;

massimo K `state_sync` falliti prima di esclusione;

`thermal_lock` ripetuto → `hardware_check_required`;

`critical_energy` persistente → no recovery finché energia non torna valida.

Questi limiti impediscono loop di recupero. Un nodo instabile deve essere isolato, non inseguito all'infinito.

Il ripristino automatico deve essere visibile nella dashboard. L'utente deve sapere quando un nodo è in recovery, perché ci è entrato, cosa sta aspettando, quali funzioni sono bloccate e se può essere reintegrato. La dashboard può mostrare stati come `recovery_pending`, `resync_required`, `thermal_recovery`, `energy_recovery`, `safety_hold`, `diagnostic_required`, `ready_for_reintegration`. Questo rende il sistema comprensibile.

Una scheda recovery può mostrare:

`recovery_type`;

`trigger_event`;

`current_recovery_step`;

`blocked_capabilities`;

`required_action`;

`time_in_recovery`;

`retry_count`;

`next_allowed_action`;

`reintegration_status`.

Per esempio:

MB-004 — `thermal_recovery` — `coil_pwm` blocked — cooldown 18s — reintegration not allowed.

Oppure:

MB-002 — `reset_recovery` — sync required — `state_sync` pending — reintegration possible after ack.

Questi stati aiutano moltissimo nel debug e nella presentazione.

Il ripristino automatico deve essere registrato nei log. Ogni recovery deve avere un recovery_id, trigger, stato iniziale, azioni eseguite, esito e durata. Questo permette di analizzare la stabilità del sistema. Un nodo che richiede molte recovery è problematico. Una run con molte recovery è meno pulita. Un pattern che causa sempre recovery termica deve essere rivisto. I log di recovery sono quindi metriche di qualità.

Un record di recovery può contenere:

recovery_id;
node_id;
trigger_event;
recovery_type;
start_time;
steps_executed;
commands_sent;
acks_received;
blocked_capabilities;
final_state;
result;
duration_ms.

Il result può essere:

recovered;
recovered_degraded;
failed;
isolated;
requires_manual_intervention.

Questa classificazione è molto utile. Non tutti i recovery finiscono nello stesso modo. Un nodo può recuperare completamente, recuperare solo in modalità passiva, fallire, essere isolato o richiedere intervento.

Il ripristino automatico deve influenzare il risultato della run. Se una run richiede recovery ma raggiunge comunque il pattern, non deve essere classificata come success_clean. Può essere success_with_recovery. Se il pattern viene degradato, partial_success_with_recovery. Se la recovery fallisce, failed_recovery. Questo rende i report onesti e tecnici.

Una classificazione possibile è:

success_clean: nessuna recovery rilevante;

success_with_recovery: recovery riuscita senza degradazione finale;

partial_success_with_recovery: recovery riuscita ma pattern degradato;

failed_recovery: recovery fallita e pattern compromesso;

aborted_after_recovery_failure: run interrotta per recovery non riuscita;

manual_intervention_required: sistema fermato in attesa operatore.

Questa classificazione aiuta a misurare maturità del sistema. Un pattern robusto dovrebbe richiedere poche recovery. Un sistema resiliente dovrebbe recuperare bene quando servono.

Il ripristino automatico deve essere testato. Come tutte le parti del Safety Layer, non basta descriverlo. Bisogna creare scenari. Si può simulare watchdog reset, perdita controller, thermal_lock, energy_low, role_mismatch, config_epoch mismatch e nodo offline. Il controller deve reagire correttamente. La dashboard deve mostrare recovery. I log devono registrare. Il nodo non deve rientrare automaticamente in ruolo attivo senza verifica.

Test importanti sono:

test_watchdog_recovery;

test_controller_lost_recovery;

test_thermal_lock_recovery;

test_low_energy_recovery;

test_critical_energy_no_auto_recovery;

test_emergency_stop_requires_clear;

test_role_mismatch_state_sync;

test_boot_id_change_resync_required;

test_recovery_retry_limit;

test_unstable_node_isolation.

Questi test possono essere implementati prima nel simulatore. Per hardware reale, si devono usare condizioni sicure. Per esempio, simulare thermal_lock senza scaldare davvero la bobina è un ottimo primo passo.

Il ripristino automatico deve essere integrato con la simulazione software. Il simulatore può generare eventi di errore e testare la risposta del controller. Questo è perfettamente coerente con la Sezione 12. Prima di affidare recovery a hardware reale, si può verificare nel modello. Il simulatore può far sparire un nodo, farlo tornare con nuovo boot_id, farlo entrare in low_energy, bloccare una capability, forzare un role_mismatch. La recovery diventa parte dei test di robustezza dello swarm.

Scenari simulativi utili:

nodo leader resetta durante linea;
connector entra in thermal_lock durante ponte;
nodo critical_energy durante griglia;
controller si interrompe e ritorna;
nodo rientra con config_epoch vecchio;
emergency_stop durante phase_attach;
ripartenza non autorizzata bloccata correttamente.

Questi scenari dimostrano che MicroBot non funziona solo quando tutto va bene.

Il ripristino automatico deve anche rispettare la differenza tra recovery automatica e recovery assistita. Alcune recovery possono essere automatiche: resync dopo perdita temporanea, request_status, state_sync, ritorno da stale a online, reintegro passivo. Altre richiedono intervento o autorizzazione: clear_safety dopo emergency_stop, riabilitazione bobina dopo fault fisico, reset di una condizione hardware incerta, rientro di un nodo con reset ripetuti. Questa distinzione è importante per la sicurezza.

Una classificazione può essere:

automatic_recovery: ammessa per errori lievi o controllati;
assisted_recovery: richiede conferma operatore o dashboard;
manual_recovery: richiede intervento fisico o verifica hardware.

Esempi:

sync_degraded temporaneo → automatic_recovery;
role_mismatch non critico → automatic_recovery;
emergency_stop → assisted_recovery;
thermal_lock ripetuto → assisted o manual;
brownout ripetuto → manual_check consigliato;
driver_fault → manual_recovery.

Questo impedisce che il sistema si riavvii automaticamente dopo eventi che meritano controllo umano.

Il ripristino automatico deve essere progettato anche per evitare rientri troppo rapidi. Dopo un errore, serve un periodo di stabilizzazione. Un nodo recuperato deve restare stabile per un intervallo minimo prima di assumere ruoli critici. Per esempio, dopo riconnessione, può inviare heartbeat per alcuni secondi. Dopo thermal_recovery, può restare in conservative_mode. Dopo reset, può essere solo passive_node nella run corrente. Questo approccio riduce rischi.

Una regola può essere:

dopo recovery, il nodo rientra prima come passive o idle; ruoli attivi richiedono stabilità prolungata.

Questo è particolarmente importante per nodi che hanno appena avuto watchdog_reset, brownout o thermal_lock. Il sistema deve fidarsi gradualmente, non subito.

Il ripristino automatico deve interagire con i ruoli. Se un nodo recuperato non può tornare al ruolo precedente, il controller può riassegnare ruoli. Per esempio, se MB-004 era active_connector ma dopo recovery ha coil_pwm blocked, può diventare passive_node. Un altro nodo può diventare active_connector. Se non esiste sostituto, il pattern può degradare. Questo è il collegamento tra recovery e adattamento dinamico.

Una policy di rientro ruolo può essere:

se ruolo precedente era non critico, reintegro possibile;

se ruolo precedente era critico, richiedere verifica completa;

se capability necessaria è bloccata, assegnare ruolo alternativo;

se nessun ruolo alternativo è utile, mantenere idle;

se pattern dipendeva da quel ruolo, fallback o abort.

Questa logica mantiene coerenza tra stato fisico e struttura.

Il ripristino automatico deve interagire con la configurazione. Se il nodo rientra con firmware diverso, protocol_version diverso o config_epoch vecchio, non deve essere reintegrato automaticamente. Serve config_sync o esclusione. Il ripristino non riguarda solo energia e temperatura, ma anche coerenza software. Un nodo con firmware incompatibile può essere online ma non sicuro per la run.

Una regola è:

recovery completa richiede protocol_version compatibile e config_epoch aggiornato.

Se questa condizione non è soddisfatta, il nodo può restare in diagnostic_mode. Questo evita errori difficili da vedere.

Il ripristino automatico deve evitare la perdita di informazioni. Quando possibile, il nodo deve inviare la causa dell'errore prima di resettare o subito dopo. Se entra in safety, deve inviare evento. Se si resetta, deve inviare reset_reason al boot. Se perde controller, deve registrare localmente lo stato e inviarlo al ritorno, se possibile. In prototipi semplici, la memoria locale può essere limitata, ma il principio resta valido: ogni recovery deve lasciare traccia.

In futuro, si può prevedere una piccola crash/recovery memory con:

last_error_code;

last_command_id;

last_mode;

last_output_state;

reset_reason;

reset_count.

Anche se non implementata subito, questa struttura è molto utile per debugging.

Il ripristino automatico deve essere coerente con l'intero Safety Layer. Corrente, bobine, temperatura, batteria, energia e watchdog non sono moduli isolati. Il recovery deve consultarli tutti. Non basta che la rete sia tornata; bisogna verificare anche thermal_state. Non basta che thermal_state sia ok; bisogna verificare energy_state. Non basta che energy_state sia ok; bisogna verificare sync e config. Il nodo può tornare operativo solo quando tutte le condizioni minime sono soddisfatte.

Una condizione generale di reintegro può essere:

$$\text{reintegrable} = \text{communication_ok} \wedge \text{sync_ok} \wedge \text{safety_ok} \wedge \text{energy_ok_or_acceptable} \wedge \text{thermal_ok_or_acceptable} \wedge \text{config_ok} \wedge \text{role_compatible}$$

Se una di queste condizioni fallisce, il nodo può restare online ma non attivo. Questa formula riassume bene il Safety Layer.

Il ripristino automatico deve avere una roadmap. Nelle prime versioni, si può implementare safe_idle, boot_id, request_status e reset detection. Poi si aggiunge state_sync. Poi si aggiungono recovery modes. Poi dashboard e log. Poi reintegro automatico controllato. Poi recovery gerarchica per pattern. Questa progressione è realistica.

Una roadmap possibile è:

v0.1: safe_idle dopo errore;

v0.2: boot_id e reset detection;

v0.3: request_status dopo recovery;

v0.4: sync recovery;

v0.5: state_sync;

v0.6: dashboard recovery state;

v0.7: log recovery completo;

v0.8: retry_limit e isolamento nodi instabili;

v0.9: reintegro automatico controllato;

v1.0: pattern recovery e fallback strutturato.

Questa roadmap chiude il Safety Layer in modo coerente con lo sviluppo incrementale di MicroBot.

Dal punto di vista concettuale, il ripristino automatico rappresenta la capacità del sistema di non rimanere bloccato nell'errore. Un sistema senza recovery può essere sicuro perché si ferma, ma non

è resiliente. Un sistema con recovery cieca può essere pericoloso perché riparte senza verifiche. MicroBot deve stare nel mezzo corretto: fermarsi quando serve, conservare stato sicuro, comunicare il problema, verificare le condizioni e ripartire solo se è coerente farlo. Questa è la differenza tra reset e ripristino controllato.

In conclusione, il ripristino automatico è il processo attraverso cui MicroBot torna da una condizione anomala a uno stato sicuro, osservabile, sincronizzato e, quando possibile, operativo. Esso gestisce reset, watchdog, perdita di comunicazione, thermal_lock, critical_energy, emergency_stop, role_mismatch, config mismatch e recovery dei pattern. La procedura corretta parte sempre dalla messa in sicurezza, passa attraverso heartbeat o recovery_event, ricostruisce boot_id, sync, stato, capabilities e configurazione, poi decide se reintegrare, degradare, isolare o richiedere intervento. Il ripristino automatico non deve essere una ripartenza cieca: deve essere una sequenza verificabile, tracciata nei log e visibile nella dashboard. Con questo punto si completa la Sezione 11, perché il Safety Layer non solo previene errori e protegge i componenti, ma definisce anche come il sistema deve comportarsi dopo un errore. La sezione successiva, dedicata alla Simulazione Software, potrà quindi mostrare come molte di queste logiche — pattern, safety, temperatura, energia, recovery e comportamento dello swarm — possano essere testate prima in ambiente virtuale, riducendo rischi e accelerando lo sviluppo sperimentale.

SEZIONE 12 — Simulazione Software

La Sezione 12 introduce la Simulazione Software del sistema MicroBot, cioè l'ambiente virtuale in cui i nodi, i pattern, i comportamenti, la comunicazione, la telemetria, il Safety Layer e le logiche di recovery possono essere progettati, osservati e validati prima di essere trasferiti su hardware reale. Dopo la Sezione 11, dedicata alla sicurezza fisica ed energetica, la simulazione assume un ruolo ancora più importante: non è soltanto una rappresentazione grafica dello swarm, ma un laboratorio controllato in cui testare condizioni normali, errori, ritardi, limiti termici, nodi mancanti, fallback, assegnazioni di ruolo e strategie di coordinazione.

Nel progetto MicroBot, la simulazione non deve essere considerata una parte secondaria o decorativa. Essa rappresenta uno dei livelli fondamentali dell'intero ecosistema, perché permette di sviluppare rapidamente idee che sarebbero lente, costose o rischiose da provare subito sull'hardware. Prima di attivare bobine reali, il sistema può simulare magnetic_pulse, cooldown, thermal_load e connessioni virtuali. Prima di coordinare molti ESP32 fisici, può simulare decine o centinaia di nodi. Prima di implementare un pattern complesso, può verificarne geometria, topologia, ruoli e metriche. Prima di far partire una run reale, può stimare se il piano è sicuro. La simulazione è quindi una fase di validazione preventiva.

La simulazione software deve avere due funzioni principali. La prima è la funzione progettuale: permette di costruire il modello del sistema, testare formule, regole locali, pattern, strutture e comportamenti emergenti. La seconda è la funzione diagnostica: permette di verificare se il controller reagisce correttamente a errori, timeout, nodi offline, ritardi, thermal_lock, low_energy, recovery e perdita di sincronizzazione. Queste due funzioni sono diverse ma complementari. La prima risponde alla domanda: “il comportamento desiderato può funzionare?” La seconda risponde alla domanda: “il sistema reagisce bene quando qualcosa non funziona?”

Un ambiente di simulazione MicroBot deve quindi rappresentare non solo il movimento o la posizione dei nodi, ma anche il loro stato operativo. Ogni nodo simulato deve avere node_id, posizione, velocità se prevista, ruolo, energia, temperatura stimata, stato safety, sync_status, phase_id, run_id, capabilities e stato comunicativo. Questo rende il nodo simulato simile al nodo fisico dal punto di vista logico. La differenza è che, nel simulatore, molte grandezze sono virtuali e

controllabili. Questa somiglianza è fondamentale perché permette al controller, alla dashboard e ai log di usare la stessa semantica sia per nodi simulati sia per nodi reali.

La simulazione deve quindi essere integrata con l'architettura generale, non separata. Il simulatore non deve essere un programma isolato che mostra pallini che si muovono. Deve poter dialogare con il controller, generare snapshot, inviare telemetria, ricevere comandi, applicare pattern, simulare safety e produrre log. In questo modo, diventa una parte del runtime. Un nodo simulato può rispondere a `set_role`, `state_sync`, `start_pattern`, `magnetic_pulse` virtuale, `request_status` e `emergency_stop` simulato. Questo permette di testare la pipeline completa anche senza hardware.

Il vantaggio principale della simulazione è l'iterazione veloce. Nell'hardware, ogni modifica richiede cablaggio, upload firmware, test fisico, attenzione alla corrente, controllo della temperatura e gestione dei componenti. Nel software, invece, si può cambiare una regola, lanciare una run, osservare la dashboard, analizzare log e modificare parametri in pochi minuti. Questo non sostituisce l'hardware, ma accelera enormemente la progettazione. Un comportamento che non funziona in simulazione difficilmente funzionerà bene nell'hardware. Un comportamento che funziona in simulazione deve comunque essere validato fisicamente, ma parte già da una base più ragionata.

La Sezione 12 seguirà quindi i punti dell'indice: ambiente di simulazione, implementazione web, visualizzazione dei MicroBot, modelli virtuali, test dei pattern, validazione del comportamento, iterazione veloce e sviluppo sperimentale. L'obiettivo è definire la simulazione come uno strumento centrale del progetto MicroBot, capace di collegare teoria, software, dashboard, safety e prototipi fisici.

12.1 Ambiente di simulazione

L'ambiente di simulazione è lo spazio software in cui il sistema MicroBot viene rappresentato, eseguito e osservato in forma virtuale. Esso deve permettere di creare nodi simulati, assegnare loro stati, ruoli e parametri, applicare pattern, far evolvere il sistema nel tempo, generare telemetria, visualizzare risultati e registrare eventi. Un ambiente di simulazione ben progettato non si limita a disegnare nodi sullo schermo, ma riproduce in modo controllato la logica fondamentale dello swarm: identità dei nodi, comportamento locale, coordinazione globale, comunicazione, sincronizzazione, safety e recovery.

Nel progetto MicroBot, l'ambiente di simulazione deve essere pensato come un laboratorio multilivello. Al livello più basso ci sono i nodi virtuali, ciascuno con posizione, stato e parametri. Al livello intermedio ci sono le regole di comportamento, come coesione, separazione, allineamento, attrazione verso target, mantenimento distanza e connessione virtuale. Al livello superiore ci sono pattern, ruoli, fasi e controller. Sopra ancora ci sono dashboard, telemetria, log e report. Questa stratificazione permette di sviluppare e testare ogni parte del sistema senza confonderla con le altre.

Un nodo simulato deve avere una struttura interna simile a quella di un nodo reale. Deve possedere un `node_id`, per esempio SIM-001, SIM-002, SIM-003. Deve avere un `mode`, come `idle`, `active`, `diagnostic`, `safety` o `offline`. Deve avere un `role`, come `leader`, `connector`, `boundary`, `core`, `terminal` o `passive_node`. Deve avere capabilities virtuali, per esempio `simulated_motion`, `simulated_magnetic_field`, `telemetry_basic`, `role_assignment`, `pattern_participation`. Deve avere uno stato energetico simulato, uno stato termico simulato e uno stato di comunicazione simulato. Questo permette di testare le stesse regole usate per i nodi fisici.

Una struttura concettuale del nodo simulato può essere:

node_id;
position;
velocity;
target_position;
mode;
role;
group_id;
pattern_run_id;
phase_id;
capabilities;
energy_state;
thermal_state;
sync_status;
safety_state;
last_command;
last_update_time.

Questa struttura è importante perché consente al simulatore di diventare compatibile con il resto dell'ecosistema. Se il controller legge un nodo simulato e un nodo fisico, deve trovare campi simili. Il nodo fisico può avere posizione non disponibile o stimata; il nodo simulato può avere posizione precisa. Il nodo fisico può avere temperatura misurata o stimata; il nodo simulato può avere thermal_load virtuale. Ma la semantica generale resta la stessa.

L'ambiente di simulazione deve gestire il tempo. Ogni simulazione evolve per step. Uno step rappresenta un avanzamento dello stato del sistema: aggiornamento delle posizioni, applicazione delle forze virtuali, controllo dei target, aggiornamento dei timer, generazione di eventi e invio di snapshot. Il tempo può essere discreto, per esempio $\text{step} = 0, 1, 2, 3$, oppure espresso in millisecondi simulati. È importante distinguere `simulation_time` da `controller_time` e `real_time`. Il simulatore può andare più veloce o più lento del tempo reale. Per una dashboard live, invece, può essere utile sincronizzare lo step con il tempo effettivo.

Si possono distinguere tre riferimenti temporali:

`simulation_time`: tempo interno del simulatore;

`real_time`: tempo effettivo del computer;

`controller_time`: tempo usato dal controller per comandi, sync e run.

In una simulazione semplice, questi tempi possono coincidere approssimativamente. In una simulazione avanzata, possono essere separati. Per esempio, il simulatore può eseguire una run accelerata per analisi, oppure rallentata per osservazione. Questa distinzione è utile anche per replay: una run registrata può essere riprodotta più lentamente o più velocemente senza cambiare gli eventi logici.

L'ambiente di simulazione deve implementare modelli di comportamento. Un comportamento può essere target-based, cioè ogni nodo cerca di raggiungere una posizione obiettivo. Può essere rule-based, cioè ogni nodo segue regole locali come coesione, separazione e allineamento. Può essere phase-based, cioè cambia in base alla fase del pattern. Può essere role-based, cioè dipende dal ruolo assegnato. Un buon simulatore MicroBot deve supportare almeno una combinazione di questi approcci.

Un comportamento target-based è utile per linee, cerchi e griglie. Il controller calcola posizioni target e ogni nodo virtuale si muove verso il proprio target. Questo permette di verificare rapidamente pattern geometrici. Un comportamento rule-based è utile per cluster, swarm e comportamento emergente. Ogni nodo reagisce ai vicini senza conoscere tutta la forma. Un comportamento phase-based permette di simulare sequenze come prepare, align, attach, stabilize e report. Un comportamento role-based permette a leader, connector, terminal e boundary di agire diversamente.

La simulazione MicroBot deve quindi poter combinare:

target attraction;

neighbor separation;

cohesion;

alignment;

connection maintenance;

role-specific rules;

phase-specific rules;

safety constraints.

Questa combinazione rende il modello più ricco. Non tutti i comportamenti devono essere implementati subito. Ma la struttura deve prevederli.

L'ambiente di simulazione deve rappresentare i pattern. Un pattern può essere caricato da configurazione o generato nel codice. Per esempio, line_basic genera target lungo un asse. circle_basic genera target su una circonferenza. grid_basic genera celle. cluster_basic genera una regione di coesione. bridge_complex combina cluster e catena. Il simulatore deve trasformare la definizione di pattern in target, ruoli, fasi e metriche. Questo permette di testare la Sezione 10 in modo operativo.

Una pipeline di simulazione pattern può essere:

carica pattern_id;

seleziona nodi partecipanti;

assegna ruoli;
calcola target geometrici;
genera topologia attesa;
avvia phase_prepare;
aggiorna nodi step dopo step;
misura errori;
applica correzioni o fallback;
produce report.

Questa pipeline è esattamente il ponte tra documentazione teorica e sistema funzionante. La simulazione non è solo “vedere qualcosa”; è verificare se il pattern definito nel documento può essere trasformato in comportamento.

L’ambiente di simulazione deve anche rappresentare la topologia. Per MicroBot, non basta sapere dove si trovano i nodi. Bisogna sapere quali nodi sono collegati, quali dovrebbero essere collegati, quali connessioni sono mancanti, quali sono indesiderate e quali sono instabili. Nel simulatore, una connessione può essere virtuale: se due nodi sono entro una distanza e rispettano condizioni di ruolo/fase, si crea un edge. Oppure può essere target-based: il pattern dichiara che MB-001 deve essere collegato a MB-002. La simulazione può confrontare edge attesi ed edge reali.

Un modello topologico può contenere:

expected_edges;
current_edges;
missing_edges;
extra_edges;
edge_strength;
edge_state;
edge_error.

Questo permette di testare catene, cerchi, reticoli e strutture complesse. Anche se l’hardware fisico non ha ancora aggancio reale, il simulatore può già validare la logica topologica. Quando arriverà l’hardware, alcune connessioni virtuali potranno essere sostituite da connessioni misurate o eventi fisici.

L’ambiente di simulazione deve includere safety virtuale. Questo punto è importantissimo dopo la Sezione 11. La simulazione non deve ignorare corrente, temperatura, batteria, cooldown e recovery. Anche se questi valori sono virtuali, devono influenzare il comportamento. Un nodo simulato può entrare in thermal_warning se riceve troppi magnetic_pulse virtuali. Può entrare in low_energy se consuma troppo. Può rifiutare comandi ad alto costo. Può andare in safety se riceve

emergency_stop. Può simulare watchdog_reset e rientrare con nuovo boot_id. Questo permette di testare il Safety Layer senza rischiare componenti reali.

Una safety simulation può includere:

thermal_load virtuale;

energy_level virtuale;

cooldown;

command rejection;

emergency_stop;

controller_lost;

watchdog_reset simulato;

recovery_state;

blocked_capabilities.

Questi elementi trasformano il simulatore in uno strumento di validazione, non solo di visualizzazione. Si può testare cosa succede se un connector entra in thermal_lock durante un ponte, o se un nodo low_energy viene escluso da una griglia, o se un leader si resetta durante una linea. Questi scenari sono fondamentali per costruire robustezza.

L'ambiente di simulazione deve permettere fault injection. Fault injection significa introdurre artificialmente errori per vedere come reagisce il sistema. In un sistema MicroBot, si possono simulare nodi offline, ritardi, pacchetti persi, jitter, energia bassa, temperatura alta, reset, ruolo sbagliato, fase sbagliata, connessione mancante, sensore rumoroso o telemetria stale. Questa capacità è centrale perché un sistema che funziona solo in condizioni ideali non è sufficiente.

Esempi di fault injection:

disattiva SIM-004 durante phase_align;

ritarda gli ack di SIM-002;

imposta SIM-003 in low_energy;

forza thermal_lock su un active_connector;

rimuovi un edge previsto;

genera role_mismatch;

genera config_epoch_mismatch;

simula controller_lost;

simula watchdog_reset.

Ogni fault deve avere una risposta attesa. Il controller deve rilevare, classificare, correggere, degradare o abortire secondo policy. Questo permette di testare la coerenza delle sezioni precedenti.

L'ambiente di simulazione deve produrre snapshot. Uno snapshot è una fotografia dello stato dello swarm in un certo istante o step. Può contenere posizione dei nodi, ruoli, pattern, metriche, stato safety, telemetria e timestamp. Gli snapshot sono utili per la dashboard live, per il debug, per il replay e per i report. Nel tuo ecosistema MicroBot, gli snapshot live sono già una logica coerente: un file come live_swarm_snapshot.json può rappresentare lo stato attuale della simulazione e permettere alla piattaforma web di leggerlo.

Uno snapshot può contenere:

simulation_id;

step;

timestamp;

mode;

pattern_run_id;

nodes;

edges;

metrics;

events_recent;

safety_summary.

Ogni nodo nello snapshot può contenere:

node_id;

position;

velocity;

role;

mode;

energy_state;

thermal_state;

sync_status;

safety_state;

target_position;

error_to_target.

Questa struttura permette alla dashboard di visualizzare lo swarm senza dover calcolare tutto da zero. Il simulatore produce lo stato; la dashboard lo mostra.

L'ambiente di simulazione deve generare metriche. Senza metriche, la simulazione resta visiva e soggettiva. Le metriche permettono di dire se un pattern sta riuscendo. Per la linea, errore laterale e spacing. Per il cerchio, errore radiale e distribuzione angolare. Per la griglia, errore delle celle e regolarità. Per il cluster, coesione e separazione. Per la safety, thermal_load, energy_state, warning, lock. Per la comunicazione simulata, packet_loss, latency e jitter. Per la recovery, numero di reset e tempo di recupero.

Metriche generali del simulatore possono essere:

pattern_error;

topology_score;

aggregation_score;

node_count;

active_nodes;

offline_nodes;

average_distance_to_target;

stability_score;

thermal_risk_score;

energy_risk_score;

recovery_count;

event_count.

Queste metriche devono essere registrate e mostrate. Un valore come aggregation_score è molto utile perché sintetizza quanto lo swarm si sta aggregando o stabilizzando. Tuttavia, ogni metrica deve avere significato chiaro. Non basta mostrare numeri; bisogna sapere cosa misurano.

L'ambiente di simulazione deve supportare modalità diverse. Una modalità live permette alla simulazione di evolvere in tempo reale e alimentare la dashboard. Una modalità batch permette di eseguire molte run rapidamente per confrontare parametri. Una modalità replay permette di rivedere una run registrata. Una modalità diagnostic permette di testare singoli comportamenti o fault. Una modalità demo permette di mostrare pattern in modo pulito e comprensibile. Queste modalità possono condividere lo stesso motore, ma servire obiettivi diversi.

Le modalità principali possono essere:

live_simulation;

batch_test;

replay_mode;

diagnostic_mode;

demo_mode;

fault_injection_mode.

Questa classificazione aiuta a organizzare il software. Non tutto deve essere interattivo. Alcuni test possono essere automatici. Alcune demo possono essere ottimizzate per chiarezza visiva. Alcune simulazioni possono essere usate solo per generare report.

L'ambiente di simulazione deve essere collegato ai file di configurazione. Parametri come numero nodi, pattern, velocità, rumore, energia iniziale, thermal_load iniziale, fault injection, durata della run e modalità di output dovrebbero poter essere configurati senza modificare codice. Questo rende il sistema più flessibile e documentabile. Un file sample_config.json può definire una run simulata. Un file initial_swarm_state.json può definire lo stato iniziale. Un file behavior_profiles.json può definire pesi comportamentali. Un file field_profiles.json può definire campi virtuali o forze. Un file experimental_runs.json può elencare esperimenti.

Una struttura coerente può essere:

sample_config.json: parametri generali della simulazione;

initial_swarm_state.json: nodi iniziali e posizioni;

behavior_profiles.json: pesi e regole comportamentali;

field_profiles.json: campi virtuali e parametri fisici;

experimental_runs.json: scenari di test e run;

sample_swarm_dataset.jsonl: dati o esempi di stati dello swarm.

Questa organizzazione è perfettamente coerente con un repository tecnico MicroBot. Ogni simulazione diventa ripetibile. Se una run produce un risultato interessante, si può salvarne la configurazione e rieseguirlo.

L'ambiente di simulazione deve distinguere tra parametri fisici e parametri visuali. Un parametro fisico influenza la dinamica del nodo, per esempio mass, damping, max_speed, cohesion_weight, magnetic_strength, separation_radius. Un parametro visuale influenza solo la rappresentazione, per esempio colore, dimensione del nodo, intensità glow, tracce, label, zoom. Questa distinzione evita confusione. Cambiare il colore di un nodo non deve cambiare il comportamento. Cambiare cohesion_weight sì.

Una separazione ordinata può essere:

simulation_parameters: dinamica, regole, tempo, sicurezza;

visualization_parameters: colori, effetti, HUD, zoom, label;

runtime_parameters: modalità, logging, output, snapshot.

Questa separazione rende il codice più pulito e la documentazione più rigorosa.

L'ambiente di simulazione deve prevedere rumore e imperfezione. Una simulazione perfetta può essere utile all'inizio, ma rischia di essere troppo distante dalla realtà. I nodi reali hanno ritardi, misure imprecise, energia limitata, reset, variazioni e guasti. Il simulatore deve poter introdurre rumore controllato. Per esempio, posizione rumorosa, ritardo di comunicazione, jitter, perdita pacchetti, errore di esecuzione, energia variabile. Questo permette di testare robustezza.

Tipi di rumore utili:

position_noise;

velocity_noise;

latency_noise;

packet_loss;

execution_delay;

sensor_noise;

energy_drain_variation;

thermal_variation.

Il rumore deve essere configurabile. In demo, si può ridurre per mostrare comportamento chiaro. In test, si può aumentare per valutare robustezza. Questa flessibilità è importante.

L'ambiente di simulazione deve supportare l'integrazione con hardware reale in modalità ibrida. In una demo ibrida, alcuni nodi sono simulati e alcuni fisici. Il simulatore mostra il comportamento completo, mentre i nodi fisici ricevono ruoli, LED, heartbeat o comandi limitati. Questa modalità è molto utile per MicroBot perché permette di dimostrare una visione ampia anche quando l'hardware fisico è ancora parziale. Per esempio, si può simulare uno swarm da 30 nodi e collegare 3 ESP32 reali come anchor, leader e connector visuali.

Una modalità ibrida può funzionare così:

simulatore genera swarm virtuale;

controller assegna ruoli a nodi virtuali e fisici;

dashboard visualizza entrambi;

nodi fisici mostrano stato tramite LED;

telemetria fisica entra nello snapshot;

eventi hardware influenzano la run, se previsto.

Questa modalità è potente perché mostra continuità tra software e hardware. Il MicroBot reale non deve essere completo per partecipare al sistema.

L'ambiente di simulazione deve essere progettato con modularità. Il motore fisico, il gestore pattern, il gestore safety, il logger, il generatore snapshot e la dashboard non dovrebbero essere un unico blocco confuso. Ogni parte deve avere responsabilità chiara. Questo permette di modificare un

comportamento senza rompere tutto. Per esempio, si può cambiare il modello di cluster senza cambiare la dashboard. Si può aggiungere `thermal_load` senza riscrivere il `renderer`. Si può testare un nuovo pattern usando lo stesso logger.

Una divisione logica può essere:

`simulation_engine;`

`node_model;`

`behavior_engine;`

`pattern_manager;`

`safety_simulator;`

`telemetry_generator;`

`snapshot_writer;`

`event_logger;`

`web_visualizer.`

Questa architettura rende il progetto più scalabile e più presentabile. Nei repository, ogni modulo può avere README e test.

L'ambiente di simulazione deve essere documentato con chiarezza. Ogni simulazione deve spiegare che cosa rappresenta, quali limiti ha, quali parametri usa, quali parti sono realistiche e quali sono astratte. Questo è importante per evitare confusione. Una simulazione di magnetismo, per esempio, può essere qualitativa e non fisicamente completa. Una simulazione di energia può essere stimata. Una simulazione di movimento può essere semplificata. Dichiarare i limiti non indebolisce il progetto; lo rende più serio.

Una buona documentazione di simulazione deve indicare:

scopo;

modello usato;

parametri principali;

assunzioni;

limiti;

output prodotti;

metriche;

modalità di esecuzione;

relazione con hardware reale.

Questa struttura permette a chi legge di capire cosa sta vedendo. È fondamentale per una tesi o per una repo pubblica.

L'ambiente di simulazione deve supportare report finali. Alla fine di una run, il simulatore può generare un report con pattern usato, numero nodi, durata, metriche finali, errori, recovery, thermal events, energy events, topology score e risultato. Questo report può essere salvato in JSON, Markdown o visualizzato nella dashboard. I report permettono di confrontare run diverse e dimostrare progressi nel tempo.

Un report può contenere:

run_id;

pattern_id;

configuration_file;

start_time;

end_time;

node_count;

final_pattern_error;

topology_score;

aggregation_score;

safety_events;

energy_events;

thermal_events;

recovery_events;

result;

notes.

Questi dati trasformano la simulazione in esperimento. Non è solo animazione, ma processo ripetibile e misurabile.

L'ambiente di simulazione deve essere utile anche per la comunicazione esterna. Quando si presenta MicroBot a una persona tecnica, a un collaboratore o a un potenziale investitore, una simulazione chiara può mostrare il concetto più velocemente di molte pagine di testo. Tuttavia, la simulazione deve essere onesta: deve distinguere tra virtuale e reale. Può mostrare pattern futuri, ma deve indicare che sono simulated. Può mostrare nodi fisici collegati, ma deve indicare quali sono reali. Questa trasparenza aumenta la credibilità.

Una demo ideale può mostrare:

swarm virtuale che forma pattern;

dashboard con metriche;

nodi fisici che ricevono ruoli;

safety event simulato;

recovery o fallback;

report finale.

Questa sequenza racconta l'intero ecosistema MicroBot: modello, controllo, visualizzazione, sicurezza e validazione.

In conclusione, l'ambiente di simulazione è il laboratorio centrale del progetto MicroBot. Esso permette di rappresentare nodi virtuali, pattern, ruoli, topologie, safety, energia, temperatura, comunicazione, fault e recovery in un contesto controllato e ripetibile. La simulazione non sostituisce l'hardware, ma riduce rischio, accelera sviluppo e rende verificabili le idee prima della realizzazione fisica. Un simulatore MicroBot maturo deve produrre snapshot, telemetria, metriche, log e report; deve supportare modalità live, batch, replay, diagnostic e fault injection; deve integrarsi con dashboard e controller; deve distinguere dati simulati, stimati e reali. Questo punto prepara direttamente il successivo, dedicato all'implementazione web, dove la simulazione diventa visibile, interattiva e presentabile attraverso interfacce browser, dashboard, canvas, pannelli di stato e strumenti di controllo.

12.2 Implementazione web

L'implementazione web rappresenta il livello attraverso cui la simulazione MicroBot diventa osservabile, interattiva e presentabile tramite browser. Dopo aver definito l'ambiente di simulazione come laboratorio centrale del sistema, è necessario descrivere come questo ambiente possa essere esposto attraverso una piattaforma web capace di mostrare nodi, pattern, telemetria, safety, eventi, metriche, snapshot e stato runtime. Nel progetto MicroBot, il web non deve essere visto soltanto come "interfaccia grafica", ma come uno strato di integrazione tra simulazione, controller, dashboard, documentazione e dimostrazione pubblica del sistema.

L'implementazione web ha una funzione tecnica e una funzione comunicativa. La funzione tecnica consiste nel permettere all'utente o allo sviluppatore di osservare lo stato dello swarm, verificare se i nodi sono online, controllare pattern, leggere metriche, vedere errori, consultare telemetria e analizzare eventi. La funzione comunicativa consiste nel rendere il progetto comprensibile a chi lo guarda dall'esterno. Una simulazione eseguita solo nel terminale può essere utile per debug, ma è molto meno efficace per raccontare il progetto. Una dashboard web, invece, può mostrare visivamente cosa sta accadendo: nodi che cambiano ruolo, pattern che si formano, stato safety, metriche che evolvono, warning termici, energia residua e log degli eventi.

Nel progetto MicroBot, l'implementazione web deve quindi essere pensata come ponte tra sistema interno e percezione esterna. Il backend può calcolare, simulare e gestire lo stato. Il frontend deve renderlo leggibile. Questo significa che il web layer non deve essere soltanto estetico, ma anche semanticamente coerente. Ogni elemento visuale deve corrispondere a un concetto reale dell'architettura: `node_id`, `role`, `mode`, `phase`, `pattern_run_id`, `sync_status`, `energy_state`, `thermal_state`, `safety_state`, `topology_score`, `aggregation_score`, `heartbeat`, `event_log`. La grafica deve aiutare a capire il sistema, non coprirlo.

Una piattaforma web MicroBot può essere composta da tre livelli principali. Il primo è il livello dati, che riceve o legge snapshot, telemetria, eventi e configurazioni. Il secondo è il livello di

visualizzazione, che disegna nodi, connessioni, pattern, pannelli e metriche. Il terzo è il livello di controllo, che permette eventualmente di avviare pattern, inviare comandi sicuri, cambiare modalità, selezionare scenari o attivare simulazioni. Questi tre livelli devono essere separati. Un frontend ben progettato non deve mescolare in modo confuso dati, rendering e comandi.

Una struttura concettuale può essere:

data layer: snapshot, telemetry, logs, config;

visual layer: canvas, grafi, pannelli, HUD, tabelle;

control layer: pulsanti, selettori, comandi, modalità demo;

safety layer web: conferme, blocco comandi pericolosi, warning, stato read-only.

Questa separazione è importante perché permette di usare la stessa interfaccia in modalità diverse. In modalità demo, il controllo può essere limitato e la visualizzazione enfatizzata. In modalità debug, si possono mostrare più dettagli tecnici. In modalità read-only, si osserva senza poter inviare comandi. In modalità hardware-safe, alcuni pulsanti possono essere disabilitati se i nodi non sono in stato sicuro.

Il cuore dell'implementazione web è lo snapshot dello swarm. Lo snapshot rappresenta lo stato corrente della simulazione o del sistema ibrido. Può essere letto periodicamente dal frontend, ricevuto via WebSocket oppure aggiornato tramite API. In una prima implementazione, il frontend può leggere un file JSON aggiornato dal simulatore. In una versione più avanzata, può ricevere aggiornamenti in tempo reale tramite WebSocket. Entrambe le soluzioni sono valide, ma hanno complessità diversa.

Una soluzione semplice è:

il simulatore scrive `live_swarm_snapshot.json`;

il frontend legge periodicamente il file o lo riceve tramite endpoint;

la dashboard aggiorna nodi, metriche e pannelli.

Una soluzione più evoluta è:

il simulatore invia eventi al backend;

il backend mantiene stato runtime;

il frontend riceve update via WebSocket;

la dashboard si aggiorna in tempo reale.

La prima soluzione è più facile da implementare e sufficiente per prototipi. La seconda è più adatta a demo live e controllo interattivo. In entrambi i casi, il formato dello snapshot deve essere stabile e documentato. Se il frontend dipende da campi casuali o non standardizzati, il sistema diventa fragile.

Uno snapshot web può contenere:

`simulation_id`;

runtime_mode;
timestamp;
step;
pattern_run_id;
pattern_id;
phase_id;
nodes;
edges;
metrics;
safety_summary;
energy_summary;
thermal_summary;
recent_events.

Il frontend può usare questi campi per disegnare lo stato. I nodi vengono rappresentati come elementi grafici. Gli edges come linee o connessioni. Le metriche come valori numerici, barre o grafici. Gli eventi come log. Lo stato safety come badge o warning. Il pattern corrente come titolo e overlay geometrico.

Ogni nodo nello snapshot deve avere una rappresentazione web chiara. Il nodo non deve essere solo un punto. Deve mostrare identità, ruolo, stato e condizioni principali. In visualizzazione compatta, può essere un cerchio con colore basato sul ruolo. In visualizzazione dettagliata, può mostrare label con node_id, role, mode e stato. Quando l'utente clicca sul nodo, può aprirsi un pannello laterale con telemetria completa. Questo rende la dashboard utile sia per demo sia per debug.

Una rappresentazione nodo può includere:

posizione sul canvas;
colore in base al ruolo;
bordo in base allo stato safety;
dimensione o glow in base all'attività;
label con node_id;
icona o badge per low_energy, thermal_warning, offline, sync_lost;
linea verso target se in modalità debug;
traccia del movimento se abilitata.

Questa visualizzazione rende il sistema leggibile. Per esempio, un leader può essere evidenziato in modo diverso da un connector. Un nodo in safety può avere bordo rosso. Un nodo in low_energy può avere icona energetica. Un nodo offline può essere grigio o trasparente. L'obiettivo non è solo fare una grafica bella, ma codificare informazioni tecniche in modo immediato.

Il web layer deve visualizzare anche le connessioni. Le connessioni possono essere `expected_edges`, `current_edges`, `missing_edges` o `extra_edges`. Nella dashboard, gli archi corretti possono essere mostrati come linee continue. Gli archi mancanti come linee tratteggiate o warning. Gli archi indesiderati come linee evidenziate in errore. Questa visualizzazione topologica è fondamentale per pattern come linee, cerchi, reticoli, ponti e strutture complesse. Senza edge visualization, l'utente vede solo punti nello spazio e perde il significato strutturale.

Una visualizzazione topologica può distinguere:

edge corretto;

edge atteso ma mancante;

edge presente ma non previsto;

edge instabile;

edge critico;

edge opzionale.

Questa distinzione è molto utile anche per la correzione degli errori. Se un ponte fallisce perché manca un `bridge_edge`, la dashboard lo mostra subito. Se una griglia ha una cella corretta ma una connessione mancante, lo si vede. Il web diventa uno strumento diagnostico, non solo estetico.

L'implementazione web deve mostrare il pattern target. Per una linea, può mostrare l'asse target e i punti desiderati. Per un cerchio, può mostrare centro, raggio e posizioni target. Per una griglia, può mostrare celle. Per un cluster, può mostrare area e raggio massimo. Per una struttura complessa, può mostrare sub-pattern e gruppi. Questo permette di confrontare stato reale e stato atteso. La dashboard dovrebbe sempre aiutare a rispondere alla domanda: "quanto siamo vicini al pattern desiderato?"

Una visualizzazione target può includere:

target positions;

target shape overlay;

expected topology;

tolerance zones;

error vectors;

pattern boundary;

cluster regions;

substructure labels.

Gli error vectors sono particolarmente utili in debug: mostrano dal nodo alla sua posizione target. In demo possono essere nascosti per non appesantire. Questa distinzione tra demo mode e debug mode è importante. Una dashboard tecnica deve poter mostrare molti dettagli, ma una presentazione deve restare pulita.

Il web layer deve includere una HUD, cioè una zona sintetica con le metriche principali. Questa HUD può mostrare numero di nodi, pattern corrente, fase corrente, aggregation_score, pattern_error, topology_score, safety_status, energy_status, thermal_status, sync_status e step della simulazione. La HUD aiuta a capire in pochi secondi lo stato del sistema. Nei tuoi siti MicroBot, questa logica HUD è molto coerente con l'estetica tecnico-futuristica già usata: pannelli, valori, indicatori, stato runtime, glow, griglia, terminale e metriche.

Una HUD può contenere:

runtime mode;

node_count;

active_nodes;

offline_nodes;

pattern_id;

phase_id;

aggregation_score;

pattern_error;

topology_score;

safety_level;

energy_level;

thermal_level;

simulation_step;

last_update_time.

Questi valori devono essere aggiornati in tempo reale o quasi. Se la dashboard mostra dati vecchi, deve indicarlo. Un campo last_update_time o snapshot_age_ms permette di capire se lo stato è fresco o stale. Questo è importante perché una dashboard con dati vecchi può ingannare l'utente.

L'implementazione web deve mostrare lo stato del Safety Layer. Dopo tutta la Sezione 11, è fondamentale che la dashboard abbia una sezione dedicata a sicurezza, energia, temperatura e recovery. Ogni nodo deve poter mostrare current_state, safety_state, thermal_state, energy_state, cooldown, watchdog status e recovery status. A livello globale, la dashboard deve mostrare se il sistema è in normal, warning, degraded, safety o recovery. Questo rende la sicurezza parte dell'interfaccia, non un dettaglio nascosto.

Una sezione Safety può mostrare:

numero nodi in thermal_warning;

numero nodi in thermal_lock;

numero nodi in low_energy;

numero nodi in critical_energy;

nodi in recovery;

emergency_stop status;

ultimo safety event;

comandi bloccati;

modalità operativa corrente.

Questi elementi sono essenziali soprattutto quando si collegano componenti fisici. In modalità solo simulazione possono essere virtuali, ma già utili per testare il comportamento. In modalità hardware, diventano indispensabili.

L'implementazione web deve supportare una console eventi. La console eventi mostra cosa sta accadendo: pattern_started, role_assigned, node_offline, sync_degraded, thermal_warning, command_rejected, recovery_started, fallback_triggered, pattern_completed. Questa console è diversa dai log completi. I log completi possono essere lunghi e salvati su file; la console mostra gli eventi recenti importanti. È utile per capire il flusso della run.

Una console eventi può mostrare:

timestamp;

severity;

event_type;

node_id se presente;

message breve;

phase_id;

action_taken.

Esempio:

```
[12:04:31] WARNING — MB-004 thermal_warning — duty limit reduced
[12:04:34] INFO — pattern line_basic_v0.2 phase_align started
[12:04:37] ERROR — MB-002 missing_edge with MB-003
[12:04:38] INFO — correction retry_attach attempt 1/2
```

Questa console rende il comportamento narrativo. L'utente vede il sistema prendere decisioni.

Il web layer deve supportare controlli sicuri. In una dashboard di simulazione, l'utente può avviare pattern, cambiare numero di nodi, selezionare scenari, attivare fault injection, resettare la

simulazione o salvare report. In una dashboard collegata a hardware reale, i controlli devono essere più prudenti. Comandi come emergency_stop devono essere sempre disponibili. Comandi ad alto consumo devono richiedere stato hardware-safe, conferma e limiti. In modalità read-only, non deve essere possibile inviare comandi.

Una sezione controlli può includere:

start simulation;

pause simulation;

reset simulation;

select pattern;

start pattern;

inject fault;

clear fault;

export snapshot;

generate report;

emergency_stop;

safe_idle_all.

Per hardware, i controlli ad alto rischio devono essere separati e protetti. Per esempio, magnetic_pulse test non deve essere un pulsante casuale vicino a start simulation. Deve essere in diagnostic mode, con parametri limitati e conferma. Questo è coerente con il Safety Layer.

L'implementazione web deve distinguere chiaramente tra simulazione e hardware reale. Se la dashboard mostra nodi simulati e fisici insieme, deve indicare il tipo di ciascun nodo. Un nodo simulato può avere badge SIM. Un nodo fisico può avere badge HW. Un nodo ibrido o virtual shadow può avere badge HYBRID. Questa distinzione evita confusione. Non bisogna far credere che 30 nodi fisici esistano se sono simulati. Allo stesso tempo, è molto positivo mostrare che alcuni nodi reali partecipano al sistema.

Una classificazione visuale può essere:

SIM: nodo virtuale;

HW: nodo fisico reale;

HYBRID: nodo fisico associato a rappresentazione simulata;

OFFLINE: nodo non disponibile;

GHOST: nodo presente in ultimo snapshot ma non aggiornato.

Questa distinzione è importante per credibilità. La dashboard può essere futuristica e bella, ma deve restare onesta tecnicamente.

L'implementazione web può essere realizzata con tecnologie semplici o avanzate. Una prima versione può usare HTML, CSS e JavaScript puro, con canvas 2D o SVG. Questo è sufficiente per disegnare nodi, linee, HUD e pannelli. Una versione più avanzata può usare Three.js per visualizzazione 3D, WebSocket per aggiornamenti live, framework frontend per stato complesso, o integrazione con backend Python/Flask/FastAPI. La scelta deve dipendere dalla fase del progetto. Non serve complicare subito tutto se una dashboard HTML/JS funziona bene.

Una progressione realistica è:

v0.1: pagina HTML statica con dati finti;

v0.2: lettura di snapshot JSON;

v0.3: canvas 2D con nodi live;

v0.4: pannelli HUD e metriche;

v0.5: event console;

v0.6: controlli simulazione;

v0.7: WebSocket live;

v0.8: integrazione backend;

v0.9: modalità hardware/simulazione;

v1.0: dashboard completa con report e replay.

Questa roadmap è coerente con lo sviluppo MicroBot. Permette di partire da qualcosa di visibile e poi aumentare integrazione.

Il backend web può avere responsabilità diverse. Può servire file statici, esporre endpoint API, leggere snapshot, mantenere stato runtime, ricevere telemetria dai nodi, inviare comandi al controller o agire esso stesso da controller leggero. Per evitare confusione, è utile separare backend dashboard e controller logic. Il backend web mostra e distribuisce dati; il controller decide comandi e pattern. In prototipi semplici possono coincidere, ma nella documentazione è meglio distinguerli.

Una possibile architettura è:

simulation_engine produce stato;

controller_runtime gestisce pattern e comandi;

web_backend espone API e WebSocket;

web_frontend visualizza e invia richieste sicure;

hardware_nodes comunicano con controller o backend secondo protocollo.

Questa architettura evita che il frontend diventi responsabile della logica critica. Il browser non deve decidere se una bobina è sicura; il controller e il nodo devono farlo. Il browser può chiedere, mostrare e inviare comandi autorizzati, ma la validazione vera resta nel runtime e nel firmware.

Gli endpoint web possono essere progettati in modo semplice. Per esempio:

GET /api/status restituisce stato generale;

GET /api/snapshot restituisce snapshot corrente;

GET /api/nodes restituisce lista nodi;

GET /api/events restituisce eventi recenti;

POST /api/pattern/start avvia un pattern simulato;

POST /api/simulation/reset resetta simulazione;

POST /api/emergency_stop invia stop globale;

GET /api/report/latest restituisce report ultimo.

Questi endpoint devono essere documentati. Anche se non vengono implementati tutti subito, definire la struttura aiuta a mantenere coerenza. In modalità hardware, gli endpoint che inviano comandi devono rispettare safety e autorizzazioni.

Il WebSocket può essere usato per aggiornamenti live. Invece di fare polling continuo, il backend può inviare snapshot_delta o eventi al frontend. Questo rende la dashboard più fluida. Tuttavia, WebSocket aggiunge complessità. In una prima versione, polling ogni 250–1000 ms può essere sufficiente. La scelta dipende dall'obiettivo: demo semplice o runtime live. La documentazione può prevedere entrambi.

Polling:

semplice;

facile da debuggare;

adatto a snapshot JSON;

meno efficiente se frequenza alta.

WebSocket:

live;

più reattivo;

adatto a eventi;

più complesso da gestire.

Una piattaforma MicroBot matura può usare WebSocket per eventi e snapshot, ma non è necessario iniziare da lì.

L'implementazione web deve gestire stati stale. Se il frontend non riceve aggiornamenti da troppo tempo, deve mostrare snapshot_stale. Se un nodo non viene aggiornato, deve diventare stale o offline. Se il backend perde connessione con simulatore, deve mostrare warning. Questo evita di visualizzare dati vecchi come se fossero attuali. In un sistema con hardware, questo è particolarmente importante.

Una regola web può essere:

se `snapshot_age_ms > soglia`, mostrare warning “snapshot stale”.

Per i nodi:

se `now - last_update < soglia_ok`, nodo online;

se supera `soglia_stale`, nodo stale;

se supera `soglia_offline`, nodo offline.

Questa logica deve essere coerente con quella del controller. La dashboard non deve inventare stati incompatibili, ma può visualizzarli chiaramente.

L'implementazione web deve supportare replay. Un replay permette di rivedere una run registrata. Questo è utile per analisi, presentazioni e debug. Il frontend può caricare una sequenza di snapshot o eventi e riprodurli con controlli play, pause, speed, step. In un replay si possono osservare pattern, errori, recovery e fallback. Questa funzione è molto potente per una tesi o una demo, perché permette di mostrare un comportamento anche senza eseguire live la simulazione.

Una modalità replay può includere:

lista snapshot;

timeline;

controlli play/pause;

velocità riproduzione;

eventi sincronizzati;

metriche nel tempo;

salto a errore o fase specifica.

Questa modalità trasforma i log in materiale visuale. Una run interessante può essere salvata e mostrata più volte.

L'implementazione web deve supportare report visuali. Dopo una run, la dashboard può mostrare riepilogo: pattern, risultato, metriche finali, errori, warning, recovery, consumo energetico stimato, thermal events e grafici. Questo report può essere esportato come JSON o Markdown. In futuro anche PDF, ma la base deve essere strutturata. Il report aiuta a trasformare ogni simulazione in un esperimento documentato.

Un report web può mostrare:

run summary;

pattern result;

node participation;

metric curves;

safety events;
energy events;
thermal events;
recovery events;
final classification;
recommendations.

Questa funzione è fondamentale per validazione del comportamento.

Il frontend deve essere coerente con l'estetica MicroBot. Dato che il progetto ha già una forte identità visiva dark, techno, HUD, grid, glow, pannelli e metriche, l'implementazione web dovrebbe mantenere questa direzione. Tuttavia, l'estetica non deve compromettere leggibilità. Colori, glow e animazioni devono aiutare a capire lo stato, non nascondere informazioni. Un'interfaccia bella ma confusa è meno utile di una più semplice ma chiara. L'obiettivo migliore è un design tecnico, pulito, scuro, con pannelli leggibili e codifica cromatica coerente.

Una UI coerente può usare:

sfondo scuro;
griglia sottile;
nodi con glow controllato;
pannelli semitrasparenti;
font tecnico leggibile;
colori coerenti per ruoli e stati;
animazioni leggere;
HUD con metriche reali;
console eventi in stile terminale.

La coerenza cromatica è importante. Se il rosso significa safety, non deve essere usato anche per ruolo normale. Se il verde significa ok o connector, deve restare coerente. Se il giallo significa warning, non deve essere usato casualmente. La UI deve diventare linguaggio.

L'implementazione web deve distinguere colori di ruolo e colori di stato. Questo è un problema sottile. Un nodo può essere connector e contemporaneamente thermal_warning. Se il colore principale indica ruolo, come mostrare warning? Una soluzione è usare colore interno per ruolo e bordo o badge per stato. Per esempio, il riempimento del nodo indica role; il bordo indica safety_state; una piccola icona indica energy_state. Così non si perde informazione.

Una codifica possibile è:

riempimento nodo = ruolo;

bordo nodo = safety/thermal status;

badge piccolo = energia;

opacità = online/stale/offline;

linea = topologia;

label = node_id.

Questa codifica rende il sistema più leggibile. Evita conflitti tra significati.

Il web layer deve essere accessibile anche come documentazione vivente. Una dashboard può includere pannelli descrittivi che spiegano pattern corrente, significato delle metriche, stato safety e modalità. Questo è utile per chi non conosce il sistema. Una sezione “About this run” può spiegare: pattern line_basic, 8 nodi, obiettivo: allineamento lungo asse, metriche: lateral_error e spacing_error, safety: nessuna attuazione reale. Questa integrazione tra documentazione e runtime è molto potente.

Un pannello descrittivo può contenere:

pattern description;

implementation mode;

what is simulated;

what is real;

current limitations;

success criteria;

safety notes.

Questo mantiene trasparenza. Una demo non deve lasciare dubbi su cosa sia reale e cosa simulato.

L'implementazione web deve essere progettata per debug progressivo. Quando qualcosa non funziona, la dashboard deve aiutare a capire dove. Se un nodo non appare, è problema snapshot, backend, simulatore o nodo? Se un pattern non parte, è problema comando o readiness? Se una metrica non cambia, è problema simulazione o visualizzazione? Per questo, il web layer deve mostrare stato delle connessioni interne: backend online, snapshot age, WebSocket status, simulation running, last event, error count.

Un pannello system status può mostrare:

frontend loaded;

backend reachable;

snapshot received;

snapshot_age_ms;

simulation_status;

websocket_status;

last_event_time;

api_error_count.

Questi dettagli sono importanti per sviluppo. Senza, si perde tempo a capire se il problema è logico o tecnico.

L'implementazione web deve essere sicura rispetto ai comandi. Anche se il progetto è locale, bisogna evitare che pulsanti o endpoint inviino comandi pericolosi senza controllo. In fase iniziale, si può limitare tutto a simulazione. Per hardware, si devono aggiungere conferme e modalità. Per esempio, la dashboard può avere un interruttore "hardware commands enabled" che di default è off. Emergency_stop deve essere sempre accessibile. I comandi attuativi devono richiedere diagnostic_mode e parametri entro limiti.

Una regola web è:

la dashboard può facilitare comandi, ma non deve bypassare validazioni di controller e nodo.

Anche se il frontend invia un comando sbagliato, il backend/controller e il firmware devono bloccarlo. La sicurezza non deve dipendere dal browser.

L'implementazione web deve permettere modalità read-only. Questa è utile per presentazioni, osservatori esterni o demo pubbliche. In read-only, la dashboard mostra stato e replay ma non invia comandi. Questo evita rischi durante una presentazione. In modalità developer, invece, si possono avere controlli avanzati. La distinzione aiuta a usare la stessa piattaforma in contesti diversi.

Modalità possibili:

read_only;

demo_control;

developer_control;

diagnostic_control;

hardware_safe_control.

Ogni modalità abilita funzioni diverse. Questo è molto professionale e coerente con un sistema estensibile.

L'implementazione web deve supportare la crescita futura verso visualizzazione 3D. I primi pattern possono essere 2D, ma MicroBot ha una visione legata anche a strutture più complesse, superfici, configurazioni e magari visualizzazioni tridimensionali. Una dashboard 2D è sufficiente per linee, cerchi, cluster e griglie. Una dashboard 3D può diventare utile per strutture avanzate, simulazioni volumetriche, campi, superfici e integrazioni AR/VR. La roadmap web può quindi prevedere un passaggio da canvas 2D a Three.js o visualizzazione 3D.

Una roadmap visuale può essere:

canvas 2D per pattern base;

SVG/canvas overlay per topologia e metriche;

timeline e replay;

Three.js per nodi 3D;

visualizzazione campi e superfici;

modalità AR/VR futura.

Non serve partire dal 3D se il 2D spiega meglio i pattern iniziali. Ma il progetto può prevedere questa crescita.

L'implementazione web deve essere collegata agli altri progetti esterni quando sarà il momento. Nelle sezioni successive parlerai di progetti esterni e integrazione. Alcuni progetti web, simulazioni, AR hand tracking, black hole simulation, clustering e dashboard possono fornire tecniche utili: visualizzazione particellare, interfacce HUD, modelli 3D, telemetria, interazione e report. Tuttavia, nella Sezione 12 bisogna restare concentrati su MicroBot. L'integrazione con altri progetti va citata come possibilità futura, non mescolata subito in modo caotico.

Una formulazione corretta è:

L'implementazione web MicroBot può riutilizzare competenze e componenti sviluppati in altri progetti, ma deve mantenere una struttura specifica per lo swarm.

Questo evita dispersione. Il web layer deve essere MicroBot-first.

L'implementazione web deve avere file e cartelle ordinati. Una possibile struttura può essere:

web/

index.html;

dashboard.html;

assets/css/style.css;

assets/js/app.js;

assets/js/render_swarm.js;

assets/js/api_client.js;

assets/js/events_panel.js;

assets/js/metrics_panel.js;

assets/js/controls.js;

data/live_swarm_snapshot.json;

data/sample_snapshot.json;

README.md.

In una versione più avanzata:

backend/

app.py;

routes/status.py;

routes/snapshot.py;

routes/patterns.py;

routes/events.py;

websocket_manager.py;

static/

templates/

Questa organizzazione aiuta a separare frontend e backend. Non tutto deve essere implementato subito, ma la struttura deve essere chiara.

Il file README dell'implementazione web dovrebbe spiegare come avviare la dashboard, quali dati legge, quali endpoint usa, quali modalità supporta, quali limiti ha e come distinguere simulazione da hardware. Deve anche indicare se la dashboard è solo visuale o se può inviare comandi. Questa documentazione è essenziale per chi apre la repo.

Un README web dovrebbe includere:

scopo;

architettura;

modalità;

input dati;

snapshot format;

avvio locale;

controlli disponibili;

sicurezza;

limitazioni;

roadmap.

Questo rende la dashboard parte del progetto, non un file isolato.

L'implementazione web deve essere testabile. Si possono creare snapshot di esempio e verificare che la dashboard li visualizzi correttamente. Si può testare un nodo offline, un nodo in safety, una connessione mancante, un thermal_warning, un pattern linea e un pattern cluster. Questo permette

di sviluppare frontend anche senza simulatore attivo. Un `sample_snapshot.json` è molto utile per questo.

Test frontend utili:

caricamento snapshot valido;

snapshot mancante;

snapshot stale;

nodo offline;

nodo in `low_energy`;

nodo in `thermal_lock`;

edge mancante;

`pattern_error` alto;

event console piena;

modalità `read-only`.

Questi test rendono il web layer più affidabile. Una dashboard che funziona solo con dati perfetti non basta.

L'implementazione web deve essere compatibile con iterazione veloce. Durante sviluppo, l'utente deve poter modificare simulazione e vedere cambiamenti rapidamente. Se ogni modifica richiede configurazioni complicate, il progetto rallenta. Una pagina locale servita con `python3 -m http.server` può essere sufficiente per frontend statico. Un backend leggero può essere aggiunto quando servono API. L'obiettivo è non bloccare la creatività, ma mantenere struttura.

Una modalità semplice di sviluppo può essere:

simulatore scrive snapshot JSON;

server statico serve dashboard;

frontend legge snapshot;

utente modifica config;

simulatore ricarica;

dashboard aggiorna.

Questa pipeline è semplice e potente. Per prototipazione MicroBot va benissimo. Solo quando serve interazione live più complessa si passa a backend completo.

L'implementazione web deve essere considerata parte della validazione. Se la dashboard mostra metriche, pattern e safety, può aiutare a decidere se una run è valida. Tuttavia, la validazione non deve dipendere solo dall'occhio umano. La dashboard mostra; le metriche misurano. L'utente vede,

ma il report registra. Questa distinzione è importante. Una bella animazione non è una prova sufficiente. Serve metriche e log.

Una regola è:

la dashboard rende visibile la run; le metriche e i log la rendono verificabile.

Questo principio deve guidare tutta l'implementazione web. Estetica e rigore devono stare insieme.

In conclusione, l'implementazione web è il livello che rende la simulazione MicroBot osservabile, interattiva e comunicabile. Essa deve visualizzare nodi, ruoli, pattern, connessioni, metriche, safety, energia, temperatura, eventi e recovery; deve distinguere simulazione e hardware; deve supportare snapshot, API, eventualmente WebSocket, replay, report e controlli sicuri; deve mantenere un'estetica coerente con l'identità MicroBot ma senza sacrificare chiarezza tecnica. La dashboard web non è un semplice ornamento: è lo strumento attraverso cui il sistema diventa leggibile, debuggabile e presentabile. Essa collega simulatore, controller, hardware, log e documentazione. Il punto successivo, dedicato alla visualizzazione dei MicroBot, entrerà nel dettaglio di come rappresentare graficamente i nodi, le connessioni, i ruoli, i pattern e gli stati operativi in modo chiaro, coerente e utile sia per sviluppo sia per dimostrazione.

12.3 Visualizzazione dei MicroBot

La visualizzazione dei MicroBot rappresenta il livello grafico e concettuale attraverso cui i nodi dello swarm diventano leggibili all'interno dell'ambiente web e della simulazione software. Dopo aver definito l'implementazione web come piattaforma di osservazione, controllo e presentazione, è necessario entrare nel dettaglio di come i singoli MicroBot debbano essere rappresentati sullo schermo. La visualizzazione non deve limitarsi a mostrare dei punti che si muovono. Ogni nodo deve comunicare identità, stato, ruolo, appartenenza al pattern, condizione energetica, condizione termica, safety, sincronizzazione, connessioni e qualità operativa. In un sistema distribuito, vedere "dove sono i nodi" non basta; bisogna capire "che cosa sono", "che cosa stanno facendo", "in che stato si trovano" e "quanto sono coerenti con il pattern".

Nel progetto MicroBot, la visualizzazione dei nodi deve essere pensata come una traduzione visiva dell'architettura. Il `node_id` diventa una label. Il ruolo diventa colore o forma. Lo stato safety diventa bordo, badge o warning. La connessione topologica diventa linea. Il target diventa punto o area di riferimento. L'errore diventa vettore o distanza evidenziata. La telemetria diventa pannello informativo. Il tempo diventa animazione o timeline. La simulazione, quindi, non mostra soltanto una scena: mostra il sistema. Questa distinzione è fondamentale. Una visualizzazione decorativa può essere bella, ma una visualizzazione tecnica deve rendere il comportamento verificabile.

Il primo elemento da rappresentare è l'identità del MicroBot. Ogni nodo deve avere un identificatore visibile o accessibile, come SIM-001, MB-001, HW-003 o un nome coerente con la struttura del sistema. L'identità deve essere stabile. Se un nodo cambia ruolo, colore o stato, il suo `node_id` resta lo stesso. Questo permette all'utente di seguirne la storia durante una run. Senza identità, i nodi diventano pallini anonimi e diventa impossibile capire quale nodo ha generato un errore, quale ha cambiato ruolo, quale è entrato in `low_energy` o quale è stato recuperato dopo un reset.

In modalità demo, la label del `node_id` può essere mostrata solo quando si passa sopra con il mouse o quando il nodo è selezionato. In modalità debug, invece, può essere sempre visibile. Questa distinzione evita di appesantire la visualizzazione durante presentazioni, ma mantiene accesso alle

informazioni quando serve analisi tecnica. Per pochi nodi, le label sempre visibili sono utili. Per molti nodi, possono diventare caotiche. Il sistema deve quindi permettere di attivarle o disattivarle.

Una modalità di visualizzazione può prevedere:

label sempre visibili per swarm piccoli;

label su hover per swarm medi;

label solo su selezione per swarm grandi;

ricerca node_id tramite pannello laterale;

evidenziazione del nodo selezionato.

Questo approccio rende la dashboard scalabile. Una visualizzazione adatta a cinque nodi non è necessariamente adatta a cento nodi. La UI deve poter cambiare densità informativa.

Il secondo elemento da rappresentare è il ruolo del nodo. Come definito nella Sezione 10, un MicroBot può essere leader, anchor, connector, boundary_node, core_node, bridge_node, terminal_node, stabilizer, seeker, passive_node, diagnostic_node o actuator_node. Il ruolo deve essere immediatamente riconoscibile. La soluzione più naturale è usare il colore principale del nodo per rappresentare il ruolo operativo. Per esempio, leader può avere un colore dominante, connector un altro, terminal un altro, core un altro. Tuttavia, questa codifica deve essere coerente e non deve sovrapporsi agli stati di sicurezza.

Il problema principale è che un nodo possiede più stati contemporaneamente. Può essere connector e nello stesso tempo thermal_warning. Può essere leader e sync_degraded. Può essere passive_node e low_energy. Se si usa un solo colore per tutto, si perde informazione. Per questo motivo, la visualizzazione deve separare graficamente ruolo e stato. Una scelta efficace è usare il riempimento del nodo per il ruolo, il bordo per la safety, un badge per energia o temperatura, e l'opacità per online/offline. In questo modo, più informazioni coesistono senza confondersi.

Una codifica semantica possibile è:

riempimento del nodo = ruolo;

bordo del nodo = stato safety o errore critico;

badge superiore = stato energetico;

badge inferiore = stato termico;

opacità = freschezza/online/offline;

glow o pulsazione = attività corrente;

linea verso target = errore o direzione di correzione.

Questa separazione è molto importante. Se il rosso indica safety, non deve indicare anche un ruolo normale. Se il giallo indica warning, non deve essere usato casualmente per estetica. La visualizzazione deve essere un linguaggio tecnico.

Il terzo elemento è lo stato operativo del nodo. Ogni MicroBot può essere idle, active, diagnostic, safety, recovery, offline, stale, low_energy o thermal_lock. Questi stati devono essere leggibili. Un nodo offline può essere grigio, trasparente o con contorno tratteggiato. Un nodo stale può essere sbiadito ma ancora visibile. Un nodo in safety deve essere evidenziato in modo forte. Un nodo in recovery può avere animazione o bordo speciale. Un nodo active può pulsare leggermente. Questi segnali aiutano l'utente a capire lo swarm senza dover leggere ogni valore numerico.

Una rappresentazione degli stati può essere:

online/active: nodo pienamente visibile;

idle: nodo visibile ma con animazione ridotta;

stale: nodo semitrasparente;

offline: nodo grigio o quasi spento;

safety: bordo rosso o segnale evidente;

recovery: bordo lampeggiante lento o badge recovery;

diagnostic: icona o badge tecnico;

low_energy: badge batteria;

thermal_warning: badge termico;

thermal_lock: badge termico critico e attuazioni bloccate.

Questi elementi devono essere accompagnati da una legenda. Una dashboard tecnica senza legenda può diventare difficile da interpretare. La legenda deve spiegare colori, bordi, badge e linee. In modalità demo, la legenda può essere compatta; in modalità debug, completa.

Il quarto elemento è la posizione del nodo. Nella simulazione, ogni nodo possiede coordinate virtuali. Queste coordinate possono essere rappresentate in 2D o 3D. Per i pattern iniziali, una visualizzazione 2D è sufficiente e probabilmente migliore: linee, cerchi, griglie e cluster sono più leggibili su un piano. La posizione deve essere coerente con lo spazio del simulatore. Se il simulatore usa coordinate normalizzate, il frontend deve convertirle correttamente in coordinate canvas. Se si usa zoom o pan, la posizione logica deve restare separata dalla posizione sullo schermo.

Una pipeline corretta è:

posizione simulata nel modello;

trasformazione spazio simulazione → spazio visuale;

applicazione di zoom, pan e scala;

rendering del nodo;

eventuale rendering di label, connessioni e target.

Questa separazione evita errori. La posizione fisica o simulata non deve essere confusa con il pixel sullo schermo. Questo sarà utile se in futuro si aggiunge replay, zoom, mappa 3D o confronto tra coordinate reali e virtuali.

Il quinto elemento è il target del nodo. Nei pattern target-based, ogni MicroBot può avere una posizione obiettivo. Visualizzare il target aiuta a capire se il nodo sta convergendo. Il target può essere mostrato come piccolo punto, croce, cerchio fantasma o marker. Una linea sottile può collegare nodo e target, indicando l'errore. In modalità demo, questi elementi possono essere nascosti per non appesantire. In modalità debug, sono estremamente utili.

Una visualizzazione target può mostrare:

target_position;

tolerance_radius;

error_vector;

distance_to_target;

target_index;

logical_position nel pattern.

Per esempio, in una linea, il target mostra dove ogni nodo dovrebbe stare lungo l'asse. In un cerchio, mostra la posizione angolare ideale. In una griglia, mostra la cella. Se il nodo è lontano dal target, l'errore è visibile immediatamente. Questo aiuta a diagnosticare problemi di convergenza.

Il sesto elemento è il raggio di interazione. In molti modelli swarm, un nodo interagisce con vicini entro una certa distanza. Visualizzare questo raggio può essere utile in debug. Per esempio, un cerchio semitrasparente intorno al nodo può indicare communication_radius, connection_radius, separation_radius o magnetic_range. Tuttavia, se tutti i nodi mostrano raggi, la visualizzazione può diventare molto carica. Quindi questi overlay devono essere attivabili.

Tipi di raggio visualizzabili:

communication_radius;

neighbor_detection_radius;

separation_radius;

connection_radius;

magnetic_effect_radius;

safety_distance.

Questi overlay sono particolarmente utili per capire perché un nodo non forma connessioni, perché un cluster resta disperso o perché si verifica una collisione simulata. In modalità presentazione, meglio nasconderli; in modalità analisi, sono preziosi.

Il settimo elemento è la connessione tra nodi. Le connessioni sono il modo in cui la topologia del pattern diventa visibile. Una connessione può rappresentare un edge atteso, una connessione

virtuale corrente, un aggancio simulato, una comunicazione, una relazione di vicinato o un vincolo strutturale. La visualizzazione deve distinguere questi tipi. Se tutte le linee sono uguali, si perde significato. Un edge atteso ma mancante non è uguale a un edge reale corretto. Un edge opzionale non è uguale a un bridge_edge critico.

Una codifica topologica può essere:

linea continua: edge presente e corretto;

linea tratteggiata: edge atteso ma mancante;

linea sottile: edge opzionale;

linea spessa: edge critico;

linea lampeggiante: edge instabile;

linea di errore: edge indesiderato;

linea curva o speciale: connessione temporanea o transition_edge.

Questa distinzione permette di leggere strutture complesse. In un ponte, il bridge_edge può essere evidenziato. In un cerchio, la chiusura tra ultimo e primo può essere mostrata chiaramente. In una griglia, si possono evidenziare celle incomplete. Il grafo dello swarm diventa parte della visualizzazione.

L'ottavo elemento è la fase corrente del pattern. La visualizzazione dei MicroBot deve cambiare in base alla fase. Durante prepare, i nodi possono essere selezionati ma non ancora attivi. Durante assign_roles, i colori dei ruoli possono apparire. Durante align, i target e gli error_vector possono essere mostrati. Durante attach, gli edge attesi possono essere evidenziati. Durante stabilize, le oscillazioni e la stabilità possono essere osservate. Durante report, la dashboard può congelare o riassumere lo stato finale.

Una visualizzazione phase-aware può essere:

prepare: nodi partecipanti evidenziati;

assign_roles: transizione cromatica dei ruoli;

align: target e vettori errore visibili;

attach: edge e connessioni evidenziati;

stabilize: indicatore stabilità e movimento ridotto;

report: metriche finali e classificazione run.

Questo rende la simulazione più narrativa e più comprensibile. L'utente vede non solo il risultato, ma la procedura. Questo è importante per spiegare MicroBot come sistema coordinato, non come animazione casuale.

Il nono elemento è il movimento dei nodi. Il movimento deve essere leggibile, non necessariamente iperrealistico nelle prime versioni. I nodi possono muoversi verso target, reagire ai vicini, formare cluster o correggere errori. La visualizzazione può mostrare velocità, traiettorie e direzione. Le

tracce sono molto utili: mostrano da dove arriva un nodo e come si è mosso. Tuttavia, troppe tracce possono creare confusione. Anche qui serve modalità attivabile.

Una visualizzazione del movimento può includere:

velocity vector;

motion trail;

previous positions;

speed indicator;

damping visualization;

movement status.

Per esempio, un nodo che oscilla può essere riconosciuto dalla traccia. Un nodo fermo ma lontano dal target indica problema di comportamento. Un nodo che si muove troppo velocemente può indicare parametri aggressivi. La visualizzazione del movimento aiuta a interpretare la dinamica.

Il decimo elemento è lo stato di sincronizzazione. Un nodo può essere sync_good, sync_ok, sync_degraded, sync_lost o out_of_phase. Questo stato non è sempre visibile nella forma geometrica, ma può spiegare perché un nodo non esegue correttamente. La dashboard deve renderlo accessibile. Un badge o un piccolo indicatore temporale può mostrare sync_status. In modalità debug, si possono visualizzare offset_ms, last_sync_age_ms e jitter. Questo collega la visualizzazione alla Sezione 9.

Una scheda nodo può mostrare:

sync_status;

clock_offset_ms;

last_sync_age_ms;

jitter_ms;

phase_id riportato;

expected_phase_id;

alignment_status.

Se un nodo è fuori fase, la visualizzazione può evidenziarlo. Per esempio, un bordo tratteggiato o un badge “PHASE” può indicare phase_mismatch. Questo è utile perché un nodo fuori fase può apparire fisicamente vicino al target ma non essere logicamente valido.

L’undicesimo elemento è lo stato energetico. Un nodo può essere energy_ok, energy_medium, low_energy o critical_energy. Questo deve essere visibile. Un badge batteria, una barra piccola o un indicatore laterale possono comunicare energia. In modalità debug, il pannello laterale può mostrare battery_source, battery_voltage, battery_level e energy_state. In modalità demo, basta un badge se il nodo è in low_energy o critical_energy.

La visualizzazione energetica deve distinguere tra dato simulato e dato reale. Se la batteria è simulata, il pannello deve indicarlo. Se è misurata, deve indicarlo. Questo evita confusioni. Per esempio:

battery_source = simulated;

energy_state = low_energy;

disabled_capabilities = coil_pwm.

Questi dati spiegano perché un nodo può essere escluso da un ruolo attivo.

Il dodicesimo elemento è lo stato termico. thermal_ok, thermal_warning, thermal_lock e thermal_recovery devono essere rappresentati. Un nodo in thermal_warning può avere badge termico giallo. Un nodo in thermal_lock può avere bordo critico e attuazioni bloccate. In modalità debug, la dashboard può mostrare thermal_load, cooldown_remaining_ms, last_pulse e blocked_capabilities. Questo collega la visualizzazione alla protezione delle bobine e alla prevenzione del surriscaldamento.

Una visualizzazione termica può includere:

thermal badge;

thermal_load bar;

cooldown ring intorno al nodo;

coil_state icon;

last thermal event.

Il cooldown ring è una soluzione interessante: un anello intorno al nodo può svuotarsi o riempirsi per indicare quanto manca alla riattivazione. Questo rende immediatamente comprensibile perché un comando è bloccato. Naturalmente, non deve essere troppo invasivo.

Il tredicesimo elemento è il safety state. Safety deve avere priorità visiva. Se un nodo è in safety, emergency_stop, fault o recovery critica, la dashboard deve mostrarlo chiaramente. Non deve essere un dettaglio nascosto in un pannello. Lo stato safety può essere rappresentato con bordo spesso, icona, badge e voce nella console eventi. Un nodo in safety non deve confondersi con un nodo normale.

Stati safety visualizzabili:

safety_ok;

safety_warning;

safety_lock;

emergency_stopped;

recovery_required;

manual_intervention_required.

Questi stati devono avere significati precisi. `safety_warning` indica attenzione. `safety_lock` indica blocco. `emergency_stopped` indica stop globale o nodo fermato. `recovery_required` indica che il nodo non può rientrare senza procedura. `manual_intervention_required` indica che non basta recovery automatica. Questa semantica deve essere coerente con i log.

Il quattordicesimo elemento è il pannello di dettaglio del nodo. Quando l'utente seleziona un MicroBot, deve apparire una scheda con tutte le informazioni principali. Questa scheda è fondamentale per debug. La visualizzazione globale mostra lo swarm; il pannello nodo mostra il singolo agente. Deve includere identità, stato, ruolo, posizione, target, errori, safety, energia, temperatura, sync, capabilities, ultimo comando e ultimi eventi.

Una scheda nodo completa può contenere:

`node_id;`

`node_type: simulated, hardware, hybrid;`

`mode;`

`role;`

`group_id;`

`pattern_run_id;`

`phase_id;`

`position;`

`target_position;`

`distance_to_target;`

`sync_status;`

`energy_state;`

`thermal_state;`

`safety_state;`

`capabilities;`

`blocked_capabilities;`

`last_command_id;`

`last_ack;`

`last_error;`

`recent_events.`

Questa scheda rende il sistema ispezionabile. Se qualcosa non va, si seleziona il nodo e si vede perché. Questo è essenziale per sviluppo serio.

Il quindicesimo elemento è la selezione e l'evidenziazione. L'utente deve poter cliccare un nodo, un edge o un gruppo. Quando un nodo è selezionato, le sue connessioni possono essere evidenziate, i suoi target mostrati, i suoi vicini marcati e il pannello aggiornato. Quando un edge è selezionato, la dashboard può mostrare edge_id, stato, criticità, nodi coinvolti e errori. Quando un gruppo è selezionato, può mostrare cluster, sub-pattern o struttura.

Funzioni utili:

click nodo;

hover nodo;

click edge;

seleziona gruppo;

filtra per ruolo;

filtra per stato;

cerca node_id;

isola sottostruttura.

Queste funzioni diventano importanti quando lo swarm cresce. Con pochi nodi si può guardare tutto insieme. Con molti, bisogna filtrare.

Il sedicesimo elemento è la rappresentazione dei gruppi e dei sub-pattern. Una struttura complessa può contenere cluster_A, cluster_B, bridge_chain, outer_ring, inner_ring, grid_row_1, boundary_nodes. La visualizzazione deve poter mostrare questi gruppi. Può usare aree semitrasparenti, etichette, colori di gruppo o pannelli laterali. Questo aiuta a capire strutture complesse. Senza gruppi, un ponte o un reticolo può sembrare solo un insieme di nodi.

Una visualizzazione di gruppo può includere:

group boundary;

group label;

subpattern color;

group metrics;

group error;

group safety summary.

Per esempio, in un ponte, cluster_A e cluster_B possono essere aree diverse, mentre bridge_chain è evidenziata. In una griglia, righe e colonne possono essere indicate. In un anello rinforzato, outer_ring e inner_ring possono essere distinguibili. Questo rende la struttura comprensibile.

Il diciassettesimo elemento è la rappresentazione degli errori. Se un nodo ha errore geometrico alto, la dashboard deve indicarlo. Se un edge è mancante, deve essere visibile. Se un nodo è

out_of_phase, deve apparire. Gli errori non devono essere nascosti nella console. La visualizzazione deve aiutare a identificare rapidamente dove il pattern fallisce.

Errori visualizzabili:

node_position_error;

spacing_error;

missing_edge;

extra_edge;

role_mismatch;

phase_mismatch;

sync_degraded;

thermal_warning;

low_energy;

command_rejected.

Una soluzione efficace è usare overlay attivabili: error overlay mostra tutte le anomalie. In modalità normale, la dashboard resta pulita. In modalità error/debug, evidenzia problemi. Questo evita sovraccarico visivo.

Il diciottesimo elemento è la rappresentazione della correzione. Quando il sistema applica una correzione, la dashboard dovrebbe mostrarlo. Per esempio, se un nodo viene riallineato, può comparire un vettore di correzione. Se un edge mancante viene ritentato, l'edge può lampeggiare. Se un ruolo viene riassegnato, il nodo può cambiare colore con transizione. Se il pattern passa a fallback, il target può aggiornarsi. Questo rende visibile l'intelligenza del sistema.

Una visualizzazione di correzione può mostrare:

correction_pending;

correction_applied;

retry_count;

fallback_active;

role_reassignment animation;

state_sync indication.

Questi elementi devono essere sobri. Lo scopo è informare, non creare caos. Una console eventi parallela può spiegare l'azione in testo.

Il diciannovesimo elemento è la rappresentazione del tempo. La dashboard dovrebbe mostrare step, simulation_time, runtime, phase_time e timeline della run. Questo aiuta a capire dove ci si trova.

Una barra timeline può indicare fasi: prepare, assign_roles, align, attach, stabilize, report. Gli eventi possono apparire sulla timeline. In replay, questa timeline diventa ancora più utile.

Una timeline può mostrare:

inizio run;

transizioni di fase;

safety events;

corrections;

fallback;

recovery;

fine run.

Questo rende la simulazione analizzabile nel tempo, non solo nello spazio. MicroBot è un sistema dinamico; la visualizzazione deve rappresentare la dinamica.

Il ventesimo elemento è la rappresentazione della stabilità. Un pattern non è riuscito solo perché i nodi hanno raggiunto una forma per un istante. Deve restare stabile per un intervallo. La dashboard può mostrare stability_score, oscillation_level o stable_for_ms. Un nodo stabile può avere glow più calmo; un nodo instabile può mostrare traccia o warning. Per strutture complesse, la stabilità può essere mostrata per gruppo o edge.

Metriche visuali di stabilità:

stable_for_ms;

movement_average;

oscillation_score;

edge_stability;

pattern_stability;

node_velocity.

Questi dati aiutano a distinguere una forma momentanea da una struttura realmente stabilizzata.

Il ventunesimo elemento è la visualizzazione dei modelli virtuali. Un MicroBot simulato può essere rappresentato come punto, cerchio, piccolo modulo, icona 3D o elemento più realistico. La scelta dipende dalla fase. Per pattern e debug, forme semplici sono migliori. Per demo esterne, un modello più curato può comunicare meglio la visione. Tuttavia, anche un modello estetico deve mantenere informazione tecnica. Non deve diventare solo un oggetto decorativo.

Una progressione può essere:

punto/circolo 2D per debug;

nodo 2D con badge per dashboard;

modulo stilizzato per demo;

modello 3D per simulazioni avanzate;

rappresentazione AR/VR futura.

Per ora, il cerchio 2D con codifica semantica è probabilmente la soluzione più efficace. È leggibile, leggero e facile da integrare.

Il ventiduesimo elemento è la distinzione tra vista globale e vista locale. La vista globale mostra tutto lo swarm. La vista locale mostra un nodo o gruppo. La vista globale è utile per pattern; quella locale per debug. La dashboard deve permettere di passare tra le due. In una struttura complessa, può essere utile zoomare su un sub-pattern. Questo prepara la scalabilità.

Una UI può avere:

swarm view;

node inspector;

group inspector;

edge inspector;

timeline;

metrics panel;

event console.

Questa struttura è ordinata e professionale. L'utente può navigare il sistema senza perdersi.

Il ventitreesimo elemento è la performance. Una visualizzazione deve restare fluida. Disegnare troppi effetti, label, glow o overlay può rallentare. Con pochi nodi si può usare più dettaglio; con molti nodi serve semplificazione automatica. Questo è importante per la Sezione 19 sulla scalabilità, ma va previsto già qui. La dashboard deve degradare visivamente in modo intelligente.

Una strategia può essere:

meno di 20 nodi: dettagli completi;

20–100 nodi: label su hover, overlay selettivi;

oltre 100 nodi: aggregazione gruppi, meno glow, metriche globali;

oltre soglia alta: cluster view invece di node view completa.

Questa logica permette alla visualizzazione di crescere con il sistema.

Il ventiquattresimo elemento è l'onestà visuale. La dashboard deve distinguere chiaramente ciò che è simulato, stimato, misurato e reale. Se la posizione è simulata, va indicato. Se la temperatura è stimata, va indicato. Se un nodo è fisico ma la sua posizione è virtuale, va indicato. Questo è fondamentale per evitare confusione in documentazione, demo e presentazioni. Una visualizzazione bella ma ambigua può far sembrare reali cose che sono ancora simulate. Questo sarebbe un errore.

Ogni dato importante dovrebbe avere source:

position_source = simulated / estimated / measured / unavailable;

energy_source = simulated / estimated / measured;

thermal_source = simulated / estimated / measured;

node_type = simulated / hardware / hybrid.

Questi campi possono essere mostrati nel pannello dettagliato. In modalità demo, si può usare badge compatto. In modalità debug, mostrare tutto.

Il venticinquesimo elemento è la coerenza con i file e gli snapshot. La visualizzazione non deve inventare nomi o stati diversi dal backend. Se lo snapshot usa energy_state, il frontend non deve chiamarlo battery_mode senza mappatura chiara. Se il protocollo usa thermal_lock, la dashboard deve mostrare thermal_lock. Questa coerenza evita errori. La UI deve essere una rappresentazione del modello dati, non un universo separato.

Una regola è:

ogni elemento visuale deve corrispondere a un campo o concetto documentato.

Se si aggiunge un badge, deve avere un significato nel modello. Se si aggiunge un colore, deve essere spiegato. Se si mostra una metrica, deve essere definita. Questo rende la dashboard scientificamente utile.

Il ventiseiesimo elemento è la leggibilità per persone esterne. Un collaboratore, un professore, un investitore o un amico tecnico deve poter guardare la visualizzazione e capire almeno: quanti nodi ci sono, che pattern stanno formando, se il sistema è stabile, se ci sono errori, quali nodi sono critici e se la run è simulata o reale. Questo richiede titoli, legenda, metriche chiare e una gerarchia visiva. Non basta una scena piena di effetti.

Una demo view dovrebbe mostrare:

titolo del pattern;

numero nodi;

stato run;

forma target;

nodi e connessioni;

metriche principali;

safety status;

nota “simulation” o “hardware/hybrid”.

Questa vista è perfetta per video, pitch e presentazioni. La debug view può essere più dettagliata. Entrambe possono usare lo stesso motore visuale.

Il ventisettesimo elemento è la possibilità di esportare immagini o snapshot visuali. Durante lo sviluppo, può essere utile catturare una schermata o esportare lo stato finale di una run. Questo materiale può entrare in documentazione, README, relazione o presentazione. La dashboard può avere un pulsante `export_snapshot_image` o `export_state_json`. Anche se non implementato subito, è una funzione utile.

Output esportabili:

immagine della dashboard;

snapshot JSON;

report metriche;

timeline eventi;

stato finale del pattern.

Questo collega visualizzazione e documentazione. Ogni run può produrre materiale archiviabile.

Il ventottesimo elemento è la visualizzazione come strumento di apprendimento. Guardando lo swarm, l'utente può capire concetti come coesione, separazione, allineamento, errore, topologia, recovery e safety. La dashboard può quindi avere anche valore didattico. Se si mostra un cluster che si disperde quando `separation_weight` è troppo alto, o collassa quando `cohesion_weight` è troppo alto, si impara il comportamento del sistema. Questo è utile per scuole, università e presentazioni.

Una modalità educational può mostrare:

parametri attivi;

effetto dei parametri;

spiegazione breve del pattern;

metriche semplificate;

visualizzazione dei vettori comportamento.

Questa modalità potrebbe diventare una delle applicazioni dimostrative di MicroBot.

Il ventinovesimo elemento è la visualizzazione delle differenze tra run. In futuro, la dashboard potrebbe confrontare due run: prima e dopo una modifica, pattern con parametri diversi, con o senza fault, con o senza safety. Questo è avanzato, ma molto utile per validazione. Si possono mostrare metriche finali e snapshot. Anche solo salvare `run_id` e report prepara questa possibilità.

Una comparison view potrebbe mostrare:

run A vs run B;

pattern_error finale;

tempo di convergenza;

safety events;

energy events;

topology_score;

stability_score.

Questo aiuta a capire se il sistema sta migliorando.

Il trentesimo elemento è la roadmap della visualizzazione. Nella prima versione, basta mostrare nodi e pattern base. Poi si aggiungono connessioni, ruoli, target, metriche, safety, eventi, pannelli, replay, gruppi e 3D. Questa progressione evita di voler fare tutto subito. L'importante è che ogni passaggio aggiunga informazione reale.

Una roadmap possibile è:

v0.1: nodi 2D e node_id;

v0.2: ruoli e colori;

v0.3: connessioni ed edge;

v0.4: target e pattern overlay;

v0.5: metriche principali;

v0.6: safety, energia e temperatura;

v0.7: pannello nodo selezionato;

v0.8: console eventi;

v0.9: timeline e replay;

v1.0: gruppi, strutture complesse e vista 3D opzionale.

Questa roadmap è coerente con lo sviluppo incrementale del progetto.

In conclusione, la visualizzazione dei MicroBot è il livello che trasforma dati, simulazione e runtime in una rappresentazione leggibile dello swarm. Ogni nodo deve comunicare identità, ruolo, posizione, stato operativo, safety, energia, temperatura, sincronizzazione, target, connessioni ed errori. La visualizzazione deve distinguere ruolo e stato, simulazione e hardware, dati stimati e misurati, edge corretti e mancanti, pattern target e stato corrente. Deve supportare modalità demo e debug, pannelli dettagliati, selezione, overlay, timeline, console eventi e report. Una buona visualizzazione non è soltanto bella: è diagnostica, didattica, verificabile e coerente con il modello dati. Essa permette di vedere il comportamento dello swarm, ma anche di capirlo. Il punto successivo, dedicato ai modelli virtuali, approfondirà come i MicroBot simulati devono essere modellati internamente, quali proprietà devono possedere, quali comportamenti possono eseguire e come possono rappresentare in modo controllato sia i nodi futuri sia quelli fisici già prototipati.

12.4 Modelli virtuali

I modelli virtuali rappresentano la struttura interna dei MicroBot simulati, cioè il modo in cui ogni unità dello swarm viene descritta, aggiornata e fatta interagire all'interno dell'ambiente software. Dopo aver definito l'implementazione web e la visualizzazione grafica dei nodi, è necessario

chiarire che cosa esiste dietro ogni punto mostrato sullo schermo. Un MicroBot virtuale non deve essere soltanto una coordinata disegnata su un canvas. Deve essere un oggetto logico con identità, stato, ruolo, parametri, capacità, vincoli, comportamento, telemetria simulata e relazione con gli altri nodi. Il modello virtuale è quindi il ponte tra la rappresentazione matematica dello swarm e la sua osservazione nella dashboard.

Nel progetto MicroBot, i modelli virtuali hanno una funzione molto importante perché permettono di simulare nodi che ancora non esistono fisicamente, oppure nodi fisici che esistono solo in parte. Un MicroBot reale iniziale può essere un ESP32 con LED, heartbeat e comandi base; un MicroBot virtuale può invece avere posizione, velocità, campo magnetico simulato, energia virtuale, temperatura stimata, connessioni topologiche, movimento e ruolo nel pattern. Questo non significa mentire sullo stato del progetto. Significa separare il livello concettuale dal livello fisico. Il modello virtuale permette di progettare come il nodo dovrebbe comportarsi prima che tutto l'hardware sia pronto. Quando l'hardware crescerà, alcune proprietà virtuali potranno essere sostituite da misure reali.

Un modello virtuale MicroBot deve quindi essere dichiarato come `simulated_node`. Questa classificazione è fondamentale. Un `simulated_node` può partecipare alla simulazione, generare telemetria virtuale, rispondere a comandi simulati e comparire nella dashboard. Tuttavia, non deve essere confuso con un `hardware_node`. Un `hardware_node` è un dispositivo fisico. Un `hybrid_node` è una combinazione: una parte fisica reale e una rappresentazione virtuale associata. Questa distinzione permette al sistema di essere potente ma onesto. La dashboard può mostrare trenta MicroBot virtuali e tre MicroBot fisici, ma deve indicare chiaramente quali sono simulati e quali sono reali.

Una classificazione corretta può essere:

`simulated_node`: nodo completamente virtuale;

`hardware_node`: nodo fisico reale;

`hybrid_node`: nodo fisico con gemello virtuale o rappresentazione simulata;

`ghost_node`: nodo visto in precedenza ma non aggiornato;

`template_node`: modello usato per generare nuovi nodi virtuali.

Questa classificazione è utile perché ogni tipo di nodo ha regole diverse. Un `simulated_node` può essere spostato dal simulatore. Un `hardware_node` no, salvo che abbia locomozione reale o posizione misurata. Un `hybrid_node` può avere LED reali ma posizione virtuale. Un `ghost_node` non deve partecipare a pattern attivi. Un `template_node` serve solo per creare configurazioni.

Il primo elemento del modello virtuale è l'identità. Ogni MicroBot virtuale deve avere un `node_id` stabile all'interno della run. Per esempio SIM-001, SIM-002, SIM-003. Questo identificatore permette al controller, alla dashboard e ai log di riferirsi al nodo in modo preciso. Anche se il nodo è virtuale, deve seguire la stessa logica di un nodo fisico. Se SIM-004 entra in `low_energy` simulato, il log deve poterlo registrare. Se SIM-002 cambia ruolo, la dashboard deve mostrarlo. Se SIM-007 viene rimosso da un pattern, il controller deve saperlo. L'identità è quindi necessaria anche nella simulazione.

Il secondo elemento è la posizione. In un modello bidimensionale, la posizione può essere descritta come coppia x, y . In un modello tridimensionale, come x, y, z . Per i primi pattern MicroBot, il modello 2D è sufficiente: linea, cerchio, griglia e cluster possono essere validati su un piano. In

futuro, modelli 3D potranno essere utili per strutture volumetriche, superfici, campi e visualizzazioni più avanzate. La posizione deve essere distinta dalla posizione grafica sullo schermo. Il modello conserva coordinate logiche; il renderer le trasforma in pixel.

Una struttura semplice può essere:

$\text{position} = \{ x, y \}$

oppure:

$\text{position} = \{ x, y, z \}$

Accanto alla posizione può esistere `target_position`, cioè la posizione desiderata dal pattern. Il nodo può muoversi verso `target_position` secondo regole di controllo. La differenza tra `position` e `target_position` genera errore geometrico. Questa differenza è una delle metriche fondamentali per validare pattern target-based.

Il terzo elemento è la velocità. Anche se il nodo fisico iniziale non si muove realmente, il nodo virtuale può avere `velocity`. La velocità permette di simulare movimento, inerzia, smorzamento e stabilità. Senza velocità, il nodo potrebbe saltare direttamente al target, rendendo la simulazione troppo artificiale. Con velocità e damping, il movimento diventa più graduale e permette di osservare convergenza, oscillazioni e instabilità. Questo è utile per testare il comportamento del sistema.

Una dinamica semplice può essere:

posizione aggiornata in base alla velocità;

velocità aggiornata in base a forze o correzioni;

damping riduce oscillazioni;

`max_speed` limita movimento eccessivo.

Il modello non deve essere fisicamente perfetto nelle prime versioni. Deve essere abbastanza coerente da mostrare come i nodi convergono verso pattern e reagiscono ai vicini. In seguito, il modello può essere raffinato.

Il quarto elemento è il ruolo. Ogni nodo virtuale può avere un ruolo come leader, connector, terminal, `core_node`, `boundary_node`, `bridge_node` o `passive_node`. Il ruolo influenza il comportamento. Un leader può definire riferimento. Un connector può cercare connessioni. Un `boundary_node` può mantenere il bordo. Un `core_node` può contribuire alla stabilità interna. Un `passive_node` può muoversi meno o limitarsi a visualizzare stato. Il modello virtuale deve quindi includere role e, se necessario, `phase_role`.

Una struttura può essere:

`run_role = connector;`

`phase_role = aligner;`

`group_id = bridge_chain.`

Questa distinzione permette di simulare comportamenti più complessi. Durante phase_align, un connector può comportarsi da aligner. Durante phase_attach, può comportarsi da active_connector. Durante phase_stabilize, può diventare stabilizer. Il modello virtuale deve supportare questa evoluzione.

Il quinto elemento è lo stato operativo. Un nodo virtuale deve avere mode: idle, active, diagnostic, safety, recovery, offline. Anche se è simulato, questi stati sono importanti perché permettono di testare la logica del controller. Un nodo offline non deve ricevere comandi. Un nodo in safety deve rifiutare attuazioni. Un nodo in recovery deve essere riallineato. Un nodo active partecipa al pattern. Un nodo idle è presente ma non impegnato. Questi stati devono essere coerenti con quelli dei nodi hardware.

Una macchina a stati virtuale può includere:

booting;

idle;

active;

diagnostic;

safety;

recovery;

offline.

Il passaggio tra stati può essere controllato dal controller, dalla simulazione o da eventi di fault injection. Per esempio, un fault può portare SIM-004 da active a offline. Un emergency_stop può portare tutti i nodi a safety. Un recovery può riportare un nodo da offline a recovery e poi a idle. Questi scenari permettono di testare il runtime.

Il sesto elemento sono le capabilities virtuali. Ogni nodo simulato deve dichiarare cosa può fare. Un nodo può avere simulated_motion, simulated_magnetic_actuation, telemetry_basic, telemetry_debug, role_assignment, pattern_participation, safety_simulation, energy_simulation e thermal_simulation. Queste capabilities non sono hardware reali, ma indicano funzioni disponibili nel modello. Il controller deve poter distinguere tra capability simulata e hardware. Questo evita di assegnare a un nodo reale una funzione che esiste solo nel simulatore.

Una capability table virtuale può essere:

simulated_motion = true;

simulated_magnetic_actuation = true;

telemetry_basic = true;

energy_simulation = true;

thermal_simulation = true;

hardware_coil = false;

hardware_led = false.

Un nodo hardware LED, invece, potrebbe avere:

hardware_led = true;

udp_command = true;

heartbeat = true;

hardware_coil = false;

simulated_motion = false, oppure true solo se associato a hybrid_node.

Questa chiarezza evita confusione tra fisico e virtuale.

Il settimo elemento è il modello comportamentale. Ogni nodo virtuale deve sapere come aggiornarsi a ogni step. Il comportamento può essere definito da forze virtuali o da regole. Le regole principali possono essere attrazione verso target, separazione dai vicini, coesione verso il gruppo, allineamento con direzione comune, mantenimento connessioni, repulsione da ostacoli e stabilizzazione. Ogni regola produce un contributo al movimento o allo stato del nodo.

Una formula concettuale può essere:

azione_totale = target_force + cohesion_force + separation_force + alignment_force +
topology_force + safety_constraints

Questa formula non deve per forza essere implementata in modo fisicamente complesso. Serve a descrivere il principio. Il nodo virtuale riceve più influenze e aggiorna il proprio stato. I pesi di queste influenze possono essere definiti in behavior_profiles.json. Per esempio, un cluster userà alta coesione e separazione. Una linea userà alta attrazione verso target e spacing. Una griglia userà target e vincoli topologici.

L'ottavo elemento è il modello di vicinato. Un nodo virtuale deve poter identificare i suoi vicini. I vicini possono essere determinati per distanza, per topologia attesa o per gruppo. Nel comportamento swarm, i vicini per distanza sono importanti per coesione e separazione. Nei pattern topologici, i vicini attesi sono definiti dal grafo del pattern. Queste due categorie non sono sempre uguali. Un nodo può essere fisicamente vicino a un nodo con cui non dovrebbe connettersi, oppure può essere lontano dal suo vicino atteso. Il modello virtuale deve distinguere.

Si possono avere:

distance_neighbors: nodi entro un raggio;

expected_neighbors: nodi definiti dal pattern;

current_edges: connessioni virtuali attuali;

forbidden_neighbors: nodi che non devono connettersi.

Questa distinzione è molto utile per simulare errori topologici. Un nodo può formare un extra_edge con un vicino fisico ma non previsto. Oppure può mancare un expected_edge. Il controller deve rilevare e correggere.

Il nono elemento è la topologia virtuale. Un modello virtuale può creare connessioni tra nodi. Una connessione può essere creata se due nodi sono entro `connection_radius`, se i loro ruoli sono compatibili, se la fase corrente lo permette e se l'edge è previsto dal pattern. Oppure il simulatore può creare connessioni in modo più semplice, confrontando `expected_edges` e distanza. L'obiettivo è poter testare catene, cerchi, griglie, cluster, ponti e reticoli.

Una regola di connessione virtuale può essere:

edge si forma se distanza tra nodi $\leq$ `connection_radius` e edge previsto dal pattern e `phase_id = attach`.

Questa regola è semplice ma efficace. Permette di simulare che un attach avviene solo quando i nodi sono abbastanza vicini e nella fase corretta. Se si vuole simulare incertezza, si può aggiungere probabilità o ritardo. Per esempio, `edge_success_probability` può dipendere da distanza, energia, temperatura o rumore.

Il decimo elemento è il modello energetico virtuale. Ogni nodo può avere `energy_level` o `energy_state`. In simulazione, l'energia può diminuire nel tempo o in base alle azioni. Muoversi virtualmente può consumare poco o essere ignorato. Attivare `simulated_magnetic_actuation` può consumare di più. Inviare telemetria può consumare una quantità simulata. L'obiettivo non è calcolare consumo reale perfetto, ma testare le policy energetiche. Se un nodo consuma troppo, entra in `low_energy`; se scende ancora, `critical_energy`. Il controller deve reagire.

Una logica semplice può essere:

`energy_level` iniziale assegnato;

ogni step consuma `base_drain`;

azioni ad alto costo consumano `extra_drain`;

se `energy_level < soglia_low`, `energy_state = low_energy`;

se `energy_level < soglia_critical`, `energy_state = critical_energy`.

Questa logica permette di testare cosa succede se un pattern è troppo costoso. Per esempio, un `active_connector` usato troppo spesso può andare in `low_energy` e venire riassegnato. Questo è molto utile per la protezione energetica.

L'undicesimo elemento è il modello termico virtuale. Come già definito nel Safety Layer, ogni nodo può avere `thermal_load`. Le attuatori virtuali aumentano `thermal_load`; il tempo di riposo lo diminuisce. Se supera una soglia, il nodo entra in `thermal_warning`; se supera una soglia maggiore, `thermal_lock`. Questo permette di testare prevenzione del surriscaldamento e protezione delle bobine anche senza hardware.

Una logica termica può essere:

`thermal_load += action_cost`;

`thermal_load -= cooling_rate × delta_time`;

se `thermal_load > warning_threshold` → `thermal_warning`;

se `thermal_load > lock_threshold` → `thermal_lock`;

se thermal_load scende sotto recovery_threshold → thermal_recovery;

se scende sotto ok_threshold → thermal_ok.

Il modello termico deve usare hysteresis, come già spiegato. Senza hysteresis, il nodo potrebbe oscillare tra warning e ok. Il simulatore deve riflettere questa logica perché il controller deve imparare a gestirla.

Il dodicesimo elemento è il modello di comunicazione virtuale. Anche in simulazione, i comandi possono avere latenza, perdita, duplicazione o ritardo. Un modello troppo perfetto non testa il sistema. Il simulatore può introdurre packet_loss, latency_ms, jitter_ms, ack_loss e command_delay. Questo permette di testare retry, no_ack, sync_degraded, stale telemetry e recovery. Il nodo virtuale può rispondere a comandi come farebbe un nodo reale, ma con errori controllati.

Parametri utili:

latency_mean_ms;

latency_jitter_ms;

packet_loss_probability;

ack_loss_probability;

duplicate_probability;

out_of_order_probability;

offline_probability.

Questi parametri devono essere configurabili. In demo si possono disattivare. In test si possono aumentare. Questo rende la simulazione utile per robustezza.

Il tredicesimo elemento è il modello di sincronizzazione virtuale. Ogni nodo simulato può avere un clock locale con offset e drift. Il controller può inviare sync e il nodo aggiornare sync_status. Nelle prime versioni, si può semplificare e assumere sync perfetto. In versioni più avanzate, si può simulare offset, jitter e perdita di sync. Questo permette di testare comandi execute_at, slot temporali e phase synchronization.

Un modello temporale può includere:

local_time_offset_ms;

clock_drift;

last_sync_time;

sync_status;

execution_error_ms.

Se un nodo ha sync_lost, non deve eseguire comandi temporizzati critici. Questo può essere testato nel simulatore. La simulazione diventa quindi uno strumento per validare la Sezione 9.

Il quattordicesimo elemento è il modello di safety virtuale. Ogni nodo simulato deve poter rifiutare comandi se lo stato non lo permette. Per esempio, se `thermal_lock` è attivo, rifiuta `magnetic_pulse`. Se `low_energy`, rifiuta comandi ad alto consumo. Se `safety_mode`, rifiuta pattern attivi. Se `command_expired`, non esegue. Questo comportamento deve essere identico alla logica prevista per firmware reale, almeno a livello semantico. Così il controller può essere testato contro nodi simulati che si comportano come nodi sicuri.

Una validazione virtuale può includere:

`capability check;`

`mode check;`

`role check;`

`phase check;`

`energy check;`

`thermal check;`

`cooldown check;`

`timestamp check;`

`duplicate command check.`

Se fallisce, il nodo simulato genera `command_rejected` con `reason`. Questa risposta deve essere loggata e visualizzata. È fondamentale per testare il controller.

Il quindicesimo elemento è il modello di recovery virtuale. Un nodo simulato può andare offline, resettarsi, cambiare `boot_id`, entrare in recovery e richiedere `state_sync`. Questo permette di testare il ripristino automatico. Per esempio, il simulatore può forzare `watchdog_reset` su SIM-005 durante `phase_attach`. Il nodo rientra con nuovo `boot_id` e `mode = safe_idle`. Il controller deve rilevare il reset, invalidare stato, inviare `sync` e decidere se reintegrarlo. Questo test è fondamentale prima dell'hardware.

Un modello di reset virtuale può fare:

`cambiare boot_id;`

`azzerare uptime;`

`perdere ruolo attivo;`

`impostare mode = safe_idle;`

`bloccare capabilities critiche temporaneamente;`

`inviare boot_event;`

`richiedere state_sync.`

Questo rispecchia il comportamento desiderato del nodo fisico. Il simulatore diventa una palestra per recovery.

Il sedicesimo elemento è il modello di sensori virtuali. In futuro, i MicroBot possono avere sensori di prossimità, temperatura, batteria, campo magnetico, IMU o altri. Il simulatore può generare letture virtuali. Anche se non sono reali, permettono di progettare la pipeline: lettura, rumore, telemetria, dashboard, decisione. Per esempio, un sensore di prossimità virtuale può indicare distanza dal vicino. Un sensore termico virtuale può restituire temperature_c stimata. Un sensore batteria virtuale può restituire voltage. L'importante è indicare source = simulated.

Una lettura sensore virtuale può contenere:

```
sensor_id;  
  
sensor_type;  
  
value;  
  
unit;  
  
source = simulated;  
  
noise_level;  
  
timestamp.
```

Questo formato prepara l'integrazione con sensori reali. Quando un sensore fisico sarà disponibile, source cambierà in measured, ma la dashboard e il controller potranno usare campi simili.

Il diciassettesimo elemento è il modello di campo virtuale. MicroBot può usare concetti di campo magnetico, attrazione, repulsione, aggregazione e forza. In simulazione, un campo può essere rappresentato come vettore o funzione nello spazio. I nodi possono rispondere a un campo globale, a campi locali generati da altri nodi o a target. Un field_profile può definire intensità, raggio, decadimento e direzione. Questo è utile per simulare aggregazione, cluster, magnetismo virtuale e comportamenti emergenti.

Un campo virtuale può essere:

```
global_field;  
  
local_node_field;  
  
target_field;  
  
repulsion_field;  
  
magnetic_field_simulated.
```

Ogni campo può influenzare il movimento o la probabilità di connessione. Non è necessario che il modello sia fisicamente perfetto. Deve essere coerente con lo scopo: testare comportamenti e pattern. Se si vuole usare un modello fisico più rigoroso, si può aggiungere in versioni successive.

Il diciottesimo elemento è la differenza tra modello semplificato e modello realistico. Un modello semplificato è utile per sviluppo veloce: pochi parametri, comportamento leggibile, risultati stabili. Un modello realistico introduce massa, attrito, rumore, ritardi, limiti fisici e vincoli più complessi. Entrambi sono utili. La cosa importante è dichiarare quale modello si sta usando. Una simulazione

demo può usare modello semplificato. Una simulazione di validazione può usare più rumore e limiti.

Si possono avere profili:

simple_model: target-based, basso rumore, comportamento chiaro;

swarm_model: coesione/separazione/allineamento;

safety_model: energia, termica e recovery;

hardware_approx_model: limiti più vicini al prototipo fisico;

stress_test_model: rumore, fault e ritardi elevati.

Questi profili possono essere configurati nei file. Così non serve modificare codice per cambiare livello di realismo.

Il diciannovesimo elemento è la parametrizzazione del modello. Un nodo virtuale deve poter essere configurato con massa, max_speed, damping, connection_radius, separation_radius, energy_level, thermal_capacity, cooling_rate, role, capabilities e comportamento. Questo permette di simulare nodi diversi. Non tutti i MicroBot devono essere identici. Alcuni possono essere più energeticamente robusti, altri più deboli, alcuni con capacità magnetica, altri solo passivi. Questa eterogeneità è molto realistica.

Parametri possibili:

mass;

max_speed;

max_acceleration;

damping;

connection_radius;

separation_radius;

cohesion_weight;

alignment_weight;

target_weight;

energy_capacity;

thermal_capacity;

cooling_rate;

capabilities;

role_preferences.

Questi parametri permettono esperimenti. Per esempio, si può testare se uno swarm con nodi eterogenei riesce comunque a formare un pattern. Oppure se nodi con bassa energia devono essere esclusi.

Il ventesimo elemento è il modello del nodo come agente. Un MicroBot virtuale può essere visto come un agente autonomo con stato interno, input, regole e output. Gli input sono comandi, vicini, campi, stato del pattern e telemetria. Le regole decidono comportamento. Gli output sono movimento, connessioni, eventi, ack e telemetria. Questa visione è utile perché MicroBot è uno swarm, non una singola animazione centralizzata.

Un agente virtuale può essere descritto così:

input: comandi, vicini, target, stato locale;

state: ruolo, mode, energia, temperatura, sync;

decision: validazione, comportamento, safety;

output: movimento, ack, eventi, telemetria.

Questo modello è molto coerente con la futura implementazione firmware: anche il nodo fisico riceve input, mantiene stato e produce output. La simulazione anticipa il firmware.

Il ventunesimo elemento è la separazione tra controllo centralizzato e comportamento locale. In alcuni modelli, il controller calcola target per tutti i nodi. In altri, i nodi seguono regole locali. I modelli virtuali devono supportare entrambe le possibilità. MicroBot può essere progettato come sistema ibrido: controller centrale per pattern e safety globale, regole locali per coesione, separazione e adattamento. La simulazione è il posto migliore per esplorare questo equilibrio.

Si possono distinguere:

centralized_target_model: controller assegna target;

local_swarm_model: nodi seguono vicini;

hybrid_model: controller assegna obiettivo globale, nodi applicano regole locali.

Il modello ibrido è probabilmente il più coerente con la visione MicroBot. Il controller definisce pattern, ruoli e fasi; i nodi virtuali applicano regole per raggiungerli e mantenere sicurezza.

Il ventiduesimo elemento è la generazione automatica dei nodi virtuali. Il simulatore deve poter creare N nodi con posizioni iniziali casuali, griglia, cluster, distribuzione circolare o da file. Questo permette test rapidi. Una configurazione può specificare node_count e spawn_mode. Spawn_mode può essere random, grid, circle, cluster, line, imported. Questo è utile per confrontare pattern da condizioni iniziali diverse.

Spawn modes possibili:

random_area;

grid_layout;

circle_layout;

clustered;

line_layout;

from_file;

from_previous_snapshot.

from_previous_snapshot è utile per continuare una run o fare replay. imported permette di caricare dataset o stati salvati.

Il ventitreesimo elemento è il modello di rumore iniziale. Anche se si genera una griglia, si può aggiungere rumore alle posizioni per vedere se il sistema corregge. Se si genera un cluster, si può variare densità. Queste condizioni iniziali influenzano la difficoltà del pattern. Un pattern che funziona solo da posizione già ordinata non è robusto. Il simulatore deve permettere condizioni iniziali realisticamente imperfette.

Parametri utili:

initial_position_noise;

initial_velocity_noise;

initial_energy_variation;

initial_sync_offset;

initial_role_missing_probability.

Questi parametri permettono stress test progressivi. Si parte facile, poi si aumenta difficoltà.

Il ventiquattresimo elemento è il modello di dataset. Una simulazione può generare dati nel tempo. Ogni step può essere salvato come record JSONL, con stato dei nodi e metriche. Questo dataset può essere usato per analisi, grafici, training futuro, report o replay. Un file sample_swarm_dataset.jsonl può contenere sequenze di stati. Questo è molto coerente con il tuo ecosistema, perché collega simulazione, data analysis e documentazione.

Un record JSONL può contenere:

run_id;

step;

timestamp;

node_id;

position;

role;

mode;

energy_state;

thermal_state;

pattern_error;

event_flags.

Oppure si può salvare uno snapshot intero per riga. La scelta dipende dall'uso. Per analisi per nodo, record per nodo è utile. Per replay, snapshot intero è più comodo.

Il venticinquesimo elemento è la validazione dei modelli virtuali. Un modello virtuale deve essere controllato con test. Se un nodo riceve target, deve muoversi verso target. Se riceve low_energy, deve rifiutare comandi high_energy. Se entra in thermal_lock, deve bloccare coil virtuale. Se perde sync, deve rifiutare execute_at critici. Se riceve emergency_stop, deve andare in safety. Questi test garantiscono che il modello virtuale rispetti le regole del sistema.

Test utili:

node_moves_to_target;

node_respects_max_speed;

node_separates_from_neighbors;

node_forms_expected_edge;

node_rejects_command_in_thermal_lock;

node_rejects_high_energy_when_low_energy;

node_enters_recovery_after_reset;

node_reports_role_mismatch;

node_updates_telemetry;

node_generates_events.

Questi test rendono la simulazione affidabile. Se il modello virtuale non rispetta safety, il controller testato contro di esso può sembrare corretto ma non esserlo davvero.

Il ventiseiesimo elemento è la compatibilità con il controller. Il controller deve poter trattare un nodo simulato quasi come un nodo reale, almeno a livello di interfaccia. Può inviare comandi, ricevere ack, leggere status, ottenere telemetry. Questo permette di testare il controller senza hardware. Tuttavia, deve sapere node_type per non confondere simulazione e realtà. Questa compatibilità è una delle forze del progetto.

Una interfaccia comune può essere:

send_command(node_id, command);

request_status(node_id);

receive_heartbeat(node_id);

get_capabilities(node_id);

```
assign_role(node_id, role);  
  
start_pattern(pattern_id);  
  
emergency_stop(node_id/all).
```

Che dietro ci sia un nodo simulato o fisico, la semantica resta simile. Questo rende il sistema estensibile.

Il ventisettesimo elemento è la compatibilità con la dashboard. Ogni modello virtuale deve produrre dati visualizzabili. Posizione, ruolo, stato, edge, target e metriche devono entrare nello snapshot. Se il modello ha dati interni ma non li esporta, la dashboard non può mostrarli. Quindi il modello virtuale deve avere una funzione di serializzazione verso snapshot. Questa funzione decide quali campi esportare.

Una serializzazione nodo può includere:

```
node_id;  
  
node_type;  
  
position;  
  
target_position;  
  
velocity;  
  
role;  
  
mode;  
  
group_id;  
  
energy_state;  
  
thermal_state;  
  
safety_state;  
  
sync_status;  
  
capabilities_summary;  
  
last_event.
```

Questi campi sono sufficienti per la dashboard base. Altri dettagli possono essere richiesti tramite node inspector.

Il ventottesimo elemento è la relazione tra modello virtuale e gemello digitale. In futuro, un nodo fisico può avere un gemello virtuale, cioè una rappresentazione software che ne segue stato, posizione stimata, energia, temperatura e ruolo. Questo hybrid_node permette di integrare hardware incompleto con simulazione. Per esempio, un ESP32 fisico può inviare heartbeat e LED state, mentre la sua posizione viene rappresentata virtualmente nella dashboard. Il gemello digitale unisce mondo fisico e simulato.

Una struttura hybrid può avere:

hardware_node_id = MB-001;

virtual_node_id = SIM-MB-001;

position_source = simulated;

heartbeat_source = hardware;

energy_source = hardware_estimated;

role_source = controller;

visual_state = combined.

Questa modalità è molto utile nelle prime demo fisiche. Mostra il nodo reale come parte dello swarm virtuale. È onesta perché indica quali dati sono reali e quali virtuali.

Il ventinovesimo elemento è la documentazione dei modelli virtuali. Ogni modello deve spiegare assunzioni, parametri, limiti e scopo. Se il modello di magnetismo è qualitativo, va detto. Se il modello termico è stimato, va detto. Se il movimento è semplificato, va detto. Questo è essenziale per rigore. La simulazione non deve essere venduta come fisica completa se non lo è. Deve essere presentata come ambiente di sviluppo e validazione progressiva.

Un documento model_notes.md può includere:

scopo del modello;

stato implementativo;

parametri;

assunzioni;

limiti;

relazione con hardware;

test disponibili;

roadmap di miglioramento.

Questa documentazione renderà la repo molto più credibile.

Il trentesimo elemento è la roadmap dei modelli virtuali. Si può partire con un modello semplice di nodi 2D target-based. Poi aggiungere regole swarm. Poi topologia. Poi energia e temperatura. Poi comunicazione virtuale. Poi fault injection. Poi hybrid nodes. Poi modelli 3D. Questa progressione è realistica e coerente.

Una roadmap può essere:

v0.1: nodo virtuale con posizione e target;

v0.2: ruoli e pattern base;

v0.3: coesione, separazione e allineamento;

v0.4: connessioni topologiche;

v0.5: energia e temperatura simulate;

v0.6: safety e command rejection;

v0.7: comunicazione virtuale con ritardi e packet loss;

v0.8: recovery e watchdog reset simulato;

v0.9: hybrid nodes collegati a ESP32 fisici;

v1.0: modelli 3D e simulazione avanzata.

Questa roadmap permette di sviluppare il simulatore senza saltare passaggi.

In conclusione, i modelli virtuali sono la base interna della simulazione MicroBot. Ogni nodo virtuale deve essere molto più di un punto grafico: deve avere identità, posizione, velocità, target, ruolo, stato operativo, capabilities, energia, temperatura, safety, sincronizzazione, comportamento, vicini, connessioni, telemetria ed eventi. Questi modelli permettono di testare pattern, topologie, recovery, fault, energia e sicurezza prima dell'hardware reale. Inoltre, preparano il concetto di `hybrid_node` e gemello digitale, fondamentale per integrare prototipi fisici parziali con uno swarm simulato più ampio. La forza dei modelli virtuali sta nella loro capacità di rendere verificabile ciò che ancora non è fisicamente completo, mantenendo però coerenza e trasparenza. Il punto successivo, dedicato ai test dei pattern, userà questi modelli per spiegare come linee, cerchi, griglie, cluster, ponti, reticoli e strutture complesse possano essere provati, misurati e validati nel simulatore prima di passare a dashboard, hardware e demo fisiche.

12.5 Test dei pattern

I test dei pattern rappresentano la fase in cui la simulazione MicroBot viene usata non soltanto per mostrare forme o movimenti, ma per verificare in modo sistematico se i pattern definiti nelle sezioni precedenti sono realmente eseguibili, misurabili, ripetibili e robusti. Dopo aver descritto i modelli virtuali dei MicroBot, cioè i nodi simulati con identità, posizione, ruolo, stato, energia, temperatura, safety e comportamento, diventa necessario spiegare come questi modelli vengano messi alla prova attraverso scenari controllati. Un pattern, infatti, non può essere considerato valido solo perché appare visivamente corretto una volta. Deve essere testato in condizioni diverse, con metriche precise, con errori simulati, con nodi mancanti, con rumore, con limiti energetici e con criteri di successo dichiarati.

Nel progetto MicroBot, testare un pattern significa trasformare una definizione teorica in un esperimento software. La linea, il cerchio, la griglia, il cluster, il ponte, il reticolo o la struttura complessa non devono restare soltanto descrizioni nel documento. Devono diventare configurazioni eseguibili nel simulatore. Il simulatore deve generare nodi, assegnare ruoli, calcolare target, avviare fasi, misurare errori, controllare topologia, applicare safety virtuale, registrare eventi e produrre un risultato finale. Questo processo permette di capire se il pattern è chiaro, se i parametri sono coerenti, se la convergenza avviene, se la topologia è corretta e se il comportamento resta stabile nel tempo.

Il primo principio dei test dei pattern è la ripetibilità. Una run eseguita una sola volta non basta. Lo stesso pattern deve poter essere ripetuto con la stessa configurazione e produrre risultati

confrontabili. Per questo motivo ogni test deve avere un `test_id`, un `pattern_id`, una configurazione iniziale, un numero di nodi, un profilo comportamentale, una durata, una lista di metriche e criteri di successo. Se il test viene modificato, deve cambiare versione o configurazione. In questo modo si può confrontare `line_basic_v0.1` con `line_basic_v0.2`, oppure `cluster_basic` con parametri diversi di coesione e separazione.

Una struttura minima di test può contenere:

```
test_id;  
  
pattern_id;  
  
pattern_version;  
  
node_count;  
  
initial_state;  
  
behavior_profile;  
  
simulation_duration;  
  
fault_profile;  
  
success_criteria;  
  
metrics_to_record;  
  
expected_result.
```

Questa struttura rende il test documentabile. Se un pattern fallisce, non si dice genericamente “non funziona”: si sa in quale configurazione ha fallito, con quanti nodi, con quali parametri, in quale fase e rispetto a quali metriche. Questo è il passaggio da animazione a sperimentazione.

Il secondo principio è la separazione tra test ideale e test robusto. Un test ideale verifica se il pattern funziona in condizioni pulite: nodi online, nessun rumore, nessuna perdita pacchetti, energia ok, temperatura ok, nessun fault. Questo test è utile come base. Se un pattern non funziona nemmeno in condizioni ideali, il problema è nella definizione del pattern, nel comportamento o nel codice. Un test robusto, invece, introduce imperfezioni: rumore nelle posizioni, ritardi, nodi mancanti, energia bassa, `thermal_warning`, connessioni fallite. Questo test verifica se il sistema è resiliente.

Si possono quindi distinguere:

```
ideal_test: condizioni perfette;  
  
noise_test: piccole perturbazioni;  
  
fault_test: errori specifici;  
  
stress_test: condizioni difficili;  
  
recovery_test: verifica del ripristino;  
  
regression_test: controllo che modifiche future non rompano pattern già funzionanti.
```


Questa classificazione è molto utile. Non tutti i test servono allo stesso scopo. Un test ideale serve a validare la logica base. Un `fault_test` serve a validare error handling. Uno `stress_test` serve a capire limiti. Un `regression_test` serve a mantenere stabilità del progetto nel tempo.

Il primo pattern da testare è la linea. La linea è il pattern base più importante perché verifica ordinamento, target, spacing, ruoli e connessione semplice. Un test `line_basic` può generare N nodi in posizioni casuali e assegnare target lungo un asse. Il sistema deve portare i nodi verso la linea, mantenere distanza tra nodi e assegnare ruoli leader, connector e terminal. Se la linea prevede topologia di catena, deve verificare anche gli edge tra vicini.

Le metriche principali per `line_basic` sono:

`lateral_error_mean`;

`spacing_error_mean`;

`order_error`;

`target_distance_mean`;

`topology_score`;

`stability_time`;

`completion_time`.

Il test è riuscito se l'errore laterale scende sotto una soglia, lo spacing è accettabile, l'ordine è corretto, gli edge attesi sono presenti e la struttura resta stabile per un intervallo minimo. Questo evita di considerare riuscita una linea che appare corretta solo per un istante.

Un test ideale della linea può essere:

5 nodi;

posizioni iniziali casuali moderate;

nessun nodo offline;

energia e temperatura ok;

pattern target lineare;

durata massima 30 secondi simulati;

successo se `lateral_error_mean` < soglia e `topology_score` > 0.95.

Un test più robusto può introdurre:

un nodo con ritardo;

rumore di posizione;

un connector inizialmente fuori asse;

un edge che fallisce al primo tentativo;

energia bassa su un nodo non critico.

Questi test permettono di verificare se la linea è soltanto geometrica o anche robusta.

Il secondo pattern da testare è il cerchio. Il cerchio verifica distribuzione angolare, raggio, chiusura topologica e assenza di estremità. È più complesso della linea perché ogni nodo deve posizionarsi rispetto a un centro e a un angolo. Inoltre, se il cerchio è topologico, l'ultimo nodo deve connettersi al primo. Il test `circle_basic` deve quindi valutare sia la forma circolare sia la chiusura del ciclo.

Le metriche principali sono:

`radial_error_mean`;

`angular_variance`;

`neighbor_distance_variance`;

`closure_score`;

`cycle_topology_score`;

`collision_count`;

`stability_time`.

Il cerchio è riuscito se i nodi restano vicini al raggio desiderato, la distribuzione angolare è abbastanza uniforme, la chiusura è presente e il ciclo resta stabile. Un cerchio visivamente accettabile ma con `closure_score` basso non è pienamente riuscito. Questo è importante perché la topologia è parte della struttura.

Un test ideale del cerchio può usare 8 nodi e target distribuiti su 360 gradi. Un test robusto può rimuovere temporaneamente un nodo, introdurre ritardo nel nodo anchor, simulare `closure_failure` e verificare fallback ad arco. Questo è un test molto importante perché collega pattern, fallback e correzione degli errori.

Il terzo pattern da testare è la griglia. La griglia verifica capacità di organizzare nodi in righe e colonne. È utile per validare assegnazione di celle, `corner_node`, `edge_node`, `internal_node` e connessioni orizzontali/verticali. Rispetto a linea e cerchio, la griglia richiede una mappa bidimensionale più strutturata. Un test `grid_basic` può generare una griglia 2x3, 3x3 o 4x4 e misurare quanto i nodi occupano correttamente le celle.

Le metriche principali sono:

`cell_assignment_error`;

`row_alignment_error`;

`column_alignment_error`;

`grid_spacing_error_x`;

`grid_spacing_error_y`;

`missing_cells`;

horizontal_edge_score;

vertical_edge_score;

deformation_score;

stability_time.

Un test ideale può usare una griglia 3x3 senza rumore. Un test robusto può simulare una cella mancante, un nodo low_energy, una riga deformata o un nodo offline. Il fallback può trasformare grid_3x3 in partial_grid o grid_2x3. Questo permette di verificare se la griglia è adattabile.

Il quarto pattern da testare è il cluster. Il cluster è diverso perché non assegna necessariamente target individuali. Misura coesione, separazione, assenza di collisioni, connettività e densità. Un test cluster_basic può iniziare con nodi distribuiti casualmente e verificare se convergono in una regione compatta senza sovrapporsi. Questo pattern è molto importante perché rappresenta un comportamento più emergente rispetto a linea e griglia.

Le metriche principali sono:

cluster_radius;

average_distance_to_center;

pairwise_min_distance;

collision_count;

isolated_nodes;

graph_connectivity;

density_score;

stability_time.

Un cluster riuscito non deve essere perfettamente geometrico. Deve essere compatto, connesso e sicuro. Se i nodi collassano tutti nello stesso punto, il cluster non è valido perché viola separazione. Se sono troppo dispersi, non è valido perché manca coesione. Il test deve quindi bilanciare coesione e separazione.

Un test robusto del cluster può introdurre un nodo isolato, un nodo seeker, energia bassa su un nodo periferico o rumore elevato. Il sistema deve tentare isolated_node_recovery o accettare partial_success se il nodo viene escluso. Questo pattern è ottimo per testare adattamento dinamico.

Il quinto pattern da testare è la catena o bridge_chain. Questo pattern è simile alla linea ma enfatizza la topologia. I nodi devono formare una sequenza di connessioni. La geometria può essere leggermente curva o non perfettamente lineare, ma la sequenza di edge deve essere corretta. Questo test è fondamentale per strutture come ponti e segmenti articolati.

Metriche principali:

chain_order_score;

missing_chain_edges;
extra_chain_edges;
chain_connectivity;
endpoint_correctness;
edge_stability;
critical_edge_failures.

Un test robusto può rimuovere un nodo centrale, generare edge extra o simulare un connector in thermal_lock. Il sistema deve decidere se riparare la catena, accorciarla o dichiarare failure. Questo test prepara le strutture complesse.

Il sesto pattern da testare è il ponte composto da due cluster e una catena centrale. Questo è un pattern complesso perché combina sub-pattern. Il test deve verificare cluster_A, cluster_B e bridge_chain separatamente, poi la connessione globale. È un ottimo esempio di test gerarchico. Il pattern può fallire localmente o globalmente. Per esempio, i cluster possono essere corretti ma la catena non collegarli; oppure la catena può esistere ma un cluster essere disperso.

Metriche principali:

cluster_A_score;
cluster_B_score;
bridge_chain_score;
interconnection_score;
global_connectivity;
critical_bridge_edges;
structure_stability;
fallback_status.

Questo test è importante perché verifica la logica delle strutture complesse. Non basta una singola metrica globale. Serve vedere quale sottostruttura ha fallito. Un report deve dire: bridge failed because bridge_chain missing edge, oppure cluster_B dispersed. Questo rende il debug preciso.

Il settimo pattern da testare è il reticolo. Il reticolo estende la griglia con topologia più articolata e possibili connessioni diagonali o rinforzi. È utile per testare strutture stabili e ridondanti. Il test deve misurare non solo celle e righe, ma anche connettività, ridondanza, deformazione e tolleranza a un nodo mancante.

Metriche principali:

lattice_geometry_error;
lattice_topology_score;

redundancy_score;
cell_deformation_score;
missing_edges;
extra_edges;
connected_components;
fault_tolerance_score.

Un test interessante può rimuovere un nodo interno e verificare se il reticolo resta connesso grazie alla ridondanza. Questo è importante per mostrare robustezza strutturale.

Ogni test dei pattern deve avere criteri di successo espliciti. Non basta dire “sembra buono”. Un criterio può essere basato su soglie numeriche. Per esempio:

pattern_error < 0.05;
topology_score > 0.95;
stability_time > 5 secondi simulati;
collision_count = 0;
thermal_lock_count = 0;
critical_energy_count = 0.

Questi criteri devono essere adattati al pattern. Per un cluster, pattern_error rigido può non avere senso, mentre cluster_radius e isolated_nodes sono più importanti. Per una griglia, cell_assignment_error è centrale. Per un cerchio, closure_score è centrale. Il test deve misurare ciò che conta per quel pattern.

Un test completo dovrebbe produrre un risultato classificato, non solo booleano. Possibili risultati:

success_clean;
success_with_warnings;
success_with_corrections;
partial_success;
partial_success_with_fallback;
failed_geometry;
failed_topology;
failed_safety;
failed_timeout;

aborted.

Questa classificazione è molto più utile di true/false. Un pattern può riuscire geometricamente ma fallire safety. Può riuscire con fallback. Può fallire solo topologia. Sapere il motivo è fondamentale.

I test devono includere il Safety Layer. Dopo la Sezione 11, sarebbe incoerente testare pattern ignorando energia, temperatura e recovery. Ogni test può avere una modalità `safety_enabled`. In questa modalità, i comandi virtuali consumano energia, aumentano `thermal_load`, rispettano cooldown e possono essere rifiutati. Questo permette di valutare se un pattern è sostenibile, non solo se è geometricamente possibile.

Metriche safety nei test:

`thermal_warning_count`;

`thermal_lock_count`;

`low_energy_count`;

`critical_energy_count`;

`command_rejection_count`;

`cooldown_violations_prevented`;

`energy_budget_used`;

`safety_events`.

Se un pattern richiede troppi impulsi virtuali e genera `thermal_lock`, non è pronto per hardware. Può funzionare in simulazione geometrica, ma non in simulazione safety. Questa distinzione è molto importante.

I test devono includere la comunicazione simulata. Un pattern può funzionare con comunicazione perfetta, ma fallire con ritardi e packet loss. Il simulatore può introdurre latency, jitter e perdita pacchetti. Il controller deve gestire ack, timeout, retry e sync. Per esempio, un test `line_basic_latency` può verificare se la linea si forma quando alcuni nodi rispondono con ritardo. Un test `circle_sync_loss` può verificare se un nodo fuori sync viene escluso da una fase temporizzata.

Metriche comunicative:

`average_latency`;

`packet_loss_rate`;

`ack_timeout_count`;

`retry_count`;

`sync_degraded_count`;

`command_expired_count`;

`stale_node_count`.

Queste metriche aiutano a capire se il pattern è robusto anche come protocollo, non solo come geometria.

I test devono includere fault injection controllata. Ogni fault deve essere definito chiaramente. Per esempio: al secondo 5, SIM-003 va offline. Al secondo 8, SIM-004 entra in low_energy. Durante phase_attach, l'edge tra SIM-002 e SIM-003 fallisce. Questo permette di testare scenari riproducibili. Il fault non deve essere casuale se lo scopo è regression test. Può essere casuale negli stress test, ma nei test di validazione deve essere deterministico.

Una definizione fault può contenere:

fault_id;

fault_type;

target_node o target_edge;

trigger_time o trigger_phase;

duration;

expected_controller_response.

Esempio:

fault_type = node_offline;
target_node = SIM-004;
trigger_phase = align;
expected_response = role_reassignment_or_pattern_degradation.

Questa struttura rende i fault testabili in modo serio.

I test dei pattern devono generare report. Ogni run deve salvare configurazione, snapshot finali, metriche, eventi, errori e risultato. Il report può essere JSON per analisi automatica e Markdown per lettura umana. Il report deve indicare anche se il test è stato ideal, fault, stress o regression. Questo permette di costruire una storia sperimentale del progetto.

Un report di test può contenere:

test_id;

run_id;

pattern_id;

configuration;

initial_conditions;

faults_applied;

metrics_summary;

events_summary;

safety_summary;
result_classification;
failure_reason se presente;
recommendations.

Questo tipo di report rende il simulatore un vero ambiente di validazione. È anche molto utile per documentazione e repo.

I test dei pattern devono supportare confronti tra parametri. Per esempio, per cluster_basic si possono provare tre valori di cohesion_weight e separation_weight. Per line_basic si possono provare diversi damping. Per circle_basic diversi angular_spacing_weight. Questo permette di capire quali parametri producono comportamento migliore. La simulazione diventa strumento di tuning.

Un test parametrico può eseguire:

run 1: cohesion = 0.4, separation = 0.6;

run 2: cohesion = 0.6, separation = 0.6;

run 3: cohesion = 0.8, separation = 0.6;

confronto di cluster_radius, collision_count e stability_score.

Il risultato può suggerire quale profilo comportamentale usare. Questo è sviluppo sperimentale serio.

I test devono distinguere tra pattern statici e pattern dinamici. Un pattern statico ha una forma finale da raggiungere e mantenere. Un pattern dinamico richiede transizione o movimento. Per esempio, line_to_circle è un pattern dinamico: i nodi passano da linea a cerchio. Cluster_split è dinamico: un cluster si divide in due. Bridge_build è dinamico: due cluster vengono collegati. I test dinamici devono misurare la qualità della transizione, non solo il risultato finale.

Metriche per pattern dinamici:

transition_time;
intermediate_error;
stability_during_transition;
role_change_count;
topology_changes_correct;
safety_events_during_transition;
final_pattern_score.

Questi test saranno importanti quando MicroBot passerà dalla formazione di pattern alla riconfigurazione.

I test devono includere condizioni iniziali diverse. Un pattern può funzionare se i nodi partono già vicini, ma fallire se sono dispersi. Per validare robustezza, si devono provare spawn_mode diversi: random, cluster, grid, circle, line, noisy_grid. Questo aiuta a capire la sensibilità del pattern.

Esempi:

line_basic_from_random;

line_basic_from_cluster;

circle_basic_from_line;

grid_basic_from_random;

cluster_basic_from_grid;

bridge_from_two_clusters.

Questi test permettono di verificare la capacità del sistema di riconfigurarsi da stati diversi. È molto più interessante di partire sempre dalla stessa condizione.

I test devono includere numero di nodi variabile. Una linea può essere testata con 3, 5, 10, 20 nodi. Un cerchio con 6, 8, 12. Una griglia con 2x3, 3x3, 4x4. Un cluster con 10, 30, 100. Questo prepara la Sezione 19 sulla scalabilità. Alcuni pattern funzionano bene con pochi nodi ma diventano difficili con molti. La simulazione deve rivelarlo.

Metriche da osservare al crescere di N:

tempo di convergenza;

errore finale;

numero di collisioni;

topology_score;

computational_cost;

event_count;

stability_score.

Questi dati mostrano i limiti del sistema. Sono fondamentali per una roadmap realistica.

I test devono essere usati anche come regression suite. Ogni volta che si modifica il simulatore, il comportamento, il controller o i pattern, si devono rieseguire test base. Se line_basic funzionava e dopo una modifica fallisce, la regressione va identificata. Questo è un approccio da software engineering serio. Anche se nelle prime fasi i test possono essere manuali, la documentazione deve prevedere una suite automatica.

Una regression suite iniziale può includere:

line_basic_5_nodes;

circle_basic_8_nodes;

grid_basic_3x3;
cluster_basic_20_nodes;
line_missing_node_fallback;
circle_closure_failure;
cluster_isolated_node_recovery;
thermal_lock_blocks_actuation;
low_energy_blocks_high_energy_command;
watchdog_reset_recovery.

Questi test coprono pattern e safety. Sono una base solida.

I test dei pattern devono distinguere tra test visuale e test numerico. Un test visuale serve a guardare il comportamento. Un test numerico serve a misurarlo. Entrambi sono utili. Il test visuale può rivelare comportamenti strani che una metrica non cattura. Il test numerico permette confronto oggettivo. Una buona pipeline usa entrambi: la dashboard mostra, il report misura.

Una frase chiave è:

la visualizzazione aiuta a capire; le metriche permettono di validare.

Questo principio mantiene rigore. Un pattern non deve essere approvato solo perché “sembra bello”.

I test devono includere tolleranze. Nel mondo reale e anche nella simulazione con rumore, non esiste perfezione assoluta. Una linea non deve avere errore zero; deve stare sotto una soglia accettabile. Un cerchio non deve avere angoli perfettamente uguali; deve avere varianza sotto soglia. Una griglia non deve essere matematica pura; deve essere entro tolleranza. Le tolleranze devono essere definite in base allo scopo del pattern.

Una tolleranza può essere:

position_tolerance;
angle_tolerance;
spacing_tolerance;
topology_tolerance;
stability_tolerance;
safety_tolerance.

Per safety, alcune tolleranze sono zero: thermal_lock_count deve essere zero per success_clean. Collision_count può essere zero in pattern fisici. Per simulazioni esplorative, alcuni warning possono essere accettati. La classificazione del risultato deve riflettere questo.

I test devono includere anche casi impossibili o non validi. Per esempio, una griglia 3x3 richiede 9 nodi; se ne vengono forniti 7, il sistema deve rifiutare o degradare, non comportarsi in modo indefinito. Un cerchio chiuso con 2 nodi non ha senso come ciclo. Un ponte senza abbastanza connector deve fallire o fallback. Questi test verificano la robustezza della validazione iniziale.

Test invalidi utili:

grid_3x3_with_7_nodes;

circle_closed_with_2_nodes;

bridge_without_bridge_nodes;

pattern_with_duplicate_node_id;

pattern_with_missing_role;

pattern_with_unsupported_capability;

pattern_with_invalid_config.

Il sistema deve rispondere con errori chiari, come `invalid_pattern_configuration`, `insufficient_nodes`, `missing_required_role` o `capability_missing`. Questo è importante per evitare bug nascosti.

I test devono verificare l'assegnazione dei ruoli. Ogni pattern deve produrre ruoli corretti. In `line_basic`, `leader` e `terminal` devono essere uno ciascuno, `connector` gli intermedi. In `grid_basic`, `corner`, `edge` e `internal` devono essere corretti. In `cluster_basic`, `core` e `boundary` devono essere coerenti. I test devono controllare `role_count` e `role_position_match`. Un pattern può fallire se assegna ruoli sbagliati anche se la geometria sembra corretta.

Metriche ruolo:

`expected_role_count`;

`actual_role_count`;

`role_mismatch_count`;

`role_position_error`;

`critical_role_missing`;

`unsupported_role_assignments`.

Questo collega i test alla Sezione 10.4.

I test devono verificare la correzione degli errori. Per esempio, un nodo viene spostato fuori linea e il sistema deve correggerlo. Un edge manca e il sistema tenta `repair`. Un nodo ha ruolo sbagliato e il controller invia `state_sync`. Questi test controllano che la correzione non sia solo descritta, ma funzioni.

Test correzione:

`line_lateral_error_recovery`;

spacing_error_recovery;
missing_edge_repair;
role_mismatch_recovery;
sync_degraded_recovery;
isolated_node_recovery;
thermal_warning_reduces_duty;
energy_low_reassigns_role.

Questi test sono molto importanti per dimostrare intelligenza operativa.

I test devono verificare fallback. Un fallback è riuscito se il sistema riconosce che il pattern originale non è più raggiungibile e passa a una forma alternativa documentata. Per esempio, circle_closed → arc_open, grid_3x3 → grid_2x3, bridge → two_clusters_unconnected. Il test deve controllare che il fallback sia permesso, che venga registrato e che il risultato sia partial_success, non success_clean.

Metriche fallback:

fallback_triggered;
fallback_type;
fallback_reason;
fallback_success;
original_pattern_failed_reason;
final_pattern_score.

Questo rende il fallback controllato e trasparente.

I test devono includere replay e analisi. Ogni test importante dovrebbe poter essere rigiocato nella dashboard. Questo permette di vedere dove è avvenuto l'errore. Il report può indicare il timestamp del primo errore; il replay può saltare lì. Questa funzione sarà utile per debug e presentazioni.

Una pipeline ideale è:

esegui test;
salva snapshot sequence;
salva events;
salva report;
apri replay;
analizza fase critica.

Anche se non viene implementata subito, è una direzione molto forte.

I test dei pattern devono essere collegati ai file di configurazione del repository. Ogni test può essere definito come JSON. Per esempio:

tests/patterns/line_basic_5_nodes.json;

tests/patterns/circle_basic_8_nodes.json;

tests/patterns/grid_3x3_noise.json;

tests/patterns/cluster_fault_isolated_node.json;

tests/patterns/bridge_connector_failure.json.

Questa organizzazione rende i test facili da versionare. Il README può spiegare come eseguirli.

Una configurazione test può contenere:

name;

description;

pattern;

nodes;

initial_state;

behavior_profile;

faults;

expected_metrics;

expected_result.

Questo formato è molto chiaro.

I test devono supportare un livello di severità. Alcuni test sono baseline e devono sempre passare. Altri sono sperimentali e possono fallire senza rompere il sistema. Altri sono stress test e servono a trovare limiti. Questa distinzione evita confusione. Se uno stress test fallisce, non significa che il sistema base sia rotto. Se un baseline test fallisce, è grave.

Categorie:

baseline;

regression;

experimental;

stress;

hardware_precheck;

demo_validation.

Questa classificazione aiuta a organizzare il lavoro.

I test dei pattern devono anche avere valore documentale. Ogni pattern importante dovrebbe avere una scheda con obiettivo, input, fasi, metriche e risultati. Questa scheda può essere inserita nella documentazione. In una tesi o relazione, mostrare che line_basic, circle_basic e cluster_basic sono stati testati con metriche rende il progetto molto più forte. Non è solo teoria: è validazione.

Una scheda pattern test può includere:

nome pattern;

scopo;

numero nodi;

condizioni iniziali;

metriche;

criteri di successo;

risultato;

problemi osservati;

sviluppi futuri.

Questa struttura è perfetta per documentazione tecnica.

I test devono essere usati anche per scegliere cosa portare su hardware. Non tutti i pattern simulati devono essere trasferiti subito. Prima si trasferiscono quelli più stabili, semplici e sicuri. Per esempio, line_basic visuale con LED, poi role assignment, poi sync flash, poi cluster visuale, poi eventuale bobina test singola. I test dei pattern aiutano a decidere priorità. Se un pattern è instabile in simulazione, non va portato subito su hardware.

Una regola è:

un pattern deve superare test simulativi baseline e safety prima di essere candidato a demo hardware.

Questo principio evita sprechi e rischi. La simulazione filtra le idee.

I test dei pattern devono essere parte della roadmap. All'inizio si testano pattern semplici. Poi fault. Poi safety. Poi strutture complesse. Poi scalabilità. Poi hardware-precheck. Questo ordine è coerente.

Una roadmap test può essere:

v0.1: line_basic e circle_basic ideali;

v0.2: grid_basic e cluster_basic;

v0.3: metriche e report;

v0.4: fault injection semplice;

v0.5: safety simulation;

v0.6: recovery tests;

v0.7: bridge e reticolo;

v0.8: stress test con molti nodi;

v0.9: hardware-precheck;

v1.0: suite automatica completa.

Questa roadmap trasforma la simulazione in disciplina di sviluppo.

In conclusione, i test dei pattern sono il processo con cui MicroBot verifica che linee, cerchi, griglie, cluster, catene, ponti, reticoli e strutture complesse possano essere generati, mantenuti, corretti e validati nel simulatore. Ogni test deve avere configurazione, criteri, metriche, fault opzionali, report e classificazione del risultato. I test devono distinguere condizioni ideali, rumore, fault, stress, recovery e regressione. Devono misurare geometria, topologia, ruoli, stabilità, comunicazione, safety, energia e temperatura. Devono produrre log e snapshot, permettere replay e aiutare a decidere quali pattern sono pronti per demo hardware. In questo modo, la simulazione non resta una semplice animazione, ma diventa un ambiente sperimentale rigoroso. Il punto successivo, dedicato alla validazione del comportamento, allargherà il discorso dai singoli pattern alla qualità complessiva del sistema, cioè alla capacità dello swarm di comportarsi in modo coerente, stabile, misurabile e sicuro nel tempo.

12.6 Validazione del comportamento

La validazione del comportamento rappresenta il livello in cui la simulazione MicroBot non si limita più a verificare se un singolo pattern viene formato, ma valuta se l'intero sistema si comporta in modo coerente, stabile, misurabile, sicuro e ripetibile nel tempo. Dopo aver definito i test dei pattern, cioè le prove specifiche su linea, cerchio, griglia, cluster, catena, ponte e reticolo, è necessario passare a una visione più ampia: un comportamento MicroBot è valido non solo quando produce una forma finale corretta, ma quando lo fa rispettando ruoli, tempi, topologia, safety, energia, temperatura, recovery, comunicazione e criteri di stabilità. La validazione del comportamento è quindi il punto in cui geometria, controllo, simulazione, telemetria e sicurezza vengono giudicati insieme.

Nel progetto MicroBot, un comportamento può essere definito come una sequenza osservabile di stati attraversati dallo swarm durante una run. Non è soltanto la forma finale. È l'intero processo: preparazione, selezione dei nodi, assegnazione dei ruoli, sincronizzazione, movimento verso target, formazione delle connessioni, stabilizzazione, eventuali correzioni, eventuali fallback, gestione degli errori e report finale. Un comportamento valido deve essere coerente dall'inizio alla fine. Una linea che compare alla fine ma nasce da nodi fuori ruolo, comandi scaduti, safety ignorata e connessioni errate non deve essere classificata come comportamento valido. Può essere una forma visivamente riuscita, ma non un comportamento sistemico corretto.

Il primo criterio di validazione è la coerenza con il pattern dichiarato. Se la run dichiara `pattern_id = line_basic_v0.1`, il comportamento osservato deve corrispondere a ciò che quel pattern richiede. I nodi devono essere in numero sufficiente, i ruoli devono essere compatibili, la geometria deve avvicinarsi alla forma prevista, la topologia deve rispettare gli edge attesi e le fasi devono avvenire

nell'ordine corretto. Se il sistema forma qualcosa di simile a una linea ma con ruoli sbagliati, edge mancanti o fasi saltate, la validazione deve rilevarlo. Il pattern non è solo una sagoma; è un contratto operativo.

Una validazione del pattern deve quindi controllare:

pattern_id corretto;

pattern_version corretta;

numero nodi compatibile;

ruoli richiesti assegnati;

fasi eseguite nell'ordine previsto;

geometria entro tolleranza;

topologia entro tolleranza;

criteri safety rispettati;

risultato finale classificato correttamente.

Questa lista mostra che la validazione è più ampia del rendering. La dashboard può mostrare una forma bella, ma il validatore deve verificare se la forma è stata raggiunta secondo le regole del sistema.

Il secondo criterio è la stabilità nel tempo. Un comportamento non è valido se il pattern appare corretto solo per un istante e poi si dissolve, oscilla o crolla. Per questo motivo, la validazione deve includere una finestra di stabilità. La struttura deve restare entro soglia per un certo intervallo minimo. Per esempio, una linea può essere considerata stabilizzata solo se `lateral_error` e `spacing_error` restano sotto soglia per almeno alcuni secondi simulati o per un certo numero di step. Un cerchio deve mantenere `radial_error`, `angular_variance` e `closure_score` stabili. Un cluster deve mantenere densità e connettività senza collisioni. Una struttura complessa deve mantenere sub-pattern e connessioni critiche.

Una metrica utile è:

`stable_for_ms` = tempo durante cui il comportamento resta entro le soglie di validità

Oppure:

`stable_steps` = numero di step consecutivi in cui le metriche restano valide

Questa metrica impedisce falsi successi. Un pattern che entra brevemente nella soglia per caso non è veramente valido. Deve rimanerci. La stabilità è una proprietà temporale, non solo spaziale.

Il terzo criterio è la convergenza. Un comportamento valido dovrebbe mostrare un miglioramento progressivo delle metriche, salvo transizioni previste. Se il `pattern_error` diminuisce nel tempo e poi si stabilizza, il comportamento è coerente. Se l'errore oscilla senza ridursi, oppure cresce, oppure migliora solo casualmente, il comportamento è fragile. La validazione deve quindi osservare la traiettoria delle metriche, non solo il valore finale. Questo è particolarmente importante per tuning dei parametri.

Una validazione di convergenza può controllare:

errore iniziale;

errore finale;

tendenza dell'errore;

tempo di convergenza;

oscillazioni;

numero di correzioni necessarie;

stabilità dopo convergenza.

Un pattern che converge rapidamente e resta stabile è migliore di un pattern che converge lentamente e richiede molte correzioni. Entrambi possono arrivare al target, ma il primo è più robusto. La validazione deve poter distinguere queste differenze.

Il quarto criterio è la coerenza dei ruoli. Come già definito, ogni nodo possiede un ruolo operativo. La validazione deve verificare che i ruoli siano stati assegnati correttamente, mantenuti quando necessario e modificati solo secondo regole documentate. Per esempio, in una linea, deve esserci un leader, un terminal e connector intermedi. In una griglia, `corner_node`, `edge_node` e `internal_node` devono corrispondere alle posizioni logiche. In un ponte, `bridge_node` e `cluster_core` devono essere coerenti con la struttura. Se un nodo cambia ruolo, il cambio deve essere registrato e motivato.

Metriche di validazione dei ruoli possono essere:

`role_assignment_success_rate`;

`role_mismatch_count`;

`critical_role_missing`;

`unsupported_role_assignment_count`;

`role_change_count`;

`unauthorized_role_change_count`;

`role_position_consistency`.

Queste metriche permettono di capire se il comportamento è organizzato. Uno swarm in cui i nodi cambiano ruolo continuamente senza motivo può apparire dinamico, ma è instabile. Uno swarm in cui i ruoli sono coerenti e tracciati è molto più controllabile.

Il quinto criterio è la coerenza topologica. In MicroBot, la topologia è fondamentale perché molte strutture dipendono dalle relazioni tra nodi. La validazione deve verificare `expected_edges`, `current_edges`, `missing_edges`, `extra_edges` e `critical_edges`. Un pattern può essere geometricamente corretto ma topologicamente sbagliato. Per esempio, un cerchio può apparire circolare ma non avere chiusura; una catena può apparire lineare ma essere disconnessa; una griglia può apparire ordinata ma avere edge mancanti; un ponte può avere cluster corretti ma collegamento centrale assente. La validazione topologica impedisce questi falsi successi.

Metriche topologiche possono essere:

topology_score;

missing_edge_count;

extra_edge_count;

critical_edge_failure_count;

connected_components;

cycle_completion_score;

bridge_connectivity_score;

edge_stability_score.

Il risultato deve distinguere failed_geometry e failed_topology. Questo è importante perché le correzioni sono diverse. Un errore geometrico richiede movimento o target correction. Un errore topologico richiede attach, edge repair, riassegnazione o fallback.

Il sesto criterio è la coerenza temporale. Il comportamento dello swarm avviene in fasi. Se i nodi eseguono comandi fuori tempo, se gli ack arrivano troppo tardi, se alcuni nodi sono in phase_align mentre altri sono in phase_attach, la run può diventare incoerente. La validazione deve quindi controllare phase synchronization, command expiration, execution timing e sync_status. Un comportamento visivamente corretto ma ottenuto con comandi fuori fase non è pienamente valido, soprattutto se in futuro dovrà controllare hardware fisico.

Metriche temporali possono essere:

phase_alignment_score;

late_ack_count;

command_expired_count;

execution_error_mean_ms;

sync_degraded_count;

sync_lost_count;

out_of_phase_nodes;

phase_transition_success_rate.

Queste metriche collegano la simulazione alla sincronizzazione. Se un pattern funziona solo con sync perfetto, la validazione robusta deve evidenziarlo. Se funziona anche con jitter moderato, è più adatto al mondo reale.

Il settimo criterio è la sicurezza. Nessun comportamento può essere considerato valido se viola il Safety Layer. Anche se forma la struttura corretta, deve rispettare corrente, bobine, temperatura, batteria, protezione energetica, watchdog e recovery. In simulazione, questi elementi possono essere virtuali, ma devono comunque essere applicati. Un pattern che raggiunge il target usando un

numero eccessivo di impulsi virtuali, generando `thermal_lock` o `critical_energy`, non è `hardware-safe`. Può essere classificato come `simulation_geometry_success`, ma non come `hardware_ready_success`.

La validazione `safety` deve controllare:

`thermal_warning_count`;

`thermal_lock_count`;

`low_energy_count`;

`critical_energy_count`;

`command_rejected_count`;

`cooldown_violation_attempts`;

`energy_budget_exceeded`;

`watchdog_reset_count`;

`emergency_stop_response`;

`safety_state_violations`.

Questa validazione è fondamentale per mantenere coerenza con la Sezione 11. Il sistema non deve premiare un comportamento che funziona solo ignorando i limiti fisici. La `safety` è parte del risultato, non un controllo separato.

L'ottavo criterio è la gestione degli errori. Un comportamento valido deve reagire correttamente quando qualcosa va storto. Questo non significa che debba sempre riuscire comunque. Significa che deve fallire bene, cioè in modo sicuro, tracciato e coerente. Se un nodo va offline, il sistema deve rilevarlo. Se un ruolo critico manca, deve segnalarlo. Se un edge non si forma, deve tentare correzione o fallback secondo policy. Se un comando viene rifiutato, deve registrare motivo e adattare. Se la recovery fallisce, deve isolare o abortire. Un comportamento che ignora errori non è valido.

Metriche di gestione errori possono essere:

`error_detection_rate`;

`error_classification_success`;

`correction_attempt_count`;

`correction_success_rate`;

`fallback_triggered`;

`fallback_success_rate`;

`recovery_success_rate`;

`unhandled_error_count`;

abort_reason_quality.

Il valore `unhandled_error_count` dovrebbe essere zero nei test controllati. Se un errore avviene e il sistema non lo classifica, la validazione deve fallire o produrre warning grave. La robustezza non richiede assenza di errori, ma capacità di gestirli.

Il nono criterio è la qualità del fallback. Un fallback non è un fallimento casuale; è una strategia documentata. Se `circle_closed` passa ad `arc_open` perché closure fallisce, questo può essere `partial_success`. Se `grid_3x3` passa a `grid_2x3` perché mancano nodi, può essere `partial_success_with_fallback`. La validazione deve verificare che il fallback sia ammesso dal pattern, che sia stato attivato per motivo corretto, che sia stato registrato e che il risultato finale sia classificato correttamente. Un sistema non deve dichiarare `success_clean` dopo fallback, perché l'obiettivo originale è cambiato.

Metriche fallback:

`fallback_allowed;`

`fallback_reason;`

`fallback_time;`

`fallback_result;`

`original_goal_preserved;`

`final_goal_valid;`

`classification_correctness.`

Queste metriche rendono il fallback trasparente. Il sistema non nasconde la degradazione; la spiega.

Il decimo criterio è la ripetibilità statistica. Un comportamento può riuscire una volta per caso. Per validarlo davvero, bisogna ripetere il test con più semi casuali, condizioni iniziali o rumori. Se `line_basic` riesce 10 volte su 10 in condizioni ideali, è stabile. Se `cluster_basic` riesce 6 volte su 10 con rumore medio, bisogna migliorare parametri. Se `bridge_complex` riesce solo 2 volte su 10, è ancora sperimentale. La validazione deve quindi includere `success_rate`, non solo singola run.

Metriche statistiche:

`success_rate;`

`average_completion_time;`

`average_pattern_error;`

`standard_deviation_error;`

`failure_modes_distribution;`

`average_corrections_per_run;`

`average_safety_events_per_run.`

Queste metriche trasformano la simulazione in analisi sperimentale. Un comportamento viene valutato su una serie, non su un episodio isolato.

L'undicesimo criterio è la classificazione del risultato. Ogni run deve finire con una classe chiara. Questo evita ambiguità. Le classi devono distinguere successo pulito, successo con correzioni, successo con warning, successo parziale, fallimento geometrico, fallimento topologico, fallimento safety, timeout, abort e necessità di intervento manuale. Questa classificazione è essenziale per report e dashboard.

Una classificazione completa può essere:

success_clean;
success_with_corrections;
success_with_warnings;
success_degraded;
partial_success_with_fallback;
failed_geometry;
failed_topology;
failed_roles;
failed_timing;
failed_safety;
failed_energy;
failed_thermal;
failed_recovery;
aborted_by_emergency_stop;
invalid_configuration;
manual_intervention_required.

Ogni classe deve avere significato preciso. Per esempio, success_clean significa che il pattern è stato raggiunto senza warning rilevanti, senza fallback e senza safety events critici. success_with_corrections significa che il pattern finale è valido ma sono state necessarie correzioni. partial_success_with_fallback significa che il target originale non è stato raggiunto, ma un target alternativo ammesso sì. failed_safety significa che la safety ha impedito completamento. Questa precisione rende il sistema molto più professionale.

Il dodicesimo criterio è la relazione tra validazione e dashboard. La dashboard deve mostrare lo stato della validazione in modo sintetico. Non deve limitarsi a mostrare nodi. Deve indicare se la run è in corso, se il comportamento è entro soglia, se la stabilità è raggiunta, se ci sono warning, se il fallback è attivo e quale risultato finale è stato classificato. In pratica, la dashboard deve avere un

pannello “Validation”. Questo pannello aiuta a capire non solo cosa si vede, ma cosa il sistema sta valutando.

Un pannello Validation può mostrare:

current_validation_status;

pattern_error;

topology_score;

role_consistency;

stability_timer;

safety_status;

energy_status;

thermal_status;

correction_count;

fallback_status;

final_result.

Durante la run, il pannello può dire “validating”, “converging”, “unstable”, “warning”, “fallback active”, “stable pending”, “validated”. Alla fine, può mostrare il risultato. Questo rende la simulazione molto più chiara.

Il tredicesimo criterio è la produzione di report di validazione. Alla fine di una run, il sistema deve generare un report che contenga configurazione, pattern, metriche, eventi, risultato e motivazione. Il report deve spiegare perché la run è stata classificata in un certo modo. Se fallisce, deve indicare failure_reason. Se riesce con warning, deve indicare quali warning. Se usa fallback, deve indicare perché. Questo è fondamentale per documentazione.

Un report di validazione può contenere:

run_id;

pattern_id;

pattern_version;

test_id;

initial_conditions;

behavior_profile;

duration;

final_result;

validation_summary;

metrics_summary;

phase_summary;

safety_summary;

error_summary;

fallback_summary;

recovery_summary;

recommendations.

Il campo recommendations è molto utile. Può suggerire di aumentare damping, ridurre cohesion, aumentare cooldown, rivedere role assignment, ridurre retry o modificare pattern. Anche se inizialmente scritto manualmente o generato in modo semplice, aggiunge valore.

Il quattordicesimo criterio è la validazione per fase. Una run non deve essere validata solo alla fine. Ogni fase può avere criteri propri. Durante prepare, i nodi devono essere disponibili. Durante assign_roles, i ruoli devono essere assegnati. Durante align, l'errore geometrico deve diminuire. Durante attach, gli edge devono formarsi. Durante stabilize, la velocità e l'errore devono restare sotto soglia. Durante report, i dati devono essere salvati. Se una fase fallisce, la run può passare a recovery o fallback.

Una validazione per fase può essere:

prepare_valid = nodi sufficienti e capabilities compatibili;

roles_valid = ruoli richiesti assegnati e confermati;

align_valid = errore geometrico sotto soglia o in diminuzione;

attach_valid = edge critici presenti;

stabilize_valid = metriche stabili per tempo minimo;

report_valid = log e snapshot salvati.

Questa struttura aiuta a diagnosticare. Se la run fallisce, si sa in quale fase. Questo è molto meglio di un fallimento generico.

Il quindicesimo criterio è la validazione dei dati. Il comportamento viene valutato sulla base di snapshot, telemetria e log. Se questi dati sono vecchi, incompleti o incoerenti, la validazione può essere sbagliata. Quindi bisogna validare anche la qualità dei dati. Uno snapshot stale non deve essere usato come stato attuale. Un nodo con telemetria outdated non deve essere considerato sano. Una metrica calcolata su dati mancanti deve essere marcata come uncertain. La validazione deve includere confidence.

Una struttura può includere:

data_freshness;

telemetry_completeness;

snapshot_age_ms;

missing_fields;

node_update_rate;

validation_confidence.

Se validation_confidence è bassa, il risultato deve essere prudente. Per esempio, “partial validation” o “validation uncertain”. Questo è importante perché un sistema reale lavora sempre con dati imperfetti.

Il sedicesimo criterio è la validazione dei nodi ibridi. Quando alcuni nodi sono hardware e altri simulati, la validazione deve distinguere le fonti. Una run può essere validata come simulation_success, hybrid_demo_success o hardware_partial_success. Se la posizione dei nodi fisici è virtuale, la geometria non è completamente hardware-validated. Se i nodi fisici confermano ruoli e heartbeat, quella parte è reale. Il report deve indicarlo. Questa chiarezza è fondamentale per non confondere la maturità del progetto.

Classificazioni possibili:

simulation_validated;

visual_hardware_validated;

hybrid_validated;

hardware_partial_validated;

hardware_not_validated.

Per esempio, una demo con 30 nodi simulati e 3 ESP32 LED può essere hybrid_validated per role assignment e dashboard, ma non hardware_validated per movimento fisico. Questa distinzione rende il progetto onesto e forte.

Il diciassettesimo criterio è la validazione rispetto alla scalabilità. Un comportamento valido con 5 nodi non è automaticamente valido con 50. La simulazione deve testare crescita del numero di unità. La validazione deve misurare come cambiano tempo di convergenza, errore, eventi, consumo computazionale, leggibilità dashboard e stabilità. Questo prepara la Sezione 19. Il sistema deve sapere quali pattern scalano bene e quali no.

Metriche di scalabilità comportamentale:

completion_time_vs_node_count;

error_vs_node_count;

event_count_vs_node_count;

topology_complexity;

simulation_step_time;

controller_load;

dashboard_frame_rate;

communication_simulated_load.

Queste metriche mostrano i limiti. Un pattern può essere valido per demo piccole ma non per swarm grandi. È importante documentarlo.

Il diciottesimo criterio è la validazione comparativa tra versioni. Ogni pattern e comportamento può evolvere. line_basic_v0.2 deve essere confrontato con line_basic_v0.1. Se la nuova versione riduce errore ma aumenta correzioni e safety warning, il miglioramento non è netto. La validazione deve permettere confronto tra versioni. Questo è molto importante per sviluppo serio.

Un confronto può analizzare:

errore finale;

tempo di completamento;

stabilità;

numero di correzioni;

warning safety;

fallimenti;

success_rate;

complessità computazionale.

Questo permette di decidere se una modifica è davvero migliore. Non basta dire “la nuova sembra più bella”.

Il diciannovesimo criterio è la validazione del comportamento emergente. Alcuni comportamenti, come clusterizzazione o aggregazione, non hanno target rigido. Devono essere validati tramite metriche diverse: coesione, separazione, densità, connettività, assenza di collisioni e stabilità. È importante non applicare a un cluster gli stessi criteri di una griglia. Ogni comportamento deve avere metriche coerenti con il suo scopo.

Per comportamento emergente, metriche utili sono:

aggregation_score;

cluster_radius;

connected_components;

isolated_nodes;

density_uniformity;

collision_count;

stability_score;

role_distribution.

La validazione deve riconoscere che un cluster può avere molte configurazioni valide. Non esiste un unico target. Quindi il criterio è una regione di comportamento accettabile, non una posizione esatta.

Il ventesimo criterio è la validazione del comportamento adattivo. Se il sistema adatta ruoli, parametri o pattern, bisogna verificare che l'adattamento sia giustificato, efficace e tracciato. Un adattamento non deve avvenire senza motivo. Deve essere scatenato da una metrica o evento. Dopo l'adattamento, il sistema deve migliorare o almeno entrare in stato più sicuro. Se l'adattamento peggiora la situazione, va registrato come failed_adaptation.

Metriche adattive:

adaptation_trigger_count;

adaptation_reason_validity;

adaptation_success_rate;

metric_improvement_after_adaptation;

unnecessary_adaptation_count;

adaptation_loop_detected.

Questo è importante perché un sistema troppo adattivo può diventare instabile. La validazione deve controllare che l'adattamento sia utile, non caotico.

Il ventunesimo criterio è la validazione della recovery. Dopo un errore, il sistema deve recuperare in modo corretto. Una recovery valida deve mettere in sicurezza, rendere il nodo osservabile, ricostruire sync, aggiornare stato, verificare capabilities e reintegrare solo se possibile. Se il nodo rientra automaticamente senza state_sync, la recovery non è valida. Se emergency_stop viene cancellato automaticamente senza conferma, non è valido. Se thermal_lock viene ignorato, non è valido. La validazione recovery deve essere severa.

Metriche recovery:

recovery_started;

recovery_steps_completed;

safe_state_reached;

sync_restored;

state_sync_success;

capabilities_updated;

reintegration_allowed;

recovery_result;

manual_intervention_required.

Queste metriche collegano la Sezione 11 alla simulazione. Il simulatore può testarle prima dell'hardware.

Il ventiduesimo criterio è la validazione del comportamento in caso di abort. Non tutti i comportamenti devono finire con successo. A volte abortire è la scelta corretta. Se c'è safety critical, config invalid, energy critical o emergency_stop, il sistema deve fermarsi. Una run abortita per una ragione valida è un comportamento sicuro, non un fallimento di progettazione. La validazione deve distinguere abort corretto da abort causato da bug.

Un abort valido deve avere:

abort_reason chiaro;

uscite critiche disattivate;

nodi in safe state;

evento registrato;

report generato;

nessun comando attuativo pendente;

recovery o manual intervention definita.

Se questi criteri sono rispettati, l'abort è safety_success anche se il pattern fallisce. Questa distinzione è importante per un sistema fisico.

Il ventitreesimo criterio è la validazione delle configurazioni. Prima di iniziare una run, il sistema deve validare input: numero nodi, pattern, capabilities, ruoli, soglie, limiti, file di configurazione. Se la configurazione è incoerente, la run non deve partire. Questo evita errori durante simulazione o hardware. La validazione pre-run è una forma di sicurezza.

Controlli pre-run:

pattern_id esiste;

pattern_version supportata;

node_count sufficiente;

ruoli richiesti disponibili;

capabilities compatibili;

safety_policy presente;

behavior_profile valido;

fault_profile valido se usato;

config_epoch coerente;

limiti energetici definiti.

Se un controllo fallisce, risultato = `invalid_configuration`. Non si deve tentare di eseguire un pattern impossibile. Questo è un punto di maturità architetturale.

Il ventiquattresimo criterio è la validazione dell'interazione con la dashboard. Se la dashboard mostra dati sbagliati, l'utente può prendere decisioni sbagliate. Quindi bisogna verificare che la visualizzazione rifletta correttamente lo stato interno. Per esempio, se un nodo è in `thermal_lock` nel modello, la dashboard deve mostrarlo. Se un edge è missing, deve essere visibile. Se un fallback è attivo, deve essere indicato. Questa è una validazione UI-runtime.

Test UI-validazione:

snapshot con nodo offline mostrato correttamente;

snapshot con `thermal_lock` mostrato correttamente;

edge missing evidenziato;

fallback attivo mostrato;

validation status aggiornato;

snapshot stale segnalato;

result finale mostrato coerentemente.

Questo collega simulazione e implementazione web.

Il venticinquesimo criterio è la validazione del comportamento rispetto alla narrativa del progetto. MicroBot vuole rappresentare un sistema modulare, distribuito, estensibile, con logica swarm e materia programmabile. La validazione deve dimostrare progressivamente questi concetti: coordinazione tra nodi, pattern emergenti, ruoli differenziati, recovery, safety, scalabilità e integrazione hardware-simulazione. Ogni comportamento validato deve poter essere collegato a una parte della visione. Non deve essere un effetto isolato.

Per esempio:

`line_basic` valida coordinazione semplice;

`circle_basic` valida chiusura topologica;

`cluster_basic` valida aggregazione;

`grid_basic` valida organizzazione spaziale;

`bridge_complex` valida composizione di sub-pattern;

safety tests validano affidabilità;

hybrid tests validano ponte software-hardware.

Questa mappa aiuta a decidere quali demo sono significative. Non tutte le simulazioni hanno lo stesso valore comunicativo.

Il ventiseiesimo criterio è la validazione documentale. Ogni comportamento validato deve poter essere descritto in modo leggibile. Il report tecnico deve spiegare obiettivo, metodo, risultati, limiti e prossimi passi. Questo è importante per tesi, README, presentazioni e invio ad Andrea o altri collaboratori. Una validazione senza documentazione resta interna; una validazione documentata diventa asset del progetto.

Una scheda di comportamento validato può includere:

nome comportamento;

scopo;

pattern associato;

modello usato;

condizioni iniziali;

metriche;

risultato;

screenshot o snapshot;

limiti;

prossima evoluzione hardware.

Questo formato può diventare parte della documentazione finale.

Il ventisettesimo criterio è la validazione progressiva. Non si deve pretendere di validare tutto subito. Prima si valida comportamento geometrico in simulazione ideale. Poi topologia. Poi safety virtuale. Poi fault. Poi recovery. Poi dashboard. Poi hybrid hardware. Poi hardware fisico parziale. Questa progressione permette di accumulare prove senza fare salti non giustificati.

Una scala di maturità può essere:

Level 0: comportamento descritto teoricamente;

Level 1: comportamento visualizzato in simulazione;

Level 2: comportamento misurato con metriche;

Level 3: comportamento validato con test ripetibili;

Level 4: comportamento robusto con fault injection;

Level 5: comportamento integrato con safety;

Level 6: comportamento integrato con dashboard e report;

Level 7: comportamento testato in modalità ibrida;

Level 8: comportamento parzialmente hardware-validato;

Level 9: comportamento hardware-ready;

Level 10: comportamento fisico completo e ripetibile.

Questa scala è molto utile per evitare allucinazioni tecniche. Ogni funzione può essere collocata a un livello di maturità.

Il ventottesimo criterio è la validazione dei limiti. Un sistema serio non documenta solo ciò che funziona, ma anche ciò che non funziona ancora. Se un pattern non scala oltre 50 nodi, va scritto. Se la safety è solo simulata, va scritto. Se la posizione dei nodi hardware è virtuale, va scritto. Se la bobina non è ancora integrata fisicamente, va scritto. La validazione deve produrre anche boundary conditions, cioè condizioni entro cui il comportamento è valido.

Un report dovrebbe indicare:

valid under these conditions;

not validated under these conditions;

known limitations;

assumptions;

future validation required.

Questa chiarezza aumenta credibilità. Un progetto che conosce i propri limiti è molto più forte di uno che promette tutto.

Il ventinovesimo criterio è la validazione come base per decisioni. I risultati dei test devono guidare cosa fare dopo. Se line_basic è stabile, si può portare in dashboard demo. Se circle_basic ha problemi di closure, si migliora topologia. Se cluster_basic collassa, si regola separazione. Se bridge_complex fallisce spesso, si semplifica. Se safety genera troppi lock, si riduce carico. La validazione non è una formalità; è uno strumento decisionale.

Ogni validazione dovrebbe rispondere a:

il comportamento è pronto per demo?

è pronto per hardware visuale?

è pronto per hardware attuativo?

serve migliorare parametri?

serve migliorare pattern?

serve migliorare safety?

serve migliorare visualizzazione?

serve ridurre ambizione?

Queste domande collegano simulazione e roadmap reale.

Il trentesimo criterio è la validazione come prova dell'ecosistema. Quando tutte le parti lavorano insieme — modelli virtuali, pattern, safety, dashboard, telemetria, report — il progetto diventa molto più forte. Non è più una collezione di file o idee, ma un ambiente sperimentale. La validazione del comportamento è ciò che dimostra che l'ecosistema ha coerenza interna. Ogni nodo, ogni comando, ogni metrica e ogni evento partecipa a una logica comune.

In conclusione, la validazione del comportamento è il processo attraverso cui MicroBot verifica che lo swarm non solo formi pattern visivamente corretti, ma si comporti in modo coerente, stabile, misurabile, sicuro e ripetibile. Essa controlla geometria, topologia, ruoli, fasi, tempo, safety, energia, temperatura, errori, fallback, recovery, dati, dashboard e report. Un comportamento valido deve rispettare il pattern dichiarato, mantenere stabilità nel tempo, convergere entro soglie, gestire errori in modo tracciato e non violare i limiti del Safety Layer. La validazione deve produrre classificazioni chiare come `success_clean`, `success_with_corrections`, `partial_success_with_fallback`, `failed_topology` o `failed_safety`, evitando giudizi ambigui. In questo modo, la simulazione MicroBot diventa un vero strumento di verifica ingegneristica. Il punto successivo, dedicato all'iterazione veloce, mostrerà come questi test e validazioni permettano di modificare rapidamente parametri, pattern, interfacce e modelli, accelerando lo sviluppo senza perdere rigore.

12.7 Iterazione veloce

L'iterazione veloce rappresenta uno dei vantaggi principali della simulazione software nel progetto MicroBot. Dopo aver definito l'ambiente di simulazione, l'implementazione web, la visualizzazione, i modelli virtuali, i test dei pattern e la validazione del comportamento, diventa necessario spiegare perché tutto questo deve essere progettato per permettere modifiche rapide, prove frequenti, confronto tra versioni e miglioramento continuo. Un sistema complesso come MicroBot non può essere sviluppato soltanto attraverso grandi blocchi teorici o test hardware lenti. Deve poter evolvere per cicli brevi: si modifica un parametro, si esegue una simulazione, si osservano metriche, si corregge, si salva il risultato, si confronta con la versione precedente e si decide il passo successivo.

Nel progetto MicroBot, l'iterazione veloce non deve essere confusa con sviluppo disordinato. Iterare velocemente non significa cambiare file a caso, aggiungere funzioni senza logica o inseguire ogni idea senza validazione. Significa costruire un ambiente in cui ogni modifica può essere provata rapidamente e valutata con criteri chiari. La velocità deve essere accompagnata da tracciabilità. Ogni esperimento deve avere input, output, configurazione, metriche e risultato. In questo modo, il progetto può crescere rapidamente senza perdere rigore.

Il principio generale è:

modifica controllata → simulazione rapida → osservazione → metrica → decisione → nuova versione

Questa sequenza descrive il ciclo di sviluppo più importante per la simulazione MicroBot. Il punto non è soltanto “provare tante cose”, ma provare tante cose in modo misurabile. Se si aumenta `cohesion_weight`, bisogna vedere se `aggregation_score` migliora, se `collision_count` aumenta, se `stability_score` peggiora o se il tempo di convergenza cambia. Se si modifica il pattern `circle_basic`, bisogna confrontare `radial_error`, `angular_variance` e `closure_score` prima e dopo. Se si introduce una nuova policy di safety, bisogna verificare se riduce `thermal_lock` senza bloccare troppo spesso il comportamento. L'iterazione veloce serve a produrre conoscenza tecnica.

La prima area in cui l'iterazione veloce è fondamentale è il tuning dei parametri. I pattern MicroBot dipendono da molti parametri: coesione, separazione, allineamento, attrazione verso target,

damping, velocità massima, raggio di connessione, tolleranza geometrica, cooldown, duty virtuale, soglie energetiche e termiche. Molti di questi valori non possono essere scelti perfettamente a priori. Devono essere provati. Una simulazione permette di eseguire rapidamente più combinazioni e confrontare risultati. Questo è molto più efficiente che provare ogni modifica su hardware reale.

Un ciclo di tuning può essere:

scegliere un parametro;

definire un intervallo di valori;

eseguire più run;

registrare metriche;

confrontare risultati;

scegliere valore candidato;

validarlo con rumore o fault;

salvare il profilo migliore.

Per esempio, in un cluster, aumentare troppo la coesione può generare collasso o collisioni. Aumentare troppo la separazione può impedire aggregazione. Serve un equilibrio. La simulazione permette di trovarlo. Il risultato può essere salvato in `behavior_profiles.json`, così il profilo non resta nella memoria dello sviluppatore ma diventa parte del progetto.

La seconda area è la modifica dei pattern. Un pattern può essere definito in un modo e poi migliorato. Una linea può passare da target statici a target ricalcolati. Un cerchio può migliorare la distribuzione angolare. Una griglia può introdurre fallback `partial_grid`. Un cluster può aggiungere seeker per nodi isolati. Un ponte può passare da catena fissa a catena adattiva. Ogni modifica deve essere provata in simulazione, non solo descritta nel documento.

Un ciclo di iterazione sui pattern può essere:

definire `pattern_v0.1`;

eseguire test ideale;

rilevare limiti;

aggiungere correzione o fallback;

creare `pattern_v0.2`;

rieseguire test baseline;

confrontare metriche;

decidere se promuovere la nuova versione.

Questo processo evita di sovrascrivere continuamente il pattern senza sapere se la nuova versione è migliore. Ogni versione deve avere motivo. Se `line_basic_v0.2` riduce `lateral_error` ma aumenta

spacing_error, non è automaticamente migliore. Serve valutazione. La simulazione rende questa valutazione possibile.

La terza area è la visualizzazione. Anche l'interfaccia web deve iterare velocemente. Può essere necessario cambiare colori, badge, pannelli, overlay, metrica principale, legenda o layout. Tuttavia, anche qui l'iterazione deve essere guidata dalla leggibilità. Una dashboard più bella ma meno chiara non è un miglioramento. La visualizzazione deve aiutare a capire nodi, ruoli, edge, safety e metriche. La simulazione permette di testare la dashboard con snapshot diversi: pattern pulito, nodo offline, thermal_lock, fallback, recovery, molti nodi, pochi nodi.

Un ciclo di iterazione UI può essere:

caricare snapshot standard;

osservare leggibilità;

attivare overlay debug;

verificare stati critici;

modificare layout;

testare su swarm piccolo e medio;

verificare che i colori restino coerenti;

salvare screenshot o nota di versione.

La dashboard deve evolvere insieme al modello dati. Se vengono aggiunti thermal_state, energy_state o recovery_state, la UI deve rappresentarli. Se vengono aggiunti gruppi o sub-pattern, la UI deve poterli mostrare. L'iterazione veloce evita che la dashboard resti indietro rispetto al motore.

La quarta area è il Safety Layer simulato. Corrente, bobine, temperatura, batteria, energia, watchdog e recovery possono essere testati in simulazione con molta più sicurezza rispetto all'hardware reale. Si può forzare thermal_lock, generare low_energy, simulare watchdog_reset, bloccare una capability, inviare un comando scaduto o creare controller_lost. Ogni scenario può essere provato rapidamente. Questo permette di migliorare le policy prima di collegare carichi reali.

Un ciclo di iterazione safety può essere:

creare scenario di rischio;

eseguire run simulata;

osservare evento safety;

verificare rifiuto comandi;

controllare dashboard;

controllare log;

modificare soglie o policy;

ripetere.

Per esempio, se `thermal_warning` blocca troppo presto un pattern semplice, forse la soglia è troppo conservativa per la simulazione. Se `thermal_lock` arriva troppo tardi, la policy è rischiosa. Se `low_energy` non riassegna il ruolo, il controller deve essere corretto. Tutti questi problemi possono emergere senza danneggiare hardware.

La quinta area è la comunicazione e la sincronizzazione. Anche se la comunicazione reale con ESP32 avrà caratteristiche proprie, il simulatore può introdurre latenza, `packet_loss`, jitter, `ack_timeout` e `sync_degraded`. Questo permette di testare il controller. L'iterazione veloce consente di provare diverse soglie di timeout, diverse strategie di retry, diversi intervalli di heartbeat e diversi limiti per dichiarare un nodo stale o offline.

Un ciclo di iterazione comunicativa può essere:

impostare profilo rete ideale;

validare pattern;

aggiungere latenza moderata;

misurare `late_ack` e `command_expired`;

aggiungere `packet_loss`;

osservare retry;

modificare timeout;

verificare che recovery non sia troppo aggressiva;

salvare `network_profile`.

Questo processo è essenziale perché un sistema distribuito non può assumere comunicazione perfetta. Meglio scoprire i problemi nella simulazione che durante una demo fisica.

La sesta area è il confronto tra varianti. L'iterazione veloce deve permettere di confrontare più versioni dello stesso comportamento. Per esempio, `cluster_basic` con profilo A, B e C; `line_basic` con damping basso, medio e alto; `circle_basic` con anchor fisso o anchor dinamico; `bridge_complex` con catena fissa o adattiva. Il confronto deve essere basato su metriche. Questo permette di scegliere non solo la variante più bella, ma quella più stabile, sicura e ripetibile.

Una tabella comparativa può includere:

versione del pattern;

profilo comportamentale;

`success_rate`;

tempo medio di convergenza;

errore finale medio;

`topology_score` medio;

correzioni medie per run;

safety events;

risultato consigliato.

Questo tipo di confronto è molto utile nella documentazione. Mostra che le scelte progettuali non sono arbitrarie, ma basate su prove.

La settima area è la gestione degli esperimenti. Se si fanno molte prove, bisogna organizzarle. Altrimenti il progetto diventa una massa di file e risultati difficili da interpretare. Ogni esperimento dovrebbe avere un nome, una descrizione, un obiettivo, una configurazione, un output e una conclusione. I file `experimental_runs.json` possono servire proprio a questo. Un esperimento può essere “testare se `cluster_basic` resta stabile con 30 nodi e rumore medio”. Un altro può essere “verificare `fallback_circle_to_arc` con nodo mancante”. Un altro ancora può essere “confrontare tre `cooldown_policy`”.

Una struttura esperimento può contenere:

`experiment_id`;

`title`;

`purpose`;

`pattern_id`;

`configuration_file`;

`behavior_profile`;

`fault_profile`;

`number_of_runs`;

`metrics`;

`expected_outcome`;

`actual_result`;

`notes`;

`next_action`.

Questa struttura trasforma l'iterazione in ricerca. Non si fanno prove casuali; si fanno esperimenti.

L'ottava area è la versioning dei file. Ogni modifica significativa a pattern, profili, configurazioni o policy dovrebbe avere una versione. Non serve creare complessità eccessiva, ma è utile distinguere `line_basic_v0.1` da `line_basic_v0.2`. Lo stesso vale per `behavior_profile`, `safety_policy`, `visualization_config` e `simulation_config`. Se non si versiona, diventa difficile capire quale configurazione ha prodotto un certo risultato.

Una convenzione può essere:

```
pattern_id = line_basic;  
  
pattern_version = v0.2;  
  
behavior_profile_id = balanced_swarm_v0.1;  
  
safety_policy_id = conservative_safety_v0.1;  
  
simulation_config_id = baseline_5_nodes_v0.1.
```

Questi identificatori devono entrare nei report. Così, quando si rilegge un risultato, si sa esattamente quale sistema lo ha generato.

La nona area è la riduzione del tempo tra idea e prova. Un ambiente ben progettato deve permettere di cambiare un file JSON, rilanciare la simulazione e vedere il risultato. Se ogni prova richiede modifiche profonde al codice, l'iterazione rallenta. Per questo motivo, pattern, parametri e scenari devono essere configurabili. Il codice deve essere il motore; i file di configurazione devono guidare l'esperimento. Questo è molto importante per MicroBot, perché il progetto ha tante componenti e deve restare gestibile.

Una pipeline agile può essere:

```
modifica sample_config.json;  
  
avvia run;  
  
dashboard aggiorna;  
  
report generato;  
  
confronto con run precedente.
```

Questo ciclo deve essere il più semplice possibile. Più è breve, più il progetto cresce velocemente.

La decima area è la separazione tra esperimenti e demo. Non tutte le simulazioni devono essere belle e pulite. Alcune devono essere brutte, rumorose, difficili, piene di errori, proprio per testare robustezza. Altre devono essere pensate per demo, quindi più stabili e leggibili. L'iterazione veloce deve supportare entrambe. Una modalità experimental può mostrare warning, overlay, fault e metriche dettagliate. Una modalità demo può usare parametri più controllati, visualizzazione pulita e narrativa chiara.

Si possono distinguere:

```
experimental_run: test tecnico, anche instabile;  
  
validation_run: test misurato con criteri precisi;  
  
demo_run: comportamento pulito e presentabile;  
  
stress_run: prova dei limiti;  
  
hardware_precheck_run: verifica prima dell'hardware.
```

Questa distinzione evita confusione. Se una run sperimentale fallisce, non significa che il progetto sia fragile. Significa che si stanno esplorando limiti. Se una demo_run fallisce, allora bisogna intervenire prima di presentarla.

L'undicesima area è il debug rapido. Quando qualcosa non funziona, il sistema deve aiutare a capire dove. Il problema può essere nel pattern, nel modello virtuale, nella visualizzazione, nella telemetria, nella safety, nella comunicazione simulata o nella configurazione. Per iterare velocemente, servono log chiari, eventi recenti, snapshot leggibili e metriche per fase. Senza strumenti di debug, ogni problema richiede troppo tempo.

Una buona iterazione richiede:

event console;

snapshot corrente;

log per fase;

pannello nodo;

pannello edge;

metriche principali;

error summary;

failure reason.

Se un cerchio non si chiude, bisogna sapere se il problema è geometrico, topologico, di ruolo, di fase o di safety. Se una griglia fallisce, bisogna sapere se mancano nodi, se celle sono sbagliate o se edge non si formano. Il debug rapido dipende dalla qualità dell'osservabilità.

La dodicesima area è la modifica controllata dei parametri durante runtime. In una dashboard avanzata, alcuni parametri possono essere modificati live: cohesion_weight, separation_weight, target_weight, simulation_speed, noise_level, overlay, fault injection. Questo è utile per esplorazione. Tuttavia, bisogna distinguere tra modifica live e configurazione validata. Una modifica live può aiutare a capire il comportamento, ma per una validazione seria il parametro finale deve essere salvato in config e rieseguito da zero.

Una regola corretta è:

le modifiche live servono a esplorare; le configurazioni salvate servono a validare.

Questo evita di basare risultati ufficiali su prove non ripetibili. Se durante una run si cambia manualmente un parametro e il pattern riesce, bisogna salvare quella configurazione e ripetere il test in modo controllato.

La tredicesima area è l'uso dei replay per iterare. Un replay permette di rivedere una run fallita e capire il punto esatto del problema. Questo riduce il tempo di debug. Invece di rilanciare continuamente la simulazione, si può fermare al momento del failure, osservare metrica, nodo, edge, evento e fase. Il replay è molto utile per pattern complessi, perché l'errore può nascere molto prima del fallimento finale.

Un replay efficace deve permettere:

play/pause;
step avanti/indietro;
salto a evento;
salto a fase;
visualizzazione metriche nel tempo;
selezione nodo;
confronto target/stato.

Questa funzione rende l'iterazione molto più potente. Una run fallita diventa materiale di studio, non solo un errore da dimenticare.

La quattordicesima area è il confronto tra snapshot. A volte è utile confrontare stato iniziale, stato intermedio e stato finale. Oppure confrontare due versioni del pattern nello stesso momento. Questo aiuta a capire miglioramenti. Per esempio, line_basic_v0.2 può avere errore minore ma richiedere più tempo. Una visualizzazione comparativa o un report comparativo possono mostrare queste differenze.

Un confronto può mostrare:

snapshot iniziale;
snapshot a metà run;
snapshot finale;
errore nel tempo;
topology_score nel tempo;
eventi principali.

Questa analisi aiuta a capire non solo se il sistema è arrivato, ma come ci è arrivato.

La quindicesima area è l'automazione dei test ripetitivi. Quando alcuni test diventano importanti, non devono essere lanciati sempre a mano. Una suite può eseguire baseline patterns, safety tests e recovery tests. Questo permette di verificare rapidamente se una modifica rompe qualcosa. Inizialmente può bastare uno script semplice. In futuro, può diventare parte della repo.

Una suite automatica può eseguire:

line_basic_5_nodes;
circle_basic_8_nodes;
grid_basic_3x3;
cluster_basic_20_nodes;
missing_node_fault;

thermal_lock_fault;

low_energy_fault;

watchdog_reset_fault.

Al termine, produce una tabella con pass/fail e metriche principali. Questo è un passo enorme verso qualità software.

La sedicesima area è la gestione dei regressions. Se una modifica peggiora un pattern già funzionante, bisogna saperlo subito. Per esempio, aggiungere una nuova regola di separazione potrebbe migliorare i cluster ma peggiorare le linee. Senza regression test, ci si accorge tardi. L'iterazione veloce deve quindi includere controlli su comportamenti già validati. Ogni nuova funzione deve essere confrontata con la suite base.

Una regola di sviluppo può essere:

nessuna modifica importante viene considerata stabile finché non supera i test baseline.

Questo non significa che ogni esperimento debba passare tutti i test. Ma significa che, prima di promuovere una modifica come nuova versione principale, bisogna verificare che non rompa il resto.

La diciassettesima area è l'iterazione sulle soglie di successo. All'inizio, alcune soglie possono essere troppo larghe o troppo strette. Se pattern_error_threshold è troppo largo, il sistema considera successi comportamenti mediocri. Se è troppo stretto, rifiuta comportamenti accettabili. Le soglie devono essere calibrate. La simulazione permette di osservare molte run e scegliere tolleranze realistiche.

Un processo di calibrazione può essere:

eseguire run ideali;

osservare errori finali normali;

eseguire run con rumore;

osservare variabilità;

definire soglia accettabile;

definire soglia warning;

definire soglia failure.

Questo rende la validazione più realistica. Non si usano soglie inventate, ma derivate dall'osservazione.

La diciottesima area è l'iterazione sulla narrativa della demo. MicroBot non è solo un sistema tecnico; deve essere anche spiegabile. La simulazione permette di costruire sequenze dimostrative: avvio nodi, assegnazione ruoli, formazione linea, errore simulato, recovery, report finale. Questa narrativa può essere iterata come un prodotto. Si prova cosa si capisce, cosa è confuso, quali metriche mostrare, quando evidenziare safety, quale pattern usare per primo. Una demo efficace nasce da iterazioni.

Una sequenza demo può essere:

mostrare swarm idle;

avviare pattern line_basic;

assegnare ruoli;

mostrare formazione;

iniettare nodo offline;

mostrare fallback;

mostrare report finale.

Questa sequenza racconta il valore del sistema: coordinazione, pattern, errore, recovery, misurazione. È molto più forte di una semplice animazione perfetta.

La diciannovesima area è l'iterazione tra simulazione e documentazione. Ogni volta che un comportamento viene validato o modificato, la documentazione deve essere aggiornata. Se la documentazione dice che circle_basic supporta fallback ad arco, il simulatore deve implementarlo. Se il simulatore mostra che il fallback ha limiti, la documentazione deve dirlo. La simulazione e il documento devono evolvere insieme. Questo è fondamentale per evitare incoerenza.

Un ciclo corretto è:

documento definisce comportamento;

simulatore implementa;

test produce risultati;

documento aggiorna limiti e metriche;

repo salva configurazione.

Questo rende il progetto compatto. La documentazione non è teoria scollegata; è specifica tecnica viva.

La ventesima area è l'iterazione tra simulazione e hardware. Quando un pattern è stabile in simulazione, può essere candidato a una demo hardware visuale. Quando una funzione hardware è disponibile, può essere integrata in modalità hybrid. Se l'hardware mostra limiti non previsti, la simulazione può essere aggiornata. Per esempio, se un ESP32 ha latenza più alta del previsto, il network_profile simulato va modificato. Se una bobina richiede cooldown più lungo, il thermal_model va aggiornato. Questo scambio rende la simulazione sempre più realistica.

Un ciclo simulation-hardware può essere:

simulazione prevede comportamento;

hardware test misura limite reale;

modello simulativo viene aggiornato;

pattern viene ritestato;

demo hardware viene riprovata.

Questo è uno dei cicli più importanti per portare MicroBot dalla teoria al fisico. La simulazione non resta fantasia; viene calibrata con dati reali.

La ventunesima area è la gestione delle priorità. L'iterazione veloce può generare moltissime idee. Il rischio è disperdersi. Per evitarlo, bisogna classificare le modifiche: core, utile, sperimentale, futura. Core sono modifiche necessarie al sistema: snapshot, nodi, pattern base, safety, report. Utili sono miglioramenti: overlay, replay, fault injection. Sperimentali sono idee da provare: 3D avanzato, AI tuning, campi complessi. Future sono integrazioni non immediate. Questa classificazione mantiene focus.

Una priorità può essere:

P0: necessario per funzionamento base;

P1: necessario per demo e debug;

P2: utile per validazione;

P3: sperimentale;

P4: futuro.

Ogni iterazione deve sapere a che priorità appartiene. Questo aiuta a non perdere mesi su effetti visivi prima che pattern e safety siano solidi.

La ventiduesima area è la definizione di "done" per una iterazione. Ogni ciclo deve avere una conclusione. Per esempio, "line_basic_v0.2 è done quando supera 10 run ideali, 10 run con rumore leggero, non genera safety events e produce report". Senza definizione di done, si continua a modificare all'infinito. Una definizione chiara permette di chiudere una versione e passare oltre.

Esempi di done:

pattern implementato;

test baseline superati;

metriche salvate;

dashboard aggiornata;

README aggiornato;

limiti documentati;

prossimi passi indicati.

Questo metodo trasforma l'iterazione in avanzamento reale.

La ventitreesima area è l'archiviazione dei risultati. Ogni iterazione importante dovrebbe lasciare traccia: config, report, screenshot, note. Non serve archiviare ogni prova minima, ma le run

significative sì. Questo crea una memoria tecnica del progetto. In futuro, quando bisogna scrivere una relazione o mostrare progressi, questi dati saranno preziosi.

Una cartella risultati può contenere:

runs/;

reports/;

snapshots/;

screenshots/;

notes/;

comparisons/.

Ogni run significativa può avere run_id. Questo rende il lavoro recuperabile. Senza archiviazione, molte scoperte si perdono.

La ventiquattresima area è l'iterazione sulle performance. Se la simulazione rallenta con molti nodi, bisogna ottimizzare. L'iterazione veloce permette di misurare step_time, frame_rate, numero di nodi, costo del rendering, costo del calcolo edge. Alcune ottimizzazioni possono essere necessarie prima della scalabilità. Per esempio, calcolare distanza tra tutti i nodi è $O(N^2)$, e con molti nodi può diventare pesante. La simulazione deve rivelare questi limiti.

Metriche performance:

simulation_step_ms;

render_frame_ms;

snapshot_size_kb;

nodes_count;

edges_count;

fps;

memory_usage;

event_rate.

Queste metriche preparano la Sezione 19. L'iterazione veloce aiuta a capire quando un approccio non scala.

La venticinquesima area è l'iterazione dei modelli matematici. Alcuni modelli iniziali saranno semplici. Poi potranno diventare più raffinati. Per esempio, un campo magnetico virtuale può iniziare come forza radiale semplificata. Poi può introdurre decadimento con distanza. Poi può considerare orientamento. Poi può essere calibrato con test fisici. L'iterazione permette di passare da modello qualitativo a modello più fisico senza bloccare il progetto all'inizio.

Una progressione modello può essere:

modello qualitativo;

modello parametrico;

modello calibrato;

modello validato parzialmente;

modello hardware-informed.

Questo è molto importante per MicroBot, perché il progetto contiene matematica, fisica e hardware. Non serve avere subito un modello perfetto; serve avere un modello migliorabile.

La ventiseiesima area è l'iterazione sulle ipotesi. Ogni simulazione si basa su assunzioni: i nodi possono muoversi, possono conoscere posizione, possono rilevare vicini, possono comunicare con certa latenza, possono stimare energia. Alcune assunzioni sono realistiche, altre future. L'iterazione veloce deve permettere di testare cosa succede se una assunzione viene indebolita. Per esempio, cosa succede se i nodi non conoscono posizione precisa? Cosa succede se la comunicazione è lenta? Cosa succede se l'energia è limitata? Questo aiuta a separare visione futura da prototipo attuale.

Ogni esperimento dovrebbe indicare:

assunzioni attive;

assunzioni simulate;

assunzioni non ancora hardware-validabili;

effetto delle assunzioni sul risultato.

Questa onestà è fondamentale. La simulazione è potente, ma solo se si conoscono le sue assunzioni.

La ventisettesima area è l'iterazione come metodo di apprendimento. Ogni ciclo insegna qualcosa: un pattern è troppo rigido, una metrica non basta, un fallback serve, una UI è confusa, una soglia è sbagliata. Questo apprendimento deve essere catturato. Può essere registrato in `notes.md` o `experimental_log.md`. Nel tempo, queste note diventano storia tecnica del progetto.

Una nota sperimentale può contenere:

data;

obiettivo;

modifica provata;

risultato osservato;

metrica principale;

problema emerso;

decisione;

prossimo passo.

Questa abitudine è molto utile per progetti grandi. Aiuta a non ripetere errori e a spiegare perché certe scelte sono state fatte.

La ventottesima area è l'iterazione con livelli di maturità. Ogni funzione può essere classificata per maturità: idea, simulata, testata, validata, integrata, hardware-parziale, hardware-validata. Questo evita di confondere ciò che è già solido con ciò che è ancora visione. L'iterazione veloce fa avanzare le funzioni da un livello al successivo.

Una scala utile è:

concept;

implemented_in_simulation;

visualized_in_dashboard;

tested_with_metrics;

validated_with_faults;

integrated_with_safety;

hybrid_demo_ready;

hardware_partial;

hardware_validated.

Per esempio, "thermal_load" può essere implemented_in_simulation e visualized_in_dashboard, ma non measured_hardware finché non c'è sensore reale. Questa chiarezza è importantissima.

La ventinovesima area è l'iterazione verso una demo stabile. A un certo punto bisogna congelare una versione demo. Non si può modificare tutto fino all'ultimo. L'iterazione veloce serve anche a scegliere una configurazione stabile, salvarla e usarla per presentazione. La demo deve essere ripetibile, non casuale. Deve avere config dedicato, snapshot iniziale, pattern, metriche e fallback controllato.

Una demo stabile dovrebbe avere:

config congelata;

pattern selezionato;

fault opzionale controllato;

dashboard testata;

report generato;

nessun bug noto critico;

istruzioni di avvio;

modalità read-only o demo_control.

Questo è essenziale se devi mostrare il progetto ad Andrea o a un collaboratore. Una demo non deve dipendere da “speriamo che funzioni”.

La trentesima area è l'iterazione come base dello sviluppo sperimentale. Il punto successivo parlerà proprio di sviluppo sperimentale, ma qui si può anticipare il legame: l'iterazione veloce è il motore che permette al progetto di crescere sperimentalmente. Senza iterazione, ogni idea resta teorica o troppo lenta da testare. Con iterazione, ogni idea può essere tradotta in configurazione, simulata, misurata e selezionata. Questo è il cuore del metodo MicroBot.

In conclusione, l'iterazione veloce è la capacità del sistema MicroBot di modificare rapidamente parametri, pattern, modelli, dashboard, safety policy e scenari, verificando ogni modifica attraverso simulazione, metriche, log e report. Essa non è sviluppo casuale, ma ciclo controllato di esperimento e miglioramento. Permette di fare tuning dei parametri, confrontare versioni, testare fault, validare safety, costruire demo, aggiornare documentazione, preparare hardware e ridurre rischi. Un ambiente di simulazione efficace deve quindi essere configurabile, osservabile, ripetibile, misurabile e facile da rilanciare. Solo così MicroBot può crescere velocemente senza perdere coerenza. Il punto successivo, dedicato allo sviluppo sperimentale, chiuderà la Sezione 12 spiegando come la simulazione diventi una metodologia continua di ricerca, prototipazione, validazione e transizione progressiva verso hardware reale, documentazione tecnica e applicazioni future.

12.8 Sviluppo sperimentale

Lo sviluppo sperimentale rappresenta il punto conclusivo della Sezione 12 e descrive il modo in cui la simulazione software diventa una metodologia continua di ricerca, prova, errore, correzione, validazione e progressivo avvicinamento all'hardware reale. Dopo aver definito ambiente di simulazione, implementazione web, visualizzazione, modelli virtuali, test dei pattern, validazione del comportamento e iterazione veloce, diventa necessario chiarire il ruolo complessivo della simulazione all'interno dell'evoluzione del progetto MicroBot. La simulazione non è soltanto uno strumento tecnico: è il luogo in cui il sistema può crescere, sbagliare, correggersi e trasformare idee teoriche in comportamenti progressivamente più solidi.

Nel progetto MicroBot, lo sviluppo sperimentale deve essere inteso come un processo ordinato. Non significa provare casualmente tante idee, ma costruire una catena di esperimenti che permetta di capire cosa funziona, cosa non funziona, cosa è ancora solo teorico, cosa è simulato, cosa è validato, cosa è pronto per una demo e cosa può essere trasferito su hardware. Questo punto è fondamentale perché MicroBot è un progetto ampio: include swarm, pattern, safety, simulazione, firmware, dashboard, telemetria, MicroBot OS, sicurezza biometrica, progetti esterni e visione teorica avanzata. Senza una metodologia sperimentale, il rischio sarebbe disperdersi. Con una metodologia sperimentale, invece, ogni parte può avanzare per livelli.

Il principio generale dello sviluppo sperimentale può essere espresso così:

ipotesi → simulazione → osservazione → misura → correzione → validazione → documentazione
→ trasferimento progressivo

Questa sequenza riassume il metodo. Prima si formula un'ipotesi: per esempio, “uno swarm può formare una linea stabile usando target individuali e spacing correction”, oppure “un nodo in low_energy deve essere escluso dai ruoli attivi”, oppure “un fallback da cerchio ad arco può mantenere partial_success quando manca un nodo”. Poi si implementa la simulazione. Poi si osserva il comportamento nella dashboard. Poi si misurano metriche. Poi si correggono parametri,

pattern o policy. Poi si valida con test. Poi si documenta il risultato. Solo alla fine si decide se portarlo verso hardware, demo o integrazione più ampia.

La simulazione diventa quindi il primo filtro della realtà. Non tutto ciò che è pensato deve essere subito costruito fisicamente. Prima deve passare da una prova virtuale. Se un pattern non converge in simulazione, non ha senso portarlo su ESP32 o bobine. Se una safety policy non blocca correttamente un comando virtuale, non deve controllare un carico reale. Se una dashboard non rappresenta chiaramente un nodo in recovery simulata, sarà ancora più difficile usarla con hardware fisico. Lo sviluppo sperimentale permette di ridurre rischio, costo e confusione.

Il primo livello dello sviluppo sperimentale riguarda le idee concettuali. MicroBot nasce da concetti ampi: unità distribuite, materia programmabile, swarm intelligence, aggregazione, pattern, ruoli, sicurezza, telemetria e sistemi riconfigurabili. Questi concetti devono essere trasformati in esperimenti. Un concetto come “forma dinamica” diventa un pattern `line_to_circle`. Un concetto come “adattamento” diventa un test con nodo mancante. Un concetto come “sicurezza distribuita” diventa un comando rifiutato localmente dal nodo. Un concetto come “energia limitata” diventa `low_energy simulation`. La simulazione serve a rendere operative le idee.

Una buona domanda sperimentale è:

quale comportamento osservabile dimostra questo concetto?

Per esempio, per dimostrare adattamento, non basta scrivere che il sistema è adattivo. Serve una run in cui un nodo fallisce e il pattern si riorganizza. Per dimostrare safety, non basta scrivere che esiste un Safety Layer. Serve una run in cui un comando non sicuro viene rifiutato e registrato. Per dimostrare telemetria, non basta dire che il sistema monitora stato. Serve una dashboard che mostra nodi, eventi e metriche. Questo è lo spirito dello sviluppo sperimentale.

Il secondo livello riguarda gli esperimenti software puri. In questa fase non serve hardware. Si lavora con nodi virtuali, snapshot, dashboard, metriche e log. Gli esperimenti software permettono di testare pattern, topologie, fault injection, safety simulata, recovery e scalabilità. Sono veloci, economici e ripetibili. Devono essere usati intensamente prima di ogni passo fisico.

Un esperimento software può essere:

`line_basic` con 10 nodi e rumore leggero;

`circle_basic` con un nodo rimosso durante la fase di chiusura;

`cluster_basic` con `isolated_node`;

`bridge_complex` con `connector in thermal_lock`;

`grid_basic` con cella mancante;

swarm da 100 nodi per testare performance;

`controller_lost` durante `phase_attach`;

`watchdog_reset` simulato su nodo leader.

Ognuno di questi esperimenti risponde a una domanda tecnica precisa. Non sono animazioni isolate: sono prove.

Il terzo livello riguarda gli esperimenti visuali. Qui l'obiettivo non è solo verificare il comportamento interno, ma renderlo comprensibile. Una simulazione può essere tecnicamente corretta ma visivamente confusa. Lo sviluppo sperimentale deve quindi includere test sulla dashboard: i nodi sono leggibili? I ruoli si distinguono? Gli edge mancanti si vedono? Il `thermal_warning` è chiaro? Il fallback è comprensibile? L'utente capisce se il sistema è in simulazione o in hardware? Queste domande sono importanti perché MicroBot deve essere anche presentabile.

Un esperimento visuale può confrontare:

visualizzazione con label sempre attive;

visualizzazione con label su hover;

colori diversi per ruoli;

bordo per safety;

badge per energia;

overlay target attivo o nascosto;

console eventi compatta o estesa.

Il risultato non è solo estetico. Una visualizzazione più chiara migliora debugging, documentazione e comunicazione esterna. Per un progetto come MicroBot, che deve essere mostrato a collaboratori, docenti, investitori o tecnici, la visualizzazione è parte del valore.

Il quarto livello riguarda gli esperimenti di safety simulata. Questi sono fondamentali perché permettono di testare condizioni rischiose senza rischio reale. Si può simulare `thermal_lock`, `critical_energy`, bobina in cooldown, comando con duty troppo alto, comando scaduto, `emergency_stop`, `reset`, `controller_lost` e `recovery`. Ogni esperimento deve verificare che il sistema reagisca correttamente. Se una bobina virtuale è in `thermal_lock`, il comando `magnetic_pulse` deve essere rifiutato. Se il nodo è in `critical_energy`, deve passare a `safe_idle`. Se arriva `emergency_stop`, tutti i nodi devono bloccare uscite critiche.

Questi esperimenti preparano direttamente l'hardware. Prima di pilotare una bobina reale, il software deve già sapere che cosa fare se qualcosa va male. Prima di avere una batteria reale, il sistema deve già gestire `low_energy`. Prima di un watchdog reale, il controller deve già sapere come reagire a `boot_id` cambiato. La simulazione non sostituisce la sicurezza fisica, ma prepara la logica di sicurezza.

Il quinto livello riguarda gli esperimenti ibridi. In una fase intermedia, alcuni nodi possono essere fisici e altri simulati. Per esempio, una dashboard può mostrare 30 MicroBot virtuali e 3 ESP32 reali che ricevono ruoli e mostrano stato tramite LED. Questa modalità è molto importante per MicroBot perché permette di collegare visione grande e prototipo reale. Il sistema può dimostrare lo swarm completo in simulazione e, allo stesso tempo, mostrare che una parte della pipeline comunica con hardware fisico.

Un esperimento ibrido può essere:

swarm virtuale con pattern linea;

tre ESP32 reali associati a leader, connector e terminal;

dashboard mostra tutti i nodi;

ESP32 lampeggiano secondo ruolo;

controller riceve heartbeat reale;

nodi virtuali formano pattern;

report distingue dati simulated e hardware.

Questo tipo di esperimento è estremamente utile per presentare il progetto in modo onesto. Non pretende che tutti i nodi siano fisici, ma mostra che l'architettura è pronta a integrare nodi reali.

Il sesto livello riguarda gli esperimenti hardware-safe. Quando si passa all'hardware, bisogna procedere lentamente. Prima LED. Poi heartbeat. Poi comandi visuali. Poi sync flash. Poi eventuale PWM su LED. Poi carico sicuro. Poi driver. Poi bobina con impulso minimo. Ogni passaggio deve essere preceduto da simulazione e seguito da log. Lo sviluppo sperimentale non deve saltare direttamente a bobine e magneti complessi. Deve costruire fiducia.

Una progressione hardware-safe può essere:

simulazione del comando;

comando visuale su dashboard;

comando LED su ESP32;

comando temporizzato con duration_ms;

emergency_stop visuale;

PWM su LED;

cooldown simulato;

cooldown firmware;

test driver senza carico critico;

test bobina con duty basso e durata breve;

integrazione con pattern solo dopo test.

Questa progressione è prudente e professionale. Permette di evitare errori banali e danni ai componenti.

Il settimo livello riguarda la documentazione sperimentale. Ogni esperimento significativo deve essere documentato. La documentazione non deve essere solo "ho provato e funziona". Deve indicare scopo, configurazione, risultato, metriche, problemi, screenshot, log e prossimo passo. Questo trasforma il lavoro in materiale tecnico riutilizzabile. È importante per la tesi, per GitHub, per eventuali collaboratori e per la memoria del progetto.

Una scheda esperimento dovrebbe contenere:

Experiment ID;

titolo;

obiettivo;

ipotesi;

configurazione;

pattern usato;

numero nodi;

modalità: simulation, hybrid, hardware;

parametri principali;

eventuali fault;

metriche osservate;

risultato;

problemi;

conclusione;

prossimo passo.

Questa struttura può diventare una cartella experiments/ nella repo. Ogni esperimento importante può avere un file Markdown e, se necessario, JSON, snapshot, screenshot e report.

L'ottavo livello riguarda la distinzione tra prova esplorativa e prova validata. Una prova esplorativa serve a capire. Può fallire, essere sporca, cambiare parametri live, usare dashboard instabile. Una prova validata, invece, deve essere ripetibile. Deve usare configurazione salvata, parametri noti, test definito e report. Questa distinzione è fondamentale. Non tutto ciò che si vede in una prova esplorativa può essere dichiarato come risultato. Prima bisogna trasformarlo in prova validata.

Una frase chiave è:

l'esplorazione produce intuizioni; la validazione produce risultati.

Questo principio mantiene rigore. Se durante una prova casuale un cluster si comporta bene, bisogna salvare la configurazione e ripetere. Solo dopo può diventare risultato documentato.

Il nono livello riguarda la maturità delle funzioni. Ogni funzione del progetto può trovarsi a un diverso livello: idea teorica, simulazione, dashboard, test, validazione, modalità ibrida, hardware parziale, hardware completo. Lo sviluppo sperimentale serve a far avanzare le funzioni da un livello al successivo. Per esempio, thermal_load può iniziare come concetto, poi diventare variabile simulata, poi apparire nella dashboard, poi bloccare comandi virtuali, poi essere implementata nel firmware, poi essere calibrata con temperatura reale. Questo è un percorso sperimentale chiaro.

Una scala di maturità può essere:

concept_only;

implemented_in_simulation;
visible_in_dashboard;
tested_with_metrics;
validated_with_faults;
integrated_with_safety;
implemented_in_firmware;
tested_on_safe_hardware;
tested_on_real_actuator;
ready_for_demo.

Questa scala evita confusione. Un elemento può essere molto avanzato concettualmente ma non ancora hardware-ready. Documentarlo così non lo indebolisce; lo rende leggibile e credibile.

Il decimo livello riguarda la gestione dei fallimenti. Lo sviluppo sperimentale deve accettare il fallimento come informazione. Se un pattern fallisce, non significa che il progetto fallisce. Significa che il pattern, il parametro o la policy deve essere modificato. Un fallimento utile è un fallimento tracciato. Deve indicare `failure_reason`: errore geometrico, topologico, safety, timeout, energia, temperatura, ruolo, comunicazione o configurazione. Senza `failure_reason`, il fallimento insegna poco. Con `failure_reason`, diventa guida.

Una buona analisi del fallimento deve rispondere a:

dove è fallito?

quando è fallito?

quale metrica è uscita soglia?

quale nodo o edge è coinvolto?

il sistema ha rilevato l'errore?

ha tentato correzione?

ha applicato fallback?

ha rispettato safety?

cosa va modificato?

Queste domande trasformano il problema in sviluppo.

L'undicesimo livello riguarda la ricerca dei limiti. Non bisogna testare solo ciò che funziona. Bisogna cercare intenzionalmente i limiti. Quanti nodi può gestire la dashboard prima di rallentare? Quanto rumore tollera il cluster? Quanta latenza sopporta il pattern? Quanti fault può gestire prima di fallire? Quanti retry sono sostenibili prima di `thermal_warning`? Queste domande sono importanti perché definiscono il confine operativo del sistema.

Esperimenti di limite possono includere:

aumentare node_count fino a perdita performance;

aumentare packet_loss fino a fallimento;

aumentare noise fino a perdita convergenza;

ridurre energy_level iniziale;

aumentare fault simultanei;

ridurre cooldown;

aumentare complessità topologica.

Il risultato non deve essere sempre successo. Il risultato è sapere dove il sistema smette di funzionare. Questo è fondamentale per una documentazione seria.

Il dodicesimo livello riguarda la transizione da simulazione a prototipo. Un comportamento simulato non diventa automaticamente hardware. Serve una procedura di trasferimento. Prima si identifica il sottoinsieme del comportamento che può essere fisicamente dimostrato. Per esempio, da line_basic completo si può trasferire role assignment e sync LED, anche se il movimento resta simulato. Da safety_layer completo si può trasferire command rejection e cooldown logico, anche se la bobina non è ancora collegata. Da pattern bridge si può trasferire dashboard e ruoli, anche senza aggancio fisico. Questo permette di fare passi reali senza pretendere subito tutto.

Una procedura di trasferimento può essere:

comportamento validato in simulazione;

identificazione della parte hardware-realizzabile;

riduzione del pattern a demo fisica parziale;

implementazione su ESP32/LED/PWM sicuro;

test hardware isolato;

integrazione con dashboard;

documentazione della differenza tra simulato e reale.

Questa procedura è molto importante perché evita l'errore di dire "non posso dimostrare tutto, quindi non dimostro niente". Si può dimostrare una parte, dichiarando chiaramente cosa è reale e cosa è simulato.

Il tredicesimo livello riguarda la costruzione di demo progressive. Ogni demo deve avere uno scopo. Una demo può mostrare identità nodi. Un'altra comunicazione. Un'altra sync. Un'altra pattern visuale. Un'altra safety. Un'altra recovery. Un'altra hybrid simulation. Non serve che ogni demo mostri tutto. Anzi, demo troppo complesse sono più fragili. Meglio avere più demo ordinate, ognuna con messaggio chiaro.

Demo progressive possibili:

Demo 1: nodo LED identificabile;

Demo 2: heartbeat e dashboard;

Demo 3: assegnazione ruoli visuale;

Demo 4: sync flash tra più nodi;

Demo 5: pattern simulato con dashboard;

Demo 6: fault injection e recovery;

Demo 7: safety command rejection;

Demo 8: hybrid swarm con nodi reali e virtuali;

Demo 9: PWM/cooldown su carico sicuro;

Demo 10: test attuativo controllato.

Questa sequenza è molto più forte di una singola demo enorme e instabile. Ogni demo costruisce credibilità.

Il quattordicesimo livello riguarda il rapporto tra simulazione e MicroBot OS. Anche se MicroBot OS sarà trattato nella Sezione 14, lo sviluppo sperimentale può anticipare il collegamento: alcune funzioni che nel simulatore sono moduli software potranno in futuro diventare servizi o componenti dell'OS embedded. Per esempio, scheduler, safety manager, telemetry manager, role manager, command dispatcher, diagnostic shell. La simulazione permette di progettare questi moduli prima di portarli nel firmware o nell'OS. In questo senso, il simulatore è anche un prototipo concettuale del sistema operativo distribuito.

La logica è:

prima si testa il comportamento nel simulatore;

poi si isola il modulo logico;

poi si implementa nel firmware;

poi può diventare servizio MicroBot OS.

Per esempio, la gestione del cooldown può nascere nel simulatore, poi passare al firmware, poi diventare parte del safety service. Questa continuità è molto potente.

Il quindicesimo livello riguarda la connessione con la telemetria. La Sezione 13 sarà dedicata proprio a telemetria, ma lo sviluppo sperimentale la prepara. Ogni esperimento deve generare dati. Questi dati devono essere raccolti, visualizzati e interpretati. Senza telemetria, la simulazione resta immagine. Con telemetria, diventa misurazione. Lo sviluppo sperimentale deve quindi prevedere già quali dati servono: stato nodi, metriche, eventi, energia, temperatura, errori, recovery, pattern score. La telemetria è la memoria numerica dell'esperimento.

Una run sperimentale dovrebbe generare:

snapshot;

eventi;
metriche;
log comandi;
stato finale;
report;
eventuali screenshot.

Questi output saranno la base della Sezione 13.

Il sedicesimo livello riguarda la connessione con i progetti esterni. La Sezione 16 parlerà dei progetti esterni: intelligenza artificiale, clustering, giochi, simulazioni evolutive, algoritmi genetici, progetti web, tool diagnostici e script di sistema. Lo sviluppo sperimentale MicroBot può sfruttare alcune competenze nate da questi progetti, ma deve farlo in modo ordinato. Per esempio, gli algoritmi genetici possono servire in futuro per ottimizzare parametri. Il clustering può servire per analizzare gruppi di nodi. Le simulazioni evolutive possono ispirare comportamenti emergenti. I progetti web possono migliorare dashboard. Ma tutto deve essere integrato solo quando serve davvero.

La regola deve essere:

i progetti esterni possono fornire strumenti, ma MicroBot deve mantenere una struttura centrale coerente.

Questo evita dispersione. Lo sviluppo sperimentale può importare idee, ma non deve diventare un miscuglio.

Il diciassettesimo livello riguarda il controllo delle ipotesi teoriche avanzate. MicroBot include anche un livello teorico su dimensioni, tempo, configurazioni e forma come evento dinamico. La simulazione è il punto in cui queste idee possono essere tradotte in modelli osservabili. Per esempio, la “quarta dimensione come tempo” può diventare timeline dei pattern. La “quinta dimensione come configurazioni” può diventare spazio degli stati o transizioni tra forme. “Forma come evento dinamico” può diventare visualizzazione della trasformazione, non solo della forma finale. Lo sviluppo sperimentale permette di dare concretezza a concetti teorici senza forzarli subito sull’hardware.

Un concetto teorico deve diventare:

modello;
metrica;
visualizzazione;
esperimento;
risultato osservabile.

Se non può diventare almeno uno di questi, resta filosofia del progetto, non ancora ingegneria. Va bene, ma va distinto. Questa distinzione rende la parte teorica più forte.

Il diciottesimo livello riguarda la pianificazione dei prossimi esperimenti. Ogni esperimento dovrebbe generare un prossimo passo. Se `line_basic` è stabile, il prossimo passo può essere `sync LED hardware`. Se `circle_basic` fallisce la chiusura, il prossimo passo è migliorare `topology repair`. Se `cluster_basic` genera collisioni, il prossimo passo è aumentare `separation` o `damping`. Se `thermal_lock` virtuale si attiva troppo spesso, il prossimo passo è rivedere `duty/cooldown policy`. Lo sviluppo sperimentale è una catena, non una lista di prove isolate.

Ogni report dovrebbe chiudere con:

`next_experiment;`

`next_parameter_to_test;`

`next_pattern_to_validate;`

`next_hardware_step;`

`next_documentation_update.`

Questo garantisce continuità.

Il diciannovesimo livello riguarda la priorità degli esperimenti. Non tutti gli esperimenti hanno lo stesso valore. Alcuni sono fondamentali per la base del sistema. Altri sono interessanti ma non urgenti. Per MicroBot, le priorità iniziali dovrebbero essere: `pattern base`, `dashboard`, `telemetry`, `safety`, `recovery`, `hybrid demo` e `hardware LED`. Solo dopo conviene spingere su modelli complessi, `AI`, `3D avanzato` o `ottimizzazione evolutiva`. Questa priorità mantiene il progetto solido.

Una priorità sperimentale sensata è:

P0: `nod`i, `snapshot`, `dashboard`, `pattern base`;

P1: `metriche`, `report`, `safety simulata`;

P2: `fault injection`, `recovery`, `replay`;

P3: `hybrid hardware/software`;

P4: `PWM` e `attuazione sicura`;

P5: `strutture complesse` e `scalabilità`;

P6: `AI`, `ottimizzazione`, `modelli avanzati`.

Questa non limita la visione; la ordina. Prima si costruisce la spina dorsale, poi si espande.

Il ventesimo livello riguarda il criterio di promozione di una funzione. Una funzione passa da sperimentale a stabile solo quando supera criteri definiti. Per esempio, un `pattern` può essere promosso quando supera `test baseline` e `noise test`. Una `safety policy` può essere promossa quando blocca tutti i comandi non validi nei test. Una `dashboard` può essere promossa quando visualizza correttamente `snapshot normali`, `errori` e `stale state`. Questo evita che funzioni immature vengano presentate come complete.

Una funzione può essere promossa se:

- ha test dedicati;
- ha documentazione;
- ha report o evidenza;
- non rompe regression suite;
- ha limiti dichiarati;
- è integrata con dashboard o log se necessario.

Questo è software engineering serio applicato al progetto.

Il ventunesimo livello riguarda la conservazione dei risultati negativi. Anche gli esperimenti falliti vanno conservati se insegnano qualcosa. Non tutti, ma quelli significativi sì. Un fallimento di `bridge_complex` può spiegare perché serve una catena più semplice. Un fallimento di cluster con alta coesione può spiegare perché serve separazione maggiore. Un fallimento di dashboard con troppi nodi può indicare limite di performance. Questi fallimenti diventano documentazione dei limiti e guida al miglioramento.

Un file `failed_experiments.md` può contenere:

- esperimento;
- configurazione;
- motivo del fallimento;
- lezione appresa;
- decisione presa.

Questo è molto utile per una tesi, perché mostra metodo scientifico.

Il ventiduesimo livello riguarda la creazione di un archivio sperimentale. Ogni run importante dovrebbe essere salvata con `run_id`. Ogni run può avere config, snapshot, report e screenshot. Nel tempo, questo archivio diventa una storia del progetto. Può mostrare evoluzione da pattern semplici a strutture complesse. Può diventare materiale per GitHub, documentazione, presentazione e pubblicazione.

Una struttura possibile è:

```
experiments/  
001_line_basic_baseline/  
  config.json  
  report.md  
  metrics.json  
  final_snapshot.json  
  screenshot.png  
002_circle_closure_fault/  
  config.json  
  report.md  
  events.json
```

003_cluster_isolated_node/
config.json
report.md

Questa struttura è molto chiara. Ogni esperimento diventa recuperabile.

Il ventitreesimo livello riguarda l'uso della simulazione per preparare pubblicazione o relazione. Una tesi o un report serio può includere esperimenti simulativi con metodi, risultati e limiti. La Sezione 12 può quindi diventare una base forte per spiegare non solo che MicroBot è stato immaginato, ma che è stato sperimentato. Anche se l'hardware è parziale, la simulazione validata ha valore: dimostra architettura, logica, safety e comportamento.

Una sezione sperimentale in relazione può includere:

obiettivo sperimentale;

modello simulato;

pattern testati;

metriche;

risultati;

limiti;

prossimi passi hardware.

Questo trasforma il progetto in lavoro tecnico serio.

Il ventiquattresimo livello riguarda il rapporto tra simulazione e startup. Se MicroBot deve essere presentato come idea imprenditoriale, la simulazione è un modo rapido per mostrare il potenziale. Però deve essere accompagnata da trasparenza: cosa è simulato, cosa è hardware, cosa è roadmap. Lo sviluppo sperimentale permette di creare demo credibili e progressivamente più fisiche. Per un primo pitch, una simulazione ben fatta con dashboard e nodi LED reali può essere molto più efficace di un hardware incompleto e instabile.

Una demo startup-oriented può mostrare:

problema/visione;

swarm simulato;

dashboard con metriche;

nodo fisico collegato;

safety/recovery;

roadmap hardware;

applicazioni future.

La simulazione diventa prototipo comunicativo, non solo tecnico.

Il venticinquesimo livello riguarda il realismo progressivo. All'inizio, la simulazione può essere molto astratta. Poi deve diventare sempre più realistica: limiti di velocità, rumore, energia, temperatura, comunicazione, fault, ritardi, capabilities. Ogni aggiunta di realismo rende il comportamento più vicino all'hardware. Tuttavia, bisogna evitare di rendere tutto complesso subito. Il realismo deve aumentare per livelli.

Una progressione di realismo può essere:

geometria ideale;

movimento con damping;

rumore di posizione;

topologia;

comunicazione con latenza;

safety virtuale;

energia e temperatura;

fault injection;

calibrazione hardware;

hybrid validation.

Questa progressione è sostenibile. Non blocca la creatività iniziale, ma spinge verso realtà.

Il ventiseiesimo livello riguarda la calibrazione con dati reali. Quando verranno raccolti dati hardware — latenza ESP32, tempi di heartbeat, limiti PWM, cooldown bobina, autonomia batteria, reset, jitter — questi dati dovranno aggiornare il simulatore. Questo è un passaggio fondamentale. Il simulatore non deve restare inventato; deve imparare dai prototipi. Se un ESP32 risponde mediamente in un certo tempo, il `network_profile` può usare quel valore. Se una bobina richiede cooldown più lungo, il `thermal_model` cambia. Se la batteria dura poco, l'`energy_model` si aggiorna.

Una calibrazione può aggiornare:

`latency_profile`;

`battery_model`;

`thermal_model`;

`cooldown_policy`;

`command_timeout`;

`heartbeat_interval`;

`hardware_capabilities`.

Questo crea un ciclo virtuoso tra software e hardware.

Il ventisettesimo livello riguarda il controllo della complessità. Lo sviluppo sperimentale deve evitare che la simulazione diventi troppo complicata prima di essere utile. Ogni nuovo modulo deve avere uno scopo. Non serve aggiungere AI se i pattern base non sono validati. Non serve 3D se la dashboard 2D non mostra bene safety. Non serve modello magnetico complesso se i comandi bobina non sono ancora sicuri. La complessità deve essere guadagnata, non accumulata.

Una regola utile è:

aggiungere complessità solo quando migliora validazione, chiarezza o transizione hardware.

Questa regola protegge il progetto dalla dispersione. MicroBot è già ampio; il metodo sperimentale deve tenerlo compatto.

Il ventottesimo livello riguarda la riproducibilità. Un esperimento deve poter essere ripetuto da te o da un collaboratore. Questo richiede file, istruzioni, versioni, configurazioni e output. Se un esperimento funziona solo perché “sul mio computer in quel momento andava”, non è riproducibile. La simulazione deve puntare a run rilanciabili. Questo è fondamentale per GitHub e per collaborazioni.

Un esperimento riproducibile richiede:

file config;

versione codice;

comando di avvio;

seed casuale se necessario;

output atteso;

criteri di successo.

Questa struttura rende il progetto più professionale.

Il ventinovesimo livello riguarda lo sviluppo sperimentale come metodo personale di lavoro. MicroBot è un progetto molto ambizioso, e proprio per questo deve procedere per esperimenti piccoli, chiusi, documentati. Ogni giorno o fase di lavoro può avere un obiettivo sperimentale preciso: “oggi valido line_basic”, “oggi faccio dashboard snapshot”, “oggi testo low_energy”, “oggi preparo hybrid demo”. Questo evita di sentirsi schiacciati dall’enormità del progetto. Il sistema cresce per blocchi.

Una giornata di sviluppo può chiudersi con:

cosa è stato testato;

cosa funziona;

cosa non funziona;

quale file è stato aggiornato;

quale esperimento viene dopo.

Questo crea continuità reale.

Il trentesimo livello riguarda la chiusura della Sezione 12. La simulazione software, nel progetto MicroBot, non è un'aggiunta estetica e non è un semplice supporto visuale. È il laboratorio in cui la teoria diventa comportamento, il comportamento diventa metrica, la metrica diventa validazione, la validazione diventa documentazione, e la documentazione guida il passaggio all'hardware. È il punto in cui pattern, safety, telemetria, dashboard, modelli virtuali e recovery vengono provati prima di diventare fisici. Senza simulazione, MicroBot rischierebbe di essere troppo teorico o troppo fragile. Con una simulazione ben progettata, invece, può evolvere in modo ordinato.

In conclusione, lo sviluppo sperimentale è il metodo con cui MicroBot cresce attraverso ipotesi, test, simulazioni, metriche, errori, correzioni, report e trasferimenti progressivi verso hardware reale. Esso permette di distinguere ciò che è concetto, ciò che è simulato, ciò che è validato, ciò che è ibrido e ciò che è fisicamente implementato. Permette di testare pattern, safety, recovery, comunicazione, energia, temperatura, dashboard e scalabilità senza rischiare componenti reali. Permette anche di costruire demo progressive, documentazione solida e una roadmap credibile. Con questo punto si chiude la Sezione 12: la simulazione software diventa il laboratorio centrale dell'ecosistema MicroBot. La sezione successiva, dedicata alla Telemetria, potrà quindi analizzare come lo stato dei nodi, gli eventi, le metriche, gli errori, l'energia, la temperatura e i risultati delle run vengano raccolti, trasmessi, visualizzati e usati per monitorare, validare e migliorare l'intero swarm.

SEZIONE 13 — Telemetria

La Sezione 13 introduce la telemetria del sistema MicroBot, cioè l'insieme dei dati raccolti, trasmessi, organizzati, visualizzati e analizzati per comprendere lo stato dello swarm durante simulazioni, demo ibride, test hardware e run sperimentali. Dopo la Sezione 12, dedicata alla simulazione software, la telemetria diventa il livello che permette di trasformare il comportamento osservato in informazione misurabile. Se la simulazione mostra ciò che accade, la telemetria registra, quantifica e rende analizzabile ciò che accade. Senza telemetria, MicroBot sarebbe visibile ma non pienamente verificabile; con la telemetria, invece, ogni nodo, comando, errore, stato energetico, condizione termica, fase, ruolo e metrica può essere tracciato.

Nel progetto MicroBot, la telemetria non deve essere vista come un semplice flusso di numeri. Essa rappresenta il sistema nervoso informativo dell'intero ecosistema. Ogni MicroBot, reale o virtuale, deve poter comunicare al controller e alla dashboard il proprio stato. Il controller deve poter sapere quali nodi sono online, quali sono offline, quali sono in safety, quali hanno energia bassa, quali hanno temperatura elevata, quali sono sincronizzati, quali hanno ricevuto comandi, quali li hanno rifiutati e quali stanno partecipando a un pattern. La telemetria permette al sistema di non essere cieco.

La telemetria è anche ciò che collega le sezioni precedenti. I pattern della Sezione 10 producono errori geometrici, topologici e comportamentali. Il Safety Layer della Sezione 11 produce stati come `thermal_warning`, `thermal_lock`, `low_energy`, `critical_energy`, `cooldown_active`, `watchdog_reset` e `recovery_required`. La simulazione della Sezione 12 produce snapshot, metriche, eventi e report. Tutti questi elementi devono essere raccolti e resi leggibili. La telemetria è quindi il livello che porta informazioni dal nodo al controller, dal simulatore alla dashboard, dal firmware ai log e dalla run al report finale.

La telemetria ha quattro funzioni principali. La prima è il monitoraggio dello stato: sapere in ogni momento in che condizione si trovano i nodi e lo swarm. La seconda è la raccolta dati in tempo reale: acquisire informazioni mentre la run avanza, senza aspettare il risultato finale. La terza è

l'analisi del sistema: trasformare i dati raccolti in metriche, diagnosi, confronti e decisioni. La quarta è il debug dello swarm: usare eventi, log e misure per capire perché un comportamento funziona, fallisce, si degrada o richiede recovery. Questi sono esattamente i quattro punti dell'indice della Sezione 13.

La telemetria deve essere progettata con un principio fondamentale:

ogni stato importante del sistema deve essere osservabile, registrabile e interpretabile.

Questo significa che un errore non deve restare nascosto. Se un comando viene rifiutato, deve esistere un evento. Se un nodo cambia ruolo, deve essere registrato. Se una bobina entra in cooldown, deve apparire nello stato. Se un nodo si resetta, il boot_id deve cambiare e il reset deve essere rilevato. Se un pattern fallisce, il report deve indicare il motivo. La telemetria rende il comportamento spiegabile.

Nel sistema MicroBot bisogna distinguere diversi tipi di telemetria. Il primo tipo è la telemetria di nodo, relativa al singolo MicroBot: mode, role, energy_state, thermal_state, sync_status, safety_state, capabilities, last_command, heartbeat, uptime, boot_id. Il secondo tipo è la telemetria di swarm, relativa al gruppo: node_count, active_nodes, topology_score, aggregation_score, pattern_error, connected_components, safety_summary. Il terzo tipo è la telemetria di pattern: pattern_id, phase_id, phase_time, success_criteria, missing_edges, target_error, correction_count, fallback_status. Il quarto tipo è la telemetria di sistema: runtime_status, simulation_step, dashboard_status, backend_status, log_status, snapshot_age, controller_state. Il quinto tipo è la telemetria sperimentale: run_id, test_id, configuration_id, metrics_summary, final_result, recommendations.

Questa distinzione è importante perché evita di mettere tutti i dati nello stesso contenitore. Un nodo non deve inviare ogni metrica globale. Il controller può calcolare metriche di swarm. Il simulatore può produrre snapshot. La dashboard può visualizzare sintesi. I log possono archiviare eventi. Ogni livello ha la sua responsabilità.

La telemetria deve essere coerente tra simulazione e hardware. Un nodo simulato può inviare energy_state simulato. Un nodo fisico può inviare energy_state stimato o misurato. Un nodo ibrido può avere posizione simulata e heartbeat reale. Tutti questi dati devono indicare la propria fonte. Questo evita confusione. Se thermal_state è simulated, lo si deve sapere. Se battery_voltage è measured, lo si deve sapere. Se position è virtuale, lo si deve sapere. La telemetria non deve soltanto dare valori; deve dire quanto quei valori sono reali, stimati o simulati.

Un campo fondamentale è quindi source. Per esempio:

temperature_source = simulated / estimated / measured / unavailable;

energy_source = simulated / estimated / measured / unavailable;

position_source = simulated / estimated / measured / unavailable;

heartbeat_source = simulated / hardware;

sync_source = controller / simulated / hardware.

Questa precisione rende il sistema più professionale. Permette di usare lo stesso formato dati senza fingere che tutto abbia lo stesso grado di realtà.

13.1 Monitoraggio dello stato

Il monitoraggio dello stato è il primo livello della telemetria MicroBot e consiste nella capacità del sistema di conoscere, aggiornare e rappresentare la condizione corrente dei nodi, dello swarm, del controller, della simulazione e della run. Monitorare lo stato significa rispondere continuamente a domande fondamentali: quali MicroBot sono online? Quali sono attivi? Quali sono in idle? Quali stanno partecipando al pattern? Quali sono in safety? Quali hanno energia bassa? Quali sono caldi? Quali hanno perso sincronizzazione? Quali sono in recovery? Quale fase del pattern è in corso? Quale comportamento è stabile e quale no?

Nel progetto MicroBot, il monitoraggio dello stato deve essere continuo ma non caotico. Non ogni dato deve essere inviato sempre alla massima frequenza. Alcuni dati sono essenziali e devono essere aggiornati spesso, come heartbeat, mode, safety_state e online status. Altri dati possono essere aggiornati con frequenza minore, come capabilities complete, firmware_version, hardware_version o configurazione. Altri ancora possono essere inviati solo quando cambiano, come role_changed, thermal_lock_triggered, low_energy_entered o watchdog_reset_detected. Questa distinzione permette di ridurre traffico e mantenere la telemetria leggibile.

Lo stato minimo di un nodo MicroBot deve includere almeno node_id, boot_id, mode, role, online_status, last_seen, safety_state, energy_state, thermal_state e sync_status. Questi campi permettono al controller di capire se il nodo è utilizzabile. Un nodo con online_status = online ma safety_state = safety_lock non è utilizzabile per pattern attivi. Un nodo con sync_status = sync_lost non deve eseguire comandi temporizzati. Un nodo con energy_state = critical_energy deve essere escluso da ruoli attivi. Un nodo con thermal_state = thermal_lock deve bloccare bobine o PWM critici. Quindi lo stato del nodo non è solo informazione descrittiva: è input decisionale.

Una struttura minima di stato può essere:

```
node_id;  
  
node_type;  
  
boot_id;  
  
online_status;  
  
mode;  
  
role;  
  
group_id;  
  
pattern_run_id;  
  
phase_id;  
  
last_seen_ms;  
  
safety_state;  
  
energy_state;  
  
thermal_state;
```

sync_status;

capabilities_summary;

last_error.

Questa struttura deve essere abbastanza piccola da poter essere aggiornata spesso, ma abbastanza ricca da supportare decisioni. I dettagli più pesanti possono essere richiesti con request_status o mostrati solo nel pannello dettagliato della dashboard.

Il campo node_type è molto importante. Può indicare simulated_node, hardware_node, hybrid_node o ghost_node. Questo permette al sistema di interpretare correttamente i dati. Un simulated_node può avere posizione precisa e battery_level simulato. Un hardware_node può avere heartbeat reale ma posizione non disponibile. Un hybrid_node può unire dati fisici e virtuali. Un ghost_node rappresenta un nodo precedentemente visto ma non aggiornato. Senza node_type, la dashboard rischia di trattare tutti i nodi allo stesso modo.

Il campo boot_id serve a distinguere sessioni diverse dello stesso nodo. Se il node_id resta MB-004 ma il boot_id cambia, il controller capisce che il nodo si è riavviato. Questo è fondamentale per il monitoraggio dello stato perché un nodo appena riavviato non deve essere considerato automaticamente allineato con la run corrente. Deve passare per recovery, sync e state_sync. Il monitoraggio dello stato deve quindi rilevare non solo se un nodo è online, ma anche se è lo stesso contesto operativo di prima.

Una regola centrale è:

stesso node_id con nuovo boot_id = nodo fisico noto, ma nuova sessione operativa.

Questo evento deve apparire nella telemetria e nei log. La dashboard può mostrare “reboot detected” o “resync required”. Il controller deve invalidare comandi pendenti e ruoli non confermati.

Il campo online_status deve distinguere almeno online, stale, offline e recovering. Online significa che il nodo ha inviato heartbeat o telemetria recentemente. Stale significa che il nodo non è stato aggiornato da un po', ma non abbastanza da essere dichiarato offline. Offline significa che il timeout è superato. Recovering significa che il nodo è tornato o sta rientrando ma non è ancora reintegrabile. Questa distinzione è molto più utile di un semplice booleano online/offline.

Una politica può essere:

online: last_seen entro soglia breve;

stale: last_seen oltre soglia breve ma sotto soglia offline;

offline: last_seen oltre soglia offline;

recovering: nodo tornato dopo offline/reset ma non ancora riallineato.

Questa logica deve essere coerente tra controller e dashboard. Un nodo stale non deve essere usato per decisioni critiche. Un nodo offline deve essere escluso. Un nodo recovering deve essere monitorato ma non ancora pienamente operativo.

Il campo mode descrive la modalità interna del nodo. Può assumere valori come booting, idle, active, diagnostic, safety, safe_idle, recovery, low_power o offline. Il mode è diverso dal ruolo. Il

ruolo dice che funzione ha il nodo nel pattern; il mode dice in quale stato operativo si trova. Un nodo può avere role = connector ma mode = safety: in quel caso non sta davvero agendo da connector. La dashboard deve mostrare entrambi.

Una distinzione fondamentale è:

role = funzione assegnata nel pattern;

mode = condizione operativa corrente del nodo.

Questa distinzione evita ambiguità. Un nodo può mantenere il ruolo assegnato ma essere temporaneamente in recovery. Oppure può essere idle senza ruolo. Oppure può essere diagnostic_node come ruolo e diagnostic come mode. I due campi si collegano ma non coincidono.

Il campo role deve essere monitorato perché l'assegnazione dei ruoli è parte del comportamento dello swarm. Il controller può aspettarsi che un nodo sia leader, connector, boundary_node, core_node, bridge_node, terminal_node, passive_node o actuator_node. Il nodo deve riportare il ruolo che ha ricevuto o che sta applicando. Se expected_role e reported_role non coincidono, c'è role_mismatch. La telemetria deve permettere di rilevarlo.

Una struttura utile può includere:

expected_role;

reported_role;

role_status;

role_assigned_at;

role_confirmed_at.

role_status può essere assigned, confirmed, mismatch, unsupported, expired o suspended. Questo permette di distinguere un ruolo appena inviato ma non ancora confermato da un ruolo rifiutato o perso. Per pattern complessi, questa precisione è molto utile.

Il campo safety_state è centrale. Può indicare safety_ok, safety_warning, safety_lock, emergency_stopped, recovery_required o manual_intervention_required. Questo stato deve avere priorità alta nella dashboard. Se safety_state non è ok, il nodo non deve essere considerato pienamente operativo. La telemetria safety deve essere immediata quando cambia stato. Per esempio, se un nodo entra in emergency_stopped, non deve aspettare il prossimo heartbeat normale; deve inviare un evento prioritario se possibile.

Il campo energy_state indica lo stato energetico: energy_ok, energy_medium, low_energy, critical_energy, energy_unknown. Come già definito nella Sezione 11, questo campo influenza ruoli e comandi. Il monitoraggio dello stato deve mostrare non solo il livello percentuale, ma lo stato interpretato. Un battery_level del 20% può significare cose diverse secondo batteria e hardware, ma low_energy ha significato operativo: limitare comandi ad alto consumo. Per questo energy_state è più importante del numero grezzo.

Il campo thermal_state indica thermal_ok, thermal_warning, thermal_lock, thermal_recovery o thermal_unknown. Anche qui lo stato interpretato è fondamentale. Un valore termico stimato può non essere preciso, ma thermal_warning significa che il sistema deve ridurre carico. thermal_lock

significa che gli attuatori termicamente critici sono bloccati. La dashboard deve mostrarlo chiaramente e il controller deve usarlo nella pianificazione.

Il campo `sync_status` indica quanto il nodo è allineato temporalmente. Può essere `sync_good`, `sync_ok`, `sync_degraded`, `sync_lost`, `unsynced` o `out_of_phase`. Questo campo è fondamentale per comandi `execute_at` e pattern sincronizzati. Un nodo con `sync_lost` può essere online e sicuro, ma non adatto a fasi temporizzate. Il monitoraggio dello stato deve quindi evitare interpretazioni semplicistiche: online non significa pronto per tutto.

Una classificazione di readiness può essere utile:

`node_ready_for_visual_commands;`

`node_ready_for_timed_commands;`

`node_ready_for_pattern;`

`node_ready_for_actuation;`

`node_ready_for_critical_role.`

Queste readiness possono essere calcolate dal controller in base a `mode`, `safety_state`, `energy_state`, `thermal_state`, `sync_status` e `capabilities`. Per esempio, un nodo può essere `ready_for_visual_commands` ma non `ready_for_actuation`. Questa informazione è estremamente utile per dashboard e pattern.

Il monitoraggio dello stato deve includere `capabilities` e `blocked_capabilities`. Le `capabilities` indicano cosa il nodo può fare; `blocked_capabilities` indicano cosa è temporaneamente non disponibile. Per esempio, un nodo può avere `coil_pwm` ma averla bloccata per `thermal_lock`. Un nodo può avere `led_status` ma ridurre `brightness` per `low_energy`. Un nodo può avere `telemetry_debug` ma disabilitarla in `critical_energy`. Questa distinzione evita errori di pianificazione.

Una struttura utile è:

`hardware_capabilities;`

`software_capabilities;`

`available_capabilities;`

`blocked_capabilities;`

`block_reason.`

Per esempio:

`hardware_capabilities = led_status, coil_pwm;`

`available_capabilities = led_status, heartbeat;`

`blocked_capabilities = coil_pwm;`

`block_reason = thermal_lock.`

Questa informazione permette al controller di non assegnare ruoli incompatibili.

Il monitoraggio dello stato deve funzionare sia tramite heartbeat periodico sia tramite eventi. L'heartbeat è un messaggio regolare che dice “sono vivo e questo è il mio stato essenziale”. Gli eventi sono messaggi inviati quando succede qualcosa. I due meccanismi sono complementari. L'heartbeat mantiene visibilità continua. Gli eventi segnalano cambiamenti importanti immediatamente. Se si usa solo heartbeat, alcuni eventi possono arrivare in ritardo. Se si usano solo eventi, si può non sapere se il nodo è ancora vivo. Servono entrambi.

Un heartbeat può contenere:

node_id;
boot_id;
uptime_ms;
mode;
role;
safety_state;
energy_state;
thermal_state;
sync_status;
last_error;
timestamp.

Un evento può contenere:

event_id;
event_type;
node_id;
severity;
timestamp;
previous_state;
new_state;
reason;
related_command_id.

Esempi di eventi sono role_changed, thermal_warning_triggered, energy_low_entered, watchdog_reset_detected, command_rejected, recovery_started, sync_lost, node_offline_detected. Questi eventi alimentano dashboard e log.

Il monitoraggio dello stato deve includere lo stato dello swarm globale. Oltre ai singoli nodi, il controller deve calcolare una sintesi. Per esempio: quanti nodi sono online, quanti attivi, quanti offline, quanti in warning, quanti in safety, quanti pronti per pattern. Questa sintesi serve alla dashboard e alle decisioni. Prima di avviare una run, il controller deve sapere se lo swarm è ready.

Uno swarm_status può contenere:

total_nodes;

online_nodes;

active_nodes;

stale_nodes;

offline_nodes;

nodes_in_safety;

nodes_in_recovery;

nodes_low_energy;

nodes_thermal_warning;

nodes_thermal_lock;

ready_for_pattern_count;

global_safety_state;

global_readiness.

global_safety_state può essere ok, warning, degraded, safety_lock o emergency_stop.
global_readiness può indicare se il sistema è pronto per una demo, una simulazione, un pattern o un test hardware. Questa sintesi rende il sistema gestibile.

Il monitoraggio dello stato deve includere lo stato della run. Una run ha run_id, pattern_id, phase_id, start_time, current_phase, elapsed_time, result_pending, metrics e stato di validazione. Senza run_status, la dashboard vede nodi ma non capisce il contesto. Un nodo active in una run line_basic ha significato diverso da un nodo active in bridge_complex. Il monitoraggio deve quindi collegare nodo e run.

Un run_status può contenere:

run_id;

pattern_id;

pattern_version;

mode: simulation, hybrid, hardware_visual, hardware_actuation;

current_phase;

phase_elapsed_ms;
participating_nodes;
validation_status;
fallback_status;
correction_count;
safety_events_count;
result_if_finished.

Questo stato permette alla dashboard di mostrare “line_basic_v0.2 — phase_align — validating — 8 nodes”. Senza queste informazioni, la visualizzazione è meno utile.

Il monitoraggio dello stato deve includere lo stato della simulazione. In modalità software, bisogna sapere simulation_id, step, simulation_time, simulation_mode, node_count, snapshot_age, running/paused, speed_factor e fault_profile. Questo è importante perché una simulazione può essere live, paused, replay o batch. La dashboard deve sapere se sta guardando una run attiva o un replay.

Uno simulation_status può contenere:

simulation_id;
step;
simulation_time_ms;
real_time_ms;
simulation_mode;
running;
paused;
speed_factor;
fault_profile_active;
last_snapshot_time;
snapshot_age_ms.

snapshot_age_ms è cruciale. Se è troppo alto, la dashboard deve indicare snapshot_stale. Questo evita di guardare dati vecchi pensando siano live.

Il monitoraggio dello stato deve includere lo stato del backend e della dashboard. Questo può sembrare secondario, ma è utile per debug. Se il frontend non riceve dati, bisogna sapere se il problema è simulatore, backend, WebSocket, snapshot o browser. Un system_status può mostrare backend_online, websocket_connected, api_error_count, last_update e source. Questo rende il sistema più facile da sviluppare.

Un `system_status` può contenere:

```
backend_status;  
  
websocket_status;  
  
snapshot_source;  
  
last_api_response;  
  
api_error_count;  
  
frontend_render_fps;  
  
data_mode;  
  
controller_status.
```

Questi dati non sono parte dello swarm, ma sono parte dell'osservabilità del sistema.

Il monitoraggio dello stato deve gestire la frequenza di aggiornamento. Non tutto deve essere aggiornato allo stesso ritmo. Per esempio, heartbeat può arrivare ogni secondo. La posizione simulata può aggiornarsi molte volte al secondo. Gli eventi arrivano quando accadono. Le capabilities complete possono essere richieste solo all'avvio o dopo recovery. La telemetria debug può essere attivata solo in `diagnostic_mode`. Questa differenziazione permette efficienza.

Una politica può essere:

```
heartbeat: frequenza regolare moderata;  
  
snapshot simulazione: frequenza più alta se dashboard live;  
  
eventi: immediati al cambio stato;  
  
status completo: on request o all'avvio;  
  
debug telemetry: solo diagnostic o test;  
  
report: fine run.
```

Questa struttura evita traffico inutile. È importante soprattutto per hardware wireless e batteria.

Il monitoraggio dello stato deve essere robusto ai dati mancanti. Un nodo può non inviare `battery_level`. Un nodo simulato può non avere `firmware_version`. Un `hardware_node` può non avere `position`. La dashboard non deve rompersi se un campo manca. Deve mostrare `unavailable` o `unknown`. Il controller deve usare policy conservative quando un dato critico è mancante. Per esempio, se `thermal_state` è `unknown` su un nodo con bobina, meglio non autorizzare attuazioni critiche.

Una regola è:

dato critico mancante → stato `unknown` → decisione conservativa.

Questo principio è molto importante. L'assenza di informazione non deve essere trattata come condizione sicura.

Il monitoraggio dello stato deve distinguere dati grezzi e dati interpretati. `battery_voltage` è dato grezzo. `energy_state` è dato interpretato. `temperature_c` è dato grezzo. `thermal_state` è interpretato. `last_seen_ms` è grezzo. `online_status` è interpretato. Entrambi sono utili. Il dato grezzo permette analisi; lo stato interpretato permette decisioni rapide. La dashboard può mostrare entrambi, ma con gerarchia: prima lo stato, poi il valore.

Esempio:

```
battery_voltage_v = 3.62;
```

```
energy_state = low_energy;
```

```
energy_source = measured.
```

Oppure:

```
thermal_load = 0.78;
```

```
thermal_state = thermal_warning;
```

```
temperature_source = estimated.
```

Questa struttura è chiara e tecnica.

Il monitoraggio dello stato deve consentire il confronto tra `expected_state` e `reported_state`. Il controller può avere una previsione: questo nodo dovrebbe essere `active`, `role connector`, `phase align`. Il nodo può riportare: `idle`, `role connector`, `phase prepare`. La differenza produce `mismatch`. La telemetria deve permettere questo confronto. È fondamentale per sistemi distribuiti, dove nodi e controller possono disallinearsi.

Una struttura utile è:

```
expected_mode;
```

```
reported_mode;
```

```
expected_role;
```

```
reported_role;
```

```
expected_phase;
```

```
reported_phase;
```

```
state_consistency.
```

`state_consistency` può essere `ok`, `pending`, `mismatch`, `stale` o `unknown`. Questo permette al controller di richiedere `state_sync` quando serve.

Il monitoraggio dello stato deve essere visualizzato in dashboard in modo gerarchico. A livello alto, si mostra stato globale: `ok`, `warning`, `degraded`, `safety`. A livello medio, si mostrano gruppi e pattern. A livello basso, si mostrano nodi. L'utente deve poter partire dalla sintesi e scendere nel dettaglio. Se ci sono 100 nodi, non si può leggere tutto. Serve una dashboard che evidenzia anomalie.

Una gerarchia visuale efficace è:

global status;

run status;

safety summary;

node list con filtri;

swarm canvas;

selected node inspector;

event console.

Questo permette di monitorare senza essere sommersi.

Il monitoraggio dello stato deve includere filtri. L'utente deve poter vedere solo nodi in warning, solo offline, solo low_energy, solo connector, solo nodi di un gruppo, solo nodi hardware. Questo è importante quando lo swarm cresce. Senza filtri, la dashboard diventa confusa.

Filtri utili:

by role;

by mode;

by safety_state;

by energy_state;

by thermal_state;

by node_type;

by group_id;

by online_status;

by error status.

Questi filtri trasformano la telemetria in strumento di debug.

Il monitoraggio dello stato deve essere integrato con il logging. Ogni cambiamento importante di stato deve generare un evento loggato. Non serve loggare ogni heartbeat se sarebbe troppo pesante, ma bisogna loggare le transizioni: online → stale, stale → offline, thermal_ok → thermal_warning, energy_ok → low_energy, role assigned, mode changed, recovery started, recovery completed. I log delle transizioni sono più utili dei valori ripetuti continuamente.

Eventi di stato importanti:

node_online;

node_stale;

node_offline;

node_recovered;
mode_changed;
role_changed;
safety_state_changed;
energy_state_changed;
thermal_state_changed;
sync_status_changed;
capability_blocked;
capability_restored.

Questi eventi permettono di ricostruire la storia della run.

Il monitoraggio dello stato deve essere collegato alla raccolta dati in tempo reale, che sarà il punto successivo. Monitorare significa sapere lo stato corrente. Raccogliere dati in tempo reale significa acquisire continuamente questi stati, trasformarli in stream, snapshot, serie temporali e log. Il monitoraggio è quindi la faccia “presente” della telemetria; la raccolta dati è la faccia “temporale”. Uno stato singolo dice cosa sta succedendo ora; una serie di stati dice come il sistema sta evolvendo.

Per esempio:

monitoraggio: MB-004 è in thermal_warning ora;

raccolta dati: MB-004 è passato da thermal_ok a thermal_warning dopo tre impulsi e ha impiegato 18 secondi per recuperare.

La seconda informazione è molto più utile per analisi. Ma la prima è indispensabile per decisione immediata. Entrambe servono.

Il monitoraggio dello stato deve essere progettato per simulazione, hardware e modalità ibrida. In simulazione, lo stato è generato dal motore. In hardware, arriva dai nodi fisici. In modalità ibrida, alcuni campi possono venire da fonti diverse. Il sistema deve unificarli senza perdere la provenienza. Questa è una delle parti più importanti dell'architettura MicroBot: unificare dati simulati e reali mantenendo trasparenza.

Un esempio ibrido:

node_id = MB-001;
node_type = hybrid_node;
heartbeat_source = hardware;
position_source = simulated;
energy_source = estimated;

role_source = controller;

thermal_source = simulated;

safety_state = hardware_reported + controller_interpreted.

Questa ricchezza permette demo intermedie molto forti. Ma richiede disciplina nei dati.

Il monitoraggio dello stato deve supportare la degradazione. Se un nodo entra in warning, il sistema non deve necessariamente fallire. Deve segnalare, ridurre o adattare. Per farlo, il controller deve monitorare lo stato in modo abbastanza rapido. Se low_energy viene rilevato troppo tardi, il nodo può diventare critical_energy. Se thermal_warning viene ignorato, può diventare thermal_lock. Se sync_degraded viene ignorato, comandi temporizzati possono scadere. Il monitoraggio è quindi una funzione preventiva.

Una regola è:

gli stati warning devono essere rilevati abbastanza presto da permettere azione correttiva prima dello stato critico.

Questo collega telemetria e Safety Layer. La telemetria non serve solo a guardare, ma a prevenire.

Il monitoraggio dello stato deve essere affidabile ma anche leggero. Se si inviano troppi dati, si consuma energia e si appesantisce rete e dashboard. Se si inviano troppo pochi dati, si perde osservabilità. Serve equilibrio. Nei nodi fisici, il profilo di telemetria deve adattarsi allo stato energetico. Nei nodi simulati, si può inviare più dati. In modalità debug, si può aumentare dettaglio. In modalità demo, si può ridurre.

Profili possibili:

telemetry_minimal;

telemetry_basic;

telemetry_normal;

telemetry_debug;

telemetry_full.

telemetry_minimal include heartbeat e safety essenziale. telemetry_basic include mode, role, energy, thermal. telemetry_normal include metriche e phase. telemetry_debug include dettagli come target, errori, capabilities. telemetry_full include tutto, ma va usato con cautela.

Il monitoraggio dello stato deve essere compatibile con future estensioni. Nelle sezioni successive si parlerà di MicroBot OS, sicurezza biometrica, progetti esterni, integrazione e scalabilità. La telemetria deve poter crescere. Potranno aggiungersi autenticazione, camera status, biometric_status, OS task status, shell diagnostics, AI module status, clustering metrics, VR state, ecc. Il modello dati deve essere estensibile senza rompere ciò che esiste.

Una struttura estensibile può usare:

core_status;

safety_status;

energy_status;

thermal_status;

sync_status;

pattern_status;

telemetry_extensions;

diagnostic_extensions.

Le estensioni permettono di aggiungere nuovi campi mantenendo il nucleo stabile. Questo è importante per un progetto grande.

Il monitoraggio dello stato deve essere documentato. Ogni campo deve avere significato. Cosa significa stale? Dopo quanti millisecondi? Cosa significa thermal_warning? Quale soglia? Cosa significa sync_degraded? Cosa significa recovery_required? Senza definizioni, i dati diventano ambigui. La documentazione della telemetria deve includere glossario, stati, transizioni e esempi.

Una documentazione telemetry_schema.md dovrebbe includere:

campi principali;

tipi di dato;

unità di misura;

valori ammessi;

significato operativo;

fonte del dato;

frequenza aggiornata;

esempi di payload.

Questo rende la telemetria implementabile e controllabile.

Il monitoraggio dello stato deve avere una roadmap. Nella prima versione si può monitorare node_id, online/offline, mode e role. Poi si aggiunge safety. Poi energy e thermal simulati. Poi sync. Poi capabilities. Poi dashboard. Poi eventi. Poi hardware heartbeat. Poi stato ibrido. Questa progressione è realistica e coerente.

Una roadmap possibile è:

v0.1: node_id, online_status, mode;

v0.2: role e pattern_run_id;

v0.3: safety_state;

v0.4: energy_state e thermal_state simulati;

v0.5: sync_status e phase_id;

v0.6: capabilities e blocked_capabilities;

v0.7: event stream;

v0.8: dashboard node inspector;

v0.9: hardware heartbeat ESP32;

v1.0: monitoraggio ibrido simulazione-hardware.

Questa roadmap permette di costruire telemetria senza voler fare tutto subito.

In conclusione, il monitoraggio dello stato è la base della telemetria MicroBot. Esso permette di sapere in ogni momento quali nodi sono online, quali ruoli hanno, in quale modalità si trovano, se sono sincronizzati, se sono sicuri, se hanno energia sufficiente, se sono caldi, se sono in recovery, se hanno capabilities disponibili e se partecipano correttamente alla run. Il monitoraggio deve distinguere nodo, swarm, run, simulazione e sistema; deve combinare heartbeat periodici ed eventi; deve trattare dati mancanti in modo conservativo; deve distinguere valori grezzi e stati interpretati; deve visualizzare sintesi e dettagli nella dashboard; deve registrare transizioni nei log. Senza monitoraggio dello stato, il sistema non può adattarsi, correggersi, proteggersi o validarsi. Il punto successivo, dedicato alla raccolta dati in tempo reale, mostrerà come questi stati vengano acquisiti durante l'esecuzione, trasformati in stream, snapshot, serie temporali e log utilizzabili per analisi, debug e report sperimentali.

13.2 Raccolta dati in tempo reale

La raccolta dati in tempo reale rappresenta il secondo livello della telemetria MicroBot e consiste nel processo attraverso cui lo stato dei nodi, dello swarm, della simulazione, dei pattern e del Safety Layer viene acquisito mentre il sistema è in esecuzione. Se il monitoraggio dello stato risponde alla domanda “che cosa sta succedendo adesso?”, la raccolta dati in tempo reale risponde a una domanda più ampia: “come sta evolvendo il sistema nel tempo?”. Questa distinzione è fondamentale, perché MicroBot non è un sistema statico. È uno swarm dinamico, distribuito, composto da nodi che cambiano ruolo, posizione, fase, energia, temperatura, connessione e stato operativo. Per comprenderlo davvero, non basta osservare un singolo istante: bisogna registrare la sequenza degli stati.

Nel progetto MicroBot, la raccolta dati in tempo reale deve essere considerata come la memoria operativa della run. Durante una simulazione o una demo hardware, ogni nodo produce informazioni: heartbeat, stato, ruolo, energia, temperatura, sync, ack, errori, eventi di safety, metriche di pattern e dati di recovery. Il controller riceve, interpreta e aggrega queste informazioni. La dashboard le visualizza. I log le conservano. I report le sintetizzano. Senza raccolta dati, la run sparisce appena termina. Con la raccolta dati, invece, diventa analizzabile, confrontabile, riproducibile e documentabile.

Il primo principio della raccolta dati in tempo reale è che non tutti i dati hanno lo stesso valore, la stessa frequenza o la stessa urgenza. Alcuni dati devono essere raccolti spesso, perché cambiano rapidamente e influenzano il comportamento immediato. Per esempio, posizione simulata, phase_id, pattern_error, heartbeat e safety_state. Altri dati cambiano raramente e possono essere raccolti meno spesso, come firmware_version, hardware_version, capabilities complete o configurazioni. Altri dati

devono essere raccolti immediatamente solo quando si verifica un evento, come `thermal_lock`, `low_energy`, `watchdog_reset`, `command_rejected` o `fallback_triggered`. La raccolta dati deve quindi essere organizzata per livelli di priorità.

Una possibile classificazione è questa:

dati continui: posizione, velocità, metriche, errore del pattern, tempo di simulazione;

dati periodici: `heartbeat`, `mode`, `role`, `energy_state`, `thermal_state`, `sync_status`;

dati su evento: errori, rifiuti comando, `recovery`, `fallback`, `reset`, `safety transitions`;

dati su richiesta: `capabilities complete`, configurazione, diagnostica estesa, `firmware info`;

dati finali: `report`, classificazione della run, metriche aggregate, risultato finale.

Questa separazione serve a evitare due problemi opposti: raccogliere troppo poco e perdere informazione importante, oppure raccogliere troppo e appesantire rete, memoria, dashboard e analisi. Un buon sistema telemetrico raccoglie abbastanza da spiegare il comportamento, ma non così tanto da diventare inutilizzabile.

Il secondo principio è la temporalità. Ogni dato raccolto deve avere un riferimento temporale. Un valore senza timestamp è quasi inutile in un sistema dinamico. Se un nodo segnala `thermal_warning`, bisogna sapere quando è accaduto. Se un comando viene rifiutato, bisogna sapere rispetto a quale fase e quale istante. Se un `pattern_error` diminuisce, bisogna poter ricostruire la curva nel tempo. Ogni messaggio di telemetria deve quindi includere `timestamp`, `simulation_step` o entrambi.

Nel sistema MicroBot possono esistere diversi riferimenti temporali:

`timestamp` reale del sistema;

`simulation_time`;

`simulation_step`;

`controller_time`;

`node_local_time`;

`phase_elapsed_time`;

`run_elapsed_time`.

In una simulazione semplice, può bastare `simulation_step` e `timestamp`. In una demo hardware, diventa importante distinguere tra tempo del controller e tempo locale del nodo. Un nodo può inviare `uptime_ms` e il controller può aggiungere `received_at`. Questo permette di stimare ritardi e freschezza del dato. La telemetria deve quindi registrare non solo il valore, ma anche il contesto temporale in cui quel valore è stato prodotto e ricevuto.

Un messaggio di telemetria ben formato può contenere:

`node_id`;

boot_id;

timestamp_node_ms;

timestamp_controller_ms;

simulation_step, se simulato;

run_id;

phase_id;

payload;

source;

sequence_number.

Il sequence_number è utile per capire se messaggi sono stati persi o arrivati fuori ordine. In un sistema distribuito, i dati possono non arrivare perfettamente ordinati. La raccolta in tempo reale deve essere pronta a gestire ritardi, duplicati e messaggi mancanti.

Il terzo principio è la distinzione tra dati grezzi, dati normalizzati e dati aggregati. I dati grezzi sono quelli prodotti direttamente dal nodo o dal simulatore: battery_voltage, thermal_load, position, last_command_id, heartbeat. I dati normalizzati sono trasformati in un formato comune: energy_state, thermal_state, online_status, readiness. I dati aggregati sono calcolati dal controller o dal sistema: node_count, topology_score, aggregation_score, safety_summary, energy_summary, success_rate. Tutti e tre i livelli sono utili, ma non devono essere confusi.

Per esempio, un nodo può inviare:

battery_voltage_v = 3.62

Il controller può interpretare:

energy_state = low_energy

Lo swarm summary può calcolare:

nodes_low_energy = 3

La dashboard può mostrare:

global_energy_status = degraded

Questi quattro livelli derivano uno dall'altro, ma non sono la stessa cosa. La raccolta dati deve conservare abbastanza informazione per permettere analisi sia locale sia globale. Se si salva solo global_energy_status, si perde dettaglio. Se si salva solo battery_voltage senza interpretazione, il sistema deve ricalcolare tutto ogni volta. L'equilibrio ideale è salvare dati essenziali grezzi e stati interpretati principali.

Il quarto principio è la fonte del dato. Ogni dato deve indicare se proviene da simulazione, stima, misura reale, controller o dashboard. Questo è particolarmente importante nel progetto MicroBot, perché molte run possono essere ibride. La posizione può essere simulata, il heartbeat reale, la

batteria stimata, la temperatura simulata, il ruolo assegnato dal controller. Senza source, la telemetria diventa ambigua.

Una struttura corretta può essere:

position = valore;

position_source = simulated;

energy_state = low_energy;

energy_source = estimated;

heartbeat_source = hardware;

thermal_state = thermal_warning;

thermal_source = simulated;

role_source = controller.

Questa precisione permette di dire con onestà che una demo è hybrid: alcuni dati sono reali, altri virtuali. È un punto di rigore tecnico fondamentale, soprattutto se il progetto viene mostrato esternamente.

Il quinto principio è la raccolta per snapshot. Uno snapshot è una fotografia completa o parziale dello stato del sistema a un certo istante. La raccolta in tempo reale può produrre una sequenza di snapshot, usata dalla dashboard per visualizzazione live e dal replay per analisi successiva. Uno snapshot può contenere tutti i nodi, gli edge, le metriche, lo stato della run e gli eventi recenti. È uno dei formati più importanti per MicroBot.

Uno snapshot completo può contenere:

snapshot_id;

run_id;

simulation_id;

timestamp;

step;

runtime_mode;

pattern_id;

phase_id;

nodes;

edges;

metrics;

safety_summary;

energy_summary;

thermal_summary;

recent_events;

validation_status.

Questo formato permette alla dashboard di aggiornarsi leggendo un unico oggetto coerente. In alternativa, in sistemi più avanzati, si possono inviare solo delta, cioè cambiamenti rispetto allo snapshot precedente. Ma nella fase iniziale, lo snapshot completo è più semplice e robusto. È più facile da salvare, leggere, debuggare e usare per replay.

Il sesto principio è la raccolta per eventi. Non tutti i dati devono essere registrati come snapshot completi. Alcune informazioni sono eventi discreti: un nodo cambia ruolo, un comando viene rifiutato, un edge si forma, una fase inizia, una recovery parte, un thermal_lock viene attivato. Questi eventi devono essere salvati come righe indipendenti in un event_log. Gli eventi raccontano la storia della run. Gli snapshot raccontano lo stato. Servono entrambi.

Un evento può contenere:

event_id;

event_type;

severity;

timestamp;

run_id;

phase_id;

node_id, se applicabile;

edge_id, se applicabile;

command_id, se applicabile;

message;

data;

action_taken.

Esempi:

role_assigned;

phase_started;

phase_completed;

node_offline;
node_recovered;
thermal_warning_triggered;
thermal_lock_triggered;
energy_low_entered;
command_rejected;
fallback_triggered;
pattern_completed.

Gli eventi devono avere severity: info, warning, error, critical. Questo permette alla dashboard di filtrare. In modalità demo si possono mostrare solo eventi importanti. In modalità debug si mostra tutto.

Il settimo principio è la raccolta per serie temporali. Alcune metriche sono utili non solo come valore finale, ma come curva nel tempo. Per esempio, pattern_error, topology_score, aggregation_score, node_count online, thermal_load medio, energy_level medio, correction_count, velocity_average, stability_score. Queste metriche possono essere salvate in un metrics_time_series. Questo permette di fare grafici, confronti e analisi post-run.

Una serie temporale può contenere record come:

step;
timestamp;
pattern_error;
topology_score;
aggregation_score;
stability_score;
active_nodes;
thermal_risk_score;
energy_risk_score;
event_count.

Queste serie sono fondamentali per capire la dinamica. Se pattern_error scende rapidamente e poi risale, la run ha un problema di stabilità. Se topology_score resta basso mentre geometry_score è alto, la forma è visivamente corretta ma topologicamente incompleta. Se thermal_risk_score cresce durante retry, bisogna ridurre attuazioni. La serie temporale mostra relazioni che un singolo snapshot non rivela.

L'ottavo principio è la raccolta dei comandi e degli ack. La telemetria non deve registrare solo stati, ma anche le interazioni: quali comandi sono stati inviati, a quali nodi, con quali parametri, quando sono stati ricevuti, se sono stati accettati, rifiutati o scaduti. Questo è essenziale per debug. Se un nodo non cambia stato, bisogna sapere se non ha ricevuto il comando, se lo ha rifiutato, se l'ack è perso o se il comando era incompatibile.

Un `command_log` può contenere:

`command_id;`
`timestamp_sent;`
`target;`
`command_type;`
`payload_summary;`
`pattern_run_id;`
`phase_id;`
`expected_response;`
`ack_received;`
`ack_timestamp;`
`result;`
`rejection_reason;`
`retry_count;`
`expires_at;`
`execution_time.`

Questo log è particolarmente importante per hardware e safety. Se un `magnetic_pulse` viene rifiutato per `cooldown_active`, il `command_log` lo registra. Se un `set_role` non viene confermato, il controller può generare `role_mismatch`. Se un comando scade, può essere escluso dalla run. La raccolta in tempo reale dei comandi rende il comportamento verificabile.

Il nono principio è la raccolta dei dati di rete. Per un sistema distribuito, la rete è parte del comportamento. Anche in simulazione, può essere utile registrare latenza, `packet_loss`, `ack_timeout`, `retry`, `jitter` e `stale status`. In hardware reale, questi dati diventano ancora più importanti. Se un nodo sembra instabile, potrebbe non essere un problema del firmware ma della rete. Se molti nodi diventano stale insieme, il problema può essere del controller o del Wi-Fi.

Metriche di rete possono includere:

`last_seen_ms;`
`round_trip_time_ms;`

ack_timeout_count;

retry_count;

packet_loss_estimated;

jitter_ms;

duplicate_message_count;

out_of_order_count;

stale_count.

Queste metriche aiutano a distinguere problemi di comunicazione da problemi di comportamento. Un pattern che fallisce per sync_lost non deve essere confuso con un pattern che fallisce per errore geometrico.

Il decimo principio è la raccolta dati adattiva. La frequenza e il dettaglio della telemetria possono cambiare in base allo stato. In condizioni normali, si raccoglie telemetria normale. Se un nodo entra in warning, si aumenta temporaneamente il dettaglio. Se il nodo è low_energy, si riduce la frequenza per risparmiare. Se si entra in diagnostic_mode, si raccoglie più informazione. Questa strategia permette di bilanciare osservabilità ed efficienza.

Una politica adattiva può essere:

normal: telemetria standard;

warning: telemetria più dettagliata sul sottosistema coinvolto;

critical: evento immediato e stato minimo sicuro;

low_energy: telemetria ridotta;

diagnostic: telemetria estesa;

replay/batch: raccolta completa se il costo non è un problema.

Per esempio, se thermal_warning si attiva, il nodo può inviare più dettagli termici. Se energy_state diventa critical, può invece inviare solo evento critico e heartbeat minimo. La telemetria deve adattarsi al contesto.

L'undicesimo principio è la gestione del buffer. In un sistema reale, i dati possono arrivare più velocemente di quanto la dashboard possa mostrarli o il disco possa salvarli. Serve una strategia: buffer circolare per eventi recenti, salvataggio su file per log completi, riduzione della frequenza per dati troppo densi, compressione o campionamento per serie lunghe. Nelle prime versioni non serve un sistema complesso, ma il concetto deve essere previsto.

Tipi di buffer utili:

recent_events_buffer;

latest_snapshot;

metrics_history_buffer;

command_history_buffer;

node_status_cache.

Il latest_snapshot serve alla dashboard. recent_events_buffer serve alla console eventi. metrics_history_buffer serve ai grafici. command_history_buffer serve al debug. node_status_cache serve al controller per decisioni immediate. Separare questi buffer rende il sistema più ordinato.

Il dodicesimo principio è la conservazione selettiva. Non tutto ciò che viene raccolto in tempo reale deve essere salvato per sempre. Alcuni dati sono utili solo live. Altri devono essere archiviati. Per esempio, snapshot completi a 60 FPS genererebbero molti dati. Si può salvare uno snapshot ogni certo intervallo, oppure solo eventi e metriche, oppure salvare snapshot completi solo per run importanti. Questo è importante per evitare cartelle ingestibili.

Una politica di salvataggio può essere:

live mode: mantiene latest_snapshot e recent_events;

debug mode: salva snapshot periodici;

validation mode: salva metrics, events e final_snapshot;

demo mode: salva report e screenshot;

stress mode: salva metriche aggregate e failure events;

full_record mode: salva tutto, solo per esperimenti specifici.

Questa distinzione aiuta a mantenere i dati utili. Non tutte le run devono produrre gigabyte di output.

Il tredicesimo principio è la qualità del dato. I dati raccolti possono essere incompleti, obsoleti, duplicati o contraddittori. Il sistema deve gestire questa realtà. Ogni dato può avere freshness, confidence e source. Se uno snapshot è troppo vecchio, deve essere marcato stale. Se un nodo non aggiorna un campo, quel campo deve restare unknown o usare ultimo valore con avviso. Se due fonti dicono cose diverse, il controller deve sapere quale ha priorità. Questo è importante soprattutto nei nodi ibridi.

Campi utili:

data_age_ms;

data_confidence;

source;

last_updated;

is_stale;

is_estimated;

is_measured.

Questi metadati trasformano la telemetria in informazione affidabile. Un valore senza contesto può essere pericoloso. Per esempio, `thermal_state = ok` ma vecchio di 30 secondi non è veramente affidabile per comandare una bobina.

Il quattordicesimo principio è la normalizzazione. Nodi diversi possono produrre dati in formati diversi, soprattutto se si integrano simulazione, ESP32, dashboard e backend. Il controller deve normalizzare i dati in uno schema comune. Per esempio, un nodo fisico può inviare “`temp_state`”, mentre il sistema usa `thermal_state`. Un simulatore può generare “`battery_sim`”, mentre lo schema usa `energy_state`. La normalizzazione evita che il resto del sistema debba gestire mille varianti.

Una pipeline di normalizzazione può essere:

ricezione messaggio grezzo;

validazione schema;

conversione campi;

aggiunta source;

aggiunta `timestamp_controller`;

calcolo stati interpretati;

aggiornamento `node_status_cache`;

emissione evento se necessario.

Questa pipeline è molto importante per mantenere pulito il sistema. I nodi possono evolvere, ma il controller deve esporre dati coerenti alla dashboard e ai log.

Il quindicesimo principio è la validazione dello schema. Ogni messaggio di telemetria deve rispettare un formato minimo. Se mancano campi essenziali, il sistema deve rifiutare o marcare il dato come incompleto. Questo evita errori silenziosi. Per esempio, un heartbeat senza `node_id` non è utilizzabile. Un evento senza `event_type` è ambiguo. Un dato di temperatura senza source può essere accettato solo con cautela. La raccolta in tempo reale deve includere controlli.

Campi minimi per heartbeat:

`type`;

`node_id`;

`boot_id`;

`timestamp` o `uptime`;

`mode`;

`safety_state`.

Campi minimi per evento:

`event_id` o generabile;

event_type;

timestamp;

severity;

source.

Campi minimi per snapshot:

snapshot_id o step;

timestamp;

nodes;

metrics o almeno stato runtime.

Questi requisiti devono essere documentati in telemetry_schema.md.

Il sedicesimo principio è l'uso dei dati in tempo reale per decisioni operative. La raccolta dati non serve solo a salvare log. Serve al controller per decidere. Se un nodo diventa stale, il controller può escluderlo. Se energy_state diventa low_energy, può riassegnare ruolo. Se thermal_warning cresce, può aumentare cooldown. Se topology_score resta basso, può applicare correzione. Se validation_status fallisce, può avviare fallback. I dati in tempo reale alimentano il comportamento adattivo.

Esempi:

telemetria thermal_warning → riduzione duty;

telemetria low_energy → ruolo passivo;

telemetria missing_edge → topology_repair;

telemetria sync_lost → resync;

telemetria node_offline → fallback;

telemetria pattern_error alto → correzione o timeout.

Questo mostra che la telemetria non è solo osservazione: è feedback di controllo.

Il diciassettesimo principio è la separazione tra telemetria critica e telemetria informativa. La telemetria critica influenza sicurezza e decisioni immediate. La telemetria informativa serve ad analisi e visualizzazione. Un thermal_lock è critico. Un colore LED attuale è informativo, salvo casi particolari. Un emergency_stop event è critico. Una metrica estetica della dashboard è informativa. La pipeline deve dare priorità ai dati critici.

Telemetria critica:

emergency_stop;

safety_lock;

thermal_lock;
critical_energy;
watchdog_reset;
controller_lost;
command_rejected for safety;
node_offline in critical role.

Telemetria informativa:

LED pattern;
debug counters;
visual preferences;
dettagli estesi del modello;
metriche secondarie.

Questa distinzione permette al sistema di reagire velocemente agli eventi importanti, anche se altri dati sono in ritardo.

Il diciottesimo principio è la raccolta dati per il debug. Quando qualcosa non funziona, bisogna avere abbastanza dati per ricostruire la causa. Il debug richiede sequence: quale comando è stato inviato? Il nodo era online? Aveva ruolo corretto? Aveva energy_state ok? Era in cooldown? Il comando è arrivato? È stato rifiutato? Quale errore ha risposto? Il pattern ha applicato fallback? Senza raccolta strutturata, queste domande restano senza risposta.

Per un buon debug servono:

command_log;
event_log;
state transitions;
last known node status;
snapshot before failure;
snapshot after failure;
metrics around failure;
recovery actions.

Questi dati permettono una diagnosi precisa. Invece di dire “non va”, si può dire “il pattern bridge_complex è fallito perché MB-004, bridge_node, è entrato in thermal_lock durante phase_attach; il controller ha tentato fallback ma mancavano nodi compatibili”. Questa è una differenza enorme.

Il diciannovesimo principio è la raccolta dati per il replay. Se si salvano snapshot ed eventi nel tempo, la run può essere riprodotta. Il replay è uno strumento potentissimo per analisi e presentazione. Permette di rivedere pattern, errori, recovery e decisioni del controller. Per supportare replay, i dati devono essere ordinati temporalmente e contenere abbastanza stato. Un semplice report finale non basta.

Un dataset replay può includere:

initial_config;

snapshot_sequence;

event_sequence;

metrics_sequence;

final_result.

Con questi dati, la dashboard può ricostruire la run. Si può saltare a un evento, rallentare una fase, analizzare un nodo. Questo rende la telemetria molto più utile.

Il ventesimo principio è la raccolta dati per confronto tra run. Se ogni run salva metriche e configurazione, si possono confrontare versioni diverse. Per esempio, line_basic_v0.1 e v0.2, cluster con due profili, safety policy diverse, network profiles diversi. Il confronto richiede dati omogenei. Quindi i report devono usare campi coerenti. Se ogni run salva metriche diverse senza schema, il confronto diventa difficile.

Metriche comuni di confronto:

final_result;

completion_time;

pattern_error_final;

topology_score_final;

stability_time;

correction_count;

fallback_count;

safety_event_count;

energy_event_count;

thermal_event_count;

recovery_count.

Queste metriche possono essere presenti in ogni report, anche se alcune non sono applicabili. Questo crea una base comune.

Il ventunesimo principio è la raccolta dati per la dashboard live. La dashboard ha bisogno di dati aggiornati e compatti. Non deve ricevere per forza tutto il log completo a ogni frame. Può ricevere `latest_snapshot`, `recent_events` e `metrics_summary`. I dettagli possono essere richiesti solo quando l'utente seleziona un nodo. Questo rende la dashboard più efficiente.

Una pipeline dashboard può essere:

`latest_snapshot` per canvas;

`metrics_summary` per HUD;

`recent_events` per console;

`node_detail` endpoint per pannello selezionato;

`report` endpoint per run finale.

Questa separazione evita di inviare dati pesanti inutilmente.

Il ventiduesimo principio è la raccolta dati per report sperimentali. Alla fine di una run, i dati live devono essere condensati in report. Il report non deve contenere necessariamente ogni singolo messaggio, ma deve riassumere ciò che conta: configurazione, metriche, eventi importanti, errori, `safety`, `recovery`, risultato e raccomandazioni. Questo report diventa documento tecnico.

Un report generato dalla raccolta dati può includere:

configurazione iniziale;

stato iniziale dello swarm;

timeline delle fasi;

metriche principali;

eventi critici;

correzioni;

fallback;

`safety summary`;

risultato finale;

lezioni apprese.

Questo è materiale perfetto per documentazione, README e tesi.

Il ventitreesimo principio è la raccolta dati hardware. Quando entrano nodi ESP32 o altri dispositivi fisici, la telemetria deve gestire limiti reali: banda, batteria, Wi-Fi, perdita pacchetti, payload piccoli, parsing semplice. Non si può pretendere che un microcontrollore invii dati enormi continuamente. La telemetria hardware deve essere compatta. I dati dettagliati possono essere richiesti solo in `diagnostic_mode`. L'heartbeat deve essere leggero.

Un heartbeat hardware compatto può contenere:

node_id;
boot_id;
uptime;
mode;
role;
safety_state;
energy_state;
thermal_state;
sync_status;
last_error_code.

Questo è sufficiente per monitoraggio. Se serve di più, request_status può chiedere dettagli. Questa scelta protegge rete ed energia.

Il ventiquattresimo principio è la raccolta dati simulata. Nei nodi simulati, il costo è minore, quindi si possono raccogliere più dati: posizione, velocità, target, errori, vicini, forze virtuali, thermal_load, energy_level, edge status. Tuttavia, anche qui bisogna evitare eccesso. Se si simulano molti nodi, salvare tutto a ogni frame può diventare pesante. Serve configurare il livello di raccolta.

Profili per simulazione:

sim_minimal: solo metriche globali;
sim_normal: nodi, edge e metriche;
sim_debug: target, errori, eventi dettagliati;
sim_full: tutti i dati per replay completo.

Questi profili permettono di adattare raccolta allo scopo.

Il venticinquesimo principio è la raccolta dati ibrida. In modalità hybrid, il sistema deve fondere dati fisici e simulati. Questo è delicato. Un nodo fisico può fornire heartbeat reale, ma posizione simulata. Il sistema deve evitare conflitti e indicare source. La raccolta deve creare un unified_node_state che combina fonti diverse, mantenendo traccia dell'origine. Questa è una delle funzioni più importanti per MicroBot.

Un unified_node_state può contenere:

node_id;
node_type = hybrid_node;
hardware_status;
virtual_status;

merged_status;

field_sources;

confidence;

last_hardware_seen;

last_virtual_update.

Questo permette alla dashboard di mostrare il nodo in modo coerente. È un ponte tra realtà e simulazione.

Il ventiseiesimo principio è la privacy e sicurezza dei dati, almeno in senso tecnico. Anche se MicroBot non tratta necessariamente dati personali nella telemetria base, la Sezione 15 introdurrà sicurezza biometrica. Se in futuro la telemetria include dati da fotocamera, landmark, autenticazione o access control, bisognerà separare dati sensibili da telemetria tecnica. Per ora, la regola è progettare la telemetria in modo modulare: i dati biometrici non devono finire nei log generici senza controllo. Questo verrà approfondito nella sezione dedicata, ma è bene preparare la struttura.

Una distinzione futura può essere:

technical_telemetry;

diagnostic_telemetry;

security_events;

biometric_processing_state;

sensitive_data_excluded_from_general_logs.

Questo mantiene ordine e sicurezza.

Il ventisettesimo principio è la raccolta dati come base per analisi futura con AI o algoritmi. I dati raccolti da run MicroBot possono diventare dataset: sequenze di stati, metriche, successi, fallimenti, configurazioni. In futuro, questi dataset potrebbero essere usati per ottimizzare parametri, classificare fallimenti, predire instabilità o suggerire pattern. Tuttavia, prima serve raccolta ordinata. Un dataset sporco non aiuta. Quindi la telemetria deve essere pensata anche come base futura per analisi più avanzata.

Dataset utili in futuro:

run_metrics_dataset;

failure_modes_dataset;

pattern_performance_dataset;

node_telemetry_dataset;

energy_thermal_dataset;

network_latency_dataset.

Questi dataset possono collegarsi ai progetti esterni di AI, clustering e analisi dati. Ma la base è la telemetria pulita.

Il ventottesimo principio è la documentazione dei formati. Ogni file prodotto dalla raccolta dati deve avere schema: snapshot.json, events.jsonl, metrics.json, command_log.jsonl, report.md. Senza schema, i file diventano difficili da usare. La documentazione deve spiegare campi, unità, valori ammessi, frequenza, fonte e esempio. Questo è essenziale per repo e collaborazione.

Una cartella telemetry può contenere:

schema_snapshot.md;

schema_events.md;

schema_metrics.md;

schema_command_log.md;

sample_snapshot.json;

sample_events.jsonl;

sample_metrics.json;

README.md.

Questa struttura rende la telemetria leggibile anche da altri.

Il ventinovesimo principio è la roadmap della raccolta dati in tempo reale. Non serve implementare tutto subito. Prima heartbeat e snapshot. Poi event log. Poi metrics time series. Poi command log. Poi dashboard live. Poi replay. Poi report. Poi hardware heartbeat. Poi hybrid merge. Questa progressione è realistica.

Una roadmap possibile è:

v0.1: snapshot statico;

v0.2: heartbeat simulato;

v0.3: event_log;

v0.4: metrics_time_series;

v0.5: dashboard live polling;

v0.6: command_log;

v0.7: report automatico;

v0.8: replay;

v0.9: hardware heartbeat ESP32;

v1.0: telemetria ibrida simulazione-hardware.

Questa roadmap collega sviluppo software e hardware.

Il trentesimo principio è la raccolta dati come fondamento della credibilità del progetto. Un sistema che non raccoglie dati può essere impressionante visivamente, ma è difficile da validare. Un sistema che raccoglie dati può dimostrare progressi, confrontare versioni, spiegare errori, produrre report, documentare test e preparare hardware. Per MicroBot, la telemetria è ciò che trasforma la simulazione in esperimento e il prototipo in sistema osservabile.

In conclusione, la raccolta dati in tempo reale è il processo attraverso cui MicroBot acquisisce stati, eventi, snapshot, metriche, comandi, ack, dati di rete, safety, energia, temperatura e recovery durante l'esecuzione. Essa deve distinguere dati continui, periodici, su evento, su richiesta e finali; deve usare timestamp, source, freshness e confidence; deve supportare snapshot, event log, metriche temporali, command log, dashboard live, replay e report. Deve funzionare in simulazione, hardware e modalità ibrida, mantenendo sempre chiara la provenienza dei dati. La raccolta dati non è solo registrazione: è feedback operativo, base della validazione, strumento di debug e memoria sperimentale del sistema. Il punto successivo, dedicato all'analisi del sistema, mostrerà come questi dati raccolti vengano trasformati in diagnosi, metriche aggregate, confronti, raccomandazioni e decisioni progettuali per migliorare progressivamente l'intero ecosistema MicroBot.

13.3 Analisi del sistema

L'analisi del sistema rappresenta il terzo livello della telemetria MicroBot e consiste nel processo attraverso cui i dati raccolti durante simulazioni, run ibride, test hardware e prove sperimentali vengono trasformati in informazioni utili per capire il comportamento complessivo dello swarm. Se il monitoraggio dello stato permette di sapere cosa sta accadendo in un dato momento, e la raccolta dati in tempo reale permette di salvare l'evoluzione della run, l'analisi del sistema serve a interpretare quei dati. Essa risponde a domande più profonde: il pattern è riuscito davvero? Quali nodi hanno lavorato di più? Quale fase ha prodotto più errori? Il comportamento è stabile o solo momentaneamente corretto? Il Safety Layer è intervenuto troppo spesso? L'energia è stata sufficiente? La temperatura è rimasta sotto controllo? La rete ha causato ritardi? La recovery ha funzionato? Il sistema sta migliorando tra una versione e l'altra?

Nel progetto MicroBot, l'analisi del sistema è fondamentale perché impedisce di giudicare il progetto solo in base all'impressione visiva. Una simulazione può sembrare corretta perché i nodi, a colpo d'occhio, formano una linea o un cluster. Tuttavia, senza analisi, non si sa se la linea è stabile, se gli edge topologici sono corretti, se i ruoli sono coerenti, se i nodi sono sincronizzati, se ci sono stati warning termici, se il pattern ha richiesto troppe correzioni o se alcuni nodi hanno consumato troppa energia. L'analisi serve proprio a trasformare la scena in valutazione tecnica.

Il principio generale dell'analisi del sistema può essere espresso così:

i dati telemetrici devono produrre diagnosi, non soltanto registrazioni.

Registrare dati è necessario, ma non sufficiente. Un file pieno di snapshot, eventi e metriche ha valore solo se permette di capire qualcosa. L'analisi deve quindi aggregare, confrontare, classificare e interpretare. Deve ridurre il rumore informativo e mostrare ciò che conta. Se una run fallisce, deve aiutare a capire perché. Se una run riesce, deve dire quanto è stata pulita, stabile e sicura. Se una versione migliora una metrica ma peggiora un'altra, deve renderlo visibile. L'analisi del sistema è quindi il livello che trasforma la telemetria in conoscenza progettuale.

La prima funzione dell'analisi è la classificazione finale della run. Ogni esecuzione del sistema deve concludersi con un risultato chiaro. Non basta dire "finito" o "ok". Una run può essere

success_clean, success_with_warnings, success_with_corrections, partial_success_with_fallback, failed_geometry, failed_topology, failed_safety, failed_energy, failed_thermal, failed_timing, failed_recovery o aborted_by_emergency_stop. Questa classificazione permette di distinguere situazioni molto diverse. Un pattern che raggiunge la forma senza errori non è uguale a un pattern che ci arriva dopo dieci correzioni, due fallback e un thermal_warning. Entrambi possono sembrare riusciti visivamente, ma hanno qualità tecnica diversa.

Una classificazione finale può essere generata analizzando:

metriche geometriche;

metriche topologiche;

coerenza dei ruoli;

stabilità temporale;

eventi safety;

eventi energetici;

eventi termici;

numero di correzioni;

fallback attivati;

recovery avvenute;

timeout;

stato finale dei nodi.

Questa classificazione deve essere spiegabile. Il report non deve limitarsi a indicare “partial_success_with_fallback”, ma deve indicare il motivo: per esempio, “circle_basic è stato degradato in arc_open perché il nodo SIM-006 è diventato offline durante phase_attach e il cycle_closure_score è rimasto sotto soglia”. Questo tipo di spiegazione rende l’analisi utile.

La seconda funzione è l’analisi delle metriche geometriche. Le metriche geometriche misurano quanto la forma osservata corrisponde alla forma desiderata. Per una linea, si analizzano errore laterale, spacing, ordine dei nodi e distanza dai target. Per un cerchio, raggio, distribuzione angolare e chiusura. Per una griglia, celle, righe, colonne e deformazione. Per un cluster, raggio, densità, isolamento e collisioni. Per un ponte, sub-pattern e connessione tra gruppi. L’analisi geometrica deve essere specifica per pattern; non esiste una metrica unica valida per tutto.

Per line_basic, l’analisi può includere:

lateral_error_mean;

lateral_error_max;

spacing_error_mean;

order_error_count;

distance_to_target_mean;

stable_for_ms.

Per circle_basic, può includere:

radial_error_mean;

angular_distribution_error;

closure_score;

cycle_topology_score;

neighbor_distance_variance.

Per cluster_basic, può includere:

cluster_radius;

average_distance_to_center;

isolated_nodes;

collision_count;

density_score;

aggregation_score.

Queste metriche permettono di capire se il pattern è realmente formato. Se l'errore medio è basso ma l'errore massimo è alto, significa che la maggior parte dei nodi è corretta ma uno o pochi nodi sono fuori posto. Se lo spacing medio è buono ma order_error_count è alto, i nodi sono sulla linea ma nell'ordine sbagliato. L'analisi deve quindi guardare più metriche insieme.

La terza funzione è l'analisi topologica. La topologia riguarda le relazioni tra nodi, cioè quali connessioni esistono, quali dovrebbero esistere e quali non dovrebbero esserci. In MicroBot questo è particolarmente importante perché molte strutture non sono definite solo dalla posizione, ma dagli edge. Una linea può essere una catena. Un cerchio può essere un ciclo. Una griglia può essere un grafo bidimensionale. Un ponte può collegare due cluster. Una struttura può sembrare geometricamente corretta ma avere topologia sbagliata.

L'analisi topologica deve considerare:

expected_edges;

current_edges;

missing_edges;

extra_edges;

critical_edges;

edge_stability;

connected_components;

topology_score.

Se expected_edges e current_edges coincidono, la topologia è corretta. Se ci sono missing_edges, alcune relazioni previste non si sono formate. Se ci sono extra_edges, il sistema ha connessioni non previste. Se un critical_edge manca, il pattern può fallire anche se molti edge secondari sono presenti. Per esempio, in un ponte, la connessione centrale può essere più importante di un edge periferico. L'analisi deve quindi pesare gli edge in base alla loro criticità.

La quarta funzione è l'analisi dei ruoli. Ogni pattern assegna ruoli ai nodi. L'analisi deve verificare se questi ruoli sono stati assegnati, confermati, mantenuti e usati correttamente. Un ruolo non è solo una label: influenza comportamento, autorizzazioni, safety e partecipazione al pattern. Se un nodo deve essere active_connector ma si trova in passive_node, il pattern può essere compromesso. Se un nodo in low_energy riceve un ruolo attivo, c'è errore di assegnazione. Se un nodo cambia ruolo troppe volte, può indicare instabilità.

L'analisi dei ruoli può includere:

role_assignment_success_rate;

role_confirmation_rate;

role_mismatch_count;

critical_role_missing;

role_change_count;

unauthorized_role_change_count;

role_energy_compatibility;

role_thermal_compatibility;

role_position_consistency.

Un risultato interessante è role_energy_compatibility. Questo misura se i ruoli attivi sono stati assegnati a nodi con energia adeguata. Se un bridge_node critico aveva low_energy, la run era fragile già dall'inizio. Allo stesso modo, role_thermal_compatibility indica se nodi in thermal_warning sono stati usati per attuazioni. Questo collega role assignment e Safety Layer.

La quinta funzione è l'analisi temporale. Il comportamento MicroBot è una sequenza nel tempo. Una run passa attraverso fasi: prepare, assign_roles, align, attach, stabilize, report. L'analisi deve valutare durata delle fasi, transizioni, ritardi, timeout, comandi scaduti e stabilità nel tempo. Una run può raggiungere il pattern finale ma impiegare troppo, oppure può restare instabile, oppure può avere ritardi eccessivi in una fase specifica. L'analisi temporale permette di identificare colli di bottiglia.

Metriche temporali utili sono:

total_run_duration_ms;

phase_duration_ms per ogni fase;

time_to_convergence;

stable_for_ms;

timeout_count;

command_expired_count;

late_ack_count;

phase_retries;

phase_transition_errors.

Per esempio, se phase_align dura molto più del previsto, il problema può essere nei parametri di movimento o nella distribuzione dei target. Se phase_attach genera molti timeout, il problema può essere nella topologia, comunicazione o safety. Se stabilize non raggiunge stable_for_ms, il pattern è troppo oscillante. L'analisi temporale trasforma il tempo in diagnosi.

La sesta funzione è l'analisi della stabilità. La stabilità misura se il comportamento resta valido nel tempo. Una struttura non deve solo formarsi: deve mantenersi. L'analisi deve osservare oscillazioni, variazioni di errore, velocità media dei nodi, edge instabili, cambi frequenti di ruolo e transizioni ripetute tra stati. Un pattern con errore finale basso ma oscillazione alta è meno affidabile di un pattern con errore leggermente maggiore ma stabile.

Metriche di stabilità possono essere:

stability_score;

error_variance_over_time;

average_node_velocity_after_convergence;

edge_stability_score;

role_flapping_count;

state_flapping_count;

oscillation_index;

stable_window_count.

Il role_flapping_count misura quante volte i ruoli cambiano avanti e indietro. Lo state_flapping_count misura oscillazioni tra stati come thermal_warning e thermal_ok, oppure sync_ok e sync_degraded. Queste oscillazioni possono indicare soglie troppo strette, hysteresis insufficiente o parametri aggressivi. L'analisi della stabilità è quindi utile anche per migliorare le policy.

La settima funzione è l'analisi del Safety Layer. Dopo la Sezione 11, ogni run deve essere valutata anche in termini di sicurezza. L'analisi deve contare e interpretare eventi di corrente, bobine, temperatura, batteria, protezione energetica, watchdog e recovery. Un pattern che funziona ma genera molti thermal_warning non è pronto per hardware attuativo. Un pattern che richiede critical_energy o reset non è sicuro. Un comando rifiutato dal Safety Layer non è necessariamente

un errore: può essere una protezione corretta. L'analisi deve distinguere protezione corretta da comportamento problematico.

Metriche safety possono includere:

safety_event_count;

thermal_warning_count;

thermal_lock_count;

low_energy_count;

critical_energy_count;

command_rejected_safety_count;

cooldown_active_rejection_count;

watchdog_reset_count;

emergency_stop_count;

recovery_required_count.

L'analisi deve interpretare questi dati. Per esempio, un singolo command_rejected perché il sistema ha bloccato un comando non sicuro può essere positivo: dimostra che la safety funziona. Ma molti command_rejected ripetuti possono indicare che il controller pianifica male o che il pattern è incompatibile con i limiti. thermal_lock deve essere trattato come evento critico; thermal_warning può essere accettabile in test stress, ma non in success_clean.

La ottava funzione è l'analisi energetica. Il sistema deve capire come viene usata l'energia. Quali nodi consumano di più? Quali ruoli sono più costosi? Il pattern supera il budget energetico? Ci sono nodi low_energy all'inizio o durante la run? Il controller riassegna ruoli quando l'energia scende? L'analisi energetica è fondamentale per preparare hardware reale e scalabilità.

Metriche energetiche possono essere:

energy_budget_used;

energy_event_count;

nodes_low_energy_count;

nodes_critical_energy_count;

high_energy_command_count;

energy_cost_by_node;

energy_cost_by_role;

energy_cost_by_phase;

commands_blocked_by_energy;

estimated_remaining_energy.

L'analisi può mostrare, per esempio, che i connector consumano molto più dei boundary_node, oppure che phase_attach è la fase energeticamente più costosa, oppure che retry ripetuti consumano troppo. Queste informazioni guidano il miglioramento dei pattern e dei ruoli.

La nona funzione è l'analisi termica. La temperatura e il thermal_load sono collegati a bobine, PWM, attuatori e carichi intensi. L'analisi termica deve mostrare quali nodi hanno raggiunto thermal_warning, quali sono entrati in thermal_lock, quanto tempo hanno passato in cooldown e quali comandi hanno generato carico. Anche se i valori sono simulati o stimati, l'analisi aiuta a progettare policy più conservative.

Metriche termiche possono essere:

max_thermal_load_by_node;

average_thermal_load;

thermal_warning_duration;

thermal_lock_duration;

cooldown_total_time;

thermal_recovery_count;

thermal_events_by_phase;

thermal_events_by_command_type;

time_to_thermal_recovery.

Queste metriche permettono di capire se il pattern è termicamente sostenibile. Se phase_attach produce sempre thermal_warning, bisogna ridurre duty virtuale, aumentare cooldown, distribuire gli slot o cambiare strategia. Se un nodo specifico si scalda sempre più degli altri, potrebbe essere sovraccaricato da ruoli critici.

La decima funzione è l'analisi della comunicazione. Un comportamento distribuito dipende dalla qualità dello scambio di dati. L'analisi deve distinguere problemi di pattern da problemi di comunicazione. Se molti ack arrivano tardi, se molti comandi scadono, se nodi diventano stale, se il jitter è alto, il sistema può fallire anche se il pattern è definito bene. L'analisi della comunicazione permette di regolare heartbeat, timeout, retry, sync e frequenza di telemetria.

Metriche comunicative possono includere:

average_rtt_ms;

max_rtt_ms;

jitter_ms;

ack_timeout_count;

retry_count;

packet_loss_estimated;

stale_node_count;

offline_transitions;

duplicate_message_count;

out_of_order_message_count.

Se ack_timeout_count è alto, il controller può aumentare timeout o ridurre frequenza di comandi. Se stale_node_count cresce con molti nodi, la telemetria è troppo pesante o la rete è fragile. Se sync_degraded è frequente, serve rivedere la strategia di sincronizzazione. Questi risultati sono molto utili per hardware futuro.

L'undicesima funzione è l'analisi della recovery. Quando un nodo o una run entra in recovery, l'analisi deve verificare se il processo è stato corretto. Il nodo è stato messo in safe state? Il controller ha rilevato boot_id cambiato? È stato eseguito state_sync? Le capabilities sono state aggiornate? Il nodo è stato reintegrato o escluso? La recovery ha rispettato limiti di retry? Questo è fondamentale perché la recovery non deve essere una ripartenza cieca.

Metriche recovery possono includere:

recovery_count;

recovery_success_count;

recovery_failure_count;

average_recovery_time;

recovery_type_distribution;

reintegration_count;

isolation_count;

manual_intervention_required_count;

recovery_retry_count;

recovery_after_watchdog_count.

L'analisi può distinguere recovery riuscita, recovery degradata, recovery fallita e recovery non necessaria. Se un nodo recupera ma viene reintegrato solo come passive_node, il risultato è recovered_degraded. Se un nodo continua a resettarsi e viene isolato, il sistema ha fatto bene a non insistere. Questi dati mostrano maturità del runtime.

La dodicesima funzione è l'analisi delle cause di fallimento. Quando una run fallisce, il sistema deve provare a classificare la causa primaria e le cause secondarie. Per esempio, failure_reason = missing_critical_edge, secondary_reason = node_low_energy. Oppure failure_reason = timeout_phase_attach, secondary_reason = high_packet_loss. Questa classificazione è fondamentale per migliorare il progetto. Senza causa, il fallimento resta generico.

Possibili cause di fallimento:

invalid_configuration;
insufficient_nodes;
capability_missing;
role_assignment_failed;
geometry_not_converged;
topology_incomplete;
critical_edge_missing;
sync_lost;
timeout;
energy_budget_exceeded;
thermal_lock;
watchdog_reset;
controller_lost;
recovery_failed;
manual_abort;
emergency_stop.

L'analisi deve individuare il momento in cui il fallimento è diventato inevitabile o più probabile. Per esempio, un nodo è andato offline al secondo 4, ma il pattern è fallito al secondo 15. La causa reale può essere il nodo offline, non il timeout finale. Questo richiede analisi della timeline.

La tredicesima funzione è l'analisi per fase. Una run complessa deve essere studiata fase per fase. Ogni fase ha obiettivi propri. Prepare deve verificare disponibilità. Assign_roles deve confermare ruoli. Align deve ridurre errore geometrico. Attach deve formare edge. Stabilize deve mantenere stabilità. Report deve salvare dati. L'analisi per fase permette di capire esattamente dove migliorare.

Una phase_analysis può contenere:

phase_id;
start_time;
end_time;
duration;
success_status;

metrics_start;

metrics_end;

events;

errors;

corrections;

safety_events;

recommendation.

Per esempio, phase_align può essere riuscita ma lenta. phase_attach può essere fallita per missing_edge. phase_stabilize può essere riuscita ma con oscillazione alta. Questo livello di dettaglio rende il report molto più utile.

La quattordicesima funzione è l'analisi per nodo. Ogni MicroBot può essere valutato singolarmente. Alcuni nodi possono essere più problematici: più spesso offline, più spesso low_energy, più spesso fuori target, più spesso coinvolti in edge mancanti. Un'analisi per nodo può identificare nodi instabili, sovraccaricati o mal configurati. In hardware, questo sarà ancora più importante: un ESP32 specifico potrebbe avere alimentazione, Wi-Fi o firmware problematico.

Metriche per nodo:

uptime_percent;

average_error_to_target;

role_history;

command_count;

rejection_count;

energy_events;

thermal_events;

recovery_count;

offline_count;

stability_score;

critical_role_count.

Se MB-004 ha molti reset, non deve essere assegnato a ruoli critici. Se SIM-007 è spesso isolato nei cluster, forse il behavior profile è sbagliato o la posizione iniziale è estrema. L'analisi per nodo aiuta a localizzare problemi.

La quindicesima funzione è l'analisi per ruolo. Alcuni ruoli possono essere più costosi o più fragili. Per esempio, active_connector può generare molti eventi termici. bridge_node può essere

criticissimo per la riuscita. leader può influenzare sincronizzazione. boundary_node può avere errore geometrico più alto. Analizzare per ruolo permette di migliorare la progettazione dei pattern.

Metriche per ruolo:

average_pattern_error_by_role;

energy_cost_by_role;

thermal_load_by_role;

command_count_by_role;

failure_rate_by_role;

recovery_count_by_role;

role_stability;

role_reassignment_frequency.

Queste metriche possono rivelare che un ruolo richiede policy speciali. Per esempio, bridge_node potrebbe richiedere energy_ok e sync_good obbligatori. Active_connector potrebbe richiedere cooldown più alto. Leader potrebbe richiedere maggiore stabilità. L'analisi per ruolo migliora il controller.

La sedicesima funzione è l'analisi per edge. Nei pattern topologici, gli edge hanno importanza centrale. Alcuni edge falliscono spesso. Alcuni sono instabili. Alcuni sono critici. L'analisi per edge permette di capire quali connessioni sono più fragili. Questo può guidare correzioni del pattern o del comportamento.

Metriche per edge:

edge_presence_time;

edge_missing_count;

edge_instability_count;

edge_formation_time;

edge_failure_reason;

critical_edge_status;

edge_retry_count.

Per esempio, in circle_basic, l'edge di chiusura tra ultimo e primo nodo può fallire spesso. L'analisi lo evidenzia e suggerisce una policy specifica per closure. In bridge_complex, un edge centrale può essere sempre il punto debole. Questo tipo di diagnosi è molto utile.

La diciassettesima funzione è l'analisi comparativa tra run. Una singola run dice poco. Il confronto tra run permette di valutare miglioramenti. Si possono confrontare versioni di pattern, behavior profiles, safety policies, network profiles o visualizzazioni. L'analisi comparativa deve usare metriche comuni e produrre conclusioni.

Un confronto può includere:

run_id;
pattern_version;
success_rate;
completion_time;
final_error;
topology_score;
safety_events;
energy_events;
thermal_events;
recovery_count;
recommendation.

Per esempio, line_basic_v0.2 può ridurre completion_time del 20% ma aumentare oscillation_score. Il sistema deve mostrare questo trade-off. Non sempre una metrica migliora senza peggiorarne un'altra. L'analisi comparativa aiuta a scegliere consapevolmente.

La diciottesima funzione è l'analisi delle regressioni. Quando si modifica codice o parametri, una funzione già valida può rompersi. L'analisi deve individuare regressioni: pattern che prima passavano e ora falliscono, metriche peggiorate oltre soglia, nuovi warning, tempi peggiori. Questo è essenziale per mantenere il progetto stabile.

Una regression_analysis può indicare:

test_name;
previous_result;
current_result;
metric_delta;
regression_detected;
severity;
likely_cause;
recommended_action.

Se cluster_basic migliora ma line_basic peggiora, bisogna decidere se creare profili diversi o correggere la regola. L'analisi delle regressioni evita che il progetto migliori in una parte e si rompa in un'altra.

La diciannovesima funzione è l'analisi della scalabilità. Anche se la Sezione 19 tratterà la scalabilità in modo completo, l'analisi del sistema deve già raccogliere dati sul comportamento al crescere dei nodi. Completion time, frame rate, step time, event rate, snapshot size, topology complexity e communication load sono dati fondamentali. Se un pattern funziona con 10 nodi ma non con 100, bisogna saperlo.

Metriche di scalabilità:

node_count;

edge_count;

simulation_step_ms;

render_frame_ms;

snapshot_size_kb;

event_rate;

controller_processing_time;

completion_time_vs_N;

topology_score_vs_N;

memory_usage.

Queste metriche aiutano a capire se il problema è comportamentale, computazionale o visuale. Una dashboard può rallentare anche se il simulatore regge. Un algoritmo $O(N^2)$ può diventare costoso. La telemetria deve permettere di scoprirlo.

La ventesima funzione è l'analisi della qualità dei dati. L'analisi stessa deve sapere quanto i dati sono affidabili. Se molti snapshot sono stale, se mancano campi, se alcuni nodi hanno source = unavailable, il report deve indicare validation_confidence. Una run con dati incompleti non può essere classificata con la stessa sicurezza di una run ben monitorata. Questo è molto importante per rigore.

Metriche di qualità dati:

telemetry_completeness;

missing_field_count;

stale_snapshot_count;

average_snapshot_age;

node_update_rate;

source_confidence;

data_gap_count;

validation_confidence.

Se `validation_confidence` è bassa, il risultato deve essere “uncertain” o “partially_validated”. Questo evita conclusioni troppo forti su dati deboli.

La ventunesima funzione è l’analisi dei trade-off. In sistemi complessi, migliorare una cosa può peggiorarne un’altra. Aumentare coesione può migliorare `aggregation_score` ma aumentare `collision_count`. Aumentare `retry` può migliorare `topology_score` ma aumentare `energy_cost` e `thermal_load`. Ridurre telemetria può risparmiare energia ma abbassare osservabilità. Aumentare `safety` può ridurre rischi ma generare più rifiuti. L’analisi deve mostrare questi trade-off.

Esempi di trade-off:

velocità di convergenza vs stabilità;

cohesion vs collision risk;

retry count vs energy consumption;

duty cycle vs thermal safety;

telemetry detail vs bandwidth/energy;

visual richness vs dashboard performance;

fallback aggressivo vs obiettivo originale.

Questi trade-off sono parte naturale del progetto. L’analisi non deve nasconderli; deve renderli chiari.

La ventiduesima funzione è la generazione di raccomandazioni. Dopo aver analizzato una run, il sistema può produrre suggerimenti. Inizialmente possono essere manuali o semi-automatici. In futuro, possono diventare più intelligenti. Le raccomandazioni devono essere basate sui dati. Se `phase_align` è lenta, suggerire aumento `target_weight` o riduzione `damping`. Se `collision_count` è alto, aumentare `separation_weight`. Se `thermal_warning` è frequente, aumentare `cooldown` o ridurre `duty`. Se `missing_edges` sono frequenti, migliorare `attach_policy` o aumentare `connection_radius` virtuale.

Esempi di raccomandazioni:

ridurre `cohesion_weight` perché `collision_count` è alto;

aumentare `stabilization_time` perché `oscillation_score` resta elevato;

ridurre `retry_count` perché `energy_budget` viene superato;

aumentare `cooldown` perché `thermal_warning` si ripete;

escludere MB-004 da ruoli critici perché ha reset frequenti;

migliorare `timeout` perché `late_ack_count` è alto;

attivare fallback più presto perché `phase_attach` supera durata media.

Le raccomandazioni non devono essere presentate come verità assolute, ma come indicazioni tecniche derivate dalle metriche.

La ventitreesima funzione è la produzione del report analitico. Il report è il documento finale dell'analisi. Deve riassumere la run in modo leggibile e tecnico. Può essere generato in Markdown, JSON o visualizzato in dashboard. Deve includere configurazione, risultato, metriche, timeline, eventi, problemi, safety, recovery, confronto con soglie e raccomandazioni. Il report è ciò che rende una run utile anche dopo che è finita.

Un report analitico può avere questa struttura:

Run Overview;

Configuration Summary;

Pattern and Phase Summary;

Node Summary;

Geometry Metrics;

Topology Metrics;

Role Analysis;

Safety Analysis;

Energy and Thermal Analysis;

Communication Analysis;

Recovery Analysis;

Failure or Success Reason;

Recommendations;

Limitations;

Next Steps.

Questa struttura è perfetta per documentazione tecnica. Può essere inclusa in repo, relazione o report sperimentale.

La ventiquattresima funzione è l'analisi per dashboard. Non tutta l'analisi deve stare in un report finale. Una parte deve essere mostrata live. La dashboard può mostrare metriche aggregate, warning, stato di validazione, fase problematica, nodo critico, edge mancante, trend di errore. L'analisi live aiuta l'utente a intervenire o capire durante la run. L'analisi post-run aiuta a documentare e migliorare dopo.

Dashboard analysis live:

pattern_error trend;

topology_score;

stability_status;

safety summary;
critical nodes;
phase health;
event severity count;
validation status.
Post-run analysis:
report completo;
timeline;
metriche finali;
failure reason;
confronti;
raccomandazioni.

Questa distinzione è importante. La dashboard live deve essere sintetica; il report può essere dettagliato.

La venticinquesima funzione è l'analisi dei comportamenti emergenti. Non tutti i comportamenti hanno target preciso. Cluster, aggregazione, dispersione controllata, coesione e pattern emergenti devono essere analizzati con metriche adatte. L'analisi deve evitare di forzare forme rigide su comportamenti flessibili. Per un cluster, può essere valido avere molte configurazioni finali diverse, purché siano dense, connesse, stabili e senza collisioni.

Metriche emergenti:

aggregation_score;
density_score;
connected_components;
isolated_nodes;
cluster_radius;
cohesion_separation_balance;
emergence_stability;
pattern_variability.

Queste metriche permettono di studiare comportamenti più vicini alla swarm intelligence. Sono importanti per la visione MicroBot.

La ventiseiesima funzione è l'analisi della modalità ibrida. In una run hybrid, alcuni dati sono reali e altri simulati. L'analisi deve separare i contributi. Per esempio, può dire: “la geometria è stata validata in simulazione; il role assignment è stato confermato da tre nodi hardware; la telemetria hardware ha confermato heartbeat e LED state; l'attuazione magnetica non è stata hardware-validata”. Questa precisione è fondamentale per presentazioni e documentazione.

Una hybrid_analysis può contenere:

simulated_nodes_count;

hardware_nodes_count;

hybrid_nodes_count;

hardware_confirmed_roles;

hardware_heartbeat_success;

simulated_geometry_validation;

hardware_visual_validation;

hardware_actuation_validation_status;

limitations.

Questo evita confusione. Una demo ibrida può essere molto valida, ma va descritta correttamente.

La ventisettesima funzione è l'analisi della maturità. Ogni modulo, pattern o funzione può avere un livello di maturità. L'analisi può aggiornare questo livello in base ai risultati. Per esempio, line_basic può passare da implemented_in_simulation a validated_with_metrics. Safety command rejection può passare da simulated a firmware_candidate. Hybrid heartbeat può passare da experimental a demo_ready. Questo aiuta a gestire la roadmap.

Una maturità può essere:

concept;

implemented;

visualized;

tested;

validated;

stress_tested;

hybrid_ready;

hardware_partial;

hardware_ready.

L'analisi non serve solo a giudicare la singola run, ma ad aggiornare lo stato del progetto.

La ventottesima funzione è l'analisi dei limiti. Ogni report deve indicare anche cosa non è stato validato. Per esempio: "la run è stata eseguita con nodi simulati; non valida locomozione fisica", oppure "thermal_state è stimato, non misurato", oppure "communication loss è simulato", oppure "hardware actuation non testata". Questo è fondamentale per rigore. Dichiarare i limiti non indebolisce il lavoro; lo rende credibile.

Un blocco limitations può indicare:

dati simulati;

dati stimati;

hardware non coinvolto;

numero nodi limitato;

assenza di sensori reali;

modello fisico semplificato;

rete simulata;

pattern non testato in stress.

Questa sezione evita affermazioni eccessive. È molto importante in un documento serio.

La ventinovesima funzione è l'analisi come guida alla roadmap. Ogni analisi deve terminare con decisioni. Se una run è valida, cosa si fa dopo? Si porta in demo? Si testa con fault? Si integra con hardware? Se fallisce, cosa si modifica? La telemetria deve guidare il lavoro successivo. Senza questa parte, il report resta descrittivo. Con essa, diventa operativo.

Possibili next steps:

promuovere pattern a demo;

ripetere test con più nodi;

aggiungere fault injection;

migliorare role assignment;

ridurre safety warnings;

aggiornare behavior_profile;

portare su hardware visuale;

aggiungere metriche mancanti;

migliorare dashboard.

Questi next steps devono essere specifici. "Migliorare sistema" è troppo generico. "Aumentare cooldown in phase_attach e ripetere bridge_complex con 10 run" è utile.

La trentesima funzione è l'analisi come ponte verso il debug dello swarm. L'analisi del sistema dà una visione complessiva. Il debug dello swarm, che sarà il punto successivo, entra invece nella diagnosi operativa dei problemi: come individuare bug, anomalie, nodi instabili, errori di configurazione, mismatch, failure di comunicazione e problemi di integrazione. In altre parole, l'analisi risponde a "che cosa è successo e quanto è stato valido"; il debug risponde a "perché è successo e come lo correggiamo nel codice, nei parametri o nell'hardware". Le due funzioni sono collegate ma diverse.

In conclusione, l'analisi del sistema è il livello della telemetria che trasforma i dati raccolti in diagnosi, metriche aggregate, classificazioni, confronti, raccomandazioni e decisioni progettuali. Essa valuta geometria, topologia, ruoli, fasi, stabilità, safety, energia, temperatura, comunicazione, recovery, qualità dei dati, modalità ibrida, scalabilità e maturità. Permette di classificare le run in modo preciso, individuare cause di fallimento, confrontare versioni, rilevare regressioni, capire trade-off e guidare i prossimi passi. Senza analisi, la telemetria sarebbe solo registrazione; con l'analisi, diventa strumento di sviluppo. Il punto successivo, dedicato al debug dello swarm, chiuderà la Sezione 13 mostrando come usare telemetria, eventi, snapshot, log, dashboard e report per individuare e correggere problemi concreti nei nodi, nei pattern, nel controller, nella simulazione e nell'integrazione hardware.

13.4 Debug dello swarm

Il debug dello swarm rappresenta il punto conclusivo della Sezione 13 e descrive il modo in cui la telemetria viene usata per individuare, interpretare e correggere problemi concreti nel comportamento dei MicroBot. Dopo aver definito monitoraggio dello stato, raccolta dati in tempo reale e analisi del sistema, diventa necessario spiegare come questi dati vengano applicati alla diagnosi operativa. Il debug non è soltanto la ricerca di un errore nel codice. In MicroBot, il debug riguarda l'intero ecosistema: nodi virtuali, nodi fisici, controller, pattern, ruoli, sincronizzazione, comunicazione, dashboard, Safety Layer, telemetria, firmware, simulazione e modalità ibrida.

In un sistema distribuito, il problema più difficile non è sempre capire che qualcosa non funziona, ma capire dove si trova la causa. Un pattern può fallire perché un nodo è offline, perché un ruolo non è stato confermato, perché un comando è scaduto, perché un edge non si è formato, perché una soglia è troppo severa, perché il nodo è in `low_energy`, perché il `thermal_lock` ha bloccato l'attuazione, perché il controller ha usato una configurazione vecchia, perché la dashboard mostra uno snapshot stale, perché il firmware si è resettato oppure perché la simulazione usa un modello incoerente. Il debug dello swarm serve proprio a separare questi livelli e a trovare la causa reale.

Il principio generale del debug MicroBot può essere espresso così:

un errore dello swarm deve essere ricostruito come sequenza di stati, comandi, eventi e decisioni.

Questo significa che non basta osservare il risultato finale. Bisogna guardare la timeline. Quale fase era attiva? Quale nodo ha cambiato stato? Quale comando è stato inviato? È arrivato? È stato accettato? È stato rifiutato? Quale metrica ha superato la soglia? Quale evento safety è stato generato? Il controller ha applicato correzione? Il fallback è stato attivato? Il nodo ha risposto? La dashboard mostrava dati aggiornati? Solo ricostruendo la sequenza si può fare debug serio.

Il primo tipo di problema riguarda i nodi non visibili o non aggiornati. Questo accade quando un MicroBot non compare nella dashboard, appare offline, rimane stale o non aggiorna il proprio stato. Nel caso di un nodo simulato, il problema può essere nel simulatore, nello snapshot, nel formato dati o nel frontend. Nel caso di un nodo fisico, il problema può essere Wi-Fi, firmware, alimentazione, heartbeat, parsing o controller. Il debug deve quindi partire dalla catena dati: il nodo

esiste nel modello? Produce telemetria? Il backend la riceve? Lo snapshot la contiene? La dashboard la legge? La visualizzazione la mostra?

Una procedura diagnostica corretta è:

verificare se il nodo esiste nella configurazione;

verificare se il simulatore o firmware lo considera attivo;

verificare se genera heartbeat;

verificare se il controller aggiorna last_seen;

verificare se lo snapshot contiene il nodo;

verificare se la dashboard riceve snapshot aggiornato;

verificare se il rendering filtra o nasconde il nodo.

Questa sequenza evita di saltare subito a conclusioni sbagliate. Se il nodo non appare, non è detto che sia offline: potrebbe essere presente nello snapshot ma nascosto da un filtro UI. Oppure potrebbe essere online nel firmware ma non raggiungere il controller. Oppure potrebbe essere escluso dalla configurazione del pattern.

Il secondo tipo di problema riguarda i ruoli incoerenti. Un nodo può ricevere un ruolo ma non confermarlo, oppure il controller può aspettarsi un ruolo diverso da quello riportato dal nodo. Questo genera role_mismatch. Nei pattern complessi, un ruolo sbagliato può causare fallimenti difficili da interpretare: un connector che resta passive_node, un bridge_node che non può attivare una connessione, un leader non sincronizzato, un terminal assegnato al nodo sbagliato. Il debug dei ruoli deve confrontare expected_role e reported_role.

La telemetria deve quindi mostrare:

expected_role;

reported_role;

role_status;

role_assigned_at;

role_confirmed_at;

last_role_command_id;

role_rejection_reason.

Se role_status = mismatch, bisogna capire se il comando assign_role non è arrivato, se è stato rifiutato, se il nodo non supporta il ruolo, se la fase è cambiata prima della conferma, se il boot_id è cambiato o se la configurazione del controller è vecchia. In un sistema MicroBot rigoroso, un ruolo non confermato non deve essere considerato valido. Il pattern deve aspettare conferma o applicare fallback.

Il terzo tipo di problema riguarda le fasi non allineate. Un nodo può trovarsi in `phase_align` mentre il controller è in `phase_attach`, oppure un comando può arrivare con `phase_id` non più valido. Questo è particolarmente pericoloso nelle azioni sincronizzate e nei pattern con attuazione. Il debug deve confrontare `expected_phase` e `reported_phase`, ma anche `timestamp`, `command_id`, `expires_at` e `sync_status`. Un nodo fuori fase non deve eseguire comandi critici.

Una diagnosi `phase_mismatch` deve controllare:

`phase_id` del controller;

`phase_id` riportato dal nodo;

tempo dell'ultima transizione di fase;

comandi pendenti;

comandi scaduti;

`sync_status`;

`last_state_sync`;

`config_epoch`.

Se il nodo non ha ricevuto `state_sync`, il problema è comunicazione o recovery incompleta. Se lo ha ricevuto ma non lo applica, il problema può essere firmware o validazione locale. Se il nodo ha `boot_id` nuovo, il controller deve trattarlo come `resync_required`. Il debug deve distinguere questi casi.

Il quarto tipo di problema riguarda i comandi rifiutati. Un comando rifiutato non è sempre un errore. Spesso è il Safety Layer che funziona correttamente. Tuttavia, troppi rifiuti indicano che il controller sta pianificando male o che il pattern non è compatibile con lo stato dei nodi. Il debug deve analizzare `reason`, `command_type`, `node_state` e `phase`. Per esempio, `magnetic_pulse` può essere rifiutato per `cooldown_active`, `thermal_lock`, `low_energy`, `role_not_authorized`, `capability_missing`, `command_expired` o `sync_lost`. Ogni motivo porta a una correzione diversa.

Un `command_rejected` deve contenere:

`command_id`;

`command_type`;

`target_node`;

`phase_id`;

`reason`;

`node_mode`;

`energy_state`;

`thermal_state`;

`sync_status`;

required_capability;

available_capabilities;

action_taken.

Se reason = cooldown_active, la soluzione è aumentare cooldown o ritardare la fase. Se reason = capability_missing, bisogna correggere role assignment o capabilities. Se reason = command_expired, bisogna correggere timing, latenza o expires_at. Se reason = role_not_authorized, bisogna verificare ruoli. Se reason = low_energy, bisogna riassegnare il nodo o ridurre carico. Il debug deve collegare ogni rejection a una decisione tecnica.

Il quinto tipo di problema riguarda le metriche geometriche. Un pattern può non raggiungere la forma desiderata. In questo caso bisogna capire se il problema è nei target, nei parametri di movimento, nella distribuzione iniziale, nei ruoli o nella topologia. Se una linea ha lateral_error alto, forse i nodi non convergono verso l'asse. Se ha spacing_error alto, forse la distanza tra nodi non viene mantenuta. Se un cerchio ha radial_error basso ma angular_distribution_error alto, i nodi stanno sul raggio ma non sono distribuiti bene. Se una griglia ha cell_assignment_error alto, i nodi non sono assegnati correttamente alle celle.

Il debug geometrico deve controllare:

target_position;

current_position;

distance_to_target;

velocity;

role;

neighbor_distance;

pattern_error per nodo;

error_vector;

phase_id.

Il pannello nodo della dashboard deve essere utile qui. Se un nodo è lontano dal target, bisogna vedere se il suo target è corretto. Un target sbagliato indica problema nel pattern generator. Un target corretto ma movimento errato indica problema nel behavior engine. Un nodo che non si muove può essere in safety, low_energy, offline o avere max_speed zero. Un nodo che oscilla può indicare damping insufficiente o forze troppo aggressive.

Il sesto tipo di problema riguarda la topologia. Un pattern può fallire perché mancano connessioni o perché si formano connessioni indesiderate. Il debug topologico deve concentrarsi su expected_edges, current_edges, missing_edges, extra_edges e critical_edges. Se un edge è mancante, bisogna capire se i nodi sono troppo lontani, se non sono nella fase corretta, se i ruoli sono incompatibili, se un nodo è in safety, se la connessione è stata tentata e fallita o se il pattern ha generato edge sbagliato.

Per ogni edge problematico, la dashboard dovrebbe poter mostrare:

edge_id;
node_a;
node_b;
expected;
current_state;
distance;
connection_radius;
edge_role_requirement;
phase_requirement;
last_attempt;
failure_reason;
criticality.

Se $distance > connection_radius$, il problema è geometrico. Se $role_requirement$ non è soddisfatto, il problema è role assignment. Se $phase_requirement$ non è soddisfatto, il problema è temporale. Se $last_attempt$ fallisce per safety, il problema è Safety Layer o pianificazione. Se $edge_id$ non esiste nel pattern ma appare come $extra_edge$, il problema è vicinato o regola di connessione troppo permissiva.

Il settimo tipo di problema riguarda la sincronizzazione. Un nodo può essere online ma non sincronizzato. Questo può causare comandi eseguiti in ritardo, phase mismatch, ack fuori tempo, edge non formati o azioni non coordinate. Il debug della sincronizzazione deve analizzare offset, jitter, $last_sync_age$, $sync_status$, command execution time e packet latency. Un nodo con $sync_degraded$ non deve essere usato per azioni temporizzate critiche.

Una diagnosi sync deve controllare:

$last_sync_time$;
 $clock_offset_ms$;
 $jitter_ms$;
 $sync_status$;
 $execute_at$;
 $actual_execution_time$;
 $execution_error_ms$;
 $command_expired_count$;
RTT;

packet_loss.

Se execution_error_ms è alto, la sincronizzazione non è sufficiente. Se last_sync_age è alto, il nodo deve essere resync. Se jitter è alto, gli slot temporali devono essere più larghi. Se i comandi scadono spesso, expires_at è troppo corto o la rete è lenta. Il debug deve trasformare questi dati in correzioni.

L'ottavo tipo di problema riguarda energia e batteria. Un nodo può smettere di partecipare perché entra in low_energy o critical_energy. Il debug energetico deve capire se il consumo è normale, se un ruolo è troppo costoso, se il pattern richiede troppi comandi high_energy, se la telemetria è troppo frequente o se la batteria è simulata/realmente bassa. Nel caso hardware, bisogna distinguere low_energy reale da stima sbagliata.

Il debug energetico deve controllare:

energy_state;

energy_source;

battery_voltage se disponibile;

high_energy_command_count;

energy_cost_by_phase;

energy_cost_by_role;

commands_blocked_by_energy;

role_reassignment_after_low_energy;

power_source;

reset o brownout events.

Se un nodo entra in low_energy sempre durante phase_attach, la fase consuma troppo. Se un nodo specifico va sempre in low_energy, forse è sovraccaricato o ha alimentazione fragile. Se energy_source = simulated, il problema può essere nel modello. Se energy_source = measured, può indicare limite reale hardware. Il debug deve mantenere questa distinzione.

Il nono tipo di problema riguarda temperatura e thermal_load. Un nodo può bloccare attuazioni perché entra in thermal_warning o thermal_lock. Il debug termico deve ricostruire quali comandi hanno aumentato thermal_load, quanto cooldown è stato applicato, se il raffreddamento è sufficiente, se i retry hanno accumulato calore e se il controller ha rispettato le soglie. In simulazione, il problema può essere nel modello termico o nei parametri; in hardware, può indicare rischio reale.

Una diagnosi termica deve controllare:

thermal_state;

thermal_source;

thermal_load;

thermal_load_history;
last_high_energy_commands;
cooldown_remaining_ms;
cooldown_policy;
thermal_events_by_phase;
commands_rejected_by_thermal;
recovery_time.

Se thermal_lock si attiva spesso, il pattern non è termicamente sostenibile. Se thermal_warning oscilla continuamente, le soglie o l'hysteresis sono sbagliate. Se cooldown non riduce thermal_load, il cooling_rate è troppo basso o il modello non è coerente. Se un comando viene accettato durante thermal_lock, c'è bug critico nel Safety Layer.

Il decimo tipo di problema riguarda recovery e reset. Un nodo che si resetta cambia boot_id e perde stato operativo. Il debug deve capire se il reset è watchdog_reset, brownout_reset, manual_reset, software_reset o unknown. Deve anche verificare se il controller ha rilevato il nuovo boot_id, se ha invalidato lo stato precedente, se ha richiesto sync e state_sync, e se ha reintegrato il nodo correttamente. Un reset ignorato è pericoloso.

La diagnosi recovery deve controllare:

old_boot_id;
new_boot_id;
reset_reason;
last_seen_before_reset;
phase_at_reset;
last_command_before_reset;
recovery_started;
safe_state_reached;
state_sync_sent;
state_sync_ack;
reintegration_decision;
recovery_result.

Se un nodo viene reintegrato senza state_sync dopo reset, c'è errore di recovery. Se reset avviene dopo comando high_energy, può essere energy_fault. Se reset avviene durante debug o parsing, può essere bug firmware. Se reset è ripetuto, il nodo deve essere marcato unstable. Il debug deve produrre questa diagnosi.

L'undicesimo tipo di problema riguarda la dashboard stessa. Una dashboard può mostrare dati sbagliati o vecchi. Questo è un problema reale, perché l'utente può prendere decisioni basate su una visualizzazione non aggiornata. Il debug deve verificare `snapshot_age_ms`, `WebSocket/polling status`, `last_update_time`, frontend filters e mapping dei campi. Se la dashboard mostra un nodo online ma il controller lo considera offline, c'è incoerenza tra frontend e runtime.

Il debug dashboard deve controllare:

backend reachable;

snapshot ricevuto;

`snapshot_age_ms`;

`node_count` nello snapshot;

`node_count` renderizzato;

filtri attivi;

mapping dei colori;

event console aggiornata;

API errors;

WebSocket status.

Spesso un problema apparentemente dello swarm è in realtà della visualizzazione. Per esempio, un nodo può non apparire perché filtrato, non perché offline. Oppure `thermal_lock` può non essere mostrato perché il frontend non mappa quel valore. La dashboard deve avere un pannello `system_status` per evitare questi dubbi.

Il dodicesimo tipo di problema riguarda lo snapshot. Se lo snapshot è incompleto, vecchio o malformato, tutta la dashboard ne risente. Il debug deve verificare `schema`, `timestamp`, `nodes array`, `edges`, `metrics`, `recent_events` e `source`. Uno snapshot senza `timestamp` non può essere valutato. Uno snapshot senza `phase_id` rende difficile interpretare pattern. Uno snapshot con `node_id` duplicati crea caos.

Validazioni snapshot importanti:

`timestamp` presente;

`step` presente;

`nodes array` valido;

`node_id` unici;

`edge node references` valide;

`metrics` coerenti;

`phase_id` presente se run attiva;

snapshot_age entro soglia;

schema_version compatibile.

Se una validazione fallisce, la dashboard deve mostrare snapshot_invalid o data_warning. Non deve procedere come se tutto fosse normale.

Il tredicesimo tipo di problema riguarda la configurazione. Molti bug derivano da configurazioni incoerenti: numero nodi insufficiente, pattern_id sbagliato, behavior_profile non compatibile, safety_policy mancante, capability non presente, soglie errate, config_epoch vecchio. Il debug deve sempre controllare config_id e config_epoch. Una configurazione sbagliata può produrre errori che sembrano runtime.

Una diagnosi config deve controllare:

config_id;

config_epoch;

pattern_id;

pattern_version;

node_count richiesto;

node_count disponibile;

required_capabilities;

available_capabilities;

behavior_profile_id;

safety_policy_id;

schema_version.

Se grid_3x3 richiede 9 nodi ma la configurazione ne fornisce 8, il problema è invalid_configuration, non comportamento. Se il pattern richiede coil_pwm ma i nodi sono LED-only, il problema è capability_missing. Il debug deve bloccare questi casi prima della run quando possibile.

Il quattordicesimo tipo di problema riguarda i dati mancanti. In un sistema reale, alcuni campi possono essere unavailable. Il debug deve distinguere tra dato assente accettabile e dato assente critico. Per esempio, position può essere unavailable per un nodo hardware LED-only; va bene se il pattern è visuale. Ma thermal_state unavailable su nodo con bobina è critico. La dashboard e il controller devono indicare data_confidence.

Il debug dei dati mancanti deve controllare:

missing_fields;

field_importance;

source;

fallback_value;

decisione conservativa;

validation_confidence.

Se mancano dati critici, la run deve essere degraded o invalidata. Se mancano dati informativi, può continuare. Questa distinzione evita blocchi inutili ma mantiene sicurezza.

Il quindicesimo tipo di problema riguarda la modalità ibrida. In hybrid mode, alcuni dati vengono dal simulatore e altri dall'hardware. Il debug deve verificare se la fusione è corretta. Un nodo può avere heartbeat hardware aggiornato ma posizione simulata stale. Oppure il gemello virtuale può avere role diverso dal nodo fisico. Oppure l'hardware può essere online ma la simulazione lo considera offline. Questi mismatch devono essere gestiti.

Una diagnosi hybrid deve controllare:

hardware_node_id;

virtual_node_id;

mapping corretto;

heartbeat_source;

position_source;

role_source;

state_merge_policy;

hardware_last_seen;

virtual_last_update;

hybrid_consistency.

Se hardware e virtuale divergono, il sistema deve decidere quale fonte ha priorità per quale campo. Per esempio, heartbeat hardware ha priorità per online_status fisico, ma posizione simulata resta source per visualizzazione. Questa logica deve essere documentata.

Il sedicesimo tipo di problema riguarda i loop di correzione. Un controller può tentare correzioni ripetute senza migliorare. Per esempio, retry_attach continuo, role_reassignment continuo, state_sync continuo, fallback che si attiva e disattiva. Questi loop possono consumare energia, creare rumore nei log e impedire stabilità. Il debug deve rilevare correction_loop o adaptation_loop.

Metriche utili:

correction_attempt_count;

same_error_repeated_count;

metric_improvement_after_correction;

retry_limit_reached;

adaptation_loop_detected;

fallback_flapping_count.

Se una correzione non migliora le metriche dopo N tentativi, il sistema deve smettere e dichiarare failure o fallback. Il debug deve mostrare questo chiaramente: “retry_attach repeated 3 times, topology_score did not improve, fallback triggered”. Questo è molto più utile di vedere solo fallimento finale.

Il diciassettesimo tipo di problema riguarda i fallback sbagliati. Un fallback può essere attivato troppo presto, troppo tardi, senza motivo valido o verso un pattern non compatibile. Il debug deve verificare fallback_reason, fallback_allowed, fallback_time, original_failure_reason e final_result. Un fallback corretto è un comportamento maturo. Un fallback casuale è un bug.

Una diagnosi fallback deve controllare:

fallback_triggered;

trigger_condition;

fallback_allowed_by_pattern;

fallback_target_pattern;

fallback_start_phase;

fallback_result;

original_goal_status;

final_classification.

Se fallback_allowed = false ma fallback viene attivato, c'è errore. Se fallback viene attivato senza failure_reason, c'è mancanza diagnostica. Se fallback produce success_clean invece di partial_success, la classificazione è sbagliata. Il debug deve correggere anche queste semantiche.

Il diciottesimo tipo di problema riguarda la classificazione finale. Il sistema può classificare una run in modo errato. Per esempio, success_clean nonostante warning safety, oppure failed_geometry quando il problema era topology, oppure partial_success senza fallback documentato. Il debug deve verificare le regole di classificazione. La classificazione è importante perché guida documentazione e decisioni. Se è sbagliata, tutto il report diventa fuorviante.

Una diagnosi classification deve controllare:

final_result;

rules_applied;

metrics_thresholds;

events_considered;

safety_events_considered;

fallback_status;

failure_reason;

validation_confidence.

Se thermal_lock_count > 0, success_clean non dovrebbe essere possibile. Se topology_score < soglia, success_clean non dovrebbe essere possibile per pattern topologico. Se validation_confidence è bassa, il risultato deve indicare incertezza. Queste regole devono essere implementate con rigore.

Il diciannovesimo tipo di problema riguarda performance e scalabilità. La simulazione o dashboard può rallentare. Questo può causare snapshot stale, frame rate basso, ritardi nella visualizzazione o perdita di eventi. Il debug performance deve misurare simulation_step_ms, render_frame_ms, snapshot_size, event_rate, memory usage e numero di nodi. Se la dashboard va lenta, non è detto che il comportamento swarm sia sbagliato; può essere un problema di rendering.

Metriche performance:

simulation_step_ms;

frontend_fps;

render_time_ms;

snapshot_size_kb;

api_latency_ms;

event_queue_length;

memory_usage;

node_count;

edge_count.

Se render_time_ms cresce troppo con gli edge, si può semplificare visualizzazione. Se snapshot_size è enorme, si può passare a delta update. Se simulation_step_ms cresce con N^2 , serve ottimizzazione del vicinato. Il debug performance prepara la scalabilità.

Il ventesimo tipo di problema riguarda la differenza tra simulazione e hardware. Un comportamento che funziona in simulazione può fallire su hardware. Il debug deve capire quale assunzione simulata non vale. Forse la latenza è maggiore. Forse il nodo non ha capacità dichiarata. Forse l'alimentazione è instabile. Forse il firmware non gestisce un comando. Forse la posizione non è misurata. Forse il Safety Layer fisico è più conservativo. Questo è normale. La simulazione deve essere aggiornata con dati reali.

Una procedura simulation-hardware debug deve confrontare:

assunzioni del modello;

dati reali misurati;

latenza reale;

limiti energetici reali;

cooldown reale;

comandi supportati dal firmware;

capabilities reali;

errori hardware osservati.

Se il comportamento hardware fallisce, non si deve forzare l'hardware ad adattarsi alla simulazione. Bisogna calibrare la simulazione e correggere il controller. Questo è un passaggio fondamentale verso maturità.

Il ventunesimo tipo di problema riguarda il firmware del nodo. Se un ESP32 non risponde, risponde male o si resetta, il debug deve scendere a livello firmware. Bisogna verificare serial log, Wi-Fi status, parsing comandi, pin mapping, watchdog, boot_id, heartbeat, modalità, safety, memoria e loop non bloccante. La telemetria deve aiutare, ma a volte serve anche log seriale.

Domande firmware:

il nodo avvia correttamente?

il node_id è corretto?

il Wi-Fi è connesso?

il controller IP/porta è corretto?

heartbeat parte?

riceve pacchetti?

parsing JSON/stringa funziona?

risponde ack?

gestisce emergency_stop?

il watchdog resetta?

le uscite sono OFF al boot?

Queste domande sono concrete e indispensabili. Il debug dello swarm deve poter arrivare fino al firmware, perché un problema distribuito può nascere da un singolo loop bloccante.

Il ventiduesimo tipo di problema riguarda il controller. Il controller può inviare comandi sbagliati, non aggiornare stati, non rispettare safety, usare snapshot vecchi, assegnare ruoli incompatibili, non gestire ack o non applicare fallback. Il debug deve quindi includere controller_logs e decision_logs. Non basta guardare i nodi. Bisogna sapere perché il controller ha preso una decisione.

Un decision_log del controller può contenere:

decision_id;

trigger_event;
current_state_summary;
rule_applied;
nodes_considered;
selected_node;
rejected_candidates;
command_generated;
expected_result;
fallback_if_failed.

Questo rende il controller spiegabile. Se assegna active_connector a un nodo low_energy, il log deve mostrare perché è successo. Forse energy_state era stale. Forse la policy non lo controllava. Forse il nodo è diventato low_energy dopo assegnazione. Il debug deve distinguere questi casi.

Il ventitreesimo tipo di problema riguarda il comportamento emergente indesiderato. In uno swarm, regole locali possono produrre effetti non previsti: oscillazioni, cluster troppo densi, nodi isolati, rotazioni, collisioni simulate, edge extra, instabilità. Il debug deve usare metriche e visualizzazione per capire quale regola causa l'effetto: coesione, separazione, target attraction, alignment, topology force, damping. Questo è un debug più vicino alla simulazione scientifica.

Una procedura può essere:

disattivare una regola alla volta;
confrontare metriche;
ridurre/aumentare peso;
osservare traiettorie;
controllare collision_count;
controllare aggregation_score;
salvare profilo migliore.

Questo metodo permette di capire il contributo delle regole. Non basta modificare parametri a caso.

Il ventiquattresimo tipo di problema riguarda i threshold. Molti stati dipendono da soglie: quando un nodo è stale, quando pattern_error è accettabile, quando thermal_warning scatta, quando fallback si attiva, quando una griglia è valida. Soglie troppo strette generano falsi errori. Soglie troppo larghe accettano comportamenti scadenti. Il debug deve analizzare threshold crossings, cioè quando e quante volte una metrica supera soglia.

Metriche utili:

threshold_cross_count;

time_above_threshold;

time_below_threshold;

margin_to_threshold;

flapping_around_threshold.

Se una metrica oscilla vicino alla soglia, serve hysteresis o smoothing. Se fallback si attiva per un superamento brevissimo, la soglia è troppo sensibile. Se safety entra troppo tardi, la soglia è troppo permissiva. Il debug delle soglie è essenziale.

Il venticinquesimo tipo di problema riguarda il logging insufficiente. A volte non si riesce a capire il problema perché manca un dato. Questo è già un risultato del debug: la telemetria non è abbastanza ricca. In quel caso, bisogna aggiungere log o eventi. Il debug deve quindi migliorare la telemetria stessa. Se non si sa perché un comando è stato rifiutato, bisogna aggiungere rejection_reason. Se non si sa perché un nodo è offline, bisogna aggiungere last_seen e heartbeat logs. Se non si sa perché fallback parte, bisogna aggiungere fallback_reason.

Una regola importante è:

ogni problema non diagnosticabile deve produrre un miglioramento dello schema telemetrico.

Questo rende il sistema sempre più osservabile. All'inizio mancheranno dati; ogni problema aiuta a capire quali dati servono.

Il ventiseiesimo tipo di problema riguarda l'eccesso di logging. Anche troppi log possono diventare un problema. Se ogni step genera migliaia di eventi, il debug diventa difficile. Serve filtrare, raggruppare e classificare. Gli eventi devono avere severity e category. I log devono distinguere info, warning, error e critical. In modalità normale, si mostrano solo eventi importanti; in debug full, si mostra tutto. Questo mantiene leggibilità.

Una struttura log utile è:

INFO: stati normali e transizioni previste;

WARNING: condizioni degradate;

ERROR: fallimenti non critici o recovery necessaria;

CRITICAL: safety, emergency, reset, lock;

DEBUG: dettagli tecnici attivabili.

Questa gerarchia rende il debug gestibile.

Il ventisettesimo tipo di problema riguarda la riproducibilità del bug. Un bug visto una volta deve poter essere riprodotto. Per farlo servono config, seed, snapshot iniziale, fault_profile e versione codice. Se non si salva nulla, il bug può sparire e tornare senza controllo. Il debug MicroBot deve quindi prevedere bug_report riproducibili.

Un bug_report utile contiene:

bug_id;

descrizione;
run_id;
config_id;
pattern_id;
seed;
snapshot iniziale;
eventi prima del problema;
risultato atteso;
risultato ottenuto;
log rilevanti;
ipotesi causa;
prossimo test.

Questa struttura trasforma il bug in lavoro tecnico ordinato. È perfetta per repo GitHub o documentazione interna.

Il ventottesimo tipo di problema riguarda la priorità dei bug. Non tutti i problemi hanno la stessa urgenza. Un colore sbagliato nella dashboard è meno grave di un comando attuativo accettato in thermal_lock. Un piccolo errore di label è meno grave di un reset ripetuto. Il debug deve classificare severità e priorità.

Una scala può essere:

P0: rischio safety o comando fisico pericoloso;
P1: perdita controllo, reset, recovery errata, dati critici falsi;
P2: pattern base non funzionante, ruoli errati, telemetria mancante critica;
P3: problema dashboard, metrica secondaria, visualizzazione incompleta;
P4: miglioramento estetico o ottimizzazione non urgente.

Questa priorità aiuta a non disperdersi. Prima safety e coerenza, poi estetica.

Il ventinovesimo tipo di problema riguarda la correzione e verifica. Dopo aver trovato la causa, bisogna correggere e poi verificare che il problema sia risolto senza introdurre regressioni. Ogni bug corretto dovrebbe avere un test. Se il bug era command_expired non gestito, si aggiunge un test. Se era thermal_lock ignorato, si aggiunge un test. Se era snapshot stale non mostrato, si aggiunge un test UI. Questo chiude il ciclo.

La sequenza corretta è:

rileva bug;

riproduci bug;
identifica causa;
correggi;
riesegui test specifico;
riesegui regression suite;
aggiorna documentazione se necessario;
chiudi bug.

Questa è disciplina software. In un progetto grande come MicroBot, è fondamentale.

Il trentesimo tipo di problema riguarda la trasformazione del debug in miglioramento progettuale. Ogni bug importante deve lasciare una lezione. Se molti bug derivano da `role_mismatch`, serve migliorare `state_sync`. Se molti derivano da `snapshot stale`, serve migliorare `data freshness`. Se molti derivano da pattern troppo complessi, serve semplificare. Se molti derivano da `safety thresholds`, serve calibrazione. Il debug non deve essere solo riparazione, ma apprendimento.

Un debug note può chiudersi con:

causa radice;
correzione applicata;
test aggiunto;
lezione progettuale;
effetto sulla roadmap.

Questa abitudine rende il progetto più maturo nel tempo.

In conclusione, il debug dello swarm è il processo attraverso cui MicroBot usa telemetria, snapshot, eventi, log, metriche, dashboard e report per individuare e correggere problemi nei nodi, nei ruoli, nei pattern, nella topologia, nella sincronizzazione, nella comunicazione, nel Safety Layer, nel firmware, nel controller, nella simulazione e nella modalità ibrida. Il debug deve ricostruire la sequenza degli eventi, distinguere causa apparente e causa reale, classificare severità, produrre bug report riproducibili e trasformare ogni errore in test o miglioramento. In un sistema distribuito, un problema può nascere in un punto e manifestarsi in un altro: per questo la telemetria deve rendere osservabile l'intera catena. Con questo punto si chiude la Sezione 13. La telemetria diventa così non solo monitoraggio, ma strumento di controllo, validazione, analisi e debug. La sezione successiva, dedicata a MicroBot OS, potrà ora descrivere il livello operativo embedded del sistema: come il firmware, le risorse, lo scheduling, la shell interna e la diagnostica possano essere organizzati in una struttura più vicina a un vero sistema operativo per i nodi MicroBot.

SEZIONE 14 — MicroBot OS

La Sezione 14 introduce il concetto di **MicroBot OS**, cioè il livello operativo embedded pensato per organizzare il comportamento interno dei nodi MicroBot in modo più strutturato rispetto a un

semplice firmware monolitico. Dopo aver definito pattern, sicurezza, simulazione e telemetria, diventa necessario descrivere come un singolo nodo possa gestire risorse, comandi, stati, task, diagnostica, comunicazione e funzioni locali attraverso una struttura ordinata. MicroBot OS non deve essere inteso, nelle prime versioni, come un sistema operativo completo paragonabile a Linux o a un OS general-purpose. Deve essere inteso come un livello software embedded progressivo, costruito sopra firmware e microcontrollore, capace di fornire astrazioni, servizi interni, scheduling, shell diagnostica e gestione dello stato del nodo.

Nel progetto MicroBot, MicroBot OS nasce dall'esigenza di evitare che il firmware del nodo diventi una massa confusa di funzioni: una parte per i LED, una per il Wi-Fi, una per la telemetria, una per i comandi, una per la safety, una per la batteria, una per il watchdog, una per la diagnostica. Finché il prototipo è molto semplice, un singolo file firmware può essere sufficiente. Ma appena il nodo comincia a gestire heartbeat, ruoli, stati, comandi, timer, cooldown, errori, telemetria, recovery e modalità operative, serve una struttura più modulare. MicroBot OS è proprio questa struttura: un insieme di servizi interni che permettono al nodo di comportarsi come una piccola unità computazionale autonoma.

L'obiettivo di MicroBot OS è dare al nodo una logica interna ordinata. Un MicroBot non deve soltanto ricevere comandi dal controller; deve interpretarli, validarli, inserirli nel proprio stato, verificarne la sicurezza, schedarne l'esecuzione, produrre telemetria, registrare errori e reagire a condizioni anomale. Questo richiede un piccolo "sistema operativo" nel senso architetturale del termine: un livello che governa le risorse locali e coordina i moduli del nodo.

Il principio generale può essere espresso così:

MicroBot OS è il livello che trasforma un microcontrollore programmato in un nodo operativo, diagnosticabile, sicuro e integrabile nello swarm.

Questa frase è importante perché distingue hardware e sistema. Un ESP32 con qualche LED è un microcontrollore. Un ESP32 che possiede identità, stato, heartbeat, comandi, ruoli, safety, telemetria, watchdog, diagnostica e recovery diventa un nodo MicroBot. MicroBot OS è il software che rende possibile questa trasformazione.

MicroBot OS deve essere progettato in modo incrementale. Nella prima versione può essere soltanto una struttura firmware con loop principale, stato del nodo, comandi seriali o Wi-Fi, heartbeat e LED di stato. In una versione successiva può aggiungere command dispatcher, task scheduler, telemetry service, safety manager e diagnostic shell. In una versione più avanzata può supportare moduli caricabili, profili di configurazione, gestione più raffinata delle risorse e interfacce astratte per sensori e attuatori. L'importante è non confondere la visione finale con la versione iniziale. MicroBot OS deve crescere insieme all'hardware.

La Sezione 14 segue i punti dell'indice: sistema operativo embedded, evoluzione delle versioni, gestione delle risorse, astrazione hardware, scheduling, shell interna e diagnostica. Questi punti definiscono il ruolo di MicroBot OS come infrastruttura locale dei nodi. Le sezioni precedenti hanno descritto cosa fa lo swarm; questa sezione descrive come un singolo nodo può essere organizzato internamente per partecipare allo swarm in modo affidabile.

14.1 Sistema operativo embedded

Il sistema operativo embedded di MicroBot rappresenta il livello software locale che organizza il comportamento del nodo al di sopra dell'hardware e al di sotto del controller globale. In termini

pratici, esso è il firmware strutturato del MicroBot, progettato però con una logica più modulare rispetto a un semplice programma lineare. Non si tratta necessariamente di un OS complesso con filesystem, processi indipendenti e memoria virtuale. Si tratta piuttosto di un runtime embedded leggero, capace di coordinare moduli, gestire stati, eseguire task periodici, rispondere a comandi, controllare uscite, inviare telemetria e mantenere condizioni di sicurezza.

Un sistema operativo embedded, nel contesto MicroBot, deve rispondere a una domanda precisa: come può un nodo piccolo, con risorse limitate, comportarsi in modo ordinato dentro uno swarm distribuito? La risposta non può essere soltanto “scrivendo codice”. Serve una struttura. Ogni funzione deve avere un posto. La comunicazione deve essere separata dalla safety. La telemetria deve essere separata dalla gestione LED. I timer devono essere gestiti senza bloccare il loop. I comandi devono passare da un dispatcher. Gli stati devono essere centralizzati. La diagnostica deve poter interrogare il nodo. Il watchdog deve verificare che i servizi critici continuino a girare. Questa organizzazione è ciò che rende il firmware più vicino a un sistema operativo.

MicroBot OS deve quindi introdurre una divisione interna in moduli. Un modulo è una parte del firmware con responsabilità definita. Per esempio, `identity_manager` gestisce `node_id`, `boot_id` e versioni. `state_manager` gestisce `mode`, `role`, `phase`, `safety_state`, `energy_state` e `thermal_state`. `command_manager` riceve e valida comandi. `telemetry_manager` prepara heartbeat e messaggi di stato. `safety_manager` controlla limiti, `emergency_stop`, cooldown e blocchi. `output_manager` controlla LED, PWM e attuatori. `scheduler` gestisce task periodici. `diagnostics_manager` espone informazioni interne. Questa separazione rende il firmware leggibile, testabile e modificabile.

Una possibile architettura interna può essere:

`identity_manager;`

`state_manager;`

`command_manager;`

`telemetry_manager;`

`safety_manager;`

`output_manager;`

`scheduler;`

`diagnostics_manager;`

`communication_adapter;`

`hardware_abstraction_layer.`

Questa lista non deve essere interpretata come obbligo immediato di creare dieci file separati già nella prima versione. È una mappa architetturale. Nelle prime versioni, alcuni moduli possono essere funzioni nello stesso file. Nelle versioni successive, possono diventare file distinti o componenti più puliti. L'importante è che la responsabilità concettuale sia chiara.

Il cuore del sistema operativo embedded è il **main loop non bloccante**. In un firmware semplice, spesso si usano delay lunghi per lampeggiare LED o aspettare eventi. In un nodo MicroBot, questo approccio diventa pericoloso. Il nodo deve controllare continuamente comandi, safety, timer, heartbeat, output e watchdog. Se un delay blocca il loop, il nodo può non ricevere `emergency_stop`,

può non spegnere una bobina al tempo giusto, può non aggiornare heartbeat o può perdere sincronizzazione. Per questo, MicroBot OS deve basarsi su una logica non bloccante.

Il main loop deve funzionare come una sequenza rapida di controlli:

aggiornare tempo locale;

leggere eventuali messaggi in ingresso;

processare comandi ricevuti;

aggiornare stato e safety;

aggiornare timer di output;

eseguire task schedulati;

preparare telemetria quando necessario;

alimentare watchdog solo se i controlli critici sono stati eseguiti;

ritornare rapidamente all'inizio del loop.

Questa struttura permette al nodo di restare reattivo. Nessuna funzione ordinaria deve bloccare il sistema per tempi lunghi. Anche un LED blink deve essere gestito tramite timer, non tramite delay bloccante. Questa è una regola importante perché prepara il passaggio da prototipo LED a nodo con attuatori reali.

Il sistema operativo embedded deve avere uno **state manager** centrale. Lo stato del nodo non deve essere disperso in variabili sparse senza controllo. Deve esistere una struttura logica che rappresenta il nodo: node_id, boot_id, mode, role, phase_id, pattern_run_id, safety_state, energy_state, thermal_state, sync_status, capabilities, blocked_capabilities, last_error, uptime. Questa struttura è la fonte locale della verità. I moduli la leggono e la aggiornano secondo regole precise.

Una struttura di stato può contenere:

node_id;

boot_id;

firmware_version;

hardware_version;

mode;

role;

group_id;

pattern_run_id;

phase_id;

safety_state;

energy_state;

thermal_state;

sync_status;

capabilities;

blocked_capabilities;

last_command_id;

last_error_code;

uptime_ms.

Questo stato deve essere coerente con la telemetria della Sezione 13. Il nodo non deve inventare campi diversi da quelli usati dal controller. Se il firmware usa mode = safe_idle, anche la dashboard deve capirlo. Se il firmware usa thermal_lock, il controller deve interpretarlo. Questa coerenza tra MicroBot OS e telemetria è fondamentale.

Il sistema operativo embedded deve gestire l'identità del nodo. Ogni MicroBot deve sapere chi è. Il node_id identifica il nodo fisico o virtuale. Il boot_id identifica la sessione corrente. Il firmware_version e hardware_version indicano compatibilità. Il protocol_version indica se il nodo può comunicare con il controller attuale. Senza identità, il controller non può distinguere i nodi, assegnare ruoli, rilevare reset o costruire log.

L'identity_manager deve quindi fornire:

node_id stabile;

boot_id generato o aggiornato a ogni avvio;

firmware_version;

hardware_version;

protocol_version;

capability_profile_id.

Il boot_id è particolarmente importante. Se il nodo si resetta, il boot_id cambia. Il controller capisce che la sessione precedente è finita e che il nodo richiede recovery. Questo collega MicroBot OS alla Sezione 11 sul watchdog e al ripristino automatico.

Il sistema operativo embedded deve avere un **command manager**. I comandi in ingresso non devono essere eseguiti direttamente appena arrivano. Devono essere ricevuti, parsati, validati, classificati e inoltrati al modulo corretto. Il command manager deve controllare command_id, command_type, target, phase_id, expires_at, capabilities richieste e stato del nodo. Solo dopo la validazione il comando può essere applicato. Questo evita esecuzioni pericolose o fuori contesto.

Una pipeline comando può essere:

ricezione messaggio;

parsing;

verifica schema;

verifica target node_id;

verifica duplicati command_id;

verifica scadenza;

verifica mode e safety;

verifica capabilities;

verifica role e phase se necessario;

inoltro al modulo corretto;

generazione ack o rejection.

Questa pipeline è fondamentale. Un comando non è un semplice testo da eseguire. È una richiesta che deve rispettare stato, sicurezza e contesto. MicroBot OS deve applicare questa disciplina localmente.

Il command manager deve distinguere comandi di basso rischio e comandi critici. Un request_status è sicuro. Un set_role è logico ma importante. Un set_led può essere relativamente sicuro. Un magnetic_pulse o PWM su carico reale è critico. Emergency_stop è critico ma sempre prioritario. Ogni classe di comando deve avere regole diverse.

Una classificazione può essere:

system_command: ping, request_status, reboot se autorizzato;

state_command: set_mode, assign_role, state_sync;

visual_command: set_led, blink_pattern;

telemetry_command: set_telemetry_level, request_debug;

actuation_command: pwm_output, magnetic_pulse;

safety_command: emergency_stop, safe_idle_all, clear_safety.

I safety_command devono avere priorità massima. Gli actuation_command devono essere filtrati dal safety_manager. I visual_command possono essere più permissivi ma comunque controllati in low_energy. Questa classificazione rende il firmware più robusto.

Il sistema operativo embedded deve avere un **safety manager** locale. Questo modulo applica le regole della Sezione 11. Prima di attivare LED intensi, PWM, bobine o qualsiasi uscita critica, il safety manager verifica energia, temperatura, cooldown, stato, ruolo, fase, watchdog, limiti e modalità. Se qualcosa non è compatibile, rifiuta il comando. Il safety manager deve poter spegnere immediatamente uscite critiche in caso di emergency_stop, thermal_lock, critical_energy o controller_lost.

Il safety manager deve controllare:

safety_state;

energy_state;

thermal_state;

cooldown;

output timers;

command limits;

emergency_stop;

controller_lost;

watchdog health;

blocked_capabilities.

Questo modulo è uno dei più importanti di MicroBot OS. Senza safety manager, il firmware sarebbe solo esecutore di comandi. Con safety manager, il nodo diventa autonomamente protetto.

Il sistema operativo embedded deve avere un **output manager**. Le uscite fisiche non devono essere controllate direttamente da qualsiasi parte del codice. LED, PWM, bobine, buzzer o altri attuatori devono passare da un modulo dedicato. Questo modulo conosce pin, modalità, timer, stato corrente, durata residua, duty cycle, cooldown e stato di sicurezza. L'output manager permette di centralizzare lo spegnimento delle uscite in caso di safety.

Un output manager può gestire:

LED di stato;

LED di ruolo;

PWM channel;

bobina o carico simulato;

timer di spegnimento;

cooldown;

output_state;

safe_off_all.

La funzione safe_off_all è fondamentale. Deve poter spegnere tutte le uscite critiche in modo immediato. Deve essere chiamabile da safety_manager, watchdog recovery, emergency_stop, critical_energy, thermal_lock e boot.

Il sistema operativo embedded deve avere un **telemetry manager**. Questo modulo prepara e invia heartbeat, status, eventi e diagnostica. Non deve essere ogni funzione del firmware a costruire messaggi casuali. Il telemetry manager deve usare lo schema definito nella Sezione 13. Deve sapere

quando inviare heartbeat, quando inviare eventi immediati, quando ridurre telemetria in low_energy e quando aumentarla in diagnostic_mode.

Il telemetry manager può gestire:

heartbeat periodico;

eventi su cambio stato;

status completo on request;

diagnostic report;

error report;

telemetry_level;

rate limiting;

payload compatto per hardware.

Questo modulo è essenziale per rendere il nodo osservabile. Un nodo che funziona ma non comunica bene è difficile da integrare nello swarm.

Il sistema operativo embedded deve avere un **scheduler** leggero. Lo scheduler non deve necessariamente essere un sistema preemptive complesso. Può essere un semplice gestore di task periodici basato su timestamp. Ogni task ha intervallo, ultima esecuzione, priorità e funzione. Per esempio: heartbeat ogni 1000 ms, safety_check ogni 50 ms, output_update ogni 10 ms, telemetry_debug ogni 500 ms, sync_check ogni 1000 ms. Il loop principale richiama lo scheduler, che esegue i task scaduti senza bloccare.

Una struttura task può contenere:

task_id;

interval_ms;

last_run_ms;

priority;

enabled;

function;

last_result;

deadline_missed_count.

Lo scheduler deve essere semplice ma disciplinato. Se un task impiega troppo, può disturbare gli altri. Se un task critico non viene eseguito, il watchdog logico deve rilevarlo. Questo collega scheduling e watchdog.

Il sistema operativo embedded deve avere una **hardware abstraction layer**, cioè un livello di astrazione hardware. Il codice alto non dovrebbe sapere direttamente che il LED è su GPIO 23 o

che PWM usa un certo canale. Dovrebbe chiamare funzioni come `set_status_led`, `set_role_led`, `set_pwm_output`, `coil_off`, `read_battery_voltage`. Questo permette di cambiare hardware senza riscrivere tutta la logica. Se un prototipo usa LED su certi pin e un altro su pin diversi, basta cambiare la configurazione hardware.

Una HAL può esporre:

`hal_init;`

`hal_set_led;`

`hal_set_pwm;`

`hal_stop_pwm;`

`hal_read_battery;`

`hal_read_temperature;`

`hal_get_uptime;`

`hal_safe_outputs_off;`

`hal_get_reset_reason.`

Questa astrazione è molto importante per la scalabilità hardware. MicroBot non resterà sempre identico. Versioni diverse avranno componenti diversi. La HAL permette al sistema operativo embedded di restare coerente.

Il sistema operativo embedded deve supportare una **diagnostic shell**, anche minima. Nelle prime versioni può essere una shell seriale tramite USB. L'utente può inviare comandi come `status`, `help`, `ping`, `led on`, `led off`, `role`, `mode`, `safety`, `telemetry`, `reset counters`. In versioni più avanzate, la shell può essere accessibile via rete o dashboard. La shell serve a ispezionare e testare il nodo senza dover sempre passare dal controller completo.

Una shell minima può supportare:

`help;`

`status;`

`identity;`

`mode;`

`role;`

`safety;`

`energy;`

`thermal;`

`capabilities;`

```
led_test;  
  
heartbeat_test;  
  
clear_error;  
  
safe_idle.
```

La shell interna è molto utile in fase di sviluppo. Permette di capire se il nodo funziona anche isolato. Se un ESP32 non risponde al controller, si può collegare seriale e interrogare lo stato. Questo riduce molto il tempo di debug.

Il sistema operativo embedded deve gestire errori in modo strutturato. Non basta stampare “error”. Ogni errore deve avere codice, categoria, severità e motivo. Per esempio: `ERR_COMMAND_EXPIRED`, `ERR_CAPABILITY_MISSING`, `ERR_THERMAL_LOCK`, `ERR_LOW_ENERGY`, `ERR_ROLE_NOT_AUTHORIZED`, `ERR_SYNC_LOST`, `ERR_INVALID_PAYLOAD`. Questi errori devono essere usati sia nel firmware sia nella telemetria.

Una struttura errore può includere:

```
error_code;  
  
error_category;  
  
severity;  
  
related_command_id;  
  
timestamp;  
  
current_mode;  
  
recovery_action;  
  
human_message opzionale.
```

Questo rende gli errori interpretabili dal controller e dalla dashboard. Un errore strutturato è molto più utile di una stringa casuale.

Il sistema operativo embedded deve gestire modalità operative. Le modalità principali possono essere `booting`, `idle`, `active`, `diagnostic`, `safe_idle`, `safety`, `recovery`, `low_power`. Ogni modalità abilita o disabilita funzioni. In `idle`, il nodo è disponibile ma non partecipa a pattern. In `active`, partecipa a una run. In `diagnostic`, permette test controllati. In `safe_idle`, uscite critiche sono spente. In `safety`, comandi attuativi sono bloccati. In `recovery`, il nodo sta rientrando dopo errore. In `low_power`, riduce attività.

Una macchina a stati può essere:

```
booting → idle;  
  
idle → active;  
  
idle → diagnostic;
```

active → safe_idle;

active → safety;

active → recovery;

recovery → idle;

safety → safe_idle dopo clear autorizzato;

low_power → idle se energia torna sufficiente.

Queste transizioni devono essere controllate. Non ogni modalità può passare direttamente a ogni altra. Per esempio, safety non deve passare direttamente ad active. Deve passare da clear_safety e safe_idle. Questo è importante per sicurezza.

Il sistema operativo embedded deve gestire configurazione locale. Alcuni parametri devono essere definiti: node_id, pin LED, intervalli heartbeat, limiti PWM, soglie energy, soglie thermal, timeout controller_lost, telemetry_level, capabilities. In una prima versione, questi parametri possono essere compilati nel firmware. In versioni successive, possono essere caricati da file, memoria non volatile o comando di configurazione. L'importante è che esista una config chiara.

Una configurazione locale può contenere:

node_id;

hardware_profile;

pin_map;

heartbeat_interval_ms;

controller_lost_timeout_ms;

max_pwm_duty;

max_pulse_duration_ms;

cooldown_ms;

telemetry_level_default;

capability_profile;

safety_policy_profile.

Questa configurazione deve essere coerente con il controller. Se il controller pensa che il nodo abbia coil_pwm ma la configurazione locale dice di no, nasce un mismatch. Per questo capabilities e config_epoch devono essere scambiati.

Il sistema operativo embedded deve essere compatibile con diverse versioni hardware. Un nodo LED-only non deve avere le stesse capabilities di un nodo con bobina. Un nodo simulato non deve avere lo stesso profilo di un nodo fisico. Un nodo con sensore temperatura avrà temperature_source

= measured; uno senza sensore userà estimated o unavailable. MicroBot OS deve supportare profili hardware. Questa è una parte essenziale della modularità.

Esempi di profili:

LED_NODE_V0;

PWM_NODE_V0;

COIL_NODE_EXPERIMENTAL;

SENSOR_NODE_V0;

HYBRID_NODE_V0;

SIMULATED_NODE_PROFILE.

Ogni profilo abilita funzioni diverse. Il controller deve interrogare il profilo prima di assegnare ruoli o comandi.

Il sistema operativo embedded deve avere un concetto di servizi. Un servizio è una funzione interna persistente del nodo: telemetry_service, safety_service, command_service, led_service, sync_service, diagnostics_service. Ogni servizio può avere stato, intervallo e health. Questo permette al watchdog logico di verificare se i servizi critici funzionano.

Una tabella servizi può contenere:

service_id;

enabled;

last_update_ms;

health_status;

error_count;

criticality.

Servizi critici come safety_service e output_service devono essere sempre aggiornati. Servizi non critici come debug_shell possono essere disabilitati in low_power. Questa gestione rende MicroBot OS più vicino a un vero runtime.

Il sistema operativo embedded deve essere progettato per risorse limitate. Un microcontrollore ha memoria, CPU e rete limitate. Quindi MicroBot OS deve restare leggero. Non deve usare strutture troppo pesanti, log enormi o parsing complesso se non necessario. Le funzioni devono essere essenziali, compatte e prevedibili. La modularità non deve diventare spreco. Un buon MicroBot OS embedded deve essere chiaro ma piccolo.

Principi di leggerezza:

payload compatti;

task brevi;

nessun blocco lungo;

buffer limitati;

eventi essenziali;

diagnostica estesa solo su richiesta;

telemetria adattiva;

config semplice.

Questo è importante soprattutto per ESP32 e nodi futuri più piccoli.

Il sistema operativo embedded deve essere testabile isolatamente. Un nodo deve poter essere acceso e interrogato senza swarm completo. La shell deve mostrare status. Il LED deve indicare boot. Heartbeat deve funzionare. Emergency_stop deve spegnere uscite. Command rejection deve funzionare. Questo permette di testare ogni nodo prima di inserirlo nello swarm.

Test base del nodo:

boot_test;

identity_test;

heartbeat_test;

status_command_test;

role_assignment_test;

led_output_test;

safe_idle_test;

emergency_stop_test;

watchdog_test;

telemetry_test.

Questi test fanno parte della maturità di MicroBot OS. Un nodo che non passa test base non deve entrare nello swarm.

Il sistema operativo embedded deve supportare logging locale minimo. Non sempre il nodo può salvare molto, ma può mantenere last_error, reset_count, last_command_id, uptime, last_reset_reason e qualche contatore. Questi dati sono utili per diagnostica. In futuro, se la memoria lo permette, si può avere un ring buffer di eventi recenti. In una prima versione, anche pochi campi bastano.

Un local_diagnostics_state può contenere:

last_error_code;

last_command_id;

last_reset_reason;

reset_count;

command_count;

rejected_command_count;

uptime_ms;

last_heartbeat_sent;

controller_lost_count.

Questi dati possono essere restituiti da request_status o shell status. Sono molto utili quando qualcosa non funziona.

Il sistema operativo embedded deve essere integrato con il watchdog. Il watchdog non deve essere alimentato casualmente. Deve essere alimentato solo se i task critici sono stati eseguiti: safety_check, output_update, command_processing e main_loop health. MicroBot OS deve quindi avere una logica watchdog-aware. Se un task critico non viene aggiornato, il nodo deve entrare in safe_idle o lasciare intervenire il watchdog.

Una logica può essere:

safety_service ok;

output_service ok;

command_service ok;

main_loop responsive;

se tutti ok, feed watchdog;

altrimenti safe_off_all e non feed watchdog.

Questa struttura rende il watchdog parte del sistema operativo embedded, non un dettaglio esterno.

Il sistema operativo embedded deve prevedere una separazione tra funzioni normali e funzioni critiche. Funzioni critiche sono safety, output critical, emergency_stop, watchdog, controller_lost. Funzioni normali sono LED decorativi, debug esteso, telemetria completa, effetti visuali. Se il nodo è sotto carico o in low_energy, le funzioni normali possono essere ridotte, ma le funzioni critiche devono restare attive. Questa priorità è fondamentale.

Una gerarchia può essere:

critical: emergency_stop, safety, output_off, watchdog;

high: heartbeat minimo, command reception, state update;

normal: role LED, telemetry normal, sync;

low: debug esteso, effetti LED, verbose logs.

Questa gerarchia guida lo scheduler e la protezione energetica.

Il sistema operativo embedded deve gestire comunicazione con adattatori. Oggi può essere seriale o Wi-Fi. Domani potrebbe essere BLE, ESP-NOW, MQTT, UDP, WebSocket o altro. Il livello alto non dovrebbe dipendere direttamente dal mezzo. Un `communication_adapter` può fornire `send_message`, `receive_message`, `is_connected`, `last_seen_controller`. In questo modo si può cambiare protocollo senza riscrivere `command_manager` e `telemetry_manager`.

Un `communication_adapter` può avere:

```
comm_init;  
  
comm_receive;  
  
comm_send;  
  
comm_available;  
  
comm_status;  
  
comm_last_error.
```

Questo è un punto architetturale importante. MicroBot deve restare estensibile.

Il sistema operativo embedded deve supportare modalità simulata. Anche su firmware o ambiente software, si può avere un MicroBot OS simulato che usa gli stessi stati ma senza hardware. Questo permette test unitari e integrazione con il simulatore. In pratica, si può creare un `simulated_os_node` che implementa `command_manager`, `state_manager` e `safety_manager` ma non usa GPIO reali. Questo aumenta la coerenza tra simulazione e hardware.

Una modalità `simulated_os` può avere:

```
HAL virtuale;  
  
telemetria virtuale;  
  
safety simulata;  
  
comandi reali del controller;  
  
output fittizi;  
  
log completo.
```

Questa modalità è molto utile per testare il controller contro nodi OS-like prima dell'hardware.

Il sistema operativo embedded deve essere pensato anche come base per MicroBot OS futuro più avanzato. Nelle tue versioni precedenti di MicroBot OS compaiono concetti come schermate, shell, diagnostica, framebuffer, stato del sistema, kernel o runtime. In questa documentazione, bisogna inserirli in modo coerente: le prime versioni non sono un OS completo, ma esperimenti e prototipi di interfaccia operativa. L'evoluzione può portare da firmware modulare a runtime embedded con shell e diagnostica più ricca. Questo sarà approfondito nel punto 14.2.

Una frase corretta è:

MicroBot OS nasce come firmware strutturato e può evolvere progressivamente verso un runtime embedded più completo, con servizi interni, shell diagnostica e astrazione delle risorse.

Questa formulazione è rigorosa perché non esagera lo stato attuale, ma mantiene la visione.

Il sistema operativo embedded deve essere documentato come architettura interna del nodo. La documentazione deve indicare moduli, stati, comandi, task, intervalli, errori, capabilities e sicurezza. Questo permette a chi legge di capire come un nodo funziona dentro. Senza questa documentazione, il firmware resta una scatola nera. Con essa, il nodo diventa parte spiegabile dell'ecosistema.

Un documento `microbot_os_architecture.md` potrebbe contenere:

scopo;

moduli principali;

state machine;

command pipeline;

scheduler;

safety manager;

telemetry manager;

HAL;

diagnostic shell;

watchdog integration;

roadmap.

Questa documentazione sarebbe molto importante in una repo tecnica.

Il sistema operativo embedded deve anche avere una relazione chiara con il controller centrale. Il controller globale decide pattern, ruoli e coordinazione. MicroBot OS esegue localmente, valida, protegge e comunica. Il controller non deve assumere controllo totale del nodo ignorando la safety locale. Il nodo non deve comportarsi come sistema indipendente fuori dallo swarm. Il rapporto corretto è cooperativo: controller globale per pianificazione, OS locale per esecuzione sicura.

Una sintesi è:

controller = coordinazione globale;

MicroBot OS = autonomia locale controllata;

firmware/HAL = accesso fisico all'hardware;

dashboard = osservazione e interazione;

telemetria = ponte informativo.

Questa divisione è essenziale per l'architettura.

Il sistema operativo embedded deve supportare un concetto di “local autonomy bounded”, cioè autonomia locale limitata. Il nodo può rifiutare comandi non sicuri, spegnere uscite, entrare in `safe_idle`, inviare eventi, mantenere watchdog e gestire cooldown. Tuttavia, non decide da solo pattern globali complessi. Questa autonomia limitata è perfetta per MicroBot: ogni nodo protegge sé stesso, ma lo swarm resta coordinato.

Esempi di autonomia locale:

rifiutare comando scaduto;

bloccare attuazione in `thermal_lock`;

entrare in `safe_idle` se `controller_lost`;

spegnere output dopo `duration_ms`;

inviare heartbeat;

segnalare `recovery_required`.

Esempi di decisioni globali del controller:

scegliere pattern;

assegnare ruoli;

pianificare slot;

attivare fallback globale;

validare run;

generare report.

Questa separazione evita sia centralizzazione fragile sia autonomia caotica.

Il sistema operativo embedded deve essere compatibile con gli stati della simulazione. Se il simulatore usa `mode`, `role`, `safety_state`, `energy_state` e `thermal_state`, il firmware deve usare concetti analoghi. Questo permette di passare da simulazione a hardware senza riscrivere tutto il linguaggio. La simulazione diventa banco di prova del MicroBot OS. Il nodo fisico diventa implementazione parziale o completa degli stessi stati.

Questa coerenza è molto importante:

stesso schema di stato;

stessi `command_type`;

stessi `error_code`;

stessi `safety_state`;

stessi `event_type`;

stesso concetto di boot_id e capabilities.

Così il controller può parlare con nodi simulati, fisici e ibridi usando logica simile.

Il sistema operativo embedded deve avere una roadmap realistica. Nella prima versione, basta boot, ID, heartbeat, LED e status seriale. Poi comandi. Poi ruoli. Poi telemetria. Poi safety. Poi scheduler. Poi shell. Poi watchdog integrato. Poi astrazione hardware. Poi recovery. Questa progressione rende il progetto gestibile.

Una roadmap possibile è:

v0.1: boot, node_id, LED status, serial status;

v0.2: heartbeat e comandi base;

v0.3: mode e role management;

v0.4: telemetry manager;

v0.5: command dispatcher strutturato;

v0.6: safety manager base;

v0.7: scheduler non bloccante;

v0.8: diagnostic shell;

v0.9: watchdog integration;

v1.0: HAL e profili hardware;

v1.1: recovery e state_sync;

v1.2: supporto nodi ibridi e test hardware-safe.

Questa roadmap si collega direttamente al punto successivo sull'evoluzione delle versioni.

In conclusione, il sistema operativo embedded di MicroBot è il livello software locale che organizza il nodo come unità operativa dello swarm. Non è necessariamente un OS general-purpose, ma un runtime embedded modulare capace di gestire identità, stato, comandi, safety, output, telemetria, scheduling, diagnostica, watchdog, comunicazione e astrazione hardware. Esso trasforma il firmware da programma monolitico a struttura ordinata, più sicura e più estensibile. MicroBot OS permette al nodo di ricevere comandi senza eseguirli ciecamente, di proteggere sé stesso, di comunicare il proprio stato e di partecipare al sistema distribuito in modo coerente. Il punto successivo, dedicato all'evoluzione delle versioni, descriverà come MicroBot OS possa crescere progressivamente da un firmware LED/ESP32 semplice fino a un runtime embedded più completo, con shell interna, diagnostica, servizi, astrazione hardware e integrazione sempre più stretta con simulazione, telemetria e controller centrale.

14.2 Evoluzione delle versioni

L'evoluzione delle versioni di MicroBot OS rappresenta il percorso attraverso cui il software interno del nodo passa da un firmware semplice, costruito per controllare pochi componenti, a un runtime embedded più strutturato, modulare, diagnosticabile e integrato con il resto dell'ecosistema

MicroBot. Questo punto è importante perché evita una confusione frequente: MicroBot OS non nasce immediatamente come sistema operativo completo. Nasce come firmware ordinato, poi cresce per livelli. Ogni versione aggiunge capacità precise, senza cancellare la versione precedente, ma costruendo sopra di essa.

Nel progetto MicroBot, questa evoluzione deve essere letta come una roadmap tecnica incrementale. All'inizio il nodo può essere soltanto un ESP32 con LED, heartbeat e comandi base. In seguito può aggiungere ruoli, telemetria, safety, scheduler, shell diagnostica, watchdog, astrazione hardware e gestione più avanzata delle risorse. Questa progressione è coerente con tutto il metodo MicroBot: prima simulare, poi visualizzare, poi misurare, poi portare una funzione su hardware in modo sicuro. MicroBot OS deve seguire lo stesso principio. Non deve provare a diventare subito complesso; deve diventare sempre più affidabile.

La prima fase può essere definita **MicroBot OS v0.1 — Boot and Identity Layer**. In questa versione, l'obiettivo principale è far sì che il nodo abbia un'identità riconoscibile e uno stato minimo osservabile. Il nodo si accende, inizializza i pin, imposta le uscite in stato sicuro, mostra un LED di boot, espone un `node_id`, genera o dichiara un `boot_id`, stampa informazioni base sulla seriale e permette di capire se il firmware è partito correttamente. Questa versione non deve ancora controllare pattern complessi. Deve semplicemente rendere il nodo riconoscibile.

In v0.1, le funzioni principali sono:

boot controllato;

`node_id` statico o configurato;

`boot_id` generato all'avvio;

LED di stato base;

uscite critiche impostate su OFF;

messaggio seriale di avvio;

versione firmware;

versione hardware;

stato idle iniziale.

Questa versione è già importante perché introduce un concetto fondamentale: ogni nodo MicroBot deve essere identificabile. Un microcontrollore generico diventa un nodo MicroBot quando possiede identità, stato e comportamento iniziale prevedibile. Anche se il nodo non comunica ancora via rete, v0.1 permette di verificarne l'avvio e la configurazione.

La seconda fase può essere definita **MicroBot OS v0.2 — Heartbeat and Basic Communication**. In questa versione, il nodo inizia a comunicare periodicamente con il controller o con un ambiente di debug. L'heartbeat indica che il nodo è vivo. Può includere `node_id`, `boot_id`, `uptime`, `mode`, `firmware_version` e stato essenziale. Se il nodo è collegato via seriale, l'heartbeat può essere stampato sulla seriale. Se è collegato via Wi-Fi, può essere inviato al controller. L'obiettivo non è ancora eseguire pattern, ma rendere il nodo visibile nel sistema.

In v0.2, le funzioni principali sono:

heartbeat periodico;
uptime_ms;
mode = idle;
invio o stampa dello stato;
ricezione di ping;
risposta pong o ack semplice;
indicatore LED per online;
controller_lost iniziale, se la comunicazione è prevista;
telemetria minima.

Questa versione collega MicroBot OS alla Sezione 13 sulla telemetria. Il nodo non è più solo acceso: è osservabile. Il controller può sapere che esiste, che è online e che non si è resettato. Il boot_id permette già di rilevare eventuali riavvii. Questa base è essenziale per qualsiasi swarm.

La terza fase può essere definita **MicroBot OS v0.3 — Command Interface**. In questa versione, il nodo non si limita a inviare heartbeat, ma riceve comandi. I comandi possono essere molto semplici: ping, request_status, set_led, blink, set_mode, safe_idle. L'obiettivo è introdurre una pipeline minima di comando: ricezione, parsing, validazione, esecuzione, risposta. Anche se i comandi sono semplici, la struttura deve già essere corretta. Il nodo non deve eseguire stringhe casuali; deve riconoscere command_type e command_id.

In v0.3, le funzioni principali sono:

command_id;
command_type;
parsing dei comandi;
ack positivo;
ack negativo o rejection;
request_status;
set_led semplice;
set_mode limitato;
safe_idle;
last_command_id;
last_error_code.

Questa fase è molto importante perché prepara il command_manager. Anche un comando LED deve passare da una logica ordinata. Se il comando è malformato, il nodo risponde invalid_command. Se

il comando è duplicato, può ignorarlo o segnalarlo. Se il comando è sconosciuto, risponde `unsupported_command`. Questa disciplina è ciò che distingue MicroBot OS da un firmware improvvisato.

La quarta fase può essere definita **MicroBot OS v0.4 — Mode and Role Management**. In questa versione, il nodo introduce modalità operative e ruoli. Il nodo può essere `idle`, `active`, `diagnostic`, `safe_idle` o `safety`. Può ricevere un ruolo come `leader`, `connector`, `terminal`, `passive_node` o `diagnostic_node`. Il ruolo viene confermato e riportato nella telemetria. Questa versione permette al controller di iniziare a organizzare nodi fisici o simulati secondo pattern logici, anche se l'hardware esegue solo LED o stato visuale.

In v0.4, le funzioni principali sono:

`mode management`;

`role assignment`;

`role confirmation`;

`role rejection` se non supportato;

`reported_role`;

`expected_role` opzionale;

`role LED pattern`;

`phase_id` iniziale;

`pattern_run_id` opzionale;

`state_status`.

Questa versione è molto utile per demo LED. Anche senza movimento fisico, il nodo può mostrare un ruolo: `leader` lampeggia in un modo, `connector` in un altro, `terminal` in un altro. Questo permette di dimostrare la logica dello swarm sul tavolo. Il ruolo diventa un'informazione reale del nodo, non solo della dashboard.

La quinta fase può essere definita **MicroBot OS v0.5 — Telemetry Manager**. In questa versione, la telemetria diventa un modulo più strutturato. Il nodo non invia più messaggi casuali, ma payload coerenti: `heartbeat`, `status`, `event`, `error_report`. I cambiamenti di stato generano eventi. Il livello di telemetria può essere `minimal`, `basic`, `normal`, `debug`. Il controller può richiedere status completo. La telemetria diventa compatibile con lo schema definito nella Sezione 13.

In v0.5, le funzioni principali sono:

`heartbeat` strutturato;

`status` completo;

eventi su cambio stato;

`error_report`;

telemetry_level;

rate limiting;

last_event;

source fields se applicabili;

schema_version.

Questa versione migliora molto la debuggabilità. Se un nodo cambia ruolo, genera role_changed. Se entra in safe_idle, genera mode_changed. Se rifiuta un comando, genera command_rejected. Il controller e la dashboard non devono dedurre tutto dagli heartbeat; ricevono eventi espliciti.

La sesta fase può essere definita **MicroBot OS v0.6 — Safety Manager Base**. In questa versione, il nodo introduce controlli di sicurezza locali. Non accetta comandi incompatibili con mode, safety_state, role o capabilities. Spegne le uscite in safe_idle. Gestisce emergency_stop. Introduce blocked_capabilities. Anche se non ci sono ancora bobine o batterie reali, la logica safety può essere simulata o applicata ai LED/PWM. Questa fase è fondamentale prima di qualsiasi attuazione fisica più seria.

In v0.6, le funzioni principali sono:

safety_state;

emergency_stop;

safe_off_all;

command validation safety-aware;

blocked_capabilities;

capability check;

mode check;

role check;

command rejection reason;

safety events.

Questa versione collega MicroBot OS alla Sezione 11. Il nodo non esegue più comandi solo perché sono validi sintatticamente. Li esegue solo se sono sicuri nel suo stato attuale. Questo è il primo vero salto di maturità.

La settima fase può essere definita **MicroBot OS v0.7 — Non-blocking Scheduler**. In questa versione, il firmware viene organizzato attorno a task periodici non bloccanti. Heartbeat, controllo safety, update LED, update output, controllo controller_lost e diagnostica vengono eseguiti a intervalli definiti. I delay lunghi vengono eliminati o fortemente ridotti. Ogni funzione deve essere breve e tornare rapidamente al main loop. Questa fase è essenziale per affidabilità.

In v0.7, le funzioni principali sono:

task scheduler leggero;

heartbeat_task;

safety_task;

output_task;

communication_task;

diagnostic_task;

controller_lost_task;

deadline monitoring;

rimozione di delay bloccanti.

Questa versione prepara il nodo a gestire più responsabilità senza perdere reattività. Un LED può lampeggiare senza bloccare la ricezione dei comandi. Un output temporizzato può spegnersi anche mentre la telemetria viene inviata. Il nodo diventa più vicino a un runtime embedded.

L'ottava fase può essere definita **MicroBot OS v0.8 — Diagnostic Shell**. In questa versione, il nodo espone una shell interna, inizialmente tramite seriale. La shell permette di interrogare stato, identità, ruoli, safety, errori, capabilities, uptime e servizi. Può permettere test controllati come led_test, heartbeat_test, safe_idle, clear_error, status_verbose. La shell è fondamentale per sviluppo hardware, perché consente di diagnosticare un nodo anche senza dashboard completa.

In v0.8, le funzioni principali sono:

shell seriale;

help;

status;

identity;

role;

mode;

safety;

capabilities;

errors;

led_test;

clear_error;

safe_idle;

diagnostic report.

Questa versione rende il nodo molto più pratico da sviluppare. Quando qualcosa non va, si può collegare il nodo al computer e chiedere direttamente il suo stato. La shell diventa la porta locale di debug del MicroBot OS.

La nona fase può essere definita **MicroBot OS v0.9 — Watchdog Integration**. In questa versione, il watchdog viene integrato nella logica del sistema operativo embedded. Il nodo non alimenta il watchdog in modo cieco, ma solo se i task critici sono stati eseguiti correttamente. Dopo un reset, il nodo imposta uscite in OFF, genera nuovo boot_id, legge reset_reason se disponibile, entra in safe_idle o recovery e segnala l'evento al controller. Questa fase collega firmware, safety e recovery.

In v0.9, le funzioni principali sono:

watchdog hardware o software;

task health check;

feed watchdog solo se safety ok;

reset_reason;

boot_id nuovo dopo reset;

safe outputs on boot;

post_reset_mode = safe_idle o recovery;

watchdog_reset event;

recovery_required.

Questa versione è importantissima per hardware reale. Un nodo che controlla uscite fisiche deve poter recuperare da blocchi del firmware. Il watchdog non è una funzione decorativa: è una barriera di sicurezza.

La decima fase può essere definita **MicroBot OS v1.0 — Hardware Abstraction Layer**. In questa versione, l'accesso all'hardware viene separato dal resto della logica. LED, PWM, lettura batteria, lettura temperatura, reset reason, pin mapping e safe output vengono gestiti tramite funzioni HAL. Questo permette di supportare versioni hardware diverse senza riscrivere tutto il sistema operativo embedded.

In v1.0, le funzioni principali sono:

hardware_profile;

pin_map;

hal_set_led;

hal_set_pwm;

hal_stop_pwm;

hal_safe_outputs_off;

hal_read_battery;

hal_read_temperature;

hal_get_reset_reason;

hal_get_uptime;

capabilities derivate dall'hardware_profile.

Questa versione segna il passaggio da firmware per un prototipo specifico a runtime adattabile a più nodi. Se un prototipo ha solo LED, il profilo abilita led_status. Se un prototipo ha PWM, abilita pwm_output. Se un futuro nodo ha sensore temperatura, abilita measured_temperature. La logica alta resta simile.

L'undicesima fase può essere definita **MicroBot OS v1.1 — State Sync and Recovery**. In questa versione, il nodo supporta state_sync dal controller e una recovery più strutturata. Dopo reset, controller_lost, safety event o mismatch, il nodo può rientrare attraverso una procedura: safe state, request_status, sync, state_sync, capability check, reintegration. Il nodo non torna automaticamente active dopo un errore. Questa versione rende MicroBot OS compatibile con la Sezione 11.8 sul ripristino automatico.

In v1.1, le funzioni principali sono:

recovery_mode;

requires_state_sync;

state_sync command;

config_epoch;

phase_id validation;

role re-confirmation;

capabilities refresh;

recovery_event;

reintegration_ready;

manual clear safety se necessario.

Questa fase è critica per sistemi distribuiti. Un nodo può tornare online, ma non è automaticamente coerente. MicroBot OS deve riconoscere la differenza tra online e reintegrabile.

La dodicesima fase può essere definita **MicroBot OS v1.2 — Energy and Thermal Runtime**. In questa versione, energia e temperatura diventano parti operative del nodo. Anche se inizialmente stimate o simulate, vengono integrate nel runtime. Il nodo gestisce energy_state, thermal_state, cooldown, thermal_load stimato, low_energy, critical_energy, thermal_warning e thermal_lock. I comandi ad alto consumo vengono filtrati da questi stati.

In v1.2, le funzioni principali sono:

energy_state;
thermal_state;
thermal_load stimato;
cooldown manager;
low_energy policy;
critical_energy policy;
thermal_warning;
thermal_lock;
command rejection energy/thermal;
telemetria energetica e termica.

Questa versione prepara il passaggio a PWM, bobine e batterie reali. Anche senza sensori, la logica può essere testata in modo conservativo. Quando arriveranno misure reali, il modello potrà essere calibrato.

La tredicesima fase può essere definita **MicroBot OS v1.3 — Actuation-Safe Runtime**. In questa versione, il nodo può gestire attuazioni più serie, ma solo sotto vincoli severi. PWM, bobina sperimentale o carico controllato vengono gestiti con durata massima, duty massimo, cooldown, emergency_stop, watchdog, output timers e logging. Questa versione non deve essere introdotta prima che safety, scheduler, watchdog e HAL siano maturi. È il primo livello veramente hardware-actuation oriented.

In v1.3, le funzioni principali sono:

pwm_output controllato;
magnetic_pulse sperimentale;
max_duty;
max_duration_ms;
auto-off timer;
cooldown obbligatorio;
safe_off_all prioritario;
rejection se safety non ok;
eventi di attuazione;
report diagnostico.

Questa versione deve essere trattata con prudenza. Ogni attuazione deve essere limitata, tracciata e testata. Il nodo non deve mai mantenere un carico critico senza timer di spegnimento. La sicurezza locale ha priorità assoluta.

La quattordicesima fase può essere definita **MicroBot OS v1.4 — Hybrid Integration**. In questa versione, il nodo fisico può essere collegato a un gemello virtuale o a una rappresentazione nella simulazione. Il nodo può inviare heartbeat reale e ricevere ruolo, mentre la sua posizione resta simulata. La dashboard lo mostra come `hybrid_node`. Questo permette demo molto potenti: pochi nodi fisici partecipano a uno swarm virtuale più ampio.

In v1.4, le funzioni principali sono:

`hybrid_node_id`;

`mapping hardware-virtual`;

`heartbeat hardware`;

`role hardware-confirmed`;

`position_source = simulated`;

`state merge`;

`hybrid telemetry`;

`capability profile ibrido`;

`dashboard integration`.

Questa fase è molto coerente con lo stato realistico del progetto: permette di dimostrare architettura, telemetria e controllo anche prima che tutti i MicroBot fisici abbiano movimento o attuazione completa.

La quindicesima fase può essere definita **MicroBot OS v1.5 — Advanced Diagnostics and Service Health**. In questa versione, MicroBot OS monitora i propri servizi interni. Ogni servizio ha `health_status`, `last_update`, `error_count`, `deadline_missed_count`. La shell e la telemetria possono mostrare lo stato di `communication_service`, `safety_service`, `output_service`, `telemetry_service`, `scheduler_service` e `diagnostic_service`. Questa fase rende il nodo molto più osservabile internamente.

In v1.5, le funzioni principali sono:

`service registry`;

`service health`;

`deadline miss counters`;

`internal diagnostics`;

`verbose status`;

`fault counters`;

stability_score locale;

diagnostic snapshots;

self-test.

Questa versione avvicina MicroBot OS a una struttura più matura, simile a un piccolo runtime monitorato. Il nodo non comunica solo “sono vivo”, ma anche “i miei servizi interni sono sani”.

La sedicesima fase può essere definita **MicroBot OS v2.0 — Modular Embedded Runtime**.

Questa è una fase futura, non necessariamente immediata. Qui MicroBot OS diventa un runtime embedded più completo: servizi separati, shell interna più ricca, profili hardware caricabili, diagnostica avanzata, possibile filesystem leggero o configurazione persistente, gestione più precisa di moduli e risorse. Non deve essere promessa come già presente, ma può essere indicata come direzione futura.

In v2.0, le funzioni possibili sono:

runtime modulare;

servizi configurabili;

profilazione delle risorse;

config persistente;

diagnostica avanzata;

shell estesa;

supporto sensori;

supporto attuatori multipli;

policy locali più ricche;

integrazione stretta con dashboard e simulatore.

Questa versione rappresenta la visione più avanzata del MicroBot OS. È il punto in cui il nodo non è più soltanto firmware, ma una piccola piattaforma embedded.

L'evoluzione delle versioni deve essere accompagnata da documentazione di compatibilità. Ogni versione deve indicare quali comandi supporta, quali stati usa, quali capabilities espone, quale protocol_version richiede e quali limitazioni ha. Questo è fondamentale perché controller, dashboard e firmware devono restare compatibili. Se il controller invia state_sync a un nodo v0.3 che non lo supporta, il nodo deve rispondere unsupported_command, non comportarsi in modo ambiguo.

Una scheda versione dovrebbe contenere:

versione MicroBot OS;

firmware_version;

protocol_version;

hardware_profile supportati;

comandi supportati;

stati supportati;

capabilities;

telemetria supportata;

safety features;

limitazioni;

compatibilità controller.

Questa scheda rende possibile lavorare con nodi di versioni diverse. In uno swarm reale, è possibile avere nodi aggiornati e nodi vecchi. Il controller deve sapere cosa aspettarsi.

L'evoluzione delle versioni deve includere anche una politica di deprecazione. Alcuni comandi o campi possono cambiare. Per evitare confusione, bisogna marcare ciò che è deprecated, cioè ancora riconosciuto ma non consigliato. Per esempio, un vecchio comando `led_on` può essere sostituito da `set_led`. Per un periodo il nodo può supportare entrambi, poi rimuovere il vecchio. Questa disciplina è importante quando il progetto cresce.

Una politica semplice può essere:

experimental: funzione instabile;

supported: funzione stabile;

deprecated: funzione vecchia ancora accettata;

removed: funzione non più supportata.

Ogni comando e campo importante può avere uno stato. Questo evita rotture improvvise.

L'evoluzione delle versioni deve essere collegata alla simulazione. Ogni nuova funzione MicroBot OS dovrebbe essere prima simulata o testata in un ambiente controllato. Per esempio, `state_sync` può essere implementato in nodi virtuali prima del firmware. `Thermal_lock` può essere simulato prima di essere collegato a una bobina reale. `Watchdog_reset` può essere simulato prima di testarlo fisicamente. Questo mantiene coerenza con la Sezione 12.

La pipeline corretta è:

definizione nella documentazione;

implementazione nel simulatore;

test con nodi virtuali;

integrazione nella dashboard;

implementazione firmware;

test hardware-safe;

validazione ibrida;

eventuale promozione a stabile.

Questa pipeline evita di mettere nel firmware funzioni non chiarite. La simulazione diventa il banco prova del MicroBot OS.

L'evoluzione delle versioni deve essere collegata alla telemetria. Ogni versione deve esporre abbastanza dati per essere monitorata e debuggata. Una funzione non osservabile è difficile da validare. Se si aggiunge `safety_manager`, bisogna aggiungere `safety_state` e `safety events`. Se si aggiunge `scheduler`, bisogna poter vedere almeno `task health` o `deadline miss` in diagnostica. Se si aggiunge `energy runtime`, bisogna esporre `energy_state`. Ogni nuovo modulo deve avere telemetria minima.

Una regola è:

ogni nuova funzione operativa deve avere stato, eventi e diagnostica minimi.

Questo evita moduli invisibili. Se qualcosa va male, si deve poter capire.

L'evoluzione delle versioni deve essere collegata al Safety Layer. Le versioni che introducono funzioni potenzialmente rischiose devono richiedere `safety` già matura. Per esempio, non si dovrebbe introdurre PWM su carico reale prima di avere `output auto-off`, `emergency_stop`, `watchdog` e `cooldown`. Non si dovrebbe introdurre batteria reale senza `low_energy policy`. Non si dovrebbe introdurre bobina senza `thermal/cooldown`. Questa dipendenza deve essere esplicita.

Una dipendenza corretta è:

`actuation_safe_runtime` richiede `safety_manager`, `scheduler`, `watchdog`, `HAL` e `telemetry events`.

Questo significa che `v1.3` non dovrebbe essere implementata seriamente se `v0.6`, `v0.7`, `v0.9` e `v1.0` non sono abbastanza solide. È una regola prudente.

L'evoluzione delle versioni deve essere testata con una suite per versione. Ogni versione ha test minimi. `v0.1` deve passare `boot_test`. `v0.2` `heartbeat_test`. `v0.3` `command_test`. `v0.4` `role_test`. `v0.6` `safety_rejection_test`. `v0.9` `watchdog_recovery_test`. `v1.3` `actuation_timer_test`. Questo permette di sapere se una versione è realmente funzionante.

Una suite per versioni può essere:

`v0.1`: `boot`, `identity`, `safe outputs`;

`v0.2`: `heartbeat`, `uptime`, `online status`;

`v0.3`: `command parsing`, `ack`, `rejection`;

`v0.4`: `role assignment`, `mode transition`;

`v0.5`: `telemetry schema`, `events`;

`v0.6`: `emergency_stop`, `safety rejection`;

v0.7: non-blocking tasks;
v0.8: shell commands;
v0.9: watchdog reset recovery;
v1.0: HAL pin abstraction;
v1.1: state_sync recovery;
v1.2: energy/thermal policy;
v1.3: PWM auto-off and cooldown.

Questi test trasformano la roadmap in verifica.

L'evoluzione delle versioni deve essere compatibile con i prototipi hardware reali. Il primo ESP32 LED può usare MicroBot OS v0.2 o v0.3. Una board con LED di ruolo può usare v0.4. Un nodo con PWM su LED può usare v0.6 o v0.7. Un nodo con bobina sperimentale deve aspettare v1.3. Questa associazione aiuta a scegliere cosa implementare per ogni prototipo.

Una mappa hardware-versione può essere:

LED breadboard prototype: v0.1–v0.4;
ESP32 role demo: v0.4–v0.5;
ESP32 telemetry demo: v0.5–v0.7;
ESP32 diagnostic node: v0.8;
ESP32 watchdog node: v0.9;
PWM safe node: v1.0–v1.2;
coil experimental node: v1.3 solo con safety completa;
hybrid demo node: v1.4.

Questa mappa è molto utile per evitare di sovraccaricare prototipi semplici con funzioni non pronte.

L'evoluzione delle versioni deve essere descritta anche come crescita concettuale. v0.x rappresenta la costruzione del nodo come entità osservabile e comandabile. v1.x rappresenta la costruzione del nodo come entità sicura, modulare e integrabile con hardware. v2.x rappresenta la visione futura di runtime embedded più completo. Questa divisione rende la roadmap più leggibile.

Una sintesi può essere:

v0.x = identità, comunicazione, comandi, ruoli, telemetria, safety base;
v1.x = HAL, recovery, energia, temperatura, attuazione sicura, ibrido, diagnostica avanzata;
v2.x = runtime modulare avanzato, servizi, configurazione persistente, estensione hardware e OS-like behavior.

Questa struttura dà ordine al progetto. Non tutto appartiene allo stesso livello di maturità.

L'evoluzione delle versioni deve essere tracciata nei log e nella dashboard. Ogni nodo deve dichiarare `microbot_os_version`, `firmware_version` e `protocol_version`. La dashboard deve poter mostrare questi campi. Se un nodo non supporta una funzione, la dashboard deve indicarlo. Questo evita errori durante demo e debug.

Un node identity payload può contenere:

`node_id;`

`boot_id;`

`microbot_os_version;`

`firmware_version;`

`protocol_version;`

`hardware_profile;`

`capability_profile;`

`build_date` opzionale;

`supported_commands.`

Questi dati devono essere disponibili tramite `request_status` o `identity command`.

L'evoluzione delle versioni deve essere coerente con le tue versioni precedenti di MicroBot OS sperimentale. Se esistono prototipi con schermate, shell, diagnostica, framebuffer, kernel o UI, possono essere interpretati come esperimenti di direzione OS-like. In questa documentazione, però, bisogna mantenere rigore: quelle versioni dimostrano esplorazione del concetto, non necessariamente un sistema operativo embedded completo per nodo fisico. Possono essere inserite come “esperimenti di interfaccia e runtime” che ispirano MicroBot OS futuro.

Una formulazione corretta è:

le versioni sperimentali di MicroBot OS già sviluppate rappresentano prove di organizzazione runtime, interfaccia, shell e diagnostica; la roadmap embedded definisce come questi concetti possano essere progressivamente adattati ai nodi fisici.

Questo permette di valorizzare il lavoro già fatto senza esagerarlo.

L'evoluzione delle versioni deve prevedere documentazione changelog. Ogni versione deve avere cosa è stato aggiunto, cosa è cambiato, cosa è rotto, cosa è sperimentale e cosa resta da fare. Il changelog è essenziale in un progetto con molte iterazioni. Senza changelog, si perde memoria tecnica.

Un changelog può avere:

`versione;`

`data;`

added;

changed;

fixed;

deprecated;

known limitations;

next version goals.

Questo formato è semplice e professionale. Può stare in CHANGELOG.md della repo MicroBot OS.

L'evoluzione delle versioni deve includere anche limitazioni dichiarate. Per esempio, v0.2 non supporta ruoli. v0.4 non supporta safety completa. v0.6 non supporta recovery avanzata. v1.2 usa energia stimata e non misurata. v1.3 è sperimentale e richiede carichi sicuri. Queste limitazioni sono importanti per evitare interpretazioni sbagliate.

Ogni versione dovrebbe indicare:

supported;

not_supported;

experimental;

requires;

unsafe_without.

Per esempio:

v1.3 supports PWM auto-off and cooldown; requires watchdog and safety manager; unsafe_without tested safe_off_all.

Questa chiarezza è molto forte tecnicamente.

L'evoluzione delle versioni deve essere guidata dalla necessità, non dalla voglia di complessità. Non serve implementare una shell avanzata se heartbeat e command pipeline non sono stabili. Non serve attuazione se safety non è pronta. Non serve configurazione persistente se il profilo hardware è ancora semplice. La roadmap deve essere ambiziosa ma ordinata.

Una regola pratica è:

ogni versione deve risolvere un problema reale della versione precedente.

v0.2 risolve l'assenza di osservabilità. v0.3 risolve l'assenza di controllo. v0.4 risolve l'assenza di ruoli. v0.5 risolve telemetria disordinata. v0.6 risolve esecuzione non filtrata. v0.7 risolve blocchi del loop. v0.8 risolve debug locale. v0.9 risolve blocchi firmware. v1.0 risolve dipendenza diretta dall'hardware. Questa logica rende l'evoluzione solida.

L'evoluzione delle versioni deve infine essere usata come strumento di presentazione del progetto. Quando si mostra MicroBot OS a un collaboratore, non bisogna dire genericamente “sto facendo un

OS”. Bisogna mostrare la progressione: prima identità, poi heartbeat, poi comandi, poi ruoli, poi telemetria, poi safety, poi scheduler, poi shell, poi watchdog, poi HAL, poi recovery, poi attuazione sicura. Questa progressione è credibile e dimostra consapevolezza ingegneristica.

Una frase sintetica può essere:

MicroBot OS non è un salto improvviso verso un sistema operativo completo, ma una roadmap progressiva che trasforma il firmware del nodo in un runtime embedded modulare, osservabile e sicuro.

Questa è la chiave concettuale della sezione.

In conclusione, l'evoluzione delle versioni di MicroBot OS definisce il percorso tecnico attraverso cui il nodo passa da firmware semplice a runtime embedded strutturato. Le prime versioni introducono identità, boot, heartbeat e comandi; le versioni successive aggiungono ruoli, telemetria, safety, scheduler, shell diagnostica e watchdog; le versioni v1.x introducono astrazione hardware, recovery, energia, temperatura, attuazione sicura e integrazione ibrida; le versioni future v2.x possono trasformare MicroBot OS in un runtime modulare più avanzato. Ogni versione deve avere funzioni, limiti, test, compatibilità e documentazione chiari. Questo approccio permette di crescere senza confondere visione e stato attuale, mantenendo MicroBot OS come infrastruttura realistica, evolutiva e coerente con simulazione, telemetria, sicurezza e hardware fisico. Il punto successivo, dedicato alla gestione delle risorse, entrerà nel dettaglio di come MicroBot OS deve amministrare tempo, memoria, energia, pin, comunicazione, task, buffer e capacità locali del nodo, evitando sprechi e garantendo comportamento stabile anche su microcontrollori con risorse limitate.

14.3 Gestione delle risorse

La gestione delle risorse rappresenta il livello di MicroBot OS dedicato al controllo ordinato di tutto ciò che il nodo possiede e consuma: tempo di elaborazione, memoria, energia, pin hardware, canali PWM, comunicazione, buffer, task, sensori, attuatori, telemetria e stati interni. Dopo aver definito MicroBot OS come runtime embedded e dopo averne descritto l'evoluzione per versioni, diventa necessario spiegare come questo sistema operativo leggero debba amministrare le risorse disponibili senza sprechi, senza conflitti e senza comportamenti imprevedibili. Un nodo MicroBot non è un computer generico con risorse abbondanti. È un microcontrollore o un modulo embedded con limiti precisi. Per questo motivo, ogni risorsa deve essere trattata come qualcosa di finito, controllato e monitorabile.

Nel progetto MicroBot, la gestione delle risorse è importante perché il nodo deve fare molte cose contemporaneamente pur restando piccolo. Deve comunicare, inviare heartbeat, ricevere comandi, aggiornare LED, controllare eventuali PWM, monitorare safety, rispettare cooldown, gestire watchdog, mantenere stato interno, rispondere alla diagnostica e, in futuro, leggere sensori o controllare attuatori. Se queste funzioni vengono aggiunte senza una logica di gestione, il firmware diventa fragile: task che si bloccano, memoria consumata senza controllo, comandi persi, output lasciati attivi, telemetria eccessiva, battery drain, reset e comportamenti non ripetibili. MicroBot OS deve prevenire questa fragilità attraverso una gestione esplicita delle risorse.

Il principio generale può essere espresso così:

ogni risorsa del nodo deve avere un proprietario logico, un limite operativo, uno stato osservabile e una politica di sicurezza.

Questa frase è centrale. Un pin GPIO non deve essere controllato da più parti del firmware senza coordinazione. Un canale PWM non deve essere attivato da un comando e disattivato da un altro

modulo senza stato condiviso. Un buffer di comandi non deve crescere indefinitamente. La telemetria non deve saturare la comunicazione. Il tempo CPU non deve essere monopolizzato da una funzione bloccante. L'energia non deve essere consumata senza budget. Ogni risorsa deve essere gestita da un modulo preciso e secondo regole note.

La prima risorsa da gestire è il tempo di esecuzione. In un microcontrollore, il tempo è una risorsa critica perché tutte le funzioni condividono lo stesso ciclo operativo. Se una funzione impiega troppo, le altre vengono ritardate. Se un task blocca il loop, il nodo può non processare `emergency_stop`, non spegnere output temporizzati, non inviare heartbeat o non aggiornare il watchdog. MicroBot OS deve quindi trattare il tempo come risorsa schedulata. Ogni task deve avere una frequenza, una durata attesa e una priorità.

Una gestione corretta del tempo richiede:

task brevi;

nessun delay bloccante nelle funzioni critiche;

scheduler basato su timestamp;

controllo delle scadenze;

misura opzionale del tempo di esecuzione;

priorità per safety e output;

watchdog collegato alla salute dei task.

Per esempio, `output_update` può dover girare molto spesso, perché controlla timer di spegnimento e PWM. Heartbeat può girare meno spesso. `Diagnostic_shell` può avere priorità più bassa. `Telemetry_debug` può essere disabilitata se il nodo è in `low_energy`. Questa differenziazione evita che una funzione secondaria impedisca una funzione critica.

La gestione del tempo deve distinguere tra task critici, task importanti e task accessori. I task critici sono quelli collegati alla sicurezza: `emergency_stop`, `output_off`, `safety_check`, `watchdog_check`, `controller_lost_check`. I task importanti sono quelli necessari alla normale integrazione: `communication_receive`, `heartbeat`, `state_update`, `command_dispatch`. I task accessori sono diagnostica estesa, effetti LED complessi, telemetria verbose, statistiche non essenziali. In condizioni normali possono girare tutti. In condizioni degradate, i task accessori possono essere ridotti o sospesi, mentre quelli critici devono restare attivi.

Una gerarchia temporale può essere:

`critical_tasks`: sempre attivi e frequenti;

`core_tasks`: attivi in modalità normale;

`diagnostic_tasks`: attivi solo se richiesti;

`visual_tasks`: riducibili in `low_energy`;

`debug_tasks`: disabilitabili in modalità conservativa.

Questa gerarchia deve essere applicata dallo scheduler. Se il nodo entra in `critical_energy`, non ha senso continuare a inviare telemetria debug estesa. Deve mantenere `safety`, `heartbeat` minimo e stato sicuro. Questo collega gestione del tempo e protezione energetica.

La seconda risorsa da gestire è la memoria. Un microcontrollore ha memoria limitata. Ogni buffer, stringa, payload, log locale, coda comandi e struttura dati consuma memoria. Se la memoria viene gestita male, il firmware può diventare instabile, rallentare o resettarsi. MicroBot OS deve usare buffer limitati, strutture compatte e log locali controllati. Non deve conservare indefinitamente eventi o comandi. Non deve costruire payload enormi se non necessario.

Una gestione prudente della memoria richiede:

- buffer con dimensione massima;

- code circolari per eventi recenti;

- payload compatti;

- evitare duplicazione inutile dello stato;

- limitare diagnostica verbose;

- cancellare comandi eseguiti o scaduti;

- non salvare log infiniti nel nodo;

- inviare dati al controller invece di accumularli localmente.

Il nodo non deve comportarsi come un database. Il suo compito è mantenere stato operativo locale e inviare telemetria. I log completi devono stare sul controller, sul backend o sul sistema di raccolta dati. Localmente basta mantenere `last_error`, ultimi eventi essenziali, contatori e stato corrente. Questa scelta è realistica per ESP32 e futuri microcontrollori.

La memoria deve essere organizzata intorno a strutture principali: `node_state`, `command_buffer`, `event_buffer`, `telemetry_buffer`, `service_status` e `hardware_state`. `node_state` conserva lo stato corrente. `command_buffer` conserva comandi in attesa, se supportati. `event_buffer` conserva pochi eventi recenti. `telemetry_buffer` prepara il prossimo messaggio. `service_status` registra salute dei task. `hardware_state` registra output attivi, PWM, LED e cooldown. Ogni struttura deve avere dimensione prevedibile.

Una possibile organizzazione è:

- `node_state`: sempre presente e compatto;

- `command_buffer`: piccolo, con limite massimo;

- `event_buffer`: ring buffer di eventi recenti;

- `telemetry_payload`: costruito e inviato, non conservato a lungo;

- `output_state`: stato di LED/PWM/attuatori;

- `diagnostic_state`: contatori e `last_error`;

service_table: stato dei task interni.

Questa struttura rende il firmware controllabile. Se un nuovo modulo richiede memoria, deve dichiarare dove la usa e quanto può crescere.

La terza risorsa è l'energia. La gestione dell'energia è già stata trattata nel Safety Layer, ma MicroBot OS deve applicarla localmente. L'energia non è solo valore di batteria; è anche budget operativo. Ogni servizio consuma: Wi-Fi, LED, telemetria, sensori, PWM, attuatori. Il sistema operativo embedded deve poter ridurre funzioni quando energy_state peggiora. Deve sapere quali task sono sacrificabili e quali no. Deve sapere quali output consumano molto. Deve impedire comandi ad alto consumo quando non ci sono condizioni sicure.

Una gestione energetica locale può includere:

energy_state;

power_source;

telemetry_level adattivo;

LED brightness ridotta;

debug disabilitato in low_energy;

attuazioni bloccate in critical_energy;

sleep futuro;

safe_idle energetico;

eventi energy_state_changed.

Il nodo deve poter passare da modalità normale a modalità conservativa. Per esempio, in low_energy può mantenere heartbeat ma ridurre frequenza. In critical_energy può disattivare LED non essenziali, bloccare PWM e inviare solo eventi critici. Questa è gestione risorse, non solo safety: il nodo decide come spendere la poca energia rimasta.

La quarta risorsa è il set di pin hardware. Ogni pin ha una funzione: LED, PWM, sensore, input, output, comunicazione. Un errore di pin mapping può rompere il comportamento o danneggiare componenti. MicroBot OS deve gestire i pin attraverso un hardware profile e una HAL. Nessun modulo alto dovrebbe manipolare direttamente pin fisici senza passare da astrazione. Questo è fondamentale quando il progetto passa da un prototipo su breadboard a versioni hardware diverse.

Un pin_map può contenere:

status_led_pin;

role_led_pin;

pwm_output_pin;

coil_driver_pin;

battery_adc_pin;

```
temperature_sensor_pin;  
  
button_pin;  
  
debug_pin;  
  
reserved_pins.
```

Ogni pin deve avere stato e proprietario. Per esempio, pwm_output_pin appartiene a output_manager. battery_adc_pin appartiene a energy_manager. status_led_pin appartiene a led_service o output_manager. Se due moduli tentano di usare lo stesso pin per funzioni diverse, la configurazione deve essere considerata invalida. Questo controllo può avvenire a compile time o durante boot.

La gestione dei pin deve includere uno stato sicuro all'avvio. Al boot, tutti i pin critici devono essere configurati in modo che non attivino carichi pericolosi. Un pin collegato a driver o bobina deve partire OFF. Un pin PWM deve partire duty zero. Questa sequenza deve avvenire prima della connessione Wi-Fi, prima della shell e prima di accettare comandi. L'hardware deve essere sicuro anche se il software centrale non è ancora collegato.

Una regola forte è:

prima inizializzare sicurezza delle uscite, poi inizializzare comunicazione e servizi.

Questo ordine è importante. Se il nodo si resetta durante una condizione anomala, deve tornare a uscite spente prima di qualsiasi altra cosa. La gestione risorse comincia dal boot safe.

La quinta risorsa è il PWM. Il PWM è una risorsa più delicata di un semplice GPIO, perché implica frequenza, duty cycle, canale, risoluzione e durata. Su ESP32, i canali PWM sono limitati e devono essere assegnati. MicroBot OS deve sapere quali canali sono in uso, quale pin controllano, quale duty massimo è permesso e se sono in cooldown. Un comando PWM non deve configurare il canale in modo arbitrario ogni volta senza controllo.

Una gestione PWM deve includere:

```
pwm_channel_id;  
  
pwm_pin;  
  
frequency_hz;  
  
resolution_bits;  
  
current_duty;  
  
max_allowed_duty;  
  
duration_ms;  
  
auto_off_time;  
  
cooldown_remaining_ms;  
  
owner_module;
```

safety_state.

Il PWM deve essere controllato dall'output_manager. Se un comando chiede duty troppo alto, viene ridotto o rifiutato. Se chiede durata troppo lunga, viene rifiutato. Se il nodo è in thermal_warning, il max_allowed_duty può essere ridotto. Se è in thermal_lock, duty deve essere zero. Questo collega PWM, safety e gestione risorse.

La sesta risorsa è la comunicazione. La comunicazione consuma tempo, energia e banda. Se il nodo invia troppa telemetria, può rallentare il sistema, consumare batteria o perdere pacchetti. Se ascolta troppo poco, può perdere comandi. MicroBot OS deve gestire intervalli, priorità, payload e stato di connessione. Deve distinguere messaggi critici da messaggi informativi.

Una gestione comunicativa deve includere:

heartbeat_interval_ms;

telemetry_interval_ms;

last_controller_seen;

controller_lost_timeout_ms;

outgoing_queue_limit;

incoming_buffer_limit;

message_priority;

retry_policy;

communication_status.

I messaggi safety devono avere priorità. Un emergency_stop ricevuto deve essere processato subito. Un thermal_lock_event deve essere inviato appena possibile. Un debug_report può aspettare. Se la coda uscente è piena, il nodo deve preferire eventi critici e scartare o comprimere dati non essenziali. Questa è gestione risorse applicata alla rete.

La settima risorsa è la capacità di elaborazione. Anche se un ESP32 è relativamente potente per molti prototipi, non bisogna sprecarlo. Parsing JSON pesante, effetti LED complessi, diagnostica verbose, crittografia futura, sensori e Wi-Fi possono consumare CPU. MicroBot OS deve mantenere il loop reattivo. Se una funzione di parsing o diagnostica richiede tempo, deve essere limitata o spezzata. La shell interna non deve bloccare safety. Il rendering non avviene sul nodo, quindi non deve pesare sul firmware.

Una politica CPU può essere:

parsing semplice e validato;

task brevi;

diagnostica verbose solo su richiesta;

nessun calcolo pesante nel loop critico;

profilazione opzionale dei task;

watchdog su task critici;

degradazione se loop_time supera soglia.

Il nodo può misurare loop_time o task_time. Se il main loop diventa troppo lento, può generare warning. Questo è utile per debug del firmware. Un nodo che mantiene heartbeat ma ha loop troppo lento può essere pericoloso per output temporizzati.

La ottava risorsa è il buffer dei comandi. Il nodo può ricevere comandi più velocemente di quanto li esegua. MicroBot OS deve decidere se accodare, rifiutare o sostituire comandi. Non tutti i comandi possono essere messi in coda. Un emergency_stop deve essere eseguito immediatamente. Un set_led può sostituire il precedente. Un magnetic_pulse non deve essere accodato se il nodo è in cooldown o se il comando scade. Un state_sync deve invalidare stato precedente. Senza policy, la coda comandi può creare comportamenti imprevedibili.

Una politica command_buffer può essere:

emergency_stop: priorità immediata, svuota comandi attuativi;

request_status: può essere risposto subito;

set_led: sostituisce comando LED precedente;

assign_role: aggiorna ruolo, invalida ruolo non confermato precedente;

actuation_command: accettato solo se può essere eseguito entro expires_at;

expired_command: scartato e loggato;

duplicate_command_id: ignorato o ack duplicato;

buffer_full: rifiuto con reason = command_buffer_full.

Questa politica è molto importante. Un nodo non deve eseguire in ritardo un comando che aveva senso solo in una fase precedente. Comandi temporizzati devono avere expires_at e phase_id. Il buffer deve rispettarli.

La nona risorsa è il buffer degli eventi. Gli eventi locali sono utili, ma non devono crescere senza limite. Il nodo può mantenere un ring buffer di ultimi N eventi. Quando il controller chiede diagnostic_status, il nodo può restituire eventi recenti. Gli eventi vecchi vengono sovrascritti. I log completi vivono fuori dal nodo. Questa scelta permette debug senza saturare memoria.

Un event_buffer locale può contenere:

event_id;

event_type;

timestamp;

severity;

related_command_id;

short_reason.

Dimensione possibile: 10, 20 o 50 eventi, a seconda della memoria. Non serve enorme. Serve abbastanza per sapere cosa è appena successo. Eventi critici possono essere inviati subito al controller.

La decima risorsa è la telemetria. La telemetria è utile, ma è anche costo. MicroBot OS deve decidere cosa inviare, quando inviarlo e con quale dettaglio. telemetry_level deve essere una risorsa configurabile. In modalità minimal, il nodo invia heartbeat essenziale. In normal, invia stato completo. In debug, invia dettagli. In low_energy, riduce. In diagnostic, aumenta temporaneamente. Questa gestione permette di adattare il carico.

Profili telemetrici:

minimal: node_id, boot_id, mode, safety_state;

basic: aggiunge role, energy_state, thermal_state;

normal: aggiunge sync_status, phase_id, last_error;

debug: aggiunge capabilities, counters, task health;

full: aggiunge eventi recenti e diagnostica estesa.

Il nodo deve poter cambiare profilo su comando, ma sempre rispettando energia e safety. Se energy_state = critical_energy, il nodo può rifiutare telemetry_full. Questo è un esempio di risorsa informativa controllata.

L'undicesima risorsa è lo stato. Lo stato interno è una risorsa logica perché deve restare coerente. Se più moduli modificano mode, role o safety_state senza regole, il nodo diventa imprevedibile. MicroBot OS deve avere state_manager come unico punto di aggiornamento controllato. Ogni transizione deve essere valida. Per esempio, safety non può passare direttamente ad active. Recovery deve passare da state_sync. Low_power deve bloccare task non essenziali.

Una gestione dello stato deve includere:

transizioni ammesse;

eventi su transizione;

validazione della transizione;

motivo della transizione;

timestamp;

modulo responsabile.

Per esempio:

mode active → safety, reason = emergency_stop;

mode safety → safe_idle, reason = clear_safety_authorized;

mode recovery → idle, reason = state_sync_completed;

mode idle → active, reason = pattern_start.

Questo rende lo stato tracciabile. Il nodo non cambia modalità senza motivo.

La dodicesima risorsa è la capacità operativa, cioè l'insieme delle capabilities disponibili. Le capabilities non sono semplici descrizioni statiche. Alcune possono essere temporaneamente bloccate. Un nodo può avere pwm_output come capability hardware, ma questa può essere blocked per thermal_lock. Un nodo può avere telemetry_debug, ma può essere blocked per low_energy. MicroBot OS deve gestire capabilities come risorse dinamiche.

Una struttura capability può contenere:

capability_name;

installed;

available;

blocked;

block_reason;

required_mode;

required_energy_state;

required_thermal_state;

owner_module.

Quando cambia safety o energia, available_capabilities deve essere aggiornato. Il controller deve ricevere eventi se una capability importante viene bloccata o ripristinata. Questo è fondamentale per role assignment.

La tredicesima risorsa sono i sensori. Anche se non tutti i prototipi li avranno subito, MicroBot OS deve prevedere la gestione di sensori come batteria, temperatura, prossimità, corrente, IMU o camera/biometria futura. I sensori devono avere frequenza di lettura, source, stato, validità e ultimo valore. Non devono essere letti continuamente senza motivo. Alcuni sensori possono consumare energia. Alcuni possono produrre dati rumorosi. Serve una politica.

Un sensor_manager può gestire:

sensor_id;

sensor_type;

enabled;

read_interval_ms;

last_value;

last_read_time;

source;

confidence;

error_state;

calibration_status.

Se un sensore è unavailable, il sistema deve usare stato unknown o stimato. Se un sensore fallisce, genera evento. Se un sensore è critico per safety e non funziona, il nodo deve entrare in modalità conservativa. Per esempio, se temperature_source passa da measured a unavailable su un nodo con bobina, la capability coil_pwm può essere bloccata o limitata.

La quattordicesima risorsa sono gli attuatori. LED, PWM, bobine, buzzer, motori o altri output devono essere gestiti con proprietà chiare. Ogni attuatore deve avere stato, limiti, proprietario, timer e safety policy. Non deve esistere attuatore “libero” controllato da codice casuale. L’output_manager deve essere il punto centrale.

Una struttura actuator_state può contenere:

actuator_id;

actuator_type;

pin_or_channel;

enabled;

current_state;

current_power_level;

max_power_level;

duration_remaining_ms;

cooldown_remaining_ms;

safety_locked;

last_command_id.

Questo permette di sapere sempre cosa è attivo. Se il nodo entra in safe_idle, output_manager spegne tutti gli actuator_state abilitati e aggiorna stato. Se un comando tenta di accendere un attuatore bloccato, viene rifiutato.

La quindicesima risorsa è la configurazione. La configurazione locale stabilisce pin, limiti, intervalli, capabilities, versioni e policy. Una configurazione sbagliata può generare molti problemi. MicroBot OS deve gestire config_id, config_epoch e validazione. Se la configurazione cambia, il nodo deve segnalarlo. Se riceve comandi basati su config_epoch vecchio, può rifiutare o richiedere sync.

Una configurazione può contenere:

config_id;

config_epoch;
hardware_profile;
safety_policy_id;
telemetry_policy_id;
pin_map;
task_intervals;
limits;
capabilities.

La configurazione deve essere validata al boot. Se manca un pin critico, se due funzioni usano lo stesso pin, se un limite è incoerente, il nodo deve entrare in diagnostic o safe mode. Meglio bloccare un nodo mal configurato che lasciarlo funzionare in modo ambiguo.

La sedicesima risorsa è il tempo di vita della sessione. boot_id, uptime e reset_count sono risorse informative che permettono di capire la continuità del nodo. Dopo un reset, alcune stime vengono perse: thermal_load in RAM, command_buffer, ruolo, sync. MicroBot OS deve sapere cosa è persistente e cosa no. Deve dichiarare recovery_required se la continuità è rotta. Questa è gestione risorse perché lo stato di sessione è una risorsa logica.

Una gestione sessione include:

boot_id;
uptime_ms;
reset_reason;
reset_count;
session_valid;
requires_state_sync;
lost_runtime_state.

Dopo reset, lost_runtime_state può includere thermal_estimate, command_queue, sync_offset, assigned_role. Il controller deve ricostruire. Il nodo non deve fingere di avere ancora stato valido.

La diciassettesima risorsa è la priorità. In un sistema embedded, priorità è una risorsa decisionale. Quando arrivano più richieste, bisogna decidere cosa viene prima. Emergency_stop prima di tutto. Safety_check prima di LED effect. Output auto-off prima di debug shell. Heartbeat minimo prima di telemetria full. MicroBot OS deve avere una politica di priorità chiara.

Una scala di priorità può essere:

P0: emergency_stop e safe_off_all;

P1: output timers, watchdog, safety_check;

P2: command_receive e controller_lost;

P3: heartbeat e state_update;

P4: role LED e pattern visuale;

P5: diagnostic shell e debug;

P6: effetti o telemetria verbose.

Questa scala deve guidare scheduler e command_manager. In condizioni di carico, si sacrificano P5/P6, non P0/P1. Questa è una garanzia di sicurezza.

La diciottesima risorsa è la banda interna tra moduli. Anche se non è una rete fisica, i moduli comunicano attraverso stato, eventi e code. Se ogni modulo accede direttamente a tutto, si crea accoppiamento. MicroBot OS deve usare interfacce chiare: state_manager per stato, event_bus locale o event_emit per eventi, command_dispatcher per comandi, output_manager per uscite. Questo riduce conflitti.

Una struttura interna ordinata può essere:

command_manager riceve comandi e invia richieste;

safety_manager autorizza o rifiuta;

output_manager esegue output;

state_manager aggiorna stato;

telemetry_manager pubblica eventi;

scheduler chiama task.

Questa separazione permette di sapere chi fa cosa. Il codice diventa più leggibile.

La diciannovesima risorsa è la diagnostica. Anche la diagnostica deve essere gestita. Non può occupare tutte le risorse del nodo. La shell può essere attiva in diagnostic_mode, ma ridotta in active o low_energy. I comandi diagnostici potenzialmente costosi devono essere limitati. Per esempio, dump_full_status può essere consentito solo quando il nodo non sta eseguendo attuazioni critiche. Questo evita che il debug stesso diventi causa di problemi.

Una politica diagnostica può essere:

diagnostic basic sempre disponibile;

diagnostic full solo in idle o diagnostic mode;

nessun comando diagnostico bloccante durante active;

shell rate-limited;

debug telemetry disattivabile;

diagnostic output compatto in low_energy.

Questa scelta mantiene il nodo reattivo anche quando viene interrogato.

La ventesima risorsa è la sicurezza operativa delle uscite. Questa è una risorsa trasversale: ogni uscita critica deve avere un limite. Non basta sapere che il pin esiste. Bisogna sapere per quanto può essere attivo, con quale duty, con quale cooldown, in quale modalità, con quale energia e con quale temperatura. MicroBot OS deve applicare questi limiti come parte della gestione risorse.

Per ogni uscita critica, si possono definire:

```
max_duration_ms;  
  
max_duty;  
  
min_cooldown_ms;  
  
allowed_modes;  
  
blocked_states;  
  
requires_role;  
  
requires_sync;  
  
auto_off_enabled;  
  
watchdog_safe_off.
```

Questi parametri rendono gli attuatori gestibili. Senza, un comando errato può lasciare output attivo o fuori limite.

La ventunesima risorsa è il numero di errori e tentativi. Anche retry e error_count sono risorse. Un nodo non deve ritentare indefinitamente. Un controller non deve inviare lo stesso comando infinite volte. MicroBot OS può mantenere counters: rejected_command_count, safety_event_count, controller_lost_count, reset_count, retry_count. Quando superano soglie, il nodo può entrare in diagnostic_required o unstable.

Una politica counters può includere:

```
max_rejected_commands_per_window;  
  
max_controller_lost_events;  
  
max_watchdog_resets;  
  
max_actuation_failures;  
  
max_sync_failures;  
  
diagnostic_required_threshold.
```

Questi contatori aiutano a evitare loop. Se il nodo continua a rifiutare comandi per thermal_lock, il problema non è il nodo: è il piano del controller. Il sistema deve fermarsi e analizzare.

La ventiduesima risorsa è la compatibilità di protocollo. Un nodo ha protocol_version. Il controller ha una versione attesa. Se non coincidono, alcune funzioni possono non essere disponibili.

MicroBot OS deve gestire la compatibilità come risorsa: quali comandi supporta, quali payload, quali eventi. Se un comando non è supportato, deve rispondere `unsupported_command`. Questo evita fallimenti silenziosi.

Una gestione protocollo può includere:

`protocol_version`;

`supported_commands`;

`supported_events`;

`schema_version`;

`deprecated_commands`;

`minimum_controller_version`.

Questo è importante quando lo swarm contiene nodi di versioni diverse. Il controller può adattarsi o escludere nodi incompatibili.

La ventitreesima risorsa è la modalità di lavoro. Un nodo in `demo_mode` non deve comportarsi come in `diagnostic_mode`. In `demo_mode` può limitare comandi per sicurezza e mostrare stati visuali. In `diagnostic_mode` può accettare test controllati. In `hardware_actuation_mode` può richiedere safety completa. MicroBot OS deve gestire la modalità come risorsa globale che abilita o disabilita funzioni.

Modalità possibili:

`development_mode`;

`demo_mode`;

`diagnostic_mode`;

`simulation_bridge_mode`;

`hardware_visual_mode`;

`hardware_actuation_mode`;

`safe_mode`.

Ogni modalità ha limiti. Per esempio, `hardware_actuation_mode` richiede watchdog, `safety_manager`, cooldown e HAL validi. `Demo_mode` può disabilitare comandi rischiosi. `Safe_mode` blocca tutto tranne status e clear autorizzato.

La ventiquattresima risorsa è la scalabilità locale. Un singolo nodo non deve essere progettato solo per un test isolato. Deve poter stare in uno swarm con molti nodi. Questo significa payload compatti, heartbeat non troppo frequenti, comandi chiari, stato minimo e possibilità di ridurre telemetria. Se ogni nodo invia troppi dati, lo swarm non scala. MicroBot OS deve quindi essere parsimonioso già dal singolo nodo.

Una regola di scalabilità locale è:

ogni nodo deve inviare solo ciò che serve al livello di telemetria corrente.

In uno swarm da pochi nodi, `telemetry_debug` è accettabile. In uno swarm grande, serve `telemetry_basic` o aggregata. La gestione delle risorse deve preparare questo passaggio.

La venticinquesima risorsa è la robustezza contro input errati. Il nodo deve trattare payload malformati, comandi sconosciuti, parametri fuori range, target sbagliato, `phase_id` non valido, `command_id` duplicato. Questo è gestione risorse perché input errati possono consumare CPU, memoria o causare stati incoerenti. MicroBot OS deve validare e rifiutare con messaggi chiari.

Controlli input:

`schema` valido;

`command_type` supportato;

parametri nel range;

target corretto;

phase compatibile;

`expires_at` non scaduto;

capability presente;

safety ok.

Se fallisce, il nodo non deve crashare. Deve rispondere con rejection strutturata. Questa robustezza è essenziale per uno swarm.

La ventiseiesima risorsa è la configurabilità. Un nodo deve poter avere parametri modificabili, ma non troppo liberamente. Cambiare `max_duty` o `cooldown` durante runtime può essere pericoloso. Alcune configurazioni devono richiedere `diagnostic_mode` o `reboot`. Altre possono essere cambiate live, come `telemetry_level`. MicroBot OS deve distinguere configurazioni runtime-safe da configurazioni critiche.

Classi di configurazione:

`runtime_safe`: `telemetry_level`, LED brightness entro limiti;

`runtime_controlled`: heartbeat interval, role LED mode;

`diagnostic_only`: test PWM basso, sensor calibration;

`boot_only`: pin_map, hardware_profile;

`locked`: safety hard limits, salvo firmware update.

Questa distinzione evita modifiche rischiose durante run attive.

La ventisettesima risorsa è la persistenza. Alcuni dati devono sopravvivere al reset, altri no. `node_id` e `hardware_profile` possono essere persistenti. `boot_id` deve cambiare. runtime role non dovrebbe essere persistente automaticamente. `last_error` può essere salvato se possibile. `reset_count` può essere persistente. MicroBot OS deve decidere cosa viene salvato e cosa viene ricostruito.

Dati persistenti possibili:

node_id;

hardware_profile;

firmware_version;

configuration;

reset_count;

last_reset_reason;

calibration data.

Dati non persistenti:

current_role;

current_phase;

command_buffer;

sync_offset;

thermal_load stimato, salvo memoria dedicata;

active output state.

Dopo reset, il nodo deve entrare in recovery perché molti dati runtime sono persi. Questa scelta è sicura.

La ventottesima risorsa è la leggibilità dello stato per l'esterno. Una risorsa gestita ma invisibile è difficile da controllare. MicroBot OS deve esporre un resource_status: memoria disponibile se possibile, task health, output state, telemetry level, capabilities, buffers, error counters. Non serve sempre inviare tutto, ma in diagnostic_mode deve essere interrogabile.

Un resource_status può contenere:

free_memory se disponibile;

loop_time_ms;

task_health;

command_buffer_used;

event_buffer_used;

telemetry_level;

active_outputs;

cooldown_status;

capabilities_available;

error_counters.

Questo aiuta molto nel debug. Se command_buffer è pieno, si vede. Se output è ancora active, si vede. Se loop_time è alto, si vede.

La ventinovesima risorsa è la semplicità. Sembra paradossale, ma anche la semplicità è una risorsa. Un MicroBot OS troppo complicato per il livello hardware attuale diventa difficile da mantenere. La gestione risorse deve quindi evitare astrazioni inutili nelle prime versioni. L'obiettivo non è creare un OS enorme, ma un firmware ordinato e scalabile. Ogni risorsa va gestita quanto basta per sicurezza, debug e crescita.

Una regola pratica è:

aggiungere gestione solo dove esiste un rischio reale di conflitto, spreco, instabilità o necessità futura chiara.

Questo mantiene il progetto concreto. LED, heartbeat, safety, output, comandi e telemetria meritano subito gestione. Funzioni più avanzate possono aspettare.

La trentesima risorsa è la documentazione delle risorse. Ogni nodo deve avere una scheda risorse: pin usati, task, intervalli, capabilities, limiti, buffer, output, sensori, profili energetici. Questa scheda rende il nodo comprensibile. È molto utile per repository, debug e sviluppo hardware.

Una scheda risorse può includere:

hardware_profile;

pin_map;

tasks;

task_intervals;

outputs;

sensors;

communication;

telemetry_profile;

safety_limits;

energy_limits;

thermal_limits;

buffer_sizes;

supported_commands;

known_limitations.

Questa documentazione deve restare aggiornata con le versioni. Un nodo v0.4 LED-only avrà scheda diversa da un nodo v1.3 con PWM sicuro.

In conclusione, la gestione delle risorse è il livello di MicroBot OS che assicura che tempo, memoria, energia, pin, PWM, comunicazione, buffer, task, sensori, attuatori, configurazioni, capabilities e stati interni vengano usati in modo controllato. Essa evita conflitti, blocchi, sprechi, reset, comandi fuori contesto e output non sicuri. Un nodo MicroBot non può essere progettato come una sequenza casuale di funzioni: deve amministrare ciò che possiede. La gestione delle risorse permette al firmware di diventare un runtime embedded affidabile, capace di lavorare dentro uno swarm senza perdere reattività, sicurezza o osservabilità. Il punto successivo, dedicato all'astrazione hardware, approfondirà come MicroBot OS possa separare la logica del nodo dai dettagli fisici di pin, sensori, LED, PWM, bobine e versioni hardware, rendendo il sistema più modulare, portatile e pronto a evolvere tra prototipi diversi.

14.4 Astrazione hardware

L'astrazione hardware rappresenta il livello di MicroBot OS che separa la logica operativa del nodo dai dettagli fisici dei componenti elettronici utilizzati. Dopo aver definito la gestione delle risorse, diventa necessario spiegare come il firmware possa restare ordinato anche quando cambiano pin, LED, sensori, canali PWM, driver, bobine, alimentazione o versioni del prototipo. Senza astrazione hardware, ogni modifica fisica obbligherebbe a riscrivere parti del codice principale. Con una buona astrazione, invece, il sistema può mantenere la stessa logica di alto livello mentre il livello più basso si occupa di adattarsi al circuito reale.

Nel progetto MicroBot, l'astrazione hardware è fondamentale perché il sistema è destinato a evolvere per prototipi successivi. Una prima versione può usare soltanto LED su breadboard. Una seconda può usare più LED per rappresentare ruoli diversi. Una terza può usare PWM per simulare intensità o attuazione. Una quarta può introdurre sensori. Una quinta può aggiungere un driver per bobina. Una sesta può integrare batteria, misura di tensione o sensore di temperatura. Se ogni versione hardware viene gestita direttamente dentro la logica del controller locale, il firmware diventa fragile e difficile da mantenere. MicroBot OS deve invece creare una separazione: sopra c'è la logica del nodo; sotto c'è il modo specifico in cui quella logica viene tradotta in segnali elettrici.

Il principio generale dell'astrazione hardware può essere espresso così:

la logica MicroBot deve chiedere funzioni operative, non manipolare direttamente componenti fisici.

Questo significa che il codice alto non dovrebbe dire “porta GPIO 23 a HIGH” in ogni punto del firmware. Dovrebbe dire “accendi LED di stato”, “imposta LED ruolo”, “porta uscita PWM a duty sicuro”, “spegni tutte le uscite critiche”, “leggi batteria”, “leggi temperatura”, “ottieni motivo del reset”. Il dettaglio del pin, del canale PWM, della risoluzione, del sensore o del profilo hardware deve stare nel livello HAL, cioè Hardware Abstraction Layer. In questo modo, se il LED passa da GPIO 23 a GPIO 2, la logica principale non cambia: cambia soltanto la configurazione hardware.

L'Hardware Abstraction Layer, o HAL, è quindi il ponte tra MicroBot OS e il circuito reale. Esso espone funzioni semplici e stabili verso il resto del firmware, mentre internamente conosce pin, canali, librerie, modalità e limiti. Il `command_manager` non deve sapere quale canale PWM viene usato. Il `safety_manager` non deve sapere il pin esatto della bobina. Il `telemetry_manager` non deve sapere come viene letta la batteria. Ogni modulo usa funzioni astratte, e la HAL traduce quelle funzioni in operazioni hardware.

Una HAL MicroBot può esporre funzioni come:

```
hal_init;  
  
hal_safe_outputs_off;  
  
hal_set_status_led;  
  
hal_set_role_led;  
  
hal_set_pwm_output;  
  
hal_stop_pwm_output;  
  
hal_read_battery_voltage;  
  
hal_read_temperature;  
  
hal_get_reset_reason;  
  
hal_get_uptime_ms;  
  
hal_get_hardware_profile.
```

Queste funzioni costituiscono il linguaggio basso livello del nodo. Anche se internamente possono cambiare, l'interfaccia verso MicroBot OS resta stabile. Questa stabilità è molto importante perché permette di sviluppare `command_manager`, `telemetry_manager`, `scheduler` e `safety_manager` senza riscriverli a ogni prototipo.

Il primo elemento dell'astrazione hardware è il profilo hardware. Un `hardware_profile` descrive che tipo di nodo si sta usando. Un nodo LED-only non ha le stesse capacità di un nodo PWM. Un nodo con sensore batteria non ha le stesse capacità di uno alimentato via USB senza misura. Un nodo con bobina sperimentale richiede limiti molto più severi. MicroBot OS deve quindi sapere quale profilo è attivo. Il profilo hardware genera le capabilities e definisce quali funzioni possono essere abilitate.

Esempi di profili hardware possono essere:

```
LED_NODE_V0;  
  
MULTI_LED_NODE_V0;  
  
PWM_LED_NODE_V0;  
  
SENSOR_NODE_V0;  
  
COIL_EXPERIMENTAL_NODE_V0;  
  
HYBRID_NODE_V0;  
  
SIMULATED_NODE_PROFILE.
```

Un `LED_NODE_V0` può supportare `status_led`, `role_led` e `heartbeat`. Un `PWM_LED_NODE_V0` può supportare `pwm_output` su LED, ma non bobina reale. Un `COIL_EXPERIMENTAL_NODE_V0` può supportare `coil_pwm`, ma solo con `safety_manager`, `cooldown`, `watchdog`, `auto-off` e limiti severi. Un `HYBRID_NODE_V0` può avere `heartbeat`

hardware e posizione simulata. Questa distinzione evita che il controller assegni comandi non supportati.

Il secondo elemento è il pin map. Il pin map è la tabella che collega funzioni logiche a pin fisici. Per esempio, `status_led_pin` può essere GPIO 2, `role_led_pin` GPIO 23, `pwm_output_pin` GPIO 5, `battery_adc_pin` un ingresso analogico. Il resto del firmware non deve usare questi numeri direttamente. Deve richiamare funzioni HAL. Il pin map può essere definito nel codice, in un file di configurazione o in un profilo compilato. Nelle prime versioni può essere statico, ma deve essere documentato.

Un pin map concettuale può includere:

```
status_led_pin;  
  
role_led_pin;  
  
led_array_pins;  
  
pwm_output_pin;  
  
coil_driver_pin;  
  
battery_adc_pin;  
  
temperature_sensor_pin;  
  
button_input_pin;  
  
debug_pin;  
  
reserved_pins.
```

Ogni pin deve avere una funzione chiara. Se un pin è riservato, non deve essere usato accidentalmente. Se due funzioni chiedono lo stesso pin, la configurazione deve essere considerata invalida. Questo controllo è importante perché molti bug hardware nascono da pin mapping confuso.

Il terzo elemento è l'inizializzazione sicura dell'hardware. Quando il nodo si avvia, MicroBot OS deve inizializzare la HAL prima di accettare comandi. La HAL deve impostare tutti i pin critici in stato sicuro. I LED possono essere spenti o impostati in boot pattern controllato. I PWM devono essere duty zero. I driver devono essere disabilitati. Le bobine devono essere OFF. Solo dopo questa inizializzazione il nodo può connettersi, inviare heartbeat e ricevere comandi. Questa sequenza è una parte fondamentale del Safety Layer applicata all'hardware.

Una sequenza corretta è:

```
avvio del microcontrollore;  
  
inizializzazione HAL;  
  
impostazione pin critici su stato sicuro;  
  
spegnimento PWM e driver;
```

inizializzazione LED di stato;

lettura reset_reason se disponibile;

inizializzazione servizi MicroBot OS;

avvio comunicazione;

invio boot_event.

Questa sequenza protegge il nodo anche dopo reset improvvisi. Se il firmware si riavvia durante un test, il primo effetto deve essere spegnere le uscite critiche, non riattivarle. L'astrazione hardware deve rendere questa azione semplice e centralizzata attraverso hal_safe_outputs_off.

Il quarto elemento è l'astrazione dei LED. Nei primi prototipi MicroBot, i LED sono fondamentali perché rappresentano visivamente nodi, ruoli, heartbeat, safety e stato. Tuttavia, i LED non devono essere gestiti direttamente in ogni modulo. Il role_manager non deve conoscere il pin del LED. Deve chiedere “mostra ruolo leader” o “mostra ruolo connector”. Il led_service o la HAL traduce questo in lampeggio, colore, intensità o pattern.

Una astrazione LED può includere:

set_status_led(state);

set_role_indicator(role);

blink_heartbeat();

show_error_code(error_code);

show_safety_state(safety_state);

set_led_brightness(level);

turn_all_leds_off().

Questa astrazione permette di cambiare da un singolo LED a più LED, o da LED normali a LED RGB, senza cambiare la logica del sistema. Se in futuro il nodo userà LED RGB, la funzione show_safety_state potrà usare rosso per safety, giallo per warning, verde per ok. Se il nodo ha solo un LED, userà pattern di lampeggio. La logica superiore resta uguale.

Il quinto elemento è l'astrazione del PWM. Il PWM può essere usato inizialmente su LED, poi eventualmente su driver o bobine. Proprio perché può controllare carichi più delicati, deve essere astratto e limitato. La logica alta non deve impostare direttamente duty e frequenza senza validazione. Deve chiedere un output PWM controllato, e la HAL o l'output_manager applicano limiti, durata massima, auto-off e cooldown.

Una astrazione PWM può includere:

hal_pwm_init(channel, pin, frequency, resolution);

hal_pwm_set_duty(channel, duty);

hal_pwm_stop(channel);

```
hal_pwm_get_state(channel);
```

```
hal_pwm_safe_stop_all();
```

```
hal_pwm_apply_limited_output(requested_duty, max_duty, duration_ms).
```

In MicroBot OS, la HAL non deve essere l'unico livello di safety: il safety_manager deve validare prima. Tuttavia, la HAL deve comunque essere prudente. Se riceve duty fuori range, deve limitarlo o rifiutarlo. Se riceve un canale non inizializzato, deve generare errore. La sicurezza deve essere ridondante.

Il sesto elemento è l'astrazione delle bobine o degli attuatori magnetici. Anche se nelle prime versioni le bobine possono essere simulate o future, la documentazione deve prevedere una struttura coerente. Una bobina non deve essere trattata come un LED. Ha vincoli termici, energetici e di sicurezza molto più severi. L'astrazione deve quindi distinguere tra output visuale e output attuativo. Un comando coil_pulse deve passare da safety_manager, output_manager e HAL. Deve avere durata limitata, duty limitato, cooldown e auto-off.

Una astrazione bobina può includere:

```
coil_init;
```

```
coil_enable_limited;
```

```
coil_disable;
```

```
coil_get_state;
```

```
coil_get_cooldown;
```

```
coil_safe_off;
```

```
coil_set_test_mode;
```

```
coil_block_due_to_thermal;
```

```
coil_block_due_to_energy.
```

Questa interfaccia permette di implementare un attuatore magnetico futuro senza contaminare tutto il firmware. Se per ora la bobina è simulata, si può avere una HAL virtuale che registra l'evento ma non attiva hardware. Quando l'hardware reale sarà pronto, la stessa interfaccia potrà controllare un driver fisico.

Il settimo elemento è l'astrazione dei sensori. Un nodo futuro può avere sensore batteria, temperatura, corrente, prossimità, campo magnetico, IMU o altri. Ogni sensore deve essere gestito da una interfaccia astratta. La logica del nodo non deve sapere se la temperatura arriva da un sensore reale, da stima o da simulazione. Deve leggere thermal_state o temperature_value con source. Questo permette di passare progressivamente da dati simulati a dati misurati.

Una astrazione sensore può includere:

```
sensor_init;
```

```
sensor_read;
```

sensor_get_status;

sensor_get_source;

sensor_get_confidence;

sensor_calibrate;

sensor_is_available.

Per esempio, battery_manager può chiedere hal_read_battery_voltage. Se il nodo non ha sensore batteria, la HAL restituisce unavailable. Se il valore è stimato, restituisce source = estimated. Se è reale, source = measured. Il resto del sistema può applicare policy conservative in base alla fonte.

L'ottavo elemento è l'astrazione della batteria. La batteria può essere assente, simulata, stimata o misurata. Un nodo alimentato via USB può avere power_source = USB_DEBUG. Un nodo con batteria ma senza misura può avere energy_source = estimated. Un nodo con partitore ADC può leggere battery_voltage. Un nodo con modulo power management può avere dati più ricchi. MicroBot OS deve usare una interfaccia comune.

Una astrazione batteria può includere:

hal_power_get_source;

hal_battery_read_voltage;

hal_battery_get_level_estimate;

hal_battery_is_charging;

hal_battery_get_status;

hal_battery_source_type.

Questa astrazione permette di scrivere energy_manager una volta e adattarlo a hardware diversi. Se il dato non è disponibile, energy_state può diventare energy_unknown e il nodo si comporta in modo conservativo. Se è misurato, può essere più preciso.

Il nono elemento è l'astrazione della temperatura. Lo stato termico può essere misurato, stimato o simulato. Un nodo con sensore può leggere temperature_c. Un nodo senza sensore può stimare thermal_load in base ai comandi. Un nodo simulato può avere thermal_load virtuale. Anche qui, l'interfaccia deve essere comune.

Una astrazione termica può includere:

hal_temperature_read;

thermal_model_update;

thermal_get_state;

thermal_get_source;

thermal_get_cooldown_remaining;

thermal_reset_estimate;

thermal_block_output;

thermal_release_output_if_safe.

Il safety_manager non deve dipendere dal tipo di sensore. Deve sapere se thermal_state è ok, warning, lock, recovery o unknown. La HAL e il thermal_manager si occupano del modo in cui quello stato viene ottenuto.

Il decimo elemento è l'astrazione della comunicazione. Un nodo può comunicare via seriale, Wi-Fi UDP, TCP, WebSocket, ESP-NOW, BLE o altro in futuro. La logica dei comandi non deve dipendere direttamente dal mezzo. Il communication_adapter deve fornire funzioni comuni: ricevi messaggio, invia messaggio, verifica connessione, aggiorna last_controller_seen. Questo permette di cambiare protocollo senza riscrivere command_manager e telemetry_manager.

Una astrazione comunicativa può includere:

comm_init;

comm_receive_message;

comm_send_message;

comm_is_connected;

comm_get_status;

comm_get_last_error;

comm_get_last_controller_seen;

comm_set_mode.

Nelle prime versioni, comm_receive_message può leggere da seriale. In una versione Wi-Fi può leggere da socket. In una simulazione può leggere da una coda interna. Il command_manager riceve comunque un messaggio normalizzato. Questa è astrazione hardware e protocollo insieme.

L'undicesimo elemento è l'astrazione del reset reason. Molti microcontrollori permettono di sapere se il reset è stato power_on, watchdog, brownout, software o unknown. MicroBot OS deve accedere a questa informazione tramite HAL. Il controller non deve dipendere da dettagli specifici del microcontrollore. Il nodo deve convertire il reset reason in valori standard.

Valori standard possono essere:

power_on_reset;

watchdog_reset;

brownout_reset;

software_reset;

manual_reset;

unknown_reset.

La HAL si occupa di leggere il registro specifico, mentre MicroBot OS usa la categoria standard. Questo permette di mantenere coerenza anche cambiando hardware.

Il dodicesimo elemento è l'astrazione del tempo locale. Il nodo deve conoscere uptime_ms, timestamp locale, intervalli e timer. Anche questo deve essere astratto, perché diverse piattaforme possono avere funzioni diverse per il tempo. MicroBot OS deve usare una funzione comune come os_time_ms o hal_get_uptime_ms. Questo evita dipendenze sparse da millis(), micros() o funzioni specifiche.

Una astrazione temporale può includere:

hal_time_ms;

hal_time_us;

hal_delay_safe, solo se ammesso;

timer_has_elapsed;

timer_reset;

uptime_ms.

Nel codice MicroBot OS, i task devono usare questa astrazione per restare portabili. Inoltre, il tempo locale deve essere usato per scheduler, heartbeat, timeout, cooldown e watchdog. È una risorsa trasversale.

Il tredicesimo elemento è l'astrazione dei profili hardware in relazione alle capabilities. Le capabilities non devono essere scritte a mano in modo incoerente. Devono derivare dal profilo hardware e dallo stato corrente. Per esempio, se hardware_profile = LED_NODE_V0, capabilities installate includono led_status e heartbeat. Se hardware_profile = PWM_LED_NODE_V0, include pwm_output_led. Se hardware_profile = COIL_EXPERIMENTAL_NODE_V0, include coil_pwm ma solo se safety_manager è attivo. MicroBot OS deve generare o dichiarare capabilities in modo coerente.

Una derivazione può essere:

hardware_profile → installed_capabilities;

safety_state + energy_state + thermal_state → available_capabilities;

thermal_lock o low_energy → blocked_capabilities.

Questa distinzione è essenziale. Un nodo può avere una capability installata ma non disponibile. L'astrazione hardware fornisce ciò che esiste; il Safety Layer decide cosa è utilizzabile ora.

Il quattordicesimo elemento è l'astrazione tra hardware reale e hardware simulato. Il simulatore può usare una HAL virtuale. Le funzioni sono le stesse, ma invece di controllare pin reali aggiornano uno stato simulato. Questo è potentissimo perché permette di testare MicroBot OS in ambiente software. Il command_manager può inviare set_led, il safety_manager può bloccare pwm, il telemetry_manager può inviare status, senza hardware fisico. La stessa architettura funziona con HAL reale o virtuale.

Una HAL virtuale può implementare:

`virtual_set_led;`

`virtual_set_pwm;`

`virtual_read_battery;`

`virtual_read_temperature;`

`virtual_get_reset_reason;`

`virtual_safe_outputs_off;`

`virtual_comm_send;`

`virtual_comm_receive.`

Questo permette di creare nodi OS-like nella simulazione. È un ponte diretto tra Sezione 12 e Sezione 14. La simulazione non è soltanto grafica: può simulare anche il sistema operativo del nodo.

Il quindicesimo elemento è l'astrazione per nodi ibridi. Un nodo ibrido può avere alcune funzioni reali e altre simulate. Per esempio, heartbeat reale, LED reale, posizione simulata, energia stimata, temperatura simulata. L'astrazione hardware deve permettere fonti miste. Ogni campo deve avere source. Il nodo ibrido può essere visto come combinazione tra HAL reale e HAL virtuale.

Un hybrid hardware layer può avere:

`real_comm_adapter;`

`real_led_output;`

`virtual_position;`

`estimated_energy;`

`simulated_thermal;`

`controller_assigned_role;`

`merged_telemetry.`

Questa struttura è molto utile per le demo MicroBot attuali e prossime. Permette di mostrare il sistema completo senza aspettare che ogni capacità sia fisicamente realizzata. L'importante è indicare chiaramente cosa è reale e cosa è simulato.

Il sedicesimo elemento è l'astrazione della configurazione hardware. Un nodo deve poter dichiarare la propria configurazione: `hardware_profile`, `pin_map`, `enabled_sensors`, `enabled_outputs`, `safety_limits`, `firmware_version`. Questa configurazione può essere compilata nel firmware o caricata. In ogni caso, deve essere leggibile da `request_status`. Il controller deve sapere con che tipo di nodo sta parlando.

Un `hardware_config` payload può contenere:

hardware_profile;
pin_map_version;
enabled_outputs;
enabled_sensors;
supported_pwm_channels;
safety_limits;
power_source_type;
temperature_source_type;
battery_source_type;
hardware_revision.

Questi dati aiutano a evitare comandi sbagliati. Se il controller vede hardware_profile = LED_NODE_V0, non invia coil_pulse. Se vede temperature_source = unavailable, evita attuazioni termicamente critiche. Questa è architettura difensiva.

Il diciassettesimo elemento è l'astrazione delle uscite sicure. Indipendentemente dall'hardware specifico, deve esistere una funzione che mette tutto in stato sicuro. Questa funzione è più importante di molte altre. Deve spegnere PWM, disabilitare driver, spegnere bobine, portare LED in stato di sicurezza e aggiornare output_state. Deve essere chiamabile da emergency_stop, watchdog recovery, thermal_lock, critical_energy, controller_lost e boot.

La funzione concettuale è:

hal_safe_outputs_off()

Questa funzione deve essere semplice, affidabile e prioritaria. Ogni profilo hardware deve implementarla. Anche se il nodo ha solo LED, spegne o imposta LED sicuri. Se ha PWM, duty zero. Se ha driver, disable. Se ha bobina, off. Non deve fallire silenziosamente.

Il diciottesimo elemento è l'astrazione dei limiti hardware. Ogni hardware ha limiti: duty massimo, corrente massima, durata massima, frequenza PWM, tensione ammessa, temperatura, pin disponibili. Questi limiti non devono essere dispersi nel codice. Devono essere definiti in hardware_profile o safety_limits. Il safety_manager li usa per validare comandi. La HAL li usa come seconda protezione.

Esempi di limiti:

max_pwm_duty;
max_pulse_duration_ms;
min_cooldown_ms;
max_active_outputs;
max_led_brightness;

battery_voltage_min;

temperature_warning_threshold;

temperature_lock_threshold;

communication_timeout_ms.

Questi limiti possono variare tra prototipi. Un LED PWM può tollerare certe cose; una bobina no. L'astrazione hardware permette di associare limiti al profilo giusto.

Il diciannovesimo elemento è l'astrazione delle unità di misura. I sensori e i modelli devono usare unità chiare: volt, gradi Celsius, millisecondi, duty percentuale, ampere se misurati, percentuale batteria. La HAL deve normalizzare le unità. Se un ADC restituisce valore grezzo, la HAL o energy_manager deve convertirlo in voltage_v o indicare raw_adc_value. Il resto del sistema deve usare unità standard.

Una buona pratica è:

battery_voltage_v;

temperature_c;

duration_ms;

frequency_hz;

duty_percent;

current_a;

energy_percent;

cooldown_remaining_ms.

Questo evita confusione. Un valore senza unità è pericoloso. In documentazione tecnica, le unità sono parte del dato.

Il ventesimo elemento è l'astrazione degli errori hardware. Un sensore può non rispondere. Un pin può essere non configurato. Un PWM può fallire inizializzazione. La lettura batteria può essere unavailable. La HAL deve convertire questi problemi in errori standard:

HAL_SENSOR_UNAVAILABLE, HAL_PWM_INIT_FAILED, HAL_INVALID_PIN,
HAL_OUTPUT_BLOCKED, HAL_UNSUPPORTED_CAPABILITY. Il resto del sistema può reagire.

Una struttura errore HAL può contenere:

hal_error_code;

module;

severity;

affected_resource;

recoverable;

recommended_action.

Per esempio, se temperature_sensor unavailable e il nodo ha coil_pwm, recommended_action può essere block_coil_output. Questo collega errore hardware e safety.

Il ventunesimo elemento è l'astrazione delle versioni hardware. Un prototipo può avere hardware_revision = 0.1, un altro 0.2. La stessa funzione può richiedere pin diversi o limiti diversi. MicroBot OS deve dichiarare hardware_revision e usarla per caricare profilo corretto. Questo è essenziale quando il progetto avrà più schede o breadboard.

Una versione hardware può descrivere:

nome board;

revision;

pin_map;

componenti presenti;

limiti;

note di cablaggio;

funzioni supportate;

funzioni non supportate.

Questa documentazione aiuta moltissimo. Se un bug appare solo su una board, si può confrontare profili. Se una board non ha un sensore, il sistema lo sa.

Il ventiduesimo elemento è l'astrazione del livello fisico rispetto al livello logico. Il livello fisico riguarda pin, tensioni, duty, sensori, output. Il livello logico riguarda ruolo, pattern, mode, safety_state. L'astrazione hardware deve impedire che questi livelli si mescolino troppo. Il ruolo connector non deve direttamente accendere pin. Il ruolo connector può richiedere una funzione operativa, che passa da safety e output manager, poi HAL. Questa catena mantiene ordine.

La catena corretta è:

controller assegna ruolo;

role_manager aggiorna ruolo;

pattern logic richiede azione;

command_manager riceve comando;

safety_manager valida;

output_manager prepara output;

HAL controlla hardware.

Se un modulo salta questi passaggi, il sistema perde sicurezza. L'astrazione hardware è quindi anche disciplina architetturale.

Il ventitreesimo elemento è l'astrazione per test. La HAL permette di testare il software senza hardware reale attraverso mock o virtual HAL. Per esempio, si può testare che emergency_stop chiami hal_safe_outputs_off. Si può testare che un comando PWM venga rifiutato se duty supera il limite. Si può testare che temperature_unavailable blocchi coil. Questi test sarebbero più difficili se il codice parlasse direttamente ai pin.

Test utili:

hal_init_sets_safe_outputs;

safe_outputs_off_disables_pwm;

invalid_pin_detected;

pwm_limit_applied;

battery_unavailable_sets_energy_unknown;

temperature_unavailable_blocks_coil;

virtual_hal_records_output_state.

Questi test rendono MicroBot OS più affidabile. La HAL non è solo astrazione; è anche testabilità.

Il ventiquattresimo elemento è l'astrazione per portabilità. Oggi il nodo può essere ESP32. In futuro potrebbe essere Arduino, RP2040, STM32 o altro. Se la logica alta usa HAL, portare MicroBot OS su un nuovo microcontrollore è più realistico. Si riscrive la HAL per la nuova piattaforma, ma state_manager, command_manager, safety_manager e telemetry_manager possono restare simili. Questo è un vantaggio enorme per un progetto che vuole evolvere.

La portabilità richiede:

interfacce HAL stabili;

tipi di dato semplici;

nessuna dipendenza diretta da librerie specifiche nel codice alto;

separazione tra platform layer e logic layer;

test comuni.

Naturalmente, non tutto sarà portabile senza modifiche. Ma l'architettura riduce il costo della migrazione.

Il venticinquesimo elemento è l'astrazione per la documentazione hardware. Ogni profilo deve avere un documento. Non basta avere pin nel codice. Serve spiegare quali componenti sono collegati, quali funzioni supportano, quali limiti hanno, quali rischi esistono e quali test sono stati fatti. Questo è particolarmente importante se il progetto viene inviato ad altri o pubblicato su GitHub.

Un documento `hardware_profile.md` può includere:

nome profilo;

scopo;

componenti;

pin map;

capabilities;

limiti;

safety requirements;

telemetria disponibile;

test eseguiti;

limitazioni;

foto o schema se disponibile.

Questa documentazione collega hardware e software. Chi legge capisce cosa può fare quel nodo.

Il ventiseiesimo elemento è l'astrazione dei componenti assenti. Una cosa importante: non tutti i nodi avranno tutti i componenti. La HAL deve gestire absent components senza crash. Se un nodo non ha batteria misurata, `hal_read_battery_voltage` restituisce `unavailable`. Se non ha sensore temperatura, `thermal_source` = `estimated` o `unavailable`. Se non ha PWM, `hal_set_pwm_output` restituisce `unsupported_capability`. Questo comportamento deve essere normale, non errore catastrofico.

Una risposta standard può essere:

```
result = unsupported;
```

```
reason = capability_missing;
```

```
capability = pwm_output;
```

```
hardware_profile = LED_NODE_V0.
```

Questo permette al controller di adattarsi. Un nodo LED-only può comunque partecipare a demo visuali. Non deve essere escluso solo perché non supporta bobina.

Il ventisettesimo elemento è l'astrazione dei livelli di rischio. Alcune funzioni hardware sono low-risk, altre high-risk. LED status è low-risk. PWM su LED è medium-risk. Bobina o driver è high-risk. Batteria e ricarica possono essere high-risk. La HAL e il profilo hardware devono dichiarare il livello di rischio delle funzioni. Questo aiuta il `safety_manager` e la dashboard.

Una classificazione può essere:

```
risk_level = visual_only;
```

risk_level = low_power_output;

risk_level = pwm_test;

risk_level = actuator_control;

risk_level = thermal_sensitive;

risk_level = energy_sensitive.

Un comando verso funzione actuator_control deve richiedere più controlli rispetto a visual_only. Questa è una forma avanzata ma utile di astrazione hardware.

Il ventottesimo elemento è l'astrazione per modalità dimostrativa. In demo, si può voler mostrare un'azione senza attivare davvero hardware rischioso. La HAL può avere simulated_actuation_mode o dry_run. In dry_run, un comando viene validato e registrato, ma non attiva fisicamente l'output. Questo è molto utile per testare controller, dashboard e telemetria senza rischi.

Modalità possibili:

real_output_mode;

dry_run_mode;

simulation_mode;

hardware_visual_only_mode;

actuation_locked_mode.

Se actuation_locked_mode è attivo, ogni comando attuativo viene rifiutato o simulato. La dashboard può mostrare che l'attuazione è bloccata. Questo permette demo sicure.

Il ventinovesimo elemento è l'astrazione come garanzia di coerenza futura. Man mano che MicroBot cresce, arriveranno nuovi componenti. Senza HAL, ogni nuova aggiunta rischia di rompere il firmware. Con HAL, ogni nuova aggiunta entra come nuova capability, nuovo profilo, nuova funzione o nuova implementazione. Questo rende il progetto più professionale e scalabile. L'astrazione hardware è quindi un investimento: costa un po' di struttura all'inizio, ma evita caos dopo.

Una regola finale è:

nessuna nuova funzione hardware deve essere integrata senza profilo, capability, limite, telemetria e safe-off.

Questo principio mantiene il sistema sicuro. Se si aggiunge un sensore, deve avere source e stato. Se si aggiunge un output, deve avere safe_off e limiti. Se si aggiunge un driver, deve avere blocchi safety. Se si aggiunge una batteria, deve avere energy policy. Ogni hardware entra nel sistema con disciplina.

Il trentesimo elemento è la roadmap dell'astrazione hardware. Nella prima versione, basta separare pin LED e funzioni led_set. Poi si aggiunge pin_map. Poi HAL per LED. Poi HAL per PWM. Poi hardware_profile. Poi capabilities derivate. Poi sensori. Poi safe_outputs_off centralizzato. Poi HAL

virtuale per simulazione. Poi profili hardware multipli. Poi modalità ibrida. Questa progressione è realistica.

Una roadmap possibile è:

v0.1: costanti pin ordinate;

v0.2: funzioni led astratte;

v0.3: pin_map centralizzato;

v0.4: HAL base per LED e tempo;

v0.5: HAL comunicazione;

v0.6: HAL PWM controllato;

v0.7: hardware_profile e capabilities;

v0.8: HAL sensori opzionali;

v0.9: safe_outputs_off centralizzato;

v1.0: HAL virtuale per simulazione;

v1.1: profili hardware multipli;

v1.2: supporto hybrid_node;

v1.3: astrazione attuatori critici con safety completa.

Questa roadmap permette di costruire astrazione senza rendere subito il firmware troppo complesso.

In conclusione, l'astrazione hardware è il livello di MicroBot OS che permette alla logica del nodo di restare indipendente dai dettagli fisici di pin, LED, PWM, sensori, bobine, batteria, comunicazione e versioni hardware. Essa introduce hardware_profile, pin_map, HAL, capabilities, limiti, safe outputs, sorgenti dei dati e modalità reali, simulate o ibride. Grazie all'astrazione hardware, MicroBot OS può funzionare su prototipi diversi mantenendo la stessa struttura logica. Un nodo LED-only, un nodo PWM, un nodo sensoriale, un nodo ibrido e un nodo futuro con attuazione magnetica possono condividere command_manager, state_manager, telemetry_manager e safety_manager, cambiando soltanto il livello hardware. Questo rende il progetto modulare, portatile, sicuro e pronto a evolvere. Il punto successivo, dedicato allo scheduling, approfondirà come MicroBot OS debba organizzare nel tempo task, heartbeat, telemetria, safety check, output update, diagnostica e comunicazione, evitando blocchi e garantendo che le funzioni critiche vengano sempre eseguite con priorità corretta.

14.5 Scheduling

Lo scheduling rappresenta il livello di MicroBot OS dedicato all'organizzazione temporale delle attività interne del nodo. Dopo aver definito l'astrazione hardware, cioè il modo in cui il firmware separa la logica dai dettagli fisici di pin, LED, PWM, sensori e attuatori, diventa necessario spiegare come tutte queste funzioni vengano eseguite nel tempo senza entrare in conflitto. Un MicroBot non deve soltanto “fare cose”: deve farle nel momento giusto, con la frequenza giusta, con la priorità giusta e senza bloccare le funzioni critiche. Lo scheduling è quindi il meccanismo che permette al

nodo di restare vivo, reattivo, sicuro e sincronizzato mentre gestisce comunicazione, telemetria, output, safety, diagnostica e comandi.

Nel progetto MicroBot, lo scheduling è fondamentale perché un nodo embedded ha risorse limitate e deve gestire più responsabilità contemporaneamente. Deve ascoltare comandi dal controller, inviare heartbeat, aggiornare LED o PWM, controllare timeout, verificare se il controller è ancora raggiungibile, aggiornare lo stato energetico e termico, rispettare cooldown, eseguire eventuali azioni temporizzate, alimentare il watchdog e rispondere alla shell diagnostica. Se queste attività vengono inserite in modo casuale nel firmware, il nodo può diventare instabile. Se una funzione usa delay lunghi, le altre restano ferme. Se la telemetria è troppo frequente, può rallentare la ricezione dei comandi. Se la diagnostica blocca il loop, un emergency_stop potrebbe essere processato in ritardo. Lo scheduling serve a evitare proprio questi problemi.

Il principio generale dello scheduling MicroBot può essere espresso così:

ogni attività del nodo deve essere eseguita secondo una frequenza, una priorità e una durata compatibili con la sicurezza del sistema.

Questa frase contiene tre concetti essenziali. La frequenza indica ogni quanto un task deve essere eseguito. La priorità indica quali task devono venire prima quando più attività competono per il tempo. La durata indica quanto un task può occupare il loop senza compromettere gli altri. Uno scheduler ben progettato non serve solo a “ordinare funzioni”, ma a garantire che le attività critiche vengano sempre controllate in tempo.

Nel contesto MicroBot, lo scheduling può iniziare in forma molto semplice. Non è necessario introdurre subito un sistema operativo real-time complesso. In una prima versione, basta uno scheduler cooperativo basato su timestamp. Questo significa che il main loop gira continuamente, controlla il tempo corrente e decide quali task devono essere eseguiti perché il loro intervallo è scaduto. Ogni task deve essere breve e non bloccante. Quando finisce, restituisce il controllo al main loop. Questa architettura è semplice, adatta ai prototipi ESP32/Arduino e coerente con la necessità di mantenere il firmware leggibile.

Una struttura concettuale di scheduler cooperativo può essere:

task_id;

enabled;

interval_ms;

last_run_ms;

priority;

expected_max_duration_ms;

last_duration_ms;

deadline_missed_count;

function_to_execute.

Ogni task viene eseguito solo quando $\text{now} - \text{last_run_ms}$ è maggiore o uguale a interval_ms . Dopo l'esecuzione, last_run_ms viene aggiornato. Se il task impiega troppo, si può incrementare un

contatore di `deadline_missed` o `task_overrun`. Questi dati possono poi essere mostrati nella diagnostica. Anche una struttura semplice come questa porta il firmware a un livello molto più maturo rispetto a un loop pieno di delay e funzioni chiamate senza criterio.

Il primo task fondamentale è il **communication_receive_task**. Questo task controlla se sono arrivati comandi dal controller, dalla seriale o da un altro canale. Deve essere eseguito molto spesso, perché i comandi importanti devono essere ricevuti rapidamente. In particolare, `emergency_stop`, `safe_idle`, `state_sync` e `request_status` non devono restare in attesa per tempi lunghi. Questo task deve essere breve: legge messaggi disponibili, li inserisce nella pipeline del `command_manager` o li processa se sono urgenti, poi torna al loop.

Il `communication_receive_task` deve avere priorità alta perché rappresenta l'ingresso del nodo nel sistema distribuito. Se il nodo non legge comandi in tempo, può restare indietro rispetto al controller. Se non legge `emergency_stop`, può diventare pericoloso. Tuttavia, anche questo task deve evitare blocchi. Non deve aspettare indefinitamente dati dalla rete o dalla seriale. Deve controllare ciò che è disponibile e proseguire. L'attesa bloccante è nemica dello `scheduling embedded`.

Il secondo task fondamentale è il **command_dispatch_task**. Dopo che i comandi sono stati ricevuti, devono essere validati ed eventualmente eseguiti. Questo task si occupa di prendere comandi dal buffer, controllare schema, target, `command_id`, scadenza, fase, ruolo, capabilities e safety. Se il comando è valido, viene inoltrato al modulo corretto. Se non è valido, viene generata una rejection strutturata. Anche questo task deve essere controllato: non deve processare una coda infinita in un solo ciclo, altrimenti potrebbe bloccare safety e `output_update`. Può processare un numero massimo di comandi per iterazione.

Una politica prudente è:

processare subito i comandi critici;

limitare il numero di comandi normali per ciclo;

scartare o rifiutare comandi scaduti;

non eseguire comandi attuativi senza safety check;

generare ack o rejection in modo compatto.

Questa scelta impedisce che un flood di comandi normali impedisca al nodo di eseguire task critici. In uno swarm, il controller deve essere disciplinato, ma anche il nodo deve proteggersi.

Il terzo task fondamentale è il **safety_check_task**. Questo task verifica periodicamente lo stato di sicurezza del nodo: `safety_state`, `thermal_state`, `energy_state`, `cooldown`, output attivi, `controller_lost`, `emergency_stop` e condizioni critiche. È uno dei task più importanti di tutto MicroBot OS. Deve girare con frequenza alta e priorità elevata. Se rileva una condizione non sicura, deve aggiornare lo stato e chiamare `output_manager` o HAL per spegnere uscite critiche.

Il `safety_check_task` non deve dipendere dalla telemetria o dalla dashboard. Deve funzionare localmente. Anche se il controller non risponde, anche se la dashboard è chiusa, anche se la rete cade, il nodo deve poter entrare in `safe_idle`. Questo task è quindi il cuore locale del Safety Layer.

Una sequenza del `safety_check_task` può essere:

verificare `emergency_stop` locale o ricevuto;

verificare controller_lost_timeout;

verificare thermal_lock o thermal_warning;

verificare critical_energy o low_energy;

verificare output oltre durata massima;

verificare cooldown;

verificare task critici e watchdog health;

applicare safe_off_all se necessario;

generare evento safety se lo stato cambia.

Questa sequenza rende il nodo capace di prevenire condizioni pericolose. Lo scheduler deve garantire che questo task non venga saltato per troppo tempo.

Il quarto task è l'**output_update_task**. Questo task gestisce LED, PWM, bobine, timer di spegnimento, duty, animazioni non bloccanti e cooldown. È diverso dal safety_check_task: il safety_check controlla se il sistema è sicuro; output_update applica lo stato agli output fisici. Se un PWM è stato avviato con duration_ms = 100, output_update deve spegnerlo quando il tempo scade. Se un LED deve lampeggiare, output_update cambia stato in base al tempo. Se una bobina è in cooldown, output_update aggiorna il countdown.

Questo task deve essere eseguito frequentemente, soprattutto se ci sono uscite temporizzate. Un output critico non deve restare attivo più del dovuto perché il firmware era occupato a fare altro. Per questo output_update_task ha priorità molto alta. In un nodo con solo LED, il rischio è basso; in un nodo con PWM o bobina, è fondamentale.

Una regola importante è:

ogni uscita critica deve avere uno spegnimento controllato dallo scheduler, non da delay bloccanti.

Questo significa che il firmware non deve fare “accendi PWM, delay, spegni PWM”. Deve fare “accendi PWM con auto_off_time” e poi lasciare che output_update lo spenga quando necessario. Così il loop resta libero di ricevere emergency_stop e aggiornare safety.

Il quinto task è l'**heartbeat_task**. Questo task invia periodicamente un messaggio di presenza al controller o alla dashboard. L'heartbeat deve contenere node_id, boot_id, uptime, mode, role, safety_state, energy_state, thermal_state, sync_status e last_error essenziale. La frequenza non deve essere troppo alta, per non sprecare banda ed energia, ma nemmeno troppo bassa, per non rendere il nodo poco osservabile. Un intervallo tipico iniziale può essere dell'ordine di centinaia di millisecondi o pochi secondi, a seconda del contesto.

Heartbeat non è solo “sono vivo”. È anche lo strumento con cui il controller rileva reset, stato safety, energia, temperatura e sincronizzazione. Tuttavia, se il nodo è in critical_energy, la frequenza può essere ridotta. Se il nodo è in diagnostic_mode, può essere aumentata o accompagnata da status più dettagliati. Quindi heartbeat_task deve essere compatibile con telemetry_level ed energy_state.

Il sesto task è il **telemetry_event_flush_task**. Gli eventi generati localmente, come command_rejected, mode_changed, thermal_warning, energy_low, role_changed o recovery_started, possono essere inviati immediatamente oppure messi in un buffer e inviati appena

possibile. Questo task svuota il buffer degli eventi verso il controller. Gli eventi critici devono avere priorità. Gli eventi informativi possono essere accorpati o inviati con frequenza minore. Questo evita che il nodo invii troppi messaggi singoli.

Una politica possibile è:

eventi critical inviati subito;

eventi warning inviati rapidamente;

eventi info accorpati o inviati nel ciclo normale;

eventi debug inviati solo in `diagnostic_mode`.

Questa politica migliora efficienza e leggibilità. Lo scheduler deve impedire che l'invio di eventi secondari blocchi i task critici.

Il settimo task è il **sync_task**. Questo task gestisce la sincronizzazione con il controller, se prevista. Può aggiornare offset temporale, verificare `last_sync_age`, controllare `sync_status` e segnalare `sync_degraded` o `sync_lost`. Nei pattern che usano comandi temporizzati, questo task è importante. Un nodo può essere online ma non sufficientemente sincronizzato per eseguire `execute_at`. Lo scheduler deve quindi aggiornare periodicamente la qualità della sincronizzazione.

Il `sync_task` può essere meno frequente di `safety_check` e `output_update`, ma deve essere abbastanza regolare. Se `sync_status` diventa `degraded`, il nodo può rifiutare comandi temporizzati critici. Se diventa `lost`, può richiedere `resync` o entrare in modalità limitata. Questo collega scheduling e coordinazione temporale dello swarm.

L'ottavo task è il **controller_lost_task**. Questo task verifica quanto tempo è passato dall'ultimo messaggio valido ricevuto dal controller. Se il tempo supera `controller_lost_timeout_ms`, il nodo deve entrare in `safe_idle` o modalità conservativa. Questo task può essere parte del `safety_check`, ma può anche essere separato per chiarezza. È fondamentale per hardware reale, perché un nodo non deve continuare ad attuare se il controller è sparito.

Una regola centrale è:

se il controller non viene visto entro la soglia, il nodo deve ridurre comportamento e spegnere output non essenziali.

Questo rende il nodo robusto contro crash del backend, rete persa, computer spento o processo interrotto. Lo scheduling assicura che questo controllo venga fatto periodicamente.

Il nono task è il **diagnostic_shell_task**. Questo task gestisce comandi locali dalla shell interna, per esempio tramite seriale. Deve avere priorità più bassa rispetto a `safety`, `output` e comunicazione. La shell è utile, ma non deve mai bloccare il nodo. Se un comando diagnostico richiede molto output, deve essere spezzato o limitato. In modalità `active`, la shell può essere ridotta; in `diagnostic_mode` può essere più ricca. In `low_energy` può essere limitata.

La diagnostic shell deve essere schedulata in modo non bloccante. Non deve restare in attesa indefinita di input. Deve controllare se c'è input, processare una riga, rispondere e tornare. Se la shell stampa un report molto lungo, può interferire con il loop; per questo è meglio mantenere output diagnostici compatti o inviarli a blocchi.

Il decimo task è il **resource_monitor_task**. Questo task controlla lo stato interno del nodo: loop_time, task_duration, buffer usage, event_count, command_buffer_used, memory se disponibile, deadline_missed_count, error counters. Non deve girare troppo spesso, ma è utile per diagnosticare problemi. Se il loop diventa troppo lento o un buffer si riempie, questo task può generare warning.

Metriche utili del resource_monitor_task sono:

main_loop_duration_ms;
max_loop_duration_ms;
task_ouerrun_count;
command_buffer_used;
event_buffer_used;
free_memory, se disponibile;
last_task_delayed;
deadline_missed_count;
watchdog_feed_status.

Questi dati sono molto utili nella shell e nella dashboard. Permettono di capire se il nodo è sotto carico o se un task sta impiegando troppo.

L'undicesimo task è il **energy_update_task**. Questo task aggiorna lo stato energetico del nodo. Se il nodo ha misura batteria, legge la tensione. Se non l'ha, aggiorna una stima o mantiene energy_unknown. Se il nodo è simulato, usa il modello virtuale. In base ai dati, aggiorna energy_state: energy_ok, energy_medium, low_energy, critical_energy. Questo task non deve girare necessariamente ad alta frequenza; l'energia cambia più lentamente rispetto agli output. Tuttavia, se energy_state cambia, deve generare evento.

Una sequenza possibile è:

leggere o stimare energia;
calcolare energy_state;
confrontare con stato precedente;
aggiornare available_capabilities;
ridurre telemetry_level se necessario;
generare energy_state_changed se cambia.

Questo task è collegato alla protezione energetica. Se entra critical_energy, safety_check o energy_update stesso può bloccare output ad alto consumo.

Il dodicesimo task è il **thermal_update_task**. Questo task aggiorna thermal_load o temperatura. Se esiste un sensore, legge il valore. Se non esiste, aggiorna un modello stimato: thermal_load aumenta

con comandi ad alto consumo e diminuisce con `cooling_rate` nel tempo. In base al valore, aggiorna `thermal_state`. Deve gestire hysteresis per evitare oscillazioni tra ok e warning.

Una sequenza possibile è:

leggere temperatura o aggiornare `thermal_load` stimato;

applicare raffreddamento virtuale/stimato;

verificare soglie warning e lock;

gestire hysteresis;

aggiornare cooldown;

bloccare o ripristinare capabilities;

generare `thermal_state_changed` se necessario.

Questo task è molto importante per versioni con PWM o bobina. Può essere meno rilevante nei LED-only node, ma può essere già presente in forma simulata o stimata.

Il tredicesimo task è il **watchdog_task**. In realtà il watchdog non deve essere solo un task separato; deve essere collegato alla salute complessiva del sistema. Tuttavia, MicroBot OS può avere una funzione che verifica se `safety_task`, `output_task`, `communication_task` e scheduler stesso hanno girato correttamente. Solo se i task critici sono ok, il firmware alimenta il watchdog hardware. Se uno dei task critici non viene eseguito entro la finestra prevista, il nodo entra in safe state o lascia intervenire il watchdog.

Una logica corretta è:

verificare `safety_task` recente;

verificare `output_task` recente;

verificare `communication_receive_task` recente;

verificare `main_loop` reattivo;

verificare nessun overrun critico;

se tutto ok, feed watchdog;

se non ok, `safe_outputs_off` e non feed watchdog o entrare in safety.

Questo evita il classico errore di alimentare il watchdog anche quando il firmware non sta facendo correttamente i controlli critici. Il watchdog deve confermare salute reale, non solo loop vivo.

Il quattordicesimo task è il **pattern_local_task**, se il nodo supporta comportamento locale legato al pattern. Nei primi prototipi, il pattern può essere gestito quasi interamente dal controller. Il nodo riceve ruoli e comandi. In versioni più avanzate, il nodo può avere piccole logiche locali: aggiornare LED in base alla fase, mantenere stato di ruolo, eseguire azioni previste in certi slot, controllare `phase_id`. Questo task deve restare leggero. Non deve trasformare il nodo in controller globale.

Una funzione locale può essere:

aggiornare indicatore ruolo;

verificare phase_id;

preparare azione programmata;

controllare se execute_at è arrivato;

annullare azione se comando scaduto;

segnalare phase_mismatch.

Questa logica permette al nodo di partecipare in modo più intelligente alla run senza assumere tutta la complessità dello swarm.

Il quindicesimo task è il **led_visual_task**. Nei prototipi attuali, LED e pattern visuali sono importanti. Tuttavia, gli effetti LED non devono bloccare il loop. Ogni lampeggio deve essere gestito con timestamp. Il led_visual_task aggiorna lo stato dei LED in base a mode, role, safety_state, heartbeat o comandi visuali. Se il nodo entra in safety, il LED pattern deve cambiare. Se entra low_energy, la luminosità o frequenza può ridursi.

Una logica LED non bloccante può gestire:

heartbeat blink;

role blink;

safety blink;

error code blink;

diagnostic blink;

low_energy dimming;

LED off in critical_energy, se necessario.

Questo task è a priorità media o bassa, salvo LED usato come safety indicator. In ogni caso non deve interferire con output critici o communication.

Il sedicesimo concetto dello scheduling è la **priorità**. Non tutti i task hanno lo stesso peso. Se due task devono essere eseguiti nello stesso momento, il sistema deve sapere quale viene prima. In uno scheduler cooperativo semplice, si può ordinare la lista dei task per priorità. I task critici vengono controllati prima. I task accessori vengono eseguiti dopo, se c'è tempo. Questo non è un real-time scheduler sofisticato, ma è già sufficiente per dare struttura.

Una priorità possibile è:

P0: emergency_stop handling e safe_outputs_off;

P1: output_update_task e safety_check_task;

P2: communication_receive_task e controller_lost_task;

P3: command_dispatch_task e watchdog_task;

P4: heartbeat_task e telemetry_event_flush_task;

P5: sync_task, energy_update_task, thermal_update_task;

P6: diagnostic_shell_task e resource_monitor_task;

P7: led_visual_task ed effetti non essenziali.

Questa scala può essere adattata, ma il principio resta: safety e output prima di estetica e diagnostica. Se il sistema è sotto carico, i task bassi possono saltare un ciclo. I task critici no.

Il diciassettesimo concetto è la **frequenza**. Ogni task deve avere un intervallo sensato. Se safety_check gira troppo raramente, il nodo reagisce lentamente. Se heartbeat gira troppo spesso, spreca banda. Se diagnostic_shell gira troppo spesso, consuma tempo inutile. Se output_update gira troppo lentamente, gli output temporizzati possono spegnersi in ritardo. La frequenza deve dipendere dal rischio e dalla velocità con cui cambiano i dati.

Una configurazione iniziale può essere concettualmente:

output_update_task: molto frequente;

safety_check_task: frequente;

communication_receive_task: frequente;

command_dispatch_task: frequente o quasi continuo;

heartbeat_task: periodico moderato;

energy_update_task: meno frequente;

thermal_update_task: medio, più frequente con attuatori;

resource_monitor_task: lento;

diagnostic_shell_task: medio-basso;

led_visual_task: secondo necessità.

I valori esatti devono essere scelti in base all'hardware e testati. La documentazione può indicare valori indicativi, ma il principio è più importante: task critici più frequenti, task informativi meno frequenti.

Il diciottesimo concetto è la **durata massima del task**. Un task non deve solo avere frequenza; deve anche avere durata controllata. Se un task dovrebbe essere leggero ma a volte impiega troppo, crea ritardi. MicroBot OS può misurare last_duration_ms e max_duration_ms per i task principali. Se un task supera expected_max_duration_ms, può generare warning. Questo è molto utile per debug.

Una struttura di health task può contenere:

task_id;

last_run_ms;

last_duration_ms;
max_duration_ms;
expected_max_duration_ms;
overrun_count;
enabled;
health_status.

Se diagnostic_shell_task ha molti overrun, bisogna ridurre output. Se telemetry_event_flush_task impiega troppo, bisogna ridurre payload. Se command_dispatch_task impiega troppo, c'è forse un problema di parsing o buffer. Questa metrica rende lo scheduler osservabile.

Il diciannovesimo concetto è il **deadline miss**. Un task ha una scadenza implicita: deve essere eseguito entro un certo intervallo. Se safety_check dovrebbe girare ogni 50 ms ma passa troppo tempo, c'è deadline_miss. Questo non significa necessariamente fallimento immediato, ma è un segnale di rischio. Per i task critici, molti deadline miss possono portare a safety state o watchdog reset.

Una politica può essere:

deadline miss su task informativo → warning diagnostico;

deadline miss ripetuto su task core → degraded mode;

deadline miss su safety/output → safe_idle o watchdog action.

Questa politica rende lo scheduling parte del Safety Layer. Non basta dire “il nodo è acceso”; deve rispettare i tempi minimi dei task critici.

Il ventesimo concetto è il **task shedding**, cioè la capacità di sospendere task non essenziali in condizioni difficili. Se il nodo è in low_energy, può ridurre LED e telemetria debug. Se è in critical_energy, può disabilitare effetti LED e diagnostica verbose. Se la comunicazione è congestionata, può ridurre eventi info. Se il loop è sovraccarico, può saltare task accessori. Questo permette di conservare risorse.

Una politica di task shedding può essere:

in normal mode: tutti i task abilitati;

in low_energy: ridurre visual_tasks e debug_tasks;

in critical_energy: mantenere solo safety, heartbeat minimo, communication essenziale;

in safety mode: bloccare attuazioni e debug pesante;

in diagnostic mode: abilitare debug ma impedire attuazioni critiche non autorizzate.

Questa gestione rende il nodo adattivo. Lo scheduler non è fisso; cambia in base allo stato operativo.

Il ventunesimo concetto è la **modalità di scheduling per stato**. Ogni mode del nodo può avere una tabella task diversa. In booting, si eseguono init e safe outputs. In idle, heartbeat, communication, shell e LED sono attivi. In active, safety, command, output e sync hanno priorità. In diagnostic, shell e resource monitor aumentano. In safety, output_off e stato sicuro dominano. In low_power, quasi tutto viene ridotto. Questa modalità permette di usare risorse in modo intelligente.

Una mappa concettuale può essere:

booting: init, safe_outputs, identity;

idle: heartbeat, communication, shell, basic telemetry;

active: communication, command, safety, output, sync, telemetry;

diagnostic: shell, status, resource monitor, controlled tests;

safety: safe_outputs, heartbeat minimal, event critical;

recovery: request_status, sync, state_sync, telemetry recovery;

low_power: heartbeat minimal, safety, communication reduced.

Questa mappa aiuta a progettare il firmware. Ogni modalità ha compiti coerenti.

Il ventiduesimo concetto è lo **scheduling degli output temporizzati**. Un comando può dire “accendi PWM per 100 ms” o “blink LED per 3 secondi”. Non si deve bloccare il firmware per quella durata. Si deve registrare un output_state con start_time, duration_ms e auto_off_time. Poi output_update_task spegne quando necessario. Questo modello è essenziale per sicurezza.

Una struttura output temporizzato può contenere:

output_id;

active;

started_at_ms;

duration_ms;

auto_off_at_ms;

current_level;

command_id;

requires_cooldown;

cooldown_until_ms.

Quando now >= auto_off_at_ms, output_update_task spegne l’uscita e avvia cooldown se richiesto. Questo permette al nodo di fare altro mentre l’output è attivo, mantenendo comunque spegnimento garantito.

Il ventitreesimo concetto è lo **scheduling dei comandi futuri**. Alcuni comandi possono essere programmati per un tempo futuro, per esempio execute_at_ms. Questo è utile per sincronizzazione.

Il nodo non deve eseguirli subito. Deve inserirli in una lista di `scheduled_commands`, validare che non siano scaduti e che il `sync_status` sia sufficiente. Quando il tempo arriva, li esegue solo se `safety` e fase sono ancora valide.

Una struttura `scheduled_command` può contenere:

`command_id`;

`command_type`;

`execute_at_ms`;

`expires_at_ms`;

`phase_id`;

`required_sync_status`;

`validated_at_receive`;

`revalidate_before_execute`;

`status`.

La regola importante è `revalidate_before_execute`. Anche se un comando era valido quando ricevuto, potrebbe non esserlo più al momento dell'esecuzione. Il nodo potrebbe essere entrato in `thermal_lock` o `low_energy`. Quindi deve ricontrollare `safety` prima di eseguire. Questo è un punto molto serio.

Il ventiquattresimo concetto è lo **scheduling degli heartbeat e della telemetria adattiva**. Heartbeat e telemetria non devono essere rigidi. Se il nodo è in `active pattern`, può inviare stato più frequente. Se è `idle`, meno frequente. Se è in `warning`, invia evento immediato. Se è in `low_energy`, riduce. Se è in `diagnostic_mode`, aumenta. Lo scheduler deve quindi permettere `interval_ms` dinamici.

Una politica può essere:

`idle heartbeat_interval` = normale;

`active heartbeat_interval` = normale o leggermente più frequente;

`warning telemetry_event` immediata;

`diagnostic telemetry_interval` ridotta per più dettaglio;

`low_energy heartbeat_interval` aumentato;

`critical_energy` solo heartbeat minimo e eventi critici.

Questo rende la telemetria compatibile con energia e banda. Lo scheduler è il meccanismo pratico che applica queste policy.

Il venticinquesimo concetto è il rapporto tra scheduler e **watchdog**. Il watchdog deve essere alimentato solo se lo scheduler sta eseguendo correttamente i task critici. Se lo scheduler si blocca, il watchdog deve intervenire. Se un task critico non gira, il sistema deve considerarlo problema

grave. Lo scheduler deve quindi registrare last_run dei task critici e fornire un health summary al watchdog.

Una condizione watchdog può essere:

safety_check_recent = true;

output_update_recent = true;

communication_recent = true;

main_loop_recent = true;

no_critical_overrun = true.

Solo se queste condizioni sono vere, il nodo alimenta il watchdog. Se una è falsa, può entrare in safe_idle o lasciare reset. Questo rende il watchdog integrato e non decorativo.

Il ventiseiesimo concetto è lo **scheduling della recovery**. Dopo reset, controller_lost, thermal_lock o state_mismatch, il nodo deve seguire una sequenza di recovery. Anche questa sequenza può essere schedulata: prima safe_outputs, poi heartbeat recovery, poi request_status, poi sync, poi state_sync, poi reintegration_ready. Non deve avvenire tutto in un blocco unico. Ogni passo può attendere ack, timeout o condizione.

Una recovery schedulata può avere stati:

recovery_start;

safe_outputs_confirmed;

boot_event_sent;

waiting_for_sync;

waiting_for_state_sync;

capabilities_reported;

ready_for_reintegration;

recovery_complete.

Lo scheduler esegue piccoli passi e controlla timeout. Se un passo fallisce, genera evento e può rimanere in recovery o richiedere intervento. Questo è molto più robusto di una recovery monolitica.

Il ventisettesimo concetto è lo **scheduling nella simulazione**. Anche i nodi virtuali possono usare una logica simile. Un simulated_node può avere task: update_position, update_energy, update_thermal, process_command, emit_telemetry. Questo rende la simulazione più coerente con MicroBot OS reale. La differenza è che nel simulatore il tempo può essere simulation_time invece di real_time. Ma la logica resta simile: task periodici, priorità, stati, eventi.

Questa coerenza è potente perché permette di testare lo scheduler in ambiente virtuale. Si può simulare task overrun, lost heartbeat, delayed command, watchdog reset e recovery. Poi le stesse idee possono essere portate su firmware.

Il ventottesimo concetto è lo **scheduling documentato**. Ogni versione di MicroBot OS dovrebbe documentare i task attivi, i loro intervalli, priorità e scopo. Questo aiuta a capire il firmware e a fare debug. Se heartbeat non arriva, si controlla heartbeat_task. Se output resta acceso, si controlla output_update_task. Se shell è lenta, si controlla diagnostic_shell_task. Una tabella task è molto utile nella documentazione.

Una tabella di scheduling può contenere:

task;

scopo;

priorità;

intervallo indicativo;

mode in cui è attivo;

criticità;

output/telemetria prodotta;

note safety.

Per esempio, safety_check_task ha priorità alta, è attivo in quasi tutti i mode, è critico e può generare safety_state_changed. diagnostic_shell_task ha priorità bassa, è attivo in idle/diagnostic, non è critico e può essere ridotto in low_energy. Questa tabella rende MicroBot OS più trasparente.

Il ventinovesimo concetto è il rischio di **over-scheduling**. Non bisogna creare troppi task troppo presto. Nelle prime versioni, pochi task ben definiti bastano. Se si creano venti task per un nodo LED-only, il firmware diventa inutilmente complesso. Lo scheduling deve crescere con le funzioni. Prima communication, heartbeat, output, safety. Poi telemetry events. Poi shell. Poi energy e thermal. Poi watchdog health. Poi resource monitor. Questo mantiene il sistema semplice ma evolutivo.

Una progressione sensata è:

v0.1: loop semplice con LED e status;

v0.2: heartbeat_task;

v0.3: communication_task e command_task;

v0.4: role_led_task;

v0.5: telemetry_task;

v0.6: safety_check_task;

v0.7: scheduler strutturato;

v0.8: diagnostic_shell_task;

v0.9: watchdog_health_task;

v1.0: resource_monitor_task;

v1.2: energy_update e thermal_update.

Questo è coerente con l'evoluzione delle versioni.

Il trentesimo concetto è lo scheduling come garanzia di comportamento deterministico. In un sistema distribuito, non si può avere un firmware che “ogni tanto” fa qualcosa quando capita. Il nodo deve avere tempi prevedibili. Non significa perfetti al millisecondo in ogni versione, ma prevedibili abbastanza da essere controllabili. Heartbeat deve arrivare a intervalli noti. Safety deve essere controllata con frequenza nota. Output deve spegnersi entro una finestra nota. Recovery deve seguire passi noti. Questa prevedibilità è ciò che rende MicroBot OS affidabile.

In conclusione, lo scheduling è il livello di MicroBot OS che organizza nel tempo le attività interne del nodo: comunicazione, command dispatch, safety, output update, heartbeat, telemetria, sincronizzazione, controller_lost, diagnostica, energia, temperatura, watchdog e recovery. Esso deve essere non bloccante, cooperativo nelle prime versioni, basato su task brevi, priorità chiare, intervalli controllati e monitoraggio della salute dei task. Le funzioni critiche devono sempre avere precedenza su diagnostica, effetti visuali e telemetria estesa. Lo scheduler impedisce che il nodo diventi lento, cieco o pericoloso a causa di delay, buffer pieni o task troppo pesanti. Con lo scheduling, MicroBot OS passa da firmware sequenziale a runtime embedded ordinato, capace di mantenere reattività, sicurezza e osservabilità. Il punto successivo, dedicato alla shell interna, descriverà come il nodo possa esporre un'interfaccia diagnostica locale per interrogare stato, errori, risorse, task, capabilities e funzioni di test, rendendo ogni MicroBot ispezionabile anche senza dashboard completa.

14.6 Shell interna

La shell interna rappresenta il livello di MicroBot OS che permette di interrogare, controllare e diagnosticare un nodo MicroBot in modo locale, diretto e strutturato. Dopo aver definito lo scheduling, cioè l'organizzazione temporale dei task interni del nodo, diventa necessario descrivere uno strumento che permetta allo sviluppatore di vedere cosa sta accadendo dentro il MicroBot senza dipendere sempre dalla dashboard, dal controller centrale o dalla simulazione. La shell interna è proprio questa interfaccia: un canale di diagnostica e controllo, inizialmente semplice, attraverso cui il nodo può ricevere comandi testuali, restituire stato, esporre errori, mostrare risorse, attivare test sicuri e aiutare il debug del firmware.

Nel progetto MicroBot, la shell interna ha un valore enorme perché trasforma il nodo da “scatola chiusa” a sistema ispezionabile. Quando un ESP32 non comunica, non lampeggia come previsto, non riceve comandi, non invia heartbeat o sembra bloccato, non basta guardare la dashboard. Bisogna poter collegare il nodo via seriale, aprire il monitor, digitare comandi come status, identity, safety, tasks, errors, capabilities, led_test, heartbeat_test, safe_idle e capire subito cosa sta succedendo. La shell interna riduce il tempo di debug e rende ogni MicroBot più controllabile.

La shell interna, nelle prime versioni, può essere implementata tramite porta seriale USB. Questo è il metodo più semplice e più adatto allo sviluppo. Il nodo riceve righe di testo dalla seriale, le interpreta e risponde con output leggibile. In versioni successive, la stessa logica potrebbe essere esposta via rete, dashboard o protocollo diagnostico. Tuttavia, la prima forma seriale è già

sufficiente per dare al nodo una forte capacità diagnostica. Non serve subito una shell complessa: serve una shell chiara, sicura e coerente con il resto di MicroBot OS.

Il principio generale può essere espresso così:

la shell interna deve permettere di osservare e testare il nodo senza bypassare la sicurezza locale.

Questa frase è fondamentale. La shell non deve diventare una porta pericolosa che permette di accendere uscite critiche senza controlli. Deve essere uno strumento diagnostico, non un modo per aggirare il Safety Layer. Se un comando shell prova ad attivare un PWM, deve comunque passare da `safety_manager` e `output_manager`. Se il nodo è in `thermal_lock`, anche la shell deve rispettarlo. Se il nodo è in `critical_energy`, la shell non deve forzare attuazioni. La shell interna deve essere potente per il debug, ma subordinata alle stesse regole del sistema.

La prima funzione della shell interna è il comando **help**. Questo comando mostra l'elenco dei comandi disponibili e una breve descrizione. È importante perché permette di usare il nodo senza ricordare tutto a memoria. In una prima versione, `help` può mostrare pochi comandi. In versioni successive può distinguere comandi base, comandi diagnostici, comandi safety, comandi output e comandi sviluppatore. Il comando `help` deve indicare anche se alcune funzioni sono disabilitate nello stato corrente.

Un `help` essenziale può includere comandi come:

`help;`

`status;`

`identity;`

`mode;`

`role;`

`safety;`

`energy;`

`thermal;`

`sync;`

`capabilities;`

`tasks;`

`errors;`

`events;`

`led_test;`

`heartbeat_test;`

`safe_idle;`

clear_error;

reboot, se supportato e sicuro.

Il comando help non deve essere solo decorativo. Deve diventare la mappa locale del nodo. Se un comando non è supportato dalla versione corrente di MicroBot OS, non deve apparire oppure deve essere indicato come unsupported. Questo evita confusione tra versioni.

La seconda funzione è il comando **status**. Questo è probabilmente il comando più importante della shell interna. Deve restituire una sintesi dello stato corrente del nodo: node_id, boot_id, uptime, mode, role, safety_state, energy_state, thermal_state, sync_status, last_error, heartbeat status, controller status e capabilities principali. Il comando status deve essere rapido, compatto e sempre disponibile, salvo guasti gravi. È la prima cosa da digitare quando qualcosa non funziona.

Un output concettuale di status potrebbe contenere:

node_id;

boot_id;

microbot_os_version;

uptime_ms;

mode;

role;

pattern_run_id;

phase_id;

safety_state;

energy_state;

thermal_state;

sync_status;

last_controller_seen_ms;

last_heartbeat_sent_ms;

last_error_code;

available_capabilities;

blocked_capabilities.

Questo comando collega direttamente la shell alla telemetria della Sezione 13. In pratica, status mostra localmente ciò che il nodo invierebbe al controller. Se la dashboard mostra dati diversi da status, allora il problema è nella comunicazione, nello snapshot o nella dashboard, non nello stato locale del nodo.

La terza funzione è il comando **identity**. Questo comando mostra l'identità del nodo: node_id, boot_id, firmware_version, protocol_version, hardware_profile, hardware_revision, capability_profile e config_epoch. È fondamentale quando si lavora con più nodi fisici. Permette di capire quale ESP32 si sta guardando, quale firmware ha e se è compatibile con il controller. Senza un comando identity, è facile confondere nodi, specialmente quando sono su breadboard simili.

Il comando identity deve rispondere a domande precise:

che nodo è questo?

quale sessione di boot è attiva?

che versione firmware sta eseguendo?

che protocollo supporta?

che profilo hardware usa?

quali capabilities dichiara?

quale configurazione è caricata?

Se il controller si aspetta protocol_version 0.5 ma il nodo dichiara 0.3, il problema diventa immediatamente visibile. Se il nodo dovrebbe essere PWM_NODE_V0 ma dichiara LED_NODE_V0, il problema è nella configurazione o nel firmware caricato. La shell permette di scoprirlo subito.

La quarta funzione è il comando **mode**. Questo comando mostra o, in alcune condizioni, modifica la modalità operativa del nodo. Le modalità possono essere idle, active, diagnostic, safe_idle, safety, recovery, low_power. Visualizzare mode è sempre sicuro. Modificare mode deve invece essere controllato. Per esempio, passare da idle a diagnostic può essere permesso. Passare da safety direttamente ad active non deve essere permesso. Passare a safe_idle deve essere sempre permesso. Passare a active potrebbe richiedere state_sync o comando del controller.

La shell deve rispettare la state machine. Se l'utente scrive mode active ma il nodo è in safety, il comando deve essere rifiutato con una motivazione chiara, per esempio transition_not_allowed oppure clear_safety_required. Questo è importante perché la shell non deve creare stati impossibili.

Una logica corretta è:

mode mostra modalità corrente;

mode diagnostic può essere permesso da idle;

mode safe_idle deve essere sempre sicuro;

mode active non deve essere forzato manualmente senza contesto;

mode safety può essere attivato per test o protezione;

mode clear non deve esistere senza procedura di clear_safety controllata.

Questo rende la shell coerente con il Safety Layer.

La quinta funzione è il comando **role**. Questo comando mostra il ruolo corrente del nodo e, se necessario, permette di assegnare un ruolo di test in `diagnostic_mode`. Il ruolo può essere `leader`, `connector`, `terminal`, `passive_node`, `diagnostic_node`, `actuator_node`, `bridge_node` o altro. In una demo LED, modificare il ruolo può cambiare il pattern di lampeggio del nodo. Tuttavia, assegnare un ruolo tramite shell deve essere considerato una funzione diagnostica, non parte normale della run distribuita. Durante una run attiva, i ruoli devono arrivare dal controller.

Il comando `role` può mostrare:

`reported_role`;

`assigned_by`;

`role_status`;

`role_assigned_at`;

`role_confirmed_at`;

`role_supported`;

`role_rejection_reason`, se presente.

Se `role` viene modificato localmente, la shell deve indicare che il ruolo è `local_override` o `diagnostic_override`. Questo evita confusione con i ruoli assegnati dal controller. Un ruolo manuale non deve essere scambiato per un ruolo valido di pattern, salvo modalità esplicitamente diagnostica.

La sesta funzione è il comando **safety**. Questo comando mostra lo stato del Safety Layer locale. Deve indicare `safety_state`, `emergency_stop` status, `blocked_capabilities`, `output_state`, `cooldown`, `last_safety_event`, `controller_lost` status, `watchdog` status e motivi di blocco. È uno dei comandi più importanti per hardware reale. Se un output non si attiva, spesso la causa è `safety`. Il comando `safety` deve spiegare perché.

Un comando `safety` può mostrare:

`safety_state`;

`emergency_stop_active`;

`controller_lost`;

`outputs_locked`;

`blocked_capabilities`;

`block_reason`;

`cooldown_remaining_ms`;

`last_safety_event`;

`last_command_rejected_reason`;

`watchdog_health`;

safe_outputs_state.

Per esempio, se un comando PWM viene rifiutato, safety può mostrare blocked_capabilities = pwm_output, block_reason = thermal_warning oppure low_energy. Questo riduce tantissimo il tempo di debug. Non bisogna andare a cercare nel codice; il nodo lo dice.

La settima funzione è il comando **energy**. Questo comando mostra la situazione energetica del nodo: power_source, energy_state, battery_voltage se disponibile, battery_level stimato, energy_source, low_energy_threshold, critical_energy_threshold e policy attiva. Nei prototipi iniziali alimentati via USB, molte informazioni possono essere unavailable o USB_DEBUG. Questo va bene, purché sia dichiarato. La shell deve distinguere dati misurati, stimati e non disponibili.

Il comando energy può restituire:

power_source;

energy_state;

energy_source;

battery_voltage_v;

battery_level_percent;

low_energy_threshold;

critical_energy_threshold;

energy_policy;

energy_limited_capabilities.

Se il nodo è alimentato via USB e non ha batteria, la shell può indicare power_source = usb_debug e battery_voltage = unavailable. Questa trasparenza è importante. Non bisogna fingere misure che non esistono.

L'ottava funzione è il comando **thermal**. Questo comando mostra thermal_state, thermal_source, thermal_load, temperature_c se disponibile, cooldown_remaining_ms, warning_threshold, lock_threshold e output bloccati per ragioni termiche. Nei nodi senza sensore temperatura, thermal_source può essere estimated o simulated. Nei nodi con sensore, measured. Come per energy, la fonte è fondamentale.

Il comando thermal può mostrare:

thermal_state;

thermal_source;

thermal_load;

temperature_c;

warning_threshold;

lock_threshold;

cooldown_remaining_ms;

last_high_energy_command;

thermal_blocked_capabilities;

thermal_recovery_status.

Se thermal_state = thermal_lock, la shell deve indicare chiaramente che attuazioni critiche sono bloccate. Se thermal_source = unavailable su nodo con capability attuativa, il sistema dovrebbe essere conservativo. Questo comando prepara il debugging di bobine, PWM e driver.

La nona funzione è il comando **sync**. Questo comando mostra lo stato di sincronizzazione con il controller: sync_status, last_sync_time, clock_offset_ms, jitter_ms, last_sync_age_ms, phase_id e out_of_phase status. Nei prototipi iniziali, la sincronizzazione può essere semplice o non implementata. In quel caso sync deve indicare unsupported o unsynced, non restituire dati finti. Quando comandi temporizzati saranno introdotti, sync diventerà molto importante.

Il comando sync può mostrare:

sync_status;

last_sync_ms;

clock_offset_ms;

jitter_ms;

phase_id;

expected_phase_id;

out_of_phase;

timed_commands_allowed.

Se timed_commands_allowed = false, il nodo non deve accettare comandi execute_at critici. La shell rende questa decisione visibile.

La decima funzione è il comando **capabilities**. Questo comando mostra cosa il nodo può fare. Deve distinguere installed_capabilities, available_capabilities e blocked_capabilities. Questa distinzione è essenziale. Un nodo può avere pwm_output installato ma bloccato per thermal_lock. Un nodo può avere led_status disponibile. Un nodo può non avere coil_pwm. La shell deve mostrare tutto in modo chiaro.

Un output capabilities può includere:

installed_capabilities;

available_capabilities;

blocked_capabilities;

block_reasons;

hardware_profile;

unsupported_capabilities_requested_recently.

Questo comando aiuta anche il controller e l'utente. Se un comando fallisce per `capability_missing`, si controlla `capabilities` e si vede subito se il nodo lo supporta. Nei prototipi LED-only, `coil_pwm` deve risultare `unsupported`, non semplicemente assente in modo ambiguo.

L'undicesima funzione è il comando **tasks**. Questo comando mostra lo stato dello scheduler: task attivi, intervalli, `last_run`, `last_duration`, `overrun_count`, `deadline_missed_count` e `health_status`. È molto utile per capire se il nodo sta rispettando il proprio scheduling. Se heartbeat non viene inviato, si controlla `heartbeat_task`. Se output non si spegne, si controlla `output_update_task`. Se safety non gira, è un problema grave.

Il comando `tasks` può mostrare:

`task_id`;

`enabled`;

`priority`;

`interval_ms`;

`last_run_ms`;

`last_duration_ms`;

`overrun_count`;

`deadline_missed_count`;

`health_status`.

In una versione semplice, può mostrare solo i task principali. In versioni avanzate, può diventare più dettagliato. Questo comando è il ponte tra shell interna e scheduling.

La dodicesima funzione è il comando **resources**. Questo comando può mostrare lo stato delle risorse interne: memoria libera se disponibile, `command_buffer_used`, `event_buffer_used`, `telemetry_level`, `active_outputs`, `loop_time`, error counters e communication status. È più generale di `tasks` e serve a capire se il nodo è sotto carico o se qualche buffer si sta riempiendo.

Un comando `resources` può mostrare:

`free_memory`;

`main_loop_time_ms`;

`command_buffer_used`;

`event_buffer_used`;

`telemetry_level`;

active_outputs_count;

communication_status;

api/message error count;

reset_count;

diagnostic_mode_status.

Non tutte queste informazioni saranno disponibili in ogni versione. La shell può indicare unavailable. L'importante è avere una struttura coerente.

La tredicesima funzione è il comando **errors**. Questo comando mostra last_error_code, error_count, error history recente, last_rejection_reason, last_fault e reset_reason. Gli errori devono essere strutturati, non stringhe casuali. Il comando errors permette di capire perché il nodo ha rifiutato comandi, perché è entrato in safety o perché si è resettato.

Il comando errors può mostrare:

last_error_code;

last_error_category;

last_error_severity;

last_error_time;

related_command_id;

rejected_command_count;

reset_reason;

watchdog_reset_count;

recent_errors.

Se il nodo ha un ring buffer di eventi, errors può mostrare solo eventi di severity error o critical. Questo è molto utile perché permette di filtrare rapidamente problemi gravi.

La quattordicesima funzione è il comando **events**. Questo comando mostra gli ultimi eventi locali: mode_changed, role_changed, command_rejected, thermal_warning, energy_low, controller_lost, recovery_started, safe_idle_entered. Gli eventi sono la timeline locale del nodo. Non sostituiscono i log del controller, ma permettono di capire cosa è successo localmente.

Un evento shell può essere mostrato in forma compatta:

timestamp;

severity;

event_type;

reason;

related_command_id.

Per esempio:

```
[123450 ms] WARNING command_rejected reason=thermal_lock command_id=CMD-18
```

Questo tipo di output è estremamente utile per debug seriale. Si vede subito la sequenza degli ultimi problemi.

La quindicesima funzione è il comando **led_test**. Nei prototipi iniziali, testare LED è fondamentale. Il comando led_test può verificare che i LED siano collegati correttamente, che il pin_map sia giusto, che i pattern di ruolo funzionino e che output_manager sia operativo. Tuttavia, anche led_test deve essere non bloccante o molto limitato. Non deve bloccare il firmware per lunghi periodi. Può avviare una sequenza breve controllata dallo scheduler.

Il comando led_test può supportare:

led_test status;

led_test role;

led_test all;

led_test stop.

Se il nodo è in critical_energy, led_test può essere rifiutato o ridotto. Se è in safety, può essere permesso solo un pattern di safety a bassa intensità. Anche i LED hanno costo energetico, quindi la shell deve rispettare le policy.

La sedicesima funzione è il comando **heartbeat_test**. Questo comando può forzare l'invio di un heartbeat o mostrare quando l'ultimo heartbeat è stato inviato. È utile quando la dashboard non vede il nodo. Se heartbeat_test funziona localmente ma il controller non riceve, il problema è nel canale di comunicazione. Se heartbeat_task non gira, il problema è scheduler o telemetry_manager.

Il comando heartbeat_test può mostrare:

last_heartbeat_sent_ms;

heartbeat_interval_ms;

heartbeat_enabled;

last_heartbeat_result;

forced_heartbeat_result.

Questo comando è molto utile nei primi test ESP32/controller.

La diciassettesima funzione è il comando **comm** o **communication**. Questo comando mostra lo stato della comunicazione: mezzo attivo, connessione Wi-Fi se prevista, last_controller_seen, messaggi ricevuti, messaggi inviati, errori di parsing, ack inviati, ack ricevuti, communication_status. Nei prototipi seriali, mostra serial active. Nei prototipi Wi-Fi, mostra stato rete. Questo comando è essenziale quando il nodo sembra non comunicare.

Può includere:

communication_mode;

connection_status;

controller_address, se applicabile;

last_controller_seen_ms;

messages_received;

messages_sent;

parse_error_count;

last_comm_error;

comm_adapter_status.

Se la comunicazione è il problema, questo comando lo rende visibile. Se comm_status è ok ma la dashboard non vede il nodo, il problema potrebbe essere backend o routing. Se comm_status è disconnected, il problema è locale o rete.

La diciottesima funzione è il comando **safe_idle**. Questo comando deve essere sempre disponibile e sicuro. Porta il nodo in safe_idle, spegne output critici, blocca attuazioni e aggiorna lo stato. È una forma locale di messa in sicurezza. Deve passare direttamente attraverso safety_manager e output_manager. Se qualcosa va storto, deve comunque tentare hal_safe_outputs_off. Questo comando è fondamentale durante sviluppo hardware.

safe_idle deve fare almeno:

spegnere output critici;

azzerare PWM;

fermare sequenze attuate;

bloccare comandi attuativi pendenti;

aggiornare mode = safe_idle;

generare evento;

stampare conferma.

Questo comando è una cintura di sicurezza. Durante test fisici, deve essere sempre facile portare il nodo in stato sicuro.

La diciannovesima funzione è il comando **emergency_stop** o **estop**, se previsto nella shell. Questo comando è ancora più forte di safe_idle. Deve portare il nodo in safety/emergency_stopped, spegnere uscite e richiedere clear_safety esplicito prima di riattivare funzioni. Deve essere usato con cautela, ma deve esistere. In fase di sviluppo, poter inviare emergency_stop dalla seriale è molto utile.

La differenza è:

safe_idle = stato sicuro operativo, potenzialmente reversibile con comandi normali;

emergency_stop = blocco di emergenza, richiede procedura esplicita di reset/clear.

Questa distinzione deve essere chiara nella shell. Non bisogna usare nomi ambigui.

La ventesima funzione è il comando **clear_error** o **clear_safety**, ma questi devono essere gestiti con molta cautela. clear_error può cancellare last_error non critico. clear_safety invece può uscire da uno stato safety, ma solo se le condizioni sono sicure. Non deve annullare emergency_stop o thermal_lock senza controlli. Se thermal_state è ancora thermal_lock, clear_safety deve rifiutare. Se critical_energy è ancora attivo, deve rifiutare. Se output non è safe, deve rifiutare.

Una procedura clear_safety deve controllare:

outputs_off;

thermal_state non lock;

energy_state non critical;

emergency condition rimossa;

controller_lost non critico o modalità locale diagnostic;

nessun comando attuativo pendente;

utente conferma, se previsto.

Solo dopo può passare da safety a safe_idle. Non deve passare direttamente ad active.

La ventunesima funzione è il comando **reboot**, se supportato. Reboot può essere utile in debug, ma è delicato. Deve essere consentito solo in modalità sicura, oppure deve prima chiamare safe_outputs_off. Dopo reboot, boot_id cambia e il controller deve rilevare recovery_required. La shell deve indicare chiaramente che reboot interrompe la sessione operativa. Nei prototipi iniziali, reboot può anche non essere implementato.

Se implementato, reboot deve fare:

safe_outputs_off;

inviare evento reboot_requested se possibile;

attendere breve flush telemetria, se sicuro;

riavviare;

al boot impostare safe_idle;

generare nuovo boot_id.

Il reboot non deve essere usato come scorciatoia per risolvere stati incoerenti senza diagnosi, ma può essere utile.

La ventiduesima funzione è il comando **self_test**. Questo comando può eseguire una serie di test locali non pericolosi: verifica identità, LED, heartbeat, stato safety, buffer, task, capabilities, HAL base. In versioni avanzate, self_test può avere profili: basic, communication, outputs, sensors. I test ad alto rischio devono essere esclusi o richiedere modalità diagnostica e conferma.

Un self_test basic può controllare:

identity presente;

boot_id valido;

HAL inizializzata;

status LED funzionante;

scheduler attivo;

heartbeat task abilitato;

safety_state leggibile;

telemetry manager operativo;

nessun errore critico.

Il risultato deve essere pass, warning o fail. Questo è molto utile prima di inserire il nodo nello swarm.

La ventitreesima funzione è il comando **pinmap** o **hardware**. Questo comando mostra hardware_profile, pin_map, output disponibili, sensori disponibili e limiti hardware. È molto utile quando si lavora su breadboard. Permette di verificare che il firmware caricato corrisponda al cablaggio reale. Se il LED è su GPIO 23 ma il firmware usa GPIO 2, lo si vede subito.

Il comando hardware può mostrare:

hardware_profile;

hardware_revision;

status_led_pin;

role_led_pin;

pwm_pin;

battery_adc_pin;

temperature_pin;

available_outputs;

available_sensors;

hardware_limits.

Questo comando collega la shell all'astrazione hardware. Aiuta a evitare bug banali ma frequenti.

La ventiquattresima funzione è il comando **output**. Questo comando mostra lo stato degli output: LED, PWM, bobina se presente, active, duty, duration_remaining, cooldown, locked. Non deve necessariamente permettere di attivare output, se non in diagnostic_mode. Ma deve sempre poter mostrare cosa è attivo. Questo è fondamentale per sicurezza.

Il comando output può mostrare:

output_id;

type;

active;

current_level;

started_at;

duration_remaining_ms;

cooldown_remaining_ms;

locked;

last_command_id.

Se un output resta active quando non dovrebbe, si vede subito. Se cooldown è attivo, si vede perché un comando viene rifiutato. Questo comando è essenziale per versioni con PWM.

La venticinquesima funzione è la gestione del parsing dei comandi shell. La shell deve essere semplice. Ogni comando può essere una parola con argomenti opzionali. Per esempio:

status;

mode diagnostic;

role connector;

led_test all;

telemetry debug;

safe_idle;

events 10.

Il parser deve essere robusto. Se l'utente scrive comando non valido, deve rispondere unknown_command. Se manca un argomento, deve rispondere missing_argument. Se l'argomento è fuori range, invalid_argument. Non deve crashare. Non deve interpretare input parziali in modo pericoloso.

Una pipeline shell può essere:

leggi riga non bloccante;

rimuovi spazi;
identifica comando;
valida argomenti;
controlla modalità e safety;
esegui funzione;
stampa risposta;
ritorna al loop.

Questa pipeline deve rispettare lo scheduling. La shell non deve bloccare.

La ventiseiesima funzione è il controllo dei permessi. Nelle prime versioni seriali, la shell può essere considerata accessibile solo allo sviluppatore collegato fisicamente. Tuttavia, se in futuro la shell sarà via rete, serviranno livelli di autorizzazione. Alcuni comandi saranno read-only, altri control, altri critical. Già nella documentazione conviene distinguere classi di comando.

Classi possibili:

read_only: status, identity, safety, tasks, errors;
diagnostic_safe: led_test, heartbeat_test, self_test basic;
control_safe: mode diagnostic, safe_idle;
critical: emergency_stop, reboot, output test;
restricted: attuazioni, configurazione limiti, clear_safety.

Questa classificazione prepara la sicurezza futura. Anche se non viene implementata subito, è importante concettualmente.

La ventisettesima funzione è il rapporto tra shell e dashboard. La shell interna non sostituisce la dashboard. La dashboard osserva lo swarm nel suo insieme; la shell osserva il nodo localmente. Le due interfacce sono complementari. Se la dashboard dice che il nodo è offline, ma la shell mostra che il nodo è vivo, il problema è comunicazione. Se la shell dice thermal_lock e la dashboard non lo mostra, il problema è telemetria o frontend. Se entrambe dicono safety_lock, il problema è reale nello stato del nodo.

Una regola utile è:

dashboard = vista globale;
shell = verità locale del nodo;
controller logs = decisioni globali;
telemetria = ponte tra le due.

Questa distinzione è molto utile nel debug dello swarm.

La ventottesima funzione è il rapporto tra shell e logging. Ogni comando shell importante deve generare un evento locale. Per esempio, mode diagnostic da shell, safe_idle da shell, clear_error da shell. Questo serve a sapere che un cambiamento è stato causato manualmente. Se un nodo cambia modalità perché l'utente lo ha fatto da shell, il controller deve poterlo sapere, almeno tramite evento o status. Altrimenti potrebbe sembrare un cambiamento autonomo.

Un evento shell_command può contenere:

command_name;

source = local_shell;

timestamp;

result;

state_changed;

reason.

Questo mantiene tracciabilità. La shell non deve essere invisibile al sistema.

La ventinovesima funzione è la roadmap della shell interna. Nelle prime versioni, basta help, status, identity, led_test e safe_idle. Poi si aggiungono role, mode, safety, errors. Poi tasks e resources. Poi energy, thermal, sync. Poi self_test. Poi output diagnostics. Poi controlli più avanzati. Questa progressione evita di fare una shell enorme prima che il firmware sia stabile.

Una roadmap realistica può essere:

v0.1: serial boot log e status base;

v0.2: help, identity, status;

v0.3: led_test e heartbeat_test;

v0.4: mode e role read-only;

v0.5: safety, errors, events;

v0.6: safe_idle ed emergency_stop locale;

v0.7: tasks e scheduler health;

v0.8: resources e buffers;

v0.9: energy, thermal, sync;

v1.0: self_test e hardware profile;

v1.1: output diagnostics;

v1.2: shell estesa con livelli di comando.

Questa roadmap è coerente con l'evoluzione di MicroBot OS. La shell cresce quando crescono i servizi interni.

Il trentesimo punto è il valore metodologico della shell interna. In un progetto complesso, la shell è uno degli strumenti più concreti per non perdersi. Permette di verificare un nodo alla volta, isolare problemi, distinguere firmware da controller, controllare risorse, testare output sicuri, vedere errori e confermare configurazioni. È un piccolo strumento, ma ha un impatto enorme sulla qualità del progetto. Ogni nodo MicroBot dovrebbe poter dire “chi sono, in che stato sono, cosa posso fare, cosa è bloccato, quali errori ho avuto e se sono sicuro”.

In conclusione, la shell interna è l’interfaccia diagnostica locale di MicroBot OS. Essa permette di interrogare identità, stato, modalità, ruolo, safety, energia, temperatura, sincronizzazione, capabilities, task, risorse, errori, eventi, comunicazione e output del nodo. Permette anche di eseguire test controllati, portare il nodo in `safe_idle`, verificare LED, heartbeat e hardware profile. Tuttavia, la shell non deve mai bypassare il Safety Layer: ogni comando deve rispettare mode, capabilities, energia, temperatura, cooldown e blocchi di sicurezza. La shell rende il MicroBot ispezionabile anche senza dashboard completa e diventa uno strumento essenziale per firmware, debug hardware, test di prototipo e sviluppo progressivo. Il punto successivo, dedicato alla diagnostica, chiuderà la Sezione 14 spiegando come MicroBot OS possa raccogliere, classificare e interpretare errori, stati interni, task health, reset, eventi, risorse e anomalie, trasformando il nodo in un sistema non solo comandabile, ma anche autodiagnosticabile.

14.7 Diagnostica

La diagnostica rappresenta il punto conclusivo della Sezione 14 e descrive la capacità di MicroBot OS di osservare sé stesso, individuare anomalie interne, classificare errori, esporre stati critici e fornire informazioni utili per correggere problemi nel nodo. Dopo aver definito sistema operativo embedded, evoluzione delle versioni, gestione delle risorse, astrazione hardware, scheduling e shell interna, diventa necessario chiudere la sezione spiegando come il nodo possa diventare non solo comandabile, ma anche autodiagnosticabile. Un MicroBot maturo non deve limitarsi a eseguire istruzioni: deve saper dire se è sano, se ha risorse sufficienti, se i task stanno girando, se gli output sono sicuri, se la comunicazione funziona, se ci sono errori ripetuti, se il watchdog è intervenuto, se la configurazione è coerente e se può partecipare allo swarm.

Nel progetto MicroBot, la diagnostica è fondamentale perché ogni nodo è parte di un sistema distribuito. Quando qualcosa non funziona, il problema può trovarsi nel singolo MicroBot, nel controller, nella rete, nella dashboard, nella simulazione, nel pattern o nella configurazione. La diagnostica locale serve a isolare la responsabilità del nodo. Se il nodo dichiara stato sano, heartbeat attivo, scheduler regolare e nessun errore critico, il problema può essere altrove. Se invece il nodo dichiara `thermal_lock`, `command_buffer_full`, `watchdog_reset`, `controller_lost` o `capability_missing`, allora il problema è interno o collegato al modo in cui il controller sta usando quel nodo. Senza diagnostica, ogni errore diventa nebbia. Con diagnostica, l’errore diventa leggibile.

Il principio generale della diagnostica MicroBot può essere espresso così:

ogni nodo deve poter descrivere il proprio stato interno in modo abbastanza chiaro da rendere diagnosticabile un guasto, una limitazione o un comportamento anomalo.

Questo principio collega diagnostica, telemetria e shell interna. La telemetria invia dati al controller e alla dashboard. La shell permette di interrogare il nodo localmente. La diagnostica è il contenuto tecnico che rende utili entrambi. Non basta avere una shell se il nodo non sa cosa riportare. Non basta avere heartbeat se l’heartbeat dice solo “sono vivo”. Un nodo veramente diagnosticabile deve riportare salute dei servizi, errori recenti, stato delle risorse, stato degli output, stato della comunicazione, reset reason, task health e capabilities disponibili o bloccate.

La prima area diagnostica è lo **stato generale del nodo**. Questo è il livello più alto e deve rispondere a una domanda semplice: il nodo è operativo, degradato, in safety, in recovery o non utilizzabile? Questo stato può essere espresso attraverso un campo `node_health` oppure `diagnostic_status`. A differenza di `mode`, che indica la modalità operativa, `diagnostic_status` sintetizza la salute complessiva. Per esempio, un nodo può avere `mode = idle` ma `diagnostic_status = degraded` perché ha molti errori di comunicazione. Oppure può avere `mode = active` ma `diagnostic_status = warning` perché `energy_state` è `low_energy`. Oppure `mode = safety` e `diagnostic_status = critical`.

Una classificazione utile può essere:

`healthy;`

`warning;`

`degraded;`

`critical;`

`recovery_required;`

`manual_intervention_required;`

`unknown.`

`healthy` indica che il nodo è utilizzabile secondo le sue capabilities. `warning` indica che esistono condizioni da osservare ma non ancora bloccanti. `degraded` indica che alcune funzioni sono limitate o alcune capabilities sono bloccate. `critical` indica che il nodo non deve essere usato per funzioni attive. `recovery_required` indica che serve `state_sync`, `resync` o procedura di ripristino. `manual_intervention_required` indica che la recovery automatica non basta. `unknown` indica che mancano dati diagnostici essenziali. Questa sintesi aiuta controller e dashboard a decidere rapidamente.

La seconda area diagnostica è la **salute dei servizi interni**. MicroBot OS può essere visto come insieme di servizi: `communication_service`, `command_service`, `safety_service`, `output_service`, `telemetry_service`, `scheduler_service`, `diagnostics_service`, `energy_service`, `thermal_service`, `sync_service`. Ogni servizio deve poter dichiarare se è attivo, quando è stato aggiornato, quanti errori ha prodotto e se ha mancato scadenze. Questa diagnostica è molto importante perché un nodo può essere apparentemente vivo ma avere un servizio interno non funzionante.

Una tabella `service_health` può contenere:

`service_id;`

`enabled;`

`health_status;`

`last_update_ms;`

`last_duration_ms;`

`error_count;`

warning_count;

deadline_missed_count;

last_error_code;

criticality.

Per esempio, se telemetry_service è degraded, il nodo può funzionare localmente ma essere poco osservabile. Se safety_service è degraded, il nodo deve entrare in safe mode, perché la sicurezza non può essere incerta. Se output_service ha errori, gli output devono essere spenti. Se communication_service è warning, il nodo può essere online ma instabile. Questa granularità permette di capire non solo che il nodo ha un problema, ma quale parte del nodo ha un problema.

La terza area diagnostica è il **controllo dello scheduler**. Lo scheduling è ciò che permette al nodo di eseguire task con frequenza corretta. La diagnostica deve quindi indicare se i task critici stanno rispettando i tempi. Ogni task importante può avere last_run_ms, interval_ms, last_duration_ms, max_duration_ms, deadline_missed_count e overrun_count. Se safety_check_task non gira abbastanza spesso, il nodo non è sicuro. Se output_update_task è in ritardo, un output potrebbe restare attivo troppo a lungo. Se heartbeat_task salta troppe scadenze, il controller può dichiarare il nodo stale.

La diagnostica scheduler deve individuare:

task mai eseguiti;

task in ritardo;

task troppo lunghi;

task disabilitati in modo non previsto;

deadline miss ripetuti;

overrun critici;

loop_time eccessivo.

Una metrica sintetica può essere scheduler_health. Se tutti i task critici sono aggiornati, scheduler_health = ok. Se task accessori saltano, scheduler_health = warning. Se safety_task o output_task saltano, scheduler_health = critical. Questo campo deve influenzare il watchdog. Un nodo con scheduler critico non deve continuare a operare come se nulla fosse.

La quarta area diagnostica è il **watchdog e reset history**. Ogni reset è un evento importante. Il nodo deve poter dichiarare reset_reason, reset_count, last_boot_time, boot_id e watchdog_reset_count. Se il nodo si è riavviato per watchdog, il controller deve saperlo. Se si è riavviato per brownout, può esserci un problema energetico. Se si è riavviato per software_reset richiesto dalla shell, è diverso. La diagnostica deve rendere distinguibili queste cause.

Un blocco reset_diagnostics può contenere:

current_boot_id;

reset_reason;

reset_count;

watchdog_reset_count;

brownout_reset_count;

software_reset_count;

uptime_ms;

last_reset_phase;

last_command_before_reset, se disponibile;

recovery_required.

Questi dati sono fondamentali. Se un nodo resetta sempre durante phase_attach, la causa potrebbe essere energia, attuazione, firmware o watchdog. Se resetta dopo un comando specifico, quel comando va controllato. Se reset_count cresce, il nodo deve essere considerato unstable. La diagnostica deve impedire che i reset vengano ignorati.

La quinta area diagnostica è il **command diagnostics**, cioè l'analisi locale dei comandi ricevuti. Il nodo deve poter indicare quanti comandi ha ricevuto, quanti sono stati accettati, quanti rifiutati, quanti duplicati, quanti scaduti, quanti non supportati e quali sono gli ultimi command_id. Questo è utilissimo per capire se il controller sta comunicando correttamente e se il nodo sta rifiutando per motivi validi.

Un blocco command_diagnostics può contenere:

commands_received_count;

commands_accepted_count;

commands_rejected_count;

duplicate_command_count;

expired_command_count;

unsupported_command_count;

invalid_payload_count;

last_command_id;

last_accepted_command_id;

last_rejected_command_id;

last_rejection_reason.

Se commands_received_count resta zero, il nodo non riceve comandi. Se invalid_payload_count cresce, il controller o il parser hanno formato incompatibile. Se expired_command_count cresce, c'è problema di timing. Se unsupported_command_count cresce, il controller sta inviando comandi non compatibili con quella versione o capability. Ogni contatore indica una possibile causa.

La sesta area diagnostica è la **diagnostica delle capabilities**. Un nodo deve sapere cosa può fare, cosa non può fare e cosa è temporaneamente bloccato. `installed_capabilities` indica le funzioni presenti. `available_capabilities` indica quelle utilizzabili ora. `blocked_capabilities` indica quelle bloccate per ragioni di safety, energia, termica, modalità o configurazione. `unsupported_capability_requests` può indicare quante volte il controller ha chiesto qualcosa che il nodo non supporta.

Un blocco `capabilities_diagnostics` può contenere:

`installed_capabilities;`

`available_capabilities;`

`blocked_capabilities;`

`block_reasons;`

`unsupported_capability_requests;`

`last_capability_error;`

`hardware_profile;`

`capability_profile_version.`

Questo blocco è essenziale per evitare errori grossolani. Se il controller chiede `coil_pwm` a un `LED_NODE_V0`, il nodo deve rispondere `capability_missing`. La diagnostica deve poi mostrarlo. Se `coil_pwm` è installata ma bloccata per `thermal_lock`, è un caso diverso. La diagnosi deve distinguere mancanza strutturale e blocco temporaneo.

La settima area diagnostica è la **diagnostica degli output**. Ogni uscita controllata dal nodo deve essere osservabile: LED, PWM, driver, bobina sperimentale, buzzer o altri attuatori. La diagnostica deve indicare quali output sono attivi, da quanto tempo, con quale comando, con quale livello, con quale `auto_off_time` e se sono in cooldown o locked. Questo è particolarmente importante per output fisici.

Un blocco `output_diagnostics` può contenere:

`output_id;`

`output_type;`

`active;`

`current_level;`

`max_allowed_level;`

`started_at_ms;`

`duration_ms;`

`remaining_ms;`

`auto_off_enabled;`

cooldown_remaining_ms;

locked;

lock_reason;

last_command_id.

Se un output resta active oltre il previsto, è un problema critico. Se auto_off_enabled = false su un output critico, è un errore di configurazione. Se max_allowed_level è ridotto per low_energy o thermal_warning, la diagnostica deve mostrarlo. Questo blocco è uno dei più importanti per la sicurezza hardware.

L'ottava area diagnostica è la **diagnostica energetica**. Il nodo deve indicare power_source, energy_state, energy_source, battery_voltage se disponibile, battery_level stimato, soglie e policy attiva. Ma la diagnostica deve andare oltre il valore corrente: deve indicare se l'energia ha causato blocchi, se ci sono stati eventi low_energy, se comandi sono stati rifiutati per energia, se ci sono stati brownout o reset energetici.

Un blocco energy_diagnostics può contenere:

power_source;

energy_state;

energy_source;

battery_voltage_v;

battery_level_percent;

low_energy_threshold;

critical_energy_threshold;

energy_event_count;

commands_blocked_by_energy;

brownout_reset_count;

energy_limited_capabilities;

last_energy_event.

Se energy_source = unavailable e il nodo dovrebbe essere alimentato a batteria, la diagnostica deve indicare confidence bassa. Se critical_energy è attivo, il nodo deve bloccare funzioni non essenziali. Se commands_blocked_by_energy cresce, il controller deve ridurre carico o riassegnare ruolo. L'energia non è solo valore: è causa decisionale.

La nona area diagnostica è la **diagnostica termica**. Come per l'energia, il nodo deve riportare thermal_state, thermal_source, thermal_load o temperature_c, soglie, cooldown e blocchi. Deve anche indicare eventi termici, tempo in warning, tempo in lock, comandi bloccati e recovery termica. Questo è essenziale per futuri PWM e bobine.

Un blocco thermal_diagnostics può contenere:

thermal_state;
thermal_source;
thermal_load;
temperature_c;
warning_threshold;
lock_threshold;
recovery_threshold;
cooldown_remaining_ms;
thermal_event_count;
thermal_warning_count;
thermal_lock_count;
commands_blocked_by_thermal;
last_high_energy_command;
time_in_thermal_warning_ms;
time_in_thermal_lock_ms.

Se thermal_lock_count è maggiore di zero, la run non dovrebbe essere success_clean. Se time_in_thermal_warning è alto, il pattern potrebbe essere troppo pesante. Se thermal_source è estimated, i risultati vanno interpretati con cautela. Questa diagnostica collega nodo, safety e validazione del comportamento.

La decima area diagnostica è la **diagnostica della comunicazione**. Il nodo deve poter indicare se la comunicazione è attiva, quando ha visto l'ultima volta il controller, quanti messaggi ha ricevuto, quanti inviato, quanti errori di parsing, quanti ack, quanti timeout. In una modalità Wi-Fi, può indicare stato connessione, rete, indirizzo controller o errori. In seriale, può indicare serial active. Questa diagnostica è fondamentale quando un nodo non appare in dashboard.

Un blocco communication_diagnostics può contenere:

communication_mode;
connection_status;
last_controller_seen_ms;
messages_received_count;
messages_sent_count;

parse_error_count;

ack_sent_count;

ack_failed_count;

last_comm_error;

controller_lost_count;

communication_health.

Se communication_health = ok ma la dashboard non vede il nodo, il problema è probabilmente dopo il nodo: backend, routing, snapshot o frontend. Se communication_health = degraded, il problema può essere rete o adapter. Se parse_error_count cresce, c'è incompatibilità di protocollo o payload. La diagnostica aiuta a localizzare.

L'undicesima area diagnostica è la **diagnostica di sincronizzazione**. Se il nodo supporta comandi temporizzati o fasi sincronizzate, deve riportare sync_status, last_sync_time, offset, jitter, execution_error e comandi temporizzati rifiutati. Un nodo non sincronizzato può essere online ma non idoneo per ruoli critici. La diagnostica deve renderlo esplicito.

Un blocco sync_diagnostics può contenere:

sync_status;

last_sync_ms;

clock_offset_ms;

jitter_ms;

sync_loss_count;

timed_commands_received;

timed_commands_executed;

timed_commands_rejected;

last_execution_error_ms;

out_of_phase_count.

Se sync_status = sync_lost, timed_commands_allowed deve essere false. Se timed_commands_rejected cresce, il controller sta usando il nodo in modo troppo temporizzato o la rete è instabile. La sincronizzazione è un requisito di contesto, non un dettaglio secondario.

La dodicesima area diagnostica è la **diagnostica di stato e transizioni**. Ogni cambiamento di mode, role, safety_state, energy_state, thermal_state e sync_status deve essere tracciabile. La diagnostica deve poter mostrare almeno l'ultima transizione rilevante e il motivo. Questo aiuta a capire perché il nodo si trova in uno stato. Per esempio, non basta sapere che è in safe_idle; bisogna sapere se ci è entrato per controller_lost, emergency_stop, thermal_lock o comando manuale.

Un blocco state_transition_diagnostics può contenere:

current_mode;

previous_mode;

mode_change_reason;

current_role;

previous_role;

role_change_reason;

last_safety_transition;

last_energy_transition;

last_thermal_transition;

last_sync_transition;

transition_count.

Questa diagnostica rende lo stato storico, non solo istantaneo. Un nodo in safety per emergency_stop è diverso da un nodo in safety per thermal_lock. La risposta del sistema deve cambiare.

La tredicesima area diagnostica è la **diagnostica degli errori strutturati**. Ogni errore deve avere error_code, category, severity, timestamp, related_command_id e recovery_action. Gli errori non devono essere stringhe generiche. Un codice come ERR_COMMAND_EXPIRED, ERR_CAPABILITY_MISSING, ERR_THERMAL_LOCK, ERR_LOW_ENERGY, ERR_INVALID_PAYLOAD, ERR_SYNC_LOST è molto più utile. Permette al controller e alla dashboard di reagire in modo coerente.

Una struttura errore può includere:

error_code;

error_category;

severity;

timestamp_ms;

related_command_id;

affected_module;

current_mode;

recovery_action;

human_readable_message.

Le categorie possono essere:

COMMAND;
SAFETY;
ENERGY;
THERMAL;
COMMUNICATION;
SYNC;
OUTPUT;
SCHEDULER;
CONFIGURATION;
HARDWARE;
RECOVERY.

Questa tassonomia rende il debug più ordinato. Se tanti errori appartengono a CONFIGURATION, il problema non è il firmware runtime. Se molti appartengono a COMMUNICATION, bisogna controllare rete o protocollo. Se sono SAFETY, il pattern può essere troppo aggressivo.

La quattordicesima area diagnostica è la **diagnostica della configurazione**. Il nodo deve poter dichiarare config_id, config_epoch, hardware_profile, pin_map_version, safety_policy_id, telemetry_policy_id e limiti principali. Se la configurazione è incoerente, deve emergere subito. Per esempio, pin duplicati, capability dichiarata ma pin mancante, safety policy assente, hardware_profile non riconosciuto.

Un blocco config_diagnostics può contenere:

config_id;
config_epoch;
config_valid;
config_errors;
hardware_profile;
pin_map_version;
safety_policy_id;
telemetry_policy_id;
limits_profile_id;
protocol_version;
schema_version.

Se `config_valid = false`, il nodo non deve entrare in active. Deve restare in diagnostic o safe mode. La diagnostica deve indicare il motivo preciso: `duplicate_pin`, `missing_required_limit`, `unsupported_profile`, `invalid_threshold`. Questo evita comportamenti indefiniti.

La quindicesima area diagnostica è la **diagnostica dell'astrazione hardware**. La HAL deve poter indicare se è inizializzata, se i pin sono validi, se PWM è disponibile, se sensori sono disponibili, se `safe_outputs_off` è stato eseguito e se esistono errori hardware. Questo blocco è importante quando si cambia prototipo fisico.

Un blocco `hal_diagnostics` può contenere:

```
hal_initialized;  
  
hardware_profile_loaded;  
  
pin_map_valid;  
  
pwm_available;  
  
sensors_available;  
  
safe_outputs_off_last_called_ms;  
  
last_hal_error;  
  
unsupported_hardware_calls;  
  
output_driver_status.
```

Se `hal_initialized = false`, il nodo deve essere critical. Se `pin_map_valid = false`, non deve accettare output. Se `unsupported_hardware_calls` cresce, qualche modulo sta chiedendo funzioni non disponibili. La HAL non deve essere invisibile: deve dichiarare salute.

La sedicesima area diagnostica è la **diagnostica della recovery**. Dopo un errore o reset, il nodo può entrare in recovery. La diagnostica deve mostrare `recovery_type`, `recovery_step`, `recovery_started_at`, `requires_state_sync`, `sync_restored`, `capabilities_reported`, `reintegration_ready` e `recovery_result`. Questo è fondamentale per evitare rientri ciechi.

Un blocco `recovery_diagnostics` può contenere:

```
recovery_active;  
  
recovery_type;  
  
recovery_step;  
  
recovery_started_ms;  
  
requires_state_sync;  
  
state_sync_received;  
  
sync_restored;  
  
safe_state_confirmed;
```

capabilities_reported;

reintegration_ready;

recovery_error;

recovery_attempt_count.

Se recovery_active = true, il nodo non deve essere trattato come completamente operativo. Se reintegration_ready = false, il controller deve aspettare o assegnare ruolo passivo. Se recovery_attempt_count supera soglia, il nodo può diventare unstable o manual_intervention_required. La diagnostica rende questa procedura trasparente.

La diciassettesima area diagnostica è la **diagnostica locale degli eventi recenti**. Il nodo non deve conservare log infiniti, ma può mantenere un piccolo ring buffer di eventi recenti. Questo buffer permette alla shell di mostrare una timeline locale. È utile soprattutto quando il nodo si comporta in modo strano ma il controller non ha ricevuto tutti gli eventi. Gli eventi recenti aiutano a ricostruire.

Un evento diagnostico locale può contenere:

event_type;

severity;

timestamp_ms;

module;

reason;

related_command_id;

state_after_event.

Il buffer può conservare ultimi 10, 20 o 50 eventi. Gli eventi critici dovrebbero avere priorità. In caso di memoria limitata, si possono conservare solo eventi warning, error e critical. Questa scelta mantiene il nodo leggero ma utile.

La diciottesima area diagnostica è la **diagnostica di readiness**. Il controller deve sapere se il nodo è pronto per certe funzioni. La diagnostica può calcolare readiness specifiche: ready_for_heartbeat, ready_for_visual_commands, ready_for_timed_commands, ready_for_pattern, ready_for_actuation, ready_for_critical_role. Queste readiness derivano da mode, safety, energy, thermal, sync, capabilities e config.

Un blocco readiness può contenere:

ready_for_status;

ready_for_visual;

ready_for_pattern;

ready_for_timed_commands;

ready_for_actuation;

ready_for_critical_role;

not_ready_reasons.

Per esempio, un nodo può essere ready_for_visual = true ma ready_for_actuation = false perché thermal_source = unavailable o energy_state = low_energy. Questa distinzione è molto più utile di un generico online. Permette assegnazioni intelligenti.

La diciannovesima area diagnostica è la **diagnostica della modalità ibrida**. Se un nodo fisico è associato a un gemello virtuale, la diagnostica deve dichiarare cosa è reale e cosa è simulato. Deve indicare mapping, sources, consistency e last update. Questo evita confusione durante demo.

Un blocco hybrid_diagnostics può contenere:

hybrid_mode_active;

hardware_node_id;

virtual_node_id;

heartbeat_source;

position_source;

energy_source;

thermal_source;

role_source;

mapping_valid;

hybrid_consistency;

last_virtual_update_ms;

last_hardware_update_ms.

Se mapping_valid = false, la dashboard non deve fidarsi della fusione. Se heartbeat reale è ok ma virtual update è stale, la rappresentazione può essere incompleta. Questa diagnostica è essenziale per mantenere onestà tecnica.

La ventesima area diagnostica è la **diagnostica di sicurezza locale** in senso ampio. Oltre a safety_state, il nodo deve poter dire se le condizioni minime di sicurezza sono rispettate: output spenti quando richiesto, emergency_stop non attivo o correttamente gestito, clear_safety possibile o no, nessun comando attuativo pendente, watchdog sano, controller visibile, limiti configurati. Questo può produrre una safety_readiness.

Un blocco safety_diagnostics può contenere:

safety_state;

emergency_stop_active;

outputs_safe;

critical_outputs_active;
clear_safety_allowed;
clear_safety_blockers;
pending_actuation_commands;
controller_lost;
watchdog_health;
safety_policy_loaded;
last_safety_action.

Questo blocco è particolarmente utile in hardware. Prima di attuazioni, il controller può richiedere safety_diagnostics. Se outputs_safe = false quando dovrebbero essere off, il nodo è critical. Se safety_policy_loaded = false, il nodo non deve attuare.

La ventunesima area diagnostica è la **diagnostica per priorità degli errori**. Non tutti i problemi hanno lo stesso peso. Il nodo deve poter classificare i problemi per severità. Un LED test fallito è meno grave di safe_outputs_off fallito. Un telemetry_debug disabilitato è meno grave di safety_service degraded. La diagnostica deve quindi produrre severity e, se possibile, priority.

Una scala può essere:

INFO: stato normale o transizione prevista;

WARNING: condizione da osservare;

ERROR: funzione compromessa ma non immediatamente pericolosa;

CRITICAL: rischio safety, output, reset, recovery o stato non sicuro.

Se esiste un errore CRITICAL attivo, diagnostic_status non può essere healthy. Se esiste un warning, può essere warning o degraded secondo tipo. Questa logica rende coerente la sintesi.

La ventiduesima area diagnostica è la **diagnostica come input per il controller**. La diagnostica non serve solo all'utente. Il controller può usarla per decisioni: escludere nodi instabili, riassegnare ruoli, ridurre carico, avviare recovery, bloccare pattern, chiedere state_sync, richiedere intervento manuale. Per questo, i dati diagnostici devono essere machine-readable, non solo testo stampato in shell. La shell può visualizzare, ma il payload diagnostico deve essere strutturato.

Un diagnostic_report inviato al controller può contenere:

node_id;
boot_id;
diagnostic_status;
service_health_summary;
readiness;

active_errors;

blocked_capabilities;

resource_summary;

reset_summary;

recommendation.

Il campo recommendation può essere semplice: continue, reduce_load, request_state_sync, keep_idle, exclude_from_pattern, manual_check_required. Anche se inizialmente generato con regole semplici, è molto utile.

La ventitreesima area diagnostica è la **diagnostica come input per la dashboard**. La dashboard deve poter mostrare salute del nodo in modo leggibile. Non deve sommergere l'utente con cento campi, ma deve evidenziare il problema principale. Per esempio: "MB-004 degraded — thermal_warning — pwm_output blocked — cooldown 12s". Oppure: "MB-002 recovery_required — boot_id changed — state_sync pending". Questo richiede che la diagnostica fornisca un summary.

Un diagnostic_summary può contenere:

status;

main_issue;

secondary_issue;

blocked_capabilities;

recommended_action;

last_event;

confidence.

Questo è molto utile per UI. L'utente vede subito cosa conta, poi può aprire dettagli.

La ventiquattresima area diagnostica è la **diagnostica locale tramite shell**. Molti dati diagnostici possono essere interrogati con comandi shell: status, tasks, resources, errors, events, safety, output, energy, thermal, comm, config, hal. La shell deve essere vista come interfaccia testuale della diagnostica. Ogni comando shell dovrebbe leggere dati da strutture diagnostiche, non inventare output separati. Questo mantiene coerenza tra shell, telemetria e dashboard.

Una regola importante è:

la shell deve visualizzare lo stesso stato diagnostico che il nodo invia al controller, solo in forma leggibile per l'utente.

Se shell e telemetria usano fonti diverse, possono contraddirsi. Questo va evitato. Meglio avere una struttura diagnostic_state centrale.

La venticinquesima area diagnostica è la **diagnostica automatica periodica**. Alcuni controlli possono essere eseguiti periodicamente senza richiesta: service health, output state, buffer usage, controller_lost, error counters. Se qualcosa supera una soglia, il nodo genera evento. Questo rende il nodo proattivo. Non aspetta che l'utente chieda.

Controlli periodici utili:

task critici aggiornati;

buffer non pieni;

output non oltre durata;

controller visibile;

error_count sotto soglia;

reset_count non eccessivo;

energy/thermal non critici;

capabilities coerenti.

Se un controllo fallisce, si genera warning o error. Se è critico, safety action. Questo collega diagnostica e protezione.

La ventiseiesima area diagnostica è la **self-test routine**. Il nodo deve poter eseguire un test automatico, specialmente all'avvio o su richiesta shell. Il self-test non deve attivare funzioni rischiose senza autorizzazione. Deve controllare identità, HAL, pin map, scheduler, servizi, telemetria, safety, output safe state. In versioni più avanzate può testare LED, sensori e comunicazione.

Un self_test_basic può controllare:

identity presente;

boot_id valido;

config valida;

HAL inizializzata;

pin_map valido;

safe_outputs_off funziona;

scheduler attivo;

safety_service healthy;

telemetry_service healthy;

communication_service inizializzato;

nessun errore critico attivo.

Il risultato può essere pass, pass_with_warnings, fail. Se fail, il nodo deve entrare in diagnostic o safe mode. Questa routine rende il nodo più affidabile prima di partecipare allo swarm.

La ventisettesima area diagnostica è la **diagnostica dei limiti e delle soglie**. Molti problemi derivano da soglie sbagliate: timeout troppo brevi, cooldown troppo corti, thermal warning troppo sensibile, heartbeat troppo lento, buffer troppo piccolo. La diagnostica deve esporre soglie correnti e contatori di superamento. Se una soglia viene superata spesso, può essere necessario ricalibrarla.

Un blocco thresholds_diagnostics può contenere:

controller_lost_timeout_ms;

heartbeat_interval_ms;

max_pwm_duty;

max_pulse_duration_ms;

cooldown_ms;

thermal_warning_threshold;

thermal_lock_threshold;

low_energy_threshold;

critical_energy_threshold;

threshold_cross_counts.

Questo permette di capire se i parametri sono realistici. Se controller_lost scatta spesso ma la rete è stabile, timeout forse troppo breve. Se thermal_warning scatta immediatamente per test LED, il modello termico è troppo aggressivo. La diagnostica delle soglie aiuta la calibrazione.

La ventottesima area diagnostica è la **diagnostica dei limiti della versione**. Ogni versione di MicroBot OS supporta solo certe funzioni. La diagnostica deve poter dire “questa funzione non è supportata in questa versione”. Questo evita ambiguità. Se un utente chiede sync ma MicroBot OS v0.3 non supporta sync, la risposta deve essere unsupported_version, non un valore vuoto.

Un blocco version_diagnostics può contenere:

microbot_os_version;

firmware_version;

protocol_version;

supported_commands;

unsupported_features;

experimental_features;

deprecated_features;

minimum_controller_version.

Questo è essenziale quando si lavora con roadmap evolutiva. Non tutti i nodi saranno allo stesso livello. La diagnostica deve rendere la maturità chiara.

La ventinovesima area diagnostica è la **diagnostica come base per bug report**. Quando un problema avviene, il nodo dovrebbe poter produrre un piccolo diagnostic snapshot. Questo snapshot può essere copiato dalla shell o inviato al controller. Deve contenere stato, errori, task, risorse, output e ultimi eventi. Questo diventa materiale per debug e documentazione.

Un diagnostic_snapshot può contenere:

identity;

status;

diagnostic_status;

active_errors;

last_events;

task_summary;

resource_summary;

output_summary;

safety_summary;

communication_summary;

reset_summary;

recommendation.

Questo snapshot è molto utile perché cattura il problema nel momento in cui avviene. Invece di descrivere a memoria “mi pare che non andasse”, si ha un blocco tecnico concreto.

La trentesima area diagnostica è la **roadmap della diagnostica**. Come tutto MicroBot OS, anche la diagnostica deve crescere per versioni. All’inizio basta status, last_error e heartbeat. Poi si aggiungono error codes, eventi, task health, resource counters, reset reason, command diagnostics, safety diagnostics, HAL diagnostics, self-test, recovery diagnostics e diagnostic snapshot. Questa progressione è realistica e mantiene il firmware gestibile.

Una roadmap possibile è:

v0.1: boot log e status minimo;

v0.2: heartbeat diagnostics;

v0.3: last_error e command counters;

v0.4: role/mode diagnostics;

v0.5: event buffer locale;
v0.6: safety diagnostics;
v0.7: scheduler/task diagnostics;
v0.8: shell diagnostics completa;
v0.9: reset/watchdog diagnostics;
v1.0: HAL/config diagnostics;
v1.1: recovery diagnostics;
v1.2: energy/thermal diagnostics;
v1.3: output/actuation diagnostics;
v1.4: hybrid diagnostics;
v1.5: diagnostic snapshot e self-test avanzato.

Questa roadmap chiude la Sezione 14 in modo coerente: MicroBot OS evolve da firmware osservabile a runtime embedded diagnosticabile.

In conclusione, la diagnostica è il livello di MicroBot OS che consente al nodo di descrivere la propria salute interna, i propri errori, le proprie risorse, i propri task, la propria configurazione, la propria comunicazione, i propri output, la propria safety e la propria capacità di partecipare allo swarm. Essa trasforma il MicroBot da semplice esecutore di comandi a sistema ispezionabile e autovalutabile. La diagnostica raccoglie service health, scheduler health, reset reason, command counters, capabilities, output state, energy, thermal, communication, sync, recovery, HAL status, config status, readiness e ultimi eventi. Questi dati possono essere mostrati nella shell, inviati nella telemetria, usati dalla dashboard e interpretati dal controller. Con la diagnostica, ogni nodo può dire non solo “sono acceso”, ma “sono in questo stato, con queste capacità, questi limiti, questi errori e questa idoneità operativa”. Con questo punto si chiude la Sezione 14 dedicata a MicroBot OS. La sezione successiva, dedicata alla Sicurezza Biometrica, potrà ora descrivere il livello di autenticazione, controllo accessi, riconoscimento facciale, landmark detection, confronto dati e decisione tramite soglia, mantenendo però la stessa logica già costruita: stato osservabile, sicurezza locale, decisioni tracciabili e integrazione progressiva nel sistema complessivo MicroBot.

SEZIONE 15 — Sicurezza Biometrica

La Sezione 15 introduce il livello di **sicurezza biometrica** del sistema MicroBot, cioè l’insieme delle funzioni pensate per controllare l’accesso al sistema attraverso una forma di riconoscimento dell’utente, in particolare tramite fotocamera, rilevamento del volto, landmark detection, confronto dei dati e decisione tramite soglia. Dopo la Sezione 14, dedicata a MicroBot OS e alla struttura interna dei nodi, questa sezione sposta l’attenzione su un livello diverso: non più soltanto il funzionamento tecnico dei MicroBot, ma il controllo di chi può interagire con il sistema, accedere alla dashboard, inviare comandi, entrare in modalità diagnostica o autorizzare funzioni sensibili.

Nel progetto MicroBot, la sicurezza biometrica non deve essere interpretata come una funzione isolata o decorativa. Non è semplicemente “mettere una webcam che riconosce una faccia”. Deve essere vista come un possibile **access control layer**, cioè un livello di autorizzazione che protegge

l'interazione con un sistema distribuito. MicroBot, infatti, non è solo una simulazione visuale: nella visione completa può includere nodi fisici, firmware, telemetria, attuatori, comandi, modalità diagnostiche, safety, dashboard e potenzialmente hardware reale. Per questo motivo, non tutte le persone dovrebbero poter accedere allo stesso livello di controllo. Un visitatore può osservare una demo; uno sviluppatore può aprire diagnostica; un operatore autorizzato può avviare pattern; un amministratore può modificare configurazioni critiche. La biometria può diventare uno dei meccanismi per distinguere questi livelli.

È importante chiarire subito un punto: la sicurezza biometrica, soprattutto nelle prime versioni del progetto, deve essere trattata come **prototipo sperimentale**, non come sistema di sicurezza definitivo per ambienti critici. Il riconoscimento facciale tramite fotocamera può essere utile per demo, autenticazione locale, controllo accessi sperimentale e interfacce intelligenti, ma richiede grande attenzione. I dati biometrici sono sensibili. Un sistema serio deve minimizzare ciò che salva, dichiarare cosa viene elaborato, evitare raccolte inutili, proteggere i dati e preferire confronti locali quando possibile. In questa documentazione, quindi, la sicurezza biometrica viene descritta come modulo progettuale progressivo, integrabile con MicroBot, ma da sviluppare con prudenza.

Il principio generale della Sezione 15 può essere espresso così:

la sicurezza biometrica deve autorizzare l'accesso al sistema senza compromettere privacy, tracciabilità, sicurezza operativa e controllo locale dei dati.

Questa frase è fondamentale. Il modulo biometrico non deve diventare una scorciatoia per bypassare il Safety Layer. Anche se un utente viene riconosciuto, non deve poter inviare comandi pericolosi senza che controller, MicroBot OS e safety_manager li validino. Il riconoscimento biometrico può autorizzare un livello di accesso, ma non sostituisce le protezioni tecniche. In altre parole, autenticazione e safety sono livelli diversi. L'autenticazione risponde alla domanda "chi sta chiedendo accesso?". Il Safety Layer risponde alla domanda "questa azione è sicura nello stato attuale?". Entrambe devono essere vere per funzioni sensibili.

La sicurezza biometrica può essere collegata a più parti dell'ecosistema MicroBot. Può proteggere la dashboard web, impedendo a utenti non riconosciuti di entrare in modalità controllo. Può proteggere la shell diagnostica remota, se in futuro esposta oltre la seriale locale. Può autorizzare l'avvio di demo hardware. Può bloccare modifiche di configurazione. Può controllare l'accesso a report o dati sperimentali. Può distinguere modalità demo, developer, diagnostic e administrator. Può anche diventare una parte interattiva dell'esperienza: il sistema riconosce l'utente, abilita il profilo corretto e configura l'interfaccia in base ai permessi.

I punti dell'indice della Sezione 15 sono: autenticazione tramite fotocamera, controllo degli accessi, riconoscimento facciale, landmark detection, confronto dei dati e decisione tramite soglia. Questi punti descrivono una pipeline completa. La fotocamera acquisisce il frame. Il sistema rileva la presenza del volto. La landmark detection individua punti caratteristici. Il modulo di riconoscimento produce una rappresentazione o embedding. Il sistema confronta questa rappresentazione con un riferimento autorizzato. La decisione finale viene presa tramite una soglia. Se il confronto supera la soglia in modo affidabile, l'accesso viene concesso secondo il livello previsto. Se non supera la soglia, l'accesso resta negato o limitato.

La pipeline può essere sintetizzata così:

acquisizione immagine;

rilevamento volto;

estrazione landmark;
normalizzazione del volto;
generazione rappresentazione biometrica;
confronto con riferimento;
calcolo score di similarità;
decisione tramite soglia;
assegnazione livello di accesso;
logging dell'evento di autenticazione.

Questa pipeline deve essere progettata in modo modulare. La fotocamera non deve decidere l'accesso. Il rilevatore del volto non deve gestire i permessi. Il modulo di confronto non deve inviare comandi ai MicroBot. Ogni livello deve avere responsabilità chiara. Questo mantiene il sistema leggibile e sicuro.

La biometria deve essere integrata con la telemetria, ma con separazione. Gli eventi di sicurezza possono essere registrati: `auth_attempt`, `auth_success`, `auth_failed`, `access_granted`, `access_denied`, `threshold_not_met`, `face_not_detected`, `multiple_faces_detected`. Tuttavia, non è necessario salvare immagini o dati biometrici grezzi nei log generici. Anzi, è meglio evitarlo. La telemetria generale deve sapere che un tentativo di autenticazione è avvenuto e quale decisione è stata presa, ma non deve necessariamente contenere dati sensibili del volto. Questo è un punto di rigore molto importante.

Una distinzione corretta può essere:

`technical_telemetry`: stato del sistema, nodi, `safety`, comandi;
`security_events`: tentativi di autenticazione e decisioni;
`biometric_processing_data`: dati temporanei usati per confronto;
`stored_biometric_reference`: eventuale riferimento protetto e minimizzato;
`general_logs`: non devono contenere immagini biometriche grezze.

Questa separazione prepara una struttura più sicura. Il modulo biometrico deve essere trattato come componente sensibile, non come normale pannello della dashboard.

La sicurezza biometrica deve anche prevedere casi di errore. Se la fotocamera non è disponibile, il sistema non deve fallire in modo ambiguo. Deve indicare `camera_unavailable`. Se nessun volto viene rilevato, `face_not_detected`. Se vengono rilevati più volti, `multiple_faces_detected` e accesso negato o richiesta di isolamento del volto. Se la qualità dell'immagine è bassa, `low_confidence`. Se il confronto è vicino alla soglia ma non sufficiente, `access_denied_uncertain`. Se il sistema non ha riferimento biometrico registrato, `no_biometric_profile_configured`. Questi stati devono essere espliciti.

Stati possibili del modulo biometrico:

camera_ready;
camera_unavailable;
face_detected;
face_not_detected;
multiple_faces_detected;
landmarks_detected;
landmarks_failed;
embedding_generated;
comparison_ready;
auth_success;
auth_failed;
auth_uncertain;
access_granted;
access_denied.

Questi stati sono coerenti con il resto del progetto: anche qui, ogni modulo deve essere osservabile, diagnosticabile e integrabile con dashboard e log.

La sicurezza biometrica deve avere una relazione chiara con i livelli di accesso. Non tutti gli accessi sono uguali. Un riconoscimento biometrico può concedere modalità diverse. Per esempio, una persona non autenticata può vedere solo la demo pubblica. Una persona riconosciuta come developer può accedere a diagnostica e test simulativi. Una persona riconosciuta come admin può modificare configurazioni. Una persona riconosciuta come operator può avviare una demo controllata. Tuttavia, anche un admin non deve poter bypassare safety hardware. La safety resta sempre superiore.

Una possibile struttura di accesso è:

guest: sola visualizzazione;
viewer: dashboard read-only;
operator: avvio demo sicure;
developer: diagnostica, replay, simulazione, log;
admin: configurazione e gestione profili;
hardware_admin: test hardware controllati, sempre sotto Safety Layer.

Questa struttura può essere implementata in futuro nella dashboard. Nelle prime versioni, può essere soltanto documentata o simulata. L'importante è separare identità, ruolo utente e autorizzazioni.

La sicurezza biometrica deve supportare anche fallback non biometrici. Un sistema serio non deve dipendere da una sola modalità, soprattutto se sperimentale. Se la fotocamera non funziona, può essere previsto accesso manuale locale, password di sviluppo, chiave fisica, token, conferma seriale o modalità read-only. Questo non significa indebolire la sicurezza: significa avere percorsi controllati. La biometria può essere comoda e dimostrativa, ma deve essere parte di un sistema di accesso più ampio.

La regola è:

il riconoscimento biometrico può autorizzare l'interfaccia, ma non deve essere l'unico meccanismo logico di sicurezza del sistema.

Questa frase è importante perché evita di sovraccaricare la biometria di responsabilità eccessive. La protezione vera del sistema nasce dalla combinazione di access control, Safety Layer, command validation, logging e modalità operative.

15.1 Autenticazione tramite fotocamera

L'autenticazione tramite fotocamera rappresenta il primo livello operativo della sicurezza biometrica MicroBot. Essa consiste nell'utilizzo di una videocamera, integrata o esterna, per acquisire l'immagine dell'utente che sta tentando di accedere al sistema, alla dashboard o a una specifica modalità di controllo. In questa fase, la fotocamera non riconosce ancora da sola l'identità dell'utente: fornisce il dato visivo da cui il sistema potrà successivamente rilevare il volto, estrarre landmark, generare una rappresentazione biometrica e confrontarla con un riferimento autorizzato. La fotocamera è quindi il punto di ingresso della pipeline, non il meccanismo decisionale finale.

Nel progetto MicroBot, l'autenticazione tramite fotocamera può essere integrata inizialmente nella piattaforma web. Una dashboard locale può richiedere l'accesso alla webcam del computer e mostrare uno stato di autenticazione prima di abilitare controlli avanzati. In una demo, l'utente apre la dashboard, il sistema attiva la fotocamera, rileva il volto, verifica se l'utente è autorizzato e poi abilita o nega funzioni. Questa integrazione è coerente con l'estetica e la struttura già costruita per MicroBot: dashboard, pannelli, telemetria, stati, eventi e livelli di accesso.

La fotocamera deve però essere gestita con attenzione. Prima di tutto, deve esserci una distinzione tra camera_stream e biometric_processing. Il camera_stream è il flusso video grezzo. Il biometric_processing è l'insieme delle operazioni che analizzano quel flusso. Idealmente, il flusso video non dovrebbe essere salvato nei log. Dovrebbe essere elaborato localmente nel browser o nel backend locale, e solo il risultato dell'autenticazione dovrebbe essere registrato come evento. Questo riduce il rischio di trattare dati sensibili inutilmente.

Una pipeline corretta può essere:

richiesta autorizzazione fotocamera;

attivazione camera_stream locale;

rilevamento del volto nel frame;

controllo qualità dell'immagine;

estrazione dati necessari;

confronto con riferimento;

decisione;

disattivazione o mantenimento controllato dello stream;

registrazione solo dell'evento di accesso, non del frame grezzo.

Questo approccio è più prudente rispetto a salvare immagini o video. Il sistema deve evitare raccolta eccessiva. La sicurezza biometrica deve essere progettata con minimizzazione dei dati.

Il primo requisito dell'autenticazione tramite fotocamera è la **disponibilità del dispositivo**. Il sistema deve verificare se una fotocamera è presente e accessibile. Se non è disponibile, deve produrre uno stato chiaro: `camera_unavailable`. Se l'utente non concede permesso al browser o all'applicazione, lo stato può essere `camera_permission_denied`. Se la fotocamera è occupata da un'altra applicazione, `camera_busy`. Questi stati devono essere mostrati nella dashboard e registrati come eventi di sicurezza, senza bloccare l'intero sistema in modo confuso.

Stati camera possibili:

`camera_initializing`;

`camera_ready`;

`camera_permission_denied`;

`camera_unavailable`;

`camera_busy`;

`camera_error`;

`camera_stream_active`;

`camera_stream_stopped`.

Questi stati permettono alla dashboard di spiegare cosa succede. Se l'autenticazione fallisce perché la fotocamera non è disponibile, non deve apparire come volto non riconosciuto. Sono due cause diverse. Il sistema deve distinguere problema tecnico e fallimento biometrico.

Il secondo requisito è il **controllo del consenso locale**. La fotocamera deve essere attivata solo quando serve. In una dashboard web, il browser chiede già autorizzazione all'utente, ma il progetto deve comunque definire una logica chiara: la fotocamera si attiva per autenticazione, non per sorveglianza continua non dichiarata. Se il sistema resta in modalità read-only, può non richiedere fotocamera. Se l'utente chiede accesso a funzioni operator o developer, può attivarla. Dopo la decisione, lo stream può essere disattivato o mantenuto solo se necessario per session monitoring, se questa funzione sarà prevista.

Una policy prudente può essere:

camera off di default;

camera on solo durante auth_attempt;

nessun salvataggio frame grezzo nei log;

nessun invio remoto non necessario;

stream disattivato dopo decisione, salvo modalità esplicita;

eventi salvati senza immagini.

Questa policy rende la biometria più accettabile e più seria. Non serve accumulare dati visivi per dimostrare l'accesso.

Il terzo requisito è il **controllo della qualità dell'immagine**. Un riconoscimento facciale affidabile dipende molto dalla qualità del frame. Illuminazione scarsa, volto parziale, movimento, sfocatura, distanza eccessiva, angolo troppo inclinato o occlusioni possono ridurre la qualità. Il sistema deve quindi valutare se il frame è adatto prima di procedere. Se non lo è, deve produrre low_image_quality o face_not_suitable, invece di prendere una decisione debole.

Indicatori di qualità possono essere:

face_visible;

face_centered;

face_size_sufficient;

brightness_ok;

blur_low;

single_face_present;

landmark_confidence;

pose_within_limits.

Questi indicatori non devono essere necessariamente tutti implementati subito. Ma concettualmente sono importanti. Un sistema biometrico non dovrebbe dire "utente non autorizzato" se il volto non è visibile correttamente. Dovrebbe dire "qualità insufficiente per autenticare". Questa distinzione migliora affidabilità e usabilità.

Il quarto requisito è la **rilevazione della presenza del volto**. Prima di riconoscere un utente, il sistema deve rilevare se nel frame è presente un volto. Questo è diverso dal riconoscimento. Face detection risponde alla domanda: "c'è un volto nel frame?". Face recognition risponde alla domanda: "di chi è questo volto?". L'autenticazione tramite fotocamera inizia dalla face detection. Se nessun volto viene rilevato, la pipeline non procede. Se più volti vengono rilevati, il sistema deve decidere cosa fare. Per sicurezza, può negare l'accesso o chiedere che una sola persona sia inquadrata.

Stati di rilevamento possono essere:

no_face_detected;
single_face_detected;
multiple_faces_detected;
face_partially_visible;
face_detection_low_confidence.

Per MicroBot, multiple_faces_detected dovrebbe portare a access_denied o auth_uncertain, almeno nelle versioni iniziali. Un sistema di accesso deve sapere chi sta autorizzando l'azione. Se ci sono più persone nel frame, la decisione diventa ambigua.

Il quinto requisito è la **gestione della sessione di autenticazione**. Un auth_attempt deve avere un identificatore, un inizio, una fine, uno stato e un risultato. Non deve essere solo una funzione anonima. Questo permette di loggare eventi in modo strutturato. Per esempio, auth_attempt_id, timestamp_start, timestamp_end, camera_status, face_detection_status, match_score, threshold, decision, access_level_granted. Non bisogna salvare immagini, ma si può salvare la decisione e i metadati non biometrici essenziali.

Una struttura auth_attempt può contenere:

auth_attempt_id;
timestamp_start;
timestamp_end;
camera_status;
face_detection_status;
landmark_status;
comparison_status;
match_score, se non sensibile secondo policy;
threshold_used;
decision;
access_level_granted;
failure_reason.

Questa struttura rende l'autenticazione tracciabile. Se l'accesso viene negato, si sa perché: camera_error, no_face_detected, multiple_faces_detected, low_confidence, score_below_threshold, no_profile_configured. Questa tracciabilità è utile senza necessariamente salvare dati biometrici grezzi.

Il sesto requisito è la **separazione tra autenticazione e autorizzazione**. La fotocamera serve per autenticare, cioè verificare l'identità o la corrispondenza con un profilo. L'autorizzazione decide cosa può fare quell'identità. Sono due passaggi diversi. Un utente può essere riconosciuto, ma avere

solo permessi viewer. Un altro può essere riconosciuto come developer. Un altro come admin. Il modulo camera non deve decidere direttamente quali comandi MicroBot inviare; deve produrre un risultato di autenticazione che il modulo access_control interpreta.

La pipeline corretta è:

camera acquisisce;

biometric module autentica;

access_control assegna livello;

dashboard abilita o disabilita funzioni;

controller e Safety Layer validano comunque ogni comando.

Questo mantiene ordine. La fotocamera non deve essere collegata direttamente ad attuazioni o modalità hardware. Deve passare dal sistema di permessi.

Il settimo requisito è la **modalità read-only senza autenticazione**. In molte demo può essere utile permettere a chiunque di vedere la dashboard in sola lettura, senza attivare la camera. Questo riduce attrito e migliora privacy. L'autenticazione tramite camera può essere richiesta solo quando si vuole passare da viewer a operator/developer/admin. Questo modello è sensato per MicroBot perché separa osservazione e controllo.

Una struttura possibile è:

dashboard pubblica locale: read-only;

click su “enable controls”: richiede autenticazione;

click su “diagnostic mode”: richiede livello developer;

click su “hardware commands”: richiede livello hardware_admin più Safety Layer;

click su “configuration”: richiede admin.

Questo rende la biometria utile senza renderla invasiva. Non serve riconoscere l'utente solo per guardare una simulazione.

L'ottavo requisito è la **gestione dei permessi dopo autenticazione**. Dopo auth_success, il sistema può creare una sessione locale con access_level, scadenza e stato. La sessione non deve durare indefinitamente. Può scadere dopo un certo tempo, dopo inattività o dopo chiusura della dashboard. Se viene rilevato logout, sessione scaduta o camera disattivata, i controlli avanzati vengono bloccati. Questo evita che una persona autenticata lasci il sistema aperto e qualcun altro usi i comandi.

Una sessione di accesso può contenere:

session_id;

authenticated_user_label, se previsto;

access_level;

issued_at;
expires_at;
last_activity;
auth_method;
session_status.

La sessione deve essere locale e controllata. Per prototipi, può essere semplice. Per sistemi più seri, dovrebbe essere protetta meglio. Ma la logica concettuale deve essere presente.

Il nono requisito è il **blocco automatico dei controlli sensibili**. Se l'autenticazione scade o fallisce, la dashboard deve disabilitare controlli sensibili. Non deve solo mostrare un messaggio. I pulsanti per start_pattern, diagnostic mode, hardware test, config edit, clear_safety o output commands devono essere disabilitati o richiedere nuova autenticazione. La UI deve riflettere chiaramente il livello di accesso.

Controlli da proteggere:

start hardware demo;
enter diagnostic mode;
send command to node;
clear_safety;
modify config;
run hardware actuation test;
change thresholds;
access biometric settings;
view sensitive security logs.

Controlli che possono restare disponibili in read-only:

view dashboard;
view simulation replay;
view public metrics;
view non-sensitive documentation;
start pure local visual demo, se non pericolosa.

Questa distinzione rende il sistema più sicuro e più usabile.

Il decimo requisito è la **gestione degli errori di camera nella dashboard**. L'interfaccia deve comunicare chiaramente lo stato della fotocamera. Se la camera non parte, deve dire cosa succede.

Se il permesso è negato, deve spiegare che non può autenticare. Se non c'è volto, deve dire di posizionarsi davanti alla camera. Se ci sono più volti, deve richiedere un solo volto. Se la qualità è bassa, deve segnalare luce o posizione. La dashboard deve essere tecnica ma comprensibile.

Messaggi possibili:

camera unavailable;

permission denied;

face not detected;

multiple faces detected;

low confidence frame;

authentication failed;

authentication successful;

access level granted: developer;

session expired.

Questi messaggi devono essere accompagnati da stati interni coerenti. Non basta testo; deve esserci anche stato per telemetria e log.

L'undicesimo requisito è la **minimizzazione dei dati biometrici**. Il sistema dovrebbe evitare di salvare frame grezzi. Se serve un riferimento biometrico, dovrebbe essere salvato in forma derivata e protetta, non come immagine libera. Per la documentazione del progetto, è sufficiente affermare che nelle versioni iniziali il sistema dovrebbe preferire elaborazione locale, salvataggio minimo e logging dei soli eventi di autenticazione. Questo è importante perché dati biometrici sono molto delicati.

Una policy di minimizzazione può essere:

non salvare video stream;

non salvare frame grezzi nei log generali;

salvare solo eventi auth_success/auth_failed;

usare reference template solo se necessario;

proteggere eventuali reference;

permettere cancellazione profili biometrici;

segnalare chiaramente quando la camera è attiva.

Questa policy rende il progetto più responsabile. Anche se è un prototipo, l'impostazione deve essere seria.

Il dodicesimo requisito è la **separazione dei log biometrici dai log tecnici**. I log tecnici del sistema MicroBot contengono nodi, pattern, safety, telemetria, errori e comandi. I log di sicurezza

contengono tentativi di accesso. I dati biometrici non devono finire nei log tecnici generali. Questo evita confusione e riduce rischi. Un `event_log` tecnico può contenere `access_granted` o `access_denied`, ma non immagini o dettagli biometrici grezzi.

Una separazione possibile è:

`microbot_events.log`: pattern, nodi, safety, telemetria;

`security_events.log`: auth attempt, access decision, access level;

`biometric_templates`: archivio separato e protetto, se usato;

`debug_camera_data`: disabilitato di default e non persistente.

Questa architettura è più pulita. La sicurezza biometrica resta un modulo separato ma integrato.

Il tredicesimo requisito è la **compatibilità con simulazione e demo**. Nelle prime versioni, la biometria può essere simulata. Per esempio, la dashboard può avere modalità `mock_auth_success` o `mock_auth_failed` per testare `access_control` senza usare davvero la camera. Questo è utile per sviluppare UI e permessi. Tuttavia, la modalità mock deve essere chiaramente indicata, per evitare confusione.

Modalità possibili:

`real_camera_auth`;

`mock_auth_success`;

`mock_auth_failed`;

`manual_developer_unlock`;

`read_only_no_auth`.

In una demo tecnica, `mock_auth` può mostrare il flusso senza trattare dati biometrici. In una demo più avanzata, `real_camera_auth` usa la fotocamera. Entrambe devono essere dichiarate. Questo è coerente con la filosofia MicroBot: distinguere simulato, reale e ibrido.

Il quattordicesimo requisito è la **gestione del profilo utente**. Se il sistema riconosce utenti diversi, deve associare profili e permessi. Un profilo può contenere `user_label`, `access_level`, `biometric_reference_id`, `enabled`, `created_at`, `last_auth_success`. Tuttavia, il progetto deve evitare di salvare dati personali inutili. Il profilo può essere locale e minimale. Per una versione prototipale, può anche esistere un solo profilo developer.

Un profilo minimale può contenere:

`profile_id`;

`display_label`;

`access_level`;

`biometric_reference_id`;

`enabled`;

created_at;

last_successful_auth;

notes opzionali.

La parte biometrica effettiva deve restare separata e protetta. Il display_label può essere “developer” o “operator”, non necessariamente nome reale, se non serve. Questo riduce sensibilità.

Il quindicesimo requisito è il **collegamento con i ruoli MicroBot**. Bisogna distinguere ruoli utente e ruoli nodo. Un utente può essere developer o admin. Un nodo può essere leader, connector o terminal. Sono concetti diversi. La documentazione deve evitare ambiguità. Il modulo biometrico assegna access_level umano; il controller assegna role ai nodi. I due sistemi possono interagire: solo un utente developer può assegnare ruoli ai nodi, ma non diventa “leader” dello swarm in senso tecnico.

Una distinzione chiara è:

user_access_level = permesso umano nella dashboard;

node_role = funzione di un MicroBot nel pattern;

system_mode = modalità globale del sistema;

node_mode = stato operativo del singolo nodo.

Questa distinzione mantiene il linguaggio ordinato.

Il sedicesimo requisito è la **protezione della modalità diagnostica**. La modalità diagnostica può esporre comandi, log, shell, configurazioni e test hardware. Non dovrebbe essere accessibile senza autorizzazione. La biometria può essere usata per sbloccare diagnostic_mode nella dashboard. Tuttavia, la shell seriale locale resta un caso diverso: è accesso fisico al nodo. In futuro, se la shell fosse remota, dovrebbe essere protetta. Per ora, la biometria può proteggere la diagnostica web.

Accesso diagnostic può richiedere:

auth_success;

access_level >= developer;

session non scaduta;

system_state non critical;

hardware commands separati da ulteriore conferma;

logging dell'accesso.

Questo rende la diagnostica controllata. Non basta aprire la pagina per mandare comandi.

Il diciassettesimo requisito è la **protezione dei comandi hardware**. Anche se un utente viene riconosciuto, i comandi hardware devono richiedere condizioni aggiuntive. Per esempio, access_level hardware_admin, nodo in diagnostic_mode, Safety Layer ok, output non bloccato,

comando entro limiti, conferma manuale. La biometria autorizza l'utente, ma la sicurezza tecnica autorizza l'azione.

Una pipeline per comando hardware può essere:

utente autenticato;

access_level sufficiente;

sessione valida;

dashboard abilita comando;

controller valida stato;

nodo valida safety locale;

output_manager applica limiti;

telemetria registra comando.

Questo doppio controllo è fondamentale. Non deve mai esserci "utente riconosciuto quindi comando sempre valido". Sarebbe pericoloso.

Il diciottesimo requisito è la **gestione dell'incertezza**. Il riconoscimento biometrico non è assoluto. Può avere score alto, medio o basso. Il sistema deve usare una soglia. Ma vicino alla soglia può esistere incertezza. Se lo score è appena sotto o appena sopra, si può richiedere una seconda acquisizione, un fattore aggiuntivo o limitare l'accesso a read-only. Non bisogna trattare tutti i casi come nettamente sì/no se la confidenza è bassa.

Stati decisionali possibili:

auth_success_high_confidence;

auth_success_low_confidence;

auth_failed;

auth_uncertain;

retry_required;

fallback_auth_required.

Per funzioni sensibili, è meglio richiedere alta confidenza. Per accesso read-only, può bastare meno o non servire biometria. Questo approccio è più robusto.

Il diciannovesimo requisito è la **protezione contro uso improprio della camera**. Anche in prototipo, bisogna evitare che la camera venga lasciata attiva senza motivo. La dashboard dovrebbe mostrare un indicatore camera_active. Dovrebbe esserci un comando stop_camera. La sessione dovrebbe poter continuare senza stream continuo, se non serve. Se si vuole session monitoring continuo, va dichiarato come funzione separata e più delicata.

Una policy semplice è:

camera_active visibile;
stop_camera disponibile;
stream spento dopo autenticazione;
nessun salvataggio automatico;
riattivazione richiesta solo per nuova autenticazione.

Questa scelta è prudente e semplice.

Il ventesimo requisito è la **compatibilità con browser e piattaforma web**. Se la fotocamera viene usata in una dashboard web, bisogna considerare che il browser richiede contesto sicuro per accesso camera in molte situazioni. In locale, localhost può essere ammesso. In deployment, servirebbe HTTPS. La documentazione può indicare che la versione prototipale è pensata per uso locale. Questo evita problemi durante sviluppo.

Stati web possibili:

running_on_localhost;
camera_permission_prompt;
camera_access_granted;
camera_access_denied;
browser_not_supported;
secure_context_required.

Questi stati aiutano a capire errori pratici. Se la camera non funziona, non è sempre un problema del codice biometrico: può essere permesso browser o contesto non sicuro.

Il ventunesimo requisito è la **separazione tra UI e decisione**. La dashboard può mostrare il volto, il box di rilevamento, lo stato e il risultato. Ma la decisione deve essere presa da un modulo chiaro, non da codice UI sparso. Il frontend può avere camera_manager, face_detector, auth_client e access_control_panel. Il backend, se usato, può avere auth_service e access_policy. Questa separazione mantiene il codice più pulito.

Una struttura possibile lato web:

camera_manager.js;
face_detection.js;
biometric_compare.js;
access_control.js;
security_events.js;
ui_auth_panel.js.

Oppure lato backend:

auth_service.py;

security_policy.py;

biometric_store.py;

security_events_logger.py.

Non serve implementare tutto subito, ma la struttura concettuale deve essere chiara.

Il ventiduesimo requisito è la **protezione del riferimento biometrico**. Se il sistema usa un riferimento del volto autorizzato, questo riferimento non dovrebbe essere un'immagine libera dentro una cartella pubblica della dashboard. Deve essere separato, protetto e minimizzato. In una prima demo, si può evitare di salvare riferimenti reali e usare mock. In una versione più avanzata, si può salvare un template locale. Ma la documentazione deve indicare che il riferimento biometrico è dato sensibile.

Una struttura prudente può prevedere:

nessun riferimento biometrico nelle repo pubbliche;

nessun volto reale nei sample;

template locale non condiviso;

possibilità di cancellazione profilo;

logging solo di esito, non di immagine.

Questo è molto importante se il progetto finirà su GitHub. Non bisogna caricare dati biometrici reali.

Il ventitreesimo requisito è la **modalità di sviluppo senza dati reali**. Per sviluppare il modulo senza usare volti reali, si possono usare mock e placeholder. Per esempio, auth_success se si preme un pulsante developer locale, o dati sintetici. Questo permette di costruire access_control, dashboard e permessi senza trattare dati biometrici. Poi la fotocamera reale può essere integrata in una fase successiva.

Modalità sviluppo:

mock profile;

manual unlock locale;

sample non-biometrico;

auth simulation;

camera disabled.

Questa scelta accelera lo sviluppo e riduce rischi.

Il ventiquattresimo requisito è la **registrazione degli eventi di accesso**. Anche se non si salvano dati biometrici grezzi, gli eventi di accesso sono utili. Il sistema può registrare quando un tentativo è stato fatto, se ha avuto successo, quale access_level è stato concesso, quale motivo di fallimento è avvenuto. Questi eventi possono essere mostrati nella dashboard sicurezza, separati dai log tecnici dello swarm.

Eventi possibili:

auth_attempt_started;

camera_permission_denied;

face_not_detected;

multiple_faces_detected;

auth_success;

auth_failed;

access_granted;

access_denied;

session_expired;

manual_lock;

manual_logout.

Questi eventi aiutano a sapere chi o cosa ha abilitato i controlli. Non serve sapere immagine o volto per avere tracciabilità funzionale.

Il venticinquesimo requisito è la **durata della sessione autorizzata**. Dopo autenticazione, l'accesso non deve restare aperto indefinitamente. Una sessione può durare, per esempio, finché la pagina resta aperta o finché l'utente è attivo. In caso di inattività, scade. In caso di cambio modalità critica, può richiedere nuova conferma. In caso di emergency_stop o safety critical, l'accesso può restare ma i comandi restano bloccati dal Safety Layer.

Una policy sessione può essere:

sessione breve per funzioni hardware;

sessione media per developer dashboard;

scadenza su inattività;

logout manuale;

re-auth per comandi critici;

revoca accesso se auth_state diventa invalid.

Questo riduce rischi di uso non autorizzato dopo autenticazione iniziale.

Il ventiseiesimo requisito è la **relazione con MicroBot OS**. La sicurezza biometrica può autorizzare comandi verso nodi, ma non deve cambiare direttamente lo stato interno di MicroBot OS senza passare dal controller e dal protocollo. Per esempio, anche se un utente admin è autenticato, `clear_safety` deve essere inviato come comando strutturato, validato dal controller e poi validato dal nodo. MicroBot OS deve restare autonomo nella safety locale.

La catena corretta è:

`auth module` riconosce utente;

`access_control` abilita azione nella dashboard;

controller crea comando autorizzato;

nodo riceve comando;

MicroBot OS valida stato e safety;

azione eseguita o rifiutata.

Questa catena mantiene coerenza. La biometria abilita il canale umano, non spegne le protezioni embedded.

Il ventisettesimo requisito è la **relazione con telemetria e debug**. Se un comando viene rifiutato nonostante l'utente sia autenticato, la dashboard deve poter spiegare perché. Per esempio: "Accesso utente valido, ma comando rifiutato dal nodo: `thermal_lock`". Questo è importante. Non bisogna confondere autorizzazione umana e validazione tecnica. La telemetria deve mostrare entrambi.

Un evento può essere:

`user_authorized_command_request`;

`controller_command_sent`;

`node_command_rejected`;

`reason = thermal_lock`.

Questa sequenza è chiara. L'utente era autorizzato, ma il nodo ha fatto bene a rifiutare. Questo dimostra maturità del sistema.

Il ventottesimo requisito è la **roadmap dell'autenticazione tramite fotocamera**. Non serve partire subito con riconoscimento completo. Una progressione sensata è: prima pannello UI camera mock, poi camera access locale, poi face detection, poi landmark detection, poi confronto sperimentale, poi soglia, poi `access_control`, poi logging, poi sessione, poi integrazione con dashboard. Questa progressione è coerente con il resto del progetto.

Una roadmap possibile:

v0.1: pannello access control mock;

v0.2: richiesta accesso fotocamera locale;

v0.3: camera status e gestione errori;

v0.4: face detection base;
v0.5: controllo singolo volto;
v0.6: landmark detection;
v0.7: confronto biometrico sperimentale;
v0.8: decisione tramite soglia;
v0.9: access_level nella dashboard;
v1.0: security events e sessione;
v1.1: protezione modalità diagnostic;
v1.2: integrazione con permessi hardware-safe.

Questa roadmap evita di saltare direttamente alla parte più delicata senza avere UI, stati e policy.

Il ventinovesimo requisito è la **documentazione della funzione**. La sezione biometrica deve spiegare scopo, limiti, dati trattati, modalità, errori, stati, access_level e policy di salvataggio. Questo è importante perché la biometria è più sensibile di altre parti del progetto. Un documento security_biometrics.md dovrebbe essere chiaro e prudente.

Un documento dedicato può includere:

scopo del modulo;
modalità prototipale;
pipeline camera;
dati elaborati;
dati non salvati;
stati camera;
stati autenticazione;
livelli di accesso;
limiti;
rischi;
roadmap.

Questa documentazione protegge il progetto da interpretazioni sbagliate.

Il trentesimo requisito è la **coerenza con la visione MicroBot**. L'autenticazione tramite fotocamera non è il cuore fisico dello swarm, ma può diventare un livello di interazione uomo-sistema. MicroBot non è solo nodi che si muovono o LED che lampeggiano: è un ecosistema con dashboard, controllo, telemetria, safety, simulazione e accesso. La biometria aggiunge un elemento di controllo

intelligente: il sistema riconosce chi sta interagendo e abilita solo ciò che è permesso. Questo rafforza l'idea di piattaforma tecnologica, non solo esperimento hardware.

In conclusione, l'autenticazione tramite fotocamera è il punto di ingresso della sicurezza biometrica MicroBot. Essa permette di acquisire localmente il volto dell'utente, verificare disponibilità della camera, controllare qualità dell'immagine, rilevare la presenza di un volto e avviare una sessione di autenticazione. Tuttavia, la fotocamera non deve essere trattata come decisione finale: è solo il primo livello della pipeline. Il sistema deve distinguere camera status, face detection, qualità, confronto, soglia e autorizzazione. Deve minimizzare i dati biometrici, evitare salvataggio inutile di frame, separare log tecnici e security events, proteggere le modalità sensibili e mantenere sempre il Safety Layer come barriera indipendente. Il punto successivo, dedicato al controllo degli accessi, approfondirà come il risultato dell'autenticazione venga trasformato in livelli di permesso, modalità abilitate, blocchi della dashboard, autorizzazioni operative e protezione delle funzioni critiche del sistema MicroBot.

15.2 Controllo degli accessi

Il controllo degli accessi rappresenta il secondo livello della sicurezza biometrica MicroBot e descrive il modo in cui il risultato dell'autenticazione viene trasformato in permessi concreti all'interno del sistema. Dopo aver definito l'autenticazione tramite fotocamera, cioè il meccanismo attraverso cui il sistema acquisisce l'immagine dell'utente e avvia la pipeline biometrica, è necessario chiarire cosa accade dopo: quali funzioni vengono abilitate, quali restano bloccate, quali richiedono ulteriori conferme e quali non devono mai essere eseguibili senza il controllo del Safety Layer. Il controllo degli accessi è quindi il ponte tra riconoscimento dell'utente e capacità operative del sistema.

Nel progetto MicroBot, il controllo degli accessi è fondamentale perché il sistema non è composto soltanto da una pagina web o da una simulazione. L'ecosistema comprende dashboard, simulatore, controller, telemetria, MicroBot OS, shell diagnostica, nodi fisici, eventuali attuatori, moduli di sicurezza, report e configurazioni. Non tutte queste parti devono essere accessibili allo stesso modo. Una persona può essere autorizzata a guardare una demo, ma non a modificare parametri hardware. Un collaboratore può accedere ai log e alla diagnostica, ma non inviare comandi attuativi. Un amministratore può modificare configurazioni, ma comunque non deve poter bypassare safety locali. Il controllo degli accessi serve a trasformare il sistema da ambiente aperto e fragile a piattaforma con livelli di autorizzazione.

Il principio generale può essere espresso così:

l'autenticazione dice chi sta tentando di accedere; il controllo degli accessi decide cosa quella persona può fare.

Questa distinzione è essenziale. Riconoscere un utente non significa concedere automaticamente ogni potere sul sistema. Il riconoscimento produce una identità o un livello di confidenza. Il controllo accessi interpreta quel risultato e lo converte in permessi. Questi permessi vengono poi applicati alla dashboard, al controller, alla telemetria, ai comandi, alla configurazione e alle funzioni diagnostiche. Infine, anche se un permesso umano è valido, il comando deve comunque essere controllato dal runtime e dal Safety Layer.

Il controllo degli accessi deve quindi essere strutturato su più livelli. Il primo livello è l'accesso all'interfaccia: l'utente può aprire la dashboard? Può vedere la simulazione? Può vedere la telemetria? Il secondo livello è l'accesso alle azioni software: può avviare una simulazione, cambiare pattern, esportare report, aprire replay? Il terzo livello è l'accesso diagnostico: può vedere

log interni, stato dei nodi, errori, risorse e comandi? Il quarto livello è l'accesso hardware: può inviare comandi a nodi fisici? Il quinto livello è l'accesso critico: può modificare soglie, safety policy, configurazioni, profili biometrici o comandi di clear_safety? Questi livelli non devono essere confusi.

Una struttura coerente dei livelli può essere:

guest;

viewer;

operator;

developer;

admin;

hardware_admin.

Il livello guest può vedere una pagina pubblica o una demo limitata. Il livello viewer può osservare dashboard e telemetria in sola lettura. Il livello operator può avviare demo sicure e pattern preconfigurati. Il livello developer può accedere a diagnostica, replay, log, simulazioni e test software. Il livello admin può modificare configurazioni e gestire profili. Il livello hardware_admin può autorizzare test hardware controllati, sempre però sotto Safety Layer. Questa gerarchia rende il sistema più leggibile.

Il livello **guest** rappresenta l'utente non autenticato. In questa modalità, il sistema deve restare sicuro e limitato. Un guest può vedere una homepage, una descrizione del progetto, una simulazione dimostrativa read-only o una dashboard pubblica senza comandi. Non deve poter modificare pattern, inviare comandi, accedere a log sensibili, aprire diagnostica o avviare hardware. Il guest serve per mostrare MicroBot senza esporre controllo. Questo è utile in contesti di presentazione, GitHub demo, pitch o esposizione pubblica.

Il livello guest può permettere:

visualizzazione di una demo;

lettura di metriche pubbliche;

accesso a documentazione non sensibile;

osservazione di replay pre-caricati;

nessun comando verso nodi;

nessuna modifica di configurazione;

nessun accesso a dati biometrici o sicurezza interna.

Questa modalità è importante perché non tutto deve richiedere autenticazione. Una demo read-only può essere aperta e semplice. L'autenticazione diventa necessaria quando si passa dal guardare al controllare.

Il livello **viewer** rappresenta un utente riconosciuto o autorizzato alla sola osservazione tecnica. Può vedere dashboard live, stato dello swarm, telemetria non sensibile, pattern in esecuzione, report e stato dei nodi. Tuttavia, non può inviare comandi. È una modalità utile per collaboratori, docenti, investitori o persone che devono valutare il sistema senza modificarlo. Il viewer ha più informazione del guest, ma nessun controllo operativo.

Il viewer può vedere:

swarm status;

node status;

pattern status;

telemetria sintetica;

eventi non critici;

report validati;

snapshot e replay;

diagnostica solo ad alto livello.

Non può fare:

start/stop pattern;

clear_safety;

modificare parametri;

inviare comandi ai nodi;

accedere alla shell;

modificare profili biometrici;

avviare test hardware.

Questa distinzione è utile perché permette di condividere il sistema senza rischiare modifiche accidentali. Un viewer può imparare, osservare e validare visivamente, ma non interferire.

Il livello **operator** rappresenta un utente autorizzato a eseguire azioni controllate e predefinite. Può avviare demo sicure, scegliere pattern già validati, mettere in pausa o fermare una run, generare report e forse attivare modalità di presentazione. Non dovrebbe modificare parametri profondi o inviare comandi arbitrari. L'operator lavora dentro percorsi già progettati. È il livello adatto a una demo stabile.

L'operator può:

avviare demo preconfigurate;

fermare una demo;

mettere in pausa simulazione;
selezionare pattern approvati;
generare report;
attivare modalità presentation;
mandare safe_stop o safe_idle;
vedere warning principali.

Non dovrebbe:

modificare soglie safety;
inviare comandi manuali ai nodi;
accedere a diagnostica completa;
cambiare hardware profile;
eseguire test attuativi;
modificare utenti o profili biometrici.

Questo livello è molto importante per rendere MicroBot presentabile. Durante una demo, non serve dare accesso completo. Serve dare comandi puliti, limitati e sicuri.

Il livello **developer** rappresenta l'utente tecnico che sviluppa, testa e debuga il sistema. Può accedere a diagnostica, log, replay, configurazioni sperimentali, test simulativi, fault injection, console eventi, metriche dettagliate e strumenti di analisi. Può modificare parametri software, ma non necessariamente comandi hardware critici. Il developer può rompere una simulazione, ma non deve poter mettere a rischio hardware fisico senza ulteriori autorizzazioni.

Il developer può:

aprire debug dashboard;
vedere log completi non sensibili;
eseguire simulazioni sperimentali;
modificare behavior_profile in ambiente software;
attivare fault injection simulata;
analizzare snapshot;
aprire node inspector;
usare replay;
generare report diagnostici;

vedere `command_log`.

Il developer non deve automaticamente poter:

modificare `safety hard limits`;

avviare attuatori fisici critici;

gestire profili biometrici;

fare `clear_safety` su hardware senza procedura;

aggiornare firmware critico senza controllo;

usare shell remota critica.

Questa separazione è importante perché sviluppo software e controllo hardware non sono la stessa cosa. Un developer può avere competenza tecnica, ma il sistema deve comunque proteggere il livello fisico.

Il livello **admin** rappresenta l'utente che può gestire configurazioni, profili, policy e accessi. Può modificare livelli di permesso, gestire utenti, cambiare configurazioni globali, aggiornare profili autorizzati, definire soglie non fisicamente pericolose e gestire la dashboard. L'admin ha controllo gestionale, ma non deve poter bypassare il Safety Layer. Anche l'admin, se invia un comando non sicuro, deve ricevere un rifiuto dal nodo.

L'admin può:

gestire profili utente;

assegnare `access_level`;

abilitare o disabilitare utenti;

modificare configurazioni della dashboard;

gestire security events;

modificare policy di accesso;

resettare sessioni;

gestire modalità demo.

L'admin non dovrebbe poter fare senza ulteriori condizioni:

superare `thermal_lock`;

disabilitare watchdog;

rimuovere `safe_off`;

impostare duty hardware fuori limite;

forzare active mode su nodo in safety;

cancellare log critici senza traccia.

Questa distinzione mantiene sicurezza anche contro errore umano. Un admin gestisce accessi, ma la sicurezza fisica resta protetta da regole tecniche.

Il livello **hardware_admin** è il più delicato. Rappresenta un utente autorizzato a eseguire test hardware controllati, per esempio PWM, driver, sensori, bobine sperimentali o prove fisiche. Questo livello deve richiedere autenticazione forte, sessione valida, modalità diagnostica, conferma esplicita e Safety Layer attivo. Non deve essere concesso automaticamente solo perché il volto è riconosciuto. Può richiedere una seconda conferma o un comando manuale locale.

L'hardware_admin può:

avviare test hardware controllati;

eseguire PWM entro limiti;

testare output con auto-off;

abilitare hardware diagnostic mode;

leggere dati sensori;

modificare alcuni parametri hardware sperimentali;

fare clear_safety solo se condizioni sicure;

eseguire self_test fisico.

Ma anche hardware_admin non può:

eseguire output senza auto-off;

superare max_duration_ms;

superare max_duty;

ignorare cooldown;

ignorare thermal_lock;

ignorare critical_energy;

disabilitare emergency_stop;

disabilitare watchdog;

forzare capability bloccate.

Questa è la regola fondamentale: il livello umano più alto non elimina la safety tecnica. Il sistema deve restare ingegneristicamente difensivo.

Il controllo degli accessi deve essere applicato nella dashboard. La UI deve mostrare solo i controlli permessi o disabilitare quelli non permessi. Per esempio, un guest vede solo demo view. Un viewer vede dashboard read-only. Un operator vede pulsanti start demo e stop. Un developer vede pannelli

debug. Un admin vede gestione profili. Un hardware_admin vede test hardware, ma solo con avvisi e stato safety ok. Questo approccio riduce errori accidentali: l'utente non deve poter cliccare comandi per cui non ha permesso.

La dashboard può usare:

access_level corrente;

session_status;

enabled_controls;

disabled_controls;

reason_for_disabled_control;

security_banner;

auth_state;

safety_state.

Il campo reason_for_disabled_control è molto utile. Se un pulsante è disabilitato, la UI può dire perché: access_level_insufficient, session_expired, safety_lock, node_not_ready, hardware_mode_required, thermal_lock, low_energy. Questo evita confusione. L'utente capisce se manca autorizzazione o se il sistema è tecnicamente bloccato.

Il controllo degli accessi deve essere applicato anche lato controller, non solo lato frontend. Non basta nascondere pulsanti nella dashboard. Un utente tecnico potrebbe comunque inviare richieste manuali se il backend non controlla. Il controller deve verificare sessione, access_level e policy prima di generare comandi. La UI è il primo filtro; il controller è il secondo. Il nodo e MicroBot OS sono il terzo, tramite safety e command validation.

La catena corretta è:

frontend disabilita azioni non permesse;

controller verifica permessi prima di creare comando;

MicroBot OS valida comando localmente;

Safety Layer può rifiutare anche se permesso umano è valido.

Questa catena a più livelli è molto più robusta. Non si basa su un solo controllo.

Il controllo degli accessi deve essere associato alle sessioni. Dopo un auth_success, il sistema crea una sessione con access_level, issued_at, expires_at e last_activity. Ogni richiesta operativa deve riferirsi a una sessione valida. Se la sessione scade, i controlli vengono bloccati. Se l'utente fa logout, la sessione viene invalidata. Se il sistema entra in emergency_stop, la sessione può restare ma i comandi hardware vengono disabilitati dalla safety. Questo permette gestione ordinata.

Una sessione può contenere:

session_id;

auth_method;
access_level;
auth_confidence;
issued_at;
expires_at;
last_activity;
session_status;
allowed_actions;
security_flags.

auth_confidence può essere utile nel caso biometrico. Un accesso con bassa confidenza può concedere solo viewer o richiedere retry per operator. Un accesso con alta confidenza può concedere il livello previsto dal profilo. Questo rende la decisione più raffinata.

Il controllo degli accessi deve prevedere scadenza. Una sessione non deve restare aperta per sempre. La scadenza può dipendere dal livello. Viewer può durare più a lungo. Hardware_admin deve durare poco. Developer può avere durata intermedia. Se la sessione resta inattiva, scade. Se l'utente vuole eseguire un'azione critica dopo molto tempo, deve riautenticarsi. Questo riduce il rischio di lasciare la dashboard aperta.

Una policy possibile:

viewer_session: durata più lunga;
operator_session: durata media;
developer_session: durata media con timeout inattività;
admin_session: durata limitata;
hardware_admin_session: durata breve e re-auth per comandi critici.

Questa struttura è coerente con il livello di rischio. Più il permesso è alto, più la sessione deve essere controllata.

Il controllo degli accessi deve distinguere tra azioni read, write, control e critical. Le azioni read leggono dati. Le azioni write modificano configurazioni o stato non critico. Le azioni control avviano o fermano run. Le azioni critical riguardano hardware, safety, clear_safety, attuatori, soglie e profili biometrici. Questa classificazione è utile perché permette policy più ordinate.

Categorie di azioni:

read_status;
read_logs;
read_security_events;

start_simulation;

start_demo;

send_node_command;

enter_diagnostic_mode;

modify_config;

modify_access_policy;

clear_safety;

hardware_test;

actuation_command;

manage_biometric_profiles.

Ogni categoria richiede un access_level minimo. Per esempio, read_status può richiedere viewer. start_demo operator. read_logs developer. modify_access_policy admin. hardware_test hardware_admin. clear_safety può richiedere admin o hardware_admin più safety conditions. Questa mappa deve essere documentata.

Una tabella dei permessi può essere descritta così:

guest: può vedere contenuto pubblico e demo read-only;

viewer: può leggere dashboard e report non sensibili;

operator: può avviare demo approvate e fermarle;

developer: può usare diagnostica software, replay, log tecnici e simulazioni sperimentali;

admin: può gestire configurazioni, utenti, policy e profili;

hardware_admin: può eseguire test hardware controllati sotto Safety Layer.

Questa tabella deve essere coerente in frontend, backend e documentazione.

Il controllo degli accessi deve essere integrato con gli eventi di sicurezza. Ogni tentativo importante deve generare un evento: access_granted, access_denied, permission_denied, session_expired, critical_action_requested, critical_action_denied, hardware_access_requested, clear_safety_requested. Questi eventi non devono contenere dati biometrici grezzi. Devono contenere metadati operativi.

Un security_event può contenere:

event_id;

event_type;

timestamp;

session_id;
access_level;
action_requested;
decision;
reason;
auth_confidence;
source_interface;
safety_state_at_request.

Questo permette di ricostruire la storia degli accessi. Se un comando hardware viene negato, si sa se il motivo era permesso insufficiente o safety_lock. Questo è molto importante per audit e debug.

Il controllo degli accessi deve distinguere tra accesso negato per identità e accesso negato per stato tecnico. Se l'utente non è autenticato, reason = unauthenticated. Se è autenticato ma non abbastanza autorizzato, reason = insufficient_access_level. Se è autorizzato ma il nodo è in thermal_lock, reason = safety_block. Se la sessione è scaduta, reason = session_expired. Se la camera non funziona, reason = auth_unavailable. Queste distinzioni migliorano molto l'esperienza utente e il debug.

Motivi di negazione possibili:

unauthenticated;
auth_failed;
auth_uncertain;
session_expired;
insufficient_access_level;
action_not_allowed_in_current_mode;
safety_block;
node_not_ready;
capability_missing;
hardware_locked;
camera_unavailable;
profile_disabled.

Questi motivi devono essere mostrati in dashboard quando utile. L'utente deve sapere se deve autenticarsi di nuovo, chiedere permesso più alto, aspettare cooldown o controllare il nodo.

Il controllo degli accessi deve supportare una modalità **lockdown**. Se si verifica un evento critico, il sistema può disabilitare tutti i comandi non essenziali, anche per utenti autorizzati. Per esempio, emergency_stop, hardware fault, multiple failed auth attempts, security module error, biometric store corrupted, controller compromise suspected. In lockdown, la dashboard può diventare read-only o consentire solo safe actions. Questo è un meccanismo prudente.

Lockdown può permettere:

view status;

safe_idle;

emergency_stop;

download diagnostic snapshot;

logout;

manual recovery secondo policy.

Lockdown deve bloccare:

start pattern;

hardware test;

config changes;

clear_safety senza procedura;

biometric profile changes;

actuation commands.

Questo stato deve essere chiaramente visibile. Non deve sembrare un bug della dashboard.

Il controllo degli accessi deve prevedere protezione contro tentativi ripetuti falliti. Se l'autenticazione fallisce molte volte, il sistema può introdurre cooldown, richiedere fallback manuale o passare a modalità read-only. Questo riduce rischio di abuso e anche errori durante demo. Non serve implementare un sistema complesso nelle prime versioni, ma la policy deve essere prevista.

Una policy semplice può essere:

dopo N auth_failed consecutivi: cooldown breve;

dopo ulteriori fallimenti: accesso solo read-only;

eventuale richiesta manual_developer_unlock;

security_event con severity warning.

Questa policy deve essere prudente ma non aggressiva. In fase sperimentale, troppi blocchi possono ostacolare sviluppo. Per questo è utile distinguere development_mode e production_like_mode.

Il controllo degli accessi deve supportare modalità di sviluppo. Durante sviluppo, potrebbe essere necessario usare `mock_auth` o `manual_unlock`. Tuttavia, questa modalità deve essere chiaramente indicata nella dashboard. Se il sistema è in `mock_auth_success`, non deve essere presentato come biometria reale. Se `manual_unlock` è attivo, deve essere loggato. Questo mantiene onestà tecnica.

Stati possibili:

`real_auth_mode`;

`mock_auth_mode`;

`manual_unlock_mode`;

`read_only_mode`;

`lockdown_mode`.

La dashboard deve mostrare il `security_mode` corrente. Questo evita confusione durante demo e documentazione.

Il controllo degli accessi deve essere collegato ai profili utente. Un profilo utente può avere `profile_id`, `access_level`, `enabled`, `auth_methods`, `last_successful_auth`, `failed_attempts` e `note`. Il profilo biometrico, se esiste, deve restare separato dai dati generali. Il sistema `access_control` usa il risultato dell'autenticazione per trovare il profilo e applicare permessi. Se il profilo è `disabled`, anche un match biometrico valido non deve concedere accesso.

Una struttura profilo può contenere:

`profile_id`;

`display_label`;

`access_level`;

`enabled`;

`auth_methods_allowed`;

`biometric_reference_id`;

`created_at`;

`last_auth_success`;

`failed_attempt_count`;

`security_notes`.

Questa struttura deve essere locale e minimale nelle prime versioni. Non serve creare un sistema utenti complesso subito, ma serve definire il concetto.

Il controllo degli accessi deve proteggere la gestione dei profili biometrici. Aggiungere, modificare o cancellare un profilo biometrico è un'azione critica. Non deve essere disponibile a operator o developer normale. Dovrebbe richiedere admin, sessione valida e conferma esplicita. Inoltre, i

profili biometrici non dovrebbero essere caricati su repository pubblici. Questo deve essere chiaro nella documentazione.

Azioni biometric_profile_management:

enroll_profile;

disable_profile;

delete_profile;

update_access_level;

rotate_reference;

export non consentito di default;

view raw biometric data non consentito.

La gestione profili è più sensibile della semplice autenticazione. Deve essere isolata.

Il controllo degli accessi deve essere legato alla modalità del sistema. Alcune azioni possono essere consentite solo se system_mode è compatibile. Per esempio, un developer può modificare parametri in simulation_mode, ma non durante hardware_actuation_mode. Un operator può avviare demo se system_state è idle, ma non durante recovery. Un admin può modificare config solo se nessuna run è attiva. Questo evita modifiche pericolose nel momento sbagliato.

Condizioni di sistema possibili:

system_idle;

simulation_running;

hardware_demo_running;

diagnostic_mode;

recovery_mode;

safety_lock;

emergency_stop;

lockdown;

maintenance_mode.

La policy access_control deve controllare sia access_level sia system_state. Il permesso umano non basta: il sistema deve essere in stato adatto.

Un esempio importante è clear_safety. Anche se l'utente ha access_level alto, clear_safety deve essere permesso solo se emergency condition rimossa, output off, thermal non lock, energy non critical, recovery non pending e nodo pronto. Il controllo accessi può autorizzare la richiesta, ma il nodo decide se tecnicamente può uscire da safety.

La catena di clear_safety deve essere:

utente autorizzato;

sessione valida;

azione clear_safety consentita;

controller verifica stato globale;

nodo verifica stato locale;

se tutte le condizioni sono ok, passa a safe_idle;

mai passare direttamente a active.

Questa è una delle regole più importanti della sezione. La biometria non deve trasformarsi in pulsante magico per superare safety.

Il controllo degli accessi deve essere compatibile con MicroBot OS. I nodi possono avere comandi supportati e unsupported. Anche se l'utente ha permesso, il controller deve verificare se il nodo supporta il comando. Se un nodo v0.3 non supporta state_sync o hardware_test, il comando viene bloccato o rifiutato. Access control non può concedere capabilities che l'hardware non ha.

La logica corretta è:

access_level autorizza categoria azione;

controller verifica protocol_version e capabilities;

nodo verifica capabilities e safety;

azione eseguita solo se tutti i livelli accettano.

Questo è un controllo multilivello. Ogni livello protegge da un tipo diverso di errore.

Il controllo degli accessi deve supportare modalità read-only anche dopo autenticazione fallita. Se l'utente non viene riconosciuto, può essere comunque permesso vedere una demo pubblica. Questo migliora l'usabilità. L'autenticazione fallita non deve per forza chiudere l'intera pagina; deve solo bloccare funzioni sensibili. Naturalmente, dopo ripetuti fallimenti, si può limitare ulteriormente.

Un modello utile è:

auth_failed → access_level = guest/read_only;

auth_success viewer → viewer;

auth_success developer → developer;

auth_uncertain → viewer temporaneo o retry_required;

auth_unavailable → fallback o guest.

Questo rende la dashboard flessibile e sicura.

Il controllo degli accessi deve distinguere tra autorizzazione temporanea e permanente. Un profilo può essere permanentemente abilitato, ma la sessione è temporanea. L'utente non deve autenticarsi ogni secondo, ma nemmeno restare autorizzato per sempre. Il sistema deve quindi controllare session_status a ogni azione importante. La dashboard può mostrare timer o stato sessione.

Stati sessione:

active;

expired;

revoked;

locked;

pending_reauth;

read_only;

elevated_temporarily.

Il concetto di elevated_temporarily è utile: un utente developer può elevare temporaneamente a hardware_admin per una singola azione critica, con conferma e scadenza immediata. Questo è più sicuro che mantenere permessi elevati a lungo.

Il controllo degli accessi deve essere visibile nella UI. L'utente deve sapere in che livello si trova. Un badge può mostrare: Guest, Viewer, Operator, Developer, Admin, Hardware Admin. Accanto può esserci session active, expires in, auth method, security mode. I comandi disabilitati devono spiegare perché. Questo evita confusione e rende il sistema professionale.

Un pannello access status può mostrare:

current_access_level;

auth_method;

session_status;

session_expires_at;

allowed_actions_summary;

security_mode;

last_security_event;

reauth_required_for_critical_actions.

Questa UI aiuta anche durante demo: si vede che il sistema non è semplicemente aperto, ma controllato.

Il controllo degli accessi deve supportare logging senza esporre troppo. I security_events devono essere consultabili solo da developer/admin, non da guest. I log possono indicare tentativi falliti, accessi concessi, sessioni scadute, azioni critiche richieste. Non devono contenere immagini, template biometrici o dati sensibili non necessari. Questo mantiene equilibrio tra audit e privacy.

Un security log leggibile può dire:

timestamp;

event_type;

access_level;

action;

decision;

reason;

source;

safety_state.

Non deve dire:

frame immagine;

embedding grezzo;

dati biometrici completi;

informazioni personali non necessarie.

Questa distinzione deve essere parte della policy.

Il controllo degli accessi deve essere progettato per errore sicuro. Se il modulo biometrico non risponde, se il profilo non è leggibile, se la sessione è corrotta, se la policy non è caricata, il sistema deve negare funzioni sensibili e restare in read-only o safe mode. Non deve concedere accesso per default. Questo principio è fondamentale.

La regola è:

in caso di errore del sistema di accesso, negare controllo e mantenere osservazione limitata.

Questo è il comportamento fail-safe. È meglio bloccare temporaneamente una funzione che concedere controllo in modo incerto.

Il controllo degli accessi deve prevedere permessi per modalità simulata diversi dalla modalità hardware. Una simulazione pura ha rischio più basso. Un developer può essere autorizzato a fare fault injection simulata o modificare parametri. Ma un comando hardware reale richiede livelli più alti. Questo evita che lo sviluppo software venga rallentato inutilmente, mantenendo protezione sul fisico.

Esempio:

developer può modificare behavior_profile simulato;

developer non può attivare PWM fisico;

operator può avviare demo simulata;

operator non può fare clear_safety hardware;

hardware_admin può fare test hardware solo con safety ok.

Questo modello è molto adatto a MicroBot, dove simulazione e hardware convivono.

Il controllo degli accessi deve essere compatibile con la roadmap. Nelle prime versioni, può essere implementato in modo semplice: guest/viewer/developer mock. Poi si aggiunge camera auth. Poi access_level. Poi sessioni. Poi blocco controlli. Poi security events. Poi protezione hardware. Poi gestione profili. Non serve partire con un sistema enterprise completo. Serve però avere architettura chiara.

Una roadmap possibile:

v0.1: dashboard read-only e controlli separati;

v0.2: mock access levels;

v0.3: sessione locale semplice;

v0.4: blocco pulsanti per access_level;

v0.5: security events;

v0.6: integrazione con camera auth;

v0.7: operator/developer/admin;

v0.8: protezione diagnostic mode;

v0.9: protezione hardware commands;

v1.0: gestione profili locale;

v1.1: re-auth per azioni critiche;

v1.2: lockdown mode.

Questa progressione è realistica e coerente con il resto del progetto.

Il controllo degli accessi deve essere documentato con una matrice dei permessi. Una matrice è uno strumento chiaro per indicare chi può fare cosa. Può essere inserita nella documentazione MicroBot. Ogni riga è un'azione, ogni colonna un livello utente. Questo rende il sistema molto più professionale e riduce ambiguità.

Una matrice può includere azioni come:

view_dashboard;

view_telemetry;

view_logs;

start_simulation;

start_demo;

run_fault_injection;

enter_diagnostic_mode;

send_node_command;

modify_config;

manage_profiles;

clear_safety;

run_hardware_test;

modify_safety_limits.

Ogni azione può essere allowed, denied, requires_reauth, requires_safety_ok o simulation_only. Questa matrice è perfetta per la documentazione.

Il controllo degli accessi deve gestire anche l'accesso ai dati. Non tutti i dati sono uguali. Telemetria tecnica può essere visibile a viewer o developer. Security events solo developer/admin. Profili biometrici solo admin, e comunque senza esposizione di dati grezzi. Log hardware critici solo developer/hardware_admin/admin. Questa separazione è importante quanto il controllo dei comandi.

Categorie dati:

public_project_info;

dashboard_live_status;

technical_telemetry;

diagnostic_logs;

security_events;

configuration_data;

hardware_test_logs;

biometric_profile_metadata;

biometric_reference_data.

Ogni categoria deve avere access_level minimo. I dati biometrici grezzi o template devono essere massimamente protetti e idealmente non esposti dalla UI normale.

Il controllo degli accessi deve prevedere logout e manual lock. L'utente deve poter chiudere la sessione. La dashboard deve tornare a read-only. I controlli devono essere disabilitati. Eventuali comandi pendenti non critici possono essere annullati. Se c'è hardware attivo, logout non deve sostituire safe_stop, ma può richiedere stop prima di uscire o mantenere Safety Layer autonomo.

Una procedura logout può fare:

invalidare sessione;

disabilitare controlli;

fermare camera se attiva;

registrare session_closed;

mantenere dashboard read-only;

non modificare stato hardware salvo policy specifica.

Se l'utente vuole lasciare hardware in sicurezza, deve usare safe_idle o stop_demo prima. Logout non deve essere ambiguo.

Il controllo degli accessi deve proteggere anche la configurazione della biometria. Attivare o disattivare camera auth, cambiare soglia, aggiungere profilo, passare a mock mode: sono tutte azioni delicate. Devono richiedere admin o developer in development_mode. La soglia di riconoscimento non dovrebbe essere modificabile da operator. Una soglia troppo bassa aumenta rischio di falso accesso; troppo alta può bloccare utenti autorizzati. Questa impostazione è critica.

Azioni biometric_config:

enable_camera_auth;

disable_camera_auth;

set_threshold;

add_profile;

delete_profile;

enable_mock_auth;

disable_mock_auth;

set_session_duration;

set_failed_attempt_policy.

Queste azioni devono essere loggate. Se mock_auth viene attivato, la dashboard deve mostrarlo chiaramente. Non deve essere nascosto.

Il controllo degli accessi deve supportare la decisione tramite soglia che sarà approfondita nel punto 15.6. Qui basta definire che il valore di confronto biometrico non assegna direttamente permessi: produce auth_success, auth_failed o auth_uncertain. Il profilo associato decide access_level. La soglia può essere diversa per livelli diversi. Per esempio, accesso viewer può richiedere soglia più bassa, hardware_admin soglia più alta o conferma aggiuntiva. Questo è un concetto importante.

Una policy può essere:

score sopra soglia alta → accesso profilo completo;

score tra soglia minima e alta → accesso limitato o retry;

score sotto soglia → denied;

multiple faces → denied;

low image quality → retry;

camera unavailable → fallback o guest.

Questa logica rende il sistema più flessibile e sicuro.

Il controllo degli accessi deve essere testabile. Bisogna poter testare casi come auth_success, developer, auth_failed, session_expired, insufficient_access_level, safety_block, mock_auth, profile_disabled, hardware_action_denied. Questi test possono essere simulati senza usare camera reale. Questo è importante per sviluppare dashboard e controller.

Test utili:

guest_cannot_send_command;

viewer_cannot_start_demo;

operator_can_start_approved_demo;

developer_can_open_diagnostics;

developer_cannot_run_hardware_test;

admin_can_manage_profiles;

hardware_admin_requires_safety_ok;

session_expired_blocks_action;

mock_auth_marked_as_mock;

safety_lock_blocks_even_admin.

Questi test dimostrano che il controllo accessi funziona come policy, non solo come UI.

Il controllo degli accessi deve avere limiti dichiarati. In una versione prototipale, non deve essere presentato come sicurezza definitiva. Potrebbe non avere cifratura completa, storage sicuro o protezione avanzata contro attacchi. Può essere un modulo sperimentale locale. La documentazione deve dichiararlo. Questo è fondamentale per rigore.

Limitazioni iniziali possibili:

uso locale;

biometria sperimentale;

nessun salvataggio frame;

mock mode per sviluppo;

non adatto a sistemi critici reali;

Safety Layer indipendente dai permessi;

hardware commands comunque limitati.

Questa onestà rende il progetto più credibile. Un prototipo può essere valido senza essere sistema di sicurezza commerciale.

In conclusione, il controllo degli accessi è il livello che trasforma l'autenticazione biometrica in permessi operativi concreti. Esso distingue guest, viewer, operator, developer, admin e hardware_admin; applica permessi alla dashboard, al controller, ai log, alla diagnostica, alle configurazioni e ai comandi hardware; gestisce sessioni, scadenze, re-auth, lockdown e security events; separa autorizzazione umana da validazione tecnica; e mantiene il Safety Layer come barriera indipendente e superiore. In MicroBot, un utente riconosciuto può ottenere accesso a funzioni coerenti con il proprio livello, ma nessun livello umano deve poter forzare comandi non sicuri, ignorare thermal_lock, superare critical_energy, disabilitare watchdog o bypassare safe_off. Il controllo degli accessi rende il sistema più maturo perché protegge non solo l'ingresso alla dashboard, ma l'intera catena di interazione tra persona, software, controller e nodi MicroBot. Il punto successivo, dedicato al riconoscimento facciale, entrerà nel dettaglio del processo tecnico con cui il volto acquisito dalla fotocamera viene rilevato, rappresentato e usato per identificare o verificare un profilo autorizzato.

15.3 Riconoscimento facciale

Il riconoscimento facciale rappresenta il terzo livello della sicurezza biometrica MicroBot e descrive il processo attraverso cui il volto acquisito dalla fotocamera viene analizzato per verificare se corrisponde a un profilo autorizzato. Dopo aver definito l'autenticazione tramite fotocamera e il controllo degli accessi, è necessario entrare nel cuore tecnico della pipeline biometrica: il sistema deve rilevare un volto, trasformarlo in una rappresentazione confrontabile, calcolare una misura di somiglianza e produrre un risultato utilizzabile dal modulo di access control. Il riconoscimento facciale non deve essere inteso come una decisione magica o assoluta, ma come una procedura probabilistica basata su dati visivi, qualità dell'immagine, modello di confronto, soglie e gestione dell'incertezza.

Nel progetto MicroBot, il riconoscimento facciale ha una funzione precisa: permettere alla dashboard o al sistema di controllo di capire se chi sta tentando di accedere è un utente autorizzato. Questa funzione può essere usata per sbloccare modalità operative, abilitare pannelli diagnostici, concedere accesso a comandi software, autorizzare l'avvio di una demo o proteggere sezioni sensibili. Tuttavia, il riconoscimento facciale non deve mai sostituire il Safety Layer. Anche se l'utente viene riconosciuto correttamente, ogni comando verso nodi fisici deve passare dal controller, dal controllo delle capabilities, da MicroBot OS e dalle verifiche locali di sicurezza. Il riconoscimento facciale autorizza la persona; non rende automaticamente sicura l'azione.

Il principio generale del riconoscimento facciale in MicroBot può essere espresso così:

il volto dell'utente viene trasformato in una rappresentazione numerica confrontabile, e il sistema concede accesso solo se il livello di somiglianza supera una soglia definita e se il contesto operativo è sicuro.

Questa frase contiene i punti principali. Il volto non viene trattato direttamente come immagine da confrontare “a occhio”. Viene prima rilevato, normalizzato e convertito in una rappresentazione. Questa rappresentazione viene confrontata con un riferimento precedentemente registrato. Il risultato non è semplicemente vero o falso: è uno score, una misura di somiglianza o distanza. La decisione finale dipende da una soglia. Se lo score è abbastanza buono, l'autenticazione può riuscire. Se è basso, fallisce. Se è vicino alla soglia o la qualità è scarsa, può essere dichiarata incerta.

Il primo passaggio del riconoscimento facciale è la **face detection**, cioè il rilevamento del volto nel frame. Questa fase risponde alla domanda: “esiste un volto visibile nell'immagine?”. Non identifica ancora la persona. Individua soltanto la regione del frame che contiene un volto. Il sistema può restituire un bounding box, cioè una finestra intorno al volto, e una confidenza di rilevamento. Se non viene rilevato nessun volto, la pipeline non può procedere. Se vengono rilevati più volti, il sistema deve considerare la situazione ambigua e negare l'accesso o chiedere di isolare una sola persona nel frame.

La face detection deve produrre stati chiari:

face_not_detected;

single_face_detected;

multiple_faces_detected;

face_detection_low_confidence;

face_partially_visible;

face_out_of_frame.

Questi stati devono essere gestiti prima del confronto biometrico. Se il volto non è presente, non ha senso calcolare somiglianza. Se ci sono più volti, non è chiaro quale identità stia autorizzando l'azione. Se il volto è parziale o la confidenza è bassa, il sistema deve chiedere un nuovo frame o dichiarare auth_uncertain. Questa prudenza evita decisioni sbagliate.

Il secondo passaggio è il **pre-processing del volto**. Una volta rilevato il volto, il sistema deve prepararlo per il riconoscimento. Il volto può essere ritagliato, ridimensionato, centrato, normalizzato in luminosità e orientamento. Se il volto è inclinato, troppo piccolo, troppo scuro, troppo sfocato o coperto, il riconoscimento diventa meno affidabile. Il pre-processing serve a ridurre variabilità non legata all'identità: distanza dalla camera, luce, dimensione dell'immagine, posizione nel frame e angolo.

Il pre-processing può includere:

ritaglio della regione del volto;

ridimensionamento a formato standard;

controllo della luminosità;

controllo della nitidezza;

normalizzazione dell'orientamento;

verifica della dimensione minima;

controllo della posa del volto;

scarto di frame non adatti.

Questa fase deve essere vista come filtro di qualità. Un sistema robusto non deve tentare riconoscimento su frame pessimi. Deve prima dichiarare `low_quality_frame` o `retry_required`. È meglio chiedere un nuovo tentativo che concedere o negare accesso sulla base di dati deboli.

Il terzo passaggio è la **rappresentazione del volto**. Il riconoscimento facciale moderno non confronta normalmente pixel grezzi, ma rappresentazioni numeriche. Una rappresentazione può essere chiamata embedding, template o feature vector. Essa è una sintesi matematica del volto, progettata per essere simile tra immagini diverse della stessa persona e diversa tra persone diverse. Nel progetto MicroBot, non è necessario entrare subito nel dettaglio di un modello specifico; è sufficiente definire il concetto architetturale: il volto acquisito viene convertito in un vettore confrontabile.

Una rappresentazione biometrica può essere descritta come:

`face_image` → `face_embedding`

Il `face_embedding` non deve essere interpretato come immagine. È una struttura numerica derivata dal volto. Tuttavia, resta dato biometrico sensibile, perché può essere usato per identificare una persona. Per questo motivo, anche gli embedding devono essere trattati con cautela: non devono essere salvati nei log generici, non devono essere caricati su repository pubbliche e devono essere minimizzati o protetti se conservati.

Il quarto passaggio è il **confronto con un riferimento autorizzato**. Per riconoscere un utente, il sistema deve avere un profilo di riferimento. Questo riferimento può essere un template biometrico registrato in precedenza, un embedding autorizzato o un modello associato a un profilo locale. Quando l'utente si presenta alla camera, il sistema genera un embedding corrente e lo confronta con il riferimento. Il confronto produce uno score. A seconda dell'algoritmo, uno score più alto può significare maggiore somiglianza oppure una distanza più bassa può significare maggiore somiglianza. La documentazione deve chiarire quale convenzione viene usata.

Una pipeline di confronto può essere:

`embedding_corrente`;

`embedding_riferimento`;

funzione di confronto;

score o distanza;

soglia;

decisione.

Per esempio, se si usa una misura di similarità, il sistema può decidere `auth_success` quando `similarity_score ≥ threshold`. Se si usa una misura di distanza, il sistema può decidere `auth_success` quando `distance_score ≤ threshold`. La logica deve essere dichiarata chiaramente perché invertire il criterio sarebbe un errore grave.

Il quinto passaggio è la **gestione del profilo riconosciuto**. Se il confronto ha successo, il sistema non deve semplicemente dire “accesso concesso” in modo generico. Deve associare il risultato a un profilo: `profile_id`, `access_level`, `enabled`, `auth_method`, `confidence`. Il profilo definisce cosa l’utente può fare. Il riconoscimento facciale stabilisce la corrispondenza; il controllo accessi decide i permessi. Se il profilo è disabilitato, anche un match corretto non deve concedere accesso. Se il profilo esiste ma ha livello viewer, non deve sbloccare funzioni developer.

Una struttura di risultato può contenere:

```
auth_result;  
  
profile_id;  
  
access_level;  
  
match_score;  
  
threshold_used;  
  
confidence_level;  
  
profile_enabled;  
  
decision_reason.
```

Questo risultato viene passato al modulo `access_control`. La dashboard può mostrare `access_level`, ma non dovrebbe mostrare dati biometrici interni dettagliati se non in modalità diagnostica e senza esporre informazioni sensibili.

Il sesto passaggio è la **gestione dell’incertezza**. Il riconoscimento facciale non è perfetto. Può sbagliare per luce scarsa, occlusioni, angoli strani, qualità bassa, cambiamenti nel volto, rumore della camera o modelli non calibrati. Per questo il sistema deve prevedere stati intermedi. Se lo score è molto sopra soglia, può essere `auth_success_high_confidence`. Se è leggermente sopra soglia, si può concedere accesso limitato o richiedere conferma aggiuntiva. Se è vicino alla soglia, `auth_uncertain`. Se è sotto soglia, `auth_failed`. Questa gestione è più matura di un semplice sì/no.

Stati decisionali possibili:

```
auth_success_high_confidence;  
  
auth_success;  
  
auth_success_low_confidence;  
  
auth_uncertain;  
  
auth_failed;  
  
retry_required;  
  
fallback_auth_required.
```


Per funzioni sensibili come `hardware_test`, `clear_safety` o gestione profili, dovrebbe essere richiesta alta confidenza o un secondo fattore. Per funzioni `read-only`, la biometria può non essere richiesta o può bastare un livello inferiore. Questo rende il sistema proporzionato al rischio.

Il settimo passaggio è il **controllo anti-ambiguità**. Se il sistema rileva più volti, se il volto è troppo piccolo, se la qualità è insufficiente, se il frame cambia troppo rapidamente o se lo score oscilla, la decisione deve essere bloccata. L'accesso non deve essere concesso in condizioni ambigue. In una dashboard MicroBot, un tentativo di accesso con più persone davanti alla camera dovrebbe generare `multiple_faces_detected` e non concedere permessi elevati. Questo è importante perché l'autorizzazione deve riferirsi a una persona specifica.

Condizioni che dovrebbero bloccare o rendere incerta l'autenticazione:

`multiple_faces_detected`;

`face_quality_low`;

`landmark_confidence_low`;

`face_pose_out_of_range`;

`score_near_threshold`;

`profile_disabled`;

`camera_stream_unstable`;

`no_reference_profile`;

`biometric_module_error`.

Questi stati devono produrre messaggi chiari nella dashboard e `security_event` nei log. Il sistema deve spiegare il motivo della mancata autenticazione, senza esporre dati sensibili.

L'ottavo passaggio è la **protezione del riferimento biometrico**. Il riconoscimento facciale richiede un riferimento con cui confrontare il volto corrente. Questo riferimento è delicato. Non deve essere trattato come un file qualsiasi. In una versione prototipale, si può evitare del tutto di salvare riferimenti reali e usare `mock_auth`. In una versione più avanzata, si può salvare un template locale, protetto, non incluso nella repo, non esposto in frontend pubblico e cancellabile. Il riferimento biometrico non deve essere condiviso con la documentazione pubblica.

Una policy corretta è:

nessun volto reale nei file sample;

nessun template biometrico nel repository;

nessun salvataggio automatico di frame;

riferimenti biometrici separati dai log tecnici;

possibilità di cancellare profilo;

accesso ai profili solo admin;

uso locale nelle prime versioni.

Questa policy è necessaria perché i dati biometrici sono più sensibili di normali dati tecnici. Anche un embedding può essere considerato dato biometrico. Va trattato con cura.

Il nono passaggio è la **distinzione tra identificazione e verifica**. Nel riconoscimento facciale esistono due modalità concettuali. La verifica risponde alla domanda: “questa persona corrisponde al profilo dichiarato?”. L’identificazione risponde alla domanda: “chi è questa persona tra più profili?”. Per MicroBot, nelle prime versioni è più prudente usare verifica, perché è più semplice e controllabile. Per esempio, la dashboard può avere un profilo developer locale e verificare se il volto corrente corrisponde a quel profilo. L’identificazione su molti profili è più complessa e richiede policy più attente.

La distinzione è:

verifica 1:1 = confronto con un profilo specifico;

identificazione 1:N = confronto con molti profili per trovare il più simile.

Per un prototipo MicroBot, verifica 1:1 o pochi profili locali sono sufficienti. L’identificazione multiutente può essere una roadmap futura. Questo riduce complessità, rischi e falsi positivi.

Il decimo passaggio è la **valutazione del falso positivo e falso negativo**. Un falso positivo avviene quando il sistema autorizza una persona non autorizzata. Un falso negativo avviene quando rifiuta una persona autorizzata. Nel contesto MicroBot, il falso positivo è più pericoloso per funzioni critiche, mentre il falso negativo è fastidioso ma meno rischioso. Per questo, le soglie per permessi elevati devono essere conservative. Meglio richiedere un nuovo tentativo che autorizzare un accesso hardware sbagliato.

La policy può essere:

per viewer: tolleranza più flessibile;

per operator: soglia moderata;

per developer: soglia più alta;

per admin: soglia alta;

per hardware_admin: soglia alta più conferma o secondo fattore.

Questa differenziazione collega riconoscimento facciale e access control. Non tutti i permessi devono avere lo stesso livello di rischio accettabile.

L’undicesimo passaggio è la **gestione della qualità nel tempo**. Un singolo frame potrebbe essere rumoroso. Il sistema può analizzare più frame e aggregare la decisione. Per esempio, può richiedere che il volto sia rilevato per alcuni frame consecutivi, che lo score sia stabile o che la qualità rimanga sufficiente. Questo riduce decisioni impulsive. Tuttavia, non bisogna rendere la procedura troppo lenta. Serve un equilibrio tra velocità e affidabilità.

Una decisione multi-frame può usare:

numero minimo di frame validi;

media degli score;

score massimo solo se qualità alta;

stabilità del volto;

assenza di più volti;

coerenza dei landmark.

Questa logica è particolarmente utile se la camera produce frame instabili. La dashboard può mostrare “scanning”, “face stable”, “verifying”, “authenticated” o “retry”. Questo rende l’esperienza più chiara.

Il dodicesimo passaggio è la **integrazione con landmark detection**. Il punto successivo della sezione sarà dedicato specificamente ai landmark, ma è utile anticipare il collegamento. La landmark detection individua punti caratteristici del volto, come occhi, naso, bocca e contorno. Questi punti possono servire per allineare il volto, controllare qualità, stimare posa e migliorare il riconoscimento. Senza landmark affidabili, il confronto biometrico può essere meno stabile. Per questo il riconoscimento facciale deve usare lo stato landmarks_detected o landmarks_failed come input.

Se landmark_status = failed, il sistema può:

richiedere nuovo frame;

abbassare confidence;

non procedere al confronto;

dichiarare auth_uncertain;

mostrare messaggio di qualità insufficiente.

Questo rende la pipeline più robusta. Il volto non viene confrontato se non è stato interpretato in modo affidabile.

Il tredicesimo passaggio è la **integrazione con la dashboard MicroBot**. Il riconoscimento facciale deve essere visibile ma non invadente. La dashboard può avere un pannello Security/Auth con camera status, face detection status, auth status, access level e session status. Non deve mostrare dati biometrici sensibili. Può mostrare un bordo intorno al volto, un indicatore di qualità e un messaggio. Dopo autenticazione, può spegnere la camera e mostrare il badge del livello di accesso.

Un pannello Auth può mostrare:

camera status;

face detection status;

quality status;

authentication status;

access level;

session timer;

last security event;

logout button.

Questa UI rende il sistema professionale. L'utente capisce se è in guest, viewer, developer o admin. I controlli si abilitano in base al livello. Se la sessione scade, il badge cambia e i controlli vengono disabilitati.

Il quattordicesimo passaggio è la **integrazione con security_events**. Ogni tentativo di riconoscimento deve generare eventi, ma senza salvare dati biometrici grezzi. Per esempio: auth_attempt_started, face_detected, auth_success, auth_failed, auth_uncertain, access_granted, access_denied. Gli eventi devono contenere metadati utili: timestamp, decision, reason, access_level, auth_method, confidence class. Non devono contenere immagini o embedding.

Un security_event può contenere:

event_id;

event_type;

timestamp;

auth_attempt_id;

decision;

reason;

access_level_granted;

confidence_class;

camera_status;

face_detection_status.

Questo permette audit e debug senza esporre dati sensibili. Se un tentativo fallisce perché no_face_detected, si vede. Se fallisce perché score_below_threshold, si vede. Se fallisce perché profile_disabled, si vede.

Il quindicesimo passaggio è la **gestione del fallimento del modulo biometrico**. Se il modello non carica, se la libreria non funziona, se la camera produce errori, se il confronto non è disponibile o se il riferimento biometrico manca, il sistema deve fallire in modo sicuro. Non deve concedere accesso elevato. Deve passare a read-only o richiedere fallback manuale controllato. Questo è un principio fail-safe.

Stati di errore del modulo:

biometric_model_unavailable;

reference_profile_missing;

comparison_failed;

camera_error;

landmark_model_failed;

auth_service_unavailable;

biometric_store_error.

In questi casi, la dashboard può dire “biometric authentication unavailable” e offrire modalità read-only o fallback developer solo se previsto. Le funzioni hardware restano bloccate.

Il sedicesimo passaggio è la **compatibilità con modalità mock**. Durante sviluppo, il riconoscimento facciale può essere sostituito da mock_auth_success, mock_auth_failed o manual_unlock. Questo permette di costruire e testare access control senza usare volti reali. Tuttavia, il sistema deve indicare chiaramente quando è in mock mode. Una demo con mock mode non deve essere presentata come riconoscimento reale. La dashboard può mostrare un badge “AUTH MOCK MODE”.

Modalità possibili:

real_face_recognition;

mock_success;

mock_failure;

manual_developer_unlock;

auth_disabled_read_only.

Queste modalità sono utili per sviluppo, ma devono essere loggate e visibili. La trasparenza è fondamentale.

Il diciassettesimo passaggio è la **privacy del riconoscimento facciale**. Questo modulo tratta dati biometrici, quindi deve essere progettato con una logica di minimizzazione. L’elaborazione dovrebbe essere locale quando possibile. I frame non dovrebbero essere salvati. I template non dovrebbero essere inclusi nel repository. Gli eventi dovrebbero essere separati. L’utente dovrebbe sapere quando la camera è attiva. La cancellazione dei profili dovrebbe essere possibile. Questi principi rendono il progetto molto più serio.

Una policy privacy-oriented può dichiarare:

camera off di default;

camera attiva solo durante auth attempt;

frame non salvati;

embedding non loggati;

template locale non pubblico;

eventi di accesso separati;

logout e stop_camera disponibili;

mock mode per sviluppo senza dati biometrici.

Questa policy deve essere inserita nella documentazione del modulo biometrico.

Il diciottesimo passaggio è la **protezione contro presentazione impropria del volto**. Un sistema di riconoscimento facciale può essere vulnerabile a foto, video o replay se non integra controlli specifici di liveness. Nelle prime versioni MicroBot, non bisogna dichiarare protezione avanzata anti-spoofing se non è implementata. Si può invece indicare che il modulo biometrico è sperimentale e non adatto a contesti critici. In futuro si possono aggiungere controlli di presenza reale, movimento, blink detection, profondità o challenge-response. Ma finché non ci sono, vanno dichiarati come roadmap.

Limitazione importante:

il riconoscimento facciale prototipale non deve essere considerato sicurezza anti-spoofing completa.

Questa frase è necessaria per rigore. Il modulo può essere utile per demo e access control locale, ma non deve essere venduto come sicurezza definitiva. Eventuali controlli di liveness saranno futuri.

Il diciannovesimo passaggio è la **relazione con i comandi MicroBot**. Il riconoscimento facciale non deve inviare direttamente comandi ai nodi. Deve solo stabilire auth_result. Il modulo access_control decide se l'interfaccia può generare un comando. Il controller crea il comando. MicroBot OS lo valida. Il Safety Layer lo autorizza o rifiuta. Questa catena deve restare sempre valida.

La catena è:

face recognition → auth_result;

access_control → allowed_actions;

dashboard → request action;

controller → command generation;

node OS → command validation;

Safety Layer → final safety decision;

telemetry → event/result.

Questo impedisce che un bug nel modulo biometrico attivi hardware direttamente. Ogni livello ha responsabilità limitata.

Il ventesimo passaggio è la **gestione dell'accesso temporaneo**. Dopo riconoscimento, il sistema può concedere una sessione. La sessione deve avere durata. Per azioni critiche, può essere richiesta riautenticazione. Per esempio, un utente riconosciuto come developer può accedere alla diagnostica, ma per hardware_test deve effettuare re-auth o conferma. Questo riduce il rischio che una sessione aperta venga usata impropriamente.

Una policy può essere:

auth_success viewer → sessione lunga read-only;

auth_success developer → sessione media;

hardware_admin action → re-auth breve;

config critical change → conferma admin;

session inactivity → scadenza.

Il riconoscimento facciale diventa quindi ingresso a una sessione, non permesso eterno.

Il ventunesimo passaggio è la **diagnostica del riconoscimento facciale**. Come tutti i moduli MicroBot, anche il riconoscimento facciale deve essere diagnosticabile. Deve poter dire se camera è pronta, se modello è caricato, se riferimento esiste, se face detection funziona, se landmark detection funziona, se comparison è disponibile, se access_control riceve risultati. La dashboard developer può mostrare questi stati senza mostrare dati sensibili.

Un biometric_diagnostics può contenere:

camera_ready;

face_detector_ready;

landmark_detector_ready;

recognition_model_ready;

reference_profiles_count;

auth_attempt_count;

last_auth_result;

last_failure_reason;

mock_mode_active;

security_policy_loaded.

Questi dati aiutano a capire perché l'autenticazione non funziona. Se recognition_model_ready = false, il problema è il modello. Se reference_profiles_count = 0, manca profilo. Se camera_ready = false, è camera. Se last_failure_reason = score_below_threshold, il modello ha confrontato ma non ha trovato match sufficiente.

Il ventiduesimo passaggio è la **valutazione del riconoscimento nel contesto di progetto**. MicroBot non ha bisogno, nelle prime versioni, di un sistema biometrico industriale. Ha bisogno di un modulo coerente con la sua architettura: camera, face detection, landmark, confronto, soglia, access_control, sessione, eventi, dashboard. Questa pipeline dimostra capacità di integrazione tra AI/vision, interfaccia web, sicurezza e sistema distribuito. È un elemento che arricchisce l'ecosistema, purché documentato con limiti chiari.

Il riconoscimento facciale può dimostrare:

interazione uomo-sistema;

access control intelligente;
protezione delle modalità sensibili;
integrazione camera-dashboard;
pipeline di computer vision;
decisioni basate su soglia;
logging di eventi security;
separazione tra autenticazione e safety.

Questi sono obiettivi coerenti e realistici. Non serve promettere sicurezza perfetta. Serve mostrare architettura robusta.

Il ventitreesimo passaggio è la **modalità locale**. Per le prime versioni, il riconoscimento dovrebbe essere pensato come modulo locale, eseguito nel browser o sul computer che ospita la dashboard, non come servizio remoto. Questo riduce complessità e rischio. Se in futuro si volesse un servizio remoto, servirebbero protezioni aggiuntive, trasporto sicuro e policy più severe. Per ora, locale è la scelta più coerente.

Una modalità locale offre:

minore esposizione dei dati;
più controllo sullo stream camera;
sviluppo più semplice;
integrazione con dashboard locale;
nessuna necessità di inviare immagini a server esterni.

Questa scelta si adatta bene ai prototipi MicroBot, che spesso girano su macchina locale con simulatori e dashboard.

Il ventiquattresimo passaggio è la **gestione del profilo singolo iniziale**. Per semplicità, il primo riconoscimento facciale può supportare un solo profilo developer. Questo evita gestione multiutente complessa. Una volta validata la pipeline, si possono aggiungere più profili e access_level diversi. Partire con un profilo singolo è una scelta sensata e riduce rischio.

Una versione iniziale può avere:

profile_id = local_developer;
access_level = developer;
camera auth locale;
soglia configurata;
no profili multipli;

no identificazione 1:N;

solo verifica 1:1.

Questa versione è sufficiente per dimostrare biometria e access control. Le versioni future possono espandere.

Il venticinquesimo passaggio è la **interazione con access levels**. Il riconoscimento non deve solo dire “sì”. Deve dire quale profilo è stato verificato e quale access_level è associato. Se il profilo è developer, la dashboard abilita diagnostica. Se è operator, abilita demo. Se è admin, abilita configurazioni. Questo collegamento deve essere chiaro.

Una decisione completa può essere:

auth_success;

profile_id = developer_local;

access_level = developer;

allowed_actions = diagnostics, simulation_control, replay, logs;

restricted_actions = hardware_test, biometric_profile_management, safety_limits;

session_duration = configurata.

Questo output è molto più utile di un booleano. Permette al controllo accessi di lavorare correttamente.

Il ventiseiesimo passaggio è la **gestione dei dati di addestramento o enroll**. Per riconoscere un volto, serve registrare un riferimento. La fase di enroll deve essere protetta. Non si deve permettere a chiunque di registrare un nuovo volto. L'enroll deve richiedere admin o development_mode esplicita. Durante enroll, il sistema può acquisire più frame, controllare qualità, creare template e salvarlo localmente. Deve essere possibile cancellarlo.

Una pipeline enroll può essere:

admin/developer autorizzato;

camera attiva;

acquisizione più frame validi;

controllo qualità;

creazione reference template;

salvataggio locale protetto;

associazione a profile_id;

test di verifica;

logging evento enroll_completed.

Questa funzione può essere futura. Nelle prime versioni, si può usare mock o profilo manuale. Ma la documentazione deve definire come potrebbe funzionare.

Il ventisettesimo passaggio è la **gestione della soglia**. Anche se la decisione tramite soglia sarà approfondita nel punto 15.6, il riconoscimento facciale deve produrre uno score che può essere confrontato. La soglia non deve essere scelta casualmente. Deve essere testata su esempi, qualità diverse e condizioni realistiche. Una soglia troppo permissiva aumenta falsi positivi. Una troppo severa aumenta falsi negativi. Per MicroBot, per funzioni critiche è meglio essere più severi.

La soglia può dipendere da:

tipo di score;

modello usato;

qualità immagine;

access_level richiesto;

numero di frame validi;

modalità demo o hardware;

confidence richiesta.

Questo significa che non esiste necessariamente una soglia unica per tutto. Si può avere threshold_viewer, threshold_developer, threshold_admin, threshold_hardware_admin. Oppure una soglia unica iniziale e policy più severe per azioni critiche.

Il ventottesimo passaggio è la **roadmap del riconoscimento facciale**. Una progressione realistica è partire da camera e face detection, poi aggiungere landmark, poi embedding, poi confronto, poi soglia, poi sessione, poi multi-profilo e diagnostica. Non serve implementare tutto insieme. La roadmap permette di integrare il modulo senza destabilizzare il sistema.

Una roadmap possibile è:

v0.1: pannello camera e mock auth;

v0.2: face detection base;

v0.3: controllo qualità frame;

v0.4: landmark detection;

v0.5: normalizzazione volto;

v0.6: generazione embedding/template;

v0.7: confronto 1:1 con profilo locale;

v0.8: decisione tramite soglia;

v0.9: access_level e sessione;

v1.0: security events;

v1.1: enroll protetto;

v1.2: multi-profilo locale;

v1.3: diagnostica biometrica;

v1.4: controlli anti-ambiguità avanzati.

Questa roadmap è coerente con tutto il progetto MicroBot: prima struttura, poi funzione, poi validazione, poi integrazione.

Il ventinovesimo passaggio è la **documentazione del modulo**. Il riconoscimento facciale deve avere una documentazione specifica che spiega pipeline, dati trattati, limiti, modalità, errori, sicurezza, soglie, profili e integrazione con access_control. Questo documento deve dichiarare chiaramente se il modulo è prototipale, locale, mock, reale o sperimentale. La documentazione è fondamentale perché la biometria è un tema sensibile.

Un documento facial_recognition_module.md può includere:

scopo;

pipeline;

camera input;

face detection;

landmark detection;

embedding;

comparison;

threshold;

auth_result;

access_control integration;

privacy policy;

limitations;

roadmap.

Questa documentazione protegge il progetto da interpretazioni sbagliate e rende il modulo più professionale.

Il trentesimo passaggio è la **collocazione del riconoscimento facciale nella visione MicroBot**. Questo modulo non è il centro meccanico o fisico dello swarm, ma è un'estensione importante dell'ecosistema. Mostra che MicroBot può integrare interazione uomo-macchina, computer vision, sicurezza, dashboard, access control e sistemi distribuiti. In una visione futura, un sistema MicroBot potrebbe essere controllato solo da operatori riconosciuti, con permessi graduati, telemetria tracciata e safety indipendente. Questa integrazione rende MicroBot più simile a una piattaforma completa.

In conclusione, il riconoscimento facciale è il processo attraverso cui il volto acquisito dalla fotocamera viene rilevato, pre-processato, trasformato in una rappresentazione biometrica, confrontato con un riferimento autorizzato e valutato tramite score e soglia. Nel sistema MicroBot, esso non deve essere trattato come decisione assoluta, ma come componente di una pipeline più ampia che include qualità dell'immagine, landmark detection, gestione dell'incertezza, access_control, sessione, security_events e Safety Layer. Il riconoscimento facciale può concedere un access_level, ma non può bypassare le protezioni tecniche dei nodi. Deve essere locale, minimizzato, diagnosticabile, documentato e dichiarato sperimentale nelle prime versioni. Il punto successivo, dedicato alla landmark detection, approfondirà il ruolo dei punti caratteristici del volto nella normalizzazione, nel controllo qualità, nella stima della posa e nel miglioramento dell'affidabilità del riconoscimento biometrico.

15.4 Landmark detection

La **landmark detection** rappresenta il quarto livello della sicurezza biometrica MicroBot e descrive il processo attraverso cui il sistema individua punti caratteristici del volto all'interno dell'immagine acquisita dalla fotocamera. Dopo aver definito il riconoscimento facciale, cioè la trasformazione del volto in una rappresentazione confrontabile con un profilo autorizzato, è necessario analizzare un passaggio intermedio molto importante: prima di confrontare un volto, il sistema deve capire se quel volto è stato rilevato correttamente, se è sufficientemente visibile, se è orientato in modo accettabile e se i suoi punti principali sono localizzabili. La landmark detection serve proprio a questo: individuare riferimenti geometrici del volto che permettono di normalizzare, controllare e rendere più affidabile il riconoscimento.

Nel contesto MicroBot, i landmark facciali possono essere interpretati come punti di riferimento del volto. Essi possono corrispondere, per esempio, alla posizione degli occhi, del naso, della bocca, del mento, delle sopracciglia, degli zigomi o del contorno facciale. Il numero esatto di landmark dipende dal modello utilizzato: alcuni sistemi usano pochi punti essenziali, altri reti più dense con decine o centinaia di riferimenti. Per la documentazione MicroBot non è necessario vincolarsi subito a un modello specifico; ciò che conta è descrivere la funzione architetturale del modulo. I landmark non sono il risultato finale dell'autenticazione, ma sono informazioni intermedie usate per migliorare la qualità della pipeline biometrica.

La landmark detection risponde a una domanda precisa:

il volto rilevato contiene punti geometrici sufficientemente affidabili per procedere al confronto biometrico?

Questa domanda è molto importante perché un volto può essere presente nel frame, ma non necessariamente adatto al riconoscimento. Se una persona è troppo inclinata, se il volto è parzialmente coperto, se un occhio non è visibile, se la bocca è fuori frame, se la camera è troppo vicina o troppo lontana, il face detector può comunque trovare una regione facciale, ma il riconoscimento potrebbe essere instabile. I landmark permettono di valutare questa situazione. Se i punti chiave non vengono rilevati con buona confidenza, il sistema può fermarsi prima del confronto e chiedere un nuovo frame.

Il primo ruolo della landmark detection è il **controllo della qualità del volto**. Il sistema può usare i landmark per capire se il volto è completo, centrato e sufficientemente visibile. Per esempio, se entrambi gli occhi sono rilevati, se il naso è al centro, se la bocca è visibile e se il contorno del volto è coerente, allora il frame può essere considerato adatto. Se invece mancano punti fondamentali o la loro confidenza è bassa, il sistema può dichiarare landmarks_failed o low_landmark_confidence. Questo evita di procedere con un riconoscimento debole.

Uno stato di qualità basato su landmark può includere:

landmarks_detected;

landmarks_partial;

landmarks_failed;

landmark_confidence_low;

face_pose_unstable;

face_not_centered;

face_too_small;

face_partially_occluded.

Questi stati devono essere trattati come parte della pipeline di autenticazione. Se landmarks_failed, il sistema non dovrebbe concedere accesso. Se landmarks_partial, può richiedere un nuovo frame. Se landmark_confidence_low, può abbassare il confidence level finale o generare auth_uncertain. La landmark detection diventa quindi un filtro prima della decisione.

Il secondo ruolo è la **normalizzazione del volto**. Quando una persona si presenta davanti alla camera, il volto può essere leggermente ruotato, spostato o inclinato. Il sistema può usare i landmark per allineare il volto in modo più stabile prima di generare l'embedding. Per esempio, la posizione degli occhi può essere usata per correggere l'inclinazione; il naso può essere usato come riferimento centrale; la distanza tra gli occhi può aiutare a normalizzare la scala. Questo rende più confrontabili immagini diverse della stessa persona.

Una pipeline di normalizzazione può seguire questo schema:

rilevamento volto;

estrazione landmark;

stima posizione degli occhi;

stima asse del volto;

correzione rotazione;

ritaglio centrato;

ridimensionamento;

generazione embedding.

Questa normalizzazione è importante perché riduce variazioni non legate all'identità. Se una persona viene ripresa leggermente più a sinistra o più inclinata, il sistema non dovrebbe interpretarla come persona diversa. I landmark aiutano a trasformare il frame grezzo in un input più regolare per il riconoscimento.

Il terzo ruolo è la **stima della posa del volto**. La posa indica l'orientamento del volto rispetto alla camera: frontale, ruotato a destra, ruotato a sinistra, inclinato verso l'alto, verso il basso o ruotato

lateralmente. Un riconoscimento facciale è generalmente più affidabile quando il volto è abbastanza frontale. Se la posa è troppo estrema, alcuni landmark diventano meno visibili e l'embedding può essere meno stabile. Il sistema può quindi usare i landmark per stimare se la posa è accettabile.

Stati di posa possibili:

pose_frontal;

pose_slightly_rotated;

pose_too_rotated;

pose_tilted;

pose_out_of_range;

pose_uncertain.

Nel sistema MicroBot, pose_too_rotated o pose_out_of_range dovrebbero portare a retry_required o auth_uncertain. Non significa che l'utente non sia autorizzato; significa che il frame non è adatto. Questa distinzione è importante per evitare falsi rifiuti. La dashboard può mostrare un messaggio semplice: "posizionare il volto più frontalmente".

Il quarto ruolo è la **verifica della presenza di un volto reale e coerente**, almeno a livello geometrico base. La landmark detection non è automaticamente un sistema anti-spoofing, ma può aiutare a rilevare alcune incoerenze. Per esempio, se i landmark sono deformati, instabili o non coerenti con un volto umano, il sistema può rifiutare il frame. Tuttavia, è importante non esagerare: senza un modulo dedicato di liveness detection, la landmark detection non garantisce che il volto appartenga a una persona fisicamente presente e non a una foto o video. Deve essere descritta come controllo geometrico e qualitativo, non come protezione completa anti-spoofing.

Una formulazione corretta è:

la landmark detection migliora il controllo geometrico e qualitativo del volto, ma non sostituisce un sistema avanzato di liveness detection.

Questa frase mantiene rigore. Il progetto può prevedere in futuro controlli di liveness, ma finché non sono implementati non devono essere dichiarati come presenti.

Il quinto ruolo è la **gestione dell'occlusione**. Un volto può essere parzialmente coperto da mano, cappello, occhiali, mascherina, capelli, ombre o oggetti. I landmark aiutano a capire se parti fondamentali sono visibili. Se mancano punti intorno agli occhi o al naso, il sistema può considerare il frame troppo incerto. In una pipeline biometrica prudente, l'occlusione non deve essere ignorata. Deve ridurre la confidenza o bloccare l'autenticazione.

Stati di occlusione possibili:

no_occlusion_detected;

minor_occlusion;

major_occlusion;

eyes_occluded;

mouth_occluded;

nose_occluded;

landmark_occlusion_uncertain.

Per MicroBot, l'occlusione maggiore dovrebbe portare a `retry_required`. Per accessi critici, anche occlusioni minori potrebbero richiedere nuovo tentativo. Questo è particolarmente importante se l'accesso sblocca diagnostica o comandi hardware.

Il sesto ruolo è la **stabilità multi-frame**. In una singola immagine, i landmark possono essere rumorosi. Analizzando più frame consecutivi, il sistema può verificare se i punti del volto restano stabili. Se i landmark cambiano drasticamente da un frame all'altro, può esserci movimento, sfocatura, rilevamento instabile o problema di qualità. In quel caso il sistema può attendere altri frame prima di decidere. Questo rende l'autenticazione più affidabile.

Una stabilità multi-frame può valutare:

numero di frame validi consecutivi;

variazione media dei landmark;

variazione della posa;

stabilità del bounding box;

stabilità della confidenza;

assenza di volti multipli;

assenza di frame persi.

La dashboard può mostrare stati come `scanning`, `stabilizing_face`, `face_stable`, `verifying`. Questo rende l'esperienza più chiara. L'utente capisce che il sistema sta cercando un volto stabile prima di confrontarlo.

Il settimo ruolo è la **separazione tra rilevamento e riconoscimento**. I landmark non identificano necessariamente una persona. Indicano la struttura geometrica del volto nel frame. Il modulo di riconoscimento usa poi questa struttura per generare un embedding o migliorare il confronto. Questa separazione deve essere chiara nella documentazione. Dire "i landmark riconoscono l'utente" sarebbe impreciso. I landmark aiutano il riconoscimento, ma non coincidono con l'identità.

La distinzione corretta è:

face detection: trova il volto;

landmark detection: trova punti caratteristici del volto;

face recognition: confronta la rappresentazione del volto con un profilo;

access control: decide cosa l'utente può fare;

Safety Layer: decide se l'azione richiesta è sicura.

Questa catena mantiene ordine concettuale. Ogni modulo ha il suo ruolo.

L'ottavo ruolo è la **produzione di uno stato diagnostico**. Come ogni modulo MicroBot, anche la landmark detection deve essere osservabile. Non deve fallire in silenzio. Se i landmark non vengono rilevati, la dashboard e i security_events devono saperlo. Se la confidenza è bassa, deve essere indicato. Se la posa è fuori range, deve essere indicato. Questo permette di distinguere un fallimento biometrico da un problema di qualità immagine.

Un landmark_status può contenere:

landmark_detector_ready;

landmarks_detected;

landmark_count;

landmark_confidence;

pose_status;

occlusion_status;

quality_status;

last_landmark_error;

frames_valid_count.

Questi dati non devono essere necessariamente mostrati tutti all'utente normale. Possono essere visibili in modalità developer. All'utente normale basta un messaggio semplice: volto non centrato, qualità insufficiente, riprovare. Ma per debug è utile avere dettagli.

Il nono ruolo è la **gestione degli errori del modello di landmark**. Il modello o libreria che rileva i landmark può non essere caricato, può fallire, può non supportare il browser, può avere prestazioni basse o può produrre output vuoti. Il sistema deve gestire questi casi con stati chiari, come landmark_model_unavailable, landmark_processing_error, unsupported_platform o landmark_timeout. In caso di errore del modulo, l'autenticazione biometrica deve fallire in modo sicuro, senza concedere accesso elevato.

Stati di errore possibili:

landmark_model_loading;

landmark_model_ready;

landmark_model_unavailable;

landmark_processing_error;

landmark_timeout;

landmark_output_invalid;

landmark_confidence_unavailable.

Se `landmark_model_unavailable`, il sistema può passare a `mock_auth` solo se `development_mode` è attivo e chiaramente indicato. In modalità reale, deve negare accesso biometrico o usare fallback autorizzato. Questo mantiene il comportamento fail-safe.

Il decimo ruolo è la **integrazione con lo score finale di autenticazione**. La landmark detection può influenzare il risultato finale non solo come blocco sì/no, ma anche come fattore di confidenza. Per esempio, se il confronto facciale produce uno score sopra soglia ma i landmark sono deboli, il sistema può classificare `auth_success_low_confidence` o `auth_uncertain`. Se i landmark sono stabili e la qualità alta, può classificare `auth_success_high_confidence`. Questo permette una decisione più fine.

Una formula decisionale concettuale può essere:

`auth_confidence` = funzione di `match_score`, `landmark_confidence`, `image_quality`, `pose_status`, `multi_frame_stability`

Non serve implementare una formula complessa nelle prime versioni. Basta definire che il risultato finale non dipende solo dallo score biometrico, ma anche dalla qualità della rilevazione. Questo è più serio tecnicamente.

L'undicesimo ruolo è la **protezione contro decisioni su dati incompleti**. Se la landmark detection produce dati parziali, il sistema deve decidere se procedere o fermarsi. Per funzioni a basso rischio, si potrebbe accettare `landmark_confidence` moderata. Per funzioni critiche, si deve richiedere `landmark_confidence` alta. Questa policy deve essere collegata al livello di accesso richiesto. Un accesso viewer può tollerare più incertezza; `hardware_admin` deve richiedere qualità alta.

Una policy può essere:

viewer access: landmark quality sufficiente;

operator access: landmark quality buona;

developer access: landmark quality buona e posa accettabile;

admin access: landmark quality alta;

hardware_admin access: landmark quality alta, volto stabile e nessuna ambiguità.

Questa differenziazione è coerente con il controllo degli accessi. Più l'azione è rischiosa, più l'autenticazione deve essere robusta.

Il dodicesimo ruolo è la **riduzione dei falsi rifiuti**. Se la landmark detection è troppo severa, può rifiutare utenti autorizzati per piccoli problemi di luce o posa. Questo genera frustrazione. Per evitarlo, il sistema può fornire feedback utile: avvicinarsi alla camera, migliorare la luce, guardare frontalmente, rimuovere occlusioni, restare fermi. Il feedback deve essere semplice e pratico. In questo modo, la landmark detection non è solo filtro, ma guida l'utente a produrre un frame migliore.

Messaggi utili nella dashboard:

volto non rilevato;

avvicinarsi alla camera;

centrare il volto;

migliorare illuminazione;

tenere il volto fermo;

rimuovere occlusioni;

una sola persona davanti alla camera;

guardare frontalmente.

Questi messaggi migliorano l'esperienza e riducono falsi fallimenti. La biometria diventa più usabile.

Il tredicesimo ruolo è la **riduzione dei falsi accessi**. Se la landmark detection è troppo permissiva, può permettere confronti su volti parziali o ambigui, aumentando rischio di falso positivo. Per funzioni sensibili, il sistema deve essere conservativo. Deve bloccare frame con più volti, landmark instabili, posa estrema o qualità molto bassa. Questo riduce probabilità di accessi sbagliati. Il bilanciamento tra usabilità e sicurezza deve essere esplicito.

Per MicroBot, una regola prudente è:

per ogni funzione che può influenzare hardware, diagnostica critica o configurazione, la landmark detection deve essere considerata valida solo se il volto è singolo, stabile, frontale e con confidenza sufficiente.

Questa regola è ragionevole e facile da spiegare. Protegge le parti più delicate.

Il quattordicesimo ruolo è la **mappatura dei landmark nel pannello developer**. In modalità sviluppatore, la dashboard potrebbe mostrare graficamente i punti rilevati sul volto, oppure mostrare un indicatore di qualità. Questo è utile per debug e demo tecnica. Tuttavia, in modalità normale, non è necessario mostrare troppi dettagli. Inoltre, bisogna evitare di salvare immagini o overlay nei log. La visualizzazione dei landmark deve essere live e temporanea.

Una UI developer può mostrare:

bounding box del volto;

punti landmark principali;

confidence globale;

pose status;

quality score;

messaggi di correzione.

Questa UI dimostra il funzionamento della pipeline. Ma deve essere distinta da una modalità pubblica o read-only. In una demo pubblica, può bastare mostrare auth status e access level.

Il quindicesimo ruolo è la **separazione tra dati temporanei e dati persistenti**. I landmark rilevati in tempo reale non dovrebbero essere salvati come dati persistenti, salvo esplicita modalità debug

controllata. Nella maggior parte dei casi, servono solo per l'autenticazione corrente. Dopo la decisione, possono essere scartati. I security_events possono registrare landmark_status e quality_status, ma non la mappa completa dei punti. Questo segue il principio di minimizzazione.

Una policy corretta è:

landmark raw data temporanei;

nessun salvataggio automatico dei punti;

nessun log dei landmark completi nei log generali;

salvare solo esito qualitativo: landmarks_detected, confidence_class, pose_status;

debug export disabilitato di default.

Questa scelta riduce esposizione dei dati biometrici. Anche i landmark, pur non essendo immagine completa, derivano dal volto e vanno trattati con cautela.

Il sedicesimo ruolo è la **integrazione con la fase di enroll**. Se il sistema registra un profilo biometrico, la landmark detection è utile anche durante l'enroll. Prima di salvare un riferimento, il sistema deve verificare che i frame di registrazione siano di buona qualità. Se un template viene creato da immagini scarse, il riconoscimento futuro sarà instabile. Quindi durante enroll, i landmark devono essere stabili e completi.

Una pipeline enroll con landmark può essere:

camera attiva;

rilevamento volto singolo;

landmark detection;

controllo qualità e posa;

acquisizione di più frame validi;

creazione template;

salvataggio riferimento;

test di verifica.

Se landmark_confidence è bassa durante enroll, il sistema deve rifiutare quel frame. È meglio spendere qualche secondo in più durante la registrazione che avere un profilo biometrico debole.

Il diciassettesimo ruolo è la **compatibilità con modelli diversi**. Esistono vari approcci alla landmark detection. Alcuni rilevano pochi punti, altri molti. La documentazione MicroBot non deve legarsi subito a un modello specifico, a meno che l'implementazione lo richieda. È meglio definire un'interfaccia astratta: il modulo landmark deve restituire stato, lista punti opzionale, confidence, pose_status e quality_status. Così in futuro si potrà cambiare libreria o modello senza riscrivere access_control.

Una interfaccia astratta può essere:

`detect_landmarks(frame) → landmark_result`

Dove `landmark_result` contiene:

`success;`

`points;`

`confidence;`

`pose_status;`

`quality_status;`

`error_code.`

Questa astrazione è coerente con il resto dell'architettura MicroBot: moduli separati, interfacce chiare, stato osservabile.

Il diciottesimo ruolo è la **gestione delle prestazioni**. La landmark detection può essere computazionalmente più costosa della semplice visualizzazione. In una dashboard web, bisogna evitare che il modulo rallenti l'interfaccia o interferisca con la simulazione MicroBot. Può essere eseguita a frequenza ridotta, su frame selezionati, o solo durante `auth_attempt`. Non deve girare continuamente se non serve. Questo riduce consumo CPU e protegge l'esperienza utente.

Una policy prestazionale può essere:

landmark detection attiva solo durante autenticazione;

frame rate ridotto per elaborazione;

stop automatico dopo decisione;

nessuna elaborazione continua in read-only;

diagnostica landmark attiva solo in developer mode.

Questo è importante perché MicroBot ha già dashboard, simulazione e telemetria. Il modulo biometrico non deve appesantire tutto inutilmente.

Il diciannovesimo ruolo è la **gestione degli stati nel security module**. La landmark detection deve avere stati integrati nella macchina a stati della sicurezza biometrica. Per esempio, la pipeline può passare da `camera_ready` a `face_detected`, poi `landmarks_detected`, poi `recognition_ready`, poi `comparison_done`, poi `access_decision`. Se `landmarks_failed`, torna a `retry_required`. Questa macchina a stati rende la pipeline più chiara.

Una possibile macchina a stati è:

`auth_idle;`

`camera_initializing;`

`camera_ready;`

`face_searching;`

face_detected;
landmarks_processing;
landmarks_valid;
landmarks_invalid;
recognition_processing;
comparison_processing;
decision_ready;
auth_success;
auth_failed;
auth_uncertain;
retry_required.

Questa struttura è molto utile per dashboard e debug. Ogni fase ha uno stato leggibile.

Il ventesimo ruolo è la **relazione con il confronto dei dati**. Il punto successivo della sezione sarà dedicato al confronto dei dati. La landmark detection prepara quel confronto. Se i landmark sono validi, il volto può essere normalizzato e convertito in embedding. Se non sono validi, il confronto potrebbe essere non affidabile. Quindi landmark_status deve essere un input per comparison_status. Il confronto non dovrebbe essere indipendente dalla qualità.

La relazione è:

landmark valido → confronto permesso;
landmark incerto → confronto possibile ma confidence ridotta o retry;
landmark fallito → confronto bloccato;
landmark multiplo/ambiguo → accesso negato o retry.

Questa logica rende la decisione più robusta. Il confronto dei dati non parte da un'immagine grezza qualunque, ma da un input controllato.

Il ventunesimo ruolo è la **classificazione degli errori landmark**. Gli errori devono essere espliciti e standardizzati. Non basta "landmark error". Bisogna distinguere no_face, multiple_faces, model_unavailable, low_confidence, pose_out_of_range, occlusion_detected, timeout, frame_invalid. Questo permette alla UI e ai log di essere precisi.

Codici di errore possibili:

LMK_NO_FACE;
LMK_MULTIPLE_FACES;

LMK_MODEL_UNAVAILABLE;

LMK_LOW_CONFIDENCE;

LMK_POSE_OUT_OF_RANGE;

LMK_OCCLUSION_HIGH;

LMK_FRAME_TOO_DARK;

LMK_FRAME_BLURRY;

LMK_TIMEOUT;

LMK_OUTPUT_INVALID.

Questi codici possono essere usati nei `security_events`. Una dashboard developer può mostrarli; una dashboard normale può tradurli in messaggi semplici.

Il ventiduesimo ruolo è la **calibrazione delle soglie landmark**. Come per il riconoscimento, anche i landmark possono avere soglie: confidenza minima, dimensione minima del volto, inclinazione massima, numero minimo di frame stabili. Queste soglie devono essere calibrate. Se sono troppo severe, l'utente fatica ad autenticarsi. Se sono troppo permissive, aumenta rischio di decisioni deboli. Le soglie possono essere diverse per livelli di accesso.

Parametri possibili:

`min_landmark_confidence`;

`min_face_size`;

`max_pose_angle`;

`min_valid_frames`;

`max_frame_blur`;

`min_brightness`;

`max_face_count` = 1.

Questi parametri possono stare in `biometric_policy`. La gestione deve essere protetta: operator non dovrebbe poterli modificare. Developer o admin possono modificarli in modalità sperimentale.

Il ventitreesimo ruolo è la **compatibilità con accesso non biometrico**. Se la landmark detection fallisce per ragioni tecniche, il sistema può permettere fallback controllato, come password locale o manual developer unlock in `development_mode`. Questo fallback deve essere loggato e non deve concedere automaticamente `hardware_admin`. La landmark detection non deve diventare punto singolo di fallimento che blocca tutto, ma se fallisce non deve nemmeno aprire permessi elevati senza controllo.

Una logica fallback può essere:

`landmark unavailable` → accesso biometrico non disponibile;

guest/viewer resta possibile;

developer fallback solo se development_mode;

hardware actions restano bloccate o richiedono altro fattore;

security_event registrato.

Questo è equilibrato: non blocca completamente la demo, ma protegge funzioni sensibili.

Il ventiquattresimo ruolo è la **protezione della modalità demo**. In una demo pubblica, si potrebbe voler mostrare la landmark detection come effetto visivo, con punti sul volto. Bisogna però evitare di salvare quei dati. Inoltre, se la demo è pubblica, non dovrebbe usare profili biometrici reali senza consenso. Una modalità demo può usare mock o mostrare solo face tracking non identificativo. Questa è una distinzione importante.

Modalità possibili:

landmark_demo_no_identity;

auth_mode_real_identity;

mock_auth_mode;

developer_debug_mode.

landmark_demo_no_identity può mostrare punti senza confrontare con profili.

auth_mode_real_identity invece tratta dati biometrici e deve essere più protetta. Questa distinzione è utile per presentazioni.

Il venticinquesimo ruolo è la **relazione con sicurezza e privacy**. I landmark sono dati derivati dal volto. Anche se non sono immagine completa, devono essere considerati parte della pipeline biometrica. Per questo è prudente non conservarli, non esportarli e non metterli nei sample. La documentazione deve dire che i landmark vengono usati temporaneamente per qualità e normalizzazione, non come log persistente.

Una policy chiara è:

i landmark vengono calcolati durante l'autenticazione;

vengono usati per qualità, posa e normalizzazione;

non vengono salvati nei log generali;

non vengono esportati nei report tecnici;

gli eventi salvano solo lo stato qualitativo;

la modalità debug deve essere esplicita e temporanea.

Questo mantiene la sezione coerente con il tema sensibile della biometria.

Il ventiseiesimo ruolo è la **testabilità del modulo**. La landmark detection deve essere testabile senza attivare tutta la pipeline. Si possono testare casi: no face, one face, multiple faces, low light, occlusion, pose angle, model unavailable. Questi test aiutano a verificare che il modulo risponda

con stati corretti. In sviluppo, si può usare mock landmark_result per testare access_control senza camera reale.

Test utili:

no_face_returns_retry;

multiple_faces_denies_auth;

low_confidence_blocks_high_access;

pose_out_of_range_requests_retry;

landmark_model_unavailable_fails_safe;

valid_landmarks_allow_recognition;

mock_landmarks_enable_ui_testing.

Questi test sono importanti perché il modulo biometrico deve essere affidabile e prevedibile.

Il ventisettesimo ruolo è la **integrazione con la documentazione tecnica**. La sezione landmark deve essere documentata in modo chiaro: scopo, input, output, stati, errori, policy di qualità, privacy, rapporto con recognition e access control. Questo evita che venga interpretata come funzione magica. È un modulo tecnico preciso.

Un documento landmark_detection_module.md può includere:

scopo;

input camera frame;

face detection dependency;

landmark output;

quality checks;

pose estimation;

occlusion checks;

multi-frame stability;

error codes;

privacy policy;

integration with recognition;

limitations;

roadmap.

Questa documentazione può essere utile anche per repository e presentazione.

Il ventottesimo ruolo è la **roadmap della landmark detection**. Anche questo modulo può crescere per versioni. Prima si può avere solo face box. Poi punti principali. Poi confidence. Poi posa. Poi qualità frame. Poi stabilità multi-frame. Poi UI developer. Poi integrazione con enroll. Poi controlli di occlusione. Poi eventuale liveness futura. Questa progressione è coerente con lo sviluppo incrementale MicroBot.

Una roadmap possibile:

- v0.1: face box e stato face_detected;
- v0.2: landmark base occhi/naso/bocca;
- v0.3: landmark_confidence;
- v0.4: controllo volto singolo;
- v0.5: pose_status;
- v0.6: quality_status;
- v0.7: multi-frame stability;
- v0.8: integrazione con recognition;
- v0.9: pannello developer con overlay;
- v1.0: integrazione con enroll;
- v1.1: occlusion_status;
- v1.2: policy per access_level;
- v1.3: controlli liveness sperimentali, se implementati.

Questa roadmap permette di migliorare il modulo senza promettere subito funzioni avanzate.

Il ventinovesimo ruolo è la **dichiarazione dei limiti**. La landmark detection non garantisce da sola identità, presenza reale o sicurezza completa. Può fallire con illuminazione scarsa, pose estreme, occlusioni, camera di bassa qualità, modelli non calibrati o differenze ambientali. Deve essere usata come supporto al riconoscimento, non come unica protezione. Questi limiti devono essere dichiarati.

Limiti principali:

- non identifica da sola l'utente;
- non garantisce liveness;
- dipende dalla qualità della camera;
- può fallire con occlusioni;
- può essere instabile con luce scarsa;
- richiede calibrazione delle soglie;

deve essere combinata con `access_control` e `Safety Layer`.

Questa dichiarazione è importante per rigore tecnico e per non sopravvalutare il modulo.

Il trentesimo ruolo è la **collocazione della landmark detection nella visione MicroBot**. Questo modulo mostra che MicroBot non è soltanto un progetto di swarm o hardware, ma un ecosistema che può integrare computer vision, sicurezza, interazione uomo-sistema e controllo modulare. La landmark detection aggiunge un livello di percezione: il sistema non riceve solo click e comandi, ma può interpretare la presenza dell'utente davanti alla camera. Tuttavia, questa percezione deve restare controllata, minimizzata e subordinata alle policy di accesso e safety.

In conclusione, la landmark detection è il processo attraverso cui il sistema MicroBot individua e valuta punti caratteristici del volto per migliorare qualità, normalizzazione, posa, stabilità e affidabilità del riconoscimento facciale. Essa non decide direttamente l'accesso, ma prepara e valida l'input per il confronto biometrico. Permette di rilevare volti parziali, pose non adatte, bassa confidenza, occlusioni, instabilità e frame non idonei. Deve produrre stati osservabili, errori chiari, policy di qualità e integrazione con `dashboard`, `security_events` e `access_control`. I dati landmark devono essere trattati come temporanei e sensibili, non salvati nei log generali. La landmark detection rende la pipeline biometrica più robusta, ma non sostituisce riconoscimento, soglia, access control o `Safety Layer`. Il punto successivo, dedicato al confronto dei dati, analizzerà come la rappresentazione del volto corrente venga confrontata con un riferimento autorizzato, come venga calcolato uno score di somiglianza o distanza e come questo score venga preparato per la decisione finale tramite soglia.

15.5 Confronto dei dati

Il **confronto dei dati** rappresenta il quinto livello della sicurezza biometrica MicroBot e descrive il processo attraverso cui le informazioni estratte dal volto dell'utente vengono confrontate con un riferimento autorizzato per stabilire se l'identità può essere considerata compatibile. Dopo aver definito autenticazione tramite fotocamera, controllo degli accessi, riconoscimento facciale e landmark detection, questo punto entra nella fase decisionale preliminare: il sistema ha acquisito un volto, lo ha rilevato, ha verificato la qualità tramite landmark e ha prodotto una rappresentazione biometrica; ora deve confrontare tale rappresentazione con un dato di riferimento. Il confronto dei dati è quindi il passaggio che trasforma l'osservazione visiva in una misura numerica di compatibilità.

Nel progetto MicroBot, questo confronto non deve essere pensato come un semplice “uguale o diverso”. Un volto umano non viene confrontato come una password testuale, dove due stringhe coincidono perfettamente oppure no. Le immagini del volto cambiano continuamente: luce, posa, distanza, camera, espressione, occlusioni e qualità del frame producono variazioni. Per questo il confronto biometrico deve lavorare con uno **score**, cioè un valore numerico che misura quanto il volto corrente sia simile al riferimento salvato. Questo score non decide da solo l'accesso; prepara la decisione finale, che verrà poi presa tramite soglia nel punto successivo.

Il principio generale può essere espresso così:

il confronto biometrico non produce una certezza assoluta, ma una misura di somiglianza o distanza che deve essere interpretata secondo soglie, qualità del dato e livello di accesso richiesto.

Questa frase è fondamentale perché impedisce di trattare il riconoscimento facciale come una verità matematica perfetta. Il confronto produce probabilità operativa, non certezza assoluta. Per questo deve essere integrato con `landmark_confidence`, `image_quality`, `pose_status`, `access_level`, `session`

policy e Safety Layer. Se lo score è molto buono ma la qualità immagine è bassa, il sistema deve essere prudente. Se lo score è vicino alla soglia, deve dichiarare incertezza. Se l'azione richiesta è hardware-critical, deve richiedere confidenza più alta.

Il primo elemento del confronto è il **dato corrente**, cioè la rappresentazione del volto appena acquisito. Questo dato deriva dalla pipeline precedente: camera frame, face detection, landmark detection, normalizzazione e generazione dell'embedding o template. Il dato corrente non dovrebbe essere il frame grezzo, ma una rappresentazione numerica compatta del volto. Questa rappresentazione viene usata solo per il confronto dell'autenticazione corrente e, nella maggior parte dei casi, non dovrebbe essere salvata nei log. Deve essere trattata come dato biometrico temporaneo.

Il dato corrente può essere descritto come:

current_face_embedding;

current_face_template;

current_biometric_vector;

current_sample;

current_auth_representation.

Il nome preciso dipende dall'implementazione, ma il concetto è sempre lo stesso: il volto corrente viene convertito in una struttura confrontabile. Questa struttura deve avere metadati associati: timestamp, qualità, landmark_confidence, source, model_version, preprocessing_version e auth_attempt_id. Senza metadati, lo score finale diventa difficile da interpretare.

Il secondo elemento è il **dato di riferimento**, cioè la rappresentazione biometrica associata a un profilo autorizzato. Il riferimento può essere creato durante una fase di enroll, cioè registrazione dell'utente. Può essere composto da uno o più embedding validi, generati da più frame di buona qualità. Usare più campioni può rendere il riconoscimento più robusto, perché tiene conto di piccole variazioni del volto. Tuttavia, più dati biometrici vengono salvati, maggiore è la responsabilità di proteggerli. Per una prima versione MicroBot, può bastare un riferimento locale singolo o mock.

Il dato di riferimento può essere descritto come:

reference_embedding;

stored_template;

authorized_profile_template;

profile_biometric_reference;

enrolled_face_vector.

Questo dato deve essere associato a un profile_id e a un access_level, ma deve essere separato dai log generali e dalla repository pubblica. Non deve essere inserito nei file di esempio reali. Se il progetto viene pubblicato, i sample devono usare dati fittizi o mock. Il riferimento biometrico è parte del modulo security, non della normale telemetria dello swarm.

Il terzo elemento è la **funzione di confronto**. Questa funzione prende il dato corrente e il dato di riferimento e produce un valore numerico. Esistono vari modi di confrontare rappresentazioni biometriche. Alcuni sistemi calcolano una similarità, dove valore alto significa maggiore somiglianza. Altri calcolano una distanza, dove valore basso significa maggiore somiglianza. La documentazione MicroBot deve dichiarare chiaramente quale convenzione viene usata, perché la decisione tramite soglia cambia completamente.

Le due convenzioni principali sono:

similarity_score alto = volti più simili;

distance_score basso = volti più simili.

Se si usa similarity_score, l'accesso può essere concesso quando $\text{score} \geq \text{threshold}$. Se si usa distance_score, l'accesso può essere concesso quando $\text{distance} \leq \text{threshold}$. Questa differenza deve essere esplicita. Un errore nel verso della soglia sarebbe gravissimo: potrebbe concedere accesso a volti poco simili o negarlo a volti corretti.

Il quarto elemento è il **modello di confronto 1:1**. Nel caso più semplice e prudente, il sistema confronta il volto corrente con un solo profilo dichiarato o previsto. Questo è chiamato verifica 1:1. Per esempio, la dashboard può verificare se il volto davanti alla camera corrisponde al profilo local_developer. Questa modalità è più controllabile rispetto all'identificazione 1:N, dove il sistema cerca automaticamente tra molti profili. Per MicroBot, soprattutto nelle prime versioni, il confronto 1:1 è più adatto.

La verifica 1:1 segue questa logica:

utente richiede accesso;

profilo atteso selezionato;

volto corrente acquisito;

embedding corrente generato;

confronto con embedding del profilo;

score prodotto;

decisione tramite soglia.

Questa modalità riduce ambiguità e complessità. Se in futuro MicroBot avrà più utenti, si potrà introdurre confronto 1:N, ma dovrà essere gestito con più attenzione.

Il quinto elemento è il **confronto 1:N**, cioè identificazione tra più profili. In questo caso, il sistema confronta il volto corrente con tutti i riferimenti autorizzati e sceglie il profilo più simile. Questa modalità può essere utile in una dashboard multiutente: il sistema riconosce automaticamente se davanti alla camera c'è operator, developer o admin. Tuttavia, è più delicata perché aumenta la possibilità di confusione tra profili, richiede gestione di soglie multiple, ranking, margine tra primo e secondo risultato e policy più rigide.

Un confronto 1:N dovrebbe calcolare:

score per ogni profilo;

best_match_profile;

best_match_score;

second_best_score;

margin_between_matches;

threshold_for_best_profile;

decisione finale.

Il margine tra primo e secondo risultato è importante. Se il migliore e il secondo migliore sono troppo vicini, il sistema deve dichiarare auth_uncertain. Non basta che il primo superi la soglia; bisogna anche evitare che due profili siano quasi equivalenti. Per le prime versioni, è meglio evitare complessità e restare su 1:1.

Il sesto elemento è la **qualità del dato confrontato**. Lo score biometrico non deve essere interpretato da solo. Deve essere pesato o almeno accompagnato dalla qualità dell'immagine, dalla confidenza dei landmark, dalla stabilità multi-frame e dalla posa del volto. Un confronto con score alto ma qualità bassa potrebbe essere meno affidabile di un confronto con score simile ma qualità ottima. Il sistema deve quindi produrre non solo match_score, ma anche comparison_confidence.

comparison_confidence può dipendere da:

image_quality;

landmark_confidence;

pose_status;

face_size;

motion_blur;

multi_frame_stability;

model_confidence;

score_margin.

In una prima implementazione, questa confidenza può essere classificata semplicemente come high, medium, low. Non serve una formula sofisticata subito. L'importante è evitare che lo score venga interpretato fuori contesto. Il confronto biometrico deve essere accompagnato da una valutazione della qualità.

Il settimo elemento è la **normalizzazione del confronto**. Se il sistema usa modelli diversi, versioni diverse o profili generati in condizioni diverse, gli score potrebbero non essere direttamente comparabili. Per questo ogni embedding o template dovrebbe avere model_version e preprocessing_version. Un riferimento creato con un modello vecchio potrebbe non essere compatibile con un embedding generato da un modello nuovo. Il sistema deve rilevare queste incompatibilità.

Un confronto dovrebbe controllare:

current_model_version;
reference_model_version;
current_preprocessing_version;
reference_preprocessing_version;
template_schema_version;
compatibility_status.

Se model_version non è compatibile, il sistema deve rifiutare il confronto o richiedere re-enroll del profilo. Non deve procedere silenziosamente. Questo è importante per un progetto che evolve per versioni. La biometria, come MicroBot OS, deve avere compatibilità dichiarata.

L'ottavo elemento è la **gestione dei profili disabilitati o non validi**. Un profilo può esistere ma essere disabled. Può essere scaduto. Può mancare del riferimento biometrico. Può avere un template corrotto. Può essere stato creato con qualità insufficiente. Il confronto deve verificare lo stato del profilo prima di calcolare o accettare il risultato. Se il profilo non è valido, anche uno score alto non deve concedere accesso.

Stati del profilo:

profile_enabled;
profile_disabled;
profile_expired;
reference_missing;
reference_invalid;
reference_low_quality;
reference_version_incompatible;
profile_requires_reenroll.

Questi stati devono generare failure_reason chiari. Per esempio, access_denied perché profile_disabled è diverso da access_denied perché score_below_threshold. Il primo riguarda amministrazione del profilo; il secondo riguarda mancata corrispondenza biometrica.

Il nono elemento è il **confronto multi-frame**. Invece di confrontare un solo frame, il sistema può acquisire più frame validi, generare più rappresentazioni e calcolare più score. Questo può rendere la decisione più stabile. Per esempio, il sistema può richiedere tre frame validi consecutivi e calcolare media o mediana degli score. Oppure può accettare il migliore solo se la qualità è alta e coerente. La scelta dipende dall'implementazione, ma concettualmente è utile.

Un confronto multi-frame può produrre:

score_frame_1;
score_frame_2;

score_frame_3;

score_mean;

score_median;

score_variance;

valid_frame_count;

final_comparison_score.

Se gli score variano molto, il sistema può dichiarare `auth_uncertain` anche se uno dei frame supera la soglia. La stabilità è una forma di affidabilità. Per accessi critici, un singolo frame fortunato non dovrebbe bastare.

Il decimo elemento è la **gestione dello score vicino alla soglia**. Questa è una zona delicata. Se lo score è molto sopra soglia, la decisione può essere più sicura. Se è molto sotto, è chiaro che l'accesso va negato. Se è vicino alla soglia, il sistema deve evitare decisioni aggressive. Può chiedere un nuovo tentativo, acquisire più frame, richiedere fallback, concedere accesso limitato o dichiarare `auth_uncertain`. Questo sarà approfondito nel punto 15.6, ma nel confronto dati bisogna già preparare il concetto.

Una classificazione possibile dello score:

`strong_match`;

`weak_match`;

`borderline_match`;

`weak_non_match`;

`strong_non_match`.

Questa classificazione è più utile di un numero grezzo. La dashboard può mostrare “verifica incerta, riprovare” invece di negare seccamente. Per `hardware_admin`, `borderline_match` non deve bastare. Per `viewer`, si può concedere accesso limitato se previsto dalla policy.

L'undicesimo elemento è la **separazione tra score e permesso**. Il confronto produce score. Il controllo accessi produce permessi. Questi due livelli non devono essere fusi. Un match biometrico con profilo `developer` non deve automaticamente inviare comandi. Deve creare `auth_result`. `access_control` legge `auth_result` e genera sessione con `allowed_actions`. Poi ogni azione passa da controller e safety. Questa separazione evita errori architetturali.

La struttura corretta è:

`comparison_result` → `auth_result`;

`auth_result` → `access_session`;

`access_session` → `allowed_actions`;

`allowed_actions` → controller request;

controller request → node command;

node command → MicroBot OS validation.

Il modulo comparison non deve sapere come accendere un LED o avviare un pattern. Deve solo dire se il volto corrente corrisponde al riferimento e con quale confidenza.

Il dodicesimo elemento è il **formato del comparison_result**. Questo oggetto deve essere chiaro e strutturato. Deve contenere auth_attempt_id, profile_id, comparison_type, score, score_type, threshold_candidate, quality_summary, confidence_class, decision_candidate e failure_reason se presente. La decisione finale può essere presa nel punto successivo, ma il comparison_result deve fornire tutti i dati necessari.

Una struttura comparison_result può contenere:

auth_attempt_id;

comparison_type: one_to_one oppure one_to_many;

profile_id_candidate;

score_type: similarity oppure distance;

score_value;

quality_status;

landmark_confidence;

pose_status;

multi_frame_stability;

comparison_confidence;

profile_status;

comparison_status;

failure_reason.

comparison_status può essere comparison_ready, comparison_done, comparison_failed, profile_missing, incompatible_template, low_quality_input. Questo oggetto è il ponte tra riconoscimento e decisione tramite soglia.

Il tredicesimo elemento è la **gestione degli errori di confronto**. Il confronto può fallire per molti motivi: embedding mancante, template mancante, modello incompatibile, vettori di dimensione diversa, qualità insufficiente, profilo disabilitato, errore del modello, dati corrotti. Ogni errore deve avere codice chiaro. Non bisogna trasformare ogni errore in “utente non riconosciuto”, perché sarebbe impreciso.

Codici possibili:

CMP_CURRENT_EMBEDDING_MISSING;

CMP_REFERENCE_MISSING;
CMP_TEMPLATE_INCOMPATIBLE;
CMP_VECTOR_DIMENSION_MISMATCH;
CMP_LOW_QUALITY_INPUT;
CMP_PROFILE_DISABLED;
CMP_MODEL_ERROR;
CMP_SCORE_UNAVAILABLE;
CMP_MULTI_FACE_AMBIGUITY;
CMP_COMPARISON_TIMEOUT.

Se il confronto fallisce per errore tecnico, la dashboard deve dire autenticazione non disponibile o riprovare, non “utente non autorizzato”. Questo migliora precisione e usabilità.

Il quattordicesimo elemento è la **protezione del dato durante il confronto**. Il dato corrente e il riferimento devono essere usati solo per il tempo necessario. Il dato corrente può essere eliminato dalla memoria dopo decisione. Il riferimento deve restare protetto. I log non devono contenere embedding completi. Gli eventi possono contenere score_class o decision, ma non vettori biometrici. Questa protezione è essenziale.

Una policy prudente è:

current_embedding volatile;
reference_template separato e protetto;
no embedding nei log generali;
no template in repository;
no export non autorizzato;
security_event con decisione e motivo, non con dati biometrici completi.

Questa policy mantiene la pipeline responsabile. Anche durante debug, bisogna evitare di stampare vettori biometrici completi nella console.

Il quindicesimo elemento è la **conservazione dello score**. Salvare lo score in un security_event può essere utile per debug, ma può anche rivelare informazioni sul confronto. Una scelta prudente è salvare una classe qualitativa, come high_confidence_match, borderline, below_threshold, invece del valore numerico preciso, almeno nei log generali. In modalità developer locale, si può mostrare score per calibrazione, ma con attenzione.

Possibili livelli di logging:

general_security_log: decision, reason, confidence_class;
developer_debug_log: score_value opzionale, senza embedding;

biometric_debug_mode: dati più dettagliati, non persistenti, solo locale;

production_like_mode: niente score grezzo, solo decisione.

Questa distinzione permette sia sviluppo sia minimizzazione dei dati. Durante la calibrazione può servire vedere score; durante uso normale no.

Il sedicesimo elemento è il **confronto come parte del sistema di accesso, non della telemetria tecnica generale**. Il confronto biometrico produce security data. Non deve essere mischiato con telemetry del nodo, come temperatura, pattern_error o heartbeat. Può generare un security_event, e la dashboard può ricevere auth_state. Ma i dettagli biometrici devono restare nel modulo security. Questa separazione mantiene ordine e riduce rischi.

La separazione corretta è:

telemetry_system: nodi, swarm, safety, pattern;

security_system: autenticazione, confronto, accessi, sessioni;

biometric_processing: dati temporanei del volto;

access_control: permessi e policy.

La dashboard può combinare le informazioni visualmente, ma i dati devono restare separati nel modello.

Il diciassettesimo elemento è la **gestione del confronto in modalità mock**. Durante sviluppo, il confronto può essere simulato. Un mock comparison_result può restituire score alto, basso o borderline. Questo permette di testare decisione tramite soglia, access_control, sessioni, UI e security_events senza usare camera o dati biometrici reali. La modalità mock deve essere evidente e loggata.

Mock utili:

mock_strong_match;

mock_borderline_match;

mock_failed_match;

mock_profile_disabled;

mock_low_quality;

mock_model_error.

Questi casi permettono test completi. Per esempio, si può verificare che hardware_admin non venga concesso su borderline_match. Oppure che profile_disabled neghi accesso anche con strong_match. Questo è molto utile per costruire il sistema in modo sicuro.

Il diciottesimo elemento è il **confronto con profili multipli e gestione del best match**. Se in futuro si introduce identificazione 1:N, il sistema deve evitare decisioni deboli. Non basta trovare il profilo con score migliore. Bisogna verificare che il migliore superi soglia e che il margine rispetto al

secondo sia sufficiente. Se il margine è piccolo, il sistema deve dichiarare incertezza. Questo evita confusione tra utenti simili o dati poco chiari.

Una logica 1:N prudente:

calcolare score per ogni profilo;

ordinare i profili per score;

identificare best_match;

controllare soglia del best_match;

controllare margine rispetto al secondo;

verificare profilo abilitato;

generare auth_success solo se tutti i controlli passano.

Se best_score supera soglia ma margin è basso, decisione = auth_uncertain. Questo è più robusto.

Il diciannovesimo elemento è la **gestione dei diversi livelli di soglia**. Il confronto può preparare soglie diverse per accessi diversi. La soglia finale non deve essere necessariamente una sola. Per esempio, per concedere viewer può bastare threshold_viewer. Per developer serve threshold_developer. Per admin threshold_admin. Per hardware_admin threshold_hardware più re-auth. Il comparison_result può quindi fornire score_value, mentre access_control decide quale soglia applicare.

Una struttura policy può contenere:

threshold_viewer;

threshold_operator;

threshold_developer;

threshold_admin;

threshold_hardware_admin;

borderline_margin;

required_quality_by_access_level.

Questa struttura collega confronto e access control in modo flessibile. Il modulo biometrico produce score; la policy decide quanto score serve per ogni livello.

Il ventesimo elemento è la **calibrazione del confronto**. La soglia deve essere scelta in base a prove. Se si usa un profilo locale, bisogna testare più condizioni: luce buona, luce scarsa, distanza diversa, volto inclinato, espressioni diverse. Bisogna anche testare volti non autorizzati, se possibile in modo etico e controllato, oppure usare mock per simulare non-match. La calibrazione serve a evitare soglie casuali. Per un progetto dimostrativo, basta una calibrazione documentata; non serve una campagna scientifica enorme, ma serve metodo.

Una calibrazione minima può registrare:

numero tentativi autorizzati;

range score autorizzato;

numero tentativi non autorizzati;

range score non autorizzato;

soglia scelta;

motivo della soglia;

limiti del test;

condizioni ambientali.

Questi dati possono essere descritti in documentazione senza salvare dati biometrici. Per esempio, si può dire che la soglia è sperimentale e va ricalibrata in ambienti diversi. Questo è rigore.

Il ventunesimo elemento è la **relazione tra confronto e falso positivo/falso negativo**. Il confronto dati è il punto in cui questi rischi emergono. Se la soglia è permissiva, aumenta il rischio di falso positivo. Se è severa, aumenta il rischio di falso negativo. Per MicroBot, le funzioni hardware e admin devono minimizzare falsi positivi. Una falsa negazione è fastidiosa, ma un falso accesso è più grave. Quindi il sistema deve privilegiare sicurezza per funzioni critiche.

Una policy ragionevole:

azioni read-only: tolleranza moderata;

azioni software non critiche: soglia normale;

diagnostica: soglia più severa;

configurazione admin: soglia alta;

hardware actuation: soglia alta più conferma/safety.

Questo significa che lo stesso score può concedere viewer ma non hardware_admin. È un approccio maturo e proporzionato.

Il ventiduesimo elemento è la **gestione della distanza temporale tra autenticazione e azione**. Il confronto dei dati avviene in un auth_attempt, ma l'utente può eseguire un comando minuti dopo. Più tempo passa, meno l'autenticazione è "fresca". Per azioni critiche, il sistema può richiedere un confronto recente. Questo si chiama re-auth. Il comparison_result deve quindi avere timestamp e valid_until. access_control deve verificare se l'autenticazione è ancora valida.

Campi utili:

comparison_timestamp;

auth_freshness_ms;

valid_until;

requires_reauth_after_ms;

last_critical_reauth.

Per hardware_test, può essere richiesto che l'autenticazione sia stata eseguita negli ultimi pochi secondi o che venga confermata. Per viewer, la sessione può durare più a lungo. Questa differenziazione riduce rischi.

Il ventitreesimo elemento è la **gestione di condizioni ambientali variabili**. Il confronto può cambiare molto se la luce cambia. Se il sistema è usato in laboratorio, camera e illuminazione possono essere abbastanza stabili. Se viene usato in ambienti diversi, la soglia potrebbe non essere adatta. Il comparison_result dovrebbe includere quality_status e environment flags, come low_light o overexposed. In condizioni scarse, il sistema può non procedere.

Stati ambientali possibili:

lighting_ok;

low_light;

overexposed;

motion_blur;

background_interference;

camera_low_resolution;

face_too_far;

face_too_close.

Questi stati non sono identità, ma influenzano lo score. Devono quindi essere parte del contesto del confronto.

Il ventiquattresimo elemento è la **gestione di profili aggiornati**. Se un profilo viene aggiornato o re-enrolled, il riferimento cambia. Il sistema deve registrare profile_version o reference_version. I vecchi comparison_result non devono essere usati per sessioni nuove. Se il profilo cambia, sessioni precedenti possono essere invalidate. Questo è importante per coerenza.

Campi utili:

profile_id;

profile_version;

reference_version;

enrolled_at;

last_updated_at;

template_status;

requires_reauth_after_profile_update.

Se un admin disabilita o aggiorna un profilo, le sessioni attive associate devono essere riesaminate o revocate. Questo è parte del controllo accessi, ma il confronto deve fornire versioni coerenti.

Il venticinquesimo elemento è la **diagnostica del confronto**. Il modulo comparison deve poter dire se funziona: modello caricato, profili disponibili, reference valido, ultimo score disponibile, ultimo failure_reason, modalità mock o reale, tempo medio di confronto. Questi dati sono utili per developer, ma non devono esporre embedding. La dashboard developer può mostrare comparison_status.

Un comparison_diagnostics può contenere:

comparison_model_ready;

reference_store_ready;

profile_count;

last_comparison_status;

last_failure_reason;

average_comparison_time_ms;

mock_mode_active;

score_type;

threshold_policy_loaded;

template_version_supported.

Se comparison_model_ready = false, l'autenticazione biometrica non è disponibile. Se threshold_policy_loaded = false, non si può decidere accesso. Se reference_store_ready = false, i profili non sono accessibili. Questi stati devono essere chiari.

Il ventiseiesimo elemento è la **sicurezza del reference store**. Il luogo dove sono conservati i riferimenti biometrici deve essere separato. In una versione locale semplice, può essere una cartella non pubblica o uno storage locale protetto. In una versione più seria, servirebbero cifratura, permessi e gestione sicura. Per la documentazione MicroBot, è importante dichiarare che i riferimenti non devono stare nella cartella pubblica web né nella repo.

Una policy reference store:

non pubblico;

non versionato su Git;

accessibile solo al modulo security;

supporta delete_profile;

supporta disable_profile;

non esporta template grezzi;

usa dati mock nei sample.

Questa policy è essenziale. Un progetto GitHub non deve includere volti o template reali. Sarebbe un errore serio.

Il ventisettesimo elemento è la **gestione del confronto come evento tracciabile**. Ogni confronto può generare un evento, ma con dati minimizzati. Per esempio, comparison_done, comparison_failed, score_class = strong_match/borderline/no_match, profile_status, decision_candidate. Non serve salvare score esatto in modalità normale. Questo permette audit senza esposizione inutile.

Eventi possibili:

comparison_started;

comparison_done;

comparison_failed;

reference_missing;

template_incompatible;

score_below_threshold;

score_borderline;

strong_match_detected;

comparison_uncertain.

Questi eventi possono essere collegati ad auth_attempt_id. La sequenza completa diventa: auth_attempt_started → face_detected → landmarks_valid → comparison_done → decision_made → access_granted/access_denied.

Il ventottesimo elemento è la **roadmap del confronto dati**. Come gli altri moduli, anche il confronto può crescere progressivamente. Si può partire da mock comparison, poi profilo singolo, poi score, poi soglia, poi qualità, poi multi-frame, poi profili multipli, poi diagnostica e calibrazione. Questa progressione riduce complessità.

Una roadmap possibile:

v0.1: mock comparison result;

v0.2: confronto 1:1 con profilo locale fittizio;

v0.3: score_value e score_type;

v0.4: threshold policy semplice;

v0.5: quality-aware comparison;

v0.6: multi-frame comparison;

v0.7: profile_status e versioning;

v0.8: security_events del confronto;

v0.9: comparison diagnostics;

v1.0: enroll + reference store locale;

v1.1: multi-profile 1:N opzionale;

v1.2: calibrazione e soglie per access_level.

Questa roadmap mantiene il modulo coerente e sviluppabile.

Il ventinovesimo elemento è la **dichiarazione dei limiti**. Il confronto dei dati non garantisce sicurezza assoluta. Dipende dal modello, dalla qualità del riferimento, dalla qualità del frame, dalla soglia, dalla camera e dal contesto. Non garantisce liveness. Non deve essere usato come unico controllo per hardware reale. Deve essere accompagnato da access_control, sessioni, logging, re-auth per azioni critiche e Safety Layer. Questi limiti devono essere dichiarati.

Limiti principali:

score probabilistico, non certezza assoluta;

dipendenza dalla qualità immagine;

dipendenza dalla qualità del profilo;

soglia da calibrare;

possibili falsi positivi e falsi negativi;

nessuna garanzia anti-spoofing se non implementata liveness;

non sostituisce safety hardware;

non deve salvare dati biometrici nei log generici.

Questa sezione dei limiti è necessaria per un documento serio.

Il trentesimo elemento è la **collocazione del confronto dati nella pipeline MicroBot**. Il confronto dei dati è il punto in cui la biometria diventa misurabile. Prima c'erano camera, volto, landmark, qualità. Dopo ci sarà la decisione tramite soglia. Questo punto è il ponte matematico e operativo: prende due rappresentazioni, produce score, indica compatibilità e confidenza. Ma la decisione finale richiede ancora soglia, access level e policy. Il confronto non è l'ultimo giudice; è il fornitore del dato numerico su cui il sistema decide.

In conclusione, il confronto dei dati è il processo attraverso cui il volto corrente dell'utente, trasformato in rappresentazione biometrica, viene confrontato con un riferimento autorizzato. Esso produce uno score di similarità o distanza, accompagnato da qualità, confidenza, stato del profilo, compatibilità del modello e motivo di eventuale fallimento. Questo score deve essere interpretato con cautela: non è certezza assoluta, ma misura probabilistica operativa. Il confronto deve proteggere dati biometrici, evitare log di embedding, distinguere errori tecnici da mancato riconoscimento, supportare mock mode per sviluppo, gestire profili e versioni, e preparare la decisione finale tramite soglia. Nel sistema MicroBot, il confronto biometrico autorizza solo il livello umano di accesso; ogni comando verso dashboard, controller e nodi resta comunque soggetto

a access_control, MicroBot OS e Safety Layer. Il punto successivo, dedicato alla decisione tramite soglia, completerà la Sezione 15 spiegando come lo score prodotto dal confronto venga trasformato in auth_success, auth_failed o auth_uncertain, e come tale decisione venga collegata ai livelli di accesso e ai limiti operativi del sistema.

15.6 Decisione tramite soglia

La **decisione tramite soglia** rappresenta il punto conclusivo della Sezione 15 e descrive il momento in cui lo score prodotto dal confronto biometrico viene trasformato in una decisione operativa: accesso concesso, accesso negato, autenticazione incerta, nuovo tentativo richiesto oppure accesso limitato. Dopo aver definito acquisizione tramite fotocamera, controllo degli accessi, riconoscimento facciale, landmark detection e confronto dei dati, resta da chiarire il passaggio finale della pipeline biometrica. Il sistema possiede un valore numerico, cioè uno score di similarità o distanza. Questo valore deve essere interpretato in modo rigoroso. Non basta dire che “sembra abbastanza simile”: bisogna definire una soglia, una regola di confronto, una gestione dell’incertezza e un collegamento con i livelli di accesso.

Nel progetto MicroBot, la decisione tramite soglia è fondamentale perché traduce un risultato biometrico in una conseguenza concreta sulla dashboard e sul sistema. Se la decisione è positiva, l’utente può ottenere un access_level, una sessione autorizzata e l’abilitazione di alcune funzioni. Se la decisione è negativa, il sistema resta in modalità guest, viewer limitato o access denied. Se la decisione è incerta, il sistema può richiedere un nuovo frame, una nuova autenticazione, un fallback manuale o un secondo fattore. Questo passaggio deve essere gestito con attenzione, perché una soglia troppo permissiva aumenta il rischio di accessi non autorizzati, mentre una soglia troppo severa aumenta i rifiuti verso utenti legittimi.

Il principio generale può essere espresso così:

la soglia non deve essere un numero scelto a caso, ma una regola operativa calibrata in base al tipo di score, alla qualità del dato, al livello di accesso richiesto e al rischio dell’azione da autorizzare.

Questa frase è il cuore del punto. La soglia non è soltanto un parametro tecnico. È una decisione di sicurezza. Stabilisce quanto il sistema deve essere severo prima di concedere accesso. Nel caso di funzioni read-only, si può essere più flessibili o non richiedere biometria. Nel caso di funzioni diagnostiche, serve più rigore. Nel caso di comandi hardware, clear_safety, configurazioni critiche o gestione profili biometrici, la soglia deve essere più severa e può richiedere conferme aggiuntive. Il valore numerico dello score deve quindi essere interpretato dentro una policy.

Il primo concetto da chiarire è la differenza tra **similarity score** e **distance score**. Se il sistema usa uno score di similarità, valori più alti indicano maggiore somiglianza tra volto corrente e riferimento. In questo caso la regola è: accesso possibile se score maggiore o uguale alla soglia. Se invece il sistema usa uno score di distanza, valori più bassi indicano maggiore somiglianza. In questo caso la regola è opposta: accesso possibile se distanza minore o uguale alla soglia. La documentazione deve dichiarare quale convenzione viene usata, perché invertire questa logica sarebbe un errore grave.

La regola può essere indicata così:

similarity_score: auth_success se score $\geq$ threshold;

distance_score: auth_success se distance $\leq$ threshold.

Questa distinzione deve essere esplicita nel modulo `biometric_policy`. Non deve essere lasciata implicita nel codice. Ogni `comparison_result` deve indicare `score_type`. Ogni `decision_result` deve indicare quale regola è stata applicata. Questo rende il sistema verificabile e riduce il rischio di errori semantici.

Il secondo concetto è la **soglia unica**. Nella versione più semplice, il sistema può usare una sola soglia biometrica. Se lo score supera la soglia, l'autenticazione riesce. Se non la supera, fallisce. Questa è la soluzione più immediata e può essere sufficiente per un primo prototipo locale. Tuttavia, una soglia unica ha un limite: tratta tutte le azioni nello stesso modo. Guardare una dashboard read-only e avviare un test hardware non hanno lo stesso rischio. Per questo, la soglia unica è accettabile nelle prime versioni, ma non è ideale nella visione completa.

Una policy iniziale può essere:

`threshold_default;`

score sopra soglia → `auth_success;`

score sotto soglia → `auth_failed;`

score vicino soglia → `auth_uncertain` o `retry_required`.

Questa versione è semplice e utile per testare la pipeline. Ma deve essere dichiarata come prototipale. Nelle versioni più mature, conviene usare soglie differenziate per `access_level` o categorie di azione.

Il terzo concetto è la **soglia differenziata per livello di accesso**. In MicroBot, i permessi non sono tutti uguali. Per concedere accesso viewer può bastare una soglia meno severa, o non servire biometria. Per concedere developer serve una soglia più alta. Per admin e hardware_admin serve una soglia ancora più alta, eventualmente un secondo controllo. Questo permette di collegare il rischio della funzione alla severità biometrica. È un approccio più maturo rispetto alla soglia unica.

Una struttura possibile è:

`threshold_viewer;`

`threshold_operator;`

`threshold_developer;`

`threshold_admin;`

`threshold_hardware_admin.`

Nel caso di `similarity_score`, `threshold_hardware_admin` dovrebbe essere più alto di `threshold_developer`, e `threshold_developer` più alto di `threshold_viewer`. Nel caso di `distance_score`, avviene il contrario: `threshold_hardware_admin` dovrebbe essere più basso, perché richiede distanza minore. La policy deve quindi essere coerente con `score_type`.

Questa impostazione permette decisioni più raffinate. Un utente può ottenere accesso viewer ma non developer se lo score è sufficiente per osservare, ma non abbastanza sicuro per diagnosticare. Può ottenere developer ma non hardware_admin se lo score è buono, ma non abbastanza alto per comandi fisici. Questo evita di trasformare il riconoscimento in un interruttore totale.

Il quarto concetto è la **zona di incertezza**. Una decisione tramite soglia non dovrebbe essere solo sopra/sotto. È utile definire una zona borderline intorno alla soglia. Se lo score è molto sopra soglia, il match è forte. Se è molto sotto soglia, il non-match è forte. Se è vicino alla soglia, il sistema dovrebbe essere prudente. Può richiedere un nuovo tentativo, usare più frame, concedere accesso limitato o chiedere fallback. Questa zona evita decisioni rigide su casi ambigui.

Per `similarity_score`, una logica può essere:

score molto sopra soglia → `strong_match`;

score poco sopra soglia → `weak_match`;

score vicino alla soglia → `borderline`;

score sotto soglia → `non_match`;

score molto sotto soglia → `strong_non_match`.

Per `distance_score`, la logica si inverte: distanza molto sotto soglia indica `strong_match`, distanza vicino alla soglia indica `borderline`, distanza sopra soglia indica `non_match`. Anche qui la documentazione deve specificare la convenzione.

La zona `borderline` può produrre:

`auth_uncertain`;

`retry_required`;

`access_limited`;

`fallback_auth_required`;

`manual_review_required`, solo in contesti futuri.

Per MicroBot, `auth_uncertain` è spesso la scelta migliore. Significa che il sistema non concede accesso elevato, ma non dichiara necessariamente che l'utente non è autorizzato. Chiede semplicemente un nuovo tentativo o un metodo alternativo. Questo è utile perché luce, posa e landmark possono influenzare molto lo score.

Il quinto concetto è la **qualità del dato come filtro prima della soglia**. Non bisogna applicare la soglia a dati di bassa qualità come se fossero equivalenti a dati puliti. Prima della decisione finale, il sistema deve controllare `face_detection_status`, `landmark_confidence`, `pose_status`, `image_quality` e `multi_frame_stability`. Se questi valori sono insufficienti, la decisione dovrebbe essere `retry_required` o `auth_uncertain`, anche se lo score sembra buono. Uno score su input debole è meno affidabile.

La pipeline decisionale dovrebbe essere:

verifica camera status;

verifica volto singolo;

verifica landmark validi;

verifica qualità immagine;
verifica profilo valido;
verifica score disponibile;
applica soglia;
applica policy access_level;
genera decisione finale.

Questo ordine è importante. La soglia non deve essere applicata prima di sapere se i dati sono affidabili. Se `multiple_faces_detected`, la decisione deve essere negata o incerta senza procedere a accesso. Se `landmark_confidence` è troppo bassa, si deve chiedere un nuovo frame. Se `profile_disabled`, uno score alto non deve concedere accesso. La soglia è una parte della decisione, non l'unico controllo.

Il sesto concetto è la **decisione multi-frame**. Una decisione basata su un solo frame può essere instabile. Per aumentare robustezza, il sistema può richiedere più frame validi e calcolare una decisione aggregata. Per esempio, può richiedere tre frame validi consecutivi. Può calcolare la media degli score, la mediana o la percentuale di frame sopra soglia. Può anche rifiutare se la varianza degli score è troppo alta. Questo riduce errori dovuti a un frame accidentale.

Una decisione multi-frame può usare:

`valid_frame_count`;
`score_mean`;
`score_median`;
`score_variance`;
`frames_above_threshold`;
`landmark_stability`;
`final_score`;
`final_confidence`.

Un criterio semplice potrebbe essere: almeno tre frame validi, almeno due sopra soglia e varianza bassa. Per accessi critici, si può richiedere tutti i frame validi sopra soglia. Per accessi meno rischiosi, può bastare media sopra soglia. La scelta deve essere proporzionata al rischio.

Il settimo concetto è la **decisione legata all'azione richiesta**. Non sempre l'utente deve essere autenticato in astratto. Spesso chiede un'azione specifica: aprire diagnostica, avviare demo, modificare config, inviare comando hardware. La soglia dovrebbe essere valutata anche rispetto all'azione. Un `auth_success` generico può creare una sessione developer, ma una `hardware_action` può richiedere re-auth recente e soglia più alta. Questo rende il sistema più sicuro.

La decisione può quindi essere divisa in due livelli:

auth_decision: identifica e autentica l'utente;

authorization_decision: decide se quella specifica azione è permessa.

Esempio:

auth_success con access_level = developer;

azione richiesta = view_logs;

decisione = allowed;

azione richiesta = hardware_pwm_test;

decisione = denied, reason = requires_hardware_admin.

Oppure:

auth_success con access_level = hardware_admin;

azione richiesta = hardware_pwm_test;

decisione access_control = allowed;

decisione Safety Layer = denied, reason = thermal_lock.

Questa distinzione deve restare sempre chiara.

L'ottavo concetto è la **decisione finale come oggetto strutturato**. Il sistema non deve restituire solo true o false. Deve produrre un decision_result con campi chiari: decision, reason, access_level_granted, score_class, threshold_used, confidence_class, session_created, allowed_actions, restrictions. Questo oggetto permette a dashboard, controller e log di capire cosa è successo.

Una struttura decision_result può contenere:

auth_attempt_id;

decision: auth_success, auth_failed, auth_uncertain, retry_required;

reason;

profile_id;

access_level_granted;

score_type;

score_value opzionale;

score_class;

threshold_used;

quality_status;

confidence_class;

session_id, se creata;

allowed_actions_summary;

restrictions;

expires_at.

Questa struttura è molto più utile di un booleano. Se l'accesso fallisce, reason spiega perché. Se riesce con confidenza bassa, restrictions può limitare azioni. Se la sessione viene creata, expires_at indica durata. La dashboard può usare questi dati per UI e controlli.

Il nono concetto è la **classe di confidenza**. Invece di mostrare sempre il valore numerico dello score, il sistema può produrre una classe: high_confidence, medium_confidence, low_confidence, uncertain. Questa classe è utile per logging e UI. In modalità developer, si può mostrare anche lo score numerico. In modalità normale, basta la classe. Questo protegge dati e rende l'interfaccia più comprensibile.

Classi possibili:

high_confidence_match;

medium_confidence_match;

low_confidence_match;

borderline;

low_confidence_non_match;

strong_non_match;

invalid_input.

Questa classificazione può essere usata nei security_events. Per esempio: auth_failed, reason = score_below_threshold, confidence_class = strong_non_match. Oppure auth_uncertain, reason = borderline_score. Questo rende i log più leggibili.

Il decimo concetto è la **soglia come parametro documentato**. La soglia deve essere registrata nella biometric_policy, non nascosta in codice sparso. Deve avere nome, valore, score_type, versione, data di calibrazione, scopo e note. Questo permette di sapere quale soglia è stata usata in una decisione. Se si cambia soglia, bisogna aggiornare la versione della policy. Questo è importante per debugging e tracciabilità.

Una biometric_policy può contenere:

policy_id;

score_type;

threshold_default;

threshold_by_access_level;

borderline_margin;
min_landmark_confidence;
min_valid_frames;
max_pose_deviation;
session_duration_by_access_level;
reauth_required_actions;
policy_version.

Questa struttura rende il sistema molto più professionale. Se un auth_attempt viene registrato, può indicare policy_id. In futuro, se si cambia soglia, si può confrontare il comportamento prima e dopo.

L'undicesimo concetto è la **calibrazione della soglia**. La soglia deve essere scelta tramite prove. In un prototipo MicroBot, non serve una validazione scientifica su dataset enorme, ma serve almeno una calibrazione interna. Bisogna provare l'utente autorizzato in condizioni diverse, osservare range degli score, provare condizioni non valide, osservare score bassi, definire margine, documentare i limiti. Questo impedisce di scegliere numeri arbitrari.

Una calibrazione minima può includere:

tentativi con volto autorizzato in luce buona;
tentativi con volto autorizzato in luce media;
tentativi con posa leggermente diversa;
tentativi con frame non validi;
tentativi con persona non autorizzata, se disponibili e con consenso;
range score positivi;
range score negativi;
soglia scelta;
margine borderline;
limitazioni.

Questa calibrazione può essere descritta senza salvare immagini o template. Si possono salvare solo statistiche aggregate. Per un progetto tecnico, questo è già un buon livello di rigore.

Il dodicesimo concetto è la **modifica controllata della soglia**. Cambiare soglia può alterare profondamente la sicurezza. Una soglia più bassa può rendere l'accesso più facile ma meno sicuro. Una più alta può rendere il sistema più sicuro ma più difficile da usare. Per questo, la soglia non deve essere modificabile da qualsiasi utente. Dovrebbe richiedere admin o development_mode. Ogni modifica deve essere loggata.

Una modifica threshold deve registrare:

old_value;

new_value;

changed_by;

timestamp;

reason;

policy_version_old;

policy_version_new;

mode: development o operational.

In una versione prototipale, changed_by può essere un profilo locale o “developer_manual”. L’importante è non cambiare soglie in modo invisibile. La tracciabilità protegge il progetto.

Il tredicesimo concetto è la **soglia e i falsi positivi**. Un falso positivo è il caso in cui il sistema concede accesso a una persona non autorizzata. Per MicroBot, questo rischio è particolarmente grave se l’accesso permette comandi hardware, diagnostica critica o modifica configurazioni. Per ridurre falsi positivi, la soglia deve essere severa per permessi elevati, la qualità deve essere alta, devono essere bloccati casi ambigui e può essere richiesta re-auth o conferma aggiuntiva.

Misure contro falsi positivi:

soglia alta per accessi critici;

blocco su multiple_faces_detected;

blocco su landmark_confidence bassa;

blocco su profilo disabilitato;

richiesta multi-frame;

sessioni brevi per hardware_admin;

Safety Layer indipendente;

re-auth per clear_safety e hardware commands.

Questo elenco mostra che la soglia è solo una difesa. Non basta da sola. Il sistema deve avere più barriere.

Il quattordicesimo concetto è la **soglia e i falsi negativi**. Un falso negativo è il caso in cui il sistema rifiuta un utente autorizzato. È meno pericoloso di un falso positivo, ma peggiora usabilità. Per ridurlo, il sistema può fornire feedback sulla qualità, richiedere nuovo frame, usare multi-frame, migliorare enroll, calibrare meglio la soglia e permettere fallback controllato. Non bisogna semplicemente abbassare troppo la soglia, perché questo aumenta falsi positivi.

Misure contro falsi negativi:

feedback su luce e posa;

retry automatico;

multi-frame decision;

enroll con frame di qualità;

soglia calibrata;

fallback manuale in development_mode;

accesso read-only se biometria fallisce.

Queste misure migliorano usabilità senza sacrificare troppo sicurezza.

Il quindicesimo concetto è la **decisione condizionata dalla modalità del sistema**. La stessa soglia può essere interpretata diversamente se il sistema è in demo_mode, development_mode, hardware_actuation_mode o lockdown. In demo_mode, si può usare mock o accesso limitato. In development_mode, si possono permettere fallback. In hardware_actuation_mode, si richiede severità. In lockdown, anche un auth_success può non concedere comandi. Questo è importante perché l'autenticazione non vive nel vuoto.

Esempi:

demo_mode: auth_success abilita solo demo controllata;

development_mode: mock_auth può essere permesso ma visibile;

hardware_actuation_mode: threshold alta e re-auth;

lockdown_mode: accesso read-only anche per admin, salvo safe actions;

recovery_mode: solo diagnostica e safe commands.

Questo collega decisione biometrica allo stato complessivo MicroBot. Il sistema deve considerare contesto e rischio.

Il sedicesimo concetto è la **decisione condizionata dalla safety**. Anche dopo auth_success, la Safety Layer può bloccare azioni. Questo deve essere chiarissimo. Un utente riconosciuto non può attivare comandi se il nodo è in thermal_lock, critical_energy, emergency_stop, controller_lost, recovery_required o capability_blocked. La decisione biometrica abilita la richiesta; la safety decide se l'esecuzione è permessa.

La catena è:

decisione biometrica = utente autorizzato;

controllo accessi = azione consentita al livello umano;

controller = comando coerente con stato globale;

MicroBot OS = comando valido per nodo;

Safety Layer = azione sicura ora.

Se uno dei livelli rifiuta, l'azione non avviene. Questo principio chiude la relazione tra sicurezza biometrica e sicurezza fisica.

Il diciassettesimo concetto è la **decisione in caso di dati mancanti**. Se manca lo score, manca il profilo, manca la camera, mancano i landmark, manca la policy o il modello non è pronto, il sistema deve fallire in modo sicuro. Non deve concedere accesso elevato. Può concedere guest/read-only. Può chiedere fallback. Può dichiarare auth_unavailable. Questa regola è fondamentale.

Casi fail-safe:

camera_unavailable → no biometric auth;

landmarks_failed → retry_required;

reference_missing → auth_unavailable o no_profile;

threshold_policy_missing → access_denied per funzioni sensibili;

score_unavailable → auth_failed o auth_uncertain;

profile_disabled → access_denied;

security_module_error → read-only o lockdown.

In caso di dubbio, non si concede controllo. Questa è una regola di sicurezza basilare.

Il diciottesimo concetto è la **decisione e la sessione**. Se auth_success viene concesso, il sistema può creare una sessione. La sessione contiene access_level, durata, metodo di autenticazione e restrizioni. La soglia usata può influenzare la durata e i permessi. Un high_confidence_match può creare sessione piena per quel profilo. Un medium_confidence_match può creare sessione limitata. Un borderline non dovrebbe creare sessione elevata.

Una sessione può essere creata così:

auth_success_high_confidence → sessione access_level pieno;

auth_success_medium_confidence → sessione con restrizioni;

auth_success_low_confidence → accesso limitato o re-auth;

auth_uncertain → nessuna sessione elevata;

auth_failed → guest/read-only.

Questo modello è più raffinato e consente controllo dinamico. Le prime versioni possono essere più semplici, ma la documentazione può prevederlo.

Il diciannovesimo concetto è la **decisione e re-auth**. Alcune azioni devono richiedere autenticazione recente. Anche se la sessione è ancora valida, per azioni critiche si può chiedere una nuova verifica. Per esempio: clear_safety, hardware_test, modifica soglie biometriche, modifica safety policy, gestione profili. La soglia per re-auth può essere alta. Questo riduce rischio di sessioni lasciate aperte.

Azioni che possono richiedere re-auth:

clear_safety;

hardware_actuation_test;

modify_safety_limits;

manage_biometric_profiles;

switch_to_hardware_admin;

disable_security_module;

enable_mock_auth;

export diagnostic sensitive data.

Il re-auth deve generare security_event. Se fallisce, l'azione resta bloccata. Anche qui, Safety Layer resta indipendente.

Il ventesimo concetto è la **decisione e logging**. Ogni decisione deve generare un security_event minimizzato. Il log deve dire cosa è successo: auth_success, auth_failed, auth_uncertain, access_granted, access_denied. Deve indicare reason, access_level, confidence_class, policy_id e timestamp. Non deve salvare immagini, embedding o landmark grezzi. In modalità developer, può mostrare score numerico per calibrazione, ma non nei log generali.

Un evento decisione può contenere:

event_type;

auth_attempt_id;

timestamp;

decision;

reason;

access_level_granted;

confidence_class;

policy_id;

session_id, se creata;

source_interface;

security_mode.

Esempi di reason:

score_above_threshold;

score_below_threshold;

score_borderline;

low_quality_input;

multiple_faces_detected;

profile_disabled;

session_created;

reauth_required;

safety_block_after_auth.

Questi eventi rendono il sistema tracciabile senza violare minimizzazione dei dati.

Il ventunesimo concetto è la **decisione e dashboard**. La dashboard deve rappresentare il risultato in modo chiaro. Non deve mostrare dettagli biometrici eccessivi a utenti non developer. Può mostrare: autenticazione riuscita, access level, sessione attiva, controlli abilitati, sessione in scadenza. Se fallisce, deve mostrare motivo semplice: volto non rilevato, qualità insufficiente, accesso negato, riprovare, sessione scaduta. In developer mode può mostrare più dettagli: score class, threshold policy, landmark status.

La UI può distinguere:

utente normale: messaggio semplice e badge accesso;

developer: dettagli diagnostici;

admin: policy e gestione profili;

security log: eventi minimizzati.

Questo mantiene usabilità e sicurezza. Non tutti devono vedere tutto.

Il ventiduesimo concetto è la **decisione e fallback**. Se la biometria fallisce o è indisponibile, può esistere un fallback controllato. Per esempio, password locale in development_mode, manual unlock, token, accesso fisico, modalità read-only. Il fallback deve essere proporzionato e loggato. Non deve concedere hardware_admin senza controllo. In una demo, può essere utile avere manual_developer_unlock chiaramente marcato come modalità di sviluppo.

Possibili fallback:

guest/read-only;

retry camera auth;

manual developer unlock in development_mode;

admin password locale, se prevista;

mock mode solo con badge visibile;

hardware actions bloccate finché non c'è auth forte.

Il fallback deve evitare sia blocco totale del lavoro sia apertura insicura. La policy deve essere esplicita.

Il ventitreesimo concetto è la **decisione e profili disabilitati**. Se il volto corrisponde a un profilo disabilitato, la decisione deve essere `access_denied`. Non deve concedere accesso solo perché lo score è alto. Questo è importante per gestione utenti. Un profilo può essere disabilitato per sicurezza, test, revoca o cambio permessi. La soglia biometrica non deve superare lo stato amministrativo.

La regola è:

`score valido + profile_disabled = access_denied, reason = profile_disabled`.

Questo dimostra che decisione biometrica e policy di accesso sono separate ma coordinate.

Il ventiquattresimo concetto è la **decisione e modifica dei livelli di accesso**. Se un profilo cambia da developer a viewer, il sistema deve applicare subito il nuovo livello. Le sessioni attive possono essere revocate o ridotte. La decisione tramite soglia deve usare `access_level` corrente del profilo, non quello vecchio. Questo richiede `profile_version` e `session validation`.

Una policy può essere:

`profile_updated` → revoke active sessions;

`access_level_changed` → require reauth;

`profile_disabled` → invalidate sessions;

`threshold_policy_changed` → require reauth for critical actions.

Questo evita incoerenza tra vecchi permessi e nuova policy.

Il venticinquesimo concetto è la **decisione e modalità mock**. In mock mode, la decisione non deriva da biometria reale. Deve essere indicata. Un mock `auth_success` può creare sessione developer per test UI, ma il sistema deve mostrare **MOCK MODE** e registrare `security_event`. In modalità non di sviluppo, mock mode dovrebbe essere disabilitato. Hardware actions dovrebbero rimanere bloccate o richiedere conferma reale, secondo policy.

Regole mock:

mock mode visibile;

`security_event` obbligatorio;

non presentare mock come biometria reale;

disabilitare mock in modalità operational;

`hardware_admin` non concesso da mock, salvo ambiente totalmente simulato;

report indicano `auth_mode = mock`.

Questo mantiene onestà tecnica. È fondamentale per presentare il progetto bene.

Il ventiseiesimo concetto è la **decisione e limiti del prototipo**. Nelle prime versioni, la decisione tramite soglia sarà sperimentale. Non bisogna dichiararla equivalente a sistemi biometrici commerciali. Può essere utile per demo, access control locale e interazione intelligente, ma deve essere accompagnata da limiti. Non deve proteggere sistemi critici reali senza ulteriori controlli. Non deve essere usata da sola per attuazioni fisiche rischiose.

Limitazioni da dichiarare:

soglie sperimentali;

dataset di calibrazione limitato;

possibili falsi positivi e negativi;

assenza di liveness avanzata, se non implementata;

uso locale;

non adatto a sistemi critici senza ulteriori livelli;

Safety Layer indipendente e sempre prioritario.

Questa dichiarazione non indebolisce il progetto. Lo rende più credibile.

Il ventisettesimo concetto è la **decisione come parte della roadmap**. La decisione tramite soglia può evolvere. Prima si usa mock threshold. Poi threshold unica. Poi zona borderline. Poi soglie per access_level. Poi multi-frame. Poi qualità integrata. Poi re-auth per azioni critiche. Poi calibrazione documentata. Poi gestione profili multipli. Questa progressione è coerente con lo sviluppo MicroBot.

Una roadmap possibile:

v0.1: decisione mock;

v0.2: threshold unica;

v0.3: score_type dichiarato;

v0.4: auth_success/auth_failed;

v0.5: auth_uncertain e retry_required;

v0.6: quality-aware decision;

v0.7: soglie per access_level;

v0.8: multi-frame decision;

v0.9: sessione con scadenza;

v1.0: re-auth per azioni critiche;

v1.1: logging minimizzato;

v1.2: calibrazione documentata;

v1.3: profili multipli e margine best-match;

v1.4: lockdown integration.

Questa roadmap chiude bene la sezione biometrica, perché mostra come passare da demo a modulo più maturo.

Il ventottesimo concetto è la **matrice decisionale**. Per documentare bene il sistema, si può creare una tabella che collega condizione, decisione e risultato. Per esempio: score alto e qualità alta → accesso concesso. Score alto ma qualità bassa → retry. Score borderline → auth_uncertain. Profilo disabilitato → access denied. Multiple faces → denied/retry. Safety lock → azione bloccata anche dopo auth. Questa matrice rende la logica molto chiara.

Una matrice concettuale può essere:

score sopra soglia, qualità alta, profilo attivo → auth_success;

score sopra soglia, qualità bassa → retry_required o auth_uncertain;

score borderline → auth_uncertain;

score sotto soglia → auth_failed;

profilo disabilitato → access_denied;

camera unavailable → auth_unavailable;

multiple faces → access_denied o retry_required;

auth_success ma access_level insufficiente → permission_denied;

auth_success ma Safety Layer blocca → action_denied_safety.

Questa matrice è molto utile nella documentazione finale.

Il ventinovesimo concetto è la **decisione come evento controllato, non come comando diretto**. La decisione tramite soglia non deve mai attivare direttamente MicroBot fisici. Deve solo modificare auth_state e access_session. L'utente poi richiede un'azione. L'azione passa da access_control, controller e nodo. Questo mantiene separazione tra sicurezza biometrica e controllo operativo. È uno dei punti più importanti della Sezione 15.

La regola finale è:

auth_success abilita la possibilità di chiedere azioni; non esegue azioni automaticamente.

Questo impedisce comportamenti pericolosi come “riconosco volto e avvio hardware”. In un sistema serio, autenticazione e comando sono sempre separati.

Il trentesimo concetto è la **chiusura della Sezione 15**. La sicurezza biometrica MicroBot è un livello di accesso intelligente costruito su fotocamera, riconoscimento facciale, landmark detection, confronto dati e decisione tramite soglia. Ma il suo valore non sta solo nel riconoscere un volto. Sta

nel modo in cui il riconoscimento viene collegato a permessi, sessioni, dashboard, security events, policy, limiti e Safety Layer. Un modulo biometrico ben progettato non rende il sistema magicamente sicuro, ma aggiunge un livello di controllo umano tracciabile e graduato.

In conclusione, la decisione tramite soglia è il passaggio in cui lo score biometrico viene interpretato secondo una policy. Essa stabilisce se l'autenticazione riesce, fallisce, resta incerta o richiede un nuovo tentativo. La soglia deve essere coerente con score_type, qualità del dato, landmark confidence, profilo, access_level, modalità del sistema e rischio dell'azione richiesta. Una decisione robusta deve prevedere zona borderline, multi-frame, soglie differenziate, re-auth per azioni critiche, logging minimizzato e fallimento sicuro in caso di dati mancanti. Nel sistema MicroBot, auth_success non bypassa mai Safety Layer, MicroBot OS o controller: abilita soltanto una sessione e un livello di accesso umano. Con questo punto si chiude la Sezione 15 dedicata alla Sicurezza Biometrica. La sezione successiva, dedicata ai Progetti Esterni, potrà ora analizzare come intelligenza artificiale, clustering, modelli, giochi, sistemi interattivi, simulazioni evolutive, algoritmi genetici, progetti web, interfacce avanzate, tool diagnostici e script di sistema possano essere collegati all'ecosistema MicroBot come competenze, moduli, ispirazioni tecniche e strumenti di supporto.

SEZIONE 16 — Progetti Esterni

La Sezione 16 introduce il ruolo dei **progetti esterni** all'interno dell'ecosistema MicroBot. Dopo aver definito le componenti principali del sistema — sicurezza, simulazione, telemetria, MicroBot OS e sicurezza biometrica — diventa necessario analizzare tutto ciò che, pur non appartenendo direttamente al nucleo fisico dello swarm, contribuisce alla crescita tecnica, metodologica e progettuale del sistema. I progetti esterni comprendono esperimenti di intelligenza artificiale, clustering, modelli, giochi, sistemi interattivi, simulazioni evolutive, algoritmi genetici, progetti web, interfacce avanzate, tool diagnostici e script di sistema. Essi rappresentano un insieme di competenze parallele che possono alimentare MicroBot, migliorarlo o ispirarne moduli futuri.

Nel contesto MicroBot, i progetti esterni non devono essere interpretati come elementi casuali o scollegati. Non sono semplicemente “altre cose fatte a parte”. Devono essere letti come una rete di esperimenti tecnici che, in modi diversi, contribuiscono alla formazione dell'ecosistema principale. Un progetto di clustering può fornire idee per analizzare gruppi di nodi. Una simulazione evolutiva può ispirare l'ottimizzazione dei comportamenti dello swarm. Un algoritmo genetico può diventare un metodo per cercare parametri migliori. Un gioco può insegnare gestione di loop, collisioni, stati, input e rendering. Un sito web avanzato può diventare base per dashboard e interfacce MicroBot. Uno script diagnostico può diventare parte degli strumenti di manutenzione. Un modello di intelligenza artificiale può supportare classificazione, previsione o controllo assistito.

Il principio generale della Sezione 16 può essere espresso così:

i progetti esterni non sostituiscono MicroBot, ma ne arricchiscono l'ecosistema fornendo competenze, strumenti, modelli, interfacce e metodi sperimentali riutilizzabili.

Questa frase è importante perché impedisce dispersione. MicroBot deve restare il progetto centrale. I progetti esterni devono essere integrati solo quando hanno una funzione chiara. Non tutto deve entrare nel core MicroBot. Alcune cose restano portfolio, studio, sperimentazione o dimostrazione di competenza. Altre possono diventare moduli reali. La distinzione è fondamentale per mantenere ordine. Un progetto esterno può essere utile come ispirazione, come libreria, come componente, come demo, come strumento o come prova di abilità, ma non deve appesantire il sistema principale senza motivo.

La Sezione 16 segue i punti dell'indice: intelligenza artificiale, clustering e modelli, giochi, sistemi interattivi, simulazioni evolutive, algoritmi genetici, progetti web, interfacce avanzate, tool diagnostici e script di sistema. Ciascuno di questi ambiti contribuisce in modo diverso. Alcuni sono più vicini alla parte scientifica e computazionale. Altri sono più vicini alla parte visuale e comunicativa. Altri ancora sono legati a debugging, automazione e manutenzione. Insieme, mostrano che MicroBot non nasce da una sola competenza, ma da una convergenza di informatica, simulazione, interazione, sistemi embedded, web design, AI e analisi dei dati.

Un punto importante è che questa sezione deve distinguere tra **integrazione diretta** e **integrazione indiretta**. L'integrazione diretta avviene quando un progetto esterno diventa parte concreta del sistema MicroBot. Per esempio, un tool diagnostico può essere inserito nella cartella scripts/ o tools/. Una dashboard web può essere riadattata come interfaccia MicroBot. Un algoritmo di clustering può essere usato per analizzare la disposizione dei nodi. L'integrazione indiretta, invece, avviene quando un progetto non entra nel codice MicroBot ma contribuisce come esperienza tecnica. Per esempio, un gioco sviluppato in Python può aver insegnato gestione degli stati, loop e collisioni; una simulazione evolutiva può aver fornito intuizioni sul comportamento emergente; un sito web può aver definito uno stile visivo.

Questa distinzione può essere sintetizzata così:

integrazione diretta = codice, modulo o strumento riutilizzato nel sistema;

integrazione indiretta = competenza o modello concettuale trasferito nel progetto;

integrazione futura = possibilità documentata ma non ancora implementata.

Questa classificazione è utile perché evita allucinazioni progettuali. Se un algoritmo genetico non è ancora collegato davvero al controller MicroBot, non bisogna scrivere che lo è già. Si può dire che rappresenta una possibile estensione futura per ottimizzare parametri. Se un progetto web non è ancora integrato nella dashboard principale, si può dire che offre base estetica o tecnica. La documentazione deve essere precisa: cosa esiste, cosa è sperimentale, cosa è potenziale.

La Sezione 16 ha anche un valore strategico. MicroBot, preso da solo, è già un progetto complesso. Ma i progetti esterni dimostrano ampiezza di competenze: AI, data analysis, web, simulazione, giochi, sistemi interattivi, tool, script, modelli e interfacce. Questo è importante per portfolio, GitHub, presentazioni e collaborazioni. Tuttavia, la presentazione deve essere ordinata. Non bisogna mostrare tutto come un mucchio di esperimenti. Bisogna spiegare come ogni progetto o famiglia di progetti contribuisce alla visione generale. La Sezione 16 serve proprio a dare questa coerenza.

Un possibile criterio di lettura è:

i progetti di AI e clustering contribuiscono alla parte analitica;

i giochi e sistemi interattivi contribuiscono alla gestione di loop, stati e input;

le simulazioni evolutive e gli algoritmi genetici contribuiscono all'idea di adattamento;

i progetti web contribuiscono a dashboard, UI e comunicazione;

le interfacce avanzate contribuiscono all'interazione uomo-sistema;

i tool diagnostici e gli script contribuiscono alla manutenzione e automazione.

Questa mappa mostra che i progetti esterni non sono rumore. Sono sottosistemi di competenza. MicroBot può assorbirli in modo selettivo.

16.1 Intelligenza artificiale

L'intelligenza artificiale rappresenta il primo ambito dei progetti esterni collegabili all'ecosistema MicroBot. In questo contesto, per intelligenza artificiale non si intende necessariamente un unico modello centrale che controlla tutto lo swarm, ma un insieme di metodi, esperimenti e strumenti capaci di analizzare dati, riconoscere pattern, classificare comportamenti, supportare decisioni, ottimizzare parametri e migliorare l'interazione tra utente e sistema. MicroBot può funzionare anche senza AI nelle sue prime versioni, ma l'AI può diventare progressivamente un livello di supporto molto potente.

È importante non forzare subito l'AI dentro il core del progetto. In una fase iniziale, MicroBot deve prima avere simulazione stabile, telemetria, safety, dashboard, pattern di base e nodi hardware riconoscibili. Solo dopo ha senso introdurre modelli intelligenti. Questo perché un modello AI senza dati affidabili rischia di produrre risultati poco utili. L'AI ha bisogno di telemetria pulita, dataset ordinati, metriche, run documentate e failure modes classificati. Per questo motivo, l'intelligenza artificiale deve essere vista come livello successivo, costruito sopra la raccolta dati e l'analisi del sistema.

Il principio generale può essere espresso così:

l'intelligenza artificiale può supportare MicroBot solo quando il sistema produce dati abbastanza strutturati da essere analizzati, confrontati e trasformati in decisioni utili.

Questo principio collega direttamente la Sezione 16 alla Sezione 13 sulla telemetria. Senza dati, l'AI diventa decorazione. Con dati ben raccolti, può diventare strumento operativo. Se ogni run MicroBot salva snapshot, eventi, metriche, errori, pattern, ruoli, safety events e risultati finali, allora questi dati possono diventare dataset. Da questi dataset, un modello può imparare a riconoscere run instabili, prevedere failure, suggerire parametri, classificare errori o identificare nodi problematici.

Una prima area di applicazione dell'AI è la **classificazione dei comportamenti dello swarm**. Durante una simulazione, lo swarm può assumere stati diversi: convergente, instabile, frammentato, bloccato, disperso, sovraccarico, in recovery, in fallback. Un modello di classificazione potrebbe ricevere metriche come `pattern_error`, `topology_score`, `aggregation_score`, `node_count`, `safety_event_count`, `correction_count` e `stability_score`, e classificare la run. Inizialmente questa classificazione può essere fatta con regole manuali. In futuro, un modello AI può imparare da esempi etichettati.

Una classificazione possibile può includere:

`stable_pattern;`

`unstable_pattern;`

`partial_convergence;`

`topology_failure;`

`geometry_failure;`

safety_degraded;
energy_limited;
thermal_limited;
communication_problem;
recovery_success;
recovery_failed.

Questa classificazione può aiutare dashboard e report. Invece di mostrare solo numeri, il sistema può dire: “la run è geometricamente buona ma topologicamente incompleta”, oppure “la run è fallita per instabilità dopo convergenza”, oppure “il comportamento è limitato da warning termici”. Questo rende l’analisi più leggibile.

Una seconda area è la **previsione dei fallimenti**. Se il sistema raccoglie dati nel tempo, può cercare segnali precoci che precedono un fallimento. Per esempio, oscillazioni crescenti del pattern_error, aumento dei retry, calo del topology_score, crescita del thermal_load, aumento dei nodi stale, role_mismatch ripetuti. Un modello predittivo potrebbe segnalare che una run ha alta probabilità di fallire prima che fallisca davvero. Questo permetterebbe al controller di intervenire prima, applicare fallback, ridurre carico o fermare la run.

Un modello di previsione potrebbe usare:

trend del pattern_error;
varianza dello stability_score;
numero di missing_edges;
numero di command_rejected;
node_stale_count;
thermal_warning_count;
energy_low_count;
phase_duration anomala;
retry_count;
sync_degraded_count.

Il risultato potrebbe essere:

failure_risk_low;
failure_risk_medium;
failure_risk_high;
recommended_action.

Questa funzione sarebbe molto utile, ma deve essere introdotta solo quando esistono abbastanza run registrate. Prima bisogna creare dataset. Poi si può addestrare o costruire un modello.

Una terza area è la **ottimizzazione dei parametri**. MicroBot può avere molti parametri: cohesion_weight, separation_weight, alignment_weight, damping, target_weight, connection_radius, cooldown, retry_count, timeout, stabilization_time, velocity_limit. Trovare valori buoni a mano può essere difficile. L'AI, o più in generale metodi di ottimizzazione, può aiutare a cercare configurazioni migliori. Inizialmente si possono usare grid search o random search. In futuro, algoritmi evolutivi o modelli predittivi possono suggerire parametri.

Parametri ottimizzabili possono essere:

forze virtuali nella simulazione;

soglie di validazione;

timeout di fase;

policy di retry;

distanze di connessione;

cooldown;

pesi di coesione e separazione;

durata della stabilizzazione;

frequenza di telemetria;

criteri di fallback.

L'obiettivo non deve essere “fare AI perché fa figo”, ma migliorare metriche concrete: maggiore success_rate, minore completion_time, minore oscillazione, minori safety_event, minore energy_cost, migliore topology_score. Ogni ottimizzazione deve essere valutata tramite report e metriche.

Una quarta area è la **analisi dei nodi problematici**. In uno swarm, alcuni nodi possono essere più instabili di altri. Possono andare spesso offline, entrare in low_energy, generare thermal_warning, rifiutare comandi, avere ruolo mismatch o causare edge mancanti. Un modello AI o un sistema di scoring può identificare nodi problematici e suggerire di escluderli da ruoli critici. Questo può essere fatto inizialmente con regole semplici, poi con modelli più raffinati.

Un node_reliability_score può usare:

uptime_percent;

reset_count;

command_rejection_rate;

thermal_event_count;

energy_event_count;

role_mismatch_count;
offline_count;
average_response_time;
recovery_success_rate;
critical_role_failure_count.

Il risultato può essere:

node_reliable;
node_degraded;
node_unstable;
node_not_suitable_for_critical_role.

Questa funzione sarebbe molto utile in hardware reale. Un ESP32 instabile non dovrebbe diventare leader o connector critico. L'AI può aiutare a decidere ruoli in modo più intelligente.

Una quinta area è la **analisi dei pattern**. Ogni pattern può avere comportamento diverso. line_basic può essere stabile. circle_basic può avere problemi di chiusura. bridge_complex può essere fragile. cluster_basic può essere denso ma rischiare collisioni. Un sistema di analisi intelligente può confrontare pattern e capire quali sono più robusti, quali richiedono più energia, quali generano più warning e quali sono migliori per demo. Questa analisi può diventare parte della dashboard.

Metriche per pattern possono includere:

success_rate;
average_completion_time;
average_pattern_error;
topology_score_final;
safety_event_rate;
energy_cost_average;
thermal_event_rate;
fallback_rate;
recovery_rate;
visual_clarity_score, se definito.

Un modulo AI potrebbe suggerire: “per una demo stabile usare line_basic o cluster_basic; evitare bridge_complex in modalità hardware; usare circle_basic solo con almeno N nodi e buona sincronizzazione”. Questo rende il sistema più intelligente e presentabile.

Una sesta area è la **interazione uomo-sistema**. La sicurezza biometrica della Sezione 15 è già una forma di integrazione con computer vision. Ma l'AI può andare oltre: riconoscere gesti, interpretare comandi vocali, assistere l'utente nella dashboard, spiegare errori, suggerire next steps. Per esempio, un assistente locale potrebbe leggere il report di una run e generare una spiegazione: "il pattern è fallito perché due nodi connector sono entrati in low_energy durante phase_attach". Oppure potrebbe guidare l'utente nel debug.

Applicazioni possibili:

spiegazione automatica dei failure;

suggerimenti di configurazione;

assistente per dashboard;

riconoscimento gesti;

classificazione livello utente;

analisi dei log;

generazione report;

supporto alla documentazione.

Queste funzioni sono molto utili ma devono restare separate dal controllo diretto dei nodi. Un assistente può suggerire, non bypassare safety. Ogni comando deve comunque passare da access_control, controller e MicroBot OS.

Una settima area è la **integrazione con pipeline BCI o segnali biologici**, se considerata come direzione futura. Nei tuoi esperimenti collegati a controllo tramite segnali cerebrali, l'AI può classificare stati o intenzioni, per esempio REST, LEFT_IMAGERY, RIGHT_IMAGERY, STOP. Questo è un ambito molto avanzato e va tenuto come future direction. Può essere collegato a MicroBot come modalità di controllo sperimentale, ma non deve essere presentato come già maturo per controllo hardware critico. Deve rimanere sotto safety e soglie di confidenza.

Una pipeline BCI concettuale può essere:

acquisizione segnale;

preprocessing;

estrazione feature;

classificazione;

confidence threshold;

smoothing temporale;

mapping comando;

safety filter;

dispatch controllato.

Questo esempio mostra che AI e MicroBot possono collegarsi anche oltre la visione facciale. Tuttavia, ogni input AI deve essere filtrato. Se la confidenza è bassa, il comando non passa. Se il comando è STOP, può avere priorità safety. Se il comando è movimento o attuazione, deve essere limitato.

Un'ottava area è la **analisi dei dati di telemetria come dataset**. Ogni run MicroBot può produrre dati. Questi dati possono essere trasformati in dataset per AI. Per esempio:

```
run_id;  
  
pattern_id;  
  
configuration_id;  
  
node_count;  
  
metrics_time_series;  
  
events;  
  
final_result;  
  
failure_reason;  
  
safety_summary;  
  
energy_summary;  
  
thermal_summary;  
  
recommendations.
```

Questo dataset può essere usato per addestrare modelli o per analisi statistica. Prima ancora dell'AI, può essere usato per grafici, confronti e tabelle. L'AI arriva dopo la disciplina del dataset. Per questo, la Sezione 13 e la Sezione 16 sono collegate profondamente.

Una nona area è la **generazione automatica di report tecnici**. L'AI può aiutare a trasformare metriche e log in testo leggibile. Per esempio, dopo una run, il sistema può generare un report: scopo, configurazione, risultato, problemi, eventi principali, causa probabile, raccomandazioni. Questo non deve sostituire la validazione umana, ma può accelerare documentazione. Per un progetto grande come MicroBot, report automatici sarebbero molto utili.

Un report generato può includere:

```
summary della run;  
  
pattern usato;  
  
risultato;  
  
metriche principali;  
  
eventi critici;  
  
failure reason;
```

nodi problematici;

raccomandazioni;

limiti;

next experiment.

Questa funzione può unire telemetria, analisi e documentazione. È uno dei modi più concreti per usare AI in MicroBot senza rischiare controllo diretto hardware.

Una decima area è la **separazione tra AI consultiva e AI operativa**. Questa distinzione è fondamentale. AI consultiva analizza, spiega e suggerisce. AI operativa decide o controlla direttamente. Nelle prime versioni, MicroBot dovrebbe usare soprattutto AI consultiva. Per esempio, suggerire parametri, classificare run, spiegare errori. L'AI operativa, che modifica in tempo reale comandi o ruoli, deve arrivare solo dopo molti test e sempre sotto vincoli di safety.

La distinzione è:

AI consultiva = suggerisce, classifica, spiega, analizza;

AI semi-operativa = propone modifiche che l'utente o controller approva;

AI operativa = modifica comportamento in tempo reale sotto limiti rigorosi;

AI non autorizzata = mai bypassa safety.

Questa distinzione protegge il progetto. Non conviene dare subito controllo diretto a un modello. Prima deve essere uno strumento di supporto.

Una undicesima area è la **AI per anomaly detection**. Il sistema può imparare cosa è normale e segnalare anomalie. Per esempio, heartbeat irregolare, pattern_error anomalo, temperatura che sale troppo, comandi rifiutati in modo insolito, latenze strane, reset frequenti. Anche senza etichette precise, si possono usare metodi di anomaly detection. Inizialmente, anche semplici soglie e regole possono funzionare. In futuro, modelli più avanzati possono migliorare.

Anomalie possibili:

node_behavior_anomaly;

telemetry_gap_anomaly;

thermal_rise_anomaly;

energy_drop_anomaly;

communication_latency_anomaly;

role_flapping_anomaly;

pattern_instability_anomaly;

unexpected_reset_anomaly.

Il risultato può essere mostrato in dashboard come warning. L'utente può poi aprire diagnostica. Questo rende il sistema più proattivo.

Una dodicesima area è la **AI per clustering dei dati**, che si collega direttamente al punto successivo della sezione. Le run possono essere raggruppate in famiglie: run stabili, run termicamente problematiche, run con failure topologico, run con failure energetico, run con rete instabile. Questo può essere fatto con clustering sui report e sulle metriche. L'obiettivo non è solo classificare manualmente, ma scoprire pattern ricorrenti nei fallimenti. Questo può aiutare la roadmap.

Cluster di run possibili:

fast_success_runs;

slow_but_stable_runs;

topology_failure_runs;

thermal_degraded_runs;

energy_limited_runs;

communication_unstable_runs;

recovery_heavy_runs.

Questi cluster possono rivelare problemi sistemici. Se molte run finiscono nel cluster thermal_degraded, bisogna rivedere cooldown o duty. Se molte finiscono in topology_failure, bisogna migliorare pattern generator. Questa è analisi intelligente.

Una tredicesima area è la **AI per simulazioni evolutive**. In futuro, MicroBot potrebbe usare AI o algoritmi evolutivi per cercare comportamenti emergenti. Invece di progettare manualmente ogni pattern, si potrebbero generare strategie, testarle in simulazione, valutarle con fitness function e selezionare quelle migliori. Questo si collega agli algoritmi genetici e alle simulazioni evolutive presenti nei progetti esterni. È una direzione affascinante, ma va considerata futura.

Una fitness function potrebbe includere:

pattern_error basso;

topology_score alto;

stability_score alto;

energy_cost basso;

thermal_event_count basso;

completion_time basso;

collision_count basso;

recovery_count basso.

L'algoritmo genera parametri, la simulazione li testa, la funzione fitness valuta, il sistema seleziona e muta. Questa idea è molto coerente con swarm intelligence, ma richiede simulazione solida e metriche affidabili.

Una quattordicesima area è la **AI per interfacce avanzate**. La dashboard potrebbe usare moduli intelligenti per filtrare dati, evidenziare problemi, ordinare nodi per criticità, suggerire viste, spiegare eventi o generare descrizioni automatiche. Per esempio, invece di mostrare 200 eventi, può raggrupparli: "5 nodi hanno perso sync durante phase_attach". Questo rende l'interfaccia più leggibile.

Funzioni possibili:

event summarization;

node ranking by risk;

automatic dashboard focus;

failure explanation;

recommendation panel;

natural language report;

query sui log;

intelligent filters.

Queste funzioni sono utili perché la telemetria può diventare molto grande. L'AI può aiutare a trasformarla in informazione comprensibile.

Una quindicesima area è la **AI come componente documentale e portfolio**. Anche se alcuni moduli non sono ancora integrati direttamente in MicroBot, i progetti AI esterni dimostrano competenza. Possono essere presentati come parte del percorso: modelli, classificatori, clustering, esperimenti, analisi dati, interfacce intelligenti. In un portfolio, questo mostra che MicroBot non nasce isolato, ma da una base ampia di esperimenti. Tuttavia, bisogna presentare con precisione: "questo progetto può supportare MicroBot in futuro", non "questo è già integrato" se non lo è.

Una classificazione documentale può essere:

già integrato;

riutilizzabile con adattamento;

ispirazione concettuale;

future direction;

portfolio separato.

Questa classificazione evita esagerazioni. È molto utile quando si prepara materiale per collaboratori o GitHub.

Una sedicesima area è la **AI e sicurezza**. Ogni modulo AI che influenza accesso o controllo deve essere considerato potenzialmente fallibile. Il riconoscimento facciale può sbagliare. Un modello di previsione può fare falsi allarmi. Un classificatore BCI può interpretare male un segnale. Un modello di anomaly detection può segnalare troppo o troppo poco. Per questo, ogni output AI deve avere confidence e deve essere filtrato. L'AI non deve essere una autorità assoluta.

Regole di sicurezza per AI:

ogni output AI deve avere confidence;

gli output incerti non devono attivare comandi critici;

AI non bypassa Safety Layer;

AI consultiva preferita nelle prime versioni;

azioni hardware richiedono validazione tradizionale;

log degli output AI;

possibilità di disabilitare modulo AI.

Queste regole proteggono il sistema e lo rendono più serio.

Una diciassettesima area è la **AI e explainability**. Se un modulo AI suggerisce una decisione, deve almeno indicare su quali metriche si basa. Per esempio, “failure risk high because topology_score decreasing, retry_count increasing, thermal_warning active”. Questo è più utile di un avviso opaco. Nel progetto MicroBot, la spiegabilità è importante perché la documentazione è centrale. Un AI black-box senza spiegazione sarebbe meno coerente con la struttura tecnica del progetto.

Un output spiegabile può essere:

risk_level = high;

main_factors = topology_score_low, missing_edges_high, retry_count_high;

recommended_action = trigger_fallback;

confidence = medium.

Questo formato è leggibile da dashboard e report. L'utente capisce il motivo.

Una diciottesima area è la **AI e versioning**. I modelli AI devono avere versione. Se un classificatore cambia, i risultati possono cambiare. Come MicroBot OS ha firmware_version e protocol_version, anche i modelli AI devono avere model_version, training_data_version, feature_schema_version e evaluation_notes. Questo è fondamentale per riproducibilità.

Un modello AI può dichiarare:

model_id;

model_version;

feature_schema_version;

training_dataset_id;
created_at;
evaluation_metrics;
supported_inputs;
limitations.

Questo impedisce confusione. Se una run viene classificata diversamente dopo aggiornamento modello, si sa perché.

Una diciannovesima area è la **AI e dataset ethics/privacy**. Se i dataset includono dati biometrici, segnali personali o log sensibili, bisogna trattarli con attenzione. Per MicroBot telemetry normale, i dati sono tecnici. Per biometria o BCI, sono sensibili. Questi dataset non devono essere pubblicati senza controllo. La Sezione 16 deve quindi distinguere AI su dati tecnici e AI su dati sensibili.

Categorie dati:

technical_swarm_data: meno sensibile;

hardware_diagnostics: tecnico ma da proteggere se contiene configurazioni;

security_events: sensibile a livello accessi;

biometric_data: altamente sensibile;

BCI/biological signals: altamente sensibile.

Questa distinzione è importante per la futura gestione dei repository. Dati biometrici reali e segnali personali non devono finire nei sample pubblici.

Una ventesima area è la **AI come estensione progressiva della roadmap MicroBot**.

L'integrazione dell'AI deve avvenire per livelli. Prima analisi offline dei report. Poi classificazione delle run. Poi suggerimenti. Poi anomaly detection. Poi ottimizzazione parametri in simulazione. Poi eventuale controllo semi-operativo. Solo molto più avanti si può valutare AI operativa in tempo reale, e comunque sotto Safety Layer.

Una roadmap sensata può essere:

AI v0.1: analisi manuale assistita dei report;

AI v0.2: classificazione regole-based delle run;

AI v0.3: dataset delle run;

AI v0.4: modello classificatore failure/success;

AI v0.5: anomaly detection;

AI v0.6: recommendation engine;

AI v0.7: ottimizzazione parametri in simulazione;

AI v0.8: dashboard assistant;

AI v0.9: semi-automatic fallback suggestions;

AI v1.0: controllo adattivo limitato e validato, solo in simulazione;

AI v1.1: integrazione hardware-safe, solo dopo test estesi.

Questa roadmap è realistica e protegge il progetto da salti troppo grandi.

Una ventunesima area è la **integrazione con i progetti AI già sviluppati esternamente**. Se esistono progetti personali di AI, modelli, classificatori, analisi dati o strumenti sperimentali, questi possono essere mappati in relazione a MicroBot. Alcuni possono diventare moduli futuri. Alcuni possono restare portfolio. Alcuni possono essere usati come prove di competenza. La cosa importante è creare una tabella di mapping: progetto esterno, funzione originale, competenza dimostrata, possibile uso MicroBot, stato attuale.

Una tabella concettuale potrebbe contenere:

nome progetto;

area tecnica;

funzione principale;

competenza acquisita;

possibile collegamento a MicroBot;

stato: esterno, riutilizzabile, integrabile, futuro.

Questo mapping sarà utile nella Sezione 17 sull'integrazione dei progetti. Per ora, la Sezione 16 introduce le famiglie.

Una ventiduesima area è la **AI nei giochi e simulazioni**. Molti giochi o simulazioni possono contenere logiche simili a quelle necessarie per MicroBot: stati, agenti, collisioni, obiettivi, punteggi, nemici, movimenti, loop. Anche senza machine learning, questi sistemi sono forme di intelligenza procedurale. Possono insegnare come gestire agenti autonomi, regole locali e comportamento emergente. Questa esperienza può essere trasferita allo swarm.

Esempi di competenze trasferibili:

state machine;

collision handling;

path logic;

agent behavior;

game loop;

event system;

score/fitness;

input handling;
real-time update;
debug visuale.

Queste competenze sono direttamente utili in simulazione MicroBot. Un MicroBot virtuale è, in molti aspetti software, un agente in un ambiente dinamico.

Una ventitreesima area è la **AI e clustering come analisi dello swarm**. Il clustering può essere usato per identificare gruppi di nodi, sottoswarm, componenti connesse, nodi isolati e regioni dense. Questo è un punto di ponte verso la prossima sottosezione. L'AI non deve necessariamente controllare i nodi: può anche interpretare la loro disposizione. Se il sistema riconosce che lo swarm si è diviso in due cluster, può generare evento: swarm_fragmented. Se riconosce nodi isolati, può suggerire recovery.

Possibili analisi:

cluster_count;
isolated_nodes;
cluster_density;
cluster_centers;
fragmentation_score;
subswarm_detection;
group_stability.

Queste metriche sono molto utili per pattern come cluster, bridge, grid e aggregation. Collegano AI, geometria e telemetria.

Una ventiquattresima area è la **AI e simulazione black-box/white-box**. Alcune funzioni AI possono essere black-box, cioè poco spiegabili, mentre altre sono white-box o rule-based. Per MicroBot, nelle prime versioni conviene usare molto rule-based e modelli interpretabili. Questo perché il progetto deve essere documentabile. Se un modello suggerisce fallback, bisogna sapere perché. I sistemi black-box possono essere studiati in futuro, ma con cautela.

Approccio consigliato:

prima regole esplicite;
poi modelli semplici interpretabili;
poi modelli più complessi solo se servono;
sempre logging e confronto con baseline;
mai sostituire safety deterministica con AI black-box.

Questo è un principio molto forte. Safety deve restare deterministica e verificabile.

Una venticinquesima area è la **AI e generazione di configurazioni sperimentali**. Il sistema può generare automaticamente configurazioni da testare: diversi node_count, pattern, noise, fault, parametri. Questo non richiede necessariamente ML avanzato, ma è automazione intelligente. Può aiutare a esplorare lo spazio delle possibilità. Un modulo può creare experimental_runs.json con combinazioni e poi analizzare risultati.

Una generazione di esperimenti può variare:

node_count;

pattern_id;

initial_distribution;

noise_level;

fault_profile;

energy_profile;

communication_latency;

behavior_profile;

safety_policy;

simulation_duration.

Questo produce dataset più ricchi. In futuro, AI può scegliere esperimenti più promettenti. Ma anche una generazione semplice è utile.

Una ventiseiesima area è la **AI e validazione delle simulazioni**. Un modello può aiutare a capire se una simulazione è realistica rispetto ai dati hardware. Quando i prototipi fisici producono misure reali, si può confrontare simulazione e hardware. Se la latenza reale è diversa, il simulatore viene calibrato. Se thermal_load reale cresce più velocemente, il modello cambia. AI/statistica può aiutare a stimare parametri.

Dati confrontabili:

latenza reale vs simulata;

energia stimata vs misurata;

cooldown stimato vs misurato;

risposta PWM reale vs virtuale;

heartbeat jitter reale vs virtuale;

failure rate hardware vs simulazione.

Questa funzione è fondamentale per rendere la simulazione meno “finta” e più aderente ai prototipi. È un’applicazione molto seria.

Una ventisettesima area è la **AI e limiti computazionali**. Non tutti i modelli possono girare su nodi MicroBot. Alcuni possono girare solo su computer centrale o backend. I microcontrollori possono avere risorse limitate. Quindi bisogna distinguere AI on-node, AI on-controller, AI offline e AI dashboard. Nelle prime versioni, quasi tutta l'AI dovrebbe girare sul controller, sul computer o offline, non sui microbot fisici.

Categorie:

AI offline: analizza run salvate;

AI backend/controller: suggerisce decisioni durante run;

AI dashboard: aiuta UI e report;

AI on-node: molto limitata, solo in futuro e con modelli leggeri.

Questa distinzione evita di progettare funzioni impossibili per hardware piccolo. MicroBot OS resta leggero; AI pesante resta fuori dal nodo.

Una ventottesima area è la **AI e documentazione dei limiti**. Ogni modulo AI deve dichiarare limiti. Per esempio: modello addestrato su dati sintetici, dataset piccolo, non validato su hardware reale, non adatto a safety, usato solo per suggerimenti, soglie sperimentali. Questa onestà è indispensabile. Un progetto tecnico maturo non nasconde i limiti.

Limitazioni da documentare:

dataset limitato;

dati sintetici;

assenza di validazione hardware;

modello sperimentale;

output consultivo;

possibili errori;

non usato per safety critica;

richiede supervisione umana.

Questa dichiarazione rende l'AI più credibile, non meno.

Una ventinovesima area è la **AI come ponte verso ricerca futura**. MicroBot può diventare, in futuro, una piattaforma per studiare swarm intelligence, ottimizzazione distribuita, comportamento emergente, controllo adattivo, interazione uomo-macchina e sistemi cyber-fisici. L'AI è uno dei ponti verso questa visione. Tuttavia, per arrivarci serve base solida: simulazione, telemetria, dataset, safety, report, hardware. La Sezione 16 deve quindi mostrare possibilità future senza confonderle con lo stato attuale.

Una formulazione corretta è:

l'AI rappresenta una direzione di espansione del sistema MicroBot, non un requisito iniziale del core hardware.

Questa frase è importante. MicroBot deve prima funzionare come sistema osservabile e sicuro. Poi può diventare intelligente in modo progressivo.

Il trentesimo punto è la **sintesi dell'intelligenza artificiale come progetto esterno integrabile**. I progetti AI esterni hanno valore perché portano metodi di classificazione, previsione, ottimizzazione, analisi, generazione report e interazione. Possono essere integrati in MicroBot in modo diretto o indiretto, ma sempre con criterio. Il loro ruolo iniziale migliore è supportare analisi e documentazione, non comandare hardware. Con il tempo, possono diventare moduli più operativi, ma solo dopo validazione.

In conclusione, l'intelligenza artificiale rappresenta un insieme di competenze e strumenti che possono rafforzare MicroBot attraverso classificazione delle run, previsione dei fallimenti, ottimizzazione dei parametri, analisi dei nodi, anomaly detection, generazione di report, assistenza alla dashboard, clustering dei comportamenti e simulazioni evolutive. Tuttavia, l'AI deve essere introdotta in modo progressivo, basata su dati telemetrici affidabili, accompagnata da versioning, explainability, limiti dichiarati e sempre subordinata al Safety Layer. Nelle prime versioni, il ruolo più coerente dell'AI è consultivo: analizzare, spiegare, suggerire e migliorare la documentazione. Solo in futuro, e solo dopo validazione estesa, potrà diventare parte semi-operativa o operativa del comportamento dello swarm. Il punto successivo, dedicato a clustering e modelli, approfondirà come i metodi di raggruppamento, classificazione e modellazione possano essere utilizzati per interpretare nodi, pattern, run sperimentali, sottoswarm, errori e configurazioni del sistema MicroBot.

16.2 Clustering e modelli

Il secondo ambito dei progetti esterni riguarda **clustering e modelli**, cioè l'insieme di tecniche utili per raggruppare dati, riconoscere strutture, descrivere comportamenti, interpretare pattern e costruire rappresentazioni formali del sistema. Dopo aver introdotto il ruolo dell'intelligenza artificiale, è necessario concentrarsi su una famiglia di strumenti più specifica e molto adatta a MicroBot: i metodi di clustering, classificazione, modellazione statistica e modellazione computazionale. Questi strumenti non servono necessariamente a "controllare" lo swarm in modo diretto, ma possono essere estremamente utili per capirlo, analizzarlo, validarlo e migliorarne il comportamento.

Nel progetto MicroBot, il clustering è particolarmente coerente perché lo swarm è composto da più unità distribuite nello spazio. Ogni MicroBot può essere visto come un punto, un nodo, un agente o un elemento di un grafo. Quando molti nodi interagiscono, possono formarsi gruppi, sottogruppi, regioni dense, catene, cluster, componenti isolate, strutture connesse o frammentazioni. Il clustering permette di leggere questa distribuzione non solo visivamente, ma anche matematicamente. In altre parole, non basta dire che "i nodi sembrano vicini": bisogna poter calcolare quanti gruppi esistono, quanto sono compatti, quali nodi sono isolati, quali sottoswarm si sono formati e se il pattern finale corrisponde alla struttura attesa.

Il principio generale può essere espresso così:

clustering e modelli permettono di trasformare la disposizione e il comportamento dei MicroBot in strutture misurabili, confrontabili e interpretabili.

Questa idea è centrale. MicroBot produce dati: posizioni, ruoli, edge, distanze, stati, metriche, eventi, errori, temperature, energia, tempi, comandi e risultati. I modelli servono a dare forma a questi dati. Il clustering trova gruppi. I modelli geometrici descrivono forme. I modelli topologici descrivono connessioni. I modelli temporali descrivono evoluzioni. I modelli statistici descrivono distribuzioni. I modelli predittivi stimano esiti futuri. Tutti questi strumenti rendono il sistema più leggibile e più scientifico.

La prima applicazione del clustering è l'**analisi spaziale dello swarm**. In una simulazione MicroBot, ogni nodo ha una posizione. A partire da queste posizioni, si può calcolare se i nodi formano un unico gruppo o più gruppi separati. Se il pattern atteso è un cluster unico, la presenza di due o tre cluster indica una frammentazione. Se il pattern atteso è un bridge tra gruppi, la presenza di due cluster con un edge centrale può essere corretta. Se il pattern atteso è una griglia, il clustering può aiutare a identificare righe, colonne o celle. La stessa tecnica può quindi avere significato diverso secondo il pattern.

L'analisi spaziale può produrre metriche come:

numero di cluster;

dimensione di ogni cluster;

centroide di ogni cluster;

densità media;

raggio del cluster;

distanza tra cluster;

nodi isolati;

frammentazione dello swarm;

stabilità dei cluster nel tempo.

Queste metriche sono estremamente utili per la telemetria e per i report. Per esempio, se durante una run cluster_basic il sistema produce cluster_count = 1, isolated_nodes = 0 e density_score alto, il pattern è probabilmente riuscito. Se cluster_count = 3 e isolated_nodes = 4, lo swarm è frammentato. Questo permette di dare giudizi oggettivi.

Una seconda applicazione è il **rilevamento dei nodi isolati**. In uno swarm, un nodo isolato può essere un problema. Può indicare che il nodo non ha ricevuto comandi, che è fuori posizione, che ha perso comunicazione, che è low_energy, che ha ruolo sbagliato o che il pattern non lo ha integrato correttamente. Il clustering permette di individuare questi nodi in modo automatico. Un nodo isolato può essere definito come nodo lontano dagli altri, senza edge, fuori dal raggio atteso o non appartenente a nessun cluster significativo.

Un isolated_node_report può contenere:

node_id;

distanza dal cluster più vicino;

edge_count;

role;
mode;
energy_state;
thermal_state;
sync_status;
last_seen;
probable_reason.

Questa analisi collega clustering e diagnostica. Se un nodo è isolato e ha sync_lost, la causa può essere comunicazione. Se è isolato ma online e con ruolo corretto, il problema può essere geometrico o comportamentale. Se è isolato e low_energy, può essere stato escluso dai comandi attivi. Il clustering fornisce il sintomo; la telemetria aiuta a trovare la causa.

Una terza applicazione è il **rilevamento della frammentazione dello swarm**. La frammentazione avviene quando lo swarm si divide in sottogruppi non collegati. In alcuni pattern può essere desiderata; in altri è un fallimento. Per esempio, in un pattern di aggregazione, la frammentazione è negativa. In un pattern multi-cluster, può essere prevista. In un bridge, due cluster separati devono essere collegati da nodi ponte. Quindi la frammentazione deve essere interpretata in base all'obiettivo del pattern.

Una metrica di frammentazione può considerare:

numero di componenti;
dimensione della componente principale;
percentuale di nodi nella componente principale;
numero di nodi fuori componente principale;
distanza tra componenti;
presenza o assenza di edge critici;
stabilità della separazione.

Se la componente principale contiene il 90% dei nodi e un nodo è isolato, il problema è minore. Se lo swarm si divide in due componenti di dimensione simile quando dovrebbe essere uno solo, il problema è grave. Questa distinzione è importante per classificare la run.

Una quarta applicazione è il **clustering temporale delle run**. Non si devono raggruppare solo nodi nello spazio, ma anche run sperimentali nel tempo. Ogni run MicroBot produce metriche finali e serie temporali. Usando queste metriche, si possono raggruppare run simili: run veloci e stabili, run lente ma riuscite, run fallite per topologia, run degradate da energia, run instabili, run con molti fallback. Questo è molto utile per capire il comportamento del sistema su molte prove.

Una run può essere rappresentata da feature come:

pattern_id;
node_count;
completion_time;
final_pattern_error;
topology_score_final;
stability_score;
safety_event_count;
energy_event_count;
thermal_event_count;
fallback_count;
recovery_count;
failure_reason.

Con queste feature, il sistema può raggruppare esperimenti. Se molte run finiscono nello stesso cluster di failure, significa che c'è un problema ricorrente. Per esempio, se bridge_complex cade spesso nel cluster topology_failure, bisogna rivedere la logica degli edge. Se cluster_basic cade spesso nel cluster collision_risk, bisogna rivedere separation_weight. Questo trasforma esperimenti isolati in conoscenza aggregata.

Una quinta applicazione è il **clustering dei failure modes**. I fallimenti MicroBot possono avere cause diverse. Alcuni sono geometrici, altri topologici, altri energetici, altri termici, altri comunicativi, altri di configurazione. Raggruppare i fallimenti permette di capire quali categorie pesano di più. Invece di vedere cento errori singoli, si vedono famiglie di problemi. Questo è molto utile per la roadmap.

Categorie di failure possono essere:

geometry_failure;
topology_failure;
role_assignment_failure;
communication_failure;
sync_failure;
energy_failure;
thermal_failure;
safety_lock_failure;
configuration_failure;

recovery_failure.

Ogni categoria può avere metriche e cause. Se il sistema mostra che il 40% dei fallimenti è topology_failure, la priorità è migliorare pattern generator o edge validation. Se il 35% è communication_failure, bisogna migliorare heartbeat, retry, timeout e network model. Se molti fallimenti sono configuration_failure, serve validazione config più forte prima della run.

Una sesta applicazione è il **modello geometrico del pattern**. Ogni pattern MicroBot può essere descritto da un modello geometrico. Una linea ha un asse, una lunghezza, una distanza tra nodi. Un cerchio ha centro, raggio, distribuzione angolare. Una griglia ha righe, colonne, celle e passo. Un cluster ha centroide, raggio e densità. Un bridge ha due gruppi e una connessione. Questi modelli permettono di valutare quanto la configurazione corrente è vicina alla forma desiderata.

Un modello geometrico può definire:

target_shape;

target_positions;

distance_constraints;

angular_constraints;

spacing_constraints;

allowed_tolerance;

validation_metric;

failure_conditions.

Per esempio, nel pattern line_basic, il modello geometrico può misurare lateral_error e spacing_error. Nel pattern circle_basic, può misurare radial_error e angular_distribution_error. Nel pattern grid_basic, può misurare cell_assignment_error. Senza modello geometrico, la validazione resta visiva. Con il modello, diventa numerica.

Una settima applicazione è il **modello topologico**. La geometria dice dove sono i nodi; la topologia dice come sono collegati. In MicroBot, molte strutture non sono definite solo dalla posizione, ma dalle relazioni. Una linea deve avere una catena di edge. Un cerchio deve avere un ciclo. Una griglia deve avere connessioni orizzontali e verticali. Un bridge deve collegare due componenti. Il modello topologico definisce expected_edges, critical_edges e vincoli di connessione.

Un modello topologico può includere:

nodes;

expected_edges;

optional_edges;

forbidden_edges;

critical_edges;

connected_components_expected;
degree_constraints;
cycle_constraints;
bridge_constraints.

Questo modello è fondamentale per validare pattern. Una disposizione può sembrare corretta ma avere edge sbagliati. Per esempio, un cerchio può avere nodi disposti bene ma non chiudere il ciclo. Un bridge può avere due cluster vicini ma nessun nodo ponte. Il modello topologico rende questi errori rilevabili.

Una ottava applicazione è il **modello dinamico**. MicroBot non è solo forma finale; è processo. Un pattern può avere fasi: prepare, assign_roles, align, attach, stabilize, report. Un modello dinamico descrive come il sistema dovrebbe evolvere nel tempo. Per esempio, durante align il pattern_error dovrebbe diminuire. Durante attach il topology_score dovrebbe aumentare. Durante stabilize la velocità media dovrebbe scendere e lo stability_score salire. Se queste tendenze non avvengono, la run è problematica.

Un modello dinamico può definire:

expected_phase_sequence;
expected_metric_trends;
maximum_phase_duration;
convergence_conditions;
stability_conditions;
allowed_retry_count;
fallback_conditions.

Questo tipo di modello è molto utile per analisi live. Se durante phase_align il pattern_error non scende, il sistema può segnalare early_warning. Se durante phase_attach missing_edges non diminuisce, può attivare fallback. Il modello dinamico trasforma il tempo in criterio decisionale.

Una nona applicazione è il **modello probabilistico dei risultati**. Dopo molte run, si possono stimare probabilità: probabilità di successo di un pattern con N nodi, probabilità di thermal_warning, probabilità di topology_failure, probabilità di recovery success. Questi modelli non devono essere complessi all'inizio. Anche statistiche semplici sono utili. Per esempio, success_rate per pattern e node_count. Questo aiuta a scegliere demo più affidabili.

Un modello probabilistico può stimare:

$P(\text{success} \mid \text{pattern_id}, \text{node_count})$;
 $P(\text{failure_type} \mid \text{configuration})$;
 $P(\text{thermal_warning} \mid \text{actuation_profile})$;

P(recovery_success | reset_reason);

P(topology_failure | pattern_complexity);

P(communication_failure | node_count).

Queste probabilità non sono verità assolute, ma indicazioni basate sui dati raccolti. Possono guidare decisioni. Se bridge_complex ha success_rate basso in modalità hardware, non è adatto a demo pubblica. Se line_basic ha success_rate alto, può essere usato per presentazione.

Una decima applicazione è il **modello di affidabilità del nodo**. Ogni nodo può avere una storia. Alcuni nodi sono stabili, altri instabili. Un modello di affidabilità può assegnare score al nodo. Questo score può influenzare role assignment. Nodi affidabili possono ricevere ruoli critici; nodi instabili possono restare passivi o essere esclusi. Questo è particolarmente utile con hardware reale.

Un node_reliability_model può usare:

uptime;

reset_count;

watchdog_reset_count;

command_rejection_rate;

response_time;

energy_stability;

thermal_events;

communication_loss_count;

recovery_success_rate;

role_failure_count.

Il risultato può essere:

reliability_score;

recommended_role_level;

critical_role_allowed;

diagnostic_required;

exclude_from_pattern.

Questo modello può essere inizialmente rule-based. Per esempio, se reset_count > soglia, il nodo non può essere leader. Se thermal_lock_count > 0 nella run corrente, non può ricevere ruolo attivo. In futuro, il modello può diventare più statistico.

Una undicesima applicazione è il **modello di complessità del pattern**. Non tutti i pattern sono uguali. Una linea è più semplice di una griglia o di un bridge. Un pattern con molti edge critici è più

fragile. Un pattern con sincronizzazione stretta è più difficile. Un modello di complessità può stimare quanto un pattern sia rischioso prima di eseguirlo. Questo può aiutare la dashboard a indicare “demo safe”, “experimental”, “hardware not recommended”.

La complessità può dipendere da:

numero nodi richiesti;

numero edge;

numero edge critici;

numero fasi;

sincronizzazione richiesta;

attuazioni richieste;

tolleranza geometrica;

tolleranza topologica;

dipendenza da ruoli critici;

energia prevista.

Un `pattern_complexity_score` può essere usato per decidere access level richiesto. Un operator può avviare pattern low complexity. Un developer può avviare experimental pattern. Un hardware_admin può testare pattern con attuazione. Questa integrazione collega modelli e controllo accessi.

Una dodicesima applicazione è il **modello energetico**. MicroBot può avere comandi e ruoli con costo energetico diverso. Un modello energetico stima consumo per fase, ruolo, comando e nodo. Anche se all’inizio è simulato o approssimato, è utile. Permette di prevedere se una run supererà un budget energetico, se certi nodi rischiano low_energy, se un pattern è troppo costoso.

Un modello energetico può includere:

base_power;

telemetry_cost;

LED_cost;

PWM_cost;

actuation_cost;

communication_cost;

role_cost_multiplier;

phase_cost;

energy_recovery o charging, se presente.

Il modello può produrre:

estimated_energy_cost;

energy_budget_remaining;

high_energy_phase;

nodes_at_risk;

recommended_load_reduction.

Questo modello è utile anche prima delle batterie reali. Permette di progettare safety e ruoli con mentalità energetica.

Una tredicesima applicazione è il **modello termico**. Per nodi con PWM, bobine o output intensi, la temperatura è critica. Un modello termico può stimare thermal_load in base ai comandi. Anche senza sensore reale, il sistema può usare un modello conservativo: thermal_load aumenta con duty e durata, diminuisce con cooldown. Questo permette di testare thermal_warning e thermal_lock in simulazione.

Un modello termico può includere:

thermal_load;

heating_rate;

cooling_rate;

warning_threshold;

lock_threshold;

recovery_threshold;

cooldown_time;

duty_factor;

command_history.

Il modello può produrre:

thermal_state;

cooldown_remaining;

commands_blocked_by_thermal;

time_to_recovery;

thermal_risk_score.

Questo modello deve essere conservativo. Se non si misura davvero la temperatura, non bisogna fingere precisione. Si può dichiarare `thermal_source = estimated`. Questo è coerente con le sezioni sulla telemetria e Safety Layer.

Una quattordicesima applicazione è il **modello comunicativo**. Lo swarm dipende dalla comunicazione. Un modello comunicativo può descrivere latenze, perdita pacchetti, jitter, ack timeout, retry e `controller_lost`. In simulazione, si possono introdurre fault profiles per testare robustezza. In hardware, si possono misurare valori reali. Questo modello permette di capire quanto la rete influenza pattern e sincronizzazione.

Un modello comunicativo può includere:

`latency_ms;`
`jitter_ms;`
`packet_loss_rate;`
`ack_timeout_ms;`
`retry_limit;`
`heartbeat_interval;`
`controller_lost_timeout;`
`message_queue_limit.`

Può produrre:

`communication_health;`
`sync_risk;`
`stale_probability;`
`recommended_timeout;`
`network_degraded_event.`

Questo modello è molto utile per scalabilità. Uno swarm con 5 nodi comunica facilmente; con 100 nodi la rete diventa un problema. Modellarla aiuta a preparare la crescita.

Una quindicesima applicazione è il **modello di sincronizzazione**. Alcuni pattern richiedono comandi temporizzati. Il modello di sincronizzazione descrive offset, jitter, `execute_at`, `sync_status` e tolleranza temporale. Se lo jitter è alto, comandi simultanei diventano imprecisi. Se offset non è aggiornato, il nodo può essere fuori fase. Questo modello permette di decidere se un nodo è idoneo per comandi timed.

Un `sync_model` può includere:

`clock_offset_ms;`
`jitter_ms;`

last_sync_age;
sync_interval;
allowed_execution_error;
timed_command_margin;
sync_status.

Può produrre:

timed_commands_allowed;
sync_degraded;
sync_lost;
resync_required;
phase_mismatch_risk.

Questo modello è essenziale per pattern avanzati. Anche se nelle prime versioni i comandi sono semplici, la documentazione deve prevedere questa evoluzione.

Una sedicesima applicazione è il **modello di accesso e sicurezza**. La Sezione 15 ha definito biometria e access control. Anche lì esistono modelli: soglia biometrica, access levels, session duration, confidence, rischio dell'azione. Questi modelli possono essere trattati con lo stesso rigore. Non sono clustering spaziale, ma modelli decisionali.

Un security_access_model può includere:

auth_score;
threshold;
confidence_class;
access_level;
session_duration;
allowed_actions;
restricted_actions;
reauth_required;
security_mode.

Questo modello collega dati biometrici a decisioni. È un esempio di come “modelli” in MicroBot non significhi solo AI, ma qualsiasi rappresentazione strutturata di un processo.

Una diciassettesima applicazione è il **modello di dashboard e interfaccia**. Anche l'interfaccia può essere modellata: quali stati mostra, quali pannelli abilita, quali warning priorizza, quali controlli

disabilita. Un modello UI può decidere che se `diagnostic_status = critical`, il pannello safety deve essere evidenziato; se `node_count` è alto, la vista deve aggregare; se `security_level = guest`, i controlli devono essere read-only.

Un `UI_state_model` può includere:

`current_access_level;`

`system_mode;`

`global_safety_state;`

`selected_node;`

`active_pattern;`

`warning_count;`

`critical_events;`

`enabled_controls;`

`recommended_view.`

Questo modello rende la dashboard coerente con lo stato del sistema. Non è AI, ma è modellazione dell'interazione.

Una diciottesima applicazione è il **clustering per visual analytics**. Quando ci sono molti nodi, la dashboard può diventare confusa. Il clustering può aiutare la visualizzazione: raggruppare nodi vicini, colorare sottoswarm, evidenziare componenti, mostrare solo cluster summary, permettere zoom su un gruppo. Questo rende l'interfaccia scalabile.

La dashboard può visualizzare:

`cluster boundary;`

`cluster centroid;`

`cluster size;`

`isolated nodes;`

`subswarm labels;`

`component connections;`

`fragmentation warning.`

Questo è molto utile quando lo swarm cresce. Senza clustering visuale, una nuvola di nodi è difficile da interpretare. Con clustering, l'utente vede struttura.

Una diciannovesima applicazione è il **clustering dei log**. I log possono diventare enormi. Raggrupparli per categoria, causa, nodo, fase o severità aiuta a leggere. Per esempio, invece di mostrare 200 `command_rejected`, la dashboard può raggruppare: "150 `command_rejected` per

cooldown_active su 5 nodi durante phase_attach”. Questo è un tipo di clustering semantico degli eventi.

Un event_cluster può contenere:

event_type;

count;

affected_nodes;

phase_id;

main_reason;

severity;

time_window;

recommended_action.

Questo rende il debug più rapido. L’utente non legge ogni evento; legge pattern di eventi. Questa funzione può essere rule-based prima che AI.

Una ventesima applicazione è il **modello di maturità del modulo**. Ogni componente MicroBot può avere stato: concept, implemented, visualized, tested, validated, hardware_partial, hardware_ready. Questo è un modello organizzativo. Permette di sapere cosa è pronto e cosa no. È molto utile per repository, roadmap e comunicazione esterna.

Un maturity_model può includere:

module_name;

current_stage;

evidence;

last_tested;

limitations;

next_step;

risk_level;

demo_ready.

Esempio: biometric_auth = prototype; MicroBot OS shell = implemented_in_concept; telemetry dashboard = tested_in_simulation; hardware PWM = experimental; coil actuation = future. Questo modello evita di presentare tutto allo stesso livello. È importantissimo per non fare confusione.

Una ventunesima applicazione è il **modello di integrazione tra progetti esterni e MicroBot**. Ogni progetto esterno può essere valutato rispetto a MicroBot: quanto è vicino, cosa può fornire, quanto lavoro serve per integrarlo, quali rischi porta. Questo modello sarà approfondito nella Sezione 17, ma nasce qui. Serve a decidere cosa portare dentro e cosa lasciare fuori.

Un integration_model può usare:

project_name;
technical_area;
current_state;
relevance_to_microbot;
integration_type;
required_adaptation;
risk;
priority;
recommended_destination.

Destinazioni possibili:

core;
simulation;
dashboard;
tools;
docs;
future_research;
portfolio_only.

Questo modello è molto utile per organizzare tanti progetti senza caos. Non tutti devono entrare nel core. Alcuni vanno in tools, altri in docs, altri restano portfolio.

Una ventiduesima applicazione è il **clustering e riduzione dimensionale per dataset MicroBot**. Se ogni run produce molte metriche, può essere utile usare tecniche di riduzione dimensionale per visualizzarle in 2D o 3D. Questo permette di vedere se run simili si raggruppano. Per esempio, run riuscite possono stare in una regione, run fallite in un'altra. Anche senza usare questi strumenti nel runtime, possono essere utili offline per analisi.

Una pipeline offline può essere:

raccolta metriche run;
normalizzazione feature;
riduzione dimensionale;
clustering;

visualizzazione;

interpretazione dei gruppi;

aggiornamento report.

Questo si collega ai progetti esterni di data analysis e machine learning. È un modo concreto per riutilizzare competenze di clustering dentro MicroBot.

Una ventitreesima applicazione è il **modello dei ruoli**. I ruoli dei nodi possono essere modellati come categorie con requisiti. Per esempio, leader richiede sync_good, energy_ok, reliability alto. Connector richiede posizione adeguata e capacità di mantenere edge. Actuator_node richiede output capability e safety ok. Passive_node richiede meno requisiti. Questo modello aiuta il controller a assegnare ruoli in modo più intelligente.

Un role_model può includere:

role_name;

required_capabilities;

required_energy_state;

required_thermal_state;

required_sync_status;

minimum_reliability_score;

allowed_modes;

priority;

fallback_role.

Esempio: bridge_node richiede reliability alto e topology importance alta. Se un nodo non soddisfa requisiti, non deve ricevere quel ruolo. Questo modello collega clustering, reliability e controller.

Una ventiquattresima applicazione è il **modello dei pattern come grafi**. Ogni pattern può essere descritto come grafo: nodi, edge, ruoli, vincoli, posizioni. Questo è molto potente. Una linea è un path graph. Un cerchio è un cycle graph. Una griglia è un grid graph. Un bridge è un grafo con due cluster collegati. Questo modello rende i pattern formalizzabili.

Un pattern_graph_model può contenere:

node_slots;

edge_slots;

slot_roles;

target_positions;

critical_edges;

optional_edges;

symmetry;

validation_rules.

Il controller può assegnare MicroBot reali o virtuali agli slot del grafo. La validazione confronta grafo atteso e grafo osservato. Questo è uno dei modelli più coerenti con MicroBot, perché unisce geometria e topologia.

Una venticinquesima applicazione è il **modello di simulazione come gemello digitale**. Un modello può rappresentare il nodo fisico nel simulatore. Questo è il concetto di digital twin o gemello virtuale. Un nodo hardware può avere stato reale per heartbeat e stato simulato per posizione. Il modello ibrido fonde i due. Questo permette di testare sistemi fisici parziali dentro uno swarm virtuale più grande.

Un hybrid_node_model può contenere:

hardware_node_id;

virtual_node_id;

real_heartbeat;

virtual_position;

merged_state;

source_fields;

confidence;

last_real_update;

last_virtual_update.

Questo modello è essenziale per demo ibride. Permette di essere onesti: alcune parti sono reali, altre simulate, e il sistema lo sa.

Una ventiseiesima applicazione è il **modello di scala**. MicroBot deve crescere da pochi nodi a molti nodi. Un modello di scala descrive come cambiano costi e limiti: numero nodi, numero edge, dimensione snapshot, carico rendering, messaggi, latenza, memoria. Questo modello aiuta a prevedere problemi prima di incontrarli.

Un scalability_model può includere:

node_count;

edge_count;

message_rate;

snapshot_size;

render_cost;

controller_processing_time;

simulation_step_time;

memory_usage;

expected_bottleneck.

Questo modello può dire, per esempio, che sopra un certo numero di nodi serve delta update invece di snapshot completo, o che il rendering degli edge deve essere semplificato. È un modello tecnico molto utile.

Una ventisettesima applicazione è il **modello di confronto tra simulazione e hardware**. Quando si avranno dati fisici, bisognerà confrontarli con simulazione. Se la simulazione prevede una risposta, ma hardware ne produce un'altra, il modello va calibrato. Questo processo è fondamentale per rendere MicroBot meno teorico e più sperimentale.

Confronti possibili:

tempo risposta comando simulato vs reale;

heartbeat jitter simulato vs reale;

consumo stimato vs reale;

thermal_load stimato vs misurato;

output timing simulato vs reale;

recovery simulata vs reale.

Questo permette di aggiornare i parametri del simulatore. Il modello non resta fisso; migliora con dati reali.

Una ventottesima applicazione è il **ruolo dei modelli nella documentazione**. I modelli non devono restare solo codice. Devono essere documentati. Ogni modello deve avere scopo, input, output, limiti, stato, versione e relazione con MicroBot. Questo rende la documentazione da tesi più solida. Un lettore deve capire che pattern_error, topology_score, cluster_count e reliability_score non sono numeri casuali, ma derivano da modelli definiti.

Un documento model_catalog.md può includere:

nome modello;

scopo;

input;

output;

formula o logica;

moduli che lo usano;

stato;

limitazioni;

versione.

Questo catalogo sarebbe molto utile per una repo MicroBot Simulation Core o MicroBot Analytics. Dà ordine ai concetti.

Una ventinovesima applicazione è la **roadmap di clustering e modelli**. Anche questo ambito deve crescere per livelli. Prima modelli semplici e metriche manuali. Poi clustering spaziale. Poi modelli topologici. Poi clustering delle run. Poi reliability model. Poi modelli energetici e termici. Poi analisi predittiva. Poi integrazione dashboard. Questa progressione evita di introdurre troppa complessità all'inizio.

Una roadmap possibile è:

v0.1: metriche geometriche base;

v0.2: cluster_count e isolated_nodes;

v0.3: topology_score e missing_edges;

v0.4: run_summary model;

v0.5: failure_reason taxonomy;

v0.6: node_reliability_score;

v0.7: energy_model stimato;

v0.8: thermal_model stimato;

v0.9: communication_model;

v1.0: clustering delle run offline;

v1.1: visual analytics nella dashboard;

v1.2: pattern_graph_model;

v1.3: hybrid_node_model;

v1.4: scalability_model;

v1.5: predictive models consultivi.

Questa roadmap rende clustering e modelli una parte ordinata dell'ecosistema, non un'aggiunta casuale.

Il trentesimo punto è la **sintesi del ruolo di clustering e modelli**. Questi strumenti sono il ponte tra dati grezzi e comprensione. Senza modelli, MicroBot produce immagini, log e numeri. Con i modelli, produce struttura: cluster, grafi, pattern, failure modes, reliability, energy, thermal, communication, scale, access, maturity. Questa è una differenza enorme. I modelli rendono il progetto più scientifico, più analizzabile e più presentabile.

In conclusione, clustering e modelli costituiscono uno degli ambiti più importanti dei progetti esterni collegabili a MicroBot. Il clustering permette di analizzare nodi, sottoswarm, frammentazioni, run sperimentali, failure modes e log. I modelli permettono di descrivere geometria, topologia, dinamica, energia, temperatura, comunicazione, affidabilità, ruoli, pattern, scala, sicurezza e integrazione. Nelle prime versioni, questi strumenti possono essere semplici e rule-based; in futuro possono diventare più statistici, predittivi o intelligenti. Il loro valore principale è trasformare lo swarm da fenomeno osservato a sistema misurabile. Il punto successivo, dedicato ai giochi, mostrerà come i progetti ludici e interattivi possano contribuire a MicroBot attraverso gestione del game loop, input, collisioni, agenti, stati, rendering, regole locali e simulazioni dinamiche.

16.3 Giochi

I **giochi** rappresentano il terzo ambito dei progetti esterni collegabili all'ecosistema MicroBot e devono essere considerati molto più importanti di quanto possa sembrare a prima vista. Un videogioco, anche semplice, non è soltanto un prodotto ludico: è un sistema software dinamico che gestisce tempo, input, stati, oggetti, collisioni, regole, punteggi, eventi, interfaccia, rendering, aggiornamenti continui e comportamento di entità autonome o semi-autonome. Per questo motivo, i progetti di gioco sviluppati esternamente possono fornire competenze direttamente trasferibili alla simulazione MicroBot, alla dashboard interattiva, al controllo degli agenti virtuali e alla costruzione di ambienti dinamici osservabili.

Nel progetto MicroBot, i giochi non devono essere inseriti nel core del sistema come elementi decorativi. Devono essere interpretati come laboratori tecnici. Un gioco come Tetris insegna gestione dello stato, griglie, collisioni, rotazioni, validazione delle mosse e aggiornamento del punteggio. Un gioco come Space Invaders insegna loop in tempo reale, proiettili, nemici, eventi, collisioni, vite, livelli e rendering dinamico. Altri giochi o simulazioni interattive insegnano movimento, input utente, entità multiple, sistemi di particelle, condizioni di vittoria e fallimento, gestione della difficoltà e feedback visuale. Tutti questi concetti sono collegabili a MicroBot, perché anche MicroBot è un sistema con agenti, stati, regole, eventi, obiettivi e visualizzazione.

Il principio generale può essere espresso così:

i giochi esterni contribuiscono a MicroBot perché insegnano a progettare sistemi dinamici, interattivi e regolati da stati, eventi e aggiornamenti continui.

Questa frase è importante perché chiarisce il valore tecnico dei giochi. Non si tratta di dire che MicroBot “diventa un gioco”, ma che molte competenze usate nei giochi sono le stesse necessarie per simulare uno swarm. Un MicroBot virtuale, nel simulatore, è simile a un'entità di gioco: ha posizione, velocità, stato, ruolo, target, collisioni o distanze da rispettare, interazioni con altri nodi, eventi e condizioni di successo. Il simulatore MicroBot, a sua volta, assomiglia a un motore di gioco specializzato: aggiorna entità nel tempo, disegna lo stato, riceve input, calcola metriche e produce eventi.

La prima competenza trasferibile dai giochi è il **game loop**. Ogni gioco in tempo reale funziona attraverso un ciclo principale: legge input, aggiorna stato, calcola logica, gestisce collisioni, disegna la scena e ripete. MicroBot ha una struttura simile. La simulazione deve aggiornare posizioni dei nodi, calcolare forze o regole locali, verificare pattern, aggiornare telemetria, controllare safety virtuale, generare eventi e ridisegnare la dashboard. La logica del game loop diventa quindi una base concettuale per il simulation loop.

Un game loop generico può essere descritto così:

lettura input;
aggiornamento tempo;
aggiornamento entità;
gestione collisioni o vincoli;
calcolo eventi;
aggiornamento UI;
rendering;
ripetizione.

Nel simulatore MicroBot, questa struttura diventa:

lettura comandi utente o controller;
aggiornamento simulation_time;
aggiornamento nodi;
calcolo distanze, edge e pattern;
generazione eventi telemetrici;
aggiornamento metriche;
rendering dello swarm;
salvataggio snapshot.

Questa somiglianza rende i progetti di gioco molto utili. Chi ha sviluppato giochi ha già affrontato il problema di mantenere un sistema dinamico coerente nel tempo.

La seconda competenza è la **gestione degli stati**. Nei giochi, un oggetto può essere alive, dead, moving, paused, falling, colliding, invincible, attacking, waiting. In MicroBot, un nodo può essere idle, active, safe_idle, diagnostic, recovery, low_energy, thermal_warning, offline. La logica è simile: ogni entità ha uno stato, e lo stato determina quali azioni sono permesse. Un oggetto in pausa non si muove; un nodo in safety non esegue comandi attuativi. Un nemico eliminato non spara; un nodo offline non riceve ruoli. La state machine dei giochi è direttamente trasferibile a MicroBot OS e simulazione.

Una state machine di gioco insegna:

transizioni ammesse;
eventi che cambiano stato;
azioni consentite per stato;
stati temporanei;

stati bloccanti;

condizioni di reset;

feedback visuale dello stato.

In MicroBot, questi concetti diventano:

mode transitions;

role transitions;

safety transitions;

thermal transitions;

energy transitions;

recovery transitions;

dashboard indicators.

Questa continuità è molto forte. Un gioco ben progettato non lascia oggetti in stati incoerenti. Allo stesso modo, MicroBot OS non deve lasciare nodi in stati ambigui. La competenza è la stessa: modellare stati e transizioni.

La terza competenza è la **gestione degli input**. Nei giochi, l'input può arrivare da tastiera, mouse, controller, touch o eventi automatici. Il sistema deve interpretarlo, filtrarlo e applicarlo allo stato. MicroBot riceve input da dashboard, controller, shell, comandi seriali, telemetria, sensori, moduli biometrici o futuri segnali BCI. Anche qui, non basta ricevere input: bisogna validarlo, classificarlo, decidere se è permesso e trasformarlo in azione.

Nei giochi, input come freccia sinistra o tasto spazio vengono trasformati in movimento o azione. In MicroBot, input come `start_pattern`, `assign_role`, `safe_idle`, `set_led` o `request_status` vengono trasformati in comandi strutturati. La differenza è il livello di sicurezza, ma la logica generale è simile. I giochi insegnano a non confondere input grezzo e azione finale. C'è sempre una pipeline: input → interpretazione → validazione → aggiornamento stato.

Questa competenza è utile per:

dashboard commands;

shell commands;

keyboard control nella simulazione;

mouse interaction con nodi;

selezione di pattern;

debug tools;

controlli live;

modalità demo.

Un simulatore MicroBot interattivo può usare input come click su nodo, drag, zoom, pause, reset, start pattern, show telemetry, inspect edge. Questa è logica tipica da game UI.

La quarta competenza è la **gestione delle collisioni e dei vincoli spaziali**. Nei giochi, collisioni tra giocatore, nemici, muri, proiettili o oggetti sono fondamentali. In MicroBot, si parla più spesso di distanze, contatti, edge, collision avoidance, spacing e aggregazione. Anche se il significato fisico è diverso, il problema software è simile: bisogna rilevare quando due entità sono troppo vicine, quando si toccano, quando superano un confine, quando entrano in una regione o quando formano una connessione.

Un sistema di collisioni insegna:

calcolo di bounding box o distanze;

rilevamento contatti;

risposta alla collisione;

vincoli di movimento;

zone proibite;

limiti dello spazio;

eventi generati dal contatto.

In MicroBot, questi concetti diventano:

neighbor_distance;

collision_count;

minimum_spacing;

connection_radius;

edge_formation;

forbidden_overlap;

boundary constraints;

cluster_density.

Per esempio, un nodo non deve sovrapporsi ad altri nella simulazione. Oppure due nodi possono formare un edge se la distanza è sotto una soglia. Oppure un cluster può essere troppo denso se molti nodi violano una distanza minima. Questi problemi sono molto vicini alla collision detection dei giochi.

La quinta competenza è la **gestione di entità multiple**. Un gioco può avere molti nemici, proiettili, blocchi, particelle, oggetti e power-up. MicroBot ha molti nodi, edge, eventi, pattern, gruppi e metriche. Gestire molte entità richiede strutture dati ordinate: liste, array, dizionari, oggetti, componenti, identificatori unici, stati e aggiornamenti. I giochi insegnano a non scrivere codice duplicato per ogni entità, ma a usare strutture generiche.

In MicroBot, ogni nodo deve essere un'entità con campi come:

node_id;
position;
velocity;
mode;
role;
energy_state;
thermal_state;
sync_status;
target;
neighbors;
last_update.

Questa struttura è molto simile a un entity object nei giochi. Il simulatore aggiorna ogni nodo a ogni step. La dashboard renderizza ogni nodo. Il controller assegna ruoli a ogni nodo. Il clustering analizza gruppi di nodi. Tutto questo richiede gestione ordinata di entità multiple, una competenza tipica da game development.

La sesta competenza è il **rendering dinamico**. Nei giochi, il rendering non è statico: la scena cambia continuamente. Oggetti si muovono, colori cambiano, animazioni indicano stati, effetti mostrano eventi, HUD mostra punteggi o vite. MicroBot ha bisogno dello stesso tipo di visualizzazione: nodi che si muovono, edge che appaiono, colori che indicano ruoli, pannelli che mostrano telemetria, warning che emergono, grafici che si aggiornano, indicatori di safety e pattern.

Il rendering di giochi insegna:

separare logica e visualizzazione;
disegnare entità in base allo stato;
usare colori o forme per categorie;
aggiornare HUD;
gestire effetti temporanei;
ottimizzare frame rate;
evitare rendering pesante inutile.

In MicroBot, questo diventa:

render_node_by_role;

render_node_by_safety_state;

render_edges;

render_target_positions;

render_cluster_boundaries;

render_warning_icons;

render_telemetry_HUD;

render_selected_node_panel.

Questa competenza è fondamentale per una dashboard efficace. Una simulazione tecnica non deve essere solo corretta; deve essere leggibile. I giochi insegnano a rendere visibili stati complessi con simboli, colori, animazioni e feedback.

La settima competenza è la **gestione dell'interfaccia HUD**. Molti giochi hanno un HUD, cioè un pannello che mostra vite, punteggio, tempo, munizioni, livello, energia, mappa o obiettivi. MicroBot ha bisogno di un HUD tecnico: node_count, active_nodes, pattern_error, topology_score, safety_state, energy_summary, thermal_summary, current_phase, run_time, event_count, connection_status. L'esperienza di costruire HUD nei giochi si trasferisce direttamente alla dashboard MicroBot.

Un HUD MicroBot può mostrare:

simulation_step;

runtime_mode;

node_count;

active_nodes;

pattern_id;

phase_id;

aggregation_score;

topology_score;

safety_summary;

telemetry_status;

last_event;

selected_node.

Il principio è lo stesso di un gioco: l'utente deve capire subito cosa sta succedendo senza leggere log enormi. Il gioco insegna sintesi visuale; MicroBot la applica a dati tecnici.

L'ottava competenza è la **gestione degli eventi**. Nei giochi, eventi sono collisioni, punti segnati, nemico eliminato, livello completato, power-up raccolto, game over. In MicroBot, eventi sono node_offline, role_changed, edge_formed, command_rejected, thermal_warning, pattern_completed, recovery_started, fallback_triggered. La struttura è analoga: un evento accade, viene registrato, può modificare stato, può essere mostrato e può generare conseguenze.

Un event system da gioco insegna:

event_type;

timestamp;

source;

target;

payload;

priority;

handling;

visual feedback;

log.

In MicroBot, event system diventa parte della telemetria. Ogni evento deve essere strutturato, classificato e visualizzato. I giochi insegnano anche a evitare che gli eventi diventino caos: bisogna filtrarli, raggrupparli e mostrare solo quelli importanti all'utente.

La nona competenza è la **gestione delle condizioni di vittoria e fallimento**. Nei giochi, esistono condizioni precise: hai vinto se completi il livello; hai perso se finisci vite; il livello termina se il tempo scade. In MicroBot, le run devono avere condizioni di successo e fallimento: pattern raggiunto, topology_score sopra soglia, stability_time sufficiente, no critical safety events, timeout non superato, fallback riuscito o fallito. Questa è una trasposizione molto chiara.

Un gioco insegna che un sistema deve sapere quando una sessione termina e con quale risultato. MicroBot deve fare lo stesso. Una run non può continuare indefinitamente. Deve avere:

success condition;

partial success condition;

failure condition;

timeout condition;

abort condition;

safety stop condition;

report generation.

Questa logica è tipica da gioco e molto utile in simulazione. Per esempio, una run line_basic può essere success se lateral_error e spacing_error restano sotto soglia per 3 secondi. Può essere failed

se timeout supera 60 secondi. Può essere aborted se emergency_stop. Questa struttura rende le simulazioni verificabili.

La decima competenza è la **gestione del punteggio e delle metriche**. I giochi usano punteggi, vite, livelli, combo, tempo, accuracy. MicroBot usa metriche tecniche: pattern_error, topology_score, stability_score, aggregation_score, safety_event_count, energy_cost, completion_time. Anche se il significato è diverso, la logica di misurazione è simile. Il sistema deve calcolare metriche durante l'esecuzione, aggiornarle e mostrarle.

Un punteggio di gioco può diventare in MicroBot:

run_score;

pattern_quality_score;

safety_score;

energy_score;

stability_score;

demo_readiness_score.

Per esempio, una run può ricevere un punteggio finale basato su successo, velocità, stabilità, safety e energia. Questo non serve a “gamificare” il progetto in modo superficiale, ma a sintetizzare qualità della run. I giochi insegnano come rendere misurabile l'esito.

L'undicesima competenza è la **gestione della difficoltà e dei livelli**. Nei giochi, i livelli aumentano difficoltà: più nemici, più velocità, più ostacoli, meno tempo. MicroBot può usare un concetto simile per test progressivi: prima 5 nodi, poi 10, poi 20; prima line_basic, poi circle_basic, poi grid, poi bridge; prima senza fault, poi con node_offline, poi con latency, poi con energy constraints. Questa logica di livelli è perfetta per una roadmap sperimentale.

Una progressione MicroBot può essere:

level 1: LED nodes e heartbeat;

level 2: role assignment visuale;

level 3: pattern line in simulazione;

level 4: cluster con metriche;

level 5: fault injection semplice;

level 6: recovery;

level 7: hybrid node;

level 8: safety events;

level 9: hardware demo controllata;

level 10: swarm più grande.

Questa idea è utile anche per presentare il progetto. Invece di mostrare tutto insieme, si mostra una progressione chiara come livelli tecnici.

La dodicesima competenza è il **debug visuale**. Nei giochi, quando qualcosa non funziona, si mostrano bounding box, coordinate, hitbox, FPS, collisioni, stati degli oggetti. MicroBot ha bisogno della stessa cosa: mostrare target, distanze, edge attesi, edge mancanti, range di connessione, cluster boundary, vettori di movimento, stato dei nodi, last command. Il debug visuale rende molto più semplice capire il comportamento dello swarm.

Un debug overlay MicroBot può mostrare:

node_id;
role;
velocity vector;
target_position;
distance_to_target;
neighbor_radius;
expected_edges;
missing_edges;
safety_state;
last_event;
telemetry_age.

Questa modalità è direttamente ispirata al debug nei giochi. È una funzione molto potente per sviluppare simulazioni e dashboard.

La tredicesima competenza è la **gestione del tempo e del frame rate**. Nei giochi, il tempo può essere gestito con delta time, FPS, pause, slow motion, time scale. MicroBot può usare concetti simili nella simulazione: simulation_step, simulation_time, time_scale, pause, resume, fast-forward, replay. Questo è fondamentale per analizzare run. Si può rallentare una fase, mettere in pausa, osservare un edge, riprodurre eventi.

Funzioni temporali utili:

pause simulation;
resume simulation;
step one frame;
time_scale;
slow_motion;
fast_forward;

replay;

jump_to_event;

snapshot_at_time.

Queste funzioni sono comuni nei giochi e molto utili in MicroBot. Una dashboard con replay e step-by-step sarebbe molto professionale.

La quattordicesima competenza è la **gestione di scene e modalità**. Un gioco ha menu principale, livello, pausa, game over, impostazioni, schermata vittoria. MicroBot può avere modalità simili: home, simulation, live dashboard, replay, diagnostics, settings, security, report. La struttura delle scene aiuta a organizzare l'interfaccia. Non tutto deve stare in una sola schermata.

Una dashboard MicroBot può avere:

overview scene;

simulation scene;

telemetry scene;

node inspector scene;

diagnostics scene;

security scene;

report scene;

settings scene.

Questa organizzazione migliora usabilità. I giochi insegnano che l'esperienza utente deve avere stati e schermate coerenti.

La quindicesima competenza è la **gestione del reset e restart**. Nei giochi, si può riavviare un livello, resettare il punteggio, ripartire da checkpoint. MicroBot può usare concetti simili: reset simulation, restart run, restore initial_swarm_state, replay from snapshot, rollback to safe state. Questa logica è molto utile per esperimenti riproducibili. Se una run fallisce, si può tornare allo stato iniziale e riprovare con parametri diversi.

Funzioni possibili:

reset_run;

restart_pattern;

load_initial_state;

save_checkpoint;

restore_checkpoint;

compare_runs;

replay_failed_run.

Queste funzioni trasformano MicroBot in un ambiente sperimentale, non solo in una demo. I giochi offrono una base mentale per gestire sessioni ripetibili.

La sedicesima competenza è la **gestione dei sistemi di particelle ed effetti visuali**. Alcuni giochi usano particelle per esplosioni, scie, energia, collisioni, fuoco, stelle. MicroBot può usare effetti visuali per rappresentare campi, comunicazione, energia, segnali, edge formati, warning, attrazione/repulsione virtuale, propagazione del comando. Questi effetti devono essere tecnici, non solo estetici. Devono aiutare a capire.

Effetti utili:

pulse su comando inviato;

glow su nodo attivo;

linea animata su edge formato;

warning ring su nodo in safety;

campo virtuale intorno a un nodo;

scia di movimento;

heat aura per thermal_warning;

telemetry pulse verso controller.

Questi effetti, se usati bene, rendono il sistema più leggibile. I giochi insegnano come visualizzare processi invisibili.

La diciassettesima competenza è la **gestione degli agenti**. Molti giochi contengono agenti con comportamento semplice: inseguono, evitano, pattugliano, attaccano, scappano, seguono percorsi. MicroBot è composto da agenti. Anche se il comportamento dello swarm è tecnico, la logica agent-based è simile. Ogni MicroBot virtuale può avere regole locali: avvicinarsi al target, mantenere distanza, formare edge, evitare collisioni, seguire ruolo, ridurre attività se low_energy.

Regole agent-based possibili:

seek target;

avoid crowding;

align with neighbors;

maintain edge distance;

follow role behavior;

enter safe state;

respond to command;

report telemetry.

Queste regole sono parenti strette delle logiche di agenti nei giochi. L'esperienza con giochi aiuta a progettare comportamenti locali senza centralizzare tutto.

La diciottesima competenza è la **separazione tra logica e rendering**. In un gioco ben progettato, la logica del mondo non deve dipendere direttamente dalla grafica. Lo stesso vale per MicroBot. La simulazione deve calcolare stato e metriche; il renderer deve visualizzarle. Se la dashboard cambia stile, la logica deve restare. Se si usa canvas, Three.js o HTML, i dati di simulazione devono essere gli stessi. Questa separazione è fondamentale per evitare codice fragile.

La separazione corretta è:

simulation_state;

update_engine;

metrics_engine;

event_system;

renderer;

UI controls;

telemetry_output.

I giochi insegnano questa architettura. Un errore comune è mischiare tutto: movimento dentro il disegno, logica dentro UI, dati dentro animazioni. MicroBot deve evitare questo.

La diciannovesima competenza è la **gestione della performance**. Nei giochi, se ci sono troppe entità o effetti, il frame rate scende. In MicroBot, se ci sono molti nodi, edge, telemetry panels e animazioni, la dashboard può rallentare. Le tecniche di ottimizzazione da gioco sono utili: ridurre rendering non necessario, aggiornare solo parti cambiate, usare livelli di dettaglio, limitare effetti, raggruppare oggetti, usare spatial partitioning.

Ottimizzazioni possibili:

renderizzare edge solo se visibili;

ridurre label con molti nodi;

limitare trail length;

usare cluster summary;

aggiornare HUD a frequenza minore;

usare delta update;

disabilitare effetti in performance mode.

Queste tecniche diventano essenziali nella Sezione 19 sulla scalabilità. I giochi offrono esperienza concreta su questi problemi.

La ventesima competenza è la **gestione della modalità replay**. Molti giochi hanno replay o registrazione. MicroBot può usare replay per analisi: salvare snapshot, eventi e metriche, poi

riprodurre la run. Questo è uno strumento potentissimo per debug e presentazione. La logica di replay da gioco si applica perfettamente: timeline, play, pause, seek, speed, event markers.

Un replay MicroBot può includere:

snapshot_sequence;

event_sequence;

metrics_sequence;

initial_config;

final_report;

timeline controls;

jump_to_event;

selected_node_history.

Questa funzione collega giochi, telemetria e dashboard. È una delle integrazioni più forti.

La ventunesima competenza è la **gestione dei livelli di astrazione visuale**. Nei giochi strategici o simulativi, si può zoomare: vista globale, vista locale, dettaglio unità. MicroBot può fare lo stesso: vista swarm globale, vista cluster, vista nodo, vista edge, vista telemetry. Questo è utile quando i nodi aumentano. L'utente non può leggere tutto sempre. Deve poter navigare livelli.

Livelli possibili:

global swarm view;

cluster view;

pattern view;

node view;

edge view;

telemetry view;

debug view.

Questa idea è molto vicina al design dei giochi strategici e simulativi. Rende MicroBot più usabile.

La ventiduesima competenza è la **gestione dell'interazione mouse**. Nei giochi, il mouse può selezionare unità, trascinare oggetti, cliccare menu, zoomare, puntare. In MicroBot, mouse interaction può essere molto utile: clic su nodo per aprire telemetria, hover su edge per vedere distanza, drag per spostare target, scroll per zoom, click su cluster per evidenziare sottoswarm. Questa è una funzione molto potente per dashboard.

Interazioni utili:

hover node → mini telemetry;

click node → inspector;

hover edge → edge status;

click pattern → pattern details;

drag target → modify simulation;

box select nodes;

zoom/pan;

right click context actions, in developer mode.

Queste funzioni rendono la simulazione esplorabile. I giochi insegnano come progettare interazioni naturali.

La ventitreesima competenza è la **gestione di AI non machine-learning nei giochi**. Molti giochi usano AI rule-based: nemici che seguono regole, stati, pathfinding, decisioni. Questa forma di AI è molto utile per MicroBot perché è interpretabile. Prima di usare machine learning, si possono usare regole locali. Per esempio, se nodo è troppo lontano dal cluster, torna verso centro; se thermal_warning, riduci attività; se missing_edge, avvicinati al neighbor. Questa è AI procedurale, non statistica.

Regole interpretabili:

if distance_to_target > threshold, move_toward_target;

if neighbor_too_close, apply_separation;

if thermal_warning, reduce_output;

if low_energy, request_passive_role;

if missing_edge, attempt_reconnect;

if sync_lost, reject_timed_commands.

Questa logica è più sicura e documentabile. I giochi insegnano a costruire agenti credibili con regole semplici.

La ventiquattresima competenza è la **gestione delle mappe e degli spazi**. Molti giochi hanno coordinate, mappe, griglie, livelli, ostacoli, aree. MicroBot può usare spazi virtuali: piano 2D, ambiente 3D, griglia, regioni magnetiche, zone proibite, target zones, area di aggregazione. L'esperienza con giochi aiuta a modellare lo spazio della simulazione.

Spazi MicroBot possibili:

2D plane;

grid space;

continuous field;

bounded arena;

target region;

obstacle region;

safe zone;

forbidden zone;

cluster zone.

Questi concetti sono utili per simulazione, pattern e dashboard. Una griglia di Tetris, per esempio, insegna vincoli discreti. Uno shooter insegna spazio continuo e collisioni. Entrambi possono ispirare MicroBot.

La venticinquesima competenza è la **gestione di modalità educative**. I giochi spesso insegnano regole attraverso livelli, tutorial, feedback, obiettivi. MicroBot può avere una modalità educativa per spiegare swarm robotics, pattern, telemetria, safety, clustering e MicroBot OS. Una demo didattica potrebbe guidare l'utente: "fase 1: i nodi ricevono ruolo; fase 2: si allineano; fase 3: formano edge; fase 4: stabilizzano". Questa è una forma di gamified explanation, ma con scopo tecnico.

Una modalità educativa può includere:

tutorial step-by-step;

highlight dei nodi;

spiegazione delle fasi;

metriche semplificate;

eventi commentati;

obiettivi visuali;

quiz o checkpoint, se utile;

replay guidato.

Questo sarebbe molto utile per presentazioni a scuole, università o collaboratori. I giochi insegnano come rendere complessi sistemi comprensibili attraverso progressione e feedback.

La ventiseiesima competenza è la **gestione di errori e feedback immediato**. Nei giochi, quando sbagli, il sistema reagisce subito: suono, animazione, perdita vita, messaggio, reset. MicroBot deve fare lo stesso tecnicamente: se un comando è rifiutato, la UI deve mostrarlo; se un nodo va offline, deve evidenziarlo; se un edge manca, deve segnalarlo; se safety entra in lock, deve essere immediatamente visibile. Feedback immediato riduce confusione.

Feedback MicroBot utili:

toast su `command_rejected`;

colore rosso su `safety_lock`;

icona warning su nodo low_energy;

edge tratteggiato se missing;

pulse verde su edge formed;

banner su controller_lost;

panel su auth_denied;

log event evidenziato.

Questa sensibilità al feedback viene molto dal game design. Un sistema senza feedback sembra rotto anche quando funziona.

La ventisettesima competenza è la **gestione della struttura dei progetti**. I giochi spesso richiedono cartelle ordinate: assets, sprites, sounds, scenes, entities, systems, configs, levels. MicroBot può usare una struttura simile: assets, simulation, nodes, systems, configs, data, logs, dashboard, tools. L'esperienza con progetti di gioco aiuta a organizzare file e moduli.

Una struttura ispirata può essere:

entities/ per nodi virtuali;

systems/ per update, physics, telemetry;

scenes/ per dashboard views;

assets/ per icone e visual;

configs/ per pattern e profili;

data/ per run e snapshot;

tools/ per debug e report.

Questa organizzazione rende il progetto più leggibile e scalabile. Anche se MicroBot non è un gioco, può imparare dalla struttura dei game projects.

La ventottesima competenza è la **trasformazione dei giochi in portfolio tecnico**. I giochi esterni possono essere presentati come prova di competenze trasferibili: loop, rendering, input, collisioni, stati, UI, performance, agenti. Non devono essere venduti come parte diretta del core MicroBot se non lo sono. Ma possono essere inseriti nella documentazione come progetti paralleli che hanno contribuito alla maturazione tecnica. Questo è molto utile per un portfolio GitHub.

Una descrizione corretta può essere:

il progetto di gioco X non è integrato direttamente nel core MicroBot, ma ha contribuito allo sviluppo di competenze in gestione del loop, collision detection, rendering e stati, successivamente riutilizzabili nella simulazione dello swarm.

Questa formulazione è rigorosa. Valorizza il lavoro senza inventare integrazioni inesistenti.

La ventinovesima competenza è la **roadmap di riuso dai giochi a MicroBot**. Non tutto va portato dentro MicroBot. Bisogna selezionare. Prima si possono riusare concetti: game loop, input,

rendering, HUD. Poi si possono riusare componenti: event system, scene manager, debug overlay, replay. Infine si possono creare demo educative ispirate ai giochi. Questa progressione è pulita.

Una roadmap possibile è:

- v0.1: trasferire concetto di simulation loop;
- v0.2: creare input controls per simulazione;
- v0.3: introdurre debug overlay;
- v0.4: migliorare HUD tecnico;
- v0.5: aggiungere pause, reset, time_scale;
- v0.6: aggiungere event feedback visuale;
- v0.7: aggiungere replay controls;
- v0.8: creare modalità tutorial;
- v0.9: ottimizzare rendering per molti nodi;
- v1.0: creare scene manager per dashboard.

Questa roadmap permette ai giochi di contribuire in modo ordinato.

Il trentesimo punto è la **sintesi del ruolo dei giochi**. I giochi sono importanti perché insegnano a gestire sistemi dinamici in tempo reale. MicroBot ha bisogno esattamente di questo: nodi che cambiano stato, scene che si aggiornano, input che arrivano, eventi che accadono, metriche che evolvono, interfacce che devono restare leggibili. I giochi non sono una deviazione dal progetto; sono una palestra tecnica. Se integrati concettualmente, aiutano a costruire simulazioni più robuste, dashboard più interattive e ambienti sperimentali più chiari.

In conclusione, i progetti di gioco esterni contribuiscono a MicroBot attraverso competenze fondamentali: game loop, gestione degli stati, input, collisioni, entità multiple, rendering dinamico, HUD, event system, condizioni di successo/fallimento, metriche, progressione, debug visuale, replay, interazione mouse, agenti rule-based e performance. Queste competenze possono essere trasferite alla simulazione dello swarm, alla dashboard, alla telemetria visuale, al replay e alle modalità educative. I giochi non devono sostituire MicroBot né essere inseriti forzatamente nel core, ma devono essere riconosciuti come laboratori tecnici che hanno preparato molte delle competenze necessarie per costruire un sistema interattivo complesso. Il punto successivo, dedicato ai sistemi interattivi, allargherà questo discorso analizzando interfacce, input avanzati, risposta in tempo reale, feedback utente, esperienze immersive e modalità di controllo che collegano la persona al sistema MicroBot in modo più diretto e dinamico.

16.4 Sistemi interattivi

I **sistemi interattivi** rappresentano il quarto ambito dei progetti esterni collegabili all'ecosistema MicroBot e riguardano tutte quelle applicazioni in cui l'utente non osserva soltanto un contenuto, ma modifica, controlla, esplora o influenza il comportamento del sistema in tempo reale. Dopo aver analizzato il ruolo dei giochi, che insegnano gestione del loop, input, collisioni, rendering e stati dinamici, è naturale estendere il discorso ai sistemi interattivi in senso più ampio: dashboard,

simulatori esplorabili, interfacce web, ambienti AR/VR, controlli tramite mouse, gesture, camera, sensori, comandi vocali, input biometrici e pannelli di telemetria. MicroBot, infatti, non è soltanto uno swarm da far funzionare; è anche un sistema che deve essere visto, compreso, controllato, spiegato e manipolato dall'utente.

Nel progetto MicroBot, l'interattività ha un ruolo centrale perché trasforma la simulazione e la documentazione in esperienza. Un sistema statico può mostrare una forma finale, un grafico o una tabella. Un sistema interattivo permette invece di selezionare un nodo, leggere la sua telemetria, modificare un parametro, cambiare pattern, vedere gli effetti in tempo reale, attivare una modalità diagnostica, osservare edge mancanti, simulare un fault, far partire un replay, confrontare due run, passare da una vista globale a una vista locale e capire perché lo swarm si comporta in un certo modo. L'interattività rende MicroBot più comprensibile e più dimostrabile.

Il principio generale può essere espresso così:

un sistema interattivo MicroBot deve permettere all'utente di osservare, esplorare e controllare il comportamento dello swarm senza compromettere sicurezza, coerenza dello stato e tracciabilità delle azioni.

Questa frase è fondamentale perché l'interattività non deve essere confusa con libertà totale. Non basta rendere cliccabili i nodi o aggiungere pulsanti. Ogni interazione deve essere collegata a uno stato, a una policy e a un effetto controllato. Se l'utente clicca un nodo, il sistema può aprire l'inspector. Se trascina un target, deve aggiornare solo la simulazione o richiedere conferma se la modifica riguarda hardware. Se preme start_pattern, il controller deve verificare access_level, configurazione, stato della run e Safety Layer. Un sistema interattivo serio non esegue azioni casuali; traduce input umani in richieste strutturate.

La prima funzione dei sistemi interattivi è la **selezione degli elementi**. In una simulazione MicroBot, l'utente deve poter selezionare un nodo, un edge, un cluster, un pattern, una fase o un evento. La selezione è la base dell'esplorazione. Senza selezione, la dashboard mostra soltanto una massa di dati. Con la selezione, l'utente può interrogare il sistema. Un click su un nodo può aprire un pannello con node_id, ruolo, stato, energia, temperatura, posizione, target, ultimi comandi, errori e capabilities. Un click su un edge può mostrare distance, expected/current status, criticality e failure_reason. Un click su un cluster può mostrare density, radius, isolated_nodes e stability_score.

Una selezione efficace deve essere contestuale. Se l'utente seleziona un nodo in modalità viewer, può vedere dati in sola lettura. Se lo seleziona in modalità developer, può vedere anche debug interno, task, eventi e resource status. Se lo seleziona in hardware mode, i comandi possibili devono essere limitati e soggetti a safety. Lo stesso gesto, cioè il click, può quindi produrre livelli diversi di dettaglio in base ad access_level e system_mode. Questo collega interazione, sicurezza biometrica e controllo accessi.

La seconda funzione è l'**ispezione in tempo reale**. Dopo aver selezionato un elemento, il sistema deve mostrare dati aggiornati. L'inspector non deve essere una scheda statica, ma un pannello vivo. Se il nodo cambia ruolo, l'inspector deve aggiornarsi. Se entra in low_energy, deve comparire il warning. Se un edge passa da missing a formed, il pannello deve cambiare. L'interattività deve essere sincronizzata con la telemetria. Questo richiede una buona architettura dati: lo stato globale dello swarm deve alimentare sia rendering sia pannelli.

Un node inspector può mostrare:

identità del nodo;

modalità operativa;

ruolo assegnato;

fase corrente;

posizione e target;

stato energetico;

stato termico;

stato safety;

capabilities disponibili;

ultimi eventi;

ultimi comandi;

diagnostic_status.

Questo pannello diventa uno degli strumenti principali di comprensione. L'utente non deve cercare nei log per sapere cosa succede a un nodo: clicca e vede. Questo rende MicroBot molto più presentabile.

La terza funzione è l'**hover telemetry**, cioè la visualizzazione rapida dei dati quando il mouse passa sopra un elemento. Questo è diverso dall'inspector completo. L'hover deve essere leggero e immediato. Passando sopra un nodo, il sistema può mostrare un piccolo tooltip con node_id, role, mode, energy, thermal, pattern_error locale. Passando sopra un edge, può mostrare distanza, stato e criticità. Passando sopra una fase nella timeline, può mostrare eventi principali. Questa interazione è molto utile perché permette esplorazione rapida senza aprire pannelli.

Un tooltip deve essere sintetico. Deve mostrare solo i dati più importanti. Il dettaglio completo resta nell'inspector. Questa distinzione evita sovraccarico visivo. Un sistema interattivo buono non mostra tutto sempre; mostra il livello giusto al momento giusto.

La quarta funzione è la **modifica controllata dei parametri**. In modalità simulazione, l'utente può voler cambiare parametri come cohesion_weight, separation_weight, node_count, target_shape, noise_level, communication_latency, energy_profile, thermal_model o fault_profile. Queste modifiche possono essere molto utili per capire come cambia il comportamento. Tuttavia, devono essere controllate. Non tutti i parametri possono essere modificati durante una run. Alcuni richiedono reset. Alcuni possono cambiare live. Alcuni sono disponibili solo in developer mode. Alcuni sono pericolosi se collegati a hardware.

Una classificazione dei parametri può essere:

live_safe_parameters;

simulation_only_parameters;

requires_restart_parameters;

developer_only_parameters;

hardware_locked_parameters;

admin_only_parameters.

Per esempio, visual_scale o show_edges possono cambiare sempre. cohesion_weight può cambiare live in simulazione, ma non su hardware reale senza policy. node_count può richiedere reset della simulazione. max_pwm_duty deve essere admin/hardware_admin e comunque limitato. Questa classificazione evita che l'interattività rompa la coerenza del sistema.

La quinta funzione è la **manipolazione diretta nella simulazione**. L'utente può trascinare nodi virtuali, spostare target, creare ostacoli, aggiungere fault, simulare un nodo offline, cambiare posizione iniziale, disegnare una forma o modificare un campo. Questa è una forma molto potente di interattività perché permette esperimenti rapidi. Tuttavia, deve essere chiaramente limitata alla simulazione. Se il sistema è in hardware mode, trascinare un nodo nella dashboard non può fisicamente spostare un MicroBot reale, a meno che esista un controllo attuativo validato. Quindi la UI deve distinguere virtual_edit e hardware_command.

Una manipolazione diretta può includere:

drag node in simulation;

drag target position;

draw formation;

add obstacle;

remove virtual node;

inject fault;

change cluster center;

move controller point;

change field intensity.

Queste funzioni sono utili per prototipazione. Permettono di vedere come lo swarm reagisce a disturbi o target diversi. Sono anche ottime per presentazioni, perché rendono il sistema vivo. Ma devono essere tracciate come eventi: user_moved_virtual_node, target_changed, fault_injected. Ogni modifica deve essere osservabile.

La sesta funzione è il **controllo della timeline**. Un sistema MicroBot interattivo deve permettere pause, resume, step, replay, slow motion, fast forward e jump_to_event. Questo è fondamentale per analisi. Una run può fallire in pochi secondi, e l'utente deve poter tornare indietro e vedere il momento esatto in cui un edge è mancato, un nodo è entrato in thermal_warning o un comando è stato rifiutato. La timeline trasforma la simulazione da evento effimero a materiale analizzabile.

Una timeline interattiva può contenere:

play;

pause;

step forward;
step backward, se replay supportato;
time_scale;
jump_to_phase;
jump_to_event;
event markers;
metric overlays;
selected_node_history.

Questa funzione collega giochi, telemetria e debug. È una delle caratteristiche più forti per una dashboard tecnica. Non basta vedere lo stato finale; bisogna poter navigare il processo.

La settima funzione è il **replay interattivo**. Il replay è diverso dalla run live. Nella run live, il sistema evolve in tempo reale. Nel replay, l'utente guarda una run registrata. Può fermarla, rallentarla, selezionare nodi e vedere metriche nel tempo. Questo è molto utile per debugging, documentazione, demo e confronto tra versioni. Un replay ben fatto può diventare una prova tecnica del comportamento MicroBot.

Un replay può usare:

snapshot registrati;
eventi registrati;
metriche time-series;
config iniziale;
pattern_id;
failure_reason;
report finale.

L'interattività nel replay deve permettere di aprire inspector su nodi anche nel passato. Per esempio, selezionando MB-004 al secondo 12, l'utente vede lo stato di quel nodo in quel momento. Questo richiede che i dati siano storicizzati. È una funzione avanzata, ma molto potente.

L'ottava funzione è il **confronto interattivo tra run**. L'utente può voler confrontare due configurazioni: prima e dopo una modifica, due pattern, due profili di comportamento, due set di parametri. Un sistema interattivo può mostrare run A e run B, metriche affiancate, differenze, grafici e report. Questo aiuta a capire se una modifica migliora davvero il sistema.

Un confronto può includere:

success/failure;
completion_time;

pattern_error;
topology_score;
safety_event_count;
energy_cost;
thermal_event_count;
fallback_count;
node_reliability;
recommendation.

L'interattività permette di filtrare: mostra solo eventi safety, mostra solo nodi problematici, confronta phase_attach, evidenzia differenze topologiche. Questo rende l'analisi molto più efficace di una tabella statica.

La nona funzione è la **gestione dei filtri**. In uno swarm con molti nodi ed eventi, non si può mostrare tutto. L'utente deve poter filtrare per ruolo, stato, energia, temperatura, errori, cluster, fase, severity, capability o access_level. I filtri sono una parte fondamentale dell'interattività. Permettono di ridurre complessità.

Filtri utili possono essere:

mostra solo nodi active;
mostra solo nodi in warning;
mostra solo edge missing;
mostra solo eventi critical;
filtra per phase_id;
filtra per role;
filtra per cluster;
filtra per energy_state;
filtra per thermal_state;
filtra per capability.

Questi filtri trasformano la dashboard in strumento di analisi. Senza filtri, la telemetria diventa rumore. Con filtri, l'utente vede ciò che conta.

La decima funzione è la **modalità focus**. L'utente può concentrarsi su un nodo, un cluster o una fase. In focus mode, la dashboard riduce tutto il resto e mostra solo gli elementi rilevanti. Per esempio, focus su un cluster mostra centroide, densità, nodi interni, edge interni e nodi isolati. Focus su un nodo mostra traiettoria, comandi, errori e vicini. Focus su una fase mostra eventi e metriche di quella fase.

Focus mode è importante perché MicroBot può diventare complesso. Non sempre serve la vista globale. A volte serve entrare dentro un dettaglio. Questa navigazione tra globale e locale è una caratteristica dei sistemi interattivi maturi.

L'undicesima funzione è la **modalità diagnostica interattiva**. In developer mode, l'utente può attivare overlay di debug: target positions, expected_edges, missing_edges, force vectors, neighbor radius, connection radius, phase boundaries, telemetry age, buffer status. Questi overlay devono essere attivabili e disattivabili. Non devono essere sempre visibili, perché renderebbero la dashboard caotica.

Overlay diagnostici utili:

node_id labels;

role labels;

target markers;

velocity vectors;

expected edges;

missing edges;

communication links;

safety rings;

thermal aura;

energy indicators;

cluster boundaries;

field vectors.

Questi strumenti rendono il sistema molto più facile da sviluppare. Sono l'equivalente tecnico delle hitbox nei giochi. L'utente vede ciò che normalmente è invisibile.

La dodicesima funzione è la **interazione tramite camera e gesture**. Nei progetti esterni possono esistere sistemi con camera, hand tracking, AR o gesture. Questi possono essere collegati a MicroBot come modalità di interazione avanzata. Per esempio, una mano può selezionare un nodo virtuale, spostare una formazione, ruotare un modello 3D, esplodere un componente o ispezionare una parte. Questo tipo di interazione è molto coerente con una demo futuristica di MicroBot.

Una pipeline gesture può essere:

camera input;

hand detection;

landmark mano;

gesture recognition;

mapping gesture → azione;

access_control check;

simulation action o UI action;

event logging.

Come per la biometria, le gesture non devono bypassare safety. Una gesture può modificare una simulazione o navigare una UI. Se deve inviare comandi hardware, serve autorizzazione, conferma e Safety Layer. La separazione resta fondamentale.

La tredicesima funzione è la **interazione AR/3D**. MicroBot può essere rappresentato in 3D: nodi, connessioni, campi, strutture, componenti, telemetria spaziale. Un sistema interattivo 3D permette rotazione, zoom, esplosione del modello, ispezione dei componenti, highlight delle connessioni e visualizzazione di campi. Questo è molto utile per spiegare architettura, hardware e swarm. I progetti esterni in Three.js, WebGL o AR possono contribuire direttamente a questa parte.

Una visualizzazione 3D interattiva può includere:

nodi come sfere o moduli;

edge come linee o canali;

campo magnetico come vettori;

telemetria come etichette;

componenti esplosi;

selezione tramite click;

rotazione camera;

zoom;

modalità wireframe;

modalità thermal view.

Questa funzione non è necessaria al core, ma ha grande valore per demo, portfolio e presentazione. Permette di far vedere MicroBot come sistema fisico e informativo.

La quattordicesima funzione è la **interazione educativa**. Un sistema interattivo può guidare l'utente attraverso concetti complessi. MicroBot può avere tutorial: "clicca su un nodo per vedere il suo stato", "attiva line_basic", "osserva come si forma un edge", "aumenta il rumore e guarda cosa succede", "simula un nodo offline", "osserva la recovery". Questa modalità rende il progetto comprensibile anche a chi non conosce swarm robotics.

Una modalità educativa può avere:

step guidati;

spiegazioni contestuali;

highlight degli elementi;

obiettivi tecnici;

mini-esperimenti;

domande di verifica;

replay commentati;

modalità semplificata.

Questa è molto utile per scuole, università, pitch e materiale divulgativo. Non sostituisce la documentazione tecnica, ma la rende esperibile.

La quindicesima funzione è la **interazione attraverso comandi testuali**. Oltre alla UI grafica, un sistema interattivo può avere una console o command palette. L'utente scrive comandi come start line_basic, show node MB-004, filter thermal_warning, replay last_run, export report. Questo è potente per developer. È simile alla shell interna di MicroBot OS, ma a livello dashboard o controller.

Una command palette può supportare:

search node;

start simulation;

load run;

filter events;

show diagnostics;

export report;

toggle overlay;

jump to event;

set time scale.

Questa interazione testuale deve essere validata. Non deve permettere comandi non autorizzati. Ogni comando deve passare da access_control. Ma può rendere il lavoro molto più veloce.

La sedicesima funzione è la **interazione vocale**, come direzione futura. Un sistema MicroBot potrebbe ricevere comandi vocali: “avvia simulazione”, “mostra nodi in errore”, “apri telemetria del nodo 4”, “ferma demo”. Tuttavia, la voce introduce rischi: riconoscimento errato, comandi accidentali, identità non verificata. Per questo, nelle prime versioni deve restare future direction o modalità demo read-only. Comandi hardware tramite voce richiederebbero conferma e access_control forte.

Una pipeline vocale sicura dovrebbe essere:

speech input;

speech-to-text;

intent recognition;

confirmation;

access_control;

command generation;

Safety Layer.

Senza conferma, la voce non deve attivare funzioni critiche. Può essere utile per query e navigazione, non per attuazione diretta.

La diciassettesima funzione è la **interazione con sensori fisici**. Alcuni progetti esterni possono usare sensori di prossimità, movimento, luce, suono o input fisici. Questi possono essere collegati a MicroBot come trigger o strumenti dimostrativi. Per esempio, un sensore di prossimità può attivare una visualizzazione, un allarme o un evento. Un sensore può simulare ambiente. Tuttavia, ogni input fisico deve essere trattato come segnale, non come comando diretto. Serve validazione e mapping.

Una pipeline sensore può essere:

sensor reading;

filtering;

threshold;

event generation;

controller interpretation;

safe action;

telemetry logging.

Questo è simile a biometria e gesture: input esterno → interpretazione → policy → azione. MicroBot deve mantenere questa disciplina.

La diciottesima funzione è la **interazione con la sicurezza biometrica**. La Sezione 15 ha definito autenticazione tramite camera. Nei sistemi interattivi, questa biometria diventa parte dell'esperienza. L'utente autenticato vede controlli diversi. La dashboard cambia badge. Le azioni si abilitano o disabilitano. La sessione scade. Una richiesta critica può chiedere re-auth. Questo rende la sicurezza visibile e integrata, non nascosta.

Un sistema interattivo deve mostrare:

access_level;

session_status;

controls enabled/disabled;

reason for disabled controls;

security events;

reauth prompts;

logout/manual lock.

Questo è importante perché sicurezza invisibile genera confusione. Se un pulsante è bloccato, l'utente deve sapere se manca accesso o se il nodo è in safety.

La diciannovesima funzione è la **interazione con il Safety Layer**. Ogni sistema interattivo deve rendere chiaro quando un'azione è bloccata per sicurezza. Se l'utente prova a inviare un comando e il nodo lo rifiuta per thermal_lock, la UI deve mostrarlo. Se critical_energy blocca output, il pulsante deve essere disabilitato o il comando deve tornare con reason. La safety deve essere una presenza visibile.

Esempi di feedback safety:

command denied: thermal_lock;

command denied: low_energy;

command denied: cooldown_active;

command denied: access_level_insufficient;

command denied: node_offline;

command denied: capability_missing;

command denied: recovery_required.

Questi messaggi devono essere tecnici ma chiari. Non basta "errore". Bisogna dire perché. Questo rende il sistema professionale.

La ventesima funzione è la **interazione tramite stati visuali**. Un buon sistema interattivo comunica lo stato anche senza testo. Colori, icone, linee, opacità, animazioni, bordi e lampeggi possono indicare active, warning, critical, selected, locked, stale, offline, simulated, hardware, hybrid. Questo è molto utile in MicroBot, dove ci sono molti elementi.

Stati visuali possibili:

nodo active;

nodo idle;

nodo safety;

nodo offline;

nodo stale;

nodo low_energy;

nodo thermal_warning;

edge expected;

edge formed;

edge missing;

cluster selected;

hardware node;

simulated node;

hybrid node.

La visualizzazione deve essere coerente e documentata. Se rosso indica critical, deve indicarlo ovunque. Se tratteggio indica missing, deve essere costante. Questa coerenza è parte dell'usabilità.

La ventunesima funzione è la **interazione con dati tabellari**. Non tutto deve essere grafico. La dashboard può avere tabelle interattive: nodi, eventi, run, metriche, profili, capabilities, errori. Le tabelle devono essere filtrabili, ordinabili e cliccabili. Una riga nodo può aprire l'inspector. Una riga evento può saltare al momento nella timeline. Una riga run può aprire report. Questa interazione collega dati e visualizzazione.

Tabelle utili:

node table;

event table;

run history;

pattern table;

capability table;

safety events;

security events;

diagnostic table.

Le tabelle sono particolarmente utili quando lo swarm cresce. La visualizzazione spaziale mostra forma; la tabella mostra dettagli ordinati. Entrambe servono.

La ventiduesima funzione è la **interazione con grafici**. MicroBot produce metriche nel tempo: pattern_error, topology_score, aggregation_score, thermal_load, energy, node_count, event_rate, latency. Grafici interattivi permettono zoom, hover, selezione, evidenziazione di eventi, confronto tra run. Questo è uno strumento analitico fondamentale.

Grafici utili:

pattern_error over time;

topology_score over time;

energy per node;

thermal_load per node;

event count over time;

latency/jitter;

cluster_count over time;

success_rate by pattern.

L'interazione deve collegare grafico e simulazione. Se l'utente clicca un picco di thermal_load, la timeline può saltare a quel momento e selezionare il nodo coinvolto. Questo tipo di collegamento rende la dashboard davvero potente.

La ventitreesima funzione è la **interazione con report**. Un report non deve essere solo un documento statico. Può avere sezioni espandibili, link agli eventi, replay collegati, nodi cliccabili, grafici dinamici, confronto con run precedenti. Questo rende il report tecnico più utile. Un report interattivo può essere esportato in forma statica, ma nella dashboard resta esplorabile.

Un report interattivo può includere:

summary cliccabile;

failure reason;

metriche espandibili;

timeline eventi;

node diagnostics;

recommendations;

link a replay;

link a config;

comparison with baseline.

Questa funzione unisce documentazione e sistema. MicroBot può diventare non solo progetto, ma piattaforma di analisi.

La ventiquattresima funzione è la **interazione con modalità di presentazione**. In una demo a un collaboratore o investitore, non serve mostrare ogni pannello tecnico. Serve una modalità presentation: vista pulita, pochi controlli, metriche chiave, animazioni chiare, spiegazione step-by-step, niente comandi pericolosi. Questa modalità può essere derivata dalla dashboard interattiva ma semplificata.

Presentation mode può mostrare:

titolo pattern;

stato run;

nodi e connessioni;

metriche chiave;

fase corrente;

success/failure;

spiegazione testuale;

nessun controllo critico;

solo start/stop demo sicuri.

Questa modalità è molto importante per comunicare MicroBot. Il sistema tecnico può essere complesso, ma la demo deve essere leggibile. I sistemi interattivi permettono di avere entrambe le viste: tecnica e presentazione.

La venticinquesima funzione è la **interazione in modalità sviluppo**. Al contrario della presentation mode, developer mode deve mostrare più strumenti: debug overlay, log, console, filtri, parametri, inspector, errori, scheduler, raw telemetry. Questa modalità è per costruire e correggere il sistema. Deve essere protetta da access_control, ma è essenziale per lo sviluppo.

Developer mode può includere:

raw events;

command log;

node diagnostics;

scheduler diagnostics;

telemetry schema;

config editor;

fault injection;

simulation parameters;

debug overlays;

export diagnostic snapshot.

Questa modalità deve essere potente ma non pericolosa. Le funzioni hardware devono restare separate e safety-limited.

La ventiseiesima funzione è la **interazione attraverso fault injection**. Un sistema interattivo può permettere di simulare errori: nodo offline, ritardo rete, pacchetto perso, low_energy, thermal_warning, role_mismatch, edge failure. Questo è molto utile per testare recovery e Safety Layer. L'utente può attivare un fault e vedere come il sistema reagisce. Questa funzione deve essere chiaramente simulation_only o developer_only.

Fault interattivi possibili:

set node offline;

inject latency;

drop messages;

force low_energy;

force thermal_warning;

remove edge;

change role;

trigger watchdog reset simulation;

disable telemetry.

Questi fault aiutano a validare il sistema. Non sono “rompere per caso”; sono test controllati. Ogni fault deve essere registrato come event con source = user_injected.

La ventisettesima funzione è la **interazione con configurazioni e preset**. L’utente può scegliere preset: small_swarm_demo, line_basic_test, cluster_fault_test, thermal_stress_sim, hybrid_node_demo. I preset rendono facile avviare scenari complessi senza impostare tutto manualmente. Sono molto utili per demo e test ripetibili.

Un preset può contenere:

pattern_id;

node_count;

initial_positions;

behavior_profile;

safety_policy;

fault_profile;

telemetry_level;

visual_mode;

expected_result.

L’interfaccia può mostrare preset come card. Ogni preset indica rischio, modalità, durata e scopo. Questo rende il sistema più ordinato.

La ventottesima funzione è la **interazione con i limiti del sistema**. Un buon sistema interattivo non deve solo permettere azioni; deve anche spiegare cosa non è possibile. Se un pattern richiede 12 nodi ma ce ne sono 8, la UI deve dirlo. Se hardware non supporta PWM, deve dirlo. Se la sessione

non ha permesso, deve dirlo. Se la simulazione è in replay, certi comandi live non hanno senso. Questo evita errori.

Esempi di blocchi spiegati:

insufficient_nodes;

capability_missing;

simulation_paused;

replay_mode_read_only;

hardware_not_connected;

access_level_insufficient;

safety_lock;

config_invalid;

pattern_not_supported.

Questa trasparenza rende la dashboard molto più professionale. L'utente non deve indovinare perché qualcosa non funziona.

La ventinovesima funzione è la **roadmap dei sistemi interattivi**. L'interattività deve crescere per livelli. Prima selezione e inspector. Poi filtri. Poi grafici. Poi timeline. Poi replay. Poi manipolazione diretta. Poi fault injection. Poi gesture/AR. Poi modalità educativa e presentation. Questo evita di costruire subito una UI enorme.

Una roadmap possibile è:

v0.1: dashboard read-only con stato swarm;

v0.2: selezione nodo e inspector;

v0.3: hover telemetry;

v0.4: filtri per stato e ruolo;

v0.5: grafici metriche base;

v0.6: timeline eventi;

v0.7: pause/resume/step simulation;

v0.8: replay interattivo;

v0.9: debug overlays;

v1.0: fault injection simulation-only;

v1.1: comparison between runs;

v1.2: presentation mode;

v1.3: educational guided mode;

v1.4: gesture/AR interaction;

v1.5: advanced command palette.

Questa roadmap rende l'evoluzione gestibile e coerente con il resto dell'ecosistema.

Il trentesimo punto è la **sintesi del ruolo dei sistemi interattivi**. L'interattività è ciò che rende MicroBot esplorabile. Senza interattività, il sistema può essere tecnicamente valido ma difficile da capire. Con interattività, ogni parte diventa interrogabile: nodi, edge, pattern, cluster, eventi, metriche, run, safety, accessi, report. L'utente non guarda soltanto; partecipa all'analisi. Questo è essenziale per sviluppo, presentazione, didattica e validazione.

In conclusione, i sistemi interattivi rappresentano un ambito fondamentale dei progetti esterni collegabili a MicroBot. Essi portano competenze in selezione, ispezione, hover telemetry, manipolazione diretta, timeline, replay, filtri, focus mode, debug overlay, gesture, AR/3D, modalità educativa, comandi testuali, grafici, report dinamici, presentazione, developer mode, fault injection e preset. Queste funzioni trasformano MicroBot da simulazione osservata a piattaforma esplorabile e controllabile. Tuttavia, ogni interazione deve essere disciplinata: deve rispettare access_control, Safety Layer, stato del sistema, modalità hardware/simulazione e tracciabilità degli eventi. Il punto successivo, dedicato alle simulazioni evolutive, analizzerà come i sistemi che cambiano nel tempo, si adattano, selezionano comportamenti e generano configurazioni emergenti possano ispirare l'evoluzione dello swarm MicroBot e la ricerca di pattern più robusti.

16.5 Simulazioni evolutive

Le **simulazioni evolutive** rappresentano il quinto ambito dei progetti esterni collegabili all'ecosistema MicroBot e riguardano quei sistemi in cui una popolazione di entità, configurazioni o comportamenti viene fatta evolvere nel tempo attraverso variazione, selezione, adattamento e valutazione. Dopo aver analizzato giochi e sistemi interattivi, cioè ambienti dinamici in cui l'utente può osservare e modificare il comportamento del sistema, le simulazioni evolutive introducono un concetto ulteriore: non solo il sistema si muove e reagisce, ma può anche migliorare, trasformarsi o cercare soluzioni migliori attraverso molte prove successive. Questo tipo di simulazione è particolarmente interessante per MicroBot perché lo swarm stesso è un sistema collettivo, composto da molte unità locali che possono produrre comportamenti globali complessi.

Nel contesto MicroBot, una simulazione evolutiva non deve essere interpretata come un elemento già obbligatorio del core. MicroBot può esistere e funzionare anche senza evoluzione automatica. Tuttavia, le simulazioni evolutive offrono una direzione molto potente per il futuro: invece di progettare manualmente ogni parametro, ogni pattern, ogni regola locale e ogni strategia di aggregazione, il sistema potrebbe testare molte configurazioni in simulazione, valutarle attraverso metriche oggettive e selezionare quelle più efficaci. In altre parole, la simulazione evolutiva può diventare uno strumento di ricerca per scoprire comportamenti, non solo per visualizzare comportamenti già decisi.

Il principio generale può essere espresso così:

le simulazioni evolutive permettono di esplorare automaticamente molte varianti di comportamento MicroBot, selezionando quelle che producono strutture più stabili, efficienti e sicure.

Questa frase contiene la funzione principale del modulo. Una simulazione classica prende una configurazione e mostra cosa succede. Una simulazione evolutiva genera molte configurazioni, le testa, le confronta e conserva quelle migliori. Questo è molto utile quando lo spazio dei parametri è grande. In MicroBot, anche una piccola regola locale può avere molti parametri: peso di coesione, peso di separazione, forza verso target, damping, raggio di connessione, tolleranza di edge, velocità massima, soglie di stabilizzazione, tempo di cooldown, gestione dei nodi isolati. Provare tutto manualmente sarebbe lento. Un sistema evolutivo può automatizzare la ricerca.

La prima componente di una simulazione evolutiva è la **popolazione iniziale**. Una popolazione è un insieme di individui. In MicroBot, un individuo non deve essere per forza un MicroBot fisico. Può essere una configurazione di parametri, una regola di comportamento, un pattern, una policy di assegnazione ruoli, una strategia di recovery o un profilo di simulazione. Per esempio, una popolazione può contenere cento diverse combinazioni di parametri per formare una linea o un cluster. Ogni combinazione viene testata e valutata.

Un individuo evolutivo MicroBot può rappresentare:

un set di parametri di movimento;

un behavior_profile;

una configurazione di pattern;

una policy di role assignment;

una strategia di fallback;

una combinazione di soglie;

una configurazione di energia e cooldown;

una regola locale di aggregazione.

Questa distinzione è importante. Non si evolve necessariamente l'hardware. Si evolvono le regole o i parametri che governano il comportamento dello swarm nella simulazione. L'hardware reale entra eventualmente dopo, quando una configurazione è stata validata e considerata sicura.

La seconda componente è il **genotipo**, cioè la rappresentazione interna dell'individuo. In un algoritmo evolutivo, il genotipo è la codifica dei parametri che possono cambiare. Nel caso MicroBot, un genotipo può essere un vettore di valori: cohesion_weight, separation_weight, target_weight, max_speed, connection_radius, damping, stabilization_time. Ogni valore influenza il comportamento della simulazione. Il sistema evolutivo modifica questi valori e osserva l'effetto sullo swarm.

Un genotipo MicroBot semplice può contenere:

cohesion_weight;

separation_weight;

alignment_weight;

target_attraction_weight;

edge_maintenance_weight;

max_velocity;

damping_factor;

connection_radius;

minimum_spacing;

stabilization_threshold;

retry_limit;

cooldown_factor.

Questi parametri non sono casuali: corrispondono a concetti già presenti nel sistema. Coesione avvicina i nodi. Separazione evita sovrapposizione. Target attraction porta i nodi verso una forma desiderata. Edge maintenance mantiene connessioni. Damping riduce oscillazioni. Connection radius definisce quando un edge può formarsi. Stabilization threshold definisce quando il pattern è considerato stabile. Il genotipo diventa quindi una configurazione comportamentale.

La terza componente è il **fenotipo**, cioè il comportamento osservabile generato dal genotipo. Due configurazioni di parametri possono produrre comportamenti molto diversi. Una può creare convergenza rapida ma instabile. Un'altra può essere lenta ma precisa. Un'altra può creare collisioni. Un'altra può formare cluster troppo densi. Il fenotipo, in MicroBot, è ciò che la simulazione mostra e misura: traiettorie, pattern finale, metriche, errori, stabilità, safety events e tempi.

Il fenotipo MicroBot può essere osservato attraverso:

forma finale ottenuta;

tempo di convergenza;

stabilità nel tempo;

numero di edge formati;

numero di edge mancanti;

oscillazioni;

collision_count;

energy_cost stimato;

thermal_load stimato;

safety_event_count;

fallback_count;

failure_reason.

La simulazione evolutiva non valuta il genotipo direttamente. Valuta il fenotipo. Una configurazione è buona solo se produce un comportamento buono secondo criteri definiti. Questo è un punto fondamentale.

La quarta componente è la **fitness function**, cioè la funzione che assegna un punteggio a ogni individuo. Nel progetto MicroBot, la fitness function deve essere costruita con molta attenzione, perché determina cosa il sistema considera “migliore”. Se si premia solo la velocità, il sistema può trovare soluzioni rapide ma instabili o insicure. Se si premia solo la precisione geometrica, può trovare soluzioni lentissime. Se si ignora la safety, può produrre comportamenti non accettabili. La fitness deve quindi combinare più obiettivi.

Una fitness function MicroBot può considerare:

pattern_error basso;

topology_score alto;

stability_score alto;

completion_time basso;

collision_count basso;

energy_cost basso;

thermal_event_count basso;

safety_event_count basso;

fallback_count basso;

recovery_success alto;

node_isolation_count basso.

Una formula concettuale potrebbe premiare successo, stabilità e topologia, penalizzando energia, collisioni e safety events. Non è necessario definire subito una formula matematica rigida, ma è importante dichiarare il principio: la migliore configurazione non è solo quella che forma la figura, ma quella che la forma in modo stabile, efficiente e sicuro. Questo è coerente con tutto il progetto MicroBot.

La quinta componente è la **selezione**. Dopo aver testato una popolazione di configurazioni, il sistema seleziona quelle con fitness migliore. Queste configurazioni diventano base per la generazione successiva. Il principio è simile alla selezione naturale: gli individui più adatti hanno maggiore probabilità di trasmettere le proprie caratteristiche. Nel caso MicroBot, una configurazione che produce una griglia stabile e con pochi errori ha più probabilità di essere mantenuta rispetto a una che produce frammentazione o oscillazioni.

La selezione può essere:

selezione dei migliori N individui;

selezione proporzionale alla fitness;

torneo tra individui;

selezione con elitismo, mantenendo sempre i migliori;

selezione multi-obiettivo, mantenendo configurazioni diverse ma valide.

Per MicroBot, è importante non mantenere solo una configurazione. A volte una configurazione è migliore per velocità, un'altra per stabilità, un'altra per energia. Una selezione multi-obiettivo può conservare più soluzioni interessanti. Questo è utile perché MicroBot può avere modalità diverse: demo veloce, demo stabile, simulazione scientifica, hardware-safe.

La sesta componente è la **mutazione**. La mutazione modifica casualmente alcuni parametri di una configurazione. Per esempio, aumenta leggermente cohesion_weight, riduce max_velocity, cambia connection_radius, aumenta damping. La mutazione serve a esplorare nuove soluzioni. Senza mutazione, il sistema resterebbe bloccato sulle configurazioni iniziali. Tuttavia, la mutazione deve essere limitata da range sicuri. Non deve generare valori assurdi o pericolosi.

Esempi di mutazione MicroBot:

cohesion_weight + 5%;

separation_weight - 3%;

connection_radius leggermente aumentato;

max_velocity ridotta;

damping_factor modificato;

stabilization_threshold più severa;

retry_limit aumentato di uno;

cooldown_factor aumentato.

Ogni parametro deve avere limiti. Per esempio, max_velocity non può diventare negativa. connection_radius non può essere infinito. cooldown non può scendere sotto una soglia di sicurezza. In simulazione si può esplorare molto, ma se una configurazione è candidata per hardware deve rispettare limiti conservativi.

La settima componente è il **crossover**, cioè la combinazione di due configurazioni. Se una configurazione è molto stabile e un'altra è molto veloce, il sistema può combinarne parti per cercare una soluzione che abbia entrambe le qualità. Per esempio, prende cohesion_weight e damping da un individuo, connection_radius e target_weight da un altro. Il crossover è utile quando i parametri hanno effetti complementari. Anche qui, bisogna validare i range dopo la combinazione.

Un crossover MicroBot può combinare:

parametri di movimento da individuo A;

parametri di connessione da individuo B;

parametri di stabilizzazione da individuo A;

parametri di cooldown da individuo B.

Il risultato è un nuovo behavior_profile. Questo profilo viene simulato e valutato. Se funziona bene, entra nella popolazione successiva. Se funziona male, viene scartato. Il processo può ripetersi per molte generazioni.

L'ottava componente è la **generazione successiva**. Una simulazione evolutiva procede per cicli. Ogni ciclo genera una popolazione, testa gli individui, calcola fitness, seleziona i migliori, produce variazioni e ripete. Nel tempo, se la fitness è ben progettata, la popolazione dovrebbe migliorare. Per MicroBot, una generazione può rappresentare un ciclo di esperimenti simulati. Ogni individuo produce una run. Ogni run produce report. Il sistema confronta e sceglie.

Una generazione evolutiva può contenere:

generation_id;

population_size;

individuals;

simulation_runs;

fitness_results;

selected_individuals;

mutations_applied;

crossover_applied;

best_individual;

average_fitness;

failure_distribution.

Questi dati devono essere salvati se si vuole analizzare il processo. La simulazione evolutiva non deve essere una scatola nera. Deve produrre report e log. Altrimenti non si capisce perché un comportamento è stato scelto.

La nona componente è la **gestione dei vincoli**. In MicroBot, non tutte le soluzioni generate sono accettabili. Alcune possono violare vincoli di sicurezza, energia, temperatura, hardware, comunicazione o topologia. Un algoritmo evolutivo potrebbe trovare una soluzione che funziona in simulazione ma richiede velocità impossibili, energia troppo alta o cooldown troppo basso. Per questo la fitness deve includere penalità e il sistema deve avere constraints rigidi. Alcuni vincoli non sono negoziabili.

Vincoli non negoziabili possono includere:

nessun output oltre limite massimo;

cooldown minimo rispettato;

thermal_lock non ignorato;

critical_energy blocca attuazione;
no collisioni sotto soglia critica;
max_velocity entro limite;
connection_radius entro range realistico;
nodi non possono teletrasportarsi;
timeout massimo della run;
Safety Layer sempre attivo.

Questi vincoli impediscono che l'evoluzione trovi scorciatoie non realistiche. Una simulazione evolutiva senza vincoli può produrre soluzioni apparentemente buone ma fisicamente inutili. MicroBot deve evitare questo.

La decima componente è la **simulazione ripetibile**. Per valutare una configurazione, la simulazione deve essere riproducibile. Se ogni run usa condizioni casuali diverse senza registrarle, non si può sapere se una configurazione è davvero migliore o solo fortunata. Serve seed, initial_state, pattern_id, fault_profile, noise_level e versioni del simulatore. Ogni individuo deve essere testato in condizioni documentate. Questo collega simulazioni evolutive e metodologia sperimentale.

Un esperimento evolutivo deve registrare:

simulation_seed;
initial_swarm_state;
pattern_id;
behavior_profile_id;
fault_profile_id;
simulation_version;
metric_schema_version;
fitness_function_version;
run_id;
generation_id;
individual_id.

Questi dati rendono le prove verificabili. Se una configurazione risulta eccellente, bisogna poterla ripetere. Se non è ripetibile, non è affidabile.

L'undicesima componente è la **valutazione su più scenari**. Una configurazione può funzionare bene in un solo scenario ma fallire in altri. Per esempio, può formare una linea con 10 nodi ma fallire con 20. Può funzionare senza rumore ma fallire con un nodo offline. Può funzionare in

cluster ma non in griglia. Per questo, le configurazioni evolute devono essere testate su più scenari prima di essere considerate robuste.

Scenari di test possono includere:

node_count diverso;

posizioni iniziali diverse;

rumore diverso;

fault_profile con nodo offline;

latenza comunicativa;

energy constraints;

thermal constraints;

pattern diversi;

densità diversa;

presenza di ostacoli virtuali.

Una configurazione robusta non è quella che vince un singolo scenario, ma quella che mantiene buone prestazioni in condizioni diverse. Questo è particolarmente importante se si vuole avvicinare il sistema all'hardware reale.

La dodicesima componente è la **multi-objective optimization**. MicroBot ha molti obiettivi: precisione, stabilità, velocità, energia, safety, topologia, robustezza. Questi obiettivi possono essere in conflitto. Aumentare velocità può ridurre stabilità. Aumentare coesione può aumentare collisioni. Aumentare separazione può rendere più difficile formare edge. Ridurre cooldown può migliorare tempo ma peggiorare safety. Per questo una fitness singola può essere troppo semplice. È utile pensare in termini multi-obiettivo.

Obiettivi possibili:

minimizzare pattern_error;

massimizzare topology_score;

massimizzare stability_score;

minimizzare completion_time;

minimizzare energy_cost;

minimizzare thermal_risk;

minimizzare safety_event_count;

massimizzare robustness_score.

Il risultato non è sempre un unico migliore assoluto. Può essere un insieme di configurazioni Pareto-efficienti: una più veloce, una più sicura, una più stabile. MicroBot può poi scegliere in base alla modalità: `presentation_mode` usa configurazione stabile e visuale; `research_mode` usa configurazione sperimentale; `hardware_mode` usa configurazione conservativa.

La tredicesima componente è la **distinzione tra evoluzione offline e online**. L'evoluzione offline avviene prima della run reale, su simulazioni salvate o ambiente di test. L'evoluzione online avviene durante il comportamento del sistema, adattando parametri in tempo reale. Per MicroBot, nelle prime versioni, l'evoluzione deve essere offline. È molto più sicura. Il sistema testa molte configurazioni in simulazione, sceglie le migliori e poi eventualmente le propone. L'evoluzione online, soprattutto su hardware, è molto più delicata e deve essere futura.

La distinzione è:

offline evolution = ricerca parametri in simulazione, senza rischio hardware;

online adaptation = modifica durante run, solo con limiti e validazione;

hardware evolution = non consigliata nelle prime versioni;

simulation-only evolution = consigliata come primo passo.

Questa regola è fondamentale. Non si deve lasciare che un algoritmo evolutivo modifichi parametri hardware reali senza controlli. La simulazione è il laboratorio sicuro.

La quattordicesima componente è la **evoluzione di pattern**. Non si devono evolvere solo parametri; si possono evolvere anche pattern. Per esempio, il sistema può provare diverse strategie per formare un bridge: prima avvicinare cluster, poi selezionare connector, poi stabilizzare edge; oppure prima selezionare leader, poi distribuire terminal, poi chiudere connessione. Ogni strategia può essere valutata. Questa è una forma più avanzata di evoluzione strutturale.

Un pattern evolutivo può variare:

ordine delle fasi;

criteri di assegnazione ruoli;

numero di nodi connector;

distanza target;

tolleranza edge;

strategia di fallback;

durata stabilizzazione;

policy di retry.

Questo permette di scoprire procedure più robuste. Tuttavia, evolvere pattern è più complesso che evolvere parametri. Va fatto dopo aver stabilizzato i modelli base.

La quindicesima componente è la **evoluzione delle regole locali**. Lo swarm MicroBot può essere governato da regole locali. Per esempio: avvicinarti ai vicini, evita sovrapposizione, mantieni edge,

seguì target. Una simulazione evolutiva può cercare combinazioni di regole o pesi che producono comportamento emergente desiderato. Questa è una direzione molto coerente con swarm intelligence. L'idea è che non si comanda ogni nodo in modo rigido, ma si evolvono regole locali da cui emerge una forma globale.

Regole locali evolvibili possono includere:

peso di attrazione verso target;

peso di attrazione verso cluster;

peso di repulsione dai vicini;

peso di mantenimento edge;

raggio di percezione;

soglia di cambio ruolo;

regola per nodo isolato;

regola per recupero edge;

regola per riduzione energia.

Questa direzione è molto potente ma richiede metriche buone. Senza metriche, non si sa cosa selezionare. Per questo telemetria, clustering e modelli sono prerequisiti.

La sedicesima componente è la **evoluzione delle strategie di recovery**. MicroBot deve gestire fallimenti: nodi offline, edge mancanti, pattern instabile, energia bassa, termica alta. Una simulazione evolutiva può confrontare strategie di recovery: retry, role reassignment, fallback pattern, esclusione nodo, riduzione target, estensione tempo. Ogni strategia può essere valutata in base a recovery_success, tempo, energia e safety.

Strategie di recovery evolvibili:

retry stesso comando;

assegna ruolo a nodo alternativo;

riduci complessità pattern;

attiva fallback line/cluster;

escludi nodo instabile;

aumenta stabilization_time;

riduci velocità;

richiedi state_sync;

passa a safe_idle.

Questa è un'applicazione molto concreta. Invece di decidere manualmente quale recovery è migliore, il sistema può testare molte possibilità in simulazione. Naturalmente, in hardware reale si applicano solo strategie validate.

La diciassettesima componente è la **evoluzione di configurazioni per modalità demo**. Una demo deve essere stabile, visivamente chiara e rapida. Una simulazione evolutiva può cercare configurazioni che massimizzano leggibilità e affidabilità, non solo performance tecnica. Per esempio, una configurazione con movimento più lento ma più pulito può essere migliore per presentazione. Una configurazione con edge visualmente ordinati può essere più comunicativa. Questo è un uso molto intelligente dell'evoluzione.

Fitness per demo può includere:

visual_clarity_score;

stable_motion;

low_oscillation;

completion_time accettabile;

few_warnings;

clear_pattern_shape;

smooth_transitions;

low_event_noise.

Questo dimostra che la fitness può cambiare secondo obiettivo. Research mode e presentation mode possono usare criteri diversi.

La diciottesima componente è la **evoluzione per robustezza**. Una configurazione veramente buona deve resistere a disturbi. Per testare robustezza, si possono introdurre fault: nodo offline, rumore posizione, latenza, energy drop, edge failure. La fitness deve premiare configurazioni che mantengono successo anche con disturbi. Questo è molto importante per MicroBot, perché hardware reale sarà sempre più rumoroso della simulazione perfetta.

Robustness testing può includere:

random initial positions;

random node failure;

communication delay;

sensor noise;

energy degradation;

thermal constraints;

role mismatch;

edge dropout;

controller delay.

Una configurazione robusta è quella che non collassa subito. Questo è più importante della performance perfetta in condizioni ideali.

La diciannovesima componente è la **evoluzione e interpretabilità**. Gli algoritmi evolutivi possono trovare soluzioni strane. Alcune funzionano ma sono difficili da capire. Per MicroBot, soprattutto in documentazione e tesi, è importante interpretare le soluzioni. Se una configurazione evolve, bisogna capire quali parametri sono cambiati e perché potrebbe funzionare meglio. Questo permette di trasformare evoluzione in conoscenza, non solo in risultato numerico.

Un report evolutivo dovrebbe spiegare:

parametri della configurazione migliore;

differenze rispetto alla baseline;

metriche migliorate;

metriche peggiorate;

possibile motivo del miglioramento;

limiti del test;

scenari in cui fallisce.

Questo rende l'evoluzione utile anche didatticamente. Non è solo “il computer ha trovato una soluzione”, ma “abbiamo capito che aumentare damping e ridurre max_speed migliora stabilità del pattern circle”.

La ventesima componente è la **evoluzione e dataset delle generazioni**. Ogni generazione produce molti dati. Questi dati possono diventare dataset per analisi successiva. Si possono visualizzare curve di fitness nel tempo, diversità della popolazione, distribuzione dei failure, parametri più frequenti nei migliori individui. Questo collega simulazioni evolutive, clustering e AI.

Metriche evolutive utili:

best_fitness_per_generation;

average_fitness_per_generation;

fitness_variance;

diversity_score;

failure_category_distribution;

parameter_distribution;

convergence_rate;

stagnation_count.

Se la fitness smette di migliorare, il sistema può essere in stagnazione. Se la diversità è troppo bassa, la popolazione si è uniformata. Queste metriche aiutano a controllare il processo evolutivo.

La ventunesima componente è la **gestione della stagnazione**. Un algoritmo evolutivo può bloccarsi su una soluzione mediocre. Se tutte le configurazioni diventano simili, non esplora più. Per evitarlo, si può aumentare mutazione, introdurre nuovi individui casuali, cambiare scenario o usare selezione meno aggressiva. Questo è importante se si vuole usare evoluzione seriamente.

Strategie anti-stagnazione:

aumentare `mutation_rate`;

iniettare individui random;

mantenere diversità;

cambiare seed/scenario;

usare multi-objective selection;

conservare soluzioni diverse;

riavviare popolazione parziale.

Nel contesto MicroBot, questo può essere utile perché lo spazio delle configurazioni è complesso. Una soluzione localmente buona non è necessariamente la migliore.

La ventiduesima componente è la **gestione dei costi computazionali**. Simulare molte configurazioni può richiedere tempo. Ogni individuo genera una run; ogni generazione contiene molti individui; molte generazioni producono centinaia o migliaia di run. Questo può essere pesante. Per questo, l'evoluzione deve essere offline e possibilmente con simulazioni leggere. Si possono usare run brevi per valutazione preliminare e run più lunghe per finalisti.

Una pipeline efficiente può essere:

screening veloce con simulazioni brevi;

selezione dei migliori;

validazione con simulazioni più lunghe;

test su più scenari;

report finale.

Questo riduce costo. Non serve testare ogni configurazione per minuti interi. Si può usare una valutazione gerarchica.

La ventitreesima componente è la **separazione tra evoluzione e Safety Layer**. La simulazione evolutiva può proporre parametri, ma non deve modificare direttamente safety hard limits. Safety hard limits devono essere vincoli, non variabili evolutive libere. Per esempio, `max_pwm_duty hardware`, `thermal_lock_threshold` e `emergency_stop behavior` non dovrebbero essere evoluti casualmente. Possono essere calibrati manualmente o con procedure molto controllate, ma non lasciati a un algoritmo che ottimizza fitness.

La regola è:

i parametri comportamentali possono evolvere; i limiti di sicurezza critici devono restare vincoli esterni.

Questo è fondamentale. Una fitness potrebbe premiare tempi rapidi e quindi favorire limiti meno sicuri. Il sistema deve impedirlo. Safety non si ottimizza sacrificandola.

La ventiquattresima componente è la **integrazione con la dashboard**. Un modulo evolutivo può avere una UI: generazioni, popolazione, miglior fitness, configurazione migliore, confronto con baseline, grafici. L'utente può osservare l'evoluzione. Può selezionare un individuo e vedere replay della sua run. Può confrontare individuo A e B. Può esportare il best_profile. Questa interattività renderebbe l'evoluzione molto più comprensibile.

Un evolution dashboard può mostrare:

generation number;

best fitness;

average fitness;

best individual parameters;

fitness chart;

failure distribution;

run replay;

parameter heatmap;

export behavior profile;

comparison with baseline.

Questa funzione unisce sistemi interattivi e simulazioni evolutive. È un ottimo esempio di integrazione tra sezioni.

La venticinquesima componente è la **esportazione dei profili evoluti**. Se una configurazione evoluta funziona bene, deve poter essere salvata come behavior_profile o pattern_profile. Tuttavia, deve essere marcata come evolved, con metadati. Non deve sostituire automaticamente i profili stabili. Deve essere revisionata, testata e validata.

Un evolved_profile può contenere:

profile_id;

source = evolutionary_simulation;

generation_id;

individual_id;

fitness_score;

tested_scenarios;

limitations;

recommended_use;

hardware_safe = false/true dopo validazione;

created_at;

fitness_function_version.

Questo rende il profilo tracciabile. Se in futuro causa problemi, si sa da dove viene. La tracciabilità è fondamentale.

La ventiseiesima componente è la **validazione prima dell'hardware**. Una configurazione evoluta in simulazione non deve essere usata direttamente su hardware fisico. Deve passare da una pipeline: test su più semi, test con fault, controllo dei limiti, revisione manuale, simulazione hardware-safe, test su LED-only, poi eventuale hardware più avanzato. Questo è indispensabile.

Una pipeline di validazione può essere:

evoluzione offline;

test multi-scenario;

analisi report;

controllo safety constraints;

review manuale;

demo simulata;

test visuale LED-only;

test hardware-safe limitato;

promozione a profile_candidate;

promozione a stable solo dopo prove.

Questo evita di portare soluzioni non controllate nel mondo fisico. È una mentalità ingegneristica corretta.

La ventisettesima componente è il **collegamento con gli algoritmi genetici**. Le simulazioni evolutive sono il contesto generale; gli algoritmi genetici sono una tecnica specifica. Il punto successivo della Sezione 16 sarà dedicato proprio agli algoritmi genetici. Qui è importante dire che le simulazioni evolutive possono usare algoritmi genetici, ma non si riducono solo a essi. Possono usare anche selezione semplice, hill climbing, random search, evolutionary strategies o altre euristiche. Gli algoritmi genetici sono una delle strade più naturali per MicroBot, perché permettono genotipo, mutazione, crossover e fitness.

La relazione è:

simulazione evolutiva = ambiente e processo sperimentale;

algoritmo genetico = metodo specifico per generare e selezionare configurazioni.

Questa distinzione prepara la sezione successiva. Prima si descrive il concetto evolutivo, poi la tecnica algoritmica.

La ventottesima componente è il **valore documentale e di ricerca**. Le simulazioni evolutive possono diventare una parte molto forte della documentazione MicroBot. Mostrano che il progetto non si limita a programmare pattern manualmente, ma può cercare comportamenti emergenti attraverso esperimenti automatizzati. Questo è vicino alla ricerca scientifica: ipotesi, simulazione, metriche, selezione, risultati, limiti. Una sezione evolutiva ben documentata può dare al progetto un livello molto più alto.

Un report evolutivo può includere:

obiettivo dell'esperimento;

parametri evoluti;

fitness function;

popolazione iniziale;

numero generazioni;

risultati migliori;

confronto con baseline;

analisi dei fallimenti;

limiti;

prossimi esperimenti.

Questo formato è perfetto per tesi, paper preliminare o documentazione tecnica avanzata. Trasforma un esperimento software in risultato analizzabile.

La ventinovesima componente è la **roadmap delle simulazioni evolutive**. Anche questo ambito deve crescere gradualmente. Prima si può fare random search su pochi parametri. Poi fitness function semplice. Poi generazioni. Poi mutazione. Poi crossover. Poi multi-scenario. Poi dashboard. Poi export di profili. Poi validazione hardware-safe. Questo è molto più realistico che partire subito con evoluzione complessa.

Una roadmap possibile è:

v0.1: random search su parametri simulativi;

v0.2: fitness function base;

v0.3: popolazione e generazioni;

v0.4: selezione dei migliori;

v0.5: mutazione controllata;

v0.6: crossover semplice;

v0.7: test su più seed;

v0.8: penalità safety/energy/thermal;

v0.9: report evolutivo;

v1.0: dashboard evolutiva;

v1.1: export evolved behavior_profile;

v1.2: multi-objective fitness;

v1.3: robustness testing;

v1.4: validazione simulation-to-hardware;

v1.5: integrazione come modulo consultivo MicroBot.

Questa roadmap rende l'evoluzione una parte seria e non improvvisata.

Il trentesimo punto è la **sintesi del ruolo delle simulazioni evolutive**. Esse servono a esplorare possibilità. MicroBot ha molti parametri e molti comportamenti potenziali. Progettare tutto a mano può essere limitante. Una simulazione evolutiva permette di provare, misurare, selezionare e migliorare. Tuttavia, deve essere confinata inizialmente alla simulazione, guidata da metriche chiare, vincolata dalla safety, documentata e validata prima di ogni uso hardware. Il suo valore principale non è automatizzare tutto, ma ampliare la capacità sperimentale del progetto.

In conclusione, le simulazioni evolutive rappresentano un ambito molto promettente per MicroBot perché permettono di generare e valutare automaticamente configurazioni, parametri, regole locali, pattern e strategie di recovery. Attraverso popolazioni, fitness function, selezione, mutazione, crossover, generazioni e test multi-scenario, il sistema può cercare soluzioni più stabili, efficienti e robuste. Tuttavia, l'evoluzione deve restare inizialmente offline e simulation-only, con vincoli di sicurezza rigidi, logging completo, riproducibilità, versioning e validazione prima di qualsiasi applicazione hardware. Le simulazioni evolutive collegano AI, clustering, giochi, sistemi interattivi e swarm intelligence, trasformando MicroBot in un laboratorio di ricerca sui comportamenti emergenti. Il punto successivo, dedicato agli algoritmi genetici, approfondirà la tecnica specifica con cui popolazioni di configurazioni possono essere codificate, mutate, incrociate, selezionate e migliorate generazione dopo generazione.

16.6 Algoritmi genetici

Gli **algoritmi genetici** rappresentano il sesto ambito dei progetti esterni collegabili all'ecosistema MicroBot e costituiscono una tecnica specifica, più formale e più implementabile, all'interno della famiglia più ampia delle simulazioni evolutive. Nel punto precedente è stato descritto il concetto generale di evoluzione simulata: una popolazione di configurazioni viene generata, testata, valutata e migliorata nel tempo. Gli algoritmi genetici forniscono un metodo concreto per realizzare questo processo attraverso una struttura ispirata all'evoluzione biologica: individui, geni, popolazione, fitness, selezione, mutazione, crossover e generazioni successive.

Nel progetto MicroBot, gli algoritmi genetici possono diventare strumenti molto utili per cercare automaticamente combinazioni di parametri più efficaci rispetto a quelle scelte manualmente. Uno swarm, anche simulato, può dipendere da molti parametri contemporaneamente: coesione, separazione, attrazione verso target, velocità massima, raggio di connessione, damping, soglie di stabilizzazione, numero massimo di retry, durata delle fasi, cooldown, tolleranze geometriche e topologiche. Modificare questi parametri a mano può essere lento e poco sistematico. Un algoritmo genetico permette invece di generare molte configurazioni, testarle nella simulazione, misurare il risultato e conservare quelle migliori.

Il principio generale può essere espresso così:

un algoritmo genetico applicato a MicroBot serve a far evolvere configurazioni simulabili dello swarm, selezionando nel tempo quelle che producono comportamento più stabile, preciso, efficiente e sicuro.

Questa frase è importante perché chiarisce cosa si evolve. Non si evolve fisicamente il MicroBot. Non si modifica hardware reale in modo casuale. Si evolve una configurazione software o comportamentale all'interno della simulazione. Solo dopo molti test una configurazione può diventare candidata per una demo o per un profilo stabile. Gli algoritmi genetici devono quindi essere trattati come strumenti di ricerca e ottimizzazione, non come meccanismi che comandano direttamente hardware senza controllo.

La prima parte di un algoritmo genetico è la definizione dell'**individuo**. In MicroBot, un individuo può essere una configurazione completa di comportamento. Ogni individuo contiene un insieme di parametri che determinano come i nodi virtuali si muovono, si aggregano, mantengono distanze, formano edge, si stabilizzano e reagiscono a errori. Un individuo non è quindi un singolo nodo, ma una possibile "ricetta" di comportamento dello swarm. Questa ricetta viene testata in una run simulata e valutata.

Un individuo MicroBot può contenere:

cohesion_weight;

separation_weight;

alignment_weight;

target_attraction_weight;

edge_maintenance_weight;

max_velocity;

damping_factor;

connection_radius;

minimum_spacing;

stabilization_threshold;

phase_timeout;

retry_limit;

cooldown_factor;

fallback_sensitivity.

Questi parametri devono essere scelti in base al modello reale della simulazione. Non bisogna inventare geni che non hanno effetto. Ogni gene deve corrispondere a una variabile usata dal simulatore o dal controller virtuale. Se un parametro non influenza il comportamento, non ha senso evolverlo. L'algoritmo genetico deve lavorare su grandezze operative, non decorative.

La seconda parte è la definizione del **gene**. Un gene è un singolo parametro modificabile. Per esempio, cohesion_weight può essere un gene; max_velocity può essere un gene; connection_radius può essere un gene. Ogni gene deve avere un tipo, un valore minimo, un valore massimo e una modalità di mutazione. Alcuni geni sono numeri continui, come un peso. Altri sono interi, come retry_limit. Altri possono essere categorie, come fallback_policy. La corretta definizione dei geni è fondamentale perché determina lo spazio di ricerca.

Un gene dovrebbe essere descritto con:

nome;

tipo;

valore minimo;

valore massimo;

valore iniziale;

step o precisione;

metodo di mutazione;

vincoli;

significato fisico o simulativo.

Per esempio, max_velocity può avere un minimo e un massimo realistici. Non deve diventare enorme, perché produrrebbe movimenti non fisici. cooldown_factor non deve scendere sotto una soglia di sicurezza. separation_weight non deve essere negativo se il modello lo interpreta come repulsione. Ogni gene deve avere vincoli coerenti.

La terza parte è il **cromosoma**. Il cromosoma è l'insieme dei geni di un individuo. In MicroBot, il cromosoma può essere rappresentato come un dizionario, un vettore, un JSON o una struttura dati. Per esempio, un behavior_profile evolutivo può essere un JSON con tutti i parametri. Questo è molto coerente con l'architettura MicroBot, perché i profili comportamentali possono essere salvati, versionati, esportati e riutilizzati.

Un cromosoma MicroBot può essere rappresentato così a livello concettuale:

behavior_profile_id;

genes;

parameter_values;

constraints;

source_generation;

fitness_score;

validation_status;

notes.

Questa struttura rende l'individuo tracciabile. Se una configurazione viene selezionata come migliore, si può salvarla con tutti i suoi geni, il suo punteggio, la generazione di origine e i test superati. Questo è importante per documentazione e riproducibilità.

La quarta parte è la **popolazione iniziale**. L'algoritmo genetico non parte da un solo individuo, ma da molti. La popolazione iniziale può essere generata casualmente entro limiti sicuri oppure partendo da configurazioni manuali già note. Una buona strategia per MicroBot è combinare individui random con individui baseline. I baseline sono profili costruiti manualmente e già ragionevoli. Gli individui random permettono esplorazione. In questo modo l'algoritmo non parte completamente alla cieca, ma conserva diversità.

Una popolazione iniziale può contenere:

profili baseline manuali;

varianti leggere dei baseline;

profili random entro limiti;

profili conservativi;

profili aggressivi solo in simulazione;

profili orientati a stabilità;

profili orientati a velocità;

profili orientati a energia.

Questa diversità iniziale è importante. Se tutti gli individui sono troppo simili, l'algoritmo esplora poco. Se sono troppo casuali e lontani da parametri realistici, molte run falliscono subito. Serve equilibrio.

La quinta parte è la **valutazione della fitness**. Ogni individuo viene testato nella simulazione. La run produce metriche: pattern_error, topology_score, stability_score, completion_time, collision_count, energy_cost, thermal_event_count, safety_event_count, fallback_count. La fitness function combina queste metriche in un punteggio. La fitness decide quali individui sono migliori e quindi quali hanno più probabilità di essere selezionati.

Una fitness function MicroBot deve premiare:

forma corretta;

topologia corretta;

stabilità;
tempo ragionevole;
pochi errori;
pochi safety events;
basso consumo energetico stimato;
basso carico termico;
robustezza a disturbi;
assenza di nodi isolati.

Deve penalizzare:

pattern_error alto;
missing_edges;
collisioni;
oscillazioni;
timeout;
thermal_warning;
critical_energy;
command_rejected;
node_isolation;
fallback frequenti;
failure finale.

Questo punto è centrale. Se la fitness è progettata male, l'algoritmo genetico ottimizzerà la cosa sbagliata. Per esempio, se si premia solo la velocità, potrebbe generare configurazioni instabili. Se si premia solo la precisione finale, potrebbe generare configurazioni lentissime o costose. La fitness deve riflettere gli obiettivi reali di MicroBot.

La sesta parte è la **fitness multi-obiettivo**. In molti casi, un singolo numero non basta. MicroBot ha obiettivi in tensione: velocità, precisione, stabilità, sicurezza, energia, robustezza. Una configurazione può essere molto veloce ma meno stabile. Un'altra può essere lenta ma sicura. Un'altra può essere ottima in simulazione ideale ma fragile con rumore. Per questo può essere utile conservare più punteggi separati e poi decidere secondo modalità d'uso.

Una valutazione multi-obiettivo può produrre:

geometry_score;

topology_score;

stability_score;

efficiency_score;

safety_score;

energy_score;

thermal_score;

robustness_score;

demo_score.

In presentation_mode si può privilegiare demo_score e visual_clarity. In research_mode si può privilegiare exploration e performance. In hardware_safe_mode si privilegiano safety_score, thermal_score ed energy_score. Questo approccio è più maturo perché evita di cercare un “migliore assoluto” che forse non esiste.

La settima parte è la **selezione**. Dopo aver calcolato la fitness, l’algoritmo decide quali individui sopravvivono e quali vengono usati per generare la generazione successiva. Le configurazioni migliori vengono conservate o hanno maggiore probabilità di riprodursi. In MicroBot, è utile usare anche elitismo, cioè conservare sempre alcuni dei migliori individui senza modificarli. Questo evita di perdere una configurazione buona per mutazioni casuali.

Metodi di selezione possibili:

elitismo;

tournament selection;

selezione proporzionale alla fitness;

selezione dei migliori N;

selezione multi-obiettivo;

selezione con diversità minima.

Il metodo più semplice è conservare i migliori N. Tuttavia, se si conserva solo il migliore, la popolazione può perdere diversità. Una strategia migliore è conservare alcuni migliori, alcuni medi promettenti e alcuni diversi. Questo permette all’algoritmo di non bloccarsi troppo presto.

L’ottava parte è il **crossover**. Il crossover combina due individui genitori per produrre un nuovo individuo figlio. Per MicroBot, questo può significare prendere i parametri di movimento da un genitore e quelli di stabilizzazione da un altro. Oppure combinare metà geni da A e metà da B. L’obiettivo è produrre configurazioni che uniscano caratteristiche utili di più soluzioni.

Un crossover MicroBot può combinare:

cohesion_weight da genitore A;

separation_weight da genitore B;

damping_factor da genitore A;

connection_radius da genitore B;

stabilization_threshold da genitore A;

retry_limit da genitore B.

Dopo il crossover, il figlio deve essere validato. I valori devono restare nei limiti. Se il crossover produce combinazioni incoerenti, il sistema deve correggere o scartare. Per esempio, connection_radius non deve diventare minore di minimum_spacing se il modello richiede il contrario. I vincoli devono essere controllati dopo ogni generazione.

La nona parte è la **mutazione**. La mutazione introduce variazioni casuali nei geni. È essenziale per esplorare nuove configurazioni. Senza mutazione, l'algoritmo ricombina solo valori già presenti. Con mutazione, può scoprire nuove regioni dello spazio dei parametri. In MicroBot, la mutazione deve essere controllata: valori piccoli e progressivi sono più realistici di salti enormi, almeno nelle fasi iniziali.

Esempi di mutazione:

aumentare cohesion_weight del 2–10%;

ridurre max_velocity;

aumentare damping_factor;

variare connection_radius;

modificare stabilization_threshold;

incrementare retry_limit;

aumentare cooldown_factor;

ridurre target_attraction_weight.

La mutazione può avere intensità diversa. Una mutazione leggera esplora vicino a soluzioni buone. Una mutazione forte introduce novità. Se l'algoritmo stagna, si può aumentare mutation_rate. Se il sistema produce troppi fallimenti, si può ridurre. Anche mutation_rate è un parametro importante.

La decima parte è la **validazione dei vincoli dopo mutazione e crossover**. Ogni individuo generato deve essere controllato prima della simulazione. Non deve contenere parametri invalidi, valori fuori range, combinazioni impossibili o limiti di sicurezza violati. Questa validazione è fondamentale perché evita di sprecare simulazioni su configurazioni assurde e protegge dalla generazione di profili non realistici.

Controlli possibili:

tutti i geni presenti;

valori nei range;

nessun parametro negativo se non ammesso;

cooldown non sotto minimo;
max_velocity entro limite;
connection_radius coerente;
minimum_spacing coerente;
safety hard limits non modificati;
schema_version compatibile.

Se un individuo fallisce validazione, può essere riparato o scartato. La riparazione consiste nel riportare i valori nei limiti. Lo scarto elimina l'individuo. Per MicroBot, nelle prime versioni, la riparazione semplice è utile, ma bisogna loggare quante riparazioni avvengono. Se molte configurazioni richiedono riparazione, i range di mutazione sono forse sbagliati.

L'undicesima parte è la **simulazione dell'individuo**. Ogni individuo deve essere testato in un ambiente controllato. Il simulatore carica il behavior_profile, inizializza lo swarm, esegue il pattern, registra metriche e produce un run_report. Questo run_report viene poi usato dalla fitness function. È importante che ogni individuo venga testato nelle stesse condizioni, almeno nella fase base, per confronto equo.

Una run evolutiva deve registrare:

individual_id;
generation_id;
behavior_profile;
pattern_id;
initial_state;
simulation_seed;
run_metrics;
events;
final_result;
fitness_result.

Questa struttura rende possibile confrontare individui. Se non si registra tutto, il processo evolutivo diventa difficile da verificare. L'algoritmo genetico deve essere tracciabile come qualsiasi altro modulo MicroBot.

La dodicesima parte è il **test su più seed**. Una configurazione può funzionare bene per una disposizione iniziale fortunata ma male per altre. Per evitare questo, ogni individuo può essere testato su più seed o più initial_swarm_state. La fitness finale può essere media, mediana o peggiore dei risultati. Questo aumenta costo computazionale, ma migliora robustezza. Per le prime versioni, si può testare su un seed; per versioni avanzate, su più scenari.

Una valutazione multi-seed può calcolare:

fitness media;

fitness minima;

success_rate;

variance;

failure_count;

robustness_score.

Una configurazione con fitness media alta ma varianza enorme è rischiosa. Una configurazione leggermente meno performante ma molto stabile può essere migliore per demo o hardware. Questo concetto è fondamentale per MicroBot.

La tredicesima parte è il **test su più pattern**. Un profilo può essere ottimo per line_basic ma pessimo per circle_basic. Bisogna decidere se l'algoritmo genetico cerca parametri specifici per un pattern o parametri generali. Entrambe le strade sono valide. Un profilo specifico può essere ottimizzato per una forma. Un profilo generale deve funzionare abbastanza bene su più pattern. La scelta dipende dall'obiettivo.

Due strategie sono:

pattern_specific_evolution;

general_behavior_evolution.

pattern_specific_evolution produce profili come line_basic_evolved_profile, circle_basic_evolved_profile, cluster_basic_evolved_profile. general_behavior_evolution produce un profilo più ampio, utile come default. Nelle prime versioni, è più facile evolvere profili specifici per pattern. In futuro si può cercare generalità.

La quattordicesima parte è la **penalità per comportamenti non sicuri**. Anche in simulazione, la fitness deve penalizzare comportamenti che sarebbero pericolosi o inutili su hardware. Per esempio, collisioni frequenti, velocità troppo alte, oscillazioni forti, comandi troppo frequenti, output stimati troppo intensi, thermal_warning, cooldown ignorato. Questa penalità evita che l'algoritmo trovi soluzioni "furbe" ma non trasferibili.

Penalità importanti:

collision_penalty;

thermal_penalty;

energy_penalty;

command_rate_penalty;

oscillation_penalty;

timeout_penalty;

safety_event_penalty;

hardware_unrealistic_penalty.

Questa parte è essenziale per il passaggio simulation-to-hardware. Un comportamento valido solo in simulazione ideale non è abbastanza. Deve rispettare limiti realistici.

La quindicesima parte è la **conservazione della diversità**. Se l'algoritmo seleziona sempre solo il migliore, la popolazione può diventare tutta uguale rapidamente. Questo riduce esplorazione. Per MicroBot, può essere utile mantenere diversità nei parametri e nei risultati. Una popolazione diversificata può contenere soluzioni veloci, soluzioni stabili, soluzioni conservative e soluzioni sperimentali. Questo è utile perché il sistema può usare profili diversi in modalità diverse.

Strategie di diversità:

mantenere individui con parametri diversi;

mantenere individui con trade-off diversi;

introdurre random immigrants;

limitare copie identiche;

penalizzare popolazione troppo uniforme;

usare clustering degli individui.

Questa diversità rende l'evoluzione più ricca. Può anche produrre insight: magari esistono due famiglie di soluzioni, una basata su alta coesione e damping alto, un'altra su target attraction forte e velocità bassa.

La sedicesima parte è la **stagnazione evolutiva**. Se la fitness non migliora per molte generazioni, l'algoritmo è in stagnazione. Questo può succedere perché la popolazione ha perso diversità, la fitness è mal progettata o lo spazio di ricerca è difficile. MicroBot deve diagnosticare anche il processo evolutivo. Non basta generare generazioni all'infinito. Serve sapere se il processo sta migliorando.

Indicatori di stagnazione:

best_fitness non migliora;

average_fitness stabile;

diversity_score basso;

failure_distribution invariata;

mutazioni non producono miglioramenti;

troppi individui simili.

Azioni possibili:

aumentare mutation_rate;

introdurre nuovi individui random;

cambiare seed;

modificare fitness;

aumentare popolazione;

ripartire da baseline diverso.

Questa diagnostica rende l'algoritmo genetico più controllabile.

La diciassettesima parte è la **versione della fitness function**. La fitness può cambiare nel tempo. Se si modifica la formula, i risultati non sono più direttamente confrontabili con quelli precedenti. Per questo ogni fitness deve avere `fitness_function_version`. Ogni run evolutiva deve registrare quale versione ha usato. Questo è molto importante per riproducibilità.

Una fitness function dovrebbe dichiarare:

`fitness_function_id`;

version;

metriche usate;

pesi;

penalità;

vincoli;

data;

note;

limiti.

Se una configurazione ha fitness 0.92 con `fitness_v1` e un'altra 0.88 con `fitness_v2`, non è detto che la prima sia migliore. Serve stessa funzione per confronto diretto. Questa disciplina è da progetto serio.

La diciottesima parte è il **versioning del simulatore**. Anche il simulatore cambia. Se fisica, regole, metriche o rendering cambiano, le fitness possono cambiare. Un individuo ottimo in `simulation_engine_v0.4` può non esserlo in `v0.6`. Per questo ogni esperimento genetico deve registrare `simulation_version`. È lo stesso principio del firmware versioning in MicroBot OS.

Ogni run evolutiva deve registrare:

`simulation_engine_version`;

`metric_schema_version`;

`behavior_schema_version`;

`fitness_function_version`;

pattern_model_version.

Questo consente di sapere in quale contesto un profilo è stato generato. Un profilo evoluto non è assoluto; è valido rispetto a un ambiente di test.

La diciannovesima parte è la **struttura dei dati per algoritmi genetici**. In una repo MicroBot, potrebbe esistere una cartella dedicata a evolutionary o optimization. Essa può contenere population configs, fitness scripts, run reports, evolved profiles e documentation. Questa struttura aiuta a mantenere separato il modulo genetico dal core runtime.

Una struttura possibile è:

optimization/

genetic/

configs/

populations/

fitness/

runs/

reports/

evolved_profiles/

docs/

Il file genetic_config.json può definire population_size, generation_count, mutation_rate, crossover_rate, selection_method, fitness_function_id e pattern_id. Il file evolved_profile.json può salvare i migliori parametri. I report possono documentare risultati. Questa separazione rende il modulo ordinato.

La ventesima parte è la **integrazione con behavior_profiles**. I risultati dell'algoritmo genetico dovrebbero essere esportati come behavior_profile. Questo permette di riutilizzarli nel simulatore senza dipendere dal modulo genetico. Un evolved_behavior_profile deve avere metadati chiari: origine, generazione, fitness, scenari testati, limiti e stato di validazione. Non deve essere trattato automaticamente come profilo stabile.

Un evolved_behavior_profile può contenere:

profile_id;

source = genetic_algorithm;

generation_id;

individual_id;

fitness_score;

fitness_function_version;

tested_patterns;
tested_seeds;
constraints_passed;
hardware_safe_status;
recommended_use;
parameters.

hardware_safe_status deve essere molto chiaro. Di default, un profilo evoluto è simulation_candidate, non hardware_ready. Solo dopo validazione può cambiare stato.

La ventunesima parte è la **integrazione con la dashboard evolutiva**. Un algoritmo genetico può essere difficile da capire se resta solo in console. Una dashboard può mostrare best fitness, average fitness, popolazione, parametri, curva di miglioramento, replay degli individui migliori e confronto con baseline. Questo rende il processo esplorabile.

La dashboard può mostrare:

generation;
best_fitness;
average_fitness;
best_individual_id;
fitness_chart;
parameter_distribution;
failure_distribution;
best_run_replay;
comparison_with_baseline;
export button.

Questa interfaccia unisce sistemi interattivi e algoritmi genetici. È molto utile per presentare il lavoro e per capire cosa sta succedendo.

La ventiduesima parte è il **confronto con baseline**. Ogni algoritmo genetico deve essere confrontato con una configurazione base manuale. Se il profilo evoluto non migliora la baseline, non è utile. Il confronto deve usare stesse condizioni. Questo è fondamentale per evitare entusiasmo ingiustificato.

Un confronto baseline può includere:

baseline_success_rate;
evolved_success_rate;

baseline_completion_time;

evolved_completion_time;

baseline_pattern_error;

evolved_pattern_error;

baseline_safety_events;

evolved_safety_events;

baseline_energy_cost;

evolved_energy_cost.

Se il profilo evoluto migliora pattern_error ma aumenta molto safety_event_count, potrebbe non essere accettabile. Il confronto deve essere multi-metrica, non solo fitness totale.

La ventitreesima parte è la **separazione tra esperimento e produzione**. Un algoritmo genetico è sperimentale. I profili generati sono candidati. Prima di usarli in demo stabile o hardware, devono passare validazione. Questo deve essere scritto chiaramente. Non bisogna far sì che il sistema scelga automaticamente il profilo evoluto più recente per una demo importante. Serve promozione manuale o pipeline controllata.

Stati di un profilo evoluto:

generated;

tested_single_scenario;

tested_multi_scenario;

reviewed;

simulation_candidate;

demo_candidate;

hardware_candidate;

stable;

deprecated.

Questa classificazione è molto utile. Un profilo generated non è uguale a stable. Il sistema deve saperlo.

La ventiquattresima parte è la **applicazione agli algoritmi genetici già sviluppati come progetto esterno**. Se esiste già un progetto esterno di genetic algorithm simulation, esso può essere collegato a MicroBot come base concettuale. Può aver già implementato popolazione, selezione, mutazione, fitness e generazioni. Anche se non è integrato direttamente, dimostra competenza. Il suo riuso può essere diretto, se il codice è adattabile, oppure indiretto, come modello di architettura.

Una descrizione rigorosa può essere:

il progetto esterno di algoritmo genetico dimostra la gestione di popolazioni, fitness, selezione e mutazione; nel contesto MicroBot, questi concetti possono essere riapplicati per ottimizzare behavior_profile e parametri simulativi, ma l'integrazione richiede adattamento dello schema dati, della fitness function e del simulatore.

Questa formulazione valorizza il lavoro senza dire che sia già integrato se non lo è.

La venticinquesima parte è la **possibile applicazione ai ruoli**. Gli algoritmi genetici possono ottimizzare non solo parametri di movimento, ma anche assegnazioni di ruolo. Per esempio, dato uno swarm con nodi di affidabilità diversa, il sistema può cercare la migliore assegnazione leader/connector/terminal per massimizzare topology_score e minimizzare rischio. Questo è più complesso, ma molto interessante.

Un cromosoma di role assignment può contenere:

node_slot_to_role mapping;

priority order;

fallback role rules;

connector selection strategy;

leader selection strategy.

La fitness valuta se la struttura risultante è stabile, connessa e robusta. Questo tipo di algoritmo può aiutare quando lo swarm cresce. Tuttavia, nelle prime versioni è meglio iniziare con parametri comportamentali, poi passare ai ruoli.

La ventiseiesima parte è la **possibile applicazione ai pattern**. Un algoritmo genetico può evolvere target positions o pattern structure. Per esempio, cercare una disposizione di nodi che massimizzi copertura, minimizzi distanza media e mantenga connessione. Questo può essere utile per generare pattern non progettati manualmente. Tuttavia, serve una fitness molto chiara e vincoli topologici.

Un cromosoma di pattern può contenere:

target_positions;

expected_edges;

role_slots;

spacing;

symmetry parameters;

cluster centers.

La fitness valuta forma, connessione e stabilità. Questa è una direzione più avanzata, utile per ricerca e generazione di strutture.

La ventisettesima parte è la **possibile applicazione alla recovery**. Come già accennato, gli algoritmi genetici possono cercare strategie di recovery. Un cromosoma può definire cosa fare in

caso di node_offline, missing_edge, thermal_warning o sync_lost. La fitness misura quanti fallimenti vengono recuperati e quanto costa il recupero. Questo è molto utile per sistemi distribuiti.

Un cromosoma di recovery policy può contenere:

retry_count;

retry_delay;

role_reassign_enabled;

fallback_pattern;

exclude_unstable_node_threshold;

state_sync_required;

safe_idle_threshold;

recovery_timeout.

La fitness valuta recovery_success_rate, completion_time_after_failure, safety_event_count e energy_cost. Questa applicazione è molto concreta ma va introdotta dopo aver stabilizzato failure taxonomy e recovery model.

La ventottesima parte è la **sicurezza degli algoritmi genetici**. Un algoritmo genetico può proporre configurazioni non desiderate. Per questo deve essere sandboxed: lavora su simulazione, con limiti, senza accesso diretto ad hardware, senza modifica di safety hard limits e con export manuale. Deve essere possibile disattivarlo. Deve produrre log. Deve dichiarare che i risultati sono candidati.

Regole di sicurezza:

simulation-only nelle prime versioni;

nessun comando diretto hardware;

nessuna modifica ai safety hard limits;

range vincolati;

fitness con penalità safety;

export manuale;

validazione prima dell'uso;

logging completo;

accesso developer/admin.

Queste regole sono fondamentali. L'algoritmo genetico è potente, ma non deve diventare incontrollato.

La ventinovesima parte è la **roadmap degli algoritmi genetici in MicroBot**. Anche qui serve progressione. Prima si può creare una random search semplice. Poi popolazione. Poi fitness. Poi

selezione. Poi mutazione. Poi crossover. Poi dashboard e report. Poi export profili. Poi validazione multi-scenario. Poi applicazioni a ruoli e recovery. Questo evita salti troppo grandi.

Una roadmap possibile è:

- v0.1: random parameter search;
- v0.2: definizione chromosome schema;
- v0.3: fitness function base;
- v0.4: popolazione iniziale;
- v0.5: selezione con elitismo;
- v0.6: mutazione controllata;
- v0.7: crossover semplice;
- v0.8: run report per individuo;
- v0.9: best profile export;
- v1.0: confronto con baseline;
- v1.1: multi-seed evaluation;
- v1.2: dashboard evolutiva;
- v1.3: multi-objective fitness;
- v1.4: evoluzione role assignment;
- v1.5: evoluzione recovery policy;
- v1.6: pipeline di validazione hardware-safe.

Questa roadmap rende l'algoritmo genetico una linea di ricerca concreta e ordinata.

Il trentesimo punto è la **sintesi del ruolo degli algoritmi genetici**. Essi sono uno strumento per cercare configurazioni migliori in uno spazio troppo grande per essere esplorato manualmente. In MicroBot possono evolvere parametri, behavior_profile, role assignment, pattern structure e recovery policy. Tuttavia, devono essere usati con rigore: fitness ben progettata, vincoli di sicurezza, simulazione ripetibile, versioning, logging, confronto con baseline, validazione multi-scenario e separazione netta dall'hardware reale. Il loro ruolo iniziale migliore è offline, consultivo e simulation-only.

In conclusione, gli algoritmi genetici rappresentano una tecnica specifica e potente per portare nel progetto MicroBot una forma di ottimizzazione evolutiva controllata. Attraverso geni, cromosomi, popolazioni, fitness, selezione, mutazione, crossover e generazioni, il sistema può esplorare automaticamente molte configurazioni comportamentali dello swarm e selezionare quelle più stabili, precise, efficienti e sicure. La loro applicazione deve partire dalla simulazione, non dall'hardware, e deve sempre rispettare vincoli, Safety Layer, tracciabilità e validazione. In questo modo, gli algoritmi genetici diventano un ponte tra progetti esterni, intelligenza artificiale,

simulazioni evolutive e ottimizzazione dello swarm MicroBot. Il punto successivo, dedicato ai progetti web, analizzerà come siti, dashboard, interfacce visuali, pagine tecniche, landing page e piattaforme browser-based possano contribuire alla comunicazione, al controllo, alla presentazione e alla gestione dell'intero ecosistema MicroBot.

16.7 Progetti web

I **progetti web** rappresentano il settimo ambito dei progetti esterni collegabili all'ecosistema MicroBot e riguardano tutti quei siti, dashboard, interfacce browser-based, landing page, pagine dimostrative, pannelli tecnici e piattaforme visuali che possono contribuire alla comunicazione, al controllo, alla presentazione e all'organizzazione del sistema. Dopo aver analizzato algoritmi genetici, simulazioni evolutive, sistemi interattivi e giochi, diventa necessario affrontare una componente molto concreta del progetto: il web. MicroBot non è soltanto firmware, simulazione e teoria; ha bisogno di essere visto, esplorato, spiegato e controllato attraverso interfacce accessibili. Il browser è uno degli strumenti migliori per farlo, perché permette di unire grafica, interazione, telemetria, animazioni, documentazione e demo in un unico ambiente.

Nel contesto MicroBot, i progetti web esterni non devono essere considerati semplici siti "di vetrina". Alcuni possono avere funzione comunicativa, come landing page, portfolio, pagine introduttive e presentazioni del progetto. Altri possono avere funzione operativa, come dashboard, pannelli di telemetria, simulatori interattivi, pagine di debug, viewer di report, replay delle run e interfacce di controllo. Altri ancora possono essere esperimenti estetici, utili per definire il linguaggio visivo del progetto: tema scuro, griglie, HUD, glow, pannelli tecnici, animazioni, microinterazioni, sfondi dinamici, card, tabelle e grafici. Tutto questo ha valore perché MicroBot, per essere compreso, deve essere visualizzato bene.

Il principio generale può essere espresso così:

i progetti web permettono di trasformare MicroBot da sistema tecnico interno a piattaforma osservabile, presentabile, controllabile e comprensibile attraverso interfacce visuali.

Questa frase è centrale. Un progetto complesso senza interfaccia resta difficile da comunicare. Si può avere un ottimo simulatore Python, un buon firmware ESP32, documentazione lunga e modelli teorici solidi, ma se l'utente non vede niente in modo chiaro, il progetto perde forza comunicativa. La parte web serve a dare forma visibile all'ecosistema: mostra lo swarm, i nodi, le connessioni, la telemetria, i pattern, i warning, le run, i report e la visione generale. È il livello in cui tecnica e narrazione si incontrano.

La prima funzione dei progetti web è la **landing page di progetto**. Una landing page non controlla direttamente MicroBot, ma lo presenta. Deve spiegare in modo chiaro che cosa è MicroBot, quale problema affronta, quale visione propone, quali componenti contiene, quali demo esistono, quali sono le direzioni future e perché il progetto è rilevante. Questa pagina è fondamentale per collaboratori, investitori, professori, amici tecnici, GitHub e portfolio. È il primo punto di accesso al progetto.

Una landing page MicroBot può contenere:

titolo del progetto;

frase di visione;

descrizione sintetica;

schema dell'ecosistema;

sezione swarm simulation;

sezione MicroBot OS;

sezione telemetria;

sezione hardware prototype;

sezione sicurezza;

sezione AI/future directions;

link a documentazione;

link a demo;

link a repository.

La landing page deve essere bella, ma anche precisa. Non deve promettere funzioni non implementate. Deve distinguere chiaramente tra ciò che è già prototipato, ciò che è simulato, ciò che è in sviluppo e ciò che è visione futura. Questa distinzione aumenta credibilità. Una frase come "hardware actuation future direction" è più seria di una pagina che fa sembrare tutto già realizzato.

La seconda funzione è la **dashboard tecnica**. Questa è una delle parti web più importanti. La dashboard è l'interfaccia attraverso cui l'utente può osservare lo stato del sistema. Può mostrare nodi, edge, pattern, telemetria, stato globale, eventi, metriche, grafici, alert e report. A differenza della landing page, la dashboard non è solo comunicativa: è operativa e diagnostica. È il centro di controllo visuale dell'ecosistema MicroBot.

Una dashboard MicroBot può includere:

vista swarm;

pannello stato globale;

node inspector;

event log;

metric charts;

pattern selector;

telemetry summary;

safety panel;

diagnostics panel;

replay controls;

security/access panel;

export report.

Questa struttura permette di passare da una visione generale a un dettaglio tecnico. L'utente vede prima lo swarm, poi può cliccare un nodo, leggere i dati, filtrare eventi, controllare errori e generare report. La dashboard diventa l'ambiente principale per sviluppo e presentazione.

La terza funzione è la **visualizzazione della simulazione**. Il web è ottimo per mostrare simulazioni 2D o 3D. Una canvas HTML, SVG, WebGL o Three.js può visualizzare nodi, edge, campi, traiettorie, cluster e pattern. Questo permette di vedere MicroBot in movimento. La simulazione web può essere autonoma, cioè solo frontend, oppure collegata a un backend Python che produce snapshot. In entrambi i casi, il browser diventa finestra sul comportamento dello swarm.

Una visualizzazione simulativa può mostrare:

nodi come punti o moduli;

edge come connessioni;

target positions;

traiettorie;

cluster boundaries;

ruoli tramite colori o forme;

stati safety tramite icone;

energia tramite indicatori;

thermal state tramite aura o overlay;

pattern phase tramite HUD.

La cosa importante è che la visualizzazione non deve essere solo estetica. Ogni elemento visuale deve corrispondere a un dato. Se un edge è rosso, deve significare missing, critical o failed secondo legenda. Se un nodo lampeggia, deve indicare warning, heartbeat o selezione. La coerenza semantica è fondamentale.

La quarta funzione è il **pannello di telemetria**. La telemetria è il flusso di dati del sistema: stato dei nodi, eventi, metriche, heartbeat, energia, temperatura, safety, ruoli, pattern, errori. Una pagina web può trasformare questa telemetria in tabelle, grafici, badge, timeline e pannelli. Senza web, la telemetria rischia di restare JSON o log testuale. Con una dashboard, diventa leggibile.

Un pannello telemetria può contenere:

node_count;

online_nodes;

offline_nodes;

active_pattern;

current_phase;

average_pattern_error;

topology_score;

aggregation_score;

safety_event_count;

energy_summary;

thermal_summary;

last_events;

telemetry_age.

La dashboard deve anche indicare la freschezza dei dati. Un valore vecchio può essere pericoloso. Se l'ultimo snapshot è stale, il sistema deve mostrarlo. Un dato non aggiornato non deve sembrare live. Questo è un punto tecnico molto importante.

La quinta funzione è il **node inspector web**. Il node inspector è una scheda dettagliata del singolo MicroBot. Quando l'utente seleziona un nodo, il pannello mostra dati locali: identificativo, ruolo, modalità, posizione, target, stato energetico, stato termico, safety, capabilities, ultimi comandi, ultimi eventi, stato comunicazione e versione firmware o simulazione. Questa funzione rende il sistema profondamente esplorabile.

Un node inspector può mostrare:

node_id;

boot_id, se nodo reale;

mode;

role;

position;

target_position;

velocity;

energy_state;

thermal_state;

safety_state;

sync_status;

capabilities;

blocked_capabilities;

last_command;

last_event;

diagnostic_status.

Questo pannello è essenziale sia per sviluppo sia per demo. In una presentazione, cliccare un nodo e vedere la sua telemetria dà subito l'idea di un sistema vivo e strutturato. In sviluppo, permette di debug rapido.

La sesta funzione è la **visualizzazione degli eventi**. Ogni sistema MicroBot produce eventi: node_online, node_offline, role_changed, edge_formed, edge_missing, command_rejected, thermal_warning, energy_low, pattern_completed, recovery_started. Una pagina web può mostrare questi eventi in una timeline o in un event log filtrabile. Questo permette di capire non solo lo stato attuale, ma la storia della run.

Un event log web dovrebbe permettere:

filtro per severity;

filtro per node_id;

filtro per event_type;

filtro per phase_id;

ricerca testuale;

jump to timestamp;

collegamento con replay;

evidenziazione critical;

raggruppamento eventi simili.

Questa funzione è molto importante perché i log grezzi diventano presto illeggibili. Un event viewer ben fatto trasforma il debug da caos a processo controllabile.

La settima funzione è la **timeline delle run**. Una run MicroBot ha un inizio, fasi, eventi, metriche e fine. La timeline mostra tutto questo nel tempo. Può indicare quando è partita la fase prepare, quando è avvenuto assign_roles, quando sono comparsi missing_edges, quando è entrato un thermal_warning, quando è stata completata la run. Una timeline interattiva permette di saltare a un evento e vedere lo stato dello swarm in quel momento.

Una timeline web può mostrare:

start_run;

phase transitions;

critical events;

metric peaks;

fallback triggers;

recovery events;

pattern_completed;

pattern_failed;

manual actions;

security events, se rilevanti.

Questa funzione collega dashboard, replay e report. È uno degli strumenti più potenti per capire comportamenti dinamici.

L'ottava funzione è il **replay viewer**. Il web può diventare un visualizzatore di run registrate. Un replay viewer carica snapshot e eventi e permette di rivedere una simulazione. Può essere usato per debugging, documentazione, presentazioni e confronto tra versioni. È una funzione estremamente utile perché consente di mostrare il comportamento anche senza eseguire live il sistema.

Un replay viewer può contenere:

play/pause;

time slider;

speed control;

event markers;

node inspector storico;

metric charts sincronizzati;

snapshot selector;

run summary;

export frame o report.

Il replay viewer trasforma una run in materiale analizzabile. Se una persona non può avviare il sistema, può comunque vedere un replay. Questo è molto utile per GitHub, demo online e documentazione.

La nona funzione è la **pagina di report**. Dopo una run, il sistema può generare un report. Il web può mostrare questo report in forma leggibile: sintesi, configurazione, metriche, grafici, eventi, failure reason, raccomandazioni, replay collegato. Un report web è più navigabile di un file testuale, ma può anche essere esportato in markdown o PDF in futuro.

Una pagina report può includere:

run_id;

pattern_id;

configuration_id;

result;
success/failure;
main metrics;
timeline summary;
node issues;
safety summary;
energy summary;
thermal summary;
failure analysis;
recommendations;
link al replay.

Questa pagina è molto utile per costruire una memoria sperimentale del progetto. Ogni test non resta isolato; diventa documento consultabile.

La decima funzione è la **pagina di documentazione tecnica integrata**. Un progetto web può ospitare sezioni documentali: architettura, protocollo, telemetria, safety, MicroBot OS, hardware profiles, pattern models, API, roadmap. Questo permette di avere un portale unico in cui si alternano demo e spiegazione. L'utente può leggere cosa significa topology_score e poi vederlo nella dashboard. Può leggere il Safety Layer e poi vedere eventi safety live.

Una documentazione web può avere:

overview;
architecture;
simulation;
telemetry;
MicroBot OS;
hardware;
safety;
biometrics;
AI modules;
external projects;
roadmap;

glossary.

Questa struttura è molto utile per presentare il progetto in modo professionale. Non sostituisce documenti lunghi, ma li rende navigabili.

L'undicesima funzione è la **piattaforma portfolio**. I progetti web possono anche servire a mostrare l'intero ecosistema personale: MicroBot, simulazioni, AI, giochi, clustering, black hole simulation, web experiments, tool diagnostici. Una piattaforma portfolio ben organizzata può spiegare come ogni progetto si collega alle competenze generali. Questo è utile per GitHub, LinkedIn, candidatura, collaboratori e pitch.

Una pagina portfolio può includere:

progetto principale MicroBot;

demo principali;

progetti AI;

progetti di simulazione;

progetti web;

giochi;

tool;

documentazione;

roadmap personale;

link repository.

Tuttavia, bisogna evitare dispersione. Il portfolio deve mostrare MicroBot come progetto centrale e gli altri come satelliti tecnici. Se tutto appare allo stesso livello, il lettore si perde. La gerarchia è importante.

La dodicesima funzione è la **coerenza estetica**. I progetti web esterni possono definire uno stile MicroBot riconoscibile: dark interface, griglia tecnica, neon/glow controllato, font monospaziato per dati, card traslucide, pannelli HUD, grafici, icone, linee sottili, microanimazioni. Questo stile non è solo estetica. Aiuta a comunicare identità tecnologica. Un progetto con stile coerente appare più maturo.

Elementi estetici ricorrenti possono essere:

background scuro;

wireframe grid;

accent color ciano/verde;

pannelli glass o HUD;

linee tecniche;

badge di stato;

font tecnico per numeri;

card modulari;

animazioni leggere;

schema responsive.

La coerenza visiva deve però restare subordinata alla leggibilità. Un'interfaccia troppo piena di glow può diventare difficile da usare. La priorità deve essere sempre chiarezza tecnica. Lo stile deve aiutare, non coprire.

La tredicesima funzione è la **interazione responsive**. Un sito MicroBot può essere visto su desktop, tablet o laptop. La dashboard tecnica probabilmente richiede desktop, ma landing page e documentazione devono essere responsive. Le pagine web esterne possono contribuire con esperienza su layout adattivi, griglie, card, sezioni e pannelli. Questo è importante se il progetto viene condiviso.

Una struttura responsive può prevedere:

desktop dashboard completa;

tablet dashboard semplificata;

mobile landing/documentazione;

mobile read-only status;

controlli critici solo desktop/developer;

grafici adattati.

Non tutto deve funzionare allo stesso modo su telefono. Una dashboard hardware non dovrebbe essere usata da mobile se l'interfaccia non è adatta. Ma la pagina di presentazione sì. Questa distinzione è matura.

La quattordicesima funzione è la **separazione tra sito statico e dashboard live**. Non tutti i progetti web hanno la stessa natura. Un sito statico presenta contenuti. Una dashboard live mostra dati aggiornati. Un simulatore web esegue logica nel browser. Un frontend con backend comunica con server Python o Node. È importante distinguere queste categorie per non confondere architettura e scopo.

Categorie web MicroBot:

landing_static_site;

documentation_site;

standalone_simulation_frontend;

live_dashboard_frontend;

backend_connected_dashboard;

replay_viewer;
report_viewer;
portfolio_site;
developer_tools_web.

Ogni categoria ha requisiti diversi. Una landing può stare su GitHub Pages. Una dashboard live può richiedere backend locale e WebSocket. Un replay viewer può caricare file JSON. Un report viewer può essere statico. Questa distinzione aiuta a organizzare repository e deploy.

La quindicesima funzione è la **connessione con backend e runtime**. Una dashboard live deve ricevere dati da un server, da un file snapshot o da WebSocket. Per esempio, un runtime Python può esporre endpoint HTTP e WebSocket. La pagina web può leggere /api/status, /api/snapshot, /api/events o ws://.../ws. Questo collegamento trasforma il web in interfaccia operativa.

Una architettura possibile è:

simulation engine produce snapshot;
runtime server espone API;
dashboard web consuma API;
WebSocket invia updates live;
frontend aggiorna vista;
event log registra eventi;
report generator salva risultati.

Questa architettura deve essere documentata. Se il sito è solo statico, va detto. Se è live dashboard, va spiegato come si avvia il backend. Questo è importante per evitare confusione.

La sedicesima funzione è il **supporto WebSocket**. Per dati live, WebSocket è spesso più adatto del polling continuo. Permette al server di inviare aggiornamenti appena disponibili. Una dashboard MicroBot può ricevere snapshot, eventi, heartbeat, metriche e warning. Questo rende l'interfaccia più reattiva.

Messaggi WebSocket possibili:

snapshot_update;
node_update;
event_update;
metric_update;
pattern_phase_update;
safety_alert;

telemetry_status;

connection_status.

Il frontend deve però gestire disconnessione. Se WebSocket cade, la dashboard deve mostrare disconnected o stale, non continuare a mostrare dati vecchi come se fossero live. La gestione dello stato di connessione è parte fondamentale della qualità web.

La diciassettesima funzione è la **API web del sistema**. Anche se la dashboard è visuale, dietro può esistere una API. Questa API deve essere coerente: status, nodes, events, patterns, runs, reports, commands. I progetti web esterni possono contribuire alla progettazione di endpoint e JSON. Una API chiara rende il sistema più modulare.

Endpoint possibili:

GET /api/status;

GET /api/snapshot;

GET /api/nodes;

GET /api/events;

GET /api/patterns;

GET /api/runs;

GET /api/reports/:id;

POST /api/commands;

POST /api/simulation/start;

POST /api/simulation/stop.

Questi endpoint devono rispettare access_control. Non tutti possono chiamare POST /api/commands. Le API read-only possono essere pubbliche o viewer. Le API operative richiedono sessione e permessi. Questa distinzione collega web e sicurezza biometrica.

La diciottesima funzione è la **gestione dei comandi web**. Un pulsante nella dashboard può generare un comando: start pattern, pause, reset, safe_idle, request_status, inject_fault. Questo comando non deve andare direttamente al nodo senza validazione. Deve essere creato come richiesta strutturata, controllata da access_control, inviata al controller e poi validata dal nodo. La UI non è autorità finale.

La pipeline corretta è:

utente clicca;

frontend crea action_request;

access_control verifica sessione;

backend/controller valida;

comando strutturato generato;

nodo riceve;

MicroBot OS valida;

Safety Layer decide;

risultato torna alla dashboard.

Questa pipeline deve essere rispettata soprattutto per hardware. Per simulation-only actions può essere più semplice, ma conviene mantenere struttura simile.

La diciannovesima funzione è la **protezione delle azioni critiche sul web**. Il web rende facile cliccare, e proprio per questo può essere pericoloso. Azioni come `clear_safety`, `hardware_test`, `modify_safety_limits`, `manage_biometric_profiles` o `enable_actuation` non devono essere pulsanti casuali. Devono richiedere `access_level`, conferma, re-auth e stato safety compatibile. La UI deve impedire click accidentali.

Azioni critiche possono richiedere:

sessione valida;

`access_level` alto;

conferma esplicita;

reauth recente;

stato sistema compatibile;

nodo selezionato;

visualizzazione del rischio;

logging obbligatorio.

Questo è fondamentale per maturità del progetto. Una dashboard tecnica deve essere bella, ma soprattutto sicura.

La ventesima funzione è la **gestione degli stati di caricamento ed errore**. Un progetto web serio non deve rompersi se i dati mancano. Se il server è offline, mostra `backend_offline`. Se snapshot manca, mostra `snapshot_unavailable`. Se telemetria è stale, mostra `stale_data`. Se API fallisce, mostra `api_error`. Se WebSocket cade, mostra `reconnecting`. Questo rende il sistema robusto.

Stati web importanti:

`loading`;

`ready`;

`backend_offline`;

`snapshot_unavailable`;

websocket_disconnected;

stale_data;

api_error;

permission_denied;

no_nodes_available;

simulation_not_running.

Questi stati sono tanto importanti quanto la grafica. Una dashboard che non gestisce errori sembra fragile. Una dashboard che spiega gli errori sembra professionale.

La ventunesima funzione è la **modalità offline o demo statica**. Per presentare MicroBot senza backend attivo, il sito può avere dati demo. Per esempio, sample snapshot, sample run, sample telemetry. Questo permette di mostrare la dashboard anche se il runtime non è avviato. Tuttavia, deve essere chiaramente indicato: demo_data_mode. Non deve sembrare live.

Modalità possibili:

live_mode;

demo_data_mode;

replay_mode;

offline_documentation_mode;

simulation_local_mode.

Questa distinzione è molto importante. Se un investitore o collaboratore vede la dashboard, deve sapere se sta guardando dati live, replay o demo statico. La chiarezza aumenta fiducia.

La ventiduesima funzione è la **integrazione con GitHub Pages o hosting statico**. La landing page, la documentazione e alcuni simulatori statici possono essere pubblicati facilmente come sito statico. Questo è utile per portfolio. La live dashboard, invece, può richiedere backend locale. La documentazione deve spiegare quali pagine sono statiche e quali richiedono runtime.

Una divisione possibile:

/docs e /landing: statici;

/demo/simulation-basic: statico;

/dashboard/live: richiede runtime;

/replay: può caricare file JSON;

/admin: richiede backend e autenticazione.

Questa struttura aiuta a progettare repository e deploy. Non tutte le pagine devono avere stesso requisito.

La ventitreesima funzione è la **struttura modulare del frontend**. Un progetto web MicroBot non dovrebbe essere un unico file enorme se cresce molto. Può partire come HTML/CSS/JS singolo per demo, ma poi conviene separare moduli: state, renderer, API client, telemetry, charts, components, controls, security. Questa modularità rende il codice mantenibile.

Una struttura frontend può essere:

index.html;

styles.css;

app.js;

api_client.js;

state_store.js;

renderer.js;

telemetry_panel.js;

node_inspector.js;

event_log.js;

charts.js;

controls.js;

security_panel.js;

utils.js.

Per demo molto semplici, un file unico può andare bene. Per dashboard avanzate, separare è meglio. La scelta dipende dallo scopo. L'importante è non mescolare tutto senza criterio.

La ventiquattresima funzione è la **accessibilità e leggibilità**. Un'interfaccia MicroBot deve essere tecnica, ma leggibile. Testi troppo piccoli, contrasti bassi, animazioni eccessive, pannelli sovrapposti o troppe informazioni rendono difficile l'uso. I progetti web esterni possono aiutare a trovare equilibrio tra estetica futuristica e chiarezza. Una dashboard deve essere usabile durante debug, non solo bella in screenshot.

Criteri di leggibilità:

contrasto sufficiente;

gerarchia visiva;

font leggibile;

numeri allineati;

stati chiari;

colori coerenti;

pannelli non troppo densi;

scroll controllato;

tooltips utili;

legende presenti.

Questo punto è molto importante. MicroBot ha molti dati; la UI deve ridurre complessità, non aumentarla.

La venticinquesima funzione è la **coerenza dei componenti UI**. Badge, card, tabelle, bottoni, grafici e alert devono avere stile e significato coerente. Se un badge warning è giallo in un pannello, deve esserlo ovunque. Se un bottone critical è rosso, non deve essere usato per azioni normali. Se una card mostra node status, tutte le card node devono avere struttura simile. Questa coerenza riduce carico cognitivo.

Componenti UI utili:

status badge;

metric card;

node card;

event row;

timeline marker;

command button;

warning banner;

inspector panel;

chart card;

modal confirm;

access level badge.

Un design system MicroBot, anche semplice, sarebbe molto utile. Permette di creare nuove pagine mantenendo identità visiva.

La ventiseiesima funzione è la **documentazione dentro il sito**. Una dashboard può avere help contestuale. Passando sopra topology_score, l'utente vede cosa significa. Cliccando su safety_state, apre spiegazione. Un pannello può avere "learn more". Questo riduce necessità di cercare documenti separati. È particolarmente utile per presentazioni.

Help contestuale può spiegare:

pattern_error;

topology_score;

aggregation_score;
safety_state;
energy_state;
thermal_state;
sync_status;
capabilities;
blocked_capabilities;
run result.

Questo rende la dashboard autoesplicativa. Una persona nuova può capire più velocemente.

La ventisettesima funzione è la **integrazione con sicurezza biometrica e access control**. I progetti web sono il luogo naturale in cui la Sezione 15 diventa visibile. La dashboard può chiedere autenticazione tramite camera, mostrare access_level, abilitare o disabilitare controlli, mostrare sessione, gestire logout e registrare security_events. Questo collega web, biometria e controllo operativo.

Una UI security può mostrare:

auth_status;
access_level;
session_timer;
camera_status;
reauth_required;
allowed_actions;
security_events;
logout;
lock controls.

Questa integrazione deve essere chiara. Un utente deve vedere se è guest, viewer, developer o admin. I controlli devono rispondere a quel livello.

La ventottesima funzione è la **valorizzazione dei progetti web esterni già sviluppati**. Se esistono già siti MicroBot, dashboard, pagine formula, siti di simulazione, portfolio o pagine con estetica avanzata, essi possono essere considerati basi o riferimenti. Alcuni possono essere integrati direttamente. Altri possono essere usati come stile. Altri possono restare portfolio. L'importante è classificarli correttamente.

Ogni progetto web esterno può essere valutato con:

nome;

scopo originale;

tecnologie usate;

stato attuale;

componenti riutilizzabili;

possibile collegamento a MicroBot;

integrazione diretta o indiretta;

lavoro necessario;

priorità.

Questa mappa aiuta a non disperdersi. Non ogni sito deve diventare dashboard ufficiale. Alcuni possono essere archiviati, altri fusi, altri trasformati in sezioni.

La ventinovesima funzione è la **roadmap dei progetti web MicroBot**. La parte web deve crescere per livelli. Prima landing page e demo statica. Poi dashboard read-only. Poi live snapshot. Poi node inspector. Poi event log. Poi grafici. Poi replay. Poi access control. Poi comandi controllati. Poi report viewer. Poi modalità presentation e developer. Questa progressione è realistica.

Una roadmap possibile è:

v0.1: landing page MicroBot;

v0.2: demo swarm statica;

v0.3: dashboard read-only con dati sample;

v0.4: collegamento a snapshot JSON;

v0.5: node inspector;

v0.6: event log filtrabile;

v0.7: metric charts;

v0.8: WebSocket live updates;

v0.9: replay viewer;

v1.0: report viewer;

v1.1: access control UI;

v1.2: simulation controls;

v1.3: developer diagnostics;

v1.4: presentation mode;

v1.5: multi-run comparison;

v1.6: hardware-safe command panel.

Questa roadmap permette di costruire la piattaforma web senza tentare tutto insieme.

Il trentesimo punto è la **sintesi del ruolo dei progetti web**. Il web è il livello che rende MicroBot visibile. Permette di comunicare il progetto, dimostrare il simulatore, osservare telemetria, ispezionare nodi, navigare eventi, generare report, controllare accessi e presentare l'ecosistema. Senza web, MicroBot resta soprattutto codice e documentazione. Con il web, diventa piattaforma. I progetti web esterni sono quindi fondamentali perché portano estetica, UI, interazione, comunicazione e architettura frontend.

In conclusione, i progetti web rappresentano un pilastro dell'ecosistema MicroBot. Essi possono assumere forma di landing page, portfolio, documentazione interattiva, simulatore browser-based, dashboard live, replay viewer, report viewer, pannello telemetrico, interfaccia di access control e ambiente developer. Il loro valore è duplice: comunicativo, perché rendono il progetto comprensibile e presentabile; operativo, perché permettono controllo, osservazione e debug. Devono però essere progettati con rigore: distinzione tra statico e live, gestione degli errori, stato stabile, API coerenti, WebSocket, access_control, Safety Layer e modalità demo/replay/live chiaramente indicate. I progetti web esterni già sviluppati possono essere riutilizzati come base estetica, tecnica o strutturale, purché integrati con ordine. Il punto successivo, dedicato alle interfacce avanzate, analizzerà come HUD, dashboard complesse, visualizzazioni 3D, AR, gesture, pannelli diagnostici, command palette e interazioni multimodali possano portare MicroBot verso un livello ancora più ricco di controllo e rappresentazione.

16.8 Interfacce avanzate

Le **interfacce avanzate** rappresentano l'ottavo ambito dei progetti esterni collegabili all'ecosistema MicroBot e descrivono tutte quelle modalità di visualizzazione, controllo e interazione che superano la semplice pagina web statica o la dashboard base. Dopo aver analizzato i progetti web, cioè il livello browser-based che permette di presentare, osservare e controllare MicroBot, è necessario approfondire le interfacce più evolute: HUD tecnici, dashboard complesse, visualizzazioni 3D, pannelli diagnostici, command palette, gesture control, interfacce AR, viste immersive, sistemi multimodali e strumenti di ispezione avanzata. Queste interfacce non sono un'aggiunta estetica superficiale, ma un modo per rendere il sistema più leggibile, esplorabile e controllabile.

Nel progetto MicroBot, il concetto di interfaccia avanzata è molto importante perché il sistema contiene molti livelli contemporaneamente: swarm simulation, nodi, ruoli, pattern, telemetria, sicurezza, MicroBot OS, safety, energia, temperatura, diagnostica, eventi, report, access control e possibili moduli AI. Una dashboard semplice può bastare per mostrare pochi dati, ma quando il sistema cresce diventa necessario creare interfacce capaci di organizzare la complessità. L'interfaccia avanzata non deve soltanto “mostrare cose belle”; deve guidare l'utente nella comprensione di un sistema distribuito complesso.

Il principio generale può essere espresso così:

un'interfaccia avanzata MicroBot deve trasformare dati complessi, stati distribuiti e processi dinamici in rappresentazioni leggibili, navigabili e tecnicamente coerenti.

Questa frase è centrale. Il problema non è solo avere tanti dati. Il problema è renderli utili. Se la dashboard mostra cento valori senza gerarchia, l'utente si perde. Se invece mostra stato globale, warning principali, nodi critici, pattern corrente, timeline, inspector e controlli contestuali, allora

l'utente capisce cosa sta succedendo. L'interfaccia avanzata deve quindi funzionare come strumento cognitivo: riduce rumore, evidenzia priorità, collega causa ed effetto e permette di passare dal generale al dettaglio.

La prima forma di interfaccia avanzata è l'**HUD tecnico**. L'HUD, o heads-up display, è una sovrapposizione informativa che mostra dati essenziali senza interrompere la visualizzazione principale. Nel caso MicroBot, l'HUD può mostrare node_count, pattern_id, current_phase, aggregation_score, topology_score, safety_state, energy_summary, thermal_summary, telemetry_status, runtime_mode e last_event. Deve essere compatto, leggibile e sempre aggiornato. L'HUD non sostituisce i pannelli dettagliati; fornisce una visione rapida dello stato generale.

Un HUD MicroBot può contenere:

runtime mode;

simulation step;

active pattern;

current phase;

online nodes;

active nodes;

topology score;

aggregation score;

safety status;

telemetry age;

last critical event.

La funzione dell'HUD è far capire subito se il sistema è sano. Se tutto è normale, l'utente vede valori stabili. Se accade qualcosa di critico, l'HUD deve evidenziarlo. Per esempio, safety_state critical deve apparire immediatamente. Telemetry stale deve essere evidente. Un nodo in thermal_lock deve emergere come warning globale. L'HUD è il primo livello di consapevolezza operativa.

La seconda forma è il **node inspector avanzato**. Un inspector base mostra dati del nodo selezionato. Un inspector avanzato può organizzare queste informazioni in sezioni: identità, stato operativo, ruolo, posizione, target, energia, temperatura, safety, comunicazione, capabilities, ultimi comandi, ultimi eventi, diagnostica MicroBot OS e storico breve. Questo permette di analizzare un singolo nodo in profondità senza aprire file o log separati.

Un inspector avanzato può includere:

identità del nodo;

boot/session info;

mode e role;

position e target;
velocity e distance_to_target;
neighbors;
expected_edges;
missing_edges;
energy_state;
thermal_state;
safety_state;
sync_status;
capabilities;
blocked_capabilities;
last command;
last event;
diagnostic status;
recommendation.

Questa interfaccia è molto utile perché un nodo MicroBot non è solo un punto sullo schermo. È un'entità con stato, ruolo, storia e vincoli. L'inspector avanzato deve far emergere questa complessità in modo ordinato. Se un comando è stato rifiutato, l'inspector deve mostrare perché. Se una capability è bloccata, deve mostrare block_reason. Se un nodo è instabile, deve mostrarne gli eventi.

La terza forma è l'**edge inspector**. In molti sistemi visuali si tende a concentrarsi sui nodi, ma in MicroBot le connessioni sono altrettanto importanti. Un edge può essere expected, formed, missing, critical, optional, unstable o forbidden. Cliccare un edge dovrebbe mostrare la distanza tra i nodi, lo stato dell'edge, il ruolo dei nodi coinvolti, la criticità, la tolleranza ammessa, la fase in cui dovrebbe formarsi e l'eventuale failure_reason. Questo è fondamentale per pattern topologici.

Un edge inspector può mostrare:

edge_id;
node_a;
node_b;
expected_status;
current_status;
distance;

target_distance;

tolerance;

criticality;

phase_required;

last_change;

failure_reason;

recommended_action.

Per esempio, in un pattern bridge, un edge mancante tra due connector può essere critico. In un cluster, un edge opzionale mancante può non compromettere il pattern. L'interfaccia deve distinguere questi casi. Senza edge inspector, l'utente vede solo una linea mancante; con edge inspector, capisce cosa significa.

La quarta forma è la **visualizzazione 3D**. Una vista 2D è sufficiente per molte simulazioni, ma una vista 3D può essere utile per rappresentare strutture, componenti hardware, campi, forze, vettori, moduli e scenari più complessi. MicroBot può usare il 3D per mostrare lo swarm nello spazio, per visualizzare un nodo esploso nei suoi componenti, per rappresentare campi magnetici o per creare una demo più immersiva. Il 3D deve però essere usato con criterio: se rende il sistema meno leggibile, non serve.

Una visualizzazione 3D può includere:

nodi come moduli tridimensionali;

edge come canali spaziali;

vettori di forza;

campo magnetico simulato;

traiettorie;

cluster nello spazio;

componenti del nodo;

vista esplosa;

wireframe;

thermal overlay;

energy overlay.

La vista 3D è particolarmente adatta per presentazioni e spiegazioni concettuali. Per debug tecnico, una vista 2D ben fatta può essere più chiara. L'interfaccia avanzata deve quindi permettere di scegliere: 2D analytics view, 3D presentation view, 3D component view, 2D debug view. Ogni vista ha uno scopo.

La quinta forma è la **vista esplosa del MicroBot**. Questa interfaccia mostra un singolo nodo come insieme di componenti: microcontrollore, alimentazione, LED, sensori, driver, bobina futura, comunicazione, firmware layer, safety layer e telemetria. Anche se alcune parti sono future o simulate, la vista esplosa può spiegare l'architettura. È molto utile per documentazione e demo, perché rende visibile ciò che normalmente è nascosto.

Una vista esplosa può mostrare:

controller embedded;

power module;

communication module;

status LED;

sensor layer;

actuator layer;

safety layer;

MicroBot OS;

telemetry output;

hardware abstraction layer.

Ogni componente può essere cliccabile. Cliccando su MicroBot OS, si vedono scheduling, shell, diagnostica. Cliccando su safety, si vedono thermal_lock, energy_state, watchdog. Cliccando su communication, si vedono heartbeat e protocollo. Questo collega interfaccia e documentazione tecnica.

La sesta forma è la **command palette**. Una command palette è un'interfaccia testuale rapida, simile a una barra di comando, che permette di cercare funzioni e lanciare azioni. In MicroBot, una command palette potrebbe accettare comandi come “show node MB-004”, “filter thermal_warning”, “start replay last run”, “toggle edge overlay”, “export report”, “jump to event 42”. Questa interfaccia è molto utile per utenti developer perché permette velocità senza navigare menu.

La command palette deve però essere soggetta ad access_control. Un utente viewer può usare comandi read-only. Un developer può usare comandi diagnostici. Un hardware_admin può usare comandi hardware-safe con conferma. La palette non deve essere un modo per bypassare i pulsanti disabilitati. Ogni comando deve passare dalla stessa pipeline di permessi.

Una command palette può supportare:

ricerca nodi;

ricerca eventi;

filtri;

apertura pannelli;

controllo replay;
toggle overlay;
export report;
azioni simulation-only;
azioni diagnostiche;
azioni critiche con conferma.

Questa interfaccia rende MicroBot più professionale perché avvicina la dashboard a strumenti tecnici moderni. È utile soprattutto quando il progetto cresce e i menu diventano numerosi.

La settima forma è il **pannello diagnostico avanzato**. Questo pannello deve raccogliere informazioni su runtime, scheduler, task, buffer, errori, risorse, API, WebSocket, telemetria, snapshot, stato backend, stato frontend e stato nodi. Deve essere pensato per sviluppo, non per presentazione. In developer mode può mostrare dati più grezzi, ma ordinati. Serve per capire se il problema è nella simulazione, nel server, nella dashboard, nella comunicazione o nei nodi.

Un pannello diagnostico avanzato può includere:

backend status;
API status;
WebSocket status;
snapshot age;
event buffer size;
telemetry rate;
frontend render FPS;
selected node diagnostics;
scheduler health;
command queue;
last API error;
last event received;
stale data indicators.

Questo pannello è fondamentale perché MicroBot è un sistema multi-layer. Quando qualcosa non funziona, bisogna localizzare il livello del problema. Il pannello diagnostico avanzato riduce il tempo di debug.

L'ottava forma è la **telemetria multilivello**. Una dashboard avanzata non deve mostrare solo telemetria globale o solo nodo singolo. Deve permettere livelli: sistema, pattern, cluster, nodo, edge,

evento. Ogni livello ha metriche diverse. A livello sistema si vedono success_rate, safety_summary, runtime. A livello pattern si vedono phase, topology_score, pattern_error. A livello cluster si vedono density, centroid, fragmentation. A livello nodo si vedono state, energy, thermal. A livello edge si vedono distanza e criticità.

Questa struttura multilivello può essere organizzata così:

global telemetry;

pattern telemetry;

cluster telemetry;

node telemetry;

edge telemetry;

security telemetry;

diagnostic telemetry.

L'interfaccia avanzata deve permettere di passare tra questi livelli senza confusione. Questo è uno dei punti più importanti per scalabilità cognitiva. Quando il sistema cresce, l'utente non può guardare ogni nodo. Deve poter leggere aggregati e poi scendere nei dettagli.

La nona forma è la **visualizzazione degli stati tramite overlay**. Un overlay è uno strato grafico attivabile sopra la simulazione. Può mostrare target positions, expected_edges, missing_edges, force vectors, cluster boundaries, heatmap energetica, heatmap termica, communication links, safety zones o telemetry freshness. L'utente può accendere e spegnere overlay in base a ciò che vuole analizzare.

Overlay possibili:

target overlay;

edge overlay;

missing edge overlay;

cluster overlay;

force vector overlay;

energy heatmap;

thermal heatmap;

communication overlay;

safety overlay;

role overlay;

sync overlay;

diagnostic overlay.

Ogni overlay deve essere semanticamente coerente. Non deve solo aggiungere colori. Deve rispondere a una domanda precisa. Per esempio, thermal heatmap risponde: quali nodi sono termicamente critici? Missing edge overlay risponde: quali connessioni attese non sono presenti? Communication overlay risponde: quali nodi sono stale o connessi male?

La decima forma è la **heatmap tecnica**. Le heatmap permettono di visualizzare intensità: energia, temperatura, errore locale, densità, rischio, latenza, stabilità. In uno swarm con molti nodi, una heatmap può essere più leggibile dei singoli valori. Per esempio, una thermal heatmap può mostrare subito una regione calda. Una density heatmap può mostrare cluster troppo compatti. Una latency heatmap può mostrare nodi con comunicazione lenta.

Heatmap utili:

energy heatmap;

thermal heatmap;

pattern error heatmap;

node density heatmap;

communication latency heatmap;

safety risk heatmap;

stability heatmap.

Le heatmap devono essere usate con legenda e valori. Non basta colorare. L'utente deve sapere cosa significa l'intensità. Inoltre, devono essere disattivabili perché possono coprire la struttura dello swarm.

L'undicesima forma è la **timeline avanzata**. Una timeline base mostra eventi. Una timeline avanzata mostra fasi, eventi critici, metriche, stato dei nodi, comandi, interventi utente, security events e cambiamenti di configurazione. Può essere zoomabile, filtrabile e collegata al replay. Se l'utente clicca un evento, la simulazione si porta a quel momento. Se seleziona una fase, la dashboard filtra gli eventi di quella fase.

Una timeline avanzata può mostrare:

phase segments;

event markers;

critical event markers;

metric curves;

user actions;

security events;

command dispatch;

node offline intervals;

recovery intervals;

safety lock intervals.

Questa interfaccia è potentissima per analisi. Permette di vedere non solo cosa è successo, ma quando e in quale ordine. In un sistema distribuito, l'ordine temporale è spesso la chiave del debug.

La dodicesima forma è il **replay sincronizzato con metriche**. Un replay avanzato non mostra solo movimento dei nodi. Sincronizza vista dello swarm, grafici, eventi, inspector e timeline. Quando il replay arriva al secondo 15, i grafici mostrano i valori del secondo 15, l'inspector mostra lo stato storico del nodo e la timeline evidenzia gli eventi. Questo trasforma il replay in uno strumento scientifico.

Un replay sincronizzato può includere:

swarm state;

metric charts;

event log;

selected node state;

phase state;

command history;

safety status;

time cursor.

Questa funzione è molto importante per documentare fallimenti. Si può vedere esattamente quando `pattern_error` è salito, quale nodo ha perso sync e quale edge è mancato. È molto più forte di un semplice video.

La tredicesima forma è la **modalità AR o realtà aumentata**. La realtà aumentata può permettere di visualizzare MicroBot sopra il mondo fisico, per esempio mostrando nodi virtuali su una superficie, componenti hardware con overlay informativi, o una struttura 3D ispezionabile con gesture. Questa è una direzione avanzata e non necessaria al core, ma molto interessante per demo e presentazione. Può collegarsi ai progetti esterni di hand tracking e Three.js.

Una modalità AR potrebbe permettere:

visualizzare uno swarm virtuale nello spazio;

selezionare nodi con la mano;

vedere telemetria sopra un modulo fisico;

mostrare campi e forze come overlay;

esplodere un modello 3D;

ispezionare componenti;

simulare aggregazione su un piano fisico.

Questa funzione deve essere trattata come futura o sperimentale se non implementata. Non deve essere presentata come core già operativo. Ma è una direzione coerente con la visione MicroBot come sistema interattivo avanzato.

La quattordicesima forma è il **gesture control**. I gesti possono essere usati per navigare la simulazione, selezionare nodi, spostare modelli, attivare overlay o controllare replay. Per esempio, pinch per selezionare, swipe per cambiare vista, due mani per esplodere un modello, mano aperta per pausa. Tuttavia, gesture control deve essere usato con cautela per azioni critiche. Non deve attivare hardware reale senza conferma.

Una pipeline gesture sicura è:

rilevamento mano;

landmark mano;

classificazione gesto;

confidence check;

mapping a intent;

access_control;

azione UI o simulazione;

conferma per azioni critiche;

logging.

Questo schema è simile alla biometria: input percettivo, interpretazione, confidenza, policy, azione. La gesture può essere molto scenografica, ma deve restare disciplinata.

La quindicesima forma è la **interfaccia multimodale**. MicroBot può combinare mouse, tastiera, camera, gesture, voce, pannelli, command palette e forse sensori. Un'interfaccia multimodale permette all'utente di scegliere il canale più naturale. Per esempio, mouse per selezionare, tastiera per comandi rapidi, voce per query read-only, gesture per presentazione, dashboard per controllo. Questa flessibilità è potente, ma può diventare caotica se non esiste una policy.

Una interfaccia multimodale deve stabilire:

quali input sono ammessi;

quali azioni possono generare;

quale access_level serve;

quale conferma è richiesta;

come vengono loggati;

come si risolvono conflitti.

Per esempio, se l'utente usa voce e mouse contemporaneamente, il sistema deve sapere cosa ha priorità. Se una gesture viene riconosciuta con bassa confidenza, non deve generare azione critica. Ogni input deve avere confidence e contesto.

La sedicesima forma è la **modalità presentation avanzata**. Una presentazione MicroBot non deve mostrare tutti i pannelli developer. Deve avere una vista pulita, narrativa, con fasi evidenziate, spiegazioni brevi, animazioni leggibili e metriche selezionate. La presentation mode può guidare lo spettatore: visione, swarm, pattern, telemetria, safety, hardware, futuro. Può essere quasi una demo interattiva controllata.

Presentation mode può includere:

hero view dello swarm;

spiegazione fase corrente;

metriche chiave;

nodi evidenziati;

animazioni lente;

nessun log rumoroso;

nessun comando critico;

preset demo;

story mode;

replay guidato.

Questa modalità è importante per comunicare MicroBot a persone non tecniche. Il sistema tecnico resta sotto, ma l'interfaccia mostra solo ciò che serve per capire.

La diciassettesima forma è la **modalità developer avanzata**. Al contrario, developer mode deve essere densa e potente. Deve offrire overlay, raw telemetry, logs, command palette, schema viewer, config editor, fault injection, performance metrics, API status, WebSocket status e diagnostics. Questa modalità è per costruire il sistema, non per presentarlo.

Developer mode può includere:

raw JSON viewer;

schema inspector;

API tester;

command log;

fault injection;

config editor;

timeline debug;
scheduler panel;
node OS diagnostics;
performance monitor;
event grouping;
export diagnostic snapshot.

Questa modalità deve essere protetta da `access_control`. Non deve essere disponibile a guest o viewer. Deve anche distinguere `simulation-only` da `hardware-connected`, per evitare errori.

La diciottesima forma è il **config editor visuale**. Un'interfaccia avanzata può permettere di modificare `pattern`, `behavior_profile`, `telemetry_level`, `visualization settings`, `simulation parameters` e `preset`. Questo editor deve essere controllato: alcune modifiche sono `live-safe`, altre richiedono `reset`, altre sono `admin-only`, altre sono `hardware-locked`. L'editor deve validare schema e mostrare errori.

Un config editor può mostrare:

`pattern config`;
`behavior profile`;
`simulation parameters`;
`fault profile`;
`visual settings`;
`telemetry settings`;
`safety policy read-only`;
`hardware profile read-only` o `admin-only`;
`schema validation`;
`diff before apply`.

Il diff è molto utile: prima di applicare una configurazione, l'utente vede cosa cambia. Per azioni critiche, serve conferma. Questo rende la modifica più sicura.

La diciannovesima forma è il **schema viewer**. MicroBot usa molti JSON: `snapshot`, `events`, `node state`, `config`, `behavior profiles`, `reports`. Un'interfaccia avanzata può mostrare lo schema di questi dati e validare i file. Questo è molto utile per developer e per documentazione. Se uno snapshot manca un campo, il viewer può segnalarlo. Se un `event_type` non è riconosciuto, può mostrarlo.

Uno schema viewer può controllare:

`snapshot schema`;
`event schema`;

node schema;
pattern schema;
behavior_profile schema;
report schema;
security_event schema;
hardware_profile schema.

Questa funzione rende il progetto più robusto. Non tutto deve essere debug a mano. La dashboard può aiutare a validare i dati.

La ventesima forma è il **performance monitor frontend**. Una dashboard avanzata deve anche monitorare sé stessa. Se il rendering scende a pochi FPS, se WebSocket riceve troppi messaggi, se la memoria cresce, se i grafici sono pesanti, l'interfaccia deve saperlo. Questo è importante per scalabilità. Un sistema che osserva lo swarm deve osservare anche il proprio carico.

Un performance monitor può mostrare:

frontend FPS;
render time;
message rate;
snapshot size;
event queue size;
memory estimate;
chart update rate;
last render delay;
dropped frames;
performance mode active.

Se lo swarm cresce, la dashboard può entrare in performance mode: meno label, meno trail, meno effetti, aggregazione per cluster. Questo rende l'interfaccia scalabile.

La ventunesima forma è la **modalità scalabile per molti nodi**. Con pochi nodi, si possono mostrare label e dettagli. Con molti nodi, bisogna aggregare. L'interfaccia avanzata deve cambiare comportamento in base al numero di nodi. Sopra una certa soglia, può nascondere label, mostrare cluster, ridurre edge, usare heatmap e filtri. Questo è essenziale per la Sezione 19 sulla scalabilità.

Modalità possibili:

small swarm mode;
medium swarm mode;

large swarm mode;

cluster aggregate mode;

performance mode;

minimal telemetry mode.

In large swarm mode, l'utente non deve vedere cento label sovrapposte. Deve vedere cluster, warning e selezione. I dettagli appaiono solo quando seleziona un nodo o filtra. Questa è progettazione avanzata dell'interfaccia.

La ventiduesima forma è la **interfaccia per digital twin ibrido**. Se MicroBot ha nodi fisici e virtuali insieme, l'interfaccia deve distinguere hardware node, simulated node e hybrid node. Deve mostrare quali dati sono reali e quali simulati. Per esempio, heartbeat reale ma posizione virtuale. Energia stimata, temperatura simulata, ruolo assegnato dal controller. Questa distinzione è fondamentale per onestà tecnica.

Una UI hybrid può mostrare:

source badge per ogni dato;

real heartbeat;

virtual position;

simulated thermal;

estimated energy;

hardware status;

mapping hardware↔virtual;

confidence;

last real update;

last virtual update.

Questo impedisce di confondere simulazione e realtà. È molto importante per demo ibride, dove alcune parti possono essere fisiche e altre virtuali.

La ventitreesima forma è il **pannello di sicurezza e accessi**. Una interfaccia avanzata deve integrare security state: auth_status, access_level, session_timer, allowed_actions, disabled_controls, security_events e reauth prompts. Questo pannello collega Sezione 15 e dashboard. L'utente deve vedere se è guest, viewer, developer o admin, e perché certe azioni sono bloccate.

Un security panel può mostrare:

current user profile o label;

access_level;

auth_method;
session status;
session expiry;
allowed actions;
restricted actions;
last security event;
reauth required actions;
logout/manual lock.

Per azioni critiche, il pannello deve mostrare conferme chiare. Non deve esserci ambiguità tra “non posso perché non sono autorizzato” e “non posso perché Safety Layer blocca”. Le ragioni devono essere separate.

La ventiquattresima forma è il **pannello Safety avanzato**. Questo pannello deve mostrare lo stato di sicurezza globale e locale: emergency_stop, thermal_lock, energy_critical, controller_lost, blocked_capabilities, active warnings, safe_outputs status, watchdog health. Deve essere sempre accessibile in hardware mode. Per MicroBot, safety non deve essere nascosta in una tabella secondaria.

Un safety panel può mostrare:

global safety state;
node safety summary;
critical nodes;
blocked capabilities;
thermal warnings;
energy warnings;
cooldown states;
watchdog state;
last safety event;
safe actions available.

Questo pannello è fondamentale per qualsiasi interfaccia che controlla hardware. Anche in simulazione, aiuta a testare la logica di safety. In hardware, diventa indispensabile.

La venticinquesima forma è la **interfaccia per report e documentazione viva**. Un’interfaccia avanzata può collegare report, replay, metriche, grafici e spiegazioni. Invece di avere un report statico separato, la dashboard può generare una pagina interattiva della run. Questa pagina può essere poi esportata. È una forma di documentazione viva, dove dati e spiegazioni sono collegati.

Un report interattivo avanzato può includere:

summary;
metric cards;
timeline;
replay embedded;
node issues;
event clusters;
failure explanation;
recommendations;
configuration diff;
export options.

Questa funzione è molto utile per costruire memoria sperimentale. Ogni run diventa conoscenza archiviata, non solo output temporaneo.

La ventiseiesima forma è la **modalità comparison avanzata**. Confrontare due run o due configurazioni è importante. Un'interfaccia avanzata può mostrare differenze nei parametri, nelle metriche, nelle timeline, nei failure modes e nei nodi problematici. Può evidenziare cosa è migliorato e cosa peggiorato. Questo è utilissimo quando si ottimizza il sistema.

Una comparison view può mostrare:

config A vs config B;
metriche affiancate;
timeline differenze;
success/failure;
pattern_error difference;
topology_score difference;
safety event difference;
energy/thermal difference;
recommendation.

Questa funzione collega algoritmi genetici, simulazioni evolutive, clustering e report. Permette di valutare se una modifica vale davvero.

La ventisettesima forma è la **interfaccia per moduli AI consultivi**. Se MicroBot integra AI per suggerimenti, classificazione run, anomaly detection o report, la dashboard deve mostrare questi

output in modo trasparente. Non deve dire solo “AI says bad”. Deve mostrare recommendation, confidence, factors e limitations. Questo rende il modulo AI interpretabile.

Un AI panel può mostrare:

risk level;

confidence;

main factors;

suggested action;

supporting metrics;

model version;

limitations;

human review required.

Questo evita di trasformare l’AI in scatola nera. L’utente può capire perché viene suggerito un fallback o una modifica.

La ventottesima forma è la **gestione del carico informativo**. Le interfacce avanzate possono facilmente diventare troppo piene. MicroBot ha molti dati, ma non tutti devono essere mostrati insieme. Serve gerarchia: overview, warnings, details, raw data. L’interfaccia deve avere progressive disclosure: mostra prima la sintesi, poi il dettaglio su richiesta. Questo è un principio fondamentale.

Una gerarchia corretta può essere:

livello 1: stato globale;

livello 2: warning e metriche principali;

livello 3: pannelli specifici;

livello 4: inspector;

livello 5: raw logs e JSON.

In questo modo, un utente non tecnico può restare ai primi livelli, mentre un developer può scendere nei dettagli. È la stessa piattaforma, ma con profondità diverse.

La ventinovesima forma è la **roadmap delle interfacce avanzate**. Anche qui serve progressione. Prima HUD e inspector. Poi overlay. Poi timeline. Poi replay sincronizzato. Poi command palette. Poi developer diagnostics. Poi 3D view. Poi gesture/AR. Poi AI panel. Questa roadmap evita di costruire interfacce enormi prima che i dati siano pronti.

Una roadmap possibile è:

v0.1: HUD tecnico base;

v0.2: node inspector avanzato;

v0.3: edge inspector;
v0.4: overlay per edge e target;
v0.5: timeline eventi;
v0.6: replay sincronizzato;
v0.7: pannello diagnostics developer;
v0.8: command palette;
v0.9: heatmap energia/termica;
v1.0: comparison view;
v1.1: config editor visuale;
v1.2: 3D swarm view;
v1.3: vista esplosa del nodo;
v1.4: gesture interaction sperimentale;
v1.5: AR/demo mode;
v1.6: AI recommendation panel;
v1.7: large swarm scalable interface.

Questa roadmap rende le interfacce avanzate una linea di sviluppo ordinata e non un accumulo casuale di effetti.

Il trentesimo punto è la **sintesi del ruolo delle interfacce avanzate**. Esse sono il livello che permette a MicroBot di essere non solo funzionante, ma comprensibile. Un sistema distribuito complesso ha bisogno di strumenti di osservazione evoluti. HUD, inspector, overlay, timeline, replay, heatmap, command palette, pannelli safety, viste 3D, AR e gesture non sono solo abbellimenti: sono strumenti per leggere lo stato, diagnosticare problemi, presentare la visione e controllare il sistema in modo responsabile.

In conclusione, le interfacce avanzate rappresentano una componente essenziale dell'ecosistema MicroBot perché trasformano dati complessi in esperienza tecnica navigabile. Esse permettono di ispezionare nodi, edge, cluster, pattern, eventi, metriche, safety, accessi, report e moduli AI attraverso HUD, pannelli, overlay, timeline, replay, viste 3D, command palette, heatmap, modalità presentation, developer mode e strumenti di confronto. Il loro sviluppo deve però restare disciplinato: ogni elemento deve avere uno scopo, rispettare access_control, distinguere simulazione e hardware, mostrare dati aggiornati e non creare confusione visiva. Le interfacce avanzate non sostituiscono il core MicroBot, ma lo rendono osservabile, spiegabile e controllabile. Il punto successivo, dedicato ai tool diagnostici, analizzerà gli strumenti software esterni e interni che permettono di controllare file, log, stati, configurazioni, errori, snapshot, report, runtime e coerenza dell'intero ecosistema MicroBot.

16.9 Tool diagnostici

I **tool diagnostici** rappresentano il nono ambito dei progetti esterni collegabili all'ecosistema MicroBot e comprendono tutti quegli strumenti software pensati per controllare, verificare, analizzare e validare lo stato del sistema. Dopo aver descritto interfacce avanzate, dashboard, visualizzazioni 3D, HUD, timeline e pannelli di ispezione, è necessario affrontare il livello più operativo e meno visibile, ma assolutamente fondamentale: gli strumenti che permettono di capire se file, configurazioni, log, snapshot, report, runtime, moduli, cartelle, dati e componenti del progetto sono coerenti. Un sistema complesso come MicroBot non può dipendere soltanto dall'osservazione manuale. Ha bisogno di tool che controllino automaticamente ciò che, a mano, diventerebbe troppo lungo, fragile e soggetto a errore.

Nel progetto MicroBot, i tool diagnostici hanno un valore enorme perché l'ecosistema è composto da molti livelli: codice Python, dashboard web, MicroBot OS, firmware, simulazioni, file JSON, documentazione, report, telemetria, configurazioni, dati sperimentali, prototipi hardware, script di avvio, moduli AI, sistemi di sicurezza e archivi di progetto. Ogni livello può generare errori diversi. Un JSON può essere malformato. Una cartella può mancare. Uno snapshot può non rispettare lo schema. Un modulo può non importare correttamente. Un file di configurazione può dichiarare una capability non supportata. Un report può mancare di metriche. Una dashboard può leggere dati vecchi. Un nodo può inviare telemetria incompleta. Un tool diagnostico serve a individuare questi problemi prima che diventino confusione durante una demo o una fase di sviluppo.

Il principio generale può essere espresso così:

un tool diagnostico MicroBot deve trasformare controlli ripetitivi, errori nascosti e incoerenze strutturali in risultati leggibili, verificabili e correggibili.

Questa frase è centrale. La diagnostica non serve solo a dire “c'è un errore”. Deve dire dove si trova, perché è rilevante, quale componente coinvolge e possibilmente come correggerlo. Un buon tool diagnostico non si limita a stampare traceback lunghi e incomprensibili. Deve produrre una sintesi: file mancante, schema non valido, campo richiesto assente, path errato, modulo non importabile, porta occupata, backend offline, snapshot obsoleto, telemetry stale, config incompatibile, report incompleto. Questa chiarezza è ciò che rende il progetto mantenibile.

La prima categoria di tool diagnostici è il **controllo della struttura delle cartelle**. MicroBot è un ecosistema grande e deve mantenere una gerarchia ordinata. Se una cartella importante manca, il sistema può rompersi o diventare difficile da navigare. Un tool diagnostico può controllare che esistano directory come core runtime, simulation engine, web platform, documentation, hardware system, MicroBot OS, AI system, analytics, releases e tools. Può verificare anche sottocartelle come configs, data, logs, reports, snapshots, assets, scripts e docs.

Un `directory_checker` può verificare:

cartelle richieste presenti;

cartelle vuote sospette;

cartelle duplicate;

nomi non coerenti;

path legacy non migrati;

file posizionati nel posto sbagliato;

cartelle temporanee da ignorare;

cartelle generate correttamente.

Questo tipo di tool è molto utile quando il progetto cresce. Se si spostano file, si generano nuove versioni o si crea una mega-repo, un controllo automatico evita caos. Il sistema può produrre un report: struttura valida, struttura parzialmente valida, cartelle mancanti, cartelle extra, suggerimenti di correzione.

La seconda categoria è il **controllo dei file obbligatori**. Ogni modulo importante dovrebbe avere alcuni file minimi: README, config, schema, entrypoint, test o documento descrittivo. Per esempio, una cartella di simulazione dovrebbe avere README.md, main script, configs, data output e docs. Una cartella hardware dovrebbe avere pin map, firmware, schema o note. Una cartella di dashboard dovrebbe avere index, styles, scripts e asset. Un tool può controllare se questi file esistono.

Un `required_files_checker` può controllare:

README.md presente;

config.json presente;

schema.json presente;

entrypoint presente;

requirements.txt o package file presente;

docs presenti;

examples presenti;

test file presenti;

manifest presente;

license o note, se richiesto.

Questo aiuta a standardizzare i repository. Un progetto grande non deve avere una cartella documentata bene e dieci cartelle senza spiegazione. I tool diagnostici possono segnalare quali moduli sono completi e quali richiedono documentazione.

La terza categoria è il **controllo dei file JSON**. MicroBot usa spesso JSON per snapshot, configurazioni, dataset registry, event logs, behavior profiles, hardware profiles, run reports, security events e mapping. Un errore JSON può bloccare la dashboard o il runtime. Un tool diagnostico deve verificare che i JSON siano sintatticamente validi, leggibili, codificati correttamente e compatibili con lo schema previsto.

Un `json_diagnostics_tool` può controllare:

JSON valido;

campi richiesti presenti;

tipi corretti;
valori nei range;
array non vuoti se richiesti;
ID coerenti;
versione schema presente;
timestamp validi;
path referenziati esistenti;
nessun campo critico mancante.

Questo tipo di tool è essenziale. Se snapshot.json manca di nodes, la dashboard non può visualizzare lo swarm. Se un behavior_profile manca di max_velocity, la simulazione non può calcolare correttamente. Se un hardware_profile dichiara pwm_output ma manca pwm_pin, il sistema deve segnalarlo prima dell'esecuzione.

La quarta categoria è il **controllo degli schemi dati**. Non basta sapere che un JSON è valido. Deve rispettare uno schema MicroBot. Uno snapshot deve avere una struttura. Un event deve avere event_type, timestamp, severity, source. Un node_state deve avere node_id, mode, role, status. Un report deve avere run_id, result, metrics, events summary. Un tool di schema validation può controllare tutto questo.

Schema importanti possono essere:

snapshot_schema;
node_state_schema;
event_schema;
run_report_schema;
behavior_profile_schema;
hardware_profile_schema;
security_event_schema;
telemetry_packet_schema;
pattern_config_schema;
diagnostic_report_schema.

Questo strumento dà ordine all'intero sistema. Se un modulo produce dati in formato sbagliato, lo schema validator lo intercetta. In un sistema distribuito, gli schemi sono il contratto tra moduli. Senza contratti, frontend, backend e simulazione rischiano di parlare linguaggi diversi.

La quinta categoria è il **controllo della telemetria**. La telemetria deve essere fresca, coerente e completa. Un tool diagnostico può leggere gli ultimi snapshot e verificare se i dati sono aggiornati,

se i nodi dichiarati sono coerenti, se i timestamp avanzano, se gli eventi hanno riferimenti validi, se i valori sono nei range e se la dashboard sta leggendo il file corretto.

Un telemetry_checker può verificare:

snapshot_age;

node_count coerente;

timestamp monotono;

node_id unici;

mode validi;

role validi;

safety_state valido;

energy_state valido;

thermal_state valido;

eventi con timestamp valido;

telemetria stale;

campi mancanti.

Se la telemetria è stale, la dashboard deve mostrarlo. Se lo snapshot non cambia da troppo tempo, il runtime può essere bloccato. Se node_count nel summary dice 10 ma l'array nodes contiene 8 nodi, c'è incoerenza. Il tool diagnostico deve intercettare questi casi.

La sesta categoria è il **controllo degli eventi**. Gli eventi sono fondamentali per capire cosa succede nel sistema. Un event log deve essere ordinato, leggibile e coerente. Un tool può controllare che ogni evento abbia event_id, timestamp, event_type, severity, source, message e payload coerente. Può anche raggruppare eventi simili e segnalare anomalie: troppi errori, troppi command_rejected, troppi reset, eventi senza source, severity non valida.

Un event_log_checker può analizzare:

eventi senza timestamp;

eventi fuori ordine;

event_type sconosciuti;

severity non valida;

payload mancante;

source non riconosciuta;

frequenza eventi anomala;

ripetizione eccessiva;

eventi critical recenti;

eventi collegati a node_id inesistenti.

Questo tool è molto utile nei test. Se una run produce molti warning, il sistema può generare un summary. Invece di leggere centinaia di righe, l'utente vede: 12 command_rejected per cooldown_active, 3 node_stale, 1 recovery_started, 1 pattern_failed.

La settima categoria è il **controllo delle configurazioni**. MicroBot può avere molti file di configurazione: simulation_config, pattern_config, behavior_profile, hardware_profile, telemetry_config, safety_policy, access_policy, AI_config. Un config_checker deve verificare che questi file siano presenti, validi, compatibili tra loro e coerenti con il sistema. Per esempio, un pattern può richiedere 12 nodi ma simulation_config ne crea 8. Oppure un behavior_profile può usare un parametro non supportato dal simulatore attuale.

Un config_checker può verificare:

config valida;

versione compatibile;

parametri nei range;

pattern compatibile con node_count;

capabilities richieste disponibili;

safety_policy caricata;

telemetry_config valida;

path di output esistente;

profili non duplicati;

nessun parametro obsoleto.

Questo strumento evita errori prima dell'avvio. È meglio scoprire subito che una config è incoerente, invece di lanciare una simulazione che fallisce dopo trenta secondi senza spiegazione.

L'ottava categoria è il **controllo delle capabilities**. I nodi MicroBot possono supportare capabilities diverse: LED, PWM, sensor_read, telemetry, shell, diagnostics, coil_future, battery_read, temperature_read. Un tool diagnostico può confrontare le capabilities richieste da un pattern o comando con quelle dichiarate dai nodi. Se un pattern richiede pwm_output ma i nodi sono LED-only, il sistema deve segnalarlo.

Un capability_checker può controllare:

capabilities installate;

capabilities disponibili;

capabilities bloccate;

capabilities richieste dal pattern;

capabilities mancanti;

capability version;

hardware_profile coerente;

blocked reasons;

unsupported commands.

Questo è particolarmente utile per evitare comandi sbagliati. Se il controller invia coil_pulse a un nodo che non supporta coil, è un errore di configurazione o assegnazione. Il tool può intercettarlo in fase di pre-run.

La nona categoria è il **controllo della sicurezza e Safety Layer**. Un safety_checker può verificare se le policy di sicurezza sono caricate, se le soglie esistono, se i valori sono conservativi, se emergency_stop è disponibile, se safe_outputs_off è definito, se watchdog è abilitato nei profili che lo richiedono, se i comandi critici hanno auto-off e cooldown. Questo tool è fondamentale per qualunque parte hardware o simulazione hardware-like.

Un safety_checker può verificare:

safety_policy presente;

emergency_stop configurato;

safe_outputs_off disponibile;

max_pwm_duty definito;

max_pulse_duration definito;

cooldown definito;

thermal thresholds presenti;

energy thresholds presenti;

watchdog policy presente;

comandi critici protetti;

nessun output critico senza auto-off.

Se un output attuativo non ha durata massima, il tool deve segnalare critical. Se un profilo hardware dichiara coil_output ma non ha thermal policy, deve essere invalidato. Questo è un controllo molto importante.

La decima categoria è il **controllo dei report**. Ogni run dovrebbe produrre report coerenti. Un report_checker può verificare che run_id, pattern_id, result, metrics, events summary, failure_reason, configuration, timestamp e versioni siano presenti. Può controllare se il report contiene tutte le sezioni richieste e se i valori principali sono coerenti con snapshot ed eventi.

Un report_checker può controllare:

run_id presente;

pattern_id presente;

result presente;

metriche principali presenti;

failure_reason se result = failed;

safety_summary presente;

node_summary presente;

event_summary presente;

config reference presente;

versioni presenti;

link a replay o snapshot presente.

Questo è importante perché i report sono memoria del progetto. Se i report sono incompleti, non si può analizzare bene l'evoluzione. Un tool può anche produrre un report_quality_score, utile per capire se una run è documentata abbastanza.

L'undicesima categoria è il **controllo dei replay**. Un replay deve avere snapshot temporali, eventi, config iniziale e metadati. Un replay_checker può verificare che la sequenza snapshot sia ordinata, che i timestamp siano coerenti, che i node_id restino consistenti, che gli eventi si riferiscano a tempi validi e che il file non sia corrotto. Questo è essenziale se la dashboard deve riprodurre run passate.

Un replay_checker può controllare:

snapshot_sequence presente;

timestamp crescenti;

node_id coerenti;

eventi dentro range temporale;

config iniziale presente;

metriche sincronizzate;

nessun frame mancante critico;

dimensione file ragionevole;

versione replay schema.

Se un replay è corrotto, la dashboard deve saperlo prima di provare a caricarlo. Il tool può dire: replay valido, replay parziale, replay non valido.

La dodicesima categoria è il **controllo dei moduli Python**. Molti componenti MicroBot possono essere in Python: simulatore, runtime, launcher, analytics, report generator, tool. Un `python_module_checker` può verificare import, dipendenze, entrypoint, percorsi, presenza di `init.py` se necessario, moduli mancanti e conflitti di path. Questo è molto utile perché errori come `ModuleNotFoundError` possono bloccare tutto.

Un `python_checker` può verificare:

file entrypoint esiste;

moduli importabili;

dipendenze installate;

requirements presenti;

path corretto;

nessun import rotto;

nessun modulo duplicato;

versione Python compatibile;

venv rilevata;

errori comuni documentati.

Questo tool può essere integrato in un launcher diagnostico: prima di avviare tutto, controlla se `pygame`, `flask`, `flask_socketio` o altre dipendenze sono presenti. Se manca qualcosa, stampa comando suggerito. Così si evita di scoprire l'errore solo a runtime.

La tredicesima categoria è il **controllo del frontend web**. Una dashboard può avere file HTML, CSS, JS, asset, moduli, API client. Un `web_checker` può verificare file presenti, riferimenti corretti, asset mancanti, script non trovati, CSS duplicato, API endpoint configurati, modalità demo/live, e possibili errori di connessione.

Un `web_checker` può controllare:

`index.html` presente;

CSS presente;

JS presente;

asset referenziati esistenti;

endpoint configurati;

WebSocket URL valido;

demo data presente;

nessun link rotto interno;

modalità live/demo dichiarata;

file troppo grandi o duplicati.

Questo aiuta molto quando si fondono più siti. Se una pagina punta a assets/logo.png ma il file è stato spostato, il tool lo segnala. Se uno script viene caricato in ordine sbagliato, può essere segnalato almeno in parte.

La quattordicesima categoria è il **controllo delle performance**. MicroBot può avere dashboard pesanti, snapshot grandi, molti eventi, simulazioni complesse. Un performance_diagnostic_tool può misurare dimensione snapshot, numero nodi, numero edge, tempo di parsing, dimensione report, frequenza eventi e carico stimato. Può suggerire quando passare a modalità performance.

Un performance_checker può misurare:

snapshot_size;

node_count;

edge_count;

event_count;

message_rate;

render_estimate;

file_size;

report_size;

replay_size;

processing_time.

Se snapshot_size diventa troppo grande, il tool può suggerire delta updates. Se event_count è enorme, può suggerire grouping. Se node_count è alto, può suggerire cluster view. Questo collega diagnostica e scalabilità.

La quindicesima categoria è il **controllo della coerenza tra documentazione e codice**. In un progetto grande, la documentazione può dire una cosa e il codice un'altra. Un doc_code_checker può verificare se i moduli citati nella documentazione esistono, se i file README menzionano endpoint reali, se i nomi delle cartelle corrispondono, se i comandi di avvio sono aggiornati. Non può capire tutto semanticamente, ma può intercettare molte incoerenze.

Un controllo utile può verificare:

path citati nei README esistono;

script di avvio citati esistono;

nomi moduli coerenti;

versioni dichiarate coerenti;

link interni validi;

file documentati presenti;

file presenti ma non documentati;

TODO aperti.

Questo tool è importantissimo per una tesi o mega-repo. La coerenza tra testo e file aumenta credibilità. Se un documento dice “avviare `run_live_demo.py`” ma il file non esiste, il lettore perde fiducia.

La sedicesima categoria è il **controllo dei link e riferimenti interni**. La documentazione MicroBot può contenere molti link a file, immagini, diagrammi, report, sezioni. Un `link_checker` può verificare che i riferimenti interni siano validi. Può controllare link relativi, immagini mancanti, anchor mancanti, file spostati. Questo è molto utile se si genera documentazione grande.

Un `link_checker` può verificare:

link Markdown validi;

immagini presenti;

anchor interne;

riferimenti a PDF;

riferimenti a report;

riferimenti a file JSON;

riferimenti a codice;

path relativi corretti.

Questo migliora moltissimo la qualità finale della documentazione. Una documentazione lunga ma piena di link rotti sembra poco curata. Un tool può prevenire il problema.

La diciassettesima categoria è il **controllo dei nomi e naming convention**. MicroBot deve evitare nomi confusi: `microbot`, `MicroBot`, `Microbot`; `core_runtime`, `CORE_RUNTIME`; `simulationEngine`, `simulation_engine`. Un `naming_checker` può segnalare incoerenze, file con spazi problematici, maiuscole/minuscole non coerenti, duplicati e nomi legacy. Questo è utile soprattutto quando si uniscono molti progetti esterni.

Un `naming_checker` può rilevare:

nomi cartelle non standard;

file duplicati con maiuscole diverse;

spazi non desiderati;

caratteri speciali;

versioni nel nome non coerenti;

prefissi mancanti;

naming legacy;

conflitti tra moduli.

Questo non è solo estetica. Su sistemi diversi, maiuscole e minuscole possono creare problemi. Path con spazi possono rompere script. Nomi incoerenti rendono il progetto più difficile da usare.

La diciottesima categoria è il **controllo dei duplicati**. Nei progetti grandi, spesso si accumulano file duplicati: vecchie versioni di index.html, copie di script, report ripetuti, immagini duplicate, dataset doppi. Un duplicate_checker può confrontare nomi, dimensioni, hash o contenuto simile. Può segnalare duplicati esatti e possibili duplicati.

Un duplicate_checker può indicare:

file identici;

file quasi identici;

nomi simili;

versioni multiple;

script duplicati;

asset duplicati;

documenti duplicati;

cartelle legacy sovrapposte.

Questo aiuta a pulire il progetto prima di pubblicarlo. Una repo con duplicati sembra meno curata e pesa di più. Naturalmente, alcuni duplicati possono essere intenzionali, ma devono essere marcati.

La diciannovesima categoria è il **controllo delle versioni**. MicroBot ha versioni di firmware, schema, simulatore, dashboard, documenti, report, behavior profiles. Un version_checker può verificare se le versioni sono presenti e coerenti. Per esempio, uno snapshot può avere schema_version 0.4 ma la dashboard richiede 0.5. Un report può mancare di simulation_version. Un firmware può non dichiarare protocol_version.

Un version_checker può controllare:

firmware_version;

protocol_version;

schema_version;

simulation_version;

dashboard_version;

behavior_profile_version;

hardware_profile_version;

report_version;

AI model version;

config version.

Questo è fondamentale per riproducibilità. Se non si sa quale versione ha prodotto un risultato, il report perde valore. Il version checker aiuta a mantenere ordine.

La ventesima categoria è il **controllo della compatibilità**. Dopo aver letto versioni e capabilities, un compatibility_checker può decidere se moduli diversi possono lavorare insieme. Per esempio, dashboard v0.8 può leggere snapshot schema v0.7? Runtime v0.5 può comunicare con node protocol v0.4? Behavior profile v1.0 è compatibile con simulation engine v0.9? Questa diagnostica evita errori sottili.

Un compatibility_checker può verificare:

dashboard ↔ snapshot schema;

runtime ↔ node protocol;

controller ↔ firmware;

simulation ↔ behavior profile;

report generator ↔ run report schema;

web UI ↔ API version;

hardware profile ↔ safety policy;

AI model ↔ feature schema.

Se incompatibile, il tool deve indicare: upgrade required, downgrade unsupported, migration needed, schema mismatch. Questo è molto utile quando il progetto evolve.

La ventunesima categoria è il **controllo dei launcher**. MicroBot può avere script di avvio per simulazione, runtime, dashboard, BCI, AI, server. Un launcher_checker può verificare che i path siano corretti, le porte libere, le dipendenze installate, le cartelle output presenti e gli entrypoint esistenti. Questo può evitare molti errori pratici.

Un launcher_checker può controllare:

entrypoint simulation presente;

entrypoint runtime presente;

dashboard path presente;

porta backend libera;

porta websocket libera;

cartelle logs/output presenti;

venv attiva o rilevata;

dipendenze installate;

permessi file;

comando avvio valido.

Se qualcosa manca, il tool deve stampare un messaggio chiaro. Per esempio: “manca flask_socketio; installare con pip install flask-socketio”. Oppure: “porta 8765 occupata”. Questi tool fanno risparmiare tempo, parecchio.

La ventiduesima categoria è il **controllo dell’ambiente di sviluppo**. Un environment_checker può verificare sistema operativo, versione Python, Node, npm, pacchetti, variabili ambiente, path, venv, permessi, spazio disco. Questo è utile quando il progetto deve girare su Mac, Windows o Linux. Molti errori non sono nel codice, ma nell’ambiente.

Un environment_checker può mostrare:

OS;

Python version;

Node version;

pip packages;

venv path;

working directory;

environment variables;

disk space;

write permissions;

GPU availability, se rilevante;

browser support, se web.

Questo tool può generare un environment_report da allegare ai bug report. Se un collaboratore ha problemi, manda il report e si capisce subito la situazione.

La ventitreesima categoria è il **controllo dei log runtime**. Un log_analyzer può leggere i log generati da simulazione, runtime, dashboard o firmware e cercare pattern noti: ModuleNotFoundError, import error, file not found, port already in use, JSON decode error, schema validation failed, WebSocket disconnected. Può raggruppare errori e produrre suggerimenti.

Un log_analyzer può rilevare:

errori import;

file mancanti;
JSON malformati;
porta occupata;
backend offline;
WebSocket error;
telemetry stale;
config invalid;
permission denied;
traceback ricorrenti.

Questo tipo di tool è molto utile perché non sempre l'utente sa interpretare traceback lunghi. Il tool può dire: problema principale = dipendenza mancante; suggerimento = installare pacchetto o usare venv corretto.

La ventiquattresima categoria è il **diagnostic snapshot dell'ecosistema**. Un tool molto potente può generare uno snapshot diagnostico completo del progetto: struttura cartelle, versioni, moduli, config, ultimi log, ultimi report, stato telemetria, stato dashboard, errori principali. Questo snapshot può essere salvato come diagnostic_snapshot.json o .md. È il corrispettivo a livello ecosistema della diagnostica di un nodo MicroBot OS.

Un ecosystem_diagnostic_snapshot può contenere:

timestamp;
project_root;
directory_status;
required_files_status;
config_status;
schema_status;
telemetry_status;
runtime_status;
web_status;
last_errors;
environment_status;
recommendations.

Questo file è utilissimo prima di mandare il progetto a qualcuno, prima di pubblicare su GitHub o prima di creare uno zip. Permette di sapere se il progetto è coerente.

La venticinquesima categoria è il **controllo pre-release**. Prima di creare una release, un tool può verificare se tutto è pronto: documentazione presente, README aggiornati, file temporanei rimossi, dati sensibili esclusi, demo funzionanti, report presenti, schema validi, duplicati ridotti, cartelle corrette. Questo è essenziale se MicroBot deve essere pubblicato o mandato a un collaboratore.

Un `pre_release_checker` può controllare:

README principale presente;

quickstart presente;

nessun file sensibile;

nessun dato biometrico reale;

nessun secret;

demo data valida;

script avvio validi;

documentazione minima completa;

licenza o note;

changelog;

release manifest;

cartelle output pulite.

Questo tool protegge da errori grossi, tipo caricare file personali, dati sensibili, cache, venv, build pesanti o template biometrici. È un passaggio importante per professionalizzare il progetto.

La ventiseiesima categoria è il **controllo di dati sensibili**. MicroBot può includere sicurezza biometrica, eventuali dati personali, path locali, log, credenziali, chiavi API, file ambientali. Un `sensitive_data_checker` deve cercare pattern per evitare che dati sensibili finiscano nella repo o nello zip. Questo è importantissimo, soprattutto se il progetto viene condiviso.

Un `sensitive_checker` può cercare:

API keys;

password;

token;

.env;

path personali eccessivi;

dati biometrici reali;

immagini volto;
template biometrici;
log con dati personali;
file di sistema;
venv;
cache;
dataset non autorizzati.

Questo tool non è perfetto, ma aiuta. In una release pubblica, dati biometrici reali non devono esserci. Nemmeno token o file .env. Questo è un controllo di sicurezza basilare ma fondamentale.

La ventisettesima categoria è il **controllo della documentazione lunga**. Se MicroBot ha documenti molto estesi, un doc_diagnostics_tool può verificare indice, sezioni mancanti, numerazione, titoli duplicati, riferimenti a figure, riferimenti a tabelle, capitoli incompleti e coerenza dei nomi. Questo è utilissimo per tesi e documentazione tecnica.

Un doc_checker può controllare:

titoli duplicati;
sezioni mancanti;
numerazione incoerente;
indice non aggiornato;
riferimenti a figure mancanti;
riferimenti a tabelle mancanti;
TODO rimasti;
terminologia incoerente;
capitoli troppo vuoti;
link rotti.

Questo strumento può essere molto utile quando la documentazione arriva a centinaia di pagine. A mano diventa impossibile controllare tutto.

La ventottesima categoria è il **tool di raccomandazione diagnostica**. Un tool avanzato non si limita a indicare errori, ma suggerisce priorità. Per esempio, se mancano README, è warning. Se manca safety_policy per hardware profile attuativo, è critical. Se ci sono duplicati, è info/warning. Se snapshot schema non è compatibile con dashboard, è error. Questo ranking aiuta a decidere cosa correggere prima.

Le severità possono essere:

INFO: nota utile;

WARNING: problema non bloccante;

ERROR: problema che impedisce una funzione;

CRITICAL: problema che può compromettere safety, release o funzionamento principale.

Il tool può produrre una lista ordinata:

prima correggere errori critical;

poi errori runtime;

poi warning di documentazione;

poi pulizia e duplicati.

Questo evita di perdersi. In un progetto grande, non tutti i problemi hanno stesso peso.

La ventinovesima categoria è la **roadmap dei tool diagnostici**. Anche questi strumenti devono crescere per livelli. Prima controlli semplici: cartelle, file, JSON. Poi schema validation. Poi telemetry checker. Poi log analyzer. Poi compatibility checker. Poi pre-release checker. Poi diagnostic snapshot. Poi dashboard diagnostica. Questa progressione è realistica e molto utile.

Una roadmap possibile è:

v0.1: directory_checker;

v0.2: required_files_checker;

v0.3: json_validator;

v0.4: schema_validator;

v0.5: config_checker;

v0.6: telemetry_checker;

v0.7: event_log_checker;

v0.8: report_checker;

v0.9: replay_checker;

v1.0: environment_checker;

v1.1: launcher_checker;

v1.2: log_analyzer;

v1.3: compatibility_checker;

v1.4: sensitive_data_checker;

v1.5: pre_release_checker;

v1.6: ecosystem_diagnostic_snapshot;

v1.7: dashboard_diagnostics_integration.

Questa roadmap trasforma i tool diagnostici in un vero sottosistema MicroBot, non in script isolati.

Il trentesimo punto è la **sintesi del ruolo dei tool diagnostici**. I tool diagnostici sono ciò che impedisce al progetto di diventare ingestibile. Più MicroBot cresce, più aumentano file, moduli, configurazioni, dati, report, dashboard, simulazioni e versioni. Senza strumenti automatici, ogni controllo diventa manuale e fragile. Con strumenti diagnostici, invece, il progetto può essere verificato, pulito, validato e preparato per demo, release, GitHub o consegna a collaboratori.

In conclusione, i tool diagnostici rappresentano una componente fondamentale dell'ecosistema MicroBot perché permettono di controllare struttura, file, JSON, schemi, telemetria, eventi, configurazioni, capabilities, safety policy, report, replay, moduli Python, frontend web, performance, documentazione, link, naming, duplicati, versioni, compatibilità, launcher, ambiente, log, dati sensibili e stato pre-release. Essi trasformano il progetto da insieme di file complessi a sistema verificabile. Un buon tool diagnostico non deve solo trovare errori, ma classificarli, spiegarli e suggerire priorità di correzione. Il punto successivo, dedicato agli script di sistema, completerà la Sezione 16 analizzando quegli script pratici che creano cartelle, generano file, avviano moduli, automatizzano build, esportano report, puliscono dati, preparano release e mantengono operativa l'intera infrastruttura MicroBot.

16.10 Script di sistema

Gli **script di sistema** rappresentano il decimo e ultimo ambito della Sezione 16 dedicata ai progetti esterni. Essi comprendono tutti quei programmi pratici, spesso scritti in Python, Bash, JavaScript o altri linguaggi di supporto, che non costituiscono necessariamente il cuore teorico di MicroBot, ma permettono al progetto di essere creato, organizzato, avviato, controllato, pulito, esportato e mantenuto nel tempo. Dopo aver analizzato tool diagnostici, interfacce avanzate, progetti web, algoritmi genetici, simulazioni evolutive, giochi, clustering e intelligenza artificiale, è necessario chiudere la sezione con una parte molto concreta: gli script che tengono insieme l'infrastruttura del progetto.

Nel contesto MicroBot, gli script di sistema sono fondamentali perché l'ecosistema è grande, modulare e composto da molte cartelle, file, configurazioni, demo, documenti, runtime, dashboard, simulatori, report, dati e strumenti. Gestire tutto manualmente sarebbe lento e rischioso. Ogni volta che si crea una nuova sezione del progetto, si rischia di dimenticare cartelle, README, file di configurazione, sottocartelle data, log, output o docs. Ogni volta che si avvia il sistema, si rischia di lanciare moduli nel posto sbagliato, usare un ambiente Python errato, dimenticare una dipendenza o non creare le cartelle di output. Gli script di sistema servono a ridurre questo caos.

Il principio generale può essere espresso così:

gli script di sistema trasformano operazioni ripetitive e fragili in procedure automatiche, riproducibili e documentabili.

Questa frase è centrale. Uno script non è solo un pezzo di codice che “fa una cosa”. È una procedura formalizzata. Se una cartella deve essere creata sempre nello stesso modo, lo script la crea. Se una demo deve essere avviata con gli stessi moduli, lo script li lancia. Se una release deve contenere sempre gli stessi file, lo script li raccoglie. Se un report deve essere esportato con struttura

coerente, lo script lo genera. Questo dà al progetto una qualità molto più professionale, perché riduce improvvisazione e aumenta ripetibilità.

La prima categoria è rappresentata dagli **script di creazione struttura**. Questi script generano cartelle, sottocartelle, file base, README, placeholder, configurazioni iniziali e manifest. Sono molto utili in MicroBot perché l'ecosistema contiene molte sezioni: core runtime, moduli, simulazione, BCI, documentazione, web platform, hardware, MicroBot OS, AI, research lab, analytics, deployment, startup material, releases. Creare a mano queste strutture è possibile, ma diventa pesante e incoerente. Uno script, invece, può generare sempre lo stesso scheletro.

Uno structure_generator può creare:

cartelle principali;

sottocartelle standard;

README.md iniziali;

config.json base;

schema.json placeholder;

docs/;

data/input;

data/output;

logs/;

reports/;

tests/;

examples/;

manifest.json.

Questo tipo di script è molto coerente con il modo in cui MicroBot sta crescendo. Ogni sezione può avere il proprio generatore. Per esempio, uno script per 08_MICROBOT_OS/boot, uno per build system, uno per diagnostics, uno per drivers, uno per docs, uno per simulation engine, uno per web platform. La cosa importante è che questi generatori non producano strutture casuali, ma rispettino una convenzione generale.

La seconda categoria è rappresentata dagli **script di inizializzazione del progetto**. Questi script preparano l'ambiente prima dell'uso. Possono creare cartelle output, verificare file minimi, generare config iniziali se mancano, controllare che il progetto sia lanciato dalla root corretta e stampare una sintesi. Sono diversi dai generatori di struttura perché non creano necessariamente un nuovo modulo; preparano il progetto esistente all'esecuzione.

Uno startup_init.py può fare:

rilevare project_root;

creare data/output se manca;
creare logs se manca;
controllare config principale;
controllare snapshot iniziale;
controllare cartelle dashboard;
controllare ambiente Python;
stampare percorsi principali;
preparare file temporanei.

Questo tipo di script evita errori banali. Se il runtime si aspetta una cartella logs e questa non esiste, può fallire. Lo script la crea prima. Se uno snapshot iniziale manca, può generarne uno vuoto. Se il progetto viene eseguito dalla directory sbagliata, può avvisare. Sono dettagli piccoli, ma fanno enorme differenza nella stabilità percepita.

La terza categoria è rappresentata dagli **script di avvio**. MicroBot può richiedere l'avvio di più moduli: simulazione, runtime, server web, dashboard, WebSocket, eventuale BCI, AI module, logging, report viewer. Avviare tutto manualmente può essere scomodo. Uno script launcher può orchestrare l'avvio, mostrare stato, aprire URL, controllare errori e fermare processi. Questo è molto utile per demo.

Un launcher MicroBot può gestire:

simulation engine;
core runtime;
dashboard server;
WebSocket bridge;
telemetry service;
report generator;
AI analyzer;
hardware bridge;
log monitor.

Il launcher deve essere chiaro. Deve stampare quali moduli sta avviando, con quali path, su quali porte e con quale interprete Python. Deve indicare se un modulo fallisce. Deve evitare di nascondere errori. Un buon launcher non deve dire soltanto “qualcosa è andato storto”, ma “simulation failed: ModuleNotFoundError pygame” oppure “dashboard port already in use”. Questo collega script di sistema e tool diagnostici.

La quarta categoria è rappresentata dagli **script di stop e cleanup runtime**. Se un launcher avvia più processi, serve anche un modo ordinato per fermarli. Uno stop script può terminare processi,

chiudere server, liberare porte, salvare log finali e generare summary. Questo è importante perché durante sviluppo si possono lasciare processi aperti, porte occupate e file temporanei. Uno script di cleanup riduce questi problemi.

Uno `stop_runtime.py` può:

fermarsi in modo graceful;

inviare shutdown event;

chiudere WebSocket;

fermare simulation process;

fermare server dashboard;

salvare ultimo snapshot;

scrivere runtime_summary;

liberare lock file;

stampare stato finale.

Questo è particolarmente utile quando si eseguono demo ripetute. Ogni run deve partire da stato pulito. Se restano processi vecchi, i risultati possono essere confusi. Lo script di stop protegge la ripetibilità.

La quinta categoria è rappresentata dagli **script di generazione dati**. MicroBot ha bisogno di dati demo, snapshot iniziali, run simulate, eventi di esempio, nodi virtuali e report fittizi per testare dashboard e documentazione. Uno script può generare questi dati in modo coerente. Questo è molto utile quando il backend live non è attivo, ma si vuole mostrare la dashboard.

Uno `data_generator` può creare:

`sample_nodes.json`;

`sample_snapshot.json`;

`sample_events.json`;

`sample_run_report.json`;

`sample_replay.json`;

`sample_telemetry_stream.json`;

`sample_security_events.json`;

`sample_pattern_config.json`.

Questi dati devono essere chiaramente marcati come demo o synthetic. Non devono essere confusi con dati reali. La dashboard deve poter indicare `demo_data_mode`. Questa distinzione è importante per onestà tecnica. I dati sintetici servono a testare interfacce, non a dimostrare hardware reale.

La sesta categoria è rappresentata dagli **script di esportazione**. MicroBot può produrre report, snapshot, grafici, documenti, pacchetti, dataset e release. Uno script export può raccogliere questi elementi e salvarli in una cartella ordinata. Per esempio, dopo una run può esportare report.md, metrics.json, events.json, replay.json, charts.png e summary.txt. Questo rende ogni esperimento archiviabile.

Uno export_run.py può generare:

run_summary.md;

metrics.json;

events.json;

snapshot_final.json;

replay.json;

node_report.csv;

charts/;

diagnostic_summary.json;

README_run.md.

Questo tipo di script è molto utile per la documentazione. Ogni run diventa una cartella con dati e spiegazioni. In futuro, queste cartelle possono essere usate per analisi AI, confronto tra versioni, grafici e tesi. L'esportazione ordinata è la base della ricerca riproducibile.

La settima categoria è rappresentata dagli **script di generazione report**. Un report generator prende dati grezzi e produce testo strutturato. Può leggere metriche, eventi, configurazione, failure reason, nodi problematici e produrre un documento Markdown o HTML. Questo collega telemetria, analytics e documentazione. È uno strumento estremamente utile per MicroBot, perché il progetto produce molti dati che devono diventare spiegazioni.

Un report_generator può includere:

introduzione della run;

configurazione usata;

pattern eseguito;

risultato finale;

metriche principali;

eventi critici;

analisi nodi;

safety summary;

energy summary;

thermal summary;

failure analysis;

raccomandazioni;

prossimi test.

Il report non deve inventare risultati. Deve basarsi sui dati disponibili. Se un dato manca, deve indicarlo. Per esempio: “thermal data unavailable” invece di creare una temperatura fittizia. Questo mantiene rigore.

L’ottava categoria è rappresentata dagli **script di conversione documentale**. MicroBot può avere documenti in Markdown, TXT, JSON, HTML, LaTeX, PDF o DOCX. Uno script può convertire contenuti da un formato all’altro, preparare sezioni, generare indici, unire file, creare export o aggiornare riferimenti. Questi script sono utili quando la documentazione cresce molto.

Uno document_converter può fare:

Markdown → HTML;

Markdown → PDF;

TXT → Markdown;

JSON report → Markdown;

più file Markdown → documento unico;

indice automatico;

estrazione titoli;

normalizzazione sezioni;

controllo numerazione.

Questi script devono essere usati con attenzione perché la documentazione lunga può rompersi facilmente. Un buon converter deve preservare titoli, paragrafi, tabelle e codice. Deve evitare di perdere formattazione. Per una tesi MicroBot, strumenti di questo tipo possono diventare fondamentali.

La nona categoria è rappresentata dagli **script di pulizia**. Un progetto grande accumula file temporanei, cache, log vecchi, build, output duplicati, **pycache**, node_modules, venv, file .DS_Store, report provvisori e cartelle di test. Uno cleanup script può rimuovere ciò che non serve o spostarlo in archive. Questo mantiene il progetto leggero e ordinato.

Uno cleanup_project.py può gestire:

pycache;

.DS_Store;

cache frontend;

log vecchi;
output temporanei;
build vecchie;
file duplicati noti;
cartelle vuote;
report provvisori;
snapshot temporanei.

Deve però essere prudente. Uno script di pulizia non deve cancellare dati importanti senza conferma o senza dry-run. Una modalità dry-run è fondamentale: mostra cosa cancellerebbe, ma non cancella. Solo dopo conferma o flag esplicito può rimuovere. Questo evita disastri, perché in un progetto grande cancellare la cartella sbagliata è una bella rottura.

La decima categoria è rappresentata dagli **script di archiviazione**. Non tutto va cancellato. Alcuni file vecchi vanno conservati ma spostati in archive. MicroBot può avere versioni legacy, vecchi esperimenti, run storiche, siti precedenti, prototipi, documenti superati. Uno script archive può spostare questi elementi in modo ordinato, mantenendo manifest e metadati.

Uno archive_tool può creare:

archive/date/;
archive/legacy_sites/;
archive/old_runs/;
archive/deprecated_configs/;
archive/old_reports/;
archive/experimental/;
archive_manifest.json.

Questo è molto utile quando si consolida il progetto. Invece di buttare via vecchi lavori, li si archivia. Così il core resta pulito, ma la storia non viene persa. È una pratica molto sana.

L'undicesima categoria è rappresentata dagli **script di migrazione**. Quando il progetto cambia struttura, i file devono essere spostati da vecchi path a nuovi path. Uno migration script può leggere una mappa old_to_new e spostare file, aggiornare riferimenti, creare redirect o manifest. Questo è fondamentale quando si uniscono molte repo o si crea una mega-repo MicroBot.

Uno migration_tool può gestire:

lettura migration_map.json;
creazione nuove cartelle;
spostamento file;

copia controllata;

aggiornamento riferimenti;

log delle operazioni;

controllo file mancanti;

report finale.

La migrazione deve essere tracciata. Ogni file spostato deve essere registrato. Se un file non viene trovato, il tool deve segnalarlo. Se un file esiste già nel nuovo path, deve decidere se saltare, rinominare o chiedere conferma. Questo evita sovrascritture accidentali.

La dodicesima categoria è rappresentata dagli **script di build**. Alcune parti del progetto possono richiedere build: frontend web, documentazione, firmware, MicroBot OS, pacchetti, asset. Uno build script può preparare il risultato finale, copiando file, minificando se necessario, generando asset, compilando o organizzando output. In MicroBot OS, build system può essere un ambito specifico. Nel web, build può preparare la cartella dist.

Uno build_script può fare:

validare config;

preparare dist/;

copiare asset;

generare bundle;

compilare documentazione;

compilare firmware, se previsto;

generare manifest;

scrivere build_info.json;

produrre log.

La build deve essere riproducibile. Se si esegue due volte con gli stessi input, dovrebbe produrre risultato coerente. Deve anche registrare versione, timestamp, commit se disponibile e ambiente. Questo aiuta molto nelle release.

La tredicesima categoria è rappresentata dagli **script di test automatico**. Anche se MicroBot è grande, alcuni test possono essere automatizzati. Testare schema, import, config, funzioni di simulazione, report generator, parser, API, frontend statico, link e tool. Questi script non sostituiscono test manuali, ma riducono errori.

Uno test_runner può eseguire:

schema tests;

config tests;

import tests;

simulation smoke test;

dashboard data load test;

report generator test;

event parser test;

API endpoint test;

diagnostic tool test;

file structure test.

Un smoke test è particolarmente utile: avvia una simulazione minima, genera snapshot, verifica che il report venga prodotto e che non ci siano errori critici. Questo dà una conferma rapida che il sistema base funziona.

La quattordicesima categoria è rappresentata dagli **script di validazione pre-demo**. Prima di mostrare MicroBot a qualcuno, serve sapere se tutto è pronto. Uno script `pre_demo_check` può controllare runtime, dashboard, dati demo, replay, report, browser path, porte, dipendenze, file mancanti e stato del progetto. È simile a un pre-release checker, ma orientato alla presentazione.

Un `pre_demo_check` può controllare:

landing page presente;

dashboard avviabile;

dati demo validi;

replay disponibili;

script launcher funzionante;

dipendenze installate;

porte libere;

nessun errore recente critical;

report demo presenti;

README demo aggiornato.

Il risultato può essere:

DEMO_READY;

DEMO_READY_WITH_WARNINGS;

DEMO_BLOCKED.

Questo è utilissimo prima di mandare materiale a qualcuno o fare una presentazione. Evita figuracce tecniche.

La quindicesima categoria è rappresentata dagli **script di preparazione release**. Una release MicroBot deve contenere file selezionati, documentazione, demo, dati sample, README, changelog e manifest. Non deve contenere cache, venv, dati sensibili, file personali, build temporanee o materiale non autorizzato. Uno script `release_builder` può creare un pacchetto ordinato.

Un `release_builder` può generare:

`release_folder/;`

`README_RELEASE.md;`

`CHANGELOG.md;`

`manifest.json;`

`docs/;`

`demo/;`

`src/;`

`configs/;`

`sample_data/;`

`reports/;`

`checksums;`

`release_notes.md.`

Questo script deve anche escludere dati sensibili. Deve avere una lista di ignore: `.env`, `token`, `biometric data`, `venv`, `node_modules`, `cache`, **`pycache`**, file personali, grandi archivi non necessari. Una release pulita è una prova di maturità del progetto.

La sedicesima categoria è rappresentata dagli **script di packaging ZIP**. Se bisogna mandare il progetto a un collaboratore, a un altro tool o archiviarlo, può servire uno zip. Uno `zip_builder` deve essere selettivo. Non deve zippare tutto ciecamente, soprattutto se il progetto è enorme. Deve poter creare pacchetti diversi: `light demo`, `full docs`, `code only`, `web demo`, `hardware package`, `research archive`.

Tipi di pacchetto possibili:

`microbot_demo_light.zip;`

`microbot_docs_full.zip;`

`microbot_web_platform.zip;`

`microbot_simulation_package.zip;`

microbot_hardware_docs.zip;

microbot_release_full.zip;

microbot_for_collaborator.zip.

Ogni pacchetto deve avere manifest. Il manifest dice cosa contiene, cosa non contiene, versione, data, scopo e note. Questo evita confusione quando si mandano zip diversi a persone diverse.

La diciassettesima categoria è rappresentata dagli **script di sincronizzazione tra moduli**. Se un pattern_config viene usato da simulazione, dashboard e report generator, serve coerenza. Uno sync script può copiare o aggiornare file condivisi, oppure verificare che le versioni siano allineate. Per esempio, se event_schema cambia, dashboard e backend devono usare la stessa versione. Uno script può controllarlo.

Uno sync_checker può verificare:

schema condivisi allineati;

config comuni aggiornati;

versioni coerenti;

file copiati nei posti giusti;

nessun modulo usa schema vecchio;

manifest aggiornato.

Questo è importante perché nei progetti modulari la duplicazione di schema crea problemi. Idealmente, gli schema stanno in un posto centrale. Se devono essere copiati, lo script deve farlo in modo controllato.

La diciottesima categoria è rappresentata dagli **script di generazione indici**. MicroBot ha molti documenti, file, moduli, report, script e asset. Uno index_generator può creare indici automatici: elenco documenti, elenco script, elenco config, elenco report, elenco demo, elenco pattern. Questo aiuta navigazione e documentazione.

Un index_generator può creare:

DOCUMENTATION_INDEX.md;

SCRIPT_INDEX.md;

CONFIG_INDEX.md;

REPORT_INDEX.md;

DEMO_INDEX.md;

PATTERN_INDEX.md;

MODULE_INDEX.md;

RELEASE_INDEX.md.

Ogni voce può includere path, descrizione, stato, ultima modifica, categoria e note. Questo è molto utile quando il progetto supera centinaia o migliaia di file. Senza indici, anche chi ha creato il progetto rischia di perdersi.

La diciannovesima categoria è rappresentata dagli **script di generazione manifest**. Un manifest è un file che descrive il contenuto di un pacchetto o modulo. Può indicare nome, versione, scopo, file principali, dipendenze, entrypoint, schema, dati, stato e limitazioni. Gli script possono generare o aggiornare manifest automaticamente. Questo rende ogni modulo più leggibile.

Un module_manifest.json può contenere:

module_name;

module_version;

description;

entrypoints;

required_files;

dependencies;

schemas_used;

configs_used;

data_inputs;

data_outputs;

status;

limitations;

last_updated.

Questo tipo di file è molto utile in un ecosistema grande. Un tool può leggere i manifest e capire cosa c'è nel progetto. La dashboard o documentazione può usarli per generare mappe.

La ventesima categoria è rappresentata dagli **script di controllo dipendenze**. Un dependency_checker può leggere requirements.txt, package.json, import Python, script JS e verificare cosa serve. Può segnalare dipendenze mancanti, inutilizzate, duplicate o non dichiarate. Questo è importante perché molti errori runtime nascono da dipendenze non installate.

Un dependency_checker può controllare:

requirements.txt presente;

package.json presente;

import Python non dichiarati;

dipendenze installate;

versioni incompatibili;

pacchetti inutilizzati;

manca di venv;

istruzioni install mancanti;

dipendenze opzionali.

Questo tool può generare un `INSTALL.md` più preciso o aggiornare una sezione requirements. Per MicroBot, dove possono convivere pygame, flask, flask-socketio, numpy, pandas, matplotlib o librerie web, è molto utile.

La ventunesima categoria è rappresentata dagli **script di monitoraggio live**. Durante una run, uno script può monitorare log, snapshot, eventi, porte, processi e file. Può stampare stato live nel terminale: runtime ok, dashboard ok, snapshot updated, events count, latest warning. Questo è utile se non si vuole aprire sempre la dashboard.

Un `live_monitor` può mostrare:

runtime status;

simulation status;

dashboard status;

last snapshot age;

event count;

critical events;

node count;

last error;

CPU/memory, se disponibile;

log tail.

Questo script può funzionare come terminal dashboard. È utile durante sviluppo, soprattutto quando la UI web è in costruzione.

La ventiduesima categoria è rappresentata dagli **script di backup**. Un progetto importante deve avere backup. Uno backup script può copiare documentazione, config, report, codici e asset in una cartella backup con timestamp. Può escludere venv, cache e file pesanti. Può creare backup leggeri e backup completi. Questo è molto pratico.

Un `backup_tool` può creare:

backup/YYYY-MM-DD_HH-MM/;

docs backup;

configs backup;
src backup;
reports backup;
manifest backup;
checksum;
backup_log.txt.

Questo script deve essere prudente e non sovrascrivere backup esistenti. Deve anche indicare dimensione finale. In un progetto enorme, backup ciechi possono occupare troppo spazio. Serve selezione.

La ventitreesima categoria è rappresentata dagli **script di confronto versioni**. Se si hanno due versioni di una configurazione, report, snapshot o struttura, uno diff_tool può mostrare differenze. Questo è utile per capire cosa è cambiato tra v0.2 e v0.3, tra due run o tra due profili evoluti.

Un diff_tool può confrontare:

config A vs config B;
behavior profile A vs B;
report A vs B;
snapshot A vs B;
directory structure A vs B;
manifest A vs B;
schema A vs B.

Il risultato può essere testuale o JSON. Per esempio: cohesion_weight cambiato da 0.7 a 0.85; max_velocity ridotto; threshold aumentata; nuovo campo sync_status aggiunto. Questo aiuta molto nella documentazione.

La ventiquattresima categoria è rappresentata dagli **script di normalizzazione**. Quando si uniscono progetti esterni, i file possono avere formati diversi, nomi diversi, struttura diversa. Uno normalize script può rinominare file, convertire encoding, sistemare estensioni, uniformare titoli, normalizzare JSON, rimuovere spazi inutili e aggiornare convenzioni.

Uno normalize_project.py può:

normalizzare nomi file;
convertire encoding UTF-8;
formattare JSON;
sistemare estensioni;

uniformare README;

rimuovere caratteri problematici;

standardizzare date;

standardizzare versioni.

Questo è molto utile quando si prepara un repository pubblico. Una struttura normalizzata è più facile da leggere e mantenere.

La venticinquesima categoria è rappresentata dagli **script di generazione template**. Molte parti del progetto richiedono file simili: README, report, config, module docs, pattern docs, run notes, hardware test notes. Uno script può generare template compilabili. Questo riduce tempo e aumenta coerenza.

Template utili:

README_MODULE.md;

PATTERN_DOC.md;

RUN_REPORT.md;

HARDWARE_TEST_REPORT.md;

SIMULATION_EXPERIMENT.md;

API_DOC.md;

SECURITY_EVENT_DOC.md;

RELEASE_NOTES.md.

Ogni template può avere sezioni predefinite. Questo aiuta a non dimenticare parti importanti. Per esempio, un hardware test report deve avere safety notes, setup, wiring, result, issues, photos, next steps. Lo script genera la base.

La ventiseiesima categoria è rappresentata dagli **script di automazione grafici**. MicroBot produce metriche e può generare grafici. Uno script può leggere metrics.json e produrre immagini o file HTML con grafici: pattern_error, topology_score, energy, thermal, event count, success rate. Questo collega script, analytics e documentazione.

Uno chart_generator può produrre:

pattern_error_over_time.png;

topology_score_over_time.png;

energy_summary.png;

thermal_summary.png;

event_distribution.png;

node_reliability.png;

run_comparison.png;

success_rate_by_pattern.png.

I grafici devono essere etichettati bene e salvati con nomi chiari. Devono indicare unità, legenda e run_id. Un grafico senza contesto serve poco. Lo script deve generare anche una breve descrizione o includere metadati.

La ventisettesima categoria è rappresentata dagli **script di integrazione con hardware**. Quando si lavora con ESP32, Arduino o altri microcontrollori, possono servire script per elencare porte seriali, caricare firmware, aprire monitor seriale, inviare comandi test, leggere output e salvare log. Questi script devono essere molto prudenti e separati dai comandi critici.

Uno hardware_helper può fare:

listare porte seriali;

rilevare board connessa;

aprire serial monitor;

inviare comando status;

salvare log seriale;

eseguire LED test;

richiedere identity;

eseguire safe_idle;

generare hardware_session_report.

Per hardware reale, gli script devono sempre includere safe defaults. Non devono inviare comandi attuativi ad alto rischio senza conferma. Devono preferire status, identity, safe_idle, test LED, diagnostica. Questo mantiene coerenza con Safety Layer.

La ventottesima categoria è rappresentata dagli **script di integrazione con Git**. Se MicroBot viene pubblicato su GitHub, script utili possono generare status, controllare file non tracciati, creare changelog, aggiornare versioni, preparare commit, controllare dimensione file e verificare che non ci siano file sensibili. Non devono sostituire il controllo umano, ma aiutano.

Uno git_release_helper può controllare:

file modificati;

file grandi;

file sensibili;

README aggiornato;

changelog aggiornato;

version bump;

branch corrente;

tag suggerito;

release notes.

Questo è utile per evitare di committare roba sbagliata, tipo venv, cache, dati personali o file enormi. In un progetto grosso, è facilissimo fare casino con Git; uno script aiuta.

La ventinovesima categoria è la **roadmap degli script di sistema**. Anche gli script devono essere sviluppati con ordine. Prima generatori struttura e launcher. Poi validator. Poi report/export. Poi cleanup. Poi release builder. Poi monitoring e automation avanzata. Se si fanno troppi script senza indice, diventano essi stessi caos. Serve uno SCRIPT_INDEX e una convenzione.

Una roadmap possibile è:

v0.1: create_project_structure.py;

v0.2: create_module_structure.py;

v0.3: init_project.py;

v0.4: launch_demo.py;

v0.5: stop_runtime.py;

v0.6: generate_sample_data.py;

v0.7: validate_json.py;

v0.8: generate_report.py;

v0.9: export_run.py;

v1.0: cleanup_project.py;

v1.1: archive_legacy.py;

v1.2: build_release.py;

v1.3: zip_package_builder.py;

v1.4: generate_indexes.py;

v1.5: environment_check.py;

v1.6: hardware_helper.py;

v1.7: pre_demo_check.py;

v1.8: pre_release_check.py.

Questa roadmap rende gli script un sottosistema ordinato. Ogni script ha scopo, input, output e documentazione. Non sono file buttati lì.

Il trentesimo punto è la **sintesi del ruolo degli script di sistema**. Essi sono la parte pratica che permette a MicroBot di restare gestibile. Se il progetto cresce fino a centinaia o migliaia di file, non si può più fare tutto manualmente. Gli script creano, controllano, avviano, esportano, puliscono, archiviano, migrano, validano e preparano release. Sono il livello di automazione che trasforma MicroBot da insieme di progetti separati a ecosistema operativo.

In conclusione, gli script di sistema completano la Sezione 16 perché rappresentano il collegamento pratico tra idee, codice, documentazione e infrastruttura. Essi permettono di generare strutture, inizializzare ambienti, avviare runtime, fermare processi, creare dati demo, esportare run, generare report, convertire documenti, pulire file, archiviare versioni, migrare cartelle, costruire release, validare configurazioni, monitorare esecuzioni, preparare backup, confrontare versioni, generare template, produrre grafici, assistere hardware e controllare Git. Nel progetto MicroBot, questi script non sono accessori secondari: sono strumenti essenziali per mantenere ordine, ripetibilità e professionalità.

Con questo punto si chiude la **Sezione 16 — Progetti Esterni**. Questa sezione ha mostrato come intelligenza artificiale, clustering e modelli, giochi, sistemi interattivi, simulazioni evolutive, algoritmi genetici, progetti web, interfacce avanzate, tool diagnostici e script di sistema possano contribuire all'ecosistema MicroBot. Il messaggio finale è che i progetti esterni non devono disperdere la visione principale, ma devono essere letti come competenze, moduli, strumenti e basi sperimentali da collegare con criterio. La sezione successiva, dedicata all'**Integrazione dei Progetti**, dovrà ora spiegare come tutti questi elementi possano essere collegati tra loro in modo ordinato, distinguendo integrazione diretta, integrazione indiretta, supporto reciproco, crescita delle competenze ed evoluzione tecnica dell'intero sistema MicroBot.

SEZIONE 17 — Integrazione dei Progetti

La **Sezione 17** introduce il tema dell'**integrazione dei progetti**, cioè il modo in cui i vari esperimenti, moduli, siti, simulazioni, tool, script, documenti e sistemi esterni possono essere collegati tra loro senza creare confusione. Dopo la Sezione 16, dedicata ai progetti esterni, diventa necessario compiere un passaggio fondamentale: non basta avere tanti progetti; bisogna capire **come metterli in relazione**, quali devono entrare nel cuore di MicroBot, quali devono restare strumenti di supporto, quali devono essere documentati come competenze e quali invece devono rimanere archiviati come esperimenti precedenti.

L'integrazione è uno dei punti più delicati dell'intero ecosistema MicroBot. Quando un progetto cresce molto, il rischio non è soltanto avere poco codice: spesso il rischio opposto è avere **troppi pezzi**, troppe versioni, troppe pagine web, troppe demo, troppi script, troppe idee e troppe cartelle non collegate. In quel caso il sistema sembra enorme, ma non necessariamente coerente. L'obiettivo della Sezione 17 è evitare proprio questo problema. MicroBot deve diventare un ecosistema leggibile, non un archivio disordinato di esperimenti.

Il principio generale della Sezione 17 può essere espresso così:

l'integrazione dei progetti deve trasformare elementi separati in un ecosistema coerente, mantenendo chiaro cosa è core, cosa è modulo, cosa è tool, cosa è demo, cosa è documentazione e cosa è ricerca futura.

Questa frase è fondamentale. Non tutto deve diventare core. Non tutto deve essere eseguito insieme. Non tutto deve comparire nella dashboard principale. Alcuni progetti possono essere integrati direttamente nel runtime. Altri possono diventare strumenti di analisi. Altri possono fornire solo stile, interfaccia o esperienza tecnica. Altri ancora possono restare portfolio. La qualità dell'integrazione sta proprio nella capacità di distinguere questi livelli.

La Sezione 17 segue i punti dell'indice: collegamento tra sistemi, integrazione funzionale, supporto reciproco, crescita delle competenze ed evoluzione tecnica. Questi punti descrivono una progressione logica. Prima si spiega come i sistemi vengono collegati tra loro. Poi si chiarisce quali funzioni possono essere integrate davvero. Poi si analizza come i progetti si supportano a vicenda. Poi si descrive come ogni progetto contribuisce alla crescita tecnica personale e dell'ecosistema. Infine si spiega come l'intero sistema può evolvere nel tempo in modo ordinato.

17.1 Collegamento tra sistemi

Il **collegamento tra sistemi** rappresenta il primo punto della Sezione 17 e descrive il modo in cui i diversi progetti, moduli e sottosistemi dell'ecosistema MicroBot possono comunicare, condividere dati, scambiarsi risultati e mantenere coerenza. Dopo aver analizzato i progetti esterni nella Sezione 16, è necessario evitare che ciascun progetto resti isolato. Un simulatore che produce dati ma non li esporta, una dashboard che mostra schermate ma non legge telemetria, un tool diagnostico che controlla file ma non comunica i risultati al report, un algoritmo genetico che genera profili ma non li salva in formato compatibile: tutti questi elementi, presi singolarmente, possono essere utili, ma il loro valore aumenta enormemente quando vengono collegati.

Nel progetto MicroBot, collegare sistemi significa creare **ponti ordinati** tra componenti diversi. Il simulatore deve poter produrre snapshot. La dashboard deve poter leggere quegli snapshot. Il report generator deve poter usare metriche ed eventi. I tool diagnostici devono poter verificare configurazioni e dati. Gli script di sistema devono poter avviare runtime e generare output. La documentazione deve poter descrivere esattamente quali file vengono prodotti e consumati. Il controller deve poter dialogare con nodi simulati o hardware. Tutto questo richiede interfacce chiare.

Il principio generale può essere espresso così:

due sistemi MicroBot sono collegati correttamente quando esiste un formato dati, un punto di scambio, una responsabilità chiara e una regola di compatibilità tra ciò che uno produce e ciò che l'altro consuma.

Questa frase è importantissima perché definisce l'integrazione in modo concreto. Non basta dire "la dashboard è collegata alla simulazione". Bisogna dire **come**. La simulazione produce **snapshot.json**. La dashboard legge **snapshot.json** o riceve aggiornamenti via WebSocket. Il formato dello snapshot segue uno schema. Lo schema ha una versione. Se la dashboard richiede una versione diversa, il compatibility checker lo segnala. Questo è collegamento reale. Senza formato, schema e responsabilità, il collegamento resta solo un'idea.

Il primo collegamento fondamentale è quello tra **simulation engine e dashboard web**. La simulazione è il motore che calcola lo stato dello swarm: nodi, posizioni, ruoli, edge, metriche, eventi e fasi. La dashboard è l'interfaccia che rende questi dati visibili. Il ponte tra i due può essere un file JSON, una API HTTP o un WebSocket. Nelle versioni iniziali, un file **snapshot.json** aggiornato periodicamente può essere sufficiente. Nelle versioni più avanzate, un backend può inviare aggiornamenti live.

Una pipeline semplice può essere:

simulation_engine calcola stato;

simulation_engine scrive snapshot;

dashboard legge snapshot;

dashboard renderizza nodi, edge e metriche;

dashboard mostra eventi e warning.

Una pipeline più avanzata può essere:

simulation_engine produce update;

runtime server riceve update;

WebSocket bridge invia update al frontend;

dashboard aggiorna stato live;

event log e charts si aggiornano in tempo reale.

Entrambe le soluzioni sono valide. La prima è più semplice e più adatta a prototipi. La seconda è più professionale e più adatta a dashboard live. L'importante è dichiarare chiaramente quale modalità viene usata. Se i dati sono sample, la dashboard deve dire demo_data_mode. Se sono live, deve dire live_mode. Se sono replay, deve dire replay_mode. Questa distinzione evita confusione.

Il secondo collegamento è quello tra **simulation engine e report generator**. Ogni simulazione dovrebbe poter generare un report. Il report non deve essere scritto a mano partendo dal nulla; deve nascere dai dati prodotti dalla run. La simulazione produce metriche, eventi, configurazione, risultato finale, failure reason e snapshot. Il report generator legge questi dati e crea un documento strutturato. In questo modo ogni esperimento diventa tracciabile.

Una pipeline report può essere:

run_config;

snapshot_sequence;

event_log;

metrics_summary;

final_result;

failure_reason;

report_generator;

report.md o report.html.

Questo collegamento è fondamentale per trasformare MicroBot in progetto documentabile. Ogni run non resta un'animazione vista una volta, ma diventa materiale tecnico: cosa è stato testato, con

quali parametri, quale risultato ha prodotto, quali errori sono emersi, quali nodi hanno avuto problemi, quali miglioramenti sono consigliati. Questo è esattamente il tipo di collegamento che rende il progetto più serio.

Il terzo collegamento è quello tra **tool diagnostici e runtime**. I tool diagnostici non devono essere separati dal sistema operativo del progetto. Devono poter controllare file, configurazioni, schema, dipendenze, ambiente, launcher, telemetria e log prima o durante l'esecuzione. Il runtime può usare questi tool in modalità pre-flight: prima di avviare demo o simulazione, controlla se tutto è pronto. Se qualcosa manca, ferma l'avvio o avvisa l'utente.

Una pipeline pre-flight può essere:

directory_check;

required_files_check;

json_validation;

config_validation;

dependency_check;

port_check;

telemetry_path_check;

launch_ready oppure launch_blocked.

Questo collegamento evita molti problemi. Invece di lanciare una demo e scoprire dopo che manca una cartella o un pacchetto, il sistema controlla prima. Se **pygame** manca, lo segnala. Se la dashboard non trova il file snapshot, lo segnala. Se la porta è occupata, lo segnala. Questo rende l'esperienza di sviluppo più stabile.

Il quarto collegamento è quello tra **script di sistema e struttura del progetto**. Gli script non devono essere file isolati messi a caso. Devono conoscere la struttura MicroBot e operare su di essa. Per esempio, uno script di generazione deve creare cartelle coerenti; uno script di export deve salvare report nel posto giusto; uno script di release deve raccogliere solo file corretti; uno script di cleanup deve ignorare dati importanti. Questo richiede path standard e manifest.

Un collegamento corretto può prevedere:

project_root riconosciuto;

manifest centrale;

cartelle standard;

output standard;

log standard;

docs standard;

config standard.

Se ogni script usa path diversi, il sistema diventa fragile. Se invece tutti leggono `project_manifest.json` o una configurazione centrale, diventano più coerenti. Il manifest diventa una mappa dell'ecosistema. Dice dove sono simulation, dashboard, reports, configs, docs, releases, tools. Gli script usano quella mappa invece di indovinare.

Il quinto collegamento è quello tra **documentazione e file reali**. Questo è un punto gigantesco. La documentazione non deve descrivere un progetto immaginario; deve collegarsi ai file esistenti. Se un documento parla di `03_SIMULATION_ENGINE`, quella cartella deve esistere. Se parla di `run_live_swarm_snapshot.py`, quel file deve esistere o deve essere marcato come planned. Se parla di dashboard, deve indicare il path corretto. Se parla di report, deve dire dove vengono generati. Il collegamento tra documentazione e struttura reale è ciò che impedisce allucinazioni progettuali.

Una documentazione integrata dovrebbe indicare:

nome del modulo;

path del modulo;

scopo;

input;

output;

file principali;

stato attuale;

dipendenze;

comandi di avvio;

limiti;

collegamenti ad altri moduli.

Questa struttura rende ogni sezione verificabile. Un lettore può passare dal testo al file. Un tool può controllare se i path citati esistono. Questo è fondamentale per una documentazione lunga e credibile.

Il sesto collegamento è quello tra **hardware e simulazione**. MicroBot deve gestire sia nodi virtuali sia prototipi fisici. Il collegamento tra hardware e simulazione non significa fingere che tutto sia fisico. Significa creare un modello chiaro in cui un nodo hardware può essere rappresentato nella dashboard, può inviare heartbeat, può dichiarare identity, può essere associato a un nodo virtuale e può contribuire alla telemetria. Questo è il concetto di sistema ibrido.

Un collegamento hardware-simulazione può contenere:

`hardware_node_id`;

`virtual_node_id`;

`real_heartbeat`;

virtual_position;
real_firmware_version;
simulated_energy oppure measured_energy;
simulated_thermal oppure measured_thermal;
capabilities reali;
stato ibrido.

Questa distinzione è fondamentale. Se la posizione è simulata ma heartbeat è reale, bisogna dirlo. Se energia è stimata, bisogna dirlo. Se temperatura non è misurata, bisogna dirlo. Il collegamento tra sistemi deve essere onesto. L'onestà tecnica aumenta la credibilità del progetto.

Il settimo collegamento è quello tra **controller e nodi**. Il controller è il livello che invia comandi, assegna ruoli, riceve telemetria e coordina lo swarm. I nodi, simulati o fisici, devono rispondere con stati coerenti. Questo collegamento richiede protocollo. Un comando deve avere command_id, target_node, command_type, payload, timestamp, timeout, access/safety context. Una risposta deve avere status, reason, node_state e timestamp. Senza protocollo, controller e nodi comunicano in modo fragile.

Una struttura comando può contenere:

command_id;
target_node_id;
command_type;
payload;
issued_at;
expires_at;
required_capability;
safety_context;
source;
access_level.

Una risposta può contenere:

command_id;
node_id;
accepted;
result;

rejection_reason;

node_mode;

safety_state;

timestamp.

Questo collegamento è il cuore del sistema distribuito. La dashboard può generare richiesta, il controller crea comando, il nodo accetta o rifiuta, la telemetria riporta il risultato. Tutto deve essere tracciabile.

L'ottavo collegamento è quello tra **sicurezza biometrica e dashboard**. La Sezione 15 ha definito autenticazione, access levels e decisione tramite soglia. Questo sistema deve essere collegato concretamente all'interfaccia web. La dashboard deve sapere se l'utente è guest, viewer, operator, developer, admin o hardware_admin. Deve abilitare o disabilitare controlli. Deve mostrare sessione attiva o scaduta. Deve richiedere re-auth per azioni critiche. Questo è un collegamento funzionale tra modulo security e UI.

Una pipeline access control può essere:

auth_attempt;

auth_result;

access_level;

session_created;

dashboard_controls_updated;

action_request;

permission_check;

controller_command oppure denied.

Questo collegamento deve essere coerente. Non basta fare login visivo. Se un utente non ha permesso, anche API/backend devono bloccare l'azione. La UI è solo un primo livello. Il controller deve verificare. Il nodo deve verificare safety. Questo collegamento multi-layer rende il sistema serio.

Il nono collegamento è quello tra **AI/analytics e telemetria**. I moduli AI non devono lavorare su dati inventati o scollegati. Devono leggere telemetria, report, run history, eventi e metriche. Se un modello classifica una run come unstable, deve basarsi su dati come pattern_error, topology_score, stability_score, safety_event_count, recovery_count. L'AI deve essere collegata alla pipeline dati, non alla fantasia.

Una pipeline analytics può essere:

run data;

metrics extraction;

feature vector;

model inference oppure rule-based analysis;

classification;

recommendation;

report/dash output.

Questo collegamento è molto importante per evitare AI decorativa. Un pannello AI che dice “rischio alto” deve spiegare quali metriche hanno portato a quel giudizio. Se non ci sono dati sufficienti, deve dire `insufficient_data`. Questa disciplina rende l’AI utile.

Il decimo collegamento è quello tra **algoritmi genetici e simulation engine**. Gli algoritmi genetici possono generare `behavior_profile`, ma il simulatore deve saperli caricare. La fitness function deve leggere i risultati delle run. I profili migliori devono essere esportati in formato compatibile. Questo è un collegamento molto concreto: schema dei geni, config del simulatore, report delle run, fitness e profili evoluti devono parlare la stessa lingua.

Una pipeline genetica può essere:

`genetic_config`;

`population`;

`behavior_profile` candidate;

`simulation run`;

`metrics output`;

`fitness calculation`;

`selection`;

`evolved_profile` export;

`validation report`.

Senza questo collegamento, l’algoritmo genetico resta progetto esterno isolato. Con questo collegamento, diventa strumento di ottimizzazione MicroBot. Però deve restare simulation-only nelle prime versioni e non comandare hardware.

L’undicesimo collegamento è quello tra **progetti web esterni e piattaforma web MicroBot**. Se esistono più siti, dashboard, demo e pagine sperimentali, bisogna mapparli. Alcuni possono essere fusi. Alcuni possono diventare sezioni. Alcuni possono fornire componenti grafici. Alcuni possono essere archiviati. Il collegamento non deve essere fatto copiando tutto dentro una pagina enorme senza criterio. Serve una mappa.

Una `website_merge_map` può indicare:

progetto sorgente;

pagina o componente;

funzione originale;
destinazione MicroBot;
stato integrazione;
modifiche necessarie;
priorità;
note.

Questo è fondamentale per evitare duplicazione. Se tre siti hanno tre versioni diverse di una dashboard, bisogna decidere quale diventa principale, quali componenti si riusano e quali si archiviano. Il collegamento tra siti deve produrre una piattaforma coerente, non un collage disordinato.

Il dodicesimo collegamento è quello tra **giochi/sistemi interattivi e simulazione MicroBot**. I giochi non devono entrare come giochi nel core, ma i loro concetti possono diventare componenti: game loop, entity system, event system, HUD, input handling, replay, collisioni, debug overlay. Il collegamento è quindi spesso concettuale e architetturale, non necessariamente di codice.

Un collegamento corretto può essere:

game loop → simulation loop;
entity object → MicroBot node model;
collision detection → spacing/collision avoidance;
HUD → telemetry HUD;
replay system → run replay viewer;
score → run_score/fitness;
debug overlay → diagnostic overlay.

Questo è un esempio di integrazione indiretta ma utile. Non si dice che Tetris fa parte di MicroBot. Si dice che la competenza di gestione griglia/stato/collisioni può ispirare moduli MicroBot. Questa precisione è importante.

Il tredicesimo collegamento è quello tra **tool diagnostici e dashboard diagnostica**. I tool possono essere CLI, ma i loro risultati possono anche essere visualizzati in una dashboard. Per esempio, un diagnostic_snapshot può essere caricato in una pagina diagnostics. La dashboard può mostrare directory status, config status, schema status, last errors, environment status, release readiness. Questo collega script/tool e interfaccia.

Una diagnostics dashboard può mostrare:

project health;
missing files;

invalid JSON;
schema errors;
dependency status;
telemetry status;
recent log errors;
release readiness;
recommendations.

Questo è molto utile per avere una vista globale dell'ecosistema. Non serve leggere output terminale ogni volta. Un pannello web può mostrare stato del progetto.

Il quattordicesimo collegamento è quello tra **report e documentazione finale**. I report delle run possono alimentare la documentazione lunga. Se una sezione parla di simulazione, può citare risultati aggregati. Se una sezione parla di safety, può includere esempi di eventi. Se una sezione parla di algoritmi genetici, può mostrare un report evolutivo. In questo modo la tesi/documentazione non è solo teorica, ma collegata a esperimenti.

Una documentazione integrata può usare:

run_report;
metrics_summary;
charts;
failure_analysis;
diagnostic_snapshot;
hardware_test_report;
evolved_profile_report;
security_test_report.

Questo collegamento è molto potente. La documentazione diventa evidenza tecnica. Ogni affermazione importante può essere sostenuta da file, report o run. Questo è il modo giusto per rendere MicroBot credibile.

Il quindicesimo collegamento è quello tra **hardware tests e documentazione hardware**. Ogni test su ESP32, LED, sensore, alimentazione o output deve produrre note. Lo script hardware_helper può salvare serial log. Il report hardware può descrivere setup, cablaggio, firmware, risultato, problemi e prossimi passi. La documentazione hardware deve poi collegare questi report. Questo evita che i test fisici restino solo memoria personale.

Un hardware_test_report può contenere:

test_id;

hardware_profile;
firmware_version;
setup;
wiring;
commands;
serial output;
result;
issues;
safety notes;
next steps.

Questo collegamento è cruciale se si vuole dimostrare che il progetto non è solo simulazione. Anche test semplici, se documentati bene, diventano prova tecnica.

Il sedicesimo collegamento è quello tra **MicroBot OS e tool/debug**. MicroBot OS deve esporre shell, diagnostica, status, identity, safety, tasks, errors. I tool esterni possono leggere o interpretare questi dati. Per esempio, un serial_log_analyzer può leggere output della shell. Una dashboard può mostrare node diagnostics. Un report può includere reset_reason e watchdog_count. Questo collega firmware e sistema superiore.

Una pipeline OS diagnostics può essere:

nodo produce diagnostic output;
controller raccoglie;
telemetry normalizza;
dashboard mostra;
report archivia;
tool analizza.

Questo è un collegamento molto importante. Il nodo fisico non deve essere una scatola chiusa. Deve poter parlare al resto del sistema attraverso diagnostica strutturata.

Il diciassettesimo collegamento è quello tra **release builder e project manifest**. Quando si prepara uno zip o una release, il builder deve sapere cosa includere. Il manifest indica moduli, file, docs, demo, sample data, esclusioni. Questo evita pacchetti casuali. Il collegamento tra manifest e release è ciò che permette di creare zip diversi: demo light, full docs, web platform, hardware docs.

Una release pipeline può essere:

leggi project_manifest;

leggi release_scope;
controlla file;
escludi sensibili;
copia contenuti;
genera manifest release;
genera checksum;
crea zip;
scrivi release_notes.

Questo collegamento è fondamentale per consegnare materiale ordinato. Uno zip senza manifest è difficile da capire. Uno zip con manifest, README e scope è molto più professionale.

Il diciottesimo collegamento è quello tra **archivio legacy e sistema attuale**. I progetti vecchi non devono sparire, ma non devono nemmeno confondere il core. Serve una mappa legacy_to_current. Per ogni vecchio progetto, si decide: archiviare, integrare, riscrivere, usare come riferimento, lasciare portfolio. Questo collegamento protegge il lavoro passato senza appesantire il presente.

Una legacy_map può contenere:

legacy_project_name;
old_path;
current_destination;
integration_status;
reason;
files_reused;
files_archived;
notes.

Questo è molto utile quando si uniscono tanti progetti. Permette di dire: questo vecchio sito ha fornito stile; questo script è stato sostituito; questa demo è archiviata; questo codice è stato riusato in dashboard. La storia del progetto resta leggibile.

Il diciannovesimo collegamento è quello tra **competenze esterne e moduli MicroBot**. Alcuni progetti non saranno integrati come codice, ma come competenza. Per esempio, un gioco insegna loop e collisioni. Un CSV analyzer insegna parsing e report. Un progetto AI insegna classificazione. Un sito web insegna UI. Questo collegamento deve essere documentato nella forma di skill_mapping.

Uno skill_mapping può contenere:

progetto esterno;

competenza sviluppata;

modulo MicroBot collegato;

tipo di integrazione;

evidenza;

stato.

Questo è molto utile per portfolio. Dimostra che i progetti esterni non sono casuali, ma hanno contribuito alla costruzione di competenze necessarie per MicroBot.

Il ventesimo collegamento è quello tra **roadmap e moduli reali**. La roadmap non deve essere una lista astratta di desideri. Deve collegarsi ai moduli esistenti. Se la roadmap dice “aggiungere replay viewer”, deve indicare quali dati servono, quale dashboard deve essere modificata, quale schema serve, quale report produce. Se dice “integrare AI”, deve dire quale dataset serve. Questo rende la roadmap eseguibile.

Una roadmap collegata deve indicare:

obiettivo;

moduli coinvolti;

file coinvolti;

input richiesti;

output attesi;

dipendenze;

rischi;

stato;

priorità.

Questo evita roadmap vaghe. Ogni passo diventa lavorabile. È la differenza tra “fare AI” e “creare run_dataset.json + feature_extractor.py + rule_based_classifier.py + dashboard panel”.

Il ventunesimo collegamento è quello tra **stato del modulo e maturità**. Ogni modulo collegato deve avere stato: concept, prototype, working, tested, documented, integrated, deprecated. Questo è necessario perché non tutti i pezzi sono allo stesso livello. Alcune cose sono reali, altre simulate, altre future. Il collegamento tra sistemi deve rispettare questa maturità.

Stati possibili:

concept;

planned;

prototype;

implemented;
tested_in_simulation;
hardware_partial;
integrated;
documented;
release_ready;
deprecated;
archived.

Questo evita promesse eccessive. Se un modulo è concept, lo si dice. Se è integrato, lo si dice. Se è archiviato, lo si dice. La trasparenza sullo stato è parte dell'integrazione.

Il ventiduesimo collegamento è quello tra **input e output dei moduli**. Ogni modulo MicroBot dovrebbe avere una scheda: cosa prende in input, cosa produce in output. Questo è uno dei modi più semplici per collegare sistemi. Se simulation_engine produce snapshot e report, la dashboard e report viewer possono consumarli. Se genetic_optimizer produce behavior_profile, il simulation_engine può consumarlo. Se diagnostic_tool produce diagnostic_snapshot, la dashboard diagnostics può consumarlo.

Una scheda modulo può contenere:

module_name;
input_files;
output_files;
input_api;
output_api;
depends_on;
consumed_by;
schema_version;
status.

Questa mappa rende l'ecosistema comprensibile. È quasi una mappa dei flussi dati. Senza questa mappa, i collegamenti restano nella testa di chi sviluppa.

Il ventitreesimo collegamento è quello tra **schema e validazione**. Ogni punto di scambio tra sistemi deve avere schema. Snapshot, events, reports, behavior profiles, security events, hardware profiles. Ogni schema deve essere validato. Questo è il modo più robusto per collegare moduli senza romperli. Se un modulo cambia output, lo schema cambia e gli altri moduli devono aggiornarsi.

La regola è:

nessun collegamento dati importante senza schema o almeno contratto documentato.

Questo è un principio ingegneristico. Anche se non si implementa subito JSON Schema completo, bisogna almeno documentare campi richiesti e significato. Il collegamento tra sistemi passa dai contratti dati.

Il ventiquattresimo collegamento è quello tra **versioni**. Ogni sistema collegato può evolvere. La dashboard può essere v0.4, lo schema snapshot v0.3, il runtime v0.5, il firmware v0.2. Il sistema deve sapere se sono compatibili. Versioning e compatibility_checker sono quindi parte del collegamento. Senza versioni, ogni aggiornamento può rompere qualcosa senza sapere perché.

Ogni payload importante dovrebbe includere:

schema_version;

producer_version;

created_at;

module_id;

compatibility_notes, se utile.

Questo permette debugging. Se la dashboard non legge snapshot, si controlla schema_version. Se un nodo rifiuta comando, si controlla protocol_version. Le versioni sono il linguaggio della compatibilità.

Il venticinquesimo collegamento è quello tra **modalità operative**. I sistemi possono essere collegati in modalità diverse: offline, demo, simulation, replay, live, hardware, hybrid, development, presentation. Ogni modalità abilita collegamenti diversi. In replay mode, la dashboard legge dati registrati. In live mode, legge WebSocket. In hardware mode, riceve dati reali. In demo mode, usa sample data. È fondamentale dichiarare la modalità.

Modalità possibili:

offline_documentation_mode;

demo_data_mode;

simulation_mode;

replay_mode;

live_dashboard_mode;

hardware_connected_mode;

hybrid_mode;

development_mode;

presentation_mode.

Ogni modalità deve avere regole. Per esempio, in `replay_mode` non si inviano comandi. In `demo_data_mode` non si dichiara live. In `hardware_connected_mode` Safety Layer è obbligatorio. In `presentation_mode` i comandi critici sono nascosti. Questo rende i collegamenti più sicuri.

Il ventiseiesimo collegamento è quello tra **utente e sistema**. L'integrazione non riguarda solo moduli software. Riguarda anche come l'utente interagisce con il sistema. L'utente entra dalla landing page, apre dashboard, si autentica, seleziona modalità, avvia simulazione, osserva telemetria, genera report, esporta dati. Questa user flow deve essere collegata ai moduli. Se l'esperienza utente è incoerente, anche il sistema tecnico sembra confuso.

Una user flow ordinata può essere:

aprire landing;

scegliere demo o dashboard;

se dashboard live, avviare runtime;

caricare simulation/replay/live;

osservare stato globale;

selezionare nodo o pattern;

aprire diagnostica;

generare report;

esportare run;

consultare documentazione.

Questo flusso deve essere documentato. Un nuovo collaboratore deve sapere da dove partire. Collegare sistemi significa anche rendere percorribile l'ecosistema.

Il ventisettesimo collegamento è quello tra **startup narrative e architettura tecnica**. MicroBot ha anche potenziale imprenditoriale e presentativo. La narrazione startup deve essere collegata alla realtà tecnica. Se si dice "piattaforma modulare per swarm robotics", bisogna mostrare moduli, dashboard, simulazione, telemetria, hardware prototype, roadmap. Se si dice "sistema estensibile", bisogna mostrare plugin, schema, moduli e manifest. La narrazione non deve essere separata dal codice.

Un collegamento corretto tra pitch e tecnica può essere:

claim: sistema modulare;

evidenza: architettura a moduli e manifest;

claim: telemetria live;

evidenza: snapshot/dashboard/eventi;

claim: safety-oriented;

evidenza: Safety Layer e command validation;

claim: estensibile verso AI;

evidenza: analytics pipeline e run dataset;

claim: prototipo hardware;

evidenza: ESP32 LED demo e report.

Questo rende il pitch molto più credibile. La startup narrative non è fumo; è spiegazione sintetica di una architettura reale.

Il ventottesimo collegamento è quello tra **architettura attuale e visione futura**. MicroBot ha una visione grande: swarm, hardware, AI, BCI, VR, programmable matter. Ma l'integrazione deve collegare il presente al futuro in modo progressivo. Non si deve saltare da LED demo a programmable matter senza livelli intermedi. Serve una catena: simulazione → telemetria → dashboard → hardware LED → protocolli → sensori → attuatori sicuri → swarm fisico → AI consultiva → interfacce avanzate → applicazioni.

Una catena evolutiva può essere:

simulazione visuale;

telemetria strutturata;

dashboard interattiva;

MicroBot OS prototipale;

nodì ESP32 LED;

hardware profile;

safety layer;

sensori;

attuatori sperimentali;

controllo swarm;

AI analytics;

interfacce AR/VR;

ricerca avanzata.

Questo collegamento protegge la visione. La rende credibile perché mostra una scala di sviluppo. Il futuro non è negato, ma viene collegato a passi concreti.

Il ventinovesimo collegamento è quello tra **integrazione e priorità**. Non tutti i collegamenti vanno fatti subito. Alcuni sono fondamentali: simulation → dashboard, simulation → report, config → validator, docs → files. Altri sono successivi: AI → analytics, genetic optimizer → profiles, AR → dashboard. Bisogna stabilire priorità. L'integrazione deve seguire valore e dipendenze.

Priorità alta:

schema snapshot;

dashboard read-only;

report generator;

config validator;

directory structure;

launcher;

documentation index;

sample data;

pre-demo check.

Priorità media:

WebSocket live;

replay viewer;

node inspector avanzato;

event filters;

diagnostic dashboard;

run comparison;

behavior profiles.

Priorità futura:

AI recommendations;

genetic optimizer;

gesture control;

AR/3D advanced;

hardware actuation panel;

BCI integration.

Questa priorità evita di perdersi. Prima si costruisce il ponte dati principale, poi si aggiungono livelli avanzati. Così l'ecosistema cresce senza collassare.

Il trentesimo punto è la **sintesi del collegamento tra sistemi**. Collegare sistemi significa creare relazioni verificabili tra componenti. Non basta mettere tutto nella stessa cartella. Serve definire

input, output, schema, versione, modalità, responsabilità, stato e compatibilità. Il valore di MicroBot non sta solo nella quantità di progetti, ma nella capacità di farli parlare tra loro in modo ordinato.

In conclusione, il collegamento tra sistemi è la base dell'integrazione MicroBot. Esso unisce simulation engine, dashboard, report generator, tool diagnostici, script di sistema, documentazione, hardware, controller, nodi, sicurezza biometrica, AI, algoritmi genetici, progetti web, interfacce avanzate, archivi legacy e roadmap. Ogni collegamento deve essere esplicito: chi produce dati, chi li consuma, in quale formato, con quale schema, con quale versione e in quale modalità operativa. Questa disciplina impedisce che l'ecosistema diventi un insieme caotico di file e lo trasforma in una piattaforma modulare, verificabile e presentabile. Il punto successivo, dedicato all'**integrazione funzionale**, dovrà spiegare quali collegamenti diventano vere funzioni operative del sistema, distinguendo ciò che è solo concettualmente collegato da ciò che può essere eseguito, testato e usato dentro MicroBot.

17.2 Integrazione funzionale

L'**integrazione funzionale** rappresenta il secondo punto della Sezione 17 e descrive il passaggio dal semplice collegamento tra sistemi alla costruzione di funzioni operative reali. Nel punto precedente è stato chiarito che i sistemi MicroBot devono poter comunicare attraverso dati, schemi, versioni, manifest, API, snapshot, report, telemetria e protocolli. Tuttavia, collegare due sistemi non significa ancora integrarli funzionalmente. Due moduli possono scambiarsi file, ma non produrre una funzione utile. L'integrazione funzionale inizia quando quel collegamento genera un comportamento concreto: una dashboard che visualizza lo swarm, un report che nasce da una run, un tool diagnostico che blocca una demo non pronta, un algoritmo genetico che produce un profilo caricabile dal simulatore, un nodo hardware che appare dentro la telemetria, una sessione biometrica che abilita controlli specifici.

Nel progetto MicroBot, questa distinzione è fondamentale. L'ecosistema contiene tanti componenti potenzialmente collegabili, ma non tutti devono diventare funzioni operative allo stesso momento. Alcuni collegamenti sono solo concettuali. Altri sono documentali. Altri sono tecnici ma non ancora usabili. Altri invece possono diventare vere funzioni del sistema. L'integrazione funzionale serve proprio a stabilire quali componenti sono già abbastanza maturi per essere eseguiti, testati, mostrati e mantenuti dentro la piattaforma MicroBot.

Il principio generale può essere espresso così:

un'integrazione diventa funzionale quando produce un risultato operativo verificabile, ripetibile e utile all'interno del sistema MicroBot.

Questa frase è decisiva. Non basta scrivere che "AI, simulazione e dashboard sono integrate". Bisogna poter indicare un flusso reale: la simulazione genera dati, la dashboard li legge, l'utente vede lo swarm, seleziona un nodo, legge telemetria, esporta un report. Questo è funzionale. Oppure: il validator legge config e blocca l'avvio se mancano campi critici. Questo è funzionale. Oppure: il replay viewer carica una run registrata e permette di navigarla. Questo è funzionale. La funzionalità deve essere visibile nel comportamento del sistema.

La prima integrazione funzionale fondamentale è **simulation engine** → **dashboard**. Questa è probabilmente una delle integrazioni più importanti dell'intero ecosistema. Il simulation engine calcola lo stato virtuale dei MicroBot; la dashboard lo rende visibile. Quando questa integrazione funziona, l'utente può osservare nodi, edge, ruoli, pattern, metriche e stati in tempo reale o tramite

snapshot. Questa funzione è centrale perché trasforma la simulazione da processo interno a esperienza visuale.

Una integrazione funzionale minima può essere:

la simulazione genera **snapshot.json**;

lo snapshot contiene nodi, posizioni, ruoli, stati ed edge;

la dashboard carica il file;

i nodi vengono renderizzati;

le metriche principali vengono mostrate;

lo stato viene aggiornato a intervalli regolari.

Una integrazione più avanzata può usare WebSocket, API, event stream e update incrementali. Ma la funzione resta la stessa: rendere osservabile lo swarm. Questa integrazione deve essere considerata prioritaria perché quasi tutte le altre funzioni visuali dipendono da essa. Senza dashboard collegata alla simulazione, il progetto resta meno comunicabile.

La seconda integrazione funzionale è **simulation engine** → **telemetria**. Non basta mostrare nodi sullo schermo. Il sistema deve produrre dati strutturati: timestamp, node_id, mode, role, position, target, energy_state, thermal_state, safety_state, sync_status, event_type, metriche globali, pattern_error, topology_score, aggregation_score. Questa telemetria è ciò che permette analisi, report, replay, AI, diagnostica e validazione. La simulazione quindi non deve essere soltanto animazione: deve diventare generatore di dati.

Una telemetria funzionale deve produrre:

stato globale della run;

stato di ogni nodo;

eventi strutturati;

metriche nel tempo;

summary finale;

eventuali failure reason.

Questa integrazione rende possibile tutto il resto. Se la telemetria è povera, la dashboard sarà povera. Se la telemetria è incoerente, i report saranno incoerenti. Se la telemetria non ha timestamp, il replay sarà difficile. La qualità della telemetria determina la qualità dell'ecosistema.

La terza integrazione funzionale è **telemetria** → **report generator**. Quando una run finisce, il sistema deve poter trasformare i dati raccolti in un documento tecnico. Questo documento può essere Markdown, HTML, JSON o, in futuro, PDF. L'importante è che il report non sia scritto a mano ogni volta, ma generato a partire da dati reali. Questa integrazione è importante perché collega esecuzione e documentazione.

Un report generato funzionalmente può includere:

identificativo della run;

pattern eseguito;

configurazione usata;

numero di nodi;

risultato finale;

metriche principali;

eventi critici;

nodi problematici;

failure reason;

raccomandazioni;

link a snapshot o replay.

Questa funzione ha valore enorme. Ogni esperimento diventa archiviabile. Ogni test lascia una traccia. Ogni miglioramento può essere confrontato con il precedente. La documentazione non resta separata dal codice, ma nasce dal comportamento del sistema.

La quarta integrazione funzionale è **event log** → **timeline/replay**. Gli eventi prodotti durante una run devono poter essere mostrati in una timeline e usati per navigare il replay. Se un evento **thermal_warning** avviene al secondo 18, la timeline deve mostrarlo. Se l'utente clicca l'evento, il replay dovrebbe portarsi a quel momento. Questa è una funzione concreta e molto utile per debug. Trasforma eventi testuali in navigazione temporale.

Una integrazione timeline funzionale richiede:

eventi con timestamp;

snapshot sincronizzati;

phase markers;

event markers;

time cursor;

collegamento tra evento e stato dello swarm.

Questa funzione rende il sistema analizzabile. Non si guarda solo il risultato finale, ma il processo. In un sistema distribuito, la causa di un fallimento spesso si trova nel tempo: un nodo va offline, un edge manca, la fase tarda, il controller ritenta, la safety interviene. La timeline permette di ricostruire questa catena.

La quinta integrazione funzionale è **node state** → **node inspector**. Ogni nodo deve poter essere selezionato e ispezionato. Il node inspector è una funzione concreta della dashboard: l'utente clicca un nodo e vede i dati specifici. Questa integrazione richiede che il modello dati del nodo sia

abbastanza ricco e che la dashboard lo sappia interpretare. È una funzione apparentemente semplice, ma tecnicamente importante perché collega visualizzazione, telemetria e diagnostica.

Un node inspector funzionale può mostrare:

- node_id;
- role;
- mode;
- position;
- target;
- neighbors;
- energy_state;
- thermal_state;
- safety_state;
- sync_status;
- capabilities;
- last_event;
- last_command;
- diagnostic_status.

Questa funzione rende i nodi reali come entità tecniche, non semplici pallini sullo schermo. Aiuta sviluppo, debug e presentazione. In una demo, selezionare un nodo e vedere la sua telemetria dà immediatamente profondità al progetto.

La sesta integrazione funzionale è **pattern model** → **validation metrics**. Ogni pattern deve avere criteri di validazione. Se il pattern è una linea, bisogna misurare allineamento, spacing e continuità. Se è un cerchio, bisogna misurare raggio, distribuzione angolare e chiusura. Se è una griglia, bisogna misurare righe, colonne e connessioni. Questa integrazione rende i pattern valutabili. Non basta vedere che “sembra giusto”. Bisogna calcolare metriche.

Una validazione funzionale può produrre:

- geometry_score;
- topology_score;
- pattern_error;
- missing_edges;
- extra_edges;

stability_score;

completion_status.

Questi valori alimentano dashboard, report, AI, algoritmi genetici e roadmap. Se un pattern non ha metriche, non può essere ottimizzato bene. L'integrazione funzionale tra pattern model e validation metrics è quindi fondamentale per rendere MicroBot misurabile.

La settima integrazione funzionale è **configurazione** → **simulazione**. Un file config deve influenzare davvero la simulazione. Se `node_count = 20`, la simulazione deve creare 20 nodi. Se `pattern_id = line_basic`, deve caricare il modello linea. Se `fault_profile = node_drop`, deve introdurre il fault previsto. Se `behavior_profile` modifica `cohesion_weight`, il comportamento deve cambiare. Questa è integrazione funzionale perché collega file e comportamento.

Una config funzionale deve controllare:

numero di nodi;

pattern;

posizioni iniziali;

parametri di comportamento;

fault profile;

durata della run;

telemetria;

visualizzazione;

safety simulation;

output path.

Se un parametro esiste ma non viene usato, è falso controllo. Va evitato. Ogni config deve avere effetto chiaro oppure essere rimossa o marcata come planned. Questo è importante per non accumulare campi inutili.

L'ottava integrazione funzionale è **validator** → **launcher**. I tool diagnostici devono entrare nel flusso di avvio. Prima di lanciare una demo, il launcher può chiamare validator. Se config, dipendenze, cartelle o schema sono invalidi, il launcher avvisa o blocca. Questo trasforma i tool diagnostici da strumenti manuali a parte operativa del sistema. È una integrazione molto utile e relativamente realistica.

Un flusso funzionale può essere:

utente lancia demo;

launcher esegue preflight checks;

validator controlla struttura, config, dipendenze;

se tutto è ok, parte runtime;

se qualcosa è error, avvio bloccato;

se warning, avvio possibile ma segnalato.

Questo rende MicroBot più robusto. La demo non parte “alla cieca”. Prima controlla la salute del progetto. Questo è molto professionale.

La nona integrazione funzionale è **diagnostic snapshot** → **dashboard diagnostics**. Un tool può generare `diagnostic_snapshot.json`; la dashboard diagnostics può leggerlo e mostrarlo. Questa funzione permette di vedere lo stato dell'intero progetto: cartelle, file, config, schema, dipendenze, ultimi errori, readiness. È un'integrazione importante tra CLI e web.

Una dashboard diagnostics funzionale può mostrare:

project health score;

missing files;

invalid configs;

schema errors;

dependency status;

environment status;

last runtime errors;

recommendations;

release readiness.

Questa funzione è molto utile quando il progetto cresce. L'utente non deve ricordarsi tutti i controlli. Apre il pannello diagnostico e vede cosa manca.

La decima integrazione funzionale è **hardware node** → **dashboard**. Quando un ESP32 o un nodo fisico invia heartbeat o status, la dashboard deve poterlo mostrare. Anche se all'inizio il nodo fisico è solo LED-based, il suo stato può apparire nel sistema. Questa integrazione è fondamentale perché collega simulazione e mondo reale. Non serve partire con swarm fisico completo: basta un nodo reale che dichiari identità, `firmware_version`, status e heartbeat.

Una integrazione hardware minima può includere:

serial connection;

identity command;

heartbeat;

`node_status`;

`firmware_version`;

capabilities;

last_seen;

dashboard badge hardware.

Questa funzione è molto forte per dimostrare che MicroBot non è soltanto web o teoria. Anche un piccolo nodo fisico, se integrato nella telemetria, diventa parte dell'ecosistema. Però bisogna essere chiari: se solo heartbeat è reale e posizione è simulata, la dashboard deve indicarlo.

L'undicesima integrazione funzionale è **hardware node** → **virtual twin**. In modalità ibrida, un nodo fisico può essere associato a un nodo virtuale. Per esempio, il nodo reale fornisce heartbeat e capabilities, mentre il simulatore fornisce posizione virtuale. La dashboard mostra un nodo hybrid. Questa funzione permette di costruire demo ibride anche quando non si hanno molti robot fisici. È molto utile ma deve essere onesta.

Un hybrid node funzionale può avere:

hardware_node_id;

virtual_node_id;

real_heartbeat;

virtual_position;

capability source;

energy source;

thermal source;

mapping_status;

hybrid_confidence.

Questa integrazione deve distinguere fonte dei dati. Non bisogna far sembrare reale ciò che è simulato. Un badge HYBRID o SIMULATED_POSITION è perfetto. La trasparenza qui è essenziale.

La dodicesima integrazione funzionale è **access control** → **comandi dashboard**. La sicurezza biometrica e l'access control devono avere effetto reale sui controlli. Se l'utente è guest, i controlli sono read-only. Se è developer, può aprire diagnostica. Se è operator, può avviare demo approvate. Se è hardware_admin, può richiedere test hardware, ma solo con safety ok. Questa è integrazione funzionale perché il risultato dell'autenticazione modifica realmente il comportamento dell'interfaccia.

Un controllo funzionale deve verificare:

auth_status;

access_level;

session_status;

requested_action;

required_access_level;

system_mode;

safety_state.

Se il permesso manca, il sistema deve rispondere con reason chiaro. Per esempio:

permission_denied: requires_developer. Se il permesso c'è ma la safety blocca, il motivo deve essere diverso: **action_denied_safety: thermal_lock.** Questa distinzione è indispensabile.

La tredicesima integrazione funzionale è **safety layer** → **command pipeline**. Ogni comando operativo deve passare da safety. Questa integrazione è più importante di quasi tutte le altre perché protegge il sistema. Se un comando riguarda output, movimento, attuazione, clear_safety, hardware test o cambio stato critico, il Safety Layer deve poterlo accettare o rifiutare. Il comando non deve essere eseguito solo perché arriva dalla dashboard o da un utente autorizzato.

Una command pipeline funzionale è:

action request;

access check;

controller validation;

capability check;

safety check;

command dispatch;

node response;

telemetry event.

Se la safety rifiuta, deve generare evento. La dashboard deve mostrarlo. Il report deve registrarlo. Questo rende la safety non solo teoria, ma funzione reale.

La quattordicesima integrazione funzionale è **algoritmo genetico** → **behavior_profile**. Un algoritmo genetico diventa funzionale quando produce un profilo realmente caricabile dal simulatore. Non basta generare numeri. Deve esportare un file conforme allo schema **behavior_profile**. Il simulatore deve poterlo usare. Il report deve indicare che quel profilo è evoluto. La dashboard può confrontarlo con baseline.

Una integrazione genetica funzionale richiede:

genetic optimizer produce profile;

profile rispetta schema;

profile ha metadati;

simulation engine lo carica;

run viene eseguita;

report confronta risultati;

profile resta simulation_candidate.

Questa funzione permette di trasformare un progetto esterno di algoritmo genetico in uno strumento MicroBot. Però deve restare vincolato alla simulazione nelle prime versioni. Non deve produrre profili hardware-ready automaticamente.

La quindicesima integrazione funzionale è **AI/analytics** → **recommendation panel**. Un modulo AI o rule-based analytics diventa funzionale quando legge dati reali di una run e produce una raccomandazione visibile. Per esempio: “topology failure likely; missing_edges high; recommend increasing connection_radius or revising role assignment”. Questa raccomandazione può apparire nel report o nella dashboard. Deve includere confidence e fattori principali.

Una raccomandazione funzionale può contenere:

recommendation_id;

risk_level;

main_factors;

suggested_action;

confidence;

source_model;

input_metrics;

limitations.

Questo rende l’AI utile e non decorativa. Se non ci sono dati sufficienti, deve produrre **insufficient_data**. Un modulo AI serio sa anche quando non può decidere.

La sedicesima integrazione funzionale è **clustering** → **swarm analysis panel**. Il clustering diventa funzionale quando calcola cluster reali dai nodi e mostra risultati: cluster_count, centroids, isolated_nodes, fragmentation_score. La dashboard può evidenziare cluster e nodi isolati. Il report può includere analisi di frammentazione. Questa integrazione è molto coerente con MicroBot.

Una funzione clustering può produrre:

cluster_count;

cluster_id per nodo;

cluster_centroids;

cluster_sizes;

isolated_nodes;

fragmentation_score;

cluster_stability.

Questi dati sono utili per pattern, validazione, failure analysis e UI. Il clustering non deve restare notebook separato; può diventare analytics module della simulazione.

La diciassettesima integrazione funzionale è **tool di esportazione** → **documentazione finale**. Uno script di export può produrre grafici, report e tabelle che entrano nella documentazione. Per esempio, una run produce `metrics.json`; il chart generator crea grafici; il report generator crea testo; la documentazione finale li include. Questa integrazione collega runtime e tesi/documentazione.

Una pipeline documentale funzionale può essere:

run;

metrics;

charts;

report;

documentation section;

release package.

Questo è molto importante per produrre materiale convincente. Il documento finale non deve essere solo descrittivo, ma supportato da output del sistema. I grafici e report diventano evidenza.

La diciottesima integrazione funzionale è **web platform** → **documentazione navigabile**. I documenti lunghi possono essere collegati a pagine web. La piattaforma web può mostrare sezioni, glossario, schemi, diagrammi, demo e report. Questa funzione rende la documentazione più accessibile. Non sostituisce il PDF o il Markdown lungo, ma lo rende navigabile.

Una documentazione web funzionale può includere:

indice;

sezioni tecniche;

schemi;

link ai moduli;

link ai report;

demo embedded;

glossario;

roadmap;

stato moduli.

Questo è utile per presentazioni e portfolio. Chi vede il progetto non deve aprire trenta file: può navigare un portale.

La diciannovesima integrazione funzionale è **legacy projects** → **archive/portfolio mapping**. Anche archiviare è una funzione. Un progetto legacy integrato funzionalmente non significa per forza eseguirlo nel core; può significare mappare il suo stato, spostarlo in archive, descriverlo, collegarlo a competenze e decidere se riusarlo. Questa funzione mantiene ordine.

Una integrazione archive funzionale può produrre:

legacy_manifest;

archive_path;

integration_status;

reused_components;

reason for archive;

related MicroBot modules;

notes.

Così i progetti vecchi non spariscono e non confondono il core. Sono valorizzati nel posto giusto. È una forma di integrazione organizzativa.

La ventesima integrazione funzionale è **script generator** → **standardizzazione dei moduli**. Se ogni nuovo modulo MicroBot viene creato con uno script standard, allora tutti avranno README, docs, config, data, logs, tests, examples. Questo produce coerenza strutturale. La funzione non è visibile nella dashboard, ma è fondamentale per mantenibilità.

Uno script generator funzionale deve:

creare struttura standard;

non sovrascrivere senza consenso;

generare README base;

creare manifest;

creare config placeholder;

creare docs placeholder;

stampare summary.

Questa funzione evita che ogni modulo nasca diverso. In un progetto grande, la standardizzazione è una forma di qualità.

La ventunesima integrazione funzionale è **pre-release checker** → **release builder**. Prima di creare uno zip o una release, il sistema deve controllare che tutto sia pronto. Se il pre-release checker trova errori critical, la release non deve essere generata o deve essere marcata come non pronta. Questa integrazione è molto utile per evitare pacchetti sporchi.

Una pipeline release funzionale può essere:

```
pre_release_check;  
  
sensitive_data_check;  
  
docs_check;  
  
schema_check;  
  
manifest_check;  
  
release_build;  
  
zip;  
  
release_report.
```

Questo flusso produce pacchetti più seri. Lo zip finale contiene ciò che serve, non cache, venv, file sensibili o dati disordinati. Per un progetto da consegnare, questa funzione è fondamentale.

La ventiduesima integrazione funzionale è **hardware test** → **hardware report**. Quando uno script o firmware esegue un test hardware, deve produrre report. Anche un test semplice, come LED blink o serial status, può generare una scheda: setup, firmware, comando, output, risultato, problemi. Questa integrazione rende il lavoro fisico documentabile.

Una funzione hardware report può produrre:

```
test_id;  
  
board;  
  
firmware_version;  
  
connection_port;  
  
commands;  
  
serial_output;  
  
result;  
  
notes;  
  
issues;  
  
next_steps.
```

Questo permette di mostrare progresso hardware in modo concreto. Non serve avere subito sei prototipi complessi; serve documentare bene quelli esistenti. Ogni test diventa evidenza.

La ventitreesima integrazione funzionale è **MicroBot OS diagnostics** → **telemetry**. La diagnostica interna del nodo deve poter essere trasmessa come telemetria. Se il nodo dichiara reset_reason, watchdog_count, command_rejection_count, capabilities, energy_state, thermal_state, questi dati devono arrivare al controller e poi alla dashboard. Questa funzione collega MicroBot OS al sistema superiore.

Una diagnostica funzionale può inviare:

identity;

firmware_version;

mode;

role;

last_error;

reset_reason;

uptime;

capabilities;

blocked_capabilities;

safety_state;

diagnostic_status.

Questa integrazione è essenziale per hardware reale. Il nodo non deve essere una scatola muta. Deve dichiarare stato e problemi. La dashboard deve poterli mostrare.

La ventiquattresima integrazione funzionale è **security events** → **audit/report**. Gli eventi di sicurezza, come auth_success, auth_failed, access_denied, session_expired, critical_action_requested, devono essere raccolti e mostrati nei report o nei pannelli security. Questa integrazione rende la sicurezza tracciabile.

Una pipeline security funzionale può essere:

auth attempt;

decision;

session update;

security_event;

dashboard security panel;

audit summary;

report section.

Questi eventi non devono contenere dati biometrici grezzi. Devono contenere decisione, reason, access_level e timestamp. La funzione è importante perché dimostra che access control non è solo UI, ma parte del sistema.

La venticinquesima integrazione funzionale è **fault injection** → **recovery analysis**. Se in simulazione si introduce un fault, il sistema deve reagire e poi analizzare la recovery. Per esempio, un nodo viene forzato offline; il controller riassegna ruolo; il report indica recovery_success o recovery_failed. Questa funzione è molto potente per testare robustezza.

Una pipeline fault/recovery può essere:

fault injected;

event logged;

controller detects;

recovery strategy starts;

new state produced;

metrics updated;

report records result.

Questa integrazione trasforma i fault da effetti casuali a test controllati. È molto utile per mostrare maturità del sistema.

La ventiseiesima integrazione funzionale è **pattern library** → **pattern selector**. Se esiste una libreria di pattern, la dashboard deve poterla leggere e mostrare un selettore. Ogni pattern deve avere nome, descrizione, requisiti, node_count minimo, complessità, modalità supportate, stato e rischi. L'utente può scegliere un pattern e avviare simulation-only o demo controllata.

Un pattern selector funzionale deve mostrare:

pattern_id;

nome;

descrizione;

required_nodes;

complexity;

supported_modes;

demo_ready;

hardware_safe;

expected_duration;

limitations.

Questa funzione rende i pattern gestibili. Non sono hardcoded in un menu casuale, ma descritti da metadata. È un'integrazione molto pulita.

La ventisettesima integrazione funzionale è **configuration diff** → **safe apply**. Quando l'utente modifica una configurazione, il sistema deve mostrare differenze e validare prima di applicare. Questo è importante soprattutto per parametri sensibili. L'integrazione tra config editor, validator e runtime crea una funzione sicura.

Una pipeline safe apply può essere:

utente modifica config;

diff generato;

schema validation;

safety validation;

compatibility check;

confirm;

apply;

event log;

runtime update o restart required.

Questo evita modifiche cieche. L'utente sa cosa cambia. Il sistema blocca valori invalidi. È una funzione avanzata ma molto utile.

La ventottesima integrazione funzionale è **modalità operative** → **interfaccia e permessi**. La dashboard deve cambiare comportamento in base alla modalità: demo, replay, simulation, live, hardware, hybrid, development, presentation. In replay mode i comandi live sono disabilitati. In presentation mode i pannelli complessi sono nascosti. In hardware mode safety panel è obbligatorio. Questa integrazione rende l'interfaccia contestuale.

Una modalità funzionale determina:

fonti dati;

controlli disponibili;

pannelli visibili;

permessi richiesti;

azioni disabilitate;

messaggi di stato;

logging.

Questa funzione evita confusione. L'utente deve sempre sapere in quale modalità si trova. MicroBot non deve sembrare live quando sta mostrando replay, e non deve permettere comandi live in replay.

La ventinovesima integrazione funzionale è **roadmap** → **issue/task generator**. In futuro, la roadmap potrebbe generare task o checklist: creare schema, implementare validator, aggiungere dashboard panel, testare pattern. Anche senza integrazione GitHub automatica, uno script può generare `TODO_NEXT_STEPS.md` o `ROADMAP_TASKS.md`. Questa funzione collega pianificazione e lavoro concreto.

Una roadmap funzionale può produrre:

task_id;

descrizione;

modulo coinvolto;

priorità;

dipendenze;

stato;

output atteso;

criterio di completamento.

Questo aiuta a non perdersi. Una roadmap enorme diventa lista lavorabile. È molto utile per progetti grandi come MicroBot.

Il trentesimo punto è la **sintesi dell'integrazione funzionale**. L'integrazione funzionale è il momento in cui le connessioni diventano strumenti reali. Non si tratta più solo di dire che i progetti sono collegati, ma di farli lavorare insieme: simulazione che alimenta dashboard, telemetria che genera report, validator che protegge launcher, hardware che appare nella dashboard, sicurezza che abilita controlli, AI che produce raccomandazioni, genetica che esporta profili, replay che usa eventi, release builder che produce zip puliti.

In conclusione, l'integrazione funzionale è ciò che trasforma MicroBot da insieme di componenti collegabili a piattaforma operativa. Ogni integrazione deve produrre una funzione verificabile: visualizzare, validare, esportare, diagnosticare, comandare, analizzare, confrontare, documentare, archiviare o rilasciare. La regola fondamentale è che ogni funzione deve avere input chiari, output chiari, schema, versione, stato, limiti e modalità operative. Non tutti i progetti esterni devono diventare funzioni del core; alcuni restano concettuali, altri documentali, altri strumenti di supporto. Ma quelli che vengono integrati funzionalmente devono essere testabili e ripetibili. Il punto successivo, dedicato al **supporto reciproco**, spiegherà come i vari progetti e moduli MicroBot possano rafforzarsi a vicenda: la simulazione supporta l'hardware, la dashboard supporta la documentazione, gli script supportano la manutenzione, i tool diagnostici supportano la release, e i progetti esterni supportano la crescita complessiva dell'ecosistema.